

COMSOL Multiphysics

Programming Reference Manual

COMSOL Multiphysics® Programming Reference Manual

© 1998–2025 COMSOL

Protected by patents listed on www.comsol.com/patents, or see Help > About COMSOL Multiphysics on the File menu in the COMSOL Desktop for less detailed lists of U.S. Patents that may apply. Patents pending.

This Documentation and the Programs described herein are furnished under the COMSOL Software License Agreement (www.comsol.com/sla) and may be used or copied only under the terms of the license agreement.

COMSOL, the COMSOL logo, COMSOL Multiphysics, COMSOL Desktop, COMSOL Compiler, COMSOL Server, and LiveLink are either registered trademarks or trademarks of COMSOL AB. All other trademarks are the property of their respective owners, and COMSOL AB and its subsidiaries and products are not affiliated with, endorsed by, sponsored by, or supported by those trademark owners. For a list of such trademark owners, see www.comsol.com/trademarks.

Version: COMSOL 6.4

Contact Information

Visit the Contact COMSOL page at www.comsol.com/contact to submit general inquiries or search for an address and phone number. You can also visit the Worldwide Sales Offices page at www.comsol.com/contact/offices for address and contact information.

If you need to contact Support, an online request form is located on the COMSOL Access page at www.comsol.com/support/case. Useful links:

- Support Center: www.comsol.com/support
- Product Download: www.comsol.com/product-download
- Product Updates: www.comsol.com/product-update
- COMSOL Blog: www.comsol.com/blogs
- Discussion Forum: www.comsol.com/forum
- Events: www.comsol.com/events
- COMSOL Video Gallery: www.comsol.com/videos
- Support Knowledge Base: www.comsol.com/support/knowledgebase
- Learning Center: www.comsol.com/support/learning-center

Part number: CM020007

C o n t e n t s

Chapter 1: Introduction

About the COMSOL API	18
Additional COMSOL API Documentation	18
Where Do I Find More Information?	18
Getting Started	20
The Model Object	20
Compiling a Model File for Java [®]	20
The Model File for Java	21
Running a Compiled Model File for Java from the Desktop	22
Running a Compiled Model File as a Batch Job from the Desktop	22
Running a Compiled Model File with the COMSOL Batch Command	22
Getting the COMSOL Installation Path from the Windows Registry	22
Setting up Eclipse for Compiling and Running a Java [®] File	22

Chapter 2: General Commands

About General Commands	26
Overview of General-Purpose Commands	26
About Name, Identifier, and Tag Properties.	28
get* and Selection Access Methods	29
set().	30
setEntry().	32
setIndex().	32
hasTag().	33
Methods Associated to Set, SetIndex, and the Various Get Methods	33
Special Property Types.	34
Selections	34
Color Themes	37
The loadFile and saveFile Methods	37
Inserting Features from Other Models.	38
ModelUtil.	39
model	44
model.attr().	47
model.attr(<tag>)	48
model.batch().	49
model.bem().	56
model.coeff().	58
model.common().	61
model.component().	82
model.constr().	85
model.coordSystem().	87
model.cpl().	94
model.elem().	101
model.elementSet().	104
model.externalInterface	106
model.extraDim().	107

model.field()	107
model.form()	108
model.frame()	108
model.func()	110
model.geom()	135
model.group()	143
model.init()	143
model.intRule()	144
model.massProp()	144
model.material()	146
model.mesh()	158
model.methodCall()	163
model.modelNode()	164
model.multiphysics()	164
model.nodeGroup()	165
model.ode()	166
model.opt()	168
Least-Squares Objective Functions	169
model.pair()	170
model.param() and model.result().param()	174
model.physics()	176
model.probe()	181
model.reduced()	184
model.result()	193
model.savePoint()	199
model.selection()	200
model.shape()	209
model.sol()	211
model.solverEvent()	215
model.study()	217
model.thermodynamics()	222
model.unitSystem()	224
model.variable()	226
model.view()	228
model.weak()	238
Plotting and Exporting Images	240
Overview.	240
Properties for the image() Objects	240
Errors and Warnings	244
Introduction.	244
Retrieving Problem Information	244

Chapter 3: Geometry

About Geometry Commands	248
Features for Creating Geometric Primitives.	248
Features for Geometric Operations.	249
Selection Features	251
Features for Virtual Operations	251
Features for Mesh Control	252

Geometric Model Information Methods	252
Working with a Geometry Sequence	255
Adding a Model Component (Geometry).	255
Adding a Geometry Feature.	255
Editing a Geometry Feature.	256
Building Geometry Features.	257
Feature Status	258
Accessing Geometry Object Names	258
Deleting and Disabling Geometry Features	259
Deleting Geometry Objects.	259
Moving and Scaling Geometry Objects.	260
Plotting a Geometry Sequence.	260
Geometry Attribute Methods	260
Geometry Settings	262
Length Unit	262
Angular Unit	262
Scale Values When Changing Unit	262
Reduction for Symmetry Boundaries	263
Geometry Representation in 3D and for Boolean Operations	263
Default Repair Tolerances	264
Automatic Rebuild	264
Constraints and Dimensions	265
Work Planes	266
Selections of Geometric Entities	267
Named Selections	267
Using Selection Features	267
Cumulative Selections	268
Vectorized Selections of Geometric Entities	268
Geometry Cleanup	269
Virtual Operations	270
About Virtual Operations	270
Mesh Control Entities	270
Geometry Object Information	271
General Information	271
Geometric Entity Counters	272
Adjacency	272
Evaluation on an Edge	273
Evaluation on a Face.	274
Geometry Representation Arrays	275
Measurements	277
Measurement Methods	277
Selecting Objects and Entities to Measure	278
Inserting Geometry Sequences from File	279
Example of Importing Geometry Sequences.	279

Exporting Geometry to File	280
Exporting to CAD File Formats	280
Exporting to an STL File	280
Compatibility for MPHBIN and MPHTXT in 2D and 3D	281
Advanced Settings for DXF Files in 2D.	281
 Creating and Using Geometry Parts	 282
 Geometry Commands	 284
AdjacentSelection	285
Array	287
BallSelection, BoxSelection, CylinderSelection, DiskSelection	289
BezierPolygon	292
Block	294
CentroidMeasurement.	297
Chamfer	297
Circle	299
CircularArc	301
CollapseEdges	302
CollapseFaces	303
CollapseFaceRegions	304
Compose, Union, Intersection, Difference	304
CompositeCurve	306
CompositeDomains	308
CompositeEdges	309
CompositeFaces	310
Cone	311
ConvertToSolid, ConvertToSurface, ConvertToCurve, ConvertToPoint.	314
CrossSection	315
CubicBezier	317
Cylinder	319
Delete	321
DistanceMeasurement	323
ECone	324
EditObject	326
Ellipse	328
Ellipsoid	330
ExplicitSelection	332
Extract.	333
Extrude	334
Fillet.	337
Finalize.	339
FromMesh	340
Helix	340
Hexahedron.	342
If, Elseif, Else, EndIf	344
IgnoreEdges	345
IgnoreFaces	346
IgnoreVertices	347
Import DXF.	348
Import Geometry Sequence.	350
Import Mesh Part or Meshing Sequence	351
Import mphbin/mphtxt.	353
InterpolationCurve	355

Interval	356
LineSegment	358
LogicalExpressionSelection	360
MergeEdges	360
MergeFaces	361
MergeVertices	362
MeshControlDomains	362
MeshControlEdges	363
MeshControlFaces	364
MeshControlVertices	364
Mirror	365
Move, Copy	367
Offset	369
ParameterCheck	370
ParametricCurve	370
ParametricSurface	372
PartInstance	374
Partition	377
PartitionDomains	379
PartitionEdges	380
PartitionFaces	381
Point	382
Polygon	384
Pyramid	386
QuadraticBezier	388
Rectangle	389
RemoveDetails	391
Revolve	392
RigidTransform	394
Rotate	396
Scale	398
Sphere	400
Split	402
Square	404
Sweep	406
Tangent	408
Tetrahedron	411
Thicken2D	412
Torus	413
UnionSelection, IntersectionSelection, DifferenceSelection, ComplementSelection	416
WorkPlane	418

Chapter 4: Mesh

About Mesh Commands	426
Working with a Meshing Sequence	428
Adding a Meshing Sequence	428
Adding a Mesh Feature	429
Editing a Mesh Feature	429
Building Mesh Features	430
Using Mesh Parts	430

Using Mesh-Based Geometries	430
Feature Status	431
Deleting Mesh Features	431
Disabling Mesh Features	431
Clearing Meshes	431
Locking Meshes	432
Units	432
Selections	432
Mesh Settings	434
Automatic Rebuild	434
Length and Angular Units.	434
Geometric Model	434
Physics-Controlled Meshing	436
Geometric Analysis, Detail Size Contribution	436
Selecting Contributing Physics	436
Adaptively Refined Meshes	438
Information and Statistics	439
Statistics	439
Number and Types of Elements	440
Quality of Elements	440
Volume of Elements and Mesh	442
Growth Rate in Mesh	442
Mesh Status	442
Mesh Plot.	442
Measurements	443
Information on the Geometric Model	443
Getting and Setting Mesh Data	444
Accessing Mesh Data	444
Setting or Modifying Mesh Data	445
Block Versions.	447
Mesh Element Numbering Conventions	448
Errors, Warnings, and Information	450
Encountering Problems when Building the Mesh	450
Continue Meshing	450
Errors Stopping the Operations	450
Retrieving Mesh Problem Information	450
Retrieving Information About the Latest Build	451
Exporting Meshes to Files	452
Exporting Mesh to a File	452
Exporting Mesh to a COMSOL Multiphysics File	452
Exporting Mesh to a NASTRAN [®] File.	453
Exporting Mesh to a Sectionwise Format.	453
Exporting Mesh to a 3MF Format.	453
Mesh Commands	454
Adapt	454
AdjacentSelection	456

Ball	457
BallSelection, BoxSelection, CylinderSelection, DiskSelection	458
BndLayer	461
BndLayerProp	463
Box	464
CollapseEntities	466
Convert	466
CopyEdge	468
CopyFace.	469
CopyDomain	470
Copy	472
CornerProp.	474
CornerRefinement	475
CreateDomains	475
CreateEdges	476
CreateFaces.	478
CreateVertices.	479
Cylinder	480
DeleteEntities	481
DetectFaces.	482
Distribution	483
Edge.	484
EdgeGroup	485
EdgeMap	486
ExplicitSelection	487
FillHoles	488
FixedMesh	489
FreeQuad	489
FreeTet	490
FreeTri	492
IdenticalMesh	493
Import.	494
Imprint.	500
IntersectLine	501
IntersectPlane	503
JoinEntities	506
LogicalExpression.	506
Map	507
MergeEntities	508
OnePointMap	509
Reference	511
Refine	512
RemeshDomains	514
RemeshEdges	514
RemeshFaces	515
Scale	516
Size	517
SizeExpression	520
Sweep	521
Transform	523
TwoPointMap	524
Union	525
UnionSelection, IntersectionSelection, DifferenceSelection, ComplementSelection	526
Vertex	527

Chapter 5: Elements and Shape Function Programming

Shape Functions and Element Types	530
Shape Function Types (Elements)	530

Chapter 6: Solvers and Study Steps

About Solver Commands	540
Features Producing and Manipulating Solutions	540
Features with Solver Settings	541
Solution Object Information Methods	542
Solution Feature Information Methods	544
Solution Object Data	545
General Information	545
Solution Data	547
SolutionInfo Object and Its Methods	548
Solution Creation	551
General Matrix Information	552
Matrix Data	553
Matrix Creation	554
The clist and cname Properties	555
Adaption	556
Advanced	556
Assemble	559
AutoRemesh	560
AWE	561
CombineSolution	564
CopySolution	568
Eigenvalue	568
EigenvalueAttrib	574
EigenvalueParam	574
FFT	575
For, EndFor	578
FullyCoupled	579
HardwareAcceleration	585
InputMatrix	585
Linear	586
LowerLimit	600
LumpedStep	600
Modal	601
ModalReduction	605
ModeFollowing	608
Optimization	609
Parametric	615
PlugFlow	616
ProperOrthogonalDecomposition	617
Previous Solution	618
Segregated	618
SegregatedStep	621
Sensitivity	624
StatAcceleration	625

StateSpace	625
Stationary	627
StopCondition	629
StoreSolution	629
StudyStep.	630
Time	631
TimeAdaption	640
TimeDiscrete	642
TimeExplicit.	644
TimeParametric	645
UpperLimit	646
Variables	647
XmeshInfo	649
 Studies and Study Steps	 652
Introduction.	652
Batch	655
Batch Sweep	657
Bidirectionally Coupled Particle Tracing	658
Bidirectionally Coupled Ray Tracing.	661
Cluster Computing	664
Cluster Sweep	667
Eigenfrequency.	669
Eigenvalue	672
Stationary Then Eigenfrequency	675
Frequency Domain and Frequency-Domain Perturbation.	676
Frequency to Time FFT	679
Function Sweep	681
Material Sweep	682
Model Reduction	683
Multigrid Level	685
Parametric Sweep	686
Ray Tracing	687
Schrödinger–Poisson	690
Sensitivity.	693
Stationary	694
Surrogate Model Training.	698
Time Dependent	703
Time Discrete	706
Time to Frequency FFT	708

Chapter 7: Results

About Results Commands	712
Commands Grouped by Function	716
 Use of Datasets	 723
 Extracting and Storing Plot Data	 726
Retrieving Plot Data.	726
Retrieving Numerical Results	727
Updating Plots, Storing and Clearing Plot Data, and Previewing Datasets	729

Solution Selection	730
About Selecting Solutions	730
Selecting Solutions by Solution Number	730
Selecting Solutions by Solution Level	730
Choosing Solution Selection Method	731
Results Configurations	731
 Custom Color Tables	 732
Overview.	732
API Methods for colorTable.	732
Properties for colorTable	732
 Stepping Through Plots	 734
API Methods for Stepping Through Plots	734
 Special Formats and Symbols in Texts	 735
Using Special Formats and Symbols	735
 Results Commands	 737
Animation	737
Annotation	742
AnnotationData	745
Array 1D, Array 2D, Array3D	747
ArrayFactor	748
ArrowData	749
ArrowVolume, ArrowSurface, ArrowLine, ArrowPoint	752
AvVolume, AvSurface, AvLine	757
Average, Integral, Maximum, Minimum.	760
Beam	762
Color	762
Comparison.	765
Contour (Plot).	765
Contour (Dataset)	770
ContourSeries	771
CoordSysLine, CoordSysSurface, CoordSysVolume, CoordSysPoint	776
CutLine2D, CutLine3D	780
CutPlane	781
CutPoint1D, CutPoint2D, CutPoint3D	782
Data.	784
Deform	787
Directivity	788
Edge2D, Edge3D	792
Embed1D, Embed2D	793
EnergyDecay	793
ErrorBars.	794
Eval	795
EvalAberration.	797
EvalGlobal	799
EvalGlobalMatrix	802
EvalGlobalSweep	805
EvalPoint	806
EvalPointMatrix	810
EvaluationGroup	813
Export.	815

Extrude1D, Extrude2D	816
Filter (Dataset)	817
Filter (Plot Attribute)	818
Filter (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing)	819
Filter (Table Graph)	819
Function	820
Function2D	824
Function3D	826
Global (Numerical)	830
Global (Plot)	832
Grain (1D Plot)	838
Grain (Dataset)	842
Grain (Evaluation)	843
GrainBin	845
GrainTrajectories.	846
GraphMarker	850
Grid1D, Grid2D, Grid3D.	853
Height, AberrationHeight, HistogramHeight, TableHeight	854
Histogram	856
Image (Export).	861
Image (Plot).	864
ImageOverlay	866
ImpulseResponse	868
InterferencePattern	872
Interp	875
IntersectionPoint2D, IntersectionPoint3D	878
IntVolume, IntSurface, IntLine	880
Isosurface (Plot)	884
Isosurface (Dataset).	888
IsosurfaceSeries	889
Join	893
JointProbabilityDistribution	894
KernelDensityEstimation	896
LayeredMaterial	897
LayeredMaterialSlice	898
Line	903
LineData	908
LineGraph	911
LineSegments	916
Marker.	919
MassProperties	922
MaterialAppearance.	925
MatrixHistogram	926
MaxMinVolume, MaxMinSurface, MaxMinLine, MaxMinPoint	930
MaxVolume, MaxSurface, MaxLine, MinVolume, MinSurface, MinLine	934
MeasureVolume, MeasureSurface, MeasureLine, MeasureDistance	939
Mesh (Plot)	942
Mesh (Dataset)	945
Mesh (Export)	946
MeshImportParameters	946
Mirror2D, Mirror3D	947
Multislice	948
Nyquist	953
OctaveBand.	957

OpticalAberration	961
Parametric1D, Parametric2D	965
ParCurve2D, ParCurve3D	965
ParSurface	966
Particle	967
Particle (1D Plot).	972
Particle (Dataset).	976
Particle (Evaluation).	977
ParticleBin	980
ParticleMass.	980
ParticleTrajectories	986
Partition	990
Pellets	991
PhasePortrait	995
Pipe	998
Plot	998
PlotGroup1D, PlotGroup2D, PlotGroup3D.	1000
PoincareMap	1011
Point	1013
PointData.	1018
PointGraph	1020
PointTrajectories.	1026
PolarGroup	1030
Polarization	1033
Principalline, PrincipalSurface, PrincipalVolume	1037
RadiationPattern	1041
RadiationPattern (Export)	1047
Ray (1D Plot)	1049
Ray (Dataset)	1054
Ray (Evaluation)	1055
RayBin	1057
RayTrajectories	1058
Receiver2D, Receiver3D	1062
ReflectionGraph, ImpedanceGraph, AdmittanceGraph.	1063
ResponseSpectrum2D, ResponseSpectrum3D	1067
Revolve1D, Revolve2D	1069
ScatterVolume, ScatterSurface	1070
Sector2D, Sector3D	1074
Selection (Dataset Attribute)	1075
Selection (Plot Attribute).	1076
Shell.	1077
Slice.	1077
SmithGroup.	1084
SobolIndex	1087
Solution	1089
SolutionArray	1090
Spatial FFT	1091
Spot Diagram	1092
StdDevVolume, StdDevSurface, StdDevLine.	1095
Streamline	1099
StreamlineMultislice.	1107
StreamlineSurface	1113
Surface (Plot)	1119
Surface (Dataset).	1124

SurfaceData	1124
SurfaceSlit	1128
SystemMatrix	1133
Table	1134
Table (Export)	1138
Table (Plot)	1138
TableAnnotation	1142
TableArrow	1144
TableColor	1146
TableContour	1148
TableDOE	1152
TableErrorBars	1154
TableHistogram	1155
TablePoint	1158
TableSurface	1162
TableUQ	1166
TimeAverage, TimeIntegral	1168
ThroughThickness	1169
Touchstone	1175
Transformation2D, Transformation3D.	1176
Transformation	1177
Transparency	1178
TubeData.	1180
VisualEffects.	1182
Volume	1183
Waterfall	1187
Whirl	1190

Chapter 8: The COMSOL File Formats

File Formats	1196
Data Formats	1197
Spreadsheet Data Format	1197
Grid Data Format	1198
Sectionwise Data Format.	1198
CGNS	1199
Supported Microsoft Excel File Types	1199
Color Tables and Color Themes	1200
About Color Tables.	1200
Continuous Color Tables.	1200
Discrete Color Tables	1200
About Color Themes	1201
Materials	1204
The Model XML-File Format	1204
Supported Material Types and Settings.	1205
Supported Property Groups and Material Properties	1210
Supported Physical Quantities for Model Inputs	1249
Supported Functions and Settings.	1251

Native Binary Data Files and Text Data Files	I 253
File Structure	I 253
Objects	I 255
Terminology	I 255
Text File Format	I 255
Binary File Format	I 255
 Serializable Classes	 I 256
BezierCurve.	I 256
BezierSurf	I 258
BezierTri	I 258
BSplineCurve	I 259
BSplineSurf	I 261
Ellipse	I 262
Geom1	I 262
Geom2	I 263
Geom3	I 265
GeomCurve.	I 268
GeomSurf	I 268
Mesh	I 268
MeshCurve	I 270
MeshSurf	I 271
PolChain	I 271
Selection	I 272
Serializable	I 273
Straight	I 273
Transform	I 273
Example of the Serialization Format.	I 274
 Index	 I 279

Introduction

This *COMSOL[®] Multiphysics Programming Reference Manual* details features and techniques that help you control COMSOL Multiphysics[®] using the application programming interface (API) for use with the Java[®] programming language. The COMSOL API can be used in the Application Builder, in model methods and the Shell window in the COMSOL Desktop[®], in a model file for Java, and from the LiveLink[™] for MATLAB[®] interface. If you are using the COMSOL API from the Application Builder, see also the *Application Programming Guide* for useful information when creating methods for applications.

In this chapter:

- [About the COMSOL API](#)
- [Getting Started](#)

About the COMSOL API

You can use the COMSOL API to develop custom applications based on COMSOL. The easiest way to create such applications is by using the Application Builder available with the COMSOL software.

You can run Java class files with COMSOL API-based applications in different ways:

- In the Application Builder as part of methods that you add using the Method Editor in the Application Builder's development environment (see also the *Application Programming Guide*).
- From the COMSOL Desktop®. A model created using a class file appears automatically in the Desktop.
- From a batch sequence in a study.
- Using the `comsol batch` command.

You can create model methods for use with the Model Builder tree when creating and running COMSOL Multiphysics models in the COMSOL Desktop. The COMSOL Desktop also includes a Shell window, from which you can run COMSOL API commands.

The LiveLink™ for MATLAB® operates using the COMSOL API and additional utility M-file functions. See the *LiveLink™ for MATLAB® User's Guide* for additional information.

Code examples for the individual API functions in this guide show what the code looks like when using Java® and when using the LiveLink™ for MATLAB®.

Additional COMSOL API Documentation

You find additional COMSOL API documentation for specialized API commands, which is available with the Model Manager and with corresponding COMSOL products, in the following COMSOL documents:

- The *Model Manager Reference Manual*.
- The *CAD Import Module User's Guide*.
- The *Design Module User's Guide*.
- The *ECAD Import Module User's Guide*.
- The CAD LiveLink™ products' *User's Guides*.

Where Do I Find More Information?

A number of online resources have more information about COMSOL, including licensing and technical information. The electronic documentation, topic-based (or context-based) help, and the Application Libraries are all accessed through the COMSOL Desktop.



If you are reading the documentation as a PDF file on your computer, the [blue links](#) do not work to open an application or content referenced in a different guide. However, if you are using the Help system in COMSOL Multiphysics, these links work to open other modules, application examples, and documentation sets.

CONTACTING COMSOL BY EMAIL

For general product information, contact COMSOL at info@comsol.com.

COMSOL ACCESS AND TECHNICAL SUPPORT

To receive technical support from COMSOL for the COMSOL products, please contact your local COMSOL representative or send your questions to support@comsol.com. An automatic notification and a case number will be sent to you by email. You can also access technical support, software updates, license information, and other resources by registering for a COMSOL Access account.

COMSOL ONLINE RESOURCES

COMSOL website	www.comsol.com
Contact COMSOL	www.comsol.com/contact
COMSOL Access	www.comsol.com/access
Support Center	www.comsol.com/support
Product Download	www.comsol.com/product-download
Product Updates	www.comsol.com/product-update
COMSOL Blog	www.comsol.com/blogs
Discussion Forum	www.comsol.com/forum
Events	www.comsol.com/events
COMSOL Application Gallery	www.comsol.com/models
COMSOL Video Gallery	www.comsol.com/videos
Learning Center	www.comsol.com/support/learning-center
Support Knowledge Base	www.comsol.com/support/knowledgebase

Getting Started

In this section:

- [The Model Object](#)
- [Compiling a Model File for Java[®]](#)
- [The Model File for Java](#)
- [Running a Compiled Model File for Java from the Desktop](#)
- [Running a Compiled Model File as a Batch Job from the Desktop](#)
- [Running a Compiled Model File with the COMSOL Batch Command](#)
- [Getting the COMSOL Installation Path from the Windows Registry](#)
- [Setting up Eclipse for Compiling and Running a Java[®] File](#)

The Model Object

In the COMSOL API you access models through the *model object*, which contains all algorithms and data structures for a COMSOL model. The COMSOL Desktop also uses the model object to represent your model. This means that the model object and the COMSOL Desktop behavior are virtually identical.

You use *methods* to create, modify, and access your model. The model object provides a large number of methods, including methods for setting up and running *sequences of operations* to create geometry, meshes, and for solving your model. The methods are structured in a tree-like way, much similar to the nodes in the model tree in the *Model Builder* window on the COMSOL Desktop. The top-level methods just return references that support further methods. At a certain level the methods perform actions, such as adding data to the model object, performing computations, or returning data.

You must have a basic understanding of the Java[®] programming language in order to fully appreciate how to work with the model object. However, the Application Builder includes tools like code recording, auto-completion, and predefined code templates that make it easier to create methods.

Compiling a Model File for Java[®]



With the COMSOL Compiler™, you can compile applications created using the Application Builder into standalone, runnable apps. See the Application Builder documentation for more information.

First make sure that COMSOL Multiphysics is installed. See the *COMSOL Multiphysics Installation Guide* for more information if required.

To test compiling a model files for Java, load `feeder_clamp.mph` from the COMSOL Multiphysics Application Library into the COMSOL Desktop.

You can learn most of the syntax for creating a model using the COMSOL API by first creating a model using the COMSOL Desktop and then saving the model as an application file for Java.

Win	To open the Application Libraries window (): • In the Home toolbar, click Windows and select Application Libraries. When the toolbar is compressed, you sometimes find it under Layout > Windows. • You can also customize the Quick Access Toolbar and then click the Application Libraries () button in the toolbar. • From the File menu select Application Libraries.
Mac Linux	To open the Application Libraries window (): • From the File or Windows menu select Application Libraries.

In the Application Libraries tree, expand **COMSOL Multiphysics** and then **Structural Mechanics**. Select the **feeder_clamp** model, then click the **Open** button to open it. To get a Java file to compile, choose **Save As** from the **File** menu and choose **Model file for Java (*.java)** as the file type. It is suggested that you save the file as **feeder_clamp.java** in your home directory.

To compile **feeder_clamp.java**, enter

```
<COMSOL path>\bin\win64\comsolcompile feeder_clamp.java
```

on Windows and

```
<COMSOL path>/bin/comsol compile feeder_clamp.java
```

on Linux and macOS, where *<COMSOL path>* is the COMSOL installation directory.

The Model File for Java

The model file for Java has the following structure:

```
import com.comsol.model.*;
import com.comsol.model.util.*;

public class feeder_clamp {

    public static void main(String[] args) {
        run();
    }

    public static Model run() {
        Model model = ModelUtil.create("Model");
        ...
        return model;
    }
}
```

Any model that you create in the COMSOL Desktop can be saved as a model file for Java.



When you compile a model file for Java into a class file and run it, COMSOL runs exactly those instructions that are included in the model file for Java. When opening an MPH file and saving it as a Java file only those sequences that have been explicitly run are run in the Java file. But saving it as a model file for Java, the file does not contain a `runAll` command for the solver sequence. To run a solver sequence, add a line similar to `model.sol("sol1").runAll();` (where `sol1` is the tag for the solver to run) at the bottom of the Java file, above the line that contains `return model;`.

Running a Compiled Model File for Java from the Desktop

Select **Open** on the **File** menu. In the **Open** dialog, next to the **File name** field, select **Compiled Model File for Java (*.class)**. Click **Open**. The file is run and appears as the model in the COMSOL Desktop.

Running a Compiled Model File as a Batch Job from the Desktop

Right-click **Job Sequences** in a study and add a study. In the added study, right-click and add **External Class** under **Other**. Then right-click the batch sequence and select **Compute**.

Runs the main function of a compiled class with the system property `cs.currentmodel` set to the tag of the model calling the class. Thus you can retrieve the current model using the steps:

```
import java.io.*;

tag = System.getProperty("cs.currentmodel");
model = ModelUtil.model(tag);
```

Running a Compiled Model File with the COMSOL Batch Command

To run the file, enter

```
<COMSOL path>\bin\win64\comsolbatch -inputfile feeder_clamp.class
```

on Windows, or enter

```
<COMSOL path>/bin/comsol batch -inputfile feeder_clamp.class
```

on Linux and Mac, where `<COMSOL path>` is the COMSOL Multiphysics installation directory.

Getting the COMSOL Installation Path from the Windows Registry

If you want to have your application find your COMSOL Multiphysics installation automatically, you can have your application examine the registry key

```
HKEY_LOCAL_MACHINE\SOFTWARE\COMSOL\COMSOL64\
```

The value name `COMSOLROOT` contains the installation path.

Setting up Eclipse for Compiling and Running a Java® File

Instead of using the COMSOL commands for compiling and running a Java® file that uses the COMSOL API one can use an Integrated Development Environment for doing these tasks. Using Eclipse makes it easier to write the Java code because Eclipse has built-in support for code completion and syntax highlighting. Furthermore, the debugger that comes as a part of Eclipse can be used to run the code line by line to verify the function of the code

and check for any programming errors. Eclipse is free and can be downloaded from www.eclipse.org. To set up Eclipse for running an exported Java file, perform the following actions in Eclipse:

- 1 Create a new Java Project and click **Next**.
- 2 Go to the **Libraries** tab, select the **Classpath** entry, and click **Add External JARs**. Add all JAR files placed in the `plugins` directory under the COMSOL installation directory (typically `C:\Program Files\COMSOL\COMSOL64\Multiphysics`). This allows Eclipse to find the definitions of the classes used by the COMSOL API and to run the code in client/server mode. Click **Finish**.
- 3 Drag and drop your exported Java file the `src` folder of your Eclipse project.
- 4 Add this line to the beginning of the main method

```
ModelUtil.initStandalone(false);
```

The argument should be false for programs that do not use graphics and true for applications that do.

- 5 To run your Java program, you can create a Run Configuration in Eclipse. You do this from **Run Configurations** on the **Run** menu in Eclipse. There, select the **Environment** tab and then click the **New** button. In the **New Environment Variable** dialog, add the following variables; then click **OK**:

- On Windows, use the **Name** `PATH` (on Windows) and enter the following text in **Value**:
`<comsolinstalldir>\lib\win64`, where `<comsolinstalldir>` is the directory where COMSOL Multiphysics is installed.
- On Linux, use the **Name** `LD_LIBRARY_PATH` and enter the following text in **Value**: `<comsolinstalldir>/lib/glnxa64:<comsolinstalldir>/lib/glnxa64/gcc:<comsolinstalldir>/ext/graphicsmagick/glnxa64:<comsolinstalldir>/ext/cadimport/glnxa64`, where `<comsolinstalldir>` is the directory where COMSOL Multiphysics is installed. Also add the variable with the **Name** `LD_PRELOAD` and enter the **Value** `<comsolinstalldir>/java/glnxa64/jre/lib/amd64/libjsig.so`. If the operating system is set to use a comma as the decimal separator, add a variable with the **Name** `LC_ALL` and the **Value** set to `C` to enforce the COMSOL software to use a period as the decimal separator.
- On macOS, use the **Name** `DYLD_LIBRARY_PATH` and enter the following text in **Value**: `<comsolinstalldir>/lib/maci64:<comsolinstalldir>/ext/graphicsmagick/maci64:<comsolinstalldir>/ext/cadimport/maci64`, where `<comsolinstalldir>` is the directory where COMSOL Multiphysics is installed.

Click **Apply**.

- 6 The Java program can now be started in either Run or Debug mode from Eclipse. The Java program is run as a single process where the COMSOL libraries are being loaded as requested. This is the preferred way of running normal, small model files.
- 7 For large simulation where the application itself has to hold many megabytes in memory in addition to the memory requirement of COMSOL, it can be beneficial to run in client-server mode. To do so, open the Java file in the editor and go to the main method. Now you have to remove the line added in step 4 and add two new lines that control the connection to the COMSOL server from your own program. The main method needs to look like this:

```
public static void main(String[] args) {  
    ModelUtil.connect("localhost", 2036);  
    run();  
    ModelUtil.disconnect();  
}
```

When you have edited the main method you must save the file. Eclipse automatically compiles the file.

- 8 You also need to call `System.exit(0)` at the end of the Java program to terminate the process.
- 9 To run the code you must first start the COMSOL server. When the server has started note the port number that is written in the console. If this number does not match the number written in the call to `ModelUtil.connect` you have to edit this call and save the file again.

The Java program can now be started in either Run or Debug mode from Eclipse. Notice that the COMSOL server window responds by writing that a connection has been set up when your application starts.

General Commands

This chapter contains reference information about general commands for creating and modifying the main parts of the model object and for creating general-purpose functionality in a model, such as functions, variables, units, coordinate systems, nonlocal couplings, and reports and presentations. It also contains information about image generation and about errors and warnings. In this chapter:

- [About General Commands](#)
- [Plotting and Exporting Images](#)
- [Errors and Warnings](#)

About General Commands

Overview of General-Purpose Commands

The following table contains the available general-purpose commands and methods:

TABLE 2-1: GENERAL COMMANDS GROUPED BY FUNCTION.

FUNCTION	PURPOSE
get* and Selection Access Methods	Access objects of the basic data types
set()	Assign objects of the basic data types
setEntry()	Set vector property value at specified entry
setIndex()	Assign objects at indices of the basic data types
hasTag()	Check if a feature has a certain tag.
Selections	Selections
ModelUtil	Model object utility methods
model	Model object
model.attr()	Model entity list methods
model.attr(<tag>)	Model entity methods
model.batch()	Batch jobs
model.bem()	Boundary elements (BEM)
model.coeff()	Coefficient form equations
model.colorTheme() , model.imageColorTheme()	Color themes, See Color Themes .
model.common()	Common definition nodes in components
model.component()	Model component nodes
model.constr()	Constraints
model.coordSystem()	Coordinate systems, PMLs, infinite elements, and absorbing layers
model.cpl()	Nonlocal couplings
model.elem()	Elements
model.elementSet()	Mesh element sets
model.extraDim()	Extra dimensions
model.field()	Fields
model.form()	Settings forms
model.frame()	Frames
model.func()	Functions
model.geom()	Geometry sequences
model.group()	Load groups and constraint groups
model.init()	Initial values
model.intRule()	Integration orders
model.massProp()	Mass properties
model.material()	Materials
model.mesh()	Meshing sequences
model.methodCall()	Model methods

TABLE 2-1: GENERAL COMMANDS GROUPED BY FUNCTION.

FUNCTION	PURPOSE
<code>model.modelNode()</code>	Model nodes (component nodes; see <code>model.component()</code>)
<code>model.multiphysics()</code>	Multiphysics features container
<code>model.nodeGroup()</code>	Group nodes in the model
<code>model.ode()</code>	Global equations
<code>model.opt()</code>	Optimization interface
<code>model.pair()</code>	Pairs
<code>model.param()</code> and <code>model.result().param()</code>	Parameters
<code>model.physics()</code>	Physics
<code>model.probe()</code>	Probes
<code>model.reduced()</code>	Reduced-order modeling
<code>model.result()</code>	Results-processing interface
<code>model.savePoint()</code>	Manage selections and hide features used by result features
<code>model.selection()</code>	Named selections
<code>model.shape()</code>	Shape functions
<code>model.sol()</code>	Solver sequences
<code>model.solverEvent()</code>	Events
<code>model.study()</code>	Studies
<code>model.thermodynamics()</code>	Thermodynamics interface
<code>model.unitSystem()</code>	Unit systems
<code>model.variable()</code>	Variables
<code>model.view()</code>	Views
<code>model.weak()</code>	Weak form equations

ABOUT VALID TAGS

A *tag* is a string that you use to refer to a model feature. When specifying a tag, it must fulfill the following format requirements: Begin with a character a–z or A–Z followed by any number of _ (underscores), numerals 0–9, or characters a–z or A–Z.

ABOUT ASSIGNING VALUES TO PROPERTIES

Even if a property is numeric, it is also possible to use a string or string array. The strings can contain expressions defined in terms of parameters defined in **Global Definitions > Parameters**. See [Table 2-4](#) under `set()` below for examples of syntaxes for assignment methods.

ABOUT FILE PATHS

In general, the file paths in methods for saving and opening files, for example, are client paths (on the client computer's file system). They can also be item version location URIs used to identify a model or file version in a

Model Manager database. The exceptions are methods that explicitly performs an operation on the server, such as `ModelUtil.loadOnServer()`, which takes a server path as its argument.

	The syntax that includes the component level, such as <code>model.component(<ctag>).geom(<tag>)</code> . . . is the default and is used throughout this chapter for parts of the model object that are stored inside a model component. To use the earlier <code>model.geom(<tag>)</code> . . . syntax, clear the Generate code using component syntax checkbox on the Application Builder > Methods page in the Preferences window. You can also run existing scripts without this syntax.
---	--

About Name, Identifier, and Tag Properties

The following tables contain information about the name, identifier, and tag properties for various features in a COMSOL model (see [Labels and Names in the Settings Window](#) in the *COMSOL Multiphysics Reference Manual* for more information). Especially, the table provides information about the relationship between names and tags, whether you can change the name in the user interface or the tag using the API, and the access methods and use of names. This information is useful for understanding how these concepts are related, especially when working with both the COMSOL Desktop and programming using the COMSOL API. In the tables below, *user interface* refers to the Settings windows in the COMSOL Desktop. The term *scope* in the **Use of Name** column in the second table means that the name is used as part of an expression access it in the model object hierarchy; for example, `comp1.ht.Cp`, where `comp1` is the name of the component, and `ht` is the name of the physics interface (see [Variable Naming Convention and Namespace](#) in the *COMSOL Multiphysics Reference Manual* for more information). The name of a user-defined tag is case sensitive, and it must begin with a lowercase or uppercase letter (a–z or A–Z). All other characters in the name must be a lowercase or uppercase letter, a number 0–9, or an underscore (_). See also [Naming Conventions, Reserved Names, and Errors](#) in the *COMSOL Multiphysics Reference Manual*.

TABLE 2-2: THE NAME AND TAG RELATIONS, AND WHETHER YOU CAN CHANGE THEM.

THE NAME/IDENTIFIER CONCEPT	NAME SAME AS TAG	CAN CHANGE NAME IN THE USER INTERFACE	CAN CHANGE TAG IN THE API
Component		Tag changes as well	√
Physics interface		Tag changes as well	√
Physics feature	√	No	√
Multiphysics coupling		Tag changes as well	√
Material	√	No	√
Property group		No	√
Definitions > Functions		Yes	√
Definitions > Coordinate systems		Yes	√
Definitions > Probes		Yes	√
Definitions > Nonlocal couplings		Yes	
Definitions > Variable utilities		Yes	√
Definitions > Pairs		Yes	√
Definitions > Density model		Yes	√
Definitions > Ambient properties		Yes	√

TABLE 2-3: ACCESS METHODS, NAME IN USER INTERFACE AND USE OF THE NAMES.

THE NAME/IDENTIFIER CONCEPT	ACCESS METHOD FOR NAME	METHOD FOR NAME DEPRECATED	NAME IN USER INTERFACE	USE OF NAME
Component	identifier identifier(<i>string</i>)	√	Name	Scope
Physics interface	identifier identifier(<i>string</i>)	√	Name	Scope
Physics feature	tag tag(<i>string</i>)	√		Scope
Multiphysics coupling	identifier identifier(<i>string</i>)	√	Name	Scope
Material	tag tag(<i>string</i>)		Name	Scope
Property group	identifier identifier(<i>string</i>)	√	Name	Scope
Definitions > Functions	get or set function name properties like funcname		Function name	Function name
Definitions > Coordinate systems	get or set name property		Name	Scope
Definitions > Probes	get or set probename property		Variable name	Variable name
Definitions > Nonlocal couplings	get or set opname property		Operator name	Operator name
Definitions > Variable utilities	get or set name property		Name	Variable or operator name
Definitions > Pairs	pairName() pairName(<i>string</i>)		Pair name	Variable suffix (<name>_p1)
Definitions > Density model	get or set name property		Name	Scope
Definitions > Ambient properties	get or set name property		Name	Scope

get and Selection Access Methods*

Use these methods to access properties of the different parts of the model object.

SYNTAX

The following syntax is used for all of these access methods (exemplified there with the `getStringArray` method for returning the value of a string array with the name <name> for the *something* object (feature).

```
something.getStringArray(<name>);
```

Note that throughout this manual, the access methods are collectively referred to as `get*(<name>)`, where `*` can be any of the basic data types used below. Use these methods to read property values. The names of the access methods indicate the data type for the data that they return.

```
something.getString(<name>)
```

returns the value as a string.

```
something.getStringArray(<name>)
```

returns the value as a string array.

```
something.getStringMatrix(<name>)
```

returns the value as a string matrix.

```
something.getInt(<name>)
```

returns the value as an integer.

```
something.getIntArray(<name>)
```

returns the value as an integer array.

```
something.getIntMatrix(<name>)
```

returns the value as an integer matrix.

```
something.getDouble(<name>)
```

returns the value as a double.

```
something.getDoubleArray(<name>)
```

returns the value as a double array.

```
something.getDoubleMatrix(<name>)
```

returns the value as a double matrix.

```
something.getBooleanArray(<name>)
```

returns the value as a boolean array.

```
something.getBooleanMatrix(<name>)
```

returns the value as a boolean matrix.

```
something.selection(<name>)
```

returns the value as a selection object, which can be edited. This is not simply an access function. It is used to obtain a selection object both for editing and for accessing data from.

In addition,

```
something.getEntryKeyIndex(<name>, <key>)
```

returns the index of a given key in a property, and

```
something.getEntryKeys(<name>)
```

returns the possible entry keys for a given property.

To return the parent entity (container) that a model entity is contained in, use

```
something.getContainer()
```

The parent entity of a feature (*something*) is in general the list in which it is contained. The parent of the list is in turn the feature that appears as parent in the Model Builder tree. Therefore, to access the parent feature of a subfeature, call *something.getContainer().getContainer()*. If no container can be found, *getContainer()* returns null.

NOTES

All arrays that are returned contain copies of the data; writing to the array does not change the data in the model object. This observation applies to all access methods of the model object that return arrays of basic data types.

SEE ALSO

[set\(\)](#)

set()

Use this method to assign values to objects of the basic data types.

SYNTAX

Use these methods to assign property values. All assignment methods return the object itself, which means that assignment methods can be appended to each other.

The basic method for assignments is

```
something.set(name,<value>);
```

The *name* argument is a string with the name of the property. The *<value>* argument can be of different types as indicated in Table 2-4, where the two different syntaxes for assignment in the COMSOL API and the LiveLink™ for MATLAB® are listed.

TABLE 2-4: SYNTAXES FOR ASSIGNMENT METHODS.

TYPE	JAVA® SYNTAX	MATLAB® SYNTAX
String	<code>set("name", "value")</code>	<code>set('name', 'value')</code>
String array	<code>set("name", new String[]{"val1", "val2"})</code>	<code>set('name', {'val1', 'val2'})</code>
double string array	<code>set("name", new String[][]{{"1", "2"}, {"3", "4"}})</code>	<code>set('name', {'1', '2'; '3', '4'})</code>
integer	<code>set("name", 17)</code>	<code>set('name', 17)</code>
integer array	<code>set("name", new int[]{1, 2})</code>	<code>set('name', [1 2])</code>
integer matrix	<code>set("name", new int[][]{{1, 2}, {3, 4}})</code>	<code>set('name', [1 2; 3 4])</code>
double	<code>set("name", 1.3)</code>	<code>set('name', 1.3)</code>
double array	<code>set("name", new double[]{1.3, 2.3})</code>	<code>set('name', [1.3 2.3])</code>
double matrix	<code>set("name", new double[][]{{1.3, 2.3}, {3.3, 4.3}})</code>	<code>set('name', [1.3 2.3; 3.3 4.3])</code>
boolean	<code>set("name", true)</code>	<code>set('name', true)</code>
boolean array	<code>set("name", new boolean[]{true, false})</code>	<code>set('name', [true, false])</code>
boolean matrix	<code>set("name", new boolean[][]{{true, false}, {false, false}})</code>	<code>set('name', [true, false; false, false])</code>

For matrix-type properties, `set(name,<string>)` splits the string at spaces and commas.

The following example shows how two set methods can be appended:

```
model.result("pg1").set("edgecolor", "black").set("edges", "on");
```

This is equivalent to:

```
model.result("pg1").set("edgecolor", "black");
model.result("pg1").set("edges", "on");
```

That is, in this case, the set method returns the plot group "pg1".

The following methods using `set` are deprecated in version 5.1 (use `setIndex()` instead):

```
set(name,pos,<value>)
set(name,pos1,pos2,<value>)
```

The following methods using `set` are deprecated in version 5.0:

```
com.comsol.model.ParameterEntity.set(String, int, double)
com.comsol.model.ParameterEntity.set(String, int, double[])
com.comsol.model.ParameterEntity.set(String, int, int)
com.comsol.model.ParameterEntity.set(String, int, int[])
com.comsol.model.ParameterEntity.set(String, int, int, double)
com.comsol.model.ParameterEntity.set(String, int, int, int)
com.comsol.model.ParameterEntity.set(String, int, int, String)
```

```
com.comsol.model.ParameterEntity.set(String, int, String)
com.comsol.model.ParameterEntity.set(String, int, String[])
```

SEE ALSO

[get*](#) and [Selection Access Methods](#), [setIndex\(\)](#)

setEntry()

Use the `setEntry` method to set property values at specified entries using a key instead of an index to locate the row to change.

SYNTAX

Use this method to assign property values of different types at specified entries:

```
setEntry(name, key, value);
```

where all input arguments are strings:

- *name*, representing the property name.
- *key*, representing the index key.
- *value*, representing the property value.

The *value* can also be a double, an integer, or (with limited applicability; primarily for part links) a boolean.

For example, if `pi1` is a part instance feature with input parameters `a` and `b` in a geometry, the following lines

```
GeomFeature f = model.component("comp1").geom("geom1").feature("pi1");
f.setEntry("inputexpr", "a", "2");
f.setEntry("inputexpr", "b", "3");
```

set the value of the input expression for parameter `a` to the string `2` and for parameter `b` to the string `3`. You can also set the values of the input expressions to numerical values (doubles):

```
f.setEntry("inputexpr", "a", 4);
f.setEntry("inputexpr", "b", 5.0);
```

`setEntry` lets you use a key to access rows in the table, defined by the content in one of the columns (in this case, the column that contains parameter names).

setIndex()

Use this method to assign values at indices in array properties of the different parts of the model object. When there are no indices, you can use `set()` instead.

SYNTAX

To use the `setIndex` method, use one of these syntaxes:

```
something.setIndex(name, <value>, <index>);
```

for array properties, or

```
something.setIndex(name, <value>, <firstIndex>, <secondIndex>);
```

for matrix properties.

Use these methods to assign values to an element in array or matrix properties, defined by specific indices (0-based). All assignment methods return the parameter object, which means that assignment methods can be appended to each other.

If *<index>* points beyond the current size of the array, then the array is extended as needed before element *<index>* is set. The values of any newly created intermediate elements are undefined.

The *name* argument is a string with the name of the property. *<value>* is a string representation of the value to set. A double array element, for example, can still be set from a string representation of the double, typically used when the property value depends on a model parameter. The values can also be a boolean or a boolean array. For example:

```
something.setIndex(name,<value>,2)
```

This code assigns the value for the element with the third index in the array (because the indices are 0-based) of an array property *name* to be the value *value*. If the parameter later changes, this property changes accordingly. You can also use an additional input argument for a second index value, for a 2-dimensional array (matrix), for example,

```
something.setIndex(name,<value>,1,4)
```

This code assigns the value *<value>* to the (1, 4) element in a matrix.

For double arrays the modifying method is also of use when assigning the value in MATLAB[®], if not all arrays have the same length. When using a cell matrix, all rows must have the same length. The method

```
something.setIndex(name,<value>,<index>)
```

can be used to get around that limitation. It inserts an array in the indexed position in the double array. The MATLAB code

```
something.setIndex('name',{'1','2','3'},0)
something.setIndex('name',{'4','5'},1)
```

is equivalent to the Java[®] code

```
something.set("name",new String[][]{{"1","2","3"},"4","5"})
```

SEE ALSO

[set\(\)](#)

hasTag()

Use this method to check whether a feature with a given tag exists in a list. The *hasTag* method returns *true* if the list contains a model entity with the given tag.

```
boolean hasTag(String tag);
```

As an example, this code checks if a block has the tag *blk1*, and if it does not exist, creates such a block:

```
boolean isBlockExists = model.geom("geom1").feature().hasTag("blk1");
if (!isBlockExists) {
    model.geom("geom1").feature().create("blk1", "Block");
}
```

Methods Associated to Set, SetIndex, and the Various Get Methods

The following methods are available where the *set*, *setIndex*, and *get<Type>* methods are available:

```
String[] properties();
```

which returns the names of all available properties.

```
boolean hasProperty(String name);
```

which returns *true* if the feature has the named property.

```
String[] getAllowedPropertyValues(String name);
```

which returns the allowed values for a named properties, if it is a finite set.

Special Property Types

In addition to the basic data types, the `set()` and `get*()` methods let you manipulate certain properties with special behavior.

REFERENCE PROPERTY

A reference property is property that stores a reference to an entity in the model object. Each reference property allows references to entities of a particular type, normally identified by the entity's tag. The property is set by passing the identifying tag to the standard `set(<name>, String)` method and read using `getString(<name>)`. But since it is the reference rather than the tag that is stored, the returned value will change if the entity's tag is changed. If the referenced entity is removed or modified in a way which no longer makes it an allowed target for the property, the return value will be `null` or the string `"none"`.

REFERENCE ARRAY

A reference array is an array of references where each position behaves like a reference property. Entity tags are set using `set(<name>, String[])` or `setIndex(<name>, String)`, and read using `getStringArray(<name>)`. Some reference arrays store a specific number of references, while others represent sets of entities. When a referenced entity is removed from the model, it will also be removed from any referring property of the latter type.

STRING MAP

A string map property is a special type of string array property which is used to specify a value for each key in a given set of allowed keys. The key-value pairs are stored interleaved `{key, value, key, value, ...}` in the array. The order of the pairs is arbitrary when setting the array using `set(<name>, String[])`. The value for a specific key can also be set using `setEntry(name, <key>, <value>)`. The keys are many cases references to a model entity specified using the entity's tag. But there are also properties using, for example, prefixed tags for entities of different types.

The complete map containing all possible keys and their current values is returned by `getStringArray(name)`. The value for a specific key is returned by `getString(name, <key>)`. If no values has been explicitly set for a given key, a default value is returned. The default may be different for each key. Values may also be references to a model entity.

Selections

This section contains information about the selection methods that are available for handling selections on the finalized geometry.

SYNTAX

This section describes the general syntax for selections on the finalized geometry. For selections in the geometry sequence, see [Geometry Object Selection Methods](#) under `model.geom()` and [Selections](#) under [Editing a Geometry Feature](#). Many objects use selections, but most of them only support a subset of the assignment methods described here. The methods supported by the named selections in `model.selection()` are listed in [model.selection\(\)](#). Other objects that use a selection support the methods relevant for the type of feature they represent. For example, a physics boundary condition feature requires a boundary selection. Therefore it does not support `selection.global()`, which makes the selection global, or `selection.allGeom()`, which makes the selection apply to the whole geometry at all levels. Using an unsupported assignment method results in an error.



The `selection` part here represents any valid selection syntax that ends with `...selection()`, such as `model.component(<ctag>).physics(<ptag>).feature(<ftag>).selection()`.

There can also be a filtering of the entities assigned to a selection. Again, take a physics boundary condition as an example. Some boundary conditions only apply to the boundaries exterior to the domains where the physics interface is active, other boundary conditions only to boundaries interior to where the physics interface is active, and so on. Therefore `selection.entities(dim)` can sometimes return less entities than have been assigned using `selection.set(<entlist>)`. On the other hand, `selection.inputEntities()` always returns all entities used in the assignment `selection.set(<entlist>)`. `selection.inputEntities()` returns the domains used as input to the selection. If the selection is of the type *interior*, *exterior*, or *meshinterior*, this method returns the unfiltered list of domains at the higher dimension that are used as the input. If the selection is not a selection of domains at a certain level, or the selection is not of the types `Explicit` or `FromGeom` (derived from a geometry feature), this method returns null.

Some selections only allow a single geometric entity, a single domain, a single boundary, edge, or point. Such selections are called *single selections*. Single selections cannot be defined by another selection and therefore do not support `selection.named(<stag>)`.

`selection.global()` sets the selection to be the global selection.

`selection.geom(<gtag>)` sets the selection to be all entities in the geometry `<gtag>`.

`selection.geom(<gtag>, dim)` specifies that subsequent calls to `all`, `set`, `add`, and `remove` refer to entities at the dimension `dim` on the geometry `<gtag>`. If there is only one possible geometry, using `selection.geom(dim)` is equivalent. Also, if there is only one allowed dimension `dim`, then `all`, `set`, and `remove` can be used directly as it is then unambiguous to which geometry and dimension their arguments apply to.

`selection.geom(<gtag>, highdim, lowdim, typelist)` specifies that subsequent calls to `all`, `set`, `add`, and `remove` refer to entities of dimension `highdim` on the geometry `<gtag>`. The entities that are obtained are those that are both of dimension `lowdim` and of any of the types listed in `<typelist>`. It is required that `highdim > lowdim`. The available types are:

- **exterior**: All entities of dimension `lowdim` that lie on the exterior of the entities at dimension `highdim`.
- **interior**: All entities of dimension `lowdim` that lie in the interior of the entities at dimension `highdim`.
- **meshinterior**: All interior mesh entities of dimension `lowdim` that lie in the interior of the entities at dimension `highdim`.

`selection.allGeom()` sets the selection to a whole geometry. Can be used instead of `selection.geom(<gtag>)` when the geometry tag is unambiguous.

`selection.geom(dim)` specifies that subsequent calls to `all`, `set`, `add`, and `remove` refer to entities of dimension `dim`. Can be used instead of `selection.geom(<gtag>, dim)` when the geometry tag is unambiguous.

`selection.all()` sets the selection to use all geometric entities in the geometry at the dimension where the selection applies. Note that you must precede this call with, for example, a `selection.geom(dim)` call to specify the dimension if there is more than one allowed dimension for the feature, which is the case for geometry features. The following example code show how to select all boundaries for a circle (disk) geometry:

```
Model model = ModelUtil.create("Model");
GeomSequence g = model.geom().create("geom1", 2);
g.create("c1", "Circle");
g.run();
g.measureFinal().selection().geom(1);
g.measureFinal().selection().all();
```

`selection.allVoids()` sets the selection to use all voids (finite voids and an infinite void, if present) in the geometry at the dimension where the selection applies. Voids can be present in models that use the boundary element method, for example. In a geometry with voids, `selection.all()` selects all domains, not the voids.

`selection.set(<entlist>)` sets the selection to use the geometric entities in `<entlist>`. Note that the list of entity numbers is always sorted in ascending order and that duplicates are removed before storing the numbers in the selection object.

`selection.add(<entlist>)` adds the geometric entities in `<entlist>` in the geometry to the set of geometric entities that the selection uses to obtain the selection.

`selection.remove(<entlist>)` removes the geometric entities in `<entlist>` in the geometry from the set of geometric entities that the selection uses.

`selection.clear()` removes all entities from the selection.

`selection.inherit(boolean)` indicates whether the selection should include all geometric entities that are specified by any of the other methods and all geometric entities at lower dimensions that are adjacent to the ones already specified.

`selection.named(<stag>)` specifies that the selection is defined by the selection `model.selection(<stag>)`.

`selection.isGlobal()` returns true if the selection is global.

`selection.isGeom()` returns true if the selection is a whole geometry.

`selection.geom()` returns the geometry tag of the selection as a string. If the selection is global, null is returned.

`selection.dimension()` returns the dimensions on a geometry where the selection applies as an integer array.

`selection.dim()` returns the largest entity dimension on which the selection applies.

`selection.entities(dim)` returns the geometric entities of the selection on the given geometry at the given dimension as an integer array. The entities are represented using unique positive integers, except (if the model geometry contains voids) for finite voids, which get unique negative integer numbers, and for an infinite void, which is represented with the entity number 0.

`selection.entities()` returns the geometric entities of the selection on the given geometry at the largest entity dimension as an integer array.

`selection.interiorEntities(dim)` returns the interior mesh domains as an integer array.

`selection.isInheriting()` returns true if the selection is inherited to lower dimension levels.

`selection.inputDimension()` returns the dimension of the entities used as input to the selection.

`selection.inputEntities()` returns the entities used as input to the selection.

If the selection is defined by another selection, `selection.named()` returns the tag of that selection. Otherwise `selection.named()` returns an empty string.

Selections of the class `XDLocalSelection` have the following additional methods:

`selection.extraDim()` returns the tag of a feature of type `AttachDimension` in `model.extraDim()`, or an empty string if no extra dimension attachment is used.

`selection.extraDim(<attachdimtag>)` sets the extra dimension attachment feature. `<attachdimtag>` must be the tag of a feature of type `AttachDimension` in `model.extraDim()` or an empty string to indicate no extra dimension attachment.

`selection.extraDimSel(<xdgeomtag>)` returns the selection in extra dimension geometry `<xdgeomtag>`. `<xdgeomtag>` should be the tag of the geometry in one of the extra dimensions attached by the extra dimension attachment feature defined by `selection.extraDim()`.

`extraDimSel()` returns all extra dimension geometry tags that are valid arguments of `extraDimSel(<xdgeomtag>)` as a string array.

NOTES

The methods `global()`, `geom(<gtag>)`, `geom(<gtag>, dim)`, `geom(<gtag>, highdim, lowdim, typelist)`, and `geom(dim)` clear the dataset by other methods.

Not all assignment methods are supported by all model entities. The list of supported methods also serves as a guide for the restriction to those named selections that can be used by that entity. All access methods are always supported.

SEE ALSO

[model.geom\(\)](#)

Color Themes

Use `model.colorTheme` to specify a color theme for selection colors.

Use `model.colorTheme(<theme>)` to set the color theme to be used in the graphics. Using `automatic` indicates that the theme specified in the preferences will be used. The other possible theme values are the names of the themes in the `data/color` folder in the COMSOL installation folder.

Use `model.colorTheme()` to return the current color theme. The value `automatic` indicates that the color theme specified in the preferences is used.

Use `model.imageColorTheme(<theme>)` to set the color theme for image export. Likewise, `model.imageColorTheme()` returns the current image export color theme.

See also the `color` and `customcolor` properties for selections and geometry features.

The loadFile and saveFile Methods

The methods `loadFile()` and `saveFile()` are available to load and save files for the following features:

- `model.param()` and `model.result().param()`
- `model.variable()` and `model.component(<ctag>).variable()`
- `model.result().table()`

You can use the following syntax for `loadFile`:

`loadFile(String path)` where the path is the path to any file type that COMSOL Multiphysics supports.

`loadFile(String path, char delim)` where `delim` is the delimiter used in the file.

`loadFile(String path, String sheet, String range)`, where the string `sheet` is the name of the sheet to read from (null or empty means the first in the file), and `range` is range of cells to read. The `range` can be a single cell, which then indicates the upper-left cell to read. If `range` is empty, it starts in the upper-left corner.

For the `saveFile` method, the following syntax is available:

`saveFile(String path)`

`saveFile(String path, char delim)`

`saveFile(String path, String sheet, String range, boolean includeHeaders, boolean overwrite)`, where `includeHeaders` determines whether to include headers or not, and `overwrite` determines if nonempty existing cells in a spreadsheet can be overwritten in the file. For example, a call like

```
model.param().saveFile(tempFile, "sheet", "C7", false, true);
```

starts saving at cell C7, does not include headers, and allows overwriting of nonempty cells.

`saveFile(String path, boolean fullPrecision, String sheet, String range, boolean includeHeaders, boolean overwrite)` additionally includes a boolean input argument, `fullPrecision`, for controlling if full precision should be used for numerical values in the exported file.

For `model.result().table`, there are no delimiter functions.

Inserting Features from Other Models

These methods are related to inserting features from other models.

SCANNING MODELS

The `ModelUtil.scanModel` method is an advanced tool that scans the raw data (XML) of the model file for a certain node type and collects the tag and label of all nodes that it finds. In most cases, the node type argument is the same as the corresponding API class name of the node. The reason for using this tool is to quickly scan several MPH files for simple information, such as a list of available materials or functions. If you need more sophisticated access to the model, it is easier to open the model file and process the content. Although you need extra time to open the model, scanning with this tool is very limited. It cannot handle backward compatibility, for example, so scanning of older files may not work.

As an option it is possible to include values of other attributes to the result, but the name of such attributes can be difficult to extract. It will be helpful to unzip the MPH file and look at the content of the `dmodel.xml` file. The example below uses "op", which corresponds to the string return by the `"getType()"` method that some node types support (for example, `PropFeature.getType()`).

Other examples of extra attributes are "entityComments" and "entityAuthor", returning the comment and author of each node, respectively.

The `scanModel` method does not support returning values of settings from, for example, `"getString(name)"` methods.

The returned result is a double string array with all found nodes in the outer level and the sequence of found attributes in the inner level, starting with tag and label. Any optional attributes follow after the tag and label.

Examples:

```
String[][] materials = scanModel("mymodel.mph", "Material", "op");
String[][] functions = scanModel("mymodel.mph", "FunctionFeature", "op");
```

produces an output like:

```
materials = [{"mat1", "Material 1", "Common"}, {"mat2", "Material 2", "Common"}]
functions = [{"an1", "Analytic 1", "Analytic"}, {"int1", "Interpolation 1",
"Interpolation"}]
```

The following parameters are supported:

- **filename:** The file path to the model.
- **type:** The type of the node to search for.
- **extraAttributes:** A list of attributes in addition to the tag and label to include in the result.

The method returns a double string array with the result.

INSERTING MATERIALS

The `insert` method can be used to insert materials with the following syntax:

```
String[][] com.comsol.model.MaterialList.insert(String filename, String[] materials,
String... password)
```

```
String[][] com.comsol.model.ComponentMaterialList.insert(String filename, String[]
materials, String... password)
```

Using this method, you can insert materials from an MPH file into the material list of this model and return the result of the operation as a double string array of length 3.

The first array contains the messages from insert, the second array contains paths to the inserted objects, and the third array contains paths to the inserted references. A pasted reference is an object that an inserted object refers to, and it is not necessarily contained by any of the inserted objects. The following example inserts the materials tagged `mat5` and `mat10` into the model's global materials:

```
model.material().insert("mymodel.mph", new String[]{"mat5", "mat10"});
```

The next example inserts the materials tagged `mat5` and `mat10` into the materials of component `comp1`:

```
String[][] ret = model.component("comp1").material().insert("mymodel.mph", new
String[]{"mat5", "mat10"});
```

It will produce the following output:

```
{{}, {" /MaterialList/mat5", " /MaterialList/mat10"}, {}}
```

The following input arguments are available:

- **filename**: The filename or model location URI used to identify a model version in a Model Manager database.
- **materials**: The tags of the materials to insert.
- **password**: Optional password required to open the file.

The `insert` method returns results from the insert operation.

ModelUtil

Model object utility methods such as methods to create and remove model objects, showing progress information, and listing and saving preferences. See also [model](#).

SYNTAX

This section describes general methods that handle the environment for the model object. It also describes methods for the client/server machinery.

```
import com.comsol.model.*;
import com.comsol.model.util.*;
```

The import statements above make all model and model utility methods available.

ModelUtil.create(<tag>): The `create` method creates a model with tag `<tag>`. Returns a reference to the model. If there is already a model with this tag the previous model is removed.

ModelUtil.remove(<tag>): The `remove` method removes the model tagged `<tag>`.

ModelUtil.clear(): The `clear` method removes all models.

ModelUtil.tags(): The `tags` method obtains the current list of model tags.

ModelUtil.model(<tag>): The `model` method returns a reference to the model tagged `<tag>`.

ModelUtil.closeWindow(<windowtag>): The `closeWindow` method closes the window tagged `<windowtag>`.

ModelUtil.closeWindows(): The `closeWindows` method closes all windows on the server.

ModelUtil.createUnique(<prefix>): The `createUnique` method creates a model with a unique tag that begins with the prefix `<prefix>`.

`ModelUtil.getComsolVersion()`: The `getComsolVersion` method returns the current COMSOL Multiphysics version as a string.

`ModelUtil.getOpenGeometryKernel()`: The `getOpenGeometryKernel` method returns the geometry kernel to use when opening models. Valid values are `model` for the geometry kernel used by the model file, and `comsol` to convert the geometry to the COMSOL kernel.

`ModelUtil.getDefaultGeometryKernel()`: The `getDefaultGeometryKernel` method returns the default geometry kernel in new models. Valid values are `comsol` for the COMSOL kernel and `cadps` for the CAD kernel (Parasolid kernel). The CAD kernel requires the CAD Import Module.

`ModelUtil.setOpenGeometryKernel(<openkernel>)`: The `setOpenGeometryKernel` method specifies the geometry kernel to use when opening models. Valid values are `model` for the geometry kernel used by the model file, and `comsol` to convert the geometry to the COMSOL kernel.

`ModelUtil.setDefaultGeometryKernel(<defaultkernel>)`: The `setDefaultGeometryKernel` method specifies the default geometry kernel to use in new models with the string `<defaultkernel>`. Valid values are `comsol` for the COMSOL kernel and `cadps` for the CAD kernel (Parasolid kernel). The CAD kernel requires the CAD Import Module.

`ModelUtil.getEntityPath(<entity>,<divider>)`: The `getEntityPath` method creates the path from the root of the model object for the given entity `<entity>`. `<divider>` specifies a divider to use between entities in path.

`ModelUtil.load(<tag>,<modelLocation>)`: The `load` method loads a model from a model location `<modelLocation>`, which can be the absolute path of a file on the file system or a model location URI identifying a model version in a Model Manager database and names it `<tag>`. Loading a model file from a directory on the file system sets the model directory. The model directory is used for saving files on the file system if you do not provide an absolute path to the file. The model directory is the directory where the model is saved. If the model has not been saved there is no model directory, you can get the model directory from a saved model using `model.getFilePath`.

`ModelUtil.loadCopy(<tag>,<filename>)`: The `loadCopy` method loads a copy of a model from a file `<filename>` in the client's file system and names it `<tag>`. The `loadCopy` method is the same as `load` except that the loaded model is not associated with the file, so `model.save()` does not work. You have to specify the filename the first time you save it again.

`ModelUtil.loadOnServer(<tag>,<filename>)`: The `loadOnServer` method works like `ModelUtil.load` except that the filename is a path on the server computer. The client does not have to have access to the file.

`ModelUtil.loadProtected(<tag>,<filename>,<password>)`: The `loadProtected` method works like `ModelUtil.load` but with password protection.

`ModelUtil.loadProtectedOnServer(<tag>,<filename>,<password>)`: The `loadProtectedOnServer` method works like `ModelUtil.loadOnServer` but with password protection.

`ModelUtil.loadRecovery(<tag>,<foldername>)`: The `loadRecovery` method loads a model from a recovery directory or folder structure in the client's file system and names it `<tag>`.

`ModelUtil.showPlots(bool)`: The `showPlots` method applies when connected to a graphics server, and `ModelUtil.showPlots(false)` will disable plotting. It will not close any existing plot windows. Use `ModelUtil.closeWindow(<tag>)` or `ModelUtil.closeWindows()` to do that.

`ModelUtil.showProgress(bool)`: The `showProgress` method with a boolean input turns on or off showing of progress in a window or on a file when running lengthy tasks when connected to a server. The return value is a boolean value that is true if showing progress is possible.

`ModelUtil.showProgress(<filename>)`: The `showProgress` method with a filename input turns on logging of progress to the file `<filename>` in the client's file system. If `<filename>` is `null` progress is logged to the standard output.

`ModelUtil.initStandalone(bool)`: The `initStandalone` method initializes the environment for using the COMSOL API in model files for Java[®]. You should *not* use this command from the LiveLink[™] for MATLAB[®]. Set the argument to true if support for plotting in a GUI using Java Swing widgets should be available.

`ModelUtil.initStandalone(bool, <guiToolkit>)` allows to specify that support for using a given Java GUI toolkit should be available. The optional `<guiToolkit>` parameter can have the values "swing" or "swt" telling that Swing widgets or widgets from the Standard Widget Toolkit (SWT) can be used.

`ModelUtil.getPreference(<prefsName>)`: The `getPreference` method returns the value of a preference.

`ModelUtil.setPreference(<prefsName>, <value>)`: The `setPreference` method sets the value of a preference.

`ModelUtil.listPreferences()`: The `listPreferences` method returns a string with a listing of the preferences names and their descriptions.

`ModelUtil.loadPreferences()`: The `loadPreferences` method loads the preferences from file. Use this in model files for Java, which do not load the preferences at launch time.

`ModelUtil.savePreferences()`: The `savePreferences` method saves the preferences to file.



The preferences are also saved when you close the **Preferences** window and when you exit COMSOL Desktop. When you run a thin client, preferences are saved when you call `ModelUtil.disconnect()` (see [Client-Server Commands](#) below).

`ModelUtil.uniquetag(String prefix)`: The `uniquetag` method returns a unique model tag with a prefix that the tag should begin as an input argument.

`ModelUtil.modelsUsedByOtherClients()`: The `modelsUsedByOtherClients` method returns the tags of models used by other clients.

`ModelUtil.blockOtherClients(boolean blockOtherClients)`, which blocks other clients from a server. This command is only meaningful when there are multiple clients connected to the server. Calling `ModelUtil.blockOtherClients(true)` will block other clients. The blocking is lifted when clients requesting the block call `ModelUtil.blockOtherClients(false)` or when it disconnects from the server.

`ModelUtil.ClearCache()`, which clears the memory and disk cache (containing, for example, geometries, meshes, rendering data, and plots).

`ModelUtil.setCacheMaxSize(long fileCache, long memoryCache)`: Set the maximum size used for caching (for example, geometries, meshes, rendering data, and plots). `fileCache` is the maximum number of megabytes (MB) used for caching in the temp file area, and `memoryCache` is the maximum number of megabytes (MB) used for caching in memory. Note that when running memory-demanding operations in the COMSOL Multiphysics software, the memory cache is automatically swapped to disk.

`getCacheMaxSizeMemory()` returns, as a long data type, the maximum amount of memory in megabytes (MB) used for caching, for example, geometries, meshes, and rendering data.

`getCacheMaxSizeDisk()` returns, as a long data type, the maximum amount of disk in megabytes (MB) used in the temp area for caching, for example, geometries, meshes, and rendering data.

License Commands

`ModelUtil` provides functionality to check availability for and control the checkout of COMSOL product licenses.

`ModelUtil.hasProduct(String... product)`: The `hasProduct` method checks if the current license allows to run the specified COMSOL products given as the input (as an array of strings).

`ModelUtil.hasProductForFile(String file)`: The `hasProductForFile` method checks if the current license allows the specified COMSOL products needed to use that COMSOL MPH file.

`ModelUtil.hasProductForFileonServer(String file)`: The `hasProductForFileonServer` method is similar to `hasProductForFile` but checks if the license allows the specified COMSOL products needed for a file on the server.

`ModelUtil.checkoutLicense(String... product)`: The `checkoutLicense` method checks out licenses for the COMSOL products given as the input (as an array of strings).

`ModelUtil.checkoutLicenseForFile(String file)`: The `checkoutLicenseForFile` method checks out the licenses needed to use that COMSOL MPH file.

`ModelUtil.checkoutLicenseForFileonServer(String file)`: The `checkoutLicenseForFileonServer` method is similar to `checkoutLicenseForFile` but checks out the licenses needed to use that COMSOL MPH file on the server.

The following table lists the available products for which licenses can be checked for availability and checked out using the names in the **Name** column:

PRODUCT	NAME
AC/DC Module	ACDC
Acoustics Module	ACOUSTICS
Battery Design Module	BATTERYDESIGN
CAD Import Module	CADIMPORT, CADREADER
CFD Module	CFD
Chemical Reaction Engineering Module	CHEM
Cluster computing functionality	CLUSTERNODE
Composite Materials Module	COMPOSITEMATERIALS
Corrosion Module	CORROSION
Design Module	DESIGN
ECAD Import Module	ECADIMPORT
Electric Discharge Module	ELECTRICDISCHARGE
Electrochemistry Module	ELECTROCHEMISTRY
Electrodeposition Module	ELECTRODEPOSITION
Fatigue Module	FATIGUE
File Import for CATIA V5	CATIA5
Fuel Cell & Electrolyzer Module	FUELCELLANDELECTROLYZER
Geomechanics Module	GEOMECHANICS
Granular Flow Module	GRANULARFLOW
Heat Transfer Module	HEATTRANSFER
Liquid & Gas Properties Module	LIQUIDANDGASPROPERTIES
LiveLink™ for AutoCAD®	LLAUTOCAD
LiveLink™ for PTC® Creo® Parametric™	LLCREOPARAMETRIC

PRODUCT	NAME
LiveLink™ for Excel®	LLEXCEL
LiveLink™ for Inventor®	LLINVENTOR
LiveLink™ for MATLAB®	LLMATLAB
LiveLink™ for Revit®	LLREVIT
LiveLink™ for Solid Edge®	LLSOLIDEDGE
LiveLink™ for SOLIDWORKS®	LLSOLIDWORKS
MEMS Module	MEMS
Microfluidics Module	MICROFLUIDICS
Mixer Module	MIXER
Molecular Flow Module	MOLECULARFLOW
Multibody Dynamics Module	MULTIBODYDYNAMICS
Nonlinear Structural Materials Module	NONLINEARSTRUCTMATERIALS
Optimization Module	OPTIMIZATION
Particle Tracing Module	PARTICLETRACING
Pipe Flow Module	PIPEFLOW
Plasma Module	PLASMA
Polymer Flow Module	POLYMERFLOW
Ray Optics Module	RAYOPTICS
RF Module	RF
Rotordynamics Module	ROTORDYNAMICS
Semiconductor Module	SEMICONDUCTOR
Structural Mechanics Module	STRUCTURALMECHANICS
Subsurface Flow Module	SUBSURFACEFLOW
Uncertainty Quantification Module.	UQ
Wave Optics Module	WAVEOPTICS

Client-Server Commands

ModelUtil provides functionality to control COMSOL client-server options. You can connect/disconnect and control connections from multiple clients to a server using, for example, the connect and disconnect methods.

ModelUtil.connect() connects to a COMSOL server (COMSOL Multiphysics server or COMSOL Server™). The COMSOL command arguments -Dcs.host=<host> and -Dcs.port=<port> can provide the hostname and port number. In case those are not provided, and the both client and server access the same file system, the host and port can be automatically transferred.

ModelUtil.connect(<host>, <port>) connects to a COMSOL server. The arguments <host> and <port> provide the hostname (a string) and port number (an integer) for the COMSOL server.

ModelUtil.connect(<host>, <port>, <user>, <password>) connects to a COMSOL server. The arguments <host>, <port>, <user> and <password> provide the hostname (a string), port number (an integer), user (a string), and password (a string) for the COMSOL server.

ModelUtil.connect(<host>, <port>, <encryption>) and

ModelUtil.connect(<host>, <port>, <encryption>, <user>, <password>) connects to a COMSOL server using encryption. With the boolean <encryption> set to false, it uses the ws WebSocket URI scheme; when set to true, it uses the secure wss (SSL) WebSocket URI scheme.

ModelUtil.disconnect() disconnects from a COMSOL server.

`ModelUtil.setServerBusyHandler(<ServerBusyHandler>)`: Use the `setServerBusyHandler` method to register a `ServerBusyHandler`. Several clients can be connected to the same server, but only one client at a time can ask the server to perform an operation. The default behavior when attempting to call the server when it is busy is to issue an exception. By registering a `ServerBusyHandler` it is possible to make the client wait until the server is free again, and to set a time-out for how long to wait.

`ServerBusyHandler` is a class which controls how long a client waits for a busy server to become free again. It also has hooks to perform any action on the client side before starting to wait and just after stopping to wait. Implement a subclass in Java to `ServerBusyHandler` to change the default behavior.

`ServerBusyHandler()` creates a server-busy handler that waits for the server to be free without any time-out.

`ServerBusyHandler(<timeOut>)` creates a server-busy handler that waits for the server to be free. The time to wait, `<timeOut>`, is given in milliseconds.

`postWaitForServer(<boolean>)` is a hook to perform any action right after waiting for the server to become free.

`preWaitForServer(<host>, <username>)` is a hook to perform any action before beginning to wait for the server to become free. The inputs are the hostname and username as strings.

`ModelUtil.setModelChangedHandler(<ModelChangedHandler>)`: The `setModelChangedHandler` method registers a handler of changes to models made by other clients. If any other client changes any model in use by this client, the model change handler is notified.

`ModelChangedHandler` is an interface for handling updates of the client when another client has modified any models in use by this client. Implement the interface in Java to change the default behavior.

`handleModelChangeOnServer(<modelChangeInfo>)` is called when another client has changed any models in use by this client. The `ModelChangedHandler` interface includes the boolean method `triggeredByChangesFromThisClient()`. It has a default implementation returning false. In this case, the method `handleModelChangeOnServer(<modelChangeInfo>)` is only called when another client changes the model. If `triggeredByChangesFromThisClient()` returns true then `handleModelChangeOnServer(<modelChangeInfo>)` is also called when the client registering the `ModelChangedHandler` changes the model.

`ModelChangeInfo` is a class with information about changes to models by other clients.

`getModelTags()` returns the tags of the models that have changed.

The scanModel Method

The `scanModel` method is an advanced tool that scans the raw data of a model file for a certain node type and collects the tag and label of all nodes that it finds. See [Scanning Models](#) for more information.

model

Model object methods that set up basic parts of a model object such as the model history and saving model files.

SYNTAX

`model` is a model object that you can create, for example, using `ModelUtil.create(<tag>)`.

`model.baseSystem(<system>)`: The `baseSystem` method sets the unit system for the entire model to the given system. The default is the SI system, which has the tag SI. Other supported unit systems are `bft` (British engineering units), `cgs`, `mpa`, `emu`, `esu`, `fps`, `ips`, and `psi`.

`model.clearThumbnail()`: The `clearThumbnail` method clears the model thumbnail image.

`model.dateModified()`: The `dateModified` method returns the modification date of the model.

Use `model.description(description)` and `description = model.description()` to set and retrieve the model's description.

`model.disableUpdates()`: The `disableUpdates` method returns the current status of the disable state for the model object.

`model.disableUpdates(boolean)` Temporarily disables and reenables the update of variables in entities that automatically generates other entities (for example, `physics` or `coordSystem`). Disable updates to speed up the evaluation of long execution sequences. Leaving this flag disabled can cause strange side effects during modeling. For example, some parameter values in a feature of a physics interface might not be valid until an update has been made. The model inputs are such parameters, which end with the suffix `_src`. Trying to set a value to any of these parameters with updates disabled might give an error message. Other effects are that the generated variables are unknown to the unit evaluator and equation view readings can be incomplete. When the disabled state goes from `true` to `false`, the program performs a full update of the variables, so the model is in a fully functional state.

Use `model.excludeComputedDataInMph(boolean exclude)` to control whether to exclude the built, computed, and plotted data when saving model files. Use `model.excludeComputedDataInMph()` to return the current value.

Use `model.excludeComputedDataInDrafts(boolean exclude)` to control whether to exclude built, computed, and plotted data when saving the model as a draft version to a database. Use `model.excludeComputedDataInDrafts()` to return whether to exclude such data when saving the model as a draft version to a database.

Use `model.excludeComputedDataInRegular(boolean exclude)` to control whether to exclude built, computed, and plotted data when saving the model as a regular version to a database. Use `model.excludeComputedDataInRegular()` to return whether to exclude such data when saving the model as a regular version to a database.

`model.fontFamily(<family>)`: The `fontFamily` method sets the font family to be used in plots. The font default is always available. If using Windows, most system fonts can also be used.

`model.fontSize(<size>)`: The `fontSize` method sets the font size to be used in plots.

`model.getComsolVersion()`: The `getComsolVersion` method returns the COMSOL Multiphysics version used to save the model or the current version if the model has never been saved. There is also a `ModelUtil.getComsolVersion()` method, which returns the current version as a string.

`model.getFilePath()` returns the absolute path of the model or an empty string if the model has not been saved or if the model is loaded from a database.

`model.getLastComputationTime(String time_format)`: The `getLastComputationTime` method returns the last computation time for the model or application as a string. Use `model.getLastComputationTime()` to get the time measured in ms, which you can then use as an input to `model.setLastComputationTime`. Other supported time formats are "hr:min:sec", "h:min:s", and "detailed", which returns the time in seconds and also includes more readable units for longer times.

`model.getUsedProducts()`: The `getUsedProducts` method returns the products that this model uses.

`model.isReadOnly()`: The `isReadOnly` method returns whether the file where the model is saved is read-only and cannot be overwritten or not. The file can be read-only for two reasons:

- The COMSOL process does not have permission to write to the file.
- On Windows, the file can be locked by another COMSOL instance.

If the model has not been saved, this method returns false.

`model.lastModifiedBy()`: The `lastModifiedBy` method returns the last user to modify the model.

`model.location()`: The `location` method returns a string that contains the location of the saved model. For a model saved on the file system, the value is the absolute path of the model file. For a model saved in a Model Manager database, the return value is the model's database location URI. If the model has never been saved, the `location` method returns `null`. The return value of the `location` method is what can be provided as-is to, for example, `ModelUtil.load(String tag, String modelLocation)` to load a model from either the file system or a database.

`model.locationUri()`: The `locationUri` method returns a database location URI for the save location of the model. If the model has never been saved, the `location` method returns `null`.

`model.modelPath(<path>)`: The `modelPath` method sets the model path. The model path is used for reading files required by the model, if no path is provided to the file. `<path>` is a list of directories separated by semicolon. When reading an external file, COMSOL Multiphysics attempts to find a file in the following locations:

- 1 The absolute path as given in the filename. If the path given in the filename is relative, it is resolved relative to the following directories.
- 2 The model directory, if provided.
- 3 If searching for a geometry part, the user part libraries. These are given by the preference property `geometry.library.userpartlibraries`.
- 4 If searching for a geometry part, the COMSOL Multiphysics installation's parts directory.
- 5 The directories defined by `model.modelPath` (ordered and semicolon separated).
- 6 The directories in the `cs.path` setting (ordered and semicolon separated).
- 7 The current directory, which is given by the Java system property `user.dir` and is the directory where you launch COMSOL Multiphysics, unless you have changed the value of `user.dir`.

The model directory is used for saving and exporting files if you do not provide an absolute path to the file.

`model.modelPath()` returns the path `<path>` set in a call to `model.modelPath(<path>)`.

`model.resetLabelsToDefault()` resets all node labels to their default values.

`model.save(<filename>)`. The `save` method saves the model as a multiphysics model file in `<filename>`. If you do not provide a path, the model is saved in the directory from where you have launched COMSOL Multiphysics or is automatically saved, if loaded from a database, as a draft version to that database.

`model.save(<filename>, <type>)` saves the multiphysics model in `<filename>`. If the type is `java`, a model file for use with Java[®] is saved. If the type is `m`, this command saves a model file as an M-file for use with the LiveLink[™] for MATLAB[®]. If the type is `vba`, this command saves a model file as a VBA-file for use with VBA (Visual Basic for Applications) in Microsoft Excel[®].

`model.save(<path>, <saveCopy>)` saves a copy of the model, if the boolean argument `<saveCopy>` is `true`. If it is set to `false`, this call results in a normal save operation. When saving as a copy, the location of that copy is not remembered, so the previous location for saving models is retained.

`model.setLastComputationTime(long time)`: The `setLastComputationTime` method sets the last computation time for the model or application as the measured computation time (in ms) that you provide as the input.

`model.setThumbnail(<image_filename>)`: The `setThumbnail` method imports the image file at the given path and sets it as the model's thumbnail image.

Use `model.title(<title>)` and `title = model.title()` to set and retrieve the model's title.

Methods for Compact History

`model.hist().complete(bool)` enables or disables history logging for methods where the arguments typically are very large objects. This setting affects some methods of `model.sol("sol")`. All methods with a name beginning with `setU` are affected by the `model.hist().complete(bool)` setting. The default for the completeness is false. The compact history operation is not affected by the completeness setting.

`model.hist().isComplete()` returns true if history logging is enabled for methods where the arguments typically are very large objects.

`model.hist().disable()` Disables logging of top-level API calls to the history. Use this method sparingly; the default is that the history is logged.

`model.hist().enable()` Removes the most recent disabling of top-level API calls to the history. Calling `enable()` can be viewed as removing an entry from a stack of disabled records; logging only occurs if the stack is empty. If you call `disable()` twice, you then have to call `enable()` twice to enable history generation.

`model.resetHist():` Compact History. The `resetHist()` method reconstructs the model history from scratch to generate a compacted model's Java- or M-file history (that is, creating a compact history). If the model has errors or invalid property values, the method fails and the old history is kept.

SEE ALSO

[model.modelNode\(\)](#), [model.unitSystem\(\)](#)

model.attr()

Model entity list methods such as copying, duplicating, clearing, and removing model entities.

SYNTAX

`model.attr()` returns a *model entity list*. The string *attr* denotes a method name for accessing the model entity list.

`model.attr().clear()` removes all tagged model entities.

`model.attr().copy(<tag>, <copytag>)` creates a new model entity with the tag *<tag>*, which is a copy of the model entity with the tag *<copytag>*. The *<copytag>* should be combination of tags separated by slashes to uniquely identify the entity. For example, `pg1/surf1/htgh1` identifies

`model.result("pg1").feature("surf1").feature("htgh1")`. How to interpret the combined tag depends on the context. The difference between `duplicate` and `copy` is that `copy` can use a source anywhere in the model, whereas `duplicate` requires that the source is in the same list. Not all model entities support the `copy` operation. The difference between `copy` and `copyTo` is that `copyTo` copies the entity to a specific position in the list, whereas `copy` copies to a default position in the list. Not all model entities support the `copyTo` operation.

`model.attr().copy(<tag>, <copytag>, <modeltag>)` creates a copy and assigns it to the model *<modeltag>*.

`model.attr().copyTo(<tag>, <copytag>, <insertafter>)` creates a copy and inserts it in the list after the entity with tag *<insertafter>*. If *<insertafter>* is an empty string, the entity is inserted first in the list. Not all model entities support the `copyTo` operation.

`model.attr().duplicate(<tag>, <copytag>)` creates a new model entity with the tag *<tag>* which is a duplicate of the model entity with tag *<copytag>*. Not all model entities support the `duplicate` operation.

`model.attr().duplicateTo(<tag>, <copytag>, <insertafter>)` creates a new model entity and inserts it in the list after the entity with tag *<insertafter>*. If *<insertafter>* is an empty string, the entity is inserted first in the list. Not all model entities support the `duplicateTo` operation.

`model.attr().get(<tag>)`. The `get` method returns the entity with tag `<tag>` from the entity list `model.attr()`.

`model.attr().remove(<tag>)`. The `remove` method removes the model entity with tag `<tag>`.

`model.attr().size()`. The `size` method returns the number of model entities.

`model.attr().tags()`. The `tags` method returns a string array with the tags of all model entities.

`model.attr().uniquetag(<tag>)`. The `uniquetag` method returns a unique tag in the list context.

SEE ALSO

[model](#)

model.attr(<tag>)

Model entity methods for adding and accessing name, tag, version, comments, date created, and the author information for a model entity.

SYNTAX

`model.attr(<tag>)` returns a *model entity* with tag `<tag>`. The string `attr` denotes a method name for accessing a model entity with tag `<tag>`.

`model.attr(<tag>).active(bool)` makes the entity with tag `<tag>` active or inactive.

`model.attr(<tag>).author()` returns the author of the entity.

`model.attr(<tag>).author(<author>)` sets the author of the entity.

`model.attr(<tag>).comments()` returns the comments of the entity.

`model.attr(<tag>).comments(<comments>)` sets the comments of the entity.

`model.attr(<tag>).dateCreated()` returns the creation date of the entity.

`model.attr(<tag>).isActive()` returns true if the entity with tag `<tag>` is active.

`model.attr(<tag>).label()` returns the label of the entity.

`model.attr(<tag>).label(<label>)` sets the label of the model entity. The label is an arbitrary nonempty string.

`model.attr(<tag>).resetAuthor(<author>)` sets the author of the entity and all its children. In particular, when used on the model itself, the method sets the author on all model entities of the model.

`model.attr(<tag>).tag()` returns the tag of the entity.

`model.attr(<tag>).tag(<newtag>)` assigns the new tag `<newtag>` to the entity `<tag>`.

`model.attr(<tag>).timeCreated()` and `model.attr(<tag>).timeModified()` return the creation time of the entity and the time when the entity was last modified, respectively. The times are reported in milliseconds since January 1, 1970, 00:00:00 GMT.

`model.attr(<tag>).version(<version>)` sets the version of the entity. The version is a user-defined string.

`model.attr(<tag>).version()` returns the version of the entity.

`model.attr(<tag>).help()` and `model.attr(<tag>).help(string)`, where `string` is the name of a type within the model object, return a query URL string for looking up HTML documentation help text for the model entity of the given type using a COMSOL Documentation server running either locally or online at doc.comsol.com.

`model.attr(<tag>).docMarker()` and `model.attr(<tag>).docMarker(string)`, where *string* is the name of a type within the model object, return the topic key for the model entity of the given type to use as the argument to the public static method `com.comsol.doc.client.DocRemoteClient.showHelp(String)` of the COMSOL Documentation application, which shows documentation with the help of a COMSOL Documentation server. If the COMSOL Documentation application is closed, it will automatically relaunch the next time you call the `showHelp(String)` method. If desired, it is possible to close a COMSOL Documentation application launched in this way programmatically by calling the method `com.comsol.doc.client.DocRemoteClient.shutdown()`.

SEE ALSO

[model](#)

model.batch()

Create batch jobs.

SYNTAX

Jobs

`model.batch().create(<tag>, jobtype)`; creates a batch job tagged *<tag>* of type *jobtype*, where *jobtype* is Parametric, Batch, or Cluster.

`model.batch().remove(<tag>)` removes a batch job.

`model.batch().size()` returns number of batch jobs.

`model.batch().tags()` returns the tags of the batch jobs.

`model.batch(<tag>).attach(<stag>)` attaches a batch job with tag *<tag>* to a study with tag *<stag>*, which makes it visible under that study.

`model.batch(<tag>).create(<jtag>, <oper>)` creates a batch job sequence.

`model.batch(<tag>).detach(<stag>)` detaches a batch job from a study with tag *<stag>*.

`model.batch(<tag>).remove(<ttag>)` removes the task.

`model.batch(<tag>).run()` runs the batch job. The run method can take an additional boolean input argument `createPlots`, which, when set to true, generates the corresponding default plots when computing a solution.

`model.batch(<tag>).run(<type>)` runs the attached job sequence without creating plots. The argument *<type>* is one of `compute`, `postprocess`, or `verify`. When the *<type>* argument is `compute` it is equivalent to calling `run()`. For an Uncertainty Quantification job, `postprocess` generates the response surface for the surrogate functions and `verify` computes verification data and verifies the surrogate functions. For all other job configurations, no action is performed when the *<type>* argument is `postprocess` or `verify`.

`model.batch(<tag>).set(jprop, <jvalue>)` sets the property *jprop* to the value *<jvalue>*.

`model.batch(<tag>).study(<stag>)` assigns a batch job to a study tag *<stag>*.

`model.batch(<tag>).study()` returns the study tag of batch job with tag *<tag>*.

Batch Job Properties

The Parametric job type has the following properties:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
accumtable	String	new	Accumulated probe table.
accumtableall	on off	off	Use all probes for the accumulated probe table.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
checkpoint	manual end	manual	Choose after how many parameters to insert a checkpoint or add no checkpoints at all.
checkpointfreq	double	4	After how many parameters to add a checkpoint per group.
control	String	user	Controlling study.
param	String array		Name of parameter and its value (output).
pdistrib	on off	off	Distributed (in parallel) the parameter values.
pname	String array		Parameter name(s) to vary.
plist	String array		Parameter values.
plot	on off	off	Update a plot group while solving.
plotgroup	String	default	Update this plot group while solving.
pwork	int	1	Limit for the number of work groups.
pworkactive	on off	off	Use a limit for the number of work groups.
stopcond	String		A stop condition expression.
err	on off	off	Stop sweep if error.
error	String array		The logged error.
errignore	true false	false	Allow the sequence to continue even if an error is encountered.
useaccumtable	on off	off	Produce an accumulated probe table while solving.

The Optimization job type sets its property through the Optimization study node, which has the following properties:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
optobj	String		Objective function to be optimized.
descr	String		Description of optimization objective function.
objectivetype	minimization maximization	minimization	Sets whether the objective should be minimized or maximized.
objectivesolution	auto first last sum min max	auto	Determines how the objective should be evaluated for studies with more than one available PDE solution, for example, time-dependent problems.
pname	String array		Names of control parameters.
initval	String array		Initial values for control parameters.
lbound	String array		Lower bounds on control parameters.
ubound	String array		Upper bounds on control parameters.
optsolver	coordsearch montecarlo neldermead bobyqa	neldermead	Optimization solver.
useseed	on off	off	Use random seed for Monte Carlo solver.
randseed	int	0	Random seed for Monte Carlo solver.
nsolvemax	int	1000	Maximum number of objective evaluations.
opttol	double	1e-2	Optimization tolerance.
useobjtable	on off	off	Produce a table with all objective evaluations.
objtable	String	new	Reference to table with objective evaluations.
convinfo	off on detailed	on	Detail of log messages from optimization solver.

The Batch job type has the following properties:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
control	String	user	Name of controlling study.
np	integer	auto	Number of cores to use.
graphics	on off	off	Enable graphics.
maxallow	integer	1	Maximum allowed number of batch jobs to start simultaneously.
maxrestart	integer	0	Maximum number of restarts before a batch job is failed.
maxalive	integer	300	Maximum number of seconds before the batch job must say it is running.
starttime	now 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23	now	The time, as an hour for a 24-hour clock, when the batch job should start.
batchmodelstoragemode	filesystem database sameascurrentmodel	filesystem	Sets the storage mode for generated models during a batch run.
batchdatastoragemode	filesystem database sameascurrentmodel	filesystem	Sets the storage mode for generated batch data during a batch run.
batchdir	String	Home directory	The directory to store files used by the batch job.
client	on off	off	Run the batch job as client.
port	integer	2036	The host port number.
host	String	localhost	Name of host.
batchfile	String	batchmodel.mph	Name of batch model file.
clear	on off	on	Clear the previous model file.
clearmesh	on off	off	Clear meshes before saving model.
clearsolution	on off	off	Clear solutions before saving model.
savefile	on off	on	Save model after run.
specbatchdir	on off	off	Specify different directory for batch process than used by the current process.
rundir	String	Home directory	The directory used by the batch job when specbatchdir is on.
speccomsoldir	on off	off	Specify different directory for the COMSOL installation than used by the current process.
comsoldir	String	COMSOL installation directory	The COMSOL installation directory used by the batch job when speccomsoldir is on.
synchsolutions	on off	off	Synchronize solutions after batch job finishes.
synchaccumprobetable	on off	off	Synchronize accumulated probe tables after batch job finishes.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
probesel	all none manual	all	The probes to compute.
probes	String array		Probes to compute.
useaccumtable	on off	off	Use the accumulated probe table.
accumtable	String	new	Name of table to use.
accumtableall	on off	on	Use all probes.
client	on off	off	Run as client.
host	String	localhost	Name of server.
port	integer		Server port number.

The Cluster job type has the following properties:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
batch	String		Tag of batch job to run.
clustertype	general whpc2008 wccs2003 sge slurm none	general	The type of cluster job.
control	String	user	Name of controlling study.
corespernode	integer	0	Minimum number of cores per node on whpc2008.
exclusive	on off	on	Demand exclusive right to nodes on whpc2008 and SLURM.
filetransfercmd	none scp user	none	Command to transfer files.
filetransferfromusercmd	String		Command to transfer files from remote location.
filetransfertousercmd	String		Command to transfer files to remote location.
hostfile	String		Path to hostfile.
memorypernode	integer	0	Minimum amount of memory per node on whpc2008 and SLURM.
mpd	on off	off	If an mpd is running on the computer or not.
mpiargs	String		Additional MPI arguments.
mpibootstrap	String		Name of bootstrap server.
mpirsh	String		Path to rsh or ssh.
nn	integer	1	Number of processes to start.
nodegran	node socket core	node	Node granularity on whpc2008.
nodegroup	String		Name of Named selection of Compute nodes in whpc2008.
perhost	integer	1	Number of processes / host.
postcmd	String		DOS/Linux command to execute after the batch job finished.
precmd	String		DOS/Linux command to execute prior to the batch job.
priority	Highest AboveNormal Normal BelowNormal Lowest	Normal	Priority of job on wccs2003 and whpc2008.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
remote	on off	off	Run on remote server.
remotecmd	none ssh user	none	Command to use when invoking a command on a remote server.
remotehosts	String		List of remote hostnames.
remoteos	native windows linux	native	OS used on remote hosts.
remoteusercmd	String		Command to run on remote server.
reqnodes	String array		Requested nodes on wccs2003, whpc2008, and SLURM.
runtime	DD:HH:MM Infinite	Infinite	Maximum time to run before stopping on wccs2003, whpc2008, and SLURM.
schedargs	String		Additional scheduler arguments.
scheduler	String	localhost	Name of the scheduler on wccs2003, whpc2008, and SLURM.
scpargs	String		Additional SCP arguments.
scpcmd	scp putty user	scp	SCP command.
scpkey	String		SCP key file.
scppath	String		Directory where SCP resides.
scpuser	String		Username used by SCP.
scpusercmd	String		Command for copying files to remote location.
sgegran	host slot manual	host	Node granularity on SGE.
sgenn	integer	1	Number of slots in SGE.
sgepriority	integer	0	Priority of job on SGE and SLURM.
sgequeue	String		Name of SGE and SLURM queue.
sshargs	String		Additional SSH arguments.
sshcmd	ssh putty user	ssh	SSH command.
sshkey	String		SSH key file.
sshpath	String		Directory where SSH resides.
sshporthost	String		Port host.
sshports	String		Ports that should be forwarded by SSH.
sshuser	String		Username used by SSH.
sshusercmd	String		User-defined SSH command.
user	String		Username on wccs2003, whpc2008, and SLURM.

Tasks

`model.batch(<tag>).create(<ttag>, tasktype);` creates a task of type *tasktype* tagged *<ttag>*. Find options for *tasktype* in [Table 2-5](#) below.

TABLE 2-5: BATCH TASK TYPE OPTIONS.

TASK TYPE	DESCRIPTION
Geomseq	A geometry sequence to build.
Meshseq	A meshing sequence to build.
Solutionseq	A solver sequence to compute.
Jobseq	A job sequence to run.

TABLE 2-5: BATCH TASK TYPE OPTIONS.

TASK TYPE	DESCRIPTION
Postseq	A post sequence to run.
Evalnumericalseq	A numerical results seq (derived value) to run (replaced Numericalseq).
Numericalseq	A numerical results seq to run (deprecated).
Exportseq	An export sequence to run.
Save	Saves the state of the model at this point in the job sequence.
Class	Runs the main function of a compiled class with the system property <code>cs.currentmodel</code> set to the name of the model calling the class.
Data	Created by batch jobs to store external process information.

Task Type Properties

`model.batch(<tag>).feature(<ttag>).set(ttprop,<tpvalue>)` sets the task type property `ttprop` to the value `<tpvalue>`.

Task type properties can have the values listed in [Table 2-6](#).

TABLE 2-6: TASK TYPE PROPERTY VALUES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
clear	on off	on	Clear the currently stored data.
filename	String		Name of file to store or open.
openfile	String array	none	Name of file that was saved.
param	String array		Name of parameter and its value.
files	String array		Name of files for each parameter.
input	String array		Input to class file.
seq	String	all	Name of sequence to run.
num	String array		Name of numerical result feature that generated value.
paramvalue	String array		Computed numerical result.
store	on off	off	Copy solution.
psol	String	none	Tag of solver sequence where solutions are stored.

The Data Task Type

The Data task type contains child nodes with process information of type Process; see [Table 2-7](#).

TABLE 2-7: DATA CHILD NODES.

TASKTYPE	DESCRIPTION
Process	Contains information about running processes.

`model.batch(<tag>).feature(<ttag>).feature(<ptag>).set(ptype,<pvalue>)` sets the property `ptype` to the value `<pvalue>`. `ptype` can have the values listed in [Table 2-8](#)

TABLE 2-8: PTYPE PROPERTY VALUES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
cmd	String		The command that started the external process.
filename	String		Name of file where model is stored.
operation	update progress cancel stop clear rerun	update	Name of operation to perform on the process.
status	String		Current status of the process.

EXAMPLE

Create a parametric sweep over a geometry sequence that creates a batch job that runs a parametric sweep that runs a solver.

Code for Use with Java

```
model.batch().create("sweep1", "Parametric");
model.batch("sweep1").set("pname", "a");
model.batch("sweep1").set("plist", new double[]{1,2});
model.batch("sweep1").create("sol", "Solutionseq");
model.batch("sweep1").feature("sol").set("seq", "sol3");
model.batch().create("batch1", "Batch");
model.batch("batch1").create("task", "Jobseq");
model.batch("batch1").feature("task").set("seq", "sweep1");
model.batch().create("sweep2", "Parametric");
model.batch("sweep2").set("pname", "b");
model.batch("sweep2").set("plist", new double[]{1,2,3});
model.batch("sweep2").create("gtask", "Geomseq");
model.batch("sweep2").feature("gtask").set("seq", "geom1");
model.batch("sweep2").create("task", "Jobseq");
model.batch("sweep2").feature("task").set("seq", "batch1");
model.batch("sweep2").run();
```

Determine the parameter names and values from a parametric sweep that has already been run.

```
model.batch(pname).feature(fname).getString("psol")
```

where `pname` is the name of the parametric sweep feature that ran and `fname` is the name of the solution feature that stored the solutions. Use

```
model.sol(sname).feature().tags()
```

to find out the tags of the stored solutions. Use

```
model.sol(sname).feature(fname).getString("sol")
```

to find the solver sequence for a parameter. Use

```
model.sol(sname).getParamNames()
```

and

```
model.sol(sname).getParamVals()
```

Code for Use with MATLAB

```
model.batch.create('sweep1', 'Parametric');
model.batch('sweep1').set('pname', 'a');
model.batch('sweep1').set('plist', [1,2]);
model.batch('sweep1').create('sol', 'Solutionseq');
model.batch('sweep1').feature('sol').set('seq', 'sol3');
model.batch.create('batch1', 'Batch');
model.batch('batch1').create('task', 'Jobseq');
model.batch('batch1').feature('task').set('seq', 'sweep1');
model.batch.create('sweep2', 'Parametric');
model.batch('sweep2').set('pname', 'b');
model.batch('sweep2').set('plist', [1,2,3]);
model.batch('sweep2').create('gtask', 'Geomseq');
model.batch('sweep2').feature('gtask').set('seq', 'geom1');
model.batch('sweep2').create('task', 'Jobseq');
model.batch('sweep2').feature('task').set('seq', 'batch1');
model.batch('sweep2').run;
```

Determine the parameter names and values from a parametric sweep that has already been run.

```
model.batch(pname).feature(fname).getString('psol')
```

where `pname` is the name of the parametric sweep feature that ran and `fname` is the name of the solution feature that stored the solutions. Use

```
model.sol(sname).feature().tags
```

to find out the tags of the stored solutions. Use

```
model.sol(sname).feature(fname).getString('sol')
```

to find the solver sequence for a parameter. Use

```
model.sol(sname).getParamNames
```

and

```
model.sol(sname).getParamVals
```

SEE ALSO

[model.sol\(\)](#), [model.study\(\)](#)

model.bem()

Create a boundary element (BEM) model.

SYNTAX

```
model.bem().create(<tag>, "CoefficientPDE");
model.bem(<tag>).set(<prop>, <value>);
model.bem(<tag>).selection();
```

`model.bem(<tag>).selection();` defines the selection for single-sided BEM boundaries. In addition, the following variants are available for double-sided boundaries:

- Use `model.bem(<tag>).selection("cont")`; for a selection of double-sided boundaries where the field is continuous.
- Use `model.bem(<tag>).selection("discont")`; for a selection of double-sided boundaries where the field is allowed to be discontinuous.
- Use `model.bem(<tag>).selection("edge")`; for a selection of BEM edges in 3D.

For a complete list of methods available under `selection()`, see [Selections](#).

The following general properties are available for `model.bem`:

TABLE 2-9: GENERAL PROPERTIES FOR BEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
background	Expression	0	Background field.
edgefluxname	String		Name of edge flux variable.
edgegradname	String[]		Names of edge gradient variables.
edgeradius	Expression		Radius of cylinders represented as edges.
fluxname	String		Name of boundary flux variable.
infval	Expression	0	Value at infinity (for Laplace's equation).
normal	String[]		Boundary normals pointing out of BEM domain.
opname	String		Name of results-processing operator.
varname	String		Name of field variable.
varnameback	String		Name of field variable on backside of double-sided boundaries.
varnamefront	String		Name of field variable on frontside of double-sided boundaries.

In addition, the following properties for the coefficient of the equation are available:

TABLE 2-10: EQUATION COEFFICIENT PROPERTIES FOR BEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
a	String	0	Absorption coefficient.
a1	String[]	{ "0", "0", "0" }	Conservative flux convection coefficient.
be	String[]	{ "0", "0", "0" }	Convection coefficient.
c	String	1	Diffusion coefficient.
cedge	String	1	Diffusion coefficient in cylinders represented as edges.
m	String	0	Condition at infinity for Helmholtz equation.

The following integration order properties are available:

TABLE 2-11: INTEGRATION ORDER PROPERTIES FOR BEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
intorderclose	String		Integration rule for close nonadjacent pairs of mesh elements.
intorderedge	String		Integration rule for pairs of mesh elements with a common edge (3D only).
intorderfar	String		Integration rule for distant pairs of mesh elements.
intordersame	String		Integration rule for pairs of mesh elements that coincide.
intordervertex	String		Integration rule for pairs of mesh elements with a common vertex.
intorderweak	String		Integration for weak equations.

The following symmetry properties are available:

TABLE 2-12: SYMMETRY PROPERTIES FOR BEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sym1	off scp user	off	Use of symmetry plane orthogonal to x-axis.
sym1plane	Expression	0	Position of symmetry plane orthogonal to x-axis.
sym2	off scp user	off	Use of symmetry plane orthogonal to y-axis.
sym2plane	Expression	0	Position of symmetry plane orthogonal to y-axis.
sym3	off scp user	off	Use of symmetry plane orthogonal to z-axis.
sym3plane	Expression	0	Position of symmetry plane orthogonal to z-axis.

Finally, the following far-field approximation properties are available:

TABLE 2-13: FAR-FIELD APPROXIMATION PROPERTIES FOR BEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
dampingparameter	Double	1	Damping parameter.
farfielddapprox	none ACA ACA+	none	Far-field approximation method.
farfielddboxsplitlimit	Integer	20	Number of mesh elements at which box splitting stops.
farfielddmindist	Double	0.5	Minimum relative distance of boxes using far-field approximation.
farfielddsvd	on off	on	Use SVD compression in far-field approximation.
farfielddtol	Double	1e-3	Tolerance used in far-field approximation.
usedamping	on off	off	Use damping parameter with iterative solver.

EXAMPLE

The following example shows how to solve for a Helmholtz equation with outgoing waves at infinity in the exterior of a geometry. The example requires that the dependent variables `u` and `bemflux` already exist on the exterior boundaries.

Code for Use with Java

```
model.intRule().create("ir1", "material1");
model.intRule("ir1").create("o1").order(4);

model.bem().create("bem1", "CoefficientPDE");
model.bem("bem1").selection().geom("geom1", 2).set(<list of exterior boundary numbers>);
model.bem("bem1").set("varname", "u");
model.bem("bem1").set("fluxname", "bemflux");
model.bem("bem1").set("normal", new String[]{"-nx", "-ny", "-nz"});
model.bem("bem1").set("a", "-1");
model.bem("bem1").set("m", "-1");
model.bem("bem1").set("opname", "bemop");
model.bem("bem1").set("intorderfar", "ir1");
model.bem("bem1").set("intorderclose", "ir1");
model.bem("bem1").set("intordersame", "ir1");
model.bem("bem1").set("intorderedge", "ir1");
model.bem("bem1").set("intordervertex", "ir1");
model.bem("bem1").set("intorderweak", "ir1");
```

Code for Use with MATLAB

```
model.intRule().create('ir1', 'material1');
model.intRule('ir1').create('o1').order(4);

model.bem.create('bem1', 'CoefficientPDE');
model.bem('bem1').selection.geom('geom1', 2).set(<list of exterior boundary numbers>);
model.bem('bem1').set('varname', 'u');
model.bem('bem1').set('fluxname', 'bemflux');
model.bem('bem1').set('normal', {'-nx', '-ny', '-nz'});
model.bem('bem1').set('a', '-1');
model.bem('bem1').set('m', '-1');
model.bem('bem1').set('opname', 'bemop');
model.bem('bem1').set('intorderfar', 'ir1');
model.bem('bem1').set('intorderclose', 'ir1');
model.bem('bem1').set('intordersame', 'ir1');
model.bem('bem1').set('intorderedge', 'ir1');
model.bem('bem1').set('intordervertex', 'ir1');
model.bem('bem1').set('intorderweak', 'ir1');
```

SEE ALSO

[model.coeff\(\)](#)

model.coeff()

Creating equations in the coefficient form. See also [model.shape\(\)](#), [model.weak\(\)](#).

SYNTAX

```
model.coeff().create(<tag>,<fields>);
model.coeff(<tag>).field(<fields>);
model.coeff(<tag>).field(<pos>,<fields>);
model.coeff(<tag>).intRule(<irlist>);
model.coeff(<tag>).intRule(<pos>,<irule>);
model.coeff(<tag>).create(<ftag>);
model.coeff(<tag>).feature(<ftag>).set(<ctype>,<cvalue>);

model.coeff(<tag>).field();
model.coeff(<tag>).intRule();
model.coeff(<tag>).feature(<ftag>).getType(<ctype>);

model.coeff(<tag>).feature(<ftag>).set(String pname, int value);
model.coeff(<tag>).feature(<ftag>).set(String pname, int pos, int value);
model.coeff(<tag>).feature(<ftag>).set(String pname, int pos, int[] value);
model.coeff(<tag>).feature(<ftag>).set(String pname, int pos1, int pos2, int value);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, String value, int index);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, String value, int firstIndex,
    int secondIndex);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, String[] value, int index);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, double value, int index);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, double value, int firstIndex,
    int secondIndex);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, double[] value, int index);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, int value, int index);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, String value, int index);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, String value, int index);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, int value, int firstIndex,
    int secondIndex);
model.coeff(<tag>).feature(<ftag>).setIndex(String name, int[] value, int index);
```

The `set()` methods index/position arguments are 1-based. The `setIndex()` methods index/position arguments are 0-based. See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.coeff(<tag>)` returns the coefficient form equations with tag `<tag>`.

`model.coeff().create(<tag>,<fields>)` creates coefficient form equations with tag `<tag>` using the fields `<fields>`. The field tags refer to the fields defined by `model.field()`. The shape functions referred to by the fields are internally also used to find the derivatives of the field variables if converting the coefficient features to the weak formulation. By default, all coefficients are designed to be noncontributing to the equation under consideration. For example, `model.coeff().create("mytag",new String[] {"u","v"})`.

`model.coeff(<tag>).field(<fields>)` sets the coefficient form field variables. `<fields>` is a string with a field tag or a vector of field tags — for example, `new String[] {"u","v"}`. Reassigning the fields has the side effect that the size of the coefficients change if the number of field variables changes.

`model.coeff(<tag>).field(<pos>,<fields>)` edits the field at position `<pos>` in the field vector `<fields>`.

`model.coeff(<tag>).intRule(<irlist>)` assigns integration rules to the coefficient form equations. The list must have the same length as the number of field variables defined by the fields or have length 1. In the latter case all equations use the same integration rule. The number of field variables is not necessarily the same as the number of strings specified in `model.coeff(<tag>).field()`.

`model.coeff(<tag>).intRule(<pos>,<irule>)` edits the integration rule at position `<pos>` in the vector `<irule>`.

`model.coeff(<tag>).feature(<ftag>)` is a coefficient form feature with tag `<ftag>` in the coefficient form equations with tag `<tag>`.

`model.coeff(<tag>).create(<ftag>)` creates a new coefficient form feature with tag `<ftag>`.

`model.coeff(<tag>).feature(<ftag>).set(ctype,<cvalue>)` sets the value of the coefficient of type `ctype` to `<value>`. All string data types that are listed in [Table 2-4](#) are supported; which argument types are applicable depends on the coefficient. `ctype` is one of `c`, `al`, `ga`, `be`, `a`, `f`, `da`, `ea`, `q`, and `g`. These coefficients are available at all dimensions. In addition at level `edim==sdim-1`, the coefficients `q` and `g` are allowed, corresponding to `a` and `f`, respectively. All coefficients have a default 0 contribution.

`model.coeff(<tag>).feature(<ftag>).selection().named(<seltag>)` assigns the coefficient form equations to the named selection `<seltag>`.

`model.coeff(<tag>).feature(<ftag>).selection().set(...)` defines a local selection that assigns the coefficient form equations to geometric entities. For a complete list of methods available under `selection()`, see [model.selection\(\)](#). Only selections at a single geometry level is allowed in the selection.

`model.coeff(<tag>).field()` returns the fields as a string array.

`model.coeff(<tag>).intRule()` returns the integration rule tags as a string array.

`model.coeff(<tag>).feature(<ftag>).getType(ctype)` returns the coefficient value. See the section [get* and Selection Access Methods](#) for available methods.

`model.coeff(<tag>).feature(<ftag>).selection().named()` returns the named selection tag, and `model.coeff(<tag>).feature(<ftag>).selection().getType()` returns domain information. See [model.selection\(\)](#) for available methods.

Special Properties for the Wave Form PDE

If you create a Wave Form PDE using, for example,

```
model.physics().create("wahw", "WaveFormPDE", "geom1", new String[][]{{"u"}});
```

then the following properties are available using the `setIndex` syntax:

TABLE 2-14: WAVE FORM PDE PROPERTIES.

PROPERTY	VALUE	DEFAULT	INDEX	DESCRIPTION
fluxmethod	fluxLF fluxGeneral	fluxLF	0	Flux method: Lax–Friedrichs or a general numerical flux.
gstar	double	1	0	General numerical flux.
tau	double	1	0	Lax–Friedrichs parameter.
filteractive	0 1	0	0	Activate filter parameters.
filter	double	36	0	Filter parameter α .
filter	double	0.6	1	Filter parameters η_c .
filter	double	3	2	Filter parameter s .

For example, to set the filter parameter s to 2.5, use

```
model.physics("wahw").feature("wafeq1").setIndex("filter", "2.5", 2);
```

EXAMPLE

Define two uncoupled Poisson-like equations on the domain `dtag`.

Code for Use with Java

```
model.coeff().create("c1",new String[]{"u","v"});
model.coeff("c1").intRule(new String[]{"gp1","gp1"});
CoeffFeature f1 = model.coeff("c1").create("f1");
```

```
f1.set("c",1,new String[]{"1","0.1","2"});
f1.set("c",2,"3");
f1.set("f",new String[]{"2","1"});
f1.selection().geom("g1",2);
f1.selection().set(1);
```

Code for Use with MATLAB

```
model.coeff.create('c1',{'u','v'});
model.coeff('c1').intRule({'gp1','gp1'});
f1 = model.coeff('c1').create('f1');
f1.set('c',1,{'1','0.1','2'});
f1.set('c',2,'3');
f1.set('f',{'2','1'});
f1.selection.geom('g1',2);
f1.selection.set(1);
```

model.common()

The common list contains nodes that have different purposes depending on the node type. The type is assigned when the node is created. All other properties are set and accessed using generic `set` and `get` methods as described under `set()` and [get* and Selection Access Methods](#). The different types may or may not use a selection. The following node types are currently defined:

TABLE 2-15: COMMON FEATURE TYPES.

TYPE	PURPOSE	DESCRIPTION
ExpressionOperator	Variable utility	Define an operator evaluating different parameterized expressions on different geometric entities.
OperatorContribution	Variable utility	Override the definition of an expression operator.
Matrix	Variable utility	Define a matrix of variables. Uses a selection.
MatrixInverse	Variable utility	Compute the inverse of a matrix. Uses a selection.
MatrixDiagonalization	Variable utility	Diagonalize a symmetric 3-by-3 matrix. Uses a selection.
MatrixDecomposition	Variable utility	Use SVD to decompose a matrix. Uses a selection.
VectorTransform	Variable utility	Transform a vector from one coordinate system to another.
MatrixTransform	Variable utility	Transform matrix indices between coordinate systems.
StateVariables	Variable utility	Define state variables and reinitialization rules.
ControlVariableField	Variable utility	Define control variables on geometric entities.
Global Dependent Variables	Equation contribution	Declare global scalar degrees of freedom.
Global Weak Contribution	Equation contribution	Add a global scalar weak contribution.
Global Constraint	Equation contribution	Add a global scalar constraint.
IntegrationRule	Equation contribution	Add an integration rule.
Dependent Variable Field	Equation contribution	Declare a dependent variable field on a selection.
Weak Contribution	Equation contribution	Add a weak contribution on a selection.
Constraint	Equation contribution	Add a constraint on a selection.
ParticipationFactors	Physics variables	Set up participation factor evaluation.

TABLE 2-15: COMMON FEATURE TYPES.

TYPE	PURPOSE	DESCRIPTION
ResponseSpectrum	Physics variables	Set up response spectrum evaluation.
AmbientProperties	Physics variables	Set up ambient thermal conditions using meteorological data.
GlobalReducedModelInputs	Control variables	Define global control variables for use as inputs to model reduction.
Control Function	Optimization	Define a control variable field and a filtered density field for topology optimization.
FixedTopologyDomain	Optimization	Define a fixed topology domain for topology optimization.
FixedTopologyBoundary	Optimization	Define a fixed topology boundary for topology optimization.
FreeShapeDomain	Optimization	Define a free shape domain for shape optimization.
FreeShapeBoundary	Optimization	Define a free shape boundary for shape optimization.
FreeShapeShell	Optimization	Define a free shape shell for shape optimization.
PolynomialShell	Optimization	Define a polynomial shell for shape optimization.
FreeShapeSymmetry	Optimization	Define a symmetry or roller condition for shape optimization.
FixedShapePoint	Optimization	Define a fixed shape point in 2D for shape optimization.
FixedShapeEdge	Optimization	Define a fixed shape edge in 3D for shape optimization.
FixedShapeBoundary	Optimization	Define a fixed shape boundary for shape optimization.
FixedShapeBoundary	Optimization	Define a mirror symmetry domain or boundary for shape optimization.
FixedShapeBoundary	Optimization	Define a sector symmetry domain or boundary for shape optimization.
GlobalLeastSquaresObjective	Optimization	Define a global least-squares objective.
LeastSquaresObjective	Optimization	Define a least-squares objective.
MaterialTopologyDomain	Optimization	Define a material topology domain for topology optimization.
MaterialTopologyBoundary	Optimization	Define a material topology boundary for topology optimization.
PolynomialBoundary	Optimization	Defined a polynomial boundary in 2D for shape optimization,.
VoidTopologyDomain	Optimization	Define a void topology domain for topology optimization.
VoidTopologyBoundary	Optimization	Define a void topology boundary for topology optimization.
MirrorTopology	Optimization	Define a mirror symmetry domain or boundary for topology optimization.
SectorTopology	Optimization	Define a sector symmetry domain or boundary for topology optimization.

TABLE 2-15: COMMON FEATURE TYPES.

TYPE	PURPOSE	DESCRIPTION
CommonInputDefault	Default model inputs	Set default values of input quantities required by materials.
CommonInputDef	Default model inputs	Override values of input quantities on selected entities.
PrescribedDeformation	Moving mesh and deformed geometry	Prescribe a mesh deformation on domains. Uses a selection.
PrescribedMeshVelocity	Moving mesh and deformed geometry	Prescribe a mesh velocity of a deforming domain boundary. Uses a selection.
RotatingDomain	Moving mesh and deformed geometry	Prescribe rotation of a domain. Uses a selection.
DeformingDomain	Moving mesh and deformed geometry	Specify free deformation of domains. Uses a selection.
FixedBoundary	Moving mesh and deformed geometry	Prescribe zero displacement of a deforming domain boundary. Uses a selection.
PrescribedMeshDisplacement	Moving mesh and deformed geometry	Prescribe displacement of a deforming domain boundary. Uses a selection.
PrescribedNormalMeshVelocity	Moving mesh and deformed geometry	Prescribe velocity of a deforming domain boundary. Uses a selection.
PrescribedNormalMeshDisplacement	Moving mesh and deformed geometry	Prescribe a normal displacement of a deforming domain boundary. Uses a selection.
RotatingBoundary	Moving mesh and deformed geometry	Prescribe rotation of a boundary. Uses a selection.
Slip	Moving mesh and deformed geometry	Prescribe mesh slip behavior of a deforming domain boundary. Uses a selection.
Symmetry	Moving mesh and deformed geometry	Prescribe symmetry on a deforming domain boundary. Uses a selection.
SurrogateModelGeometrySampling	Surrogate modeling	Geometry sampling for surrogate modeling.



All types with an optimization purpose require the Optimization Module.



Moving mesh and deformed geometry features exist in two versions which differ in type ID and in which frame they control, but are otherwise identical. The deformed geometry version controlling the material frame uses a type ID with `DeformedGeometry` appended.

SYNTAX

```
model.component(<ctag>).common().create(<tag>,type);
model.component(<ctag>).common(<tag>).set(property,<value>);
model.component(<ctag>).common(<tag>).image()
```

DESCRIPTION

`model.component(<ctag>).common().create(<tag>,type)` creates a common feature node with the given tag and type.

`model.component(<ctag>).common(<tag>).set(property,<value>)` sets a named property in the common feature with tag `<tag>` in component `<ctag>`.

ExpressionOperator

`model.component(<ctag>).common().create(<tag>,"ExpressionOperator")` creates an expression operator feature. The following properties are available.

TABLE 2-16: PROPERTIES FOR EXPRESSIONOPERATOR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
argument	String array	{}	List of argument formal variable names.
dimensions	String array	{}	List of unit expressions for each argument.
expression	String	Empty	An expression in terms of formal arguments and other variables.
passbyval	String array of on or off	String array with all elements set to off.	Pass the arguments by reference (when set to "off") or by value (when set to "on").

OperatorContribution

`model.component(<ctag>).common(<exop>).create(<tag>,"OperatorContribution")` creates an Operator Contribution feature as a subfeature to the Expression Operator feature with tag <exop>. The following properties are available.

TABLE 2-17: PROPERTY FOR OPERATORCONTRIBUTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
expression	String	Empty	An expression in terms of formal arguments and other variables.

Matrix

`model.component(<ctag>).common().create(<tag>,"Matrix")` creates a matrix variable feature. For a Matrix definition, the following properties are available.

TABLE 2-18: PROPERTIES FOR MATRIX.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
format	full symmetric hermitian	full	Matrix format.
matrix	String matrix	{{"1", "0", "0"}, {"0", "1", "0"}, {"0", "0", "1"}}	The matrix elements of a square matrix.
size	1 2 3 4 4 6 7 8 9 userDefined	3	Matrix size. Values 1–9, representing 1-by-1 to 9-by-9 matrices, or userDefined
userDefined Size	Integer	3	Matrix size when userDefined. Sizes up to 50-by50 are allowed.

MatrixInverse

`model.component(<ctag>).common().create(<tag>,"MatrixInverse")` creates a matrix inverse feature. For a MatrixInverse definition, the following properties are available.

TABLE 2-19: PROPERTIES FOR MATRIXINVERSE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
format	full symmetric hermitian	full	Matrix format.
matrix	String matrix	{{"1", "0", "0"}, {"0", "1", "0"}, {"0", "0", "1"}}	The matrix elements of the square matrix to invert.

TABLE 2-19: PROPERTIES FOR MATRIXINVERSE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
size	1 2 3 4 4 6 7 8 9 userDefined	3	Matrix size. Values 1–9, representing 1-by-1 to 9-by-9 matrices, or userDefined
userDefined Size	Integer	3	Matrix size when userDefined. Sizes up to 50-by50 are allowed.

MatrixDiagonalization

`model.component(<ctag>).common().create(<tag>,"MatrixDiagonalization")` creates a matrix diagonalization feature. For a MatrixDiagonalization definition, the following properties are available.

TABLE 2-20: PROPERTIES FOR MATRIXDIAGONALIZATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
computeExponential	true false	false	Compute matrix exponential.
ignoreJacobianContributions	true false	true	Ignore solution dependencies.
matrix	String matrix	{{"1", "0", "0"}, {"0", "1", "0"}, {"0", "0", "1"}}	The matrix elements of the square matrix to diagonalize.

MatrixDecomposition

`model.component(<ctag>).common().create(<tag>,"MatrixDecomposition")` creates a matrix decomposition (SVD) feature. For a MatrixDecomposition definition, the following properties are available.

TABLE 2-21: PROPERTIES FOR MATRIXDECOMPOSITION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
computeLeftSingularVectors	true false	true	Compute left singular vectors.
computeRightSingularVectors	true false	true	Compute right singular vectors.
computeRotationMatrix	true false	true	Compute rotation matrix.
computeStretchMatrix	true false	true	Compute stretch matrix.
format	full symmetric hermitian	full	Matrix format.
matrix	String matrix	{{"1", "0", "0"}, {"0", "1", "0"}, {"0", "0", "1"}}	The matrix elements of the square matrix to decompose.
size	1 2 3 4 4 6 7 8 9 userDefined	3	Matrix size. Values 1–9, representing 1-by-1 to 9-by-9 matrices, or userDefined
userDefinedSize	Integer	3	Matrix size when userDefined. Sizes up to 50-by50 are allowed.

VectorTransform

`model.component(<ctag>).common().create(<tag>,"VectorTransform", <component>)` creates a vector transformation feature that transforms vectors from one coordinate system to another. For a VectorTransform definition, the following properties are available.

TABLE 2-22: PROPERTIES FOR VECTORTRANSFORM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
componentChange	none raiseIndex lowerIndex	none	Raise or lower index of output vector.
inputComponents	contravariant covariant	contravariant	Input component type.
inputReferenceFrame	mesh geometry material spatial	spatial	Reference frame for relative input coordinate system.
inputSystem	global:spatial global:material <coordsys_tag>	global:spatial	Coordinate system to which input components refer.
inputVolumeReferenceSystem	global:spatial global:material <coordsys_tag>	global:spatial	Input volume reference for density transforms.
metric	material spatial	spatial	Metric for raising or lowering indices.
outputReferenceFrame	mesh geometry material spatial	spatial	Reference frame for relative output coordinate system.
outputSystem	global:spatial global:material <coordsys_tag>	global:spatial	Coordinate system to user for output vector components.
outputVolumeReferenceSystem	global:spatial global:material <coordsys_tag>	global:spatial	Output volume reference for density transforms.
transform	vector vectorDensity scalarFlux directedAreaElement	vector	Transform rule to apply.
vector	String array	{"0","0","0"}	Input vector components.

MatrixTransform

`model.component(<ctag>).common().create(<tag>,"MatrixTransform", <component>)` creates a matrix transformation feature that transforms matrix indices from one coordinate system to another. For a MatrixTransform definition, the following properties are available.

TABLE 2-23: PROPERTIES FOR MATRIXTRANSFORM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
componentChange	none raiseIndex lowerIndex	none	Raise or lower index of output vector.
format	full symmetric	full	Form of input matrix.
inputColumnComponents	contravariant covariant	contravariant	Input column index component type.
inputColumnReferenceFrame	mesh geometry material spatial	spatial	Reference frame for relative column input system.
inputColumnSystem	global:spatial global:material <coordsys_tag>	global:spatial	Coordinate system to which input column components refer.
inputComponents	contravariant covariant	contravariant	Input component type for symmetric matrix.

TABLE 2-23: PROPERTIES FOR MATRIXTRANSFORM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
inputReferenceFrame	mesh geometry material spatial	spatial	Reference frame for relative symmetric input coordinate system.
inputRowComponents	contravariant covariant	contravariant	Input row index component type.
inputRowReferenceFrame	mesh geometry material spatial	spatial	Reference frame for relative row input coordinate system.
inputRowSystem	global:spatial global:material <coordsys_tag>	global:spatial	Coordinate system to which input row components refer.
inputSystem	global:spatial global:material <coordsys_tag>	global:spatial	Coordinate system to which symmetric input components refer.
inputVolumeReferenceSystem	global:spatial global:material <coordsys_tag>	global:spatial	Input volume reference for density transforms.
matrix	String matrix	{{"0", "0", "0"}, {"0", "0", "0"}, {"0", "0", "0"}}	Input matrix components.
columnMetric	material spatial	spatial	Metric for raising or lowering column indices.
rowMetric	material spatial	spatial	Metric for raising or lowering row indices.
outputColumnReferenceFrame	mesh geometry material spatial	spatial	Reference frame for relative output column coordinate system.
outputColumnSystem	global:spatial global:material <coordsys_tag>	global:spatial	Coordinate system to user for output matrix column indices.
outputRowReferenceFrame	mesh geometry material spatial	spatial	Reference frame for relative output row coordinate system.
outputRowSystem	global:spatial global:material <coordsys_tag>	global:spatial	Coordinate system to user for output matrix row indices.
outputVolumeReferenceSystem	global:spatial global:material <coordsys_tag>	global:spatial	Output volume reference for density transforms.
transform	vector tensorDensity vectorFlux	tensor	Transform rule to apply.

StateVariables

`model.component(<ctag>).common().create(<tag>,"StateVariables")` creates a state variable definition feature that declares state variables and specifies initialization and update expressions. For a State Variables definition, the following properties are available.

TABLE 2-24: PROPERTIES FOR STATEVARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
description	String array	{}	State descriptions.
initialValue	String array	{}	Initial value expressions.
order	0 2 4 6 8 10 12 14 userDefined	4	Integration point order.

TABLE 2-24: PROPERTIES FOR STATEVARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
orderExpression	String	4	User-defined order
quantity	none <i><quantity_id></i>	dimensionless	State physical quantity
state	String array	{}	State names.
unit	<i><unit></i>	1	State unit
update	beforeStep afterPredictor afterStep onlyInitialization	beforeStep	When to apply the update expressions.
updateExpression	String array	{}	Update expressions.
valueType	real complex	complex	Value type when using splitting of complex variables

Global Dependent Variables

`model.component(<ctag>).common().create(<tag>,"GlobalDependentVariables")` creates a global dependent variables definition feature that declares global degrees of freedom. For a Global Dependent Variables feature, the following properties are available.

TABLE 2-25: PROPERTIES FOR GLOBALDEPENDENTVARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
description	String array	{}	DOF descriptions.
initialTimeDerivative	String array	{}	Initial time derivative expressions.
initialValue	String array	{}	Initial value expressions.
quantity	none <i><quantity_id></i>	dimensionless	State physical quantity.
timeDerivatives	off solver frequency	solver	Definition of time derivatives.
unit	<i><unit></i>	1	State unit.
valueType	real complex	complex	Value type when using splitting of complex variables.

Global Weak Contribution

`model.component(<ctag>).common().create(<tag>,"GlobalWeakContribution")` creates a global weak contribution feature that adds one or more global scalar weak expression to the overall system of equations. For a Global Weak Contribution feature, the following properties are available.

TABLE 2-26: PROPERTIES FOR GLOBALWEAKCONTRIBUTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
weakExpression	String array	{}	Weak expressions.

Global Constraint

`model.component(<ctag>).common().create(<tag>,"GlobalConstraint")` creates a global constraint feature that adds a single scalar constraint to the overall system of equations. For a Global Constraint feature, the following properties are available.

TABLE 2-27: PROPERTIES FOR GLOBALCONSTRAINT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
constraint	String	0	Constraint expression.
constraintForce	String	0	Constraint force expression.
reactionTerms	symmetric userDefined	symmetric	Application of reaction forces.

IntegrationRule

`model.component(<ctag>).common().create(<tag>,"IntegrationRule")` creates a custom integration rule feature that specifies the integration point locations and weights for all element shapes. For an Integration Rule feature the following properties are available:

TABLE 2-28: PROPERTIES FOR INTEGRATIONRULE.

NAME	VALUE	DEFAULT	DESCRIPTION
interval	DoubleMatrix	{{1/2}}	Interval
intervalWeight	DoubleArray	{1}	Interval
triangle	DoubleMatrix	{{1/3,1/3}}	Triangle
triangleWeight	DoubleArray	{1/2}	Triangle
quad	DoubleMatrix	{{1/2,1/2}}	Quadrilateral
quadWeight	DoubleArray	{1}	Quadrilateral
tetrahedron	DoubleMatrix	{{1/4,1/4,1/4}}	Tetrahedron
tetrahedronWeight	DoubleArray	{1/6}	Tetrahedron
block	DoubleMatrix	{{1/2,1/2,1/2}}	Block
blockWeight	DoubleArray	{1}	Block
prism	DoubleMatrix	{{1/3,1/3,1/2}}	Prism
prismWeight	DoubleArray	{1/2}	Prism
pyramid	DoubleMatrix	{{1/2,1/2,1/3}}	Pyramid
pyramidWeight	DoubleArray	{1/3}	Pyramid

Dependent Variable Field

`model.component(<ctag>).common().create(<tag>,"DependentVariableField")` creates a dependent variable field definition feature that declares a scalar or vector field on a selection. For a Dependent Variable field definition, the following properties are available.

TABLE 2-29: PROPERTIES FOR DEPENDENTVARIABLEFIELD.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
boundaryFlux	on off	on	Define accurate boundary flux variables for results processing.
boundaryFluxSmoothing	on off	on	Smooth boundary flux variables.
componentFrame	mesh geometry material spatial allCovariant allContravariant	spatial	Frame of vector variable components.
components	String array	{<field><x>,<field><y>,<field><z>}	Component names when declared on a single frame.
condition	String	0	Boolean condition true in elements where shape functions are defined.
conditionActive	on off	off	Use element set condition.
derivativeFrame	all mesh geometry material spatial	all	Frames where gradient component variables will be defined.
fieldType	scalar inPlaneVector outOfPlaneVector threeComponentVector	scalar	Number of field components and interpretation.

TABLE 2-29: PROPERTIES FOR DEPENDENTVARIABLEFIELD.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initialTimeDerivative	String	0	Initial time derivative of scalar field.
initialValue	String	0	Initial value of scalar field.
initialVector	String array	{"0", "0", "0"}	Initial value of vector field.
initialVectorTimeDerivative	String array	{"0", "0", "0"}	Initial value of vector field.
interiorFluxEstimate	String array	{"0", "0", "0"}	Domain expression estimating the flux in the domain.
elementOrder	0 1 2 3 4 5 6 7 user defined	2	Element order. Available values depend on the shape function type.
elementOrderExpression	String	2	User-defined element order
quantity	none <quantity_id>	dimensionless	State physical quantity
referenceFrame	mesh geometry material spatial	spatial	Frame in which the interiorFluxEstimate components are entered.
shapeFunctionType	<shape_function_id>	shlag	Shape function type ID.
slits	Selection	empty	Selection where a discontinuity is introduced.
timeDerivatives	off solver frequency	solver	Definition of time derivatives.
unit	<unit>	1	State unit
valueType	real complex	complex	Value type when using splitting of complex variables

Weak Contribution

`model.component(<ctag>).common().create(<tag>,"WeakContribution")` creates a weak contribution feature that adds a weak expression on a selection to the overall system of equations. For a Weak Contribution feature, the following properties are available.

TABLE 2-30: PROPERTIES FOR WEAKCONTRIBUTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
weakExpression	String	0	Weak expression
integrationOrder	0 2 4 6 8 userDefined	4	Integration order.
integrationOrderExpression	String	4	User-defined integration order.
quadratureFrame	mesh geometry material spatial	spatial	Frame in which the integration is performed.
quadratureMethod	String	gauss	Quadrature method: built-in or user-defined.

Constraint

`model.component(<ctag>).common().create(<tag>,"Constraint")` creates a constraint feature that adds a scalar or vector constraint on a selection. The constraint can be added to the overall system of equations as a

pointwise constraint, a weak constraint, or be enforced by a penalty term. For a Constraint feature, the following properties are available.

TABLE 2-31: PROPERTIES FOR CONSTRAINT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
constraint	String	0	Constraint expression.
constraintForce	String	0	Constraint force expression.
constraintMethod	elemental nodal	elemental	Pointwise constraint method.
constraintType	pointwise weak Nitsche	pointwise	Constraint implementation.
excludedBoundaries	Selection	empty	Boundaries excluded from a domain constraint.
excludedEdges	Selection	empty	Edges excluded from a domain or boundary constraint.
excludedPoints	Selection	empty	Points excluded from a constraint.
formulation	symmetric skew nonsymmetric	skew	Nitsche method formulation.
integrationOrder	0 2 4 6 8 user defined	4	Integration order for weak and Nitsche implementations.
integrationOrderExpression	String	4	User-defined integration order for weak and Nitsche implementations.
interiorFlux	String array	{"0", "0", "0"}	Interior flux in equation being constrained by the Nitsche method.
elementOrder	0 1 2 3 4 5 6 7 user defined	2	Element order for pointwise and weak constraints.
elementOrderExpression	String	2	User-defined element order for pointwise and weak constraints.
lmName	String	lm_<tag>	Lagrange multiplier variable name for weak constraint.
penaltyFactor	String	24*dvol_spatial/ mean(dvol_spatial)	User-defined penalty factor for the Nitsche method.
penaltyFactorControl	auto userDefined	auto	Nitsche method penalty factor selection strategy.
quadratureFrame	mesh geometry material spatial	spatial	Frame in which integration of weak and Nitsche contributions is performed.
quadratureMethod	String	gauss	Quadrature method weak and Nitsche contributions.
reactionTerms	symmetric user defined	symmetric	Application of reaction forces.
referenceFrame	mesh geometry material spatial	spatial	Frame in which the Nitsche interior flux is interpreted.

TABLE 2-31: PROPERTIES FOR CONSTRAINT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
shapeFunctionType	<code><shape_function_id></code>	shlag	Shape function type ID.
vectorConstraint	String array	{"0", "0", "0"}	Vector constraint expression.
vectorConstraintForce	String array	{"0", "0", "0"}	Vector constraint force expression.

ControlVariableField

`model.component(<ctag>).common().create(<tag>, "ControlVariableField")` creates a control variable field feature that adds a control variable to some geometric entity for use in a sensitivity analysis. For a control variable field definition, the following properties are available, in addition to the [Selections](#).

TABLE 2-32: PROPERTIES FOR CONTROLVARIABLEFIELD.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initialValue	String array	{}	Initial value expressions.
lbound	double	0	Lower bound.
order	1 2 3 4	1	Element order.
scale	double	1	Scaling of the control variable.
shapeFunctionType	shlag shdisc	shlag	The shape function type: Lagrange or discontinuous Lagrange.
ubound	double	0	Upper bound.
useBounds	true false	true	Use bounds for the control variable.

ParticipationFactors

`model.component(<ctag>).common().create(<tag>, "ParticipationFactors")` creates a participation factors feature setting up variables for participation factor evaluation. For a ParticipationFactors definition, the following properties are available.

TABLE 2-33: PROPERTIES FOR PARTICIPATIONFACTORS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
cor	com user	com	Center of rotation definition: Center of mass or user defined.
point	String array	{"0", "0", "0"}	The point for the center of rotation, if cor is set to user.

ResponseSpectrum

`model.component(<ctag>).common().create(<tag>, "ResponseSpectrum")` creates a response spectrum feature preparing a structural model for response spectrum evaluation. For a Response Spectrum feature, the following properties are available.

TABLE 2-34: PROPERTIES FOR RESPONSESPECTRUM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
eigStudy	String	none	Tag of a Study containing at least one Eigenfrequency study step, or none.

BucklingImperfection

`model.component(<ctag>).common().create(<tag>, "BucklingImperfection")` creates a buckling imperfection feature preparing a structural model for performing a buckling analysis with an initial imperfection

based on a preceding linear buckling analysis. For a Buckling Imperfection feature, the following properties are available:

TABLE 2-35: PROPERTIES FOR BUCKLINGIMPERFECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
LoadParameter	String	Any existing parameter	The load parameter for the nonlinear buckling study.
NonlinearBucklingStudy	String	new	Tag of the study to use as the linear buckling study, or new to create a new study.
Study	String	none	Tag of the study to use as the linear buckling study, or none.

This feature requires a license for the Structural Mechanics Module.

P-Norm

`model.component(<ctag>).common().create(<tag>,"PNorm")` creates a P-norm feature which is differentiable approximation of maximum operator, which is particularly useful in the context of gradient-based optimization. For a P-norm feature, the following properties are available:

TABLE 2-36: PROPERTIES FOR P-NORM

PROPERTY	VALUE	DEFAULT	DESCRIPTION
name	String	pnorm1	The name of the p-norm variable.
expression	String	0	The expression to compute the p-norm of
norm	String	10	The norm to use
manualNorm	String	10	The norm to use when the norm property is set to userDefined
integrationOrder	String	4	The integration order to use when computing the integrals or userDefined.
integrationOrderExpression	String	4	The integration order to use when the integrationOrder property is set to userDefined
quadratureFrame	String	spatial	The frame in which the integration is performed
quadratureMethod	String	gauss	Quadrature method: built-in or user-defined.
axisSymmetry	boolean	false	Determines whether $2\pi r$ is to be multiplied for axisymmetric models to mimic a volumetric integral.

Standard Deviation

`model.component(<ctag>).common().create(<tag>,"StandardDeviation")` creates a standard deviation which can be used to quantify the variation over a selection. For a Standard Deviation feature, the following properties are available:

TABLE 2-37: PROPERTIES FOR STANDARD DEVIATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
name	String	std1	The name of the standard deviation variable.
expression	String	0	Tag of the study to use as the linear buckling study, or new to create a new study.
integrationOrder	String	4	The integration order to use when computing the integrals or userDefined.
integrationOrderExpression	String	4	The integration order to use when the integrationOrder property is set to userDefined.
quadratureFrame	String	spatial	The frame in which the integration is performed.

TABLE 2-37: PROPERTIES FOR STANDARD DEVIATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
quadratureMethod	String	gauss	Quadrature method: built-in or user-defined.
axisymmetry	boolean	false	Determines whether $2\pi r$ is to be multiplied for axisymmetric models to mimic a volumetric integral.

This feature requires a license for the Structural Mechanics Module.

AmbientProperties

`model.component(<tag>).common().create(<tag>,"AmbientProperties")` creates an ambient properties feature. Ambient properties can be set manually, or imported from meteorological data. For an Ambient Properties feature, [Table 2-38](#) lists the properties that are available for the default setting of the AmbientData property.

TABLE 2-38: PROPERTIES FOR AMBIENTPROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
AmbientData	UserDef MeteorologicalData MeteorologicalData2017	UserDef	Source of ambient data.
T_amb	double	293.15 [K]	Ambient temperature.
p_amb	double	1 [atm]	Ambient absolute pressure.
phi_amb	double	0	Ambient relative humidity.
v_amb	double	0 [m/s]	Wind velocity.
Isn_amb	double	1000 [W/m ²]	Clear sky noon beam normal irradiance.
Ish_amb	double	0 [W/m ²]	Clear sky noon diffuse horizontal irradiance.

GlobalReducedModelInputs

`model.component(<tag>).common().create(<tag>,"GlobalReducedModelInputs")` creates a reduced-model inputs feature defining control variables for use as inputs when training a reduced model. For a Global Reduced-Model Inputs feature definition, the following properties are available.

TABLE 2-39: PROPERTIES FOR GLOBALREDUCEDMODELINPUTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
name	String array	{}	Names of global scalar variables to be made available as reduced-model inputs.
expression	String array	{}	Value expressions, one for each variable in name.

CommonInputDefault

The Common Model Inputs feature is a default singleton feature (with tag `cminput`) that cannot be removed. It controls the default values of model input quantities required as input values to materials and physics features. For the Common Model Inputs definition, the following properties are available.

TABLE 2-40: PROPERTIES FOR COMMONINPUTDEFAULT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
modified	String matrix	{{}}	Pairs of quantity name and defining expression

CommonInputDef

`model.component(<ctag>).common().create(<tag>,"CommonInputDef")` creates a common model input definition feature overriding the value of a model input quantity on a selection. For a Model Input definition, the following properties are available.

TABLE 2-41: PROPERTIES FOR COMMONINPUTDEF.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
minpDefName	String	dimensionless	Name of quantity to be defined.
minpScalar	String	Empty	Scalar definition of current quantity.
minpVector	String array	{"1", "2", "3"}	Vector definition of current quantity.

Note that changing the minpDefName property resets the corresponding value property to a default value specific to the particular quantity.

PrescribedDeformation

`model.component(<ctag>).common().create(<tag>,"PrescribedDeformation")` or `model.component(<ctag>).common().create(<tag>,"PrescribedDeformationDeformedGeometry")` creates a prescribed deformation feature acting on the spatial or material frame mesh, respectively. For a PrescribedDeformation definition, the following property is available.

TABLE 2-42: PROPERTY FOR PRESCRIBEDDEFORMATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prescribedDeformation	String array	{"0", "0", "0"}	The prescribed deformation vector.

PrescribedMeshVelocity

`model.component(<ctag>).common().create(<tag>,"PrescribedMeshVelocity")` or `model.component(<ctag>).common().create(<tag>,"PrescribedMeshVelocityDeformedGeometry")` creates a prescribed mesh velocity feature acting on the spatial or material frame mesh, respectively. For a PrescribedMeshVelocity definition, the following property is available.

TABLE 2-43: PROPERTY FOR PRESCRIBEDDEFORMATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prescribedMeshVelocity	String array	{"0", "0", "0"}	The prescribed mesh velocity vector.

RotatingDomain

`model.component(<ctag>).common().create(<tag>,"RotatingDomain")` or `model.component(<ctag>).common().create(<tag>,"RotatingDomainDeformedGeometry")` creates a rotating domain feature acting on the spatial or material frame mesh, respectively. For a RotatingDomain definition, the following properties are available.

TABLE 2-44: PROPERTY FOR ROTATINGDOMAIN.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angularVelocity	String	0	The angular velocity when rotationType is set to rotationalVelocity.
appliedMoment	String	0	The applied moment when rotationType is set to rigidBody.
initialAngle	String	0	The initial angle when rotationType is set to rotationalVelocity or rigidBody.

TABLE 2-44: PROPERTY FOR ROTATINGDOMAIN.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initialAngularVelocity	String	0	The initial angular velocity when rotationType is set to rigidBody.
momentOfInertia	String	0	The moment of inertia when rotationType is set to rigidBody.
revolutionsPerTime	String	0	The revolutions per time when rotationType is set to rotationalVelocity.
rotationAngle	String	0	The rotational angle when rotationType is set to userDefined.
rotationAxis	String array	{"0", "0", "0"}	The rotation axis (3D only).
rotationAxisBasePoint	String array	{"0", "0", "0"} (3D); {"0", "0"} (2D)	The rotation axis base point.
rotationType	userDefined rotationalVelocity rigidBody		
rotationalVelocity Expression	generalAngularVelocity constantAngularVelocity constantRevolutionsPerTime generalRevolutionsPerTime	constantAngularVelocity	The rotational velocity expression to use when rotationType is set to rotationalVelocity.

DeformingDomain

`model.component(<ctag>).common().create(<tag>,"DeformingDomain")` or

`model.component(<ctag>).common().create(<tag>,"DeformingDomainDeformedGeometry")` creates a deforming domain feature acting on the spatial or material frame mesh, respectively. For a DeformingDomain definition, the following properties are available.

TABLE 2-45: PROPERTY FOR DEFORMINGDOMAIN.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initialDeformation	String array	{"0", "0", "0"}	The initial deformation of the domain.
smoothingType	laplace winslow hyperelastic yeoh	laplace	The mesh smoothing type: Laplace, Winslow, hyperelastic, or Yeoh.
stabilization	none huhu huhululu	none	Additional stabilization terms
stiffeningFactor	String	10	Coefficient for the quadratic term in the Yeoh material model
tuningParameter	String	1e-6*dGeomChar^2	Stabilization tuning parameter
volumetricStrain	linear logarithmic	linear	Volumetric strain definition

FixedBoundary

`model.component(<ctag>).common().create(<tag>,"FixedBoundary")` or

`model.component(<ctag>).common().create(<tag>,"FixedBoundaryDeformedGeometry")` creates a fixed

boundary feature acting on the spatial or material frame mesh, respectively. This feature has a boundary selection only.

PrescribedMeshDisplacement

`model.component(<ctag>).common().create(<tag>,"PrescribedMeshDisplacement")` or
`model.component(<ctag>).common()`

`.create(<tag>,"PrescribedMeshDisplacementDeformedGeometry")` creates a prescribed mesh displacement feature acting on the spatial or material frame mesh, respectively. For a PrescribedMeshDisplacement definition, the following property is available.

TABLE 2-46: PROPERTY FOR PRESCRIBEDMESHDISPLACEMENT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prescribedMeshDisplacement	String array	{"0", "0", "0"}	The prescribed mesh displacement.

PrescribedNormalMeshDisplacement

`model.component(<ctag>).common().create(<tag>,"PrescribedNormalMeshDisplacement")` or
`model.component(<ctag>).common()`

`.create(<tag>,"PrescribedNormalMeshDisplacementDeformedGeometry")` creates a prescribed normal mesh displacement feature acting on the spatial or material frame mesh, respectively. For a PrescribedNormalMeshDisplacement definition, the following property is available.

TABLE 2-47: PROPERTY FOR PRESCRIBEDNORMALMESHDISPLACEMENT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prescribedNormalDisplacement	String	0	The prescribed normal mesh displacement.

PrescribedNormalMeshVelocity

`model.component(<ctag>).common().create(<tag>,"PrescribedNormalMeshVelocity")` or
`model.component(<ctag>).common()`

`.create(<tag>,"PrescribedNormalMeshVelocityDeformedGeometry")` creates a prescribed normal mesh velocity feature acting on the spatial or material frame mesh, respectively. For a PrescribedNormalMeshVelocity definition, the following property is available.

TABLE 2-48: PROPERTY FOR PRESCRIBEDNORMALMESHVELOCITY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prescribedNormalVelocity	String	0	The prescribed normal mesh velocity.

RotatingBoundary

`model.component(<ctag>).common().create(<tag>,"RotatingBoundary")` or
`model.component(<ctag>).common().create(<tag>,"RotatingBoundaryDeformedGeometry")` creates a rotating boundary feature acting on the spatial or material frame mesh, respectively. For a RotatingBoundary definition, the following properties are available.

TABLE 2-49: PROPERTY FOR ROTATINGBOUNDARY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angularVelocity	String	0	The angular velocity when rotationType is set to rotationalVelocity.
appliedMoment	String	0	The applied moment when rotationType is set to rigidBody.

TABLE 2-49: PROPERTY FOR ROTATINGBOUNDARY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initialAngle	String	0	The initial angle when rotationType is set to rotationalVelocity or rigidBody.
initialAngularVelocity	String	0	The initial angular velocity when rotationType is set to rigidBody.
momentOfInertia	String	0	The moment of inertia when rotationType is set to rigidBody.
revolutionsPerTime	String	0	The revolutions per time when rotationType is set to rotationalVelocity.
rotationAngle	String	0	The rotational angle when rotationType is set to userDefined.
rotationAxis	String array	{"0", "0", "0"}	The rotation axis (3D only).
rotationAxisBasePoint	String array	{"0", "0", "0"} (3D); {"0", "0"} (2D)	The rotation axis base point.
rotationType	userDefined rotationalVelocity rigidBody		
rotationalVelocity Expression	generalAngularVelocity constantAngularVelocity constantRevolutionsPerTime generalRevolutionsPerTime	constantAngularVelocity	The rotational velocity expression to use when rotationType is set to rotationalVelocity.

Slip

`model.component(<ctag>).common().create(<tag>,"Slip")` or
`model.component(<ctag>).common().create(<tag>,"SlipDeformedGeometry")` creates a mesh slip feature acting on the spatial or material frame mesh, respectively. This feature has a boundary selection only.

Symmetry

`model.component(<ctag>).common().create(<tag>,"Symmetry")` or
`model.component(<ctag>).common().create(<tag>,"SymmetryDeformedGeometry")` creates a symmetry feature acting on the spatial or material frame mesh, respectively. This feature has a boundary selection only.



The ControlFunction, DensityTopology, FixedTopologyDomain, FixedTopologyBoundary, FreeShapeDomain, FreeShapeBoundary, FreeShapeShell, FreeShapeSymmetry, FixedShapeBoundary, FixedShapeEdge, FixedShapePoint, GlobalLeastSquaresObjective, LeastSquaresObjective, MaterialTopologyDomain, MaterialTopologyBoundary, PolynomialBoundary, PolynomialShell, VoidTopologyDomain, and VoidTopologyBoundary features below require the Optimization Module.

Control Function

`model.component(<ctag>).common().create(<tag>,"ControlFunction")` creates a control function feature for general optimization.

DensityTopology

`model.component(<ctag>).common().create(<tag>,"DensityTopology")` creates a density model feature for topology optimization.

FixedTopologyDomain

`model.component(<ctag>).common().create(<tag>,"FixedTopologyDomain")` creates a fixed topology domain feature for topology optimization.

FixedTopologyBoundary

`model.component(<ctag>).common().create(<tag>,"FixedTopologyBoundary")` creates a fixed topology boundary feature for topology optimization.

FreeShapeDomain

`model.component(<ctag>).common().create(<tag>,"FreeShapeDomain")` creates a free shape domain feature for shape optimization.

Transformation

`model.component(<ctag>).common().create(<tag>,"Transformation")` creates a transformation feature for shape optimization.

FreeShapeBoundary

`model.component(<ctag>).common().create(<tag>,"FreeShapeBoundary")` creates a fixed shape boundary feature for shape optimization.

FreeShapeShell

`model.component(<ctag>).common().create(<tag>,"FreeShapeShell")` creates a free shape shell feature for shape optimization.

PolynomialShell

`model.component(<ctag>).common().create(<tag>,"PolynomialShell")` creates a polynomial shell feature for shape optimization.

FreeShapeSymmetry

`model.component(<ctag>).common().create(<tag>,"FreeShapeSymmetry")` creates a free shape symmetry feature for shape optimization.

FixedShapePoint

`model.component(<ctag>).common().create(<tag>,"FixedShapePoint")` creates a fixed shape point feature for shape optimization in 2D only.

FixedShapeEdge

`model.component(<ctag>).common().create(<tag>,"FixedShapeEdge")` creates a fixed shape edge feature for shape optimization in 3D only.

FixedShapeBoundary

`model.component(<ctag>).common().create(<tag>,"FixedShapeBoundary")` creates a fixed shape boundary feature for shape optimization.

MirrorShape

`model.component(<ctag>).common().create(<tag>,"MirrorShape")` creates a mirror shape feature for shape optimization.

SectorShape

`model.component(<ctag>).common().create(<tag>,"SectorShape")` creates a sector shape boundary feature for shape optimization.

GlobalLeastSquaresObjective

`model.component(<ctag>).common().create(<tag>,"GlobalLeastSquaresObjective")` creates a global least-squares objective.

LeastSquaresObjective

`model.component(<ctag>).common().create(<tag>,"LeastSquaresObjective")` creates a least-squares objective.

MaterialTopologyDomain

`model.component(<ctag>).common().create(<tag>,"MaterialTopologyDomain")` creates a material topology domain feature for topology optimization.

MaterialTopologyBoundary

`model.component(<ctag>).common().create(<tag>,"MaterialTopologyBoundary")` creates a material topology boundary feature for topology optimization.

PolynomialBoundary

`model.component(<ctag>).common().create(<tag>,"PolynomialBoundary")` creates a polynomial boundary feature for shape optimization in 2D only.

Transformation

`model.component(<ctag>).common().create(<tag>,"Transformation")` creates a linear shape domain feature for shape optimization.

VoidTopologyDomain

`model.component(<ctag>).common().create(<tag>,"VoidTopologyDomain")` creates a void topology domain feature for topology optimization.

VoidTopologyBoundary

`model.component(<ctag>).common().create(<tag>,"VoidTopologyBoundary")` creates a void topology boundary feature for topology optimization.

MirrorTopology

`model.component(<ctag>).common().create(<tag>,"MirrorTopology")` creates a mirror symmetry feature for topology optimization.

SectorTopology

`model.component(<ctag>).common().create(<tag>,"SectorTopology")` creates a sector symmetry feature for topology optimization.

The following properties are available for the common optimization features.

TABLE 2-50: PROPERTIES FOR COMMON OPTIMIZATION FEATURES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
beta	double	8	The projection slope when projectionType is set to TanhProjection.
d0	String array	{"0","0","0"}	Initial control displacement.
discretization	linear constant		The discretization: linear or constant.

TABLE 2-50: PROPERTIES FOR COMMON OPTIMIZATION FEATURES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
filterType	Helmholtz No_filter	Helmholtz	The filter type for filtering of the density: Helmholtz filtering or no filtering.
filterLengthType	Mesh Custom	Mesh	L_min is taken as h when filterLengthType is set to Mesh.
filterRadiusType	Small Medium Large Custom	Large	L_min is taken as 1, 1.5, and 2 times the maximumDisplacement, when the filterRadiusType is set to Small, Medium, and Large, respectively.
fixedDensityType	Custom Material Void	Material	theta_fix is taken as 0 when fixedDensityType is set to Void, and it is taken as 0 when fixedDensityType is set to Material.
interpolationType	SIMP Darcy Linear_interp RAMP Custom	SIMP	The interpolation type: SIMP, Darcy, linear, RAMP, or user defined.
L_min	String	h	Filter radius when filterType is set to Helmholtz.
maximumDisplacement	double	5% of geometry bounding box	Maximum displacement (in each coordinate direction).
normal	double array	{0,0,1}	Prescribed normal vector.
order	positive integer	2	Polynomial order.
p_SIMP	double	3	SIMP exponent, when interpolationType is set to SIMP.
projectionType	No_projection TanhProjection	No_projection	The projection type: No projection or hyperbolic tangent projection.
q_Darcy	double	0.01	Darcy interpolation, when interpolationType is set to Darcy.
q_RAMP	double	3	RAMP parameter, when interpolationType is set to RAMP.
simpExponentType	Auto Custom	Auto	p_SIMP is taken as 3, when simpExponentType is set to Auto.
theta0	double	0.5	Initial value for the discretization.
thetaMinType	Auto Custom	Auto	theta_min is taken as 0.001 when thetaMinType is set to Auto.
theta_beta	double	0.5	The projection point when projectionType is set to TanhProjection.
theta_fix	double	1	Material volume factor constraint.
theta_min	double	0.001	Minimum penalized volume fraction, when interpolationType is set to SIMP.
theta_u	String	1-dtopo#.theta	User-defined interpolation
type	bernstein lagrange	bernstein	Polynomial type.

SurrogateModelGeometrySampling

`model.component(<tag>).common().create(<tag>,"SurrogateModelGeometrySampling")` creates a geometry sampling feature for surrogate model training.

The following properties are available for a `SurrogateModelGeometrySampling` definition:

TABLE 2-51: PROPERTIES FOR THE SURROGATEMODELGEOMETRYSAMPLING FEATURE.

VALUE	VALUE	DEFAULT	DESCRIPTION
samplemethod	selection grid mesh	selection	Use selection to sample on all mesh vertices in the selection of the computational mesh, use grid to uniformly sample on a grid and use mesh to interpolate the solution on a different mesh.
frame	mesh geometry material spatial	material	The coordinate frame to sample in.
dim1min	double	0	The lower bound of the grid in the first dimension when samplemethod is set to grid.
dim1max	double	1	The upper bound of the grid in the first dimension when samplemethod is set to grid.
dim1resolution	positive integer	1000 (1D), 100 (2D), 30 (3D)	The number of points the grid is uniformly discretized with in the first dimension when samplemethod is set to grid.
dim2min	double	0	The lower bound of the grid in the second dimension when samplemethod is set to grid.
dim2max	double	1	The upper bound of the grid in the second dimension when samplemethod is set to grid.
dim2resolution	positive integer	1000 (1D), 100 (2D), 30 (3D)	The number of points the grid is uniformly discretized with in the second dimension when samplemethod is set to grid.
dim3min	double	0	The lower bound of the grid in the third dimension when samplemethod is set to grid.
dim3max	double	1	The upper bound of the grid in the third dimension when samplemethod is set to grid.
dim3resolution	positive integer	1000 (1D), 100 (2D), 30 (3D)	The number of points the grid is uniformly discretized with in the third dimension when samplemethod is set to grid.
geometryselection	selection	all()	The selection to use when samplemethod is set to selection.
mesh	reference to mesh		The mesh to interpolate on when samplemethod is set to mesh.

model.component()

Model component nodes.

A component node has one of three types:

- Component: Component nodes in the model builder tree have this type.
- ExtraDim: Extra dimension nodes under global definitions have this type.
- MeshComponent: Mesh parts nodes under global definitions have this type.

All three types are included, if present, in the list returned by `model.component()`.

	The <code>model.component</code> syntax replaces the earlier <code>model.modelNode</code> syntax, which is still available for backward compatibility. The list object returned by <code>model.component()</code> still has type <code>ModelNodeList</code> and individual components returned from <code>model.component(<tag>)</code> are of type <code>ModelNode</code>.
	Component nodes control which frames are created as individual <code>Frame</code> objects in the <code>model.frame()</code> list once a geometry is added to the component. If the <code>defineAllFrames</code> flag is set, all four logical frames (spatial, material, geometry, and mesh frame) are created. Otherwise a single frame object is created, representing all four logical frames. When a model is created in the GUI, <code>defineAllFrames</code> is always set to true. This is required by some Moving Mesh and Deformed Geometry functionality, which otherwise becomes inaccessible. Working with a single frame object is still allowed from the API, for compatibility reasons.

SYNTAX

```
model.component().create(<tag>);
model.component().create(<tag>, <basetag>);
model.component().create(<tag>, <type>);
model.component().create(<tag>, boolean);

model.component(<tag>).defineAllFrames();
model.component(<tag>).defineAllFrames(boolean);
model.component(<tag>).getType();
model.component(<tag>).scope();
model.component(<tag>).baseSystem();
model.component(<tag>).baseSystem(<system>);
model.component(<tag>).sorder();
model.component(<tag>).sorder(<stype>);
model.component(<tag>).defineLocalCoord();
model.component(<tag>).defineLocalCoord(boolean);
model.component(<tag>).curvedInterior();
model.component(<tag>).curvedInterior(boolean);
model.component(<tag>).geometricModel();
model.component(<tag>).geometricModel(<sequencetag>);
model.component(<tag>).measure().measurementMethod;
```

DESCRIPTION

`model.component(<tag>)` represents a component node in the model tree.

`model.component().create(<tag>)` creates a component node of type `Component` with the given tag and `defineAllFrames` set to false.

`model.component().create(<tag>, <type>)` creates a component node with the tag `<tag>` of one of the following types, set as the string `<type>`: `Component`, for a normal geometry component; `ExtraDim`, for an extra dimension; or `MeshComponent`, for a mesh component. For example, to create a mesh component:

```
model.component().create("mcomp1", "MeshComponent");
model.component("mcomp1").geom().create("mgeom1", 3);
model.component("mcomp1").mesh().create("mpart1", "mgeom1");
```

The `defineAllFrames` flag is set to false.

`model.component().create(<tag>, boolean)` creates a model component node of type `Component` with the specified value of the `defineAllFrames` flag.

`model.component(<tag>).defineAllFrames()` returns the current value of the `defineAllFrames` flag.

`model.component(<tag>).defineAllFrames(boolean)` sets the `defineAllFrames` flag controlling whether all four logical frames (spatial, material, geometry, and mesh frame) should be created as separate `Frame` objects in `model.frame()` when a geometry is added to the component.

`model.component(<tag>).getType()` returns the type of component that the component with the tag `<tag>` is an instance of: `Component`, for a normal model component; `ExtraDim` for an extra dimension component; and `MeshComponent` for a mesh component.

`model.component(<tag>).scope()` returns the fully qualified scope name.

`model.component(<tag>).baseSystem(<system>)` use the given base system as unit system for the component node. This overrides the global unit system specified for the entire model object. To use global system again, set the base system of the component node to `null`.

`model.component(<tag>).sorder()` returns the geometry shape function used for the component node and its descendants.

`model.component(<tag>).sorder(<stype>)` sets the geometry shape function. Allowed values are `automatic` (the default), `linear`, `quadratic`, `cubic`, `quartic`, `quintic`, `sextic`, `septic`, `linear_serp`, `quadratic_serp`, `cubic_serp`, and `quartic_serp` (the latter four values represent serendipity elements). With an automatic shape function, the physics interfaces under the component node decide the most optimum shape function. The shape function set here is also used for the discretization of the mesh displacement when using ALE functionality.

`model.component(<tag>).defineLocalCoord()` returns true if element local coordinate variables exist. By default, this is the case.

`model.component(<tag>).defineLocalCoord(boolean)` sets a flag that determines whether element local coordinate variables exist.

`model.component(<tag>).curvedInterior()` returns true if curved interior mesh is activated in the component.

`model.component(<tag>).curvedInterior(boolean)` sets the curved interior mesh setting in the component. If true, the interior domain mesh elements can be curved to avoid inverted elements.

`model.component(<tag>).geometricModel()` returns the component's geometric model on which the physics is defined. The returned tag is prefixed with `geom/` or `mesh/` to indicate if it is a geometry or mesh.

`model.component(<tag>).geometricModel(<sequencetag>)` specifies the geometric model used in the component. Enter the tag for the geometry or meshing sequence `<sequencetag>` to be specified as the component's geometric model. Use the prefix `/geom` or `/mesh` if the tag only is ambiguous. Only meshing sequences that define their own geometric models can be specified as the component's geometric model. For more information, see [Geometric Model](#).

`model.component(<tag>).measure().measurementMethod` returns the volume, area, length, vertex coordinates, average coordinates, or distance between two vertices according to the entity selection. For a list of the available measurement methods to use, see [Measurement Methods](#). The measurements are done on the finalized geometry or mesh that defines the component's geometric model. See [Geometric Model](#) for more information. For information about how to select entities, see [Selections](#).

EXAMPLE

Create a component node and assign it to a geometry and an analytic function.

Code for Use with Java

```
model.component().create("comp1");
model.component("comp1").geom().create("geom1", 3);
model.component("comp1").func().create("an1", "Analytic");
```

Code for Use with MATLAB

```
model.component.create('comp1');
model.component('comp1').geom.create('geom1', 3);
model.component('comp1').func.create('an1', 'Analytic');
```

model.constr()

Creating and modifying constraints in a model.

SYNTAX

```
model.constr().create(<tag>,<shtags>);
model.constr().create(<tag>,<nglobal>);
model.constr(<tag>).shape(<shtags>);
model.constr(<tag>).shape(<pos>,<shtags>);
model.constr(<tag>).global(<nglobal>);
model.constr(<tag>).create(<ftag>);
model.constr(<tag>).feature(<ftag>).set(ctype,<value>);

model.constr(<tag>).shape();
model.constr(<tag>).global();
model.constr(<tag>).feature(<ftag>).getType(ctype);

model.constr(<tag>).feature(<ftag>).set(String pname, int value);
model.constr(<tag>).feature(<ftag>).set(String pname, int pos, int value);
model.constr(<tag>).feature(<ftag>).set(String pname, int pos, int[] value);
model.constr(<tag>).feature(<ftag>).set(String pname, int pos1, int pos2, int value);
model.constr(<tag>).feature(<ftag>).setIndex(String name, String value, int index);
model.constr(<tag>).feature(<ftag>).setIndex(String name, String value, int firstIndex,
int secondIndex);
model.constr(<tag>).feature(<ftag>).setIndex(String name, String[] value, int index);
model.constr(<tag>).feature(<ftag>).setIndex(String name, double value, int index);
model.constr(<tag>).feature(<ftag>).setIndex(String name, double value, int firstIndex,
int secondIndex);
model.constr(<tag>).feature(<ftag>).setIndex(String name, double[] value, int index);
model.constr(<tag>).feature(<ftag>).setIndex(String name, int value, int index);
model.constr(<tag>).feature(<ftag>).setIndex(String name, String value, int index);
model.constr(<tag>).feature(<ftag>).setIndex(String name, String value, int index);
model.constr(<tag>).feature(<ftag>).setIndex(String name, int value, int firstIndex,
int secondIndex);
model.constr(<tag>).feature(<ftag>).setIndex(String name, int[] value, int index);
```

The `set()` methods index/position arguments are 1-based. The `setIndex()` methods index/position arguments are 0-based. See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.constr(<tag>)` returns the constraint with tag `<tag>`.

`model.constr().create(<tag>,<shtags>)` creates a constraint with tag `<tag>` using the shape functions `<shtags>`.

`model.constr().create(<tag>,<nglobal>)` creates a global constraint with tag `<tag>` expecting `<nglobal>` components.

`model.constr(<tag>).shape(<shTags>)` points to the shape functions associated with the constraint. Reassigning the shape functions can have the side effect of modifying the constraints since the number of constraints can change as the size of each constraint vector can change.

`model.constr(<tag>).global(<nglobal>)` specifies that the constraint is global and sets the expected number of components.

`model.constr(<tag>).feature(<ftag>)` is a feature in the constraint with tag <tag>.

`model.constr(<tag>).create(<ftag>)` creates a constraint feature.

`model.constr(<tag>).feature(<ftag>).set(<ctype>, <value>)` sets the parameter *ctype* to <value>, where *ctype* is either `constr` or `constrf`, and <value> is a single constraint expression or a list of constraint expressions. The number of elements in the constraint expression depends on the number of global constraint components or shape functions specified, and on the shape function type. A Lagrange shape function or global constraint component requires a single item, whereas a vector shape function requires one item for each space dimension. The supported set methods are the ones for double string arrays defined in [Table 2-4](#).

`model.constr(<tag>).feature(<ftag>).selection().named(<selTag>)` assigns the constraint to the named selection <selTag>.

`model.constr(<tag>).feature(<ftag>).selection().set(...)` defines a local selection that assigns the constraint to geometric entities. For a complete list of methods available under `selection()`, see [model.selection\(\)](#). Only selections at a single geometry level is allowed in the selection.

`model.constr(<tag>).shape()` returns the shape function tags as a string array.

`model.constr(<tag>).global()` returns the number of components if the constraint is global, otherwise -1.

`model.constr(<tag>).feature(<ftag>)).`

`getAllowedPropertyValues(property)` returns the set of allowed values for a property if the set is a finite set of strings; otherwise, it returns null.

`model.constr(<tag>).feature(<ftag>).getType(<ctype>)` returns the constraint or constraint force value. For available methods, see [get* and Selection Access Methods](#).

`model.constr(<tag>).feature(<ftag>).selection().named()` returns the named selection tag, and `model.constr(<tag>).feature(<ftag>).selection().getType()` returns domain information. For available methods, see [Selections](#).

`model.constr(<tag>).feature(<ftag>).selection(<estype>).set(...)` defines a subselection of a given lower-dimensional excluded selection type that should be excluded from the constraint selection. Excluded selection type can be `exclude0` for points, `exclude1` for edges, and `exclude2` for excluded face subselections. The constraint is not enforced on the specified excluded subselections. Excluded subselections have to have lower dimension than the constraint selection. For a complete list of methods available under `selection(<estype>)`, see [model.selection\(\)](#).

EXAMPLES

Code for Use with Java

Set several constraint by using multiple constraints:

```
model.constr().create("c1", new String[]{"shu", "shv"});
ConstrFeature f = model.constr("c1").create("f1");
f.set("constr", new String[]{"u-1", "v"});
f.selection().geom("geom1", 1);
f.selection().all();
```

Vector elements need a set of constraints:

```

model.constr().create("c2",new String[]{"shE"});
ConstrFeature f = model.constr("c2").create("f1");
f.set("constr",new String[]{"Ex-1","Ey-0","Ez-0"});
f.selection().geom("geom1",1);
f.selection().all();

```

Code for Use with MATLAB

```

model.constr.create('c1',{'shu','shv'});
f = model.constr('c1').create('f1');
f.set('constr',{'u-1','v'});
f.selection.geom('geom1',1);
f.selection.all;

```

Vector elements need a set of constraints:

```

model.constr.create('c2',{'shE'});
f = model.constr('c2').create('f1');
f.set('constr',{'Ex-1','Ey-0','Ez-0'});
f.selection.geom('geom1',1);
f.selection.all;

```

SEE ALSO

[model.shape\(\)](#)

model.coordSystem()

Add coordinate systems, perfectly matched layers, infinite elements, and absorbing layers. Perfectly matched layers, infinite elements, and absorbing layers are all available with a set of add-on products only.



The syntax that includes the component level, such as `model.component(<ctag>).coordSys()` ... is the default and is used throughout this chapter. To use the earlier `model.coordSys()` ... syntax, clear the **Generate code using component syntax** checkbox on the **Application Builder>Methods** page in the **Preferences** window.

SYNTAX

```

model.component(<ctag>).coordSystem().create(<tag>,<gtag>,type);
model.component(<ctag>).coordSystem(<tag>).set(property,<value>);
model.component(<ctag>).component(<ctag>).coordSystem(<tag>).
    setIndex(property,<value>,row);
model.component(<ctag>).coordSystem(<tag>).setIndex(property,<value>,row,col);
model.component(<ctag>).coordSystem(<tag>).selection();

model.component(<ctag>).coordSystem(<tag>).coord()
model.component(<ctag>).coordSystem(<tag>).isOrthonormal()
model.component(<ctag>).coordSystem(<tag>).isLinear()
model.component(<ctag>).coordSystem(<tag>).image()

```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.component(<ctag>).coordSystem().create(<tag>,<gtag>,type)` creates a coordinate system with tag `<tag>` on geometry `<gtag>` of type `type`. There are the following types of coordinate systems: mapped system (Mapping), base-vector system (VectorBase), rotated system (Rotated), boundary system (Boundary), scaling system (Scaling), cylindrical system (Cylindrical), and system from geometry (SystemFromGeometry). The boundary system only applies to boundaries. In addition, the perfectly matched layers (PMLs), infinite elements, and absorbing layers are also implemented as types of coordinate systems: PML, InfiniteElement, and AbsorbingLayer, respectively.

`model.component(<ctag>).coordSystem(<tag>).selection().named(<seltag>)` assigns the coordinate system to the named selection `<seltag>`.

`model.component(<ctag>).coordSystem(<tag>).selection().set(...)` defines a local selection that assigns the coordinate system to geometric entities. For a complete list of methods available under `selection()`, see [Selections](#). The selection method is only available for coordinate systems of the following types: `Scaling`, `PML`, `InfiniteElement`, and `AbsorbingLayer`.

`model.component(<ctag>).coordSystem(<tag>).set("orthonormal", "on")` specifies that this is an orthonormal system. This affects the internal calculation of systems, so some simplifications on expressions can be made. It is recommended to use this option when possible. Boundary systems, rotated systems, and cylindrical system are always orthonormal.

TABLE 2-52: COMMON PROPERTIES FOR COORDINATE SYSTEMS.

PROPERTY	VALUE	DESCRIPTION
name	String	Coordinate system name.

Mapping

`model.component(<ctag>).coordSystem().create(<tag1>, <gtag>, "Mapping")` creates a mapped system. In a mapped system you specify the coordinate mapping given in some of the available frame coordinates (usually `x`, `y`, `z`).

TABLE 2-53: PROPERTIES FOR MAPPING SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	String matrix	[(x1,x2,x3)]	Coordinate names.
map	String array	(x,y,z)	The map.
orthonormal	boolean	false	If the system is orthonormal.
frametype	mesh material spatial geometry	spatial	The frame type.

`model.component(<ctag>).coordSystem(<tag1>).setIndex("map", "x+1", 0)` sets the mapping of the first coordinate system coordinate to be a function of the first frame coordinate, `x`.

`model.component(<ctag>).coordSystem(<tag1>).setIndex("map", "y+1", 2)` sets the mapping of the third coordinate system coordinate to be a function of the second frame coordinate `y`.

VectorBase

`model.component(<ctag>).coordSystem().create(<tag2>, "VectorBase")` creates a base-vector system. In a base-vector system you specify the base vectors given as components of a frame system. If the components are independent of frame coordinates this is a linear system and can be applied for any frame.

TABLE 2-54: PROPERTIES FOR BASE VECTOR SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	String matrix	[(x1,x2,x3)]	Coordinate names, set using <code>setIndex</code> syntax.
base	String matrix	[(1,0,0) (0,1,0) (0,0,1)]	Base vectors.
frametype	mesh material spatial geometry	spatial	Frame type for the specification of the origin.
makeorthonormal	boolean	false	Make the system orthonormal.
origin	String array	(0,0,0)	Origin of system; set using <code>setIndex</code> syntax.
orthonormal	boolean	false	If the system is orthonormal or not.

TABLE 2-54: PROPERTIES FOR BASE VECTOR SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outofplane	String	2 in 2D, 1, 2 in 1D	Out-of-plane index.
workplanesrc	this part-Instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

`model.component(<ctag>).coordSystem(<tag2>).setIndex("base", "1", 0, 1)` sets the first base vector's second component to one. As an alternative, it is possible to specify the full base-vector matrix using the following syntax:

`model.component(<ctag>).coordSystem(<tag2>).set("base", new String[][]{{"0", "1", "0"}, {"0", "0", "1"}, {"1", "0", "0"}})` sets the base vector matrix so the first base vector is equal to the *y*-axis of the frame system, the second is the *z*-axis, and so on. In 2D, you only use a two rows and two columns from the full base vector matrix for the in-plane base vectors. As an option, it is therefore possible to specify which of the coordinate system base vectors that corresponds to the out-of-plane axis in the frame system. Internally, this base vector always gets the components {"0", "0", "1"}. The third column is also set using these components. To make a general 3D system in 2D, you must use the mapped system.

`model.component(<ctag>).coordSystem(<tag2>).set("outofplane", "2")` sets the third base vector to represent the out-of-plane vector (*z*-axis in 2D). The value is zero based. In 1D the out-of-plane index is set using the syntax "1,2" to set second and third base vectors to represent the out-of-plane vector.

Rotated

`model.component(<ctag>).coordSystem().create(<tag3>, "Rotated")` creates a rotated system. In 3D you specify the Euler angles, which correspond to sequential rotation first about, by default, the *z*-axis, then the *x*-axis, and finally the *z*-axis again. In 2D you can either specify a full 3D rotation or a rotation only about a selected out-of-plane axis.

TABLE 2-55: PROPERTIES FOR ROTATED SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angle	String array	(0,0,0)	Rotation angles.
coord	String matrix	[(x1,x2,x3)]	Coordinate names.
method	String	inPlane	Input method in 2D.
frametype	mesh material spatial geometry	spatial	Frame type for the specification of the origin.
inPlaneAngle	String	0	Rotation angle when method is inPlane.
origin	String array	(0,0,0)	Origin of system; set using setIndex syntax.
outofplane	String	2 in 2D, 1, 2 in 1D	Out-of-plane index.
rotationSequence	XYX XYZ XZX XZY YXY YXZ YZX YZY ZXY ZXZ ZYX ZYZ	ZXZ	The rotation sequence for the Euler angles.
workplanesrc	this part-Instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

`model.component(<ctag>).coordSystem(<tag3>).setIndex("angle", "12[deg]", 0)` sets the first rotation about the z-axis to 12 degrees. The default unit for angles are radians.

Boundary

`model.component(<ctag>).coordSystem().create(<tag4>, <gtag>, "Boundary")` creates a new boundary system, which is a local base vector system on 2D boundaries ($\mathbf{t}$, $\mathbf{n}$) and on 3D boundaries ($\mathbf{t}_1$, $\mathbf{t}_2$, $\mathbf{n}$). There is always one boundary system added by default for each geometry.

TABLE 2-56: PROPERTIES FOR BOUNDARY SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	String matrix	[(x1, x2, x3)]	Coordinate names,
frametype	mesh material spatial geometry	spatial	Frame type,
reversenormal	boolean	false	Reverse normal direction.
tangent	String array		Tangent direction.
mastersystem	manual globalCartesian <tag>	globalCartesian	Which system to create first tangential direction from.
mastercoordsyscomp	String	"2" in axisymmetry, "3" otherwise	Which axis to create first tangential direction from.

`model.component(<ctag>).coordSystem(<tag4>).set("reversenormal", "on")` flips the normal direction for this system, so that it is opposite to the normal direction given by the geometry.

`model.component(<ctag>).coordSystem(<tag4>).set("mastersystcomp", "2")` sets the first tangential direction from the second axis of the specified master system.

`model.component(<ctag>).coordSystem(<tag4>).set("mastersystem", "manual")` specifies that no master system is used and that the tangential direction must be entered by the user.

`model.component(<ctag>).coordSystem(<tag4>).setIndex("tangent", "1")` sets the first component of the first tangential direction.

In addition, you can add `ReverseNormal` and `DomainNormal` subfeatures to reverse the normal for some boundaries and switch the normal direction on the exterior of some domains, respectively.

`model.component(<ctag>).coordSystem(<tag4>).create(<rntag>, "ReverseNormal")` creates a `ReverseNormal` subfeature. To it, you then assign a boundary selection. For boundary 3, for example, use `model.component(<ctag>).coordSystem(<tag4>).feature(<rntag>).selection().set(3);`

`model.component(<ctag>).coordSystem(<tag4>).create(<dntag>, "DomainNormal")` creates a `DomainNormal` subfeature. To it, you then assign a domain selection. For domain 2, for example, use `model.component(<ctag>).coordSystem(<tag4>).feature(<dntag>).selection().set(3);`. There is one property for the `DomainNormal` subfeature: `normalDirection`, which can be a string outward (the default) or inward.

Cylindrical

`model.component(<ctag>).coordSystem().create(<tag5>, <gtag>, "Cylindrical")` creates a cylindrical coordinate system, which you can use in 2D and 3D where rotational symmetry about the axis is required. You can specify the origin, axis direction and radial base vector.

TABLE 2-57: PROPERTIES FOR CYLINDRICAL SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	String matrix	[(r, phi, a)]	Coordinate names.
origin	String array	(0,0,0)	Origin of system.

TABLE 2-57: PROPERTIES FOR CYLINDRICAL SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	String array	(0,0,1)	Axis direction.
radialbasevector	String array	(1,0,0)	Radial base vector direction a $j = 0$.
workplanesrc	this part-Instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

`model.component(<ctag>).coordSystem(<tag5>).set("origin", new String[]{"1","0","0"})` sets the origin to (1,0,0).

Spherical

`model.component(<ctag>).coordSystem().create(<tag6>,<gtag>,"Spherical")` creates a spherical coordinate system, which you can use in 3D to define a field or property using spherical coordinates. You can specify the origin, zenith axis ($\theta = 0$), and azimuth axis ($\theta = \pi/2$, $\varphi = 0$).

TABLE 2-58: PROPERTIES FOR SPHERICAL SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	String matrix	[(r, phi, a)]	Coordinate names.
origin	String array	(0,0,0)	Origin of system.
axis	String array	(0, 0, 1)	Zenith axis direction.
radialbasevector	String array	(1, 0, 0)	Azimuth axis direction.
workplanesrc	this part-Instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

`model.component(<ctag>).coordSystem(<tag6>).set("origin", new String[]{"1","0","0"})` sets the origin to (1,0,0).

FromGeometry

`model.component(<ctag>).coordSystem().create(<tag7>,<gtag>,"FromGeometry")` creates a coordinate system taken from a work plane in a 3D geometry or defined in a 3D geometry part that is included in the geometry as a part instance. You can specify which work plane to use:

TABLE 2-59: PROPERTIES FOR SYSTEM FROM GEOMETRY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	String matrix	[(r, phi, a)]	Coordinate names.
frametype	string (mesh material spatial geometry)	material	The frame type.
workplane	String	xyplane	Name of work plane to use. The default value represents a global Cartesian coordinate system.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

Scaling

`model.component(<ctag>).coordSystem().create(<tag8>,<gtag>,"Scaling")` creates a scaling coordinate system that maps the geometry, as represented by the independent coordinates of an underlying frame, onto a virtual geometry represented by virtual scaling system coordinates. Physics interfaces that support infinite elements or perfectly matched layers accept the scaling system coordinates as being the physical domain, in which the underlying frame coordinates are seen as a parameterization. Therefore, using a scaling coordinate system you can arbitrarily deform the domain.

TABLE 2-60: PROPERTY FOR SCALING SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
map	String array	(x,y,z)	Coordinate mapping.

`model.component(<ctag>).coordSystem(<tag8>).setIndex("map", "y+1", 1)` sets the second coordinate mapping to y+1.

Combined

`model.component(<ctag>).coordSystem().create(<tag9>,<gtag>,"Combined")` creates a combined coordinate system that makes it possible to use different coordinate systems in different domains, for example. To add a coordinate system to the combined system and define it on domain 2, use

```
model.component(<ctag>).coordSystem(<tag9>).create(<tag91>,"VectorBase");
model.component("comp3").coordSystem(<tag9>).feature(<tag91>).selection().
    set(new int[] {2});
```

You can specify the frame and the coordinate names for the combined system:

TABLE 2-61: PROPERTIES FOR A COMBINED SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	String matrix	[(r, phi, a)]	Coordinate names.
frametype	string (mesh material spatial geometry)	material	The frame type.

Composite

`model.component(<ctag>).coordSystem().create(<tag9>,<gtag>,"Composite")` creates a composite coordinate system that defines a new coordinate system by multiplying the transformation matrices of two other coordinate systems.

TABLE 2-62: PROPERTIES FOR A COMBINED SYSTEM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
baseSystem	none <coordsys_tag>	none	Reference to a base system.
relativeSystem	none <coordsys_tag>	none	Reference to a system defining its axes relative to the base system axes.

PML

`model.component(<ctag>).coordSystem().create(<tag10>,<gtag>,"PML")` creates a PML (perfectly matched layer), which acts as an artificial absorbing layer in a domain for a wave simulation. You can specify the type and scaling of the PML using the following properties:

TABLE 2-63: PROPERTIES FOR PML

PROPERTY	VALUE	DEFAULT	DESCRIPTION
d	String	"x"	The distance function for each stretching direction, if <code>ScalingType</code> is set to <code>userDefined</code> . Set using the <code>setIndex</code> function.
directions	1, 2, or 3	1	Number of directions, if <code>ScalingType</code> is set to <code>userDefined</code> .
dmax	String	"1 [m]"	The thickness for each stretching direction, if <code>ScalingType</code> is set to <code>userDefined</code> . Set using the <code>setIndex</code> function.
imagFunction	none, or any defined function	none	Imaginary part of stretching function when <code>stretchingType</code> is set to <code>userDefined</code> .
PMLfactor	double	1	PML scaling factor.
PMLgamma	double	1	PML scaling curvature factor.
realFunction	none, or any defined function	none	Real part of stretching function when <code>stretchingType</code> is set to <code>userDefined</code> .
ScalingType	Cartesian Cylindrical Spherical userDefined	Cartesian	The PML scaling type. Spherical is only available in 3D.
stretchingType	polynomial rational userDefined	polynomial	The coordinate stretching type for the PML scaling.
typicalWavelength	double	1	Typical wavelength for the waves.
wavelengthSourceType	fromPhysics userDefined	fromPhysics	Take the wavelength from the physics or a user-defined wavelength.

InfiniteElement

`model.component(<ctag>).coordSystem().create(<tag11>,<gtag>,"InfiniteElement")` creates an infinite element, which acts as an unbounded domain for a simulation. You can specify the type and scaling of the infinite element using the following properties:

TABLE 2-64: PROPERTIES FOR INFINITE ELEMENT

PROPERTY	VALUE	DEFAULT	DESCRIPTION
r0	double array	0; 0 (2D) 0; 0; 0 (3D)	Center coordinates for cylindrical infinite elements.
raxis	double array	0; 0; 0	Center axis direction for cylindrical infinite elements in 3D,
pole	double	dGeomChar	Pole distance
ScalingType	Cartesian Cylindrical Spherical userDefined	Cartesian	The scaling type for the infinite element. Spherical is only available in 3D.
width	double	1e3*dGeomChar	Physical width.
directions	integer	1	Number of stretching directions (for <code>userDefined</code> scaling).

TABLE 2-64: PROPERTIES FOR INFINITE ELEMENT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
d	String array	x; y, z	Distance function for stretching directions (1–3).
dmax	double array	1; 1, 1	Thickness function for stretching directions (1–3).

AbsorbingLayer

`model.component(<ctag>).coordSystem().create(<tag12>,<gtag>,"AbsorbingLayer")` creates an absorbing layer, which acts as an effective nonreflecting-like boundary condition in transient simulations using a time-explicit solver. You can specify the type and scaling of the absorbing layer using the following properties:

TABLE 2-65: PROPERTIES FOR ABSORBING LAYER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
r0	double array	0; 0 (2D) 0; 0; 0 (3D)	Center coordinates for cylindrical absorbing layers.
raxis	double array	0; 0; 0	Center axis direction for cylindrical absorbing layers in 3D,
pole	double	dGeomChar	Pole distance
ScalingType	Cartesian Cylindrical Spherical userDefined	Cartesian	The scaling type for the absorbing layer. Spherical is only available in 3D.
width	double	1e3*dGeomChar	Physical width.
directions	integer	1	Number of stretching directions (for userDefined scaling).
d	String array	x; y, z	Distance function for stretching directions (1–3).
dmax	double array	1; 1, 1	Thickness function for stretching directions (1–3).

EXAMPLE

Create a cylindrical coordinate system with a radial base vector direction that is (0, 1, 0); that is, a coordinate that points in the global y direction:

Code for Use with Java

```
model.component("comp1").coordSystem().create("sys2", "geom1", "Cylindrical");
model.component("comp1").coordSystem("sys2").setIndex("radialbasevector", "1", 1);
model.component("comp1").coordSystem("sys2").setIndex("radialbasevector", "0", 0);
```

Code for Use with MATLAB

```
model.component('comp1').coordSystem.create('sys2', 'geom1', 'Cylindrical');
model.component('comp1').coordSystem('sys2').setIndex('radialbasevector', '1', 1);
model.component('comp1').coordSystem('sys2').setIndex('radialbasevector', '0', 0);
```

model.cpl()

Add nonlocal couplings.

SYNTAX

```
model.component(<ctag>).cpl().create(<tag>,type);
model.component(<ctag>).cpl().create(<tag>,type,<gttag>);
model.component(<ctag>).cpl(<tag>).set(property,<value>);
model.component(<ctag>).cpl(<tag>).set("opname",<opname>);
model.component(<ctag>).cpl(<tag>).selection(property).named(<seltag>);
model.component(<ctag>).cpl(<tag>).selection(property).set(...);
model.component(<ctag>).cpl(<tag>).create(<subtag>,subtype);
model.component(<ctag>).cpl(<tag>).feature(<subtag>).set(property,<value>);

model.component(<ctag>).cpl(<tag>).getType(property,<value>);
model.component(<ctag>).cpl(<tag>).feature(<subtag>).getType(property,<value>);
model.component(<ctag>).cpl(<tag>).image();

model.component(<ctag>).cpl(<tag>).set(property,<value>);
model.component(<ctag>).cpl(<tag>).setIndex(property,<value>,<index1>);
model.component(<ctag>).cpl(<tag>).setIndex(property,<value>,<index1>,<index2>);
```

The `set()` methods index/position arguments are 1-based. The `setIndex()` methods index/position arguments are 0-based. See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.component(<ctag>).cpl().create(<tag>,type)` creates a nonlocal coupling of type *type*.
`model.component(<ctag>).cpl().create(<tag>,type,<gttag>)` creates a nonlocal coupling on the geometry *<gttag>*. The supported types are `GeneralExtrusion`, `LinearExtrusion`, `BoundarySimilarity`, `IdentityMapping`, `GeneralProjection`, `LinearProjection`, `Integration`, `Average`, `Maximum`, and `Minimum`. The nonlocal coupling operators provide coupling of values, typically from a source to a destination between or within model components.

`model.component(<ctag>).cpl(<tag>).selection().named(<seltag>)` assigns the nonlocal coupling's source to the named selection *<seltag>*. `model.cpl(<tag>).selection().set(...)` defines a local selection that assigns the nonlocal coupling's source to geometric entities. For a complete list of methods available under `selection()`, see [Selections](#).

`model.component(<ctag>).cpl(<tag>).set(property,<value>)` specifies properties relevant for the selected nonlocal coupling type; see below.

`model.component(<ctag>).cpl(<tag>).set("opname",<opname>)` sets the operator name of the nonlocal coupling. The default nonlocal coupling operator name is *<tag>*.

`model.component(<ctag>).cpl(<tag>).selection(property).named(<seltag>)` assigns the nonlocal coupling's selection property to the named selection *<seltag>*.

`model.component(<ctag>).cpl(<tag>).selection(property).set(...)` defines a local selection that assigns the nonlocal coupling's selection property to geometric entities. For a complete list of methods available under `selection()`, see [model.selection\(\)](#).

`model.component(<ctag>).cpl(<tag>).create(<subtag>,subtype)` creates a subfeature of type *subtype*. This can only be done when the nonlocal coupling *type* is `BoundarySimilarity`. The supported values of subtype are `OnePointMap`, `TwoPointMap`, and `EdgeMap`.

`model.component(<ctag>).cpl(<tag>).selection().named()` returns the named source selection of the coupling.

`model.component(<ctag>).cpl(<tag>).selection().getType(...)` queries the source selection.

`model.component(<ctag>).cpl(<tag>).getType(property)` returns the value of a specified property.

`model.component(<ctag>).cpl(<tag>).selection(property).named()` returns the named selection tag of the selection property.

`model.component(<ctag>).cpl(<tag>).selection(property).getType(...)` queries a selection property.

Use `model.component(<ctag>).cpl(<tag>).image()` method to create a plot or export images. See [Plotting and Exporting Images](#).

Notation: `srcedim` = dimension of source selection; `srcsdim` = space dimension of source geometry.

INTEGRATION COUPLINGS

Integration

By default, a *nonlocal integration coupling* operator `oper` integrates an expression `e` over the source selection. The resulting value `oper(e)` can be used anywhere. If `method=summation`, the expression is instead summed over the nodes in the source selection.

TABLE 2-66: INTEGRATION COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axisym	on off	on	Multiply by $2\pi r$ in axisymmetric geometries.
frame	mesh material spatial geometry	spatial	Frame to integrate in (determines volume element).
intorder	String	4	Integration order.
method	integration summation	integration	Method of computation.

Average

A *nonlocal average coupling* operator `oper` integrates an expression `e` over the source selection and divides with the measure of the source selection. The resulting value `oper(e)` can be used anywhere.

TABLE 2-67: AVERAGE COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axisym	on off	on	Multiply by $2\pi r$ in axisymmetric geometries.
frame	mesh material spatial geometry	spatial	Frame to integrate in (determines volume element).
intorder	String	4	Integration order.

Maximum/Minimum

A *nonlocal maximum* or *minimum coupling* operator `oper` finds the maximum or minimum of an expression `e` over the source selection. The resulting value `oper(e)` can be used anywhere. An optional second argument is evaluated at the point where the first argument has its maximum or minimum. Use `x`, `y`, or `z`, for example, to get the coordinate location of the maximum or minimum.

TABLE 2-68: MAXIMUM/MINIMUM COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
intorder	String	4	Integration rule.
lagrange	String	2	Lagrange order.
points	node integration lagrange	node	Type of point.

The maximum or minimum is approximated by evaluating the expression in the specified points.

Line Integration

By default, a *line integration coupling* operator *oper* evaluates the integral of a general operator argument expression, the integrand, on a line through a meshed source domain selection in 2D or 3D. The resulting value *oper*(**e**) can be used anywhere.

TABLE 2-69: LINE INTEGRATION COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axisym	on off	on	Multiply by $2\pi r$ in axisymmetric geometries.
direxprs	sdim-by-1 string array	{"1", "0", "0"} (in 3D)	Direction expressions, when dirinput is set to direxprs.
dirinput	direxprs opargs	direxpr	Use fixed expressions or operation arguments for the line direction.
frame	mesh material spatial geometry	spatial	Frame to integrate in (determines volume element).
intorder	String	4	Integration order.
linetype	unidirectional bidirectional	unidirectional	Method of computation.

EXTRUSION COUPLINGS

A *nonlocal extrusion coupling* operator *oper* maps an expression **e** defined on (a part of) the source selection to an expression *oper*(**e**) that can be evaluated on (a part of) the destination geometries. For each point p_s in the source selection, there can be zero, one, or several corresponding points p_d in the destination. The inverse mapping $p_s = m(p_d)$ is always one-to-one. The value of *oper*(**e**) at the point p_d is defined as the value of **e** at the point p_s .

The inverse mapping *m* is specified as the composition of a *destination map* m_d and the inverse of a *source map* m_s : $p_s = m(p_d) = m_s^{-1}(m_d(p_d))$. In other words, $m_s(p_s) = m_d(p_d)$ — both the destination map and the source map into the same *intermediate space*. For all operator types except GeneralExtrusion, the intermediate space coincides with the source geometry. The source map is always one-to-one. By default, the source map is the identity.

The operator type determines the type of destination map:

TABLE 2-70: EXTRUSION COUPLING TYPES.

COUPLING TYPE	DESTINATION MAP
GeneralExtrusion	Nonlinear map described by expressions.
LinearExtrusion	Linear map described by vertex mapping.
BoundarySimilarity	Similarity transformation described by mapping of boundaries. Also used by copy mesh.
IdentityMapping	Identity map.

For most of these coupling types, a source map described by (possibly nonlinear) expressions can be used.

TABLE 2-71: EXTRUSION COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
exttol	double	0.3	Extrapolation tolerance in mesh search.
manualsearchdist	on off	off	Use manual control of search distance.
method	usetol closest walk	usetol	Mesh search method: A tolerance, closest point, or walk in a connected component.
searchdist	positive distance	0.01	Search distance in mesh search if manualsearchdist is set to on.
usenan	on off	off	Use NaN instead of error message when source point is outside selection.

If *method*=usetol, *oper*(**e**) is defined when the source point p_s is within the source selection, or if it is slightly outside. The tolerance is given in the property *exttol*, which is a distance in mesh element local coordinates; that

is, it is a measure relative to the mesh element size. If `oper(e)` is not defined, an error message is given (if `usenon=off`), or the value NaN is returned (if `usenon=on`).

If `method=closest`, a brute force search method is used, which makes `oper(e)` defined everywhere (the nearest point to p_s in the source selection is used).

If `method=walk`, a walk through the source mesh is made. This method should not be used if the component is not connected.

If `manualsearchdist=on` and the source selection is a boundary, edge, or point selection, `searchdist` gives the search distance in directions orthogonal to the source selection. The default unit is the component's length unit. If `manualsearchdist=off`, the search distance is equal to `exttol` times the mesh element size. If `method=closest`, `exttol` and `searchdist` are not used.

Depending on the coupling type, additional properties are available (see below).

GeneralExtrusion

A *nonlocal general extrusion coupling* operator maps an expression defined on a source to an expression that can be evaluated on any destination geometry where the destination map expressions are valid.

TABLE 2-72: GENERAL EXTRUSION MAP PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>dstmap</code>	String array	spatial coordinates	Expressions for destination map $m_d(p_d)$.
<code>srcframe</code>	mesh material spatial	spatial	Frame for source mesh.
<code>srcmap</code>	String array	spatial coordinates	Expressions for source map $m_s(p_s)$.
<code>usesrcmap</code>	on off	off	Use source map.

Trailing empty expressions in the properties `dstmap` and `srcmap` are ignored. The remaining expressions must be equal in number, and this determines the dimension `idim` of the intermediate space. Requirement: `srcdim <= idim <= srcdim`. Changing the source selection has the side effect of changing `dstmap` and `srcmap` so that this requirement is satisfied. By default, `idim=srcdim`.

The source mesh is viewed in the frame `srcframe`. The source mapping is taken to be linear within each source mesh element.

If `usesrcmap=off`, the `srcmap` property is not used. In this case, `dstmap` is a mapping from the destination to the source (viewed in the frame `srcframe`), and `idim=srcdim`.

LinearExtrusion

A *nonlocal linear extrusion coupling* operator linearly maps an expression defined on a source to an expression that can be evaluated in the destination.

TABLE 2-73: LINEAR EXTRUSION MAP PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>srcvertexN</code>	Selection		Source vertex number N
<code>dstgeom</code>	String	source geometry	Destination geometry
<code>dstvertexN</code>	Selection		Destination vertex number N
<code>srcframe</code>	mesh material spatial	spatial	Frame for evaluation of source vertex coordinates
<code>dstframe</code>	mesh material spatial	spatial	Frame for evaluation of destination vertex coordinates
<code>usesrcmap</code>	on off	off	Use source map
<code>srcmap</code>	string[srcdim]	spatial coordinates	Expressions for source map $m_s(p_s)$

The number of selections `srcvertexN` and `dstvertexN` is 4. These are used only for $1 \leq N \leq \text{dim}+1$, where `dim` is a number less than or equal to $\min(\text{srcdim}, \text{dstdim})$. The remaining $4 - \text{dim}$ selections should be empty.

The destination map is the following linear (affine) map from the destination geometry to the source geometry:

- 1 First, if $\text{dim} < \text{dstdim}$, an orthogonal projection onto the affine space spanned by the destination vertices. The number of destination vertices is $\text{dim}+1$. Thus, $\text{dim}=2$ gives a plane, and $\text{dim}=1$ gives a line.
- 2 Then, a linear (affine) map mapping the destination vertices onto the source vertices.

BoundarySimilarity (3D)

A *nonlocal boundary similarity coupling* operator maps an expression defined on a part of a boundary to another part of a boundary with the same shape.

TABLE 2-74: BOUNDARY SIMILARITY PROPERTIES IN 3D.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>destination</code>	Selection		Destination face.
<code>usesrcmap</code>	on off	off	Use source map.
<code>srcmap</code>	string[srcdim]	spatial coordinates	Expressions for source map $m_s(p_s)$.

The destination transformation is a similarity transformation that maps a destination face (`destination`) onto a set of source faces (the source selection). The mesh is always viewed in the mesh frame.

By default, the algorithm automatically chooses a transformation when symmetries make several transformations possible. To control this choice, one of the following subfeatures can be added in 3D.

TABLE 2-75: SUBFEATURE TYPES.

SUBFEATURE	REMARKS
EdgeMap	Specify how one source edge is mapped.
OnePointMap	Specify how one source vertex is mapped.
TwoPointMap	Specify how two source vertices are mapped.

EdgeMap

An *edge map* specifies that a certain destination edge should be mapped onto a certain source edge. Their relative direction is given by the property `direction`. The edges must be adjacent to the given faces.

TABLE 2-76: PROPERTIES FOR EDGEMAP SUBFEATURE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>srcedge</code>	Selection		Source edge.
<code>dstedge</code>	Selection		Destination edge.
<code>direction</code>	auto same opposite	auto	Edge direction.

OnePointMap

A *one-point map* specifies that a certain destination vertex should be mapped onto a certain source vertex.

TABLE 2-77: PROPERTIES FOR ONEPOINTMAP SUBFEATURE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>srcpoint1</code>	Selection		Vertex on source face.
<code>dstpoint1</code>	Selection		Vertex on destination face.

TwoPointMap

A *two-point map* specifies that two destination vertices should be mapped onto two source vertices.

TABLE 2-78: PROPERTIES FOR ONEPOINTMAP SUBFEATURE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
srcpoint1	Selection		Vertex 1 on source face.
srcpoint2	Selection		Vertex 2 on source face.
dstpoint1	Selection		Vertex 1 on destination face.
dstpoint2	Selection		Vertex 2 on destination face.

BoundarySimilarity (2D)

TABLE 2-79: BOUNDARY SIMILARITY PROPERTIES IN 2D.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
destination	Selection		Destination edge.
direction	auto same opposite	auto	Edge direction.
usesrcmap	on off	off	Use source map.
srcmap	string[srcsdim]	spatial coordinates	Expressions for source map $m_s(p_s)$.

The destination transformation is a similarity transformation that maps a destination edge (*destination*) onto a set of source edges (the source selection). Their relative direction is given by the property *direction*. The mesh is always viewed in the mesh frame.

IdentityMapping

A *nonlocal identity mapping coupling* operator maps between geometric entities that overlap, possibly when viewed in different frames. The destination transformation is an identity mapping between the given frames.

TABLE 2-80: IDENTITY MAPPING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
dstframe	mesh material spatial	spatial	Frame for evaluation of destination coordinates.
srcframe	mesh material spatial	spatial	Frame for evaluation of source coordinates.

PROJECTION COUPLINGS

A projection coupling operator *oper* maps an expression *e* defined on (a part of) the source selection to an expression *oper* (*e*) that can be evaluated on (a part of) the destination geometries. It does so by performing integration along curves in the source selection. These curves correspond to lines in an *intermediate space*, whose dimension is equal to *srcdim*. There is a *source map* m_s mapping the source selection into the intermediate space, and a *destination map* m_d mapping the destination geometries into the subspace of intermediate space where the last coordinate is zero. The source map is always one-to-one. The value of *oper* (*e*) at a destination point p_d is defined as follows:

- 1 In the intermediate space, consider the line that is parallel to the last coordinate axis and goes through the point $m_d(p_d)$.
- 2 Map this line to a curve in the source selection using the inverse of the source map.
- 3 Integrate the expression *e* over this curve.

This implies that the value of *oper* (*e*) at the destination point p_d is the integral of *e* along a curve through the source point $p_s = m_s^{-1}(m_d(p_d))$.

The coupling type determines the type of the maps:

TABLE 2-81: PROJECTION OPERATOR TYPES.

COUPLING TYPE	MAP TYPES
GeneralProjection	Nonlinear map described by expressions.
LinearProjection	Linear map described by vertex mapping.

TABLE 2-82: PROJECTION COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
intorder	int	4	Order of integration formula.

Additional properties are available depending on the coupling type; see below.

GeneralProjection

Use a *nonlocal general projection coupling* to define integration along curves.

TABLE 2-83: GENERAL PROJECTION COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
dstmap	string[srcdim-1]	spatial coordinates	Expressions for destination map $m_d(p_d)$.
srcframe	mesh material spatial	spatial	Frame for source mesh.
srcmap	string[srcdim]	spatial coordinates	Expressions for source map $m_s(p_s)$.

LinearProjection

A *nonlocal linear projection coupling* operator defines a mapping between destination and source that is given by a linear map defined by vertices. Let v be the vector from the first source vertex to the last source vertex. The value of `oper(e)` at a point p_d is equal to the integral of e over the line through the point $p_s = m_s^{-1}(m_d(p_d))$ with direction vector v .

TABLE 2-84: LINEAR PROJECTION COUPLING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
srcvertexN	Selection		Source vertex number N.
dstgeom	String	source geometry	Destination geometry.
dstvertexN	Selection		Destination vertex number N.
srcframe	mesh material spatial	spatial	Frame for evaluation of source vertex coordinates.
dstframe	mesh material spatial	spatial	Frame for evaluation of destination vertex coordinates.

The number of selections `srcvertexN` is 4. These are used only for $1 \leq N \leq \text{srcdim}+1$. The remaining selections should be empty. The number of source vertices is `srcdim+1`. The source map is a linear (affine) map that maps the source vertices onto the points $0, e_1, e_2, \dots, e_{\text{srcdim}}$ in the intermediate space, where e_i is the i th unit vector.

The number of selections `dstvertexN` is 4. These are used only for $1 \leq N \leq \text{srcdim}$. The remaining selections should be empty. The number of destination vertices is `srcdim`. The destination map is the following linear (affine) map from the destination geometry to the intermediate space:

- 1 First, if `srcdim-1 < dstsdim`, an orthogonal projection onto the affine space spanned by the destination vertices. Thus, `srcdim=3` gives a plane, and `srcdim=2` gives a line.
- 2 Then, a linear (affine) map mapping the destination vertices onto the points $0, e_1, e_2, \dots, e_{\text{srcdim}-1}$ in the intermediate space, where e_i is the i th unit vector.

model.elem()

Creating and modifying different types of elements.

SYNTAX

```
model.elem().create(<tag>,eltype);
model.elem(<tag>).set(<ftag>,value);
model.elem(<tag>).field().create(<ftag>,"record");
model.elem(<tag>).field(<ftag>).set(<ftag>,value);
model.elem(<tag>).field().create(<atag>,"array");
model.elem(<tag>).field(<atag>).pos().create("string",value);
model.elem(<tag>).field(<atag>).pos().create("array");
model.elem(<tag>).field(<atag>).pos(pos).create("string",value);
model.elem(<tag>).src().create(<fttag>);
model.elem(<tag>).src(<fttag>).set(<ftag>,value);
model.elem(<tag>).src(<fttag>).field().create(<ftag>,"array");
model.elem(<tag>).geomdim().create(<fttag>);
```

DESCRIPTION

`model.elem().create(<tag>,eltype)` creates a new element of type `eltype`, for example `elinterp`, `elpric`, `elvar`, `elode`, and so on.

`model.elem(<tag>).set(<ftag>,value)` sets the field tagged `<ftag>` to `value`. Examples of fields and values are `file` and `solution_interp.txt`, `global` and `1`, and so on.

`model.elem(<tag>).field().create(<ftag>,"record")` creates a new field tagged `<ftag>` of type `record` under the element tagged `<tag>`.

`model.elem(<tag>).field(<ftag>).set(sname,value)` sets the field tagged `sname` to `value`. The field is located under the record field tagged `<ftag>`.

`model.elem(<tag>).field(<ftag>).field().create(<rtag>,"record")` creates a new record field tagged `<rtag>` that is a field under the record field `<ftag>`.

`model.elem(<tag>).field().create(<atag>,"array")` creates a new array field tagged `<atag>`.

`model.elem(<tag>).field(<atag>).pos().create("array")` adds a new element of type `array` to the array tagged `<atag>`.

`model...field(<atag>).pos(1).pos().create("string","1")` adds a new array element of type `string` with value `1` to the first array element of the array stored in the field `<atag>`.

In the text below, all occurrences of `src` can be replaced with `geomdim`.

`model.elem(<tag>).src().create(<fttag>)` creates a `src` feature tagged `<fttag>` under the element. A feature must have a domain selection.

`model.elem(<tag>).src(<fttag>).selection().dim(2).set(gname)` assigns all domains of dimension 2 from geometry `gname` to the selection of feature `<fttag>`.

`model.elem(<tag>).src(<fttag>).set(<ftag>,value)` sets the field `<ftag>` to `value` under the feature `<fttag>`.

`model.elem(<tag>).src(<fttag>).field().create(rname,"record")` adds a new record field `rname` under the feature `<fttag>`.

EXAMPLES

Specifies an interpolation element that takes its data from a file named `solution_data.txt`.

Code for Use with Java

```
model.elem().create("fun1","elinterp");
model.elem("fun1").set("name",new String[]{"sol"});
model.elem("fun1").set("file","solution_data.txt");
model.elem("fun1").set("fileindex",new String[]{"1"});
```

```

model.elem("fun1").set("defvars",new String[]{"true"});
model.elem("fun1").set("method",new String[]{"linear"});
model.elem("fun1").set("extmethod",new String[]{"const"});

```

Code for Use with MATLAB

```

model.elem.create('fun1','elinterp');
model.elem('fun1').set('name',{'sol'});
model.elem('fun1').set('file','solution_data.txt');
model.elem('fun1').set('fileindex',{'1'});
model.elem('fun1').set('defvars',{'true'});
model.elem('fun1').set('method',{'linear'});
model.elem('fun1').set('extmethod',{'const'});

```

The example below creates two integration couplings.

Code for Use with Java

```

model.elem().create("elem1","elcplscalar");
model.elem("elem1").set("var",new String[]{"aa","bb"});
model.elem("elem1").set("global",new String[]{"1","2"});
model.elem("elem1").src().create("feat1");
model.elem("elem1").src("feat1").selection().geom("g",2).set(new int[]{1});
model.elem("elem1").src("feat1").set("expr",new String[][]{{"1"},"2"});
model.elem("elem1").src("feat1").set("ipoints",new String[][]{{"2"},"2"});
model.elem("elem1").src("feat1").set("frame",new String[][]{{"spatial"},"spatial"});

```

Code for Use with MATLAB

```

model.elem.create('elem1','elcplscalar');
model.elem('elem1').set('var',{'aa','bb'});
model.elem('elem1').set('global',{'1','2'});
model.elem('elem1').src.create('feat1');
model.elem('elem1').src('feat1').selection.geom('g',2).set(1);
model.elem('elem1').src('feat1').set('expr',{'1'},{'2'})
model.elem('elem1').src('feat1').set('ipoints',{'2'},{'2'})
model.elem('elem1').src('feat1').set('frame',{'spatial'},{'spatial'});

```

This complicated example creates a constr element with two constraints (usually done with constraint features):

Code for Use with Java

```

model.elem().create("elem1","elsconstr");
feat = model.elem("elem1").geomdim().create("feat1");
feat.selection().geom("g",2).set(new int[]{1});
feat.set("constr",new String[][]{{"Ex","Ey","Ez"}});
feat.set("cshape",new String[]{"1"});
feat.field().create("shelem","record");
feat.field("shelem").set("case",new String[0]);
feat.field("shelem").set("mind",new String[0]);
feat.field("shelem").field().create("default","array");
feat.field("shelem").field("default").pos().create("array");
feat.field("shelem").field("default").pos(1).pos().create("array");
feat.field("shelem").field("default").pos(1).pos(1).pos().create("string","edg");
feat.field("shelem").field("default").pos(1).pos(1).pos().create("string","shcurl");
feat.field("shelem").field("default").pos(1).pos(1).pos().create("record");
feat.field("shelem").field("default").pos(1).pos(1).pos(3).set("order","2");
feat.field("shelem").field("default").pos(1).pos(1).pos(3)
.set("compnames",new String[]{"Ex","Ey","Ez"});
feat.field("shelem").field("default").pos(1).pos(3).set("frame","ref");
feat.field("shelem").field("default").pos(1).pos(1).pos().create("string","edg2");
feat.field("shelem").field("default").pos(1).pos(1).pos().create("string","shcurl");
feat.field("shelem").field("default").pos(1).pos(1).pos().create("record");
feat.field("shelem").field("default").pos(1).pos(1).pos(6).set("order","2");
feat.field("shelem").field("default").pos(1).pos(1).pos(6)
.set("compnames",new String[]{"Ex","Ey","Ez"});
feat.field("shelem").field("default").pos(1).pos(1).pos(6).set("frame","ref");

```

Code for Use with MATLAB

```
model.elem.create('elem1','elsconstr');
feat = model.elem('elem1').geomdim().create('feat1');
feat.selection().geom('g',2).set(1);
feat.set('constr',{{'Ex','Ey','Ez'}});
feat.set('cshape',{'1'})
shelem = feat.field.create('shelem','record');
shelem.set('case','');
shelem.set('mind','');
shelem.field().create('default','array');
shelem.field('default').pos.create('array');
shelem.field('default').pos(1).pos.create('array');
shelem.field('default').pos(1).pos(1).pos.create('string','edg');
shelem.field('default').pos(1).pos(1).pos.create('string','shcurl');
shelem.field('default').pos(1).pos(1).pos.create('record');
shelem.field('default').pos(1).pos(1).pos(3).set('order','2');
shelem.field('default').pos(1).pos(1).pos(3).set('compnames',{'Ex','Ey','Ez'});
shelem.field('default').pos(1).pos(1).pos(3).set('frame','ref');
shelem.field('default').pos(1).pos(1).pos.create('string','edg2');
shelem.field('default').pos(1).pos(1).pos.create('string','shcurl');
shelem.field('default').pos(1).pos(1).pos.create('record');
shelem.field('default').pos(1).pos(1).pos(6).set('order','2');
shelem.field('default').pos(1).pos(1).pos(6).set('compnames',{'Ex','Ey','Ez'});
shelem.field('default').pos(1).pos(1).pos(6).set('frame','ref');
```

For all records, the statement

```
model...set("frame","ref");
```

is the equivalent to

```
model...field().create("frame","string","ref");
```

The statement

```
model...set("expr",new String[][]{{"1"},"2"});
```

is equivalent to

```
model...field().create("expr","array");
model...field("expr").pos().create("array");
model...field("expr").pos(1).create("string","1");
model...field("expr").pos().create("array");
model...field("expr").pos(2).create("string","1");
```

so the `set` method is often a much more convenient way to create simple fields.

model.elementSet()

Mesh element sets.

SYNTAX

```
model.elementSet().create(<tag>);
model.elementSet(<tag>).set(<var>,<expr>);
model.elementSet(<tag>).remove(<var>);
model.elementSet(<tag>).model(<mtag>);

model.elementSet(<tag>).varnames();
model.elementSet(<tag>).get(<var>);
model.elementSet(<tag>).model();
model.elementSet(<tag>).scope();
```


DESCRIPTION

`model.elementSet(<tag>)` returns an element set. It contains one or several definitions of sets of mesh elements. Each element set is identified with an *element set variable* name — the variable evaluates to 1 on mesh elements that belong to the element set, and it evaluates to 0 on other mesh elements. The variable has a defining expression that is evaluated at the midpoint of each mesh element to determine whether the mesh element belongs to the element set. This evaluation is done once at the beginning of the solution process, so the expression must not depend on variables that change during the solution process. All element sets in `model.elementSet(<tag>)` are subsets of the selection `model.elementSet(<tag>).selection()`.

`model.elementSet().create(<tag>)` creates a new element set with tag `<tag>`.

`model.elementSet(<tag>).set(<var>, <expr>)` sets the defining expression for the element set variable `<var>` to `<expr>`.

`model.elementSet(<tag>).remove(<var>)` removes the element set variable `<var>`.

`model.elementSet(<tag>).model(<mtag>)` sets the model component node.

`model.elementSet(<tag>).selection().selMethod` manipulates the geometric entity selection; see [Selections](#) for a description of the available methods.

`model.elementSet(<tag>).varnames()` returns all element set variables as a string array.

`model.elementSet(<tag>).get(<var>)` returns the defining expression of element set variable `<var>`.

`model.elementSet(<tag>).model()` returns the model component node tag.

`model.elementSet(<tag>).scope()` returns the fully qualified scope name.

EXAMPLE

Let A be an element set consisting of all mesh triangles that are not adjacent to boundaries 3 or 4 in square, plus all mesh triangles that are adjacent to boundaries 1 or 2. Let the dependent variable u be defined on A. Solve Poisson's equation with Dirichlet conditions on boundaries 1 and 2:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
model.component("comp1").geom().create("geom1", 2);
model.component("comp1").geom("geom1").create("sq1", "Square");
model.component("comp1").mesh().create("mesh1", "geom1");
model.component("comp1").mesh("mesh1").run();
model.elementSet().create("es1");
model.elementSet("es1").set("A", "!bndadj(3,4) || bndadj(1,2)");
model.elementSet("es1").selection().geom(2).all();
model.shape().create("sh1", "material1");
model.shape("sh1").create("f1", "shlag");
model.shape("sh1").feature("f1").set("basename", "u").set("order", 1);
model.shape("sh1").selection().geom(2).all();
model.shape("sh1").elementSet("A");
model.field().create("field1", "u");
model.field("field1").shape(new String[]{"sh1"});
model.intRule().create("ir1", "material1");
model.intRule("ir1").create("o2").order(2);
model.weak().create("weak1");
model.weak("weak1").weak("if(A,ux*test(ux)+uy*test(uy)-test(u),0)");
model.weak("weak1").intRule("ir1");
model.weak("weak1").selection().geom(2).all();
model.constr().create("constr1", new String[]{"sh1"});
model.constr("constr1").create("f1");
model.constr("constr1").feature("f1").set("constr",1,new String[]{"u"});
model.constr("constr1").feature("f1").set("constrf",1,new String[]{"test(u)"});
```

```

model.constr("constr1").feature("f1").selection().geom(1).set(1,2);
model.study().create("std1");
model.study("std1").create("stat", "Stationary");
model.study("std1").run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
model.component('comp1').geom.create('geom1', 2);
model.component('comp1').geom('geom1').create('sq1', 'Square');
model.component('comp1').mesh.create('mesh1', 'geom1');
model.component('comp1').mesh('mesh1').run;
model.elementSet.create('es1');
model.elementSet('es1').set('A', '!bndadj(3,4) || bndadj(1,2)');
model.elementSet('es1').selection().geom(2).all;
model.shape.create('sh1', 'material1');
model.shape('sh1').create('f1', 'shlag');
model.shape('sh1').feature('f1').set('basename', 'u').set('order', 1);
model.shape('sh1').selection.geom(2).all;
model.shape('sh1').elementSet('A');
model.field.create('field1', 'u');
model.field('field1').shape({'sh1'});
model.intRule.create('ir1', 'material1');
model.intRule('ir1').create('o2').order(2);
model.weak.create('weak1');
model.weak('weak1').weak('if(A,ux*test(ux)+uy*test(uy)-test(u),0)');
model.weak('weak1').intRule('ir1');
model.weak('weak1').selection().geom(2).all;
model.constr.create('constr1', {'sh1'});
model.constr('constr1').create('f1');
model.constr('constr1').feature('f1').set('constr',1,{ 'u' });
model.constr('constr1').feature('f1').set('constrf',1,{ 'test(u)' });
model.constr('constr1').feature('f1').selection().geom(1).set([1,2]);
model.study.create('std1');
model.study('std1').create('stat', 'Stationary');
model.study('std1').run;

```

SEE ALSO

[model.shape\(\)](#)

model.externalInterface

Create an external interface for cosimulation with Simulink, for example.

SYNTAX

```

model.externalInterface().create(<tag>,<type>);
model.externalInterface(<tag>).set(<property>,<value>);
model.externalInterface(<tag>).run();

```

DESCRIPTION

`model.externalInterface().create(<tag>,<type>)` creates an extra dimension feature of the given type. The supported type is `SimulinkCosimulation` for a cosimulation with Simulink.

`model.externalInterface(<tag>).set(<property>,<value>)` specifies properties relevant for the selected external interface type.

`model.externalInterface(<tag>).run()` exports a file from the external interface.

model.extraDim()

Create attachments of extra dimensions.

SYNTAX

```
model.extraDim().create(<tag>,<type>);
model.extraDim(<tag>).set(<property>,<value>);
model.extraDim(<tag>).model(<mtag>);
model.extraDim(<tag>).selection();
model.extraDim(<tag>).selection("point");
model.extraDim(<tag>).getType(<property>);
model.extraDim(<tag>).model();
```

DESCRIPTION

`model.extraDim().create(<tag>,<type>)` creates an extra dimension feature of the given type. The supported types are `AttachDimension`, `PointsToAttach`, and `Integration`.

`model.extraDim(<tag>).set(<property>,<value>)` specifies properties relevant for the selected extra dimension feature type.

- Properties for `AttachDimension`: `extradim`
- `PointsToAttach` has no properties.
- Properties for `Integration`: `opname`, `intrule`, `intorder`, `frame`, `axisym`

`model.extraDim(<tag>).model(<mtag>)` sets the model component node. For features of type `AttachDimension`, this defines the base model component node. For features of type `PointsToAttach` and `Integration`, this defines the extra dimension component node.

```
model.extraDim(<tag>).selection();
```

Returns the selection of the feature (for features of type `AttachDimension` and `Integration`).

`model.extraDim(<tag>).selection("point")` returns the selection of points to attach (for features of type `PointsToAttach`).

`model.extraDim(<tag>).getType(<property>)` returns the value of the specified property.

`model.extraDim(<tag>).model()` returns the model component node tag.

model.field()

Create and define properties for fields with shape functions that defined field variables.

SYNTAX

```
model.field().create(<tag>,<fname>);
model.field(<tag>).field(<fname>);
model.field(<tag>).shape(<shlist>);

model.field(<tag>).field();
model.field(<tag>).shape();
model.field(<tag>).geom();
```

DESCRIPTION

`model.field().create(<tag>,<fname>)` creates a field with tag `<tag>` with the field name `<fname>`.

`model.field(<tag>).field(<fname>)` sets the field name.

`model.field(<tag>).shape(<shlist>)` sets the shape functions defining the field variables. *<shlist>* is a list of shape function tags. Each shape function defines one or more field variables. Together the shape functions specify which field variables there are in the field.

`model.field(<tag>).field()` returns the field name as a string.

`model.field(<tag>).shape()` returns the shape function tags as a string array.

`model.field(<tag>).geom()` returns the geometry associated with the field.

SEE ALSO

[model.shape\(\)](#), [model.coeff\(\)](#)

model.form()

Create settings forms for use in the Model Builder.

SYNTAX

`model.form().create(<tag>, <fname>);`

`model.form(<tag>).update();`

DESCRIPTION

`model.form().create(<tag>, <fname>)` creates a new form instance using the Application Builder Form Feature with the tag *<fname>*.

`model.form(<tag>).update()` updates the Settings Form instance *<tag>* with the current definition of the Application Builder Form Feature it references.

To change the value of an input field in a settings form you can change the value of the source data tied to the input field and let the data binding update the value of the input field.

SEE ALSO

[model.methodCall\(\)](#)

model.frame()

Create and define properties for different types of frames: spatial frames, material frames, mesh frames, and geometry frames.

SYNTAX

```
model.frame().create(<tag>,<gtag>);
model.frame(<tag>).coord(<coordlist>);
model.frame(<tag>).coord(<pos>,<coord>);
model.frame(<tag>).meshFrame();
model.frame(<tag>).materialFrame();
model.frame(<tag>).geometryFrame();
model.frame(<tag>).spatialFrame();
model.frame(<tag>).sshape.create(<stag>,type);
model.frame(<tag>).sshape(<stag>).type(type);
model.frame(<tag>).sshape(<stag>).sorder(order);
model.frame(<tag>).sshape(<stag>).coorddof(<dofs>);
model.frame(<tag>).sshape(<stag>).coorddof(<pos>,<dof>);
model.frame(<tag>).sshape(<stag>).refframe(<ftag>);
model.frame(<tag>).sshape(<stag>).coordexpr(<exprs>);
model.frame(<tag>).sshape(<stag>).coordexpr(<pos>,<expr>);
```

```
model.frame(<tag>).coord();
model.frame(<tag>).identifier();
model.frame(<tag>).varNameSuffix();
model.frame(<tag>).geom();
model.frame(<tag>).isMeshFrame();
model.frame(<tag>).isGeometryFrame();
model.frame(<tag>).isMaterialFrame();
model.frame(<tag>).isSpatialFrame();
model.frame(<tag>).sshape(<stag>).type();
model.frame(<tag>).sshape(<stag>).sorder();
model.frame(<tag>).sshape(<stag>).coorddof();
model.frame(<tag>).sshape(<stag>).refframe();
model.frame(<tag>).sshape(<stag>).coordexpr();
```

DESCRIPTION

`model.frame().create(<tag>,<gtag>)` creates a new frame and assigns it to geometry `<gtag>`.

`model.frame(<tag>).coord(<coordlist>)` defines `<coordlist>` as a list of independent variables. (Formerly `sdim`.)

`model.frame(<tag>).coord(<pos>,<coord>)` edits the coordinate at position `<pos>` in the coordinate list.

`model.frame(<tag>).meshFrame()` sets this frame to be the mesh frame. Each geometry requires exactly one mesh frame. The first one added becomes the mesh frame. When assigning one frame to be the mesh frame, this flag is cleared in the previous frame being the mesh frame.

`model.frame(<tag>).geometryFrame()` sets this frame to be the geometry frame. Each geometry requires exactly one geometry frame. The first one added becomes the geometry frame. When assigning one frame to be the geometry frame, this flag is cleared in the previous frame being the geometry frame.

`model.frame(<tag>).materialFrame()` sets this frame to be the material frame. Each geometry requires exactly one material frame. The first one added becomes the material frame. When assigning one frame to be the material frame, this flag is cleared in the previous frame being the material frame.

`model.frame(<tag>).spatialFrame()` sets this frame to be the spatial frame. Each geometry requires exactly one spatial frame. The first one added becomes the spatial frame. When assigning one frame to be the spatial frame, this flag is cleared in the previous frame being the spatial frame.

`model.frame(<tag>).sshape().create(<stag>,type)` creates a frame feature of the given type. Possible types are `fixed` (default), `moving_abs`, `moving_rel`, and `moving_expr`.

`model.frame(<tag>).sshape(<stag>).type(type)` sets the type of the frame feature.

`model.frame(<tag>).sshape(<stag>).sorder(order)` sets the geometry shape function for `<stag>` to `order`.

`model.frame(<tag>).sshape(<stag>).coorddof(<dofs>)` sets the spatial coordinates for <stag> when the `moving_rel` type is used.

`model.frame(<tag>).sshape(<stag>).coorddof(<pos>, <dof>)` edits the coordinate name at position <pos> in the degree of freedom list.

`model.frame(<tag>).sshape(<stag>).refframe(<ftag>)` sets the reference frame for <stag> when the `moving_rel` type is used.

`model.frame(<tag>).sshape(<stag>).coordexpr(<exprs>)` sets the expressions for the mesh displacement for <stag>.

`model.frame(<tag>).sshape(<stag>).coordexpr(<pos>, <expr>)` edits the expression at position <pos> in the expression list.

`model.frame(<tag>).sshape(<stag>).selection().named(<seltag>)` assigns the frame feature to the named selection <seltag>.

`model.frame(<tag>).sshape(<stag>).selection().set(...)` defines a local selection that assigns the frame feature to geometric entities. For a complete list of methods available under `selection()`, see [model.selection\(\)](#). All types of selections are supported except the global one and selections containing interior mesh boundaries.

`model.frame(<tag>).coord()` returns the coordinate names as a string array.

`model.frame(<tag>).identifier()` returns the frame's identifier as a string.

`model.frame(<tag>).varNameSuffix()` returns the variable name suffix as a string.

`model.frame(<tag>).geom()` returns the geometry name as a string.

`model.frame(<tag>).isMeshFrame()` returns true if this frame is the mesh frame.

`model.frame(<tag>).isGeometryFrame()` returns true if this frame is the geometry frame.

`model.frame(<tag>).isMaterialFrame()` returns true if this frame is the material frame.

`model.frame(<tag>).isSpatialFrame()` returns true if this frame is the spatial frame.

`model.frame(<tag>).sshape(<stag>).type()` returns the type as a string.

`model.frame(<tag>).sshape(<stag>).sorder()` returns the spatial approximation order as an integer.

`model.frame(<tag>).sshape(<stag>).coorddof()` returns the spatial coordinates as a string array.

`model.frame(<tag>).sshape(<stag>).refframe()` returns the reference frame as a string.

`model.frame(<tag>).sshape(<stag>).coordexpr()` returns the spatial coordinate expressions as a string array.

`model.frame(<tag>).sshape(<stag>).selection().named()` returns the named selection tag.

`model.frame(<tag>).sshape(<stag>).selection().getType()` returns domain information. For available methods, see [model.selection\(\)](#).

SEE ALSO

[model.shape\(\)](#)

model.func()

Add different types of functions.

SYNTAX

```
model.func().create(<tag>,<type>);
model.func(<tag>).create(<tag>,<type>);
model.func(<tag>).createPlot(<pgtag>)
model.func(<tag>).label(<label>)
model.func(<tag>).model(<mtag>)
model.func(<tag>).set(property,<value>);
model.func(<tag>).set("funcname",<funcname>)
model.func(<tag>).discardData()
model.func(<tag>).importData()
model.func(<tag>).refresh()
model.func(<tag>).image()
model.func(<tag>).run()

model.func(<tag>).model()
model.func(<tag>).getType(property);
model.func(<tag>).functionNames()
```

DESCRIPTION

`model.func().create(<tag>,<type>)` creates a new function of type `<type>` with the tag `<tag>`. The types can be one of the following strings: Analytic, Interpolation, Piecewise, GaussianPulse, Ramp, Rectangle, Step, Triangle, Wave, NormalDistribution, Random, External, MATLAB (requires LiveLink™ for MATLAB®), Elevation, Image, CGNS (requires the Acoustics Module or the CFD Module, supported on Windows® only), LeastSquares, GaussianProcess (requires the Uncertainty Quantification Module to create and train but not to use a created and trained function), PolynomialChaosExpansion (requires the Uncertainty Quantification Module to create and train but not to use a created and trained function), DNN, and PartialFractionFit. In addition, `model.create(<tag>,"FunctionSwitch")` creates a function switch. You can add other functions to a function switch:

```
model.func().create("sw1","FunctionSwitch");
model.func("sw1").create("int1","Interpolation");
model.func("sw1").create("an1","Analytic");
model.func("sw1").create("rn1","Random");
```

Use `model.func("sw1").set("definecases",true)`; to instantiate all functions for all cases in a function sweep. By default, not all of them are instantiated.

`model.func(<tag>).createPlot(<pgtag>)` creates a plot group with the tag `pgtag` with a plot of the function. The method returns the plot group.

`model.func(<tag>).label(<label>)` sets a label for the function.

`model.func(<tag>).model(<mtag>)` sets the model component node of the function.

`model.func(<tag>).set(property,<value>)` sets the value of a property of the function. See the available properties for each type of function below.

`model.func(<tag>).set("funcname",<funcname>)` sets the operator name of the function. The default operator name is `<tag>`.

`model.func(<tag>).model()` returns the model component node tag.

`model.func(<tag>).getType(property)` retrieves a value of a function property.

`model.func(<tag>).importData()` imports the file that the function references into the model. This is possible for interpolation, elevation, and image functions. The `importData()` method also works for some physics features.

`model.func(<tag>).discardData()` discards the data imported with `importData()`. This is possible for interpolation, elevation, and image functions. The `discardData()` method also works for some physics features.

`model.func(<tag>).refresh()` reevaluates the file for functions that read files (Elevation, Image, and Interpolation).

Use the `model.func(<tag>).image()` methods for plotting and exporting images showing plots of the functions. See [Plotting and Exporting Images](#).

`model.func(<tag>).run()` performs optimization or training for functions that need to process their input data before they can be evaluated (Least-Squares Fit, Gaussian Process, Polynomial Chaos Expansion, and DNN).

`model.func(<tag>).continueRun()` continues performing optimization or training using the previous call to `run()` or `continueRun()` as the starting point for the computation (Least-Squares Fit, Gaussian Process, Polynomial Chaos Expansion, and DNN).

`model.func(<tag>).export(<filePath>)` exports the internal representation of the function to the target file path in a standardized format to use with external tools and runtimes (DNN).

`model.func(<tag>).runTest()` tests the performance of the previously trained function (DNN) on test data set.

`model.func(<tag>).functionNames()` returns an array containing the function names that the function feature defines. Most functions always return an array of length one, but interpolation function features, for example, can define an arbitrary number of function names.

Function features can have associated problem features. To access the list of problem features for a function feature, use:

```
model.func(<tag>).problem();
```

To access a specific problem feature, use:

```
model.func(<tag>).problem(<problem_tag>);
```



For functions on the component level, use the same syntax but add the component level, such as `model.component(<ctag>).func().create(<tag>, <type>)`



When using a local table the interpolation function uses the `funcname` property to set the function name. When the data comes from a file or a result table, the name is specified in the `funcs` string matrix property. This is necessary because there can be more than one function.

What properties that are available depends on the type of function. The following function types are available:

Analytic

Create an *analytic* function using a symbolic expression.

TABLE 2-85: ANALYTIC PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
argders	Nx2 String array	{}	(argument, partial derivative) pairs if <code>dermethod</code> is <code>manual</code> .
args	String array	{}	The arguments to the function.
complex	boolean	false	True if the function can produce complex results for real inputs.
dermethod	automatic manual	automatic	Automatic differentiation or manual control over the derivatives.
expr	String	None	The expression defining the function.
funcname	String	The tag name	The name of the function.

TABLE 2-85: ANALYTIC PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
periodic	boolean	false	True if the function should be extended to a periodic function.
periodiclower	String	0	The lower limit of the interval that is extended periodically.
periodicupper	String	1	The upper limit of the interval that is extended periodically.
plotargs	N-by-3 array		Contains one row for each function argument. Each row contains the argument name, the lower limit for plotting, and the upper limit for plotting.
plotaxis	boolean array	{}	Contains one value for each function argument. A false value means that the corresponding function argument will be constant when plotting. The constant value is taken from the lower limit value in the plotargs property. Constant arguments do not use an axis in the plot. Up to 3 axes are supported when plotting. Specify using <code>setIndex("plotaxis", false, 2)</code> , for example.
pname	String array	{}	Local parameter names, using <code>setIndex</code> .
plist	String array	{}	Local parameter expressions, using <code>setIndex</code> .
argunit	String		A comma-separated list of required units for each argument.
fununit	String		The unit of the function's result.

Interpolation

Generate an *interpolation* function. You can use several interpolation and extrapolation methods.

TABLE 2-86: INTERPOLATION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
adaptol	double	1e-3	The tolerance used when sampling is set to adaptive.
arange	string matrix	{[0,1],...,[0,1]}	A matrix where the <i>i</i> th array of length 2, which must evaluate to a double, defines the argument range for the <i>i</i> th function argument.
defineinv	on off	off	If source is set to table: Whether to define the inverse function.
definerandom	on off	off	Whether to define a random function.
defvars	boolean	false	If source is file and defvars is set to true, the spatial coordinate variables are used as default arguments to the function if no arguments are supplied in a call to it.
dseparator	point comma	point	Decimal separator in interpolation data file, when source is set to file.
extrap	none const interior linear value	const	The extrapolation method.
extrapvalue	double	0	The extrapolation value if extrap is set to value.
filename	String		The file that contains the data if source is set to file.

TABLE 2-86: INTERPOLATION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
frame	String	spatial	Frame used for defining the spatial coordinates if defvars is set to true.
funcinvname	String		If source is table and defineinv is on: The name of the inverse function.
funcname	String	The tag name	The name of the function if source is table or function.
funcnametable	String matrix		Used if source is file: the first column contains function names and the second column contains the positions in the file where the corresponding function is defined.
interp	neighbor linear piecewisecubic cubicspline	linear	The interpolation method.
leftend	double	0	The left end of the range for the random function, if randomrange = manual.
modelres	String		If sourcetype is model, specifies the model resource that contains the interpolation data
nargs	integer (1, 2, 3)	1	The number of function arguments if struct is spreadsheet or source is resultTable.
plotfuncname	String	Name of the original interpolation function	The name of the function to plot, if you have defined an inverse, primitive, or random function.
plotleftextrap	boolean	true	Include left extrapolation in plots.
plotrighextrap	boolean	true	Include right extrapolation in plots.
points	positive integer	1000	The number of uniform sampling points, if sampling is set to uniform.
primfunname	String		Define a primitive function with the name give as primfunname.
randomname	String		Define a primitive function with the name give as primfunname.
randomnargs	integer	1	The number of arguments for the random function.
randomrange	automatic manual	automatic	Whether to define a range for the random function.
reinterp	boolean	false	If true, reinterpolate interpolation data on computational mesh. Available if defvars is true and frame is set to mesh.
reinterporder	positive integer	1	Lagrange shape function order for reinterpolation, if reinterp is set to true.
resultTable	String		The tag of the result table to use (tbl1, for example).
rightend	double	1	The right end of the range for the random function, if randomrange = manual.

TABLE 2-86: INTERPOLATION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
sampling	automatic uniform adaptive	automatic	Both automatic and uniform use uniform sampling, in the former case with 1000 sampling points, in the latter with the number of sampling points specified by points. For adaptive, the tolerance specified by adaptol (default 10^{-3}) must satisfy the condition $0.25(b-a)\text{abs}(2f((b-a)/2)-(f(a)+f(b))) \leq \text{arange} \cdot \text{adaptol}$ for each linear-interpolation subinterval $[a,b]$ or else the subinterval is divided in half.
scaledata	auto on off	auto	Apply scaling of data if the bounding box of the interpolation points has a bad aspect ratio (auto), always apply the scaling (on), or turn off scaling altogether (off).
source	table file resultTable function	table	If sourcetype is user, specifies whether the data is entered in a local table, read from a file, taken from a results table, or is based on another function.
sourcetype	model user	user	Specifies if the data for the function is stored in the model or provided by the user.
scrfun	String		Name (tag) of the function used as source when source is set to function.
scrfunname	String		Function name of the function used as source when source is set to function.
struct	grid sectionwise spreadsheet	spreadsheet	The data format if source is set to file.
table	Nx2 String array	Empty	Contains the point/value pairs if source is set to table.
argunit	String		A comma-separated list of required units for each argument.
fununit	String		The unit of the function's result.
argtrans	none logarithmic	none	No or logarithmic transformation of the argument.
valtrans	none logarithmic	none	No or logarithmic transformation of the function.

Piecewise

Generate a *piecewise* interpolation function, which is created by splicing together several functions, each defined on one interval.

TABLE 2-87: PIECEWISE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
arg	String	x	The argument to the function.
extrap	const interior none periodic value	const	The extrapolation method.
extrapvalue	double	0	The extrapolation value if extrap is value.
funcname	String	The tag name	The name of the function.
pieces	N-by-3 String array	Empty	(left, right, expression) for each interval.

TABLE 2-87: PIECEWISE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
pname	String array	{}	Local parameter names, using setIndex.
plist	String array	{}	Local parameter expressions, using setIndex.
smooth	none cont contd1 contd2	none	The type of smoothing.
smoothzone	double	0.1	The relative size of the smoothing zone if smoothing is enabled.
argunit	String		A comma-separated list of required units for each argument.
fununit	String		The unit of the function's result.

GaussianPulse

Generate a *Gaussian pulse* function. This function is the common bell-shaped curve (Gaussian function).

TABLE 2-88: GAUSSIAN PULSE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
baseline	double	0	The baseline for the function.
funcname	String	The tag name	The name of the function.
integralvalue	double	1	The integral value, when normalization is set to integral.
location	String	0	Where the pulse peaks.
peakvalue	double	1	The peak value, when normalization is set to peak.
plotlimitsactive	true false	false	Set custom range for arguments in preview plots.
plotlowerlimit	double		The lower limit for arguments in preview plots, if plotlimitsactive is set to true.
plotupperlimit	double		The upper limit for arguments in preview plots, if plotlimitsactive is set to true.
sigma	String	1	The standard deviation of the underlying normal distribution.
normalization	integral peak	integral	The normalization method to use.

Ramp

Generate a *ramp* function.

TABLE 2-89: RAMP PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
baseline	double	0	The baseline for the function.
cutoffactive	boolean	false	If true, then the ramp ends when it reaches the cutoff value.
cutoff	double	1	If cutoffactive is true, the level where the ramp ends.
funcname	String	The tag name	The name of the function.
location	String	0	Where the ramp starts.
slope	String	1	The slope of the ramp.
ncontder	1 or 2	2	The number of continuous derivatives if smoothing is enabled.
plotlimitsactive	true false	false	Set custom range for arguments in preview plots.
plotlowerlimit	double		The lower limit for arguments in preview plots, if plotlimitsactive is set to true.

TABLE 2-89: RAMP PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
plotupperlimit	double		The upper limit for arguments in preview plots, if plotlimitsactive is set to true.
smoothzonecutoffactive	boolean	false	Smooth the transition where the ramp ends at the cutoff.
smoothzonelocactive	boolean	false	Smooth the transition where the ramp starts.
smoothzonecutoff	double	0.1	The relative size of the smoothing zone for the cutoff, if smoothing is enabled.
smoothzoneloc	double	0.1	The relative size of the smoothing zone where the ramp starts, if smoothing is enabled.

Rectangle

Generate a *rectangle*-shaped function.

TABLE 2-90: RECTANGLE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
amplitude	double	1	The amplitude for the function.
baseline	double	0	The baseline for the function.
funcname	String	The tag name	The name of the function.
lower	String	-0.5	Where the high zone begins.
ncontder	1 or 2	2	The number of continuous derivatives if smoothing is enabled.
plotlimitsactive	true false	false	Set custom range for arguments in preview plots.
plotlowerlimit	double		The lower limit for arguments in preview plots, if plotlimitsactive is set to true.
plotupperlimit	double		The upper limit for arguments in preview plots, if plotlimitsactive is set to true.
smooth	boolean	true	Smooth the transitions?
smoothzone	String	0.1	The size of the smoothing zone on both sides of the transitions.
upper	String	0.5	Where the high zone ends.

Step

Generate a *step* function.

TABLE 2-91: STEP PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
baseline	double	0	The baseline for the function.
from	String	0	The value to the left of the location.
funcname	String	The tag name	The name of the function.
location	String	0	Where the step is located.
locationdef	beginning middle end	middle	Location for the smoothing: beginning of step, middle of step, or end of step.
ncontder	1 or 2	2	The number of continuous derivatives if smoothing is enabled.
plotlimitsactive	true false	false	Set custom range for arguments in preview plots.
plotlowerlimit	double		The lower limit for arguments in preview plots, if plotlimitsactive is set to true.

TABLE 2-91: STEP PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
plotupperlimit	double		The upper limit for arguments in preview plots, if plotlimitsactive is set to true.
smooth	boolean	true	Smooth the transition?
smoothzone	String	0.1	The size of the smoothing zone on both sides of location.
to	String	1	The value to the right of the location.

Triangle

Generate a *triangle*-shaped function.

TABLE 2-92: TRIANGLE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
amplitude	double	1	The amplitude for the function.
baseline	double	0	The baseline for the function.
funcname	String	The tag name	The name of the function.
lower	String	-0.5	Where the high zone begins.
ncontder	1 or 2	2	The number of continuous derivatives if smoothing is enabled.
plotlimitsactive	true false	false	Set custom range for arguments in preview plots.
plotlowerlimit	double		The lower limit for arguments in preview plots, if plotlimitsactive is set to true.
plotupperlimit	double		The upper limit for arguments in preview plots, if plotlimitsactive is set to true.
smooth	boolean	true	Smooth the transitions.
smoothzone	String	0.1	Size of smoothing zone on both sides of the transitions.
upper	String	0.5	Where the high zone ends.

Wave

Use a *wave* function to generate a wave-shaped function (waveform). The wave shape can be a sawtooth, sine wave, square wave, or triangle wave.

TABLE 2-93: WAVE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
amplitude	String	1	The amplitude for the function.
baseline	String	0	The baseline for the function.
delay	String	2*2*pi	The delay in the modulation, if modul is set to gaussian.
dutycycle	String	0.5	The duty cycle of the function (a value between 0 and 1). Available for Square functions (as the fraction of a period that the function has the high value) and for Triangle functions (as the fraction of a period that the function is rising).
funcname	String	The tag name	The name of the function.
modul	none gaussian	none	No modulation or Gaussian modulation, if type is set to sine.
ncontder	1 or 2	2	The number of continuous derivatives if smoothing is enabled.
period	String	2*pi	The period.
phase	String	0	The phase.

TABLE 2-93: WAVE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
plotlimitsactive	true false	false	Set custom range for arguments in preview plots.
plotlowerlimit	double		The lower limit for arguments in preview plots, if plotlimitsactive is set to true.
plotupperlimit	double		The upper limit for arguments in preview plots, if plotlimitsactive is set to true.
sigma	String	0.5/(2*pi)	The standard deviation for the modulation, if modul is set to gaussian.
smooth	boolean	true	Smooth the transitions. (Only used for wave forms with discontinuous function values or derivatives.)
smoothzone	String	0.1	The size of smoothing zone on both sides of the transitions.
type	sawtooth sine square triangle	sine	The type of waveform.

NormalDistribution

Generate a *normal distribution* function.

TABLE 2-94: NORMAL DISTRIBUTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
cumfuncname	String	The tag name + _cum	The name of the cumulative function.
funcname	String	The tag name	The name of the function.
invcumfuncname	String	The tag name + _cum_inv	The name of the quantile function.
mean	double	0	The mean of the function.
nargs	positive integer	1	The number of arguments to the random function.
randomname	String	rn_ + the tag name	The name of the random function.
seed	positive integer		A seed for the random function.
seedactive	boolean	false	Use a custom random seed?
sigma	double	1	The standard deviation of the function.

Random

Generate a *random* function. The random function can have a uniform or normal distribution.

TABLE 2-95: RANDOM PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
funcname	String	The tag name	The name of the function.
mean	String	0	The average value.
nargs	integer	1	The number of arguments.
normalsigma	String	1	The standard deviation if type is Normal.
seed	String	Unique for each random function	Random seed, if seedtype is set to manual.
seedactive	boolean	false	If true, the random seed will be used.
seedtype	manual currenttime	manual	The random seed type, if seedactive is true.

TABLE 2-95: RANDOM PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
type	uniform normal	uniform	The distribution type.
uniformrange	String	1	The range if type is Uniform.

External

Generate an *external* function that interfaces to other external functions written in the C language.

TABLE 2-96: EXTERNAL PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
ders	N-by-3 string array		(function name, argument, partial derivative) triplets.
funcs	String array		The functions defined by the library.
init	String		The string that is sent to the library when the function feature is initialized.
path	String		The path to the shared library that defines the functions.

An external function is a function defined in a shared library written by the user. The shared library must define the following three functions with C linkage:

- `int init(const char *str)` is called when the function is initialized with the string from the **Initialization data** field. It returns a nonzero value in case of success and zero in case of failure. This function might be called several times; it is always called before solving a model that uses the function.
- `int eval(const char *func, int nArgs, const double **inReal, const double **inImag, int blockSize, double *outReal, double *outImag)` is called for elementwise evaluation of the function `func` called with `nArgs` arguments of length `blockSize`. The array `inReal` contains the real parts of the arguments; it has length `nArgs`, and each element has length `blockSize`.

If the arguments are all-real, then `inImag` is null; otherwise, it contains the imaginary parts of the arguments. If the function evaluation is successful, 1 is returned if it resulted in an all-real array and 2 is returned if it resulted in a complex array. The function should return 0 in case of error. In case of a real result, the function values should be written to the array `outReal`. In case of a complex result, the real parts of the function should be written to `outReal` and the imaginary parts to `outImag`. The `outReal` and `outImag` arrays both have length `blockSize`. All matrices are allocated and deallocated by COMSOL.

- `const char *getLastError()` returns the last error that has occurred. A null or empty string is returned if no error has occurred. Calling `init()` or `eval()` must set the last error string to "" or null. All memory allocation of this string is handled by the shared library. There is no localization of the error messages.

If you are using Microsoft Visual Studio to compile your library, you can declare the functions as

`__declspec(dllexport)` to export them from the DLL.

An example of a library that defines a function called `extsinc` that computes the `sinc` function ($\sin(x)/x$):

```
#include <math.h>
#include <stdlib.h>
#include <string.h>

#ifdef _MSC_VER
#define EXPORT __declspec(dllexport)
#else
#define EXPORT
#endif

static const char *error = NULL;

EXPORT int init(const char *str) {
    return 1;
}
```



```

}

EXPORT const char * getLastError() {
    return error;
}

EXPORT int eval(const char *func,
                int nArgs,
                const double **inReal,
                const double **inImag,
                int blockSize,
                double *outReal,
                double *outImag) {

    if (strcmp("extsinc", func) == 0) {
        if (nArgs != 1) {
            error = "One argument expected";
            return 0;
        }
        for (int i = 0; i < blockSize; i++) {
            double x = inReal[0][i];
            outReal[i] = (x == 0) ? 1 : sin(x) / x;
        }
        return 1;
    }
    else {
        error = "Unknown function";
        return 0;
    }
}

```

To compile this function into a library, place it in `ext.c` and proceed as follows depending on platform:



See www.comsol.com/system-requirements for information about supported compiler versions.

- 64-bit Windows with Microsoft Visual Studio 2022:
 - Start **Visual Studio 2022** > **Visual Studio Tools** > **VC > x64 Native Tools Command Prompt for VS 2022** from the Windows Start menu.
 - `cd` to the directory that contains `ext.c`
 - `cl /MT /c ext.c`
 - `link /OUT:ext.dll /DLL ext.obj`
- 64-bit Linux with Intel Compiler:
 - `cd` to the directory that contains `ext.c`
 - `icx -fPIC -c ext.c`
 - `icx -shared -fPIC -Wl,-z -Wl,defs -o ext.so ext.o -ldl`
- 64-bit Linux ARM64 with GCC:
 - `cd` to the directory that contains `ext.c`
 - `gcc -fPIC -c ext.c`
 - `gcc -shared -fPIC -o ext.so ext.o -ldl`

- 64-bit macOS (Intel or ARM64) with Clang:
 - `cd` to the directory that contains `ext.c`
 - `clang -fPIC -c ext.c`
 - `clang -shared -fPIC -o ext.dylib ext.o`

For other compilers, refer to the compiler's documentation for instructions how to compile and create a shared library.

MATLAB

Declare use of function in *MATLAB*. This requires the LiveLink™ for MATLAB®.

TABLE 2-97: MATLAB PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>ders</code>	Nx3 string array		(function name, argument, partial derivative) triplets.
<code>funcs</code>	String array		The functions defined by MATLAB.

Elevation

Generate an *elevation* function by importing geospatial elevation data from digital elevation models (DEM files).

TABLE 2-98: ELEVATION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>extrap</code>	<code>const</code> <code>interior</code> <code>linear</code> <code>value</code>	<code>const</code>	The extrapolation method.
<code>extrapvalue</code>	<code>double</code>	<code>0</code>	The extrapolation value if <code>extrap</code> is <code>value</code> .
<code>filename</code>	String		The name of the DEM file.
<code>funcname</code>	String	The tag name	The name of the function.
<code>interp</code>	<code>neighbor</code> <code>linear</code>	<code>linear</code>	The interpolation method.

Image

Generate an *image* function from a BMP, GIF, JPEG, PNG, or TIFF file.

TABLE 2-99: IMAGE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>argunit</code>	String		The unit of the function arguments.
<code>clipmaxx</code>	<code>double</code>	<code>1000</code>	If clipping is manual: The maximum pixel x-coordinate that is kept.
<code>clipminx</code>	<code>double</code>	<code>0</code>	If clipping is manual: The minimum pixel x-coordinate that is kept.
<code>clipmaxy</code>	<code>double</code>	<code>1000</code>	If clipping is manual: The maximum pixel y-coordinate that is kept.
<code>clipminy</code>	<code>double</code>	<code>0</code>	If clipping is manual: The minimum pixel y-coordinate that is kept.
<code>clipping</code>	<code>none</code> <code>manual</code>	<code>none</code>	The clipping method.
<code>extrap</code>	<code>const</code> <code>interior</code> <code>linear</code> <code>value</code>	<code>const</code>	The extrapolation method.
<code>extrapvalue</code>	<code>double</code>	<code>0</code>	The extrapolation value if <code>extrap</code> is <code>value</code> .
<code>fununit</code>	String		The unit of the function value.
<code>filename</code>	String		The name of the DEM file.
<code>flipx</code>	<code>boolean</code>	<code>false</code>	If <code>inplace</code> is <code>false</code> : Whether to flip the image horizontally when mapping it to the xy-plane.

TABLE 2-99: IMAGE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
flipy	boolean	false	If inplace is false: Whether to flip the image vertically when mapping it to the xy-plane.
funcname	String	The tag name	The name of the function.
inplace	boolean	false	If true, the image is mapped to the xy-plane without scaling; 1 length unit corresponds to 1 pixel.
interp	neighbor linear	linear	The interpolation method.
manualexpr	String	$(r+g+b)/3$	If scaling is manual: The scaling function expressed in terms of the red (r), green (g), and blue (b) pixel intensities.
scaling	automatic manual	automatic	The method used for computing function values from pixel colors.
xmax	double	1	If inplace is false: The maximum x-coordinate of the region to which the image is mapped.
xmin	double	0	If inplace is false: The minimum x-coordinate of the region to which the image is mapped.
ymax	double	1	If inplace is false: The maximum y-coordinate of the region to which the image is mapped.
ymin	double	0	If inplace is false: The minimum y-coordinate of the region to which the image is mapped.

CGNS

Generate a CGNS (CFD General Notation System) function by importing CFD analysis data from a file.

TABLE 2-100: CGNS PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
filename	string		The name of the CGNS file.
funcs	string array		The functions defined by the CGNS data file.
plotargs	N-by-3 array		Contains one row for each argument. Each row contains the argument name, the lower limit for plotting, and the upper limit for plotting.

LeastSquares

Create a function based on a least-squares fit of the function's input data to a parameterized function.

TABLE 2-101: LEAST SQUARES PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
args	string array	{}	The argument names for the defined function(s), as an array of alternating column names from the input data and argument names. Using the <code>setEntry</code> , <code>getEntryKeys</code> , <code>getEntryKeyIndex</code> methods makes it easier to work with this array.
columnType	string array	{}	Defines the types of columns in the input data, as an array of alternative column names and column types. Valid column types are <code>none</code> , <code>arg</code> , and <code>value</code> . See also the description of the <code>args</code> property.
dseparator	point comma	point	Decimal separator in input data file, when source is set to <code>file</code> .

TABLE 2-101: LEAST SQUARES PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
exprs	string array	{}	The expression for the defined function(s), as an array of alternating column names from the input data and expressions. Using the <code>setEntry</code> , <code>getEntryKeys</code> , <code>getEntryKeyIndex</code> methods makes it easier to work with this array.
filecolumns	integer	0	Number of columns in the input data. This property is updated automatically when input data properties are changed.
fileheaders	string array	{}	Custom column names to be displayed in the column settings table. This property is updated automatically when file input data properties are changed. Column header names are taken from the last line in the file that starts with a '%' character. The line is split on " " (double space), tab character, and comma character, and each part is used as one custom column name.
filename	string		The file that contains the data if source is file.
lbound	double		The lower bound for parameters, if used. Set using <code>setIndex</code> .
ubound	double		The upper bound for parameters, if used. Set using <code>setIndex</code> .
unit	string array	{}	The units for the arguments and function values, as an array of alternating column names from the input data and unit expressions. See also the description of the <code>args</code> property.
pname	string		Parameter name, set using <code>setIndex</code> .
plist	double		Parameter value, set using <code>setIndex</code> .
plotargs	N-by-3 array		Contains one row for each argument. Each row contains the argument name, the lower limit for plotting, and the upper limit for plotting.
plotaxis	boolean array	{}	Contains one value for each argument. A false value means that the corresponding function argument will be constant when plotting. The constant value is taken from the lower limit value in the <code>plotargs</code> property. Constant arguments do not use an axis in the plot. Up to 3 axes are supported when plotting. Specify using <code>setIndex("plotaxis", false, 2)</code> , for example.
plotfuncname	string		The name of the function to plot.
resultTable	string		The results table that contains the data if source is <code>resultTable</code> .
scale	double	1	Scale for parameters, set using <code>setIndex</code> .

TABLE 2-101: LEAST SQUARES PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
source	file table resultTable	file	Specifies whether the data is read from a file, specified in a table, or taken from a results table.
table	N-by-2 string array	Empty	Contains the argument/function value pairs if the source is table.
unit	string array	{}	The units for the arguments and function values, as an array of alternating column names from the input data and unit expressions. See also the description of the args property.

GaussianProcess

Generate a Gaussian process regression (Kriging) function. Creating or training a Gaussian process function requires the Uncertainty Quantification Module. Using an already created and trained Gaussian process function does not require the Uncertainty Quantification Module.

TABLE 2-102: GAUSSIAN PROCESS PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
args	string array	{}	The argument names for the defined function(s), as an array of alternating column names from the input data and argument names. Using the setEntry, getEntryKeys, getEntryKeyIndex methods makes it easier to work with this array.
columnType	string array	{}	Defines the types of columns in the input data, as an array of alternative column names and column types. Valid column types are none, arg, and value. See also the description of the args property.
covfunction	se matern32 matern52 nn	matern32	Type of covariance function to use in the Gaussian process regression. Use se for Squared exponential, matern32 for Matérn 3/2, matern52 for Matérn 5/2 and nn for Single-layer neural network.
definestddev	boolean	false	If true, the related error estimation function(s) are made available.
descr	string array	{}	The description for each function argument and function value, as an array of alternating function arguments or values and their descriptions. Using the setEntry, getEntryKeys, getEntryKeyIndex methods makes it easier to work with this array.
dseparator	point comma	point	Decimal separator in input data file, when source is set to file.
filecolumns	integer	0	Number of columns in the input data. This property is updated automatically when input data properties are changed.

TABLE 2-102: GAUSSIAN PROCESS PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
fileheaders	string array	{}	Custom column names to be displayed in the column settings table. This property is updated automatically when file input data properties are changed. Column header names are taken from the last line in the file that starts with a '%' character. The line is split on " " (double space), tab character, and comma character, and each part is used as one custom column name.
filename	string		The file that contains the data if source is file.
fraction	double	0.1	The fraction of input data to set aside for validation of the trained function(s). Used when validation is one of random, fraction, last.
funcs	string array	{}	The names of the defined function(s), as an array of alternating column names from the input data and function names. See also the description of the args property.
ignorenaninf	boolean	false	If true, causes lines in the training data that contain any NaN or Inf values to be ignored. If false, NaN and Inf values are treated as an error.
lastinternalseed	nonnegative integer	1014	Random number seed used for training the function(s). Used when useseed is manual.
lastinternalseedtest	nonnegative integer	108714	Random number seed used for validation of the trained function(s). Used when validation is random and useseedtest is manual.
manualrestarthypergpnnumber	positive integer	10	Number of restart points for training. Use more restart points to increase the chance to find the global optimum during training. Used when restarthypergp is manual.
maxmatsize	positive integer	2000	Maximum number of allowed training data points. More data points require more memory and longer training time.
meanfunction	const linear quadratic	const	Defines the overall trend of the trained function(s).
plotargs	N-by-3 array		Contains one row for each function argument. Each row contains the argument name, the lower limit for plotting, and the upper limit for plotting.

TABLE 2-102: GAUSSIAN PROCESS PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
plotaxis	boolean array	{}	Contains one value for each function argument. A false value means that the corresponding function argument will be constant when plotting. The constant value is taken from the lower limit value in the plotargs property. Constant arguments do not use an axis in the plot. Up to 3 axes are supported when plotting. Specify using <code>setIndex("plotaxis", false, 2)</code> , for example.
plotfixedvalue	double array	{}	Contains one value for each function argument. For arguments that do not have an axis in the plot (see <code>plotaxis</code>), this property specifies the fixed value to use in the plot.
plotfuncname	string		The name of the function whose value should be plotted.
restarthypergp	automatic manual	automatic	Controls how the number of restart points during training is determined. If automatic, the number is calculated from the number of function arguments. If manual, the number is given by the <code>manualrestarthypergpnumber</code> property.
resultTable	string		The results table that contains the data if source is resultTable.
source	file resultTable	file	Specifies whether the data is read from a file or taken from a results table.
stddevsuffix	string	_stddev	Function name suffix to add to error estimation functions. Used when <code>definestddev</code> is true.
testerrortable	none new results table	none	If validation is not none, specifies the table where verification error summary is stored. Use none to not generate the table data. Use new to create a new table for the data. Use a results table tag to store the data in an existing table.
testtable	none new results table	new	If validation is not none, specifies the table where detailed verification error information is stored. Use none to not generate the table data. Use new to create a new table for the data. Use a results table tag to store the data in an existing table.
unit	string array	{}	The units for the arguments and function values, as an array of alternating column names from the input data and unit expressions. See also the description of the <code>args</code> property.
useseed	manual currenttime	manual	Specifies how the random seed for training is determined. If manual, the seed is given by the <code>lastinternalseed</code> property. If currenttime, the seed is computed from the current time when training is started.

TABLE 2-102: GAUSSIAN PROCESS PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
useseedtest	manual currenttime	manual	Specifies how the random seed for validation is determined. If manual, the seed is given by the lastinternalseedtest property. If currenttime, the seed is computed from the current time when training is started.
validation	none random fraction last table	none	Specifies what data to use for validation of the trained function(s). none: No validation is performed. random: Use a random sample of the input data and exclude the corresponding values from the training data. The size of the sample is fraction times the number of input data points. fraction: Use every 1/fraction values from the input data and exclude the corresponding values from the training data. last: Use the last part of the input data and exclude the corresponding values from the training data. The size of the last part is fraction times the number of input data points. table: Use a results table as validation data.
validationtable	string		The results table to take validation data from. Used when validation is table.

Changes in the following properties take effect without retraining the function: `definestddev`, `stddevsuffix`, `funcs`, `plotargs`, and `unit`.

The *Uncertainty Quantification Module User's Guide* explains the theory behind Gaussian process regression. See [Surrogate Models — Gaussian Process](#).

PolynomialChaosExpansion

Generate a *polynomial chaos expansion (PCE)* function. Creating or training a PCE function requires the Uncertainty Quantification Module. Using an already created and trained PCE function does not require the Uncertainty Quantification Module.

TABLE 2-103: POLYNOMIAL CHAOS EXPANSION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
args	string array	{ }	The argument names for the defined function(s), as an array of alternating column names from the input data and argument names. Using the <code>setEntry</code> , <code>getEntryKeys</code> , <code>getEntryKeyIndex</code> methods makes it easier to work with this array.
columnType	string array	{ }	Defines the types of columns in the input data, as an array of alternative column names and column types. Valid column types are none, arg, and value. See also the description of the <code>args</code> property.

TABLE 2-103: POLYNOMIAL CHAOS EXPANSION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
descr	string array	{}	The description for each function argument and function value, as an array of alternating function arguments or values and their descriptions. Using the <code>setEntry</code> , <code>getEntryKeys</code> , <code>getEntryKeyIndex</code> methods makes it easier to work with this array.
distributionselection	string array	{}	Defines the probability distributions for the function arguments, as an array of alternative column names and distribution types. Valid distribution types are <code>uniform</code> , <code>normal</code> , <code>lognormal</code> , <code>gamma</code> , <code>beta</code> , <code>weibull</code> , and <code>gumbel</code> . See also the description of the <code>args</code> property.
dseparator	point comma	point	Decimal separator in input data file, when source is set to <code>file</code> .
filecolumns	integer	0	Number of columns in the input data. This property is updated automatically when input data properties are changed.
fileheaders	string array	{}	Custom column names to be displayed in the column settings table. This property is updated automatically when file input data properties are changed. Column header names are taken from the last line in the file that starts with a '%' character. The line is split on " " (double space), tab character and comma character and each part is used as one custom column name.
filename	string		The file that contains the data if source is <code>file</code> .
fraction	double	0.1	The fraction of input data to set aside for validation of the trained function(s). Used when validation is one of <code>random</code> , <code>fraction</code> , <code>last</code> .
funcs	string array	{}	The names of the defined function(s), as an array of alternating column names from the input data and function names. See also the description of the <code>args</code> property.
ignorenaninf	boolean	false	If true, causes lines in the training data that contain any NaN or Inf values to be ignored. If false, NaN and Inf values are treated as an error.
lastinternalseedtest	nonnegative integer	108714	Random number seed used for validation of the trained function(s). Used when validation is <code>random</code> and <code>useseedtest</code> is <code>manual</code> .

TABLE 2-103: POLYNOMIAL CHAOS EXPANSION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
lboundselection	string array	{}	Defines how the lower bounds of the valid range for the function arguments are determined, as an array of alternative column names and CDF (cumulative distribution function) values. Valid CDF values are 0.3, 0.1, 0.01, 0.001, 1e-4, 1e-5, 1e-6, 1e-7, and manual. Not used if distributionselection is uniform or beta. See also the description of the args property.
maxmatsize	positive integer	2000	Maximum number of allowed training data points. More data points require more memory and longer training time.
outofrange	warning cancel	warning	Decides how to handle out-of-range and extrapolated training data. If outofrange is cancel, the training is canceled. If outofrange is warning, the training continues, but a warning is added under the function feature. In either case, details about the problematic training data points are logged in the log window.
pcesettings	auto manual	automatic	If pcesettings is set to auto, the training will automatically determine the required polynomial degree needed to obtain suitable accuracy. If pcesettings is set to manual, the maximum polynomial degree is determined by the polydegreespc and qnorm settings.
plotargs	N-by-3 array		Contains one row for each function argument. Each row contains the argument name, the lower limit for plotting, and the upper limit for plotting.
plotaxis	boolean array	{}	Contains one value for each function argument. A false value means that the corresponding function argument will be constant when plotting. The constant value is taken from the lower limit value in the plotargs property. Constant arguments do not use an axis in the plot. Up to 3 axes are supported when plotting. Specify using setIndex("plotaxis", false, 2), for example.
plotfixedvalue	double array	{}	Contains one value for each function argument. For arguments that do not have an axis in the plot (see plotaxis), this property specifies the fixed value to use in the plot.
plotfuncname	string		The name of the function whose value should be plotted.
polydegreespc	positive integer	30	The maximum polynomial degree to use when pcesettings is set to manual. See also the <i>Uncertainty Quantification Module User's Guide</i> .

TABLE 2-103: POLYNOMIAL CHAOS EXPANSION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
qnorm	double	0.5	Controls the hyperbolic truncation of polynomial terms used for functions having more than one argument. Only used when pcesettings is set to manual. See also the <i>Uncertainty Quantification Module User's Guide</i> .
s1selection	string array	{}	Defines the first probability distribution parameter for the function arguments as an array of alternative column names and distribution parameters. Not used if distributionselection is set to uniform. See also the description of the args property.
s2selection	string array	{}	Defines the second probability distribution parameter for the function arguments as an array of alternative column names and distribution parameters. Not used if distributionselection is set to uniform. See also the description of the args property.
source	file resultTable	file	Specifies whether the data is read from a file or taken from a results table.
surrogatetol	double	1e-3	The tolerance used when deciding the required polynomial degree of the trained function. See also the <i>Uncertainty Quantification Module User's Guide</i> .
testerrortable	none new results table	none	If validation is not none, specifies the table where verification error summary is stored. Use none to not generate the table data. Use new to create a new table for the data. Use a results table tag to store the data in an existing table.
testtable	none new results table	new	If validation is not none, specifies the table where detailed verification error information is stored. Use none to not generate the table data. Use new to create a new table for the data. Use a results table tag to store the data in an existing table.
uboundselection	string array	{}	Defines the upper bound of the valid input range for the function arguments as an array of alternative column names and upper bounds. Only relevant if distributionselection is set to uniform or beta, or if ucdfselection is set to manual. See also the description of the args property.

TABLE 2-103: POLYNOMIAL CHAOS EXPANSION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
ucdfselection	string array	{}	Defines how the upper bounds of the valid range for the function arguments are determined as an array of alternative column names and CDF (cumulative distribution function) values. Valid CDF values are 0.7, 0.9, 0.99, 0.999, 1-1e-4, 1-1e-5, 1-1e-6, 1-1e-7, and manual. Not used if distributionselection is set to uniform or beta. See also the description of the args property.
unit	string array	{}	The units for the arguments and function values, as an array of alternating column names from the input data and unit expressions. See also the description of the args property.
useseedtest	manual currenttime	manual	Specifies how the random seed for validation is determined. If manual, the seed is given by the lastinternalseedtest property. If currenttime, the seed is computed from the current time when training is started.
validation	none random fraction last table	none	Specifies what data to use for validation of the trained function(s). none: No validation is performed. random: Use a random sample of the input data and exclude the corresponding values from the training data. The size of the sample is fraction times the number of input data points. fraction: Use every 1/fraction values from the input data and exclude the corresponding values from the training data. last: Use the last part of the input data and exclude the corresponding values from the training data. The size of the last part is fraction times the number of input data points. table: Use a results table as validation data.
validationtable	string		The results table to take validation data from. Used when validation is table.

Changes in the following properties take effect without retraining the function: funcs, plotargs, unit.

The *Uncertainty Quantification Module User's Guide* explains the theory behind PCE. See [Surrogate Models — Polynomial Chaos Expansion](#).

DNN

Generate a *deep neural network* (DNN) function. A DNN function provides training and validation using a deep neural network (DNN) for use with surrogate model training, for example. Deep neural networks form a class of

machine learning algorithms similar to the artificial neural network and aims to mimic the information processing of the brain.

TABLE 2-104: DEEP NEURAL NET PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
activation	string array	{"none", "tanh"}	Array of activation functions to use for each layer. Choose from none, relu, elu, sigmoid, tanh, softplus, leakyrelu and gelu.
args	string array	{}	The argument names for the defined function(s), as an array of alternating column names from the input data and argument names. Using the setEntry, getEntryKeys, getEntryKeyIndex methods makes it easier to work with this array.
batchsize	positive integer	512	The batch size for the DNN training.
colscale	String map with keys col1, col2, col3, ... and values to01, std, log, or no.	{}	The scaling type for each argument or function argument as a string map of alternating keys and values.
descr	string array	{}	The description for each function argument and function value, as an array of alternating function arguments or values and their descriptions. Using the setEntry, getEntryKeys, getEntryKeyIndex methods makes it easier to work with this array.
epochs	integer	1000	The number of epochs to train for. In each epoch all input data is processed once.
filename	string		The full filename of the data source, when source is set to file.
fraction	scalar, 0–1	0.1	The validation data fraction.
gputraining	boolean	false	Use a supported GPU for accelerated DNN training.
ignorenaninf	boolean	true	Ignore data points that are NaN or Inf.
ignorenonpositive	boolean	false	Ignore nonpositive data for the data columns for which colscale is log.
layertype	array of dense	{dense}	The type of layer in the DNN.
lr	positive scalar	1e-3	The learning rate for the training.
loss	mse mae	mse	The loss function; root-mean-square error or a mean absolute error.
momentum	nonnegative scalar	0	The momentum, if optmethod is set to sgd.
optmethod	adam sgd	adam	Training optimization method: Adam or a stochastic gradient descent (SGD) method
outfeatures	array of positive integers	{1}	The number of output features from a layer.
plotargs	N-by-3 array		Contains one row for each function argument. Each row contains the argument name, the lower limit for plotting, and the upper limit for plotting.

TABLE 2-104: DEEP NEURAL NET PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
plotaxis	boolean array	{}	Contains one value for each function argument. A false value means that the corresponding function argument will be constant when plotting. The constant value is taken from the lower limit value in the plotargs property. Constant arguments do not use an axis in the plot. Up to 3 axes are supported when plotting. Specify using <code>setIndex("plotaxis", false, 2)</code> , for example.
plotfixedvalue	double array	{}	Contains one value for each function argument. For arguments that do not have an axis in the plot (see plotaxis), this property specifies the fixed value to use in the plot.
resultTable	result table tag		The result table to use as source, if source is set to resultTable.
rndseed	double	0	The random seed for training, if useseed is set to manual.
rndseedvalidation	double	0	The random seed for validation, if useseedvalidation is set to manual.
source	file resultTable	file	Specify whether the data source is read from a file or taken from a results table.
testloss	double		The test loss of the last trained network. This property is updated automatically when the test has completed.
trainingloss	double		The training loss of the last trained network. This property is updated automatically when training has completed.
useseed	manual currenttime	manual	Specifies how the random seed for training is determined. If manual, the seed is given by the rndseed property. If currenttime, the seed is computed from the current time when training is started.
useseedvalidation	manual currenttime	manual	Specifies how the random seed for validation is determined. If manual, the seed is given by the rndseed property. If currenttime, the seed is computed from the current time when training is started.
validation	random fraction last table	random	The validation data: random sample of data values, every N:th data value, last part of data values, or taken from a separate table.
validationloss	double		The validation loss of the last trained network. This property is updated automatically when training has completed.
validationtable	table tag (string)		The tag of the table used as validation data, if validation is set to table.
weightdecay	double	0	Nonnegative number to penalize complexity by adding the squares of all the parameters to the loss function.

PartialFractionFit

Create an *partial fraction fit* function.

TABLE 2-105: PARTIAL FRACTION FIT PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
asymterm	double	0	Asymptotic term for the poles and residues.
dseparator	point comma	point	Decimal separator in the data file, when source is set to file.
froissart	boolean	false	Automatically detect and remove Froissart doubles.
funcname	String	The tag name	The name of the function.
iter	positive integer	3	The maximum number of iterations for the partial fraction fit, if stopcondition is set to iter or itertol.
plotargs	N-by-3 array		Contains one row for each argument. Each row contains the argument name, the lower limit for plotting, and the upper limit for plotting.
plotaxis	boolean array	{}	Contains one value for each argument. A false value means that the corresponding function argument will be constant when plotting. The constant value is taken from the lower limit value in the plotargs property. Constant arguments do not use an axis in the plot. Up to 3 axes are supported when plotting. Specify using <code>setIndex("plotaxis", false, 2)</code> , for example.
source	file resultTable	file	Specify whether the data source is read from a file or taken from a results table.
stopcondition	tol iter itertol	tol	Stop condition: tolerance, iterations, or iterations or tolerance.
threshold	double	1e-3	The threshold for the detection of Froissart doubles, if froissart is set to true.
tol	double	1e-3	The tolerance for the partial fraction fit, if stopcondition is set to tol or itertol.

COMPATIBILITY

For the Wave function, the `freq` property with a default value of 1 in previous versions of COMSOL Multiphysics has been replaced by `period` with a default value of 2π in version 6.0.

SEE ALSO

`model.material()`

model.geom()

Creating and specifying general properties for 1D, 2D, and 3D geometries.



For more information about the different methods, see [Geometry](#).

SYNTAX

```
model.component(<ctag>).geom().create(<tag>,<sdim>);
model.component(<ctag>).geom().create(<tag>,<meshtag>,<filename>);
model.component(<ctag>).geom(<tag>).model(<mtag>);
model.component(<ctag>).geom(<tag>).model();
model.component(<ctag>).geom(<tag>).axisymmetric(boolean);
model.component(<ctag>).geom(<tag>).isAxisymmetric();
model.component(<ctag>).geom(<tag>).lengthUnit(<unit>);
model.component(<ctag>).geom(<tag>).lengthUnit();
model.component(<ctag>).geom(<tag>).angularUnit(<unit>);
model.component(<ctag>).geom(<tag>).angularUnit();
model.component(<ctag>).geom(<tag>).scaleUnitValue(boolean);
model.component(<ctag>).geom(<tag>).scaleUnitValue();
model.component(<ctag>).geom(<tag>).repairTol(<relTol>);
model.component(<ctag>).geom(<tag>).repairTol();
model.component(<ctag>).geom(<tag>).geomRep(geomrep);
model.component(<ctag>).geom(<tag>).geomRep();
model.component(<ctag>).geom().remove(<tag>);

model.component(<ctag>).geom(<tag>).create(<ftag>,type);
model.component(<ctag>).geom(<tag>).createAfter(<ftag>,<type>,<postag>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).create(<ftag2>,type);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property).selMethod;
model.component(<ctag>).geom(<tag>).feature(<ftag>).geom().geomMethod;
model.component(<ctag>).geom(<tag>).feature(<ftag>).active(boolean);
model.component(<ctag>).geom(<tag>).feature(<ftag>).isActive();
model.component(<ctag>).geom(<tag>).feature().move(<ftag>,<position>);
model.component(<ctag>).geom(<tag>).feature().remove(<ftag>);

model.component(<ctag>).geom(<tag>).run(<ftag>);
model.component(<ctag>).geom(<tag>).runPre(<ftag>);
model.component(<ctag>).geom(<tag>).runCurrent();

model.component(<ctag>).geom(<tag>).run();
model.component(<ctag>).geom().run();

model.component(<ctag>).geom(<tag>).current();
model.component(<ctag>).geom(<tag>).feature(<ftag>).status();
model.component(<ctag>).geom(<tag>).feature(<ftag>).message();

model.component(<ctag>).geom(<tag>).objectNames();
model.component(<ctag>).geom(<tag>).feature(<ftag>).objectNames();
model.component(<ctag>).geom(<tag>).obj(<objname>).geomInfoMethod
model.component(<ctag>).geom(<tag>).geomInfoMethod
model.component(<ctag>).geom(<tag>).selection(<seltag>).selMethod;

model.component(<ctag>).geom(<tag>).measure().measurementMethod;

model.component(<ctag>).geom(<tag>).cleanup().cleanupMethod;

model.component(<ctag>).geom(<tag>).export(<filename>);
model.component(<ctag>).geom(<tag>).exportFinal(<filename>);

model.component(<ctag>).geom(<tag>).defeaturing(<tooltag>).defeaturingMethod;
model.component(<ctag>).geom(<tag>).feature(<ftag>).find();
model.component(<ctag>).geom(<tag>).feature(<ftag>).detail().selMethod;

model.component(<ctag>).geom(<tag>).insertFile(<filename>,<gttag>);
model.component(<ctag>).geom(<tag>).insertSequence(<mtag>,<gttag>);

model.component(<ctag>).geom().create(<tag>, "Subsequence", sDim);
model.component(<ctag>).geom(<tag>).inputParam().set(<name>, <expr>, <descr>);
```



```

model.component(<ctag>).geom(<tag>).localParam().set(<name>, <expr>, <descr>);
model.geom(<gtag>).create(<ftag>, "PartInstance");
model.geom(<gtag>).stepInto(<ftag>);
model.geom(<gtag>).feature(<ftag>).geom().run(<ftag2>);

```

The last group of syntaxes are only applicable for geometry subsequences (see [Creating and Using Geometry Parts](#)), but `model.geom(<gtag>).feature(<ftag>).geom()` also exists if `<ftag>` is a work plane feature.

```

model.geom().load(<tags>, <filename>, <subsequenceTagsInFile>);
model.geom(<gtag>).loaded();

```

```

model.geom(<gtag>).filename();
model.geom(<gtag>).filename(<filename>);
model.geom(<gtag>).tagInFile();
model.geom(<gtag>).dateModifiedInFile();
model.geom(<gtag>).commentsInFile();
model.geom(<gtag>).labelInFile();
model.geom(<gtag>).versionInFile();
model.geom(<gtag>).reload();

```

The last group of syntaxes are only applicable for geometry subsequences (see [Creating and Using Geometry Parts](#)).

DESCRIPTION

Geometry Sequences and Geometry Objects

`model.component(<ctag>).geom(<tag>)` returns a geometry sequence consisting of geometry features. The geometry sequence also contains geometry objects resulting from building the geometry sequence.

Creating and Deleting a Geometry

`model.component(<ctag>).geom().create(<tag>, <sdim>)` creates a geometry sequence of space dimension `<sdim>` and assigns it the tag `<tag>`.

`model.component(<ctag>).geom().create(<tag>, <meshtag>, <filename>)` creates a geometry sequence tagged `<tag>` and a corresponding meshing sequence tagged `<meshtag>`. The parameter `<filename>` specifies a file that contains a geometry or a mesh, and an import feature is inserted into the geometry or meshing sequence.

`model.component(<ctag>).geom().remove(<tag>)` deletes the geometry tagged `<tag>`.

General Geometry Settings

`model.geom(<tag>).model(<mtag>)` sets the model component node of the geometry `<tag>` to `<mtag>`.

`String mtag = model.geom(<tag>).model()` returns the model component node tag of the geometry.

`model.component(<ctag>).geom(<tag>).axisymmetric(boolean)` indicates if the geometry is axisymmetric. This is only applicable for 1D and 2D geometries.

`model.component(<ctag>).geom(<tag>).isAxisymmetric()` returns true if the geometry is axisymmetric and false otherwise.

`model.component(<ctag>).geom(<tag>).lengthUnit(<unit>)` sets the length unit.

`String unit = model.component(<ctag>).geom(<tag>).lengthUnit()` returns the length unit.

`model.component(<ctag>).geom(<tag>).angularUnit(<unit>)` sets the angular unit.

`String unit = model.component(<ctag>).geom(<tag>).angularUnit()` returns the angular unit.

`model.component(<ctag>).geom(<tag>).scaleUnitValue(boolean)` sets the geometry to scale property values when units are changed.

`model.component(<ctag>).geom(<tag>).scaleUnitValue()` returns true if the geometry is set to scale property values when units are changed.

`model.component(<ctag>).geom(<tag>).repairTol(<relTol>)` sets the default relative repair tolerance to use when creating new features.

`double relTol = model.component(<ctag>).geom(<tag>).repairTol()` returns the default relative repair tolerance.

`model.component(<ctag>).geom(<tag>).geomRep(geomrep)` sets the geometry representation to use in a 3D geometry. The *geomrep* string can be `comsol`, meaning the COMSOL kernel or `cadps` (requires the CAD Import Module), meaning the CAD kernel (Parasolid).

`String geomrep = model.component(<ctag>).geom(<tag>).geomRep()` returns the geometry representation.

`model.component(<ctag>).geom(<tag>).useConstrDim(enable)` enables or disables the constraints and dimensions functionality in a 2D geometry (requires the Design Module).

`boolean enabled = model.component(<ctag>).geom(<tag>).useConstrDim()` returns true if the constraints and dimensions functionality is enabled.

`model.component(<ctag>).geom(<tag>).constrDimBuild(value)` determines what constraint and dimension features to use when building a 2D geometry (requires the Design Module). Valid values are:

- "all": All (enabled) constraint and dimension features are used. This is the default.
- "none": No constraint and dimension features are used.
- "uptotarget": All (enabled) constraint and dimension features up to the feature to build are used.

`String value = model.component(<ctag>).geom(<tag>).constrDimBuild()` returns the constraints and dimensions to use when building. The default is "all".

`String status = model.component(<ctag>).geom(<tag>).constrDimStatus()` gets a description of the overall status of the constraints and dimensions in a 2D geometry (requires the Design Module).

Creating, Editing, Disabling, and Deleting Features

`model.component(<ctag>).geom(<tag>).create(<ftag>, type)` adds a geometry feature *<ftag>* of type *type* to the geometry *<tag>*, after the current feature.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property, <value>)` sets a property in the geometry feature *<ftag>*. All data types listed in [Table 2-4](#) are supported; the applicable data types differ between the properties. String expressions can use parameters from `model.param()`.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property)` returns the value of a property in the geometry feature *<ftag>*.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property).selMethod` manages the geometry object selection property *property*. The available selection methods are described in [Geometry Object Selection Methods](#).

`model.component(<ctag>).geom(<tag>).feature(<ftag>).geom().geomMethod` manipulates the 2D geometry sequence corresponding to the work plane feature *<ftag>*. The available methods are the same as for a 2D geometry `model.geom(<gtag>)`.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).active(false)` disables the feature *<ftag>*.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).active(true)` enables the feature *<ftag>*.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).isActive()` returns true if the feature *<ftag>* is enabled, and false otherwise.

`model.component(<ctag>).geom(<tag>).feature().remove(<ftag>)` removes the feature *<ftag>*.

`model.component(<ctag>).geom(<tag>).feature().move(<ftag>,<position>)` moves the feature <ftag> to the zero indexed position <position> in the sequence.

`String[] remaining = model.component(<ctag>).geom(<tag>).deleteEdges(<objname>, <edges>)` deletes the given edges from the given object in a 2D geometry by using a combination of the following methods:

- Removing the corresponding geometry feature.
- Removing points from a Polygon feature.
- Splitting a Polygon or Composite Curve feature into several features.
- Adding a Delete Entities feature.

The return array contains the tags of the remaining (if any) and created (if any) features.

Getting Tags of Named Selections

`model.component(<ctag>).geom(<tag>).feature(<ftag>).componentSelection()` returns tags of selection output objects for use in selections in the component <ctag>. It returns an empty string array if there are no defined selections.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).outputSelection()` returns tags of selection output objects for use in selections in the geometry sequence <tag>. It returns an empty string array if there are no defined selections.

Building Features

After each build operation, the current feature is set as the last of the active features that were built. The current state contains all objects that are generated by these features.

`model.component(<ctag>).geom(<tag>).run(<ftag>)` builds all features up to (and including) the feature <ftag>.

`model.component(<ctag>).geom(<tag>).runPre(<ftag>)` builds all features preceding the feature <ftag>; for example, with <ftag> being a work plane feature in a 3D geometry, `model.geom(<tag>).runPre(<ftag>)` builds all 3D geometry features preceding the work plane.

`model.component(<ctag>).geom(<tag>).runCurrent()` builds all features up to (and including) the current feature.

`model.component(<ctag>).geom(<tag>).run()` builds all features. The finalized geometry and all selections are also updated.

`model.component(<ctag>).geom().run()` builds the finalized geometry in all geometries.

Getting Build Status

`String fTag = model.component(<ctag>).geom(<tag>).current()` returns the tag of the current feature. If the current state is before the first feature, the empty string "" is returned.

`String status = model.component(<ctag>).geom(<tag>).feature(<ftag>).status()` returns the status of the feature <ftag>. The status is `built`, `warning`, `needs_rebuild`, `edited`, or `error`.

`String msg = model.component(<ctag>).geom(<tag>).feature(<ftag>).message()` returns the warning/error message of the feature <ftag>.

Getting Information About Geometry Objects

`String[] n = model.component(<ctag>).geom(<tag>).objectNames()` returns the names of all objects that exist in the current state.

`String[] n = model.component(<ctag>).geom(<tag>).feature(<ftag>).objectNames()` returns the names of the output object generated by the feature <ftag>.

`model.component(<ctag>).geom(<tag>).obj(<objname>).geomInfoMethod` returns information about the object `<objname>`. The available methods are described in [Geometry Object Information](#).

`model.component(<ctag>).geom(<tag>).geomInfoMethod` returns information about the finalized geometry of geometry `<tag>`.

Getting Information About Named Selections

`model.component(<ctag>).geom(<tag>).selection(<seltag>).selMethod` returns information about the named selection `<seltag>`. See [Selections of Geometric Entities](#) and [Geometry Object Selection Methods](#) for more information.

Geometric Measurements

Use `model.component(<ctag>).geom(<tag>).measure().selection().selMethod` to specify the domains, boundaries, or edges in geometry objects that you want to measure. You can also specify one, two, or several vertices to get the coordinates of the vertex, the distance between the two vertices, or the average coordinates of the vertices, respectively. The available selection methods are described in [Geometry Object Selection Methods](#).

`model.component(<ctag>).geom(<tag>).measure().measurementMethod` returns the volume, area, length, vertex coordinates, or distance between two vertices according to the selection. The available measurement methods are described in [Measurement Methods](#).

Use `model.component(<ctag>).geom(<tag>).measure().getBoundingBox` to return the bounding box of the selected geometry objects or entities. They are returned as a double array of [xmin, xmax, ymin, ymax, ...] depending on the space dimension.

Exporting Geometry Objects

`model.component(<ctag>).geom(<tag>).export().selection().selMethod` can be used to select a number of geometry objects to export to file. The available selection methods are described in [Geometry Object Selection Methods](#).

`model.component(<ctag>).geom(<tag>).export(<filename>)` exports the selected objects to a file.

`model.component(<ctag>).geom(<tag>).exportFinal(<filename>)` exports the finalized geometry to a file.

CAD Defeaturing

If you have a license for the CAD Import Module, or a LiveLink™ product for CAD software, the following functionality is available. For details, see the *CAD Import Module User's Guide*.

`model.component(<ctag>).geom(<tag>).defeaturing(tooltag).defeaturingMethod` uses a defeaturing tool to create a feature that deletes small details. Available tools are listed in the *CAD Import Module User's Guide*.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).find()` searches for small details, for a defeaturing feature `<ftag>`.

`model.component(<ctag>).geom(<tag>).feature(<ftag>).detail().selMethod` manipulates the selection of details to remove, for a defeaturing feature `<ftag>`.

Geometry Object Selection Methods

For a geometry object selection `sel`, the following methods are available:

`sel.init()` sets the selection to be a selection of whole geometry objects. Subsequent calls to `set`, `add`, and `remove` select objects.

`sel.init(dim)` sets the selection property to be a selection of geometric entities of dimension `dim`. Subsequent calls to `all`, `set`, `add`, `remove`, and `clear` select entities.

The following three methods are applicable when the selection consists of whole objects. The argument `<onames>` can be an array of strings, or several string arguments.

`sel.all()` sets the selection to be all available objects.

`sel.set(<onames>)` sets the selection to be the objects <onames>.

`sel.add(<onames>)` adds the objects <onames> to the selection.

`sel.allNonconstruction()` sets the selection to be all available nonconstruction objects.

`sel.remove(<onames>)` removes the objects <onames> from the selection.

The following seven methods are applicable when the selection consists of geometric entities. The argument <entities> can be an array of integers or several integer arguments.

`sel.all(<oname>)` sets the selection to be all the entities of object <oname>. The selections on other objects are not affected.

`sel.all()` sets the selection to be all entities (of the given dimension) of all available objects.

`sel.set(<oname>, <entities>)` sets the selection on object <oname> to be <entities>. The selections on other objects are not affected.

`sel.add(<oname>, <entities>)` adds the entities <entities> to the selection on object <oname>. The selections on other objects are not affected.

`sel.remove(<oname>, <entities>)` removes the entities <entities> from the selection on object <oname>. The selections on other objects are not affected.

`sel.clear()` removes all entities from the selection.

The `set`, `add`, and `remove` methods can all be vectorized. See [Selections of Geometric Entities](#) for more information.

`sel.clear(<oname>)` clears the selection on object <oname>. The selections on other objects are not affected.

To let the selection be defined by a named selection, use:

`sel.named(<seltag>)` where <seltag> is the trimmed tag of a named selection defined by a preceding feature in the geometry sequence. See [Selections of Geometric Entities](#) for more information.

To get information about the selection, use:

`String[] onames = sel.objects()` returns the names of the selected objects.

`int dim = sel.dim()` returns the dimension for the entities in the selection if the selection consists of geometric entities; otherwise, it returns -1.

`int[] dims = sel.dimension()` returns an array of length 1 containing the entity dimension if the selection consists of geometric entities; otherwise, it returns an empty array.

`int[] ent = sel.entities(<oname>, dim)` returns the entities in the selection on object <oname> at the given dimension, if the selection consists of geometric entities.

`int[] ent = sel.entities(<oname>)` returns the entities in the selection on object <oname>, if the selection consists of geometric entities.

`String[] seltag = sel.named()` returns the trimmed tag of the named selection that this selection refers to, or an empty string if the selection does not refer to a named selection.

If Statements

Use `model.geom(<tag>).create(<ftag>, <type>)` to add an If, Else If, Else, or End If feature after the current feature.

Use `model.geom(<tag>).createAfter(<ftag>, <type>, <postag>)` to add an If, Else If, Else, or End If feature after the feature tagged `<postag>`.

Insert Sequence

`model.geom(<tag>).insertFile(<filename>, <gtag>)`; inserts a geometry sequence, with tag `<gtag>`, from another model file, with the filename `<filename>`, into the geometry sequence with tag `<tag>`.

`model.geom(<tag>).insertSequence(<mtag>, <gtag>)`; inserts a geometry sequence, with tag `<gtag>`, from another model, with tag `<mtag>`, into the current geometry sequence with tag `<tag>`.

EXAMPLE

Create a 2D geometry model as the union of a circle and rectangle.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
g.create("r1", "Rectangle");
g.feature("r1").set("size", new double[] {0.5, 1});
g.feature("r1").set("pos", new double[] {-1, 0});
g.create("c1", "Circle");
g.feature("c1").set("r", 0.5);
g.feature("c1").set("pos", new double[] {0.5, 0});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component().create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
g.create('r1', 'Rectangle');
g.feature('r1').set('size', [0.5, 1]);
g.feature('r1').set('pos', [-1, 0]);
g.create('c1', 'Circle');
g.feature('c1').set('r', 0.5);
g.feature('c1').set('pos', [0.5, 0]);
g.run;
```

COMPATIBILITY

From version 5.6, for a selection of entities, the method `objects()` returns feature tags for objects that are created by a feature that creates exactly one geometry object.

From version 5.1, the access method `argument()` is deprecated and replaced with `inputParam()`, and the access method `param()` is deprecated and replaced with `localParam()`. The methods `link(...)`, `linked()`, and `relink()` are deprecated and replaced with `load(...)`, `loaded()`, and `reload()`, respectively.

From version 4.4, the method

```
model.geom(<tag>).runAll();
```

is deprecated. Instead, use

```
model.component(<ctag>).geom(<tag>).run(<ftag>);
```

to specify to which feature node in the geometry sequence you want to run.

From version 4.3a, the methods

```
model.geom(<tag>).object(<objname>)
String[] onames = sel.object()
```

are deprecated and replaced by the following methods:

```
model.component(<ctag>).geom(<tag>).obj(<objname>)
```

```
String[] onames = sel.objects()
```

SEE ALSO

[model.mesh\(\)](#)

model.group()

Add load groups and constraint groups.

SYNTAX

```
model.group().create(<tag>,type);  
model.group(<tag>).identifier(<id>);
```

```
model.group(<tag>).type();  
model.group(<tag>).identifier();
```

DESCRIPTION

`model.group().create(<tag>,type)` creates a new group of the specified type, which can be either `LoadGroup` or `ConstraintGroup`.

`model.group(<tag>).identifier(<id>)` sets the group identifier, which is used for defining a corresponding parameter, group.<id>.

`model.group(<tag>).type()` returns the group type as a string.

`model.group(<tag>).identifier()` returns the group identifier.

model.init()

Creating and defining initial values.

SYNTAX

```
model.init().create(<tag>);  
model.init(<tag>).set(<fieldname>,<expr>);  
model.init(<tag>).remove(<fieldname>);
```

```
model.init(<tag>).varnames();  
model.init(<tag>).get(<fieldname>);
```

DESCRIPTION

`model.init().create(<tag>)` creates a new initial value with tag <tag>.

`model.init(<tag>).set(<fieldname>,<expr>)` defines the expression <expr> as the initial value for the dependent variable (field variable) <fieldname>.

`model.init(<tag>).remove(<fieldname>)` removes the field variable <fieldname> from the initial value with tag <tag>.

`model.init(<tag>).selection().named(<seltag>)` assigns the initial value to the named selection <seltag>. `model.init(<tag>).selection().set(...)` defines a local selection that assigns the initial value to geometric entities. For a complete list of methods available under `selection()`, see [Selections](#). Only selections at a single geometry level is allowed except for ODE states which require the global selection.

`model.init(<tag>).varnames()` returns the names of the variables for the initial value with tag <tag> as a string array.

`model.init(<tag>).get(<fieldname>)` returns the initial value for the field variable <fieldname> as a string.

`model.init(<tag>).selection().named()` returns the named selection tag.

`model.init(<tag>).selection().getType()` returns domain information for the initial value with tag <tag>; see [Selections](#) for available methods.

model.intRule()

Integration rules.

SYNTAX

```
model.intRule().create(<tag>,<ftag>);
model.intRule(<tag>).frame(<ftag>);
model.intRule(<tag>).create(<ftag>);
model.intRule(<tag>).feature(<ftag>).order(gporder);
model.intRule(<tag>).frame();
```

DESCRIPTION

`model.intRule().create(<tag>,<ftag>)` creates an integration rule for the frame <ftag>.

`model.intRule(<tag>).frame(<ftag>)` sets the frame for the integration rule.

`model.intRule(<tag>).create(<ftag>)` creates an integration rule feature.

`model.intRule(<tag>).feature(<ftag>).order(gporder)` specifies the integration order of the integration rule.

`model.intRule(<tag>).frame()` returns the frame as a string.

EXAMPLE

Specify two integration rules, one with the integration order 2 and one with the integration order 4.

Code for Use with Java

```
model.intRule().create("ir1","f");
model.intRule("ir1").create("ir1").order(2);
model.intRule("ir1").create("ir2").order(4);
```

Code for Use with MATLAB

```
model.intRule.create('ir1','f');
model.intRule('ir1').create('ir1').order(2);
model.intRule('ir1').create('ir2').order(4);
```

SEE ALSO

[model.shape\(\)](#)

model.massProp()

Compute mass properties and add mass contributions.

SYNTAX

```
model.component(<ctag>).massProp().create(<tag>,"MassProperties");
model.component(<ctag>).massProp(<tag>).selection();
model.component(<ctag>).massProp(<tag>).set(<pname>,<expr>);
model.component(<ctag>).massProp(<tag>).image();

model.component(<ctag>).massProp(<tag>).create(<mctag>,"MassContributions");
model.component(<ctag>).massProp(<tag>).feature(<mctag>).selection();
model.component(<ctag>).massProp(<tag>).feature(<mctag>).set(<pname>,<expr>);
```


DESCRIPTION

Creating Mass Properties and Providing Geometry Source Selections

`model.component(<ctag>).massProp().create(<tag>,"MassProperties")` creates a mass properties object that computes variables for mass properties such as total mass and the center of mass.

`model.component(<ctag>).massProp(<tag>).selection().set(...)` defines a local selection that assigns the mass properties to geometric entities that act as sources. For a complete list of methods available under `selection()`, see [model.selection\(\)](#).

Specifying Density and Mass Properties

You can specify the density to be taken from a physics. For example,

```
model.component(<ctag>).massProp(<tag>).set("densitySource", "fromSpecifiedPhysics")
model.component(<ctag>).massProp(<tag>).feature("mc1").set("physics", "solid");
```

takes the density from the Solid Mechanics interface `solid`.

You can also specify a user-defined density. For example, to specify the density to be 1107 kg/m^3 , use

```
model.component(<ctag>).massProp(<tag>).set("densitySource", "userDefined");
model.component(<ctag>).massProp(<tag>).set("expr", "1107[kg/m^3]");
```

To create variables for the center of mass and to not create variables for the moment of inertia:

```
model.component(<ctag>).massProp(<tag>).set("createCenterOfMass", "on");
model.component(<ctag>).massProp(<tag>).set("createMomentOfInertia", "off");
```

Adding a Mass Contribution

You can add mass contributions from other parts of the geometry (an adjacent boundary, for example) by creating a mass contribution:

```
model.component(<ctag>).massProp(<tag>).create("mc1", "MassContributions");
model.component(<ctag>).massProp(<tag>).feature("mc1").set("expr", "3");
```

For the mass contributions, you can specify source selections and define the density in the same way as for the mass properties.

Properties and Variables for massProp

The `massProp` feature accepts the following properties:

TABLE 2-106: MASSPROP PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>createCenterOfMass</code>	on off	on	Create center of mass variables.
<code>createMass</code>	on off	on	Create mass variable.
<code>createMassContribution</code>	on off	on	Add a mass contribution.
<code>createMomentOfInertia</code>	on off	on	Create moment of inertia variables.
<code>createPrincipalInertia</code>	on off	on	Create principal moment of inertia variables.
<code>createVolume</code>	on off	on	Create volume variable.
<code>densitySource</code>	fromSpecifiedPhysics userDefined	userDefined	Source for the density values.
<code>expr</code>	String		User-defined density expression.
<code>includeLowerPh</code>	on off	on	Include adjacent entities of lower dimension for density values taken from physics.
<code>densityFrame</code>	frame	Material	The frame to which the user-defined density refers.

TABLE 2-106: MASSPROP PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
intorder	4	Positive integer	Integration order.
outputFrame	frame	Material	Integration frame for output variables.

The created variables for different mass properties have the following default names:

- `mass1.mass` and `mass1.volume` for the mass and volume, respectively.
- `mass1.CMX`, and so on, for the coordinates of the center of mass.
- `mass1.IXY`, and so on, for the components of the moment of inertia.
- `mass1.Ip1`, and so on, for the moment of inertia principal values.
- `mass1.Ip1X`, and so on, for the moment of inertia principal directions.

model.material()

Materials and material property groups.

SYNTAX

To create a global material of the type `<type>`, use the syntax

```
model.material().create(<tag>,<type>,"");
```

To create a material for the component with tag `<ctag>`, use instead

```
model.component(<ctag>).material().create(<tag>);
model.component(<ctag>).material().create(<tag>,<type>);
```



For materials and material library links defined on the global level, omit `component(<ctag>)` from the syntax examples below.

```
model.component(<ctag>).material(<tag>).info(<itag>);
model.component(<ctag>).material(<tag>).info();
model.component(<ctag>).material().insert(...)
model.component(<ctag>).material(<tag>).label(<label>);
model.component(<ctag>).material(<tag>).materialType(<mattype>);
model.component(<ctag>).material(<tag>).propertyGroup(<mtag>);
model.component(<ctag>).material(<tag>).propertyGroup();
model.component(<ctag>).material(<tag>).prefix(<prefix>);
model.component(<ctag>).material(<tag>).prefix();
model.component(<ctag>).material(<tag>).selection();
model.component(<ctag>).material(<tag>).image();
model.component(<ctag>).material(<tag>).set(<pname>,<expr>);
model.component(<ctag>).material().move(<tag>,<position>);

model.MaterialList.export(<filename>,<materials>);
model.ComponentMaterialList.export(<filename>,<materials>);

MaterialModel mm = model.component(<ctag>).material(<tag>).propertyGroup().
    create(<mtag>,<descr>);
mm.addInput(<quantity>);
mm.descr(<pname>,<descr>);
mm.func();
mm.func(<ftag>);
mm.setMixingRule(<pname>,<mixRule>);
mm.getPropertyInfo(<pname>);
mm.getString(<pname>);
mm.getStringArray(<pname>);
mm.getValueType(<pname>);
mm.hasParam(<pname>);
mm.info(<itag>);
mm.info();
mm.input();
mm.isOutput(<pname>);
mm.param();
mm.removeInput(<quantity>);
mm.set(<pname>,<expr>);
mm.setMixingRule(<pname>,<mixRule>);
mm.setPropertyInfo(<pname>,<info>);
mm.size(<pname>);
mm.suffix(<suffix>);
mm.suffix();

mm.info().create(<itag>,<descr>);
mm.info(<itag>).title(<title>);
mm.info(<itag>).title();
mm.info(<itag>).body(<body>);
mm.info(<itag>).body();
```

DESCRIPTION

A material is a collection of property groups, where each property group defines a set of material properties, material functions, and model inputs that can be used to define a temperature-dependent material property, for example. A property group usually defines properties used by a particular material model to compute a fundamental quantity. A material property can either be a visible output property or a local parameter. The output property is visible for physics interfaces; local properties are only visible inside the property group. If two property groups define the same output property, the last property group determines the value of the output property. The material function is used

by the property group to calculate a property or parameter value as a function of other variables, usually model inputs. The model input is a quantity that the material model recognizes as an input variable (temperature, for example). The actual variable that represents the model input is not known until the model is solved, and it can also be different between physics interfaces.

Material library links and material links are also described in this section.

There are two types of property groups, user-defined and specialized. When a material is created, there is always one default user-defined property group present. To this property group it is possible to add output properties from a predefined list of quantities. These quantities are recognized by all physics interfaces as material properties — for example, thermal conductivity, electric conductivity, and density. The full list is presented in the physics interface for the default property group. The specialized property groups are built in and usually define few output properties that only some physics interfaces can access. These output properties are not necessarily part of the allowed properties for the default property group. An example of such a specialized group is the refractive index material model, which defines the real and imaginary part of the refractive index as output properties. These properties can only be accessed by the Electromagnetic Waves interface.

`model.component(<ctag>).material().create(<tag>)` creates a new material for the model component with the tag <ctag>.

`model.component(<ctag>).material().create(<tag>, "Common")` also creates a new material for the model component with the tag <ctag>.

`model.component(<ctag>).material().create(<tag>, "Switch")` creates a material switch for the model component with the tag <ctag>. You can add materials to the material switch:

```
model.component(<ctag>).material().create("sw1", "Switch", "");
model.component(<ctag>).material("sw1").feature().create("mat1", "Common", "");
model.component(<ctag>).material("sw1").feature().create("mat2", "Common", "");
```

`model.component(<ctag>).material().create(<tag>, "Link")` creates a material link for the model component with the tag <ctag>.

`model.component(<ctag>).material().create(<ptag>, "PorousMedia")` also creates a new local porous material for the model component with the tag <ctag>. Use `model.component(<ctag>).material(<ptag>).feature().create(<subtag>, <subfeaturename>, <ctag>);` where <subfeaturename>

is one of "Fluid", "Pellet", "Solid", or "ImFluid" to add a subfeature to the porous material feature representing a fluid, pellet, solid, or immobile fluid, respectively.

`model.component(<ctag>).material().create(<tag>, "External")` creates an external material that sets up an interface between a physics feature and functions in an external shared library (a DLL, .so, or .dylib file.). For an external material,

`model.component(<ctag>).material(<tag>).set("path", <path to external material file>)` sets the path to the external shared library. Also, use `model.component(<ctag>).material(<tag>).set("threadSafe", "off")` if the DLL is not thread safe (default is, "on"; that is, the DLL is thread safe).

`model.material().create(<tag>, <type>)` creates a global material, material switch, or material link.

`model.component(<ctag>).material(<tag>).info(<itag>)` returns an information item for a material.

`model.component(<ctag>).material(<tag>).input()` returns the list of model inputs.

For `model.component(<ctag>).material().insert(...)`, see [Inserting Materials](#).

`model.component(<ctag>).material(<tag>).materialType("solid")` or `model.component(<ctag>).material(<tag>).materialType("nonSolid")` sets the material type to a solid or a nonsolid (that is, fluid or liquid) material, respectively.

`model.component(<ctag>).material(<tag>).propertyGroup(<mtag>)` gets the property group named `<mtag>` for the material.

`model.component(<ctag>).material(<tag>).selection()` returns the selection of the material. The selection determines which geometry the material belongs to.

`model.component(<ctag>).material(<tag>).set("family", <appearance>)` sets the appearance to a family of materials (as strings; "water", for example): air, aluminum, brick, concrete, copper, defaultmaterial, defaultplot, gold, iron, lead, magnesium, plastic, steel, titanium, water, as a few of the available material types, or custom. You can set a "color" property for plastic, defaultplot, and some other material types. With custom you can set these additional parameters for the appearance:

TABLE 2-107: PROPERTIES FOR CUSTOM APPEARANCE OF MATERIALS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alpha	double	1	Diffuse and ambient color opacity.
ambient	string[] (a color or custom)	custom	Ambient light color.
anisotropy	double (0–1)	0	Add anisotropic specular highlighting.
anisotropyaxis	string array of length 3.	0, 0, 1	The axis for the anisotropy.
clearcoat	double (0–1)	0.15	Clear coat for Cook–Torrance lighting model.
colornoise	true false	false	Add an additional color to the appearance using noise.
colornoisebrush	0 1 2 3 4 5 6 7	0	No brush lines (0); brush lines around x, y, and z (1, 2, 3, respectively); brush lines along x, y, and z (4, 5, 6, respectively), if colornoise is true.
colornoisefrequency	double	1	Additional color noise frequency, if colornoise is true.
colornoisenormalscale	double	0	Normal vector noise scale for the added color, if colornoise is true.
colornoisescale	double	0	Additional color noise scale, if colornoise is true.
colornoisetype	0 or 1	0	White noise (0) or simplex noise (1), if colornoise is true.
customambient	double[]		RGB values for a custom ambient color.
customdiffuse	double[]		RGB values for a custom diffuse color.
customspecular	double[]		RGB values for a custom specular color.
diffuse	string[] (a color or custom)	custom	Diffuse light color.
diffusewrap	double (0–1)	0.27	Diffuse wrap for Cook–Torrance lighting model.
flipanisotropy	true false	false	Flip the anisotropy for anisotropic specular highlighting.
fresnel	double (0–1)	0	Reflectance at normal incidence for Cook–Torrance.
lighting	phong cooktorrance simple	phong	Lighting model.
metallic	double (0–1)	0.1	Metallic surface for Cook–Torrance lighting model.
noise	true false	false	Add noise to the appearance.

TABLE 2-107: PROPERTIES FOR CUSTOM APPEARANCE OF MATERIALS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
noisecolor	string[] (a color or custom)	custom	The color for an additional color, if colornoise is true.
noisecolorblend	double (0–1)	0	The color blend, if colornoise is true.
noisefreq	double	1	Normal vector noise frequency.
noisescale	double	0	Normal vector noise scale.
normalnoisebrush	0 1 2 3 4 5 6 7	0	No brush lines (0); brush lines around x, y, and z (1, 2, 3, respectively); brush lines along x, y, and z (4, 5, 6, respectively).
normalnoisetype	0 or 1	0	White noise (0) or simplex noise (1).
pearl	double (0–1)	0.015	Pearl surface for Cook–Torrance lighting model.
reflectance	double (0–1)	0	Reflectance for Cook–Torrance lighting model.
roughness	double (0–1)	0.45	Surface roughness for Cook–Torrance lighting model.
shininess	double	64	Specular exponent for Blinn–Phong lighting model.
specular	string[] (a color or custom)	custom	Specular light color.

`model.component(<ctag>).material().move(<tag>,<position>)` moves the material `<tag>` to the zero-indexed position `<position>` in the list.

Material Library Links

For a material library link, `model.component(<ctag>).material().create("mat1", "LibraryLink");` creates a material library link. As long as the link is active, you cannot change any material properties in the material library link or add any user-defined property groups. Otherwise, it supports the same API properties as a material. To reload the material properties from the material library (if they have changed there), use

```
model.component(<ctag>).material("mat1").reload();
```

To specify the link to the material library use

```
model.component(<ctag>).material("mat3").set("materialPath",
"comsol_basic_material_lib.silicon");
```

As an example here, the link is to silicon in the Built-in material library.

To unlink the material library link, use

```
model.component(<ctag>).material("mat1").set("isLinked", false);
```

If you unlink the material library link, it then works just like any other material.

Material Links

For a material link, `model.component(<ctag>).material(<tag>).set("link",<linktag>);` creates a material link from a component to the global material with the tag in `<linktag>`. For example,

```
model.component("comp1").material().create("matlnk1", "Link");
model.material("matlnk1").set("link", "mat1");
```

creates a material link in the component `comp1` and then links it to the global material `mat1`. The material link can also have a selection; for example, `model.material("matlnk1").selection().all();`

Importing Materials

You can import materials as global materials and as materials in a component.

To import global materials as `mat1`, `mat2`, and `mat3` from `myfile.xml`, for example:

```
String[][] result = model.material().insert("myfile.xml",
    new String[]{"mat1", "mat2", "mat3"});
```

To import the same materials to the component comp1:

```
String[][] result = model.component("comp1").material().insert("myfile.xml",
    new String[]{"mat1", "mat2", "mat3"});
```

The `insert` method inserts materials from an MPH- or XML-file into the material list and returns the result of the operation as a double string array of length 3. The first array contains messages from the insert operation, the second array contains paths to the inserted objects, and the third array contains paths to the inserted references. An inserted reference is an object that an inserted object refers to and is not necessarily contained by any of the inserted objects. There can only be inserted references when importing from an MPH file. The first parameter is the filename, and the second is the tags of the materials to insert. For protected MPH files, you can supply a password as an optional third parameter.

See [Materials](#) in *The COMSOL File Formats* chapter for information about the XML-file format for materials that you can import and export.

Exporting Materials

Materials and their data can be exported to the Model XML-File format using the following methods:

```
String[][] com.comsol.model.MaterialList.export(String filename, String[] materials)
String[][] com.comsol.model.ComponentMaterialList.export(String filename, String[] materials)
```

The following two input arguments are available:

- **filename:** The filename or model location URI used to identify a model version in a Model Manager database.
- **materials:** The tags of the materials to export.

The return value is an array of length 2 containing nested string arrays. The first dimension determines the type of messages that are found in the nested array:

- The string array at 0 contains warning messages.
- The string array at 1 contains messages from model export operations.

The following example shows how to export the materials tagged with `mat5` and `mat10` to a file at the path `C:\temp\material_export.xml`:

```
String[][] responses = model.material().export("C:\temp\material_export.xml", new String[]{"mat5",
    "mat10"});
```

The same export can be accomplished at the level of a component, assuming that the materials `mat5` and `mat10` belong to the component with tag `comp1`:

```
String[][] responses = model.component("comp1").material().export("C:\temp\material_export.xml", new
    String[]{"mat5", "mat10"});
```

If successful, these commands can often produce an empty output: `responses = {{}}, {{}}`. Messages in the second string array are typically not a sign of problems.

See [Materials](#) in *The COMSOL File Formats* chapter for information about the XML-file format for materials that you can import and export.

Property Groups

The following syntax examples describe the methods available for property groups added to a material (not available for material switches or material links).

`mm = model.component(<ctag>).material(<mtag>).propertyGroup().create(<tag>, <descr>)` returns a property group with the given tag and description and stores it in the variable `mm`. If `<tag>` is equal to any of the

known built-in types, a property group of that type is created, except if a built-in property group matching the tag/type already exists, in which case that property-group instance is returned.

`mm = model.component(<ctag>).material(<mtag>).propertyGroup().create(<tag>, type, <descr>)` creates and returns a property group of the type *type* with the given tag and description. If *type* is a built-in type and a property group of this type already exists in the model, the method throws an error.

To define a method to set an output property of a material, use the following syntax (in this example you specify a density for the basic property group `def`):

```
model.component(<ctag>).material("mat1").propertyGroup("def").  
set("density", String[] arg);
```

where *arg* is the string value to be defined. For a list of available physical property names (such as `density`), see [Table 2-108](#).

`mm.addInput(<quantity>)` adds a new model input to the property group of the given quantity.

`mm.descr(<pname>, <descr>)` adds the description *<descr>* to the local material property with the name *<pname>*.

`mm.func(<ftag>)` returns the function object named *<ftag>*. For information about how to add and modify functions, see [model.func\(\)](#).

`mm.getMixingRule(<pname>)` returns the mixing rule for the given material property.

`mm.getPropertyInfo(<pname>)` returns the property info (references, temperature conditions, and so on) for the given material property.

`mm.getString(<pname>)` returns the string value of the given property. If it is a vector or matrix quantity, the first value is returned.

`mm.getStringArray(<pname>)` returns the string array value of the given property. Matrix values are returned in a column-wise order.

`mm.getValueType(<pname>)` returns the main data type that a property can return. The data types are: `String`, `StringArray`, and `StringMatrix`.

`mm.hasParam(<pname>)` returns true if the property is defined by the property group.

`mm.info(<itag>)` returns the information object for the property group.

`mm.isOutput(<pname>)` returns true if the given local material parameter is an output property. For user-defined property groups the method returns true for all predefined material properties known to all physics interfaces. For the specialized property groups, it can also return true for other properties.

`mm.param()` returns a list of all parameters stored in the property group.

`mm.removeInput(<quantity>)` removes the given quantity from the list of model inputs.

`mm.set(<pname>, <expr>)` sets the expression for the given property. The expression can use local names for the properties, parameters, and model inputs. For vector and matrix properties, the expression can be string arrays of varying size. Isotropic matrices only require one element or a string, diagonal matrices require three elements, and so forth. Vectors always require three elements.

`mm.setMixingRule(<pname>, <mixRule>)` sets the mixing rule for the given material property.

`mm.setPropertyInfo(<pname>, <info>)` sets the property info (references, temperature conditions, and so on) for the given material property.

`mm.size(<pname>)` returns the size of the stored property, which usually is 1-by-1, 3-by-1, or 3-by-3, but other sizes are supported.

`model.component(<ctag>).material(<tag>).propertyGroup()` returns a list of all property groups in the material.

`model.component(<ctag>).material(<tag>).propertyGroup(<tag>).info().create(<tag>,<descr>)` creates a new information object that can contain detailed information about this property group. This could, for example, be used by the Material Library to define the Phase/Condition and Orientation/Condition fields.

`mm.info(<itag>).title(<title>)` sets the title of the information object.

`mm.info(<itag>).title()` returns the title.

`mm.info(<itag>).body(<body>)` sets the body text of the information object.

`mm.info(<itag>).body()` returns the body text.



The term *material model* is sometimes used instead of *property group* in some contexts. The graphical user interface uses the term property group.

TABLE 2-108: AVAILABLE PHYSICAL QUANTITIES.

PHYSICAL QUANTITY	NAME
Absorbed dose	absorbeddose
Absorption coefficient	absorption
Acceleration	acceleration
Activation energy	activationenergy
Angular acceleration	angularacceleration
Angular frequency	angularfrequency
Area	area
Bulk viscosity	bulkviscosity
Capacitance	capacitance
Catalytic activity	catalyticactivity
Characteristic acoustic impedance	acousticimpedance
Charge	charge
Coefficient of hygroscopic swelling	hygroscopicswellingcoefficient
Coefficient of thermal expansion	thermalexpansioncoefficient
Collisional power loss	inelasticpowerloss
Compliance	compliance
Compressibility of fluid	compressibility
Concentration	massconcentration
Concentration	concentration
Conductance	conductance
Corrected pressure	correctedpressure
Corrected velocity field	correctedvelocity
Current	current
Current density	currentdensity
Current source	currentsource

TABLE 2-108: AVAILABLE PHYSICAL QUANTITIES.

PHYSICAL QUANTITY	NAME
Damping constant per unit area	dampingconstantperarea
Damping constant per unit length	dampingconstantperlength
Damping constant per unit volume	dampingconstantpervolume
Density	density
Diffusion coefficient	diffusion
Dimensionless	dimensionless
Dipole moment	dipolemoment
Dispersed phase volume fraction	dispersedphasevolume fraction
Displacement field	displacement
Displacement of shell normals	dimensionless_displacement
Dose equivalent	doseequivalent
Dynamic viscosity	dynamicviscosity
EEDF	eedf
Edge load	edgeload
Effective gas density	effectivegasdensity
Effective plastic strain	effectiveplasticstrain
Elasticity	elasticity
Elastoresistive coupling	couplingelastoresistive
Electric displacement field	displacementfield
Electric field	electricfield
Electric permittivity	permittivity
Electric potential	electricpotential
Electric conductivity	electricconductivity
Electrolyte conductivity	electrolyteconductivity
Electrolyte potential	electricpotentialionicphase
Electron density	electrondensity
Electron energy density	electronenergydensity
Electron mobility	electronmobility
Energy	energy
Energy density	energydensity
Entropy	entropydensity
External free energy	externalfreeenergy
Extinction coefficient	extinctioncoefficient
Face load	faceload
Flow rate out from source per unit length	areapertime
Flow resistivity	pressure timeperarea
Fluid conductance	fluidconductance
Force density	forcedensity
Force load	force
Force potential	forcepotential
Fowler–Nordheim coefficient	fowlernordheimcoefficient

TABLE 2-108: AVAILABLE PHYSICAL QUANTITIES.

PHYSICAL QUANTITY	NAME
Frequency	frequency
Frequency factor	frequencyfactor
Head	head
Heat capacity at constant pressure (molar)	molarheatcapacity
Heat capacity at constant pressure	heatcapacity
Heat source	powerdensity
Heat transfer coefficient	heattransfercoefficient
Henry's constant	henrysconstant
Hydraulic conductivity	hydraulicconductivity
Inductance	inductance
Initial curvature	planeangleperlength
Initial electron density	initialelectrondensity
Intensity (RMS)	intensity
Inward heat flux	heatflux
Isotropic structural loss factor	lossfactor
Kinematic viscosity	kinematicviscosity
Length	length
Level set variable	levelsetvariable
Line charge	linecharge
Line current source	linecurrentsource
Log mass fraction	logmassfraction
Log of electron density	logelectrondensity
Log of electron energy density	logelectronenergydensity
Logarithmic ratio	logarithmicratio
Logarithmic ratio per unit length	logarithmicratiooperunitlength
Luminous intensity	luminousintensity
Magnetic field	magneticfield
Magnetic flux	magneticflux
Magnetic flux density	magneticfluxdensity
Magnetic permeability	permeability
Magnetic scalar potential	magneticscalarpotential
Magnetic vector potential	magneticvectorpotential
Mass	mass
Mass flow	massflow
Mass flux	massflux
Mass fraction	massfraction
Mass per unit area	massperarea
Mass per unit length	massperlength
Mass source	masssource
Mass transfer coefficient	masstransfercoefficient
Mean electron energy	meanelectronenergy

TABLE 2-108: AVAILABLE PHYSICAL QUANTITIES.

PHYSICAL QUANTITY	NAME
Mean flow velocity potential	meanflowvelocitypotential
Mean molar mass	molarmass
Molar enthalpy	energyperamount
Molar flux	molarflux
Molar surface flux	molarsurfaceflux
Molar volume	molarvolume
Moment body load	torquepervol
Moment edge load	torqueperlength
Moment face load	torqueperarea
Natural logarithmic ratio	naturallogarithmicratio
Natural logarithmic ratio per unit length	naturallogarithmicratioperunitlength
Normal electron current density	normalelectroncurrentdensity
Normal ion current density	normalioncurrentdensity
Number density	numberdensity
Particle momentum	momentum
Particle position	position
Permeability	hydraulicpermeability
Phase field help variable	phasefieldhelpvariable
Phase field variable	phasefieldvariable
Piezoelectric coupling d (strain-charge)	couplingstraincharge
Piezoelectric coupling e (stress-charge)	couplingstresscharge
Piezoresistive coupling	couplingpiezoresistive
Plane angle	planeangle
Point current source	pointcurrentsource
Poiseuille coefficient	poiseuillecoefficient
Poisson's ratio	poissonsratio
Porosity	porosity
Power	power
Power flow	powerflow
Power per unit charge	powerpercharge
Power per unit length	powerpermeter
Power per unit mass	powerpermass
Pressure	pressure
Production rate	productionrate
Production/absorption coefficient	heatproduction
Radiative intensity	radiativeintensity
Radioactivity	radioactivity
Ratio of specific heats	ratioofspecificeat
Reaction rate	reactionrate
Reciprocal area	reciprocalarea
Reciprocal initial interface distance	reciprocallength_i

TABLE 2-108: AVAILABLE PHYSICAL QUANTITIES.

PHYSICAL QUANTITY	NAME
Reciprocal wall distance	reciprocallength
Recombination rate (domain)	recombinationratedomain
Reduced electric field	reducedelectricfield
Reduced electron diffusivity	reduceddiffusivity
Reduced electron mobility	reducedmobility
Relative permeability	relpermeability
Relative permittivity	relpermittivity
Resistance	resistance
Resistivity	resistivity
Scattering coefficient	scattering
Secondary emission energy flux	energyflux
Secondary emission flux	particleflux
Seebeck coefficient	seebeckcoefficient
Solid angle	solidangle
Space charge density	spacechargedensity
Specific dissipation rate	specificdissipationrate
Specific energy	specificenergy
Speed of sound	soundspeed
Spring constant per unit area	springconstantperarea
Spring constant per unit length	springconstantperlength
Spring constant per unit volume	springconstantpervolume
Squared slip velocity	slipvelocity
Storage	storage
Strain energy per unit area	energydensityperarea
Strain energy per unit length	energydensityperlength
Strain reference temperature	strainreferencetemperature
Stress tensor	stress
Substance	substance
Surface capacitance	surfacecapacitance
Surface charge density	surfacechargedensity
Surface current density	surfacecurrentdensity
Surface electric conductivity	surfaceconductivity
Surface emissivity	emissivity
Surface energy density	surfaceenergydensity
Surface magnetic current density	surfacemagneticcurrentdensity
Surface resistance	surfaceresistance
Surface site concentration	surfaceconcentration
Surface tension coefficient	surfacetensioncoefficient
Temperature	temperature
Thermal conductivity	thermalconductivity
Time	time

TABLE 2-108: AVAILABLE PHYSICAL QUANTITIES.

PHYSICAL QUANTITY	NAME
Time change in pressure head	timechangeinpressurehead
Torque	torque
Total damping constant	dampingconstant
Total spring constant	springconstant
Trap density distribution (boundary)	trapdensityboundary
Trap density distribution (domain)	trapdensitydomain
Turbulent dissipation rate	turbulentdissipationrate
Turbulent kinetic energy	turbulentkineticenergy
Undamped turbulent kinematic viscosity	turbulentkinematicviscosity
Velocity field	velocity
Velocity potential	velocitypotential
Volume	volume
Volume fraction	volumefraction
Volume per time	volumepertime
Volumetric heat capacity	volumetricheatcapacity
Wave number	wavenumber
Wavelength	wavelength
Young's modulus	youngsmodulus

SEE ALSO

`model.func()`, `model.physics()`

model.mesh()

Meshing sequences.

	See the Mesh section for more information.
---	--

SYNTAX

```
model.component(<ctag>).mesh().create(<tag>,<gtag>);
model.component(<ctag>).mesh().remove(<tag>);
model.component(<ctag>).mesh(<tag>).create(<ftag>,operation);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag2>,operation);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature().meshMethod
model.component(<ctag>).mesh(<tag>).current(<ftag>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).active(boolean);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).isActive();
model.component(<ctag>).mesh(<tag>).feature().move(<ftag>,<position>);
model.component(<ctag>).mesh(<tag>).feature().remove(<ftag>);
model.component(<ctag>).mesh(<tag>).feature().image();
model.component(<ctag>).mesh(<tag>).clearMesh();
model.component(<ctag>).mesh(<tag>).geom();
model.component(<ctag>).mesh(<tag>).image();

model.component(<ctag>).mesh(<tag>).run(<ftag>);
model.component(<ctag>).mesh(<tag>).run();
model.component(<ctag>).mesh().run();
model.component(<ctag>).mesh().runForStudy(<studyStep>);

model.component(<ctag>).mesh(<tag>).current();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).status();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).message();

model.component(<ctag>).mesh(<tag>).feature(<ftag>).componentSelection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).outputSelection();

model.component(<ctag>).mesh(<tag>).meshGetMethod
model.component(<ctag>).mesh(<tag>).data().meshModificationMethod
model.component(<ctag>).mesh(<tag>).stat().meshStatisticsMethod
model.component(<ctag>).mesh(<tag>).measure().measurementMethod

model.component(<ctag>).mesh(<tag>).export(<filename>);
```

DESCRIPTION

Creating and Deleting a Meshing Sequence

`model.component(<ctag>).mesh().create(<tag>,<gtag>)` creates a meshing sequence (or just mesh) for the geometry sequence `<gtag>` and assigns it the tag `<tag>`.

`model.component(<ctag>).mesh().remove(<tag>)` removes the meshing sequence `<tag>`.

Creating, Editing, and Deleting Features

`model.component(<ctag>).mesh(<tag>).create(<ftag>,operation)` adds a feature `<ftag>` of type `operation` to the meshing sequence `<tag>`, after the current feature.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>)` sets the property `property` defined for the feature `<ftag>` to the value `<value>`. All data types listed in [Table 2-4](#) are supported; the applicable data types differ between the properties. String expressions can use parameters from `model.param()`.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property)` returns the value of a property in the feature `<ftag>`.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().selMethod` manages the selection of the feature `<ftag>`. The available selection methods are described in [Selection Methods](#).

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property).selMethod` manipulates the selection of the property *property*. The available selection methods are described in [Selection Methods](#).

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature().meshMethod` manages the entity list for the feature *<ftag>*.

`model.component(<ctag>).mesh(<tag>).current(<ftag>)` sets the current feature to be *<ftag>*.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).active(false)` disables the feature *<ftag>*.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).active(true)` enables the feature *<ftag>*.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).isActive()` returns `true` if the feature *<ftag>* is enabled, and `false` otherwise.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>)).`

`getAllowedPropertyValues(property)` returns the set of allowed values for a property if the set is a finite set of strings; otherwise, it returns null.

`model.component(<ctag>).mesh(<tag>).feature().move(<ftag>,<position>)` moves the feature *<ftag>* to the zero indexed position *<position>* in the sequence.

`model.component(<ctag>).mesh(<tag>).feature().remove(<ftag>)` removes the feature *<ftag>*.

`model.component(<ctag>).mesh(<tag>).clearMesh()` removes all features from the sequence and clears the mesh.

Building Features

After each build operation, the current feature is set as the last of features that were built. The mesh is updated to be the mesh generated by these features.

`model.component(<ctag>).mesh(<tag>).run(<ftag>)` builds all features up to (and including) the feature *<ftag>*.

`model.component(<ctag>).mesh(<tag>).run()` builds all features.

`model.component(<ctag>).mesh().run()` builds all meshing sequences.

`model.component(<ctag>).mesh(<tag>).runForStudy(<studyStep>)` builds the physics-controlled mesh with the settings from the specified study. If the mesh is not physics-controlled, it builds all mesh features.

Getting Build Status

`String fTag = model.component(<ctag>).mesh(<tag>).current()` returns the tag of the current feature. If the current state is before the first feature, the empty string `" "` is returned.

`String status = model.component(<ctag>).mesh(<tag>).feature(<ftag>).status()` returns the status of the feature *<ftag>*. The status is `built`, `warning`, `needs_rebuild`, `edited`, or `error`.

`String msg = model.component(<ctag>).mesh(<tag>).feature(<ftag>).message()` returns the warning/error message of the feature *<ftag>*.

Getting Tags of Selections

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).componentSelection()` returns tags of selection output objects for use in selections in the component *<ctag>*. It returns an empty string array if there are no defined selections.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).outputSelection()` returns the tags of selections output objects for use in selections within the meshing sequence. It returns an empty string array if there are no defined selections.

Getting and Setting Mesh Data

`model.component(<ctag>).mesh(<tag>).meshGetMethod` gets mesh data from the mesh `<tag>`. The available methods are described in [Accessing Mesh Data](#).

`model.component(<ctag>).mesh(<tag>).data().meshModificationMethods` are used to modify mesh data on a low level. You can access and modify individual elements. The available methods are described in [Accessing Mesh Data](#).

`model.component(<ctag>).mesh(<tag>).data().createMesh()` transfers the modified data into to the mesh `<tag>`. See also [Setting or Modifying Mesh Data](#).

Mesh Statistics

`model.component(<ctag>).mesh(<tag>).stat().selection().selMethod` can be used to select a number of geometric entities for which statistics is wanted. The available selection methods are described in [Selection Methods](#).

`model.component(<ctag>).geom(<tag>).stat().meshStatisticsMethod` returns mesh statistics about the selected geometric entities. The available methods are described in [Information and Statistics](#).

Exporting a Mesh to File

`model.component(<ctag>).mesh(<tag>).export(<filename>)` exports the mesh `<tag>` to an MPHBIN- or MPHTXT-file.

Plotting a Mesh

Use the `model.component(<ctag>).mesh(<tag>).image()` and `model.component(<ctag>).mesh(<tag>).feature().image()` methods to plotting and exporting mesh images. See [Plotting and Exporting Images](#).

Selection Methods

`selection.allGeom()` sets the selection to be the entire geometry (that is, all geometric entities).

`selection.remaining()` sets the selection to be the geometric entities that remains to be meshed when the feature is about to be built.

`selection.geom(<dim>).all()` sets all geometric entities in dimension `<dim>`.

`selection.geom(<dim>).set(<entities>)` sets the selection to be the geometric entities specified in the integer array `<entities>` in dimension `<dim>`.

`selection.geom(<dim>).add(<entities>)` adds the geometric entities specified in the integer array `<entities>` in dimension `<dim>` to the selection.

`selection.geom(<dim>).remove(<entities>)` removes the geometric entities specified in the integer array `<entities>` in dimension `<dim>` from the selection.

`selection.geom(<dim>).clear()` clears the selection.

To access the selections use:

`int[] dims = selection.dimension()` returns the geometric entity level in `dims[0]` for the entities in the selection. If `dims` is empty the selection defines the entire geometry.

`selection.isRemaining()` returns true if the selection specifies the remaining entities, otherwise false.

`selection.dom(<dim>)` returns the geometric entities in dimension `<dim>` for the selection.

Getting the Geometry Tag

`model.mesh(<tag>).geom()` returns the geometry tag. This can be useful when working with several geometries in the same model.

Measuring the Geometric Entities

`model.component(<tag>).measure().measurementMethod` returns the volume, area, length, vertex coordinates, average coordinates, or distance between two vertices according to the entity selection. For a list of the available measurement methods to use, see [Measurement Methods](#). For information about how to select entities, see [Selections](#).

EXAMPLE

Create a 2D geometry by the union of a circle and square. Build a triangle mesh with $h_{\max} = 0.1$ in domains 1 and 3, and $h_{\max} = 0.01$ in domain 2.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("c1", "Circle");
g.create("sq1", "Square");
g.create("uni1", "Union");
g.feature("uni1").selection("input").set(new String[]{"c1", "sq1"});
m.create("size1", "Size");
m.feature("size1").selection().geom(2).set(new int[]{1, 3});
m.feature("size1").set("hmax", "0.1");
m.create("size2", "Size");
m.feature("size2").selection().geom(2).set(new int[]{2});
m.feature("size2").set("hmax", "0.025");
m.create("ftri1", "FreeTri");
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('c1', 'Circle');
g.create('sq1', 'Square');
g.create('uni1', 'Union');
g.feature('uni1').selection('input').set({'c1', 'sq1'});
m.create('size1', 'Size');
m.feature('size1').selection.geom(2).set([1, 3]);
m.feature('size1').set('hmax', '0.1');
m.create('size2', 'Size');
m.feature('size2').selection.geom(2).set(2);
m.feature('size2').set('hmax', '0.025');
m.create('ftri1', 'FreeTri');
m.run;
```

SEE ALSO

[model.geom\(\)](#)

model.methodCall()

Calling model methods. You can create and run model methods to extend the functionality of a COMSOL Multiphysics simulation using custom methods. See the *Application Programming Guide* for more information about creating and using methods.



The Model Java-file history for running a method call in the COMSOL Desktop contains the history produced while running the method call and not the method itself.

SYNTAX

```
model.methodCall().create(<tag>,<methodname>);
model.methodCall(<tag>).run();
model.methodCall(<tag>).inputNames();
model.methodCall(<tag>).methodName();
model.methodCall(<tag>).methods();
model.methodCall(<tag>).set(<param>,<expr>);
```

DESCRIPTION

`model.methodCall(<tag>)` represents a method call for a model method.

`model.methodCall().create(<tag>,<methodname>)` creates a method call for the model method `<methodname>` with the given tag.

`model.methodCall(<tag>).inputNames();` returns the names of input parameters of a method reference by a given method call.

`model.methodCall(<tag>).methodName` returns the name of the method that the method call runs.

`model.methodCall(<tag>).methods();` returns the names of all methods in the model.

`model.methodCall(<tag>).run()` runs the method call with the tag `<tag>`.

The following examples show how you can specify the names of an input parameter, the value of a 1D double array parameter, and the value of a boolean parameter:

```
model.methodCall(<tag>).set("paramname","p1");
model.methodCall(<tag>).set("paramvalues",new double[] {2.3 3.7 5.6 7.1 11.3 17.5});
model.methodCall(<tag>).set("sendmail",true);
```

Parameter names are created automatically with the appropriate types according to the inputs that have been specified in the method. This means that you can then use a natural set syntax as shown in the examples above.

ARGUMENTS TO METHOD CALLS

The following table describes the names of arguments that you can use to change the value of inputs to a method call before running it:

TABLE 2-109: ARGUMENT NAMES FOR METHOD CALLS.

ARGUMENT NAME	SAMPLE VALUES	DESCRIPTION
-methodinputnames	<i>size, times</i>	Comma-separated list of names of inputs (as strings) to change for the method call to run.
-methodinputvalues	0.8,{0.1,0.3,0.8}	Comma-separated list of the corresponding values. Arrays and 2D arrays are entered using curly braces.
-methodinputfile	Path to file	A file to read the inputs to the method call from. This file has the same format as the one used together with -appargsfile (see the Application Builder documentation about application arguments),

Specifying method call inputs both from a file and individually on the command line can be combined. If an input is given both in a file and on the command line, the value given on the command line overrides the value given in the file. If a value is given several times in the file or on the command line, the last given value overrides any previous given value.

	To use units in values passed as arguments to <code>-methodinputvalues</code> , enclose them in single quotation marks. For example, <code>" ' 500[m] ' , ' 45[s] ' "</code>
---	--

USING METHOD CALLS FROM A MODEL JAVA-FILE

You can use the model method in a method call from a Model Java-file if the file starts by loading an MPH file (including its model method) and then calls a run of a method call. However, in this context, model methods do not support user interface commands or file schema. The table below table lists user interface commands that are not supported:

TABLE 2-110: UNSUPPORTED USER INTERFACE COMMANDS IN MODEL JAVA-FILES.

COMMAND	RESULT
Alert	Command is ignored
Confirm	Command causes an exception
DebugLog	Command is ignored
FileOpen	Command causes an exception
FileSaveAs	Command causes an exception
ImportFile	Command causes an exception
Message	Command is ignored
OpenURL	Command is ignored
Request	Command causes an exception
SelectNode	Command is ignored
SetProgress	Command is ignored
SetProgressInterval	Command is ignored
ZoomExtents	Command is ignored

SEE ALSO

[model.form\(\)](#)

model.modelNode()

Deprecated alias for [model.component\(\)](#) for accessing model component nodes.

	This syntax is still used when you turn off the component syntax (clear the Generate code using component syntax checkbox on the Application Builder>Methods page in the Preferences window). Otherwise, the code that COMSOL Multiphysics creates uses <code>model.component().create(<tag>)</code> instead of <code>model.modelNode().create(<tag>)</code> , and so on. See model.component() .
---	---

model.multiphysics()

Add multiphysics features to this feature container.

SYNTAX

```
model.multiphysics().create(<tag>,...);  
model.multiphysics().image()
```

DESCRIPTION

`model.multiphysics().create(<tag>,<coupling>,<geom>,<sdim>)`; adds a multiphysics coupling with the tag and the name to a geometry and a space dimension. Use -1 as the space dimension to indicate a coupling valid in the entire model.

EXAMPLE

In a model with a Solid Mechanics interface and a Heat Transfer in Solids interface, add a Thermal Expansion multiphysics coupling on the domain level and a Temperature Coupling on a model-wide level in a geometry `geom1`:

Code for Use with Java

```
Model model = ModelUtil.create("Model");  
model.component().create("comp1");  
model.component("comp1").geom.create("geom1", 3);  
model.multiphysics().create("te1", "ThermalExpansion", "geom1", 3);  
model.multiphysics().create("tc1", "TemperatureCoupling", "geom1", -1);
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');  
model.component.create('comp1');  
model.component('comp1').geom.create('geom1', 3);  
model.multiphysics.create('te1', 'ThermalExpansion', 'geom1', 3);  
model.multiphysics.create('tc1', 'TemperatureCoupling', 'geom1', -1);
```

model.nodeGroup()

Node groups.

You can create node groups to structure the nodes in the model tree. It can be useful in this context when editing a Model Java-file created in the COMSOL Desktop and then opens it in the COMSOL Desktop again. It can also be useful in model methods.

SYNTAX

```
model.nodeGroup().create(<tag>,<type>);  
model.nodeGroup().create(<tag>,<type>,<context>);  
  
model.nodeGroup(<tag>).add(<nodetag>);  
model.nodeGroup(<tag>).add(<parenttag>,<nodetag>);  
  
model.nodeGroup().ungroup(<tag>);
```

There is also a component list `model.component("comp1").nodeGroup()` with the groups belonging to a component.

DESCRIPTION

`model.nodeGroup(<tag>)` represents a node group in the model tree.

`model.nodeGroup().create(<tag>,<type>)` creates a node group of the specified type. For example,

```
model.nodeGroup().create("grp1", "GlobalDefinitions");
```

creates a node group with the tag "grp1" under the **Global Definitions** node in the model tree.

`model.nodeGroup().create(<tag>,<type>,<context>)` creates a group of the specified type in the specified context. For example,

```
model.nodeGroup().create("g", "Geometry", "geom1")
```

creates a group in a geometry sequence.

Use `model.nodeGroup().ungroup(<tag>)` to ungroup (remove) a group. Removing the group does not remove its members from the model.

For a node group, the following methods are available:

- Use `nodeGroup.add(<type>,<tag>)` to add a node with the tag <tag> of the type <type> to the group. For example `group.add("func", "an1")` adds `model.func("an1")` to the group.
- Use `nodeGroup.remove(<type>,<tag>)` to remove a node with the tag <tag> of the type <type> from the group.

EXAMPLE

The following example creates a node group under Definitions in a Component, adds two Model Input features to it, and then removes it by the ungroup method:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1", true);
model.component("comp1").geom().create("geom1", 3);
model.component("comp1").common().create("minpt1", "CommonInputDef");
model.component("comp1").common().create("minpt2", "CommonInputDef");
model.nodeGroup().create("grp1", "Definitions", "comp1");
model.nodeGroup("grp1").set("type", "commondef");
model.nodeGroup("grp1").add("common", "minpt1");
model.nodeGroup("grp1").add("common", "minpt2");
model.nodeGroup().ungroup("grp1");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1',true);
model.component('comp1').geom.create('geom1',3);
model.component('comp1').common.create('minpt1','CommonInputDef');
model.component('comp1').common().create('minpt2','CommonInputDef');
model.nodeGroup.create('grp1','Definitions','comp1');
model.nodeGroup('grp1').set('type','commondef');
model.nodeGroup('grp1').add('common','minpt1');
model.nodeGroup('grp1').add('common','minpt2');
model.nodeGroup.ungroup('grp1');
```

model.ode()

Create global equations (ODEs and DAEs).

SYNTAX

```
model.ode().create(<tag>);  
model.ode(<tag>).state(<statelist>);  
model.ode(<tag>).state(<pos>,<state>);  
model.ode(<tag>).ode(<state>,<equation>);  
model.ode(<tag>).descr(<state>,<descr>);  
model.ode(<tag>).weak(<wlist>);  
model.ode(<tag>).weak(<pos>,<wexpr>);  
model.ode(<tag>).discrete(<boolean>);  
model.ode(<tag>).valueType(prop);
```

```
model.ode(<tag>).state();  
model.ode(<tag>).ode(<state>);  
model.ode(<tag>).descr(<state>);  
model.ode(<tag>).weak();  
model.ode(<tag>).discrete();  
model.ode(<tag>).valueType();
```

DESCRIPTION

`model.ode(<tag>)` returns a global equation (or an ODE or a DAE) with tag `<tag>`.

`model.ode().create(<tag>)` creates a global equation with tag `<tag>`.

`model.ode(<tag>).create(<tag>,<fname>)` creates an ODE field with tag `<tag>` with the field name `<fname>`.

`model.ode(<tag>).state(<statelist>)` sets the states of the global equation tagged `<tag>` according to the list `<statelist>`.

`model.ode(<tag>).state(<pos>,<state>)` edits the state at position `<pos>` in the state vector for the global equation `<tag>`.

`model.ode(<tag>).ode(<state>,<equation>)` sets the equation for the state `<state>`. If the state variable has not previously been added using `model.ode(<tag>).state(<statelist>)` then `<state>` is appended to the list of state variables.

`model.ode(<tag>).descr(<state>,<descr>)` sets the description for the state `<state>`.

`model.ode(<tag>).weak(<wlist>)` set weak equations. `<wlist>` is a list of weak expressions.

`model.ode(<tag>).weak(<pos>,<wexpr>)` sets the weak expression at position `<pos>` in the list of weak expressions.

`model.ode(<tag>).field(<fname>)` sets the ODE field name.

`model.ode(<tag>).field()` returns the ODE field name as a string.

`model.ode(<tag>).state()` returns the state variables as a string array.

`model.ode(<tag>).ode(<state>)` returns the global equation for the state variable `<state>` as a string.

`model.ode(<tag>).descr(<state>)` returns the description of the state variable `<state>` as a string.

`model.ode(<tag>).weak()` returns the weak equations as a string vector of weak expressions.

`model.ode(<tag>).discrete(true)` specifies that the global equation contains event states.

`model.ode(<tag>).valueType(prop)` specifies value type as real or complex when splitting of complex variables in real and imaginary parts has been turned on.

`model.ode(<tag>).valueType()` returns the value type.

EXAMPLE

Define a global equations with the variables u and w , the ODEs $u_t + 1 = 0$ and $v_t + 1 = 0$, where the subscript t indicates the derivative with respect to time. Also define a weak expression $\text{test}(u) \cdot v$.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.ode().create("ode1");
model.ode("ode1").ode("u", "ut+1");
model.ode("ode1").ode("v", "vt-1");
model.ode("ode1").weak(new String[] {"test(u)*v"});
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.ode.create('ode1');
model.ode('ode1').ode('u', 'ut+1');
model.ode('ode1').ode('v', 'vt-1');
model.ode('ode1').weak({'test(u)*v'});
```

SEE ALSO

[model.init\(\)](#), [model.solverEvent\(\)](#)

model.opt()

Optimization interface.

SYNTAX

```
model.opt().objective().create(<tag>, type)
model.opt().objective(<tag>).set(property, <value>)
```

```
model.opt().constr().create(<tag>)
model.opt().constr(<tag>).etc
```

```
model.opt().gconstr().create(<tag>)
model.opt().gconstr(<tag>).constr(<constrExpr>)
model.opt().gconstr(<tag>).lbond(<lbondExpr>)
model.opt().gconstr(<tag>).ubound(<uboundExpr>)
```

DESCRIPTION

The purpose of `model.opt` is to manage information relating to optimization and sensitivity analysis. Most of the fields under `model.opt` are read and interpreted directly by the optimization and sensitivity solvers. They never affect the result of other solvers.

`model.opt().objective().create(<tag>, type)` adds an objective function of the specified type. The supported types are `Global` and `LeastSquares`.

`model.opt().objective(<tag>).set(property, <value>)` sets an objective function property. Objective functions of type `Global` support the single property `expr`, which takes a globally defined expression as value. Allowed properties for objectives of type `LeastSquares` are described below.

`model.opt().objective(<tag>).selection().named(<seltag>)` assigns the objective function to the named selection `<seltag>`.

`model.opt().objective(<tag>).selection().set(...)` defines a local selection that assigns the objective function to geometric entities. For a complete list of methods available under `selection()`, see [model.selection\(\)](#). Only objective functions of type `LeastSquares` require a selection. See further below.

`model.opt().constr().create(<tag>)` adds a pointwise (mesh-based) constraint on the control variables. The syntax is shared with [model.constr\(\)](#) with the exception that the `ctype` parameter expects values `constr`, `lbond`, and `ubound` for constraint, lower bound, and upper bound, respectively.

`model.opt().gconstr().create(<tag>)` registers a global constraint with the optimization solvers. Such constraints consist of a globally available expression, which can depend both on optimization variables and on the forward PDE solution, together with likewise global expressions for lower and upper bound.

`model.opt().gconstr(<tag>).constr(<constrExpr>)` specifies a global constraint expression.

`model.opt().gconstr(<tag>).lbound(<lboundExpr>)` sets lower bound for the constraint.

`model.opt().gconstr(<tag>).ubound(<lboundExpr>)` sets upper bound for the constraint.

Least-Squares Objective Functions

Least-squares objective functions are specified in terms of measured values, stored on file, together with information about how corresponding expressions can be evaluated for the current control variable values. An overview of the allowed properties is given in the table below.

TABLE 2-111: PROPERTIES FOR OBJECTIVE FUNCTION TYPE LEASTSQUARES.

PROPERTY	VALUE	DESCRIPTION
filename	String	Full path of the measurement data file.
paramnames	String array	Parameters used in the experiment.
paramexprs	String array	Values of the given parameters.
columntypes	String array	List of column type indicators.
columnexprsweights	String array	Column contribution weights.
columnexprs	String array	Measurement expressions.

In principle, you must specify the following for each measured value:

- To which experiment the value belongs and parameters for that experiment
- Which expression to evaluate
- Where the expression must be evaluated
- For which time or parameter value the evaluation must be performed

Each *experiment* corresponds to a solution of the forward problem with a given set of parameter values. In practice, measurements for each experiment must be stored in a separate file, and specified as a separate `LeastSquares` objective feature where you give the full path of the measurement data file in the `filename` property. Parameters specified in the `paramnames` property are given the values specified using `paramexprs` property during the forward solution. One forward solution is performed for each unique set of parameter names and values.

The required measurement data file format is row- and column-oriented. Entries on each row must be separated by commas or semicolons, while rows are separated by line feeds. Use the `columntypes` property to specify the content of each column, in the order that they appear in the data file, according to the following table:

TABLE 2-112: ALLOWED COLUMN TYPES.

TYPE	COLUMN CONTENTS
time	Actual measurement times
param	Actual parameter values
coord	Actual measurement coordinates
value	Measured values
none	Ignored column

Columns of type `time` are only allowed for transient problems. The measurements on the same row are assumed to be made at the specified time. Forward model values are interpolated to the given times. There must only be one column of type `time`, and it requires no further parameters.

Columns of type `param` contain parameter values for which the measurements on the same row have been made, and for which the forward problem must be solved. A data file can contain multiple parameter columns. Corresponding parameter names must be given in the `columnnames` property.

Columns of type `coord` contain global coordinates where the measurements on the same row have been made. The coordinate columns must be coupled to a coordinate variable by specifying the coordinate variable name in the `columnnames` property for the given column and the frame tag `spatial`, `material`, `mesh`, or `geometry` in the `columnexprs` property. For example, in a 3D model, you need three columns of type `coord` with `columnnames` entries `x`, `y`, and `z`, respectively.

A `value` column contains measured data. For each `value` column, a corresponding expression to be evaluated must be specified in the `columnexprs` property. Entries in `value` columns are interpreted as real numbers when possible. Anything else, including for example hash marks (`#`) and the literal strings `nan`, `Nan`, `NaN`, and `NAN` is interpreted as an illegal value which is excluded from the least squares objective function evaluation. A weight for the objective contribution from a column, multiplying the squared difference between the measured value and the expression, can be specified as a positive globally expression that can be evaluated using the `columnexprsweights` property. To exclude a measurement from a comma-separated file, you can also simply leave a `value` column empty.

Columns of type `none` can be used to exclude columns from the data file.

Coordinates are interpreted as global in the context of the objective feature's selection. This means that the `value` column expressions are evaluated at the points within the selection that best match the given coordinates. If the interpolation fails for some point because its coordinates lie too far outside the selection, the corresponding value is ignored.

model.pair()

Create and define identity pairs, contact pairs, general contact pairs, and sector symmetry pairs for assemble geometries.

SYNTAX

```
model.component(<ctag>).pair().create(<tag>,type,<gtag>);
model.component(<ctag>).pair(<tag>).type(type);
model.component(<ctag>).pair(<tag>).type();
model.component(<ctag>).pair(<tag>).pairName(<pname>);
model.component(<ctag>).pair(<tag>).pairName();
model.component(<ctag>).pair(<tag>).source().selMethod;
model.component(<ctag>).pair(<tag>).source().named(<seltag>);
model.component(<ctag>).pair(<tag>).source().named();
model.component(<ctag>).pair(<tag>).destination().selMethod;
model.component(<ctag>).pair(<tag>).destination().named(<seltag>);
model.component(<ctag>).pair(<tag>).destination().named();
model.component(<ctag>).pair(<tag>).swap();
model.component(<ctag>).pair(<tag>).manualFrame(manual);
model.component(<ctag>).pair(<tag>).manualFrame();
model.component(<ctag>).pair(<tag>).srcFrame(<frame>);
model.component(<ctag>).pair(<tag>).srcFrame();
model.component(<ctag>).pair(<tag>).dstFrame(<frame>);
model.component(<ctag>).pair(<tag>).dstFrame();
model.component(<ctag>).pair(<tag>).hasAutoSelection();
model.component(<ctag>).pair(<tag>).manualSelection(manual);
model.component(<ctag>).pair(<tag>).manualSelection();
model.component(<ctag>).pair(<tag>).searchMethod(method);
model.component(<ctag>).pair(<tag>).searchMethod();
model.component(<ctag>).pair(<tag>).manualDist(manual);
model.component(<ctag>).pair(<tag>).manualDist();
model.component(<ctag>).pair(<tag>).searchDist(<dist>);
model.component(<ctag>).pair(<tag>).searchDist();
model.component(<ctag>).pair(<tag>).opName(src2dst);
model.component(<ctag>).pair(<tag>).mphOpName(src2dst);
model.component(<ctag>).pair(<tag>).gapName(src2dst);
model.component(<ctag>).pair(<tag>).extTol(tol);
model.component(<ctag>).pair(<tag>).manualSectorSymmetry(boolean);
model.component(<ctag>).pair(<tag>).nSector(integer);
model.component(<ctag>).pair(<tag>).basePoint(string[]);
model.component(<ctag>).pair(<tag>).axis(string[]);
model.component(<ctag>).pair(<tag>).active(boolean);
model.component(<ctag>).pair(<tag>).isActive();
model.component(<ctag>).pair().remove(<tag>);
```

Not all of the syntaxes above are available for all pair types. See the following section for more information.

DESCRIPTION

`model.component(<ctag>).pair().create(<tag>,type,<gtag>)` creates a pair with tag `<tag>` in the geometry with tag `<gtag>` in the component with tag `<ctag>`. The type `type` is `Contact`, `GeneralContact`, `Identity`, or `SectorSymmetry`. `model.component(<ctag>).pair(<tag>).type()` returns the pair type as a string. `model.component(<ctag>).pair(<tag>).type(type)` changes the pair type.

`model.component(<ctag>).pair(<tag>).pairName(<pname>)` sets the pair name, which is used as a suffix in operator names and variable names. By default, the pair name is the same as the tag.

`model.component(<ctag>).pair(<tag>).pairName()` returns the pair name.

`model.component(<ctag>).pair(<tag>).source().named(<seltag>)` assigns the source boundaries to the named selection `<seltag>`.

`model.component(<ctag>).pair(<tag>).source().set(...)` defines a local selection that assigns the source boundaries to geometric entities. For a complete list of methods available under `model.component(<ctag>).pair(<tag>).source()`, see [model.selection\(\)](#).

`model.component(<ctag>).pair(<tag>).destination().named(<seltag>)` assigns the destination boundaries to the named selection `<seltag>`.

`model.component(<ctag>).pair(<tag>).destination().set(...)` defines a local selection that assigns the destination boundaries to geometric entities. For a complete list of methods available under `model.component(<ctag>).pair(<tag>).destination()`, see [model.selection\(\)](#).

`model.component(<ctag>).pair(<tag>).swap()` swaps the source and destination selections.

`model.component(<ctag>).pair(<tag>).srcFrame(<frame>)` and `model.pair(<tag>).dstFrame(<frame>)` sets the source and destination frames for the identity mapping; the `srcFrame` property is also available for the frame setting for sector symmetry pairs. Manual control of frames is automatically activated when these methods are used. The argument `<frame>` can have the values `geometry`, `material`, `mesh` or `spatial`. The default is `spatial`. These frames are only used for identity pairs.

`model.component(<ctag>).pair(<tag>).srcFrame()` and `model.component(<ctag>).pair(<tag>).dstFrame()` returns the frame tags.

`model.component(<ctag>).pair(<tag>).hasAutoSelection()` returns true if the contact pair was created automatically, using the create pairs checkbox in the finalize geometry node.

`model.component(<ctag>).pair(<tag>).manualFrame(manual)` enables or disables manual control of the frames for source and destination of an identity pair. The argument *manual* is a boolean.

`model.component(<ctag>).pair(<tag>).manualFrame()` returns true if manual control of frames is enabled, and false otherwise. Note that `model.component(<ctag>).pair(<tag>).srcFrame(<frame>)` and `model.component(<ctag>).pair(<tag>).dstFrame(<frame>)` enables manual control of frames.

`model.component(<ctag>).pair(<tag>).manualSelection(manual)` enables or disables manual control of the selections for a pair that was created automatically.

`model.component(<ctag>).pair(<tag>).manualSelection()` returns true if manual control of selections is enabled, and false otherwise.

`model.component(<ctag>).pair(<tag>).searchMethod(method)` sets the search method for a contact pair. The argument *method* can be `hierarchical`, `fast`, or `direct`. The default is `hierarchical` in 2D and 3D and `fast` in 1D, where `hierarchical` is not available.

`model.component(<ctag>).pair(<tag>).searchMethod()` returns the search method.

`model.component(<ctag>).pair(<tag>).manualDist(manual)` enables or disables manual control of the search distance for a pair. The argument *manual* is a boolean. The default value `false` means that the search distance is determined automatically. `model.component(<ctag>).pair(<tag>).manualDist()` returns true if manual control of search distance is enabled and false otherwise.

`model.component(<ctag>).pair(<tag>).searchDist(<dist>)` sets the search distance for a contact or identity pair when manual control of the search distance is enabled. The argument `<dist>` is a string whose default unit is the geometry's length unit. The default is `1e-2`.

`model.component(<ctag>).pair(<tag>).searchDist()` returns the search distance as a string.

`model.component(<ctag>).pair(<tag>).opName(src2dst)` returns the name of the operator transferring an expression from source to destination (if *src2dst*=true) or from destination to source (if *src2dst*=false).

`model.component(<ctag>).pair(<tag>).mphOpName(src2dst)` returns the name of the multiphysics operator transferring an expression from source to destination (if *src2dst*=true) or from destination to source (if *src2dst*=false). When the test operator is applied on this operator, it does not give any contribution (reaction force) for the structural mechanics interfaces' degrees of freedom due to the variable point mapping. These operators are available only for contact pairs.

`model.component(<ctag>).pair(<tag>).gapName(src2dst)` returns the name of the geometric gap variable seen from the destination (if *src2dst*=true) or seen from the source (if *src2dst*=false). These variables are available only for contact pairs.

`model.component(<ctag>).pair(<tag>).elementwise(mapping)` controls the element mapping for compatible meshes. *mapping* can be `on`, `off`, or `require`, where you use the last option to require compatible meshes.

`model.component(<ctag>).pair(<tag>).compatibleMeshesTol(tol)` provides the ability to specify a user-defined relative tolerance *tol* for compatible meshes, if the `elementwise` property is set to `on` or `require`. The default tolerance is 0.001.

`model.component(<ctag>).pair(<tag>).contactSelectionType(type)` specifies if the contact selection should be `all`, for all selections, or `manual`, for determining selections manually for general contact pairs.

`model.component(<ctag>).pair(<tag>).excludeSource(boolean)` specifies if source boundaries should be excluded from selections from other pairs (if `true`) for general contact pairs.

`model.component(<ctag>).pair(<tag>).excludeDestination(boolean)` specifies if destination boundaries should be excluded from selections from other pairs (if `true`) for general contact pairs.

`model.component(<ctag>).pair(<tag>).mapping(type)` specifies if the mapping method for general contact pairs: `deformed` for a deformed configuration or `initial` for an initial configuration.

`model.component(<ctag>).pair(<tag>).contactSearchDist(type)` specifies the search distance for general contact pairs: `automatic`, `manual`, or `tolerance`. For all three types, an extrapolation tolerance value can be given as `model.component(<ctag>).pair(<tag>).searchTol(tol)`. For `manual`, you can also specify a search distance as `model.component(<ctag>).pair(<tag>).searchDist(dist)`. For `tolerance`, you can also specify a contact search tolerance as `model.component(<ctag>).pair(<tag>).contactSearchTol(tol)`;

`model.component(<ctag>).pair(<tag>).selfIntersectionType(type)` specifies the type of self-intersection check for general contact pairs: `automatic`, `exhaustive`, or `off`.

`model.component(<ctag>).pair(<tag>).manualSectorSymmetry(boolean)` specifies if the sector symmetry settings are set automatically (if `false`) or manually (if `true`) for sector symmetry pairs. If set to `true`, the following three properties are available:

`model.component(<ctag>).pair(<tag>).nSector(integer)` specifies the number of sector for sector symmetry pairs.

`model.component(<ctag>).pair(<tag>).basePoint(string[])` species the axis base point for the rotation for sector symmetry pairs. Give an *x*-coordinate in 1D, *x*- and *y*-coordinates in 2D, and *x*-, *y*-, and *z*-coordinates in 3D.

`model.component(<ctag>).pair(<tag>).axis(string[])` species the axis of rotation for sector symmetry pairs in 3D. Give the *x*-, *y*-, and *z*-coordinates.

`model.component(<ctag>).pair(<tag>).extTol(tol)` species the extrapolation tolerance for sector symmetry pairs.

`model.component(<ctag>).pair(<tag>).active(boolean)` enables or disables the pair.

`boolean enabled = model.component(<ctag>).pair(<tag>).isActive()` returns `true` if the pair is enabled, and `false` otherwise.

`model.component(<ctag>).pair().remove(<tag>)` deletes the pair.

EXAMPLE

Create a contact pair in the geometry `geom1` with source boundaries 4 and 6 and destination boundaries 10 and 12.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
```

```

GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("blk1", "Block");
g.create("blk2", "Block");
g.feature("blk2").set("pos", new String[]{"0.5", "0.5", "1"});
g.feature("fin").name("Form Assembly");
g.feature("fin").set("action", "assembly");
g.feature("fin").set("imprint", true);
g.feature("fin").set("createpairs", false);
g.run();

model.component("comp1").pair().create("p1", "Contact", "geom1");
model.component("comp1").pair("p1").source().set(new int[]{4, 6});
model.component("comp1").pair("p1").destination().set(new int[]{10, 12});

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('blk1', 'Block');
g.create('blk2', 'Block');
g.feature('blk2').set('pos', {'0.5', '0.5', '1'});
g.feature('fin').name('Form Assembly');
g.feature('fin').set('action', 'assembly');
g.feature('fin').set('imprint', true);
g.feature('fin').set('createpairs', false);
g.run();

model.component('comp1').pair.create('p1', 'Contact', 'geom1');
model.component('comp1').pair('p1').source().set([4, 6]);
model.component('comp1').pair('p1').destination.set([10, 12]);

```

model.param() and model.result().param()

Add, define, and remove global parameters. For parameters in results, `model.result().param()` works in the same way as `model.param()`.

SYNTAX

```

model.param().set(<param>, <expr>);
model.param().set(<param>, <expr>, <descr>);
model.param().descr(<param>, <descr>);
model.param().remove(<param>);
model.param().clear();
model.param().rename(<oldpname>, <newpname>);

model.param().varnames();
model.param().get(<param>);
model.param().descr(<param>);
model.param().evaluate(<param>);
model.param().evaluateComplex(<param>);
model.param().evaluateUnit(<param>);

model.param().loadFile(<tempFile>, ...);
model.param().saveFile(<tempFile>, ...);

```

The following syntaxes are only applicable for global parameters and not for parameters in results:

```
model.param().create(<ptag>);
model.param().group().create(<ptag>);
model.param(<ptag>);
model.param().group(<ptag>);

model.param(<ptag>).paramCase().create(<pctag>);
model.param(<ptag>).create(<pctag>);
model.param(<ptag>).paramCase(<pctag>).set(<param>, <expr>);
model.param(<ptag>).setFromCase(<param>, <pctag>);
model.param(<ptag>).setFromCase(<params>, <pctag>);
model.param(<ptag>).setFromCase(<pctag>);

model.param(<ptag>).setShowInParamSel(true|false);
model.param(<ptag>).showInParamSel();

model.param().move(<param>, <ptag>);
model.param().group(<ptag>).move(<param>);
```

DESCRIPTION

`model.param()` is a collection of global model parameters. Likewise, `model.result().param()` is a collection of model parameters in results.

`model.param().create(<ptag>)` or `model.param().group().create(<ptag>)` creates a global model parameters node named `<ptag>`.

`model.param().group()` lists all global parameters nodes. The default parameters node belongs to a group with the fixed named `default`.

`model.param(<ptag>)` and `model.param().group(<ptag>)` both return the parameters node `<ptag>`.

The following methods apply equally for `model.param()`, `model.param(<ptag>)`, and `model.result().param()`:

`model.param().set(<param>, <expr>)` defines the parameter `<param>` as `<expr>`.

`model.param().set(<param>, <expr>, <descr>)` defines the parameter `<param>` as `<expr>` and assigns it the description `<descr>`.

`model.param().descr(<param>, <descr>)` sets the description for the parameter `<param>`.

`model.param().remove(<param>)` removes the parameter `<param>`. `model.param().clear()` removes all parameters.

`model.param().rename(<oldpname>, <newpname>)` renames the parameter `<oldpname>` to `<newpname>`.

`model.param().varnames()` returns the names of all parameters as a string array.

`model.param().get(<param>)` returns the parameter value as a string.

`model.param().descr(<param>)` returns the parameter description as a string.

`model.param().evaluate(<param>)` evaluates the value of the parameter `<param>` as a double real-valued floating-point value. For complex-valued parameters, use the `evaluateComplex` method instead.

`model.param().evaluateComplex(<param>)` evaluates the value of the parameter `<param>` as a double floating-point array with the real and imaginary part of a complex-valued parameter.

`model.param().evaluateUnit(<param>)` returns the unit of the parameter `<param>` if defined. It returns `null` if the parameter has no unit defined, or if the model does not use any unit system.

For `model.param().loadFile()` and `model.param().saveFile()`, see [The loadFile and saveFile Methods](#).

The following methods are available only for global model parameters nodes:

`model.param(<ptag>).paramCase().create(<pctag>)` or `model.param(<ptag>).create(<pctag>)` creates a parameter case for the parameters node `<ptag>`. You can create several parameter cases, where you can use the `.set(<param>, <expr>)` syntax to specify another expression for any existing parameter `<param>`. Then use `model.param(<ptag>).setFromCase(<param>, <pctag>)` to specify the parameter case `<pctag>` as the source for the value of the parameter `<param>`. You can set more than one parameter value in this way by using an array of parameters names as the first argument. Use `model.param(<ptag>).setFromCase(<pctag>)` to use the case `<pctag>` as the source for all parameters in the `<ptag>` node.

Use `model.param(<ptag>).setShowInParamSel(false)` to exclude the parameters in the parameters node in `<ptag>` in parameter selections. The default is that `setShowInParamSel` is `true`; that is, all the parameters are included in parameter selections. Use `model.param(<ptag>).showInParamSel()` to get the current state.

`model.param().move(<param>, <ptag>)` or `model.param(<ptag>).move(<param>)` moves the parameter `<param>` to the parameters node `<ptag>`.

EXAMPLE

Define the parameter `c` in terms of another parameter `a` and then remove `c`.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.param().set("c", "1+a");
model.param().remove("c");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.param.set('c', '1+a');
model.param.remove('c');
```

SEE ALSO

[model.variable\(\)](#)

model.physics()

Create and define properties for a physics interface.

SYNTAX

```
model.component(<ctag>).physics().create(<tag>,physint);
model.component(<ctag>).physics().create(<tag>,physint,<geomtag>);
model.component(<ctag>).physics().create(<tag>,physint,<geomtag>,<varnames>);
model.component(<ctag>).physics(<tag>).model(<mtag>);
model.component(<ctag>).physics(<tag>).field(fieldname).fieldname(<namelist>);
model.component(<ctag>).physics(<tag>).field(fieldname).fieldname(<pos>,<name>);
model.component(<ctag>).physics(<tag>).prop(propname).set(property,<value>);
model.component(<ctag>).physics(<tag>).create(<ftag>,feature);
model.component(<ctag>).physics(<tag>).create(<ftag>,feature,<dim>);
model.component(<ctag>).physics(<tag>).feature(<ftag>).create(<ftag2>,feature);
model.component(<ctag>).physics(<tag>).feature(<ftag>).create(<ftag2>,feature,<dim>);
model.component(<ctag>).physics(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).physics(<tag>).feature().move(<ftag>,<position>);

model.component(<ctag>).physics(<tag>).feature(<ftag>).feature();
model.component(<ctag>).physics(<tag>).feature(<ftag>).feature(<ftag2>);
model.component(<ctag>).physics(<tag>).feature(<ftag>).featureInfo();
model.component(<ctag>).physics(<tag>).feature(<ftag>).featureInfo("info");
feature = model.component(<ctag>).physics(<tag>).feature(<ftag>);
feature.featureInfo("info").set(variable,<value>);
feature.featureInfo("info").getInfoTable(id);

model.component(<ctag>).physics(<tag>).model();
model.component(<ctag>).physics(<tag>).field(fieldname).fieldname();
model.component(<ctag>).physics(<tag>).scope();
model.component(<ctag>).physics(<tag>).geom();
model.component(<ctag>).physics(<tag>).image();
model.component(<ctag>).physics(<tag>).prop(propname).getType(<pname>);
model.component(<ctag>).physics(<tag>).prop(propname).param();
model.component(<ctag>).physics(<tag>).feature(<ftag>).getType(<pname>);
model.component(<ctag>).physics(<tag>).feature(<ftag>).param();

model.component(<ctag>).physics(<tag>).feature(<tag>).set(String pname, int value);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    set(String pname, int pos, int value);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    set(String pname, int pos, int[] value);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    set(String pname, int pos1, int pos2, int value);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, String value, int index);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, String value, int firstIndex, int secondIndex);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, String[] value, int index);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, double value, int index);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, double value, int firstIndex, int secondIndex);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, double[] value, int index);
model.component(<ctag>).physics(<tag>).feature(<tag>).setIndex(String name, int value, int
    index);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, String value, int index);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, String value, int index);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, int value, int firstIndex, int secondIndex);
model.component(<ctag>).physics(<tag>).feature(<tag>).
    setIndex(String name, int[] value, int index);
model.component(<ctag>).physics(<tag>).feature(<tag>).importData();
model.component(<ctag>).physics(<tag>).feature(<tag>).discardData();
```

```

model.component(<ctag>).physics(<tag>).feature(<tag>).image();

model.component(<ctag>).physics(<tag>).prop(<tag>).set(String pname, int value);
model.component(<ctag>).physics(<tag>).prop(<tag>).set(String pname, int pos, int value);
model.component(<ctag>).physics(<tag>).prop(<tag>).set(String pname, int pos, int[] value);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    set(String pname, int pos1, int pos2, int value);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, String value, int index);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, String value, int firstIndex, int secondIndex);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, String[] value, int index);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, double value, int index);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, double value, int firstIndex, int secondIndex);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, double[] value, int index);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, int value, int index);
model.physics(<tag>).prop(<tag>).setIndex(String name, String value, int index);
model.physics(<tag>).prop(<tag>).setIndex(String name, String value, int index);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, int value, int firstIndex, int secondIndex);
model.component(<ctag>).physics(<tag>).prop(<tag>).
    setIndex(String name, int[] value, int index);

```

The `set()` methods index/position arguments are 1-based. The `setIndex()` methods index/position arguments are 0-based. See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.component(<ctag>).physics().create(<tag>, physint)` creates and returns a physics interface.

`model.component(<ctag>).physics().create(<tag>, physint)` or `model.component(<ctag>).physics().create(<tag>, physint, <geomtag>)` adds a physics interface to the model and initializes it with defaults. The *physint* argument specifies which physics interface to create. There can be several different values of *physint* which create the same internal physics interface class, but which set different defaults. The constructor without the <geomtag> argument can only be used (and should be used) by 0D (space-independent) interfaces.

`model.component(<ctag>).physics().create(<tag>, physint, <geomtag>, <varnames>)` adds an interface with the field variable names <varnames>. Only interfaces supporting a varying number of field variables considers this argument. Providing the variable names in the `create` method rather than changes them afterward using `model.component(<ctag>).physics(<tag>).field(fieldname).fieldname(<namelist>)` ensures that the default features are correct.

`model.component(<ctag>).physics(<tag>).field(fieldname).fieldname(<namelist>)` sets a name of a dependent variable. The entity *fieldname* (which could be, for example, `temperature`, `x-velocity`, `electric field`) specifies which dependent variable to set the name for. The available fields are provided by the physics interface. The argument <namelist> can be a list of names for physics interfaces supporting an arbitrary number of dependent variables. The physics interfaces provide default names for the dependent variables.

`model.component(<ctag>).physics(<tag>).field(fieldname).fieldname(<pos>, <name>)` changes the name at position <pos> in the list of field names.

`model.component(<ctag>).physics(<tag>).selection().named(<seltag>)` specifies that the physics interface is active on the named selection <seltag>.

`model.component(<ctag>).physics(<tag>).selection().set(...)` defines a local selection that makes the physics interface active on the selection's geometric entities. For a complete list of methods available under `selection()`, see [Selections](#). The selection must apply to the physics interface's maximum geometry level. The `create()` method makes the physics interface active in all domains. 0D interfaces are always active globally and do not support these methods.

`model.component(<ctag>).physics(<tag>).prop(propname).set(pname,<value>)` sets the value of some property parameter. All string types listed in [Table 2-4](#) are supported.

`model.component(<ctag>).physics(<tag>).create(<ftag>,feature)` adds a new feature instance to the physics interface and initializes the feature with defaults. The available features are given by the physics interface.

`model.component(<ctag>).physics(<tag>).create(<ftag>,feature,<dim>)` adds a new feature instance to the physics interface and initializes the feature with defaults. The feature is assigned to the domain level `<dim>`. Use this constructor for features which can be applied to more than one domain level. The constructor without the `<dim>` argument assigns the feature to the highest domain level, which the feature supports.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).set(pname,<value>)` sets a parameter value. All string types listed in the section [Table 2-4](#) are supported.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).selection().named(<seltag>)` assigns the physics feature to the named selection `<seltag>`.

`model.component(<ctag>).physics(<tag>).selection().set(...)` defines a local selection that assigns the physics feature to geometric entities. For a complete list of methods available under `selection()`, see [Selections](#). 0D features need no domain selection.

`model.component(<ctag>).physics(<tag>).feature().move(<ftag>,<position>)` moves the feature `<ftag>` to the zero indexed position `<position>` in the list. A feature cannot be moved before a default feature and the default features cannot be moved.

`model.component(<ctag>).physics(<tag>).create(<itag>,"init")` creates an initial value feature, using the reserved feature ID `init`.

`model.component(<ctag>).physics(<tag>).feature(<itag>).set(varname,<value>)` specifies an initial value. The variable names are the field variables. For wave problems, the time derivatives of the field variables are also included in the list of variables.

`model.component(<ctag>).physics(<tag>).model()` returns the model component node tag of the interface.

`model.component(<ctag>).physics(<tag>).field(fieldname).fieldname()` returns the field names as a string array.

`model.component(<ctag>).physics(<tag>).scope()` returns the fully qualified scope name.

`model.component(<ctag>).physics(<tag>).geom()` returns the geometry tag as a string.

`model.component(<ctag>).physics(<tag>).selection().named()` returns the selection tag as a string.

`model.component(<ctag>).physics(<tag>).selection().getType()` returns domain information. See [Selections](#) for available methods.

`model.component(<ctag>).physics(<tag>).prop(propname).getType(pname)` returns the parameter value. See [get* and Selection Access Methods](#) for available methods.

`model.component(<ctag>).physics(<tag>).prop(propname).param()` returns the parameter names as a string array.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).getType(<pname>)` returns the parameter value. See [get* and Selection Access Methods](#) for available methods.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).param()` returns the parameter names as a string array.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).selection().named()` returns the selection tag as a string array.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).feature()` returns the list of feature attributes. This list supports the same methods as `model.component(<ctag>).physics(<tag>).feature()`.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).feature(<ftag2>)` returns the feature attribute `<ftag2>`. The feature attributes support the same methods as `model.component(<ctag>).physics(<tag>).feature(<ftag>)`.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).featureInfo()` returns a list of info objects.

`model.component(<ctag>).feature(<ftag>).featureInfo("info")` returns the info object that contains information about the variables, weak expressions, and constraints that a feature generates. The `model.component(<ctag>).physics(<tag>)` and `model.coordSystem(<tag>)` objects also have this list that you access with `model.component(<ctag>).physics(<tag>).featureInfo("info")`. These objects do not support the `set` method, which only works for the object `model.component(<ctag>).physics(<tag>).feature(<ftag>)`.

`feature.featureInfo("info").set(variable, <value>)` locks the named variable to the given expression. The expression must be given as a string array.

`feature.featureInfo("info").getInfoTable(id)` returns a table that lists all information about a certain table id. The supported IDs are Expression, Shape, Weak, and Constraint.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).importData()` imports the file that the physics feature references into the model. This is only allowed for specific physics features that allow external files to be used, such as the Release from Data File feature for the particle tracing interfaces.

`model.component(<ctag>).physics(<tag>).feature(<ftag>).discardData()` Discards the external file imported by the `importData()` command. This only has an effect if `importData()` has been called previously for the physics feature. This is only allowed for specific physics features that allow external files to be used, such as the Release from Data File feature for the particle tracing interfaces.

EXAMPLE

This example creates an Electrostatics interface. It sets boundaries 3 and 8 to the ground potential and assigns the electric potential of 1 V at boundary 4.

When the physics interface is created a couple of default features are automatically added. One of them is the Charge Conservation feature, which has the tag `ccn1`. The relative permittivity of this feature is set to 1.

Code for Use with Java

```
model.component("comp1").physics().create("es", "Electrostatics", "geom1");
model.component("comp1").physics("es").create("gnd1", "Ground", 2);
model.component("comp1").physics("es").feature("gnd1").selection().set(new int[]{3, 8});
model.component("comp1").physics("es").create("pot1", "ElectricPotential", 2);
model.component("comp1").physics("es").feature("pot1").selection().set(new int[]{4});
model.component("comp1").physics("es").feature("pot1").set("V0", "1");
model.component("comp1").physics("es").feature("ccn1").set("epsilon_r_mat", "userdef");
model.component("comp1").physics("es").feature("ccn1").set("epsilon_r", "1");
```

Code for Use with MATLAB

```
model.component('comp1').physics.create('es', 'Electrostatics', 'geom1');
model.component('comp1').physics('es').create('gnd1', 'Ground', 2);
model.component('comp1').physics('es').feature('gnd1').selection().set([3, 8]);
model.component('comp1').physics('es').create('pot1', 'ElectricPotential', 2);
```

```

model.component('comp1').physics('es').feature('pot1').selection.set(4);
model.component('comp1').physics('es').feature('pot1').set('V0', '1');
model.component('comp1').physics('es').feature('ccn1').set('epsilon_r_mat', 'userdef');
model.component('comp1').physics('es').feature('ccn1').set('epsilon_r', '1');

```

COMPATIBILITY

From version 4.3 the methods

```

model.physics(<tag>).feature(<ftag>).params();
model.physics(<tag>).prop(propname).params();

```

are deprecated and replaced by the methods

```

model.physics(<tag>).feature(<ftag>).param();
model.physics(<tag>).prop(propname).param();

```

SEE ALSO

[model.material\(\)](#), [model.study\(\)](#)

model.probe()

Create and define properties for probes, which you can add to a model to monitor some quantity (real or complex-valued number) during a time-dependent, frequency-domain, or parametric simulation.

SYNTAX

```

model.probe().create(<tag>, type);
model.probe(<tag>).model(<mtag>);
model.probe(<tag>).set(property, <value>);
model.probe(<ptag>).create(<etag>, "PointExpr");
model.probe(<ptag>).feature(<etag>).set(eproperty, <evalue>);
model.probe(<tag>).genResult(String sol)
model.probe(<tag>).image()

```

DESCRIPTION

`model.probe().create(<tag>, type)` creates a probe of type *type* with tag *<tag>*. [Table 2-113](#) lists the supported probe types.

`model.probe(<tag>).model(<ctag>)` sets the probe's model component node to *<ctag>*.

`model.probe(<tag>).set(property, <value>)` sets *property* to *<value>*.

`model.probe(<ptag>).selection("dom")` gets the source selection for a boundary point probe.

`model.probe(<ptag>).create(<etag>, "PointExpr")` creates a point probe expression of tag *<etag>*. This is supported for probes of the types `DomainPoint` and `BoundaryPoint`.

`model.probe(<ptag>).feature(<etag>).set(eproperty, <evalue>)` sets the property *eproperty* on the point probe expression *<etag>* to the value *<evalue>*.

`model.probe(<tag>).genResult(String sol)` where *sol* is a solver sequence tag. This function prepares for using a probe while solving or during results processing. The command is invoked automatically when a solver or study is run from the COMSOL Desktop for all active probes but needs to be invoked explicitly when run through the API. The function `genResult(String sol)` sets up result features for evaluating the corresponding probe using the solver sequence *sol*. If `null` is used, the default solver sequence for a solution dataset is used. When *sol* is `null`, the corresponding probe solution dataset does not refer to any solver sequence. This means, for example, that the current model is used for selections used for this probe. When a solver sequence is run, then solution to use for the probes is always reset to use the current solver.

Use the `model.probe(<tag>).image()` methods for plotting and exporting probe plot images. See [Plotting and Exporting Images](#).

Boundary Probes, Domain Probes, Edge Probes, Point Probes, Global Variable Probes, and Point Probe Expressions

Probes can be of the following types:

TABLE 2-113: PROBE TYPES.

TYPE	DESCRIPTION
Boundary	Probe that defines a value as an integral, maximum, minimum, or average over boundaries.
Domain	Probe that defines a value as an integral, maximum, minimum, or average over domains.
Edge	Probe that defines a value as an integral, maximum, minimum, or average over edges (in 3D).
Point	Probe that defines a value as an integral, maximum, minimum, or average at points.
GlobalVariable	Probe that defines a value using a global variable.
PointExpr	Probe that defines a value by interpolation of an expression at a probe point. The probe point's coordinates are defined by the parent, which is either a <code>DomainPoint</code> or a <code>BoundaryPoint</code> probe.

Boundary Point Probes and Domain Point Probes

These are probes that can be defined at arbitrary points in the modeling domain or on its boundaries. There are two types:

TABLE 2-114: POINT PROBE TYPES.

TYPE	DESCRIPTION
BoundaryPoint	Defines a probe coordinate on a boundary in 3D.
DomainPoint	Defines a probe coordinate in a domain.

Probes of the types listed in [Table 2-113](#) take the following properties:

TABLE 2-115: PROBE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
descr	String		Description of the probe. Used for <code>model.result()</code> .
descriptive	on off	off	Manual control of description.
expr	String		The expression defining the probe.
frame	String	spatial frame	Frame used for defining the probe.
intorder	Integer	4	Integration order, when method is set to integration.
intsurface	boolean	false	Compute surface integral for 1D axisymmetric <code>DomainProbe</code> and 2D axisymmetric <code>BoundaryProbe</code> average and integral probe types.
intvolume	boolean	false	Compute volume integral for 2D axisymmetric <code>DomainProbe</code> average and integral probe types.
lagrange	String	2	Lagrange order when points is set to lagrange.
method	integration summation	integration	Method used when type is set to average or integral.
pintorder	String	4	Integration rule when points is set to integration.
points	node integration lagrange	node	Type of point when type is set to maximum or minimum.

TABLE 2-115: PROBE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
probename	String	probe tag	Probe variable name.
table	String	default	Table to use for probe evaluation.
type	average maximum minimum integral	average	Type of probe (Domain, Boundary, Edge and Point probes).
unit	String	unit of expr	Unit for the probe. Used for model.result().
window	String	default	The plot window to use for the probe.

A point probe of the type DomainPoint takes the following properties:

TABLE 2-116: PROBE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
bndsnap1	on off	off	Snap to nearest point (1D).
bndsnap2	on off	off	Snap to nearest boundary point (2D).
bndsnap3	on off	off	Snap to nearest boundary point (3D).
coords1	Matrix of doubles		Probe coordinates (1D).
coords2	Matrix of doubles		Probe coordinates (2D).
coords3	Matrix of doubles		Probe coordinates (3D).
depthpointnormal	double	0	Depth along line defined by the pointnormal method.
depthpointdirection	double	0	Depth along line defined by the pointdirection method.
depthtwopoints	double	0	Depth along line defined by the twopoints method.
dimension	1 2 3	3	The spatial dimension in which the point resides.
first	double array		The coordinates of the first point on the probe line.
frame	String	spatial frame	Frame used for defining the probe.
method	pointnormal pointdirection twopoints none	pointnormal	Line entry method.
second	double array		The coordinates of the second point (for method=twopoints)
twopointscurrent	first second	first	Point selector (for method=twopoints)

A point probe of BoundaryPoint type takes the following properties:

TABLE 2-117: PROBE PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
coords	double array	0,0,0	Probe coordinates.
frame	String	spatial frame	Frame used for defining the probe.
rawcoords	double array	0,0,0	Full-precision probe coordinates.
snapcoords	double array	0,0,0	The boundary-snapped coordinates.

In addition, boundary point probes support the selection property "dom", through which a Selection object containing the boundary on which the probe point resides can be retrieved.

EXAMPLES

Retrieve the entities for the boundary selection of a boundary point probe.

Code for Use with Java

```
int[] entities = model.probe("pbnd1").selection("dom").entities();
```

Code for Use with MATLAB

```
entities = model.probe('pbnd1').selection('dom').entities();
```

Here, the length of the array `entities` is either 0 or 1.

Retrieve the coordinates for a domain point probe in 2D.

Code for Use with Java

```
model.probe("pdom1").set("coords",new double[][]{{-0.17134, 0.0155763}});  
double[][] coords = model.component("comp1").probe("pdom1").getDoubleMatrix("coords");
```

Code for Use with MATLAB

```
model.probe('pdom1').set('coords',[0.8878505 0.0778816]);  
coords = model.component('comp1').probe('pdom1').getDoubleMatrix('coords');
```

model.reduced()

Reduced-order modeling.

There are reduced-order models of different types. The properties listed below are set and accessed using generic `set` and `get` methods described in [set\(\)](#) and [get*](#) and [Selection Access Methods](#). The following reduced-order model types are currently supported:

TABLE 2-118: REDUCED-ORDER MODEL TYPES.

TYPE	DESCRIPTION
ModalFrequency	Created by the Model Reduction study step for a parametric problem using the modal solver.
ModalTransient	Created by the Model Reduction study for a time-dependent problem using the modal solver.
PODFrequency	Created by the Model Reduction study step for a frequency-dependent problem using POD reduction.
PODTransient	Created by the Model Reduction study for a time-dependent problem using POD reduction.
AWE	Created by the Model Reduction study step for a parametric problem using the asymptotic waveform evaluation (AWE) solver.
RandomVibration	Uses a Frequency-Domain, Modal Reduced-Order Model to compute model response to random excitations.

SYNTAX

```
model.reduced()  
model.reduced().create(<tag>, <type>);  
  
model.reduced().getM(String)  
model.reduced().getN(String)  
model.reduced().getNnz(String)  
model.reduced().getOutputs()  
model.reduced().getParameters()  
model.reduced().getSparseMatrixCol(String)  
model.reduced().getSparseMatrixRow(String)  
model.reduced().getSparseMatrixVal(String)  
model.reduced().getSparseMatrixValImag(String)  
model.reduced().getVector(String)  
model.reduced().getVectorImag(String)  
model.reduced().isReal(String)
```

DESCRIPTION

`model.reduced().create(<tag>,<type>)` creates a reduced-model container of the specified type.

Depending on the type of reduced-order model, the following properties are available:

TABLE 2-119: PROPERTIES FOR REDUCED-ORDER MODEL TYPES.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
depvars	boolean	false	Controls if models with output variables should define degrees of freedom.	
depvarnames	String array	{}	Names of dependent variables.	
information	String	Empty	Model information.	√
logMessage	String	Empty	The Build log.	√
matrices	String matrix	{{}}	Names and descriptions of matrices.	√
metaCOMSOLVersion	String	Empty	String description of the COMSOL version when the model has been created.	√
metaCOMSOLVersionInt	String	Empty	Numerical representation of the COMSOL version when the model has been created.	√
outputDescr	String array	{}	Descriptions of output variables.	√
outputExprs	String array	{}	Values or expressions of output variables.	√
outputNames	String array	{}	Names of output variables.	√
reconstructors	String array	{}	Names of reconstruction variables.	√
reconstructorDescr	String array	{}	Descriptions of reconstruction variables.	√
vectors	String matrix	{{}}	Names and descriptions of vectors.	√

Note that only metaCOMSOLVersion and metaCOMSOLVersionInt are defined for the RandomVibration reduced-order model..

TABLE 2-120: PROPERTIES FOR THE MODALFREQUENCY REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
constrStateDescrs	String array	Constraint mode Y, where "Y" represents the index of the constraint mode among all constraint modes	Description of the Constraint mode.	Read only in the stateless case; editable in the stateful case.
constrStateExprs	String array	{"0"}	Constraint expression. This is the Model Control Input expression if one was found. Else this expression is 0 per default.	If the constraint variable is mapped to an input, then its expression is read only.
constrStateNames	String array	romX.constrStateY, where "Y" represents the index of the constraint mode among all constraint modes	Constraint DOF variable name (alias). This variable will evaluate to the ROM operator <code>rom1.state(x)</code> , with <code>x</code> being the internal state DOF number.	√
constrStateModesIndices	String array	{}	The actual mode index of constraint modes among all modes.	√

TABLE 2-120: PROPERTIES FOR THE MODALFREQUENCY REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
constrStateUsed	String array	{"off"}	Flag for enabling and disabling constraints for the constraint variables.	
controls	String array	{}	Values or expressions of control variables.	
controlDescr	String array	{}	Descriptions of control variables.	√
controlNames	String array	{}	Names of control variables.	√
dampingRatio	String	"0"	Damping ratio used by Uniform damping.	
dampingRatioPerMode	String array	{"0"}	Damping ratio for each eigenmode.	
form	Automatic Frequency	Automatic	Equation form (when interf is stateful): study controlled (Automatic) or frequency domain (Frequency).	
freq	String	freq	Value or expression for the frequency, when freq_src is userdef or interf is stateless.	
freq_src	fromSolver userdef	fromSolver	Frequency source: from the solver or a user-defined frequency, when form is Frequency.	
interf	stateful stateless	stateless	Use a stateless or stateful interface.	
loadfact	String	1	Load factor value or expression.	

TABLE 2-120: PROPERTIES FOR THE MODALFREQUENCY REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
matrices	String matrix	<pre>{ {"Kr", "Stiffness matrix"}, {"Kr_c", "Constrained part of stiffness matrix"}, {"Dr", "Damping matrix"}, {"Dr_c", "Constrained part of damping matrix"}, {"Dra", "Damping ratio matrix"}, {"Dra_c", "Constrained part of damping ratio matrix"}, {"Er", "Mass matrix"}, {"Er_c", "Constrained part of mass matrix"}, {"Br", "Input matrix"}, {"Cr", "Output matrix"}, {"F", "Input feedback matrix"}} </pre>	Names and descriptions of matrices.	√
modalfrequencies	String array		Frequency for each eigenmode.	√
modalDamping	String	noDamping	Damping of Modal reduced-order model.	
phase	String	phase	Phase value or expression, when interf is stateless. If given, the reconstructed solution will be multiplied by the factor $\exp(i \cdot \text{phase})$ prior to expression evaluation with the <code><rom>.eval(<expr>)</code> operator.	

TABLE 2-120: PROPERTIES FOR THE MODALFREQUENCY REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
solveElimSystem	boolean	false	Solve an eliminated system with constraints from constrain modes being removed, when interf is set to stateful.	
vectors	String matrix	{{"L", "Load vector"}, {"YO", "Output bias"}, {"EPartSol", "Mass matrix times particular solution"}, {"DPartSol", "Damping matrix times particular solution"}}, {"CImap", "Constraint modes to inputs map"}	Names and descriptions of vectors.	√

TABLE 2-121: PROPERTIES FOR THE MODALTRANSIENT REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
constrStateDescrs	String array	Constraint mode Y, where "Y" represents the index of the constraint mode among all constraint modes	Description of the Constraint mode.	Read only in the stateless case; editable in the stateful case.
constrStateExprs	String array	{"0"}	Constraint expression. This is the Model Control Input expression if one was found. Else this expression is 0 per default.	If the constraint variable is mapped to an input, then its expression is read only.
constrStateNames	String array	romX.constrStateY, where "Y" represents the index of the constraint mode among all constraint modes	Constraint DOF variable name (alias). This variable will evaluate to the ROM operator <code>rom1.state(x)</code> , with <code>x</code> being the internal state DOF number.	√
constrStateModesIndices	String array	{}	The actual mode index of constraint modes among all modes.	√
constrStateUsed	String array	{"off"}	Flag for enabling and disabling constraints for the constraint variables.	

TABLE 2-121: PROPERTIES FOR THE MODALTRANSIENT REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
controls	String array	{}	Values or expressions of control variables.	
controlDescr	String array	{}	Descriptions of control variables.	√
controlNames	String array	{}	Names of control variables.	√
dampingRatio	String	0	Damping ratio used by Uniform damping.	
dampingRatioPerMode	String array	{0}	Damping ratio for each eigenmode.	
form	Automatic Transient	Automatic	Equation form (when interf is stateful): study controlled (Automatic) or time domain (Transient).	
interf	stateful stateless	stateless	Use a stateless or stateful interface.	
loadfact	String	"1"	Load factor value or expression.	

TABLE 2-121: PROPERTIES FOR THE MODALTRANSIENT REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
matrices	String matrix	<pre>{{"Kr","Stiffness matrix"}, {"Kr_c", "Constrained part of stiffness matrix"}, {"Kr", "Stiffness matrix"}, {"Dr", "Damping matrix"}, {"Dr_c", "Constrained part of damping matrix"}, {"Dra", "Damping ratio matrix"}, {"Dra_c", "Constrained part of damping ratio matrix"}, {"Er", "Mass matrix"}, {"Er_c", "Constrained part of mass matrix"}, {"Br", "Input matrix"}, {"Cr", "Output matrix"}, {"F", "Input feedback matrix"}, {"BOr","Initial value input matrix"}, {"BOrdot", "Initial value time derivative input matrix"}, {"Brdot", "Time derivative input matrix"}, {"Brdotdot", "Second time derivative input matrix"}, {"Mc", "Mass matrix"}, {"Mc_c", "Constrained part of mass matrix"},</pre>	Names and descriptions of matrices.	√

TABLE 2-121: PROPERTIES FOR THE MODALTRANSIENT REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
matrices		{ "MA", "Stiffness matrix"}, { "MA_c", "Constrained part of stiffness matrix"}, { "MB", "Input matrix"}, { "D", "Input feedback matrix"}, { "C", "Output matrix"}, { "C_c", "Constrained part of output matrix"}}	Continued from the previous page.	√
modalfrequencies	String array		Frequency for each eigenmode.	√
modalDamping	String	noDamping	Damping of Modal reduced-order model.	
rtol	double	0.01	Relative tolerance.	
solveElimSystem	boolean	false	Solve an eliminated system with constraints from constrain modes being removed, when interf is set to stateful.	
time	String	t	Value or expression for the time, when interf is stateless.	
vectors	String matrix	{ {"L", "Load vector"}, {"Y0", "Output bias"}, {"U0", "Output bias"}, {"Udot0", "Initial derivative vector"}, {"Kud", "Stiffness matrix times ud"}, {"x0", "Initial value vector"}}, {"CImap", "Constraint modes to inputs map"}	Names and descriptions of vectors.	√

TABLE 2-122: PROPERTIES FOR THE AWE REDUCED-ORDER MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION	READ ONLY
freq	String	freq	Value or expression for the frequency.	
phase	String	phase	Phase value or expression. If given, the reconstructed solution will be multiplied by the factor $\exp(i \cdot \text{phase})$ prior to expression evaluation with the <code><rom>.eval(<expr>)</code> operator.	

TABLE 2-123: PROPERTIES FOR THE RANDOM VIBRATION MODEL.

NAME	VALUE	DEFAULT	DESCRIPTION
correlationMatrix	upper-triangle of a matrix	all 0s	The correlation matrix, when <code>correlationType</code> is <code>crossCorrelated</code> .
correlationType	uncorrelated fullyCorrelated crossCorrelated	uncorrelated	The correlation type.
fHigh	positive double	1000	Upper frequency limit as an integration bound.
fLow	positive double	1	Lower frequency limit as an integration bound.
frequencyResponseModel	name of frequency-domain, modal reduced-order model		The frequency response model to use. Specify its components using the <code>setIndex</code> method.
integrationMethod	automatic userDefined	automatic	Use an automatic or user-defined integration method.
integrationPoints	integer > 1	1001	Number of integration points, if <code>integrationMethod</code> is set to <code>userDefined</code> .
powerSpectralDensity	double scalar vector	all 0s	The power spectral density for each control name. Specify each power spectral density using the <code>setIndex</code> method.

For information about the `getM`, `getN`, `getNnz`, `isReal`, and the `getSparseMatrix` and `getVector` methods, see the matrix data tables [Table 6-6](#) and [Table 6-7](#) in the *Solvers and Study Steps* chapter.

COMPATIBILITY

The `constrStateMap` property has been removed in version 6.3.

The `parameters` property is deprecated in version 6.1 and has been replaced by the `freq` or `time` property. It may be removed in future versions, but until then, for backward compatibility, changing the `parameters` property automatically triggers a corresponding change of `freq` or `time` (but not the other way around).

The following methods are deprecated in version 5.5 and may be removed in future versions:

```

getOutputs()
getControls()
getParameters()
getValues(String[])
getVectors()
getMatrices()
getDescription(String[])
getString(String)
setControl(String, String)

```


The following methods are not supported:

```
create(<tag>)
create(<tag>, filepath)
setParameter(String, String)
renameControl(String, String)
renameOutput(String, String)
```

model.result()

Postprocessing and results interface.

SYNTAX

```
model.result();
model.result().create(<pgtag>, dim);
model.result().create(<pgtag>, ftype);
model.result(<pgtag>).set(property, <value>);
model.result(<pgtag>).run();
model.result().run();

model.result(<pgtag>).create(<ftag>, ftype);
model.result(<pgtag>).feature(<ftag>).getPlotGroup();
model.result(<pgtag>).feature(<ftag>).getType();
model.result(<pgtag>).feature(<ftag>).getSDim();
model.result(<pgtag>).feature(<ftag>).isPlotGroup();
model.result(<pgtag>).feature(<ftag>).prepareView(<value>);
model.result(<pgtag>).feature(<ftag>).selection("selection");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();

model.result(<pgtag>).feature(<ftag>).create(<attrtag>, attrtype);
model.result(<pgtag>).feature(<ftag>).feature(<attrtag>).set(property, <value>);

model.result().dataset().create(<dtag>, dtype);
model.result().dataset(<dtag>).create(<dtag2>, dtype);
model.result().dataset(<dtag>).selection("selection");
model.result().dataset(<dtag>).set(property, <value>);

model.result().export().create(<etag>, <pgtag>, ctype);
model.result().export(<etag>).create(<e2tag>, ctype);
model.result().export(<etag>).set(property, <value>);
model.result().export(<etag>).run();

model.result().numerical().create(<ntag>, ntype);
model.result().numerical(<ntag>).selection("selection");
model.result().numerical(<ntag>).set(property, <value>);
model.result().numerical(<ntag>).run();
model.result().numerical().run();

model.result().table().create(<ftag>, ntype);
model.result().table(<ftag>).setColumnHeaders(<headers>);
model.result().table(<ftag>).setTableData(<realData>, <imagData>);
model.result().table(<ftag>).getColumnHeaders();
model.result().table(<ftag>).getReal();
model.result().table(<ftag>).getImag();
model.result().table(<ftag>).isComplex();
model.result().table(<ftag>).clearTableData();
model.result().table(<ftag>).save(<filename>);
```

```

model.result().report().create(<rtag>);
model.result().report(<rtag>).set(<rprop>,<value>);
model.result().report(<rtag>).set("template", <value>);
model.result().report(<rtag>).generate();
model.result().report(<rtag>).create(<r2tag>,<rtype>);
model.result().report(<rtag>).feature(<r2tag>).set(<rprop>,<value>);
model.result().report(<rtag>).feature(<r2tag>).create(<r3tag>,<rtype>);
model.result().report(<rtag>).feature(<r2tag>).feature(<r3tag>).set(<rprop>,<value>);
model.result().report(<rtag>).run();

```

DESCRIPTION

`model.result(<pgtag>)` returns a plot group with tag `<pgtag>`.

`model.result().create(<pgtag>,dim)` creates a plot group with the tag `<pgtag>`, of dimension `dim`, where `dim` can be 1, 2, or 3.

`model.result().create(<pgtag>,ftype)` creates a plot group of type `ftype`. The valid types are "PlotGroup1D", "PlotGroup2D", "PlotGroup3D", "PolarGroup", and "SmithGroup".

`model.result(<pgtag>).create(<ftag>,ftype)` creates a plot feature of type `ftype` tagged `<ftag>` belonging to the plot group `<pgtag>`.

`model.result(<pgtag>).feature(<ftag>).getType()` returns the type of the feature `<ftag>`. This is the same string `ftype` that was used to create the feature.

`model.result(<pgtag>).feature(<ftag>).getSDim()` returns the spatial dimension of the plot group.

`model.result(<pgtag>).feature(<ftag>).isPlotGroup()` returns `true` if the feature is a plot group. This method is also available on the child features.

`model.result(<pgtag>).feature(<ftag>).getPlotGroup()` returns the plot group the feature belongs to. This method is also available on the child features.

`model.result(<pgtag>).feature(<ftag>).create(<attrtag>,attrtype)` creates an attribute feature with the tag `<attrtag>` of type `attrtype`, belonging to the feature `<ftag>`.

`model.result(<pgtag>).run()` plots the plot group with the tag `<pgtag>`.

`model.result().run()` plots all plot groups in the model.

`model.result(<pgtag>).feature(<ftag>).prepareView(<value>)` returns the view to use when plotting the plot group to which the feature belongs. The argument is a boolean, and if `true`, the view is created if needed.

`model.result(<pgtag>).feature(<ftag>).selection("selection")` returns the Selection object for the feature `<ftag>`, if applicable.

`model.result().dataset().create(<dtag>,dtype)` creates a dataset feature with the tag `<dtag>` and the type `dtype`.

`model.result().dataset(<dtag>).selection("selection")` returns the Selection object for the feature `<dtag>`, if applicable.

`model.result().export().create(<etag>,<pgtag>,etype)` creates an export feature with the tag `<etag>`, belonging to plot group `<pgtag>` and of export type `etype`.

`model.result().numerical().create(<ntag>,ntype)` creates a numerical results feature with the tag `<ntag>` of the numerical feature type `ntype`.

`model.result().numerical(<ntag>).selection("selection")` returns the Selection object for the feature `<ntag>`, if applicable.

`model.result().numerical(<ntag>).run()` evaluates the numerical results feature with the tag <ntag>.

`model.result().numerical().run()` evaluates all numerical results features in the model.

`model.result().dataset(<dtag>).refresh()` updates Grid dataset using data from functions that read files (Elevation, Image, and Interpolation).

`model.result().table().create(<ftag>, ntype)` creates a table feature with the tag <ftag>. The set and get methods used to manipulate tables are described in [Table](#).

The data extraction methods used to retrieve plot or numerical data are described in [Results](#).

`model.result().report().create(<rtag>, "Report")` creates a report with tag <rtag>.

`model.result().report().create(<ptag>, "Presentation")` creates a presentation with tag <ptag>.

`model.result().report().create(<ttag>, "Template")` creates a report template with tag <ttag>. Such report templates are customized alternatives to the built-in templates: brief, intermediate, and complete. A complete report template contains a single instance of each of the regular model-contents report features. When a model-contents report feature occurs in a template, the available properties match those of the corresponding regular report feature. However, some regular report-feature properties, such as the `noderef` property for referencing the model feature to report on, are not applicable and therefore not available.

To set up a template `template`, you can build it feature by feature or use one of the built-in templates as the starting point and then generate it before customizing the settings:

```
template.set("level", lvalue);
template.set("absentnodes", state);
template.generate();
```

The allowed values for the `level` property are "brief", "intermediate" (default), and "complete". The value of this property is also used to define the initial settings when building the template manually. The `absentnodes` property can take two values:

- "exclude" (default) — template nodes that are absent from or disabled in the template are *excluded* when generating a report using the template;
- "include" — conversely, template nodes that are absent from or disabled in the template are *included* when generating a report using the template.

Having created the report `report`, specify which template to use and then generate contents:

```
report.set("template", value);
report.generate();
```

The allowed values for the `template` property depends on the availability of custom templates in the model, the installation, and the user's report templates directory:

- The built-in templates, "brief", "intermediate" (default), and "complete", are always available.
- If the model contains report templates with tags `tmp11`, ..., `tmp1n`, these can be chosen as templates for `report` by setting the `template` property to any of the values "model.tmp11", ..., "model.tmp1n", with `model` being a fixed namespace prefix.
- If the directory `data/reporttemplates` under the COMSOL Multiphysics installation root directory contains MPH files with report templates, these can be chosen by setting `template` to

"installation.<TemplateFilename>.<ttag>". Here `installation` is a fixed namespace prefix, `<TemplateFilename>.mph` is the name of the MPH file, and `<ttag>` is a report template tag.

- Finally, if the `reporttemplates` directory under the user settings directory `.comsol/v64` under your local home directory contains MPH files with report templates, these are chosen using the same pattern as for installation templates with the difference that the namespace prefix is `user`.

Note that if `report` already has child nodes when `report.generate()` is called, these nodes will be removed before the generation of new contents.

`model.result().report().create(<ttag>, "PresentationTemplate")` creates a presentation template with tag `<ttag>`. Such presentation templates are customized alternatives to the built-in templates: default and detailed. A complete presentation template contains a single instance of each of the regular model-contents presentation features. When a model-contents presentation feature occurs in a template, the available properties match those of the corresponding regular presentation feature. However, some regular presentation-feature properties, such as the `noderef` property for referencing the model feature to include, are not applicable and therefore not available.

`report.create(<tptag>, "TitlePage")` adds a title page to the report `report`. Only one title-page feature can be added.

`report.create(<tptag>, "TitleSlide")` adds a title slide to the presentation `presentation`. Only one title-slide feature can be added.

`report.feature(<tptag>).set(prop, value)` sets the title-page property `prop` to the value `value`.

`report().create(<toc>, "TableOfContents")` adds a table of contents to the report `report`. Only one table-of-contents feature can be added.

`report.create(<stag>, "Section")` adds an additional section level to the report.

`report.create(<stag>, "SectionSlide")` adds a section slide to the presentation.

`report.feature(<stag>).set(prop, value)`

`report.feature(<stag>).create(<ftag>, feature)`

`report.feature(<stag>).feature(<ftag>).set(prop, value)`

To add a report or presentation contents feature — that is, a feature corresponding to content in the report or presentation — to a section feature `section`, type, `section.create(<frtag>, frtype, ...)`. Depending on the feature type `frtype`, the create operation includes zero, one, or two tags that refer to the model feature to include. The tags must refer to an existing feature of the correct type. The feature types that are available for reporting on the model contents are listed in [Table 2-124](#). For details on their usage, see the section [Model Contents — Report Components](#) in the *COMSOL Multiphysics User's Guide*.

TABLE 2-124: MODEL-CONTENTS REPORT FEATURES.

REPORT FEATURE	DESCRIPTION
Model	Prints information about the model root, such as model file.
ModelNode	Prints information about a model component.
Parameter	Reports on a global parameters feature.
Variables	Reports on a variables feature.
Functions	Reports on a function feature.
ThermoPackage	Reports on a thermodynamics property-package feature (requires a Chemical Reaction Engineering Module license).
MethodCall	Report on a method-call feature.
Group	Report on a constraint- or load-group feature.

TABLE 2-124: MODEL-CONTENTS REPORT FEATURES.

REPORT FEATURE	DESCRIPTION
ReducedModel	Report on a reduced-model feature.
GlobalReducedModelInputs	Report on a global reduced-model inputs feature.
GeometryPart	Report on a geometry part.
MeshPart	Report on a mesh part.
ExtraDim	Prints information about an extra-dimension model component.
ExpressionOperator	Report on an expression-operator feature.
MatrixVariable	Report on a matrix-variable feature (Matrix, Matrix Inverse, Matrix Diagonalization, Matrix Decomposition (SVD), Vector Transform, or Matrix Transform).
EquationContribution	Report on an equation-contribution feature (State Variables, Dependent Variable Field, Weak Contribution, or Constraint).
ParticipationFactors	Report on a participation-factors feature.
PNorm	Report on a P-norm feature.
StandardDeviation	Report on a standard-deviation feature.
BucklingImperfection	Report on a buckling-imperfection feature.
ResponseSpectrum	Report on a response-spectrum feature.
MassProperties	Reports on a mass-properties feature.
Probe	Reports on a probe feature.
ComponentCoupling	Reports on a component-coupling feature.
Selection	Reports on a selection feature.
Pair	Reports on a pair feature.
CoordinateSystem	Reports on a coordinate system feature.
MovingMesh	Report on a moving-mesh feature.
DeformedGeometry	Report on a deformed-geometry feature.
Optimization	Report on an optimization feature.
ParameterEstimation	Report on a parameter-estimation feature.
GeometrySampling	Report on a surrogate-model geometry sampling feature.
AbsorbingLayer	Reports on an absorbing-layer feature.
PML	Reports on a perfectly-matched-layer feature.
InfiniteElements	Reports on an infinite-element-domain feature.
MultiphysicsProp	Reports on a multiphysics-properties feature.
ControlVariableField	Reports on a control-variable-field feature.
ControlFunction	Reports on a control-function feature.
Geometry	Reports on a geometry.
Material	Reports on a material feature.
Physics	Reports on a physics interface and its features.
Multiphysics	Reports on a multiphysics coupling and its features.
Mesh	Reports on a mesh.
Study	Reports on a study.
Solver	Reports on a solver.
ResultParameter	Reports on a result-parameters feature.
Configuration	Reports on a configuration feature.
DataSet	Reports on a dataset feature.

TABLE 2-124: MODEL-CONTENTS REPORT FEATURES.

REPORT FEATURE	DESCRIPTION
DerivedValues	Reports on a derived-values feature.
Table	Includes a results table in the report.
ColorTable	Includes a custom color table in the report.
PlotGroup	Includes a plot group in the report.
EvaluationGroup	Reports on an evaluation-group feature.
Export	Includes an export feature in the report.

In addition, the custom feature types listed in [Table 2-125](#) are also available for building reports and presentations. Their usage is described in the section [Custom Report, Documentation, and Presentation Components](#) in the *COMSOL Multiphysics User's Guide*.

TABLE 2-125: CUSTOM-CONTENTS REPORT FEATURES.

REPORT FEATURE	DESCRIPTION
Equation	Adds an equation to the report.
Heading	Adds a heading to the report.
Image	Adds an image to the report.
List	Adds a list to the report.
ListItem	Adds an item to a list.
Tbl	Adds a custom table to the report.
TblHRow	Adds a heading row to a custom table.
TblRow	Adds a body row to a custom table.
Text	Adds a text paragraph to the report.
Code	Adds a text paragraph with code formatting.

Finally, a number of report feature types are provided for creating reports and presentations for apps created in the Application Builder. These are listed in [Table 2-126](#). For further details, see the sections [Arrays and Scalars](#) and [Declaration Components](#) in the *COMSOL Multiphysics User's Guide* and references therein.

TABLE 2-126: ARRAYS, SCALARS, AND DECLARATION-CONTENTS REPORT FEATURES.

REPORT FEATURE	DESCRIPTION
Arrays	Adds a customized table for Array 1D and Array 2D declaration nodes defined under the Declarations branch in the Application Builder.
Scalars	Adds a table where the columns to include and the table data rows can be customized.
ChoiceList	Reports on a choice list.
UnitSet	Reports on a unit set.
StringDataField, BooleanDataField, IntegerDataField, DoubleDataField	Reports on scalar data declarations.
StringArrayDataField, BooleanArrayDataField, IntegerArrayDataField, DoubleArrayDataField	Reports on 1D array data declarations.
StringMatrixDataField, BooleanMatrixDataField, IntegerMatrixDataField, DoubleMatrixDataField	Reports on 2D array data declarations.

To point a report feature `rFeature` to another feature with tag `<ftag>` in the tree, use the method `rFeature.set("noderef", <ftag>)` method. A report contents feature must point to a feature of the type it is designed to report on; see the table above. Instead of a feature tag, set "noderef" to "none" to clear a reference.

```
model.result().report(<rtag>).feature(<stag>).feature(<frtag>).set(frprop, <value>)
```

to set a property in a report feature.

EXAMPLES

Create a dataset and set it to point to the tagged solution `sol1` from a solver sequence:

Code for Use with Java

```
model.result().dataset().create("dset", "Solution");
model.result().dataset("dset").set("solution", "Sol1");
```

Code for Use with MATLAB

```
model.result.dataset.create('dset', 'Solution');
model.result.dataset('dset').set('solution', 'Sol1');
```

Create a 3D plot group containing a streamline plot and a plane with a contour plot on:

Code for Use with Java

```
result().create("pg1", 3);
result("pg1").set("data", "dset");
result("pg1").create("stream", "Streamline");
model.result("pg1").feature("stream").set("expr", new String[]{"2-x", "0", "z"});
model.result("pg1").feature("stream").selection().set(new int[]{2});
result().dataset().create("cutp1", "CutPlane");

result("pg1").create("cont1", "Contour");
result("pg1").feature("cont1").set("data", "cutp1");
result("pg1").run();
```

Code for Use with MATLAB

```
result.create('pg1', 3);
result('pg1').set('data', 'dset');
result('pg1').create('stream', 'Streamline');
model.result('pg1').feature('stream').set('expr', {'2-x', '0', 'z'});
model.result('pg1').feature('stream').selection.set(2);
result.dataset.create('cutp1', 'CutPlane');

result('pg1').create('cont1', 'Contour');
result('pg1').feature('cont1').set('data', 'cutp1');
result('pg1').run;
```

model.savePoint()

Manage selections and hide features used by result features.

SYNTAX

```
model.savePoint(<tag>).geom(<gtag>)
model.savePoint(<tag>).geom(<gtag>).selection(<stag>)
model.savePoint(<tag>).geom(<gtag>).view(<vtag>)
```

DESCRIPTION

`model.savePoint(<tag>)` is a container of selections and hide features used by result features. When solving, a copy of the model is made — a save point model — which is used in results and analysis. The selections and hide features contained in `model.savePoint(<tag>)` refer to the geometry in this copy.

Editing the data in `model.savePoint(<tag>)` can only be done in the following circumstances.

- The geometry on which the analysis is done has been modified after solving. In this case the selections and hide feature can be edited but not created or removed.
- The geometry on which the analysis is done has been removed. In this case the selections and hide features can be both edited, created, and deleted.

In all other circumstances, edit the selections in `model.component(<ctag>).selection()`, and the hide features in `model.component(<ctag>).view()`. Changes there are synchronized with the data in `model.savePoint()`.

`model.savePoint(<tag>).geom(<gtag>)` returns a container with selections and views with hide features for a geometry in the save point model.

`model.savePoint(<tag>).geom(<gtag>).selection(<stag>)` returns a selection.

`model.savePoint(<tag>).geom(<gtag>).view(<vtag>)` returns a view. Contrary to the views in `model.component(<ctag>).view()`, only the hide features in `view.hideEntities()` can be edited.

SEE ALSO

[model.selection\(\)](#), [model.weak\(\)](#)

model.selection()

Named selections.

SYNTAX

```
model.selection().create(<tag>);
model.selection().create(<tag>,<type>);
model.selection(<tag>).model(<mtag>);
model.selection(<tag>).set(property,<value>);
model.selection(<tag>).geom(<gtag>,dim);
model.selection(<tag>).geom(<gtag>,highdim,lowdim,typelist);
model.selection(<tag>).geom(dim);
model.selection(<tag>).all();
model.selection(<tag>).set(<entlist>);
model.selection(<tag>).add(<entlist>);
model.selection(<tag>).remove(<entlist>);
model.selection(<tag>).clear();
model.selection(<tag>).inherit(bool);
```

```
model.selection(<tag>).model();
model.selection(<tag>).isGeom();
model.selection(<tag>).geom();
model.selection(<tag>).dimension();
model.selection(<tag>).dim();
model.selection(<tag>).entities(dim);
model.selection(<tag>).entities();
model.selection(<tag>).interiorEntities(dim);
model.selection(<tag>).isInheriting();
model.selection(<tag>).inputDimension();
model.selection(<tag>).inputEntities();
model.selection(<tag>).image();
```

DESCRIPTION

`model.component(<ctag>).selection(<tag>)` returns a named selection. Anywhere where you define a selection, you can point to a named selection by using its tag, for example, `selection.named(<tag>)`.

`model.component(<ctag>).selection().create(<tag>)` creates a named selection of type Explicit.

`model.component(<ctag>).selection().create(<tag>,<type>)` creates a named selection of type `<type>`. The following types are available: "Explicit", "Union", "Intersection", "Difference", "Complement", "Adjacent", "Ball", "Box", "Cylinder", "Disk", and "LogicalExpression".

`model.selection(<tag>).model(<mtag>)` sets the model component node of the selection.

`model.selection(<tag>).model()` returns the model component node tag of the selection.

`model.component(<ctag>).selection(<tag>).set(property,<value>)` sets a property value for the selection. Which properties are available for the different selection types are listed on the following pages. All other assignment methods are only supported by the Explicit selection type.

Use the `model.selection(<tag>).image()` methods for plotting and exporting selection images. See [Plotting and Exporting Images](#).

All other methods are explained in the section [Selections](#).

Other entities can use any of the selections in `model.component(<ctag>).selection()` when defining its selection. For example, create a selection `sel1`:

```
model.component("comp1").selection().create("sel1");
```

Then, for example, a variable entities can use this selection:

```
model.component("comp1").variable().create("var1");
model.component("comp1").variable("var1").model("mod1");
model.component("comp1").variable("var1").selection().named("sel1");
```

What properties are available depends on the type of selection. The following selection types are available:

Explicit

Selection defined by an explicit set of geometric entities such as domains or boundaries.

Use the method

```
model.selection(<tag>).geom(<gtag>,highdim,lowdim,typelist);
```

with `highDim` set to `sdim` and `lowDim` set to `sdim-1` to select outputs boundaries that are adjacent to the selected domains. The `typeList` argument determines the types of adjacent boundaries included:

- `exterior` selects all exterior boundaries of the domain selection.
- `exterioroutside` selects all exterior boundaries that are adjacent to the outside (the infinite region of the complement) of the domain selection.
- `exteriorinside` selects all exterior boundaries that are adjacent to the inside (a finite region of the complement) of the domain selection.
- `interior` selects all interior boundaries of the domain selection.
- `meshinterior` selects all interior mesh boundaries of the domain selection.

Listing both `exterioroutside` and `exteriorinside` is equivalent to just listing `exterior`.

TABLE 2-127: EXPLICIT SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
angletol	double	5	Angle tolerance for continuity evaluation.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the <code>customcolor</code> property.

TABLE 2-127: EXPLICIT SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
groupcontang	on off	off	Continuous tangent mode.

When groupcontang is set to on, the set, add, and remove methods operate on groups of adjacent entities that have continuous tangents at their junctions.

The angletol property defines the tolerance for the continuity evaluation.

Ball

Selection of entities that are inside or intersect a ball.

TABLE 2-128: BALL SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
entitydim	0 1 2 3	sdim	Dimension of entities to select.
angletol	double	5	Angle tolerance for continuity evaluation.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
groupcontang	on off	off	Continuous tangent mode.
groupcontang	on off	off	Continuous tangent mode.
inputent	all selections	all	Use all entities or entities defined by input property.
input	String[]	{}	Input selections.
condition	intersects inside somevertex allvertices	intersects	Condition for inclusion of an entity.
posx	double	0	Center of ball, first coordinate.
posy	double	0	Center of ball, second coordinate.
posz	double	0	Center of ball, third coordinate.
r	double	0	Radius.

The posx, posy, and posz properties define the center of the ball, and r defines the radius. These properties take their units from the corresponding geometry sequence.

When condition is intersects, all entities that intersect the ball are included in the selection. The rendering mesh is used for the calculation. You can set the accuracy of the rendering mesh using

```
ModelUtil.setPreference("graphics.rendering.detail", <detail>);
```

where <detail> is coarse, normal, fine, or wireframe.

When condition is inside, all entities that are completely inside the ball are included in the selection. The rendering mesh is used for the calculation.

When condition is somevertex, all entities that have at least one adjacent vertex inside the ball are included in the selection.

When condition is allvertices, all entities that have all adjacent vertices inside the ball are included in the selection.

When `inputent` is `selections`, the selection is restricted to the entities in the selections defined by the input property. When `inputent` is `all`, all entities in the geometry are considered.

When `groupcontang` is set to `on`, the selection operates on groups of entities that have continuous tangents at their junctions.

The `angletol` property defines the tolerance for the continuity evaluation.

Box

Selection of entities that are inside or intersect a box.

TABLE 2-129: BOX SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>entitydim</code>	0 1 2 3	<code>sdim</code>	Dimension of entities to select.
<code>angletol</code>	double	5	Angle tolerance for continuity evaluation.
<code>color</code>	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the <code>customcolor</code> property.
<code>customcolor</code>	RGB-triplet	Next available theme color	The color to use. Active when <code>color</code> is set to custom.
<code>groupcontang</code>	on off	off	Continuous tangent mode.
<code>groupcontang</code>	on off	off	Continuous tangent mode.
<code>inputent</code>	all selections	all	Use all entities or entities defined by input property.
<code>input</code>	String[]	{ }	Input selections.
<code>condition</code>	intersects inside somevertex allvertices	intersects	Condition for inclusion of an entity.
<code>xmax</code>	double	inf	Maximum x-coordinate of box.
<code>xmin</code>	double	-inf	Minimum x-coordinate of box.
<code>ymax</code>	double	inf	Maximum y-coordinate of box.
<code>ymin</code>	double	-inf	Minimum y-coordinate of box.
<code>zmax</code>	double	inf	Maximum z-coordinate of box.
<code>zmin</code>	double	-inf	Minimum z-coordinate of box.

The `xmax`, `xmin`, `ymax`, `ymin`, `zmax`, and `zmin` properties define the box. These properties take their units from the corresponding geometry sequence.

When `condition` is `intersects`, all entities that intersect the box are included in the selection. The rendering mesh is used for the calculation. You can set the accuracy of the rendering mesh using

```
ModelUtil.setPreference("graphics.rendering.detail", <detail>);
```

where `<detail>` is `coarse`, `normal`, `fine`, or `wireframe`.

When `condition` is `inside`, all entities that are completely inside the box are included in the selection. The rendering mesh is used for the calculation.

When `condition` is `somevertex`, all entities that have at least one adjacent vertex inside the box are included in the selection.

When `condition` is `allvertices`, all entities that have all adjacent vertices inside the box are included in the selection.

When `inputent` is `selections`, the selection is restricted to the entities in the selections defined by the input property. When `inputent` is `all`, all entities in the geometry are considered.

When `groupcontang` is set to `on`, the selection operates on groups of entities that have continuous tangents at their junctions.

The `angletol` property defines the tolerance for the continuity evaluation.

Cylinder

Selection of entities that are inside or intersect a cylinder in 3D.

TABLE 2-130: CYLINDER SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>angle1</code>	double	0	Start angle
<code>angle2</code>	double	360	End angle (default: 360 degrees; that is, a full cylinder)
<code>color</code>	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the <code>customcolor</code> property.
<code>customcolor</code>	RGB-triplet	Next available theme color	The color to use. Active when <code>color</code> is set to custom.
<code>groupcontang</code>	on off	off	Continuous tangent mode
<code>entitydim</code>	0 1 2 3	3	Dimension of entities to select
<code>angletol</code>	double	5	Angle tolerance for continuity evaluation
<code>groupcontang</code>	on off	off	Continuous tangent mode
<code>inputent</code>	all selections	all	Use all entities or entities defined by input property
<code>input</code>	String[]	{}	Input selections
<code>condition</code>	intersects inside somevertex allvertices	intersects	Condition for inclusion of an entity
<code>pos</code>	double[]	{0,0,0}	Cylinder base point
<code>axis</code>	double[]	{0,0,1}	Direction of the cylinder axis. Vector has length 3 if <code>axistype</code> is cartesian and length 2 if <code>axistype</code> is spherical. Not used if <code>axistype</code> is x, y, or z.
<code>axistype</code>	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with <code>axis</code> .
<code>top</code>	double	inf	Coordinate of upper face in local coordinate system
<code>bottom</code>	double	-inf	Coordinate of lower face in local coordinate system
<code>r</code>	double (nonnegative)	0	Outer radius
<code>rin</code>	double (nonnegative)	0	Inner radius

The `pos` property defines the center of the cylinder and the `axis` property defines the cylinder axis. The `top`, `bottom`, `r`, and `rin` properties define the size of the cylinder. Setting `rin` equal to `r` corresponds to a cylindrical surface. These properties take their units from the corresponding geometry sequence. Using the `angle1` and `angle2` properties, you can create a cylinder segment.

When `condition` is `intersects`, all entities that intersect the cylinder are included in the selection. The rendering mesh is used for the calculation. You can set the accuracy of the rendering mesh using

```
ModelUtil.setPreference("graphics.rendering.detail",<detail>);
```

where <detail> is coarse, normal, fine, or wireframe.

When condition is inside, all entities that are completely inside the cylinder are included in the selection. The rendering mesh is used for the calculation.

When condition is somevertex, all entities that have at least one adjacent vertex inside the cylinder are included in the selection.

When condition is allvertices, all entities that have all adjacent vertices inside the cylinder are included in the selection.

When inputent is selections, the selection is restricted to the entities in the selections defined by the input property. When inputent is all, all entities in the geometry are considered.

When groupcontang is set to on, the selection operates on groups of entities that have continuous tangents at their junctions.

The angletol property defines the tolerance for the continuity evaluation.

Disk

Selection of entities that are inside or intersect a disk.

TABLE 2-131: DISK SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
angle1	double	0	Start angle
angle2	double	360	End angle (default: 360 degrees; that is, a full disk)
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
groupcontang	on off	off	Continuous tangent mode.
entitydim	0 1 2	sdim	Dimension of entities to select.
angletol	double	5	Angle tolerance for continuity evaluation.
groupcontang	on off	off	Continuous tangent mode.
inputent	all selections	all	Use all entities or entities defined by input property.
input	String[]	{}	Input selections
condition	intersects inside somevertex allvertices	intersects	Condition for inclusion of an entity.
posx	double	0	Center of disk, first coordinate.
posy	double	0	Center of disk, second coordinate.
r	double (nonnegative)	0	Outer radius.
rin	double (nonnegative)	0	Inner radius.

The posx and posy properties define the center of the disk, and r and rin define the outer and inner radius, respectively. These properties take their units from the corresponding geometry sequence. Using the angle1 and angle2 properties, you can create a disk segment.

When condition is intersects, all entities that intersect the disk are included in the selection. The rendering mesh is used for the calculation. You can set the accuracy of the rendering mesh using

```
ModelUtil.setPreference("graphics.rendering.detail", <detail>);
```

where *<detail>* is coarse, normal, fine, or wireframe.

When condition is inside, all entities that are completely inside the disk are included in the selection. The rendering mesh is used for the calculation.

When condition is somevertex, all entities that have at least one adjacent vertex inside the disk are included in the selection.

When condition is allvertices, all entities that have all adjacent vertices inside the disk are included in the selection.

When inputent is selections, the selection is restricted to the entities in the selections defined by the input property. When inputent is all, all entities in the geometry are considered.

When groupcontang is set to on, the selection operates on groups of entities that have continuous tangents at their junctions.

The angletol property defines the tolerance for the continuity evaluation.

Union

Selection defined by the union of a set of selections.

TABLE 2-132: UNION SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
entitydim	0 1 2 3	sdim	Dimension of entities to select.
input	String array	{}	Selections to add.

Intersection

Selection defined by the intersection of a set of selections.

TABLE 2-133: INTERSECTION SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
groupcontang	on off	off	Continuous tangent mode.
entitydim	0 1 2 3	sdim	Dimension of entities to select.
input	String[]	{}	Selections to intersect.

Difference

Selection defined by the difference between two sets of selections.

TABLE 2-134: DIFFERENCE SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.

TABLE 2-134: DIFFERENCE SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
groupcontang	on off	off	Continuous tangent mode.
entitydim	0 1 2 3	sdim	Dimension of entities to select.
add	String[]	{}	Selections to add.
subtract	String[]	{}	Selections to subtract.

Complement

Selection defined by the complement of a set of selections.

TABLE 2-135: COMPLEMENT SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
groupcontang	on off	off	Continuous tangent mode.
entitydim	0 1 2 3	sdim	Dimension of entities to select.
input	String[]	{}	Selections to invert.

Adjacent

Selection of entities that are adjacent to entities in another selection.

TABLE 2-136: ADJACENT SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
groupcontang	on off	off	Continuous tangent mode
entitydim	0 1 2 3	sdim	Dimension of entities to select.
input	String[]	{}	Input selections.
outputdim	0 1 2 3	sdim-1	Dimension of output entities.
exterior	on outside inside off	on	on: Select all exterior entities. outside: Select all exterior boundaries that are adjacent to the infinite region (connected component) of the complement of the domain selection. inside: Select all exterior boundaries that are adjacent to a finite region (connected component) of the complement of the domain selection. off: Select no exterior entities. Values outside and inside are only supported when entitydim is sdim and outputdim is sdim-1.
interior	on off	off	Include interior boundaries/edges.

LogicalExpression

Selection defined by a logical expression in terms of other named selections.

TABLE 2-137: LOGICAL EXPRESSION SELECTION PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
entitydim	0 1 2 3	sdim	Dimension of entities to select.
expression	String		The logical expression to use. It can contain parentheses and the , &&, and ! Boolean operators, operating on the tags of existing named selections.

EXAMPLES

Define the selection equ1 as the domain of a rectangle and the selection bnd1 as the boundary of the rectangle.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
model.component("comp1").geom().create("geom1",2);
model.component("comp1").geom("geom1").create("f1","Rectangle");
model.component("comp1").geom("geom1").run("f1");
model.component("comp1").selection().create("equ1").geom(2);
model.component("comp1").selection("equ1").all();
model.component("comp1").selection().create("bnd1").geom(1);
model.component("comp1").selection("bnd1").all();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
model.component('comp1').geom.create('geom1',2);
model.component('comp1').geom('geom1').create('f1','Rectangle');
model.component('comp1').geom('geom1').run('f1');
model.component('comp1').selection.create('equ1').geom(2);
model.component('comp1').selection('equ1').all;
model.component('comp1').selection.create('bnd1').geom(1);
model.component('comp1').selection('bnd1').all;
```

The (outer) boundaries for the model can be set with the following selection:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
model.component("comp1").geom().create("geom1",2);
model.component("comp1").geom("geom1").create("r1","Rectangle");
model.component("comp1").selection().create("outer").
    geom("geom1",2,1,new String[]{"exterior"});
model.component("comp1").selection("outer").all();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
model.component('comp1').geom.create('geom1',2);
model.component('comp1').geom('geom1').create('r1','Rectangle');
model.component('comp1').selection.create('outer').geom('geom1',2,1,{'exterior'});
model.component('comp1').selection('outer').all;
```


Create a selection for all boundaries of a block intersecting a ball with radius 0.5 and center (1,1,1):

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
model.component("comp1").geom().create("g1",3).create("blk1", "Block");
model.component("comp1").geom("g1").run();
model.component("comp1").selection().create("ball1", "Ball");
model.component("comp1").selection("ball1").set("entitydim", "2");
model.component("comp1").selection("ball1").set("posx", "1");
model.component("comp1").selection("ball1").set("posy", "1");
model.component("comp1").selection("ball1").set("posz", "1");
model.component("comp1").selection("ball1").set("r", "0.5");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
model.component('comp1').geom.create('g1',3).create('blk1', 'Block');
model.component('comp1').geom('g1').run;
model.component('comp1').selection.create('ball1', 'Ball');
model.component('comp1').selection('ball1').set('entitydim', '2');
model.component('comp1').selection('ball1').set('posx', '1');
model.component('comp1').selection('ball1').set('posy', '1');
model.component('comp1').selection('ball1').set('posz', '1');
model.component('comp1').selection('ball1').set('r', '0.5');
```

Create a selection of all edges adjacent to the boundaries in the ball selection:

Code for Use with Java

```
model.component("comp1").selection().create("adj1", "Adjacent");
model.component("comp1").selection("adj1").set("entitydim", "2");
model.component("comp1").selection("adj1").set("outputdim", "1");
model.component("comp1").selection("adj1").set("input", new String[]{"ball1"});
```

Code for Use with MATLAB

```
model.component('comp1').selection.create('adj1', 'Adjacent');
model.component('comp1').selection('adj1').set('entitydim', '2');
model.component('comp1').selection('adj1').set('outputdim', '1');
model.component('comp1').selection('adj1').set('input', {'ball1'});
```

SEE ALSO

[Selections](#)

model.shape()

Create and define shape functions for the field variables' elements.

SYNTAX

```
model.shape().create(<tag>,<frame>);
model.shape(<tag>).elementSet(<condition>);
model.shape(<tag>).frame(<ftag>);
model.shape(<tag>).create(<ftag>,<func>);
model.shape(<tag>).feature(<ftag>).set(property,<value>);
model.shape(<tag>).slits().named(<seltag>);
model.shape(<tag>).slits().set(...);
model.shape(<tag>).upFlux(<varName>);
model.shape(<tag>).downFlux(<varName>);
model.shape(<tag>).domainFlux(<expressions>,<frame>);

model.shape(<tag>).elementSet();
model.shape(<tag>).frame();
model.shape(<tag>).feature(<ftag>).shape();
model.shape(<tag>).feature(<ftag>).getType(<property>);
model.shape(<tag>).fieldVariable();
model.shape(<tag>).slits().named();
model.shape(<tag>).slits().getType();
model.shape(<tag>).upFlux();
model.shape(<tag>).downFlux();
```

DESCRIPTION

`model.shape(<tag>)` returns a shape function.

`model.shape().create(<tag>,<frame>)` creates a shape function with tag `<tag>` and assigns the frame `<frame>` to it.

`model.shape(<tag>).frame(<ftag>)` assigns frame `<ftag>` to the shape function. See `model.frame()` for a discussion on the default frame.

`model.shape(<tag>).create(<ftag>,<func>)` creates a shape feature with the shape function expression `func`. `func` can be a shape function name (`shlag`, for example) or a shape function with arguments (`shlag(2,u)`, for example). The latter is interpreted as an assignment of some property values.

`model.shape(<tag>).feature(<ftag>).set(property,<value>)` sets a property for the shape function. Of the data types listed in [Table 2-4](#), the ones supported are those for integers, strings, and string arrays. Which ones are applicable differs for each property.

`model.shape(<tag>).selection().named(<seltag>)` assigns the shape function to the named selection `<seltag>`.

`model.shape(<tag>).selection().set(...)` defines a local selection that assigns the shape function to geometric entities. For a complete list of methods available under `selection()`, see [Selections](#).

`model.shape(<tag>).upFlux(<varName>)` and `model.shape(<tag>).downFlux(<varName>)` set the names of the up and down boundary flux variables. `model.shape(<tag>).domainFlux(<expressions>,<frame>)` sets expressions for the domain flux in a given frame. This is required to make the boundary flux variables produce accurate results. Only Lagrange shape functions support boundary flux variables.

`model.shape(<tag>).frame()` returns the frame tag as a string.

`model.shape(<tag>).feature(<ftag>).shape()` returns the shape function expression as a string.

`model.shape(<tag>).feature(<ftag>).getType(property)` returns a property value. For available data types, see [get* and Selection Access Methods](#).

`model.shape(<tag>).fieldVariable()` returns the field variables which the shape functions define.

`model.shape(<tag>).selection().named()` returns the selection tag as a string.

`model.shape(<tag>).elementSet(<condition>)` sets the element set condition to the given string. The condition should be an expression containing *element set variables* (defined in `model.elementSet()`) and the logical operators `&&`, `||`, and `!`. The shape function is defined only on the mesh elements for which the condition is true (nonzero).

`model.shape(<tag>).elementSet()` returns the element set condition. An empty string means no condition.

`model.shape(<tag>).selection().getType()` returns domain information. For available methods, see [model.selection\(\)](#).

`model.shape(<tag>).slits()` returns a selection used to generate a slit on the shape. Works exactly as selections.

`model.shape(<tag>).upFlux()` and `model.shape(<tag>).downFlux()` return the names of the up and down flux variables (an empty string if the variable names have not been set.)

EXAMPLE

Define the shape function `shlag(2, "u")`:

Code for Use with Java

```
model.shape().create("shu", "f");
model.shape("shu").create("f1", "shlag");
model.shape("shu").feature("f1").set("order", 2);
model.shape("shu").feature("f1").set("basename", "u");
model.shape("shu").selection().named("equ1");
```

Code for Use with MATLAB

```
model.shape.create('shu', 'f');
model.shape('shu').create('f1', 'shlag');
model.shape('shu').feature('f1').set('order', 2);
model.shape('shu').feature('f1').set('basename', 'u');
model.shape('shu').selection.named('equ1');
```

See also [Shape Functions and Element Types](#) for information and syntax examples for all shape functions (element types).

SEE ALSO

[model.coeff\(\)](#), [model.intRule\(\)](#), [model.weak\(\)](#) and the [Elements and Shape Function Programming](#) chapter.

model.sol()

Solver sequences.



Solvers and Study Steps

SYNTAX

```
model.sol().create(<tag>)
model.sol().create(<tag>,<studytag>)
model.sol().create(<tag>,<studytag>,<varstag>)
model.sol().remove(<tag>)

model.sol(<tag>).create(<ftag>,<oper>)
model.sol(<tag>).feature().remove(<ftag>)
model.sol(<tag>).feature(<ftag>).create(<f2tag>,<oper>)
model.sol(<tag>).feature(<ftag>).set(property,<value>)
model.sol(<tag>).attach(<stag>)
model.sol(<tag>).isAttached()
model.sol(<tag>).detach(<stag>)
model.sol(<tag>).study(<stag>)
model.sol(<tag>).clearSolutionData()
model.sol(<tag>).copySolution(<ctag>)
model.sol(<tag>).createAutoSequence(<stag>)
model.sol(<tag>).createSolution()
model.sol(<tag>).updateSolution()
model.sol(<tag>).adaptationStudyFeature()
model.sol(<tag>).adaptationStudyFeature(<stag>)
model.sol(<tag>).getDefaultSolnum()
model.sol(<tag>).setClusterStorage
model.sol(<tag>).getClusterStorage
model.sol(<tag>).isEmpty()
model.sol(<tag>).isInitialized()
model.sol(<tag>).run(<ftag>,[<createPlots>])
model.sol(<tag>).runFrom(<ftag>,[<createPlots>])
model.sol(<tag>).runFromTo(<ftagstart>,<ftagstop>,[<createPlots>])
model.sol(<tag>).runAll([<createPlots>])
model.sol(<tag>).runAllWithoutModelUpdate()
model.sol(<tag>).continueRun()
model.sol(<tag>).continueWithoutModelUpdate()
```

DESCRIPTION

`model.sol().create(<tag>)` adds a solver sequence to the model.

`model.sol().create(<tag>,<studytag>)` adds a solver sequence to the model. The constructor adds one feature of the type `StudyStep` to the solver sequence with the tag `<studytag>`. This `StudyStep` feature is connected to a study step (see `model.study()`).

`model.sol().create(<tag>,<studytag>,<varstag>)` adds a solver sequence to the model. The constructor adds one feature of the type `StudyStep` with the tag `<studytag>` and one feature of the type `Variables` with the tag `<varstag>` to the solver sequence.

`model.sol().remove(<tag>)` removes a solver sequence from the model.

`model.sol(<tag>).create(<ftag>,<oper>)` creates a solver feature. Each solver feature is a solver operation.

`model.sol(<tag>).feature().remove(<ftag>)` removes the solver feature `<ftag>`.

`model.sol(<tag>).feature(<ftag>).set(property,<value>)` sets the property `property` for the feature `<ftag>`.

`model.sol(<tag>).attach(<stag>)` attaches a solver sequence with tag `<tag>` to a study with tag `<stag>`, which makes it visible under that study. You can use `attach` to make a solver sequence part of the study sequence. Using `model.sol(<tag>).study(<stag>)`, on the other hand, only associates a solver sequence with a study. The `attach()` operation implies calling `study(<tag>)`. You can only have one solver sequence attached to each study (but you can have multiple batch features).

`model.sol(<tag>).isAttached()` returns true if the solver sequence with tag `<tag>` is attached to a study.

`model.sol(<tag>).detach(<stag>)` detaches a solver sequence with tag `<tag>` from study with tag `<stag>`.

`model.sol(<tag>).clearSolutionData()` clears computed solution data associated with the solver sequence `<tag>`. Solution selection settings and settings in results features associated with the solution are not modified.

`model.sol(<tag>).getDefaultSolnum()` returns which solnum the call `getU()` returns (compare with `getU(<solnum>)`; see [Solution Data](#)).

`model.sol(<tag>).setClusterStorage(<value>)` sets the solution storage format used on clusters. Use the `<value>` "all" to store the solution on all cluster nodes and the `<value>` "single" to store the solution only on a single cluster node.

`model.sol(<tag>).getClusterStorage()` returns "all" if the solution is stored on all cluster nodes and "single" if the solution is only stored on a single cluster node.

`model.sol(<tag>).copySolution(<ctag>)` copies the solution data associated with the solver sequence `<tag>` to a new solver sequence `<ctag>`. The features are not copied.

`model.sol(<tag>).createAutoSequence(<stag>)` creates a solver sequence of features automatically from the study `<stag>`. The sequence of study steps are used as input to the sequence generation algorithm but also the physics used in the study steps are used to automatically adopt the solver settings.

`model.sol(<tag>).createSolution()` creates a solution object from one or more set operations (`setU(...)`, ...), see [Solution Creation](#) for details.

`model.sol(<tag>).updateSolution()` updates a solution data associated with the solver sequence to make it consistent with the current model, without recomputing the solution.

`model.sol(<tag>).adaptationStudyFeature()` returns the path to the study feature with adaptation that controls this sequence or empty string if none. `model.sol(<tag>).adaptationStudyFeature(<ftag>)` sets the study feature with adaptation, `<ftag>`, that controls the solver sequence `<tag>`. Use an empty string to disable study control.

`model.sol(<tag>).isEmpty()` is true if there is no solution data or if all solution data has been cleared.

`model.sol(<tag>).isInitialized()` is true if the solution is a valid (initialized) object. Even if the solution has been cleared, `isInitialized` is true (use `isEmpty` to check for cleared solution data).

`model.sol(<tag>).run(<ftag>)` runs the features for a solver sequence up to and including the feature `<ftag>`. It corresponds to clicking **Compute** on the solver sequence feature node in the COMSOL Desktop.

`model.sol(<tag>).runFrom(<ftag>)` runs the features for a solver sequence from and including the feature `<ftag>`.

`model.sol(<tag>).runFromTo(<ftagstart>,<ftagstop>)` runs the features for a solver sequence from and including the feature `<ftagstart>` to and including the feature `<ftagstop>`.

`model.sol(<tag>).runAll()` runs all the features for a solver sequence.

All the `run`, `runFrom`, `runFromTo`, and `runAll` methods can take an additional boolean input argument `createPlots`, which, when set to true, generates the corresponding default plots when computing a solution.

`model.sol(<tag>).runAllWithoutModelUpdate()` runs all the features for a solver sequence without updating the model, skipping the discretization phase.

`model.sol(<tag>).continueRun()` continues to run a solver sequence that has been stopped.

`model.sol(<tag>).continueWithoutModelUpdate()` continues to run a stopped solver sequence without updating the model, skipping the discretization phase.

EXAMPLES

Assume that a study `st1` represents one stationary study step with the tag `stat1` for some equations.

Code for Use with Java

```
model.sol().create("s", "step1", "vars1");
model.sol("s").feature("step1").set("study", "st1");
model.sol("s").feature("step1").set("studystep", "stat1");
model.sol("s").create("solver1", "Stationary");
```

Code for Use with MATLAB

```
model.sol.create('s', 'step1', 'vars1');
model.sol('s').feature('step1').set('study', 'st1');
model.sol('s').feature('step1').set('studystep', 'stat1');
model.sol('s').create('solver1', 'Stationary');
```

Assume that a second study step with frequency response is added to the study with tag `freq1` and that you want to make a frequency sweep from 10 to 1000 using the parametric solver and the solution above as the linearization point (bias solution).

Code for Use with Java

```
model.sol("s").create("step2", "StudyStep");
model.sol("s").feature("step2").set("study", "st1");
model.sol("s").feature("step2").set("studystep", "freq1");
model.sol("s").create("vars2", "Variables");
SolverFeature s2 = (SolverFeature) model.sol("s").create("solver2", "Stationary");
s2.set("nonlin", "linper"); // (*)
s2.set("linpmethod", "sol");
s2.set("linpsol", "s");
s2.set("storelinpoint", "on");
s2.create("par", "Parametric");
s2.feature("par").set("pname", "freq");
s2.feature("par").set("plist", new double[]{10, 1000});
s2.runAll();
```

Code for Use with MATLAB

```
model.sol('s').create('step2', 'StudyStep');
model.sol('s').feature('step2').set('study', 'st1');
model.sol('s').feature('step2').set('studystep', 'freq1');
model.sol('s').create('vars2', 'Variables');
s2 = model.sol('s').create('solver2', 'Stationary');
s2.set('nonlin', 'linper'); // (*)
s2.set('linpmethod', 'sol');
s2.set('linpsol', 's');
s2.set('storelinpoint', 'on');
s2.create('par', 'Parametric');
s2.feature('par').set('pname', 'freq');
s2.feature('par').set('plist', [10, 1000]);
s2.runAll;
```

At this point the solution `s` is associated to the study step `freq1` (but it depends indirectly on the bias study step `stat1` as well).

(*) Uses the small-signal study functionality, which makes it possible to access also the linearization point for results processing together with the small-signal solution. Here it is assumed that the bias problem and the small-signal problem can be set up independently for the two study steps.

COMPATIBILITY

From version 6.4, the methods

```
model.sol(<tag>).run()
model.sol(<tag>).run([<createPlots>])
```

are deprecated. Use the following methods instead.

```
model.sol(<tag>).runAll()
model.sol(<tag>).runAll([<createPlots>])
```

From version 5.3a, the method

```
model.sol(<tag>).clearSolution()
```

is deprecated and replaced by the method

```
model.sol(<tag>).clearSolutionData()
```

since `clearSolutionData` generally works as expected, while `clearSolution` clears settings unexpectedly.

SEE ALSO

[model.study\(\)](#)

model.solverEvent()

Create and define events for the solver.

SYNTAX

```
model.solverEvent().create(<tag>,evtype);
model.solverEvent(<tag>).start(expr);
model.solverEvent(<tag>).start();
model.solverEvent(<tag>).period(expr);
model.solverEvent(<tag>).period();
model.solverEvent(<tag>).condition(expr);
model.solverEvent(<tag>).condition();
model.solverEvent(<tag>).reinit();
model.solverEvent(<tag>).reinit().create(<tag>);
model.solverEvent(<tag>).reinit(<tag>).set(<var>,expr);
model.solverEvent(<tag>).useConsistentInit(isConsistent);
```

DESCRIPTION

Create events and control event settings. There are two types of events; **Explicit** and **Implicit**.

`model.solverEvent().create(<tag>,evtype)` creates a new event of type *evtype*, either **Explicit** and **Implicit**.

There are also *event sequences*, which you create using the *evtype* set as **EventSequence** to which you can add *sequence members*. See [Event Sequences](#).

Explicit Events

Explicit events triggers on a predefined timing.

`model.solverEvent(<tag>).start(expr)` sets the start time for an explicit event.

`model.solverEvent(<tag>).period(expr)` sets the period for an explicit event. After the start time, the event then triggers after each period.

Implicit Events

Implicit events trigger when a condition goes from false to true.

`model.solverEvent(<tag>).condition(expr)` sets the condition for an implicit event.

Event Sequences

You can also add event sequences to specify a sequence of events that will take place in the order that you add them. Event sequences simplify the setup of a sequence of events by automatically defining explicit and implicit events. The individual steps in the sequence are defined using sequence member subnodes:

```
model.solverEvent().feature(<estag>).create<tag>(SequenceMember);
```

For a sequence member you can set the property `endConditionOptions` to `logicalExpression` and then provide a logical expression using the `condition` property that is true when it evaluates to something that is >0 , or set the property `endConditionOptions` to `duration` and then provide a time duration using the `duration` property to control for how long a step runs.

Reinitialization

When an event is triggered, any degree of freedom can be reinitialized. This typically means that they get a new value. You specify these values with a reinitialization method, `reinit(...)`, which has the same syntax as `model.init(...)`.

`model.solverEvent(<tag>).reinit().create(<tag>)` adds a new reinit feature to the event. In most cases you only need one, but you need more when you have reinitialization conditions on several geometric entity levels, for example on a global selection and a domain selection.

Event State Variables

An event needs state variables in most cases. There are discrete states and indicator states. Discrete states are just ODE states that only change during reinitialization, and can only have a zero-valued equation (or no equation). The indicator states are needed for implicit events and are ODE states with nonzero equations.

`model.ode().create(<tag>).type(<ode type>)` creates a new global equation that contains event state variables if you set the ode type to `discrete` for discrete states and `quadrature` for indicator states.

`model.ode(<tag>).state(<states>)` adds a new discrete states to the global equation.

`model.ode(<tag>).ode(<state>,"sin(2*pi*t)")` adds a new indicator state and its right-hand side to the global equation. The left-hand side of the equation is the state variable, so the full equation for the indicator state becomes `nojac(sin(2*pi*t))-<state>`.

`model.solverEvent(<tag>).useConsistentInit(isConsistent)` controls how consistency is enforced after reinitialization. If `isConsistent = 0` for all of the triggered events, then consistency will not be enforced after reinitialization. If `isConsistent = 1` for at least one of the triggered events and `Time.consistent = bweuler` (see [Table 6-74](#)), then consistency is enforced using an artificial step with the backward Euler method. See [About the Reinitialization Process](#) in the *COMSOL Reference Manual* for additional details. The solver log indicates whether and how consistency has been enforced after each reinitialization.

EXAMPLE

Example of an idealized bouncing ball using implicit events.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.study().create("std1");
model.study("std1").create("time1", "Transient");
model.study("std1").feature("time1").set("tlist", "0 10");
model.study("std1").feature("time1").set("rtol", 1e-6); // Nondiscrete states
model.ode().create("ode1");
model.ode("ode1").ode("y", "-2*y-ytt");
model.init().create("ode1");
model.init("ode1").selection().global();
model.init("ode1").set("y", "1");
// Discrete states
model.ode().create("ode2").type("quadrature");
model.ode("ode2").ode("z1", "y");
// Implicit event
model.solverEvent().create("impl1", "Implicit");
model.solverEvent("impl1").condition("!(z1>=0)");
model.solverEvent("impl1").reinit().create("reinit");
model.solverEvent("impl1").reinit("reinit").selection().global();
model.solverEvent("impl1").reinit("reinit").set("y", "y");
```



```

// Bounce reverts velocity
model.solverEvent("impl1").reinit("reinit").set("yt", "-yt");
model.sol().create("sol1");
model.sol("sol1").createAutoSequence("std1");
// Special solver settings for events
model.sol("sol1").feature("t1").set("tout", "tsteps");
model.sol("sol1").feature("t1").set("atolglobal", "1e-6");
model.sol("sol1").feature("t1").set("initialstepbdfactive", "on");
model.sol("sol1").feature("t1").set("initialstepbdf", "1e-6");
model.sol("sol1").feature("t1").set("eventtol", "2e-6");
model.sol("sol1").feature("t1").set("ewtrescale", "off");
model.sol("sol1").runAll();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.study.create('std1');
model.study('std1').create('time1', 'Transient');
model.study('std1').feature('time1').set('tlist', '0 10');
model.study('std1').feature('time1').set('rtol', 1e-6);
% Nondiscrete states
model.ode.create('ode1');
model.ode('ode1').ode('y', '-2*y-ytt');
model.init.create('ode1');
model.init('ode1').selection().global();
model.init('ode1').set('y', '1');
% Discrete states
model.ode.create('ode2').type('quadrature');
model.ode('ode2').ode('z1', 'y');
% Implicit event
model.solverEvent.create('impl1', 'Implicit');
model.solverEvent('impl1').condition('!(z1>=0)');
model.solverEvent('impl1').reinit().create('reinit');
model.solverEvent('impl1').reinit('reinit').selection().global();
model.solverEvent('impl1').reinit('reinit').set('y', 'y');
% Bounce reverts velocity
model.solverEvent('impl1').reinit('reinit').set('yt', '-yt');
model.sol().create('sol1');
model.sol('sol1').createAutoSequence('std1');
% Special solver settings for events
model.sol('sol1').feature('t1').set('tout', 'tsteps');
model.sol('sol1').feature('t1').set('atolglobal', '1e-6');
model.sol('sol1').feature('t1').set('initialstepbdfactive', 'on');
model.sol('sol1').feature('t1').set('initialstepbdf', '1e-6');
model.sol('sol1').feature('t1').set('eventtol', '2e-6');
model.sol('sol1').feature('t1').set('ewtrescale', 'off');
model.sol('sol1').runAll();

```

SEE ALSO

[model.ode\(\)](#), [model.init\(\)](#)

model.study()

Create and define studies.

SYNTAX

```
model.study().create(<tag>);
model.study(<tag>).create(<ftag>,type);
model.study(<tag>).feature().move(<ftag>,position);

model.study(<tag>).run([<createPlots>])
model.study(<tag>).runNoGen()
model.study(<tag>).runNoGen(type)
model.study(<tag>).createAutoSequences(type)
model.study(<tag>).showAutoSequences(type);
model.study(<tag>).getSolverSequences(type);
model.study(<tag>).isGenConv();
model.study(<tag>).isGenIntermediatePlots();
model.study(<tag>).isGenPlots();
model.study(<tag>).isPlotUndefVals();
model.study(<tag>).isStoreSolution();
model.study(<tag>).setEntry(...);
model.study(<tag>).setGenConv(boolean);
model.study(<tag>).setGenIntermediatePlots(boolean);
model.study(<tag>).setGenPlots(boolean);
model.study(<tag>).setPlotUndefVals(boolean);
model.study(<tag>).setStoreSolution(boolean);
model.study(<tag>).feature(<ftag>).loadFile(filePath);
model.study(<tag>).feature(<ftag>).saveFile(filePath);

step = model.study(<tag>).feature(<ftag>);

step.discardData();
step.discretization(<physpath>,<discr>);
step.exportData(<filePath>);
step.importData(<filePath>);
step.loadFile(<filePath>);
step.mesh(<geom>,<mesh>);
step.refresh();
step.saveFile(<filePath>);
step.setSolveFor(<entityPath>,boolean)

step.type();
step.discretization(<physpath>);
step.mesh(<geom>);
step.solveFor(<entityPath>);

step.mglevel.create(<mglevel>);
step.mglevel(<mglevel>).mesh(<geom>,<mesh>);
step.mglevel(<mglevel>).discretization(<physpath>,<discr>);

step.mglevel(<mglevel>).mesh(<geom>);
step.mglevel(<mglevel>).discretization(<physpath>);
```

DESCRIPTION

`model.study` stores a list of studies, each of which consists of a number of study steps. Each study step, in turn, defines a solver-ready problem. This means that a study step can be turned into an extended mesh, and a basic solver (Stationary, Time, Eigenvalue, Modal, AWE, or Optimization) can be applied, resulting in a solution object.

The central property of a study step is its *study type*, which on one hand controls the equations generated by physics interfaces, and on the other hand triggers automatic selection of a suitable solver. Another important property of a study step is which mesh to use (for each geometry in the model). Other fundamental simulation parameters can also be found among the study step settings, like the time span for a Time Dependent study type and frequency range for a Frequency Domain study type.

Under a study step, you can add *multigrid levels*. The parent node still defines the problem to be solved (for example, the study type and the mesh). Therefore, the added multigrid levels must necessarily be coarser than the parent study step.

Most physics features and also some other parts of the model object (for example, expression features) must support a `step` member, which (in analogy to the spatial `selection`) controls for which study steps the feature is active. In many ways, the study selection can be seen as a fourth, discrete, dimension.

The *StudyList* is the list of studies in a model. For instance, `model.study("std1")` gives the study with tag `std1`. `model.study().create(<tag>)` creates a new study sequence.

`model.study(<tag>).run()` computes the study. The `run` method can take an additional boolean input argument `createPlots`, which, when set to true, generates the corresponding default plots when computing a solution.

`model.study(<tag>).runNoGen()` runs the attached solver sequence without regenerating it. If the study does not have an attached solver sequence, a new one is generated.

`model.study(<tag>).runNoGen(type)` runs the attached solver sequence without regenerating it. If no sequence exists, a new one is generated. The argument *type* is one of `compute`, `postprocess`, or `verify`. When the *type* argument is `compute` it is equivalent to calling `runNoGen()` on all study steps. For an Uncertainty Quantification study, `postprocess` generates the response surface for the surrogate functions and `verify` computes verification data and verifies the surrogate function. For all other studies, no action is performed when the *type* argument is `postprocess` or `verify`.

`model.study(<tag>).createAutoSequences(type)` creates an attached solver sequence or job using default solver settings if the solver sequence has not been edited. This command is similar to **Compute** in the COMSOL Desktop. The argument *type* is one of `all`, `jobs`, or `sol`, corresponding to creating both jobs and solver sequences or one of them.

`model.study(<tag>).showAutoSequences(type)` generates a new attached solver sequence or job using default solver settings. This command is similar to **Show Default Solver** in the COMSOL Desktop; that is, it always creates unedited solver sequences. See `createAutoSequences` above for information about the *type* argument.

`model.study(<tag>).getSolverSequences(type)` returns a list of tags for solver sequences (see `model.sol()`) connected to this study. The *type* argument is one of `SolverSequence`, `CopySolution`, `ParametricStore`, `Stored`, `Parametric`, `None`, or `All`.

`model.study(<tag>).setGenConv(boolean)` controls the generation of convergence plots. Use `model.study(<tag>).isGenConv()` to check if convergence plots will be generated.

`model.study(<tag>).setGenIntermediatePlots(boolean)` controls the generation of default plots for intermediate solutions. Use `model.study(<tag>).isGenIntermediatePlots()` to check if plots will be generated for intermediate solutions.

`model.study(<tag>).setGenPlots(boolean)` controls the generation of default plot for the final solution after the last step in a study sequence. Use `model.study(<tag>).isGenPlots()` to check if plots will be generated for the final solution.

`model.study(<tag>).setPlotUndefVals(boolean)` controls the creation of a plot that indicates the location of undefined values such as Inf and NaN. Use `model.study(<tag>).isPlotUndefVals()` to check if plots of undefined values will be generated.

`model.study(<tag>).setStoreSolution(boolean)` inserts a Solution Store node between each study step in a multistep study if set to true. If set to false, Solution Store nodes are only inserted in certain cases. Use

`model.study(<tag>).isStoreSolution()` to check if a Solution Store node is inserted between each study step (it then returns `true`).

`model.study(<tag>).create(<ftag>,type)` creates a new study step of the given type within the specified sequence. The set of allowed values should be limited to study types supported by at least one physics interface present in the model (Stationary, Time, Frequency, and Eigenvalue should always be allowed).

`model.study(<tag>).feature().move(<ftag>,position)` moves the feature `<ftag>` to the zero indexed position `<position>` in the list.

The following two lines sets the outputs to store from a study to the selections with tags `sel1` and `sel2`:

```
model.study(<tag>).setEntry("outputmap", "solid", "selection");
model.study(<tag>).setEntry("outputselectionmap", "solid", "sel1;sel2");
```

See [setEntry\(\)](#) for more information about how it works.

`boolean model.study(<tag>).feature(<ftag>).loadFile(String filePath)` and `boolean model.study(<tag>).feature(<ftag>).SaveFile(String filePath)` both work if the type of feature is a Parametric Sweep and then loads or saves information concerning either All Combinations or Specified Combinations. These methods return `false` if the type of feature is not a Parametric Sweep. They also return `false` if the operation is not successful.

`step = model.study(<tag>).feature(<ftag>)` obtains a reference to a specified study step.

`step.discardData()` discards any experimental data imported into a Parameter Estimation study step.

`step.discretization(<phys>,<discr>)` assigns discretization for a physics interface. The string `<phys>` is the tag of a physics interface. The string `<discr>` is a tag of a discretization feature under a physics mode or the string `physics` to use the discretization settings in the physics interface itself.

`step.exportData(<filePath>)` exports any experimental data previously imported to a Parameter Estimation study step to the specified file.

`step.importData(<filePath>)` imports experimental data into a Parameter Estimation study step from the specified `<filePath>`.

`step.loadFile(<filePath>)` loads parameter values from the specified `<filePath>` into a Parametric Sweep study step.

`step.mesh(<geom>,<mesh>)` specifies which mesh to use for geometry `<geom>` in the model.

`step.refresh()` refreshes, that is, reloads, any experimental data files used by a Parameter Estimation study step.

`step.saveFile(<filePath>)` saves parameter values from a Parametric Sweep study step to `<filePath>`.

`step.setSolveFor(<entityPath>,solve)` controls whether degrees of freedom defined by the specified entity will be solved for in the study step. When `solve` is set to `false`, the degrees of freedom will not be solved for; when set to `true`, they will be solved for only if the entity can generate equations for the study-step type. The correct `<entityPath>` argument corresponding to a model entity is given by `<entity>.resolveModelPath()`.

`step.type()` returns the study type.

`step.discretization(<phys>)` returns the tag of the discretization feature under a physics interface with tag `<phys>` that specifies the shape functions to use for the physics interface in this study step, or the string `physics` to indicate that the discretization settings in the physics interface itself are used.

`step.mesh(<geom>)` returns the tag of the mesh to be used in geometry `<geom>` in this study step.

`step.solveFor(<entityPath>)` returns true if degrees of freedom defined by the specified entity will be solved for in the study step. The return value accounts for both the state of the `activate` property and whether the entity

allows the study step type. The correct `<entityPath>` argument corresponding to a model entity is given by `<entity>.resolveModelPath()`.

`step.mglevel.create(<mglevel>)` adds a (coarser) multigrid level to a study.

`step.mglevel(<mglevel>).mesh(<geom>, <mesh>)` specifies a mesh for the multigrid level. The set of allowed values must, in addition to the actual meshes, include an option “from parent”. This should be the default choice and indicates that the multigrid level uses the same mesh as the parent study.

`step.mglevel(<mglevel>).discretization(<physpath>, <discr>)` assigns discretization for a multigrid level. The string `<physpath>` is the path of a physics interface. The string `<discr>` is a tag of a discretization feature under a physics mode. The default `<discr>` is the physics interface tag. It can be changed to the tag of a discretization node under a physics interface.

`step.mesh(<geom>)` returns the mesh selected for the given geometry.

`step.mglevel(<mglevel>).mesh(<geom>)` returns the mesh for the selected multigrid level and geometry.

`step.mglevel(<mglevel>).discretization(<discpath>)` returns activation status of a discretization feature.

EXAMPLE

The following code sets up a study sequence to analyze the influence of structural deformation on a waveguide with a numerical port boundary condition. It consists of three steps: stationary structural mechanics followed by an eigenvalue study for the port and finally a wave propagation problem solved with manual multigrid levels (to get nested meshes).

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
model.component("comp1").geom().create("geom1", 3);
model.component("comp1").geom("geom1").create("blk1", "Block");
model.component("comp1").geom().run();
model.component("comp1").mesh().create("mesh1", "geom1");
model.component("comp1").mesh().create("mesh2", "geom1");
model.component("comp1").mesh().create("mesh3", "geom1");

model.physics().create("rfw1", "ElectromagneticWaves", "geom1");
model.study().create("seq1");
Study s1 = model.study("seq1");
s1.create("struct", "Stationary");
s1.feature("struct").mesh("geom1", "mesh1");
s1.create("port", "BoundaryModeAnalysis");
s1.feature("port").set("PortName", "port1");
s1.feature("port").mesh("geom1", "mesh2");
s1.create("wave", "Frequency");
s1.feature("wave").mesh("geom1", "mesh2");
s1.feature("wave").mglevel().create("mgl1");
s1.feature("wave").mglevel().create("mgl2");
s1.feature("wave").mglevel("mgl2").mesh("geom1", "mesh3");
model.physics("rfw1").create("mgl1", "Discretization");
model.physics("rfw1").feature("mgl1").set("order", "1");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
model.component('comp1').geom.create('geom1', 3);
model.component('comp1').geom('geom1').create('blk1', 'Block');
model.component('comp1').geom.run;
model.component('comp1').mesh.create('mesh1', 'geom1');
model.component('comp1').mesh.create('mesh2', 'geom1');
model.component('comp1').mesh.create('mesh3', 'geom1');
```

```

model.physics.create('rfw1', 'ElectromagneticWaves', 'geom1');
model.study.create('seq1');
s1 = model.study('seq1');
s1.create('struct', 'Stationary');
s1.feature('struct').mesh('geom1', 'mesh1');
s1.create('port', 'BoundaryModeAnalysis');
s1.feature('port').set('PortName', 'port1');
s1.feature('port').mesh('geom1', 'mesh2');
s1.create('wave', 'Frequency');
s1.feature('wave').mesh('geom1', 'mesh2');
s1.feature('wave').mglevel.create('mgl1');
s1.feature('wave').mglevel.create('mgl2');
s1.feature('wave').mglevel('mgl2').mesh('geom1', 'mesh3');
model.physics('rfw1').create('mgl1', 'Discretization');
model.physics('rfw1').feature('mgl1').set('order', '1');

```

In this case, the only settings that must be applied in the study members of other features are the ones relating to multigrid levels. The physics interfaces' equation form is by default set to automatic, which means that they respond suitably to the study type each time an extended mesh (xmesh) is created.

SEE ALSO

[model.batch\(\)](#), [model.physics\(\)](#), [model.sol\(\)](#)

model.thermodynamics()

Create constants and functions interfacing to a thermodynamics property package.

SYNTAX

Creating a thermodynamics property package feature.

```
model.thermodynamics().create(<ptag>, "PropertyPackage");
```

Setting and getting properties in a thermodynamics property package feature:

```

model.thermodynamics().feature(<ptag>).set(<prop>, <value>);
model.thermodynamics().feature(<ptag>).getString(<prop>);

```

TABLE 2-138: PROPERTIES SUPPORTED BY A THERMODYNAMICS PROPERTY PACKAGE.

NAME	TYPE	DESCRIPTION
manager_id	String	Thermodynamics manager ID.
manager_version	String	Thermodynamics manager version.
package_id	String	Thermodynamics package ID.
package_desc	String	Thermodynamics package description. Only for display in the user interface.

```
model.thermodynamics().feature(<ptag>).storePersistenceData();
```

Calling this method after a property package feature has been created, and `manager_id` and `package_id` have been set, stores information about how the thermodynamics package was created in the COMSOL model. If the model is later opened on a computer with the thermodynamics manager installed but without the property package, this information can be used to create the required property package.

```
model.thermodynamics().feature(<ptag>).create(<ftag>, <type>);
```

Creates a thermodynamics constant, function, or flash calculation feature. Possible types are `CompoundConstant`, `TemperatureDependentProperty`, `PressureDependentProperty`, `OnePhaseProperty`, `TwoPhaseProperty`, and `FlashCalculationProperty`.

```

model.thermodynamics().feature(<ptag>).feature(<ftag>).set(<prop>, <value>);
model.thermodynamics().feature(<ptag>).feature(<ftag>).getString(<prop>);
model.thermodynamics().feature(<ptag>).feature(<ftag>).getStringArray(<prop>);

```

```
model.thermodynamics().feature(<ptag>).feature(<ftag>).getStringMatrix(<prop>);
```

Set and get properties in a thermodynamics constant, function, or flash calculation feature.

TABLE 2-139: PROPERTIES SUPPORTED BY ALL THERMODYNAMICS CONSTANT, FUNCTION, AND FLASH CALCULATION FEATURES.

NAME	TYPE	DESCRIPTION
prop_basis	String	Basis ("mass" or "mole") for evaluated properties. Only relevant for some properties.

Thermodynamics Constant Features

The thermodynamics constant features are features with the type "CompoundConstant", and are used to define constants that get their value from a thermodynamics package.

TABLE 2-140: PROPERTIES SUPPORTED BY THERMODYNAMICS CONSTANT FEATURES.

NAME	TYPE	DESCRIPTION
funcname	String	Name of constant; that is, a variable name that can be used in expressions.
compound	String	Identifier for compound in the thermodynamics property package.
property	String	Identifier for property in the thermodynamics property package.
value	String	Numerical value of constant. This value is only displayed in the user interface. When the constant is evaluated in an expression, the value is obtained by calling the thermodynamics property package.

Thermodynamics Function Features

A thermodynamics function feature defines a function that can be used to evaluate properties that depend, for example, on temperature or pressure. Different types are TemperatureDependentProperty, PressureDependentProperty, OnePhaseProperty, and TwoPhaseProperty.

TABLE 2-141: PROPERTIES SUPPORTED BY THERMODYNAMICS FUNCTION FEATURES.

NAME	TYPE	DESCRIPTION
funcname	String	Name of function.
derivatives	String array	Names of partial derivatives of function.
compounds	String array	Identifiers for compounds in the thermodynamics property package.
comp_basis	String	Basis ("mass" or "mole") for amounts of compounds. Only relevant for functions of type "OnePhaseProperty" or "TwoPhaseProperty."
property	String	Identifier for the property to evaluate in the thermodynamics property package.
phase	String	Identifier for phase in the thermodynamics property package. Only for features of type "OnePhaseProperty."
phase1	String	Identifier for first phase in the thermodynamics property package. Only for features of type "TwoPhaseProperty."
phase2	String	Identifier for second phase in the thermodynamics property package. Only for features of type "TwoPhaseProperty."
args	String matrix	Names, units, and descriptions for the function arguments. Only for display in the user interface.
unit	String	Unit of the function. Only for display in the user interface.

Functions of type TemperatureDependentProperty and PressureDependentProperty have a single argument, which is the temperature or pressure, respectively.

Functions of type OnePhaseProperty have temperature and pressure as their first two arguments. If there is more than one compound, there are additional arguments for the fraction of each compound.

Functions of type TwoPhaseProperty have temperature and pressure as their first two arguments. If there is more than one compound, there are additional arguments for the fraction of each compound in each phase.

Thermodynamics Flash Calculation Features

Thermodynamics flash calculation features are used as an interface for flash calculations, which take amounts of different compounds and two conditions (for example, temperature and pressure) as input and compute the fraction of each compound that is present in each phase.

TABLE 2-142: PROPERTIES SUPPORTED BY THERMODYNAMICS FLASH CALCULATION FEATURES.

NAME	TYPE	DESCRIPTION
compounds	String array	Identifiers for compounds in the thermodynamics property package.
cond1	String	First flash condition.
cond2	String	Second flash condition.
temperature	String	Name of function evaluating the temperature (if temperature is not one of the flash conditions).
pressure	String	Name of function evaluating the pressure (if pressure is not one of the flash conditions).
inphase	String	Base name for functions evaluating presence of each phase. Function names for each phase are formed by appending _<phase> to the base name.
amounts	String	Base name for functions evaluating amount of each phase. Function names for each phase are formed by appending _<phase> to the base name.
composition	String	Base name for functions evaluating fraction of each compound in each phase. Function names are formed by appending _<phase>_<compound> to the base name.
soltype	String	Solution type (“undefined”, “normal,” or “retrograde”).
args	String matrix	Names, units, and descriptions for the function arguments. Only for display in the user interface.
phases	String array	Names of all phases. Only for display in the user interface. The phases used in the flash calculation are determined by the thermodynamics property package.

Each flash calculation feature defines a number of functions. All of the functions take the values of two flash conditions as their first two arguments, followed by arguments for the total amount of each compound.

SEE ALSO

[model.func\(\)](#)

model.unitSystem()

Unit systems.

SYNTAX

```
UnitSystem us = model.unitSystem().create(<tag>);
us.baseUnit().create(<tag>, <symbol>, <quantity>)
us.derivedUnit().create(<tag>, <units>, <powers>);
us.additionalUnit().create(<tag>, <dim>);
model.unitSystem().builtInTags();

us.baseUnit(<tag>);
us.derivedUnit(<tag>);
us.additionalUnit(<tag>)
us.derivedUnit(<tag>).aliases();
us.baseUnit(<tag>).dimension();
us.derivedUnit(<tag>).quantity();
us.derivedUnit(<tag>).offset();
us.derivedUnit(<tag>).scale();
us.derivedUnit(<tag>).symbol();
us.derivedUnit(<tag>).definition(<units>, <powers>);
us.additionalUnit(<tag>).aliases(<aliases>);
us.additionalUnit(<tag>).quantity(<quantity>);
us.additionalUnit(<tag>).offset(<offset>);
us.additionalUnit(<tag>).scale(<scale>);
us.additionalUnit(<tag>).offset(<offset>);
us.additionalUnit(<tag>).symbol(<symbol>);
```

DESCRIPTION

`model.unitSystem().create(<uname>)` creates a unit system *<uname>*.

`us.baseUnit().create(<tag>, <symbol>, <quantity>)` creates a base unit for the quantity *<quantity>*, tagged *<tag>* with the symbol *<symbol>*. The quantity is any of the seven base dimensions (length, mass, time, current, temperature, substance, and intensity).

`us.derivedUnit().create(<tag>, <units>, <powers>)` creates a new derived unit tagged *<tag>* and derived from the units in *<units>* each to the power of the powers in *<powers>*.

`us.derivedUnit(<tag>).definition(<units>, <powers>)` sets the definition of a derived unit in powers of other units. The resulting dimension must agree with any previously specified dimension for this unit. Use the `create` method to define a dimension from the derived units.

`us.additionalUnit().create(<tag>, <dim>)` creates a new additional unit.

All methods below are valid for all units, no matter what unit list they belong to. Furthermore, only the set methods are described here, but there is also a corresponding get method.

`model.unitSystem().builtInTags()` returns the tags of the built-in unit systems. The method `model.unitSystem().tags()` returns the tags of the user-defined unit systems. Both sets of tags can be used to retrieve the unit system using `model.unitSystem(<tag>)`.

`us.additionalUnit(<tag>).aliases(<aliases>)` sets alternative names for the unit that can be used in unit expressions.

`us.additionalUnit(<tag>).quantity(<quantity>)` assigns a physical quantity to the given unit.

`us.additionalUnit(<tag>).scale(<scale>)` sets the scale of the additional unit.

`us.derivedUnit(<tag>).symbol(<symbol>)` sets the symbol of the derived unit.

`us.derivedUnit(<tag>).offset(<offset>)` sets the offset of the derived unit.

NOTES

You can set the base unit system for the entire model using `model.baseSystem(<utag>)` or separately for each component node using `model.component(<tag>).baseSystem(<utag>)`.

The SI system is read only and always created by default.

EXAMPLE

Create a `cgs2` unit system with the base unit for length set to centimeter (cm). Also add meter/second (m/s) as a derived unit for speed and degrees Celsius as an additional unit for temperature:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
UnitSystem us = model.unitSystem().create("cgs2");
model.baseSystem("cgs2");
us.baseUnit().create("centimeter", "cm", "length");
us.derivedUnit().create("meter_per_second", new int[]{1,0,-1,0,0,0,0,0});
Unit du = us.derivedUnit("meter_per_second");
du.definition(new String[]{"meter", "second"}, new int[]{1,-1,0,0,0,0,0,0});
Unit au = us.additionalUnit().create("celsius", new int[]{0,0,0,0,1,0,0,0});
au.offset(273.15);
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
us = model.unitSystem.create('cgs2');
model.baseSystem('cgs2');
us.baseUnit.create('centimeter', 'cm', 'length');
us.derivedUnit.create('meter_per_second', [1,0,-1,0,0,0,0,0]);
du = us.derivedUnit('meter_per_second');
du.definition({'meter', 'second'}, [1,-1,0,0,0,0,0,0]);
au = us.additionalUnit.create('celsius', [0,0,0,0,1,0,0,0]);
au.offset(273.15);
```

SEE ALSO

[model.physics\(\)](#)

model.variable()

Create, define, and remove variables.

SYNTAX

```
model.variable().create(<tag>);
model.variable(<tag>).set(<var>, <expr>);
model.variable(<tag>).set(<var>, <expr>, <descr>);
model.variable(<tag>).descr(<var>, <descr>);
model.variable(<tag>).remove(<var>);
model.variable(<tag>).clear();
model.variable(<tag>).model(<mtag>);

model.variable(<tag>).varnames();
model.variable(<tag>).get(<var>);
model.variable(<tag>).descr(<var>);
model.variable(<tag>).model();
model.variable(<tag>).scope();
model.variable(<tag>).loadFile(tempFile, ...);
model.variable(<tag>).saveFile(tempFile, ...);
```



For variables on the component level, use `model.component(<ctag>).variable().create(<tag>)`, and so on, instead of the syntax above for global variables.

DESCRIPTION

`model.variable(<tag>)` returns a variable collection. Each variable collection can contain several variables, but only one selection.

`model.variable().create(<tag>)` creates a variables node with tag `<tag>`.

`model.variable(<tag>).set(<var>, <expr>)` defines the variable `<var>` by the expression `<expr>`.

`model.variable(<tag>).set(<var>, <expr>, <descr>)` defines a variable and gives it a description.

`model.variable(<tag>).descr(<var>, <descr>)` defines a description for the variable `<var>`.

`model.variable(<tag>).model(<mtag>)` sets the model component node.

`model.variable(<tag>).selection().named(<seltag>)` assigns the variable node to the named selection `<seltag>`.

`model.shape(<tag>).selection().set(...)` defines a local selection that assigns the variable collection to geometric entities. Before assigning a selection, the variable's model must be set using

`model.variable(<tag>).model(<mtag>)`. Only the global selection and selections on a geometry in the model can be used. For a complete list of methods available under `selection()`, see [Selections](#).

`model.variable(<tag>).remove(<var>)` removes a variable from the variable collection.

`model.variable(<tag>).clear()` removes all variables from the variable collection.

`model.variable(<tag>).varnames()` returns the names of all expressions as a string array.

`model.variable(<tag>).get(<var>)` returns the variable value as a string.

`model.variable(<tag>).descr(<var>)` returns the variable description as a string.

`model.variable(<tag>).model()` returns the model component node tag.

`model.variable(<tag>).scope()` returns the fully qualified scope name.

`model.variable(<tag>).selection().named()` returns the selection tag as a string.

`model.variable(<tag>).selection().getType()` returns domain information. For available methods, see [model.selection\(\)](#).

For `model.param().loadFile()` and `model.param().saveFile()`, see [The loadFile and saveFile Methods](#).

EXAMPLES

Define the expression `e` as `x+1` in Domains 1 and 2 and as `x-1` in Domain 3.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
model.component("comp1").geom().create("geom1",3);
model.component("comp1").geom("geom1").create("blk1", "Block");
model.component("comp1").geom("geom1").run();
model.component("comp1").variable().create("e1").set("e", "x+1");
model.component("comp1").variable("e1").selection().geom("geom1",2);
model.component("comp1").variable("e1").selection().set(new int[] {1,2});
model.component("comp1").variable().create("e2").set("e", "x-1");
model.component("comp1").variable("e2").selection().geom("geom1",2);
model.component("comp1").variable("e2").selection().set(3);
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
model.component.geom.create('geom1',3);
model.component.geom('geom1').create('blk1', 'Block');
```

```

model.component.geom('geom1').run;
model.component.variable('e1').set('e','x+1');
model.component.variable('e1').selection.geom('geom1',2);
model.component.variable('e1').selection.set([1,2]);
model.component.variable.create('e2').set('e','x-1');
model.component.variable('e2').model('mod1');
model.component.variable('e2').selection().geom('geom1',2);
model.component.variable('e2').selection().set(3);

```

SEE ALSO

[model.selection\(\)](#)

model.view()

Create and manage views.



For views defined on the global level, under Results, omit `component(<ctag>)`. from the syntax such as `model.component(<ctag>).view().create(<vtag>,<gtag>)` in the syntax examples below.

SYNTAX

```

model.component(<ctag>).view().create(<vtag>,<gtag>)
model.component(<ctag>).view().create(<vtag>,<viewdim>)
model.component(<ctag>).view().create(<vtag>,<gtag>,<workplane>)
model.component(<ctag>).view(<vtag>).set(<pname>,<pvalue>)
model.component(<ctag>).view(<vtag>).getType(<pname>)

model.component(<ctag>).view(<vtag>).axis().set(<pname>,<pvalue>)
model.component(<ctag>).view(<vtag>).axis().getType(<pname>)
model.component(<ctag>).view(<vtag>).camera().set(<pname>,<pvalue>)
model.component(<ctag>).view(<vtag>).camera().getType(<pname>)
model.component(<ctag>).view(<vtag>).copyToGeometry()
model.component(<ctag>).view(<vtag>).copyToWorkPlane()
model.component(<ctag>).view(<vtag>).geom()
model.component(<ctag>).view(<vtag>).getHiddenEntities()
model.component(<ctag>).view(<vtag>).getHiddenEntities(<mesh>)
model.component(<ctag>).view(<vtag>).getSDim()
model.component(<ctag>).view(<vtag>).isCurrent()
model.component(<ctag>).view(<vtag>).light().create(<ltag>,<ltype>)
model.component(<ctag>).view(<vtag>).light(<ltag>).set(<pname>,<pvalue>)
model.component(<ctag>).view(<vtag>).light(<ltag>).getType(<pname>)
model.component(<ctag>).view(<vtag>).hideObjects().create(<htag>)
model.component(<ctag>).view(<vtag>).hideObjects(<htag>).set(<pname>,<pvalue>)
model.component(<ctag>).view(<vtag>).hideObjects(<htag>).getType(<pname>)
model.component(<ctag>).view(<vtag>).hideEntities().create(<htag>)
model.component(<ctag>).view(<vtag>).hideEntities(<htag>).set(<pname>,<pvalue>)
model.component(<ctag>).view(<vtag>).hideEntities(<htag>).getType(<pname>)
model.component(<ctag>).view(<vtag>).hideEntities(<htag>).image()
model.component(<ctag>).view(<vtag>).hideMesh().create(<htag>)
model.component(<ctag>).view(<vtag>).hideMesh(<htag>).set(<pname>,<pvalue>)
model.component(<ctag>).view(<vtag>).hideMesh(<htag>).getType(<pname>)
model.component(<ctag>).view(<vtag>).hideMesh(<htag>).image()

```

DESCRIPTION

View Settings

`model.component(<ctag>).view()` returns a list of view that can be used when viewing geometry/mesh and plot groups. Each view has an axis and some properties on the top level. In 3D, a view also has a camera and a list of lights. All views also have a list of hide features.

`model.component(<ctag>).view().create(<vtag>,<gtag>)` creates a view tied to the geometry with the given tag. The dimension of the view is the same as the dimension for the geometry.

`model.component(<ctag>).view().create(<vtag>,<viewdim>)` creates a view with the given tag for the given dimension (1, 2, or 3). These views are not tied to any geometry and show up under the Views node under Results in the COMSOL Desktop.

`model.component(<ctag>).view().create(<vtag>,<gtag>,<workplane>)` creates a view tied to the work plane with the given tag in the geometry sequence with the given tag. The dimension of the view is 2.

`model.component(<ctag>).view(<vtag>).set(<pname>,<pvalue>)` sets the given property to the given value.

`model.component(<ctag>).view(<vtag>).geom()` returns the geometry sequence (or null for the views not associated with a geometry).

`model.component(<ctag>).view(<vtag>).getSDim()` returns the view's space dimension.

`model.component(<ctag>).view(<vtag>).getType(<pname>)` returns the property with the given name of type *Type*.

TABLE 2-143: VIEW PROPERTIES.

NAME	VALUE	DIMENSIONS	DESCRIPTION
default	true false	1, 2, 3	If this is the default view to use when viewing the geometry and mesh.
headlight	true false	3	If the light in the camera should be turned on.
locked	true false	1, 2, 3	If the settings should be updated from interactive changes or not.
rendermesh	true false	2, 3	If mesh rendering should be used (default: true).
scenelight	true false	3	If the background lights as specified by the added lights should be turned on (default: true).
showmaterial	true false	2, 3	If the material color and texture should appear or not (default: false).
showselection	true false	2, 3	If the selection colors should appear or not (default: true).
showunits	true false	1, 2, 3	If the axis units should appear or not.
transparency	true false	3	If the plot show be transparent (default: false).
transparencylevel	0 to 1	3	The transparency level, where 0 is fully opaque and 1 is fully transparent (default: 0.5).
uniformblending	true false	3	If the transparent plot should use Fresnel transmittance (default: false).
uniformblendinglevel	0 to 1	3	The Fresnel transmittance (default: 0.5).
wireframe	true false	3	If wireframe rendering should be used.
xscale	double	2, 3	Scaling factor for x-axis when using a manual view scale.
yscale	double	2, 3	Scaling factor for y-axis when using a manual view scale.
zscale	double	3	Scaling factor for z-axis when using a manual view scale.

Axis Settings

The axis settings apply to 2D views with an *x*- and a *y*-axis.

`model.component(<ctag>).view(<vtag>).axis().set(<pname>,<pvalue>)` sets the given axis property to the given value. Which axis properties that are available in the different dimensions are given in the table below.

`model.component(<ctag>).view(<vtag>).axis().getType(<pname>)` returns the axis property with the given name.

TABLE 2-144: AXIS PROPERTIES.

NAME	VALUE	DIMENSIONS	DESCRIPTION
auto	on off	1, 2, 3	Set if axis settings should be automatically stored and updated from interactive changes using mouse and toolbar buttons.
autocontext	autofit isotropic anisotropic	2	The automatic view scaling when <code>viewscaletype</code> is set to automatic: autofit (the default), isotropic, or anisotropic.
equal	on off	2, 3	Should the same scaling be used for all directions.
logx	on off	1	Should log scale be used for the x-axis.
logy	on off	1	Should log scale be used for the y-axis.
manualspacing	on off	2	Should manual spacing be used for x and y grid lines.
manuallimits	on off	1, 2, 3	Should manual axis limits be used. If not a zoom extents is performed each time something is plotted into the axis.
viewscaletype	none manual automatic	2	The view scale specification: none (the default), manual, or automatic.
xextra	double array	2	An array with extra x grid lines.
xmax	double	1, 2, 3	The maximum x-coordinate.
xscale	double	2	A positive scalar value for the x scale when <code>viewscaletype</code> is set to manual.
xspacing	double	2	Manual spacing for x grid lines.
xweight	double	2	A positive scalar value for the x weight when <code>autocontext</code> is set to anisotropic.
yextra	double array	2	An array with extra y grid lines.
ymin	double	2, 3	The minimum y-coordinate.
ymax	double	2, 3	The maximum y-coordinate.
yscale	double	2	A positive scalar value for the yscale when <code>viewscaletype</code> is set to manual.
yspacing	double	2	Manual spacing for y grid lines.
yweight	double	2	A positive scalar value for the yweight when <code>autocontext</code> is set to anisotropic.
zmin	double	3	The minimum z-coordinate.
zmax	double	3	The maximum z-coordinate.

Camera Settings

These settings apply to the camera for 3D views.

	Some camera settings are affected by the preference settings for default 3D views.
---	--

`model.component(<ctag>).view(<vtag>).camera().set(<pname>,<pvalue>)` sets the given camera property to the given value.

`model.component(<ctag>).view(<vtag>).camera().getType(<pname>)` returns the camera property with the given name.

TABLE 2-145: CAMERA PROPERTIES.

NAME	VALUE	DESCRIPTION
autocontext	isotropic anisotropic	Use an isotropic automatic cube scaling or an anisotropic automatic block scaling with x, y, and z direction relative scales.
autoupdate	true false	Use automatic update of the view when viewscaletype is automatic.
projection	perspective orthographic	Use perspective or orthographic projection.
manualspacing	on off	Should manual spacing be used for grid lines.
position	double array	The position of the camera.
target	double array	The point the camera looks at.
up	double array	The up direction.
rotationpoint	double array	The center of rotation.
viewscaletype	none automatic manual	Control the view scale to achieve a suitable axis scaling (default: None).
xextra	double array	An array with extra x grid lines.
xspacing	double	Manual spacing for x grid lines.
xweight	double	Relative weight in the x direction for anisotropic automatic view scale.
yextra	double array	An array with extra y grid lines.
yspacing	double	Manual spacing for y grid lines.
yweight	double	Relative weight in the y direction for anisotropic automatic view scale.
zextra	double array	An array with extra z grid lines.
zspacing	double	Manual spacing for z grid lines.
zoomanglefull	double	The full field of view angle in degrees.
zweight	double	Relative weight in the z direction for anisotropic automatic view scale.

Lighting Settings

These settings control the different types of lighting — direction light, spotlight, headlight, and point light — that you can add to a 3D view.

`model.component(<ctag>).view(<vtag>).light().create(<ltag>,<ltype>)` creates a light with the given tag and type. `<ltype>` can be any of 'DirectionalLight', 'PointLight', 'SpotLight', and 'HeadLight'.

`model.component(<ctag>).view(<vtag>).light(<ltag>).set(<pname>,<pvalue>)` sets the given light property to the given value. Different properties are available for the different types of lights according to the table below.

`model.component(<ctag>).view(<vtag>).light(<ltag>).getType(<pname>)` returns the light property with the given name.

TABLE 2-146: LIGHT PROPERTIES.

NAME	VALUE	LIGHT TYPES	DESCRIPTION
angleattenuationscale	positive scalar	SpotLight	Angle attenuation scale factor for spotlights.
castlight	boolean	all	Cast light.
castshadows	boolean	all	Cast shadows, if castlight is set to true.
castshadowsonfloor	boolean	all	Cast shadows on floor.

TABLE 2-146: LIGHT PROPERTIES.

NAME	VALUE	LIGHT TYPES	DESCRIPTION
color	string or RGB triplet	all	The color of the light.
cameracoord	on off	all	If the light should be defined in the camera coordinate system or the world coordinate system.
direction	double array	DirectionalLight, SpotLight	The direction from which the light shines or is directed at.
distanceattenuationscale	positive scalar	PointLight, SpotLight	Distance attenuation scale factor for spotlights and point lights.
position	double array	PointLight, SpotLight	The position from which the light shines.
spreadangle	double	SpotLight	The spread angle for the light.

The following table lists the light properties for 3D views:

TABLE 2-147: LIGHT PROPERTIES IN 3D VIEWS.

NAME	VALUE	DEFAULT	DESCRIPTION
ambientcolor	custom black blue cyan gray green magenta red white yellow none	white	The coloring of the intersection highlight, if globalambient is set to true.
castshadows	true false	true	Cast shadows from lights when direct shadows are active.
globalambient	true false	true	Include ambient light.
scenelight	true false	true	Include scene light. Must be true for the other properties to be active.
showmarker	true false	true	Show light marker.
totambient	double (0–1)	0.3	The ambient intensity.
totlightintensity	double (0–1)	1	The light intensity.
usediffuse	true false	true	Include diffuse light.
usespecular	true false	true	Include specular light.

Visual Effects Settings: Ambient Occlusion, Shadows, and Gamma Correction

The following table lists the properties for 3D views that control visual effects such as ambient occlusion, direct and floor shadows, and gamma correction:

TABLE 2-148: VISUAL EFFECTS PROPERTIES IN 3D VIEWS.

NAME	VALUE	DEFAULT	DESCRIPTION
displayoutput	boolean	false	Use gamma correction.
displayoutput brightness	double (0–1)	0	The brightness, if displayoutputpreset is set to custom.
displayoutputcontrast	double (0–4)	1	The contrast, if displayoutputpreset is set to custom.
displayoutputexposure	double (0–2)	0.0	The exposure value, if displayoutputpreset is set to custom.

TABLE 2-148: VISUAL EFFECTS PROPERTIES IN 3D VIEWS.

NAME	VALUE	DEFAULT	DESCRIPTION
displayoutputgamma	double (1–4)	2.2	The gamma value, if displayoutputpreset is set to custom.
displayoutputhue	double (0–1)	0	The hue, if displayoutputpreset is set to custom.
displayoutputpreset	default custom	default	Use a preset gamma correction or specify a custom gamma correction.
displayoutputsaturation	double (0–4)	1	The saturation, if displayoutputpreset is set to custom.
displayoutputtonemap	Integer string ("0" to "4")	"1" (Clamp)	The tone map to use if displayoutputpreset is set to custom. The "0" to "4" correspond to Filmic, None (the default), Extreme, Bright, and Natural, respectively.
displayoutputvibrance	double (0–1)	0	The
flooreffect	boolean	false	Add floor shadows.
flooreffectambientocclusion	boolean	true	Use ambient occlusion.
flooreffectangle	double	5	Pitch angle for the floor shadows (in degrees), if flooreffectnormalsettings is set to angle.
flooreffectnormal	double array	{0, 0, 1}	The floor effect normal vector, if flooreffectnormalpreset is set to custom.
flooreffectnormalpreset	xyplane yzplane zxplane yzplane zyplane xzplane custom	xyplane	The floor shadow normal preset, if flooreffectnormalsettings is set to explicit.
flooreffectnormalsettings	angle explicit	angle	The floor effect normal setting: derive from pitch angle or specify explicitly.
flooreffectoffset	double	0	Additional floor shadow effect offset; a positive number moved the floor farther away.
flooreffectorigin	double array	{0, 0, 0}	The floor effect origin, if flooreffectoriginsettings is set to explicit.
flooreffectoriginsettings	farthestvertex boundsphere boundingbox explicit	farthestvertex	Floor origin settings: offset by farthest vertex, bounding sphere, bounding box, or explicit.
flooreffectshadow	boolean	true	Use direct floor shadows
flooreffectshadowblur	double (0–1)	0	Additional blur, if flooreffectshadowblur is set to true.
flooreffecttransparency	double (0–1)	0	The transparency of the floor shadows.
shadowmapping	boolean	false	Add direct shadows.

TABLE 2-148: VISUAL EFFECTS PROPERTIES IN 3D VIEWS.

NAME	VALUE	DEFAULT	DESCRIPTION
shadowmappingaccurate depthcomparison	boolean	false	Use accurate depth comparison, when shadowmappingpreset is set to preset.
shadowmappingbias settings	default custom	default	Use the default bias settings for direct shadows, or use custom settings.
shadowmapping constantdepthbias	double (0–1)	0.001	Constant depth bias, if shadowmappingbiassettings is set to custom.
shadowmappinglimit lightviewfrustums	boolean	false	Limit light view frustums, when shadowmappingpreset is set to preset.
shadowmappingmulti samplingeverywhere	boolean	true	Use multisampling everywhere, when shadowmappingpreset is set to preset.
shadowmappingnumber ofoccludersamples	integer (1–200)	8	Number of occluder samples, when shadowmappingpreset is set to preset.
shadowmappingnormal awaresmoothing	boolean	false	Use normal-aware smoothing, if shadowmappingaccuratedepthcomparison is set to true.
shadowmappingnormal offsetbias	double (0–1)	0.003	Normal offset bias, if shadowmappingbiassettings is set to custom.
shadowmappingnumber ofsamples	integer (1–200)	16	Number of samples, when shadowmappingpreset is set to preset.
shadowmappingpreset	low medium high custom	low	Quality preset for direct shadows: low, medium, or high quality, or use custom settings.
shadowmapping resolution	double (0–1)	0.5	Resolution, when shadowmappingpreset is set to preset.
shadowmappingslope depthbias	double (0–1)	0.001	Slope depth bias, if shadowmappingbiassettings is set to custom.
shadowmapping softness	double (0–3)	0.5	Strength of the direct shadows.
shadowmapping strength	double (0–1)	0.5	Softness of the direct shadows.
ssao	boolean	false	Add ambient occlusion.
ssaokernelrotations texturewidth	double (0–8)	4	Kernel rotations texture width, when ssaopreset is set to custom.
ssaomagnitude	double (0–10)	1	Shadow strength for ambient occlusion.
ssaonormalaware smoothing	boolean	false	Add normal-aware smoothing for ambient occlusion, when ssaopreset is set to custom.
ssaonsamples	integer (1–300)	64	Number of samples, when ssaopreset is set to custom.

TABLE 2-148: VISUAL EFFECTS PROPERTIES IN 3D VIEWS.

NAME	VALUE	DEFAULT	DESCRIPTION
ssaopreset	low medium high custom	medium	Quality preset for ambient occlusion: low, medium, or high quality, or use custom settings.
ssaoradiusexplicit	double (>0)	0.4	Maximum distance to occluder, when ssaoradiustype is set to explicit.
ssaoradiusrelative	double (>0)	0.4	Fraction of scene bounding box, when ssaoradiustype is set to relative.
ssaoradiustype	relative explicit	relative	Radius type, for ambient occlusion when ssao is set to true.
ssaoresolution	double (0–1)	1	Resolution, when ssaopreset is set to custom.
ssaoroughness	double (0–1)	1	Roughness, when ssaopreset is set to custom.
ssaosmooth	double (0–8)	2	Smooth, when ssaopreset is set to custom.
ssaosqueeze	double (0–10)	1	Shadow tightness for ambient occlusion.

Hiding Geometry Objects, Geometric Entities, and Imported Meshes

`model.component(<ctag>).view(<vtag>).hideObjects().create(<htag>)` creates a hide feature of geometric objects in the geometry sequence. The API for controlling it is similar to the API for selection in the geometry sequence; see [Geometry Object Selection Methods](#) under `model.geom()`.

`model.component(<ctag>).view(<vtag>).hideEntities().create(<htag>)` creates a hide feature of geometric entities in the analyzed geometry used, for example, for the physics. The API for controlling it is similar to the API for selections on the finalized geometry; see `model.selection()`.

`model.component(<ctag>).view(<vtag>).hideMesh().create(<htag>)` creates a hide feature of geometric entities in the analyzed geometry from an imported mesh used, for example, for the physics. The API for controlling it is similar to the API for selections on the finalized geometry; see `model.selection()`. For example, the following code hides boundary 4 in the geometry based on the mesh in mesh1:

```
// Create mesh hide object in view 1
model.component("comp1").view("view1").hideMesh().create("hide1");
// Select mesh1
model.component("comp1").view("view1").hideMesh("hide1").mesh("mesh1");
// Hide boundary 4 (3D is assumed)
model.component("comp1").view("view1").hideMesh("hide1").geom(2).set(4);
```

`model.component(<ctag>).view(<vtag>).getHiddenEntities()` returns an integer array of hidden entities in each dimension. The entity numbers refer to the entities of the finalized geometry.

`model.component(<ctag>).view(<vtag>).getHiddenEntities(<mesh>)` returns an integer array of hidden entities in each dimension. The entity numbers refer to the entities of specified meshing sequence with the tag `<mesh>` in its current state.

For plotting or exporting images of views with hidden objects, use the

`model.component(<ctag>).view(<vtag>).hideEntities(<htag>).image()` and `model.component(<ctag>).view(<vtag>).hideMesh(<htag>).image()` methods. See [Plotting and Exporting Images](#).

Copying Views

To copy a view to a geometry or work plane, you can use the `copyToGeometry` and `copyToWorkPlane` methods. For example,

```
model.component("comp1").view("view1").copyToWorkPlane();
```

copies the view `view1` to a work plane.

Clipping Tool Settings

These settings control the different types of clipping tools — clip plane, clip box, clip sphere, and clip cylinder — that you can add to a 3D view.

`model.component(<ctag>).view(<vtag>).clip().create(<ctag>,<ctype>)` creates clipping tool with the given tag and type. `<ctype>` can be any of 'ClipPlane', 'ClipBox', 'ClipSphere', and 'ClipCylinder'.

`model.component(<ctag>).view(<vtag>).clip(<ctag>).set(<pname>,<pvalue>)` sets the given clipping tool property to the given value. Different properties are available for the different types of clipping tools according to the table below.

`model.component(<ctag>).view(<vtag>).clip(<ctag>).getType(<pname>)` returns the clipping tool property with the given name.

TABLE 2-149: CLIPPING PROPERTIES FOR THE VIEW FEATURES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>clipapplyclipping</code>	true false	true	Set to false to disable the clipping action for the clipping tools.
<code>clippingactive</code>	true false	true	If the clipping tools are active. The other properties in this table are not available if <code>clippingactive</code> is set to false.
<code>clipedges</code>	true false	true	Clip edges in the geometry.
<code>clipfaces</code>	true false	true	Clip faces in the geometry.
<code>cliphighlightintersection</code>	true false	true	Highlight the intersection when clipping.
<code>clipintersectionhighlightcolor</code>	custom fromtheme black blue cyan gray green magenta red white yellow none	fromtheme	The coloring of the intersection highlight, if <code>cliphighlightintersection</code> is set to true.
<code>clippoints</code>	true false	true	Clip points in the geometry.
<code>clipprimaryhovereffect</code>	true false	true	Use clip primary hover effect.
<code>clipsecondaryhovereffect</code>	true false	false	Use clip contextual hover effect.
<code>clipshowcappedfaces</code>	true false	false	Show cross sections.
<code>clipshowcappedfacescolorize</code>	true false	true	Colorize cross sections, if <code>clipshowcappedfaces</code> is true.
<code>clipshowcappedfacescolorizeper</code>	domain object	domain	Colorize per domain or per geometry object.
<code>clipcappedfaceshighlightoverlappingdomains</code>	true false	true	Highlight overlapping domains, if <code>clipshowcappedfaces</code> is true.
<code>clipcappedfaceshighlightoverlappingdomainscolor</code>	custom fromtheme black blue cyan gray green magenta red white yellow none	fromtheme	The coloring of the overlapping domains, if <code>clipcappedfaceshighlightoverlappingdomains</code> is true.
<code>clipcappedfacestransparency</code>	double (0–1)	0.2	The amount of transparency for the cross sections.

TABLE 2-149: CLIPPING PROPERTIES FOR THE VIEW FEATURES.

NAME	VALUE	DEFAULT	DESCRIPTION
clipcappedfacestransparency enabled	true false	false	Use transparency for the cross sections.
clipshowframes	true false	true	Show the frames of the clipping tools.
clipshowgizmos	true false	true	Show gizmos for the clipping tools.

The following table lists the properties available for the clipping tools. Properties that are not available for all clipping tools have a note about availability in the **Description** column.

TABLE 2-150: CLIPPING PROPERTIES FOR THE CLIPPING TOOLS.

NAME	VALUE	DEFAULT	DESCRIPTION
applyclipping	true false	true	Set to false to disable the clipping action for the clipping tools.
disableclipping	true false	false	Disable the clipping.
drawasintersectionwithgridbox	true false	true	Draw as intersection with grid box.
invertclipping	true false	false	Invert the clipping.
length	double		The length of the clipping tool. For ClipCylinder only.
position	1-by-3 double		The position of the clipping tool.
radius	double		The radius of the clipping tool. For ClipSphere and ClipCylinder only.
showframe	true false	true	Show a frame.
showgizmo	true false	true	Show a gizmo.

Environment Mapping Settings

These settings control the environment mapping that can provide reflections on the model geometry and as a skybox to provide a background in the view. You add environment mapping by, for example, specifying one of the built-in environment maps (images), such as envmap_park2:

```
model.component(<ctag>).view(<vtag>).set("environmentmap", "envmap_park2") adds an environment map that shows a city park. To see its reflection on a model geometry you need to clear any selections and also activate the setting for showing material color and texture:
model.component(<ctag>).view(<vtag>).set("showmaterial", true);
```

You can also add an user-defined environment map by specifying its name followed by the suffix (Local) or (Model), depending on where the environment map is stored.

TABLE 2-151: ENVIRONMENT MAPPING PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
environmentmap	environment map or none	none	The environment map to use. Use none to turn off environment mapping.
environmentquality	low medium high ultra	high	The quality of the environment map.
environmentreflections	true false	true	Add environment reflections.
rotateenvironment	true false	false	Rotate the environment map.
skybox	true false	false	Activate a skybox to make the environment mapping for a background for the view.

TABLE 2-151: ENVIRONMENT MAPPING PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
skyboxblend	double (0–1)	1	The blending of the skybox, if skybox is set to true. 1 means no blending.
skyboxblurriness	double (0–1)	0	The blurriness of the skybox, if skybox is set to true. 0 means no blurriness.
skyboxfov	double (1–180)	110	Skybox field of view, in degrees, if skyboxprojection is set to special.
skyboxprojection	special fromcamera	special	The skybox projection, if skybox is set to true. With fromcamera, the camera's projection is used.
skydirection	positivex negativex positivey negativey positivez negativez	positivey	The sky direction.
skyrotation	skyrotationzero skyrotationninetty skyrotationhundredeighty skyrotationtwohundredseventy	skyrotationzero	Rotate the environment map by 0, 90, 180, or 270 degrees.

SEE ALSO`model.result()`*model.weak()*

Weak form equations.

SYNTAX

```

model.weak().create(<tag>);
model.weak(<tag>).weak(<wlist>);
model.weak(<tag>).weak(<pos>,<wexpr>);
model.weak(<tag>).intRule(<irlist>);
model.weak(<tag>).intRule(<pos>,<irule>);
model.weak(<tag>).condition(<condition>);

```

```

model.weak(<tag>).weak();
model.weak(<tag>).intRule();
model.weak(<tag>).condition();

```

DESCRIPTION

`model.weak(<tag>)` returns the weak form equations with tag `<tag>`.

`model.weak().create(<tag>)` creates weak form equations with tag `<tag>`.

`model.weak(<tag>).weak(<wlist>)` sets the equations. You can supply a single weak expression or a list of weak expressions. `<wlist>` is a string or a string array.

`model.weak(<tag>).weak(<pos>,<wexpr>)` sets the equations at position `<pos>` in the list.

`model.weak(<tag>).intRule(<irlist>)` assigns the integration rules to the weak form equations. The list of integration rules must have the same length as the list of equations, or be of length 1. In the latter case all weak expressions use the same integration rule.

`model.weak(<tag>).intRule(<pos>,<irule>)` sets the integration rule at position `<pos>` in the integration rule list.

`model.weak(<tag>).condition(<condition>)` introduces conditional assembly. The feature is assembled if `<condition>` is true.

`model.weak(<tag>).selection().named(<seltag>)` assigns the weak equations to the named selection `<seltag>`.

`model.weak(<tag>).selection().named(<seltag>)` defines a local selection that assigns the weak equations to geometric entities. Before assigning a selection, the variable's model must be set using

`model.variable(<tag>).model(<mtag>)`. Only the global selection and selections on a geometry in the model can be used. For a complete list of methods available under `selection()`, see [Selections](#).

EXAMPLE

Define the weak expressions `u*test(u)` and `v*test(v)` on the selection `dom1`, using the integration rule `gp1` and the frame `ref`.

Code for Use with Java

```
model.weak().create("w1").selection().named("dom1");
model.weak("w1").intRule("gp1");
model.weak("w1").weak(new String[]{"u*test(u)", "v*test(v)"});
```

Code for Use with MATLAB

```
model.weak.create('w1').selection.named('dom1');
model.weak('w1').intRule('gp1');
model.weak('w1').weak({'u*test(u)', 'v*test(v)'});
```

SEE ALSO

[model.coeff\(\)](#), [model.shape\(\)](#)

Plotting and Exporting Images

Overview

For a number of model entities, two methods for plotting and exporting images are available:

- `image().plot()` plots the model entity in a window. The plotting is available when running a graphics server. The `plot()` method does nothing when run from a model method.
- `image().export()` exports an image of the model entity to file.

To set properties related to the plotting or export of images, use the standard `set` method on the `image()` object.

The following examples show the basic usage of these methods.

This example plots a geometry sequence in a window:

```
model.geom("geom1").image().plot();
```

This example exports a physics interface to a PNG file.

```
model.physics("es").image().set("pngfilename", "C:\physics.png");  
model.physics("es").image().export();
```

The `image()` method is available for many objects in, for example, the lists `model.common()`, `model.cpl()`, `model.coordSys()`, `model.func()`, `model.geom()` (including individual geometry features), `model.material()`, `model.mesh()`, `model.multiphysics()`, `model.pair()`, `model.physics()`, `model.physics(<tag>).feature()`, `model.probe()`, and `model.selection()`.

For mesh features, you can use the `set` property to specify the selection for the `image()` object:

```
model.mesh(<mtag>).feature(<ftag>).image().set("selection", "main")
```

It specifies that the feature's main selection, `model.mesh(<mtag>).feature(<ftag>).selection()`, will appear in the plot. This is the default. Use

```
model.mesh(<mtag>).feature(<ftag>).image().set("selection", <propname>)
```

to specify that a property selection, `model.mesh(<mtag>).feature(<ftag>).selection(<propname>)`, will appear in the plot.

For mesh features that do not have any selection, the selection property is ignored when plotting.

Properties for the `image()` Objects

The following properties are available for the export of images:

TABLE 2-152: EXPORT PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
antialias	on off	on*	Enable or disable antialiasing.
axes1d	on off	on*	If options1d is on; enable/disable display of the coordinate axes in 1D.
axes2d	on off	on*	If options2d is on; enable/disable display of the coordinate axes in 2D.
axisorientation	on off	on*	If options is on; enable or disable display of the axis orientation indicator. Used for 3D plots.
background	current color transparent	color*	The background color.

TABLE 2-152: EXPORT PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bmpfilename	String		The name of the output file if imagetype is bmp.
customcolor	double array	{1, 1, 1}*	If background is color: the red, green, and blue components of the background color.
epsfilename	String		The name of the output file if imagetype is eps.
fontsize	integer	9*	The font size.
giffilename	String		The name of the output file if imagetype is gif.
grid	on off	on*	If options is on: enable or disable display of the coordinate grid. Used for 3D plots.
height	integer	480 px*	The height of the image.
lockratio	on off	off*	If on, the aspect ratio of the image is preserved when the width or the height is changed.
imagetype	bmp eps jpeg png tiff gif	png*	The type of image to export. eps can only be used for 1D plots.
jpegfilename	String		The name of the output file if imagetype is jpeg.
legend1d	on off	on*	If options1d is on: enable or disable display of the legend in 1D.
legend2d	on off	on*	If options2d is on: enable or disable display of the legend in 2D.
legend3d	on off	on*	If options3d is on: enable or disable display of the legend in 3D.
logo1d	on off	on*	If options1d is on: enable or disable display of the logo in 1D.
logo2d	on off	on*	If options2d is on: enable or disable display of the logo in 2D.
logo3d	on off	on*	If options3d is on: enable or disable display of the logo in 3D.
options1d	on off	off*	Enable or disable optional components of the image in 1D.
options2d	on off	off*	Enable or disable optional components of the image in 2D.
options3d	on off	off*	Enable or disable optional components of the image in 3D.
pngfilename	String		The name of the output file if imagetype is png.
resolution	integer	480 px*	The image resolution in dots per inch.
size	current web print presentation custom any custom size settings	current*	How to specify the size and resolution of the image. The value current gives a 800-by-600 image with the resolution 96 dots per inch. The values manualweb and manualprint let you give a size and resolution suitable for the web or printing, respectively, but also let you specify the size and resolution manually.
tifffilename	String		The name of the output file if imagetype is tiff.
title1d	on off	on*	If options1d is on: enable or disable display of the title in 1D.
title2d	on off	on*	If options2d is on: enable or disable display of the title in 2D.

TABLE 2-152: EXPORT PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
title3d	on off	on*	If options3d is on: enable or disable display of the title in 3D.
unit	px mm in	px*	The unit for the dimensions of the image.
width	integer	480 px*	The width of the image when size is manualweb.

* When making an image export, the value of the image feature doing the export will be stored and used as the default value when creating a new image feature. The image feature is created the first time the `image()` method is called.

The following properties are related to plotting of images:

TABLE 2-153: PROPERTIES FOR PLOTTING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
window	new window tag	new	The window to plot in. The window tags must be of the form <code>windowN</code> , where N is an integer (for example, <code>window3</code>).
windowtitle	String	Plot N	The window's title. The default is Plot N , where N is taken from the window tag.

The following properties are used for both export and plotting of images:

TABLE 2-154: PROPERTIES FOR EXPORT AND PLOTTING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
renderwireframe	on off fromview	on	Whether to use wireframe rendering for the geometry. The value <code>fromview</code> specifies that the view controls the rendering. The other values override the view setting. This property is only available in 3D and only for the features that plot a selection. This excludes geometries, meshes, and functions.
view	auto view tag	auto	The view settings to use when displaying this image. <code>auto</code> indicates that the view is selected automatically and will be the current view for the geometry being displayed.
zoomextents	on off	on	Whether to zoom the image to its extents. This property is only available for image objects displaying a geometry or mesh.
zooming	nozooming zoomextents zoomtoselection	px*	How to zoom the image. This property is available for all image objects except for those displaying a geometry, mesh, or function.
zoomlevel	integer (in the range from -15 to 15)	0	The number of zoom-in (if positive) or zoom-out (if negative) action steps to perform after zooming or zoomextents has been applied.

The tables below contain properties that are available for `image()` objects attached to a certain parent type.

The following property is used when attached to a geometry object:

TABLE 2-155: GEOMETRY PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
showmaterial	on off fromview	fromview	Enable or disable material color and texture.
showselection	on off fromview	fromview	Enable or disable material color and texture.

The only difference for the `showmaterial` and `showselection` properties for the geometry is that the values are taken from the view by default- They are `off` by default for all other features (such as probes and physics features).

The following property is used when attached to a pair object:

TABLE 2-156: PAIR PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selection	srcanddst source destination	srcanddst	Whether to display the source and destination selection, only the source selection, or only the destination selection.

The following property is used when attached to a physics feature or multiphysics coupling object:

TABLE 2-157: PHYSICS FEATURE OR MULTIPHYSICS COUPLING PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selection	main selection name	main	Which selection to display. Some physics features and multiphysics couplings have multiple selections. The value <code>main</code> corresponds to the selection retrieved by calling <code>selection()</code> on the physics feature or coupling, and the other values are the names to obtain a selection using <code>selection(<name>)</code> .

The following property is used when attached to a nonlocal coupling object:

TABLE 2-158: PAIR PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selection	srcanddst source destination	srcanddst	A property with these values is available for the Boundary Similarity, Edge Map, and Point Map nonlocal coupling features. Whether to display the source and destination selection, only the source selection, or only the destination selection.
selection	source srcvertices dstvertices	source	A property with these values is available for the Linear Extrusion and Linear Projection nonlocal coupling features. Whether to display the source selection, the source vertices selection, or the destination vertices selection.

Errors and Warnings

Introduction

For errors and warnings in the mesh and solver sequences, the following methods are available to retrieve an error message or warning message:

```
String warning = model.sol(<stag>).getWarningMessage();
```

returns the warning message as a string, and

```
String error = model.sol(<stag>).getErrorMessage();
```

returns the error message as a string. Here, <stag> is a solver sequence tag.

In addition, which the following section shows examples of, the following methods are available in the meshing sequences and solver sequences:

- `hasProblems()`, a boolean method that is true if problems exist.
- `problems()`, a `String[]` array with names of features with problems (that is, the union of warnings and errors)
- `errors()` and `warnings()`, `String[]` arrays with information about error and warning, respectively.

Retrieving Problem Information

Problems here means the union of warnings and errors.

EXAMPLE OF RETRIEVING PROBLEM INFORMATION IN A MESH

The following example shows how to retrieve information about a problem in the mesh (the same syntax also works for problems in the geometry):

1 Build a geometry:

```
Model mdl = ModelUtil.create("Model");
mdl.component().create("comp1");
mdl.component("comp1").geom().create("g",3);
mdl.component("comp1").geom("g").create("cyl1","Cylinder").set("h",3.0);
mdl.component("comp1").geom("g").create("cyl2","Cylinder").set("h",3.0).set("r",0.95);
mdl.component("comp1").geom("g").create("co1","Difference");
mdl.component("comp1").geom("g").feature("co1").selection("input").set("cyl1");
mdl.component("comp1").geom("g").feature("co1").selection("input2").set("cyl2");
```

2 Build a mesh:

```
MeshSequence ms = mdl.component("comp1").mesh().create("m","g");
ms.feature("size").set("hauto",9);
ms.create("ftri1","FreeTri");
ms.feature("ftri1").selection().geom(2).set(1,2,7,10);
ms.create("ftet1","FreeTet");
ms.feature("ftet1").create("ms1","Size");
ms.run();
```

3 Check if the mesh was built with problems:

```
boolean problem = ms.hasProblems();
```

4 Get the names of the features with problems. In this case, the feature `ftet1`:

```
String[] problemFeatures = ms.problems();
MeshFeature problematicFeature = ms.feature(problemFeatures[0]);
```

5 Get error information:

```
String[] errors = problematicFeature.errors();
```

```
for (String tag : errors) {
    analyzeProblems(problematicFeature.problem(tag));
}
```

Here, `analyzeProblems` is the following method to recursively retrieve error information:

```
private static void analyzeProblems(MeshProblemFeature problem) {
    String errorMessage = problem.message();
    System.out.println(errorMessage);

    if (problem.hasSelection()) {
        MeshSelection sel = problem.selection();
        System.out.println("Selection: " + sel);
    }

    double[][] allCoord = null;
    if (problem.hasProperty("allcoordinates")) {
        allCoord = problem.getDoubleMatrix("allcoordinates");
    }

    if (allCoord == null || allCoord.length == 0) {
        if (problem.hasProperty("coordinate")) {
            allCoord = new double[][]{problem.getDoubleArray("coordinate")};
        }
        else if (problem.hasProperty("coord")) {
            allCoord = new double[][]{problem.getDoubleArray("coord")};
        }
    }

    if (allCoord != null) {
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < allCoord.length; i++) {
            for (int j = 0; j < allCoord[i].length; j++) {
                builder.append(String.format("%10g", allCoord[i][j]));
            }
            builder.append(System.lineSeparator());
        }
        System.out.println(builder);
    }

    String[] problemDetails = problem.problems();
    for (String tag : problemDetails) {
        MeshProblemFeature detail = problem.problem(tag);
        // Recursively analyze subproblems
        analyzeProblems(detail);
    }
}
```

6 Get warning information:

```
String[] warnings = problematicFeature.warnings();
for (String tag : warnings) {
    analyzeProblems(problematicFeature.problem(tag));
}
```

where `analyzeProblems` is the same method as described in the previous step.

EXAMPLE OF RETRIEVING PROBLEM AND WARNING INFORMATION IN A SOLVER

The same technique is also available for retrieving information about problems and warnings in a solvers sequence:

1 For a solver sequence `ss1`, check if there are any problems:

```
SolverSequence ss1
boolean problem = ss1.hasProblems();
```

2 Get the names of the features with problems:

```
String[] problemNames = ss1.feature().problemNames();
```

3 Get error information:

```
String[] errorNames = ss1.feature(problemNames[0]).problem().errorNames();  
SolverFeature errorFeature = ss1.feature(problemNames[0]).problem(errorNames[0]);  
String errorMessage = errorFeature.getString("message");
```

4 Get warning information:

```
String[] warningNames = ss1.feature(problemNames[0]).problem().warningNames();  
SolverFeature warningFeature = ss1.feature(problemNames[0]).problem(warningNames[0]);  
String warningMessage = warningFeature.getString("message");
```

Geometry

This chapter includes reference information about the geometry commands and how to work with a geometry sequence and the geometry objects to create the model geometry. In this chapter:

- [About Geometry Commands](#)
- [Working with a Geometry Sequence](#)
- [Geometry Settings](#)
- [Work Planes](#)
- [Selections of Geometric Entities](#)
- [Geometry Cleanup](#)
- [Virtual Operations](#)
- [Geometry Object Information](#)
- [Measurements](#)
- [Inserting Geometry Sequences from File](#)
- [Exporting Geometry to File](#)
- [Creating and Using Geometry Parts](#)
- [Geometry Commands](#)

About Geometry Commands

	• Features for Creating Geometric Primitives • Features for Geometric Operations • Features for Virtual Operations • Features for Mesh Control • Geometric Model Information Methods
---	--

	For documentation of additional geometry commands available in the Design Module, CAD Import Module, ECAD Import Module, and LiveLink™ for CAD products, see the documentation for each of those products.
---	--

	model.geom() — Methods for the main <code>model.geom</code> command.
---	--

Features for Creating Geometric Primitives

Table 3-1 is an overview of the features for creating 3D geometric primitives:

TABLE 3-1: 3D GEOMETRIC PRIMITIVES.

NAME	DESCRIPTION
BezierPolygon	Chain of connected line segments, quadratic curves, or cubic curves
Block	Right-angled parallelepiped (box)
Cone	Right circular cone or cone frustum
CubicBezier	Rational Bézier curve of degree 3
Cylinder	Right circular cylinder
ECone	Oblique cone or cone frustum with elliptic base
Ellipsoid	Ellipsoid
Helix	Helix solid, surface, or curve
Hexahedron	Hexahedron bounded by bilinear faces
LineSegment	Line segment between two vertices or points
ParametricCurve	Curve defined by coordinate expressions
ParametricSurface	Surface defined by coordinate expressions
Point	One or several points
Polygon	Chain of connected line segments
Pyramid	Rectangular pyramid
QuadraticBezier	Rational Bézier curve of degree 2
Sphere	Sphere or ball
Tetrahedron	Tetrahedron
Torus	Torus

Table 3-2 is an overview of the features for creating 2D geometric primitives:

TABLE 3-2: 2D GEOMETRIC PRIMITIVES.

NAME	DESCRIPTION
BezierPolygon	Chain of connected line segments, quadratic curves, or cubic curves
Circle	Circle or disc
CircularArc	Circular arc
CubicBezier	Rational Bézier curve of degree 3
Ellipse	Ellipse
LineSegment	Line segment between two vertices or points
ParametricCurve	Curve defined by coordinate expressions
Point	One or several points
Polygon	Chain of connected line segments
QuadraticBezier	Rational Bézier curve of degree 2
Rectangle	Rectangle
Square	Square

Table 3-3 is an overview of the features for creating 1D geometric primitives:

TABLE 3-3: 1D GEOMETRIC PRIMITIVES.

NAME	DESCRIPTION
Interval	One interval, or a chain of connected intervals
Point	One or several points

Features for Geometric Operations

The [Import](#) feature imports geometry objects from a file or from another geometry. The [FromMesh](#) feature constructs a geometry object from a (deformed) mesh.

[Table 3-4](#) through [Table 3-8](#) list the features that create new geometric objects from existing ones, [Table 3-9](#) lists programming features for construction of geometry parts with conditionally active geometry features, for example.

TABLE 3-4: WORK-PLANE RELATED FEATURES (ONLY 3D, EXCEPT FOR CROSSSECTION).

NAME	DESCRIPTION
WorkPlane	Create a work plane for drawing 2D objects that are embedded into 3D
Extrude	Extrude planar faces in 3D
Revolve	Revolve planar faces in 3D
Sweep	Sweep one or several faces along a spine curve to create a solid in 3D
CrossSection	Create 2D geometry from intersection of 3D geometry with work plane

TABLE 3-5: BOOLEAN AND PARTITIONING OPERATIONS.

NAME	DESCRIPTION
Compose	Compose geometry objects using a set formula
Difference	Subtract geometry objects from geometry objects
Intersection	Intersect geometry objects
Union	Unite geometry objects
Partition	Partition a 2D or 3D geometry using tool objects or (3D only) a work plane
PartitionDomains	Partition domains in 2D or 3D geometries along some partitioning lines, edges, or faces

TABLE 3-5: BOOLEAN AND PARTITIONING OPERATIONS.

NAME	DESCRIPTION
PartitionEdges	Partition edges in 2D or 3D geometries at some partitioning vertices along the edges
PartitionFaces	Partition faces in 3D geometries at some partitioning curves on the edges

See [Compose](#), [Union](#), [Intersection](#), [Difference](#) for information about those Boolean operations.

TABLE 3-6: LINEAR TRANSFORMATIONS.

NAME	DESCRIPTION
Array	Rectangular or linear array of geometry objects
Mirror	Reflect objects in a plane (3D), a line (2D), or a point (1D)
RigidTransform	Make a rigid transform (translation and rotation) of 3D geometry objects
Rotate	Rotate geometry objects about a centerpoint
Scale	Scale geometric objects about a centerpoint
Move	Translate geometry objects
Copy	Make a displaced copy of geometry objects

See [Move](#), [Copy](#) for details about those linear transformations.

TABLE 3-7: OBJECT TYPE CONVERSIONS.

NAME	DESCRIPTION
ConvertToSolid	Unite and convert objects to a single solid object
ConvertToSurface	Unite and convert 3D objects to a single surface object
ConvertToCurve	Unite and convert 2D or 3D objects to a single curve object
ConvertToPoint	Unite and convert objects to a single point object

See [ConvertToSolid](#), [ConvertToSurface](#), [ConvertToCurve](#), [ConvertToPoint](#) for information about those conversion operations.

TABLE 3-8: OTHER OPERATIONS.

NAME	DESCRIPTION
Chamfer	Chamfer corners in 2D geometry objects
Fillet	Fillet corners in 2D geometry objects
Tangent	Line segment tangent to an edge in 2D
Delete	Delete entities (domains, boundaries, edges, or points) from objects, or delete entire geometry objects
Split	Split geometry objects into their constituent entities
Finalize	Form union or assembly by combining all geometry objects

TABLE 3-9: PROGRAMMING AND PARTS FEATURES.

NAME	DESCRIPTION
If , ElseIf , Else , EndIf	Construct an If statement, enabling or disabling features depending on conditions in terms of parameters
ParameterCheck	Check the value of parameters.
PartInstance	Create an instance of a geometry part.

Selection Features

Table 3-10 lists the features that correspond to selections:

TABLE 3-10: SELECTIONS.

NAME	DESCRIPTION
AdjacentSelection	Selection of entities or objects that are adjacent to given selections
ExplicitSelection	Explicit selection of entities or objects
BallSelection	Selection of entities or objects that (partly) lie inside a ball
BoxSelection	Selection of entities or objects that (partly) lie inside a box
CylinderSelection	Selection of entities or objects that (partly) lie inside a cylinder
ComplementSelection	Selection of entities or objects that is the complement of the input selections
DifferenceSelection	Selection of entities or objects that is the difference of the input selections
IntersectionSelection	Selection of entities or objects that is the intersection of the input selections
UnionSelection	Selection of entities or objects that is the union of the input selections

See [BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#) and [UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#) for information about those selections.

Features for Virtual Operations

Table 3-11 lists the features that correspond to virtual operations:

TABLE 3-11: VIRTUAL GEOMETRY RELATED FEATURES (ONLY 2D AND 3D).

NAME	DESCRIPTION
IgnoreVertices	Virtually remove isolated vertices or vertices adjacent to two edges only
IgnoreEdges	Virtually remove isolated edges or edges adjacent to precisely two faces or between two domains
IgnoreFaces	Virtually remove isolated faces or faces between two domains
CompositeEdges	Form virtual composite edges from sets of connected edges by ignoring the vertices between the edges in each set
CompositeFaces	Form virtual composite faces from sets of connected faces by ignoring the edges between the faces in each set
CompositeDomains	Form virtual composite domains from sets of connected domains by ignoring the boundaries between the domains in each set
CollapseEdges	Virtually collapse each edge into a vertex by merging its adjacent vertices
CollapseFaces	Virtually collapse faces
CollapseFaceRegions	Virtually detect and collapse regions of faces narrower than a specified size
MergeEdges	Virtually merge edges adjacent to face
MergeVertices	Virtually merge one adjacent vertex of an edge with the other adjacent vertex
RemoveDetails	Automatically remove small details in a geometry by using virtual operations

Features for Mesh Control

Table 3-12 lists the features that correspond to mesh control operations:

TABLE 3-12: MESH CONTROL RELATED FEATURES (ONLY 2D AND 3D).

NAME	DESCRIPTION
MeshControlVertices	Use vertices for mesh control only
MeshControlEdges	Use edges for mesh control only
MeshControlFaces	Use faces for mesh control only
MeshControlDomains	Use domains for mesh control only

Geometric Model Information Methods

Every geometry object in a geometry sequence, as well as the finalized geometry, have an associated `GeomInfo` object for which you can get information about the geometric model.

To get the `GeomInfo` object for a geometry object `<objtag>`, use:

```
GeomInfo info = model.component(<ctag>).geom(<tag>).obj(<objtag>);
```

To get the `GeomInfo` object for the finalized geometry or for the geometric model of a component that is defined by a mesh, use:

```
GeomInfo info = model.component(<ctag>).geom(<tag>).info();
```

For more information about getting information for meshes, see [Information on the Geometric Model](#).

	For a <code>GeomSequence</code> whose component's geometric model is defined by a <code>MeshSequence</code> , the <code>GeomInfo</code> methods return information about that geometric model, not about the <code>GeomSequence</code> . The behavior in this case might change in a future version.
---	--

GENERAL INFORMATION

TABLE 3-13: GENERAL GEOMETRY INFORMATION METHODS.

METHOD	DESCRIPTION
check	Throw an exception if the geometry or mesh is invalid.
getBoundingBox	Return bounding box as the bounding-box coordinates (xmin, xmax, ymin, ymax, ...).
getSDim	Get the space dimension.
getType	Get the geometry object type (solid, surface, curve, point, mixed, or empty).
hasCadRep	Check if this is a 3D geometry represented using the CAD kernel.
isAssembly	Check if this is a geometry of assembly type (originating from a Form Assembly feature), including virtual geometries based on a real assembly geometry.
isMesh	Return true if this geometric model is represented by a mesh (rather than a geometry).
isVirtual	Return true if this is a virtual geometry.

GEOMETRIC ENTITY COUNTERS

TABLE 3-14: GEOMETRIC ENTITY COUNTERS.

METHOD	DESCRIPTION
getNEntities	Get number of entities of different dimensions
getNVertices	Get number of vertices
getNEdges	Get number of edges
getNFaces	Get number of faces
getNBoundaries	Get number of boundaries
getNDomains	Get number of domains
getNEntitiesMesh	Get number of entities of different dimensions in the geometry used for meshing

ADJACENCY

TABLE 3-15: ADJACENCY BETWEEN GEOMETRIC ENTITIES.

METHOD	DESCRIPTION
getStartEnd	Get start and end vertices of edges
getUpDown	Get up and down domain indices
getUpDownExt	Get up and down extended domain indices
getVertexDomain	Get domain index for isolated vertices
getSD	Get domain index for isolated vertices
getAdj	Get adjacency matrices
getAdjOrient	Get adjacency orientation

EDGE EVALUATION

TABLE 3-16: EDGE EVALUATION METHODS.

METHOD	NOTES	DESCRIPTION
edgeParamRange		Get parameter range of edge
edgeX	1	Evaluate coordinates.
edgeDX	1	Evaluate first derivative.
edgeDDX	1	Evaluate second derivative.
edgeDDDX	1	Evaluate third derivative.
edgeNormal	1	Evaluate normal vector in 2D.

TABLE 3-16: EDGE EVALUATION METHODS.

METHOD	NOTES	DESCRIPTION
edgeCurvature	1	Evaluate curvature.
edgeTorsion	1	Evaluate torsion in 3D.

¹ For a mesh, the evaluation is done on a smooth interpolated edge curve.

FACE EVALUATION

TABLE 3-17: FACE EVALUATION METHODS.

METHOD	NOTES	DESCRIPTION
faceParamRange	1, 2	Get parameter ranges of face.
faceX	1, 2	Evaluate coordinates.
faceDX	1, 2	Evaluate first derivatives.
faceDDX	1, 2	Evaluate second derivatives.
faceNormal	1, 2	Evaluate normal vector.
faceFF1	1, 2	Evaluate first fundamental form.
faceFF2	1, 2	Evaluate second fundamental form.
faceGaussCurvature	1, 2	Evaluate Gauss curvature.
faceMeanCurvature	1, 2	Evaluate mean curvature.

¹ Not supported for geometric models defined by a mesh.

² Only supported for faces that are not virtual.

GEOMETRY REPRESENTATION ARRAYS

TABLE 3-18: GET ARRAYS IN GEOMETRY REPRESENTATION.

METHOD	NOTES	DESCRIPTION
getVertex	1, 3	Get vertex matrix.
getEdges	1, 3	Get edge matrix.
getFaces	1, 3	Get face matrix.
getPVertex	1, 3	Get parameter vertices (embeddings of vertices in faces).
getPEdge	1, 3	Get parameter edges (embeddings of edges in faces).
getVertexCoord		Get vertex coordinates.

¹ Not supported for geometric models defined by a mesh.

³ Only supported on geometry objects using the COMSOL kernel, and not for assembly geometries or virtual geometries.



In the `GeomSequence` interface, the `GeomInfo` methods give information about the finalized geometry of the geometry sequence. However, for a `GeomSequence` whose component's geometric model is defined by a `MeshSequence`, the `GeomInfo` methods instead return information about that geometric model. The behavior in this case might change in a future version.

`GeomSequence.obj` gives information about a geometry object resulting from a geometry feature.

You can use `GeomSequence.axisymmetric(boolean)` to change its axisymmetry status.

Working with a Geometry Sequence

This section describes how to construct geometries using Java[®] methods. A *geometry* is defined by a *geometry sequence* consisting of *geometry features*. Each feature generates a set of *output geometry objects* when you *build* the feature. An *operation feature* takes previously generated geometry objects as input and usually deletes them. You can create named selections by adding *selection features*. Each geometry sequence in 1D ends with a **Finalize** feature that forms a single output object by uniting all existing geometry objects. A geometry sequence in 2D or 3D also contains a **Finalize** feature, but in 2D and 3D it is possible to add features corresponding to *virtual operations* after the **Finalize** feature (see [Virtual Operations](#) for more information). The output object of the last feature of a sequence is referred to as the *finalized geometry*. The finalized geometry is used for meshing and physics modeling.

In 3D, you can also add work planes (see [Work Planes](#) for more information), where you can add 2D geometry sequences that build 2D geometries that you can use to embed, extrude, and revolve in the 3D geometry.

	Methods on <code>model.component(<ctag>).geom(<tag>)</code> can also be used in work planes on <code>model.component(<ctag>).geom(<tag>).feature(<wptag>).geom()</code> .
	The syntax that includes the component level, such as <code>model.component(<ctag>).geom(<tag>)</code> ... is the default and is used throughout this chapter. To use the earlier <code>model.geom(<tag>)</code> ... syntax, clear the Generate code using component syntax checkbox on the Application Builder > Methods page in the Preferences window.
	Geometry Modeling and CAD Tools in the <i>COMSOL Multiphysics Reference Manual</i>

Adding a Model Component (Geometry)

To add a new geometry to the model object `model`, enter

```
model.component(<ctag>).geom().create(<tag>, sDim);
```

where `<tag>` is the geometry's tag (an identifier of your choice), and `sDim` is its space dimension (1, 2, or 3).

The geometry is added to the last created model component. If no model component exists in the model, a model component node tagged `comp1` is automatically created for you. A physics interface using the geometry must belong to the same model component as the geometry.

You can change the model component of a geometry by entering

```
model.component(<ctag>).geom(<tag>);
```

where `<ctag>` is the tag of a component node.

Adding a Geometry Feature

To add a feature to a geometry tagged `<tag>`, enter

```
model.component(<ctag>).geom(<tag>).create(<ftag>, ftype);
```

where `<ftag>` is the feature's tag (an identifier of your choice), and `ftype` is the feature's type. Feature types are capitalized and case-sensitive, for example **Rectangle**.

When you add a feature, it is inserted after the *current feature*. You can get the tag of the current feature type by entering

```
String ftag = model.component(<ctag>).geom(<tag>).current();
```

If `ftag` is the empty string, the current feature is the beginning of the geometry sequence, that is, the empty state before all features. When the feature has been added, it automatically becomes current, but it is not built automatically.

All properties in a new feature get a default value.

Editing a Geometry Feature

To change a property value in a feature, enter

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
```

where *property* is a property name and *<value>* is a property value.

All numeric properties can be given either as a numeric value or as a string expression that can contain parameters defined in `model.param()`. When building the feature, the string expressions are evaluated using the current values of the parameters.

To get the value of a property, enter one of the following, depending on the type of the property:

```
double d = model.component(<ctag>).geom(<tag>).feature(<ftag>).getDouble(property);
String s = model.component(<ctag>).geom(<tag>).feature(<ftag>).getString(property);
double[] da = model.component(<ctag>).geom(<tag>).feature(<ftag>).
getDoubleArray(property);
String[] sa = model.component(<ctag>).geom(<tag>).feature(<ftag>).
getStringArray(property);
double[][] dm = model.component(<ctag>).geom(<tag>).feature(<ftag>).
getDoubleMatrix(property);
String[][] sm = model.component(<ctag>).geom(<tag>).feature(<ftag>).
getStringMatrix(property);
```

If you request a numerical value for a string property, it is evaluated using the current values of the parameters in `model.param()`.

	• get* and Selection Access Methods • set()
---	---

SELECTIONS

There are primitive features and operation features. Operations features take existing geometry objects as input and create new geometry objects from them. The input objects are usually specified in the input selection:

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input").
set(inputObjects);
```

where `inputObjects` is a string array with object or feature names. If `inputObjects` contains a feature name, it refers to all objects generated by this feature. If you have defined a named selection of objects, you can refer to it using the named method (see [Selection Features](#) below).

	Geometry Object Selection Methods
---	---

Usually, the input objects of an operation feature is removed when building the feature. To change this behavior, a property `keep` is available for many operations features. If `keep` is set to `on`, the input objects are kept when building the feature.

ATTRIBUTES

Attributes are string values that can be assigned to geometric entities or geometry objects. The available attributes are described in [Geometry Attribute Methods](#). To set an attribute on all resulting objects of a feature, enter

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(<attribute>,<value>);
```

To get the attribute value set in a feature, enter

```
String value = model.component(<ctag>).geom(<tag>).feature(<ftag>).
    getAttribute(<attribute>);
```

For a work plane, you can in addition set and get attributes on the 2D features in the Plane Geometry sequence as follows, for example:

```
model.component("comp1").geom("geom1").feature("wp1").geom().feature("r1").
    setAttribute("construction", "on");
String value = model.component("comp1").geom("geom1").feature("wp1").geom().
    feature("r1").getAttribute("construction");
```

Building Geometry Features

To generate the output objects of a feature, you must *build* the feature. Enter

```
model.component(<ctag>).geom(<tag>).run(<ftag>);
```

to build the feature `<ftag>` and all its preceding features (the features are built in the order from the first to the last). When the build has completed, the feature `<ftag>` becomes current.

To build all preceding features of the feature `<ftag>`, enter

```
model.component(<ctag>).geom(<tag>).runPre(<ftag>);
```

To build all features, including the Finalize feature and the virtual operations, enter

```
model.component(<ctag>).geom(<tag>).run();
```



If the current feature was before the Finalize feature, this operation sets the current feature to be the feature preceding Finalize.

ERRORS

If an error occurs when building a feature, the build stops, and the feature before the failing feature becomes current. The failing feature gets an *error feature* appended, which contains the error message. To access the error message, enter

```
String msg = model.component(<ctag>).geom(<tag>).feature(<ftag>).message();
```

To access the error feature, its message and detailed message, enter

```
String msg = model.component(<ctag>).geom(<tag>).feature(<ftag>).
    problem("error").getString("message");
String det = model.component(<ctag>).geom(<tag>).feature(<ftag>).
    problem("error").getString("details");
```

WARNINGS

After a successful build, a feature can get warning features appended, which contain warning messages. To access the first warning message, enter

```
String msg = model.component(<ctag>).geom(<tag>).feature(<ftag>).message();
```

To access the warning features, their messages and detailed messages, enter

```
String msg = model.component(<ctag>).geom(<tag>).feature(<ftag>).
    problem(<wtag>).getString("message");
String det = model.component(<ctag>).geom(<tag>).feature(<ftag>).
    problem(<wtag>).getString("details");
```

where *<wtag>* is warning1, warning2, and so on.

RETRIEVING INFORMATION ABOUT THE LATEST BUILD

The following methods are available to retrieve the information specific to the build of a geometry feature.

To see the COMSOL version number (and architecture) used for the latest build of the feature *<ftag>*, use

```
String version = model.component(<ctag>).geom(<tag>).feature(<ftag>).
    buildComsolVersion();
```

To get access to the date and time of the last time a feature was built, enter

```
String date = model.component(<ctag>).geom(<tag>).feature(<ftag>).buildDate();
```

The methods above return empty strings if the feature has not yet been built. Use

```
int time = model.component(<ctag>).geom(<tag>).feature(<ftag>).buildTime();
```

to get the time (in seconds) it took to build the feature the last time it was built. In case the feature has not been built, it will return -1.

Feature Status

The *status* of a feature can be one of the following:

- *Built* or *warning*. This means that the none of the feature's properties have changed since the feature was last built, and the features of the input objects are all built. If the status is *warning*, the feature contains warning messages.
- *Edited*. This means that some of the feature's properties have changed since the feature was last built.
- *Needs rebuild*. This means that the feature generating some input object is not built.
- *Error*. This means that the feature contains an error message.

You can examine the status of a feature by entering

```
String status = model.geom(<tag>).feature(<ftag>).status();
```

Accessing Geometry Object Names

Each feature produces one or several output geometry objects. To get the names of these objects, enter

```
String[] oNames = model.component(<ctag>).geom(<tag>).feature(<ftag>).objectNames();
```

To get the names of all currently existing geometry objects (the geometry objects that were generated by the last build), enter

```
String[] oNames = model.component(<ctag>).geom(<tag>).objectNames();
```

To access one of these objects, you can enter

```
GeomObject go = model.component(<ctag>).geom(<tag>).obj(<objname>);
```

where the string *<objname>* is an object name. If *<objname>* does not exist in the current state, you get an error message. You can get information about the geometry object *go* by using the *geometry information methods*, for example,

```
int numberOfFaces = go.getNFaces();
```



Geometry Object Information

To access the finalized geometry (the output of the last feature), use

```
model.component(<ctag>).geom(<tag>)
```

NAMING OF GEOMETRY OBJECTS

The names of the output objects of a feature are formed by appending characters after the feature's tag, in one of the following ways:

- `ftag(index)`. For example, `split1(1)`, `split1(2)`, `split1(3)` if the feature tagged `split1` has three output objects. This method is used for most features.
- `ftag(i1,i2,...)`. For example, `arr1(1,1)`, `arr1(1,2)`, `arr1(2,1)`, `arr1(2,2)` for a 2-by-2 array feature tagged `arr1`. This method is only used for the `Array` feature.
- `ftag.objectNameIn2D`. For example, `wp1.r1`, `wp1.pt1(1)`, `wp1.pt1(2)` if the work-plane feature `wp1` contains the 2D objects `r1`, `pt1(1)`, and `pt1(2)`. This method is only used for the `WorkPlane` feature.
- `ftag.objectNameIn3D`. For example, `cro1.blk1` and `cro1.cyl1`. This method is used for the `CrossSection` feature.
- `ftag.objectName`. This method can be used for the `Import` feature, and then `objectName` is taken from the CAD file.

Deleting and Disabling Geometry Features

To delete a feature, enter

```
model.component(<ctag>).geom(<tag>).feature().remove(<ftag>);
```

To disable a feature, enter

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).active(false);
```

The disabled feature does not affect the finalized geometry — its output is empty. To enable a disabled feature, enter

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).active(true);
```

You can get the enabled/disabled status of a feature by entering

```
boolean isEnabled = model.component(<ctag>).geom(<tag>).feature(<ftag>).active();
```

Deleting Geometry Objects

You can use the following operation to delete objects from the geometry sequence.

```
model.component(<ctag>).geom(<tag>).delete(String[]);
```

In the input string array you specify the names of the objects to delete. The operation deletes objects that correspond to primitive geometry features by removing these features from the sequence. The operation then deletes the remaining objects by adding and building a `Delete` feature with the objects in its selection.



When using the delete operation the status of the current feature and all its preceding features must be built.

You can use the following operation to delete some edges from a Polygon or CompositeCurve object in a 2D geometry:

```
String[] remaining = model.component(<ctag>).geom(<tag>).deleteEdges(<objname>, <edges>);
```

This operation deletes the given edges by using a combination of the following methods:

- Removing the geometry feature.
- Removing points from a Polygon feature.
- Splitting a Polygon or Composite Curve feature into several features.

The return array contains the tags of the remaining (if any) and created (if any) features.

Moving and Scaling Geometry Objects

You can use the following operations to move or scale objects from the geometry sequence.

```
model.component(<ctag>).geom(<tag>).move(String[] obj, double[] dist);
model.component(<ctag>).geom(<tag>).
    scale(String[] obj, double[] factor, double[] center);
model.component(<ctag>).geom(<tag>).scale(String[] obj, double factor, double[] center);
```

The input array `obj` specifies the objects to move or scale. The `dist` array specifies the move distance in each axis direction. The `factor` array specifies an anisotropic scaling and the `factor` scalar specifies an isotropic scaling. The `center` array specifies the scaling centerpoint. When possible, the move and scale operations modify the corresponding geometry features in the sequence. Not all features can be moved or scaled by modifying their properties, in which case move or scale features are added to the geometry sequence instead.



When using the move or scale operations the status of the current feature and all its preceding features must be built.

Plotting a Geometry Sequence

Use the `image().plot()` method for plotting the geometry sequence in a window:

```
model.geom("geom1").image().plot();
```

See [Plotting and Exporting Images](#) for more information.

Geometry Attribute Methods

Attributes are string values that can be assigned to geometric entities or geometry objects. Currently, there are only two types of attributes:

- The Construction geometry attribute. See [Construction Geometry](#) for more information.
- The PCB component attribute that can appear when importing components from some ECAD files. The PCB component attribute also has a field that has the tag `package`. This field assigns a component type to each component.

Get the type of the attribute, for example `Construction` or `PCBComponent`. The method returns a string.

```
model.geom(<gtag>).attribute(<atag>).getType();
```

where `<atag>` is the tag of the attribute, for example `imp1.component`.

Get the tag/name of the attribute, for example `construction`, `imp1.component`. The method returns a string.

```
model.geom(<gtag>).attribute(<atag>).tag();
```

Get the (localized) label of the attribute, for example Construction geometry or PCB component. The method returns a string.

```
model.geom(<gtag>).attribute(<atag>).label();
```

Get the whole objects that have the given attribute value <value> in the current state of the geometry sequence. The method returns a string array.

```
model.geom(<gtag>).attribute(<atag>).objects(<value>);
```

Get the objects that have some entity in dimension <dim> that have the given attribute value <value> in the current state of the geometry sequence. The method returns a string array.

```
model.geom(<gtag>).attribute(<atag>).objects(<value>, <dim>);
```

Get the entities in dimension <dim> in the object <oname> that have the given attribute value <value> in the current state of the geometry sequence. The method returns an integer array.

```
model.geom(<gtag>).attribute(<atag>).entities(<value>, <oname>, <dim>);
```

For the finalized geometry: Get the entities in dimension <dim> that have the given attribute value <value>. The method returns an integer array.

```
model.geom(<gtag>).attribute(<atag>).entities(<value>, <dim>);
```

Get all values of the attribute that exist in the current state of the geometry sequence. The method returns a string array.

```
model.geom(<gtag>).attribute(<atag>).values();
```

Get all values of the attribute that exist in the finalized geometry. The method returns a string array.

```
model.geom(<gtag>).attribute(<atag>).valuesFinal();
```

Get all tags of the fields of the attribute. The method returns a string array.

```
model.geom(<gtag>).attribute(<atag>).fieldTags();
```

Get the field value of a given field for a given attribute value. The method returns a string.

```
model.geom(<gtag>).attribute(<atag>).fieldValue(<attributeValue>, <fieldTag>);
```

Geometry Settings

You can control the following general settings for a geometry:

- [Length Unit](#)
- [Angular Unit](#)
- [Scale Values When Changing Unit](#)
- [Reduction for Symmetry Boundaries](#)
- [Geometry Representation in 3D and for Boolean Operations](#)
- [Default Repair Tolerances](#)
- [Automatic Rebuild](#)
- [Constraints and Dimensions](#) (with the Design Module)

Length Unit

The default length unit is meter. To change the length unit, enter

```
model.component(<ctag>).geom(<tag>).lengthUnit(newLengthUnit);
```

where *newLengthUnit* is a string like "mm", "in", or "ft".

To get the current length unit, enter

```
String currentUnit = model.geom(<tag>).lengthUnit();
```

The length unit is used in fields for lengths and for visualization of the geometry. In fields you can override the unit (for example, by entering 13[mm]). When solving the model, all lengths are converted to meters.

Angular Unit

The default angular unit is degrees. To change the angular unit, enter

```
model.component(<ctag>).geom(<tag>).angularUnit(newAngularUnit);
```

where *newAngularUnit* is **deg** or **rad**.

To get the current angular unit, enter

```
String currentUnit = model.component(<ctag>).geom(<tag>).angularUnit();
```

The angular is used in fields for angles. You can override the unit, for example by entering 0.3[rad]. Numeric inputs and outputs of trigonometric functions are always assumed to be in radians, though.

Scale Values When Changing Unit

When you change the length unit or angular unit there are two possibilities to interpret pure numeric values in the geometry and meshing sequences. The first possibility is to reinterpret the numeric value in the new unit; a circle of radius 1.0 (meter) becomes a circle of radius 1.0 (millimeter), assuming the length unit changes from meter to millimeter.

The other possibility is to scale the numbers; a circle of radius 1.0 (meter) becomes a circle of radius 1000.0 (millimeter). To control the behavior, use

```
model.component(<ctag>).geom(<tag>).scaleUnitValue(newScaleValue);
```

where *newScaleValue* is **true** if you want values to be scaled, and **false** otherwise. The default value is **false**.

To get the currently used method, enter

```
boolean currentScaleValue = model.component(<ctag>).geom(<tag>).scaleUnitValue();
```

Reduction for Symmetry Boundaries

Use the methods `symmetricX()`, `symmetricY()`, and `symmetricZ()` to set one or several of the space coordinates to be symmetric, meaning that the part of the geometry that lies in the negative region of the space coordinate will be removed by the Form Union/Assembly feature, and that a corresponding boundary selection will be created. Note that the second space coordinate in 2D axisymmetry is Z.

```
model.component(<ctag>).geom(<tag>).symmetricZ(value);
```

where `value` is set to `true` or `false`. The method `symmetricX()` throws an exception if `isAxisymmetric()` is `true`. The method `symmetricY()` throws an exception in 1D and in 2D axisymmetry. The method `symmetricZ()` throws an exception in 1D and in 2D if not axisymmetric.

Use the methods `symmetricX()`, `symmetricY()`, and `symmetricZ()` to get the values previously set. If the methods return `true`, the part of the geometry that lies in the negative region of the space coordinate will be removed by the Form Union/Assembly feature, and that a corresponding boundary selection will be created. The method `symmetricX()` always returns `true` if `isAxisymmetric()` is `true`.

```
boolean b1 = model.component(<ctag>).geom(<tag>).symmetricX();
```

Geometry Representation in 3D and for Boolean Operations

This settings is only relevant if you have a license for the CAD Import Module. The geometry representation determines which kernel (geometric modeler) that COMSOL uses to represent and operate on the geometry objects: the CAD Import Module's kernel (Parasolid) or COMSOL's own kernel. To change the geometry representation, enter

```
model.component(<ctag>).geom(<tag>).geomrep(newGeomRep);
```

where `newGeomRep` is `comsol` or `cadps`.

- If you choose `cadps`, all objects and operations that support the CAD kernel (Parasolid kernel) use it. For example, the Work Plane, Extrude, and Revolve features currently do not support this kernel.
- If you choose `comsol`, all objects are represented using the COMSOL kernel.

When you change the geometry representation, all nodes that support the CAD kernel get an *edited* status. To rebuild the geometry using the new kernel, use the `run` method.

To get the geometry representation of the geometry sequence `<tag>`, enter

```
model.component(<ctag>).geom(<tag>).geomrep();
```

When you create a new model (application), its default geometry representation is controlled by the preference setting **Geometry>Geometry representation>In new applications** (`geometry.geomrep.default`). To change or read this preference setting, enter

```
ModelUtil.setDefaultGeometryKernel(defaultGeomRep);  
ModelUtil.getDefaultGeometryKernel();
```

where `defaultGeomRep` is `cadps` or `comsol`.

When you open an existing model, you normally use the geometry representation used in the model. To always convert the geometry to the COMSOL kernel, change the preference setting **Geometry>Geometry representation>When opening an existing application** to **Convert to COMSOL kernel** (`geometry.geomrep.open`). To change or read this preference setting, enter

```
ModelUtil.setOpenGeometryKernel(openGeomRep);  
ModelUtil.getOpenGeometryKernel();
```

where *openGeomRep* is `model` or `comsol`.

USING THE DESIGN MODULE BOOLEAN OPERATIONS

When you use the CAD kernel in 3D and the license includes the Design Module, you can use the Boolean operations from the Design Module kernel (Parasolid kernel) by entering

```
model.component(<ctag>).geom(<tag>).designBooleans(true);
```

The default setting for Design Module Boolean operations is controlled by the preference setting **Geometry>Design Module Boolean operations>Use in new models**.

Default Repair Tolerances

The repair tolerance for the applicable geometry operations can be of three types: automatic (the default), relative, or absolute. The automatic repair tolerance provides suitable settings when using the CAD kernel; when using the COMSOL kernel, it sets a relative repair tolerance of $1\text{e-}6$. You specify the default repair tolerance type by entering:

```
model.component(<ctag>).geom(<tag>).repairTolType(<newRepairTolType>)
```

where *<newRepairTolType>* is any of `auto`, `relative`, or `absolute`.

The *default relative repair tolerance* is $1\text{e-}6$ (it can be any positive scalar value smaller than 0.1). You can change it by entering

```
model.component(<ctag>).geom(<tag>).repairTol(<newRelativeRepairtol>);
```

When doing so, the software also sets the `repairTolType` to `relative`. To get the current default relative repair tolerance, enter

```
double reptol = model.component(<ctag>).geom(<tag>).repairTol();
```

The *default absolute repair tolerance* is $1\text{e-}6$ (it can be any positive scalar value). You can change it by entering

```
model.component(<ctag>).geom(<tag>).absRepairTol(<newAbsoluteRepairtol>);
```

When doing so, the software also sets the `repairTolType` to `absolute`. To get the current default absolute repair tolerance, enter

```
double reptol = model.component(<ctag>).geom(<tag>).absRepairTol();
```

The default repair tolerance is the default value that is used when you add a new feature that has the repair tolerance properties — for example, Boolean operations and conversions. Changing the default repair tolerance does not affect the tolerances in existing features. Adjust the repair tolerance if you experience problems with a Boolean operation.

Automatic Rebuild

The `autoRebuild` setting controls if the geometry sequence is automatically rebuilt when clicking a node in the model tree outside the geometry sequence. You can change it by entering:

```
model.component(<ctag>).geom(<tag>).autoRebuild(<newAutoRebuild>);
```

where *<newAutoRebuild>* is `on` or `off`.

The default geometry representation is controlled by the preference setting **Geometry>Automatic rebuild when leaving geometry>Default in new geometries**.

The `autoBuildNew` setting controls if certain geometry operations will be built automatically when you add them, provided that you have preselected a sufficient number of entities or objects. You can change it by entering:


```
model.component(<ctag>).geom(<tag>).autoBuildNew(<AutoBuildNew>);
```

where *<AutoBuildNew>* is true or false.

The default for building geometry operations automatically is controlled by the preference setting **Geometry>Build new operations automatically>Default in new geometries**.

Constraints and Dimensions



These settings are only available if the geometry is 2D and you have a license for the Design Module.

To enable or disable the constraints and dimensions functionality, enter

```
model.component(<ctag>).geom(<tag>).useConstrDim(enable);
```

To determine whether the constraints and dimensions functionality is enabled, enter

```
boolean enabled = model.component(<ctag>).geom(<tag>).useConstrDim();
```

`model.component(<ctag>).geom(<tag>).constrDimBuild(value)` determines what constraint and dimension features to use when building the geometry. Valid values are:

- "all": All (enabled) constraint and dimension features are used. This is the default.
- "none": No constraint and dimension features are used.
- "uptotarget": All (enabled) constraint and dimension features up to the feature to build are used.

`String value = model.component(<ctag>).geom(<tag>).constrDimBuild()` returns the constraints and dimensions to use when building. The default is "all".

`String status = model.component(<ctag>).geom(<tag>).constrDimStatus()` gets a description of the overall status of the constraints and dimensions.

Work Planes

In 3D, you can create 3D objects by defining 2D objects in *work planes* and then *extruding* and *revolving* these into 3D objects. You can also get a *cross section* 2D object by intersecting a 3D object with the work plane.

To add a WorkPlane feature, use

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"WorkPlane");
```

You access a work planes 2D geometry sequence by the `geom()` method. To add a 2D feature to a work plane, use

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).geom().create(<ftag1>,ftype);
```

where `<ftag>` refers to a WorkPlane feature and `ftype` refers to a 2D feature type.



The **Finalize** feature is optional for work planes but is necessary if you want to add virtual operations to the work plane sequence. It then needs to be created, as specified in [Finalize](#).

A work plane's geometry sequence inherits its settings from its 3D sequence.



For more information on the settings for a geometry sequence see [Geometry Settings](#).

When you build a work-plane feature, its corresponding 2D sequence builds automatically and the geometry objects defined by the 2D sequence *embed* into 3D geometry objects in the 3D sequence. You can then extrude or revolve these embedded point, curve, or surface objects into curve, surface, or solid objects, respectively, using **Extrude** or **Revolve** features, respectively.



Methods on `model.component(<ctag>).geom(<tag>)` can also be used on `model.component(<ctag>).geom(<tag>).feature(<wptag>).geom()`.

Selections of Geometric Entities



The following sections contain information about selections of geometric entities in geometry objects; that is, those geometry objects (such as geometry primitives) and the entities that they consist of, which is useful for the preprocessing stages. For information about selection of geometric entities in the finalized geometry (domains, boundaries, edges, and points) used for specifying materials and physics, for example, see [Selections](#) and [model.selection\(\)](#).

Named Selections

You can create named selections of entities in geometry objects or of whole geometry objects in two ways:

- By setting the `selresult` property to `on` in an arbitrary geometry feature. This creates a selection containing the feature's output objects, and derived selections of entities for each entity type (domain, boundary, edge, and point). For the Import feature, there is an additional `selindividual` property, which creates selections corresponding to individual geometry objects.
- By adding a selection feature, see [Using Selection Features](#). If this defines a selection of whole objects, derived selections of entities are also created for each entity type (domain, boundary, edge, and point).

You get the tags of the created named selections by

```
String[] selTags = model.component(<ctag>).geom(<tag>).feature(<ftag>).  
outputSelection();
```

You can access a named selection by `model.geom(<gtag>).selection(<seltag>)`, where `<seltag>` is the selection's tag. Usually, `<seltag>` is the same as the tag of the feature that created the selection, but derived selections of entities have a suffix `.dom`, `.bnd`, `.edg`, or `.pnt`.

You can use a named selection as an input selection in a geometry feature that comes after the feature that created the selection in the geometry sequence, as follows:

```
model.component(<ctag>).geom(<gtag>).feature(<ftag2>).selection(<propname>).  
named(<trimmedseltag>);
```

Here, `<trimmedseltag>` is the selection's tag without the suffix.

Each named selection of entities in the geometry sequence can also be used as a named selection on the finalized geometry. You can access this named selection by `model.component(<ctag>).selection(<gtag>_<seltag>)`. However, if the selection was derived from a selection of whole objects, you access the corresponding selection on the finalized geometry by `model.component(<ctag>).selection(<gtag>_<trimmedseltag>_<lvl>)`, where `<lvl>` is one of `dom`, `bnd`, `edg`, or `pnt`. Named selections on the finalized geometry are described in [model.selection\(\)](#).

To get the selected objects for a named selection, use

```
String[] so = model.component(<ctag>).geom(<gtag>).selection(<seltag>).objects();
```

Using Selection Features

A selection features in the geometry sequence creates a selection of geometric entities or objects that is a subset of all entities or objects that were generated by the sequence of features preceding the selection feature.

- Use the `ExplicitSelection` feature to create a selection of entities or objects that you specify explicitly.

- Use the `BallSelection`, `BoxSelection`, `CylinderSelection`, or `DiskSelection` feature to create a selection of entities or objects that partly or completely lie inside a given ball, box, cylinder, or disk. The input entities or objects to select among can be all entities or objects generated by the preceding features, or a subset of these entities or objects defined by a set of input selections.
- Use the `UnionSelection`, `IntersectionSelection`, `DifferenceSelection`, and `ComplementSelection` features to combine a set of input selections using a Boolean operation.
- Use `AdjacentSelection` to create a selection of entities of a given dimension that are adjacent to entities in a set of input selections.

When using input selections, the input selections must be defined by features that come before the selection feature in the geometry sequence. You set the input selections in the `input` property (add and subtract for `DifferenceSelection`), where you use trimmed tags (that is, tags without the suffix) to refer to the input selections. Usually, the trimmed tag is the same as the tag of the feature that defined the input selection. It can happen that an input selection selects entities or objects that no longer exist after building the feature preceding the selection feature. In this case, the input selection is interpreted by using an associative mapping to existing entities or objects.

Cumulative Selections

To create a cumulative selection tagged `<seltag>`, use

```
model.component(<ctag>).geom(<tag>).selection().create(<seltag>,"CumulativeSelection");
```

To make a geometry feature contribute its selection to a cumulative selection, use

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).set("contributeto",<seltag>);
```

To remove the contribution of a geometry feature to a cumulative selection, use

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).set("contributeto","none");
```

To control whether the cumulative selection is available outside the geometry sequence, use

```
model.component(<ctag>).geom(<tag>).selection(<seltag>).show(<boolean>);
```

To query whether the cumulative selection is available outside the geometry sequence, use

```
boolean show = model.component(<ctag>).geom(<tag>).selection(<seltag>).show();
```

Vectorized Selections of Geometric Entities

For all selections of geometric entities, you can use the `add` method in a way that the following example demonstrates:

```
model.view("view1").hideObjects("hide1").add(new String[]{"arr1(2,1,1)", "arr1(3,1,1)"},
new int[][]{{1, 2, 3, 4, 5, 6}, {1, 2, 3, 4}});
```

This code adds all given entities for each object to the selection. In the same way, you can use vectorized calls to the `set` and `remove` methods.

Geometry Cleanup

In 3D it is possible to remove small details in the geometry by running a *geometry cleanup*. Gaps, narrow domain regions, narrow face regions, sliver faces, small faces, and short edges that are smaller than the estimated, or manually set, detail size are considered to be small details and are therefore considered unwanted. This will typically reduce the number of elements in the mesh and therefore reduce computation time and memory consumption. If the geometry is a single vertex, cleanup is not available.

To access the cleanup tool for the geometry `<gtag>`, enter

```
model.geom(<gtag>).cleanup();
```

To estimate the detail size that is used for the geometry analysis and automatic cleanup, use the method below. Details smaller than the returned value are considered unwanted by default.

```
model.geom(<gtag>).cleanup().estimateDetailSize();
```

It is possible to manually set a different detail size using the `detailsize` property.

```
model.geom(<gtag>).cleanup().set("detailsize", <value>);
```

The default value is 0 or an estimated detail size. The detail size `<value>` must be larger than $1e-6$ times the size of the geometry and less than 0.1 times the size of the geometry. If `<value>` is set to 0 when you run the analysis or automatic cleanup, it will be set to the estimated detail size.

To run the geometry analysis to detect small details with the current detail size, use the method below.

```
model.geom(<gtag>).cleanup().analyze();
```

Use the method below to get the time it took to analyze or clean up the geometry. It returns -1 if the analysis has not been built.

```
model.geom(<gtag>).cleanup().buildTime();
```

Get a list of all detail types that were found in the last analysis by entering

```
model.geom(<gtag>).cleanup().details();
```

and get the estimated max and min size for each detail of the specified type `<type>` that were found in the last analysis by entering

```
model.geom(<gtag>).cleanup().detailSizes(<type>);
```

To run automatic cleanup with the current detail size, enter

```
model.geom(<gtag>).cleanup().run();
```

To revert changes to the model done by cleanup, enter

```
model.geom(<gtag>).cleanup().revert();
```

The automatic cleanup can add Mesh-Based Geometries for some of the cleaning. See [Using Mesh-Based Geometries](#) for more information.

Virtual Operations

About Virtual Operations

In 2D and 3D it is possible to reduce the number of vertices, edges, faces, and domains of the geometry by using *virtual operation features*. To add the first virtual operation feature to a sequence you need to build the finalize feature by entering

```
model.component(<ctag>).geom(<tag>).run("fin");
```

You can then add the virtual operation feature by entering

```
model.component(<ctag>).geom(<tag>).create(<ftag>, ftype);
```

where *<ftag>* is the feature's tag (an identifier of your choice), and *ftype* is the feature's type. To build the feature, enter

```
model.component(<ctag>).geom(<tag>).run(<ftag>);
```

To build all features, including the finalize feature and all virtual operation features, and to create the finalized geometry, enter

```
model.component(<ctag>).geom(<tag>).run();
```

The finalized geometry of a sequence that contains virtual operation features is referred to as a *virtual geometry*. If you form a composite edge, face, or domain by using a [CompositeEdges](#), [CompositeFaces](#), or [CompositeDomains](#) feature, respectively (or the analogues [IgnoreVertices](#), [IgnoreEdges](#), or [IgnoreFaces](#) features) the resulting edge, face, or domain is referred to as a *virtual composite edge*, *virtual composite face*, or *virtual composite domain*, respectively, or more generally, a *virtual composite entity*.

Mesh Control Entities

Sometimes it is desirable to use certain geometric entities only when constructing the mesh. For example, you can add a curve inside a domain to control mesh element size there. If you mark this curve as a *mesh control entity*, it is not included in the geometry used when defining the physics. An advantage is that the final mesh need not respect this curve exactly; it is used only to control element size. You can use the `keepformesh` property of the [Composite](#) and [Ignore](#) features described above to define mesh control entities. Alternatively, you can use the [MeshControlDomains](#), [MeshControlVertices](#), [MeshControlEdges](#), or [MeshControlFaces](#) features.

Geometry Object Information

You can get the geometry object named `<objname>` via

```
model.component(<ctag>).geom(<tag>).obj(<objname>)
```



Accessing Geometry Object Names

The geometry itself,

```
model.component(<ctag>).geom(<tag>)
```

works as an object, namely the final geometry resulting from the sequence. To get information about these objects, you can apply the methods described in this section.



A geometry part does not have a finalized geometry, so these methods are not applicable for geometry parts. See [Creating and Using Geometry Parts](#) for information about applicable methods when working with geometry parts.

General Information

TABLE 3-19: GENERAL GEOMETRY INFORMATION METHODS.

METHOD	OUTPUT TYPE
<code>check()</code>	void
<code>exists()</code>	boolean
<code>getBoundingBox()</code>	double[sdim*2]
<code>getSDim()</code>	int
<code>getType()</code>	String
<code>hasCadRep()</code>	boolean

- `check()` issues an exception if the object is invalid.
- `exists()` returns `true` if an object exists.
- `getBoundingBox()` returns a bounding box for the object in the order `xmin`, `xmax`, `ymin`, `ymax`, `zmin`, and `zmax`.
- `getSDim()` returns the space dimension of the geometry.
- `getType()` returns the object type: `solid`, `surface`, `curve`, `point`, `mixed`, or `empty`.
- `hasCadRep()` returns `true` if the object is represented using the CAD kernel (Parasolid).

Geometric Entity Counters

The following geometric entity counter methods are available:

TABLE 3-20: GEOMETRIC ENTITY COUNTER METHODS.

METHOD	1D	2D	3D	OUTPUT TYPE
getNEntities()	√	√	√	int[]
getNVertices()	√	√	√	int
getNEdges()	√	√	√	int
getNFaces()			√	int
getNBoundaries()	√	√	√	int
getNDomains()	√	√	√	int
getNEntitiesMesh()	√	√	√	int[]

- `getNEntities` returns a vector of length 2 in 1D, length 3 in 2D, and length 4 in 3D. The vectors contain the number of geometric entities for each entity dimension. The methods `getNVertices`, `getNEdges`, `getNFaces`, `getNBoundaries`, and `getNDomains` return the number of entities of the specified type.
- `getNEntitiesMesh` returns a vector of length 2 in 1D, length 3 in 2D, and length 4 in 3D. The vectors contain the number of geometric entities for each entity dimension in the geometry used for meshing. If there are no mesh control entities in the geometry, the output is identical to that of `getNEntities`.

Adjacency

The following geometry adjacency information methods are available:

TABLE 3-21: GEOMETRY ADJACENCY INFORMATION METHODS.

METHOD	1D	2D	3D	OUTPUT TYPE
getStartEnd()	√	√	√	int[2][]
getUpDown()	√	√	√	int[2][]
getUpDownExt()	√	√	√	int[2][]
getVertexDomain()		√	√	int[]
getSD()		√	√	double[]
getAdj(int,int)	√	√	√	int[][]
getAdj(int,int,int)	√	√	√	int[]
getAdjOrient(int,int)	√	√	√	int[][]
getAdjOrient(int,int,int)	√	√	√	int[][]

- `getStartEnd` returns the start and end vertices of all edges in the first and second row of the returned matrix.
- `getUpDown` returns the up and down domain number for all boundaries in the first and second row of the returned matrix. All void regions have the domain number 0.
- `getUpDownExt` returns the up and down domain number for all boundaries in the first and second row of the returned matrix, using an extended domain numbering where void regions have different domain numbers. The infinite void region has domain number 0. The finite void regions have negative domain numbers. However, if `voidsAreLabeled()` returns `false`, the extended domain numbering is not available, and then all void regions have domain number 0.
- `getVertexDomain` returns the domain index for each vertex. For nonisolated vertices, the domain index is -1.
- `getSD` returns the domain index for each vertex. For nonisolated vertices, the domain index is NaN.

- `a = getAdj(fromDim, toDim)` returns a matrix where `a[fromIdx]=getAdj(fromDim,toDim,fromIdx)` contains the entities in dimension `toDim` that are adjacent to entity `fromIdx` in dimension `fromDim`.
- `ao = getAdjOrient(fromDim, toDim)` returns a matrix where `ao[fromIdx]=getAdjOrient(fromDim, toDim,fromIdx)` contains the orientation flag for the entities in `getAdj(fromDim,toDim,fromIdx)`. The orientation flag is 1 if the adjacent entities have the same orientation, and -1 if they have the opposite orientation, and 2 if the relative orientation cannot be determined (for instance, for an edge interior to a face).

Evaluation on an Edge

The following edge evaluation methods are available in 2D and 3D:

TABLE 3-22: EDGE EVALUATION METHODS IN 2D AND 3D.

METHOD	2D	3D	OUTPUT TYPE
<code>edgeParamRange(int)</code>	✓	✓	<code>double[2]</code>
<code>edgeX(int, double[])</code>	✓	✓	<code>double[][D]</code>
<code>edgeDX(int, double[])</code>	✓	✓	<code>double[][D]</code>
<code>edgeDDX(int, double[])</code>	✓	✓	<code>double[][D]</code>
<code>edgeDDDX(int, double[])</code>	✓	✓	<code>double[][D]</code>
<code>edgeNormal(int, double[])</code>	✓		<code>double[][D]</code>
<code>edgeCurvature(int, double[])</code>	✓	✓	<code>double[]</code>
<code>edgeTorsion(int, double[])</code>		✓	<code>double[]</code>

The first input argument of all methods is the edge number. The second input argument, when it exists, is an array of parameter values for which to perform evaluation on the edge. For all but the first method, the first index in the output corresponds to the different parameter values, and the second index corresponds to the spatial coordinates.

- `edgeParamRange` returns the parameter range for evaluation on the edge.
- `edgeX` evaluates the parameters to coordinate values.
- `edgeDX` evaluates the parameters to first-order derivative values.
- `edgeDDX` evaluates the parameters to second-order derivative values.
- `edgeDDDX` evaluates the parameters to second-order derivative values.
- `edgeDDDX` evaluates the parameters to third-order derivative values.
- `edgeNormal` evaluates the parameters to normal vector values.
- `edgeCurvature` evaluates the parameters to curvature values.
- `edgeTorsion` evaluates the parameters to torsion values.

The following edge evaluation method is available in 3D:

TABLE 3-23: EDGE EVALUATION METHOD IN 3D.

METHOD	OUTPUT TYPE
<code>edgeMaxFaceAngle(int[], int[], int[])</code>	<code>double[]</code>

This method computes the maximum along a 3D edge of the angle (in radians) between the normal vectors of two faces that are adjacent to the edge. The input arguments are a vector of edge indices, a vector of indices for the first face, and a vector of indices for the second face. The output is a double array, where the maximum is defined as the value closest to $\pi/2$ radians. The method is vectorized so that the N th entry in the returned array is the maximum along `edge[N]` of the angle between faces `face1[N]` and `face2[N]`. The edge and face numbers are one-based.

Evaluation on a Face

Use the following methods for face evaluation in 3D. They do not work on composite faces in virtual geometry objects.

TABLE 3-24: FACE EVALUATION METHODS IN 3D.

METHOD	OUTPUT TYPE
<code>faceParamRange(int)</code>	<code>double[4]</code>
<code>faceX(int, double[][2])</code>	<code>double[][3]</code>
<code>faceDX(int, double[][2])</code>	<code>double[][3][2]</code>
<code>faceDDX(int, double[][2])</code>	<code>double[][3][2][2]</code>
<code>faceNormal(int, double[][2])</code>	<code>double[][3]</code>
<code>faceFF1(int, double[][2])</code>	<code>double[][2][2]</code>
<code>faceFF2(int, double[][2])</code>	<code>double[][2][2]</code>
<code>faceGaussCurvature(int, double[][2])</code>	<code>double[]</code>
<code>faceMeanCurvature(int, double[][2])</code>	<code>double[]</code>

The first input argument of all methods is the face number. The second input argument, when it exists, is a matrix of parameter points, for which to perform evaluation. For all but the first method, the first index in the output corresponds to the different parameter points.

- `faceParamRange` returns two parameter ranges for evaluation on the face.
- `faceX` evaluates the parameters to coordinate values.
- `faceDX` evaluates the parameters to first order derivative values.
- `faceDDX` evaluates the parameters to second order derivative values.
- `faceNormal` evaluates the parameters to normal vector values.
- `faceFF1` evaluates the parameters to the first fundamental form values.
- `faceFF2` evaluates the parameters to the second fundamental form values.
- `faceGaussCurvature` evaluates the parameters to Gauss curvature values.
- `faceMeanCurvature` evaluates the parameters to mean curvature values.

Geometry Representation Arrays

Use the following methods to access the arrays in the internal representation of COMSOL Multiphysics geometry objects. They do not work on objects represented using the CAD kernel, assembly geometries, or virtual geometries.

TABLE 3-25: GET ARRAYS IN GEOMETRY REPRESENTATION.

METHOD	1D	2D	3D	OUTPUT TYPE
<code>getVertex()</code>	√	√	√	<code>double[][]</code>
<code>getEdge()</code>		√	√	<code>double[][]</code>
<code>getFace()</code>			√	<code>double[][]</code>
<code>getPVertex()</code>			√	<code>double[][]</code>
<code>getPEdge()</code>			√	<code>double[][]</code>
<code>getVertexCoord()</code>	√	√	√	<code>double[][]</code>
<code>voidsAreLabeled()</code>	√	√	√	<code>boolean</code>

- In 2D and 3D, `getVertex` returns (sdim+2)-by-nv matrix representing the vertices of the object. The first sdim rows are the coordinates of the vertices. Row sdim+1 contains the domain number if the vertex is isolated and is unspecified otherwise. The last row contains a relative local tolerance for the vertex. For nontolerant vertices the tolerance is NaN. This method does not work on virtual geometry objects.
- In 1D, `getVertex` returns a 3-by-nvtx matrix representing the vertices of the 1D object. Row 1 provides the coordinates of the vertices. Rows 2 and 3 provide the up and down domain numbers, respectively.
- `getPVertex` returns a 6-by-npv matrix containing embeddings of vertices in faces. Row 1 contains the vertex index (that is, column from `getVertex`), rows 2 and 3 contain (s, t) coordinates of the vertex on the face, row 4 contains a face index, and row 5 contains the surface index into the surfaces. Row 6 contains a relative local tolerance for the vertex. This method does not work on virtual geometry objects.
- In 3D, `getEdge` returns a 7-by-ne matrix representing the edges of the 3D object. Rows 1 and 2 contain the start and end vertex indices of the edge (0 if they do not exist), respectively. Rows 3 and 4 give the parameter values of these vertices. Row 5 gives the index of a domain if the edge is not adjacent to a face, and is unspecified otherwise. Row 6 gives a sign and an index to the underlying curve. The sign indicates the direction of the edge relative the curve. Finally, row 7 contains a relative local tolerance for the edge. This method does not work on virtual geometry objects.
- In 2D, `getEdge` returns a 8-by-ne matrix representing the edges of the 2D object. Rows 1 and 2 contain the start and end vertex indices of the edge, respectively (0 if they do not exist). Rows 3 and 4 give the parameter values of these vertices. Rows 5 and 6 contain the left and right domain number of the edge, respectively. Row 7 gives a sign and an index to the array of underlying curves. The sign indicates the direction of the edge relative the curve. Row 8 contains a relative local tolerance for the edge.
- `getPEdge` returns a 10-by-npe matrix representing the embeddings of the edges in faces. The first row gives the index of the edge in `getEdge`. Rows 2 and 3 contain the start and end vertex indices from `getPVertex`, respectively. Rows 4 and 5 give the parameter values of these vertices. Row 6 and 7 give the indices of the faces to the left and right of the edge, respectively. Row 8 gives a sign and index to the parameter curve (if any), and row 9 gives the index to the surface. Row 10 contains a relative local tolerance for the edge. This method does not work on virtual geometry objects.
- `getFace` returns a 4-by-nf matrix representing the faces of the 3D geometry. Rows 1 and 2 contain the up and down domain index of the face, respectively, and row 3 contains the surface index of the face. Row 4 contains a relative local tolerance for the face. This method does not work on virtual geometry objects.

- `getVertexCoord` returns a matrix with the vertex coordinates. Its dimension is the space dimension times the number of vertices.
- `voidsAreLabeled` returns `true` if all finite void regions are labeled with negative domain indices in the serialization (mphtxt or mphbin file). This also affects the domain indices in the following methods: `getAdjExt()`, `getAdjOrientExt()`, `getUpDownExt()`, `getFace()`, `getEdge()`, and `getVertex()`. It returns `false` if some finite void regions might be denoted with 0 in the serialization, like in version 4.2.

Measurements

Geometric measurements is a tool to measure geometric entities and objects. You access it by entering

```
model.component(<ctag>).geom(<tag>).measure();
```

using the `GeomSequence.measure()` method.

This command measures objects that exist in the current build state (corresponding to what you would see in the COMSOL Desktop). You cannot measure objects not yet built or object that are built but has been consumed as input in another operation.

The geometric measurements tool supports the applicable measurement options (for computing a volume, for example) used with the `measure` method in the section below.

See also [CentroidMeasurement](#) and [DistanceMeasurement](#) that creates parameters to be used in geometry features and when setting up the physics.

Measurement Methods

All the measurement methods assume that you have selected one or several entities that you want to measure.

To get the volume, area, or length of a selection of entities, enter

```
double vol = model.component(<ctag>).geom(<tag>).measure().getVolume();
```

To get the surface area surrounding selected 3D domains or the perimeter of selected 3D boundaries or 2D domains, enter

```
double bndVol = model.component(<ctag>).geom(<tag>).measure().getBoundaryVolume();
```

To get the area of a selection of faces (3D) or domains (2D), enter

```
double area = model.component(<ctag>).geom(<tag>).measure().getArea();
```

To get the surrounding boundary area of a selection of 3D domains, enter

```
double surrBndArea = model.component(<ctag>).geom(<tag>).measure().getBoundaryArea();
```

To get the perimeter of a selection of faces in 3D or 2D domains, enter

```
double perimeter = model.component(<ctag>).geom(<tag>).measure().getPerimeter();
```

To get the length of a selection of edges or 1D domains, enter

```
double length = model.component(<ctag>).geom(<tag>).measure().getLength();
```

The volumes, areas, and lengths are approximate because they are based on approximating the geometry with a triangular or polygonal mesh (corresponding to a rendering mesh with the rendering detail set to fine).

If you have selected two vertices, you can get their distance by entering

```
double[] d = model.component(<ctag>).geom(<tag>).measure().getVtxDistance();
```

`d[0]` is the distance, and `d[i]` is the distance in the *i*th coordinate (*i* = 1, 2, 3).

If you have selected one vertex, you can get its coordinates by entering

```
double[] coord = model.component(<ctag>).geom(<tag>).measure().getVtxCoord();
```

Select two adjacent edges (or any two straight edges) to get the angle between them.

```
double angle = model.component(<ctag>).geom(<tag>).measure().getEdgeAngle();
```

The method compares the tangents at the common vertex. For a pair of straight edges, the angle is computed even if the edges are not adjacent and the direction of the edges is ignored so that the returned angle is at most 90 degrees. If the angle is not defined for the selected edges, NaN is returned.

To get the total number of entities in the selected objects, enter

```
int[] entitiesPerDimension = model.component(<ctag>).geom(<tag>).measure().  
getNEntities();
```

To get the total number of finite voids in the selected objects or finalized geometry, enter

```
int finiteVoids = model.component(<ctag>).geom(<tag>).measure().getNFiniteVoids();
```

Selecting Objects and Entities to Measure

To select entities you want to measure, enter

```
model.component(<ctag>).geom(<tag>).measure().selection().init(entDim);  
model.component(<ctag>).geom(<tag>).measure().selection().set(<objname>,entities);
```

where *entDim* is the dimension of the entities, *<objname>* is the object name, and *entities* is an integer array containing the entity numbers.

For more information about selecting geometry objects, see [Geometry Object Selection Methods](#).

Inserting Geometry Sequences from File

To insert a geometry sequence from an MPH file, enter

```
model.component(<ctag>).geom(<tag>).insertFile(<filename>, <sequencename>);
```

where <filename> and <sequencename> are strings. The <filename> can also be a model location URI used to identify a model version in a Model Manager database.

To insert a geometry sequence from a different model component, enter

```
model.component(<ctag>).geom(<tag>).insertSequence(<ctag2>, <sequencename>);
```

where <ctag2> and <sequencename> are strings.

Example of Importing Geometry Sequences

The following sequence imports three different geometry sequences from two different files:

Code for Use with Java

```
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("g", 2);
g.insertFile("filename", "geom1");
ModelUtil.load("Model2", "filename2");
g.insertSequence("Model2", "geom1/wp1");
g.insertSequence("Model2", "geom1/wp2");
```

Code for Use with MATLAB

```
model.component.create('comp1');
g = model.geom.create('g', 2);
g.insertFile('filename', 'geom1');
ModelUtil.load('Model2', 'filename2');
g.insertSequence('Model2', 'geom1/wp1');
g.insertSequence('Model2', 'geom1/wp2');
```

Exporting Geometry to File

To export the finalized geometry to a file, enter

```
model.component(<ctag>).geom(<tag>).exportFinal(<filename>);
```

where *<filename>* is a string for a file system path or a file location URI referencing a file version in a Model Manager database.

To export selected geometry objects to a file, first select the objects to export using

```
model.component(<ctag>).geom(<tag>).export().selection().set(<objnames>);
```

Set the file format using

```
model.component(<ctag>).geom(<tag>).export().setType(<format>);
```

where *<format>* can be set to any of the type values in [Table 3-26](#).

Check which file format is set using

```
String formatType = model.component(<ctag>).geom(<tag>).export().getType();
```

Export the file by entering

```
model.component(<ctag>).geom(<tag>).export(<filename>);
```

The file can be of any of the following formats:

TABLE 3-26: VALID FILE FORMATS.

FILE FORMAT	NOTE	FILE EXTENSIONS	TYPE VALUE
COMSOL Multiphysics Binary		.mphbin	nativebin
COMSOL Multiphysics Text		.mphtxt	nativeascii
Parasolid Binary (3D)	1	.x_b, .xmt_bin	parasolidbin
Parasolid Text (3D)	1	.x_t, .xmt_txt	parasolidascii
ACIS Binary (3D)	1	.sab	acisbin
ACIS Text (3D)	1	.sat	acisascii
IGES File (3D)	1	.igs, .iges	iges
STEP File (3D)	1	.step, .stp	step
STL Binary (3D)		.stl	stlbin
STL Text (3D)		.stl	stlascii
DXF (2D)		.dxf	dxf
¹ This format requires a license for the CAD Import Module, Design Module, or a LiveLink product for a CAD package.			

Exporting to CAD File Formats

See the section [Export](#), [ExportFinal](#) in the documentation for the *CAD Import Module*.

Exporting to an STL File

An alternative way of setting the STL file formats is to use

```
model.component(<ctag>).geom(<tag>).export().setSTLFormat(<format>);
```


where `<format>` is string with only two allowed values: `binary` and `text`. The methods `setType(<format>)` and `setSTLFormat(<format>)` will set the correct value for the other, so you only need to use one of them.

Use the following methods to select domains or boundaries to export:

```
model.component(<ctag>).geom(<tag>).export().selection().init(<edim>);
model.component(<ctag>).geom(<tag>).export().selection().set(<objnames>, <entlst>);
```

Use the following methods to select objects to export:

```
model.component(<ctag>).geom(<tag>).export().selection().init();
model.component(<ctag>).geom(<tag>).export().selection().set(<objnames>);
```

Finish the export by using the following line

```
model.component(<ctag>).geom(<tag>).export(<filename>);
```

Compatibility for MPHBIN and MPHTXT in 2D and 3D

If you want to open a COMSOL Multiphysics geometry file in an earlier version of COMSOL Multiphysics, you might need to set the COMSOL Multiphysics version using

```
model.component(<ctag>).geom(<tag>).export().setCompat(<ver>);
```

where `<ver>` is a string `4.0` (only 3D), `4.0a` (only 3D), `4.2a`, `4.3b`, `5.1`, `5.5`, or `6.3`. Check the currently set version using

```
model.component(<ctag>).geom(<tag>).export().getCompat();
```

Advanced Settings for DXF Files in 2D

Set the curve representation for DXF export using

```
model.component(<ctag>).geom(<tag>).export().setDxfCurveRep(<rep>);
```

where `<rep>` is set to `exact`, `approxspline`, or `polygon`.

Get the curve representation for DXF export using the following method. The method returns a string value.

```
model.component(<ctag>).geom(<tag>).export().getDxfCurveRep();
```

Use the below method to set the approximation detail for the DXF export.

```
model.component(<ctag>).geom(<tag>).export().setDxfApproxDetail(<detail>);
```

where `<detail>` is `graphics`, `coarse`, `normal`, or `fine`. This is used when `getDxfCurveRep()` is `approxspline` or `polygon`.

Get approximation detail for DXF export. The method returns a string value.

```
model.component(<ctag>).geom(<tag>).export().getDxfApproxDetail();
```

Set how to handle edges surrounding domains using the following method

```
model.component(<ctag>).geom(<tag>).export().setDxfDomainHandling(<handling>);
```

where `<handling>` is set to `edgesdomainwise` or `intedgesonce`.

Get domain handling for DXF export. The method returns a string value.

```
model.component(<ctag>).geom(<tag>).export().getDxfDomainHandling();
```

Creating and Using Geometry Parts

For a description of geometry parts, see [Using Geometry Parts](#) in the *COMSOL Multiphysics Reference Manual*.

To create a geometry part, enter

```
model.geom().create(<tag>, "Part", sDim);
```

where *<tag>* is the parts tag, and *sDim* is its space dimension (1, 2, or 3).

To add an input parameter to the part, enter

```
model.geom(<tag>).inputParam().set(<name>, <expr>, <descr>);
```

where the description *<descr>* can be omitted.

Similarly, add a local parameter by

```
model.geom(<tag>).localParam().set(<name>, <expr>, <descr>);
```

The containers `model.component(<ctag>).geom(<tag>).inputParam()` and `model.component(<ctag>).geom(<tag>).localParam()` also support the other methods listed in [model.param\(\)](#) and [model.result\(\).param\(\)](#).

To add a local function of the type *<type>*, enter

```
model.geom(<tag>).func().create(<ftag>, <type>);
```

See [model.func\(\)](#) for more information. To return a list of all local functions, enter

```
model.geom(<tag>).func();
```



The **Finalize** feature is optional for geometry parts but is necessary if you want to add virtual operations to the part sequence. It then needs to be created, as specified in [Finalize](#).

To load one or more geometry parts, enter

```
model.geom().load(<tags>, <filename>, <partTagsInFile>);
```

where *<tags>* is a list of part tags, *<filename>* is the filename of the MPH file where the parts are defined or a model location URI used to identify a model version in a Model Manager database, and *<partTagsInFile>* is a list of the parts' tags in that file. If `model.geom(<gtag>)` is a geometry part, `model.geom(<gtag>).loaded()` returns true if the part was created by loading it from a file.

To get the filename of a loaded part, enter

```
model.geom(<gtag>).filename();
```

To change the filename of a loaded part, enter

```
model.geom(<gtag>).filename(<filename>);
```

where *<filename>* is the new filename or a model location URI used to identify a model version in a Model Manager database.

For a loaded part, to return the comments from the MPH file, enter

```
model.geom(<gtag>).commentsInFile();
```

For a loaded part, to return the last modification date from the MPH file, enter

```
model.geom(<gtag>).dateModifiedInFile();
```

To get the tag that a loaded part has in the MPH file, enter

```
model.geom(<gtag>).tagInFile();
```

To get the label that a loaded part has in the MPH file, enter

```
model.geom(<gtag>).labelInFile();
```

For a loaded part, to return the version from the MPH file, enter

```
model.geom(<gtag>).versionInFile();
```

To reload (update) a loaded part after its definition has been changed, enter

```
model.geom(<gtag>).reload();
```

To make keeping selections active by default in part instances, use

```
model.geom(<gtag>).keepSelInInstances(true);
```

By default, this property is set to `false`. To see the current setting, use

```
boolean keep = model.geom(<gtag>).keepSelInInstances();
```

To call a geometry part in a component geometry, add a `PartInstance` feature:

```
model.component(<ctag>).geom(<gtag>).create(<ftag>, "PartInstance");
```

See [PartInstance](#) for details.

To debug a call to part, you can step into it using

```
model.component(<ctag>).geom(<gtag>).stepInto(<ftag>);
```

You can then apply the usual geometry sequence methods on the local part instance, for example,

```
model.component(<ctag>).geom(<gtag>).feature(<ftag>).geom().run(<ftag2>);
```

to build the feature `<ftag2>` in the local part instance.

To make the part a part variant, enter

```
model.geom(<tag>).partVariant(true);
```

To check if the part is a part variant, enter

```
model.geom(<tag>).partVariant();
```

Geometry Commands

Geometry Commands (A to L)

- `AdjacentSelection`
- `Array`
- `BallSelection`, `BoxSelection`,
`CylinderSelection`, `DiskSelection`
- `BezierPolygon`
- `Block`
- `CentroidMeasurement`
- `Chamfer`
- `Circle`
- `CircularArc`
- `CollapseEdges`
- `CollapseFaces`
- `Compose`, `Union`, `Intersection`, `Difference`
- `CompositeCurve`
- `CompositeDomains`
- `CompositeEdges`
- `CompositeFaces`
- `Cone`
- `ConvertToSolid`, `ConvertToSurface`,
`ConvertToCurve`, `ConvertToPoint`
- `CrossSection`
- `CubicBezier`
- `Cylinder`
- `Delete`
- `DistanceMeasurement`
- `ECone`
- `EditObject`
- `Ellipse`
- `Ellipsoid`
- `ExplicitSelection`
- `Extract`
- `Extrude`
- `Fillet`
- `Finalize`
- `FromMesh`
- `Helix`
- `Hexahedron`
- `If`, `ElseIf`, `Else`, `EndIf`
- `IgnoreEdges`
- `IgnoreFaces`
- `IgnoreVertices`
- `Import DXF`
- `Import Geometry Sequence`
- `Import Mesh Part or Meshing Sequence`
- `Import mphbin/mphtxt`
- `InterpolationCurve`
- `Interval`
- `LineSegment`
- `LogicalExpressionSelection`

Geometry Commands (M to Z)

- [MergeEdges](#)
- [MergeFaces](#)
- [MergeVertices](#)
- [MeshControlDomains](#)
- [MeshControlEdges](#)
- [MeshControlFaces](#)
- [MeshControlVertices](#)
- [Mirror](#)
- [Move, Copy](#)
- [Offset](#)
- [ParameterCheck](#)
- [ParametricCurve](#)
- [ParametricSurface](#)
- [PartInstance](#)
- [Partition](#)
- [PartitionDomains](#)
- [PartitionEdges](#)
- [PartitionFaces](#)
- [Point](#)
- [Polygon](#)
- [Pyramid](#)
- [QuadraticBezier](#)
- [Rectangle](#)
- [RemoveDetails](#)
- [Revolve](#)
- [RigidTransform](#)
- [Rotate](#)
- [Scale](#)
- [Sphere](#)
- [Split](#)
- [Square](#)
- [Sweep](#)
- [Tangent](#)
- [Tetrahedron](#)
- [Thicken2D](#)
- [Torus](#)
- [UnionSelection, IntersectionSelection, DifferenceSelection, ComplementSelection](#)
- [WorkPlane](#)

AdjacentSelection

Create a selection of entities or objects that are adjacent to given selections.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"AdjacentSelection");  
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

`AdjacentSelection` creates a selection of all entities of dimension `outputdim` that are adjacent to at least one entity in the input selections. If the output selection has lower dimension than the input selections, you can use the `exterior` and `interior` properties to exclude or include output entities that are exterior/interior to the union of the input selections.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
entitydim	0 1 2 3	space dimension	Dimension of input entities.
exterior	on outside inside off	on	on: Select all exterior entities. outside: Select all exterior boundaries that are adjacent to the infinite region (connected component) of the complement of the domain selection. inside: Select all exterior boundaries that are adjacent to a finite region (connected component) of the complement of the domain selection. off: Select no exterior entities. Values outside and inside are only supported when entitydim is sdim and outputdim is sdim-1.
input	String[]	{}	Tags of input selections.
interior	on off	off	Include output entities that are interior to the union of the input selections.
outputdim	0 1 2 3	space dimension - 1	Dimension of entities to select.
selkeep	on off	on	Keep the selection within the geometry sequence.
selshow	on off	on	Show selection in physics, materials, and so on; in part instances; or in 3D from a plane geometry.
contributeto	String	none	Tag of cumulative selection to contribute to.

See [Selections of Geometric Entities](#) for general information about selections.

EXAMPLE

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("sph1", "Sphere");
g.run("sph1");
g.create("sel1", "ExplicitSelection");
g.feature("sel1").selection("selection").init(0);
g.feature("sel1").selection("selection").set("sph1", new int[]{4});
g.create("adjsel1", "AdjacentSelection");
g.feature("adjsel1").set("entitydim", 0);
g.feature("adjsel1").set("input", new String[]{"sel1"});
g.run("adjsel1");
g.create("del1", "Delete");
g.feature("del1").selection("input").named("adjsel1");
g.run("del1");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
```

```

g.create('sph1', 'Sphere');
g.run('sph1');
g.create('sel1', 'ExplicitSelection');
g.feature('sel1').selection('selection').init(0);
g.feature('sel1').selection('selection').set('sph1', 4);
g.create('adjsel1', 'AdjacentSelection');
g.feature('adjsel1').set('entitydim', 0);
g.feature('adjsel1').set('input', 'sel1');
g.run('adjsel1');
g.create('del1', 'Delete');
g.feature('del1').selection('input').named('adjsel1');
g.run('del1');

```

SEE ALSO

[BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#), [ExplicitSelection](#), [UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#)

Array

Create a block-shaped (3D), rectangular (2D, 3D), or linear array of geometry objects.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Array");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Array")` to create an array of geometry objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects to array. The default selection is empty.

The following properties are available:

TABLE 3-27: VALID PROPERTIES FOR ARRAY.

NAME	VALUE	DEFAULT	DESCRIPTION
displ	double[sdim]	1	Displacements in axis directions.
input	Selection		Objects to array.
propagatesel	on off	on	Propagate selections from input objects to moved objects.
size	int	1	Specify array size. Used for sdim=1.
type	linear rectangular three-dimensional	rectangular or three-dimensional	Specify the type of array for sdim 2 and 3. For sdim=2, use rectangular or linear. For sdim=3, use three-dimensional or linear.
fullsize	double[sdim]	{1,1} or {1,1,1}	Specify array size. Used when type is rectangular or three-dimensional.
linearsize	double	1	Specify array size. Used when type is linear.
indexattr	on off	off	Create index attributes.

TABLE 3-27: VALID PROPERTIES FOR ARRAY.

NAME	VALUE	DEFAULT	DESCRIPTION
indexattrtag	String		Tag of index attribute. Used if indexattr is on and the space dimension is 1 or type is linear.
indexattrtags	String[]		Tags of index attributes. Used if indexattr is on and type is rectangular (sdim=2) or three-dimensional (sdim=3).
selresult	on off	off	Create selections of all resulting objects of this feature.
selresultshow	all obj dom bnd edg pnt off	The highest available entity level except obj; usually dom.	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.

1D and when type is linear: A linear (oblique) array with `size` or `linearsize` copies of the input objects is constructed. The displacement between two consecutive copies is given by the vector `displ`. The names of the output objects are `ftag(i)`, where `ftag` is the tag of the feature, and `i` is a 1-based index. If there are more than one input object, the output objects are named `ftag(i,in)`, where `in` is a 1-based index corresponding to the input objects.

2D: If type is rectangular, a rectangular array with `fullsize[0]`-by-`fullsize[1]` copies of the input object is constructed. The *x*- and *y*-displacements are `displ[0]` and `displ[1]`, respectively. The names of the output objects are `ftag(i1,i2)`, where `ftag` is the name of the feature, and `i1` and `i2` are 1-based indices. If there are more than one input object, the output objects are named `ftag(i1,i2,in)`, where `in` is a 1-based index corresponding to the input objects.

3D: If type is three-dimensional, a three-dimensional (block shaped) array with `fullsize[0]`-by-`fullsize[1]`-by-`fullsize[2]` copies of the input object is constructed. The *x*-, *y*-, and *z*-displacements are `displ[0]`, `displ[1]`, and `displ[2]`, respectively. The names of the output objects are `ftag(i1,i2,i3)`, where `ftag` is the name of the feature, and `i1`, `i2`, and `i3` are 1-based indices. If there are more than one input object, the output objects are named `ftag(i1,i2,i3,in)`, where `in` is a 1-based index corresponding to the input objects.

The input object is deleted and an identical object is constructed as a part of the array.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Array](#).

The following attributes are available:

TABLE 3-28: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use <code>inherit</code> to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

In COMSOL Multiphysics 5.2, the `selresult` property replaced the `createselection` property. `createselection` is still supported for backward compatibility.

`model.geom(<tag>).create(<ftag>,"arrayr")` constructs an Array feature

EXAMPLE

The following sequence creates a block with four equally-sized holes:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("cyl1", "Cylinder");
g.create("arr1", "Array");
g.feature("arr1").selection("input").set("cyl1");
g.feature("arr1").set("displ", "4 4 0");
g.feature("arr1").set("size", "2 2 1");
g.create("blk1", "Block");
g.feature("blk1").set("size", "10 14 5");
g.feature("blk1").set("pos", "-3 -5 -4");
g.create("dif1", "Difference");
g.feature("dif1").selection("input").set("blk1");
g.feature("dif1").selection("input2").set("arr1");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('cyl1','Cylinder');
g.create('arr1','Array');
g.feature('arr1').selection('input').set('cyl1');
g.feature('arr1').set('displ','4 4 0');
g.feature('arr1').set('size','2 2 1');
g.create('blk1','Block');
g.feature('blk1').set('size','10 14 5');
g.feature('blk1').set('pos','-3 -5 -4');
g.create('dif1','Difference');
g.feature('dif1').selection('input').set('blk1');
g.feature('dif1').selection('input2').set('arr1');
g.run;
```

SEE ALSO

[Move](#), [Copy](#)

BallSelection, BoxSelection, CylinderSelection, DiskSelection

Create selections of geometric entities or objects that (partly) lie inside a ball, box, cylinder, or disk.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"BallSelection");
model.component(<ctag>).geom(<tag>).create(<ftag>,"BoxSelection");
model.component(<ctag>).geom(<tag>).create(<ftag>,"CylinderSelection");
model.component(<ctag>).geom(<tag>).create(<ftag>,"DiskSelection");
model.component(<ctag>).geom(<tag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).getType(property);
```

DESCRIPTION

The following general properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angletol	double	5	Angle tolerance for continuity evaluation.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
condition	intersects inside somevertex allvertices	intersects	Condition for inclusion of an entity.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
entitydim	-1 0 1 2 3	space dimension	Dimension of entities to select. -1 means Object.
groupcontang	on off	off	Continuous tangent mode.
input	String[]	{}	Tags of input selections, only used when inputent is selections.
inputent	all selections	all	Select among all entities or entities defined by the input property.
selkeep	on off	on	Keep the selection within the geometry sequence.
selshow	on off	on	Show selection in physics, materials, and so on; in part instances; or in 3D from a plane geometry.
contributeto	String	none	Tag of cumulative selection to contribute to.

For BallSelection, you define the ball using the following properties (using the geometry sequence's length unit):

PROPERTY	VALUE	DEFAULT	DESCRIPTION
posx	double	0	Center of ball, first coordinate.
posy	double	0	Center of ball, second coordinate.
posz	double	0	Center of ball, third coordinate.
r	double	0	Radius of ball.

For BoxSelection, you define the box using the following properties (using the geometry sequence's length unit):

PROPERTY	VALUE	DEFAULT	DESCRIPTION
xmax	double	inf	Maximum x-coordinate of box.
xmin	double	-inf	Minimum x-coordinate of box.
ymax	double	inf	Maximum y-coordinate of box.
ymin	double	-inf	Minimum y-coordinate of box.
zmax	double	inf	Maximum z-coordinate of box.
zmin	double	-inf	Minimum z-coordinate of box.

For CylinderSelection, you define the cylinder using the following properties (using the geometry sequence's length unit):

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angle1	double	0	Start angle.
angle2	double	360	End angle (default: 360 degrees; that is, a full cylinder).

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axistype	x y z cartesian spherical	z	Type of axis or coordinate system used for axis. The value is synchronized with axis.
axis	double[]	{0,0,0}	Direction of cylinder axis. Vector has length 3 if axistype is cartesian and length 2 if axistype is spherical. Not used if axistype is x, y, or z. See also ax3 and ax2.
ax2	double[]	{0,0}	Can be used instead of axis when axistype is spherical
ax3	double[]	{0,0,1}	Can be used instead of axis axis when axistype is cartesian
bottom	double	-inf	Coordinate of bottom face in local coordinate system.
pos	double[]	{0,0,0}	Base point.
r	double (nonnegative)	0	Outer radius.
rin	double (nonnegative)	0	Inner radius.
top	double	inf	Coordinate of top face in local coordinate system.

For DiskSelection, you define the disk using the following properties (using the geometry sequence's length unit):

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angle1	double	0	Start angle.
angle2	double	360	End angle (default: 360 degrees; that is, a full disk).
posx	double	0	Center of disk, first coordinate.
posy	double	0	Center of disk, second coordinate.
r	double (nonnegative)	0	Outer radius.
rin	double (nonnegative)	0	Inner radius.

You select the input entities or objects to select among using the properties `entitydim`, `inputent`, and `input`. For a boundary or edge selection in 2D or 3D, you can force the selection to select whole groups of entities by setting the property `groupcontang` to `on`. Each group consists of adjacent entities that meet at an angle less than `angletol`.

The output entities/objects are determined by the property `condition`:

- **intersects:** All entities/objects that intersect the ball/box/cylinder are included.
- **inside:** All entities/objects that are completely inside the ball/box/cylinder are included.
- **somevertex:** All entities/objects that have at least one adjacent vertex inside the ball/box/cylinder are included.
- **allvertices:** All entities/objects that have all adjacent vertices inside the ball/box/cylinder are included.

For **intersects** and **inside**, the rendering mesh is used for the calculation. You can set the resolution of the rendering mesh using

```
ModelUtil.setPreference("graphics.rendering.detail", <detail>);
```

where `<detail>` is `coarse`, `normal`, `fine`, or `wireframe`.

See [Selections of Geometric Entities](#) for general information about selections.

EXAMPLE

In a 10-by-10 array of squares, delete the squares that lie in the box $x > 9.5, y > 9.5$.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
g.create("sq1", "Square");
g.create("arr1", "Array");
g.feature("arr1").selection("input").set("sq1");
g.feature("arr1").set("fullsize", new int[]{10,10});
g.feature("arr1").set("displ", new double[]{2,2});
g.run("arr1");
g.create("boxsel1", "BoxSelection");
g.feature("boxsel1").set("entitydim", -1);
g.feature("boxsel1").set("xmin", 9.5);
g.feature("boxsel1").set("ymin", 9.5);
g.create("del1", "Delete");
g.feature("del1").selection("input").init();
g.feature("del1").selection("input").named("boxsel1");
g.run("del1");
// g.objectNames().length = 75
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
g.create('sq1', 'Square');
g.create('arr1', 'Array');
g.feature('arr1').selection('input').set('sq1');
g.feature('arr1').set('fullsize', [10,10]);
g.feature('arr1').set('displ', [2,2]);
g.run('arr1');
g.create('boxsel1', 'BoxSelection');
g.feature('boxsel1').set('entitydim', -1);
g.feature('boxsel1').set('xmin', 9.5);
g.feature('boxsel1').set('ymin', 9.5);
g.create('del1', 'Delete');
g.feature('del1').selection('input').init;
g.feature('del1').selection('input').named('boxsel1');
g.run('del1');
% length(g.objectNames)= 75
```

SEE ALSO

[AdjacentSelection](#), [ExplicitSelection](#), [UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#)

BezierPolygon

Create a curve or solid polygon consisting of Bézier segments in 2D or 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"BezierPolygon");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>, "BezierPolygon")` to create a Bézier polygon or a line segment. The following properties are available:

TABLE 3-29: VALID PROPERTY/VALUE PAIRS FOR BEZIERPOLYGON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
degree	int[] int	1	Degree of Bézier segments.
p	double[][]		Control points.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom in 2D edg in 3D.	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
type	solid open closed	solid (2D) open (3D)	Object type. solid is not available in 3D.
w	double[]		Weights.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

If type is open or closed, a curve consisting of line, quadratic, or cubic rational Bézier segments is constructed. If type is solid, the solid enclosed by such a closed polygon is constructed. If type is closed or solid, but the first and last control points are different, an extra linear segment is added to close the curve.

The degree of the *n*th segment is `degree[n]`, and it must be 1 (linear), 2 (quadratic), or 3 (cubic). The *n*th segment has `degree[n]+1` control points and weights. The weights are stored consecutively in the array `w`, which has length `degree[0]+...+degree[N-1]+N`, where *N* is the number of segments. The *i*th coordinates of the control points are stored consecutively in the array `p[i]`. Adjacent segments share the common control point, which means that `p[i]` has length `degree[0]+...+degree[N-1]+1`.

For a linear or cubic segment, the default weights are 1. For a quadratic segment, the default weights are 1, $1/\sqrt{2}$, 1.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [BezierPolygon](#).

The following attributes are available:

TABLE 3-30: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

From version 5.5, the BezierPolygon feature is deprecated. Instead of BezierPolygon, use the [CircularArc](#), [CompositeCurve](#), [CubicBezier](#), [InterpolationCurve](#), [Polygon](#), or [QuadraticBezier](#) feature.

EXAMPLE

Construct a solid triangle b1 and an elliptic arc b2:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("b1","BezierPolygon");
g.feature("b1").set("p", new double[][]{{0, 0, 2}, {1, 0, 0}});
g.create("b2","BezierPolygon");
g.feature("b2").set("type","open");
g.feature("b2").set("degree",2);
g.feature("b2").set("p", new double[][]{{0, 1, 0}, {1, 2, 0}});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('b1','BezierPolygon');
g.feature('b1').set('p', [[0, 0, 2]; [1, 0, 0]]);
g.create('b2','BezierPolygon');
g.feature('b2').set('type','open');
g.feature('b2').set('degree',2);
g.feature('b2').set('p', [[0, 1, 0]; [1, 2, 0]]);
g.run;
```

SEE ALSO

[CircularArc](#), [CompositeDomains](#), [CubicBezier](#), [Polygon](#), [QuadraticBezier](#)

Block

Create a right-angled solid block, surface block, our bounding box in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Block");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Block")` to create a block. The following properties are available:

TABLE 3-31: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	double[]	{0,0,1}	Direction of the edge on the local z-axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.

TABLE 3-31: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
base	corner center	corner	Positions the object either centered about pos or with one corner in pos.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
createpar	on off	off	Create parameters.
definedby	size bounding	size	Specify if the block is to be defined by its size and position or be a bounding box. See Table 3-32 when definedby is bounding.
layer	double[]		Thicknesses of layers.
layertop	on off	off	Apply layers on top.
layerbottom	on off	on	Apply layers on bottom.
layerleft	on off	off	Apply layers to the left.
layerright	on off	off	Apply layers to the right.
layerfront	on off	off	Apply layers on front.
layerback	on off	off	Apply layers on back.
size	double[]	{1,1,1}	Edge lengths.
pos	double[]	{0,0,0}	Position of the object.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
rot	double	0	Rotational angle about axis.
type	solid surface	solid	Object type.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

TABLE 3-32: ADDITIONAL PROPERTIES WHEN DEFINED BY IS BOUNDING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>orientdef</code>	<code>manual principal</code>	<code>principal</code>	Specify orientation definition.
<code>input</code>	<code>Selection</code>		Input objects or entities.
<code>keep</code>	<code>on off</code>	<code>on</code>	Keep input objects. Only used if <code>input</code> is an object selection.
<code>leftmargin</code>	<code>double</code>	<code>0</code>	Left margin around bounding box.
<code>rightmargin</code>	<code>double</code>	<code>0</code>	Right margin around bounding box.
<code>frontmargin</code>	<code>double</code>	<code>0</code>	Front margin around bounding box.
<code>backmargin</code>	<code>double</code>	<code>0</code>	Back margin around bounding box.
<code>bottommargin</code>	<code>double</code>	<code>0</code>	Bottom margin around bounding box.
<code>topmargin</code>	<code>double</code>	<code>0</code>	Top margin around bounding box.

The following attributes are available:

TABLE 3-33: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
<code>construction</code>	<code>on off</code>	<code>off</code>	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"block2")` constructs a solid block.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"block3")` constructs a surface block.

The following properties are also available:

TABLE 3-34: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>ax2</code>	<code>double[]</code>	<code>{0,0}</code>	Alias for <code>axis</code> when <code>axistype</code> is spherical.
<code>ax3</code>	<code>double[]</code>	<code>{0,0,1}</code>	Alias for <code>axis</code> when <code>axistype</code> is cartesian.
<code>lx, ly, lz</code>	<code>double</code>	<code>1</code>	Alias for <code>size</code> .
<code>x, y, z</code>	<code>double</code>	<code>0</code>	Alias for <code>pos</code> .

The property `const` is no longer available.

EXAMPLE

The following commands create a solid and surface block, where the position is defined in the two alternative ways.

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("b1","Block");
g.feature("b1").set("size","1 2.1 0.5");
g.feature("b1").set("base","center");
g.feature("b1").set("pos","1 0 1");
g.feature("b1").set("axis","1 0 0");
g.feature("b1").set("rot",30);
double[] a = g.feature("b1").getDoubleArray("pos");
g.create("b2","Block");
g.feature("b2").set("type","surface");
g.feature("b2").set("size","1 2.1 0.5");
```



```
g.feature("b2").set("pos",a);
String b = g.feature("b2").getString("pos");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('b1','Block');
g.feature('b1').set('size','1 2.1 0.5');
g.feature('b1').set('base','center');
g.feature('b1').set('pos','1 0 1');
g.feature('b1').set('axis','1 0 0');
g.feature('b1').set('rot',30);
a = g.feature('b1').getDoubleArray('pos');
g.create('b2','Block');
g.feature('b2').set('type','surface');
g.feature('b2').set('size','1 2.1 0.5');
g.feature('b2').set('pos',a);
b = g.feature('b2').getString('pos');
```

SEE ALSO

[Hexahedron](#)

CentroidMeasurement

Create parameters for the average position of a selection of vertices.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CentroidMeasurement");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CentroidMeasurement")` to create parameters for the average position of vertices.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("ent").set()` to select the vertices to measure. The default selection is empty.

Use `double[] parvalues = model.component(<ctag>).geom(<tag>).feature(<ftag>).getDoubleArray("parval")`; to access the parameter values.

TABLE 3-35: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ent	Selection		Specify the vertex selection.
parname	String[]		Parameter names. The array has the length of the space dimension, sdim
parval	double[]		To access the parameter values (read-only).

SEE ALSO

[DistanceMeasurement](#), [Measurement Methods](#)

Chamfer

Create flattened corners in 2D objects. The Design Module also supports 3D chamfers.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Chamfer");  
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Chamfer")` to chamfer corners in 2D.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("point")` to select the corners to chamfer. The default selection is empty.

TABLE 3-36: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
dist	double	0	Distance from vertex to chamfer.
point	Selection		Vertices to chamfer.
pointinsketch	Selection		Vertices to chamfer in sketch geometry.
selectinsketch	on off	on	Determines whether you can use the pointinsketch property. It can be off in models made in version 6.0 or earlier.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The point property contains the vertices to chamfer in the usual geometry visualization. The pointinsketch property contains the vertices to chamfer in the sketch visualization. These properties are automatically kept synchronized.

Additional properties are available with the Design Module, see [Chamfer](#).

The following attributes are available:

TABLE 3-37: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

EXAMPLE

Chamfer a rectangle.

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("r1","Rectangle");
g.create("cha1","Chamfer");
g.feature("cha1").selection("point").set("r1(1)",new int[]{1,2,3,4});
g.feature("cha1").set("dist",0.1);
g.run();
```

Code for use MATLAB

```
Model model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('r1','Rectangle');
g.create('cha1','Chamfer');
g.feature('cha1').selection('point').set('r1(1)',{1,2,3,4});
g.feature('cha1').set('dist',0.1);
g.run;
```

DIAGNOSTICS

If a chamfer cannot be created according to the specified properties, this vertex is ignored. When the chamfers generate intersections with other edges in the geometry, an error message is given.

SEE ALSO

[Fillet](#)

Circle

Create a circle, disk, or bounding disk in 2D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Circle");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Circle")` to create a disk in 2D. The following properties are available:

TABLE 3-38: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angle	double	360	Circle sector angle.
base	corner center	center	Positions the object either centered about pos or with the lower-left corner of a surrounding box in pos
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
createpar	on off	off	Create parameters.

TABLE 3-38: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
definedby	size bounding	size	Specify if the circle is to be defined by its size and position or be a bounding disk. See Table 3-39 when definedby is bounding.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
r	double	1	Radius.
rot	double	0	Rotational angle about pos.
type	solid curve	solid	Object type.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances/3D (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

TABLE 3-39: ADDITIONAL PROPERTIES WHEN DEFINEDBY IS BOUNDING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
centerdef	manual centroid	centroid	Specify center definition.
input	Selection		Input objects or entities.
keep	on off	on	Keep input objects. Only used if input is an object selection.
margin	double	0	Margin around bounding disk.

Additional properties are available with the Design Module, see [Circle](#).

The following attributes are available:

TABLE 3-40: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"circ2")` creates a solid disk.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"circ1")` creates a circle curve.

The following properties are also available:

TABLE 3-41: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
x, y	double	0	Alias for pos.

The property `const` is no longer available.

EXAMPLE

The sequence below creates a unit disk (solid circle object).

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("c1","Circle");
g.feature("c1").set("pos",new double[] {2,3});
String base = g.feature("c1").getString("base");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('c1','Circle');
g.feature('c1').set('pos',[2,3]);
base = g.feature('c1').getString('base');
g.run;
```

SEE ALSO

[Ellipse](#)

CircularArc

Create a circular arc in 2D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CircularArc");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CircularArc")` to create a circular arc in 2D. The following properties are available:

TABLE 3-42: VALID PROPERTY/VALUE PAIRS FOR A CIRCULAR ARC.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angle1	double	0	Start angle (in degrees).
angle2	double	90	End angle (in degrees).
center	double[2]	{0,0}	Center coordinates.
clockwise	on off	off	Clockwise direction.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
point1	double[2]	{1,0}	Starting point.
point2	double[2]	{0,1}	Endpoint.

TABLE 3-42: VALID PROPERTY/VALUE PAIRS FOR A CIRCULAR ARC.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
specify	center endsr endsangle1 endsangle2	center	Properties to specify.
r	double	1	Radius.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
shortarc	on off	on	Use the shorter arc.
type	solid curve	solid	Object type.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [CircularArc](#).

The following attributes are available:

TABLE 3-43: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The sequence below creates a circular arc that is a half circle with the center in (0.5, 0.5) and endpoints in (0, 0) and (1, 1), specified using the center, radius, and angles.

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("ca1", "CircularArc");
g.feature("ca1").set("center", new double[] {0.5, 0.5});
g.feature("ca1").set("r", 0.707106781187);
g.feature("ca1").set("angle1", 225);
g.feature("ca1").set("angle2", 45);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('ca1','CircularArc');
g.feature('ca1').set('center',[0.5, 0.5]);
g.feature('ca1').set('r', 1/sqrt(2));
g.feature('ca1').set('angle1', 225);
g.feature('ca1').set('angle2', 45);
g.run;
```

SEE ALSO

[Circle](#), [CompositeCurve](#)

CollapseEdges

Collapse edges.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CollapseEdges");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CollapseEdges")` to collapse edges.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the edges to collapse. The default selection is empty.

The feature collapses an edge by removing it, merging its adjacent vertices to the vertex with lowest index, and reconnecting the adjacent edges to the merged vertex.

The output object is a virtual geometry.

The following properties are available:

TABLE 3-44: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Edges to collapse.
ignoremerged	on off	on	Specifies if the operation tries to ignore the resulting merged vertices.

SEE ALSO

[MergeVertices](#), [CollapseFaces](#), [CollapseFaceRegions](#)

CollapseFaces

Collapse faces.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CollapseFaces");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CollapseFaces")` to collapse faces.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the faces to collapse. The default selection is empty.

The feature collapses a face by removing it, merging its adjacent opposite edges into one or more edges or collapsing all adjacent edges into one vertex, and reconnecting the adjacent faces to the merged edges or vertex.

The output object is a virtual geometry.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
collvxtol	auto manual	auto	Use an automatic or manual tolerance for the maximum perimeter of a face to be collapsed into a vertex.
input	Selection		Faces to collapse.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ignoremerged	on off	off	Specifies if the operation tries to ignore the resulting merged entities.
maxfaceperimeter	double	0.001	The maximum perimeter of a face to be collapsed into a vertex when collvxtol is set to manual.

SEE ALSO

[MergeEdges](#), [CollapseEdges](#), [CollapseFaceRegions](#)

CollapseFaceRegions

Collapse face regions.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CollapseFaceRegions");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CollapseFaceRegions")` to collapse narrow face regions.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the faces where narrow regions should be collapsed. The default selection is empty.

The feature collapses narrow face regions by determining narrow regions of a face and then collapsing those resulting sliver faces.

The output object is a virtual geometry.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
input	Selection		Faces for which narrow regions should be collapsed.
maxwidth	double	0.001	The maximum width of a face region to be collapsed when narrowtol is set to manual.
narrowtol	auto manual	auto	Use an automatic or manual tolerance for the maximum width of a face region to be collapsed.

SEE ALSO

[MergeEdges](#), [CollapseEdges](#), [CollapseFaces](#)

Compose, Union, Intersection, Difference

Compose objects using a Boolean set formula.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Compose");
model.component(<ctag>).geom(<tag>).create(<ftag>,"Union");
model.component(<ctag>).geom(<tag>).create(<ftag>,"Intersection");
model.component(<ctag>).geom(<tag>).create(<ftag>,"Difference");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,operationName)` to combine geometric objects in different ways.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property)` to select the objects to combine. The default selection is empty.

The following properties are available:

TABLE 3-45: VALID PROPERTIES FOR THE COMPOSE AND BOOLEAN OPERATIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
absrepairtol	double	<code>...geom(<tag>).absRepairTol()</code>	Absolute repair tolerance.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
formula	String		Set formula (only for Compose feature).
input	Selection		Objects to compose.
input2	Selection		Objects to subtract (only for the Difference feature).
intbnd	on off	on	Keep interior boundaries.
keep	on off	off	Keep input objects (not for the Difference feature).
keepadd	on off	off	Keep input objects to add (only for the Difference feature).
keeplowerdim	on off	off	Keep lower-dimensional intersections (only for the Intersection feature).
keepssubtract	on off	off	Keep input objects to subtract (only for the Difference feature).
repairtol	double	<code>...geom(<tag>).repairTol()</code>	Relative repair tolerance, relative to size of union of inputs.
repairtoltype	auto relative absolute	<code>...geom(<tag>).repairTolType()</code>	Repair tolerance type: automatic, relative, or absolute.
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-45: VALID PROPERTIES FOR THE COMPOSE AND BOOLEAN OPERATIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
contributeto	String	none	Tag of cumulative selection to contribute to.

The following Boolean operation is performed:

- For **Compose**, the input objects are combined using the set formula in the property formula. The operators +, *, and - correspond to the set operations union, intersection, and difference, respectively. The precedence of the operators + and - are the same. * has higher precedence.
- For **Union**, the objects in input are united.
- For **Intersection**, the objects in input are intersected.
- For **Difference**, the objects in input2 are subtracted from the union of the objects in input to form a set difference.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-46: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

The following properties are also supported, see the [Delete](#) feature:

TABLE 3-47: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
edge	all none	none	Delete isolated edges on a face (3D). Delete interior edges and edges not adjacent to a domain (2D; alias for indbnd).
face	all none	none	Delete interior faces (3D; alias for intbnd).
point	all none	none	Delete isolated vertices (points) on a face (3D) or in a domain (2D).

The property out is no longer available.

SEE ALSO

[ConvertToSolid](#), [ConvertToSurface](#), [ConvertToCurve](#), [ConvertToPoint](#), [Finalize](#), [Partition](#)

CompositeCurve

Form composite curves in 2D that has a sequence of child features of the following types: Polygon, CircularArc, InterpolationCurve, QuadraticBezier, and CubicBezier.

SYNTAX

```
model.component(<ctag>).geom(<tag>).feature().compositeCurves(<ftags>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).feature().
    removeCurveComponents(<childtags>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

String remaining = model.component(<ctag>).geom(<tag>).feature().compositeCurves(<ftags>) creates or extends a CompositeCurve or Polygon feature by joining the curve features tagged <ftags>. These input features must be of one of the following types: Polygon, CircularArc, InterpolationCurve, QuadraticBezier, CubicBezier, or CompositeCurve. The returned string is the tag of the created or remaining feature.

String[] remaining = model.component(<ctag>).geom(<tag>).feature(<ftag>).feature().removeCurveComponents(<childtags>) removes the children with the given tags from the geometry feature list of the CompositeCurve feature tagged <ftag>. This can result in the CompositeCurve feature being split into several features. The remaining children can be reordered. The return array contains the tags of the remaining or created features.

The following properties are available:

TABLE 3-48: VALID PROPERTIES FOR COMPOSITECURVE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
type	solid curve	curve	Object type (in the case when the curve is closed).

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-49: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

Create a composite curve that includes two previously created circular arcs, "ca1" and "ca2".

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom", 2);
g.create("ca1", "CircularArc");
g.feature("ca1").set("angle1", 225);
g.feature("ca1").set("angle2", 45);
g.create("ca2", "CircularArc");
g.feature("ca2").set("angle1", 225);
g.feature("ca2").set("angle2", 180);
g.feature().compositeCurves("ca1", "ca2");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom', 2);
g.create('ca1', 'CircularArc');
g.feature('ca1').set('angle1', 225);
g.feature('ca1').set('angle2', 45);
g.create('ca2', 'CircularArc');
g.feature('ca2').set('angle1', 225);
g.feature('ca2').set('angle2', 180);
g.feature().compositeCurves(['ca1'; 'ca2']);
g.run;
```

SEE ALSO

[CircularArc](#), [CubicBezier](#), [InterpolationCurve](#), [Polygon](#), [QuadraticBezier](#)

CompositeDomains

Form composite domains.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CompositeDomains");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CompositeDomains")` to form composite domains.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the domains to composite. The default selection is empty.

The feature forms a composite domain for each connected domain component of the selected domains by ignoring the boundaries between the domains. The output object is a virtual geometry.

The following properties are available:

TABLE 3-50: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
ignoreadj	on off	on	Ignore edges (3D only) and vertices on boundary.
input	Selection		Edges to composite.
keepformesh	on off	off	Keep input domains for mesh control.

Use `ignoreadj` to specify if the feature also removes the ignorable edges (3D only) and vertices on the boundary of each resulting composite domain.

Use `keepformesh` to keep the input domains while meshing, to help you in constructing the mesh.

EXAMPLE

Create a composite domain of domain 2 and 3.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom", 2);
g.create("r1", "Rectangle");
g.create("c1", "Circle");
g.run("fin");
g.create("cmd1", "CompositeDomains");
g.feature("cmd1").selection("input").set("fin", 2, 3);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom', 2);
g.create('r1', 'Rectangle');
g.create('c1', 'Circle');
g.run('fin');
g.create('cmd1', 'CompositeDomains');
g.feature('cmd1').selection('input').set('fin', 2, 3);
g.run;
```

SEE ALSO

[CompositeEdges](#), [CompositeFaces](#), [IgnoreEdges](#), [IgnoreFaces](#)

CompositeEdges

Form composite edges.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>, "CompositeEdges");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property, <value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>, "CompositeEdges")` to form composite edges.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the edges to concatenate. The default selection is empty.

The feature forms a composite edge for each connected edge component (of manifold type) of the selected edges by ignoring the vertices between the edges. The output object is a virtual geometry.

The following properties are available:

TABLE 3-51: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Edges to composite.
keepformesh	on off	off	Keep input edges for mesh control.

Use `keepformesh` to keep the input edges while meshing, to help you in constructing the mesh.

Note that the operation never forms composite edges that are closed loops or periodic, that is, every resulting edge has distinct start and end vertices.

EXAMPLE

Compose edges 2 and 4 of a circle into one edge.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
g.create("c1", "Circle");
g.run("fin");
g.create("cme1", "CompositeEdges");
g.feature("cme1").selection("input").set("fin", 2, 4);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
g.create('c1', 'Circle');
g.run('fin');
g.create('cme1', 'CompositeEdges');
g.feature('cme1').selection('input').set('fin', 2, 4);
g.run;
```

SEE ALSO

[CompositeDomains](#), [CompositeFaces](#), [IgnoreVertices](#)

CompositeFaces

Form composite faces.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CompositeFaces");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CompositeFaces")` to form composite faces.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the faces to concatenate. The default selection is empty.

The feature forms a composite face for each connected face component (of manifold type) of the selected faces by ignoring the edges between the faces. The output object is a virtual geometry.

The following properties are available:

TABLE 3-52: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Faces to composite.
ignorevtx	on off	on	Ignore vertices on boundary.
keepformesh	on off	off	Keep input faces for mesh control.

Use `ignorevtx` to specify if the feature also removes the ignorable vertices on the boundary of each resulting composite face.

Use `keepformesh` to keep the input faces while meshing, to help you in constructing the mesh.

EXAMPLE

A COMSOL Multiphysics standard cone has six faces. Using the following composite face operation, the result is a cone with three faces: top, bottom, and side.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("cone1", "Cone");
g.run("fin");
g.create("cmf1", "CompositeFaces");
g.feature("cmf1").selection("input").set("fin", 1, 2, 5, 6);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('cone1', 'Cone');
g.run('fin');
g.create('cmf1', 'CompositeFaces');
g.feature('cmf1').selection('input').set('fin', [1, 2, 5, 6]);
g.run;
```

SEE ALSO

[CompositeDomains](#), [CompositeEdges](#), [IgnoreEdges](#)

Cone

Create a right circular cone or cone frustum (conical frustum, truncated cone) in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Cone");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Cone")` to create a cone. The following properties are available:

TABLE 3-53: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ang	double	arctan(1/2) (about 26.565 degrees)	The cone's semiangle; that is, the angle between the axis and a generator of the conical surface.
axis	double[]	{0,0,1}	Direction of the axis. Vector has length 3 if <code>axistype</code> is cartesian, and length 2 if <code>axistype</code> is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with <code>axis</code> .

TABLE 3-53: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
h	double	1	Height.
layer	double[]		Thicknesses of layers.
layertop	on off	off	Apply layers on top.
layerbottom	on off	off	Apply layers on bottom.
layerside	on off	on	Apply layers on side.
pos	double[]	{0,0,0}	Center of the bottom circle.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
r	double	1	Radius of bottom circle.
rot	double	0	Rotational angle about axis.
rtop	double	0.5	Radius of top circle.
specifytop	angle radius	angle	If axistype is angle, the radius of the top circle is given by the ang property. If axistype is radius, the radius of the top circle is given by the rtop property.
type	solid surface	solid	Object type.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-54: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"cone3")` creates a solid cone.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"cone2")` creates a surface cone.

The following properties are also available:

TABLE 3-55: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ax2	double[]	{0,0}	Alias for axis when axistype is spherical.
ax3	double[]	{0,0,1}	Alias for axis when axistype is cartesian.
x, y, z	double	0	Alias for pos.

The property `const` is no longer available.

EXAMPLE

Create a cone with an apex:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
double h = 3;
double r = 2;
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.angularUnit("rad");
g.create("c1","Cone");
g.feature("c1").set("r",r);
g.feature("c1").set("h",h);
g.feature("c1").set("ang", Math.atan(r/h));
double ang = g.feature("c1").getDouble("ang");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
h = 3;
r = 2;
g = model.component('comp1').geom.create('geom1',3);
g.angularUnit('rad');
g.create('c1','Cone');
g.feature('c1').set('r',r);
g.feature('c1').set('h',h);
g.feature('c1').set('ang', atan2(r,h));
ang = g.feature('c1').getDouble('ang');
```

Create a truncated and rotated cone:

Code for Use with Java

```
g.create("c2","Cone");
g.feature("c2").set("pos", "1 -2 4");
g.feature("c2").set("axis", "1 -1 0.3");
g.feature("c2").set("rot",Math.PI/3);
g.run();
```

Code for Use with MATLAB

```
g.create('c2','Cone');
g.feature('c2').set('pos', '1 -2 4');
g.feature('c2').set('axis', '1 -1 0.3');
g.feature('c2').set('rot', pi/3);
g.run;
```

SEE ALSO

[Cylinder](#), [ECone](#)

ConvertToSolid, ConvertToSurface, ConvertToCurve, ConvertToPoint

Unite and convert objects to a solid, surface, curve, or point object.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"ConvertToSolid");
model.component(<ctag>).geom(<tag>).create(<ftag>,"ConvertToSurface");
model.component(<ctag>).geom(<tag>).create(<ftag>,"ConvertToCurve");
model.component(<ctag>).geom(<tag>).create(<ftag>,"ConvertToPoint");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,convertOperation)` to reduce or extend the topological dimension of objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects to convert. The default selection is empty.

The following properties are available:

TABLE 3-56: VALID PROPERTIES FOR THE CONVERSION OPERATIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
absrepairtol	double	<code>...geom(<tag>).absRepairTol()</code>	Absolute repair tolerance.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
input	Selection		Objects to convert.
keep	on off	off	Keep input objects.
repairtol	double	<code>...geom(<tag>).repairTol()</code>	Relative repair tolerance, relative to size of union of inputs. Only used if unite is on.
repairtoltype	auto relative absolute	<code>...geom(<tag>).repairTolType()</code>	Repair tolerance type: automatic, relative, or absolute. Only used if unite is on.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-56: VALID PROPERTIES FOR THE CONVERSION OPERATIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd edg pnt off	dom for ConvertToSolid; bnd for ConvertToSurface; bnd (2D) or edg (3D) for ConvertToCurve; and pnt for ConvertToPoint	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
unite	on off	on	Union of input objects.

The input objects are united, and the resulting object is then converted to the requested type.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-57: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Compose](#), [Union](#), [Intersection](#), [Difference](#)

CrossSection

Create a 2D geometry from a cross section of a 3D geometry.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"CrossSection");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

In a 2D geometry, use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CrossSection")` to create 2D geometry objects by intersecting 3D geometry objects with a work plane. Select the work plane using the `workplane` property, whose value is the 3D sequence's tag followed by a slash and the work-plane feature's tag, for example `geom1/wp1`. By default, you get the last work plane in the last 3D geometry.

In a 2D sequence of a work-plane feature, use `model.component(<ctag>).geom(<tag>).feature(<wptag>).geom().create(<ftag>,"CrossSection")` to create 2D geometry objects by intersecting 3D geometry objects with the work plane.

By default, you get the intersection for all 3D objects that were generated by the features preceding the work-plane feature. To select a subset of these objects, set the `intersect` property to `selected`, and use the property input to select the 3D objects to intersect.

It is recommended to add a command to run the node before building the `CrossSection`. Building the previous feature is done automatically in the user interface, but needs to be done manually in the API to make sure the geometry is updated if you, for example, run a parametric sweep changing dimensions in the geometry.

The following properties are available:

TABLE 3-58: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
absrepairtol	double	<code>...geom(<tag>).absRepairTol()</code>	Absolute repair tolerance.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
input	Selection	empty	Selection of objects to intersect.
intersect	all selected	all	Intersect all objects or selected objects.
repairtol	double	<code>...geom(<tag>).repairTol()</code>	Relative repair tolerance, relative to size of each input object.
repairtoltype	auto relative absolute	<code>...geom(<tag>).repairTolType()</code>	Repair tolerance type: automatic, relative, or absolute.
selffrom3D	true false	false	Create selections from the 3D geometry.
selffrom3dshow	true false	false	Show created selections from 3D in, for example, material and physics settings. Not available for in work planes.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
workplane	String		Work plane to intersect with.

The following attributes are available:

TABLE 3-59: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

EXAMPLE

Create a work plane though the axis of a torus. In a 2D axisymmetric geometry, create the cross section of the torus using the work plane. Note that the last `run()` command removes the part of the cross section that falls in the region $r < 0$.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g1 = model.component("comp1").geom().create("geom1", 3);
g1.create("tor1", "Torus");
g1.run("tor1");
g1.create("wp1", "WorkPlane");
g1.feature("wp1").set("planetype", "circularedge");
```

```

g1.feature("wp1").selection("circedge").set("tor1", 15);

model.component().create("comp2");
GeomSequence g2 =model.component("comp2").geom().create("geom2", 2);
g2.axisymmetric(true);
g2.create("cro1", "CrossSection");
g2.run("cro1"); // Two circles
g2.run(); // One circle

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g1 = model.component('comp1').geom.create('geom1', 3);
g1.create('tor1', 'Torus');
g1.run('tor1');
g1.create('wp1', 'WorkPlane');
g1.feature('wp1').set('planetype', 'circularedge');
g1.feature('wp1').selection('circedge').set('tor1', 15);

model.component.create('comp2');
g2 = model.component('comp2').geom.create('geom2', 2);
g2.axisymmetric(true);
g2.create('cro1', 'CrossSection');
g2.run('cro1'); % Two circles
g2.run(); % One circle

```

SEE ALSO

[Projection](#), [WorkPlane](#)

CubicBezier

Create a rational Bézier curve of degree 3 in 2D or 3D.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"CubicBezier");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"CubicBezier")` to create a cubic Bézier curve in 2D or 3D. The following properties are available:

TABLE 3-60: VALID PROPERTY/VALUE PAIRS FOR A CUBIC BÉZIER CURVE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
p	double[sdim][4]	0	Control points.
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-60: VALID PROPERTY/VALUE PAIRS FOR A CUBIC BÉZIER CURVE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
w	double[4]	1	Weights for the control points.
workplanesrc	this part-instance feature	this	Par-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [CubicBezier](#).

The following attributes are available:

TABLE 3-61: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following commands creates a cubic Bézier curve in 2D with control points in (0, 0); (1, 3); (2, -2); and (3, 0):

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("cb1", "CubicBezier");
g.feature("cb1").setIndex("p", 1, 0, 1);
g.feature("cb1").setIndex("p", 3, 1, 1);
g.feature("cb1").setIndex("p", 2, 0, 2);
g.feature("cb1").setIndex("p", -2, 1, 2);
g.feature("cb1").setIndex("p", 3, 0, 3);
g.run();
```

The zero values are default values and are therefore not set.

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('cb1','CubicBezier');
g.feature('cb1').setIndex('p', 1, 0, 1);
g.feature('cb1').setIndex('p', 3, 1, 1);
g.feature('cb1').setIndex('p', 2, 0, 2);
g.feature('cb1').setIndex('p', -2, 1, 2);
g.feature('cb1').setIndex('p', 3, 0, 3);
g.run;
```

The zero values are default values and are therefore not set.

SEE ALSO

[CompositeCurve](#), [QuadraticBezier](#)

Cylinder

Create a solid cylinder, hollow (surface) cylinder, or bounding cylinder in 3D. The cylinder is a right circular cylinder; that is, a cylinder that has circles as bases aligned one directly above the other.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Cylinder");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Cylinder")` to create a cylinder. The following properties are available:

TABLE 3-62: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	double[]	{0,0,1}	Direction of the axis. Vector has length 3 if <code>axistype</code> is cartesian, and length 2 if <code>axistype</code> is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with <code>axis</code> .
createpar	on off	off	Create parameters.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the <code>customcolor</code> property. Coloring is only available when <code>selresult</code> is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when <code>color</code> is set to custom.
definedby	size bounding	size	Specify if the cylinder is to be defined by its size and position or be a bounding cylinder. See Table 3-63 when <code>definedby</code> is bounding.
layer	double[]		Thicknesses of layers.
layertop	on off	off	Apply layers on top.
layerbottom	on off	off	Apply layers on bottom.
layerside	on off	on	Apply layers on side.
h	double	1	Height.
pos	double[]	{0,0,0}	Center of the bottom circle.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when <code>postype</code> is vertex.
r	double	1	Radius of bottom circle.
rot	double	0	Rotational angle about axis.
type	solid surface	solid	Object type.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances (used if <code>sellayer</code> is on)
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-62: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

TABLE 3-63: ADDITIONAL PROPERTIES WHEN DEFINEDBY IS BOUNDING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axisdef	manual principallongest principalshortest	principallongest	Specify axis definition.
axispos	double[]	{0,0,0}	Point on axis. Used when axisdef is manual.
input	Selection		Input objects or entities.
keep	on off	on	Keep input objects. Only used if input is an object selection.
radialmargin	double	0	Margin around bounding cylinder.
bottommargin	double	0	Bottom margin around bounding cylinder.
topmargin	double	0	Top margin around bounding cylinder.

The following attributes are available:

TABLE 3-64: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"cylinder3")` creates a solid cylinder.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"cylinder2")` creates a surface cylinder.

The following properties are also available:

TABLE 3-65: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ax2	double[]	{0,0}	Alias for axis when axistype is spherical
ax3	double[]	{0,0,1}	Alias for axis when axistype is cartesian
x, y, z	double	0	Alias for pos

The property const is no longer available.

EXAMPLE

The following commands generate a surface cylinder and a solid cylinder:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.angularUnit("rad");
g.create("c2","Cylinder");
g.feature("c2").set("type","surface");
g.feature("c2").set("r",0.5);
g.feature("c2").set("h",4);
g.feature("c2").set("pos","1 1 0");
g.feature("c2").set("axis","pi/2 0");
g.create("c3","Cylinder");
g.feature("c3").set("r",20);
g.feature("c3").set("h",40);
g.feature("c3").set("pos","0 0 -100");
g.feature("c3").set("axis","1 1 1");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.angularUnit('rad');
g.create('c2','Cylinder');
g.feature('c2').set('type','surface');
g.feature('c2').set('r',0.5);
g.feature('c2').set('h',4);
g.feature('c2').set('pos','1 1 0');
g.feature('c2').set('axis','pi/2 0');
g.create('c3','Cylinder');
g.feature('c3').set('r',20);
g.feature('c3').set('h',40);
g.feature('c3').set('pos','0 0 -100');
g.feature('c3').set('axis','1 1 1');
g.run;
```

SEE ALSO

[Cone](#), [ECone](#)

Delete

Delete vertices, edges, faces, domains, or geometric objects.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Delete");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Delete")` to delete geometric entities.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the entities to delete. To set the selection level for the selection object, and to select the input entities see [Geometry Object Selection Methods](#).

The default selection is empty.

TABLE 3-66: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
input	Selection		Vertices, edges, faces, domains, or objects to delete.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
compat	4.2a 4.3	4.3	Algorithm version.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

Deleting a domain, face, or edge automatically deletes all lower-dimensional adjacent entities, except those needed to bound surviving entities.

In 2D and 3D, vertices that are adjacent to an edge cannot be deleted.

In 3D, an edge can be deleted if it has no adjacent faces, or if it is interior to a face.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-67: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

In version 4.3, the algorithm was changed slightly. The main difference is that the old algorithm preserved the object type for solid, surface, and curve objects. To get the old behavior, set compat to 4.2a.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"del")` creates a Delete feature.

EXAMPLE

Delete face 5 from a surface block:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g =model.component("comp1").geom().create("geom1",3);
g.create("blk1","Block");
g.feature("blk1").set("type", "surface");
g.run("blk1");
```

```

g.create("del1", "Delete");
g.feature("del1").selection("input").set("blk1",5);
g.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('blk1', 'Block');
g.feature('blk1').set('type', 'surface');
g.run('blk1');
g.create('del1', 'Delete');
g.feature('del1').selection('input').set('blk1',5);
g.run;

```

SEE ALSO

[Compose](#), [Union](#), [Intersection](#), [Difference](#), [Extract](#)

DistanceMeasurement

Create parameters that measure the shortest Euclidean distance and vectorial distances between two points or between a point and an edge.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"DistanceMeasurement");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"DistanceMeasurement")` to create parameters for distance and vectorial distance.

Use the following methods to specify the dimension of the second entity as well as selecting it:

```

model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("ent2").init()
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("ent2").set()

```

to select the end edge, boundary or point of the measurement. The default selection is empty.

Use `double[] parvalues = model.component(<ctag>).geom(<tag>).feature(<ftag>).getDoubleArray("parvalvec")`; to access the parameter values (similar for `parval`).

TABLE 3-68: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ent1	Selection		Specify a vertex as first entity.
ent2	Selection		Specify a second entity. Dimension 0 and 1 supported.
extendent	on off	off	Extend a straight or circular edge specified in ent2.
parname	String		Parameter names for the Euclidean distance.
parnamevec	String[]		Parameter names for the vectorial distances. The array has the length of the space dimension, <code>sdim</code>
parval	double		To access parameter value of Euclidean distance (read-only).
parvalvec	double[]		To access parameter values of vectorial distances (read-only).

SEE ALSO

[CentroidMeasurement](#), [Measurement Methods](#)

Create a solid or surface eccentric oblique cone or frustum in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"ECone");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"ECone")` to create an eccentric oblique cone. The following properties are available:

TABLE 3-69: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angle	double	360	Ellipse sector angle.
axis	double[]	{0,0,1}	Direction of the normal to the bottom ellipse. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
displ	double[2]	{0,0}	Displacement of top ellipse relative to bottom ellipse in the local coordinate system.
h	double	1	Height.
pos	double[3]	{0,0,0}	Center of the bottom ellipse.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
r	double	1	Radius of bottom ellipse.
rat	double	0.5	Ratio between perimeter for top ellipse and bottom ellipse.
rot	double	0	Rotational angle about axis.
semiaxes	double[2]	{1,1}	Semiaxes of bottom ellipse.
type	solid surface	solid	Object type.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

TABLE 3-69: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-70: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"econe3")` creates a solid eccentric cone.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"econe2")` creates a surface eccentric cone.

The following properties are also available:

TABLE 3-71: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
a, b	double	1	Alias for semiaxes.
ax2	double[]	{0,0}	Alias for axis when axistype is spherical.
ax3	double[]	{0,0,1}	Alias for axis when axistype is cartesian.
x, y, z	double	0	Alias for pos.

The property const is no longer available.

EXAMPLES

Create a truncated eccentric cone with the base face in the *xy*-plane:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("e1","ECone");
g.feature("e1").set("semiaxes","10 40");
g.feature("e1").set("h",20);
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('e1','ECone');
g.feature('e1').set('semiaxes','10 40');
g.feature('e1').set('h',20);
```

Create an eccentric cone with an apex, that is, a singular patch, on top:

Code for Use with Java

```
g.create("e2","ECone");
g.feature("e2").set("semiaxes","1 2");
g.feature("e2").set("h",4);
g.feature("e2").set("rat",0);
```

```

g.feature("e2").set("displ","1 1");
g.feature("e2").set("pos","100 100 100");
g.feature("e2").set("axis","0 1 4");
g.feature("e2").set("rot",45);
g.run();

```

Code for Use with MATLAB

```

g.create('e2','ECone');
g.feature('e2').set('semiaxes','1 2');
g.feature('e2').set('h',4);
g.feature('e2').set('rat',0);
g.feature('e2').set('displ','1 1');
g.feature('e2').set('pos','100 100 100');
g.feature('e2').set('axis','0 1 4');
g.feature('e2').set('rot',45);
g.run;

```

SEE ALSO

[Cone](#), [Cylinder](#)

EditObject

Create an edit object feature in 2D.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"EditObject");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).vertexNew();
model.component(<ctag>).geom(<tag>).feature(<ftag>).vertexDelete(<vertex>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).vertexSnap(<vertex>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).startVertexDisconnect(<edge>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).endVertexDisconnect(<edge>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).edgeNew();
model.component(<ctag>).geom(<tag>).feature(<ftag>).edgeDelete(<edge>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"EditObject")` to create an edit object feature.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).vertexNew()` to add a new vertex to the object.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).vertexDelete(<vertex>)` to delete `<vertex>` from the object.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).vertexSnap(<vertex>)` to delete `<vertex>` from the object, and move any adjacent edges to the closest remaining vertex.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).startVertexDisconnect(<edge>)` to create a new vertex and use this vertex as the start vertex for `<edge>`.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).endVertexDisconnect(<edge>)` to create a new vertex and use this vertex as the end vertex for `<edge>`.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).edgeNew()` to add a new edge to the object.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).edgeDelete(<edge>)` to delete <edge> from the object.

The following properties are available:

TABLE 3-72: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
input	Selection		Geometry object to edit.
vertex	integer " "		Vertex to edit.
xvertex	double	0	x-coordinate of vertex being edited.
yvertex	double	0	y-coordinate of vertex being edited.
edge	integer " "		Edge to edit.
x	double[]		x-coordinates of control points of edge being edited.
y	double[]		y-coordinates of control points of edge being edited.
weights	double[]		Weights of control points of edge being edited.
knots	double[]		Knots of NURBS curve for edge being edited.
degree	1 2 3		Degree of edge being edited.
start	integer " "		Start vertex of edge being edited.
end	integer " "		End vertex of edge being edited.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-73: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

EXAMPLE

The following sequence edits a circle, setting the degree of one edge to one to create a straight edge:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
```

```

g.create("c1", "Circle");
g.run("c1");
g.create("edo1", "EditObject");
g.feature("edo1").selection("input").set(new String[]{"c1"});
g.feature("edo1").set("edge", "1");
g.feature("edo1").set("degree", "1");
g.run("edo1");

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
g.create('c1', 'Circle');
g.run('c1');
g.create('edo1', 'EditObject');
g.feature('edo1').selection('input').set({'c1'});
g.feature('edo1').set('edge', '1');
g.feature('edo1').set('degree', '1');
g.run('edo1');

```

SEE ALSO

[BezierPolygon](#)

Ellipse

Create a solid or curve ellipse in 2D.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Ellipse");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Ellipse")` to create an ellipse. The following properties are available:

TABLE 3-74: VALID PROPERTIES FOR ELLIPSE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
base	corner center	center	Positions the object either centered about pos or with the lower left corner of surrounding box in pos.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
layer	double[]		Thicknesses of layers.
pos	double[]	{0,0}	Position of the object.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.

TABLE 3-74: VALID PROPERTIES FOR ELLIPSE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
rot	double	0	Rotational angle about pos.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances/3D (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
semiaxes	double[]	{1,1}	Semiaxes.
type	solid curve	solid	Object type.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Ellipse](#).

The following attributes are available:

TABLE 3-75: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"ellip2")` is a solid ellipse.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"ellip1")` is an ellipse curve.

The following properties are also available:

TABLE 3-76: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
a, b	double	1	Alias for semiaxes.
x, y	double	0	Alias for pos.

The property `const` is no longer available.

EXAMPLE

The following sequence creates a solid ellipse:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("e1","Ellipse");
g.feature("e1").set("semiaxes","1 0.3");
g.feature("e1").set("rot",45);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('e1','Ellipse');
g.feature('e1').set('semiaxes','1 0.3');
g.feature('e1').set('rot',45);
```

```
g.run;
```

SEE ALSO

[Circle](#)

Ellipsoid

Create a solid or surface ellipsoid in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Ellipsoid");  
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Ellipsoid")` to create an ellipsoid. The following properties are available:

TABLE 3-77: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	double[]	{0,0,1}	Direction of the local z-axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
layer	double[]		Thicknesses of layers.
pos	double[]	{0,0,0}	Center.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
rot	double	0	Rotational angle about axis.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
semiaxes	double[3]	{1,1,1}	Semiaxes.
type	solid surface	solid	Object type.

TABLE 3-77: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-78: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"ellipsoid3")` creates a solid ellipsoid.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"ellipsoid2")` creates a surface ellipsoid.

The following properties are also available:

TABLE 3-79: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
a, b, c	double	1	Alias for semiaxes.
ax2	double[]	{0,0}	Alias for axis when axistype is spherical.
ax3	double[]	{0,0,1}	Alias for axis when axistype is cartesian.
x, y, z	double	0	Alias for pos.

The property const is no longer available.

EXAMPLE

The following commands create a surface and solid ellipsoid, where the position and semiaxes are defined in two alternative ways:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("e2","Ellipsoid");
g.feature("e2").set("type","surface");
g.feature("e2").set("pos","0 1 0");

g.create("e3","Ellipsoid");
g.feature("e3").set("semiaxes","12 10 8");

g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('e2','Ellipsoid');
g.feature('e2').set('type','surface');
g.feature('e2').set('pos','0 1 0');

g.create('e3','Ellipsoid');
```

```
g.feature('e3').set('semiaxes','12 10 8');
g.run;
```

SEE ALSO

[Sphere](#)

ExplicitSelection

Create explicit selections of geometric entities or objects.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"ExplicitSelection");
model.component(<ctag>).geom(<tag>).feature().selection("selection");
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);
model.component(<ctag>).geom(<tag>).feature().getType(property);
```

DESCRIPTION

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angletol	double	5	Angle tolerance for continuity evaluation.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
groupcontang	on off	off	Continuous tangent mode.
selection	Selection	empty	Selection of entities or objects.
selkeep	on off	on	Keep the selection within the geometry sequence.
selshow	on off	on	Show selection in physics, materials, and so on; in part instances; or in 3D from a plane geometry.

Use the selection methods described in the section “Geometry Object Selection Methods” under `model`.

`component(<ctag>).geom()` to specify the selection `model.component(<ctag>).geom(<tag>).feature().selection("selection")`.

For a boundary or edge selection in 2D or 3D, you can force the selection to select whole groups of entities by setting the property `groupcontang` to `on`. Each group consists of adjacent entities that meet at an angle less than `angletol`.

See [Selections of Geometric Entities](#) for general information about selections.

COMPATIBILITY

The following alias can also be used:

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Selection");
```

EXAMPLE

The sequence below creates a block and a cylinder and creates a selection of face 4 of the block. This corresponds to faces 9 and 13 in the finalized geometry.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("blk1", "Block");
g.run("blk1");
g.create("sel1", "ExplicitSelection");
g.feature("sel1").selection("selection").init(2);
g.feature("sel1").selection("selection").set("blk1", new int[]{4});
g.create("cyl1", "Cylinder");
g.run();
int[] faces = model.selection("geom1_sel1").entities(2);
// faces = 9, 13
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('blk1', 'Block');
g.run('blk1');
g.create('sel1', 'ExplicitSelection');
g.feature('sel1').selection('selection').init(2);
g.feature('sel1').selection('selection').set('blk1', 4);
g.create('cyl1', 'Cylinder');
g.run;
faces = model.selection('geom1_sel1').entities(2);
% faces = 9, 13
```

SEE ALSO

[AdjacentSelection](#), [BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#), [UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#)

Extract

Extract vertices, edges, faces, domains to new objects of the corresponding level, or extract geometric objects to keep only the extracted objects and delete all other objects.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Extract");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Extract")` to extract geometric entities and objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the entities to extract. To set the selection level for the selection object, and to select the input entities see [Geometry Object Selection Methods](#).

The default selection is empty.

TABLE 3-80: VALID PROPERTY/VALUE PAIRS FOR EXTRACT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
input	Selection		Vertices, edges, faces, domains, or objects to extract.
inputhandling	keep remainder remove	keep	Select how to handle the input objects.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-81: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Delete](#)

Extrude

Extrude planar faces into 3D objects.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Extrude");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Extrude")` to extrude objects from a work plane or planar faces in the 3D geometry.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the work plane objects to extrude. The default selection is all available objects from the last preceding work plane.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("inputface")` to select the faces to extrude. Faces are extruded when the `workplane` property is none; otherwise work plane objects are extruded.

The following properties are available:

TABLE 3-82: VALID PROPERTY/VALUE PAIRS FOR EXTRUDE.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the <code>customcolor</code> property. Coloring is only available when <code>selresult</code> is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
crossfaces	on off	on	Keep cross-sectional faces.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when <code>color</code> is set to custom.
displ	double[n_d][2]	{0,0}	Displacement (parallel to work plane) of extrusion top (of each layer) in local coordinate system.
distance	double double[n_d]	1	Extrusion distance(s), that is, local z-coordinate for the top (of each layer). Used if <code>specify</code> is set to distances.
extrudefrom	workplane faces edges vertices	workplane, if available, otherwise faces	Extrude work plane objects, planar faces, edges, or vertices in 3D.
includeinput	boolean	true	Include (planar) input faces when forming the extruded object.
input	Selection	all objects	Objects to extrude.
inputedge	Selection		Edges to extrude. The selected edges must span a plane. Used if <code>extrudefrom</code> is edges.
inputface	Selection		Faces to extrude. Used if <code>extrudefrom</code> is faces.
inputvertex	Selection		Vertices to extrude. The selected vertices must span a plane. Used if <code>extrudefrom</code> is vertices.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
reverse	on off	off	Reverse the extrude direction.
scale	double[n_d][2]	{1,1}	Scale of extrusion top (of each layer).
selextrude	on off	off	Extrude selections from work plane (used if <code>extrudefrom</code> is workplane)
selextrudeshow	on off	on	Show extruded selections in physics/instances (only used when <code>selextrude</code> is on)
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-82: VALID PROPERTY/VALUE PAIRS FOR EXTRUDE.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
specify	distances vertices	distances	Specification of extrusion distances: as distances using the distance property or by specifying vertices in the 3D geometry using the vertex property.
twist	double double[n_d]	0	Twist angle (of each layer).
vertex	Selection		Vertices to extrude to. Used if specify is set to vertices.
workplane	String		Work plane to extrude.

Each planar input is extruded in n_d layers defined by a local coordinate system. By default, $n_d=1$. The property distance is the extrusion distance (of each layer) in the z -axis direction of the local system. The properties displ, scale, and twist define the translation displacements, scale factors and rotation of the top (of each layer) with respect to the bottom of the extruded object. The last array dimension in the properties displ, scale, and twist can be omitted if the same value is desired for all layers.

When extruding work plane objects, the local system is defined as the local system of the work plane. When extruding faces, the local system is defined by the face with the smallest face number in the object that comes first in the geometry sequence. The local z -axis is parallel to the face normal and located at the center of the face. The local x -axis is defined by the tangent direction corresponding to the first parameter in the surface representation for the face.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-83: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

The cubic interpolated extrusion is no longer supported.

The following property is also supported:

TABLE 3-84: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
face	all none	all	Cross-sectional faces to delete, alias for crossfaces.
keep	on off	off	Alias for unite property with opposite value.

EXAMPLE

Creation of a cylinder of height 1.3:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
```



```

GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("wp1","WorkPlane");
g.feature("wp1").geom().create("c1","Circle");
g.run("wp1");
g.create("e1","Extrude");
g.feature("e1").set("distance",1.3);
g.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('wp1','WorkPlane');
g.feature('wp1').geom.create('c1','Circle');
g.run('wp1');
g.create('e1','Extrude');
g.feature('e1').set('distance',1.3);
g.run;

```

SEE ALSO

[Revolve](#), [WorkPlane](#)

Fillet

Create circular rounded corners (fillets) in 2D geometry objects. The Design Module supports 3D fillets.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Fillet");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Fillet")` to round corners in 2D.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("point")` to select which corners to round. The default selection is empty.

TABLE 3-85: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
point	Selection		Vertices to fillet.
pointinsketch	Selection		Vertices to fillet in sketch geometry.
radius	double	0	Radius of fillet.
selectinsketch	on off	on	Determines whether you can use the pointinsketch property. It can be off in models made in version 6.0 or earlier.
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-85: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The point property contains the vertices to fillet in the usual geometry visualization. The pointinsketch property contains the vertices to fillet in the sketch visualization. These properties are automatically kept synchronized.

Additional properties are available with the Design Module, see [Fillet](#).

The following attributes are available:

TABLE 3-86: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

EXAMPLE

Fillet a rectangle object:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("r1","Rectangle");
g.create("fil1","Fillet");
g.feature("fil1").selection("point").set("r1(1)",new int[]{1,2,3,4});
g.feature("fil1").set("radius",0.1);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('r1','Rectangle');
g.create('fil1','Fillet');
g.feature('fil1').selection('point').set('r1(1)',1:4);
g.feature('fil1').set('radius',0.1);
g.run;
```

DIAGNOSTICS

If Fillet does not succeed in creating a rounded corner according to the specified radius, the vertex is skipped. When a fillet intersects another edge, the function generates an error message.

SEE ALSO

[Chamfer](#)

Finalize

Form a union or assembly by combining all geometry objects. The only allowed tag for the Finalize feature is "fin".

SYNTAX

```
model.component(<ctag>).geom(<tag>).create("fin", "FormUnion");
model.component(<ctag>).geom(<tag>).create("fin", "FormAssembly");
model.component(<ctag>).geom(<tag>).feature("fin").set(property,<value>);
model.component(<ctag>).geom(<tag>).feature("fin").getType(property);
```

DESCRIPTION

The Finalize feature (Form Union/Assembly) combines all available geometry objects in the sequence to form a single geometry object. There can only be one Finalize feature per geometry sequence and is added by default for all geometry sequences except for geometry parts and work plane sequences. For 2D, 3D, work plane, and geometry part sequences, you can modify the finalized object by using virtual operations.



The Finalize feature is optional for work plane and geometry part sequences but is necessary if you want to add virtual operations. It then needs to be manually created.

The output of the last geometry feature is the *finalized geometry* used when meshing and when setting up physics. If the property action is set to union, and multiple geometry objects are present in the geometry sequence, the objects are combined into a single object with multiple domains corresponding to the input objects and overlaps between these. In a 1D and 2D axisymmetric geometry, the union action also removes the part of the geometry that falls in the region $r < 0$.

Set the property action to *assembly* to keep multiple objects in the finalized geometry. Use this option when modeling physics that needs separate geometry objects, for example, when modeling mechanical contact.

TABLE 3-87: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
absrepairtol	double	1e-6	Absolute repair tolerance.
action	union assembly	union	Handling of multiple objects,
createpairs	on off	on	Create pairs. Used if action is assembly,
fastpairdetection	on off	on	Generate pairs in a fast way for stacked objects. Used in 3D and if createpairs is on.
frame	mesh geometry material spatial	spatial	Frame used for automatically generated Identity pair features.
imprint	on off	off	Create imprints. Used if action is assembly,
repairtol	double	1e-6	Relative repair tolerance, relative to size of union of inputs.
repairtoltype	auto relative absolute	auto	Repair tolerance type: automatic, relative, or absolute.
pairtype	identity contact	identity	Type of pairs to create,
splitpairs	on off	off	Create one pair for each connected set of touching boundaries,

SEE ALSO

[Compose](#), [Union](#), [Intersection](#), [Difference](#)

FromMesh

Create geometry (deformed configuration) from a (deformed) mesh.

SYNTAX

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
```

DESCRIPTION

To create a geometry sequence from a deformed mesh, use the `createDeformedConfig` method on a solution dataset, see [Solution](#). Such a geometry sequence contains a `FromMesh` feature. This feature has the following properties

TABLE 3-88: VALID PROPERTIES FOR FROMMESH.

NAME	VALUE	DESCRIPTION
solution	String	Tag of solver sequence to take the deformed mesh from, or none.
enableupdate	on off none	Enable update of the deformed configuration using the <code>importData()</code> method.
outertype	solnum none	If the solver used a parametric sweep, then outertype is solnum. Only used if enableupdate is on.
outersolnum	integer	The index of the outer solution to use. Only used if outertype is solnum.
solvertype	solnum none	If you can choose among the solver's solution vectors using a solution number, then solvertype is solnum. Only used if enableupdate is on.
solnum	integer interp	The index of the solution vector to use. Only used if solvertype is solnum.
timeinterp	on off	The timeinterp property is on if solnum is interp.
t	double	Time to interpolate solution at. Only used if timeinterp is on.
solnumdescr	String	Description of the chosen solution. Only used if enableupdate is off.

These properties determine the solution from which the deformed mesh is taken. You should not change the properties `solution`, `enableupdate`, `outertype`, `solvertype`, `timeinterp`, or `solnumdescr`. Only change the properties `outersolnum`, `solnum`, and `t` (but only if `enableupdate` is on).

`model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()` updates the geometry based on the current value of the solution in the feature's corresponding solver sequence.

Helix

Create a solid, surface, or curve helix (coil) with a circular cross section in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Helix");  
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Helix")` to create a helix. The following properties are available:

TABLE 3-89: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axialpitch	double	0.3	Axial pitch.
axis	double[]	{0,0,1}	Direction of the helix axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
chirality	right left	right	Chirality.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
endcaps	paraaxis perpaxis perpspine	paraaxis	Direction of end caps.
grep	bezier spline	spline	Geometry representation.
pos	double[]	{0,0,0}	Position of the object.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
radialpitch	double	0	Radial pitch.
rmaj	double	1	Major radius.
rmin	double	0.1	Minor radius.
rot	double	0	Rotational angle about axis.
rtol	double	1e-4	Relative tolerance.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
turns	double	3	Number of turns.
type	solid surface	solid	Object type.
twistcomp	on off	on	Twist compensation.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-90: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following sequence generates a surface helix and a solid helix:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
GeomFeature h = g.create("h1","Helix");
h.set("type","surface");
h.set("rmaj",2);
h.set("rmin",0.3);
h.set("axialpitch",1);

GeomFeature h2 = g.create("h2","Helix");
h2.set("rmaj",10);
h2.set("rmin",2);
h2.set("axialpitch",1);
h2.set("pos","0,0,-100");
h2.set("axis","1,1,1");
h2.set("rot",60);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
h = g.create('h1','Helix');
h.set('type','surface');
h.set('rmaj',2);
h.set('rmin',0.3);
h.set('axialpitch',1);

h2 = g.create('h2','Helix');
h2.set('rmaj',10);
h2.set('rmin',2);
h2.set('axialpitch',1);
h2.set('pos','0,0,-100');
h2.set('axis','1,1,1');
h2.set('rot',60);
g.run;
```

SEE ALSO

[Torus](#), [Sweep](#)

Hexahedron

Create a solid or surface hexahedron bounded by bilinear faces.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Hexahedron");  
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Hexahedron")` to create a general hexahedron. The following properties are available:

TABLE 3-91: VALID PROPERTY/VALUE PAIRS FOR HEXAHEDRON.

PROPERTY NAME	PROPERTY VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
p	double[3][8]	{ {0,0,1,1,0,0,1,1}, {0,1,1,0,0,1,1,0}, {0,0,0,0,1,1,1,1} }	Corner coordinates.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
type	solid surface	solid	Object type.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For a hexahedron approximately aligned to the coordinate planes, the points in `p` are ordered as follows:

- The first four points and the last four points projected down to the (x,y) -plane defines two negatively oriented quadrangles (quadrilaterals).
- The corresponding plane for the second quadrangle must lie above the plane of the first quadrant in the z direction.
- Generally oriented hexahedra have the points of `p` ordered in a similar way, except for a rigid transformation of the defining point set.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-92: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following command generates a solid hexahedron object:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("h1", "Hexahedron");
g.feature("h1").set("p",new double[][]
    {{0,0.0,1,1.0,0,0,1.0,1},
     {0,0.8,1,0.0,0,1,1.2,0},
     {0,0.1,0,0.2,1,1,2.0,1}});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('h1', 'Hexahedron');
g.feature('h1').set('p',...
    [[0,0.0,1,1.0,0,0,1.0,1];...
     [0,0.8,1,0.0,0,1,1.2,0];...
     [0,0.1,0,0.2,1,1,2.0,1]]);
g.run;
```

SEE ALSO

[Block](#), [Pyramid](#), [Tetrahedron](#)

If, ElseIf, Else, EndIf

Construct an If statement, enabling or disabling features depending on conditions in terms of parameters.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,<type>);
model.component(<ctag>).geom(<tag>).createAfter(<ftag>,<type>,<postag>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,<type>)` to add an If, ElseIf, Else, or EndIf feature after the current feature.

Use `model.component(<ctag>).geom(<tag>).feature().createAfter(<ftag>,<type>,<postag>)` to add an If, ElseIf, Else, or EndIf feature after the feature tagged `<postag>`.

The following property is available for If and ElseIf only:

TABLE 3-93: VALID PROPERTY.

NAME	VALUE	DEFAULT	DESCRIPTION
condition	double	1	Logical condition in terms of parameters.

EXAMPLE

Build a block if `variant = 1`, else build a cone:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.param().set("variant", "1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
```



```

g.create("if1", "If");
g.feature("if1").set("condition", "variant==1");
g.create("blk1", "Block");
g.create("else1", "Else");
g.create("cone1", "Cone");
g.create("endif1", "EndIf");
g.run();
model.param().set("variant", "2");
g.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model1');
model.param.set('variant', '1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('if1', 'If');
g.feature('if1').set('condition', 'variant==1');
g.create('blk1', 'Block');
g.create('else1', 'Else');
g.create('cone1', 'Cone');
g.create('endif1', 'EndIf');
g.run;
model.param.set('variant', '2');
g.run;

```

IgnoreEdges

Ignore edges by removing selected edges that are isolated, adjacent to precisely two faces, or between two domains.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"IgnoreEdges");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"IgnoreEdges")` to ignore edges.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the edges to ignore. The default selection is empty.

The feature removes the selected edges that are isolated, that are adjacent to precisely two faces, or that are between two domains. If an edge is adjacent to two faces in 3D, the operations forms a composite face, if an edge is between two domains in 2D, the operation forms composite domain. The output object is a virtual geometry.

The following properties are available:

TABLE 3-94: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Edges to ignore.
ignorevtx	on off	on	Ignore vertices on boundary.
keepformesh	on off	off	Keep edges for mesh control.

Use `ignorevtx` to specify if the feature also removes the ignorable vertices on the boundary of each resulting composite face.

Use `keepformesh` to keep the ignored edges while meshing, to help you in constructing the mesh.

EXAMPLE

Create a sphere and ignore all edges and, implicitly, all vertices.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("sph1", "Sphere");
g.create("ige1", "IgnoreEdges");
g.feature("ige1").selection("input").set("fin(1)", 1,2,3,4,5,6,7,8,9,10,11,12);
g.run("ige1");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('sph1', 'Sphere');
g.create('ige1', 'IgnoreEdges');
g.feature('ige1').selection('input').set('fin(1)', 1:12);
g.run('ige1');
```

SEE ALSO

[CompositeFaces](#), [IgnoreFaces](#), [IgnoreVertices](#), [MeshControlEdges](#)

IgnoreFaces

Ignore faces by removing the selected faces that are isolated or that are between two domains.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"IgnoreFaces");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"IgnoreFaces")` to ignore faces in 3D.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the faces to ignore. The default selection is empty.

The feature removes the selected faces that are isolated or that are between two domains. In the latter case, the operation forms a composite domain. The output object is a virtual geometry.

The following properties are available:

TABLE 3-95: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Faces to ignore.
ignoreadj	on off	on	Ignore edges and vertices on boundary.
keepformesh	on off	off	Keep faces for mesh control.

Use `ignoreadj` to specify if the feature also removes the ignorable edges and vertices on the boundary of each resulting composite domain.

Use `keepformesh` to keep the ignored faces while meshing, to help you in constructing the mesh.

EXAMPLE

Ignore faces to form one composite domain. The operation also creates composite faces and composite edges.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("blk1", "Block");
g.create("cyl1", "Cylinder");
g.run("fin");
g.create("igf1", "IgnoreFaces");
g.feature("igf1").selection("input").set("fin", 6, 7, 10);
g.run("igf1");
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component().create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('blk1', 'Block');
g.create('cyl1', 'Cylinder');
g.run('fin');
g.create('igf1', 'IgnoreFaces');
g.feature('igf1').selection('input').set('fin', [6, 7, 10]);
g.run('igf1');
```

SEE ALSO

[CompositeDomains](#), [IgnoreEdges](#), [IgnoreVertices](#), [MeshControlFaces](#)

IgnoreVertices

Ignore vertices by removing the selected vertices that are isolated or that are adjacent to precisely two edges.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"IgnoreVertices");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"IgnoreVertices")` to ignore vertices.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the vertices to ignore. The default selection is empty.

The feature removes the selected vertices that are isolated or that are adjacent to precisely two edges. If a vertex is adjacent to two edges, the operation forms a composite edge. The output object is a virtual geometry.

The following properties are available:

TABLE 3-96: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Vertices to ignore.
keepformesh	on off	off	Keep vertices for mesh control.

Use `keepformesh` to keep the ignored vertices while meshing, to help you in constructing the mesh.

EXAMPLE

Create an ellipse and ignore vertices 1 and 3, which gives you two remaining edges in the final geometry.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
```

```

GeomSequence g = model.component("comp1").geom().create("geom1", 2);
model.mesh().create("mesh1", "geom1");
g.create("e1", "Ellipse");
g.run("fin");
g.create("igv1", "IgnoreVertices");
g.feature("igv1").selection("input").set("fin", 1, 3);
g.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
model.mesh.create('mesh1', 'geom1');
g.create('e1', 'Ellipse');
g.run('fin');
g.create('igv1', 'IgnoreVertices');
g.feature('igv1').selection('input').set('fin', 1, 3);
g.run();

```

SEE ALSO

[CompositeEdges](#), [IgnoreEdges](#), [IgnoreFaces](#), [MeshControlVertices](#)

Import DXF

Import geometry objects from a DXF file to a 2D geometry.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Import");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Import")` to create a geometry import feature. When the property filename is set to a file recognized as a DXF CAD drawing, the property type is set to dxf and the following properties are available:

TABLE 3-97: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alllayers	String[]		Read-only property that returns all layers in the DXF file. Access it using <code>model.component(<ctag>).geom(<tag>).feature(<ftag>).getStringArray('alllayers')</code> ;
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
convert	solid curve off	solid	Options to unite all objects in each layer and make all void regions solid.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
filename	String		Filename.

TABLE 3-97: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
layers	String[]	all layers	Layers to import.
repairgeom	on off	on	Repair geometry.
repairtol	double	1e-5	Repair tolerance, relative to size of union of imported objects.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
selindividual	on off	off	Create selections of individual objects.
selindividualshow	all dom bnd pnt off	dom	Show selections of individual objects in physics, materials, and so on; in part instances; or in 3D if in a Work Plane's Plane Geometry, when selindividual is on.
type	dxf		Type of import.

The file specified by `filename` can be of any of the following formats:

TABLE 3-98: SUPPORTED FILE FORMATS.

FILE FORMAT	FILE EXTENSIONS
DXF	.dxf

The imported objects are represented using the COMSOL geometry modeler.

The method

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()
```

imports the file again.

If `selresult` is set to `on`, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point), for use in the geometry sequence. To access the object selection, use `model.geom(<tag>).selection(<ftag>)`, where `<tag>` is the geometry tag and `<ftag>` is the feature tag. To access the other selections, use `model.geom(<tag>).selection(<ftag>.<lvl>)`, where `<tag>` is the geometry tag, `<ftag>` is the feature tag, and `<lvl>` is one of `dom`, `bnd`, `edg`, or `pnt` (`edg` is not available for DXF import in 2D). If, in addition, `selresultshow` is set to a value other than `off`, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<ftag>_<lvl>)`, where `<tag>` is the geometry tag, `<ftag>` is the feature tag, and `<lvl>` is one of `dom`, `bnd`, `edg`, or `pnt` (`edg` is not available for DXF import in 2D).

If `selindividual` is set to `on`, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point) of each individual object, for use in the geometry sequence. To access the object selections, use `model.geom(<tag>).selection(<otag>)`, where `<otag>` is a tag derived from the name of the imported object. For standard object names of the form `<ftag>(<n>)`, where `<n>` is an object number, the corresponding `<otag>` is `<ftag>_<n>`. To access the other selections, use `model.geom(<tag>).selection(<otag>.<lvl>)`, where `<otag>` is a tag derived from the name of the imported object. If, in addition, `selindividualshow` is set to a value other than `off`, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<otag>_<lvl>)`, where `<otag>` is a tag derived from the name of the imported object. For standard object names of the form `<ftag>(<n>)`, where `<n>` is an object number, the corresponding `<otag>` is `<ftag>_<n>`.

The following attributes are available:

TABLE 3-99: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

The following property is also supported:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coercion	solid face curve off	solid	Alias for convert. The value face is equivalent to solid.

Import Geometry Sequence

Import geometry objects from another geometry sequence.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Import");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Import")` to create a geometry import feature. Set the property mesh to the tag of a meshing sequence of another model component in the model.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
sequence	String		Tag of other geometry sequence.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
selindividual	on off	off	Create selections of individual objects.
selindividualshow	all dom bnd edg pnt off	dom	Show selections, when selindividual is on, of individual objects in physics, materials, and so on; in part instances; or in 3D from a plane geometry.
type	sequence		Type of import.

When building, the import feature takes all the existing objects in the specified sequence and imports them into the feature's sequence.

The method

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()
```

imports the sequence again. The imported objects are represented using the COMSOL Multiphysics geometry modeler or the CAD Import Module's geometry modeler (Parasolid).

If `selresult` is set to `on`, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point), for use in the geometry sequence. To access the object selection, use `model.geom(<tag>).selection(<ftag>)`, where `<tag>` is the geometry tag and `<ftag>` is the feature tag. To access the other selections, use `model.geom(<tag>).selection(<ftag>.<lvl>)`, where `<tag>` is the geometry tag, `<ftag>` is the feature tag, and `<lvl>` is one of `dom`, `bnd`, `edg`, or `pnt` (`edg` is not available for DXF import in 2D). If, in addition, `selresultshow` is set to a value other than `off`, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<ftag>_<lvl>)`, where `<tag>` is the geometry tag, `<ftag>` is the feature tag, and `<lvl>` is one of `dom`, `bnd`, `edg`, or `pnt` (`edg` is not available for DXF import in 2D).

If `selindividual` is set to `on`, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point) of each individual object, for use in the geometry sequence. To access the object selections, use `model.geom(<tag>).selection(<otag>)`, where `<otag>` is a tag derived from the name of the imported object. For standard object names of the form `<ftag>(<n>)`, where `<n>` is an object number, the corresponding `<otag>` is `<ftag>_<n>`. To access the other selections, use `model.geom(<tag>).selection(<otag>_<lvl>)`, where `<otag>` is a tag derived from the name of the imported object. If, in addition, `selindividualshow` is set to a value other than `off`, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<otag>_<lvl>)`, where `<otag>` is a tag derived from the name of the imported object. For standard object names of the form `<ftag>(<n>)`, where `<n>` is an object number, the corresponding `<otag>` is `<ftag>_<n>`.

The following attributes are available:

TABLE 3-100: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

SEE ALSO

[Import 3D CAD](#), [Import DXF](#), [Import of ECAD Files](#), [Import Mesh Part or Meshing Sequence](#), [Import mphbin/mphtxt](#)

Import Mesh Part or Meshing Sequence

Create a geometry object from an imported mesh.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Import");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>, "Import")` to create a geometry import feature. Set the property sequence to the tag of another geometry sequence in the model.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
mesh	String	none	Tag of meshing sequence or mesh part to import, or none to create a new mesh part from a mesh file.
meshfilename	String		Path to mesh file to import when mesh is set to none.
type	Mesh		Type of import.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
selindividual	on off	off	Create selections of individual objects.
selindividualshow	all dom bnd edg pnt off	dom	Show selections of individual objects in physics, materials, and so on, or in part instances, when selindividual is on.
contributeto	String	none	Tag of cumulative selection to contribute to.
defectremoval	double	1.0	Relative size factor for identification local defects (3D only).
simplifymesh	on off	on	Boolean specifying if the original mesh should be simplified (3D only).
simplifytol	double	0.01	Relative simplification tolerance (3D only).

When building, the import feature takes the finalized mesh (if the sequence imports a mesh) or the current mesh (if it is not an imported mesh) of the specified sequence and constructs a corresponding geometry object.

The method

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()
```

imports the sequence again. The imported objects are represented using the COMSOL Multiphysics geometry modeler. The CAD Import Module's geometry modeler (Parasolid) does not support these types of geometries.

If `selresult` is set to on, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point), for use in the geometry sequence. To access the object selection, use `model.geom(<tag>).selection(<ftag>)`, where `<tag>` is the geometry tag and `<ftag>` is the feature tag. To access the other selections, use `model.geom(<tag>).selection(<ftag>.<lvl>)`, where `<tag>` is the geometry tag, `<ftag>` is the feature tag, and `<lvl>` is one of dom, bnd, edg, or pnt. If, in addition, `selresultshow` is set to a value other than off, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<ftag>_<lvl>)`, where `<tag>` is the geometry tag, `<ftag>` is the feature tag, and `<lvl>` is one of dom, bnd, edg, or pnt.

If `selindividual` is set to `on`, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point) of each individual object, for use in the geometry sequence. To access the object selections, use `model.geom(<tag>).selection(<otag>)`, where `<otag>` is a tag derived from the name of the imported object. For standard object names of the form `<ftag>(<n>)`, where `<n>` is an object number, the corresponding `<otag>` is `<ftag>_<n>`. To access the other selections, use `model.geom(<tag>).selection(<otag>_<lvl>)`, where `<otag>` is a tag derived from the name of the imported object. If, in addition, `selindividualshow` is set to a value other than `off`, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<otag>_<lvl>)`, where `<otag>` is a tag derived from the name of the imported object. For standard object names of the form `<ftag>(<n>)`, where `<n>` is an object number, the corresponding `<otag>` is `<ftag>_<n>`.

The following attributes are available:

TABLE 3-101: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

Import mphbin/mphtxt

Import geometry objects from a file using COMSOL Multiphysics geometry formats: binary or text.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Import");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Import")` to create a geometry import feature. When the property filename is set to a file recognized as an MPHBIN- or MPHTXT-file, the property type is set to `native` and the following properties are available:

TABLE 3-102: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when <code>selresult</code> is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when <code>color</code> is set to <code>custom</code> .
filename	String		Filename.
includevirtual	boolean	true	Include virtual operations when importing the geometry. Not available in 1D and in work planes, where virtual operations are always excluded.
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-102: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
selindividual	on off	off	Create selections of individual objects.
selindividualshow	all dom bnd edg pnt off	dom	Show selections, when selindividual is on, of individual objects in physics, materials, and so on; in part instances; or in 3D from a plane geometry.
type	native		Type of import.

The file specified by filename can be of any of the following formats:

TABLE 3-103: SUPPORTED FILE FORMATS.

FILE FORMAT	FILE EXTENSIONS
COMSOL Multiphysics Binary	.mphbin
COMSOL Multiphysics Text	.mphtxt

The imported objects are represented using COMSOL's geometry kernel or the CAD Import Module's geometry kernel (Parasolid).

The method

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()
```

imports the file again.

If selresult is set to on, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point), for use in the geometry sequence. To access the object selection, use `model.geom(<tag>).selection(<ftag>)`, where <tag> is the geometry tag and <ftag> is the feature tag. To access the other selections, use `model.geom(<tag>).selection(<ftag>.<lvl>)`, where <tag> is the geometry tag, <ftag> is the feature tag, and <lvl> is one of dom, bnd, edg, or pnt (edg is not available for DXF import in 2D). If, in addition, selresultshow is set to a value other than off, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<ftag>_<lvl>)`, where <tag> is the geometry tag, <ftag> is the feature tag, and <lvl> is one of dom, bnd, edg, or pnt (edg is not available for DXF import in 2D).

If selindividual is set to on, a selection is created for all resulting entities of each type (object, domain, boundary, edge, and point) of each individual object, for use in the geometry sequence. To access the object selections, use `model.geom(<tag>).selection(<otag>)`, where <otag> is a tag derived from the name of the imported object. For standard object names of the form <ftag>(<n>), where <n> is an object number, the corresponding <otag> is <ftag>_<n>. To access the other selections, use `model.geom(<tag>).selection(<otag>_<lvl>)`, where <otag> is a tag derived from the name of the imported object. If, in addition, selindividualshow is set to a value other than off, all or some of these selections appear for use outside the geometry sequence. To access these selections, use `model.selection(<tag>_<otag>_<lvl>)`, where <otag> is a tag derived from the name of the imported object. For standard object names of the form <ftag>(<n>), where <n> is an object number, the corresponding <otag> is <ftag>_<n>.

The following attributes are available:

TABLE 3-104: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

InterpolationCurve

Create a curve interpolating or approximating a sequence of points in 2D or 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"InterpolationCurve");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property)
model.component(<ctag>).geom(<tag>).feature(<ftag>).importToTable();
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

To create an interpolation curve use `model.component(<ctag>).geom(<tag>).create(<ftag>,"InterpolationCurve")`. The following properties are available:

TABLE 3-105: VALID PROPERTY/VALUE PAIRS FOR INTERPOLATIONCURVE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
endcond	none zerocurv tangent	none	Condition at endpoint: none, a tangent condition, or zero curvature
endtang	double[sdim]	{1,0} or {1,0,0}	Tangent direction at endpoint (if endcond is tangent).
filename	String		If source is file, the file that contains the data.
rtol	double	0	Maximum relative error. 0 implies interpolation.
source	table file vectors	table	Whether data is specified as vectors, a table, or read from a file.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom in 2D; edg in 3D	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
startcond	none zerocurv tangent	none	Condition at starting point: none, a tangent condition, or zero curvature.
starttang	double[sdim]	{1,0} or {1,0,0}	Tangent direction at starting point (if startcond is tangent).
struct	sectionwise spreadsheet	spreadsheet	The data format if source is file.
table	double[][]		Data points, size N*sdim.
type	open closed solid	open	Type of curve.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).

TABLE 3-105: VALID PROPERTY/VALUE PAIRS FOR INTERPOLATIONCURVE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.
x	double[]	{}	x-coordinates for data points.
y	double[]	{}	y-coordinates for data points.
z	double[]	{}	z-coordinates for data points.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).importToTable()` to read data from the file defined by the `filename` property and store the data in the `table` property. The `source` property is also changed to `table`.

When building the feature, if the start condition or end condition is zero curvature, the corresponding (currently inactive) tangent direction property should be set to the tangent vector of the resulting curve.

If `source` is `file`, the interpolation curve is not automatically rebuilt when the data in the file changes. Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()` to rebuild the interpolation curve after such a change.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [InterpolationCurve](#).

The following attributes are available:

TABLE 3-106: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following commands create a curve interpolating four points in 2D:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
g.create("ic1", "InterpolationCurve");
g.feature("ic1").set("table", new double[][]{{0,0}, {1,0}, {1,1}, {0,1}});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
g.create('ic1', 'InterpolationCurve');
g.feature('ic1').set('table', [[0,0]; [1,0]; [1,1]; [0,1]]);
g.run();
```

SEE ALSO

[CubicBezier](#), [QuadraticBezier](#)

Interval

Create one or several connected intervals in 1D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Interval");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Interval")` to create one or more intervals. The following properties are available:

TABLE 3-107: VALID PROPERTY/VALUE PAIRS FOR INTERVAL.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
contributeto	String	none	Tag of cumulative selection to contribute to.
coord	double[]	{0,1}	Coordinates in table (used when coordsource is set to table).
coordsource	table vector	table	Data source for coordinates: a table or a vector of coordinates (used when specify is set to coord).
coordvec	double[]	{0,1}	Coordinates in table (used when coordsource is set to table).
left	double	0	Left endpoint (used when specify is set to len).
lensource	table vector	table	Data source for interval lengths: a table or a vector of lengths (used when specify is set to len).
len	double[]	{1}	Lengths in table (used when lensource is set to table).
lensource	table vector	table	Data source for interval lengths: a table or a vector of lengths (used when specify is set to len).
lenvec	double[]	{1}	Lengths in vector field (used when lensource is set to table).
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
specify	coord len	coord	Specify coordinates or interval lengths.

To specify one interval, set the properties p1 and p2. Then, `intervals` is automatically set to one.

To specify a sequence of connected intervals, set the property p. Then, `intervals` is automatically set to many.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"solid1")` creates an interval.

The properties above were added in version 5.4; there is backward compatibility with respect to the previous properties as follows:

- The property `intervals` set to one corresponds to `coordsource` set to `table`.
- The property `intervals` set to many corresponds to `coordsource` set to `vector`.
- Setting or getting the value of the property p1 operates on the first element of the `coord` array.
- Setting or getting the value of the property p2 operates on the last (or second) element of the `coord` array.
- Setting or getting the value of the property p operates on the `coordvec` array.

EXAMPLE

The following commands create a solid consisting of two intervals:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",1);
g.create("i1","Interval");
g.feature("i1").set("p","0 1 3");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',1);
g.create('i1','Interval');
g.feature('i1').set('p','0 1 3');
g.run;
```

SEE ALSO

[BezierPolygon](#)

LineSegment

Create line segments in 2D and 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"LineSegment");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"LineSegment")` to create a line segment.

The following properties are available:

TABLE 3-108: VALID PROPERTY/VALUE PAIRS FOR LINESEGMENT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
coord1	double[]	zero vector	Coordinates for starting point (with specify1 set to coord).
coord2	double[]	zero vector	Coordinates for endpoint (with specify2 set to coord).
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
p2	Selection	1	Endpoint (with specify2 set to vertex).
selresult	on off	off	Create selections of all resulting objects.

TABLE 3-108: VALID PROPERTY/VALUE PAIRS FOR LINESEGMENT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selresultshow	all obj dom bnd off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
specify1	vertex coord	vertex	Type of starting point specification.
specify2	vertex coord	vertex	Type of endpoint specification.
vertex1	Selection	0	Starting point (with specify1 set to vertex).
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D and for coordinates only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D and for coordinates only). The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [LineSegment](#).

The following attributes are available:

TABLE 3-109: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following commands create a block in 3D and a line segment from a vertex in that block to a point with the coordinates (0, 1, 2):

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("blk1", "Block");
g.create("ls1", "LineSegment");
g.feature("ls1").set("specify1", "vertex");
g.feature("ls1").set("specify2", "coord");
g.feature("ls1").selection("vertex1").set("blk1(1)", new int[]{1});
g.feature("ls1").set("coord2", new double[]{0, 1, 2});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('blk1', 'Block');
g.create('ls1', 'LineSegment');
g.feature('ls1').set('specify1', 'vertex');
g.feature('ls1').set('specify2', 'coord');
g.feature('ls1').selection('vertex1').set('blk1(1)', 1);
g.feature('ls1').set('coord2', [0, 1, 2]);
g.run;
```

SEE ALSO

[BezierPolygon](#)

LogicalExpressionSelection

Combine selections of entities or objects using a logical expression.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"LogicalExpressionSelection");  
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature().getType(property);
```

DESCRIPTION

For LogicalExpressionSelection, the following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
entitydim	-1 0 1 2 3	space dimension	Dimension of entities to select. -1 means Object.
expression	String		The logical expression to use. It can contain parentheses and the , &&, and ! Boolean operators, operating on the tags of existing named geometry selections.
selkeep	on off	on	Keep the selection within the geometry sequence.
selshow	If the level is not Object, the allowed values are on off. If the level is Object, the allowed values are all obj dom bnd edg pnt off.	If the level is not Object, the default value is on. If the level is Object, the default value is all in a component's geometry, obj in a part, and dom in a work plane's Plane Geometry.	Show selection in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry.

See [Selections of Geometric Entities](#) for general information about selections.

SEE ALSO

[AdjacentSelection](#), [BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#), [ExplicitSelection](#), [UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#)

MergeEdges

Merge edges adjacent to faces.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"MergeEdges");  
model.component(<ctag>).geom(<tag>).feature().selection(property);  
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature().getType(property);
```


DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"MergeEdges")` to merge edges adjacent to face.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("keepedge")` to select the edges to keep. The default selection is empty.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("removeedge")` to select the edges to remove. The default selection is empty.

The feature merges the edges by collapsing the face between the edges and reconnecting the faces adjacent to the removed edges to the resulting merged edges.

The output object is a virtual geometry.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
keepedge	Selection		Edges to keep.
removeedge	Selection		Edges to remove.

SEE ALSO

[CollapseFaces](#), [CollapseFaceRegions](#)

MergeFaces

The feature merges the selected faces by collapsing the gap between the faces. The output object is a virtual geometry.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"MergeFaces");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"MergeFaces")` to merge faces using the following properties:

TABLE 3-110: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
pairing	auto manual	auto	Specifies how to pair the faces in the merge.
input	Selection		Faces to merge. Used if pairing is auto.
keepfac	Selection		Faces to keep. Used if pairing is manual.
removefac	Selection		Faces to remove. Used if pairing is manual.
distancetype	auto relative manual	auto	Specifies how to enter the maximum distance to bridge.
maxreldistance	Double	0.01	Maximum distance between faces to be merged, relative to size of geometry. Used if distancetype is relative.
maxabsdistance	Double		Maximum absolute distance between faces to be merged. Used if distancetype is absolute.
method	imprint one-to-one	imprint	Specifies how to pair the entities. The option imprint creates an imprint of the entities to remove on the keep side. Used when the geometry representation is cadps.

TABLE 3-110: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
precision	[0,1]	0.5	Specifies the precision with which to make the imprints. A value close to 1 gives a more exact imprint. Used when method is imprint.
forceonetoone	on off	off	Forces the faces on the remove side to be matched with the keep side without any imprints. Used when the geometry representation is comsol.
createdom	on off	on	Specify if to create domains for each watertight finite void region.

Read more about the geometry representation in the section [Geometry Representation in 3D and for Boolean Operations](#).

SEE ALSO

[Geometry Cleanup](#)

MergeVertices

Merge two vertices.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"MergeVertices");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"MergeVertices")` to merge two vertices.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("keepvtx")` to select the vertex to keep. The default selection is empty.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("removevtx")` to select the vertex to remove. The default selection is empty.

The feature merges the two vertices by collapsing the edge between the vertices and reconnecting the edges adjacent to the removed vertex to the resulting merged vertex.

The output object is a virtual geometry.

The following properties are available:

TABLE 3-111: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
keepvtx	Selection		Vertex to keep.
removevtx	Selection		Vertex to remove.

SEE ALSO

[CollapseEdges](#)

MeshControlDomains

Define mesh control domains.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlDomains");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlDomains")` to define mesh control domains.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the domains to include. The default selection is empty.

The feature creates a composite domain by removing all faces (in 3D) or edges (in 2D) between the selected domains and adjacent domains. The removed entities are kept for mesh control.

The following property is available:

TABLE 3-112: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Edges to ignore.

SEE ALSO

[MeshControlFaces](#), [MeshControlEdges](#)

MeshControlEdges

Define mesh control edges.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlEdges");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlEdges")` to define mesh control edges.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the edges to include. The default selection is empty.

The feature removes the selected edges that are isolated, that are adjacent to precisely two faces (in 3D), or that are between two domains (in 2D). The edges are kept for mesh control.

The following properties are available:

TABLE 3-113: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Edges to ignore.
includevtx	on off	on	Include start and end vertices.

Use `includevtx` to specify if the feature also removes the ignorable start and end vertices of the edge.

SEE ALSO

[IgnoreEdges](#), [MeshControlDomains](#), [MeshControlFaces](#), [MeshControlVertices](#)

MeshControlFaces

Define mesh control faces.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlFaces");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlFaces")` to define mesh control faces in 3D.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the faces to include. The default selection is empty.

The feature removes the selected faces that are isolated or that are between two domains. The faces are kept for mesh control.

The following properties are available:

TABLE 3-114: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Faces to include.
includeadj	on off	on	Include edges and vertices on boundary.

Use `includeadj` to specify if the feature also includes the ignorable edges and vertices on the boundary of each resulting composite domain.

SEE ALSO

[IgnoreFaces](#), [MeshControlDomains](#), [MeshControlEdges](#), [MeshControlVertices](#)

MeshControlVertices

Define mesh control vertices.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlVertices");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"MeshControlVertices")` to define mesh control vertices.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the vertices to include. The default selection is empty.

The feature removes the selected vertices that are isolated or that are adjacent to precisely two edges. The vertices are kept for mesh control.

The following properties are available:

TABLE 3-115: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
input	Selection		Vertices to include.

SEE ALSO

[IgnoreVertices](#), [MeshControlFaces](#), [MeshControlEdges](#)

Mirror

Reflect (mirror) objects in a plane (3D), a line (2D), or a point (1D).

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Mirror");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Mirror")` to mirror geometry objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects to mirror. The default selection is empty.

The following properties are available:

TABLE 3-116: VALID PROPERTY/VALUE PAIRS FOR MIRROR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active. 2D and 3D only.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom. 2D and 3D only.
edge	Selection		Edge selection. Used when specify is set to edge.
input	Selection		Objects to reflect.
keep	on off	off	Keep input objects.
pos	double[]	0	A point to be fixed during reflection.
axis	double[]	{0 0 1} (3D) {1 0} (2D) {1} (1D)	Vector in the direction to reflect.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.

TABLE 3-116: VALID PROPERTY/VALUE PAIRS FOR MIRROR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
specify	pointnormal edge	pointnormal	Specify the line of reflection as a point and normal or as a straight edge in 2D.
contributeto	String	none	Tag of cumulative selection to contribute to.

In 3D, the input objects are reflected in the plane through pos with normal vector axis. In 2D, the input objects are reflected in the line through pos with normal vector axis. In 1D, the input objects are reflected in the point pos.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Mirror](#).

The following attributes are available:

TABLE 3-117: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

The property out is no longer available.

EXAMPLE

A 2D example, mirroring a rectangle:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("r1","Rectangle");
g.create("m1","Mirror");
g.feature("m1").selection("input").set("r1");
g.feature("m1").set("pos","2 2");
g.feature("m1").set("axis","1 1");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('r1','Rectangle');
g.create('m1','Mirror');
g.feature('m1').selection('input').set('r1');
g.feature('m1').set('pos','2 2');
g.feature('m1').set('axis','1 1');
g.run;
```

A 3D example, mirroring a block:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("blk1","Block");
g.create("m1","Mirror");
g.feature("m1").selection("input").set("blk1");
g.feature("m1").set("pos","2 2 2");
```

```
g.feature("m1").set("axis","1 1 1");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('blk1','Block');
g.create('m1','Mirror');
g.feature('m1').selection('input').set('blk1');
g.feature('m1').set('pos','2 2 2');
g.feature('m1').set('axis','1 1 1');
g.run;
```

SEE ALSO

[Move](#), [Copy](#), [Rotate](#), [Scale](#), [RigidTransform](#)

Move, Copy

Move or copy geometry objects by translation.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Move");
model.component(<ctag>).geom(<tag>).create(<ftag>,"Copy");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Move")` to move geometry objects.

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Copy")` to move a copy of geometry objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects to move or copy. The default selection is empty.

The following properties are available:

TABLE 3-118: VALID PROPERTIES FOR MOVE AND COPY.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
displx	double[]	0	x displacement. Used when specify is displ.
disply	double[]	0	y displacement. Used when specify is displ.
displz	double[]	0	z displacement. Used when specify is displ.
indexattr	on off	off	Create index attributes.
indexattrtag	String		Tag of index attribute. Used if indexattr is on.
newpos	vertices coord	vertices	The new (destination) position. Used when specify is pos.

TABLE 3-118: VALID PROPERTIES FOR MOVE AND COPY.

NAME	VALUE	DEFAULT	DESCRIPTION
newposvertices	Selection		Vertices to move to. Used when newpos is vertices.
newposx	double[]	0	x-coordinates of new positions. Used when newpos is coord.
newposy	double[]	0	y-coordinates of new positions. Used when newpos is coord.
newposz	double[]	0	z-coordinates of new positions. Used when newpos is coord.
oldpos	vertex coord	vertex	The old (original) position. Used when specify is pos.
oldposvertex	Selection		Vertex to move to. Used when oldpos is vertex.
oldposcoord	double[sdim]	0	Coordinates of old position. Used when oldpos is coord.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
specify	displ pos	displ	Specify displacement vector or positions.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

If `displ` is a one-dimensional array, a single copy of each input object is created using the translation vector `displ`. If `displ` is a two-dimensional array, several copies can be created, where the `n`th copy has translation `displ[i][n]` in the `i`th coordinate.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Move](#), and [Copy](#).

The following attributes are available:

TABLE 3-119: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use <code>inherit</code> to set the construction geometry attribute only if all input objects are construction geometry.

EXAMPLE

The sequence below moves a circle from the origin to (2, 3):

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
g.create("c1", "Circle");
g.create("m1", "Move");
```



```
g.feature("m1").selection("input").set("c1");
g.feature("m1").set("displ", new double[][]{{2},{3}});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('c1','Circle');
g.create('m1','Move');
g.feature('m1').selection('input').set('c1');
g.feature('m1').set('displ', [2,3]);
g.run;
```

SEE ALSO

[Array](#), [Mirror](#), [Rotate](#), [Scale](#), [RigidTransform](#)

Offset

Offset the edges of 2D curve or solid objects in the normal direction.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Offset");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Offset")` to create an offset feature tagged `<ftag>` in the 2D geometry sequence with the tag `<tag>`.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the entities to offset. To set the selection level for the selection object, and to select the input entities see [Geometry Object Selection Methods](#). For example, use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input").init(1)` to set the selection to be a selection of boundaries (geometric entities of dimension 1) for the offset feature with the tag `<ftag>` in the 2D geometry sequence with the tag `<tag>`.

The following properties are available:

TABLE 3-120: VALID PROPERTY/VALUE PAIRS FOR OFFSET.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
input	Selection		Objects, domains, or boundaries to offset.
keep	on off	on	Keep input objects.
distance	double	0	Offset distance.
reverse	on off	off	Reverse the offset direction.
convexcorner	fillet tangent extend noconnection	fillet	Handling of offset edges in convex corners.
trim	on off	on	Trim the offset edges in concave corners.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; or in 3D from a plane geometry. obj is not available in a component's geometry

TABLE 3-120: VALID PROPERTY/VALUE PAIRS FOR OFFSET.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
contributeto	String	none	Tag of cumulative selection to contribute to.

The following attributes are available:

TABLE 3-121: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Thicken2D](#)

ParameterCheck

Check the values of parameters.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"ParameterCheck");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"ParameterCheck")` to check parameter values and issue an error if the check condition is fulfilled (being nonzero); for example, the condition `r>30[mm]` results in an error if the value of parameter `r` is larger than 30 mm. The following properties are available:

TABLE 3-122: VALID PROPERTY/VALUE PAIRS FOR PARAMETERCHECK.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
condition	double	1	The condition that checks some value of the parameters. The error appears if the value of the condition is nonzero.
message	string	empty string	The error or warning message that is displayed if condition is fulfilled.
messagetype	error warning	error	Specifies if to display an error or warning message if condition is fulfilled.

ParametricCurve

Create a parametric curve defined by coordinate expressions in 2D or 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"ParametricCurve");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"ParametricCurve")` to create a parametric curve. Self-intersecting curves are not supported, except the case of a closed curve (that is, when the starting point and endpoint coincide). The following properties are available:

TABLE 3-123: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	double[]	{0,0,1}	Direction of the z-axis of the local coordinate system. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
coord	String[2] String[3]	empty	Coordinates of parametric curve as function of parameter.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
maxknots	int	1000	Maximum number of knots.
parname	String	s	Parameter name.
parmax	double	1	Maximum parameter value.
parmin	double	0	Minimum parameter value.
pos	double[]	{0,0,0}	Position of the object.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
reparameterize	true false	false	Reparameterize the curve using the arc length.
rot	double	0	Rotational angle about axis.
rtol	double	1e-6	Relative tolerance.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	bnd in 2D; edg in 3D	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

TABLE 3-123: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

The expressions in `coord` can contain functions defined in the model. If the definition of such a function is changed, the parametric curve is not automatically rebuilt. Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()` to rebuild the parametric curve after such a change.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-124: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following commands create a parametric curve in 3D with the shape of a helix:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("pc1", "ParametricCurve");
g.feature("pc1").set("parmax", "2*pi");
g.feature("pc1").set("coord", new String[]{"cos(s)", "sin(s)", "s*0.2"});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('pc1', 'ParametricCurve');
g.feature('pc1').set('parmax', '2*pi');
g.feature('pc1').set('coord', {'cos(s)', 'sin(s)', 's*0.2'});
g.run;
```

SEE ALSO

[BezierPolygon](#), [ParametricSurface](#)

ParametricSurface

Create a parametric surface defined by coordinate expressions in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>, "ParametricSurface");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property, <value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).importData();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute, <value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"ParametericSurface")` to create a parametric surface. Self-intersecting surfaces are not supported. The following properties are available:

TABLE 3-125: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	double[]	{0,0,1}	Direction of the z-axis of the local coordinate system. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
coord	String[3]	empty	Coordinates of parametric surface as function of parameters.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
maxknots	int	10	Maximum number of knots in each parameter coordinate.
parname1	String	s1	First parameter name.
parname2	String	s2	Second parameter name.
parmax1	double	1	Maximum value of first parameter.
parmax2	double	1	Maximum value of second parameter.
parmin1	double	0	Minimum value of first parameter.
parmin2	double	0	Minimum value of second parameter.
pos	double[]	{0,0,0}	Position of the object.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
rot	double	0	Rotational angle about axis.
rtol	double	1e-6	Relative tolerance.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	bnd	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

The expressions in `coord` can contain functions defined in the model. If the definition of such a function is changed, the parametric surface is not automatically rebuilt. Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()` to rebuild the parametric surface after such a change.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-126: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following commands create a parametric surface in 3D with the shape of a twisted rectangle:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("ps1", "ParametricSurface");
g.feature("ps1").set("parmin1", "-1");
g.feature("ps1").set("parmax2", "pi");
g.feature("ps1").set("coord", new String[] {"s1*cos(s2)", "s1*sin(s2)", "s2"});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('ps1', 'ParametricSurface');
g.feature('ps1').set('parmin1', '-1');
g.feature('ps1').set('parmax2', 'pi');
g.feature('ps1').set('coord', {'s1*cos(s2)', 's1*sin(s2)', 's2'});
g.run;
```

SEE ALSO

[ParametricCurve](#)

PartInstance

Create an instance of a geometry part.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"PartInstance");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).geom().geomSequenceMethod;
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

This feature creates an instance of a geometry part with new values of its input parameters. Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).geom()` to access its local copy of the part.

The following properties are available:

TABLE 3-127: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
part	String		Tag of part to call, or local to use a local part.
inputname	String[]		Names of the input parameters (read-only).
inputexpr	String[]		Expressions for the input parameters.

The default for part is the first part in `model.geom()`, or local if there is none.

In 3D, the following additional properties are available to control the positioning of the output objects:

TABLE 3-128: VALID PROPERTY/VALUE PAIRS IN 3D.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
ax2	double[2]	{0,0}	Spherical angles theta and phi, used if axistype is spherical.
ax3	double[3]	{0,0,1}	Axis vector, used if axistype is cartesian.
axis	double[]	{0,0,1}	Rotation axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical. Alias for ax2 and ax3.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. Used if specify is set to axis.
displ	double[3]	{0,0,0}	Displacement vector.
edge	Selection		Edge selection. Used when specify is set to edge.
eulerang	double[3]	{0,0,0}	Intrinsic Z-X-Z Euler angles α , β , and γ , used if specify is set to eulerang.
rot	double	0	Rotation angle, used if specify is set to axis.
specify	axis eulerang edge	axis	Specify an axis of rotation, Euler angles (Z-X-Z) or a straight edge.
workplanepart	String	xyplane	Tag of work plane in the geometry part, or xyplane.
workplanesrc	String	this	Tag of PartInstance feature to take work plane from, or this to take it from this sequence.
workplane	String	xyplane	Tag of work plane to match, or xyplane.

In 2D, the following additional properties are available to control the positioning of the output objects:

TABLE 3-129: VALID PROPERTY/VALUE PAIRS IN 2D.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
displ	double[2]	{0,0}	Displacement vector.
rot	double	0	Rotation angle.

For each geometric entity level (object, domain, boundary, edge, and point) that exists in the geometry, there is in addition the following properties related to the output selections for that level:

TABLE 3-130: VALID PROPERTY/VALUE PAIRS FOR OUTPUT SELECTIONS.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
selcolor _{level}	String[]	all none	Colors of selections (read-only). This property is only available for domains and boundaries in 3D and domains in 2D.
selkeepnoncontr	on off	on	Keep all noncontributing selections.
seltag _{level}	String[]		Tags of selections (read-only).
selname _{level}	String[]		Names of selections (read-only).

TABLE 3-130: VALID PROPERTY/VALUE PAIRS FOR OUTPUT SELECTIONS.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
<code>selcontributeto$level$</code>	<code>String[]</code>	all none	Tags of cumulative selections to contribute to, or none to not contribute.
<code>selkeep$level$</code>	<code>String[]</code> with on/off values	all off	Keep selection from part (only used when <code>selkeepnoncontr</code> is off).
<code>selshow$level$</code>	<code>String[]</code> with on/off values	all on	Show selection in physics, materials, and so on; in part instances; or in 3D from a plane geometry.

where *level* is obj, dom, bnd, edg, or pnt for geometry objects, domains, boundaries, edges, and points, respectively.

The following attributes are available:

TABLE 3-131: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

In version 5.1, the following properties were deprecated and replaced:

TABLE 3-132: DEPRECATED PROPERTIES.

PREVIOUS NAME	NEW NAME IN 5.1
arg	inputname
argexpr	inputexpr
argvalue	inputvalue
argdescr	inputdescr
subsequence	part
workplanesub	workplanepart

EXAMPLE

Create a geometry part that makes a torus of revolution angle α , where α is an argument (default value: 90 degrees). Then add work planes for the two planar face to make it easy to position the result:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
GeomSequence p = model.geom().create("part1", "Part", 3);
p.inputParam().set("a", 90);
p.create("tor1", "Torus");
p.feature("tor1").set("angle", "a");
p.run("tor1");
p.create("wp1", "WorkPlane");
p.feature("wp1").set("planetype", "faceparallel");
p.feature("wp1").selection("face").set("tor1", new int[]{1});
p.feature("wp1").set("reverse", "on");
p.create("wp2", "WorkPlane");
p.feature("wp2").set("planetype", "faceparallel");
p.feature("wp2").selection("face").set("tor1", new int[]{6});
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
p = model.geom.create('part1', 'Part', 3);
p.inputParam.set('a', 90);
p.create('tor1', 'Torus');
p.feature('tor1').set('angle', 'a');
p.run('tor1');
```



```

p.create('wp1', 'WorkPlane');
p.feature('wp1').set('planetype', 'faceparallel');
p.feature('wp1').selection('face').set('tor1', 1);
p.feature('wp1').set('reverse', 'on');
p.create('wp2', 'WorkPlane');
p.feature('wp2').set('planetype', 'faceparallel');
p.feature('wp2').selection('face').set('tor1', 6);

```

Create two part instances of this geometry part. The first has $\alpha = 90$ (the default value), and the second has $\alpha = 120$. The objects are positioned so that the two circular faces match, with a rotation angle of 50 degrees.

Code for Use with Java

```

model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("pi1", "PartInstance");
g.create("pi2", "PartInstance");
g.feature("pi2").setIndex("inputexpr", 120, 0);
g.feature("pi2").set("workplanepart", "wp1");
g.feature("pi2").set("workplanesrc", "pi1");
g.feature("pi2").set("workplane", "wp2");
g.feature("pi2").set("rot", 50);
g.run("pi2");

```

Code for Use with MATLAB

```

model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('pi1', 'PartInstance');
g.create('pi2', 'PartInstance');
g.feature('pi2').setIndex('inputexpr', 120, 0);
g.feature('pi2').set('workplanepart', 'wp1');
g.feature('pi2').set('workplanesrc', 'pi1');
g.feature('pi2').set('workplane', 'wp2');
g.feature('pi2').set('rot', 50);
g.run('pi2');

```

SEE ALSO

[If, ElseIf, Else, EndIf](#)

Partition

Partition 2D and 3D geometry objects using tool objects or a work plane.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Partition");
model.component(<ctag>).geom(<tag>).feature().selection(property);
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);
model.component(<ctag>).geom(<tag>).feature().getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

The Partition Boolean operation partitions each input object using a set of tool objects or (in 3D only) a work plane. That is, within the input object new boundaries, edges, and vertices are created that come from the tools or the work plane.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
absrepairtol	double	<code>...geom(<tag>).absRepairTol()</code>	Absolute repair tolerance.
contributeto	String	none	Tag of cumulative selection to contribute to.
input	Selection	empty	Objects to partition.
keepinput	on off	off	Keep input objects.
keeptool	on off	off	Keep tool objects.
partitionwith	objects workplane	objects	Partition with tool objects or a work plane.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
repairtol	double	<code>...geom(<tag>).repairTol()</code>	Relative repair tolerance, relative to size of union of inputs.
repairtoltype	auto relative absolute	<code>...geom(<tag>).repairTolType()</code>	Repair tolerance type: automatic, relative, or absolute.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
tool	Selection	empty	Tool objects.
workplane	String		Work plane to partition with.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-133: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

The keep property has been deprecated in version 5.3a. Instead, use the new properties keepinput and keeptool. If you set keep to on, both keepinput and keeptool. If you get the value of keep, it is on if keepinput and keeptool are on.

EXAMPLE

Create an interior boundary in a cylinder by partitioning it with an oblique work plane:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("cyl1", "Cylinder");
g.feature("cyl1").set("h", 10);
g.create("wp1", "WorkPlane");
g.feature("wp1").set("planetype", "general");
g.feature("wp1").setIndex("genpoints", 4, 0, 2);
```

```

g.feature("wp1").setIndex("genpoints", 5, 1, 2);
g.feature("wp1").setIndex("genpoints", 5, 2, 2);
g.create("par1", "Partition");
g.feature("par1").selection("input").set("cyl1");
g.feature("par1").set("partitionwith", "workplane");
g.run("par1");

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom', 3);
g.create('cyl1', 'Cylinder');
g.feature('cyl1').set('h', 10);
g.create('wp1', 'WorkPlane');
g.feature('wp1').set('planetype', 'general');
g.feature('wp1').setIndex('genpoints', 4, 0, 2);
g.feature('wp1').setIndex('genpoints', 5, 1, 2);
g.feature('wp1').setIndex('genpoints', 5, 2, 2);
g.create('par1', 'Partition');
g.feature('par1').selection('input').set('cyl1');
g.feature('par1').set('partitionwith', 'workplane');
g.run('par1');

```

SEE ALSO

[Compose](#), [Union](#), [Intersection](#), [Difference](#), [WorkPlane](#), [PartitionDomains](#), [PartitionEdges](#)

PartitionDomains

Partition domains in 2D or 3D geometries with curves and surfaces defined in various ways.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"PartitionDomains");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);
model.component(<ctag>).geom(<tag>).feature().getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

The Partition Domains operation partitions selected domains using curves or surfaces defined by vertices, edges, faces, work planes, or objects.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
absrepairtol	double	...geom(<tag>).absRepairTol()	Absolute repair tolerance.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
domain	Selection		Selection of domains to partition.
edge	Selection		Edges that define the partitioning curves (2D).

PROPERTY	VALUE	DEFAULT	DESCRIPTION
extendededge	Selection		Edges whose underlying curves define the partitioning curves (2D).
extendedface	Selection		Faces whose underlying surfaces define the partitioning surfaces (3D).
face	Selection		Faces that define the partitioning surfaces (3D).
keepobject	on off	on	Keep objects used to partition the geometry with, when <code>partitionwith</code> is set to <code>objects</code> .
object	Selection		Geometry objects used to partition domains.
partitionwith	linesegments lines edges extendededges objects in 2D workplane faces extendedfaces objects in 3D	linesegments in 2D, workplane in 3D	Method for partitioning the domains. The <code>objects</code> option is not available for partitioning domains after a form union/assembly operation.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
repairtol	double	<code>...geom(<tag>).repairTol()</code>	Relative repair tolerance, relative to size of union of inputs.
repairtoltype	auto relative absolute	<code>...geom(<tag>).repairTolType()</code>	Repair tolerance type: automatic, relative, or absolute.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if <code>selresult</code> is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. <code>obj</code> is not available in a component's geometry. <code>dom</code> , <code>bnd</code> , and <code>edg</code> are not available in all features.
vertexsegment	Selection		Vertices that define the line segments (2D).
vertexline	Selection		Vertices that define the lines (2D).
workplane	String		Work plane to partition with (3D).

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-134: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use <code>inherit</code> to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Partition](#), [PartitionEdges](#), [PartitionFaces](#)

PartitionEdges

Partition edges in 2D or 3D geometries at some positions along the edges.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"PartitionEdges");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);
model.component(<ctag>).geom(<tag>).feature().getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

The Partition Edges operation partitions selected edges at specified locations. You can specify the positions using parameters based on the arc length or existing vertices whose orthogonal projections on the edges specify the positions.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
edge	Selection		Selection of edges to partition.
position	arclength projection	arclength	Specifies the position along the selected edges.
param	double[]		Relative arc length parameters.
vertexproj	Selection		Vertices to project on the selected edges.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-135: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Partition](#), [PartitionDomains](#), [PartitionFaces](#)

PartitionFaces

Partition faces in 3D geometries at some positions on the faces.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"PartitionFaces");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);
model.component(<ctag>).geom(<tag>).feature().getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

The Partition Faces operation partitions selected faces at specified locations. You can specify the positions using vertices to define curve segments, adjacent edges that are extended, or a work plane.

The following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
extendededge	Selection		Planar edges defining partitioning lines, circles, or planes.
face	Selection		Faces to partition.
partitionwith	workplane curvesegments extendededges	curvesegments	Method for partitioning the faces.
vertexsegment	Selection		Vertices that define the curve segments.
workplane	String		Work plane to partition with.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-136: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Partition](#), [PartitionDomains](#), [PartitionEdges](#)

Point

Create a point object in 1D, 2D, or 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Point");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Point")` to create one or more points. The following property is available:

TABLE 3-137: VALID PROPERTY/VALUE PAIR.

PROPERTY NAME	PROPERTY VALUE	DEFAULT	DESCRIPTION
p	double[] double[][]	0	Coordinates.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj bnd pnt off	pnt in 2D and 3D; bnd in 1D	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

If `p` is a one-dimensional array, a single point with these coordinates is constructed. If `p` is a two-dimensional array, a point object containing several points is constructed, where the *n*th point has *i*th coordinate `p[i][n]`.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Point](#).

The following attributes are available:

TABLE 3-138: VALID ATTRIBUTES.

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry. Available only in 2D and 3D.

COMPATIBILITY

The following aliases work in 1D, 2D, and 3D, respectively:

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"point1");
model.component(<ctag>).geom(<tag>).create(<ftag>,"point2");
model.component(<ctag>).geom(<tag>).create(<ftag>,"point3");
```

EXAMPLE

The following commands generate a point at (1, 2) in a 2D geometry:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("p1","Point");
```

```
g.feature("p1").set("p",new double[][]{{1},{2}});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('p1','Point');
g.feature('p1').set('p',[1,2]);
g.run;
```

Polygon

Create curve or solid polygon consisting of line segments in 2D or 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Polygon");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Polygon")` to create a polygon or a line segment. The following properties are available

TABLE 3-139: VALID PROPERTY/VALUE PAIRS FOR POLYGON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
filename	String		If source is file, the file that contains the vertex coordinates.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom in 2D; edg in 3D	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
source	table file vectors	vectors	Whether vertex coordinates are specified as vectors, a table, or read from a file.
table	double[][]		The vertex coordinates when source is table, size N*sdim.
type	solid open closed	solid (2D) open (3D)	Object type. solid is not available in 3D.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

TABLE 3-139: VALID PROPERTY/VALUE PAIRS FOR POLYGON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
x	double[]	{}	x-coordinates for vertices.
y	double[]	{}	y-coordinates for vertices.
z	double[]	{}	z-coordinates for vertices.

If `type` is `open` or `closed`, a curve consisting of line segments is constructed. If `type` is `solid`, the solid enclosed by such a closed polygon is constructed. If `type` is `closed` or `solid`, but the first and last control points are different, an extra segment is added to close the curve.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).importToTable()` to read data from the file defined by the `filename` property and store the data in the `table` property. The `source` property is also changed to `table`.

If `source` is `file`, the polygon is not automatically rebuilt when the data in the file changes. Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).importData()` to rebuild the polygon after such a change.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Polygon](#).

The following attributes are available:

TABLE 3-140: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"line1")` constructs an open polygon.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"poly1")` constructs a closed polygon.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"line2")` or `model.component(<ctag>).geom(<tag>).create(<ftag>,"poly2")` constructs a solid polygon.

EXAMPLE

Construct a solid triangle `pol1`:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("pol1","Polygon");
g.feature("pol1").set("x","0,0,2").set("y","1,0,0");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('pol1','Polygon');
g.feature('pol1').set('x','0,0,2').set('y','1,0,0');
g.run;
```

SEE ALSO

[CubicBezier](#), [QuadraticBezier](#)

Pyramid

Create solid or surface rectangular pyramid or frustum in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Pyramid");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Pyramid")` to create a pyramid. The following properties are available:

TABLE 3-141: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
a, b	double	1	Side lengths for bottom rectangle.
axis	double[]	{0,0,1}	Direction of the axis orthogonal to the bottom rectangle. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
h	double	1	Height.
pos	double[]	{0,0,0}	Center of the bottom rectangle.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
rat	double	0.5	Ratio of perimeter of top rectangle and bottom rectangle.
rot	double	0	Rotational angle about axis.
type	solid surface	solid	Object type.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-142: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"pyramid3")` creates a solid pyramid.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"pyramid2")` creates a surface pyramid.

The following properties are also available:

TABLE 3-143: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ax2	double[]	{0,0}	Alias for axis when <code>axistype</code> is spherical.
ax3	double[]	{0,0,1}	Alias for axis when <code>axistype</code> is cartesian.
x, y, z	double	0	Alias for pos.

The property `const` is no longer available.

EXAMPLE

Create a pyramid frustum with the base face in the *xy*-plane:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("e1","Pyramid");
g.feature("e1").set("a",10).set("b",40);
g.feature("e1").set("h",20);
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('e1','Pyramid');
g.feature('e1').set('a',10).set('b',40);
g.feature('e1').set('h',20);
```

Create a pyramid with an apex:

Code for Use with Java

```
g.create("e2","Pyramid");
g.feature("e2").set("a",1).set("b",2);
g.feature("e2").set("h",4);
g.feature("e2").set("rat",0);
g.feature("e2").set("pos","100 100 100");
g.feature("e2").set("axis","0 1 4");
g.feature("e2").set("rot",45);
g.run();
```

Code for Use with MATLAB

```
g.create('e2','Pyramid');
g.feature('e2').set('a',1).set('b',2);
g.feature('e2').set('h',4);
g.feature('e2').set('rat',0);
g.feature('e2').set('pos','100 100 100');
```

```
g.feature('e2').set('axis','0 1 4');
g.feature('e2').set('rot',45);
g.run;
```

SEE ALSO

[Cone](#), [ECone](#)

QuadraticBezier

Create a rational Bézier curve of degree 2 in 2D or 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"QuadraticBezier");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"QuadraticBezier")` to create a quadratic Bézier curve in 2D or 3D. The following properties are available:

TABLE 3-144: VALID PROPERTY/VALUE PAIRS FOR A QUADRATIC BÉZIER CURVE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
p	double[sdim][3]	0	Control points.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
w	double[3]	{1, 1/sqrt(2), 1}	Weights for the control points.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [QuadraticBezier](#).

The following attributes are available:

TABLE 3-145: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

EXAMPLE

The following commands creates a quadratic Bézier curve in 2D with control points in (4, 2); (8, 8); and (16, 4):

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("qb1", "QuadraticBezier");
g.feature("qb1").setIndex("p", 4, 0, 0);
g.feature("qb1").setIndex("p", 2, 1, 0);
g.feature("qb1").setIndex("p", 8, 0, 1);
g.feature("qb1").setIndex("p", 8, 1, 1);
g.feature("qb1").setIndex("p", 16, 0, 2);
g.feature("qb1").setIndex("p", 4, 1, 2);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('qb1','QuadraticBezier');
g.feature('qb1').setIndex('p', 4, 0, 0);
g.feature('qb1').setIndex('p', 2, 1, 0);
g.feature('qb1').setIndex('p', 8, 0, 1);
g.feature('qb1').setIndex('p', 8, 1, 1);
g.feature('qb1').setIndex('p', 16, 0, 2);
g.feature('qb1').setIndex('p', 4, 1, 2);
g.run;
```

SEE ALSO

[CubicBezier](#), [CompositeCurve](#)

Rectangle

Create a solid rectangle, curve rectangle, or bounding box in 2D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Rectangle");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>, "Rectangle")` to create a rectangle. The following properties are available:

TABLE 3-146: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
base	corner center	corner	Positions the object either centered about pos or with the lower-left corner in pos.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
createpar	on off	off	Create parameters.
definedby	size bounding	size	Specify if the rectangle is to be defined by its size and position or be a bounding box. See Table 3-147 when definedby is bounding.
layer	double[]		Thicknesses of layers.
layerleft	on off	off	Apply layers to the left.
layerright	on off	off	Apply layers to the right.
layertop	on off	off	Apply layers on top.
layerbottom	on off	on	Apply layers on bottom.
pos	double[]	{0,0}	Position of the object.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
rot	double	0	Rotational angle about pos.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances/3D (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
size	double[]	{1,1}	Side lengths.
type	solid curve	solid	Object type.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

TABLE 3-147: ADDITIONAL PROPERTIES WHEN DEFINEDBY IS BOUNDING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
orientdef	manual principal	principal	Specify orientation definition.
input	Selection		Input objects or entities.
keep	on off	on	Keep input objects. Used if input is an object selection.
leftmargin	double	0	Left margin around bounding box.
rightmargin	double	0	Right margin around bounding box.

TABLE 3-147: ADDITIONAL PROPERTIES WHEN DEFINED BY IS BOUNDING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottommargin	double	0	Bottom margin around bounding box.
topmargin	double	0	Top margin around bounding box.

Additional properties are available with the Design Module, see [Rectangle](#).

The following attributes are available:

TABLE 3-148: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"rect2")` creates a solid rectangle.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"rect1")` creates a curve rectangle.

The following properties are also available:

TABLE 3-149: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
lx, ly	double	1	Alias for size.
x, y	double	0	Alias for pos.

The property `const` is no longer available.

RemoveDetails

Remove small details from the geometry.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"RemoveDetails");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"RemoveDetails")` to automatically remove small details from the geometry. You can also add extra local virtual operations using the following syntax (in this example, adding an `IgnoreEdges` operation):

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).create("IgnoreEdges", "ige1");
```

You can then build that local virtual operation using

```
model.component(<ctag>).geom(<tag>).run("<ftag>/ige1");
```

You can also use the selection property input:

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input");
```

If the `RemoveDetails` operation is in the automatic state, use the `.problems()` syntax to retrieve information on warnings (if any). The problem list is cleared if you switch to the manual state.

The following properties are available:

TABLE 3-150: VALID PROPERTY/VALUE PAIRS FOR REMOVEDDETAILS.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
automatic	on off	on	Mode of operation.
contangletol	double	5 degrees	Maximum allowed angular tangent deviation across a vertex or edge to be ignored.
contvertices	boolean	true	Specifies if vertices with continuous tangent are ignored.
detailsizetype	auto relative absolute	auto	Detail size type: automatic, relative, or absolute.
input	Selection		Entities for which to remove small details.
maxrelsize	double	0.001	Maximum relative detail size, relative to size of geometry.
maxabssize	double		Maximum absolute detail size.
selection	geometry entities	geometry	Remove small details from the entire geometry or from geometric entities specified using the input.
shortedges	boolean	true	Specifies if short edges are removed.
sliverfaces	boolean	true	Specifies if sliver faces are removed.
smallfaces	boolean	true	Specifies if small faces are removed.
thindomains	boolean	true	Specifies if thin domains (that is, domains with a thickness less than the specified detail size) are removed.

Revolve

Revolve planar faces in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Revolve");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Revolve")` to revolve objects from a work plane.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the work plane objects to revolve. The default selection is all available objects from the last preceding work plane.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("inputface")` to select the faces to revolve. Faces are revolved when the `workplane` property is none; otherwise, work plane objects are revolved.

The following properties are available:

TABLE 3-151: VALID PROPERTY/VALUE PAIRS FOR REVOLVE.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
angle1	double	0	Start revolution angle.
angle2	double	0	End revolution angle.
angtype	specang full	specang	Type of specification.
axis	double[2]	{0,1}	Direction of axis of revolution (in local coordinate system).

TABLE 3-151: VALID PROPERTY/VALUE PAIRS FOR REVOLVE.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
axis3	double[3]	{0,1,0}	Direction of axis of revolution (in 3D coordinate system).
axistype	2d 3d edge	2d	Type of revolution axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
edge	Selection		Specifies a straight edge used as revolution axis when axistype is edge.
input	Selection	all objects	Objects to revolve.
intputedge	Selection		Edges to revolve. Used if revolvefrom is edges.
intputface	Selection		Faces to revolve. Used if revolvefrom is faces.
intputvertex	Selection		Vertices to revolve. Used if revolvefrom is vertices.
origfaces	on off	on	Keep original faces.
pos	double[2]	{0,0}	A point on the axis of revolution (in work plane's coordinate system).
pos3	double[3]	{0,0,0}	A point on the axis of revolution (in 3D coordinate system).
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
revolvefrom	workplane faces edges vertices	workplane, if available, otherwise faces	Revolve work plane objects, faces, edges, or vertices from 3D objects.
selrevolve	on off	off	Revolve selections from work plane (used if revolvefrom is workplane)
selrevolveshow	on off	on	Show revolved selections in physics/instances (only used when selrevolve is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
unite	on off	on	Unite revolved objects with input objects.
workplane	String		Work plane to revolve or none to revolve faces.

Each 2D object in **input** or planar face in **inputface** is revolved about the revolution axis. The range of angles is given by the properties **angle1** and **angle2**. If **axistype** is 2d, the revolution axis is defined in a local coordinate system. The revolution axis goes through **pos** with direction **axis**. If **axistype** is 3d, the revolution axis is defined in the 3D coordinate system. The revolution axis goes through **pos3** with direction **axis3**.

When revolving work plane objects, the local system is defined as the local system of the work plane. When revolving faces, the local system is defined by the face with the smallest face number in the object that comes first

in the geometry sequence. The local z -axis is parallel to the face normal and located at the center of the face. The local x -axis is defined by the tangent direction corresponding to the first parameter in the surface representation for the face.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-152: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use <code>inherit</code> to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

Additional properties:

TABLE 3-153: VALID PROPERTY/VALUE PAIRS FOR COMPATIBILITY.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
angles	double double[2]	2*pi	Alias for <code>angle1</code> and <code>angle2</code> .
keep	on off	off	Alias for <code>unite</code> property with opposite value.
revaxis	double[2][2]	{{0,0},{0,1}}	Alias for <code>pos</code> (first column) and <code>axis</code> (second column).

EXAMPLE

Create a torus about the y -axis:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.geom().create("geom1",3);
g.create("wp1", "WorkPlane");
g.feature("wp1").geom().create("c1", "Circle");
g.feature("wp1").geom().feature("c1").set("pos", "2 0");
g.run("wp1");
g.create("r1", "Revolve");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
g = model.geom.create('geom1',3);
g.create('wp1', 'WorkPlane');
g.feature('wp1').geom.create('c1', 'Circle');
g.feature('wp1').geom.feature('c1').set('pos', '2 0');
g.run('wp1');
g.create('r1', 'Revolve');
g.run;
```

SEE ALSO

[Extrude](#), [WorkPlane](#)

RigidTransform

Perform a rigid transform (translation and rotation) of 3D geometry objects. You can use this feature to position and orient geometry objects by matching a coordinate system (work plane) with another coordinate system (work plane) and then optionally performing a rotation and translation in the work plane's coordinate system.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"RigidTransform");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"RigidTransform")` to make a rigid transform of 3D geometry objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects for which to perform a rigid transform. The default selection is empty.

The following properties are available:

TABLE 3-154: VALID PROPERTIES FOR RIGIDTRANSFORM.

NAME	VALUE	DEFAULT	DESCRIPTION
ax2	double[2]	{0,0}	Spherical angles theta and phi, used if axistype is spherical.
ax3	double[3]	{0,0,1}	Axis vector, used if axistype is cartesian.
axis	double[]	{0,0,1}	Rotation axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical. Alias for ax2 and ax3.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. Used if specify is set to axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
displ	double[3]	{0,0,0}	Displacement.
edge	Selection		Edge selection. Used when specify is set to edge.
eulerang	double[3]	{0,0,0}	Intrinsic Z-X-Z Euler angles α , β , and γ , used if specify is set to eulerang.
input	object selection		Geometry objects to perform a rigid transformation of.
keep	on off	off	Keep input objects.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
rot	double[]	0	Rotation angles for one or more rotations of the input objects, used if specify is set to axis.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
specify	axis eulerang edge	axis	Specify an axis of rotation, Euler angles (Z-X-Z), or a straight edge.

TABLE 3-154: VALID PROPERTIES FOR RIGIDTRANSFORM.

NAME	VALUE	DEFAULT	DESCRIPTION
workplane	xyplane work-plane feature tag	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.
workplaneobj	xyplane work-plane feature tag	xyplane	The work plane for the geometry objects.
workplanesrc	this part-instance feature tag	this	The source to take the work plane from.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-155: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use <code>inherit</code> to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Mirror](#), [Move](#), [Copy](#), [Scale](#), [Rotate](#)

Rotate

Rotate objects about a point in 2D or an axis in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Rotate");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Rotate")` to rotate geometry objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects to rotate. The default selection is empty.

The following properties are available:

TABLE 3-156: VALID PROPERTIES FOR ROTATE.

NAME	VALUE	DEFAULT	DESCRIPTION
ax2	double[2]	{0,0}	Spherical angles theta and phi, used if <code>axistype</code> is spherical.
ax3	double[3]	{0,0,1}	Axis vector, used if <code>axistype</code> is cartesian.
axis	double[]	{0,0,1}	Rotation axis. Vector has length 3 if <code>axistype</code> is cartesian, and length 2 if <code>axistype</code> is spherical. Alias for <code>ax2</code> and <code>ax3</code> .
axistype	x y z cartesian spherical	z	Coordinate system used for axis. Used if specify is set to axis.

TABLE 3-156: VALID PROPERTIES FOR ROTATE.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
edge	Selection		Edge selection. Used when specify is set to edge.
eulerang	double[3]	{0,0,0}	Intrinsic Z-X-Z Euler angles α , β , and γ , used if specify is set to eulerang.
indexattr	on off	off	Create index attributes.
indexattrtag	String		Tag of index attribute. Used if indexattr is on.
input	Selection		Objects to rotate.
keep	on off	off	Keep input objects.
pos	double[]		Center of rotation.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
rot	double[]	0	Rotation angles for one or more rotations of the input objects, used if specify is set to axis.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
specify	axis eulerang edge	axis	Specify an axis of rotation, Euler angles (Z-X-Z), or a straight edge.
specifypoint	coord vertex	coord	Specification of point on axis of revolution (in 3D only). Used when specify is axis.
vertex	Selection		Vertex for point on axis of revolution. Used when specifypoint is vertex.
workplane	xyplane work-plane feature tag	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.
workplanesrc	this part-instance feature tag	this	Part-instance feature to take the work plane from (in 3D only).

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Rotate](#).

The following attributes are available:

TABLE 3-157: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

The possibility to set and get a rotation matrix has been removed.

The property out is no longer available.

EXAMPLE

The commands below create and then rotate an ellipse by 10 degrees about (2, 3):

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("e1","Ellipse");
g.feature("e1").set("semiaxes","1 3");
g.create("r1","Rotate");
g.feature("r1").selection("input").set("e1");
g.feature("r1").set("rot",10);
g.feature("r1").set("pos","2 3");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('e1','Ellipse');
g.feature('e1').set('semiaxes','1 3');
g.create('r1','Rotate');
g.feature('r1').selection('input').set('e1');
g.feature('r1').set('rot',10);
g.feature('r1').set('pos','2 3');
g.run;
```

SEE ALSO

[Mirror](#), [Move](#), [Copy](#), [Scale](#), [RigidTransform](#)

Scale

Scale objects around a point.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Scale");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Scale")` to scale geometry objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects to scale. The default selection is empty.

The following properties are available:

TABLE 3-158: VALID PROPERTIES FOR SCALE.

NAME	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
factor	double double[]	1	Scale factor(s).
input	Selection		Objects to scale.
keep	on off	off	Keep input objects.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
pos	double[]	0	Center of scaling.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
specifycenter	coord vertex	coord	Specification of center of scaling
vertex	Selection		Vertex for center. Used when specifycenter is vertex.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from (in 3D only).
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system (in 3D only). The default, xyplane, is the global Cartesian coordinate system.

If factor is an array, the inputs are scaled by the factor[i] in the ith coordinate.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Scale](#).

The following attributes are available:

TABLE 3-159: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

EXAMPLE

The sequence below scales the unit circle by (1, 2) about (2, 3):

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
```

```

GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("c1","Circle");
g.create("s1","Scale");
g.feature("s1").selection("input").set("c1");
g.feature("s1").set("factor", "1,2");
g.feature("s1").set("pos",new double[]{2,3});
g.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('c1','Circle');
g.create('s1','Scale');
g.feature('s1').selection('input').set('c1');
g.feature('s1').set('factor', '1,2');
g.feature('s1').set('pos',[2,3]);
g.run;

```

COMPATIBILITY

The property out is no longer available.

SEE ALSO

[Mirror](#), [Move](#), [Copy](#), [Rotate](#), [RigidTransform](#)

Sphere

Create a solid ball, surface sphere, or bounding ball in 3D.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Sphere");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Sphere")` to create a sphere. The following properties are available:

TABLE 3-160: VALID PROPERTY/VALUE PAIRS FOR SPHERE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	double[]	{0,0,1}	Direction of the local z-axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
createpar	on off	off	Create parameters.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.

TABLE 3-160: VALID PROPERTY/VALUE PAIRS FOR SPHERE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
definedby	size bounding	size	Specify if the sphere is to be defined by its size and position or be a bounding ball. See Table 3-161 when definedby is bounding.
layer	double[]		Thicknesses of layers.
pos	double[]	{0,0,0}	Center.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
r	double	1	Radius.
rot	double	0	Rotational angle about axis.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
type	solid surface	solid	Object type.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

TABLE 3-161: ADDITIONAL PROPERTIES WHEN DEFINEDBY IS BOUNDING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
centerdef	manual centroid	centroid	Specify center definition.
input	Selection		Input objects or entities.
keep	on off	on	Keep input objects. Only used if input is an object selection.
margin	double	0	Margin around bounding ball.

The following attributes are available:

TABLE 3-162: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"sphere3")` creates a solid sphere.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"sphere2")` creates a surface sphere.

The following properties are also available:

TABLE 3-163: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ax2	double[]	{0,0}	Alias for axis when axistype is spherical.
ax3	double[]	{0,0,1}	Alias for axis when axistype is cartesian.
x, y, z	double	0	Alias for pos.

The property `const` is no longer available.

EXAMPLE

The following commands create a surface and solid sphere, where the position and radius are defined differently:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("s2","Sphere");
g.feature("s2").set("type","surface");
g.feature("s2").set("pos","0 1 0");
g.create("s3","Sphere");
g.feature("s3").set("r",4);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('s2','Sphere');
g.feature('s2').set('type','surface');
g.feature('s2').set('pos','0 1 0');
g.create('s3','Sphere');
g.feature('s3').set('r',4);
g.run;
```

SEE ALSO

[Ellipsoid](#)

Split

Split (explode) objects into domains, faces, edges, or vertices.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Split");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Split")` to split geometry objects.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the objects to split. The default selection is empty.

TABLE 3-164: VALID PROPERTY/VALUE PAIRS FOR SPLIT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
input	Selection		Objects to split.
keep	on off	off	Keep input objects.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

- A solid object is split into solids corresponding to its domains.
- A surface object is split into surface objects corresponding to its faces.
- A curve object is split into curve objects corresponding to its edges.
- A point object is split into point objects corresponding to its vertices.
- A general (mixed) object is split into solids (corresponding to the domains), surface objects (corresponding to faces not adjacent to a domain), curve objects (corresponding to edges not adjacent to a face or domain), and point objects (corresponding to vertices not adjacent to an edge, face, or domain).

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-165: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use <code>inherit</code> to set the construction geometry attribute only if all input objects are construction geometry.

EXAMPLE

Split union of a solid circle and a solid rectangle.

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("r1","Rectangle");
g.create("c1","Circle");
g.create("u1","Union");
```

```

g.feature("u1").selection("input").set(new String[]{"r1","c1"});
g.create("spl1","Split");
g.feature("spl1").selection("input").set("u1");
g.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('r1','Rectangle');
g.create('c1','Circle');
g.create('u1','Union');
g.feature('u1').selection('input').set({'r1','c1'});
g.create('spl1','Split');
g.feature('spl1').selection('input').set('u1');
g.run();

```

SEE ALSO

[Compose](#), [Union](#), [Intersection](#), [Difference](#), [Delete](#)

Square

Create a solid or curve square in 2D.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Square");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Square")` to create a square. The following properties are available:

TABLE 3-166: VALID PROPERTY/VALUE PAIRS FOR SQUARE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
base	corner center	corner	Positions the object either centered about pos or with the lower left corner in pos.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult in active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
layer	double[]		Thicknesses of layers.
layerleft	on off	off	Apply layers to the left.
layerright	on off	off	Apply layers to the right.
layertop	on off	off	Apply layers on top.
layerbottom	on off	on	Apply layers on bottom.
pos	double[]	{0,0}	Position of the object.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
rot	double	0	Rotational angle about pos.

TABLE 3-166: VALID PROPERTY/VALUE PAIRS FOR SQUARE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
size	double	1	Side length.
type	solid curve	solid	Object type.
sellayer	on off	off	Create layer selections.
sellayershow	on off	on	Show layer selections in physics/instances/3D (used if sellayer is on)
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
contributeto	String	none	Tag of cumulative selection to contribute to.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

Additional properties are available with the Design Module, see [Square](#).

The following attributes are available:

TABLE 3-167: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"square2")` creates a solid square.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"square1")` creates a curve square.

The following properties are also available:

TABLE 3-168: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
l	double	1	Alias for size.
x, y	double	0	Alias for pos.

The property `const` is no longer available.

EXAMPLE

The sequence below creates a unit solid square:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("sq1","Square");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom',2);
g.create('sq1','Square');
g.run;
```

SEE ALSO

[Rectangle](#)

Sweep

Sweep one or several faces, edges, or points along a spine curve into solids, surfaces, or curves in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Sweep");
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Sweep")` to sweep faces, edges, or points along a spine curve.

Use the following methods to set the dimension of the entities to sweep and to select the entities:

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).
    selection("enttosweep").init(entDim);
model.component(<ctag>).geom(<tag>).feature(<ftag>).
    selection("enttosweep").set(<objname>,entities);
```

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("edge")` to select the edges to sweep along.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("diredge")` to select the edge whose direction defines the positive sweep direction. If this selection is empty, it is automatically set when the edge selection is set. The `diredge` selection can be empty if the `edge` selection contains a single edge.

The following properties are available:

TABLE 3-169: VALID PROPERTY/VALUE PAIRS FOR SWEEP.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
adjustlen	double	0.1	Adjustment parameter length at start of spine. Only used if <code>spineperpstart</code> is on.
adjustlenend	double	0.1	Adjustment parameter length at end. Only used if <code>spineperpend</code> is on.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the <code>customcolor</code> property. Coloring is only available when <code>selresult</code> is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when <code>color</code> is set to custom.
crossfaces	on off	off	Create cross-sectional faces. Only used if <code>enttosweep</code> has geometric entity level 2.
diredge	Selection		Direction-defining edge.
edge	Selection		Edges that form spine curve of the sweep.
enttomatch	Selection		Entities to match (with level 2, 1, or 0). Only used if <code>spineperpend</code> is on.

TABLE 3-169: VALID PROPERTY/VALUE PAIRS FOR SWEEP.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
enttosweep	Selection		Entities to sweep (supported levels 2, 1, or 0).
grep	bezier spline	spline	Geometry representation.
includefinal	on off	off	Include all used input objects in Finalize operation.
keep	on off	on	Keep input objects.
manualdir	on off	on	Manual control of sweep direction.
matchvertices	on off	off	Add twisting and scaling to match vertices.
maxknots	int	1000	Maximum number of knots.
movetospine	on off	off	Move entities to sweep to spine start and match work plane coordinate system with spine.
parameterization	arclength normalizedarclength internal	arclength	Parameterization of the spine curve: arc length, normalized arc length, or the internal parameterization in the geometry's data structures.
parname	String	s	Parameter name.
projvector	String[]	{0,0,1}	The vector to follow when controlling the twisting of the cross section. Only used if twisting is projvector.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
reversedir	on off	off	Reverse sweep direction. Only used if manualdir is on.
rtol	double	1e-4	Relative tolerance.
scale	String	1	Scale factor for cross section.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
smooth	on off	on	Smooth edge connections.
spineperpend	on off	off	Make spine perpendicular to entities to match at end of spine curve.
spineperpstart	on off	off	Make spine perpendicular to entities to sweep at start of spine curve.
twist	String	0	Twist angle for cross section.
twisting	none projvector curvature	none	Select how the twisting of cross section is done.
vertexinenttosweep	Selection		Vertex in entity to sweep. Only used if matchvertices is on.
vertexismatch	Selection		Vertex to match at end. Only used if matchvertices is on.

If `includefinal` is `off`, input objects are automatically removed in the Finalize (Form Union/Assembly) operation if they are completely used by this feature. Objects used in `enttosweep` are considered completely used if all of their entities of the selected dimension (2, 1, or 0) are included in `enttosweep`. Objects used in `edge` are considered completely used if all their edges are included in `edge`. If an object is considered completely used by one

property but not completely used by another property, the object is not removed in the Finalize operation. If `includefinal` is on, input objects are not removed in the Finalize operation.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-170: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use <code>inherit</code> to set the construction geometry attribute only if all input objects are construction geometry.

COMPATIBILITY

In COMSOL Multiphysics 4.2a and earlier versions, the positive sweep direction was defined as the curve direction instead of the edge direction.

EXAMPLE

Create a half torus about the *y*-axis using a sweep operation:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("wp1", "WorkPlane");
g.feature("wp1").geom().create("c1", "Circle");
g.create("pc1", "ParametricCurve");
g.create("swe1", "Sweep");
g.feature("pc1").set("parmax", "pi");
g.feature("pc1").set("coord", new String[]{"(cos(s)-1)*3", "0", "sin(s)*3"});
g.feature("swe1").selection("enttosweep").set("wp1.c1", new int[]{1});
g.feature("swe1").selection("edge").set("pc1(1)", new int[]{1});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('wp1', 'WorkPlane');
g.feature('wp1').geom.create('c1', 'Circle');
g.create('pc1', 'ParametricCurve');
g.create('swe1', 'Sweep');
g.feature('pc1').set('parmax', 'pi');
g.feature('pc1').set('coord', {'(cos(s)-1)*3', '0', 'sin(s)*3'});
g.feature('swe1').selection('enttosweep').set('wp1.c1', 1);
g.feature('swe1').selection('edge').set('pc1(1)', 1);
g.run;
```

SEE ALSO

[Extrude](#), [Helix](#), [Revolve](#), [WorkPlane](#)

Tangent

Create a tangent line segment to one or two 2D edges.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Tangent");  
model.component(<ctag>).geom(<tag>).feature(<ftag>).selection(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Tangent")` to create a line segment tangent to two edges or tangent to one edge with a fixed endpoint. The following properties are available:

TABLE 3-171: VALID PROPERTY/VALUE PAIRS FOR TANGENT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
contributeto	String	none	Tag of cumulative selection to contribute to.
edge	Selection		Edge in a geometry object to find tangent to.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj bnd edg pnt off	bnd	Show selections, if selresult is on, in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
start	double	0.5	Start guess for parameter value of point of tangency.
type	edge point coord	edge	Type of tangent.

If type is edge a common tangent line to two edges are constructed. Then, the following additional properties are available:

TABLE 3-172: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
edge2	Selection		Second edge in some geometry object to find tangent to.
start2	double	0.5	Start guess for parameter value of point of tangency.

If type is point a tangent line through a given point is constructed. Then, the following additional property is available:

TABLE 3-173: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
point	Selection		Point in some geometry object.

If type is coord a tangent line through a point with given coordinates are constructed. Then, the following additional property is available:

TABLE 3-174: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coord	double[]	{0,0}	Coordinates.

If a tangent cannot be found, a tangent to some adjacent edge is constructed, if possible.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-175: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(gname).create(fname, "tangent")` creates a Tangent feature.

The following properties are no longer supported:

TABLE 3-176: OBSOLETE PROPERTY/VALUE PAIRS.

PROPERTY NAME	PROPERTY VALUE	DEFAULT	DESCRIPTION
edim1	0 1	geometry dependent	Starting point element dimension: 0 for vertex, 1 for edge.
edim2	0 1	geometry dependent	Ending point element dimension: 0 for vertex, 1 for edge.
dom1	integer	1	Starting point entity number.
dom2	integer	1	Ending point entity number.
out	cell array of Strings	{}	Additional output data.
start1	double	0.5	Starting point parameter value on specified edge.

EXAMPLE

The following sequence generates a tangent from the unit circle to the point (2, 0):

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("c1", "Circle");
g.run("c1");
g.create("tan1", "Tangent");
g.feature("tan1").set("type", "coord");
g.feature("tan1").selection("edge").set("c1",3);
g.feature("tan1").set("coord", "2 0");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('c1', 'Circle');
g.run('c1');
g.create('tan1', 'Tangent');
g.feature('tan1').set('type', 'coord');
g.feature('tan1').selection('edge').set('c1',3);
g.feature('tan1').set('coord', '2 0');
g.run;
```

The following sequence generates a common tangent between two circles:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",2);
g.create("c1", "Circle");
g.create("c2", "Circle");
g.feature("c2").set("pos", "2 2");
```

```

g.run("c2");
g.create("tan1", "Tangent");
g.feature("tan1").selection("edge").set("c1",4);
g.feature("tan1").selection("edge2").set("c2",4);
g.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',2);
g.create('c1','Circle');
g.create('c2','Circle');
g.feature('c2').set('pos', '2 2');
g.run('c2');
g.create('tan1','Tangent');
g.feature('tan1').selection('edge').set('c1',4);
g.feature('tan1').selection('edge2').set('c2',4);
g.run;

```

SEE ALSO

[BezierPolygon](#)

Tetrahedron

Create a solid or surface tetrahedron in 3D.

SYNTAX

```

model.component(<ctag>).geom(<tag>).create(<ftag>,"Tetrahedron");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);

```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Tetrahedron")` to create a tetrahedron. The following properties are available:

TABLE 3-177: VALID PROPERTY/VALUE PAIR FOR TETRAHEDRON.

PROPERTY NAME	PROPERTY VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
p	double[3][4]	{0,0,1,0}, {0,1,0,0}, {0,0,0,1}}	Corner coordinates.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
type	solid surface	solid	Object type.

TABLE 3-177: VALID PROPERTY/VALUE PAIR FOR TETRAHEDRON.

PROPERTY NAME	PROPERTY VALUE	DEFAULT	DESCRIPTION
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the selresult and contributeto properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-178: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

model.component(<ctag>).geom(<tag>).create(<ftag>,"tetrahedron3") creates a solid tetrahedron.

model.component(<ctag>).geom(<tag>).create(<ftag>,"tetrahedron2") creates a surface tetrahedron.

EXAMPLE

The following commands generate a solid tetrahedron object:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("t1","Tetrahedron");
g.feature("t1").set("p", new double[][]{{0,0,1,0},{0,0.8,1,0},{0,0.1,0,0.2}});
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('t1','Tetrahedron');
g.feature('t1').set('p', [[0,0,1,0];[0,0.8,1,0];[0,0.1,0,0.2]]);
g.run;
```

SEE ALSO

[Hexahedron](#), [Pyramid](#)

Thicken2D

Thicken the edges of 2D curve or solid objects in the normal direction to convert them into solid objects.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Thicken2D");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use model.component(<ctag>).geom(<tag>).create(<ftag>,"Thicken2D") to create a thicken feature tagged <ftag> in the 2D geometry sequence with the tag <tag>.

Use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input")` to select the entities to thicken. To set the selection level for the selection object, and to select the input entities see [Geometry Object Selection Methods](#). For example, use `model.component(<ctag>).geom(<tag>).feature(<ftag>).selection("input").init(1)` to set the selection to be a selection of boundaries (geometric entities of dimension 1) for the thicken feature with the tag `<ftag>` in the 2D geometry sequence with the tag `<tag>`.

The following properties are available:

TABLE 3-179: VALID PROPERTY/VALUE PAIRS FOR THICKEN2D.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
input	Selection		Objects or boundaries to thicken.
keep	on off	off	Keep input objects.
offset	symmetric asymmetric	symmetric	Type of thicken.
totalthick	double	0	Total thickness. Used if offset is symmetric.
downthick	double	0	Upside thickness. Used if offset is asymmetric.
upthick	double	0	Downside thickness. Used if offset is asymmetric.
ends	straight circular	straight	Determines if the edges at the ends of the thickened object are straight or circular. Used if offset is symmetric.
convexcorner	fillet tangent extend noconnection	fillet	Handling of offset edges in convex corners.
propagatesel	on off	on	Propagate selections from input objects to resulting objects.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd pnt off	dom	Show selections, if selresult is on, in physics, materials, and so on; or in 3D from a plane geometry. obj is not available in a component's geometry
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
contribute to	String	none	Tag of cumulative selection to contribute to.

The following attributes are available:

TABLE 3-180: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off inherit	inherit	Designate the resulting objects as construction geometry. Use inherit to set the construction geometry attribute only if all input objects are construction geometry.

SEE ALSO

[Offset](#), [Thicken](#)

Torus

Create a solid or surface torus in 3D.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"Torus");  
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);  
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

Use `model.component(<ctag>).geom(<tag>).create(<ftag>,"Torus")` to create a torus. The following properties are available:

TABLE 3-181: VALID PROPERTY/VALUE PAIRS FOR TORUS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angle	double	360	Revolution angle.
axis	double[]	{0,0,1}	Direction of the revolution axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
axistype	x y z cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property. Coloring is only available when selresult is active.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
intfaces	on off	off	Create cross section faces inside the torus.
pos	double[]	{0,0,0}	Center coordinates.
postype	coord vertex	coord	Specify position type as coordinate or vertex.
posvertex	Selection		Vertex for position. Used when postype is vertex.
rmaj	double	1	Directrix radius.
rmin	double	0.5	Generatrix radius.
rot	double	0	Rotational angle about axis.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj dom bnd edg pnt off	dom	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.
type	solid surface	solid	Object type.
workplanesrc	this part-instance feature	this	Part-instance feature to take the work plane from.
workplane	xyplane work-plane feature	xyplane	Work-plane feature that defines the coordinate system. The default, xyplane, is the global Cartesian coordinate system.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#).

The following attributes are available:

TABLE 3-182: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

`model.component(<ctag>).geom(<tag>).create(<ftag>,"torus3")` creates a solid torus.

`model.component(<ctag>).geom(<tag>).create(<ftag>,"torus2")` creates a surface torus.

The following properties are also available:

TABLE 3-183: VALID PROPERTY/VALUE PAIRS FOR TORUS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ax2	double[]	{0,0}	Alias for axis when axistype is spherical.
ax3	double[]	{0,0,1}	Alias for axis when axistype is cartesian.
x, y, z	double	0	Alias for pos.

The property `const` is no longer available.

EXAMPLE

The following sequence generates a surface torus and a solid torus:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("t2","Torus");
g.feature("t2").set("type","surface");
g.feature("t2").set("rmaj",2);
g.feature("t2").set("rmin",1);

g.create("t3","Torus");
g.feature("t3").set("rmaj",10);
g.feature("t3").set("rmin",2);
g.feature("t3").set("pos","0,0,-100");
g.feature("t3").set("axis","1,1,1");
g.feature("t3").set("rot",60);
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1',3);
g.create('t2','Torus');
g.feature('t2').set('type','surface');
g.feature('t2').set('rmaj',2);
g.feature('t2').set('rmin',1);

g.create('t3','Torus');
g.feature('t3').set('rmaj',10);
g.feature('t3').set('rmin',2);
g.feature('t3').set('pos','0,0,-100');
g.feature('t3').set('axis','1,1,1');
g.feature('t3').set('rot',60);
g.run;
```

SEE ALSO

[Cylinder](#)

UnionSelection, IntersectionSelection, DifferenceSelection, ComplementSelection

Combine selections of entities or objects using a Boolean operation.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"UnionSelection");
model.component(<ctag>).geom(<tag>).create(<ftag>,"IntersectionSelection");
model.component(<ctag>).geom(<tag>).create(<ftag>,"DifferenceSelection");
model.component(<ctag>).geom(<tag>).create(<ftag>,"ComplementSelection");
model.component(<ctag>).geom(<tag>).feature().set(property,<value>);
model.component(<ctag>).geom(<tag>).feature().getType(property);
```

DESCRIPTION

Use UnionSelection to get all entities/objects that belong to at least one of the input selections. Use IntersectionSelection to get all entities/objects that belong to all input selections. Use DifferenceSelection to get all entities/objects that belong some of the add selections, but do not belong to any of the subtract selections. Use ComplementSelection to get all entities/objects of the given dimension that do not belong to any input selection.

For DifferenceSelection, the following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
add	String[]	{}	Tags of selections to add.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
entitydim	-1 0 1 2 3	space dimension	Dimension of entities to select. -1 means Object.
selkeep	on off	on	Keep the selection within the geometry sequence.
selshow	on off	on	Show selection in physics, materials, and so on; in part instances; or in 3D from a plane geometry.
subtract	String[]	{}	Tags of selections to subtract.

For the other selections, the following properties are available:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
contributeto	String	none	Tag of cumulative selection to contribute to.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
entitydim	-1 0 1 2 3	space dimension	Dimension of entities to select. -1 means Object.
input	String[]	{}	Tags of input selections.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selkeep	on off	on	Keep the selection within the geometry sequence.
selshow	If the level is not Object, the allowed values are on off. If the level is Object, the allowed values are all obj dom bnd edg pnt off.	If the level is not Object, the default value is on. If the level is Object, the default value is all in a component's geometry, obj in a part, and dom in a work plane's Plane Geometry.	Show selection in physics, materials, and so on; in part instances; or in 3D from a plane geometry. obj is not available in a component's geometry.

See [Selections of Geometric Entities](#) for general information about selections.

EXAMPLE

In an array of blocks, select all vertices whose distance to the origin is between 2.5 and 3.5. This results in 22 vertices in 7 different objects.

Code for Use with Java

```

Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
g.create("blk1", "Block");
g.create("arr1", "Array");
g.feature("arr1").selection("input").set("blk1");
g.feature("arr1").set("size", new int[]{3,3,1});
g.feature("arr1").set("displ", new double[]{1.5,1.5,0});
g.run("arr1");
g.create("ballsel1", "BallSelection");
g.feature("ballsel1").set("entitydim", 0);
g.feature("ballsel1").set("r", 3.5);
g.feature().duplicate("ballsel2", "ballsel1");
g.feature("ballsel2").set("r", 2.5);
g.create("difs1", "DifferenceSelection");
g.feature("difs1").set("entitydim", 0);
g.feature("difs1").set("add", new String[]{"ballsel1"});
g.feature("difs1").set("subtract", new String[]{"ballsel2"});
g.run("difs1");
String[] obj = g.selection("difs1").objects();
int nVtx = 0;
for (int i=0; i<obj.length; ++i)
    nVtx += g.selection("difs1").entities(obj[i],0).length;
// obj.length = 7, nVtx = 22

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
g.create('blk1', 'Block');
g.create('arr1', 'Array');
g.feature('arr1').selection('input').set('blk1');
g.feature('arr1').set('size', [3,3,1]);
g.feature('arr1').set('displ', [1.5,1.5,0]);
g.run('arr1');
g.create('ballsel1', 'BallSelection');
g.feature('ballsel1').set('entitydim', 0);
g.feature('ballsel1').set('r', 3.5);
g.feature.duplicate('ballsel2', 'ballsel1');
g.feature('ballsel2').set('r', 2.5);
g.create('difs1', 'DifferenceSelection');

```

```
g.feature('difsel1').set('entitydim', 0);
g.feature('difsel1').set('add', {'ballsel1'});
g.feature('difsel1').set('subtract', {'ballsel2'});
g.run('difsel1');
obj = g.selection('difsel1').objects;
nVtx = 0;
for i=1:length(obj)
    nVtx = nVtx + length(g.selection('difsel1').entities(obj(i),0));
end
% length(obj) = 7, nVtx = 22
```

SEE ALSO

[AdjacentSelection](#), [BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#), [ExplicitSelection](#)

WorkPlane

Create a work plane in 3D for drawing 2D objects that can be extruded, revolved, or embedded.

SYNTAX

```
model.component(<ctag>).geom(<tag>).create(<ftag>,"WorkPlane");
model.component(<ctag>).geom(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).geom(<tag>).feature(<ftag>).geom().geomSequenceMethod
model.component(<ctag>).geom(<tag>).feature(<ftag>).geom().feature();
model.component(<ctag>).geom(<tag>).feature(<ftag>).setAttribute(attribute,<value>);
model.component(<ctag>).geom(<tag>).feature(<ftag>).getAttribute(attribute);
```

DESCRIPTION

A work plane embeds 2D objects in 3D. The sections below describe how to define the location of the work plane and how to create 2D objects in it. You can also use a work plane in the [CrossSection](#) and [Partition](#) features. In that case, you do not need to draw anything in the work plane.

Unite Objects

There is an option to unite all objects in the work plane before using the 2D geometry in 3D. Uniting all objects can improve the handling of the 2D geometry when extruding it, for example. You can control the union of 2D objects using the following properties:

TABLE 3-184: VALID PROPERTIES, UNITE OBJECTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
absrepairtol	double	See below	Absolute repair tolerance.
repairtol	double	See below	Relative repair tolerance, relative to size of union of inputs.
repairtoltype	auto relative absolute	See below	Repair tolerance type: automatic, relative, or absolute.
unite	on off	off	Unite objects.

The tolerance settings are active when `unite` is set to `on`. The default values for the repair tolerance is taken from the geometry sequence's default repair tolerance.

Visualization

To control the in-plane visualization of the 3D geometry, the visualization of the local coordinate system, and to activate the ability to draw directly on the work plane in 3D, use the following properties:

TABLE 3-185: VALID PROPERTIES, VISUALIZATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
showcoincident	boolean	true	Show coincident 3D geometry.
showintersection	boolean	true	Show intersection of 3D geometry.
showcoordsys	boolean	true	Show local coordinate system.
showprojection	boolean	true	Show projection of 3D geometry.
workplane3d	boolean	true	Draw on work plane in 3D.

Defining the Location of the Work Plane

A work plane has a local coordinate system that is orthonormal and positively oriented (right-handed). The work plane coincides with the *xy*-plane in the local coordinate system. The following properties control how the work plane is defined.

TABLE 3-186: VALID PROPERTIES, LOCATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
planetype	quick faceparallel edgeparallel edgeangle circleperpendicular normal vertices coordinates transformed	quick	Type of data defining the work plane.
contributeto	String	none	Tag of cumulative selection to contribute to.

For information about the `selresult` and `contributeto` properties, see [Selections of Geometric Entities](#). Note that in a work plane's Plane Geometry, the `selresultshow` and `selindividualshow` properties are not available.

Depending on `planetype`, additional properties are available.

Quick

This creates a work plane parallel to one of the global coordinate planes.

TABLE 3-187: VALID PROPERTIES, QUICK.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
quickplane	xy yz zx yx zy xz	xy	Coordinate plane.
quickx	double	0	x-coordinate for work plane (used when plane is yz or zy).
quicky	double	0	y-coordinate for work plane (used when plane is xz or zx).
quickz	double	0	z-coordinate for work plane (used when plane is xy or yx).
quickoffsettype	distance vertex	distance	Type of offset specification.
offsetvertex	Selection		Vertex for offset.
quickorigin	global vertexproj	global	Origin of local coordinate system.
originvertex	Selection		Vertex for origin.
quickaxis	natural vertexproj	natural	Local x-axis.
axisvertex	Selection		Vertex for axis.
displ	double[2]	{0,0}	Displacement of local coordinate system.
rot	double	0	Rotation angle of local coordinate system.

Face Parallel

This creates a work plane that is parallel to a planar face in a geometry object

TABLE 3-188: VALID PROPERTIES, FACE PARALLEL.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
face	Selection		Planar face.
offset	double	0	Signed offset in the direction of the local z-axis.
reverse	on off	off	Reverse direction of local z-axis.
offsettype	distance vertex	distance	Type of offset specification.
offsetvertex	Selection		Vertex for offset.
origin	facecenter boxcorner vertexproj	facecenter	Origin of local coordinate system.
originvertex	Selection		Vertex for origin.
faceparallelaxis	s1 s2 vertexproj	s1	Local x-axis.
axisvertex	Selection		Vertex for axis.
displ	double[2]	{0,0}	Displacement of local coordinate system.
rot	double	0	Rotation angle of local coordinate system.

Edge Parallel

This creates a work plane that is parallel to a planar edge in a geometry object.

TABLE 3-189: VALID PROPERTIES, FACE PARALLEL.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
edge	Selection		Planar edge.
offset	double	0	Signed offset in the direction of the local z-axis.
origin	edgecenter boxcorner	edgecenter	Origin of local coordinate system.
reverse	on off	off	Reverse direction of local z-axis.
offsettype	distance vertex	distance	Type of offset specification.
offsetvertex	Selection		Vertex for offset.
edgeparallelorigin	startvertex endvertex vertexproj	startvertex	Origin of local coordinate system.
originvertex	Selection		Vertex for origin.
edgeparallelaxis	tangent vertexproj	tangent	Local x-axis.
axisvertex	Selection		Vertex for axis.
displ	double[2]	{0,0}	Displacement of local coordinate system.
rot	double	0	Rotation angle of local coordinate system.

Edge Angle

This creates a work plane through a straight edge of a geometry object. The work plane makes a given angle with the tangent plane of a face in the same geometry object. The face must be adjacent to the edge, and its tangent plane must be the same at all points on the edge. The origin of the local coordinate system coincides with the start vertex (if `reverse` is `off`) or end vertex (if `reverse` is `on`) of the edge. The direction of the local *x*-axis coincides with the direction of the edge (if `reverse` is `off`) or its opposite (if `reverse` is `on`). If the property `angle` is zero,

the direction of the local y -axis points into the face. In general, the local coordinate system is rotated by `angle` about the local x -axis.

TABLE 3-190: VALID PROPERTIES, EDGE ANGLE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>angle</code>	double	0	Angle between face and work plane.
<code>edge</code>	Selection		Straight edge.
<code>adjface</code>	Selection		Face adjacent to edge in the same object.
<code>reverse</code>	on off	off	Reverse direction of local x -axis.
<code>displ</code>	double[2]	{0,0}	Displacement of local coordinate system.
<code>rot</code>	double	0	Rotation angle of local coordinate system.

Circle Perpendicular

This creates a work plane that is perpendicular to a given circular edge. The origin of the local coordinate system is at the circle's center. By default, the local x -axis goes through the edge's start vertex. Thus, if the geometry is rotationally symmetric, the symmetry axis coincides with the local y -axis.

TABLE 3-191: VALID PROPERTIES, CIRCLE PERPENDICULAR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>circedge</code>	Selection	empty	Circular edge.
<code>circpoint</code>	startvertex endvertex othervertex	startvertex	Point on plane.
<code>circvertex</code>	Selection		Vertex on plane.
<code>circoffset</code>	double	0	Offset angle.
<code>reverse</code>	on off	off	Reverse direction of local x -axis.
<code>displ</code>	double[2]	{0,0}	Displacement of local coordinate system.
<code>rot</code>	double	0	Rotation angle of local coordinate system.

Normal Vector

This creates a work plane defined by a normal vector and a point on the plane. Together with a [RigidTransform](#) feature, this makes it possible to orient an object so that one of its planar faces gets a prescribed normal vector.

TABLE 3-192: VALID PROPERTIES, CIRCLE PERPENDICULAR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>normalvector</code>	double[3]	{0,0,1}	Normal vector.
<code>normalpoint</code>	coord vertex	coord	Point on plane (defined using coordinates or as a geometry vertex).
<code>normalcoord</code>	double[3]	{0,0,1}	Coordinates of point, used when <code>normalpoint</code> is set to <code>coord</code> .
<code>normalvertex</code>	Vertex selection		Vertex for point, used when <code>normalpoint</code> is set to <code>vertex</code> .
<code>displ</code>	double[2]	{0,0}	Displacements xw and yw .
<code>rot</code>	double	0	Rotation angle.

Vertices

This creates a work plane parallel to a plane through three vertices `v1`, `v2`, and `v3`. When `offset=0`, the origin of the local coordinate system coincides with the first vertex `v1`. The x -axis of the local coordinate system is in the

direction $v_2 - v_1$. The direction of the local z -axis is given by the cross product $(v_2 - v_1) \times (v_3 - v_1)$ or its opposite (if `reverse` is on).

TABLE 3-193: VALID PROPERTIES, VERTICES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>offset</code>	double	0	Signed offset in the direction of the local z -axis.
<code>reverse</code>	on off	off	Reverse direction of local z -axis.
<code>vertex1</code>	Selection		First vertex.
<code>vertex2</code>	Selection		Second vertex.
<code>vertex3</code>	Selection		Third vertex.
<code>displ</code>	double[2]	{0,0}	Displacement of local coordinate system.
<code>rot</code>	double	0	Rotation angle of local coordinate system.

Coordinates

This creates a work plane through three points `p1`, `p2`, and `p3`. The origin of the local coordinate system coincides with the first point `p1`. The x -axis of the local coordinate system is in the direction `p2 - p1`. The direction of the local z -axis is given by the cross product $(p_2 - p_1) \times (p_3 - p_1)$.

TABLE 3-194: VALID PROPERTY, COORDINATES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>genpoints</code>	double[3][3]	{{0,0,0},{1,0,0},{0,1,0}}	Points.

`genpoints[n][i]` is the i th coordinate of the n th point.

Transformed

This creates a work plane as a transformation of another work plane, using a displacement and a rotation.

TABLE 3-195: VALID PROPERTIES, TRANSFORMED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>transax2</code>	double[2]	{0,0}	Spherical angles θ and ϕ , used if <code>transaxistype</code> is spherical.
<code>transax3</code>	double[3]	{0,0,1}	Axis vector, used if <code>transaxistype</code> is cartesian.
<code>transaxis</code>	double[]	{0,0,1}	Rotation axis. Vector has length 3 if <code>transaxistype</code> is cartesian, and length 2 if <code>transaxistype</code> is spherical. Alias for <code>transax2</code> and <code>transax3</code> .
<code>transaxistype</code>	x y z cartesian spherical	z	Coordinate system used for axis. Used if <code>transspecify</code> is set to axis.
<code>transdispl</code>	double[3]	{0,0,0}	Displacement of local coordinate system.
<code>transedge</code>	Selection		Edge selection. Used when <code>transspecify</code> is set to edge.
<code>transeulerang</code>	double[3]	{0,0,0}	Intrinsic Z-X-Z Euler angles α , β , and γ , used if <code>transspecify</code> is set to <code>eulerang</code> .
<code>transrot</code>	double	0	Rotation angle, if <code>transspecify</code> is not set to <code>eulerang</code> .
<code>transspecify</code>	axis eulerang edge	axis	Specify an axis of rotation, Euler angles (Z-X-Z), or a straight edge.
<code>workplanesrc</code>	String	this	Tag of PartInstance feature to take work plane from, or this to take work plane from this sequence.
<code>workplane</code>	String	xyplane	Tag of input work plane, or xyplane.

Part Instances

In a geometry part, the following property is available:

TABLE 3-196: VALID PROPERTY IN A FEATURE IN A GEOMETRY PART.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
showworkplane	on off	on	Show work plane in part instances. This property is only available if the work plane is in a geometry part.

Selections of Resulting Entities

For selections of resulting entities, the following properties are available:

TABLE 3-197: VALID PROPERTIES FOR SELECTIONS OF RESULTING ENTITIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selplaneshow	on off	off	Show selections from Plane Geometry in physics or part instances.
selresult	on off	off	Create selections of all resulting objects.
selresultshow	all obj bnd edg pnt off	bnd	Show selections, if selresult is on, of resulting objects in physics, materials, and so on, or in part instances. obj is not available in a component's geometry. dom, bnd, and edg are not available in all features.

Creating 2D Objects in the Work Plane

The work plane owns a geometry sequence that contains the features that define the 2D objects you draw in the work plane. You access this geometry sequence by

```
model.component(<ctag>).geom(<tag>).feature(<ftag>).geom()
```

where <ftag> is the name of the work-plane feature. You can add geometry features in this 2D sequence as usual.

Attributes

The following attributes are available:

TABLE 3-198: VALID ATTRIBUTES

NAME	VALUE	DEFAULT	DESCRIPTION
construction	on off	off	Designate the resulting objects as construction geometry.

COMPATIBILITY

The plane type `circularedge` from earlier versions is still valid as an alternative to its replacement `circleperpendicular`, and the plane type `general` from earlier versions is still valid as an alternative to its replacement `coordinates`.

EXAMPLE

Create a work plane with a rectangle. When the work plane is built, the rectangle is embedded in the space of the 3D sequence:

Code for Use with Java

```
Model model = ModelUtil.create("Model1");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1",3);
g.create("wp1","WorkPlane");
g.feature("wp1").set("quickplane","yz");
g.feature("wp1").geom().create("r1","Rectangle");
g.feature("wp1").geom().feature("r1").set("pos","1 1");
g.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model1');
```

```
model.component.create('comp1');  
g = model.component('comp1').geom.create('geom1',3);  
g.create('wp1','WorkPlane');  
g.feature('wp1').set('quickplane','yz');  
g.feature('wp1').geom.create('r1','Rectangle');  
g.feature('wp1').geom.feature('r1').set('pos','1 1');  
g.run;
```

SEE ALSO

[CrossSection](#), [Extrude](#), [Partition](#), [Revolve](#), [Sweep](#)

Mesh

Details include reference information about the mesh commands and utility methods.

In this chapter:

- [About Mesh Commands](#)
- [Working with a Meshing Sequence](#)
- [Mesh Settings](#)
- [Physics-Controlled Meshing](#)
- [Adaptively Refined Meshes](#)
- [Information and Statistics](#)
- [Getting and Setting Mesh Data](#)
- [Errors, Warnings, and Information](#)
- [Exporting Meshes to Files](#)
- [Mesh Commands](#)

About Mesh Commands

Mesher can be either

- conforming with a geometry, or
- define their own geometric model.

The first category involves models that have a geometry sequence with either imported CAD or created in the software. One or several meshes are then generated based on this geometry. Mesher that define their own geometric model typically involve imported meshes or operations that edit a mesh. If such a mesh defines the component's geometric model, the physics can then be defined on the geometric entities of the mesh. For more information, see [Geometric Model](#).

[Table 4-1](#) is an overview of all available mesh operations. The two rightmost columns give information about if an operation is supported for meshes conforming with geometry and meshes defining their own geometric model.

TABLE 4-1: AVAILABLE MESH OPERATIONS.

FEATURE TYPE	DESCRIPTION	DEFAULT TAG IN GUI	MESH CONFORMING WITH GEOMETRY	MESH DEFINING ITS OWN GEOMETRIC MODEL
Adapt	Adaptive mesh modification	ada	√	√
Ball	Partition geometric entities of the mesh by an element set defined by a ball	ball	√	√
BndLayer	Generate boundary layer mesh	bl	√	√
Box	Partition geometric entities of the mesh by an element set defined by a box	box		√
CollapseEntities	Collapse geometric entities of the mesh	col		√
Convert	Convert nonsimplex elements in the mesh to simplex elements	conv	√	√
Copy	Copy a mesh from a meshing sequence	copy	√	
CopyEdge	Copy mesh between edges	cpe	√	
CopyFace	Copy mesh between faces	cpf	√	
CopyDomain	Copy mesh between domains	cpd	√	
CreateDomains	Create one domain for each finite void region defined by the mesh	dom		√
CreateEdges	Create new edges in the mesh	edg		√
CreateFaces	Create new faces in the mesh	fac		√
CreateVertices	Create new geometric vertices in the mesh	vtx		√
Cylinder	Partition geometric entities of the mesh by an element set defined by a cylinder	cyl		√
DeleteEntities	Delete geometric entities from the mesh	dele		√
DetectFaces	Partition faces of the mesh by searching for shapes that are likely to constitute separate faces	detf		√
Edge	Generate mesh for edges	edg	√	
FillHoles	Repair holes in faces of the mesh	fill		√
FreeTet	Generate unstructured tetrahedral mesh	ftet	√	√
FreeTri	Generate unstructured triangular mesh	ftri	√	√

TABLE 4-1: AVAILABLE MESH OPERATIONS.

FEATURE TYPE	DESCRIPTION	DEFAULT TAG IN GUI	MESH CONFORMING WITH GEOMETRY	MESH DEFINING ITS OWN GEOMETRIC MODEL
FreeQuad	Generate unstructured quadrilateral mesh	fq	√	√
Import	Import mesh from file or other source	imp		√
IntersectLine	Intersect the mesh with a line (2D)	lin		√
IntersectPlane	Intersect the mesh with a plane (3D)	pln		√
JoinEntities	Join adjacent geometric entities of the mesh	join		√
LogicalExpression	Partition geometric entities of the mesh by an element set defined by a logical expression	le		√
Map	Generate structured quadrilateral mesh	map	√	√
MergeEntities	Merge geometric entities in the mesh	mrg		√
Vertex	Generate a point mesh	vtx	√	
Reference	Refer to another meshing sequence	rf	√	
Refine	Refine the mesh	ref	√	√
Sweep	Generate swept mesh	swe	√	√
Union	Unite geometric entities by intersecting mesh elements and merging overlapping elements	uni		√

Table 4-2 contains an overview of mesh attributes to control the mesh element sizing, layer distributions, scaling, and position of an imported mesh. The two rightmost columns give information about if an attribute is supported for meshes conforming with geometry and meshes defining their own geometric model.

TABLE 4-2: AVAILABLE MESH ATTRIBUTES.

FEATURE TYPE	DESCRIPTION	DEFAULT TAG IN GUI	MESH CONFORMING WITH GEOMETRY	MESH DEFINING ITS OWN GEOMETRIC MODEL
BndLayerProp	Boundary Layer Properties	blp	√	√
CornerRefinement	Corner Refinement	cr	√	
Distribution	Distribution	dis	√	√
EdgeGroup	Edge Groups	eg	√	
EdgeMap	Edge Map	em	√	
OnePointMap	One-Point Map	pm	√	
Scale	Scale	sca	√	
Size	Size	size	√	√
SizeExpression	Size Expression	se	√	
TwoPointMap	Two-Point Map	ppm	√	
Transform	Position, rotate, or scale a mesh.	tr		√

Working with a Meshing Sequence

This section describes how to build meshes using Java[®] methods. A *mesh* is defined by a *meshing sequence* consisting of *mesh features*. A meshing feature is either an *attribute feature* or an *operation feature*. Each operation feature modifies the mesh when you *build* the feature using properties defined by attribute features.

An attribute feature is defined on a geometric entity selection and has a set of properties. Running an attribute feature does not change the mesh, but affects the subsequent operation features in the sequence. For example, the **FreeTet** operation feature, that creates a tetrahedral mesh, uses properties from the **Distribution**, **Scale**, and **Size** attribute features, that define the size and distribution of the mesh elements. You can add an attribute feature directly to the meshing sequence, this is referred to as a *global attribute feature*, or add it to an operation feature, this is referred to as a *local attribute feature*. Properties defined in local attribute features of an operation feature overrides corresponding properties defined in preceding global feature properties (on the same selection).

An operation features makes operations on the mesh as defined by the meshing sequence. Some operation features, like **FreeTet** and **Sweep** *generate* new mesh. Other operation features, like **Refine** and **Convert** *modify* existing mesh.

In this section:

- [Adding a Meshing Sequence](#)
- [Adding a Mesh Feature](#)
- [Editing a Mesh Feature](#)
- [Building Mesh Features](#)
- [Using Mesh Parts](#)
- [Using Mesh-Based Geometries](#)
- [Feature Status](#)
- [Deleting Mesh Features](#)
- [Disabling Mesh Features](#)
- [Clearing Meshes](#)
- [Locking Meshes](#)
- [Units](#)
- [Selections](#)

	Meshing in the COMSOL Multiphysics Reference Manual
	model.mesh() — Information about the main <code>model.mesh</code> command.

Adding a Meshing Sequence

To add a new meshing sequence to a model object `model`, enter

```
model.component(<ctag>).mesh().create(<mtag>,<gtag>);
```

where `mTag` is the mesh's tag (an identifier of your choice) and `gTag` is the tag of the associated geometry. If you want to import a mesh, you must specify an empty geometry sequence; the geometry is then defined by the mesh.



The syntax that includes the component level, such as `model.component(<ctag>).mesh()....` is the default and is used throughout this chapter. To use the earlier `model.mesh()....` syntax, clear the **Generate code using component syntax** checkbox on the **Application Builder > Methods** page in the **Preferences** window.

Adding a Mesh Feature

To add a feature to a mesh with tag `<tag>`, enter

```
model.component(<ctag>).mesh(<tag>).create(<ftag>, ftype);
```

where `<ftag>` is the feature's tag (an identifier of your choice), and `ftype` is the feature's type. Feature types are capitalized and case-sensitive (for example, `FreeTet`).

When you add a feature, it is inserted after the *current feature*. You can get the tag of the current feature type by entering

```
String ftag = model.component(<ctag>).mesh(<tag>).current();
```

If `ftag` is the empty string, the current feature is the beginning of the meshing sequence, that is, the empty state before all features. Adding a meshing feature, it automatically becomes current, but it is not built automatically.

For some operation features it is possible to add attribute features. To add an attribute feature to an operation feature, enter

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>, ftype);
```

where `<ftag1>` is the attribute feature's tag (an identifier of your choice), and `ftype` is the attribute feature's type.

All properties in a new feature get a default value.

Editing a Mesh Feature

To change a property value in a feature, enter

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property, <value>);
```

where `property` is a property name and `<value>` is a property value.

All numeric properties can be given either as a numeric value or as a string expression that can contain parameters defined in `model.param()`. When building the feature, the string expressions are evaluated using the current values of the parameters.

To get the value of a property, enter one of the following, depending on the property type:

```
double d = model.component(<ctag>).mesh(<tag>).feature(<ftag>).getDouble(property);
String s = model.component(<ctag>).mesh(<tag>).feature(<ftag>).getString(property);
double[] da = model.component(<ctag>).mesh(<tag>).feature(<ftag>).
    getDoubleArray(property);
String[] sa = model.component(<ctag>).mesh(<tag>).feature(<ftag>).
    getStringArray(property);
```

If you request a numerical value for a string property, it is evaluated using the current values of the parameters in `model.param()`.

Building Mesh Features

To modify the mesh, you must *build* an operation feature. Enter

```
model.component(<ctag>).mesh(<tag>).run(<ftag>);
```

to build the feature *<ftag>* and all its preceding features (the features are built in the order from the first to the last). When the build has completed, the feature *<ftag>* becomes the current feature.

To build all features, enter

```
model.component(<ctag>).mesh(<tag>).run();
```

Use the method

```
model.component(<ctag>).mesh(<tag>).runForStudy(<studyStep>);
```

to build the mesh using settings for the specified study to generate a physics-controlled mesh (if the mesh is physics-controlled). *<studyStep>* is the path of tags to the study feature, for example *std/freq*. If the mesh is user-controlled, the method builds all mesh features.

Using Mesh Parts

For a description of mesh parts, see [Mesh Parts](#) in the *COMSOL Multiphysics Reference Manual*.

A mesh part is defined by a model component of `MeshComponent` type (a *mesh component*). A mesh component contains a single meshing sequence and a geometry that defines the dimension, selections, and geometrical properties such as the length and angular unit. When you work with a Mesh Part node in the user interface, you can modify the meshing sequence of the mesh component. When working with the API it is important to be aware of the corresponding mesh component and its geometry.

To create a mesh part, enter

```
model.modelNode().create(<mcomptag>, "MeshComponent");  
model.geom().create(<mgeomtag>, sDim);  
model.mesh().create(<tag>, <mgeomtag>);
```

where *<mcomptag>* is the tag of the mesh component, *<mgeomtag>* is the tag of the geometry, *sDim* is its space dimension (1, 2, or 3), and *<tag>* is the mesh part's tag.

Use `model.component(<ctag>).geom(<mgeomtag>)` to access the geometry properties, such as length units (see `model.geom(<tag>)`).

To work with the mesh part, use `model.mesh(<tag>)`, see the sections above in [Working with a Meshing Sequence](#) and [Table 4-1](#), the column with header Mesh Defining its Own Geometric Model, for details.

For using a mesh part in a model component, see [Import Mesh Part or Meshing Sequence](#) under Geometry Commands to create a geometry from the part, or [Import](#) under Mesh Commands to use the resulting mesh as an imported mesh.

To remove a mesh part, use `model.modelNode().remove(<mcomptag>)`;

Using Mesh-Based Geometries

A mesh-based geometry is a geometric model defined by a mesh, in contrast to a standard geometry that is defined through operations in a geometry sequence. The mesh in a mesh-based geometry cannot be used for computations but must be meshed using a meshing sequence that operates on the mesh-based geometry. For more information, see [Mesh-Based Geometry](#). See also [Geometric Model](#) and [Geometry Cleanup](#).

To create a meshing sequence or a mesh-based geometry sequence, use

```
model.mesh().create(<mtag>,<type>,<gttag>);
```

where *<mtag>* is the mesh's tag (an identifier of your choice) and *<gttag>* is the tag of the associated geometry. Set *<type>* to **Sequence** to create an ordinary meshing sequence and set it to **Geometry** to create a mesh-based geometry sequence.

Use

```
model.mesh(<mtag>).isGeometry();
```

to check if the meshing sequence *<mtag>* is a mesh-based geometry.

Feature Status

The *status* of a feature can be one of the following:

- *Built*. This means that none of the feature's properties have changed since the feature was last built, and the features of the input objects are all built. The feature can contain warning messages.
- *Edited*. This means that some of the feature's properties have changed since the feature was last built.
- *Needs rebuild*. This means that any of the preceding features is edited.
- *Error*. This means that the feature contains an error message.
- *Warning*. This means that the feature contains a warning message.

You can examine the status of a feature by entering

```
boolean built = model.component(<ctag>).mesh(<tag>).feature(<ftag>).isBuilt();  
boolean edited = model.component(<ctag>).mesh(<tag>).feature(<ftag>).isEdited();  
boolean hasError = model.component(<ctag>).mesh(<tag>).feature(<ftag>).hasError();  
boolean hasWarning = model.component(<ctag>).mesh(<tag>).feature(<ftag>).hasWarning();  
boolean needsRebuild = !(built || edited || hasError || hasWarning);
```

Deleting Mesh Features

To delete a feature, enter

```
model.component(<ctag>).mesh(<tag>).feature().remove(<ftag>);
```

Disabling Mesh Features

To disable a feature, enter

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).active(false);
```

To enable a disabled feature, enter

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).active(true);
```

You can get the enabled/disabled status of a feature by entering

```
boolean isEnabled = model.component(<ctag>).mesh(<tag>).feature(<ftag>).active();
```

Clearing Meshes

To clear the built mesh of a sequence while keeping all features and settings in the sequence, enter

```
model.component(<ctag>).mesh(<tag>).clearMesh();
```

To clear the built mesh and remove all features in a sequence, enter

```
model.component(<ctag>).mesh(<tag>).feature().clear();
```

To clear all built meshes in a component while keeping all features and settings in the sequences, enter

```
model.component(<ctag>).mesh().clearMeshes();
```

To clear all built meshes in a model while keeping all features and settings in the sequences, enter

```
model.mesh().clearMeshes();
```

Locking Meshes

Lock a mesh to make it independent of any other data in the model. Locking a mesh is useful if you, for example, want to keep an adapted mesh but need to clear the solution data that generated the adapted mesh. To lock a mesh, enter

```
model.component(<ctag>).mesh(<tag>).lockMesh();
```

You can examine the status of a mesh by entering

```
boolean locked = model.component(<ctag>).mesh(<tag>).isLocked();
```

Use the command

```
model.component(<ctag>).mesh(<tag>).geometricModel("");
```

to disconnect the mesh from the geometry. See the section [Geometric Model](#) for more information.

Units

The meshing sequence uses the same base unit system as the geometry sequence. The string versions of setters and getters support units and unit conversion using the standard machinery.

Selections

Most mesh features have entity selections, to specify where they operate. To access a feature's entity selection, use the syntax

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
```

To specify the entire geometry, write

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().allGeom();
```

To specify all geometric entities in dimension *<dim>*, write

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().geom(<dim>).all();
```

To specify the geometric entities that remains to be meshed when the feature is about to be built, use

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().remaining();
```

It is not possible to retrieve the geometric entities of this selection, unless the feature is built.

If *entities* is an integer array of geometric entities in dimension *dim*, use the following syntax to select these entities

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().geom(dim).set(entities);
```

For example, to selection domain 1 and 2 in a 3D geometry, write

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().geom(3).  
set(new int[]{1,2});
```

To add the geometric entities specified in the integer array *<entities>* in dimension *<dim>* to the selection, write


```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().geom(<dim>).
    add(<entities>);
```

To remove the geometric entities specified in the integer array *<entities>* in dimension *<dim>* from the selection, write

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().geom(<dim>).
    remove(<entities>);
```

To clear the selection in dimension *<dim>*, write

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().geom(<dim>).clear();
```

Some features have more than one selection, for example *sweep*, where it is possible to specify source and destination faces. Use the following syntax to access these selections.

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(<property>);
```

Thus, to specify boundary 5 as source face on the sweep feature *swe1*, write

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection("sourceface").
    geom(2).set(5);
```

NAMED SELECTIONS

To specify a named selection, *<stag>*, for the selection of the feature *<ftag>*, use

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection().named(<stag>);
```

For meshes that are coupled to a geometry, you can use selections defined under definitions and selections defined by the geometry. See [model.selection\(\)](#) and [Named Selections](#) for more information about selections defined under definitions and by the geometry, respectively.

For meshes that define their own geometric model, use the following methods to get hold of the tags *<stag>* for named selections. For more information, see [Geometric Model](#). Both methods return empty string arrays if there are no named selections. Selections defined under Definitions cannot be used.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).outputSelection()` returns the tags of named selections (selection output objects) that are defined.

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).componentSelection()` returns tags of the selection in `model.selection()` that are derived from the selections in the feature. If the sequence does not define the geometrical model of the component, `componentSelection()` returns an empty result. For more information about using named selections in the component, see the section [Selections](#) in the [General Commands](#) chapter.

Mesh Settings

This section describes general settings that relates to a meshing sequence. In this section:

- [Automatic Rebuild](#)
- [Length and Angular Units](#)
- [Geometric Model](#)

Automatic Rebuild

The `autoRebuild` setting is a GUI setting which controls if the mesh sequence is automatically rebuilt when clicking a node in the model tree outside the mesh sequence where the mesh is used. You can change it by entering:

```
model.component(<ctag>).mesh(<tag>).autoRebuild(<newAutoRebuild>);
```

where `<newAutoRebuild>` is `on` or `off`. The setting will only be available in the GUI and have effect if the mesh `<tag>` defines its own geometric model and the physics is defined on this geometric model. To control on which geometric model to define the physics, use:

```
model.component(<ctag>).definedByMesh(<tag>);
```

where `<tag>` corresponds to the same mesh as for `autoRebuild`.

Length and Angular Units

The length and angular units are determined by the corresponding geometry properties. See the sections [Length Unit](#) and [Angular Unit](#) for more information.

Geometric Model

A meshing sequence that defines its own geometric model defines its own set of entities and is not connected to a geometry sequence. The most common example is a meshing sequence including an [Import](#) feature. The meshing features listed in the [See Also](#) section can only be added to meshing sequences that define their own geometric model.

	For more information about meshes that define their own geometric model, see The Mesh Node in the <i>COMSOL Multiphysics Reference Manual</i> .
	To get information, such as number of entities, adjacency, and more on the geometric model, see Information on the Geometric Model .

To get the geometric model used in the meshing sequence `<tag>`, use

```
String geomModel = model.component(<ctag>).mesh(<tag>).geometricModel();
```

The method returns a string with the tag of the geometry or meshing sequence with the prefix `geom/` or `mesh/` to indicate if it is a geometry or mesh. It returns an empty string if the sequence defines its own geometric model.

Enter

```
model.component(<ctag>).mesh(<tag>).geometricModel(String);
```

to specify if the meshing sequence *<tag>* defines its own geometric model, is operating on a geometry, or is operating on a mesh. For example, use

```
model.component("comp1").mesh("mesh1").geometricModel("geom/geom1");
```

to specify that mesh1 is operating on the geometric model defined by geom1. Use

```
model.component("comp1").mesh("mesh2").geometricModel("");
```

to specify that mesh2 is defining its own geometric model. Use

```
model.component("comp1").mesh("mesh3").geometricModel("mesh/mesh1");
```

to specify that mesh3 is operating on the geometric model of mesh1. See also [Using Mesh-Based Geometries](#).

The physics is defined on the component's geometric model. For more information, see [model.component\(\)](#).

SEE ALSO

[Import](#), [Ball](#), [Box](#), [CreateDomains](#), [CreateEdges](#), [CreateFaces](#), [CreateVertices](#), [Cylinder](#), [DeleteEntities](#), [DetectFaces](#), [FillHoles](#), [IntersectLine](#), [IntersectPlane](#), [JoinEntities](#), [LogicalExpression](#), [Union](#)

Physics-Controlled Meshing



When a physics-controlled sequence is built, a sequence of ordinary meshing features is created. This sequence can be customized by editing these features. However, do not assume the existence of a certain feature in a Java program designed to run with future versions of COMSOL Multiphysics. The actual contents of the sequence might change.

A physics-controlled meshing sequence examines the physics to automatically determine size attributes and sequence operations needed to create a mesh adapted to the geometry and the problem. The physics-controlled sequence is based on heuristics and knowledge built-in by mesh and application experts. It is not adapted by numerical error estimates — that type of adaptation is provided by mesh adaptation in the solver sequence.

When a mesh is built or a problem solved, the physics-controlled sequence is updated to match the currently active physics. If the sequence is in any other state than physics-controlled, it is not updated or modified before it is built.

By default, a meshing sequence is in the physics-controlled state. If you manually add a feature to the sequence or edit a feature, the sequence automatically switches to the user-controlled state. It is also possible to explicitly switch to user-controlled state by entering

```
model.component(<ctag>).mesh(<tag>).automatic(false);
```

To switch back to physics-controlled mesh, enter

```
model.component(<ctag>).mesh(<tag>).automatic(true);
```

The current sequence is then modified or overwritten next time the sequence is built or the problem is solved.

Use `model.component(<ctag>).mesh(<tag>).isAutomatic()` to determine in which state the sequence is.

You can adjust the overall size of a physics-induced mesh by using the method

```
model.component(<ctag>).mesh(<tag>).autoMeshSize(<size>);
```

The value 5 of `<size>` corresponds to the default size, the values 4, 3, 2, and 1 give you an increasingly finer mesh, whereas the values 6, 7, 8, and 9 give you a coarser mesh. You can use any nonnegative value for the predefined mesh size. Use `Double.POSITIVE_INFINITY` to create the coarsest possible mesh. The method `model.component(<ctag>).mesh(<tag>).autoMeshSize()` returns the current size adjustment.

Geometric Analysis, Detail Size Contribution

Use the method below to control if the geometric analysis will contribute to the physics-controlled mesh.

```
model.component(<ctag>).mesh(<tag>).contribute("geom/detail",<contribute>);
```

See also the section [Selecting Contributing Physics](#) for more methods to use.

Selecting Contributing Physics

You can control which physics interface that contributes to the physics-controlled mesh using

```
model.component(<ctag>).mesh(<tag>).contribute(<interface>,<contribute>);
```

The argument `<contribute>` (true or false) value is used to specify if the interface contributes or not.

The interface can be a physics interface, a multiphysics coupling, or a common feature, specified according to the following table:

TABLE 4-3: INTERFACE TYPES.

TYPE	INTERFACE SPECIFICATION	EXAMPLE
Physics interface	<code><tag></code> or <code>physics/<tag></code>	ht
Multiphysics coupling	<code>multiphysics/<tag></code>	multiphysics/emb1
Common feature	<code>common/<tag></code>	common/free1

Not all common features can contribute to the physics-controlled mesh.

To check if an interface is currently contributing to the physics controlled mesh, use

```
model.component(<ctag>).mesh(<tag>).contributing(<interface>);
```

An interface is contributing by default, when added. To get the tags of all available contributors, use

```
model.component(<ctag>).mesh(<tag>).contributors();
```

The resulting string array will also contain the tag `geom/detail`. See [Geometric Analysis, Detail Size Contribution](#) for more information.

The following code shows some examples of how to use these methods, assuming there is a Heat Transfer interface tagged `ht` and an Electromagnetic Heating multiphysics coupling tagged `emh1`:

```
String[] contrTags = model.component(<ctag>).mesh("mesh1").contributors();

model.component(<ctag>).mesh("mesh1").contribute("ht", true);
model.component(<ctag>).mesh("mesh1").contribute("multiphysics/emh1", false);

boolean isActivated = model.component(<ctag>).mesh("mesh1").contributing("ht");
boolean physicsActive = model.component(<ctag>).physics("ht").isActive();
```

Where `contrTags` will be `{"physics/ht", "multiphysics/emh1"}` and `isActivated` will have the value `true`. If `physicsActive` is `true`, then the Heat Transfer interface contributes to the physics-controlled mesh, otherwise not.

Adaptively Refined Meshes

A meshing sequence can represent an adaptively refined mesh controlled by an [Adaption](#) study feature. The following method is available to return the tag of the study and study feature with adaptation that controls the meshing sequence:

```
model.component(<ctag>).mesh(<tag>).adaptationStudyFeature();
```

If no such study exists, this method returns an empty string.

To specify the study feature with adaptation that controls the meshing sequence, use

```
model.component(<ctag>).mesh(<tag>).adaptationStudyFeature(<studytag>)
```

where *<studytag>* is a path of tags to the Adaptation study feature. Use an empty string to disable study control.

Information and Statistics

In this section:

- [Statistics](#)
- [Number and Types of Elements](#)
- [Quality of Elements](#)
- [Volume of Elements and Mesh](#)
- [Mesh Status](#)
- [Mesh Plot](#)
- [Measurements](#)
- [Information on the Geometric Model](#)

Statistics

Use the `stat()` method on the meshing sequence to determine the number of elements of different types and the quality of elements. For information about number of entities, see [Information on the Geometric Model](#).

The `stat()` method returns an object with a collection of methods that can be queried for statistical information about the current mesh. There is also a selection,

```
model.component(<ctag>).mesh(<tag>).stat().selection()
```

which is used to select geometric entities for which the statistics is calculated. The default selection is the entire geometry. The methods described below also exist directly on the meshing sequence. These methods always return statistics for the entire geometry.

Statistics can be requested per element type. The type is given as a string, denoted *type*, and the possible types are listed in the following table.

TABLE 4-4: ELEMENT TYPES.

STRING	ELEMENT	ELEMENT DIMENSION
vtx	Vertex element	0
edg	Edge element	1
tri	Triangular element	2
quad	Quadrilateral element	2
tet	Tetrahedral element	3
pyr	Pyramid element	3
prism	Prism element	3
hex	Hexahedral element	3
all	All elements of maximal dimension in the selection	

The parameter string `all` gives statistics for all elements with the same dimension as the maximal dimension of the current selection. For example, if the entire geometry is selected in 2D, the parameter `all` provides combined statistics for triangular and quadrilateral elements.

To specify the mesh quality measure to use, use the `setQualityMeasure` method. For example,

```
model.component(<ctag>).mesh(<tag>).stat().setQualityMeasure("maxangle")
```

To get the current mesh quality measure, use the `getQualityMeasure` method:

```
String model.component(<ctag>).mesh(<tag>).stat().getQualityMeasure()
```

The following mesh quality measures are available:

TABLE 4-5: MESH QUALITY MEASURES.

NAME	DESCRIPTION
skewness	Skewness. This measure is based on a calculation of the mesh elements' equiangular skew.
maxangle	Maximum angle. This measure is based on the largest angle in the element.
volcircum	Volume versus circumradius. This measure is based on a quotient of the element volume and the radius of the circumscribed sphere (or circle) of the element.
vollength	Volume versus length. This measure is based on a quotient of element edge lengths and element volume.
condition	Condition number. This measure is based on the element dimension divided by the condition number (in the Frobenius norm) of the matrix transforming the element to a reference element.
growth	Neighbor growth rate. This measure is based on a mesh element growth rate calculation.
curvedskewness	Curved skewness. This measure is defined as the elementwise product of skewness and <code>reldetjacmin</code> , which is a measure of the deformation when generating the higher-order element.
custom	A custom mesh quality expression.

	• Geometry Object Information • Selections
---	---

Number and Types of Elements

To determine the number of elements of a certain type, use

```
int model.component(<ctag>).mesh(<tag>).stat().getNumElem(type);
```

To get the number of elements of all types, use

```
int model.component(<ctag>).mesh(<tag>).stat().getNumElem();
```

To determine the element types, use

```
String[] model.component(<ctag>).mesh(<tag>).stat().getTypes();
```

If the current selection is not the entire geometry, only elements and types in the current selection is returned. You can also use the methods

```
int model.component(<ctag>).mesh(<tag>).getNumElem(String type);
int model.component(<ctag>).mesh(<tag>).getNumElem();
int[] model.component(<ctag>).mesh(<tag>).getGeomEntities(String type);
String[] model.component(<ctag>).mesh(<tag>).getTypes();
```

to obtain information about the entire mesh.

To check whether the mesh contains any second-order element, use

```
boolean model.component(<ctag>).mesh(<tag>).hasSecondOrderElements();
```

Quality of Elements

COMSOL Multiphysics includes several different mesh quality measures. The absolute value of the mesh element quality is always between 0 and 1, where 0.0 represents a degenerated element and 1.0 represents the best possible element. A negative value means a contradiction to the COMSOL Multiphysics numbering convention for mesh

element vertices (see [Mesh Element Numbering Conventions](#)), and the element is then referred to as an *inverted element*. The following mesh quality measures are available:

- The *skewness* (`skewness`), which is based on the mesh elements' equiangular skew.
- The *maximum angle* (`maxangle`), which is based on the largest angle in the element. If no angle is larger than the largest angle of the corresponding optimal element, the quality is one; otherwise, the measure shows how much larger the angle is. This quality measure is insensitive to element anisotropy.
- The *volume versus circumradius* (`volcircum`), which is the default quality measure, is based on the ratios of the inscribed and circumscribed circles' or spheres' radii for the simplex corresponding to each corner of the element. If the simplex cannot be clearly determined (an apex of the pyramid, for example), the corresponding corner is excluded from the consideration.
- The *volume versus length* (`vollength`), which is based on a quotient of element edge lengths and element volume. This quality measure is primarily sensitive to anisotropy.
- The *condition number* (`condition`), which is based on the element dimension divided by the condition number (in the Frobenius norm) of the matrix transforming the element to a reference element.
- The *growth rate* (`growth`), which is based on the mesh elements' local (anisotropic) growth rate.
- The *curved skewness* (`curvedskewness`), which is defined as the elementwise product of skewness and `reldetjacmin`, which is a measure of the deformation when generating the higher-order element.

There is also a *custom* quality measure (`custom`), which is based on a user-defined expression for the mesh element quality.

To retrieve the minimal quality, use

```
double model.component(<ctag>).mesh(<tag>).stat().getMinQuality(type);
double model.component(<ctag>).mesh(<tag>).stat().getMinQuality();
```

To retrieve the mean quality, use

```
double model.component(<ctag>).mesh(<tag>).stat().getMeanQuality(type);
double model.component(<ctag>).mesh(<tag>).stat().getMeanQuality();
```

To calculate a distribution of qualities, use the `getQualityDistr` method.

```
int[] model.component(<ctag>).mesh(<tag>).stat().getQualityDistr(type, <size>);
int[] model.component(<ctag>).mesh(<tag>).stat().getQualityDistr(<size>);
```

The size parameter is a positive integer determining how detailed the distribution is and equals the size of the output array. The distribution can be used to plot a histogram of the element quality. For example, if size equals 10, the first entry in the returned array is the number of elements with quality less than 0.1, and the last entry is the number of elements with quality better than 0.9.

The following methods are available directly on the sequence and provide mesh quality statistics, using the volume versus circumradius quality measure, for the entire geometry:

```
double model.component(<ctag>).mesh(<tag>).getMinQuality(type);
double model.component(<ctag>).mesh(<tag>).getMinQuality();
double model.component(<ctag>).mesh(<tag>).getMeanQuality(type);
double model.component(<ctag>).mesh(<tag>).getMeanQuality();
int[] model.component(<ctag>).mesh(<tag>).getQualityDistr(type, <size>);
int[] model.component(<ctag>).mesh(<tag>).getQualityDistr(<size>);
```

The following methods are available for retrieving and specifying the mesh quality measure, respectively:

```
String model.component(<ctag>).mesh(<tag>).getQualityMeasure();
model.component(<ctag>).mesh(<tag>).setQualityMeasure(String measure);
```

Volume of Elements and Mesh

To determine minimum element volume, area, or length of a certain type, use the method `getMinVolume`:

```
double model.component(<ctag>).mesh(<tag>).stat().getMinVolume(type);
double model.component(<ctag>).mesh(<tag>).stat().getMinVolume();
```

To determine maximum element volume, area, or length of a certain type, use the method `getMaxVolume`:

```
double model.component(<ctag>).mesh(<tag>).stat().getMaxVolume(type);
double model.component(<ctag>).mesh(<tag>).stat().getMaxVolume();
```

To determine the volume, area, or length of the mesh, use the method `getVolume`:

```
double model.component(<ctag>).mesh(<tag>).stat().getVolume(type);
double model.component(<ctag>).mesh(<tag>).stat().getVolume();
```

The following methods are available directly on the sequence and provide volume information about the entire geometry:

```
double model.component(<ctag>).mesh(<tag>).getMinVolume(type);
double model.component(<ctag>).mesh(<tag>).getMinVolume();
double model.component(<ctag>).mesh(<tag>).getMaxVolume(type);
double model.component(<ctag>).mesh(<tag>).getMaxVolume();
double model.component(<ctag>).mesh(<tag>).getVolume(type);
double model.component(<ctag>).mesh(<tag>).getVolume();
```

Growth Rate in Mesh

The growth rate value is a local measure greater than or equal to 1 indicating the maximum element size growth rate between two neighboring elements.

To retrieve the maximal growth rate value for a selection, use

```
double model.component(<ctag>).mesh(<tag>).stat().getMaxGrowthRate();
```

To retrieve the average growth rate for a selection, use

```
double model.component(<ctag>).mesh(<tag>).stat().getMeanGrowthRate();
```

The following methods are available directly on the sequence and provide statistics for the entire geometry:

```
double model.component(<ctag>).mesh(<tag>).getMaxGrowthRate();
double model.component(<ctag>).mesh(<tag>).getMeanGrowthRate();
```

Mesh Status

You can check if the entire selected geometry has a mesh by calling the `isComplete` method.

```
boolean model.component(<ctag>).mesh(<tag>).stat().isComplete();
```

To check if the entire geometry is meshed, use

```
boolean model.component(<ctag>).mesh(<tag>).isComplete();
```

You can also check if the selected geometry has an empty mesh by calling the `isEmpty` method.

```
boolean model.component(<ctag>).mesh(<tag>).stat().isEmpty();
```

To check if the entire geometry has an empty mesh, use

```
boolean model.component(<ctag>).mesh(<tag>).isEmpty();
```

Mesh Plot

Create a mesh plot by calling the `createPlot` method.

```
model.component(<ctag>).mesh(<tag>).createPlot(<dtag>, <ptag>);
```

where `dtag` is the tag of the mesh dataset that the plot will refer to. If such a dataset already exists, and it refers to a mesh sequence, it is used. Otherwise, a new mesh dataset with tag `dtag` is created. The method creates and returns a plot group with the tag `ptag`. The mesh plot is generated from one or more mesh plot features in this plot group.

Measurements

To measure geometric entities in a mesh, use

```
model.component(<ctag>).mesh(<tag>).measure().measurementMethod;
```

where `<ctag>` is the component and `<tag>` is the mesh.

The methods return the volume, area, length, vertex coordinates, average coordinates, or distance between two vertices according to the entity selection. Measurements are performed on the output mesh from the last built mesh feature. For a list of the available measurement methods to use, see [Measurement Methods](#). For information about methods to select entities, see the section [Selections](#).

Information on the Geometric Model

Use the `info()` method to get information on the geometric model (number of entities, adjacency, and more). For a mesh that defines its own geometric model, use `infoCurrent()` to obtain topological information about the current (last built) mesh feature. Use `info()` to obtain the same information about the finalized mesh. See [Geometric Model](#) for more information about meshes that define their own geometric models.

```
int nDomains = model.component(<ctag>).mesh(<tag>).infoCurrent().getNDomains();
int nFaces = model.component(<ctag>).mesh(<tag>).infoCurrent().getNFaces();
int nEdges = model.component(<ctag>).mesh(<tag>).infoCurrent().getNEdges();
int nVertices = model.component(<ctag>).mesh(<tag>).infoCurrent().getNVertices();
```

See [Geometric Model Information Methods](#) for a complete list of available methods.

For a mesh that is conforming with a geometry sequence, `info()` returns information about the currently meshed entities only. This means that, for example, the adjacency relations can be a subset of the adjacency relations in the full geometric model. Since the numbering of entities is the same as in the complete geometric model, there might be gaps or unused entries in the returned data arrays.

For a mesh that defines its own geometric model, `infoCurrent()` returns geometric information on the output of the last built mesh operation, while `info()` returns geometric information for the finalized mesh. The geometric model of the finalized mesh is the geometric model used by physics and it is updated only when the finalize feature is built.

Getting and Setting Mesh Data

The data in the mesh object can be accessed and manipulated via *getters and setters*. You can get vertex coordinates, elements, and for each element the number of its geometric entity. The element matrix consists of indexes into the vertex list. The entity list contains the entity number of each element. There is one element matrix and one entity number list for each type.

Accessing Mesh Data

To get the number of mesh vertices, use

```
int model.component(<ctag>).mesh(<tag>).getNumVertex();
```

To get the coordinates of the mesh vertices, use

```
double[][] model.component(<ctag>).mesh(<tag>).getVertex();
```

which gives you a matrix where each column corresponds to a mesh vertex.

To get the element types in the mesh, use

```
String[] model.component(<ctag>).mesh(<tag>).getTypes();
```

The following table lists the possible types. See [Mesh Element Numbering Conventions](#) for an explanation of each type.

TABLE 4-6: MESH ELEMENT TYPES.

STRING	ELEMENT	DIMENSION	NUMBER OF NODES
vtx	Vertex element	0	1
edg	Edge element	1	2
tri	Triangular element	2	3
quad	Quadrilateral element	2	4
tet	Tetrahedral element	3	4
pyr	Pyramid element	3	5
prism	Prism element	3	6
hex	Hexahedral element	3	8

To get the number of elements of a specific type, use

```
int model.component(<ctag>).mesh(<tag>).getNumElem(type);
```

To get the elements for a specific type, use

```
int[][] model.component(<ctag>).mesh(<tag>).getElem(type);
```

which gives you a matrix where each column contains the mesh vertex indices of an element's corners.

To get the geometric entity number for the elements of a specific type, use

```
int[] model.component(<ctag>).mesh(<tag>).getElemEntity(type);
```

To return the tags of imported mesh selections, use

```
String[] outputSelection();
```

The tag for the corresponding selection feature can then be derived by adding component tag and feature tag in front of the mesh selection tag. For example, if a mesh selection feature is imported by a feature `imp1` in component

comp1, and its tag (returned by the function `outputSelection()`) is mytag, the tag of the selection feature is comp1_imp1_mytag.



Information and Statistics

Setting or Modifying Mesh Data

You can modify the mesh object of a meshing sequence via the `data()` method. Using this method you access a temporary object (`MeshData`) storing mesh data. When you use the `data()` method the first time the `MeshData` object is empty. You can fill it with mesh data by using various set methods or by transferring mesh data from the mesh of the meshing sequence. Call the method `data().createMesh` to construct a complete mesh from the `MeshData` object and store it in the meshing sequence. If the geometry is not empty, the new mesh is checked to ensure that it matches the geometry. Thus, to create an arbitrary mesh, you need to create an empty geometry sequence and a corresponding empty meshing sequence and construct the mesh on the empty meshing sequence.

To set the mesh vertices, use

```
model.component(<ctag>).mesh(<tag>).data().setVertex(double[][]);
```

where each column of the input matrix contains the coordinates of a mesh vertex.

To set the elements of a specific type, use

```
model.component(<ctag>).mesh(<tag>).data().setElem(type, int[][]);
```

where each column of input element matrix contains the mesh vertex indices of an element's corners.

If you want to specify the geometric entity number for the elements of a specific type, use

```
model.component(<ctag>).mesh(<tag>).data().setElemEntity(type, int[]);
```

The `MeshData` object has the same access methods as the meshing sequence.

```
int model.component(<ctag>).mesh(<tag>).data().getNumVertex();
double[][] model.component(<ctag>).mesh(<tag>).data().getVertex();
String[] model.component(<ctag>).mesh(<tag>).data().getTypes();
int model.component(<ctag>).mesh(<tag>).data().getNumElem(type);
int[][] model.component(<ctag>).mesh(<tag>).data().getElem(type);
int[] model.component(<ctag>).mesh(<tag>).data().getElemEntity(type);
```



Accessing Mesh Data

It is also possible to fill the `MeshData` object with mesh data from the mesh of a meshing sequence. To transfer the mesh from the current meshing sequence into the `MeshData` object, use

```
model.component(<ctag>).mesh(<tag>).data().transferMesh();
```

To transfer the mesh from another meshing sequence, specified by `mtag`, into the `MeshData` object, use

```
model.component(<ctag>).mesh(<tag>).data().transferMesh(mtag);
```

To clear the `MeshData` object, use

```
model.component(<ctag>).mesh(<tag>).data().clearData();
```

To create a complete mesh from the `MeshData` object and store it in the sequence, use

```
model.component(<ctag>).mesh(<tag>).data().createMesh();
```

This method uses several properties when creating a complete mesh from the specified mesh data. To set a property, use

```
model.component(<ctag>).mesh(<tag>).data().set(property, <value>);
```

To get a property, use

```
model.component(<ctag>).mesh(<tag>).data().getType(property);
```

Running the `createMesh()` method from provided, possibly incomplete, mesh data is equivalent to importing possibly incomplete mesh data from file. See [Importing Externally Generated Mesh Data](#) in the *COMSOL Multiphysics Reference Manual* for more information.

The following properties are available.

TABLE 4-7: VALID PROPERTY/VALUE PAIRS.

PROPERTY	VALUE	DEFAULT	2D	3D	DESCRIPTION
extrangle	double	0.6 degrees		√	Maximum angle between boundary element normal and extrusion plane that causes the element to be a part the extruded face if possible.
faceangle	double	360 degrees		√	Maximum angle between any two boundary elements in the same face.
facecleanup	double	0.01		√	Avoid creating small faces. Faces with an area less than Facecleanup * the mean face area, are merged with adjacent faces.
facecurv	double	10 degrees		√	Maximum relative angle deviation between any two boundary elements in the same face.
minareaecurv	double	1		√	Minimum relative area of face to be considered as a face with constant curvature.
minareaextr	double	0.05		√	Minimum relative area of face to be considered extruded.
minareaeplane	double	0.005	√	√	Minimum relative area of face to be considered planar.
neighangle	double	20 degrees	√	√	Maximum angle between a boundary element and a neighbor that causes the elements to be part of the same boundary domain if possible.
planarangle	double	0.6 degrees	√	√	Maximum angle between boundary element normal and a neighbor that causes the element to be a part the planar face if possible

EXAMPLES OF SETTING OR MODIFYING MESH DATA

The following examples create a triangular mesh on a square, extracts the vertices and the triangles. Then the vertices are transformed and inserted into a new meshing sequence.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");

model.component("comp1").geom().create("geom1", 2);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

// Create a rectangle and a mesh
model.component("comp1").geom("geom1").create("r1", "Rectangle");
m.create("ftri1", "FreeTri");
m.run();

double[][] vtx = m.getVertex();
int[][] tri = m.getElem("tri");

// Transform x-coordinates
```

```

for (int k=0; k<vtx[0].length; k++)
    vtx[0][k] *= 0.5;

// Create a new geometry and mesh
model.component("comp1").geom().create("geom2", 2);
MeshSequence m2 = model.component("comp1").mesh().create("mesh2", "geom2");

// Insert vertices and triangles and create mesh
m2.data().setElem("tri", tri);
m2.data().setVertex(vtx);
m2.data().createMesh();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');

model.component('comp1').geom.create('geom1', 2);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

% Create a rectangle and a mesh
model.component('comp1').geom('geom1').create('r1', 'Rectangle');
m.create('ftri1', 'FreeTri');
m.run;

vtx = m.getVertex;
tri = m.getElem('tri');

% Transform x-coordinates
vtx(1,:) = vtx(1,:)*0.5;

% Create a new geometry and mesh
model.component('comp1').geom.create('geom2', 2);
m2 = model.component('comp1').mesh.create('mesh2', 'geom2');

% Insert vertices and triangles and create mesh
m2.data.setElem('tri', tri);
m2.data.setVertex(vtx);
m2.data.createMesh;

```

Block Versions

Since the amount of available Java memory might be limited, there are block versions of the mesh setters and getters, which sets or gets a subset of the data. The getters take a *position* argument, which specifies the first item to get, and a *number* argument, which specifies the number of items to get. The setters takes only the position argument; the number of items is determined by the size of the provided data. When working with the setters, remember that it is more efficient to set the data at the last position first, since sufficient space is then allocated directly and no copying and reallocation is needed.

```

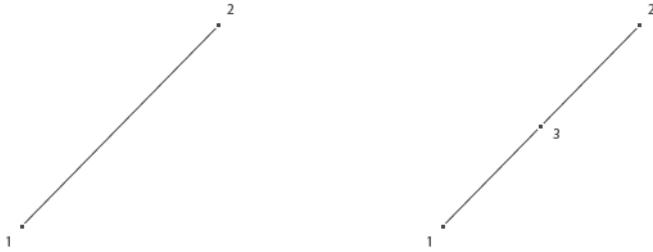
double[][] model.component(<ctag>).mesh(<tag>).getVertex(int position, int number);
int[][] model.component(<ctag>).mesh(<tag>).getElem(String type, int position, int number);
int[] model.component(<ctag>).mesh(<tag>).getElemEntity(String type, int position,
    int number);
model.component(<ctag>).mesh(<tag>).data().setVertex(double[][], int position);
model.component(<ctag>).mesh(<tag>).data().setElem(String type, int position, int[][]);
model.component(<ctag>).mesh(<tag>).data().setElemEntity(String type, int[], int position);
double[][] model.component(<ctag>).mesh(<tag>).data().getVertex(int position, int number);
int[][] model.component(<ctag>).mesh(<tag>).data().getElem(String type, int position,
    int number);
int[] model.component(<ctag>).mesh(<tag>).data().getElemEntity(String type, int position,
    int number);

```

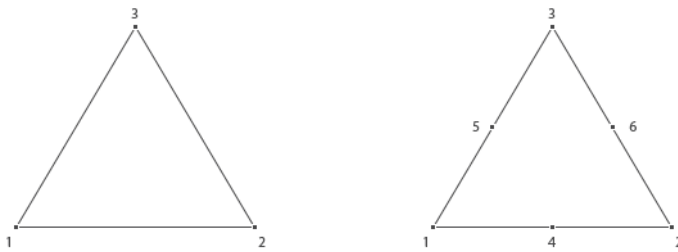
Mesh Element Numbering Conventions

The (local) numbering of the corners of a mesh element is defined according to the following.

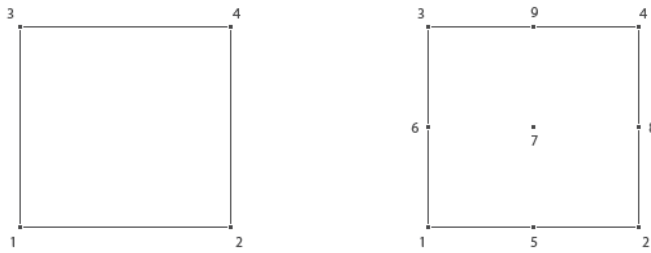
Edge element (**edg**) as a first-order element (left) and a second-order element (right):



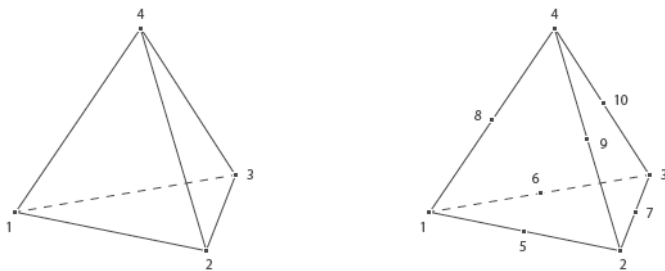
Triangular element (**tri**) as a first-order element (left) and a second-order element (right):



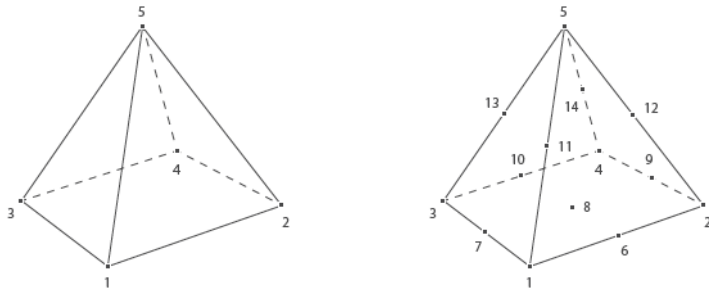
Quadrilateral element (**quad**) as a first-order element (left) and a second-order element (right):



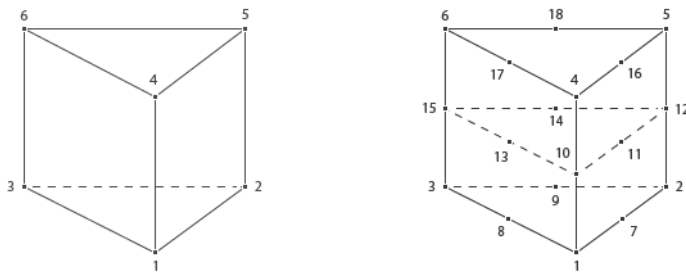
Tetrahedral element (**tet**) as a first-order element (left) and a second-order element (right):



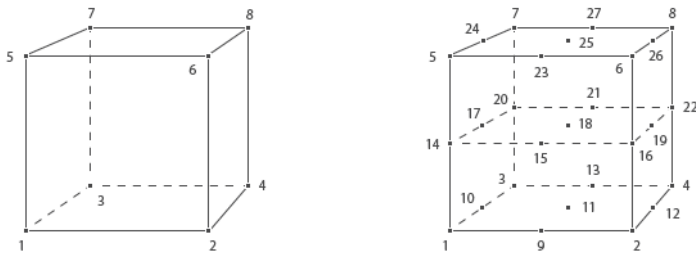
Pyramid element (**pyr**) as a first-order element (left) and a second-order element (right):



Prism element (**prism**) as a first-order element (left) and a second-order element (right):



Hexahedral element (**hex**) as a first-order element (left) and a second-order element (right):



Errors, Warnings, and Information

COMSOL Multiphysics treats problems encountered when building a meshing feature in two different ways depending on if it is possible to avoid the problem and continue the operation or if the operation must be stopped.

See also [Errors and Warnings](#) in the *General Commands* chapter.

Encountering Problems when Building the Mesh

In most cases when you build a mesh feature that generates mesh and problems are detected on some geometric entities, those entities and adjacent entities are meshed if possible, otherwise they are left unprocessed. When the mesher fails to mesh some entities, the operation continues meshing the remaining entities and stores information about the encountered problems in the feature. A feature that encountered this type of problems during the build gets a warning status (but with error nodes). If you build several mesh features in a sequence, the build is not stopped by a feature that fails to process some of its entities. However, some errors are considered as fatal and therefore stop the build process. Failure to process one or more entities in some operation will always result in an issue exception when all specified features are built, even if the build process was not stopped directly.

Continue Meshing

Use the `continue` property to control whether a meshing feature should avoid encountered problems and continue to mesh or if it should stop at the first encountered problem.

Errors Stopping the Operations

When you build a feature other than any of the `BndLayer`, `FreeTri`, `FreeQuad`, `FreeTet`, `Map`, or `Sweep` features, the operation always stops if an error is encountered. This means that no changes are made to the mesh. The feature gets an error status and if it is part of a sequence build the build stops and the preceding feature becomes the current feature. Information on the error is stored in the feature. Refer to the section [Retrieving Mesh Problem Information](#) for more information on accessing this information.

Retrieving Mesh Problem Information

There are three categories of problems that can appear in a meshing sequence: `MeshInformation`, `MeshWarning`, and `MeshError` features. They all contain a message describing the problem and can be equipped with a selection defining the geometric entities or coordinate values specifying a position related to the error. All of these features can have a subfeature of the same type that contains low-level problem information. This means that a problem can be represented by a stack of features that reflects the stack trace of the problem.

For more information about the severity of the problems and possible ways to fix them, see the section [Information, Warnings, and Errors for Meshing Operations](#).

Use

```
boolean problem = model.component(<ctag>).mesh(<tag>).feature(<ftag>).hasProblems();
```

to find out if the feature `<ftag>` has any reported problems. There are similar methods, `hasWarnings()` and `hasErrors()`, for warnings and errors, respectively. The following method returns the tags of the problems as well as tags that refer to information:

```
String[] problemTags = model.component(<ctag>).mesh(<tag>).feature(<ftag>).problems();
```

Similarly, use the methods `warnings()` and `errors()` to get access to only warning and error tags, respectively.

The following two methods return the message and any entity selection of the problem `<ptag>`.

```
String problemMessage = model.component(<ctag>).mesh(<tag>).feature(<ftag>).  
    problem(<ptag>).message();  
  
MeshSelection sel = model.component(<ctag>).mesh(<tag>).feature(<ftag>).  
    problem(<ptag>).selection();
```

To retrieve the full stack trace, repeat the above methods until all levels of reported problems have been accessed. See the section [Retrieving Problem Information](#) for an example of how to retrieve mesh warnings recursively.

Retrieving Information About the Latest Build

The following methods are available to retrieve the information specific to the latest build of a mesh sequence or a mesh feature. Unless stated otherwise, the output of the methods will be empty if the feature or sequence has not yet been built.

Enter

```
String outSeq[][] = model.component(<ctag>).mesh(<tag>).buildOutput();  
String outFeat[][] = model.component(<ctag>).mesh(<tag>).feature(<ftag>).buildOutput();
```

to get a matrix with output information from the build of a mesh sequence (also locked meshes) or mesh feature, respectively. The matrix can contain information about generated, imported or modified entities, and elements. See [Information, Warnings, and Errors for Meshing Operations](#) in the *COMSOL Multiphysics Reference Manual* for more information.

To see the COMSOL version number (and architecture) used for the latest build of a mesh sequence or a feature, use (for mesh sequence)

```
String versionSeq = model.component(<ctag>).mesh(<tag>).buildComsolVersion();
```

To get access to the date and time of the last time a sequence or feature was built, enter

```
String dateSeq = model.component(<ctag>).mesh(<tag>).buildDate();
```

Use

```
int timeSeq = model.component(<ctag>).mesh(<tag>).buildTime();
```

to get the time (in seconds) it took to build a sequence (or feature) the last time it was built. In case the feature has not been built, it will return -1.

For information messages specific to a particular feature, use

```
String infoFeat[] = model.component(<ctag>).mesh(<tag>).feature(<ftag>).buildInfo();
```

Note that other information may be presented in `MeshInformation` features. See [Retrieving Mesh Problem Information](#) for more information on accessing this information.

Exporting Meshes to Files

Exporting Mesh to a File

To export a mesh to a file, enter

```
model.component(<ctag>).mesh(<tag>).export(<filename>);
```

where *<filename>* is a string or a file location URI used to identify a file version in a Model Manager database. The file can be any of the following formats.

TABLE 4-8: VALID FILE FORMATS.

FILE FORMAT	NOTE	FILE EXTENSIONS
COMSOL Multiphysics Binary		.mphbin
COMSOL Multiphysics Text		.mphtxt
NASTRAN file		.nas, .bdf, .nastran, .dat
STL Binary (3D)	1	.stl
STL Text (3D)	1	.stl
Sectionwise		.txt
PLY Binary (3D)	2	.ply
PLY Text (3D)	2	.ply
3MF (3D)		.3mf

¹ Use `model.mesh(<tag>).export().set("stlformat",<format>)` to specify the STL file format ("binary" or "text")

² Use `model.mesh(<tag>).export().set("plyformat",<format>)` to specify the PLY file format ("binary" or "text")

Exporting Mesh to a COMSOL Multiphysics File

To specify the dimensions of the elements to export or to choose to include or exclude the geometric entity information, enter

```
model.component(<ctag>).mesh(<tag>).export().set(<property>,<value>);
```

The following table lists the available properties:

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bndelem	on off	on	Specify if boundary elements are exported.
domelem	on off	on	Specify if domain elements are exported.
edgelem	on off	on	Specify if edge elements are exported (3D only).
fileversion	v44 v64	v64	Specify the file version with which the MPHTXT file will be compatible.
vtxelem	on off	on	Specify if vertex elements are exported (2D and 3D only).
geominfo	on off	on	Specify if geometric entity information for each element is exported.
nativequadratic	on off	off	Specify if elements are exported as linear or second-order (quadratic) elements.
selection	on off	off	Specify if mesh selections are exported.

Exporting Mesh to a NASTRAN® File

To specify the dimensions of the elements to export, to choose to include or exclude the geometric entity information, or to set the file field format or element order, enter

```
model.component(<ctag>).mesh(<tag>).export().set(<property>,<value>);
```

The following table lists the available properties:

TABLE 4-9: AVAILABLE NASTRAN MESH EXPORT PROPERTIES AND THEIR VALID VALUES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
solidelem	on off	on	Specify if domain elements are exported.
shellelem	on off	off	Specify if boundary elements are exported (3D only).
geominfo_nastran	on off	on	Specify if geometric entity information for each element is exported.
fieldformat	small large free	large	Specify file field format.
nastranquadratic	on off	on	Specify if elements are exported as linear or second-order (quadratic) elements.

Exporting Mesh to a Sectionwise Format

To specify the geometric entity level for the sectionwise format, use

```
model.component(<ctag>).mesh(<tag>).export().set(<property>,<value>);
```

The following table lists the available property:

TABLE 4-10: AVAILABLE COMSOL SECTIONWISE MESH EXPORT PROPERTY AND ITS VALID VALUES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
level	domain boundary edge	domain	Specify the geometry entity level.

Exporting Mesh to a 3MF Format

To specify the 3MF data to export on the 3MF format, use

```
model.component(<ctag>).mesh(<tag>).export().set(<property>,<value>);
```

The following table lists the available property:

TABLE 4-11: AVAILABLE 3MF MESH EXPORT PROPERTY AND ITS VALID VALUES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
3mfdata	domain boundary	boundary	Specify if the data to export.

Mesh Commands

The following list includes the available commands for creating and modifying meshes (listed in alphabetical order):

- `Adapt`
- `AdjacentSelection`
- `Ball`
- `BallSelection`, `BoxSelection`,
`CylinderSelection`, `DiskSelection`
- `BndLayer`
- `BndLayerProp`
- `Box`
- `CollapseEntities`
- `Convert`
- `CopyEdge`
- `CopyFace`
- `CopyDomain`
- `Copy`
- `CornerProp`
- `CornerRefinement`
- `CreateDomains`
- `CreateEdges`
- `CreateFaces`
- `CreateVertices`
- `Cylinder`
- `DeleteEntities`
- `DetectFaces`
- `Distribution`
- `Edge`
- `EdgeGroup`
- `EdgeMap`
- `ExplicitSelection`
- `FillHoles`
- `FixedMesh`
- `FreeQuad`
- `FreeTet`
- `FreeTri`
- `IdenticalMesh`
- `Import`
- `Imprint`
- `IntersectLine`
- `IntersectPlane`
- `JoinEntities`
- `LogicalExpression`
- `Map`
- `MergeEntities`
- `OnePointMap`
- `Reference`
- `Refine`
- `RemeshDomains`
- `RemeshEdges`
- `RemeshFaces`
- `Scale`
- `Size`
- `SizeExpression`
- `Sweep`
- `Transform`
- `TwoPointMap`
- `Union`
- `UnionSelection`, `IntersectionSelection`,
`DifferenceSelection`, `ComplementSelection`
- `Vertex`

Adapt

Set up an adaptive mesh refinement.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Adapt");  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).importData();  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Adapt")` to set up a mesh adaptation based on some expressions and criteria. The adaptive mesh refinement feature is also available for imported meshing sequences.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify geometric entities to perform adaptive mesh refinement in. If you do not specify the selection, the feature operates on the entire geometry.

To use an anisotropic metric for the type of expression to base the adaptive mesh refinement on, use the following code for a 2D anisotropic expression in the local mesh size h :

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set("method", "modify");  
model.component(<ctag>).mesh(<tag>).feature("ada1").set("exptype", "metric");  
model.component(<ctag>).mesh(<tag>).feature("ada1").  
    set("metric", new String[][]{{"2/h", "0"}, {"0", "1/h"}});
```

You can use `mesh.feature(<ftag>).importData()` to rebuild the adapted mesh, taking an updated model into account.

The following properties are available.

TABLE 4-12: AVAILABLE PROPERTIES FOR ADAPT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapsolnum	Integer array (positive values)	1	Solution number indices, determining which solutions in a parametric or eigenvalue simulation to use for the adaptation.
allowcoarsening	on off	on	Controls if the mesh can be coarsened by the general modification method (that is, method set to modify). For $sdim \geq 2$.
elementspar	Positive scalar	Empty	Controls refinement if <code>elselect = elements</code> .
elselect	globalmin worst elements	Empty	Method to select elements to refine.
errorexpr	String	Empty	Error expression.
exptype	size error metric	error, if added in a component and a solution exists; otherwise, size	Type of expression for the adaptive mesh generation: an absolute size, an error expression, or (2D and 3D) an anisotropic metric.
facerep	curved linear	curved	In 3D, specify face representation when placing moved or new mesh vertices for meshes that define their own geometric models.
globalminparam	Positive scalar	Empty	Controls refinement if <code>elselect = globalmin</code> .
horder	Double array	0	Error orders (see below).
maxcoarsening	Positive integer	5	The maximum coarsening factor (if method is set to modify and <code>allowcoarsening</code> is on). For $sdim \geq 2$.

TABLE 4-12: AVAILABLE PROPERTIES FOR ADAPT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
maxrefinement	Positive integer	5	The maximum number of mesh refinements.
method	modify regular longest	longest	For <code>sdim</code> >=2, set the refinement method for mesh adaptation (general mesh modification, regular refinement, or longest edge refinement). In 1D, method is regular.
metric	2-by-2 (2D) or 3-by-3 (3D) symmetric string matrix	{{"1/h", "0"}, {"0", "1/h"}}	
selection	first last all manual	last	Solution selection: the first or last solution, a sum of the selection, or manual, using weights and solution number indices in <code>adapcsolnum</code> .
sizeexpr	String	Empty	Mesh size expression.
solution	String	Empty	The tag of the solution defining the mesh adaptation, or none to evaluate on the input mesh.
updatecondition	A parameter name	Empty	Name of a parameter used to trigger an update.
weights	Double array (positive values)	1.0	Weight for each selected solution.
worstpar	Positive scalar	Empty	Controls refinement if <code>elselect</code> = worst.

The `facerep` property is only used for meshes that define their own geometric model. For example, when having an imported mesh. Use `curved` to place new or moved mesh vertices on a curved surface approximation of the input mesh. Use `linear` to place new or moved mesh vertices on the input mesh.

SEE ALSO

[Refine](#), [SizeExpression](#)

AdjacentSelection

Define a selection of entities that are adjacent to a specified selection.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"AdjacentSelection");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Supported for meshes that define their own geometric model, such as imported meshes. For more information, see [Geometric Model](#).

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"AdjacentSelection")` to create a named selection for one or more entities.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the input entities for the selection. For information about specifying a named selection as input, see [Selections](#).

The following properties are available:

TABLE 4-13: AVAILABLE PROPERTIES FOR ADJACENTSELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angletol	double	15	Angle tolerance for continuity evaluation. Used when groupcontang is on.
groupcontang	on off	off	Specify to group faces (in 3D) or edges by continuous tangent.
entitydim	3 2 1 0	space dimension	Dimension of entities to select.
exterior	on off	on	Include output entities that are exterior to the union of the input selections.
interior	on off	off	Include output entities that are interior to the union of the input selections.
inputent	all selections	all	Select among all entities or entities defined by the input property.
input	String[]	{}	Tags of input selections. Used when inputent is selections.
outputdim	0 1 2 3	space dimension - 1	Dimension of entities in the output selection.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
selshow	on off	on	Show selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part.

SEE ALSO

[UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#), [BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#), [ExplicitSelection](#)

Ball

Split geometric entities of an imported mesh by a ball.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Ball");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Ball")` to split geometric entities of an imported 2D or 3D mesh by an element set defined by a ball.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify geometric entities to split. If you do not specify the selection, the feature operates on the entire geometry.

The following properties are available:

TABLE 4-14: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
condition	allvertices somevertex	allvertices	Condition for inclusion of an element.
posx	double	0	Center, first coordinate.
posy	double	0	Center, second coordinate.
posz	double	0	Center, third coordinate.
r	double	1	Radius.
selinside	on off	off	Create selection of all entities inside the ball.
seloutside	on off	off	Create selection of all entities outside the ball.

TABLE 4-15: AVAILABLE ADDITIONAL PROPERTIES WHEN SELINSIDE AND SELOUTSIDE ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorinside	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorinside property.
coloroutside	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcoloroutside property.
customcolorinside	RGB-triplet	Next available theme color	The color to use. Active when colorinside is set to custom.
customcoloroutside	RGB-triplet	Next available theme color	The color to use. Active when coloroutside is set to custom.
selinsideshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.
seloutsideshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.

SEE ALSO

[Import](#), [Box](#), [Cylinder](#), [DetectFaces](#), [LogicalExpression](#)

BallSelection, *BoxSelection*, *CylinderSelection*, *DiskSelection*

Create selections of geometric entities that (partly) lie inside a ball, box, cylinder, or disk.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"BallSelection");
model.component(<ctag>).mesh(<tag>).create(<ftag>,"BoxSelection");
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CylinderSelection");
model.component(<ctag>).mesh(<tag>).create(<ftag>,"DiskSelection");
model.component(<ctag>).mesh(<tag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).getType(property);
```

DESCRIPTION

The following general properties are available:

TABLE 4-16: AVAILABLE GENERAL PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
entitydim	3 2 1 0	space dimension	Dimension of entities to select.
inputent	all selections	all	Select among all entities or entities defined by the input property.
input	String[]	{}	Tags of input selections. Used when inputent is selections.
condition	intersects inside somevertex allvertices	intersects	Condition for inclusion of an entity.
groupcontang	on off	off	Continuous tangent mode.
angletol	double	15	Angle tolerance for continuity evaluation (value is converted if radians is used).
selshow	on off	on	Show selection in physics, materials, and so on. For mesh parts, this option shows selection in instances of the part.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.

For BallSelection, you define the ball using the following properties:

TABLE 4-17: ADDITIONAL PROPERTIES FOR BALLSELECTION.

NAME	VALUE	DEFAULT	DESCRIPTION
posx	double	0	Center of ball, first coordinate.
posy	double	0	Center of ball, second coordinate.
posz	double	0	Center of ball, third coordinate.
r	double (nonnegative)	0	Radius of ball.

For BoxSelection, you define the box using the following properties:

TABLE 4-18: ADDITIONAL PROPERTIES FOR BOXSELECTION.

NAME	VALUE	DEFAULT	DESCRIPTION
xmax	double	inf	Maximum x-coordinate of box.
xmin	double	-inf	Minimum x-coordinate of box.
ymax	double	inf	Maximum y-coordinate of box.
ymin	double	-inf	Minimum y-coordinate of box.
zmax	double	inf	Maximum z-coordinate of box in 3D.
zmin	double	-inf	Minimum z-coordinate of box in 3D.

For `CylinderSelection`, you define the cylinder using the following properties:

TABLE 4-19: ADDITIONAL PROPERTIES FOR CYLINDERSELECTION.

NAME	VALUE	DEFAULT	DESCRIPTION
angle1	double	0	Start angle
angle2	double	360 or 2π	End angle (default is a full cylinder)
axistype	x y z cartesian spherical	z	Type of axis or coordinate system used for axis. The value is synchronized with axis.
bottom	double	-inf	Coordinate of bottom face in local coordinate system.
pos	double[]	{0,0,0}	Base point.
r	double (nonnegative)	0	Outer radius.
rin	double (nonnegative)	0	Inner radius.
top	double	inf	Coordinate of top face in local coordinate system.
ax3	double[]	{0,0,1}	Direction of cylinder axis. Used when axistype is set to cartesian.
ax2	double[]	{0,0}	Direction of cylinder axis. Used when axistype is set to spherical.

For `DiskSelection`, you define the disk using the following properties:

TABLE 4-20: ADDITIONAL PROPERTIES FOR DISKSELECTION.

NAME	VALUE	DEFAULT	DESCRIPTION
angle1	double	0	Start angle
angle2	double	360	End angle (default is a full disk)
posx	double	0	Center of disk, first coordinate.
posy	double	0	Center of disk, second coordinate.
r	double (nonnegative)	0	Outer radius.
rin	double (nonnegative)	0	Inner radius.

You select the input entities to select among using the properties `entitydim`, `inputent`, and `input`. For a boundary or edge selection in 2D or 3D, you can force the selection to select whole groups of entities by setting the property `groupcontang` to on. Each group consists of adjacent entities that meet at an angle less than `angletol`.

The output entities/objects are determined by the property condition:

- **intersects:** All entities that intersect the ball/box/cylinder/disk are included.
- **inside:** All entities that are completely inside the ball/box/cylinder/disk are included.
- **somevertex:** All entities that have at least one adjacent vertex inside the ball/box/cylinder/disk are included.
- **allvertices:** All entities that have all adjacent vertices inside the ball/box/cylinder/disk are included.

SEE ALSO

[UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#), [AdjacentSelection](#), [ExplicitSelection](#)

BndLayer

Create a boundary layer mesh. Supported for domains in 2D and 3D as well as for faces in 3D. For imported meshes, the domains must contain a mesh. Use [FreeTet](#) to fill unmeshed domains before creating a boundary layer mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"BndLayer");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"BndLayerProp");
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"BndLayer")` to create a boundary layer mesh in domains in 2D and 3D, and on faces in 3D.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the domain or boundary selection. If you do not specify the selection, the feature operates on all domains or on all boundaries depending on the selection dimension of the corresponding `BndLayerProp` attribute.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"BndLayerProp")` to add a `BndLayerProp` attribute feature defining the locations and properties of the boundary layers.

The feature reads properties from the `BndLayerProp` attribute feature.

The following properties are available:

TABLE 4-21: AVAILABLE PROPERTIES FOR BNDLAYER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sharpcorners	none split trim	split	Specifies the handling of sharp corners in 2D and sharp edges in 3D.
layerdec	integer	2	Maximum difference in number of boundary layers between neighboring points on boundary layer boundaries. For all values of sharpcorners.
method	latest legacy60 legacy61	latest	Specify the boundary layer method to use.
smoothtransition	on off	on	Specifies if the operation smooths the transition to interior mesh.
splitangle	double	240[deg]	Minimum angle between boundary layer boundaries to introduce a boundary layer split. When sharpcorners is split.
splitdivangle	double	100[deg]	Maximum angle per split. When sharpcorners is split.
trimmaxangle	double	50[deg]	Maximum angle for boundary layer trimming in narrow corners. For all values of sharpcorners.
trimminangle	double	240[deg]	Minimum angle between boundary layer boundaries for boundary layer trimming. When sharpcorners is trim.

TABLE 4-21: AVAILABLE PROPERTIES FOR BNDLAYER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. When smoothtransition is on.
smoothmaxdepth	integer	6	Specifies the maximum element smoothing depth. When smoothtransition is on.

EXAMPLES

Insert boundary layers to an existing mesh containing both quadrilateral elements and triangular elements.

Code for Use with Java

```

Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("sq1", "Square");
g.create("sq2", "Square");
g.feature("sq2").setIndex("pos", "1", 0);
g.create("c1", "Circle");
g.feature("c1").set("r", "0.2");
g.feature("c1").setIndex("pos", "1.5", 0);
g.feature("c1").setIndex("pos", "0.5", 1);
g.create("co1", "Compose");
g.feature("co1").selection("input").init().set(new String[]{"c1", "sq1", "sq2"});
g.feature("co1").set("formula", "sq1+sq2-c1");
g.run();

m.create("map1", "Map");
m.feature("map1").selection().geom("geom1", 2);
m.feature("map1").selection().set(new int[]{1});
m.create("ftri1", "FreeTri");
m.create("bl1", "BndLayer");
m.feature("bl1").create("blp", "BndLayerProp");
m.feature("bl1").feature("blp").selection().set(new int[]{2, 3, 5, 6, 8, 9, 10, 11});
m.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
m = model.component('comp1').mesh.create('mesh1', 'geom1');
g.create('sq1', 'Square');
g.create('sq2', 'Square');
g.feature('sq2').setIndex('pos', '1', 0);
g.create('c1', 'Circle');
g.feature('c1').set('r', '0.2');
g.feature('c1').setIndex('pos', '1.5', 0);
g.feature('c1').setIndex('pos', '0.5', 1);
g.create('co1', 'Compose');
g.feature('co1').selection('input').init.set({'c1', 'sq1', 'sq2'});
g.feature('co1').set('formula', 'sq1+sq2-c1');
g.run();

m.create('map1', 'Map');
m.feature('map1').selection.geom('geom1', 2);
m.feature('map1').selection.set(1);
m.create('ftri1', 'FreeTri');
m.create('bl1', 'BndLayer');
m.feature('bl1').create('blp', 'BndLayerProp');
m.feature('bl1').feature('blp').selection.set([2, 3, 5, 6, 8, 9, 10, 11]);
m.run();

```

Create a boundary layer mesh consisting of prism elements along the boundary layer boundaries and tetrahedral elements in the interior:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.feature("blk1").setIndex("size", "10", 0);
g.feature("blk1").setIndex("size", "5", 1);
g.feature("blk1").setIndex("size", "5", 2);
g.create("sph1", "Sphere");
g.feature("sph1").setIndex("pos", "3", 0);
g.feature("sph1").setIndex("pos", "2.5", 1);
g.feature("sph1").setIndex("pos", "2.5", 2);
g.create("dif1", "Difference");
g.feature("dif1").selection("input").init().set(new String[]{"blk1"});
g.feature("dif1").selection("input2").init().set(new String[]{"sph1"});
g.run();

m.create("ftet1", "FreeTet");
m.create("bl1", "BndLayer");
m.feature("bl1").create("blp", "BndLayerProp");
m.feature("bl1").feature("blp").selection().
    set(new int[]{2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13});
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('blk1', 'Block');
g.feature('blk1').setIndex('size', '10', 0);
g.feature('blk1').setIndex('size', '5', 1);
g.feature('blk1').setIndex('size', '5', 2);
g.create('sph1', 'Sphere');
g.feature('sph1').setIndex('pos', '3', 0);
g.feature('sph1').setIndex('pos', '2.5', 1);
g.feature('sph1').setIndex('pos', '2.5', 2);
g.create('dif1', 'Difference');
g.feature('dif1').selection('input').init.set({'blk1'});
g.feature('dif1').selection('input2').init.set({'sph1'});
g.run();

m.create('ftet1', 'FreeTet');
m.create('bl1', 'BndLayer');
m.feature('bl1').create('blp', 'BndLayerProp');
m.feature('bl1').feature('blp').selection.set(2:13);
m.run();
```

SEE ALSO

[BndLayerProp](#), [Map](#), [Sweep](#)

BndLayerProp

Set the boundary layer meshing properties.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"BndLayerProp");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"BndLayerProp")` to define boundary layer properties for the `BndLayer` feature `<ftag>`.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the boundary or edge selection. If you do not specify the selection, it is empty.

The following properties are available:

TABLE 4-22: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>blnlayers</code>	Integer	8	Number of boundary layers
<code>blstretch</code>	Double	1.2	Boundary layer stretching factor
<code>inittype</code>	<code>blhminfact</code> <code>blhmin</code> <code>blhtot</code>	<code>blhminfact</code>	Specifies how the thickness of the boundary layer mesh is calculated.
<code>blhminfact</code>	Double	1	Factor used to adjust the default thickness. Used when <code>inittype</code> is <code>blhminfact</code> .
<code>blhmin</code>	Double		Thickness of first boundary layer. Used when <code>inittype</code> is <code>blhmin</code> .
<code>blhtot</code>	Double		Total thickness of the boundary layer mesh. Used when <code>inittype</code> is <code>blhtot</code> .

The value of `blnlayers` is a positive integer scalar, or a string that evaluates to a positive integer.

The values of `blhmin`, `blhminfact`, `blhtot`, and `blstretch` are positive real scalars, or strings that evaluate to positive real scalars.

Use the properties `blhmin`, `blstretch`, and `blnlayers` to specify the distribution of the boundary layers. `blhmin` specifies the thickness of the initial boundary layer, `blhtot` specifies the total thickness of the boundary layer mesh, `blstretch` a stretching factor, and `blnlayers` the number of boundary layers. The thickness of the m th boundary layer ($m=1$ to `blnlayers`) is $\text{blstretch}^{(m-1)}\text{blhmin}$. The number of boundary layers and the thickness of the boundary layers might be automatically reduced in thin regions.

The default values of `blhmin` and `blhtot` are determined from the specified element size in the sequence.

It is also possible to specify the thickness of the initial layer by using the `blhminfact` property. Use this property to specify a scaling factor that multiplies the thickness of the first boundary layer.

The property `inittype` determines which of `blhminfact`, `blhmin`, or `blhtot` that is used. You do not need to set it explicitly because the feature automatically changes it when you set one of `blhminfact`, `blhmin`, or `blhtot`.

SEE ALSO

[BndLayer](#), [Scale](#), [Size](#)

Box

Split geometric entities of an imported mesh by a box.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Box");  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Box")` to split geometric entities of an imported 2D or 3D mesh by an element set defined by a box.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify geometric entities to split. If you do not specify the selection, the feature operates on the entire geometry.

The following properties are available:

TABLE 4-23: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
condition	allvertices somevertex	allvertices	Condition for inclusion of an element
xmin	double	-inf	Minimum x-coordinate of box
xmax	double	inf	Maximum x-coordinate of box
ymin	double	-inf	Minimum y-coordinate of box
ymax	double	inf	Maximum y-coordinate of box
zmin	double	-inf	Minimum z-coordinate of box
zmax	double	inf	Maximum z-coordinate of box
selinside	on off	off	Create selection of all entities inside the box.
seloutside	on off	off	Create selection of all entities outside the box.

TABLE 4-24: AVAILABLE ADDITIONAL PROPERTIES WHEN SELINSIDE AND SELOUTSIDE ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorinside	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorinside property.
coloroutside	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcoloroutside property.
customcolorinside	RGB-triplet	Next available theme color	The color to use. Active when colorinside is set to custom.
customcoloroutside	RGB-triplet	Next available theme color	The color to use. Active when coloroutside is set to custom.
selinsideshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.
seloutsideshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.

SEE ALSO

[Import](#), [Ball](#), [Cylinder](#), [DetectFaces](#), [LogicalExpression](#)

CollapseEntities

Collapse geometric entities of the mesh. The operation can collapse edges and boundaries smaller than a tolerance. If all boundaries around an unmeshed domain are collapsed, the domain will also be collapsed.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CollapseEntities");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"CollapseEntities")` to collapse entities in 3D mesh. Supported input: entire geometry, boundaries, and edges.

The following properties are available:

TABLE 4-25: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sizetype	auto relative absolute	auto	Specifies the type of maximum size.
maxabssize	double		Specifies the absolute size when sizetype is absolute.
maxrelsize	double	0.01	Specifies the relative size when sizetype is relative.

SEE ALSO

[MergeEntities](#), [JoinEntities](#), [DeleteEntities](#), [Import](#)

Convert

Convert a mesh to a simplex mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Convert");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Convert")` to convert nonsimplex elements in a 2D or 3D mesh to simplex elements, that is, triangles and tetrahedra. The convert feature is also available for imported meshing sequences.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the domain or face selection. If you do not specify the selection the feature converts all quadrilateral, pyramidal, prismatic, and hexahedral elements in the mesh.

The following properties are available:

TABLE 4-26: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
splitmethod	diagonal center	diagonal	Split method for quadrilateral and hexahedral elements.

Use the property `splitmethod` to specify how to split quadrilateral and hexahedral elements into triangular and tetrahedral elements, respectively. Use the `diagonal` option to split each quadrilateral element into two triangular elements and each hexahedral element into five tetrahedral element. Use the `center` option to split each quadrilateral element into four triangular elements and each hexahedral element into 28 tetrahedral elements. The conversion also affects quadrilateral elements on the boundaries of the specified domains in 3D, which are converted into two triangular elements (when the option `diagonal` is used) or four triangular elements (when the option `center` is used).

EXAMPLES

Create a mapped quad mesh on a unit rectangle and convert each quadrilateral element into four triangular elements:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("r1", "Rectangle");
g.run();

m.create("map1", "Map");
m.create("conv1", "Convert");
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('r1', 'Rectangle');
g.run;

m.create('map1', 'Map');
m.create('conv1', 'Convert');
m.run;
```

Create a prism mesh and then convert each prism into three tetrahedral elements:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.run();

m.create("ftri1", "FreeTri");
m.feature("ftri1").selection().set(new int[]{1});
m.create("swe1", "Sweep");
m.create("conv1", "Convert");
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
m = model.component('comp1').mesh.create('mesh1', 'geom1');
```

```

g.create('blk1', 'Block');
g.run;

m.create('ftri1', 'FreeTri');
m.feature('ftri1').selection().set(1);
m.create('swe1', 'Sweep');
m.create('conv1', 'Convert');
m.run;

```

SEE ALSO

[BndLayer](#), [Map](#), [Refine](#), [Sweep](#)

CopyEdge

Copy an edge mesh to copy meshes on edges.

SYNTAX

```

model.component(<ctag>).mesh(<tag>).create(<ftag>,"CopyEdge");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);

```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"CopyEdge")` to copy mesh between edges in a 2D or 3D geometry.

The following properties are available:

TABLE 4-27: AVAILABLE PROPERTIES FOR COPYEDGE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
copymethod	auto singlecopy arraycopy	auto	Type of copy operation.
direction	auto same opposite	auto	Direction of the copied mesh.
source	Selection	Empty	Source edges.
destination	Selection	Empty	Destination edges.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.

Use the properties `source` and `destination` to specify the source and destination edges. The `copymethod` property determines which type of copy is used: single copy (all-to-one), array copy (one-to-one), or automatic detection. The value `auto` lets the software choose between single copy (all-to-one), array copy (one-to-one), or a mixture of the two. The `direction` property controls the orientation of the copied mesh, and is relative the direction of the source edge with smallest number and the direction of the destination edge.

Copying a mesh is only possible if the destination edge is not adjacent to a meshed domain. The copy feature overwrites any existing mesh on the destination edge.

EXAMPLE

Mesh Edge 1 and copy the mesh to Edges 2, 3, and 4.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("sq1", "Square");
g.run();

m.create("edg1", "Edge");
m.feature("edg1").selection().set(new int[]{1});
m.create("cpe1", "CopyEdge");
m.feature("cpe1").selection("source").set(new int[]{1});
m.feature("cpe1").selection("destination").set(new int[]{2, 3, 4});
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('sq1', 'Square');
g.run;

m.create('edg1', 'Edge');
m.feature('edg1').selection().set(1);
m.create('cpe1', 'CopyEdge');
m.feature('cpe1').selection('source').set(1);
m.feature('cpe1').selection('destination').set(2:4);
m.run;
```

SEE ALSO

[IdenticalMesh](#), [CopyFace](#), [CopyDomain](#), [Copy](#)

CopyFace

Copy a face mesh to copy meshes on faces using a rigid body transformation with a scale factor. Use the attributes [EdgeMap](#), [OnePointMap](#), [TwoPointMap](#) to control the orientation of the source mesh on the destination.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CopyFace");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,maptype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"CopyFace")` to copy mesh between faces in a 3D geometry.

If you want to specify the orientation of the source mesh on the destination, use

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,maptype)` to add an [EdgeMap](#), [OnePointMap](#), or [TwoPointMap](#) attribute feature.

The following properties are available:

TABLE 4-28: AVAILABLE PROPERTIES FOR COPYFACE

PROPERTY	VALUE	DEFAULT	DESCRIPTION
copymethod	auto singlecopy arraycopy	auto	Type of copy operation.
source	Selection	Empty	Source boundaries.

TABLE 4-28: AVAILABLE PROPERTIES FOR COPYFACE

PROPERTY	VALUE	DEFAULT	DESCRIPTION
destination	Selection	Empty	Destination boundary.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.

Use the properties `source` and `destination` to specify the source and destination boundaries. The `copymethod` property determines which type of copy is used: single copy (all-to-one), array copy (one-to-one), or automatic detection. The value `auto` lets the software choose between single copy (all-to-one), array copy (one-to-one), or a mixture of the two.

EXAMPLE

Mesh Face 1 of a block and copy the mesh to the opposite Face 6.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
MeshSequence m= model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.run();

m.create("ftri1", "FreeTri");
m.feature("ftri1").selection().set(new int[]{1});
m.create("cpf1", "CopyFace");
m.feature("cpf1").selection("source").set(new int[]{1});
m.feature("cpf1").selection("destination").set(new int[]{6});
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('blk1', 'Block');
g.run;

m.create('ftri1', 'FreeTri');
m.feature('ftri1').selection().set(1);
m.create('cpf1', 'CopyFace');
m.feature('cpf1').selection('source').set(1);
m.feature('cpf1').selection('destination').set(6);
m.run;
```

SEE ALSO

[IdenticalMesh](#), [CopyEdge](#), [CopyDomain](#), [Copy](#), [EdgeMap](#), [OnePointMap](#), [TwoPointMap](#)

CopyDomain

Copy a domain mesh to copy meshes on domains using a rigid body transformation with a scale factor. Use the attributes [EdgeMap](#), [OnePointMap](#), [TwoPointMap](#) to control the orientation of the source mesh on the destination.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CopyDomain");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,maptype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"CopyDomain")` to copy mesh between domains in a 2D or 3D geometry.

If you want to specify the orientation of the source mesh on the destination, use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,maptype)` to add an `EdgeMap`, `OnePointMap`, or `TwoPointMap` attribute feature.

The following properties are available:

TABLE 4-29: AVAILABLE PROPERTIES FOR COPYDOMAIN.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
copymethod	auto singlecopy arraycopy	auto	Type of copy operation.
source	Selection	Empty	Source domains.
destination	Selection	Empty	Destination domains.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.

Use the properties `source` and `destination` to specify the source and destination boundaries. The `copymethod` property determines which type of copy is used: single copy (all-to-one), array copy (one-to-one), or automatic detection. The value `auto` lets the software choose between single copy (all-to-one), array copy (one-to-one), or a mixture of the two.

EXAMPLE

Mesh block 1 and copy the mesh to block 2.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.create("blk2", "Block");
g.feature("blk2").setIndex("pos", "2", 0);
g.run();

m.create("ftet1", "FreeTet");
m.feature("ftet1").selection().set(new int[]{1});
m.create("cpd1", "CopyDomain");
m.feature("cpd1").selection("source").set(new int[]{1});
m.feature("cpd1").selection("destination").set(new int[]{2});
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
```

```

model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('blk1', 'Block');
g.create('blk2', 'Block');
g.feature('blk2').setIndex('pos', '2', 0);
g.run;

m.create('ftet1', 'FreeTet');
m.feature('ftet1').selection().set(1);
m.create('cpd1', 'CopyDomain');
m.feature('cpd1').selection('source').set(1);
m.feature('cpd1').selection('destination').set(2);
m.run;

```

SEE ALSO

[IdenticalMesh](#), [CopyEdge](#), [CopyFace](#), [Copy](#), [EdgeMap](#), [OnePointMap](#), [TwoPointMap](#)

Copy

Copy a mesh between edges, boundaries, domains, or between different meshing sequences. In 2D and 3D, use the attributes [EdgeMap](#), [OnePointMap](#), [TwoPointMap](#) to control the orientation of the source mesh on the destination.

SYNTAX

```

model.component(<ctag>).mesh(<tag>).create(<ftag>,"Copy");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,maptype);

```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Copy")` to copy a mesh between meshing sequences. Any meshing sequence can be used as the source of the operation, whereas the destination sequence cannot contain an imported mesh. The dimension of the source sequence must be less than or equal to the dimension of the destination sequence.

The following properties are available (for 1D meshes, only the `mesh` property is available):

TABLE 4-30: AVAILABLE PROPERTIES FOR COPY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	String none	Native sequence (none in 1D)	Specifies the source mesh.
copymethod	auto singlecopy arraycopy	auto	Type of copy operation.
dimension	all, 1, 2, or 3 (in 3D)	2 in 2D, 3 in 3D	Specifies the dimension for the operation. all means that the entire mesh should be copied.
source	Selection	Empty	Specifies the selection of source entities.
buildsource	on off	off	Build source mesh automatically.
destination	Selection	Empty	Specifies the selection of destination entities.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.

TABLE 4-30: AVAILABLE PROPERTIES FOR COPY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.

Use the properties `source` and `destination` to specify the geometric entities of the source and destination (except when the dimension is set to copy the entire geometry). The `copymethod` property determines which type of copy is used: single copy (all-to-one), array copy (one-to-one), or automatic detection. The value `auto` lets the software choose between single copy (all-to-one), array copy (one-to-one), or a mixture of the two.

EXAMPLE

The following example shows how to use the Copy feature with a modified geometry from an imported mesh:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence geom1 = model.component("comp1").geom().create("geom1", 2);
geom1.create("c1", "Circle");
MeshSequence mesh1 = model.component("comp1").mesh().create("mesh1", "geom1");
mesh1.run();

model.component().create("comp2");
GeomSequence geom2 = model.component("comp2").geom().create("geom2", 2);
GeomFeature imp1 = geom2.create("imp1", "Import");
imp1.set("type", "mesh");
imp1.set("mesh", "mesh1");
GeomFeature r1 = geom2.create("r1", "Rectangle");
r1.set("size", new String[]{"3", "3"});
r1.set("base", "center");

MeshSequence mesh2 = model.component("comp2").mesh().create("mesh2", "geom2");
MeshFeature copy1 = mesh2.create("copy1", "Copy");
copy1.set("mesh", "mesh1");
copy1.set("dimension", 2);
copy1.selection("source").set(1);
copy1.selection("destination").set(2);
mesh2.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
geom1 = model.component('comp1').geom.create('geom1', 2);
geom1.create('c1', 'Circle');
mesh1 = model.component('comp1').mesh.create('mesh1', 'geom1');
mesh1.run();

model.component.create('comp2');
geom2 = model.component('comp2').geom.create('geom2', 2);
imp1 = geom2.create('imp1', 'Import');
imp1.set('type', 'mesh');
imp1.set('mesh', 'mesh1');
r1 = geom2.create('r1', 'Rectangle');
r1.set('size', {'3', '3'});
r1.set('base', 'center');

mesh2 = model.component('comp2').mesh.create('mesh2', 'geom2');
copy1 = mesh2.create('copy1', 'Copy');
copy1.set('mesh', 'mesh1');
copy1.set('dimension', 2);
copy1.selection('source').set(1);
```

```
copy1.selection('destination').set(2);
mesh2.run;
```

SEE ALSO

[IdenticalMesh](#), [CopyEdge](#), [CopyFace](#), [CopyDomain](#), [EdgeMap](#), [OnePointMap](#), [TwoPointMap](#)

CornerProp

To override the corner settings of [BndLayer](#) for a selection of corners.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"CornerProp");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"CornerProp")` to override the corner settings for the feature `<ftag>` that can be of the type `BndLayer`.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the vertex selection in 2D and the vertex or edge selection in 3D. The selection is empty by default.

The following properties are available:

TABLE 4-31: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
cornerhandling	trim split none	trim	Specify how to handle sharp corners for vertices in 2D and edges in 3D.
cornerhandlingvtx	trim none	trim	Specify how to handle sharp corners for vertices in 3D. Used if the parent <code>BndLayer</code> feature has a domain selection.
splitcondition	on off	off	Specify if to use an angle condition for the split. Used if <code>cornerhandling</code> is split. Not supported for vertices in 3D when the parent <code>BndLayer</code> feature has a domain selection.
splitdivangle	double	100	Maximum angle for split. Used if <code>cornerhandling</code> is split.
splitminangle	double	240	Split for angles greater than specified value. Used if <code>splitcondition</code> is on.
trimcondition	on off	off	Specify if to use angle conditions for the trimming. Used if <code>cornerhandling</code> is trim. Not supported for vertices in 3D when the parent <code>BndLayer</code> feature has a domain selection.
trimmaxangle	double	50	Trim for angles less than specified value. Used if <code>trimcondition</code> is on.
trimminangle	double	240	Trim for angles greater than specified value. Used if <code>trimcondition</code> is on.

It can be useful to filter out the corners of `CornerProp` in a [CornerRefinement](#) feature to either include them in the refinement if `trim` is used, or to exclude them if `split` or `none` is used.

SEE ALSO

[BndLayer](#), [BndLayerProp](#), [CornerRefinement](#)

CornerRefinement

Decrease element size at sharp corners.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CornerRefinement");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).vmesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"CornerRefinement");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"CornerRefinement")` to decrease the element size defined in the sequence at vertices in 2D and edges in 3D that define a sharp corner. Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"CornerRefinement")` to decrease the element size for the feature `<ftag>` that can be any of the types `Edge`, `FreeQuad`, `FreeTri`, or `FreeTet`.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` or `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the domain selection. If you do not specify any selection, the feature is defined on the entire geometry.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection("corner").set()` to specify a selection of corners to include or exclude from the automatic selection of corners.

The following properties are available:

TABLE 4-32: VALID PROPERTIES.

NAME	VALUE	DEFAULT	DESCRIPTION
boundary	Selection		Boundary selection.
corner	Selection		Corners to include or exclude.
filter	include exclude		Exclude or include only selected corners.
minangle	double	240 [deg]	Minimum angle between two boundaries (sharp corner) for decreasing the element size.
refinement	double	0.25 in 2D, 0.35 in 3D	Factor multiplying the element size at sharp corners. The valid range is [0 1].
usefilter	on off	off	Filter implicitly defined corners.

SEE ALSO

[BndLayer](#), [BndLayerProp](#), [CornerProp](#)

CreateDomains

Create a domain for each (connected) finite void region that is defined by a 3D mesh that defines its own geometric model, such as an imported mesh. For more information, see [Geometric Model](#).

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CreateDomains");
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>, "CreateDomains")` to create a domain for each (connected) finite void region that is defined by an imported 3D mesh. There are no additional selections or properties for the CreateDomains operation.

TABLE 4-33: VALID PROPERTIES

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colordom	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolordom property. Used if seldom is on.
customcolordom	RGB-triplet	Next available theme color	The color to use. Active when colordom is set to custom.
seldom	on off	on	Specifies if to create a selection of resulting domains.
seldomshow	on off	on	Show domain selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if seldom is on.

SEE ALSO

[Import](#), [CreateEdges](#), [CreateFaces](#), [CreateVertices](#)

CreateEdges

Create geometrical edges of a selection of mesh edges or between two vertices.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>, "CreateEdges");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property, <value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>, "CreateEdges")` to create additional edges in a mesh that defines its own geometric model. For more information, see [Geometric Model](#).

Choose between creating new meshed edges between vertices or converting existing mesh edges into edge elements. In the general case, converting mesh edges is most easily done by clicking in the Graphics window, as the midpoint coordinates must be exact.

When specifying start/end vertices, specify the size distribution in four different ways: by specifying the number of elements only, by specifying the maximum element size, by specifying the element distribution explicitly, or by specifying the number of elements together with properties determining the distribution of the elements. The property type determines which of the four alternatives you want to use.

The following properties are available:

TABLE 4-34: AVAILABLE PROPERTIES FOR CREEDEGES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
edgespec	vertices meshedge	vertices	Specifies how to create edges. Either give selections of start/end vertices, or give exact coordinates of a mesh edge's midpoint.
start	Selection		Starting vertices to create edges from when edgespec is vertices.

TABLE 4-34: AVAILABLE PROPERTIES FOR CREATEEDGES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
end	Selection		End vertices to create edges to when edgespec is vertices.
type	size number explicit predefined	size	Size distribution type when edgespec is vertices.
coordsel	double[]	{}	Enter exact midpoint coordinates of mesh edge when edgespec is meshedges.
groupcontang	on off	off	Specifies if a selection of a mesh edge is propagated to tangent edges. Used if edgespec is meshedge.
angletol	double	10	Specifies which mesh edges are considered tangent when groupcontang is on.
seldom	on off	off	Specifies if to create a selection of resulting small domains. Used in 2D.
seledg	on off	off	Specifies if to create a selection of resulting edges.
selfac	on off	off	Specifies if to create a selection of resulting small faces. Used in 3D.

TABLE 4-35: AVAILABLE ADDITIONAL PROPERTY WHEN TYPE IS SET TO SIZE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
size	double	0.1*(size of bounding box) (3D); (size of bounding box)/15 (2D)	Maximum element size.

TABLE 4-36: AVAILABLE ADDITIONAL PROPERTY WHEN TYPE IS NUMBER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
numelem	integer	1	Number of elements.

TABLE 4-37: AVAILABLE ADDITIONAL PROPERTIES WHEN TYPE IS EXPLICIT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
explicit	double[]	{0, 1}	Specify the relative placement of vertices along the edge.
reverse	on off	on	Reverse the direction of the explicit distribution.

TABLE 4-38: AVAILABLE ADDITIONAL PROPERTIES WHEN TYPE IS PREDEFINED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
elemcount	integer	5	Number of elements.
elemratio	double	1	Specify the ratio in size between the last element and first element along the edge.
method	arithmetic geometric	arithmetic	Specifies if the element size is linearly growing (arithmetic sequence) or exponentially growing (geometric sequence).
reverse	on off	off	Specify if the distribution is defined in the opposite edge direction for the edge in the selection with lowest index.
symmetric	on off	off	Specify if the distribution is made symmetric.

TABLE 4-39: AVAILABLE ADDITIONAL PROPERTY WHEN SELEDG IS SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
seledgshow	on off	on	Show edge selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if seledg is on.

TABLE 4-40: AVAILABLE ADDITIONAL PROPERTIES WHEN SELFAC AND SELDOM ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorcompl	none custom integer between 1 and the number of colors in the current theme	none	The color of the entity selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorcompl property. Used if selcompl is on.
colorfac	none custom integer between 1 and the number of colors in the current theme	none	The color of the entity selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorfac property. Used if selfac is on.
customcolorcompl	RGB-triplet	Next available theme color	The color to use. Active when colorcompl is set to custom.
customcolorfac	RGB-triplet	Next available theme color	The color to use. Active when colorfac is set to custom.
selareafraction	double	0.1	Maximum area of a small face (3D) or domain (2D) as a fraction of the total area of the input faces. Used when selfac or seldom, respectively, is on.
selcompl	on off	off	Specifies if to create a complement selection of resulting faces (3D) or domains (2D) when selfac or seldom, respectively, is on.
selcomplshow	on off	on	Show entity selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selcompl is on.
seldomshow	on off	on	Show domain selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if seldom is on.
selfac	on off	off	Specifies if to create a selection of resulting small faces. Used in 3D.
selfacshow	on off	on	Show face selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selfac is on.

SEE ALSO

[CreateDomains](#), [CreateFaces](#), [CreateVertices](#)

CreateFaces

Create additional faces in a 3D mesh that defines its own geometric model. For more information, see [Geometric Model](#).

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CreateFaces");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"CreateFaces")` to create additional faces by selecting bounding edges. Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the bounding edges. If you do not specify the selection, it is left empty.

The following property is available:

TABLE 4-41: AVAILABLE PROPERTY FOR CREATEFACES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
createdom	on off	off	Specify if to create domains for each (connected) finite void region.
groupadjedg	on off	on	Group adjacent edges.
seldom	on off	on	Specifies if to create a selection of resulting domains.
selfac	on off	on	Specifies if to create a selection of resulting faces.

TABLE 4-42: AVAILABLE ADDITIONAL PROPERTIES WHEN SELDOM AND SELFAC ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colordom	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolordom property.
colorfac	none custom integer between 1 and the number of colors in the current theme	none	The color of the face selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorfac property.
customcolordom	RGB-triplet	Next available theme color	The color to use. Active when colordom is set to custom.
customcolorfac	RGB-triplet	Next available theme color	The color to use. Active when colordom is set to custom.
seldomshow	on off	on	Show domain selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if seldom is on.
selfacshow	on off	on	Show face selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selfac is on.

SEE ALSO

[Import](#), [CreateDomains](#), [CreateEdges](#), [CreateVertices](#), [FillHoles](#)

CreateVertices

Create vertices in a mesh that defines its own geometric model.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"CreateVertices");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"CreateVertices")` to create additional vertices in a mesh that defines its own geometric model. For more information, see [Geometric Model](#).

Convert existing mesh vertices into vertex elements, refine existing elements by inserting points into them, or create new vertex elements in void or in unmeshed domains. In the general case, converting mesh vertices is most easily done setting the `coord` value due to the snapping tolerance. This option also makes it possible to create vertices that are not connected to an existing mesh.

The following properties are available:

TABLE 4-43: AVAILABLE PROPERTIES FOR CREATEVERTICES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
vertexspec	coord meshvertex	coord	Specify if to enter approximate point coordinates or to specify the exact mesh vertex coordinates.
coordformat	component point	component	Specify if to enter arrays of the x, y, and z components or if to specify a matrix containing all points. Use when vertexspec is coord.
x	double[]	{}	Enter x-coordinates when coordformat is component.
y	double[]	{}	Enter y-coordinates when coordformat is component.
z	double[]	{}	Enter z-coordinates (3D only) when coordformat is component.
points	double[][]	{}	Enter the point coordinates when coordformat is point. Specify, x, y, and z (3D only) values.
relnapto1	double	0.001	The snapping tolerance relative to diameter of the mesh bounding box diameter. Enter value between 0 and 1. Is used when vertexspec is coord.
coordsel	double[][]	{}	Enter exact coordinates of mesh vertex when vertexspec is meshvertex. Specify, x, y, and z (3D only) values.
selvtx	on off	on	Specifies if to create a selection of resulting vertices.
selvtxshow	on off	on	Show vertex selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selvtx is on.

SEE ALSO

[CreateDomains](#), [CreateEdges](#), [CreateFaces](#)

Cylinder

Split geometric entities of an imported mesh by a cylinder.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Cylinder");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Cylinder")` to split geometric entities of an imported 3D mesh by an element set defined by a cylinder.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify geometric entities to split. If you do not specify the selection, the feature operates on the entire geometry.

The following properties are available:

TABLE 4-44: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
r	double	1	Radius of the cylinder.
top	double	inf	Coordinate of upper boundary circle in local coordinate system.
bottom	double	-inf	Coordinate of lower boundary circle in local coordinate system.
pos	double[]	{0,0,0}	Position of the cylinder.
axistype	x y z Cartesian spherical	z	Coordinate system used for axis. The value is synchronized with axis.
axis	double[]	{0,0,1}	Direction of the axis. Vector has length 3 if axistype is cartesian, and length 2 if axistype is spherical.
condition	allvertices somevertex	allvertices	Condition for inclusion of an element.
selinside	on off	off	Create selection of all entities inside the cylinder.
seloutside	on off	off	Create selection of all entities outside the cylinder.

TABLE 4-45: AVAILABLE ADDITIONAL PROPERTIES WHEN SELINSIDE AND SELOUTSIDE ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorinside	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorinside property.
coloroutside	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcoloroutside property.
customcolorinside	RGB-triplet	Next available theme color	The color to use. Active when colorinside is set to custom.
customcoloroutside	RGB-triplet	Next available theme color	The color to use. Active when coloroutside is set to custom.
selinsideshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.
seloutsideshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.

SEE ALSO

[Import](#), [Ball](#), [Box](#), [DetectFaces](#), [LogicalExpression](#)

DeleteEntities

Delete geometric entities from an imported mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"DeleteEntities");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"DeleteEntities")` to delete geometric entities from an imported 2D or 3D mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify geometric entities to delete.

The following property is available:

TABLE 4-46: AVAILABLE PROPERTY FOR DELETEENTITIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
unmesheddom	boolean	false	Specifies whether to remove domain elements, while keeping the domains as unmeshed as well as keeping adjacent meshed boundaries. Only available in 3D and only valid when dimension of the selection is 3.
deleteadj	boolean	true	Specifies if the operation removes lower dimensional adjacent entities. Ignored if unmesheddom is valid and set to true

SEE ALSO

[Import](#), [JoinEntities](#)

DetectFaces

Split geometric boundary entities by detecting faces in the mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"DetectFaces");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"DetectFaces")` to split geometric boundary entities of an imported 3D mesh by detecting shapes in the mesh that are likely to constitute faces.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the boundary entities to split. If you do not specify the selection, it is left empty.

The following properties are available:

TABLE 4-47: VALID PROPERTY/VALUE PAIRS FOR DETECTFACES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
detectadjfillets	on off	on	Whether to detect cylindrical faces adjacent to the detected planar faces.
detectfacesplanar	on off	on	Whether to detect planar faces.
facemaxangle	double	40 degrees	Maximum tolerated angle between neighboring boundary elements in the same face.

TABLE 4-47: VALID PROPERTY/VALUE PAIRS FOR DETECTFACES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
planarfacemaxangle	double	0.6 degrees	Maximum tolerated angle between neighboring boundary elements in the same planar face.
planarfaceminareafraction	double	0.005	Minimum relative area for a planar face to be created.

SEE ALSO

[Import](#), [Ball](#), [Box](#), [Cylinder](#), [LogicalExpression](#)

Distribution

Mesh attribute to specify an element distribution along an edge (3D), a boundary (2D), or along the direction to sweep a mesh in a domain (3D).

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Distribution");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"Distribution");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Distribution")` to specify element distribution properties in the sequence. Use

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"Distribution")` to specify element distribution properties for the feature `<ftag>` that can be any of the types `Edge`, `FreeQuad`, `FreeTri`, `FreeTet`, `Map`, or `Sweep`.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` or `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the edge (3D), boundary (2D), or domain selection (1D and 3D).

You can specify a mesh element distribution in three different ways: by specifying the number of elements only, by specifying the number of elements together with properties determining the distribution of the elements, or by specifying the element distribution explicitly. The property `type` determines which of the three alternatives you want to use. However, you need not set `type` manually since it is automatically updated when you set a property from one of the three groups below.

TABLE 4-48: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
type	number explicit predefined	number	Specifies the distribution method: fixed number of elements, explicit, user-defined element distribution, or predefined distribution.

The following group of properties are available:

TABLE 4-49: AVAILABLE PROPERTIES WHEN TYPE IS NUMBER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
numelem	integer	5	Number of elements.
equidistant	on off	off	Specifies if to create an equidistant mesh along edges in 3D and boundaries in 2D.

Use the property `numelem` to specify the number of elements, but let the algorithm determine a suitable distribution, taking geometry and surrounding mesh into account.

TABLE 4-50: AVAILABLE PROPERTIES WHEN TYPE IS EXPLICIT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
explicit	double[]	{0, 1}	Specifies the relative placement of vertices along the edge.
reverse	on off	off	Reverse the direction of the explicit distribution.

Use the `explicit` property to specify an explicit element distribution. The value of this property is an array with increasing values starting at 0.

TABLE 4-51: AVAILABLE PROPERTIES WHEN TYPE IS PREDEFINED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
elemcount	integer	5	Number of elements.
elemratio	double	1	Specifies the ratio in size between the last element and first element along the edge.
method	arithmetic geometric	arithmetic	Specifies if the element size is linearly growing (arithmetic sequence) or exponentially growing (geometric sequence).
reverse	on off	off	Specifies if the distribution is defined in the opposite edge direction for the edge in the selection with lowest index.
symmetric	on off	off	Specifies if the distribution is made symmetric.

When the type is predefined, the distribution is calculated from the parameters given above.

This Distribution feature can be assigned to edges in 2D and 3D, or domains in 1D and 3D. The `FreeTet`, `FreeTri`, `FreeQuad`, `Edge`, and `Map` features use this property when defined on edges, the `Sweep` feature uses this property defined on domains.

SEE ALSO

[Scale, Size](#)

Edge

Create an edge mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Edge");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Edge")` to create an edge mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the edge selection. If you do not specify a selection, the feature creates a mesh on the remaining entities in 1D. In 3D and 2D, the default selection is empty.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype)` to add **Size** or **Distribution** attribute features.

The following properties are available:

TABLE 4-52: AVAILABLE PROPERTIES FOR EDGE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.

SEE ALSO

[Distribution](#), [Vertex](#), [Size](#)

EdgeGroup

Define edge groups for mapped meshes.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"EdgeGroup");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"EdgeGroup")` to define edge groups for the Map feature <ftag>.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the domain.

The following properties are available:

TABLE 4-53: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
edge1	Selection		First group of edges.
edge2	Selection		Second group of edge.
edge3	Selection		Third group of edge.
edge4	Selection		Fourth group of edge.

The value of each property is an edge selection that combines edges to defines a logical side of the corresponding domain (in 2D) or boundary (in 3D). No specific ordering of the edges is required.

SEE ALSO

[Distribution](#), [Map](#), [Size](#)

Specify an edge map for the copy operations and the IdenticalMesh feature to control the orientation of the source mesh on the destination.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag1>, "EdgeMap");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property, <value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>, "EdgeMap")` to define an edge mapping for CopyFace or CopyDomain feature <ftag>.

The following properties are available:

TABLE 4-54: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
direction	auto same opposite	auto	The direction of dstedge relative srcedge.
dstedge	Selection		Edge on destination face/domain.
srcedge	Selection		Edge on source face/domain.

Use the EdgeMap feature if you need to control how the source and destination faces/domains are matched in a copy mesh operation and in an IdenticalMesh feature. When this feature is present, the source mesh is transformed so that srcedge is mapped onto dstedge. The relative orientation of the edges is specified by the direction property.

EXAMPLE

Create a block and then mesh Face 1 with a fine mesh on Edge 1. Copy this mesh to face 6 and ensure that the fine mesh of Edge 1 ends up on Edge 12.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.run();

m.create("ftri1", "FreeTri");
m.feature("ftri1").selection().set(new int[]{1});
m.feature("ftri1").create("size1", "Size");
m.feature("ftri1").feature("size1").selection().geom("geom1", 1).set(new int[]{1});
m.feature("ftri1").feature("size1").set("hmax", "0.01");

m.create("cpf1", "CopyFace");
m.feature("cpf1").selection("source").set(new int[]{1});
m.feature("cpf1").selection("destination").set(new int[]{6});
m.feature("cpf1").create("em1", "EdgeMap");
m.feature("cpf1").feature("em1").selection("dstedge").set(new int[]{1});
m.feature("cpf1").feature("em1").selection("dstedge").set(new int[]{12});

m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
```

```

model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('blk1', 'Block');
g.run;

m.create('ftri1', 'FreeTri');
m.feature('ftri1').selection().set(1);
m.feature('ftri1').create('size1', 'Size');
m.feature('ftri1').feature('size1').selection().geom('geom1', 1).set(1);
m.feature('ftri1').feature('size1').set('hmax', '0.01');

m.create('cpf1', 'CopyFace');
m.feature('cpf1').selection('source').set(1);
m.feature('cpf1').selection('destination').set(6);
m.feature('cpf1').create('em1', 'EdgeMap');
m.feature('cpf1').feature('em1').selection('dstedge').set(1);
m.feature('cpf1').feature('em1').selection('dstedge').set(12);

m.run;

```

SEE ALSO

[Copy](#), [CopyFace](#), [CopyDomain](#), [OnePointMap](#), [TwoPointMap](#), [IdenticalMesh](#)

ExplicitSelection

To create a named selection for individual geometric entities on the geometric entity level chosen. Supported for meshes that define their own geometric model.

SYNTAX

```

model.component(<ctag>).mesh(<tag>).create(<ftag>,"ExplicitSelection");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);

```

DESCRIPTION

Supported for meshes that define their own geometric model, such as imported meshes. For more information, see [Geometric Model](#).

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"ExplicitSelection")` to create a named selection for one or more entities.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the input entities for the selection.

The following properties are available:

TABLE 4-55: AVAILABLE PROPERTIES FOR EXPLICITSELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
angletol	double	15	Angle tolerance for continuity evaluation. Used when groupcontang is on.
groupcontang	on off	off	Specify to group faces (in 3D) or edges by continuous tangent.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.

TABLE 4-55: AVAILABLE PROPERTIES FOR EXPLICITSELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.
selshow	on off	on	Show selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part.

For more information, see the section [Selections](#).

SEE ALSO

[UnionSelection](#), [IntersectionSelection](#), [DifferenceSelection](#), [ComplementSelection](#), [BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#), [AdjacentSelection](#)

FillHoles

Repair an imported surface mesh that might have minor holes by filling those holes.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"FillHoles");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"FillHoles")` to fill small holes on surfaces (boundaries) of an imported mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the faces for which the holes are filled. If you do not specify the selection, it is left empty.

The following properties are available:

TABLE 4-56: AVAILABLE PROPERTIES FOR FILLHOLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colordom	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
createdom	on off	off	Specify if to create domains for each (connected) finite void region.
customcolordom	RGB-triplet	Nextavailable theme color	The color to use. Active when colordom is set to custom.
fillholestol	auto manual	auto	Use an automatic or manual tolerance for the maximum perimeter of a hole to be filled.
join	on off	on	Join with filled holes.
perimeter	double scalar	0.01	The maximum perimeter of a hole to be filled when fillholestol is set to manual.
seldom	on off	on	Specifies if to create a selection of resulting domains.
seldomshow	on off	on	Show domain selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if seldom is on.

SEE ALSO

[Import](#), [CreateFaces](#)

FixedMesh

Use `FixedMesh` to keep the mesh fixed on edges (3D) or boundaries (2D) while remeshing the faces (3D) or domains (2D).

SYNTAX

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"FixedMesh");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"FixedMesh")` to keep meshes on a selection of edges (3D) or boundaries (2D) fixed for the feature `<ftag1>` that can be any of the types [RemeshFaces](#) or [RemeshDomains](#), respectively.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the edge (3D) or boundary (2D) selection.

SEE ALSO

[RemeshDomains](#), [RemeshFaces](#)

FreeQuad

Create an unstructured quadrilateral mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"FreeQuad");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"FreeQuad")` to create an unstructured quadrilateral mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the domain (boundary in 3D) selection. If you do not specify any selection the feature creates a mesh on the remaining geometric entities in 2D. In 3D, the default selection is empty.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype)` to add attribute features.

The following properties are available:

TABLE 4-57: AVAILABLE PROPERTIES FOR FREEQUAD.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
defectremoval	double	1.0	Relative size factor for identification local defects. Only available for imported meshes in 3D when method is quad or any of the legacy versions.
method	quaddominant quad legacy52 legacy52a legacy54	quaddominant	The quad meshing algorithm to use.
simplifymesh	on off	on	Boolean specifying if the original mesh should be simplified. Only available for imported meshes in 3D when method is quad or any of the legacy versions.
simplifytol	double	0.01	Relative simplification tolerance. Only available for imported meshes in 3D when method is quad or any of the legacy versions.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.
xscale	double	1	Scale geometry in x direction before meshing.
yscale	double	1	Scale geometry in y direction before meshing.
zscale	double	1	Scale geometry in z direction before meshing.

See the [FreeTet](#) feature for more information on the properties.

The following attribute features are used:

TABLE 4-58: ATTRIBUTE FEATURES FOR FREEQUAD.

FEATURE	REMARKS
CornerRefinement	Used when generating mesh on faces. Only available when generating mesh for a geometry.
Distribution	Used when defined on edges.
Size	All properties are used.
SizeExpression	All properties are used. Only available when generating mesh for a geometry.

For imported meshes, there is a default size attribute under the FreeQuad feature with the tag `size`. You can access it using `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature("size")`.

COMPATIBILITY

See [FreeTet](#).

SEE ALSO

[Size](#), [SizeExpression](#), [CornerRefinement](#), [Distribution](#), [IdenticalMesh](#), [FreeTri](#), [RemeshFaces](#)

FreeTet

Create an unstructured tetrahedral mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"FreeTet");  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);  
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"FreeTet")` to create an unstructured tetrahedral mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the domain selection. If you do not specify any selection the feature creates a mesh on the remaining geometric entities.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype)` to add attribute features.

The following properties are available:

TABLE 4-59: AVAILABLE PROPERTIES FOR FREETET.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
method	auto del dellegacy52	auto	Delaunay meshing algorithm to use.
optlevel	basic medium high	basic	Optimization level for the mesh element quality.
optcurved	on off	on	Avoid inverted curved mesh elements.
optlarge	on off	off	Avoid mesh elements that are too large.
optsmall	on off	off	Avoid mesh elements that are too small.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.
xscale	double	1	Scale geometry in x direction before meshing.
yscale	double	1	Scale geometry in y direction before meshing.
zscale	double	1	Scale geometry in z direction before meshing.

The mesher does not stop if it encounters an error. Instead, it continues to mesh remaining entities. Before finishing, all errors are collected and reported as feature problems. You can use the output to visually examine the partial mesh; this can help you understand what the problems are and how they can be fixed.

The `method` property determines the Delaunay tessellation algorithm to use. The default is `auto`, which makes the `FreeTet` mesh generator determine the best algorithm for each domain. The `del1` Delaunay algorithm is the Delaunay algorithm used in earlier COMSOL versions. The `del2` Delaunay algorithm is an alternative version of the algorithm, which under some conditions can modify the boundary mesh to simplify the meshing.

The properties `xscale`, `yscale`, and `zscale` specify scalar factors in each axis direction that the geometry is scaled by before meshing. The resulting mesh is then scaled back to fit the original geometry. The values of other properties correspond to the scaled geometry. By default, no scaling is done.

The following attribute features are used:

TABLE 4-60: ATTRIBUTE FEATURES FOR FREETET.

FEATURE	REMARKS
CornerRefinement	Used when generating mesh on faces. Only available when generating mesh for a geometry,
Distribution	Used when defined on edges. Only available when generating mesh for a geometry.
Size	All properties are used.
SizeExpression	All properties are used. Only available when generating mesh for a geometry.

For imported meshes, there is a default size attribute under the FreeTet feature with the tag size. You can access it using `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature("size")`.

SEE ALSO

[Size](#), [SizeExpression](#), [CornerRefinement](#), [Distribution](#), [IdenticalMesh](#), [FreeTri](#)

FreeTri

Create an unstructured triangular mesh in domains in 2D and on faces in 3D. Can also be used to remesh faces in a 3D surface mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"FreeTri");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"FreeTri")` to create an unstructured triangular mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the domain (boundary in 3D) selection. If you do not specify any selection the feature creates a mesh on the remaining geometric entities in 2D. In 3D, the default selection is empty.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype)` to add attribute features.

The following properties are available:

TABLE 4-61: AVAILABLE PROPERTIES FOR FREETRI.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
defectremoval	double	1.0	Relative size factor for identification local defects. Only available 3D meshes that define their own geometric model.
method	auto af del	auto	Triangulation method to use.
narrowreg	on off	off	Specifies if to resolve the mesh with respect to adjacent narrow domain regions.
simplifymesh	on off	on	Boolean specifying if the original mesh should be simplified. Only available 3D meshes that define their own geometric model.
simplifytol	double	0.01	Relative simplification tolerance. Only available 3D meshes that define their own geometric model.

TABLE 4-61: AVAILABLE PROPERTIES FOR FREETRI.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.
xscale	double	1	Scale geometry in x direction before meshing.
yscale	double	1	Scale geometry in y direction before meshing.
zscale	double	1	Scale geometry in z direction before meshing.

Use the property method to specify the method used to triangulate domains in 2D and faces in 3D. A Delaunay based method is used if the property is set to `del` and an advancing front method is used if the property is set to `af`. If `method` is set to `auto`, the program tries to choose the best method for each geometric entity.

The following attribute features are used:

TABLE 4-62: ATTRIBUTE FEATURES FOR FREETRI.

FEATURE	REMARKS
CornerRefinement	Used when generating mesh on faces. Only available when generating mesh for a geometry.
Distribution	Used when defined on edges.
Size	All properties are used.
SizeExpression	All properties are used. Only available when generating mesh for a geometry.

For imported meshes, there is a default size attribute under the FreeTri feature with the tag `size`. You can access it using `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature("size")`.

COMPATIBILITY

See [FreeTet](#).

SEE ALSO

[Size](#), [SizeExpression](#), [CornerRefinement](#), [Distribution](#), [IdenticalMesh](#), [FreeTet](#), [FreeQuad](#), [RemeshFaces](#)

IdenticalMesh

Use this attribute to generate identical mesh on pairs of faces (3D) or edges.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"IdenticalMesh");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"IdenticalMesh")` to generate identical mesh on pairs of faces or edges in meshes that are conforming with geometry.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag1>).create(<ftag>,"IdenticalMesh")` generate identical mesh on pairs of faces or edges in meshes that define their own geometric models.

IdenticalMesh is then a local attribute to the feature `<ftag1>` that can be any of the types [RemeshFaces](#), and [RemeshEdges](#). For more information, see [Geometric Model](#).

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property).set()` or `model.component(<ctag>).mesh(<tag>).feature(<ftag1>).feature(<ftag>).property).set()` to specify the geometric entity selection for the properties group1 and group2.

The following properties are available:

TABLE 4-63: FEATURE PROPERTIES DEFINED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
dimension	1 2	2	Entity level of the groups. Only used in 3D.
group1	Selection		Selection first entity group.
group2	Selection		Selection second entity group.

SEE ALSO

[OnePointMap](#), [TwoPointMap](#), [EdgeMap](#), [FreeTet](#), [FreeTri](#), [FreeQuad](#), [RemeshFaces](#), [RemeshDomains](#)

Import

Import mesh from a file, from another meshing sequence, from a geometry sequence, or from a [Filter \(Dataset\)](#) or [Partition](#) dataset. You can import a mesh from a COMSOL Multiphysics file and Sectionwise file. In 3D, you can also import meshes from 3MF, NASTRAN, PLY, STL, and VRML files. In 2D, you can also import meshes from NASTRAN (the third coordinate must then be the same for all mesh points) and COMSOL's native formats.

Use a [Transform](#) attribute to position, rotate, or scale an imported mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Import");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).importData();
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Import")` to import a mesh into a meshing sequence that defines its own geometrical model. See the section [Geometric Model](#) for more information. If the sequence already contains a mesh, the imported mesh is added to the existing mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).importData()` to import the file of the `<ftag>` Import feature again.

Use

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).setString(property,<value>,index);
```

to select if to keep a selection and show it in physics or not. The `index` is the (0-based) number of the selection in the table.

The following mesh formats are supported:

TABLE 4-1: SUPPORTED MESH FORMATS.

FORMAT	FILE EXTENSION
COMSOL Multiphysics text file	.mphant
COMSOL Multiphysics binary file	.mphbin
NASTRAN file	.nas .bdf .dat
Sectionwise file	.txt
STL text file	.stl
STL binary file	.stl
PLY text file	.ply
PLY binary file	.ply
3MF file	.3mf
VRML file	.vrl .vrm

The available properties are listed in the tables below. [Table 4-2](#) lists properties common for many of the import sources:

TABLE 4-2: AVAILABLE PROPERTIES FOR THE MESH IMPORT.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
source	3mf file geom sequence dataset nastran native ply stl vrm sectionwise	file	Source for the import. The value file specifies any supported file type. The value native specifies COMSOL Multiphysics file import.
facepartition	auto minimal detectfaces manual	auto	Boundary partitioning algorithm. detectfaces is only available in 3D, and manual is only available in 2D. Available when source is native, nastran, stl, sectionwise, ply, 3mf, vrm, and dataset.
filename	String		Specify the filename when source is file, native, nastran, stl, sectionwise, ply, or 3mf.
keepsel_dom	on off	on	Keep domain selection.
keepsel_bnd	on off	on	Keep boundary selection.
keepsel_edg	on off	on	Keep edge selections. Only in 3D.
keepsel_pnt	on off	on	Keep point selection.
showsel_dom	on off	on	Show domain selection in physics, materials, and so on. Used if keepsel_dom is on.
showsel_bnd	on off	on	Show domain selection in physics, materials, and so on. Used if keepsel_bnd is on.
showsel_edg	on off	on	Show domain selection in physics, materials, and so on. Used if keepsel_edg is on. Only in 3D.
showsel_pnt	on off	on	Show domain selection in physics, materials, and so on. Used if keepsel_pnt is on.

The following tables list additional properties depending on which source property is used.

TABLE 4-3: ADDITIONAL PROPERTIES WHEN SOURCE IS NASTRAN.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
allowshellpartition	on off	on	Specifies whether the boundary entities that are defined based on the data in the file can be split into smaller parts.
data	all mesh	all	Specifies the data to import from the NASTRAN file.
domelemnasttran	on off	on	Specifies if domain elements are imported. Use when source is nastran.
linearelem	on off	off	Specifies if the elements in are imported as linear elements. Available in 2D and 3D.
materialsplitted	on off	on	Specifies if material data in the NASTRAN file is used to determine the domain partitioning of the domain elements.
selcreation	on off	on	Specifies whether selections corresponding to the groups of domain and boundary elements in the NASTRAN file should be created.

TABLE 4-4: ADDITIONAL PROPERTIES WHEN SOURCE IS 3MF, PLY, STL, OR VRML.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
createdom	on off	off	Specify if to create domains for each (connected) finite void region. Use when source is 3mf, ply, stl, or vrml.
selectionstl	on off	on	Create boundary selections from an STL file.
stltolabs	Positive scalar	1e-8	Absolute tolerance for STL import, when stltoltype is set to absolute.
stltolrel	Positive scalar, 1.0 or smaller	1e-8	Relative tolerance for STL import, when stltoltype is set to relative.
stltoltype	auto relative absolute	auto	STL file import tolerance type: automatic, relative, or absolute.
mergepoints	on off	off	Specify if to merge coinciding points. Use when source is 3mf, ply, or vrml.
toltype	auto relative absolute	auto	Specifies the type of tolerance to enter. Use when mergepoints is on.
reltol	Positive scalar	1e-8	Relative tolerance for merging points. Use when toltype is set to relative
abstol	Positive scalar	1e-8	Absolute tolerance for merging points. Use when toltype is set to absolute.

TABLE 4-5: ADDITIONAL PROPERTIES WHEN SOURCE IS NATIVE, DATASET, OR SECTIONWISE.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
dataset	String		Specify the tag of the dataset when source is dataset.
domelem	on off	on	Specifies if domain elements are imported. Use when source is native.
domelemdataset	on off	off	Specifies if domain elements are imported. Use when source is dataset.
domelemsectionwise	on off	on	Specifies if domain elements are imported. Use when source is sectionwise.

TABLE 4-5: ADDITIONAL PROPERTIES WHEN SOURCE IS NATIVE, DATASET, OR SECTIONWISE.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
linearelem	on off	off	Specifies if the elements in are imported as linear elements. Available in 2D and 3D when source is native.
selection	on off	on	Import mesh selections when source is native or dataset.

TABLE 4-6: ADDITIONAL PROPERTIES WHEN SOURCE IS SEQUENCE.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
buildsource	on off		Specifies if source mesh is automatically rebuilt and imported when source is sequence. Default is on when sequence specifies a mesh part and off for other meshing sequences.
domelemsequence	on off	on	Specifies if domain elements are imported. Use when source is sequence.
sequence	String		Specify tag of the meshing sequence when source is sequence.
unmesheddom	on off	off	Specifies if unmeshed domains are imported. Use when source is sequence

TABLE 4-7: ADDITIONAL PROPERTIES WHEN SOURCE IS GEOM.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
detail	fine normal coarse	normal	Specifies the level of detail to use when meshtype is visualization.
geom	none String	none	Specify tag of the geometry sequence when source is geom.
meshtype	freetri visualization	freetri	Specifies the triangulation to use for the geometry surfaces when source is geom.
meshsize	1–9	5	Specifies the predefined element size to use when meshtype is freetri. 1 is the finest and 9 is the coarsest predefined size setting. You can use any nonnegative value for the predefined element size. Use Double.POSITIVE_INFINITY to create the coarsest possible mesh.
narrowreg	on off	on	Specifies if narrow domain regions will be resolved by the mesh when meshtype is freetri.
resdetail	on off	on	Specifies if geometric details will be resolved by the mesh when meshtype is freetri.

Properties for STL File Import

The properties `stlto1abs`, `stlto1rel`, `stlto1type`, and `selectionstl` are only used for import of STL files. If `selectionstl` is active, the following tags are assigned to the created selections:

- If a selection is named in the file, its tag is the name.
- If it is unnamed, it gets the tag `sel` + the number of the unnamed selection (started with 1). If there is only one selection and it is unnamed, the tag is just `sel`.

The tags of the imported selections can also be obtained by calling the `outputSelection()` function on the mesh import feature.

The tag of the model selection becomes `<meshing sequence tag>_<import node tag>_<selection tag>`, such as `mesh1_imp1_sel`.

Properties for Meshing Sequence Import

The property `buildsource` is only used when `source` is `sequence`. If `buildsource` is active, the source sequence is always built and the current version of the source mesh is imported. This is required when running a parametric sweep that influences the source meshing sequence. If the property is deactivated, run the method `importData()` to manually build and reimport the source mesh sequence.

Properties for NASTRAN File Import

The properties `linearelem`, `materialsplitted`, and `data` are only used for import of NASTRAN files. The properties `selcreation` and `allowshellpartition` are only used when `materialsplitted` is on.

Additional Properties

`linearelem` specifies if the elements in the NASTRAN or COMSOL Multiphysics file are imported as linear elements. If the value is `on` all imported elements are linear. Otherwise, the order of the imported elements is determined from the order of the elements in the file. The default value is `off`.

`materialsplitted` determines if material data in the file is used (if available) to determine the domain partitioning of the domain elements. If the value is `off` all domain elements in the imported mesh belongs to the same domain if possible. The default value is `off`.

If you set `facepartition` to `minimal`, the operation keeps the original partition from the file (if any), adding minimal partitioning in order to satisfy topological requirements.

If you set `facepartition` to `manual`, you can use the following properties. If you set any of these properties without setting `facepartition` to `manual`, the operation automatically switches `facepartition` to `manual`.

TABLE 4-8: VALID PROPERTY/VALUE PAIRS FOR `FACEPARTITION = MANUAL`.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>faceangle</code>	<code>double</code>	<code>360 degrees</code>	Maximum angle between any two boundary elements in the same face.
<code>minareaplane</code>	<code>double</code>	<code>0.005</code>	Minimum relative area of face to be considered planar.
<code>neighangle</code>	<code>double</code>	<code>40 degrees</code>	Maximum angle between a boundary element and a neighbor that causes the elements to be part of the same boundary domain if possible.
<code>planar</code>	<code>on off</code>	<code>on</code>	Detect planar faces.
<code>planarangle</code>	<code>double</code>	<code>0.6 degrees</code>	Maximum angle between boundary element normal and a neighbor that causes the element to be a part the planar face if possible.

The following properties are available in 3D when `facepartition` is set to `detectfaces`:

TABLE 4-9: VALID PROPERTY/VALUE PAIRS FOR `FACEPARTITION = DETECTFACES`.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>detectedjfillets</code>	<code>on off</code>	<code>on</code>	Whether to detect cylindrical faces adjacent to the detected planar faces.
<code>detectedgesplanar</code>	<code>on off</code>	<code>on</code>	Whether to detect planar edges.
<code>detectedgesstraight</code>	<code>on off</code>	<code>on</code>	Whether to detect straight edges.
<code>detectfacesplanar</code>	<code>on off</code>	<code>on</code>	Whether to detect planar faces.
<code>edgemaxangle</code>	<code>double</code>	<code>60 degrees</code>	Maximum angle between an edge element and a neighbor for the two elements to belong to the same edge.
<code>facemaxangle</code>	<code>double</code>	<code>40 degrees</code>	Maximum tolerated angle between neighboring boundary elements in the same face.

TABLE 4-9: VALID PROPERTY/VALUE PAIRS FOR FACEPARTITION = DETECTFACES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
minlengthtype	auto relative absolute	auto	Minimum edge length type: automatic, relative, or absolute.
minrellength	double	0.01	Minimum relative edge length, relative to size of geometry, if minlengthtype = relative.
minabslength	double	0	Minimum absolute edge length, if minlengthtype = absolute.
planaredgeparam	double	0.5	Parameter for planar edge detection (0–1).
planarfacemaxangle	double	0.6 degrees	Maximum tolerated angle between neighboring boundary elements in the same planar face.
planarfaceminareafraction	double	0.005	Minimum relative area for a planar face to be created.
straightedgeparam	double	0.5	Parameter for straight edge detection (0–1).

The table below specifies the properties for selections imported when source is file, stl, or dataset.

TABLE 4-10: VALID PROPERTY/VALUE PAIRS WHEN SOURCE IS FILE, STL, OR DATASET.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outsel_dom	String array		Names of domain selections.
outsel_bnd	String array		Names of boundary selections.
outsel_edg	String array		Names of edge selections in 3D.
outsel_pnt	String array		Names of point selections in 3D and 2D.

Default values in the arrays are based on the names of the selections in the source. The length of the array can be determined by using

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getStringArray(property).length;
```

If no selection of a certain dimension is imported, the length of the corresponding array is 0.

The values of the selection properties can be set using individual names accompanied by an index between 0 and length-1

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).setIndex(property,<nname>,<index>);
```

where <nname> is the new name for the selection on place <index> in the *property* array. An alternative is to set the entire string array at once

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<narray>);
```

where <narray> is a new array of strings of correct length.

Values in each of the string arrays must be unique, while different string arrays may contain the same value. Values in the string arrays must be nonempty.

The table below specifies the supported NASTRAN bulk data entries.

TABLE 4-11: SUPPORTED NASTRAN BULK DATA ENTRIES.

BULK DATA ENTRY				
CBAR	CBEAM	CORD2C	CQUAD4	GRID
CHEXA	CORD2R	CQUAD8	MAT1	MAT10
CORDIC	CORD2S	CTETRA	PSHELL	PSOLID
CORDIR	CPENTA	CTRIA3	RBE2	RBE3
CORDIS	CPYRAM	CTRIA6		

The RBE entries (RBE2 and RBE3) define a fixed point and one or several dependent points (For RBE3, only the first set of dependent points is taken). Dependent points that are second-order points are ignored. A point selection is created for the fixed points of each RBE entry. In addition, a selection of each entity level may be created for each RBE entry in such a way that each mesh element, whose all first-order vertices are dependent points for the RBE entry, is part of the selection. If a dependent point selection associated with a RBE entry is not empty, the fixed point is also added to the selection.

The NASTRAN bulk data format uses reduced second-order elements; that is, the center node on quadrilateral mesh faces (`quadNode`) and the center node of hexahedral elements (`hexNode`) are missing. Importing a NASTRAN mesh with second-order elements, COMSOL Multiphysics interpolates the coordinates of these missing node points from the surrounding node points using the following formulas: $\text{quadNode} = 0.5 \cdot \text{quadEdgeNodes} - 0.25 \cdot \text{quadCornerNodes}$, where `quadEdgeNodes` is the sum of the coordinates of the surrounding 4 edge nodes and `quadCornerNodes` is the sum of the coordinates of the surrounding 4 corner nodes, and $\text{hexNode} = 0.25 \cdot \text{hexEdgeNodes} - 0.25 \cdot \text{hexCornerNodes}$, where `hexEdgeNodes` is the sum of the coordinates of the surrounding 12 edge nodes and `hexCornerNodes` is the sum of the coordinates of the surrounding 8 corner nodes.



The **Import** feature does not handle NASTRAN files in free field format where the data fields are separated by blanks.

COMPATIBILITY

The source property value `stlvrml` has been replaced by separate `stl` and `vrml` values in version 5.5. The value `stlvrml` is also accepted for backward compatibility. If a filename is given and it ends with `.vrml`, `source` is then set to `vrml`. Otherwise, `source` is set to `stl`.

The `elemsplit` property from earlier versions is no longer available from version 5.3.

For 3D meshing sequences, the setting `manual` of the property `facepartition` in the mesh **Import** feature, as well as all the properties associated with this setting, are deprecated as of COMSOL 5.1 and may be removed in a future version. In COMSOL 5.1, these properties are still available with unchanged behavior for backward compatibility.

SEE ALSO

[Transform](#), [Ball](#), [Box](#), [CreateVertices](#), [DeleteEntities](#), [DetectFaces](#), [JoinEntities](#), [LogicalExpression](#), [Filter \(Dataset\)](#), [Partition](#)

Imprint

Make imprints of faces, edges, or vertices onto another face in a 3D mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Imprint");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Imprint")` to imprint entities onto faces in a 3D mesh. Supported source input: boundaries, edges, or points. Supported destination input: boundaries. Specify the source and destination entities of the imprint as specified in [Table 4-12](#).

The following properties are available:

TABLE 4-12: AVAILABLE PROPERTIES FOR IMPRINT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
source	Selection		Entity selection of dimension 0,1, or 2.
destination	Selection		Entity selection of dimension 2.
symmetric	on off	off	Specify if to imprint on destination only (off) or both source and destination (on). Used when source is of dimension 2.
placement	linear curved	curved	Specifies the placement of mesh vertices on the destination.
precision	[0, 1]	0.5	Specifies the precision with which to make the imprints. A value close to 1 gives a more exact imprint. Used when source is of dimension 2.
groupadjedg	on off	off	Group adjacent edges that seem to be part of the loop. Used when source is of dimension 1.
distancetype	auto absolute relative	auto	Specify how to control the maximum distance to bridge when imprinting.
maxabsdistance	double		Absolute maximum distance between source and destination. Used when distancetype is absolute.
maxreldistance	double	0.01	Specify a relative maximum distance. The value is relative to the length of the longest side of the mesh's bounding box. Used when distancetype is relative.

SEE ALSO

[MergeEntities](#), [DeleteEntities](#), [Import](#)

IntersectLine

Intersect an imported 2D mesh with a straight line. The operation will partition geometric entities and intersect mesh elements to introduce straight boundaries in an intersected domain.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"IntersectLine");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"IntersectLine")` to partition geometric entities and mesh elements of an imported mesh by specifying a straight line.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the geometric entities to split with the line. If you do not specify the selection, the feature operates on the entire geometry.

The following properties are available:

TABLE 4-13: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
point	vertex coord	coord	How the point is specified.
coord	double[]	{0 0}	Point on line, if point is coord.

TABLE 4-13: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
vertex	vertex selection		Specify the point by selection of vertex.
direction	vertex coord vector	vector	How the direction is specified.
coord2	double[]	{0 1}	Second point on line, if direction is coord.
dirvector	double[]	{0 1}	Direction of line, if direction is vector.
vertex2	vertex selection		Second point on line, if direction is vertex.
toltype	auto relative absolute	auto	Snapping tolerance specification.
abstol	double		Absolute point snapping tolerance, if toltype is absolute.
reltol	double	0.01	Relative point snapping tolerance, if toltype is relative.
placement	linear curved	curved	Specifies the placement of mesh vertices in the line intersection.
selabove	on off	off	Specifies if to create a selection of resulting domains above the line.
selbelow	on off	off	Specifies if to create a selection of resulting domains below the line.
selinter	on off	off	Specifies if to create a selection of resulting intersection edges.

TABLE 4-14: AVAILABLE ADDITIONAL PROPERTIES WHEN SELABOVE, SELBELOW, AND SELINTER ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorabove	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorabove property.
colorbelow	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorbelow property.
customcolorabove	RGB-triplet	Next available theme color	The color to use. Active when colorabove is set to custom.
customcolorbelow	RGB-triplet	Next available theme color	The color to use. Active when colorbelow is set to custom.
selaboveshow	off all pnt bnd dom	dom	Specifies on which geometric level the selection is shown in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selabove is on.
selbelowshow	off all pnt bnd dom	dom	Specifies on which geometric level the selection is shown in physics,, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selbelow is on.
selintershow	on off	on	Show edge selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selinter is on.

SEE ALSO

[Import](#), [IntersectPlane](#), [Ball](#), [Box](#), [Cylinder](#), [LogicalExpression](#)

IntersectPlane

Intersect an imported 3D mesh with a plane. The operation will partition geometric entities and intersect mesh entities to introduce planar intersections. Supported entities are unmeshed domains, boundaries, and edges. Meshed faces are created within the intersection edges if the edges form closed loops.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"IntersectPlane");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"IntersectPlane")` to partition geometric entities and mesh elements of an imported 3D mesh by specifying a plane.

The following properties are available:

TABLE 4-15: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
planetype	quick faceparallel edgeparallel normalvector vertices coordinates	quick	How the plane is specified. See tables Table 4-22 to Table 4-28 for additional properties.
additionalplane	on off	off	Specify if to add additional parallel planes
plannedistance	double[]		Sequence of numbers specifying desired offsets of additional planes. Both positive and negative values are allowed.
reversedistance	on off	off	Generate the additional planes in the opposite direction
createface	on off	on	Generate faces for closed loops in the intersection
toltype	auto relative absolute	auto	How snapping tolerance is specified.
abstol	double		Absolute point snapping tolerance, if toltype is absolute.
reltol	double	0.01	Relative point snapping tolerance, if toltype is relative.
cleanup	on off	on	Cleanup of resulting mesh by collapsing mesh edges within specified toltype tolerance.
placement	linear curved	curved	Specifies the placement of mesh vertices in the plane intersection.
selabove	on off	off	Specifies if to create a selection of resulting domains above the plane.
selbelow	on off	off	Specifies if to create a selection of resulting domains below the plane.
selinter	on off	off	Specifies if to create a face selection of resulting boundaries in intersection plane.
selinteredg	on off	off	Specifies if to create an edge selection of resulting intersection edges in the plane.

Quick

This defines a plane parallel to one of the global coordinate planes.

TABLE 4-16: VALID PROPERTIES, QUICK.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
quickplane	xy yz zx yx zy xz	xy	Specify which coordinate plane to use.
quickx	double	0	x-coordinate for plane (used when plane is yz or zy).
quicky	double	0	y-coordinate for plane (used when plane is xz or zx).
quickz	double	0	z-coordinate for plane (used when plane is xy or yx).
quickoffsettype	distance vertex	distance	Type of offset specification.
offsetvertex	Selection		Specify vertex when quickoffsettype is vertex.

Face Parallel

This specifies a plane that is parallel to a planar face in a mesh.

TABLE 4-17: VALID PROPERTIES, FACEPARALLEL.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
face	Selection		Planar face selection.
offset	double	0	Specify offset in the normal direction of plane.
reverse	on off	off	Reverse normal direction of plane.
offsettype	distance vertex	distance	Type of offset specification.
offsetvertex	Selection		Specify vertex when offsettype is vertex.

Edge Parallel

This defines a plane that is parallel to a planar edge in a mesh.

TABLE 4-18: VALID PROPERTIES, EDGEPARALLEL.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
planaredge	Selection		Planar edge selection.
offset	double	0	Specify offset in the normal direction of plane.
reverse	on off	off	Reverse normal direction of plane.
offsettype	distance vertex	distance	Type of offset specification.
offsetvertex	Selection		Specify vertex when offsettype is vertex.

Normal Vector

This specifies a plane defined by a normal vector and a point on the plane.

TABLE 4-19: VALID PROPERTIES, NORMALVECTOR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
normalvector	double[3]	{0,0,1}	Normal vector specification.
normalpoint	coord vertex	coord	Point on plane (defined using coordinates or as a geometry vertex).
normalcoord	double[3]	{0,0,1}	Coordinates of point, used when normalpoint is set to coord.
normalvertex	Selection		Vertex for point, used when normalpoint is set to vertex.

Vertices

This creates a plane parallel to a plane through three vertices v1, v2, and v3.

TABLE 4-20: VALID PROPERTIES, VERTICES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
vertex1	Selection		Specification of first vertex.
vertex2	Selection		Specification of second vertex.
vertex3	Selection		Specification of third vertex.
offset	double	0	Specify offset in the normal direction of plane.
reverse	on off	off	Reverse normal direction of plane.

Coordinates

This creates a plane through three points p1, p2, and p3.

TABLE 4-21: VALID PROPERTY, COORDINATES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
genpoints	double[3][3]	{{0,0,0},{1,0,0},{0,1,0}}	Specification of points.

genpoints[n][i] is the *i*th coordinate of the *n*th point.

TABLE 4-22: AVAILABLE ADDITIONAL PROPERTIES WHEN SELABOVE, SELBELOW, SELINTER AND SELINTEREDG ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorabove	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorabove property.
colorbelow	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorbelow property.
colorinter	none custom integer between 1 and the number of colors in the current theme	none	The color of the face selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorinter property.
customcolorabove	RGB-triplet	Next available theme color	The color to use. Active when colorabove is set to custom.
customcolorbelow	RGB-triplet	Next available theme color	The color to use. Active when colorbelow is set to custom.
customcolorinter	RGB-triplet	Next available theme color	The color to use. Active when colorinter is set to custom.
selaboveshow	off all pnt edg bnd dom	dom	Specifies on which geometric level the selection is shown in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selabove is on.
selbelowshow	off all pnt edg bnd dom	dom	Specifies on which geometric level the selection is shown in physics,, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selbelow is on.

TABLE 4-22: AVAILABLE ADDITIONAL PROPERTIES WHEN SELABOVE, SELBELOW, SELINTER AND SELINTEREDG ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selintershow	on off	on	Show face selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selinter is on.
selinteredgshow	on off	on	Show edge selection in physics, materials, and so on. For mesh parts, this option shows selection outside the part. Used if selinteredg is on.

SEE ALSO

[Import](#), [IntersectLine](#), [Ball](#), [Box](#), [Cylinder](#), [LogicalExpression](#)

JoinEntities

Join geometric entities of an imported mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"JoinEntities");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"JoinEntities")` to join adjacent geometric entities of an imported 2D or 3D mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify geometric entities to join.

The following properties are available:

TABLE 4-23: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
joinadj	boolean	true	Specifies if the operation joins lower dimensional adjacent entities.

SEE ALSO

[Import](#), [DeleteEntities](#)

LogicalExpression

Split geometric entities of an imported mesh by specifying a logical expression.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"LogicalExpression");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"LogicalExpression")` to split entities of an imported mesh by specifying an element set based on a logical expression.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the geometric entities for which you want to define an element selection. If you do not specify the selection, the feature operates on the entire geometry.

The following properties are available:

TABLE 4-24: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
expression	String	1	Logical expression, where the valid variables are: x, y, and z; h and qual; and istri, isquad, istet, ispyr, isprism, ishex, and meshelement.
condition	allvertices somevertex	allvertices	Condition for inclusion of an element.
seltrue	on off	off	Create selection of all entities in the region where the logical expression is true.
selfalse	on off	off	Create selection of all entities in the region where the logical expression is false.

TABLE 4-25: AVAILABLE ADDITIONAL PROPERTIES WHEN SELTRUE AND SELFALSE ARE SET TO ON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colortrue	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolortrue property.
colorfalse	none custom integer between 1 and the number of colors in the current theme	none	The color of the domain selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolorfalse property.
customcolortrue	RGB-triplet	Next available theme color	The color to use. Active when colortrue is set to custom.
customcolorfalse	RGB-triplet	Next available theme color	The color to use. Active when colorfalse is set to custom.
seltrueshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.
selfalseshow	all dom bnd pnt off	dom	Show the selection on the specified entity level in physics, materials, and so on. For mesh parts, this option shows selection outside the part.

SEE ALSO

[Import](#), [Ball](#), [Box](#), [Cylinder](#), [DetectFaces](#)

Map

Create a structured (mapped) quadrilateral mesh in domains in 2D and on faces in 3D. The mapped mesher maps a regular grid defined on a logical unit square onto each domain or face. The mapping method is based on transfinite interpolation. The operation can also be used to remesh meshes that define their own geometric model.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Map");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>, "Map")` to create a structured quadrilateral mesh.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the domain (boundary in 3D) selection. If you do not specify any selection the feature creates a mesh on the remaining domains in 2D. In 3D, the default selection is empty.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>, ftype)` to add a Size, Distribution, or EdgeGroup attribute feature.

The following properties are available:

TABLE 4-26: AVAILABLE PROPERTIES FOR MAP.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adjustedgdistr	on off	off	When enabled, the mapped mesher adjusts evenly distributed edge mesh.
interpmethod	auto transfinite2D transfinite3D	auto	Interpolation method (3D meshes only). Only available for meshes conforming with geometry.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.

The following attribute features are used:

TABLE 4-27: ATTRIBUTE FEATURES USED.

FEATURE	REMARKS
EdgeGroup	Defined on the domain/face to be meshed.
Distribution	Used when defined on edges.
Scale	Scales Size and Distribution.
Size	Defined on domain/face. Uses only hauto and hmax.

SEE ALSO

[Distribution](#), [Size](#), [EdgeGroup](#), [FreeTri](#), [RemeshFaces](#), [RemeshDomains](#)

MergeEntities

Merge geometric entities of the mesh. The operation can merge points, edges, and boundaries within a tolerance.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>, "MergeEntities");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property, <value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>, "MergeEntities")` to merge entities in 3D mesh. Supported input: boundaries, edges, or points. Specify the entities to keep and the entities to be removed by the merge.

The following properties are available:

TABLE 4-28: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
deformation	[0, 1]	0	Specifies how the deformation propagates in the mesh adjacent to the removed entities. 1 specifies a translation of the remove side.
dimension	0 1 2	2	Specifies the dimension of what to merge. 2 specifies boundaries.
distancetype	auto relative absolute	auto	Specifies the type maximum distance to enter.
keep	Selection		Specification of the entities to keep after the merge when pairing is manual.
maxabsdistance	double		Maximum absolute distance between entities to merge. Used when distancetype is maxabsdistance.
maxreldistance	double	0.01	Maximum relative distance between entities to merge. Used when distancetype is maxreldistance.
method	imprint one-to-one	imprint	Specifies how to pair the entities. The option imprint creates an imprint of the entities to remove on the keep side.
pairing	auto manual	auto	Specify how to pair the entities in the merge, automatically or manual specification of entities to keep and remove.
placement	linear curved	curved	Specifies the placement of mesh vertices on the entities to keep.
precision	[0, 1]	0.5	Specifies the precision with which to make the imprints. A value close to 1 gives a more exact imprint. Used when method is imprint.
remove	Selection		Specification of the entities to remove. Used when pairing is manual.

SEE ALSO

[CollapseEntities](#), [JoinEntities](#), [DeleteEntities](#), [Imprint](#), [Import](#)

OnePointMap

Specify a one-point map for the copy operations and the `IdenticalMesh` feature to control the orientation of the source mesh on the destination.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"OnePointMap");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"OnePointMap")` to define a one-point map for `CopyFace` or `CopyDomain` feature `<ftag>`.

The following properties are available:

TABLE 4-29: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
srcpoint1	Selection		Point on source face/domain.
dstpoint1	Selection		Point on destination face/domain.

Use the `OnePointMap` feature if you need to control how the source and destination faces/domains are matched in a copy operation or for an `IdenticalMesh` feature. When this feature is present, the source mesh is transformed so that `srcpoint1` is mapped to `dstpoint1`.

EXAMPLE

Create a block and mesh face 4 with a fine mesh near point 8. Copy this mesh onto face 3 and ensure that the fine mesh near point 8 ends up near point 3:

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.run();

m.create("ftri1", "FreeTri");
m.feature("ftri1").selection().set(new int[]{4});
m.feature("ftri1").create("size1", "Size");
m.feature("ftri1").feature("size1").selection().geom("geom1", 0).set(new int[]{8});
m.feature("ftri1").feature("size1").set("hmax", "0.01");
m.create("cpf1", "CopyFace");
m.feature("cpf1").selection("source").geom("geom1", 2).set(new int[]{4});
m.feature("cpf1").selection("destination").geom("geom1", 2).set(new int[]{3});
m.feature("cpf1").create("pm1", "OnePointMap");
m.feature("cpf1").feature("pm1").selection("srcpoint1").set(new int[]{8});
m.feature("cpf1").feature("pm1").selection("dstpoint1").set(new int[]{3});
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom().create('geom1', 3);
m = model.component('comp1').mesh().create('mesh1', 'geom1');

g.create('blk1', 'Block');
g.run();

m.create('ftri1', 'FreeTri');
m.feature('ftri1').selection().set(4);
m.feature('ftri1').create('size1', 'Size');
m.feature('ftri1').feature('size1').selection().geom('geom1', 0).set(8);
m.feature('ftri1').feature('size1').set('hmax', '0.01');
m.create('cpf1', 'CopyFace');
m.feature('cpf1').selection('source').geom('geom1', 2).set(4);
m.feature('cpf1').selection('destination').geom('geom1', 2).set(3);
m.feature('cpf1').create('pm1', 'OnePointMap');
m.feature('cpf1').feature('pm1').selection('srcpoint1').set(8);
m.feature('cpf1').feature('pm1').selection('dstpoint1').set(3);
m.run();
```

SEE ALSO

[Copy](#), [CopyFace](#), [CopyDomain](#), [EdgeMap](#), [TwoPointMap](#), [IdenticalMesh](#)

Refer to another meshing sequence.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Reference");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).expand();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Reference")` to refer to another meshing sequence. Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype)` to add Scale attribute features.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).expand()` to replace the reference with a copy of the referred sequence, where the attributes have been scaled with the scale attribute features of the reference.

The following properties are available:

TABLE 4-30: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sequence	String		Tag of referred sequence.

Use the sequence property to specify another meshing sequence on the same geometry. When running the feature, all features of the specified sequence are run in the current context.

It is not allowed to introduce circular references.

EXAMPLE

Create a mixed mesh with quads and triangles on a geometry. Create a second meshing sequence with a scale feature and a reference to the first meshing sequence. The result is a coarser version of the first mesh.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
MeshSequence m1 = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("sq1", "Square");
g.create("sq2", "Square");
g.feature("sq2").set("size", "0.5");
g.run();

m1.create("map1", "Map");
m1.feature("map1").selection().geom("geom1", 2).set(new int[]{1});
m1.create("ftri1", "FreeTri");
m1.feature("ftri1").selection().geom("geom1", 2).set(new int[]{2});
m1.run();

MeshSequence m2 = model.mesh().create("mesh2", "geom1");
m2.create("sca1", "Scale");
m2.feature("sca1").set("scale", "2");
m2.create("rf1", "Reference");
m2.feature("rf1").set("sequence", "mesh1");
m2.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
m1 = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('sq1', 'Square');
g.create('sq2', 'Square');
g.feature('sq2').set('size', '0.5');
g.run();

m1.create('map1', 'Map');
m1.feature('map1').selection().geom('geom1', 2).set(1);
m1.create('ftri1', 'FreeTri');
m1.feature('ftri1').selection().geom('geom1', 2).set(2);
m1.run();

m2 = model.mesh().create('mesh2', 'geom1');
m2.create('sca1', 'Scale');
m2.feature('sca1').set('scale', '2');
m2.create('rf1', 'Reference');
m2.feature('rf1').set('sequence', 'mesh1');
m2.run();
```

SEE ALSO

[Scale](#)

Refine

Refine a mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Refine");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Refine")` to refine the mesh. Mesh refinement is available both for generated and imported meshes.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the domain selection. The default selection is the entire geometry, meaning that all elements in the mesh are refined.

The following properties are available:

TABLE 4-31: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
rmethod	longest regular	see below	Refinement method.Used when <code>sdim</code> >=2.
numrefine	int or int[]	1	Number of refinements.
facerep	curved linear	curved	In 3D, specify face representation when placing moved or new mesh vertices for meshes that define their own geometric models.

TABLE 4-31: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
boxcoord	on off	off	For <code>sdim</code> >=2, use coordinates of a bounding box to determine the elements to refine.
xmax, xmin, ymax, ymin, zmax, zmin	double		Coordinates of bounding box. zmax and zmin are only used in 3D

The default refinement method in 2D is regular refinement, where all edges of the element are bisected. Longest edge refinement, where the longest edge of each specified element is bisected, can be selected by giving `longest` as `rmethod`. Using `regular` as `rmethod` results in regular refinement. Some elements outside of the specified set can also be refined due to propagation.

In 3D, the default refinement method is `longest`. If the mesh contains nonsimplex elements, consider using `regular` refinement instead because this method preserves the structure of the mesh.

In 1D, regular refinement, where each element is divided into two elements of the same shape, is always used.

By default, all elements are refined once. The `numrefine` property specifies how many times the elements is refined.

The `facerep` property is only used for meshes that define their own geometric model. For example, when having an imported mesh. Use `curved` to place new or moved mesh vertices on a curved surface approximation of the input mesh. Use `linear` to place new or moved mesh vertices on the input mesh.

Use the `boxcoord` property to refine elements inside a bounding box. To define the bounding box, set the properties `xmin`, `xmax`, `ymin`, `ymax`, `zmax`, and `zmin` on the feature, where `(xmin,ymin,zmin)` defines the lower-left corner, and `(xmax,ymax,zmax)` defines the upper-right corner of the bounding box. The elements that have all its corner points in the bounding box are refined once. `boxcoord` is automatically set to `on` if one of the coordinates are set.

EXAMPLE

Mesh two squares with an unstructured mesh. Refine the mesh on `sq2` once and refine the elements inside a box in `sq1` twice.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 2);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("sq1", "Square");
g.create("sq2", "Square");
g.feature("sq2").setIndex("pos", "1", 0);
g.run();

m.create("ftri1", "FreeTri");
m.create("ref1", "Refine");
m.feature("ref1").selection().geom("geom1", 2).set(new int[]{2});
m.create("ref2", "Refine");
m.feature("ref2").set("xmin", "0.2");
m.feature("ref2").set("xmax", "0.8");
m.feature("ref2").set("ymin", "0.2");
m.feature("ref2").set("ymax", "0.6");
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component().create('comp1');
g = model.component('comp1').geom.create('geom1', 2);
m = model.component('comp1').mesh.create('mesh1', 'geom1');
```

```

g.create('sq1', 'Square');
g.create('sq2', 'Square');
g.feature('sq2').setIndex('pos', '1', 0);
g.run();

m.create('ftri1', 'FreeTri');
m.create('ref1', 'Refine');
m.feature('ref1').selection().geom('geom1', 2).set(2);
m.create('ref2', 'Refine');
m.feature('ref2').set('xmin', '0.2');
m.feature('ref2').set('xmax', '0.8');
m.feature('ref2').set('ymin', '0.2');
m.feature('ref2').set('ymax', '0.6');
m.run();

```

SEE ALSO

[Adapt](#), [Convert](#)

RemeshDomains

Remesh domain in 2D meshes that define their own geometric model, such as imported meshes.

SYNTAX

```

model.component(<ctag>).mesh(<tag>).create(<ftag>,"RemeshDomains");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);

```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"RemeshDomains")` remesh a selection of domains in 2D meshes that define their own geometric model. For more information, see [Geometric Model](#).

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the faces to remesh. The default selection is empty.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype)` to add attribute features. Supported attributes are listed under the See Also section.

TABLE 4-32: AVAILABLE PROPERTIES FOR REMESHDOMAINS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
quad	on off	off	Specifies if to generate a quad-dominant mesh.

SEE ALSO

[CornerRefinement](#), [Distribution](#), [FixedMesh](#), [IdenticalMesh](#), [Size](#), [RemeshEdges](#)

RemeshEdges

Remesh edges in 2D and 3D meshes that define their own geometric model, such as imported meshes.

SYNTAX

```

model.component(<ctag>).mesh(<tag>).create(<ftag>,"RemeshEdges");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);

```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>, "RemeshEdges")` remesh a selection of edges in 2D or 3D meshes that define their own geometric model. For more information, see [Geometric Model](#).

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the faces to remesh. The default selection is empty.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>, ftype)` to add attribute features. Supported attributes are listed under the See Also section.

The following properties are available:

TABLE 4-33: AVAILABLE PROPERTIES FOR REMESHFACES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
preservefaceshape	on off	on	Specifies if to use the shape of the adjacent faces to improve the shape of the edge.

SEE ALSO

[CornerRefinement](#), [Distribution](#), [IdenticalMesh](#), [Size](#), [RemeshFaces](#), [Import](#)

RemeshFaces

Remesh boundaries in 3D meshes that define their own geometric model, such as imported meshes.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>, "RemeshFaces");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property, <value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>, ftype);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>, "RemeshFaces")` remesh a selection of faces in 3D meshes that define their own geometric model. For more information, see [Geometric Model](#).

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the faces to remesh. The default selection is empty.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>, ftype)` to add attribute features. Supported attributes are listed under the See Also section.

The following properties are available:

TABLE 4-34: AVAILABLE PROPERTIES FOR REMESHFACES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
facerep	curved linear	curved	Specify face representation when placing moved or new mesh vertices for meshes that define their own geometric models.
narrowreg	on off	on	Specifies if to resolve the mesh with respect to adjacent narrow domain regions.
quad	on off	off	Specifies if to generate a quad-dominant mesh.

After the faces are remeshed, use [FreeTet](#) to fill the domains with a tetrahedral mesh.

SEE ALSO

[CornerRefinement](#), [Distribution](#), [FixedMesh](#), [IdenticalMesh](#), [Size](#), [RemeshEdges](#), [Import](#)

Scale mesh size properties.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Scale");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"Scale");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Scale")` to scale size properties defined in the sequence and use `model.mesh(<tag>).feature(<ftag>).create(<ftag1>,"Scale")` to scale size properties defined in the sequence referred to by the Reference feature `<ftag>`.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` or `model.mesh(<tag>).component(<ctag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the geometric entity selection or the entire geometry (which is default).

The following properties are available:

TABLE 4-35: FEATURE PROPERTIES DEFINED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
scale	double	1	Scale factor.

Scale is a positive number. The feature scales mesh size properties, distribution properties, and boundary layer properties affecting mesh elements generated by features following the scale feature. The scale feature also affects size properties defined by `Size`, `Distribution`, and `BndLayerProp` features occurring later in the sequence.

A scale less than 1 gives smaller (more) elements; a scale greater than 1 gives larger (fewer) elements. The scale feature has no effect on any mesh generated earlier in the sequence.

If two or more scale features exist on the same selection, the resulting scale on that selection is the product of the given scales.

EXAMPLE

Create a block and mesh it with 10-by-10-by-10 hexahedra. Setting `scale` to 2 gives you a block with 5-by-5-by-5 hexahedra and setting the scale to 0.5 gives you a block with 20-by-20-by-20 hexahedra.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g = model.component("comp1").geom().create("geom1", 3);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.run();

m.create("sca1", "Scale");
m.create("map1", "Map");
m.feature("map1").selection().set(new int[]{1});
m.create("swe1", "Sweep");
m.run();
```

```

m.feature("sca1").set("scale", "2");
m.run();

m.feature("sca1").set("scale", "0.5");
m.run();

```

Code for Use with MATLAB

```

model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('blk1', 'Block');
g.run;

m.create('sca1', 'Scale');
m.create('map1', 'Map');
m.feature('map1').selection().set(1);
m.create('swe1', 'Sweep');
m.run;

m.feature('sca1').set('scale', '2');
m.run;

m.feature('sca1').set('scale', '0.5');
m.run;

```

SEE ALSO

[BndLayerProp](#), [Distribution](#), [Size](#)

Size

Specify mesh size properties.

SYNTAX

```

model.component(<ctag>).mesh(<tag>).create(<ftag>,"Size");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"Size");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);

```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Size")` to specify element size properties in the sequence. Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"Size")` to specify element size properties for the feature `<ftag>` that can be any of the types `Edge`, `FreeQuad`, `FreeTri`, `FreeTet`, `Map`, or `Sweep`.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` or `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the geometric entity selection. If you do not specify any selection the size feature is defined on all geometric entities. The selection is not available for the *default size feature*, tagged `size`.

The following properties are available:

TABLE 4-36: FEATURE PROPERTIES DEFINED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
custom	on off	off	Set custom to on to activate all mesh parameters.
hauto	1, 2, ...,9 (or any nonnegative scalar)	5	Specifies automatic settings for all mesh parameters when custom is off. 1 is the finest and 9 is the coarsest predefined size setting. You can use any nonnegative value for the predefined element size. Use Double.POSITIVE_INFINITY to create the coarsest possible mesh.
hcurve	double	0.3 0.6	Curvature mesh size.
hcurveactive	on off	on	Specifies if hcurve is used.
hgrad	double	1.3 1.5	Element growth rate.
hgradactive	on off	on	Specifies if hgrad is used.
hmax	double	geometry dependent	Maximum element size.
hmaxactive	on off	on	Specifies if hmax is used.
hmin	double	geometry dependent	Minimum element size.
hminactive	on off	on	Specifies if hmin is used.
hnnarrow	double	0.5	Resolution of narrow regions.
hnnarrowactive	on off	on	Specifies if hnnarrow is used.
table	cfid default plasma semi coarseadaptation timeexplicitwave	default	Specifies for which physics the element size is calibrated.



The properties with names ending in active are not available for the default size feature.

The property `table` specifies the physics for which the element size is calibrated.

`Hauto` is a nonnegative scalar. This value is used to set several mesh parameters in order to get a mesh of desired size. Smaller values of `hauto` generate finer meshes with more elements. The integers between 1 and 9 has a special interpretation; they correspond to the mesh settings **Normal**, **Fine**, **Coarse**, and so forth in COMSOL Multiphysics. The value 5 correspond to **Normal**. When you set the property `hauto`, all other properties are set to their default value, according to the following tables (for `table` set to `default`). Other noninteger values provide mesh parameters that are interpolated from the values in the tables.

TABLE 4-37: MESH PARAMETERS SET BY THE PROPERTY HAUTO IN 2D (FOR DEFAULT TABLE).

HAUTO	HMAXFACT	HCURVE	HGRAD	HMINFACT	HNNARROW
1	0.01	0.2	1.1	2e-5	1
2	0.02	0.25	1.2	7.5e-5	1
3	0.037	0.25	1.25	1.25e-4	1
4	0.053	0.3	1.3	3e-4	1
5	0.067	0.3	1.3	3e-4	1
6	0.1	0.4	1.4	0.002	1
7	0.13	0.6	1.5	0.006	1

TABLE 4-37: MESH PARAMETERS SET BY THE PROPERTY HAUTO IN 2D (FOR DEFAULT TABLE).

HAUTO	HMAXFACT	HCURVE	HGRAD	HMINFACT	HNARROW
8	0.2	0.8	1.8	0.016	1
9	0.33	1	2	0.05	0.9

TABLE 4-38: MESH PARAMETERS SET BY THE PROPERTY HAUTO IN 3D (FOR DEFAULT TABLE).

HAUTO	HMAXFACT	HCURVE	HGRAD	HMINFACT	HNARROW
1	0.02	0.2	1.3	2e-4	1
2	0.035	0.3	1.35	0.0015	0.85
3	0.055	0.4	1.4	0.004	0.7
4	0.08	0.5	1.45	0.01	0.6
5	0.1	0.6	1.5	0.018	0.5
6	0.15	0.7	1.6	0.028	0.4
7	0.19	0.8	1.7	0.04	0.3
8	0.3	0.9	1.85	0.054	0.2
9	0.5	1	2	0.07	0.1

The property `hcurve` is a real value that relates the mesh size to the curvature of the geometry boundaries. The Gaussian radius of curvature is multiplied by the `hcurve` factor to obtain the mesh size along the boundary. The specified `hcurve` is only used if `hcurveactive` is on, otherwise `hcurve` is taken from a preceding size feature in the sequence. In the default size feature, tagged `size`, `hcurve` is always active and there is no `hcurveactive` property.

The property `hgrad` tells how fast the element size — measured as the length of the longest edge of the element — can grow from a region with small elements to a region with larger elements. If two elements lie one unit length apart, the difference in element size can be at most `hgrad`. The specified `hgrad` is only used if `hgradactive` is on, otherwise `hgrad` is taken from a preceding size feature in the sequence. In the default size feature, `hgrad` is always active and there is no `hcurvegrad` property.

The `hmax` parameter controls the size of the elements in the mesh. The algorithm aims at creating a mesh where no element size exceeds `hmax`. The default `hmax` value is `hmaxfact * maxdist`, where `maxdist` is the longest axis parallel distance in the geometry. The specified `hmax` is only used if `hmaxactive` is on, otherwise `hmax` is taken from a preceding size feature in the sequence. In the default size feature, `hmax` is always active and there is no `hmaxactive` property.

You can use `hmin` to control the minimum size of the elements. The main purpose of this parameter is to prevent the generation of many small elements near small curved parts of the geometry. The default `hmin` value is `hminfact * maxdist`, where `maxdist` is the longest axis parallel distance in the geometry. The specified `hmin` is only used if `hminactive` is on, otherwise `hmin` is taken from a preceding size feature in the sequence. In the default size feature, `hmin` is always active and there is no `hminactive` property.

The `hnarrow` parameter controls the size of the elements in narrow regions. Increasing values of this property decrease the size of the elements in narrow regions. If the value of `hnarrow` is less than one, elements that are anisotropic in size might be generated in narrow regions. The specified `hnarrow` is only used if `hnarrowactive` is on, otherwise `hnarrow` is taken from a preceding size feature in the sequence. In the default size feature, `hnarrow` is always active and there is no `hnarrowactive` property.

The values of `hauto`, `hcurve`, `hgrad`, `hmax`, `hmin`, and `hnarrow` are positive real scalars, or strings that evaluate to positive real scalars, given the evaluation context provided by `model.param()`.

It is not possible to specify coarser size settings on the boundary of a domain than on the domain. The finer settings on the domain is inherited by its boundaries and, in 3D, edges. A warning is issued when settings are overwritten

by inheritance. If you need to create coarser mesh on a boundary, you should first mesh the boundary then add the finer size settings on the domain for the corresponding [FreeTet](#), [FreeTri](#), or [FreeQuad](#) operation.

SEE ALSO

[Distribution](#), [Scale](#), [SizeExpression](#)

SizeExpression

Specify a mesh size expression.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"SizeExpression");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).importData();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"SizeExpression");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).
    set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"SizeExpression")` to specify a mesh element size expression in the sequence. Use

`model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"SizeExpression")` to specify an element size expression for the feature `<ftag>` that can be any of the types `Edge`, `FreeQuad`, `FreeTri`, or `FreeTet`.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` or `model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection()` to specify the geometric entity selection. If you do not specify any selection, the size expression feature is defined on all geometric entities.

You can use `mesh.feature(<ftag>).importData()` to reevaluate the size expression, taking an updated model into account.

The following properties are available.

TABLE 4-39: FEATURE PROPERTIES FOR SIZEEXPRESSION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapsolnum	Array of integers>0	1	Solution number indices.
cellsize	Positive scalar	Geometry bounding box size / 25	Grid cell size.
elementspar	Positive scalar		Controls refinement if <code>elselect = elements</code> .
elselect	<code>globalmin</code> <code>worst</code> <code>elements</code>		Method to select elements to refine.
errorexpr	String		Error expression.
exprtype	<code>size</code> <code>error</code>	<code>size</code>	Type of expression for the adaptive mesh generation: an absolute size or an error expression.
evaltype	<code>grid</code> <code>solution</code> <code>initialexpression</code>	<code>grid</code>	Specification of mesh to evaluate on.

TABLE 4-39: FEATURE PROPERTIES FOR SIZEEXPRESSION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
globalminparam	Positive scalar		Controls refinement if elselect = globalmin.
gridtype	cellresolution cellsize	cellresolution	Grid specification.
hmeshgrad	Scalar, 1.0 or greater	1.4	Maximum size field growth rate (1.4 means that the mesh can grow by 40%, for example).
horder	Double array	0	Error orders (see below).
mesh	auto tag of other mesh	auto	Mesh for evaluation when evaltype is initialcondition.
numcell	Integer	25	Number of cells per dimension.
selection	first last all manual	last	Solution selection: the first or last solution, all solutions, or manual, using weights and solution number indices in adapsolnum.
sizeexpr	String	Geometry bounding box size	Size expression to evaluate for the mesh size.
solution	String		The solution defining the mesh adaptation.
studystep	none tag path to study step	none	The study step to use. Available when evaltype is initialcondition.
updatecondition	A parameter name		Name of a parameter used to trigger an update.
weights	Double[] (positive values)	1.0	Weight for each selected solution.
worstpar	Positive scalar		Controls refinement if elselect = worst.

For the horder property, its value is automatically calculated in the adaptation algorithm. It is used only when the Element selection method is set to Rough global minimum.

SEE ALSO

[Adapt](#), [Distribution](#), [Scale](#), [Size](#)

Sweep

Create a swept mesh in 3D by sweeping the mesh from the source face along the domain to an opposite destination face.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Sweep");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype);
```

DESCRIPTION

Use model.component(<ctag>).mesh(<tag>).create(<ftag>,"Sweep") to create a swept mesh in 3D.

Use model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection() to specify the domain selection. If you do not specify any selection the feature creates a mesh on the remaining domains.

Use model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,ftype) to add a Size or Distribution attribute feature.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the 3D domain selection. If you do not specify any selection the feature creates a mesh on the remaining domains.

The following properties are available:

TABLE 4-40: AVAILABLE PROPERTIES FOR SWEEP.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
remeshsourceface	on off	on	Specifies if source faces are remeshed when the mesh defines its own geometric model.
element	hexdominant hex prism hexlegacy63 hexlegacy54 hexlegacy52a hexlegacy52	hexdominant	Specifies the face meshing method. For mesh-based geometries and imported meshes, it is used when remeshsourceface is on.
mapinterpmethod	auto transfinite2D transfinite3D	auto	Interpolation method for linking faces.
smoothcontrol	auto on off	auto	Specifies if the operation smooths the mesh across removed control entities. Only available for meshes conforming with geometry.
smoothmaxiter	integer	4	Specifies the number of smoothing iterations. Used if smoothcontrol is on.
smoothmaxdepth	integer	4	Specifies the maximum element smoothing depth. Used if smoothcontrol is on.
sourceface	Selection		Source faces selection.
sweepath	auto straight circular general	auto	Sweep path calculation.
targetface	Selection		Destination face selection.
targetmesh	auto rigid morph project	auto	Destination mesh method.

For more information about meshes that define their own geometric model, see the section [Geometric Model](#).

Use the property `sourceface` and `targetface` to specify the source faces and the destination faces of the sweep, respectively. For domains in the feature selection where none of the surrounding faces are specified as either a source or a destination face, the software automatically tries to determine these faces.

Use the property `sweepath` if you want to specify the shape of the sweep path. The string is either `auto`, `straight`, `circular`, or `general`. `straight` means that all interior mesh points are located on straight lines between the corresponding source and destination points. `circular` means that all interior mesh points are located on circular arcs between the corresponding source and destination points. `general` means that the positions of the interior mesh points are determined by a general interpolation procedure. `auto`, which is default, means that the sweeping algorithm automatically tries to determine if the sweep path is straight or circular. If this is the case `sweepath` is set to `straight` or `circular`, respectively. Otherwise, `sweepath` is set to `general`.

Any source face that is not meshed, is meshed automatically. The property `facemethod` controls which face meshing method is used:

- If `facemethod` is `auto`, the algorithm tries to mesh the source faces with quads, but switches to triangles if quad meshing fails or if the element quality becomes too low.
- If `facemethod` is `quad`, you get quadrilateral face mesh and therefore hexahedral domain mesh.
- If `facemethod` is `tri`, you get triangular face mesh and prism elements in the domain.

Use the property `targetmesh` if you want to specify the method to be used for transferring the source mesh to the destination. See [Swept](#) for more information about the values and examples where they would be used.

The following attribute features are used:

TABLE 4-41: ATTRIBUTE FEATURES USED.

FEATURE	REMARKS
Distribution	Used when defined on domains.
Scale	Scales Size and Distribution.
Size	Defined on domain.

If a [Distribution](#) feature is defined on a domain, it is used to determine the distribution of element layer in the sweep direction. Otherwise, equidistant element layers are generated.

SEE ALSO

[Distribution](#), [Size](#), [FreeQuad](#), [FreeTri](#), [Map](#)

Transform

Transform an imported mesh by specifying a displacement, rotation angle, or scaling factor by adding an attribute to [Import](#).

SYNTAX

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"Transform");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>)
    .set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"Transform")` to position, rotate, or scale an imported mesh.

The following properties are available:

TABLE 4-42: FEATURE PROPERTIES DEFINED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
displ	double[sdim]	(0,0,0)	Specify a displacement offset.
base	origin corner center bottom top	origin	Base point position in source mesh.
anchor	origin vertex		Anchor point position to match.
vertex	Selection		Select anchor point, if anchor is vertex
axistype	x y z cartesian spherical	z	Axis type, 3D only.
ax3	double[3]	(0,0,1)	Specify cartesian axis. If axistype is cartesian.
ax2	double[2]	(0,0)	Specify theta and phi. If axistype is spherical.
rot	double	0	Rotation angle.
scaletype	isotropic anisotropic	isotropic	Type of scaling.

TABLE 4-42: FEATURE PROPERTIES DEFINED.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
isotropic	double	1	Scaling factor
anisotropic	double[sdim]	(1, 1, 1)	Specify x, y, and z (if 3D) scaling factors

Anchor is only valid for the import operations succeeding the first import operation in a meshing sequence.

The displacement offset is applied after the positions have been matched.

	The positioning of imported mesh objects is supported for static positioning only. To parameterize the geometry and move objects around in a parametric sweep, use the option of Creating Geometry from Mesh to apply the parameterization in the geometry sequence.
---	--

The mesh is translated so that the specified position in source is moved to the position to match. After that, any specified rotation or scaling is applied, and, lastly, if an offset is specified, it is added to the translation.

A scaling factor less than 1 gives reduces the size of the geometry; a scale greater than 1 increases the geometry. Enter a negative scale to mirror the mesh. The scale is applied prior to any translation or rotation; the center used is the point specified using the `base` property.

SEE ALSO

[Import](#)

TwoPointMap

Specify a two-point map for the copy operations and the `IdenticalMesh` feature to control the orientation of the source mesh on the destination.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"TwoPointMap")
model.component(<ctag>).mesh(<tag>).feature(<ftag>).feature(<ftag1>).selection(property)
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).create(<ftag1>,"TwoPointMap")` to define a two-point map for the `CopyFace` or `CopyDomain` feature `<ftag>`.

The following properties are available:

TABLE 4-43: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DESCRIPTION
srcpoint1	Selection	First point on source face/domain.
srcpoint2	Selection	Second point on source face/domain.
dstpoint1	Selection	First point on destination face/domain.
dstpoint2	Selection	Second point on destination face/domain.

Use the `TwoPointMap` feature if you need to control how the source and destination faces/domains are matched in a copy mesh operation and in an `IdenticalMesh` feature. When this feature is present, the source mesh is transformed so that `srcpoint1` is mapped to `dstpoint1` and `srcpoint2` is mapped to `dstpoint2`.

EXAMPLE

Create a block and mesh face 2. Copy this mesh onto the opposite face 5 and ensure that point 6 is mapped to point 4 and point 5 is mapped to Point 8.

Code for Use with Java

```
Model model = ModelUtil.create("Model");
model.component().create("comp1");
GeomSequence g= model.component("comp1").geom().create("geom1", 3);
MeshSequence m = model.component("comp1").mesh().create("mesh1", "geom1");

g.create("blk1", "Block");
g.run();

m.create("ftri1", "FreeTri");
m.feature("ftri1").selection().set(new int[]{2});
m.feature("ftri1").create("size1", "Size");
m.feature("ftri1").feature("size1").selection().geom("geom1", 1).set(new int[]{9});
m.feature("ftri1").feature("size1").set("hmax", "0.01");
m.create("cpf1", "CopyFace");
m.feature("cpf1").selection("source").geom("geom1", 2).set(new int[]{2});
m.feature("cpf1").selection("destination").geom("geom1", 2).set(new int[]{5});
m.feature("cpf1").create("ppm1", "TwoPointMap");
m.feature("cpf1").feature("ppm1").selection("srcpoint1").set(new int[]{6});
m.feature("cpf1").feature("ppm1").selection("dstpoint1").set(new int[]{4});
m.feature("cpf1").feature("ppm1").selection("srcpoint2").set(new int[]{5});
m.feature("cpf1").feature("ppm1").selection("dstpoint2").set(new int[]{8});
m.run();
```

Code for Use with MATLAB

```
model = ModelUtil.create('Model');
model.component.create('comp1');
g = model.component('comp1').geom.create('geom1', 3);
m = model.component('comp1').mesh.create('mesh1', 'geom1');

g.create('blk1', 'Block');
g.run;

m.create('ftri1', 'FreeTri');
m.feature('ftri1').selection().set(2);
m.feature('ftri1').create('size1', 'Size');
m.feature('ftri1').feature('size1').selection().geom('geom1', 1).set(9);
m.feature('ftri1').feature('size1').set('hmax', '0.01');
m.create('cpf1', 'CopyFace');
m.feature('cpf1').selection('source').geom('geom1', 2).set(2);
m.feature('cpf1').selection('destination').geom('geom1', 2).set(5);
m.feature('cpf1').create('ppm1', 'TwoPointMap');
m.feature('cpf1').feature('ppm1').selection('srcpoint1').set(6);
m.feature('cpf1').feature('ppm1').selection('dstpoint1').set(4);
m.feature('cpf1').feature('ppm1').selection('srcpoint2').set(5);
m.feature('cpf1').feature('ppm1').selection('dstpoint2').set(8);
m.run;
```

SEE ALSO

[Copy](#), [CopyFace](#), [CopyDomain](#), [EdgeMap](#), [OnePointMap](#), [IdenticalMesh](#)

Union

Unite imported 3D surface meshes with each other. The operation will partition geometric entities and intersect mesh elements where the meshes intersect.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Union");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection(property);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Union")` to unite meshed entities in 3D. Supported input: entire geometry, unmeshed domains, boundaries adjacent to unmeshed domains, isolated boundaries, isolated edges, and isolated points.



Use `feature.selection()` when dimension is 1, 2, or 3. Use `feature(<ftag>).selection(property)` when dimension is mixed.

Use [DeleteEntities](#) to delete domain elements while keeping the unmeshed domains. If a subset of the faces are selected for the operation, it might not be possible to update any existing domain information. Use [CreateDomains](#) to recreate the domains, if needed.

The following properties are available:

TABLE 4-44: AVAILABLE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
abstol	double		Absolute point snapping tolerance, if toltype is absolute.
cleanup	on off	on	Cleanup of the mesh before and after uniting the selected entities, for example by collapsing small and sliver elements.
dimension	1 2 3 all mixed	all	Specifies the entity dimension to be united. Default is entire geometry.
domain			Specifies the domain selection if dimension is mixed.
edge			Specifies the edge selection if dimension is mixed.
face			Specifies the face selection if dimension is mixed.
placement	linear curved	curved	Specifies the placement of mesh vertices in the intersection between united entities.
reltol	double	0.001	Relative point snapping tolerance, if toltype is relative.
toltype	auto relative absolute	auto	How the snapping tolerance is specified.
vertex			Specifies the vertex selection if dimension is mixed.

SEE ALSO

[CreateDomains](#), [DeleteEntities](#), [IntersectLine](#), [IntersectPlane](#), [Import](#)

UnionSelection, IntersectionSelection, DifferenceSelection, ComplementSelection

Combine selections of entities using a Boolean operation.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"ComplementSelection");
model.component(<ctag>).mesh(<tag>).create(<ftag>,"DifferenceSelection");
model.component(<ctag>).mesh(<tag>).create(<ftag>,"IntersectionSelection");
model.component(<ctag>).mesh(<tag>).create(<ftag>,"UnionSelection");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
model.component(<ctag>).mesh(<tag>).feature(<ftag>).set(property,<value>);
model.component(<ctag>).mesh(<tag>).feature(<ftag>).getType(property);
```

DESCRIPTION

Use `UnionSelection` to get all entities that belong to at least one of the input selections. Use `IntersectionSelection` to get all entities that belong to all input selections. Use `DifferenceSelection` to get all entities that belong to some of the add selections, but do not belong to any of the subtract selections. Use `ComplementSelection` to get all entities of the given dimension that do not belong to any input selection.

The following properties are available:

TABLE 4-45: AVAILABLE PROPERTIES FOR DIFFERENCESELECTION

NAME	VALUE	DEFAULT	DESCRIPTION
entitydim	3 2 1 0	space dimension	Dimension of entities to select.
add	String[]	{}	Tags of selections to add
subtract	String[]	{}	Tags of selections to subtract
selshow	on off	on	Show selection in physics, materials, and so on. For mesh parts, this option shows selection in instances of the mesh part.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.

TABLE 4-46: AVAILABLE PROPERTIES FOR UNIONSELECTION, INTERSECTIONSELECTION, AND COMPLEMENTSELECTION.

NAME	VALUE	DEFAULT	DESCRIPTION
entitydim	3 2 1 0	space dimension	Dimension of entities to select.
input	String[]	{}	Tags of input selections
selshow	on off	on	Show selection in physics, materials, and so on. For mesh parts, this option shows selection in instances of the mesh part.
color	none custom integer between 1 and the number of colors in the current theme	none	The color of the selection, either given as an integer indicating a color in the color theme, or as a custom color in the customcolor property.
customcolor	RGB-triplet	Next available theme color	The color to use. Active when color is set to custom.

SEE ALSO

[BallSelection](#), [BoxSelection](#), [CylinderSelection](#), [DiskSelection](#), [AdjacentSelection](#), [ExplicitSelection](#)

Vertex

Create a vertex mesh.

SYNTAX

```
model.component(<ctag>).mesh(<tag>).create(<ftag>,"Vertex");
model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection();
```

DESCRIPTION

Use `model.component(<ctag>).mesh(<tag>).create(<ftag>,"Vertex")` to mesh geometrical points.

Use `model.component(<ctag>).mesh(<tag>).feature(<ftag>).selection()` to specify the point selection.

If you do not specify any selection the feature creates a mesh on the remaining points.

SEE ALSO

[Edge](#)

Elements and Shape Function Programming

This chapter contains reference information for using the API available for the shape functions (elements) in COMSOL Multiphysics. See also `model.shape()` in the *General Commands* chapter for details about the syntax for specifying shape functions. For an overview of the elements and shape functions, see [Elements and Shape Functions](#) in the *COMSOL Multiphysics Reference Manual*.

Shape Functions and Element Types

This section describes the available shape functions (element types) with their properties and syntax examples.

Shape Function Types (Elements)

THE LAGRANGE ELEMENT (SHLAG)

Specify Lagrange elements in the `model.shape` field of the model object. The constructor of the Lagrange shape function is `shlag`. The following properties are allowed:

TABLE 5-1: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHLAG SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
basename	Variable name		Base variable name
order	Positive integer		Basis function order
sorder	Positive integer	Determined by frame	Geometry shape function order

It is not possible to abbreviate the property names, and you must write them in lowercase letters enclosed in quotation marks. For example:

```
model.shape().create("shu","f");
model.shape("shu").create("f1","shlag");
model.shape("shu").feature("f1").set("order",2);
model.shape("shu").feature("f1").set("basename","u");
```

The Lagrange element defines the following variables. Denote `basename` with u , and let x and y denote (not necessarily distinct) spatial coordinates. The variables are (`sdim` = space dimension and `edim` = mesh element dimension):

- u
- ux , meaning the derivative of u with respect to x , defined on `edim` = `sdim`
- uxy , meaning a second derivative, defined on `edim` = `sdim`
- uTx , the tangential derivative variable, meaning the x -component of the tangential projection of the gradient, defined on `edim` < `sdim`
- $uTxy$, meaning xy -component of the tangential projection of the second derivative, defined when `edim` < `sdim`

When calculating the derivatives, the global spatial coordinates are expressed as polynomials of degree (at most) `sorder` in the local coordinates.

THE NODAL SERENDIPITY ELEMENT (SHNSERP)

Specify serendipity shape functions in the `model.shape` field of the model object. The constructor of the serendipity shape function is `shnserp`. The following properties are allowed:

TABLE 5-2: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHNSERP SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
basename	Variable name		Base variable name
order	Integer, 2–4		Basis function order
sorder	Positive integer	Determined by frame	Geometry shape function order

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```
model.shape().create("shu","f");
model.shape("shu").create("f1","shnserp");
```

```
model.shape("shu").feature("f1").set("order",3);
model.shape("shu").feature("f1").set("basename","u");
```

The nodal serendipity element defines the following field variables. Denote `basename` with u , and let x and y denote (not necessarily distinct) spatial coordinates. The variables are (`sdim` = space dimension and `edim` = mesh element dimension):

- u
- ux , meaning the derivative of u with respect to x , defined when `edim` = `sdim` or `edim`=0
- uxy , meaning a second derivative, defined when `edim` = `sdim`
- uTx , the tangential derivative variable, meaning the x -component of the tangential projection of the gradient, defined when $0 < \text{edim} < \text{sdim}$
- $uTxy$, meaning xy -component of the tangential projection of the second derivative, defined when `edim` < `sdim`

When calculating the derivatives, the global spatial coordinates are expressed as polynomials of degree (at most) `sorder` in the local coordinates.

THE ARGYRIS ELEMENT (SHARG_2_5)

Specify Argyris shape functions in the `model.shape` field of the model object. The constructor of the Argyris shape function is `sharg_2_5`. The following properties are allowed:

TABLE 5-3: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHARG SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
basename	Variable name		Base variable name

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```
model.shape().create("shu","f");
model.shape("shu").create("f1","sharg_2_5");
model.shape("shu").feature("f1").set("basename","u");
```

The Argyris element defines the following degrees of freedom (where u is the base name and x and y are the spatial coordinate names):

- u at corners
- ux and uy at corners, meaning derivatives of u
- uxx , uxy , and uyy at corners, meaning second derivatives
- un at side midpoints, meaning a normal derivative. The direction of the normal is to the right if moving along an edge from a corner with lower mesh vertex number to a corner with higher number

The Argyris element defines the following field variables (where `sdim` = space dimension = 2 and `edim` = mesh element dimension):

- u
- ux , meaning the derivative of u with respect to x
- uxy , meaning a second derivative, defined for `edim` = `sdim` and `edim` = 0
- $uxTy$, the tangential derivative variable, meaning the y -component of the tangential projection of the gradient of ux , defined for $0 < \text{edim} < \text{sdim}$

When calculating the derivatives, the global spatial coordinates are always expressed with shape order 1 in the Argyris element.

THE HERMITE ELEMENT (SHHERM)

Specify Hermite shape functions in the `model.shape` field of the model object. The constructor of the Hermite shape function is `shherm`. The following properties are allowed:

TABLE 5-4: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHHERM SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
basename	Variable name		Base variable name
order	Integer ≥ 3		Basis function order
sorder	Positive integer	Determined by frame	Geometry shape function order

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```
model.shape().create("shu","f");
model.shape("shu").create("f1","shherm");
model.shape("shu").feature("f1").set("order",3);
model.shape("shu").feature("f1").set("basename","u");
```

The Hermite element defines the following degrees of freedom:

- The value of the variable `basename` at each Lagrange node point that is not adjacent to a corner of the mesh element.
- The values of the first derivatives of `basename` with respect to the global spatial coordinates at each corner of the mesh element. The names of these derivatives are formed by appending the spatial coordinate names to `basename`.

The Hermite element defines the following field variables. Denote `basename` with u , and let x and y denote (not necessarily distinct) spatial coordinates. The variables are (`sdim` = space dimension and `edim` = mesh element dimension):

- u
- ux , meaning the derivative of u with respect to x , defined when `edim` = `sdim` or `edim`=0
- uxy , meaning a second derivative, defined when `edim` = `sdim`
- uTx , the tangential derivative variable, meaning the x -component of the tangential projection of the gradient, defined when $0 < \text{edim} < \text{sdim}$
- $uTxy$, meaning xy -component of the tangential projection of the second derivative, defined when `edim` < `sdim`

When calculating the derivatives, the global spatial coordinates are expressed as polynomials of degree (at most) `sorder` in the local coordinates.

BUBBLE ELEMENTS (SHBUB)

Specify bubble shape functions in the `model.shape` field of the model object. The constructor of a bubble shape function is `shbub`. The following properties are allowed:

TABLE 5-5: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHBUB SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
basename	variable name		Base variable name
mdim	nonnegative integer	<code>sdim</code>	Dimension of the mesh elements on which the bubble exist
sorder	positive integer	Determined by frame	Geometry shape function order

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```
model.shape().create("shu","f");
model.shape("shu").create("f1","shbub");
model.shape("shu").feature("f1").set("mdim",2);
```

```
model.shape("shu").feature("f1").set("basename","u");
```

The bubble element has a single degree of freedom, `basename`, at the midpoint of the mesh element.

The bubble element defines the following field variables. Denote `basename` with u , and let x and y denote (not necessarily distinct) spatial coordinates. The variables are (`sdim` = space dimension and `edim` = mesh element dimension):

- u , defined when $\text{edim} \leq \text{mdim}$, $u = 0$ if $\text{edim} < \text{mdim}$.
- ux , meaning the derivative of u with respect to x , defined when $\text{edim} = \text{mdim} = \text{sdim}$.
- uTx , the tangential derivative variable, meaning the x -component of the tangential projection of the gradient, defined when $\text{mdim} < \text{sdim}$ and $\text{edim} \leq \text{mdim}$. $uTx = 0$ if $\text{edim} < \text{mdim}$.
- $uTxy$, meaning the xy -component of the tangential projection of the second derivative, defined when $\text{mdim} < \text{sdim}$ and $\text{edim} \leq \text{mdim}$. $uTxy = 0$ if $\text{edim} < \text{mdim}$.

THE CURL ELEMENT (SHCURL)

Specify curl shape functions in the `model.shape` field of the model object. The constructor of the curl shape function is `shcurl`. The following properties are allowed:

TABLE 5-6: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHCURL SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
fieldname	string		Field name
compnames	string array	derived from fieldname	Names of vector field components
dofbasename	string	See below	Base name of degrees of freedom
dcompnames	string array	See below	Names of the anti-symmetrized components of the gradient of the vector field
order	integer		Basis function order
sorder	positive integer	Determined by frame	Geometry shape function order

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```
model.shape().create("shu","f");
model.shape("shu").create("f1","shcurl");
model.shape("shu").feature("f1").set("fieldname","E");
model.shape("shu").feature("f1").set("order",2);
model.shape("shu").feature("f1").set("compnames",new String[]{"Ex","Ey"});
model.shape("shu").feature("f1").set("dofbasename","tE");
```

The default for `compnames` is `fieldname` concatenated with the spatial coordinate names. The default for `dofbasename` is `tallcomponents`, where `allcomponents` is the concatenation of the names in `compnames`.

The property `dcompnames` lists the names of the component of the antisymmetric matrix

$$dA_{ij} = \frac{\partial A_j}{\partial x_i} - \frac{\partial A_i}{\partial x_j},$$

where A_i are the vector field components and x_i are the spatial coordinates. The components are listed in row order. If a name is the empty string, the field variable corresponding to that component is not defined. If you have provided `compnames`, the default for the entries in `dcompnames` is `compnames(j) sdimnames(i) compnames(i) sdimnames(j)` for off-diagonal elements. If only `fieldname` has been given, the default for the entries are `dfieldname sdimnames(i) sdimnames(j)`. Diagonal elements are not defined per defaults. For example, `shcurl('order',3,'fieldname','A','dcompnames',{' ',' ','curlAy','curlAz',' ',' ','curlAx',' '})`.

The curl element defines the following degrees of freedom: `dofbasename d c`, where $d = 1$ for DOFs in the interior of an edge, $d = 2$ for DOFs in the interior of a surface, and so forth, and c is a number between 0 and $d - 1$.

The curl element defines the following field variables (where `comp` is a component name from `compnames`, and `dcomp` is a component from `dcompnames`, `sdim` = space dimension and `edim` = mesh element dimension):

- `comp`, meaning a component of the vector, defined when `edim = sdim`.
- `tcomp`, meaning one component of the tangential projection of the vector onto the mesh element, defined when `edim < sdim`.
- `comp $x$` , meaning the derivative of a component of the vector with respect to global spatial coordinate x , defined when `edim = sdim`.
- `tcomp $x$` , the tangential derivative variable, meaning the x component of the projection of the gradient of `tcomp` onto the mesh element, defined when `edim < sdim`. Here, x is the name of a spatial coordinate.
- `dcomp`, meaning a component of the anti-symmetrized gradient, defined when `edim = sdim`.
- `tdcomp`, meaning one component of the tangential projection of the anti-symmetrized gradient onto the mesh element, defined when `edim < sdim`.

For performance reasons, use `dcomp` in expressions involving the curl rather than writing it as the difference of two gradient components.

For the computation of components, the global spatial coordinates are expressed as polynomials of degree (at most) `sorder` in the local coordinates.

THE CURL TYPE 2 ELEMENT (SHCURL2)

Specify curl type 2 shape functions in the `model.shape` field of the model object. The constructor of the curl type 2 shape function is `shcurl2`. The usage of the curl type 2 element is the same as the curl element (`shcurl1`) described above, apart from the difference in the naming of the constructor.

DISCONTINUOUS LAGRANGE (SHDISC) AND NODAL DISCONTINUOUS LAGRANGE ELEMENTS (SHHWDISC)

Specify discontinuous Lagrange shape functions in the `model.shape` field of the model object. The constructor of the discontinuous Lagrange shape functions is either `shdisc`, for discontinuous Lagrange shape functions, or `shhwdisc`, for nodal discontinuous Lagrange shape functions. The difference between these two is that the latter has optimal placement of degrees of freedom on triangular and tetrahedral meshes with respect to certain interpolation error estimates, whereas the former is available on all types of mesh elements with arbitrary polynomial order k . However, the available numerical integration formulas usually limits the usefulness to $k \leq 5$ ($k \leq 4$ for tetrahedral meshes). The following properties are allowed:

TABLE 5-7: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHDISC SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>basename</code>	variable name		Base variable name
<code>order</code>	integer		Basis function order
<code>mdim</code>	nonnegative integer	<code>sdim</code>	Dimension of the mesh elements where the discontinuous element exists
<code>sorder</code>	positive integer	Determined by frame	Geometry shape function order

The `shhwdisc` (nodal discontinuous Lagrange) shape function has the same properties as the `shdisc` (nodal discontinuous Lagrange) shape function, except that the mesh element dimension `mdim` cannot be set; it is instead assumed equal to `sdim`. That is, `shhwdisc` shape functions are only usable on the top dimension of the geometry.

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```
model.shape().create("sh1", "frame1");
```

```

model.shape("sh1").create("f1", "shdisc");
model.shape("sh1").feature("f1").set("order", 2);
model.shape("sh1").feature("f1").set("basename", "u");

```

The discontinuous element defines the following field variables. Denote **basename** with u , and let x denote the spatial coordinates. The variables are (edim is the mesh element dimension):

- u , defined when $\text{edim} = \text{mdim}$.
- ux , meaning the derivative of u with respect to x , defined when $\text{edim} = \text{mdim} = \text{sdim}$.
- uTx , the tangential derivative variable, meaning the derivative of u with respect to x , defined when $\text{edim} = \text{mdim} < \text{sdim}$.

DENSITY ELEMENTS (SHDENS)

Specify density shape functions in the `model.shape` field of the model object. The constructor of the density shape function is `shdens`. The following properties are allowed:

TABLE 5-8: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHDENS SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
basename	variable name		Base variable name
order	integer		Basis function order
sorder	positive integer	Determined by frame	Geometry shape function order

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```

model.shape().create("shu", "f");
model.shape("shu").create("f1", "shdens");
model.shape("shu").feature("f1").set("order", 2);
model.shape("shu").feature("f1").set("basename", "u");

```

The density element defines the following field variables. Denote **basename** with u , and let x denote the spatial coordinates. The variables are (edim is the mesh element dimension):

- u , defined when $\text{edim} = \text{sdim}$.
- ux , meaning the derivative of u with respect to x , defined when $\text{edim} = \text{sdim}$.

GAUSS POINT DATA ELEMENTS (SHGP)

Specify Gauss point data shape functions in the `model.shape` field of the model object. The constructor of the density shape function is `shgp`. The following properties are allowed:

TABLE 5-9: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHGP SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
basename	variable name		Base variable name
order	integer		Basis function order
mdim	integer		Element dimension
valuetype	real complex	complex	Value type in case of using split representation of complex variables ^a

a. The value type is ignored when split representation of complex variables is not used.

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

The following code creates a Gauss point data shape function declaring the degree of freedom u at integration points of order 4 in three-dimensional mesh elements.

```

model.shape().create("shu", "f");
model.shape("shu").create("f1", "shgp");
model.shape("shu").feature("f1").set("order", 4);
model.shape("shu").feature("f1").set("basename", "u");
model.shape("shu").feature("f1").set("mdim", "3");

```

The Gauss point data element defines the following field variables. Denote `basename` with u and let `edim` be the evaluation dimension:

- u , defined when `edim` $\leq$ `mdim`.

DIVERGENCE ELEMENTS (SHDIV)

Syntax for Divergence Elements (shdiv)

Specify divergence shape functions in the `model.shape` field of the model object. The constructor of the divergence shape function is `shdiv`. The following properties are allowed:

TABLE 5-10: VALID PROPERTY NAME/VALUE PAIRS FOR THE SHDIV SHAPE FUNCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>fieldname</code>	variable name		Name of vector field
<code>compnames</code>	string array	Derived from <code>fieldname</code>	Names of vector field components
<code>dofbasename</code>	string	see below	Base name of degrees of freedom
<code>divname</code>	string	see below	Name of divergence field
<code>order</code>	integer	1	Basis function order
<code>sorder</code>	positive integer	Determined by frame	Geometry shape function order

The property names cannot be abbreviated and must be written in lowercase letters enclosed in quotation marks.

```
model.shape().create("shu","f");
model.shape("shu").create("f1","shdiv");
model.shape("shu").feature("f1").set("order",2);
model.shape("shu").feature("f1").set("compnames",new String[]{"Bx","By"});
model.shape("shu").feature("f1").set("dofbasename","nB");
```

The default for `compnames` is `fieldname` concatenated with the spatial coordinate names. The default for `dofbasename` is `nallcomponents`, where `allcomponents` is the concatenation of the names in `compnames`.

The vector element defines the following degrees of freedom: `dofbasename` on element boundaries, and `dofbasename` `sdim` c , $c = 0, \dots, \text{sdim} - 1$ for DOFs in the interior.

The divergence element defines the following field variables (where `comp` is a component name from `compnames`, `divname` is the `divname`, `sdim` = space dimension and `edim` = mesh element dimension):

- `comp`, meaning a component of the vector, defined when `edim` = `sdim`.
- `ncomp`, meaning one component of the projection of the vector onto the normal of mesh element, defined when `edim` = `sdim` - 1.
- `comp $x$` , meaning the derivative of a component of the vector with respect to global spatial coordinate x , defined when `edim` = `sdim`.
- `ncomp $Tx$` , the tangential derivative variable, meaning the x component of the projection of the gradient of `ncomp` onto the mesh element, defined when `edim` < `sdim`. Here, x is the name of a spatial coordinate. `ncomp $Tx$` = 0.
- `divname`, means the divergence of the vector field.

For performance reasons, prefer using `divname` in expressions involving the divergence rather than writing it as the sum of `sdim` gradient components.

For the computation of components, the global spatial coordinates are expressed as polynomials of degree (at most) `sorder` in the local coordinates.

DIVERGENCE TYPE 2 ELEMENTS (SHDIV2)

Specify divergence type 2 shape functions in the `model.shape` field of the model object. The constructor of the divergence type 2 shape function is `shdiv2`. The usage of the divergence type 2 element is the same as the divergence element (`shdiv`) described above, apart from the difference in the naming of the constructor.

Solvers and Study Steps

This chapter contains reference information about the solver command and utility commands for producing and handling solutions. Information about study steps is also included.

In this chapter:

- [About Solver Commands](#)
- [Solution Object Data](#)
- [Studies and Study Steps](#)

About Solver Commands

The following sections describe the solver commands:

	• Features Producing and Manipulating Solutions • Features with Solver Settings • Solution Object Information Methods • Solution Feature Information Methods • Studies and Solvers in the <i>COMSOL Multiphysics Reference Manual</i>
---	---

Features Producing and Manipulating Solutions

Table 6-1 is an overview of the available features for producing and handling solution objects.

TABLE 6-1: SOLUTION OBJECT FEATURES.

FEATURE	PURPOSE
Assemble	Assembles and stores the matrices generated during assembly
AWE	Solve parametric problem with asymptotic waveform evaluation
CombineSolution	Combine solutions by concatenation or summation
CopySolution	Copy solution
Eigenvalue	Solve eigenvalue problem
EigenvalueAttrib	Eigenvalue solver attribute to Sensitivity and Optimization solvers
EigenvalueParam	Solve parametric eigenvalue problem
FFT	Perform a forward or inverse fast Fourier transform (FFT or IFFT)
InputMatrix	Input matrices or vectors (for example, load vectors or stiffness matrices) to a solver
Modal	Solve time-dependent or parametric problem with modal analysis
ModalReduction	Solve time-dependent or parametric problem with modal reduction
ModeFollowing	Follow eigenfunctions of parametric eigenvalue problem
Optimization	Solve optimization problem
PlugFlow	Solve stationary plug flow problem
ProperOrthogonalDecomposition	Solve time-dependent or frequency-domain problem with POD reduction
StateSpace	Assembles and stores matrices that describe a model as a dynamic system
Stationary	Solve stationary problem
StudyStep	Specifies which problem to compile
Time	Solve time-dependent problem with implicit or explicit time stepping
TimeDiscrete	Solve time-dependent problem with user's own time stepping
TimeExplicit	Solve time-dependent problem with explicit time stepping
TimeParametric	Handle properties for parameter stepping for a time-dependent problem.
Variables	Handle variables solved for (initial values, scaling) and not solved for (prescribed values)

Features with Solver Settings

Table 6-2 is an overview of the available features for solver settings.

TABLE 6-2: SOLVER SETTING FEATURES.

FEATURE	SETTINGS HANDLED
Adaption	Adaptation subfeature (created by the study step)
Advanced	Advanced general settings
AutoRemesh	Automatically remesh deformed geometries
ControlField	Control fields (a set of control variables)
ControlState	Set of global control variables
Field	Fields (a set of dependent variables)
FullyCoupled	Fully coupled nonlinear solution approach
HardwareAcceleration	Hardware acceleration using GPU for dG-based transient acoustics.
LowerLimit	Lower limits.
LumpedStep	Lumped steps.
Parametric	Parameter stepping
Previous Solution	Previous solution solvers
Segregated	Segregated nonlinear solution approach
SegregatedStep	Segregated steps
Sensitivity	Sensitivity analysis
StatAcceleration	Stationary acceleration
StopCondition	Stop conditions
TimeAdaption	Time-dependent adaptive mesh refinement
UpperLimit	Upper limits.
See Linear for the following:	
Direct	Direct linear system solvers
DirectPreconditioner	Direct linear system solvers as preconditioners
DomainDecomposition	Domain decomposition solver (Schwarz)
DomainDecompositionSchur	Domain decomposition solver (Schur)
HierarchicalLU	Hierarchical LU linear system preconditioner (for BEM)
IncompleteLU	Incomplete factorization preconditioners
Iterative	Iterative linear system solvers
Jacobi	Jacobi linear system preconditioners
KrylovPreconditioner	Krylov linear system preconditioners
SchurKrylovPreconditioner	Krylov linear system preconditioners for a Schur solver
Multigrid	Multigrid linear system preconditioners
SCGS	SCGS linear system preconditioners
SOR	SOR linear system preconditioners
SORGauge	SOR Gauge linear system preconditioners
SORLine	SOR Line linear system preconditioners
SORVector	SOR Vector linear system preconditioners
SchurLocal	Localized Schur solver
SchurSpLocal	Sparse localized Schur solver

TABLE 6-2: SOLVER SETTING FEATURES.

FEATURE	SETTINGS HANDLED
State	Sets of global dependent variables
Vanka	Vanka linear system preconditioners

Solution Object Information Methods

The following tables are an overview of the solution object information methods.

GENERAL INFORMATION

TABLE 6-3: GENERAL SOLUTION INFORMATION METHODS.

METHOD	DESCRIPTION
getType	Get solution type
getSize	Get number of dynamic solutions and length of solution vector
getSizeMulti	Get number of local solution objects and total number of solutions
getMesh	Get mesh name associated with solution and geometry
getNU	Get number of solutions of a certain solution data type
getPNames	Get parameter names
getParamName	Get parameter names for parametric sweep
getParamVals	Get parameter values for parametric sweep

SOLUTION DATA

TABLE 6-4: SOLUTION DATA ACCESS METHODS.

METHOD	DESCRIPTION
getU	Get real part of solution vector
getUDot	Get real part of the first time-derivative solution vector
getUImag	Get imaginary part of solution vector
getUDotImag	Get imaginary part of first time-derivative solution vector
getPVals	Get the real part of the parameter values
getPValsImag	Get the imaginary part of the parameter values
getUBlock	A blocked version of the getU method
getUDotBlock	A blocked version of the getUDot method
getUImagBlock	A blocked version of the getUImag method
getUDotImagBlock	A blocked version of the getUDotImag method

SOLUTION CREATION

TABLE 6-5: SOLUTION CREATION METHODS.

METHOD	DESCRIPTION
setU	Set real part of solution vector
setUDot	Set real part of the first time-derivative solution vector
setUImag	Set imaginary part of solution vector
setUDotImag	Set imaginary part of first time-derivative solution vector
setPVals	Set the real part of the parameter values
setPValsImag	Set the imaginary part of the parameter values
setUBlock	A blocked version of the setU method
setUDotBlock	A blocked version of the setUDot method
setUImagBlock	A blocked version of the setUImag method
setUDotImagBlock	A blocked version of the setUDotImag method

ERRORS AND WARNINGS

See [Errors and Warnings](#) in the *General Commands* chapter for handling of errors and warnings in solver sequences.

Solution Feature Information Methods

Some solver features, including Assemble, Modal, ModalReduction, and StateSpace, create and store matrices and vectors. These are accessed by name using the following common methods.

GENERAL INFORMATION

TABLE 6-6: GENERAL MATRIX INFORMATION METHODS.

METHOD	DESCRIPTION
isReal	Check if matrix is real
getM	Get number of rows
getN	Get number of columns
getNnz	Get number of nonzeros in sparse matrix

MATRIX DATA

TABLE 6-7: MATRIX ACCESS METHODS.

METHOD	DESCRIPTION
getSparseMatrixVal	Get matrix values
getSparseMatrixValImag	Get the imaginary matrix values
getSparseMatrixCol	Get column indices of matrix values
getSparseMatrixRow	Get row indices of matrix values
getVector	Get the vector associated with the matrix type
getVectorImag	Get the imaginary part of the vector associated with the matrix type
getSparseMatrixValBlock	A blocked version of getSparseMatrixVal
getSparseMatrixValImagBlock	A blocked version of getSparseMatrixValImag
getSparseMatrixColBlock	A blocked version of getSparseMatrixCol
getSparseMatrixRowBlock	A blocked version of getSparseMatrixRow
getVectorBlock	A blocked version of getVector
getVectorImagBlock	A blocked version of getVectorImag

MATRIX CREATION

The InputMatrix solver feature allows declaring and defining input matrices using the following methods.

TABLE 6-8: MATRIX CREATION METHODS.

METHOD	DESCRIPTION
createSparseMatrix	Create sparse matrix
addSparseMatrixVal	Add matrix values to the created matrix
addSparseMatrixValImag	Add imaginary matrix values to the created matrix
createVector	Create vector
setVector	Set the vector associated with the matrix type
setVectorImag	Set the imaginary part of the vector associated with the matrix type
setVectorBlock	A blocked version of setVector
setVectorImagBlock	A blocked version of setVectorImag

Solution Object Data

The solver sequence works as a solution object itself. The solution object data produced by running the sequence (partially or in whole) can be obtained by a number of access methods on the sequence. See [Table 6-3](#) and [Table 6-4](#) for an overview.

In this section:

- [General Information](#)
- [Solution Data](#)
- [SolutionInfo Object and Its Methods](#)
- [Solution Creation](#)
- [General Matrix Information](#)
- [Matrix Data](#)
- [Matrix Creation](#)

General Information

This section contains information about general solution object information methods, listed in the following table and further described below.

TABLE 6-9: GENERAL SOLUTION OBJECT INFORMATION METHODS.

METHOD	OUTPUT TYPE
<code>isAttached()</code>	boolean
<code>isRealU()</code>	boolean
<code>isRealU(int)</code>	boolean
<code>isRealU(int,string)</code>	boolean
<code>isRealU(int,string,int)</code>	boolean
<code>isRealUDot()</code>	boolean
<code>isRealUDot(int)</code>	boolean
<code>isRealPVals()</code>	boolean
<code>getType()</code>	String
<code>getSize()</code>	int[2]
<code>getSize(int)</code>	int[2]
<code>getSizeMulti()</code>	int[2]
<code>getMesh(string)</code>	String
<code>getMesh(string,int)</code>	String
<code>getNU(String)</code>	int
<code>getPNames()</code>	String[]
<code>getPVals()</code>	double[]
<code>getParamNames()</code>	String[]
<code>getParamVals()</code>	double[]
<code>getSolutioninfo()</code>	SolutionInfo

- `model.sol(<tag>).isAttached()` returns true if the solution solver sequence is attached to a study.
- `model.sol(<tag>).isRealU()` returns true if the solution is real.
- `model.sol(<tag>).isRealU(<solnum>)` returns true if solution <solnum> is real.

- `model.sol(<tag>).isRealU(<solnum>, <uType>)` returns true if the solution `<solnum>` of type `<uType>` is real.
- `model.sol(<tag>).isRealU(<solnum>, <uType>, <uNum>)` returns true if the solution `<solnum>` of type `<uType>` and solution index `<uNum>` is real.
- `model.sol(<tag>).isRealUDot()` returns true if the first time derivative is real.
- `model.sol(<tag>).isRealUDot(<solnum>)` returns true if first time derivative `<solnum>` is real.
- `model.sol(<tag>).isRealPVals()` returns true if the parameter values are real.
- `model.sol(<tag>).isRealU()` returns true if the solution vector is real.
- `model.sol(<tag>).getType()` returns a string for the solution type which can be any of the strings; Stationary, Parametric, Time, Eigenvalue, and None.
- `model.sol(<tag>).getSequenceType()` returns a string for the solver sequence type, which can be any of the strings SolverSequence, CopySolution, ParametricStore, Stored, Parametric, and None.
- `model.sol(<tag>).getSize()` returns an array of sizes for the solution data. The number of degrees of freedoms is stored in the first position and the number of solutions (`solnums`) in the second.
- `model.sol(<tag>).getSize(<iMulti>)` returns an array of sizes for the solution number `<iMulti>` of the multisolution. A *multisolution* includes multiple solutions from automatic remeshing in single solution object. The number of degrees of freedoms is stored in the first position and the number of solutions (`solnums`) in the second.
- `model.sol(<tag>).getSizeMulti()` returns an array of sizes for the multisolution. The number of local solution objects is stored in the first position and the total number of solutions (`solnums`) in the second.
- `model.sol(<tag>).getMesh(<geom>)` returns the mesh name associated with the solution and the geometry `<geom>`.
- `model.sol(<tag>).getMesh(<geom>, <iMulti>)` returns the mesh name associated with the solution number `<iMulti>` of the multisolution and the geometry `<geom>`.
- `model.sol(<tag>).getNU(<uType>)` returns the number of solutions stored of the type `<uType>`. Here `<uType>` is the solution type as a string: Sol (main solution), Reacf (reaction force), Adj (adjoint solution), Fsens (functional sensitivity), and Sens (forward sensitivity).
- `model.sol(<tag>).getPNames()` returns the parameter names from continuation solver as an array of strings.
- `model.sol(<tag>).getParamNames()` returns the parameter names from a parametric sweep as an array of strings.
- `model.sol(<tag>).getParamVals()` returns the parameter values from a parametric sweep as an array of double.

Solution Data

TABLE 6-10: SOLUTION DATA ACCESS METHODS, REAL PART.

METHOD	OUTPUT TYPE
getU(int,string,int)	double[]
getU(int,string)	double[]
getU(int)	double[]
getU()	double[]
getUDot()	double[]
getUDot(int)	double[]
getPVals()	double[]
getPVals(int)	double[]

- `model.sol(<tag>).getU(<solnum>,<uType>,<uNum>)` returns the real part of the solution vector for solution number `<solnum>`, the solution type `<uType>`, and the 1-based solution index `<uNum>`. Here, $1 \leq \text{<uNum>} \leq N$, where $N = \text{model.sol}(\text{<tag>}).\text{getNU}(\text{<uType>})$. The `<uType>` type is the solution type as a string: `Sol` (main solution), `Reacf` (reaction force), `Adj` (adjoint solution), `Fsens` (functional sensitivity), and `Sens` (forward sensitivity).
- `model.sol(<tag>).getU(<solnum>,<uType>)` returns the real part of the solution vector for the solution number `<solnum>` and the solution data type `<uType>`. The solution index `<uNum>=1`. The `<uType>` type is the solution type as a string: `Sol` (main solution), `Reacf` (reaction force), `Adj` (adjoint solution), `Fsens` (functional sensitivity), and `Sens` (forward sensitivity).
- `model.sol(<tag>).getU(<solnum>)` returns the real part of the solution vector for solution number `<solnum>`. The solution data type `<uType>=Sol` and the solution index `<uNum>=1`.
- `model.sol(<tag>).getU()` returns the real part of the solution vector. For a Time-dependent and Parametric type, the last solution number is used, and for an Eigenvalue type the first solution number. The solution data type `<uType>=Sol` and the solution index `<uNum>=1`.
- `model.sol(<tag>).getUDot()` returns the real part of the first time-derivative solution vector for a Time-dependent type and if the time derivatives have been stored. The last solution number is used. For other types and if the time derivatives have not been stored, an error message is given.
- `model.sol(<tag>).getUDot(<solnum>)` returns the real part of the first time-derivative solution vector for the solution number `<solnum>`.
- `model.sol(<tag>).getPVals()` returns for a solution of a Parametric type the real part of all the parameter values stored. For multiple parameters all parameter tuples are concatenated. For a solution of a Time-dependent type, this is the time for which solution data is stored. For a solution of an Eigenvalue type, this is the real part of the eigenvalues stored. For a Time-dependent and Parametric type, the last solution number is used, and for an Eigenvalue type the first solution number.
- `model.sol(<tag>).getPVals(<solnum>)` returns for a solution of a Parametric type the real part of the parameter tuples stored for solution number `<solnum>`. For a solution of a Time-dependent type, this is the time for solution number `<solnum>`. For a solution of an Eigenvalue type, this is the real part of the eigenvalue stored at solution number `<solnum>`.

TABLE 6-11: SOLUTION DATA ACCESS METHODS, IMAGINARY PART.

METHOD	OUTPUT TYPE
getUImag()	double[]
getUImag(int)	double[]
getUImag(int,string)	double[]

TABLE 6-11: SOLUTION DATA ACCESS METHODS, IMAGINARY PART.

METHOD	OUTPUT TYPE
<code>getUImag(int,string,int)</code>	<code>double[]</code>
<code>getUDotImag()</code>	<code>double[]</code>
<code>getUDotImag(int)</code>	<code>double[]</code>
<code>getPValsImag()</code>	<code>double[]</code>
<code>getPValsImag(int)</code>	<code>double[]</code>

- `model.sol(<tag>).getUImag()` returns the imaginary part of the solution vector. The same `<solnum>`, `<uType>`, and `<uNum>` is used as for the method `getU()`. And similarly for the other `Imag` methods.

TABLE 6-12: SOLUTION DATA ACCESS METHODS, BLOCKED VERSIONS.

METHOD	OUTPUT TYPE
<code>getUBlock(int,int)</code>	<code>double[]</code>
<code>getUBlock(int,int,int)</code>	<code>double[]</code>
<code>getUBlock(int,string,int,int)</code>	<code>double[]</code>
<code>getUBlock(int,string,int,int,int)</code>	<code>double[]</code>
<code>getUDotBlock(int,int)</code>	<code>double[]</code>
<code>getUDotBlock(int,int,int)</code>	<code>double[]</code>
<code>getUImagBlock(int,int)</code>	<code>double[]</code>
<code>getUImagBlock(int,int,int)</code>	<code>double[]</code>
<code>getUImagBlock(int,string,int,int)</code>	<code>double[]</code>
<code>getUImagBlock(int,string,int,int,int)</code>	<code>double[]</code>
<code>getUDotImagBlock(int,int)</code>	<code>double[]</code>
<code>getUDotImagBlock(int,int,int)</code>	<code>double[]</code>

- `model.sol(<tag>).getUBlock(<startpos>,<endpos>)` returns a subset of the vector returned by `getU()`, the subarray from position `<startpos>` to the position `<endpos>`. And similarly for the other `Block` methods.

SolutionInfo Object and Its Methods

For each solver sequence, there is an associated `SolutionInfo` object, which can be accessed by the function `getSolutionInfo()`. This object has several methods to access the solution data generated by a parametric sweep. Such parametric sweep generated solution data is normally stored in solver sequences of type `Parametric` or of type `SolverSequence`, depending on whether so-called outer parametric sweeps have been used or not. It can also be used to convert between so-called loop-level settings and solution numbers. The following methods in [Table 6-13](#) are supported.

TABLE 6-13: SOLUTIONINFO METHODS.

METHOD	OUTPUT TYPE
<code>getIndices(int, int[])</code>	<code>int[]</code>
<code>getISol(int, double)</code>	<code>int</code>
<code>getISol(int, int)</code>	<code>int[]</code>
<code>getLevelDescription(int)</code>	<code>String</code>
<code>getLevelNames()</code>	<code>String[]</code>
<code>getLevels()</code>	<code>int</code>
<code>getMaxInner(int[])</code>	<code>int</code>
<code>getMaxLevels()</code>	<code>int</code>

TABLE 6-13: SOLUTIONINFO METHODS.

METHOD	OUTPUT TYPE
getName(int)	String
getOuterSolnum()	int[]
getPNamesOuter()	String[]
getPUnitsOuter()	String[]
getSol(int)	String
getSolDescriptions(int, int[], boolean, boolean)	String[]
getSolnum(int, boolean)	int[]
getSplitLevelDescriptions()	String[]
getSplitLevelNames()	String[]
getSplitLevelUnits()	String[]
getSplitName(int)	String[]
getSplitUnit(int)	String[]
getUnit(String)	String
getVals(int, int[])	double[][]
getValsImag(int, int[])	double[][]
isStructured()	boolean
isValid()	boolean
mapToLevel(int[], int[], boolean)	int[][]
mapToSolnum(int[][][], boolean)	int[][]

```
info = model.sol(<tag>).getSolutioninfo()
```

- **getIndices(int level, int[] levels)** Returns the one-based indices available for the loop level, `level` (index-zero based). If the format is structured, `levels` can be set to `null` as it does not affect the result. If the format is unstructured, and defining $X = \text{levels}[i]$, then the level i is set to its X -th value or tuple. The returned values are $1, \dots, N$, where N is the number of values or tuples for the given level. When `level=getMaxLevel()-1`, the indices can be the result of an outer product between levels. When the format is unstructured, and when `levels` is set, then the unstructured list of indices is returned. When the format is unstructured, and when `levels=null` or `levels.length=0`, an error is given if `level` is such that there are no structured data to return (currently `level=0` and multiple inner parameter names).

An example: A parameter `p` taking the values 1 and 2, for which a time-dependent simulation is done. All time steps from the solver are saved. Assume that `p` is affecting the time stepping. Using the Time Parametric solver (or a Parametric Sweep for a time-dependent study) will result in a `SolutionInfo` object with two levels, one for the time `t` (level 0) and one for `p` (level 1). To access the indices for the solutions on level 0, use the method `getIndices`:

```
SolverSequence sol = model.sol("sol1");
SolutionInfo info = sol.getSolutioninfo();
int [] indx_1 = info.getIndices(0, int [] {1,1});
int [] indx_2 = info.getIndices(0, int [] {1,2});
```

- **getISol(int outersolnum, int innersolnum)** returns the index zero-based multisolution object number and the index zero based solution number within it, for the index one based outer and inner solution numbers. The solution object number is returned in the first position and the corresponding solution number in the second. The solution object number is normally the same (0) for all `innersolnum`, but can vary for time-dependent adaptation or for automatic remeshing.

- `getISol(int outersolnum, double t)` returns the index-zero based multisolution object number for the one-based outer solution number `outersolnum` and time value `t`. The returned solution number is normally the same (0) for all `t` but can vary for time-dependent adaptation or for automatic remeshing.
- `getLevelDescription(int level)` returns a description of the index zero-based loop level, `level`.
- `getLevelNames()` returns the names of the different loop levels. Some of these can be a concatenated string such as "p1,p2".
- `getLevels()` returns the number of loop levels, `getLevels()<=getMaxLevels()`.
- `getMaxInner(int[] outersolnum)` returns the maximum number of inner solutions for the given index one based outer solution numbers. If `outersolnum` is null, the maximum is taken over all outer solutions.
- `getMaxLevels()` returns the maximum number of used loop levels, `getLevels()<=getMaxLevels()`.
- `getName(int level)` returns the parameter name for the index zero based loop level `level`. This name can be a concatenated string such as "p1, p2".
- `getOuterSolnum()` returns the one based indices for the outer solutions. If there are no outer parameters or added corresponding parameter values the array is empty.
- `getPNamesOuter()` returns the subset of parameter names that are looped by a job sequence parametric sweep.
- `getSol(int outersolnum)` returns the solver sequence tag for the index one based outer solution number `outersolnum`. If the solution number is invalid, null is returned.
- `getSolDescriptions(int level, int[] levels, boolean paramInclusion, boolean indexInclusion)` returns the descriptions for the solutions for the index-zero based loop level, `level`. The current level setting can be given in `levels` (index one based). One string for each solution is returned. When `paramInclusion` is true, the description always includes the parameter name, even if this level contains only one. When `paramInclusion` is false, the parameter name is only included when there is more than one parameter name on this level. When the format is unstructured, and when `levels` is set, then the unstructured list of descriptions is returned. When the format is unstructured, and when `levels==null` or `levels.length==0`, an error is given if `level` is such that there are no structured data to return.
- `getSolnum(int outersolnum, boolean strict)` returns the one based inner solution numbers for the index one based outer solution number `outersolnum`. If `strict` is true the inner solution numbers is returned if `outersolnum` is a valid outer solution number and else a zero array is returned. If `strict` is false and if the `outersolnum` does not match, then the solution numbers for the containing solution object is returned.
- `getSplitLevelDescriptions()` returns the description of the different parameters, split into an array for the case when there is more than one parameter for a loop level.
- `getSplitLevelNames()` returns the names of the different parameters, split into an array for the case when there is more than one parameter for a loop level.
- `getSplitLevelUnits()` returns the units for the parameters. It returns a vector of the same length and order as `getSplitLevelDescriptions`, with null as the contents when units are not used or defined.
- `getSplitNames(int level)` returns the parameter names for the index-zero based loop level `level`.
- `getUnit(String name)` returns the unit of the sweep parameter name.
- `getVals(int level, int[] levels)` returns the parameter values for the index-zero based loop level, `level`. The current level setting can be given in `levels` (index-one based). For `level < getMaxLevels()-1` this is just the values of the parameters for this level. The number of rows is the same as the number of parameters for this level. The columns are the values. For `level = getMaxLevels()-1` the values are expanded into tuples for the case that levels have been merged. When the format is unstructured, and when `levels` is set, then the unstructured lists of values are returned. When the format is unstructured, and when `levels==null` or `levels.length=0`, an error is given if `level` is such that there are no structured data to return.

- `getValsImag(int level, int[] levels)` returns the imaginary parts of the parameter values for the index-zero based loop level, `level`. The current level setting can be given in `levels` (index-one based). For `level < getMaxLevels() - 1` this is just the values of the parameters for this level. The number of rows is the same as the number of parameters for this level. The columns are the values. For `level = getMaxLevels() - 1` the values are expanded into tuples for the case that levels have been merged. When the format is unstructured, and when `levels` is set, then the unstructured lists of values are returned. When the format is unstructured, and when `levels==null` or `levels.length=0`, an error is given if `level` is such that there are no structured data to return.
- `isStructured()` returns true unless the underlying solution object/objects has a parameter variation that depends on the solution process itself. Examples are time-dependent simulations where the output is determined by the steps taken by the solver or eigenvalue simulations.
- `isValid()` returns true if the underlying solution data is consistent with this info object.
- `mapToLevel(int[] outersolnum, int[] innersolnum, boolean compressedOutput)` returns the index one based level representation of the index-one based outer and inner solution numbers, `outersolnum` and `innersolnum` respectively. The number of rows of the returned data is equal to the number of levels. When `compressedOutput` is false, the columns represent the tuples, which is the most general format. When `compressedOutput` is true, the level settings are made unique on each level.



When `compressedOutput` is true and if the compressed representation does not match the input, an array with the right number of rows, but each with zero length, are returned.

- `mapToSolnum(int[][] levelSetting, boolean expandInput)` returns the one based solution number representation of a loop level setting `levelSetting`. The first row in the output is the inner and the second the outer solution numbers. The `levelSettings` must have the same number of rows as there are levels. On each row, index one based settings for each level should be given. If `expandInput` is false the number of columns must be the same and the columns are treated as level-tuples. If `expandInput` is true, the number of columns can be different and the output is expanded to the outer product of each levels setting.

Solution Creation

TABLE 6-14: SOLUTION CREATION METHODS, REAL PART.

METHOD	OUTPUT TYPE
<code>setU(double[])</code>	
<code>setU(int,double[])</code>	
<code>setPNames(String[])</code>	
<code>setPVals(double[])</code>	
<code>setPVals(int,double[])</code>	
<code>createSolution()</code>	

- `model.sol(<tag>).setU(<vals>)` sets the real part of the solution vector to `<vals>`.
- `model.sol(<tag>).setU(<solnum>,<vals>)` sets the real part of solution vector `<solnum>`, to `<vals>`.
- `model.sol(<tag>).setPNames(<pnames>)` sets parameter names of the solution vectors to `<pnames>`.
- `model.sol(<tag>).setPVals(<vals>)` sets the parameter values to `<vals>`.
- `model.sol(<tag>).setPVals(<solnum>,<vals>)` sets the parameter values `<solnum>` to `<vals>`.

- `model.sol(<tag>).createSolution()` creates solutions based on the input from the vectors previously set. The solution is created at this stage. Afterward the user input is cleared. If a created solution is used before this function is run the result is unpredictable.

TABLE 6-15: SOLUTION CREATION METHODS, IMAGINARY PART.

METHOD	OUTPUT TYPE
<code>setUImag(double[])</code>	
<code>setUImag(int,double[])</code>	
<code>setPValsImag(double[])</code>	
<code>setPValsImag(int,double[])</code>	

- `model.sol(<tag>).setUImag(<solnum>,<vals>)` sets the imaginary part of solution vector `<solnum>` to `<vals>` (and similarly for the other `Imag` methods).

TABLE 6-16: SOLUTION CREATION METHODS, REAL PART.

METHOD	OUTPUT TYPE
<code>setUBlock(double[],int)</code>	
<code>setUBlock(int,double[],int)</code>	
<code>setUBImaglock(double[],int)</code>	
<code>setUImagBlock(int,double[],int)</code>	
<code>setUDotBlock(double[],int)</code>	
<code>setUDotBlock(int,double[],int)</code>	
<code>setUImagBlock(double[],int)</code>	
<code>setUImagBlock(int,double[],int)</code>	

- `model.sol(<tag>).setUBlock(<solnum>,<vals>,<start>)` sets the real part of solution vector `<solnum>`, the subarray from position `<start>` to position `<start>+<vals>.length-1` to `<vals>` (and similarly for the other `Imag` methods).

General Matrix Information

TABLE 6-17: GENERAL MATRIX OBJECT INFORMATION METHODS.

METHOD	OUTPUT TYPE
<code>isReal(String)</code>	<code>boolean</code>
<code>getM(String)</code>	<code>int</code>
<code>getN(String)</code>	<code>int</code>
<code>getNnz(String)</code>	<code>int</code>

- `model.sol(<tag>).feature(<ftag>).isReal(<mname>)` returns true if the matrix `<mname>` is real.
- `model.sol(<tag>).feature(<ftag>).getM(<mname>)` returns number of rows in the matrix `<mname>`.
- `model.sol(<tag>).feature(<ftag>).getN(<mname>)` returns number of columns in the matrix `<mname>`.
- `model.sol(<tag>).feature(<ftag>).getNnz(<mname>)` returns number of nonzero entries in the matrix `<mname>`.

The following table lists available matrices and vectors (availability depends on the solvers used in the simulation):

TABLE 6-18: AVAILABLE MATRIX AND VECTOR NAMES.

PROPERTY	DESCRIPTION
K	Stiffness matrix
L	Load vector

TABLE 6-18: AVAILABLE MATRIX AND VECTOR NAMES.

PROPERTY	DESCRIPTION
M	Constraint vector
N	Constraint Jacobian
D	Damping matrix
E	Mass matrix
EL	Lumped mass matrix
NF	Constraint force Jacobian
NP	Optimization constraint Jacobian
MP	Optimization constraint vector
MLB	Lower bound constraint vector
MUB	Upper bound constraint vector
Kc	Eliminated stiffness matrix
Lc	Eliminated load vector
Dc	Eliminated damping matrix
Ec	Eliminated mass matrix
ELc	Eliminated lumped mass matrix
Null	Constraint null-space basis
Nullf	Constraint force null-space basis
ud	Particular solution ud
uscale	Scale vector

Matrix Data

TABLE 6-19: MATRIX DATA ACCESS METHODS, REAL PART.

METHOD	OUTPUT TYPE
getSparseMatrixVal(String)	double[]
getSparseMatrixCol(String)	int[]
getSparseMatrixRow(String)	int[]
getVector(String)	double[]

See [Ref. 6-18](#) above for a list of available matrix and vector names.

- `model.sol(<tag>).feature(<ftag>).getSparseMatrixVal(<mname>)` returns the real part of the sparse matrix values of matrix `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “K”, “D”, “E”, “EL”, “N”, “NF”, “NP”, “Kc”, “Dc”, “Ec”, “ELc”, “Null”, “Nullf”, “Mc”, “MA”, “MB”, or “C”.
- `model.sol(<tag>).feature(<ftag>).getSparseMatrixCol(<mname>)` returns the column numbers of the sparse matrix values of matrix `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “K”, “D”, “E”, “EL”, “N”, “NF”, “NP”, “Kc”, “Dc”, “Ec”, “ELc”, “Null”, “Nullf”, “Mc”, “MA”, “MB”, or “C”.
- `model.sol(<tag>).feature(<ftag>).getSparseMatrixRow(<mname>)` returns the row numbers of the sparse matrix values of matrix `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “K”, “D”, “E”, “EL”, “N”, “NF”, “NP”, “Kc”, “Dc”, “Ec”, “ELc”, “Null”, “Nullf”, “Mc”, “MA”, “MB”, or “C”.

- `model.sol(<tag>).feature(<ftag>).getVector(<mname>)` returns the real part of the vector `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “L”, “M”, “MP”, “MLB”, “MUB”, “ud”, “uscale”, or “x0”.

TABLE 6-20: MATRIX DATA ACCESS METHODS, IMAGINARY PART.

METHOD	OUTPUT TYPE
<code>getSparseMatrixValImag(String)</code>	<code>double[]</code>
<code>getVectorImag(String)</code>	<code>double[]</code>

- `model.sol(<tag>).feature(<ftag>).getSparseMatrixValImag(<mname>)` returns the imaginary part of the sparse matrix values of matrix `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “K”, “D”, “E”, “EL”, “N”, “NF”, “NP”, “Kc”, “Dc”, “Ec”, “ELc”, “Null”, “Nullf”, “Mc”, “MA”, “MB”, “C”.
- `model.sol(<tag>).feature(<ftag>).getVectorImag(<mname>)` returns the imaginary part of the vector `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “L”, “M”, “MP”, “MLB”, “MUB”, “ud”, “uscale”, or “x0”.

TABLE 6-21: MATRIX DATA ACCESS METHODS, BLOCKED VERSIONS.

METHOD	OUTPUT TYPE
<code>getSparseMatrixValBlock(String,int,int)</code>	<code>double[]</code>
<code>getSparseMatrixColBlock(String,int,int)</code>	<code>int[]</code>
<code>getSparseMatrixRowBlock(String,int,int)</code>	<code>int[]</code>
<code>getVectorBlock(String,int,int)</code>	<code>double[]</code>
<code>getSparseMatrixValImagBlock(String,int,int)</code>	<code>double[]</code>
<code>getVectorImagBlock(String,int,int)</code>	<code>double[]</code>

- `model.sol(<tag>).feature(<ftag>).getSparseMatrixValBlock(<mname>,<startpos>,<endpos>)` returns a subset of the real part of the sparse matrix values returned by `getSparseMatrixVal(<mname>)`, the subarray from the position `<startpos>` to the position `<endpos>`. And similarly for the other **Block** methods.
- `model.sol(<tag>).feature(<ftag>).getVectorBlock(<mname>,<vals>)` returns a subset of the real part of the vector values returned by `getVector(<mname>)`, the subvector from the position `<startpos>` to the position `<endpos>`. Here, `<mname>` is “L” or “M”.

Matrix Creation

TABLE 6-22: MATRIX DATA CREATION METHODS, REAL PART.

METHOD	OUTPUT TYPE
<code>createSparseMatrix(String,int,int,int,boolean)</code>	<code>void</code>
<code>addSparseMatrixVal(String,int[],int[],double[])</code>	<code>void</code>
<code>createVector(String,int,boolean)</code>	<code>void</code>
<code>setVector(String,double[])</code>	<code>void</code>

- `model.sol(<tag>).feature(<ftag>).createSparseMatrix(<mname>,<M>,<N>,<Nnz>,<isReal>)` creates a sparse matrix `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “K”, “D”, “E”, “N”, or “NF”; `<M>` is the number of rows; `<N>` is the number of columns; `<Nnz>` is the number of nonzeros; and `<isReal>` is true if the matrix is real.
- `model.sol(<tag>).feature(<ftag>).addSparseMatrixVal(<mname>,<row>,<col>,<val>)` adds the values stored in `<val>` to the sparse matrix `<mname>` stored in the feature `<ftag>`. Here `<mname>` is one of “K”, “D”, “E”, “N”, or “NF”; `<row>` lists the rows; `<col>` lists the columns; and `<val>` are the values of the entries.
- `model.sol(<tag>).feature(<ftag>).createVector(<mname>,<M>,<isReal>)` creates a vector `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of “L”, or “M”; `<M>` is the size of the vector; and `<isReal>` is true if the vector is real.

- `model.sol(<tag>).feature(<ftag>).setVector(<mname>,<val>)` sets the real part of the vector `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is "L" or "M" and `<val>` are the values to store in the vector.

TABLE 6-23: MATRIX DATA CREATION METHODS, IMAGINARY PART.

METHOD	OUTPUT TYPE
<code>addSparseMatrixValImag(String,int[],int[],double[])</code>	void
<code>setVectorImag(String,double[])</code>	void

- `model.sol(<tag>).feature(<ftag>).addSparseMatrixValImag(<mname>,<row>,<col>,<val>)` creates the imaginary part of the sparse matrix values of matrix `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is one of "K", "D", "E", "N", or "NF"; `<row>` lists the rows; `<col>` lists the columns; and `<val>` are the imaginary values of the entries.
- `model.sol(<tag>).feature(<ftag>).setVectorImag(<mname>,<val>)` sets the imaginary part of the vector `<mname>` stored in the feature `<ftag>`. Here, `<mname>` is "L" or "M" and `<val>` are the imaginary values to store in the vector.

TABLE 6-24: MATRIX DATA CREATION METHODS, BLOCKED VERSIONS.

METHOD	OUTPUT TYPE
<code>setVectorBlock(String,double[],int)</code>	void
<code>setVectorImagBlock(String,double[],int)</code>	void

- `model.sol(<tag>).feature(<ftag>).setVectorBlock(<mname>,<vals>,<startpos>)` sets a subset of the real part of the vector values set by `setVector(<mname>,<vals>)`, the subvector from the position `<startpos>`. Here, `<mname>` is "L" or "M".

The *clist* and *cname* Properties

The `cname` property is a string array where you provide names of constants as input parameters. The `clist` property is a string array where you provide values for constants as input parameters. The `clist` array must have the same length as the `cname` array. These constants overrule any previous definition (for example, from Global Definitions). Each string is evaluated as a list of values. This means, for example, that the `range()` operator, units, and global expressions are supported. The first entry in each of these lists will be used to temporarily set the corresponding parameter as given by `cname`. This parameter can be a new or an existing global parameter. The constant is temporary in the sense that it is only defined during the solver run. It cannot be evaluated during results processing; in fact, it will be unknown if it is not an existing global parameter, and it will use its default value if it is. This is similar to other solver parameters, like `t` and `timestep`, behave.

For example, a "bootstrapped" definition like

```
"cname" = {"foo", "bar"}
"clist" = {"5", "foo+2"}
```

also works, but in the COMSOL Desktop you will get an error about "foo" for not being recognized in the evaluation context. And if you use a circular dependency like

```
"cname" = {"foo", "bar"}
"clist" = {"bar", "foo+2"}
```

you will get warnings about this if the parameters are used, and the evaluation will be NaN (not-a-number).

If "a" and "b" are already defined as global parameters and, say, `b=a+1`; then redefining "a" to a new value through `cname` and `clist` leads to the value for "b" being redefined (correctly).

You cannot override parameters used in the following parts of the model:

- Mesh and geometry

- Selections
- Probes
- Functions
- Materials
- Pairs
- Solver sequences
- Parameter-dependent lists

Also, the Parametric and Time Dependent solvers overrule any definition of solver constants.

Constants settings for a solver node do not carry over to results processing.

Some examples of when it can be useful to define constants for a solver:

- When doing more advanced solver sequences, where constants need to be changed between calls (for example, in for-loops).
- When you do not want to change the global definition of a parameter or when you cannot or do not want to use a Parametric Solver feature.

When you want to define auxiliary parameters that are part of the equations like CFLCMP or niterCMP and where the solver does not define these parameters.

Adaption

Adaptive mesh refinement.

DESCRIPTION

The Adaption feature can be created from study steps of Eigenvalue or Stationary types See [Stationary](#) for the properties that control mesh adaptation and error estimates. See also [TimeAdaption](#) for the feature for time-dependent adaptation.

Advanced

Handle advanced general solver properties.

SYNTAX

```
model.sol(sname).feature(solv).create(fname,"Advanced")
model.sol(sname).feature(solv).feature(fname).set(pname,value)
```

DESCRIPTION

Feature for a number of advanced solver settings and assembly settings.

TABLE 6-25: VALID GENERAL PROPERTY/VALUE PAIRS FOR ADVANCED.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
assemdistrsol	auto true false	auto	Use distributed solution for assembly.
assemloc	true false	true	Optimize data locality on clusters.
asemt01	0 or a positive double	1e-12	Drop tolerance for local matrix contributions.
autorescale	on off	on	Automatic rescaling of linear equations (for the Stationary solver).

TABLE 6-25: VALID GENERAL PROPERTY/VALUE PAIRS FOR ADVANCED.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
blocksize	positive integer auto	auto	Assembly block size.
cachepattern	true false	false	Reuse sparsity pattern during assembly.
cachepatternmethod	auto expr	auto	User an expression or automatic setting with distributed solution for assembly.
cachepatternexpr	String		The expression for a distributed solution for assembly, if cachepatternmethod is set to auto.
checkmatherr	on off	off	Check for undefined numerical values after each operation.
complexfun	on off	off	Use complex-valued functions with real input.
convinfo	on detailed off	on	Print info to log.
D, E, K, L, M, N	on off		Manual control of reassembly.
elemdatainit	boolean	true	Optimize element data initialization for contact problems and reduced-order modeling.
keep	on off		Manual control of reassembly.
logsampling	double, zero or positive	0.005	Minimum time (in seconds) for log sampling of time-dependent solvers.
matherr	on off	on	Error for undefined operations.
matrixformat	auto sparse filled matrixfree	auto	Matrix format.
optthread	boolean	false	Optimize thread scheduling.
orthonormallimit	double	1e7	Limit, as an estimate of the complexity (number of operations), for using the orthonormal method in the automatic or hybrid null-space function.
nullfun	fnullorth flspnull auto explicitorth explicitsp flhybrid	auto	Null-space function.
rankreduction	fast robustge robustall	fast	Rank reduction rules for unidirectional constraints when nullfun is auto or flhybrid.
rankreductionorth	fast robustall	fast	Rank reduction rules for unidirectional constraints when nullfun is fnullorth or explicitorth.
.recoverysaveevery	nonnegative double	1800	The interval for which to save recovery files (in seconds), if recoverysaveforce is set to manual.
recoverysaveforce	frompreferences manual	frompreferences	Force recovery file saving at an interval: take from the preferences or specify manually.

TABLE 6-25: VALID GENERAL PROPERTY/VALUE PAIRS FOR ADVANCED.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
recoverysavinginfo	frompreferences oneveryiteration ratio	frompreferences	Recovery saving frequency: from preferences, on every iteration, or using a saving ratio.
recoverysavingratio	double (0–1)	0.1	The recovery file saving ratio, if recoverysavinginfo is set to ratio.
reusnullspacedata	on off	off	Reuse null-space data.
rowscale	on off	on	Equilibrate rows.
statblockdistr	boolean	false	Use static distribution of assembly blocks.
storeresidual	off solving solvingandoutput	off	Store the latest residual while solving or while solving and in the output.
symmetric	auto on off hermitian	auto	Symmetric matrices.

The [Advanced](#) section in the *COMSOL Multiphysics Reference Manual*, describes the functionality corresponding to the properties `blocksize`, `complexfun`, `nullfun`, `orthonormallimit`, and `rowscale`.

You can use the property `symmetric` to tell the solver that the model is symmetric or Hermitian, or you can use the automatic feature to find out (see [Advanced](#) in the *COMSOL Multiphysics Reference Manual*).

You can set `convinfo=detailed` to print more detailed information about the solver process in the log window. For example information about individual linear iterations or the scales per field computed by the automatic scaling algorithm. When `convinfo=off`, only minimal information about the solution process is printed.

By default, COMSOL Multiphysics gives an error message if the solver encounters an undefined mathematical operation when solving the model, for instance, $0/0$ or $\log(0)$. If you instead want the solver to proceed, put the property `matherr=off`. Then $0/0=\text{NaN}$ (not a number) and $\log(0)=-\text{Inf}$.

The properties `keep` and `D`, `E`, `K`, `L`, `M`, and `N` allow manual control of reassembly. If `keep=on`, each of the other properties controls reassembly of a specific matrix or vector. Setting the property value to `on`, means that the quantity is constant, and therefore can be assembled once and then kept. The letters have the following meaning: `E`=constant mass, `D`=constant damping, `K`=constant Jacobian, `L`=constant load, `M`=constant constraint, `N`=constant constraint Jacobian.

The `autorescale` property control if the automatically computed scales should be recomputed. This property only affects stationary nonlinear problems and fields that are using the automatic scaling method and for the constant damping technique. The initially computed scales are based on the initial assembled matrix. When `autorescale=on` the scales are recomputed in each nonlinear iteration based on the current solution.

You can use the property `matrixformat` to tell the solver which matrix format to store the system matrices in:

- `sparse` stores the matrix in a sparsely populated matrix format.
- `filled` stores the matrix in a densely populated format.
- `matrixfree` does not store matrices. The effects of matrix operations are assembled when needed.

When you specify `auto` (the default), the format is automatically determined based on the solver used.

By default, COMSOL does not check for undefined numerical values (for example, from numerical overflow) after each numerical operation. Set the property `checkmatherr` to `on` to make COMSOL check for such undefined numerical values, which will give more accurate error messages if such undefined numerical values occur.

Assemble and store the matrices generated during assembly.

SYNTAX

```
model.sol(sname).create(fname, "Assemble")
model.sol(sname).feature(fname).set(pname, value)
model.sol(sname).feature(fname).getSparseMatrixVal(mname)
model.sol(sname).feature(fname).getSparseMatrixValImag(mname)
model.sol(sname).feature(fname).getSparseMatrixRow(mname)
model.sol(sname).feature(fname).getSparseMatrixCol(mname)
model.sol(sname).feature(fname).getVector(vname)
model.sol(sname).feature(fname).getVectorImag(vname)
model.sol(sname).feature(fname).getSparseMatrixValBlock(mname, start, stop)
model.sol(sname).feature(fname).getSparseMatrixValImagBlock(mname, start, stop)
model.sol(sname).feature(fname).getSparseMatrixRowBlock(mname, start, stop)
model.sol(sname).feature(fname).getSparseMatrixColBlock(mname, start, stop)
model.sol(sname).feature(fname).getVectorBlock(vname, start, stop)
model.sol(sname).feature(fname).getVectorImagBlock(vname, start, stop)
model.sol(sname).feature(fname).isReal(mname)
model.sol(sname).feature(fname).getM(mname)
model.sol(sname).feature(fname).getN(mname)
```

DESCRIPTION

Assemble feature.

TABLE 6-26: VALID GENERAL PROPERTY/VALUE PAIRS.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
K	on off	off	Assemble the stiffness matrix
L	on off	off	Assemble the load vector
M	on off	off	Assemble the constraint vector
N	on off	off	Assemble the constraint Jacobian
D	on off	off	Assemble the damping matrix
E	on off	off	Assemble the mass matrix
EL	on off	off	Assemble the lumped mass matrix
NF	on off	off	Assemble the constraint force Jacobian
NP	on off	off	Assemble the optimization constraint Jacobian
MP	on off	off	Assemble the optimization constraint vector
MLB	on off	off	Assemble the lower bound constraint vector
MUB	on off	off	Assemble the upper bound constraint vector
Kc	on off	off	Assemble the eliminated stiffness matrix
Lc	on off	off	Assemble the eliminated load vector
Dc	on off	off	Assemble the eliminated damping matrix
Ec	on off	off	Assemble the eliminated mass matrix
ELc	on off	off	Assemble the eliminated lumped mass matrix
Null	on off	off	Assemble the constraint null-space basis
Nullf	on off	off	Assemble the constraint force null-space basis
ud	on off	off	Assemble the particular solution ud
uscale	on off	off	Assemble the scale vector
clist	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .

TABLE 6-26: VALID GENERAL PROPERTY/VALUE PAIRS.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
cname	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use <code>clist</code> . See The <code>clist</code> and <code>cname</code> Properties .
eiguse	on off	off	Assemble an eigenvalue problem using the <code>eigname</code> as the eigenvalue
eigname	String	lambda	The name of the eigenvalue
eigref	String	0	Value of eigenvalue linearization point
message	String		The log message from the last assembly process

The assemble feature assembles the matrices specified as output matrices and stores them in the feature. The output is stored in the feature. You can access the result using the matrix and vector access methods. The linearization point is determined by the current solution (that is, the solution computed by the previous feature in the sequence). The linearization point is stored in the sequence after the run. For information about the eliminated system, see [Advanced](#) in the *COMSOL Multiphysics Reference Manual*.

AutoRemesh

Iteratively and automatically create deformed geometries and remesh these geometries. In each step, map the solution and restart the simulation.

SYNTAX

```
model.sol(sname).feature(tname).create(fname, "AutoRemesh")
model.sol(sname).feature(tname).feature(fname).set(pname, pvalue)
```

DESCRIPTION

Operation feature. The following property/values are accepted:

TABLE 6-27: VALID PROPERTY/VALUE PAIRS FOR AUTOREMESH.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
autoremeshgeom	String		Name of geometry sequence.
consistentremesh	on off	off	Consistent initialization after remesh.
initialstepremesh	positive scalar	0.001	Initial time step size after remesh.
initialstepremeshactive	on off	off	Use initialstepremesh.
remeshtimes	double array		Times for remeshing, if <code>stepcondtype</code> is <code>spectime</code> .
remeshtimestep	double		Constant time step for remeshing, if <code>stepcondtype</code> is <code>specstep</code> and <code>remeshtimesteptype</code> is <code>const</code> .
remeshtimesteptype	const expr	const	Type of time step.
remeshtimestepexpression	String	0.1	Time-step expression for remeshing, if <code>stepcondtype</code> is <code>specstep</code> and <code>remeshtimesteptype</code> is <code>expr</code> .
solutionremesh	tout timestep	timestep	Solution to use for remeshing.
stopcondtype	quality distortion general spectime specstep	quality	Type of condition for remeshing: mesh quality, distortion, general, specified times, or specified time step.
stopdistexpr	String		Distortion expression, if <code>stepcondtype</code> is <code>distortion</code> .
stopdistval	String	2	Maximum distortion allowed, if <code>stepcondtype</code> is <code>distortion</code> .

TABLE 6-27: VALID PROPERTY/VALUE PAIRS FOR AUTOREMESH.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
stopexpr	String		Mesh quality expression, if stepcondtype is quality.
stopgenexpr	String		Logical condition for remeshing, if stepcondtype is general.
stopval	String	0.2	Minimal mesh quality, if stepcondtype is quality.
storesolmesh	on off	on	Store solutions when new meshes are created.

The automatic remeshing solver works in one geometry at a time. You specify the name of the geometry sequence in the property `autoremeshgeom`. Automatic remeshing is available for Time-Dependent studies and is intended for use with the moving meshes and deformed geometries.

Use the `stopcondtype` property to select the type of condition for when remeshing should occur:

- **quality.** This means that the solver remeshes when the value of `stopexpr` becomes smaller than the value of `stopval`.
- **distortion.** This means that the solver remeshes when the value of `stopdistexpr` becomes larger than the value of `stopdistval`.
- **general.** This means that the solver remeshes when the `stopgenexpr` property becomes true (nonzero).
- **spectime.** This means that the solver remeshes at the times given in `remeshtimes`.
- **specstep.** This means that the solver remeshes at the time step given in `remeshtimestep` if `remeshtimesteptype` is `const` for a constant time step, or in `remeshtimestepexpression` if `remeshtimesteptype` is `expr` for a, possibly nonconstant, time-step expression.

The `solutionremesh` property determines which previous solution is used for the remeshing:

- **tout** means that remeshing is done on the last solution that would have been stored by the solver if remeshing would not have occurred. This setting discards any solver progress done since the last output.
- **tstep** means that the remeshing is done using the solution from the last solver step before the condition for remeshing became fulfilled. Only the very last solver step, at which the condition was triggered, is discarded. Typically this setting is preferred because then the progress of the automatic remeshing does not depend on the solver's list of output times.

After each remeshing, the time integration is restarted and you can control the time stepping by the `Time` type analogous properties `consistentremesh` and `initialstepremesh`.

If the time integrator runs into problems, the computation is restarted at the beginning of the previous time interval using stricter time stepping controls.

AWE

Solve a parametric problem using asymptotic waveform evaluation (AWE).

SYNTAX

```
model.sol(sname).create(fname, "AWE")
model.sol(sname).feature(fname).set(pname, pvalue)
```

DESCRIPTION

Operation feature. The following properties are accepted:

TABLE 6-28: VALID AWE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
acceptshort	on off	on	If on, the solver accepts short intervals unconditionally.
atol	scalar	0.001	Absolute tolerance for parameter sweep.
aweassemble	all one	all	Either assemble all the needed matrices at once, or one at a time.
awefunc	String vector of strings		Expression(s) used in the search algorithm.
clist	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .
cname	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use clist. See The clist and cname Properties .
control	String	user	Name of controlling study step.
expeval	scalar numeric vector	range(0.1, 0.1, 0.9)	Where to check the error in each subinterval.
expsize	scalar	3	Number of terms in expansion.
exptype	pade taylor	pade	Use Padé or Taylor expansions to approximate the unknown.
keeplog	on off	off	Keep warnings in stored log.
linearity	false linper	false	Use linear linearity or a linear perturbation.
linplistsolnum	integer array	new int[] {1}	List of solutions to use for the linearization point, when linpsolnum is set to from_list.
linpmanualsolnum	solution number	new int[] {1}	Solution number (index) to use for the linearization point, when linpsolnum is set to manual.
linpmethod	sol init	init	Method used for linearization point, which for sol is given by the linpsol property.
linpsol	zero solution object	zero	Linearization point solution.
linpsolnum	all auto first last from_list interp manual solution number	auto	Indices to solutions to use as linearization points. The default value from a default solver is case dependent.
linpsoluse	current manual solution store tag	current	Linearization point solution to use.
linpsolusesolnum	integer		Index for the solution to use when linpsoluse is set to manual.
linpt	double	0	Interpolated point for the linearization point when linpsolnum is set to interp.
minint	scalar	0	The shortest allowed subinterval length, if minintmethod is set to manual.
minintmethod	auto manual	auto	The minimum interval length method: If auto, rtol times the parameter span is used. If manual, minint is used.
message	String		The log message from the last solution process.
output	solution rom	solution	Output the solution or a reduced-order model.

TABLE 6-28: VALID AWE PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outsollinearized	du u	du	Store the total solution (u) or deviation and linearization point (du), when storelinpoint is set to off.
plist	scalar numeric vector		Parameter list.
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
pname	vector of strings		Parameter names.
pout	plist psteps	plist	When pout is set to plist, output solutions for the parameters in plist. When pout is set to psteps, output solutions at the parameter values chosen by the solver, which will coincide with the adaptively found expansion points.
probesel	all none manual	all	The probes to compute.
probes	vector of strings		Probes to use when probesel>manual.
romdata	String	new	Tag of the target container for the reduced model (new for a new reduced model), when output is set to rom.
rtol	scalar	0.01	Relative tolerance for parameter sweep.
storelinpoint	on off	off	Whether to store the linearization point.

The AWE solver computes expansions of an underlying problem around certain parameter values. In the first step the largest and smallest values of `plist` are used as expansion points. Using these expansions, the values of one or more functionals at intermediate parameter values are computed. If the two expansions give similar enough functional values at these internal points, the interval is accepted and no subdivisions of that particular interval are deemed necessary.

The property `awefunc` is used to specify the functionals of interest, and the property `expeval` determines at which internal points these functionals are to be evaluated. The values for `expeval` are given relative the interval. That is, a value of 0.5 means that the functionals are evaluated at the midpoint of each interval. When the functional values from the two expansions are compared, a check is performed to see if they fulfill the specified tolerances `atol` and `rtol`. If neither of the tolerances are fulfilled, the interval is bisected, and the process is repeated for each subinterval. Before a bisection is performed a check is made to make sure that the new intervals are not shorter than the shortest allowed. By default the shortest allowed interval is given by the relative tolerance times the length of the interval defined by `plist` (when `mininactive` is set to off). If `mininactive` has been set to on the value of `minint` is the shortest interval allowed. If `minint` has been specified, the value of `mininactive` is on by default. The property `acceptshort` determines how to handle intervals that are shorted than allowed. If `acceptshort` is set to off and a short interval is detected, the solver is interrupted with an error/warning. If `acceptshort` is set to on and a short interval is detected, the solver accepts the interval even if the tolerances have not been fulfilled.

In AWE several matrices are needed to compute each expansion. There are two options when it comes to assembling these matrices: With `aweassemble` set to `all` everything is assembled in a single call to the `Xmesh`. With `aweassemble` set to `one`, the matrices are assembled one at a time. The first option is faster but requires more memory.

COMPATIBILITY

The `mininactive` property has been replaced with the `minintmethod` property since version 6.4. For models created before version 6.4 where `mininactive` was set to off, `minintmethod` is set to `auto`. For models created before version 6.4 where `mininactive` was set to on, `minintmethod` is set to `manual`.

The `romReconstruct` property has been removed since version 5.5.

The `soltypesonline` and `soltypesol` properties have been removed since version 5.6.

CombineSolution

Operate on solutions using concatenation, convert outer to inner solutions, summation, weighted summation, general summation, or remove solutions.

SYNTAX

```
model.sol(sname).create(fname,"CombineSolution")
model.sol(sname).feature(fname).set("soloper","remsol")
```

DESCRIPTION

This feature manipulates solutions via concatenation, summation or removal. The concatenation can, for example, take two time-dependent solutions solver over different time spans and combine them into a single time-dependent solution including all times from both solutions. You can also remove solutions. Stationary solutions are not possible to use. The following properties are accepted:

TABLE 6-29: VALID COMBINESOLUTION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>addinterpparam</code>	boolean	true	Add interpolated solution at intersection points for continuous parameter intervals.
<code>addinterp</code>	boolean	true	Add interpolated solution at intersection times.
<code>clearscrsol</code>	boolean	false	Clear the source solution when creating a weighted summation or removing solutions.
<code>concatmethod</code>	<code>firstprecedence</code> <code>includeall</code>	<code>firstprecedence</code>	For <code>soloper</code> set to concatenation, make first solution have precedence or include all solutions.
<code>conjugatepairconsecutivesort</code>	boolean	true	Use a conjugate-pair consecutive sort of eigenvalues regardless of the sorting method.
<code>cssol</code>	current any available solution	current	The solution to sum, when <code>soloper</code> is set to summation.
<code>cssol1</code>	current any available solution	current	The first solution to concatenate, when <code>soloper</code> is set to concatenation.
<code>cssol2</code>	current any available solution	current	The second solution to sum, when <code>soloper</code> is set to concatenation.
<code>cssolouter</code>	current any available solution	current	The source solution when <code>soloper</code> is set to <code>convertouter</code> .
<code>cssolwghtsum</code>	current any available solution	current	The solution to use for a weighted solution, when <code>soloper</code> is set to <code>wghtsum</code> .

TABLE 6-29: VALID COMBINESOLUTION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
excludeorinclude	explicit implicit	explicit	Exclude or include selected solutions when soloper is set to remsol.
excmethod	explicit implicit	explicit	Exclude method: explicit or implicit selection.
incmethod	explicit implicit	explicit	Include method: explicit or implicit selection.
interpandsort	auto manual	auto	Automatic or manual interpolation and sort settings if soloper is set to concatenation.
keeplog	true false	false	Keep warnings in stored log.
listsolnum	vector of integers	{1}	Indices to solutions to use as linearization points when solnum = from_list.
manualoutersolnumcssol1	vector of positive integers	{1}	Identifies the outer solutions to include when outersolnumcssol1 = manual.
manualoutersolnumcssol2	vector of positive integers	{1}	Identifies the outer solutions to include when outersolnumcssol2 = manual.
manualoutersolnumpssol	vector of positive integers	{1}	Identifies the outer solutions to include when outersolnumpssol = manual.
manualpname1	String array	new String[]{} 	Name of the outer parameters from the parametric sweep solution when outerparammethod1 is manual.
manualpname2	String array	new String[]{} 	Name of the outer parameters from the parametric sweep solution when outerparammethod2 is manual.
manualplist1	String array		Value of the outer parameters from the parametric sweep solution when outerparammethod1 is manual.
manualplist2	String array		Value of the outer parameters from the parametric sweep solution when outerparammethod2 is manual.

TABLE 6-29: VALID COMBINESOLUTION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
manualpunit1	String array	new String[]{}	Unit of the outer parameters from the parametric sweep solution when outerparammethod1 is manual.
manualpunit2	String array	new String[]{}	Unit of the outer parameters from the parametric sweep solution when outerparammethod2 is manual.
manualsolnum	vector of positive integers	{1}	Identifies the solutions used when solnum = manual.
outerlistsolnumcssol1	any available parametric store solution		Identifies the outer solutions to include when outersolnumcssol1 = from_list.
outerlistsolnumcssol2	any available parametric store solution		Identifies the outer solutions to include when outersolnumcssol2 = from_list.
outerlistsolnumpssol	any available parametric store solution		Identifies the outer solutions to include when outersolnumpssol = from_list.
outerparammethod1	auto manual	auto	How to define the outer parameter values for the simple solution when concatenating with a parametric sweep solution. Available when cssol1 is a simple solution.
outerparammethod2	auto manual	auto	How to define the outer parameter values for the single solution when concatenating with a parametric sweep solution. Available when cssol2 is a simple solution.
outersolnumcssol1	all from_list manual	all	Selection of outer solutions to include when cssol1 is a parametric sweep solution: All, from list, or manual.
outersolnumcssol2	all from_list manual	all	Selection of outer solutions to include when cssol2 is a parametric sweep solution: All, from list, or manual.
outersolnumpssol	all from_list manual	all	Selection of outer solutions to include when pssol is a parametric sweep solution: All, from list, or manual.

TABLE 6-29: VALID COMBINESOLUTION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pssol	current any available solution	current	The parametric sweep solution to concatenate, when soloper is set to convertouter.
removesol	current any available solution	current	The solution to remove from, when soloper is set to remsol.
remsolfromexprxc	String		Logical expression for solutions to exclude, when soloper is set to remsol.
remsolfromexprinc	String		Logical expression for solutions to include, when soloper is set to remsol.
solnum	all from_list manual	all	Selection of solutions to exclude or include, when soloper is set to remsol: All, from listsolnum, or from manualsolnum.
soloper	concatenation convertouter summation wgtsum gensum remsol	concatenation	Combine solutions using concatenation, convert outer to inner solutions, summation, weighted summation, general summation, or remove solutions.
solutiontype	parametric time eigenvalue	parametric	Specify a solution type if interpandsort is set to manual.
sortdiscontinparams	boolean	true	Sort discontinuous single-parameter values.
sorteigexpression	String		The expressions used to sort the eigenvalues or eigenfrequencies. Specified using setIndex() .
sorteigmethod	predefined user	predefined	Use a predefined or user defined sorting method, if sorteigvalues is set to true.
sorteigorder	ascend descend	ascend	Sort eigenvalues in ascending or descending order, if sorteigvalues is set to true.
sorteigprimarily	realpart imagpart realpartmagnitude imagpartmagnitude absval	See description	Primary sorting expression, if sorteigmethod is set to predefined. For eigenvalues, the default is realpart; for eigenfrequencies, the default is imagpart.

TABLE 6-29: VALID COMBINESOLUTION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sorteigsecondarily	realpart imagpart realpartmagnitude imagpartmagnitude absval	See description	Secondary sorting expression, if sorteigmethod is set to predefined. For eigenvalues, the default is imagpart; for eigenfrequencies, the default is imagpartmagnitude.
sorteigvalues	boolean	true	Sort eigenvalues.
sortontransformedeig	true false	true	Sort based on transformed eigenvalues, if sorteigmethod is set to predefined.
weightsmethod	oneexpr listexpr	oneexpr	Use one expression or a list of expressions for the weighted summation.
weightoneexpr	String	1	A single expression as weight for the summation (when weightsmethod is set to oneexpr).

When weightsmethod is set to listexpr, use setIndex with the properties wsolnum, weightlistexpr, and weightlistexpractive to specify the index (solution number), lists of weights, and active flags, respectively, for the case with a list of weights for a weighted summation. See [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

CopySolution

Handle a solution copy from another solver.

SYNTAX

```
model.sol(sname).create(fname, "CopySolution")
```

DESCRIPTION

This feature gives access to a copy of a solution from another solver. The following property is accepted:

TABLE 6-30: VALID COPYSOLUTION PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sol	String	none	Name of solution to copy, or "none" if empty.

Eigenvalue

Solve a PDE eigenvalue problem.

SYNTAX

```
model.sol(sname).create(fname, "Eigenvalue")
model.sol(sname).feature(fname).set(pname, value)

model.sol(sname).feature(fname).create(fname2, LinearType)

model.sol(sname).feature(fname).create(fname2, "Advanced")
```

Here LinearType is any of the allowed linear solver feature types.

DESCRIPTION

Operation feature.

For both linear and nonlinear problems, the eigenvalue problem is that of the linearization about a solution U_0 . If the eigenvalue appears nonlinearly, COMSOL Multiphysics reduces the problem to a quadratic approximation around a value λ_0 specified by the property `eigref`. The discretized form of the problem reads

$$KU - (\lambda - \lambda_0)DU + (\lambda - \lambda_0)^2EU = -N_F\Lambda$$

$$NU = M$$

where K , D , E , N , and N_F are evaluated for $U = U_0$ and $\lambda = \lambda_0$. Λ is the Lagrange multiplier vector, and λ is the eigenvalue. The eigenvalue name can be given by the property `eigname`. The linearization point U_0 can be given with the property `U`. The shift, described below, is compensated according to the linearization point for the eigenvalue. Therefore, changing the linearization point has no effect at all for linear or quadratic eigenvalue problems. The eigenvalue search method can be manual or a region in the complex plane (controlled by the property `eigmethod`) for the ARPACK eigenvalue solver or by stochastic estimation or a manually defined half or full contour in the complex plane for the FEAST eigenvalue solver. There is also a LAPACK eigenvalue for small filled matrices.

The `eigenvalue` feature accepts the following properties:

TABLE 6-31: VALID EIGENVALUE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
<code>appnreigs</code>	positive integer		Approximate number of eigenvalues (for <code>eigmethod</code> = region).
<code>approxnep</code>	<code>tol</code> <code>fixed</code>	<code>tol</code>	Taylor expansion truncation using a tolerance or a fixed truncation, when <code>eigsolver</code> is set to <code>arpacknonlinear</code> .
<code>chkeigregion</code>	<code>true</code> <code>false</code>	<code>false</code>	Perform eigenvalue region consistency check when <code>eigsolver</code> is <code>arpack</code> and <code>eigmethod</code> is <code>region</code> .
<code>clist</code>	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use <code>cname</code> . See The <code>clist</code> and <code>cname</code> Properties .
<code>cname</code>	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use <code>clist</code> . See The <code>clist</code> and <code>cname</code> Properties .
<code>computeandstoreleft eig</code>	<code>auto</code> <code>on</code> <code>off</code>	<code>auto</code>	Compute and store left eigenvectors.
<code>conangle</code>	integer	0	Rotation angle of the ellipse contour (–180 – 180 degrees), when <code>eigsolver</code> is <code>feast</code> and <code>searchcon</code> is <code>wholecon</code> .
<code>conjugatepairconsecutivesort</code>	<code>true</code> <code>false</code>	<code>true</code>	Use a conjugate-pair consecutive sort regardless of the sorting method.

TABLE 6-31: VALID EIGENVALUE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
conlbdy	real scalar	0	Lower bound of search interval, when eigsolver is feast and searchcon is halfcon.
conmid	real scalar	0	Center of the ellipse contour, when eigsolver is feast and searchcon is wholecon.
conrad	real scalar	1	Horizontal radius of the ellipse contour, when eigsolver is feast and searchcon is wholecon.
conradyhm	real scalar	0.15	Imaginary semiaxis, when eigsolver is feast and searchcon is halfcon.
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
conubdy	real scalar	1	Upper bound of search interval, when eigsolver is feast and searchcon is halfcon.
distrlnsol	true false	false	Distribute linear system solution.
eighermchk	true false	false	Real symmetric or Hermitian eigenvalue solver consistency check, if eigsolver is feast.
eigli	real scalar	0	Largest imaginary value of search region when eigsolver is arpack.
eiglrr	real scalar	0	Largest real value of search region when eigsolver is arpack.
eigmethod	manual region	manual	Eigenvalue search method when eigsolver is arpack: around a shift or using a rectangular region.
eigname	String	lambda	Name of eigenvalue variable.
eigref	String	0	Linearization point for the eigenvalue.
eigreschk	true false	false	Add an a posteriori residual check.
eigscalingexpression	String	1	The expression used to scale the eigenvectors, when eigvfunscale is set to manual.
eigsi	real scalar	0	Smallest imaginary value of search region when eigsolver is arpack.
eigsr	real scalar	0	Smallest real value of search region when eigsolver is arpack.
eigsolver	arpack feast lapack arpacknonlinear	arpack	Which eigenvalue solver to use: ARPACK, FEAST, LAPACK (filled matrix), or ARPACK nonlinear.
eigsymchk	true false	false	Real symmetric eigenvalue consistency check. Active if usesymeig is auto.
eigunit	valid unit	rad/s	The unit for the eigenvalue.

TABLE 6-31: VALID EIGENVALUE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
eigvfunscale	average maximum mass manual	average	The scaling of eigenvectors: RMS (average), maximum, so that the modal masses become unity, or by using an expression.
eigvfunscaleparam	positive real scalar	1	The maximum absolute value for the eigenvector scaling, when eigvfunscale is maximum.
eigwhich	lm lr sr li si	lm	Eigenvalue search direction (for eigmethod = manual).
expdegneq	Integer	3	Degree of the Taylor expansion, if approxneq is set to fixed.
filtereigdescription	array of strings		The description of the associated entry in filtereigexpression. This is just for usability.
filtereigexpression	array of strings		The expressions used to filter the eigenvalues or eigenfrequencies. Specified using <code>setIndex()</code> .
intgtypehm	auto gaussf trapez zolot	auto	Integration type for estimation: automatic, Gauss, or trapezoidal, when eigsolver is feast. Also, you can choose Zolotarev (zolot) when searchcon is halfcon.
keeplog	on off	off	Keep warnings in stored log.
krylovdim	positive integer	300	Dimension of Krylov space when eigsolver is arpack.
linpmethod	init sol	init	Method used for linearization point.
linpsol	zero solution object	zero	Linearization point solution.
linpsoluse	current manual solution store tag	current	Linearization point solution to use.
linpsolusesolnum	integer		Index for the solution to use when linpsoluse is set to manual.
maxdegneq	Integer	5	The maximum degree for the Taylor expansion truncation, if approxneq is set to tol.
maxeigit	positive integer	300	Maximum number of eigenvalue iterations when eigmethod is region.
maxmatsize	positive integer	2000	Maximum matrix size, when eigsolver is set to lapack.
maxnreigs	positive integer	200	Maximum number of eigenvalues when eigmethod is region.
maxnumberstoredeigsol	positive integer	1000	Maximum number of stored solutions, if storeeigsol is set to firstnth.
message	String		The log message from the last solution process.
modfollowing	on off	off	Whether to follow eigenmodes in parametric eigenvalue problems.

TABLE 6-31: VALID EIGENVALUE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
neigs	positive integer	6	Number of eigenvalues sought when eig solver is arpack.
neigsmanual	positive integer	6	Approximate number of eigenvalues, when neigsmethodnhm is set to manual, and size of initial search subspace for estimation, when neigsmethodnhm is stochest.
neigsmethodnhm	stochest manual	stochest	The number of eigenvalues, determined using a stochastic estimation algorithm or manually, when eig solver is feast.
nintgptsmethodhmest	auto manual	auto	Number of integration points for estimation: automatic or manual, when eig solver is feast.
nintgptsmethodhm	auto manual	auto	Number of integration points for eigenvalue solver: automatic or manual, when eig solver is feast.
nintgptshm	positive integer	16	Number of integration points, when nintgptsmethodhm is manual.
nintgptshmest	positive integer	6	Number of integration points for estimation, when nintgptsmethodhmest is manual.
qnmaxitnep	positive integer	20	Maximum number of iterations for quasi-Newton refinement, when qnrefinementnep is set to true.
qnrefinementnep	boolean	true	Use quasi-Newton refinement, when eig solver is set to arpacknonlinear.
rtol	scalar	1e-6	Relative tolerance.
searchcon	halfcon wholecon	wholecon	Specify the eigenvalue search contour when eig solver is feast: a whole contour or a half contour (Hermitian problem).
shift	scalar	0	Eigenvalue search location when eig solver is arpack.
shiftactive	on off	off	Specify eigenvalue search location when eig solver is arpack.
shiftregman	scalar	0	Eigenvalue shift (when shiftregselect = manual).
shiftregselect	auto manual	auto	Search for eigenvalues around (auto = center of search region) when eig solver is arpack.
solnum	auto all positive integer	auto	The solution numbers to use.
solutiontype	parametric type eigenvalue	parametric	The solution type for solutions to concatenate, when interpdnsort is set to manual.
sorteigexpression	String		The expressions used to sort the eigenvalues or eigenfrequencies. Specified using setIndex() .

TABLE 6-31: VALID EIGENVALUE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
sorteigmethod	predefined user	predefined	Use a predefined or user defined sorting method.
sorteigorder	ascend descend	ascend	Sort eigenvalues in ascending or descending order.
sorteigprimarily	realpart imagpart realpartmagnitude imagpartmagnitude absval	See description	Primary sorting expression, if sorteigmethod is set to predefined. For eigenvalues, the default is realpart; for eigenfrequencies, the default is imagpart.
sorteigsecondarily	realpart imagpart realpartmagnitude imagpartmagnitude absval	See description	Secondary sorting expression, if sorteigmethod is set to predefined. For eigenvalues, the default is imagpart; for eigenfrequencies, the default is imagpartmagnitude.
sortontransformedeig	true false	true	Sort based on transformed eigenvalues, if sorteigmethod is set to predefined.
stopwhennoeigfound	on off	on	Allows the solver to return an empty (NaN) solution if set to off.
storeeigsol	all firstnth	all	Store all converged solution or only the first Nth solutions.
storefact	true false	false	Store linear system factorization.
storelinpoint	on off	off	Whether to store the linearization point.
termfeast	relertrace maxrelres	relertrace	Termination criterion: relative error on the trace or maximum relative residual, when eigsolver is feast.
tolfactqnep	double	1	Tolerance factor for quasi-Newton refinement, when qnrefinementnep is set to true.
toltruncationnep	double	1E-12	Tolerance for the Taylor expansion truncation, if approxnep is set to tol.
transcontour	true false	true	Enter transformed values, when eigsolver is feast and eigmethod is manual.
transeigref	true false	true	Transform eigenvalue linearization point and eigmethod is region.
transform	String	none	Eigenvalue transformation.
transshift	true false	true	Enter transformed values, when eigsolver is arpack.
usesymeig	auto false	auto	Use real symmetric eigenvalue solver when eigsolver is arpack.

Specify where to look for the desired eigenvalues with the property shift. Enter a real or complex scalar; the default value is 0, meaning that the solver tries to find eigenvalues close to 0.

For more information about the eigenvalue solver, see [Eigenvalue Solver](#) in the *COMSOL Multiphysics Reference Manual*.

EigenvalueAttrib

An attribute to Sensitivity and Optimization solvers. It has the same properties as the Eigenvalue solver. See [Eigenvalue](#) for more information.

EigenvalueParam

Handle properties for parameter stepping for a parametric eigenvalue problem.

SYNTAX

```
model.sol(sname).create(fname, "Eigenvalue")
model.sol(sname).feature(fname).create(parname, "EigenvalueParam")
model.sol(sname).feature(fname).feature(parname).set(pname, pvalue)
```

DESCRIPTION

Attribute feature.

TABLE 6-32: EIGENVALUE PARAMETRIC PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
addsolutions	boolean	true	Add solution data points from the general noncontinuation parameter values lists, if <code>uselsqdata</code> is set to true.
auxsweepeigval	default preveigval	default	ARPACK starting vector: the default or from previous eigenvalues.
auxsweepeigvalshift	average closest	closest	ARPACK shift from previous eigenvalues: average of converged eigenvalues or eigenvalue closest to the previous shift, if <code>auxsweepeigvec</code> is set to <code>preveigvec</code> .
auxsweepeigvec	default preveigvec	default	ARPACK shift: as specified (default) or based on eigenvalues from last parameter (<code>preveigvec</code>).
auxsweepeigvecstart	sum closest	sum	ARPACK starting vector from previous eigenvalues: summation of all eigenvalues or eigenvector with eigenvalue closest to the shift, if <code>auxsweepeigval</code> is set to <code>preveigval</code> .
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
excludelsqvalues	boolean	true	Create bounding box from general parameter value lists, if <code>uselsqdata</code> is set to true.
pdistrib	on off	off	If the solver should distribute the parameter sweep.
peigonerror	stop skip empty	stop	Stop, skip the error and then continue, or add an empty solution and then continue when an error occurs.
plist	real array		List of parameter values. Obsolete, use <code>plistarr</code> instead.
plistarr	real matrix		Lists of parameter values. One row of values for each parameter name.
pname	vector of strings		Parameter names.
pwork	integer	1	Maximum number of distributed groups.

TABLE 6-32: EIGENVALUE PARAMETRIC PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sweeptype	sparse filled	sparse	Method for doing the parameter variation. For sweeptype=sparse, the parameter tuples defined by the columns in plistarr are solved for. This method requires equal length for the rows. For sweeptype=filled, all parameter combinations given by plistarr are solved for.
uselsqdata	boolean	true	Use parameters from least-squares objectives (requires the Optimization Module).

FFT

Compute a fast Fourier transform (FFT) or inverse fast/nonuniform Fourier transform (IFFT/INFT) using an FFT study step and solver,

SYNTAX

```
model.sol(sname).create(fname,"FFT")
model.sol(sname).feature(fname).set(pname,pvalue)
```

DESCRIPTION

Operation feature. The following properties are accepted:

TABLE 6-33: VALID FFT SOLVER PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
addstatsol	on off	off	Add stationary solution.
control	String	user	Set control for FFT solver by study.
fftendf	double	1.0	Endpoint of frequency interval, only applicable when fftsteptypef = interval.
fftendtime	double	1.0	End time for the time interval in a forward FFT.
fftextend	on off	on	For an inverse NFT/FFT, extend the input data samples by adding complex conjugate pairs.
fftfile	on off	off	Store intermediate FFT data on disk.
fftinputdata	fftperiodic fftnotperiodic ffthalfperiodic	fftperiodic	Interpret the input data as periodic, not periodic, or half-periodic (antisymmetric). The first two options replaces the Boolean fftperiodic property in earlier versions.
fftinputmethod	sol init	sol	Prescribe the input values using a solution or an initial expression.
fftmaxfreq	double	10	Maximum output frequency in forward FFT.
fftoutorder	nat sym	nat	Output order (natural or symmetric) of frequencies for forward FFT.
fftouttrange	double array		Output times for inverse NFT/FFT.
fftphaseinexpr	String		Expression for input phase function, can be expressed in terms of t, freq, and niterFFTin (if applicable).
fftphaseoutexpr	String		Expression for output phase function, can be expressed in terms of t, freq, and niterFFTout (if applicable).
fftphasetypein	none fromexpr	none	Method for input phase function.
fftphasetypeout	none fromexpr	none	Method for output phase function.

TABLE 6-33: VALID FFT SOLVER PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
fftrealtstore	on off	on	Do not store negative frequencies for real input.
fftscaling	cont discrete	cont	Use a discrete or continuous scaling for the Fourier transform.
fftstarttf	double	0.0	Starting point of frequency interval, only applicable when fftsteptypef = interval.
fftstarttime	double	0.0	Start time for the time interval in a forward FFT.
fftsteptypef	allfreqs interval	allfreqs	Method for selecting frequencies from range.
fftstoretimes	out store	out	Store time steps taken by the solver (out) or output times stored in fftstoretrange (store).
fftstoretrange	String (vector of output times)		Output times for the inverse FFT if fftstoretimes is set to store.
ffttranstype	transfft transifft	transfft (depends on type of FFT study step)	Type of FFT transformation: forward or inverse.
fftwinalpha	double	0.5	Window parameter for a Tukey window.
fftwincenterfw	double	0.5	Window center for a Gaussian window function in a forward FFT.
fftwincenterinv	double	50	Window center for a Gaussian window function in an inverse NFT/FFT.
fftwindev	double	1	Standard deviation for a Gaussian window function.
fftwincutoff	double	1.0	Cutoff fraction for window function in [0, 1].
fftwindowfw	on off	off	Use window function in forward FFT.
fftwindowinv	on off	off	Use window function in inverse NFT/FFT.
fftwinexpr	String		Expression for window function, can be expressed in terms of t, freq, niterFFTin, and niterFFTout (if applicable).
fftwinmaxfw	double	1	Maximum (end) value for window in forward FFT.
fftwinmaxinv	double	100	Maximum (end) value for window in inverse NFT/FFT.
fftwinminfw	double	0	Minimum (start) value for window in forward FFT.
fftwinmininv	double	0	Minimum (start) value for window in inverse NFT/FFT.
fftwintypefw	fromexpr cutoff rectangle gauss hamming hanning blackman tukey	fromexpr	Method for window function in a forward FFT.
fftwintypeinv	fromexpr cutoff rectangle gauss hamming hanning blackman tukey	fromexpr	Method for window function in an inverse NFT/FFT.
keeplog	on off	off	Keep warnings in stored log.

TABLE 6-33: VALID FFT SOLVER PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
punit	String	Hz	Frequency unit.
statmethod	init sol	sol	For an added stationary solution, use a solution or initial expression.
statsol	String		Solution to use as an added stationary solution.
statsolnum	String		The solution number to use for the solution used as an added stationary solution.
statsoluse	a stored solution or current	current	Use the current solution or a stored solution as the added stationary solution.
statstudy	String		Study from which the added stationary solution is chosen.
statt	double		Time value at which the added stationary solution is chosen.
statmanualsolnum	integer		Solution number of the added stationary solution.
tunit	String	s	Time unit.
winpunit	unit	Hz	Frequency unit for window in an inverse NFT/FFT.

EXAMPLE*Code for Use with Java*

```

model.study("std2").create("tdfft", "TimeToFreqFFT");
model.study("std2").feature("tdfft").set("fftinputstudy", "std1");
model.study("std2").feature("tdfft").set("fftstarttime", "0.5");
model.study("std2").feature("tdfft").set("fftendtime", "2.0");
model.study().create("std3");
model.study("std3").create("fdfft2", "FreqToTimeFFT");
model.study("std3").feature("fdfft2").set("fftinputstudy", "std2");

...

model.sol("sol7").create("fft1", "FFT");
model.sol("sol7").feature("fft1").set("ffttranstype", "transfft");
model.sol("sol7").feature("fft1").set("fftstarttime", "0.5");
model.sol("sol7").feature("fft1").set("fftendtime", "2.0");
model.sol("sol7").feature("fft1").set("control", "tdfft");

...

model.sol("sol9").feature("fft1").set("ffttranstype", "transfft");
model.sol("sol9").feature("fft1").set("control", "fdfft2");

model.study("std3").feature("fdfft2").set("fftinputmethod", "sol");
model.study("std3").feature("fdfft2").set("fftinputstudy", "std2");
model.study("std3").feature("fdfft2").set("fftinputsol", "current");
model.study("std3").feature("fdfft2").set("fftinputsoluse", "current")

...

model.sol("sol12").feature("fft1").set("fftstarttime", "0.5");
model.sol("sol12").feature("fft1").set("fftendtime", "2.0");
model.sol("sol12").feature("fft1").set("fftperiodic", "off");
model.sol("sol12").feature("fft1").set("fftoutorder", "nat");

...

```

```

model.sol("sol2").feature("ft1").set("fftphase", "on");
model.sol("sol2").feature("ft1").set("fftwintypefw", "fromexpr");
model.sol("sol2").feature("ft1").set("fftwinexpr", "(niterFFTin<5)*niterFFTin");
model.sol("sol2").feature("ft1").set("fftphasetypein", "fromexpr");
model.sol("sol2").feature("ft1").set("fftphaseinexpr", "5+i*2*niterFFTin");
model.sol("sol2").feature("ft1").set("fftphasetypeout", "fromexpr");
model.sol("sol2").feature("ft1").set("fftphaseoutexpr", "2+i*4*niterFFTout");

```

COMPATIBILITY

The property `fftperiodic` has been replaced by the property `fftinputdata` since version 6.2.

The properties `fftbwalgtype` and `fftmeasure` from earlier versions are no longer available since version 5.2.

In version 5.2, the property `fftwintype` for the window type in earlier versions was replaced by `fftwintypefw` and `fftwintypeinv` for the window type for a forward FFT and for an inverse NFT/FFT, respectively.

In version 5.2, the property `fftwindow` for switching a window on and off in earlier versions was replaced by `fftwindowfw` and `fftwindowinv` for the window type for a forward FFT and for an inverse NFT/FFT, respectively.

In version 5.2, the property `tlist` for the input time range to a forward FFT in earlier versions was replaced by the properties `fftstarttime` and `fftendtime` for the start time and end time, respectively.

For, EndFor

Add for loops to a solver sequence.

SYNTAX

```

model.sol("sol1").create("for1", "For");
model.sol("sol1").create("endfor1", "EndFor");

```

DESCRIPTION

`model.sol("sol1").create("for1", "For")` adds the start of a for loop.

`model.sol("sol1").create("endfor1", "EndFor")` adds the end of a for loop.

Use `For` and `EndFor` to enclose a sequence of solver commands that you want to iterate in a for loop. You can add more than one for loop, but they must be balanced so that each `For` ends with a corresponding `EndFor`.

You control the number of iterations in the loop using the following properties:

TABLE 6-34: PROPERTIES FOR THE FOR LOOP.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
expr	String	1	Global expression used to compute relative error when <code>method=convergence</code> .
iter	Positive integer	5	Number of iterations of the solver loop when <code>method=iterations</code> .
maxiter	Positive integer	25	Maximum number of iterations in the solver loop when <code>method=convergence</code> .
method	convergence iterations	iterations	Choose whether termination of the solver loop is based on the convergence of a global variable or a fixed number of iterations.
miniter	Positive integer	1	Minimum number of iterations in the solver loop when <code>method=convergence</code> .

TABLE 6-34: PROPERTIES FOR THE FOR LOOP.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
rtolterm	Positive real number	0.001	Relative tolerance for termination of the solver loop when method=convergence.
rtolthresh	Positive real number	1	Threshold used to avoid division by zero while computing the relative error when method=convergence.

FullyCoupled

Handle the fully coupled nonlinear solution approach.

SYNTAX

```
model.sol(sname).feature(solv).create(fname, "FullyCoupled")
model.sol(sname).feature(solv).feature(fname).set(pname, value)
```

DESCRIPTION

This feature can be used as an attribute to the Time and Stationary features. The nonlinear solver is an affine invariant form of the damped Newton method.

The following properties are available:

TABLE 6-35: VALID FULLY COUPLED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
aaccdelay	positive integer	0	Number of iterations between pseudo time stepping becomes inactive and Anderson acceleration becomes active when stabacc = aacc.
aaccdim	positive integer	10	Dimension of Anderson iteration space when stabacc = aacc.
aaccmix	scalar 0–1	1	Mixing parameter when segstabacc = segaacc.
adaptcfltol	boolean	true	Use an adaptive target error estimate for pseudo time stepping.
adapttol	boolean	false	Use adaptive tolerance in the linear solver if set to true. Available if dtech is set to bcktrack or const.
adapttolerrweight	boolean	true	Apply nonlinear error weighting to the adaptive tolerance for models that do not use pseudo time stepping and if adapttol is set to true.
adapttolcflweight	boolean	true	Apply CFL weighting to the adaptive tolerance for models using pseudo time stepping and if adapttol is set to true.

TABLE 6-35: VALID FULLY COUPLED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
atolchoice	fullerrorest partialerrorest nonlinresidual onlycfl onlyerr	fullerrorest	The error estimation method to use if adapttol is set to true. fullerrorest uses the full information about the linear and nonlinear error estimates to determine the tolerance. partialerrorest uses partial information about the linear and nonlinear error estimates to determine the tolerance. nonlinresidual uses the nonlinear residual only to determine the tolerance. onlycfl applies CFL suppression to the initial tolerance for models using pseudo time stepping. onlyerr applies a suppression mechanism based on the nonlinear error estimate for models with a stationary solver and not using pseudo time stepping.
atolthreshold	positive value, 0–1	0.1	The threshold for using the safeguards when determining the tolerance. The higher the value, the faster the safeguards get disabled. Only available when adapttol is set to true.
backdampstep	scalar (0–1)	0.5	Damping in one step, when dttech is set to bcktrack and backmethod is set to conststep.
backmethod	fullest conststep	fullest	Backtracking method when dttech is set to bcktrack: full estimation or constant step.
backtrackonce	boolean	false	When set to true, it forces the solver to apply at least one backtracking step even if the algorithm does not strictly require it. Available only when dttech is set to bcktrack.
cflaa	boolean	false	Use Anderson acceleration when the CFL number is under a CFL threshold set by cflaacfl. Only available when pseudo time stepping is used (stabacc=cflcmp).
cflaacfl	positive scalar	100	CFL threshold when stabacc = cflcmp.
cflaadelay	nonnegative integer	0	Number of iterations between pseudo time stepping becomes inactive and Anderson acceleration becomes active when segstabacc = cflcmp.
cflaadim	positive integer	10	Dimension of the iteration space for Anderson Acceleration when used with pseudo time stepping (stabacc = cflcmp and cflaa = on).

TABLE 6-35: VALID FULLY COUPLED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
cflaafact	positive scalar	1	Threshold for Anderson step for pseudo time stepping (stabacc = cflcmp and cflaa = on).
cflaamix	real scalar 0–1	1	Mixing parameter for Anderson acceleration when segstabacc = cflcmp.
cflaaset	pid lock	pid	Decides if the PID controller should be disabled and the CFL number set to the target CFL number when Anderson acceleration is enabled (lock) or if the PID controller should be used also when Anderson acceleration is enabled (pid). Available when pseudo time stepping is used (stabacc = cflcmp and cflaa = on).
cfljtech	boolean	false	Override Jacobian update for step when stabacc = cflcmp.
cfljtechval	onfirst minimal	onfirst	Jacobian update on first iteration or minimal when stabacc = cflcmp and cfljtech = true.
cfltech	simple interp	simple	When simple is used, the PID controller is using the classical PID control theory. The interp option combines the classical PID controller with a linear interpolation method. Available when stabacc is set to cflcmp.
cfltol	positive scalar	0.1	Target error estimate for pseudo time stepping.
damp	positive real	1	Damping factor for the damped Newton method.
dampexponent	real scalar (1–2)	2	The exponential factor used to determine the adaptive tolerance. if atolchoice is set to nonlinresidual.
dampfactor	real scalar (0–1)	1	The linear damping factor used to determine the adaptive tolerance, if atolchoice is set to nonlinresidual.
ddoginitdamp	nonnegative scalar	1	Initial damping factor for dtech set to ddog.
ddogrestart	positive integer	7	Number of iterations before restart for double dogleg solver.
dtech	const auto bcktrack hnlin ddog	auto const (Time)	Damping technique. For more information, see below this table.
etamax	scalar (0–1)	0.9	Maximal tolerance for the adaptive tolerance algorithm, if adapttol is set to true.
forcecfl	boolean	true	Limit to target CFL number for pseudo time stepping.

TABLE 6-35: VALID FULLY COUPLED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initcfl	positive scalar	5.0	Initial CFL number for pseudo time stepping.
initiallntol	scalar (0–1)	0.5	Initial tolerance for the adaptive tolerance algorithm, if adapttol is set to true.
initstep	nonnegative scalar	1	Initial damping factor for dtech set to auto.
initsteph	nonnegative scalar	1e-4	Initial damping factor for dtech set of hnlin.
jtech	minimal once onevery	onevery minimal (Time)	Jacobian update technique for dtech set to const or bcktrack.
jtechcfl	positive scalar	100	CFL threshold for Jacobian update when segstabacc = cflcmp and cfljtech = true.
kdpid	positive scalar	0.05	PID controller, derivative for pseudo time stepping.
kipid	positive scalar	0.05	PID controller, integral for pseudo time stepping.
kppid	positive scalar	0.65	PID controller, proportional for pseudo time stepping.
maxdamp	scalar (0, 1]	1.0	The maximum damping factor. If backtracking returns a damping larger than this value, maxdamp is used. Available only if dtech is set to bcktrack.
maxiter	positive integer	25 4 (Time)	Maximum number of Newton iterations.
mincfl	positive scalar	10000	Target CFD number for pseudo time stepping.
mindamp	positive scalar	0.1	The minimum damping factor. If backtracking returns a damping smaller than this value, mindamp is used. Values above 1 just mean that no damping is allowed. Available only if dtech is set to bcktrack.
minstep	scalar (0–1)	1.0e-4	Minimum damping factor for dtech set to auto.
minsteph	scalar (0–1)	1.0e-8	Minimum damping factor for dtech set to hnlin.
minsteprecovery	positive scalar	0.75	Recovery damping factor. Available when useminsteprecovery is set to auto or on.
niter	positive integer	1	Fixed number of iterations.
ntermauto	tol itertol	tol	Termination techniques for dtech set to auto/hnlin.
ntermconst	iter tol itertol	tol	Termination techniques for dtech set to const or bcktrack.
ntolfact	positive scalar	1	Tolerance factor.

TABLE 6-35: VALID FULLY COUPLED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ratelimit	positive scalar	0.9 (dtech set to const)	Limit on nonlinear convergence rate.
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probesel	all none manual	all	The probes to compute.
probes	vector of strings		Probes to use when probesel>manual.
ratelimitactive	on off	off on (Time)	Use limit on nonlinear convergence rate.
relaxationressmooth	positive scalar	0.1	Relaxation factor for residual smoothing when ressmooth is set to true.
reserrfact	positive scalar	1000	Residual factor for termonres if set to auto or both.
resscale	scalefieldwise scaleuniform	scalefieldwise	Residual scaling technique for dtech set to ddog.
ressmooth	boolean	false	Use residual smoothing for pseudo time stepping (when stabacc is set to cflcmp).
rstep	positive scalar	10	Restrictions for step-size update (limits how much the damping factor is allowed to change in a Newton iteration).
rstepabs	positive scalar 0–1	1	Restrictions for step-size increase (maximum for the allowed absolute increase in the damping factor for a Newton iteration).
stabacc	none cflcmp aacc	none	Stabilization and acceleration: None, pseudo time stepping (for stationary solvers), or Anderson acceleration.
termonres	off on auto both	auto/off	Termination criterion: Solution, residual, solution or residual, or solution and residual. The default is auto for stationary problems and off for time-dependent problems.
updweightsauto	true false wthresh	true	Updates the weights for automatic scales if they change two orders of magnitude and then restart the Newton solver from the current solution if true. If set to wthresh, you can use thresholds for the weights. All updweights properties are available for stationary and parametric studies and for Newton solvers only (that is, dtech is not ddog, bcktrack, or const).

TABLE 6-35: VALID FULLY COUPLED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
updweightsdamp	current manual	current	Available if updweightsauto is wthresh. Use fraction of the current damping factor at update (current) or a constant update damping factor (manual).
updweightsdampval	positive scalar	1	Fraction of current damping factor. If updweightsauto is wthresh and updweightsdamp is current.
updweightsdampvalconst	positive scalar	0.1	Update damping factor. If updweightsauto is wthresh and updweightsdamp is manual.
updweightsfac	scalar	100	Weights threshold factor for update. If updweightsauto is wthresh.
useheuristicfact	boolean	false (Time)	Use pre-6.2 linear heuristic for adaptive tolerance.
useminsteprecovery	auto on off	auto	Use recovery damping factor. The recovery damping is used when the damping is reaching its minimum value, and would otherwise return an error (about minimum damping factor reached). Available when dtech is set to auto or hnlin.

The property `dtech` controls which damping factor to use in the damped Newton iterations.

For `dtech` set to `auto`, the solver determines an appropriate damping factor automatically. For this method the initial and minimally allowed damping factors are controlled by the properties `initstep` and `minstep`, respectively. The termination technique is controlled by the property `ntermauto`.

For `dtech` set to `hnlin`, the solver determines an appropriate damping factor automatically but treat the problem as being highly nonlinear. This option can be tried if there is no convergence with `dtech` set to `auto`. For this method the initial and minimally allowed damping factors are controlled by the properties `initsteph` and `minsteph` respectively. Moreover, certain internal control structures are adapted. Especially, the error control is biased from a more absolute norm toward a relative norm. So this parameter is also useful if a solution with components of highly varying orders of magnitudes are present. In the context of parameter stepping, you can also try this option if the step sizes in the parameter seem to be too small.

When `dtech` is set to `const`, the constant damping factor specified in the property `damp` is used. For this method the termination technique is controlled by the property `ntermconst`. Furthermore, the property `jtech` can be used to control how often the Jacobian is updated. With `jtech=minimal`, the Jacobian is updated as seldom as possible (only once for a stationary problem and at most once per time step for a time-dependent problem). For time-dependent problems, the choice `jtech=once` makes the solver update the Jacobian once per time step. With `jtech=onevery`, the Jacobian is updated on every Newton iteration. The default is `onevery` for stationary problems and `minimal` for time-dependent problems.

When `dtech` is set to `ddog` (stationary problems), the double dogleg solver is used. The initial damping factor is controlled by the property `ddoginitdamp` and the property `resscale` controls the residual scaling. The option `resscale=scalefieldwise` scales the equations based on the field-wise sizes of the initial residual. When the option `resscale=scaleuniform` is selected the algorithm terminates on the relative residual based on the initial residual.

When `dtech` is set to `bcktrack`, backtracking is used as an automated way of finding the “optimal” damping factor used in the nonlinear solvers.

The tolerance `ntol` gives the criterion for convergence for a stationary problem; see [Stationary Solver](#) in the *COMSOL Multiphysics Reference Manual*.

The property `ntolfact` controls how accurately the nonlinear system of equations is solved. The value given in `ntolfact` is multiplied with the main solver tolerance and used in the convergence criteria. Also, the solution process is interrupted (and the Jacobian updated or the time step reduced) if the convergence is too slow. This can be disabled by setting `useratelim`=off. When `useratelim`=on, the property `ratelimit` controls what is considered as too slow in this context. The solution process is interrupted if the estimated linear convergence rate (of all steps, when the segregated solver is used) becomes larger than the value given in `ratelimit`.

The property `stabacc` enables or disables pseudo time stepping (for stationary problems) or Anderson acceleration. When enabled the pseudo time stepping is controlled by the scalar-valued controller parameters `cf1tol`, `initcf1`, `kdpid`, `kipid`, and `kppid`. For the Anderson acceleration, the parameter `aacddim` specifies the dimension of the Anderson iteration space.

The property `termmonres` controls the termination criterion for stationary problems when `dtech`=const or `bcktrack` (and `ntermconst` is not iter), `auto`, or `hnl`. When `termmonres`=off the estimated error is solution based, with `termmonres`=on it is based on a relative residual and for `termmonres`=auto the estimated error is the minimum of the solution and residual based errors. For `termmonres`=auto the property `reserrfact` is a scalar factor multiplying the relative residual error. For `termmonres`=both, both methods are used.

COMPATIBILITY

The property `usecf1cmp` from earlier versions of COMSOL Multiphysics is not used in version 5.0. Use the property `stabacc` instead.

HardwareAcceleration

Hardware acceleration using a supported GPU for transient acoustics simulations based on the discontinuous Galerkin method.

SYNTAX

```
model.sol(sname).feature(solv).create(fname, "HardwareAcceleration")
model.sol(sname).feature(solv).feature(fname).set(pname, value)
```

DESCRIPTION

This feature can be used as an attribute to the `Time` feature to accelerate the solution time if you are simulating transient acoustics using a discontinuous Galerkin method. It requires access to a supported GPU.

The following properties are available:

TABLE 6-36: VALID HARDWARE ACCELERATION PROPERTIES

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>dgacc</code>	boolean	false	Use an accelerated solver for discontinuous Galerkin.
<code>gpuresidual</code>	boolean	true	Compute residual on GPU, if <code>dgacc</code> is set to true.
<code>gpusingleprecision</code>	boolean	false	Use single precision on GPU, if <code>gpuresidual</code> is set to true. If false, double precision is used.

InputMatrix

Input matrices and vectors to the linear solvers.

SYNTAX

```
solver=model.sol(sequenceTag).feature(solverTag)
solver.create(fname,"InputMatrix")
solver.feature(fname).set(pname,value)
solver.feature(fname).addSparseMatrixVal(mname,row,col,val)
solver.feature(fname).addSparseMatrixValImag(mname,row,col,val)
solver.feature(fname).createSparseMatrix(mname,M,N,Nnz,isReal)
solver.feature(fname).createVector(mname,M,isReal)
solver.feature(fname).getSparseMatrixVal(mname)
solver.feature(fname).getSparseMatrixValImag(mname)
solver.feature(fname).getSparseMatrixRow(mname)
solver.feature(fname).getSparseMatrixCol(mname)
solver.feature(fname).getVector(vname)
solver.feature(fname).getVectorImag(vname)
solver.feature(fname).getSparseMatrixValBlock(mname,start,stop)
solver.feature(fname).getSparseMatrixValImagBlock(mname,start,stop)
solver.feature(fname).getSparseMatrixRowBlock(mname,start,stop)
solver.feature(fname).getSparseMatrixColBlock(mname,start,stop)
solver.feature(fname).getVectorBlock(vname,start,stop)
solver.feature(fname).getVectorImagBlock(vname,start,stop)
solver.feature(fname).setVector(vname,val)
solver.feature(fname).setVectorImag(vname,val)
solver.feature(fname).setVectorBlock(vname,val,start)
solver.feature(fname).setVectorImagBlock(vname,val,start)
solver.feature(fname).isReal(mname)
solver.feature(fname).getM(mname)
solver.feature(fname).getN(mname)
```

DESCRIPTION

The InputMatrix feature can be used to create the raw data of an assembled matrix or vector from Java[®]. The InputMatrix feature can exist as a subfeature of the Eigenvalue, Stationary, and Time solver features. These solver features automatically pick up matrices from the InputMatrix subfeature instead of automatically assembling the matrices. The matrices are not stored in the model when the model is saved. They must be created before computing the solver features.

TABLE 6-37: VALID INPUTMATRIX PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
K	on off	off	Input the stiffness matrix
L	on off	off	Input the load vector
M	on off	off	Input the constraint vector
N	on off	off	Input the constraint Jacobian
D	on off	off	Input the damping matrix
E	on off	off	Input the mass matrix
NF	on off	off	Input the constraint force Jacobian

EXAMPLE

For an example of how to access system matrices, see [Accessing System Matrices and Vectors](#) in the *Application Programming Guide*.

SEE ALSO

[Matrix Data](#), [Matrix Creation](#)

Linear

Handle linear system solvers with three different attribute features: Direct, Iterative, and Multigrid.

SYNTAX

```
sol.feature(fname).create(lname,"Direct")
sol.feature(fname).feature(lname).set(pname,value)

sol.feature(fname).create(lname,"Iterative")
sol.feature(fname).feature(lname).set(pname,value)
sol.feature(fname).feature(lname).create(pcname,PType);
sol.feature(fname).feature(lname).feature(pcname).set(pname,value)

sol.feature(fname).feature(lname).create(pname,"Multigrid")
sol.feature(fname).feature(lname).feature(pname).feature("pr").create(pname,SType)
sol.feature(fname).feature(lname).feature(pname).feature("po").create(postname,SType)
sol.feature(fname).feature(lname).feature(pname).feature("cs").create(cname,CType)
sol.feature(fname).feature(lname).feature(pname).feature("pr").feature(pname).
    set(pname,value)
sol.feature(fname).feature(lname).feature(pname).feature("po").feature(postname).
    set(pname,value)
sol.feature(fname).feature(lname).feature(pname).feature("cs").feature(cname).
    set(pname,value)
```

PType is any of the allowed preconditioner feature types. These types are Direct preconditioner, Multigrid, Domain decomposition, SCGS, SOR, SOR Gauge, SOR Vector, Vanka, SOR Line, AMS, Auxiliary-Space AMG, Incomplete LU, Krylov, and Block Navier–Stokes. *SType* is any of the allowed smoother types. These are the same as the *PType* except Domain decomposition, Multigrid, AMS, and Auxiliary-Space AMG. *CType* is any of the allowed coarse grid solver types: Direct and all types listed for *PType*.

Presmoothing, *AMG Presmoothing*, *Postsmoothing*, *AMG Postsmoothing*, *Coarse Solver*, *AMG Coarse Solver*, *SchurSolver*, *SchurSourceSolver*, *PressureSolver*, and *VelocitySolver*

Specify the Presmoothing and AMG Presmoothing, Postsmoothing and AMG Postsmoothing, Coarse Solver and AMG Coarse Solver, Schur Solver, Schur Source Solver, Pressure Solver, and Velocity Solver via `feature("pr")`, `feature("po")`, `feature("cs")`, `feature("sso")`, `feature("ssso")`, `feature("ps")`, and `feature("vs")`, respectively. These features are for administrative purposes only and do not contain any settings themselves.

DESCRIPTION

Three attribute features for linear system solvers.

EXAMPLES:

GMRES with ILU as preconditioner:

Code for Use with Java

```
SolverFeature solver = sol.feature(fname).create("iter1","Iterative");
solver.set("solver","gmres");
solver.create("ilu","IncompleteLU");
```

Code for Use with MATLAB

```
solver = sol.feature(fname).create('iter1','Iterative');
solver.set('solver','gmres');
solver.create('ilu','IncompleteLU');
```

Change the preconditioner to GMG/SORVector/SPOOLES:

Code for Use with Java

```
solver.create("gmg","Multigrid");
solver("gmg").set("solver","gmg");
solver("gmg").feature("pr").create("p1","SORVector");
solver("gmg").feature("po").create("p1","SORVector");
SolverFeature csolver = solver("gmg").feature("csolver").create("c1","Direct");
csolver.set("solver","spooles");
csolver.set("errorchkd","on");
```

Code for Use with MATLAB

```
solver.create('gmg','Multigrid');
solver('gmg').set('solver','gmg');
solver('gmg').feature('pr').create('p1','SORVector');
solver('gmg').feature('po').create('p1','SORVector');
csolver = solver('gmg').feature('csolver').create('c1','Direct');
csolver.set('solver','spooles');
csolver.set('errorchkd','on');
```

Use Conjugate Gradients instead of GMRES:

Code for Use with Java

```
solver.set("solver","cg");
```

Code for Use with MATLAB

```
solver.set('solver','cg');
```

Use the sparse approximate inverse (SAI) preconditioner using a sparsity pattern of SAI defined by a power of 3 of the system matrix:

Code for Use with Java

```
model.sol("sol1").feature("t1").create("i1", "Iterative");
model.sol("sol1").feature("t1").feature("i1").set("linsolver", "cg");
model.sol("sol1").feature("t1").feature("i1").create("sai1", "SAI");
model.sol("sol1").feature("t1").feature("i1").feature("sai1").set("saisymm", "on");
model.sol("sol1").feature("t1").feature("i1").feature("sai1").
    set("saipattern","saipowa");
model.sol("sol1").feature("t1").feature("i1").feature("sai1").set("saipowera", "3");
```

Code for Use with MATLAB

```
iter = model.sol('sol1').feature('t1').create('i1', 'Iterative');
iter.set('linsolver', 'cg');
iter.create('sai1', 'SAI');
iter.feature('sai1').set('saisymm', 'on');
iter.feature('sai1').set('saipattern','saipowa');
iter.feature('sai1').set('saipowera', '3');
```

THE PREFUN PROPERTY

The `prefun` property for the solver (preconditioner) accepts a different set of values depending on the context. In the section below, its possible values and default value is listed for each preconditioner that supports it.

DIRECT PROPERTIES

TABLE 6-38: VALID DIRECT PROPERTIES (FOR ALL SOLVERS).

PROPERTY	VALUES	DEFAULT	DESCRIPTION
errorchk	off on auto	auto	Check error estimate.
errorratiobound	nonnegative scalar	0.5	Error ratio bound (typical values in the range of 0–1).
iterrefine	on off	on off (eigenvalue solver)	Iterative refinement.
linsolver	mumps pardiso spooles dense cudss	mumps	Method to use: MUMPS, PARDISO, SPOOLES, Dense matrix, or NVIDIA cuDSS.
maxrefinesteps	nonnegative integer	15	Maximum number of iterative refinement steps.
nliniterrefine	on off	off	Use iterative refinement in nonlinear solver.
prefun	mumps pardiso spooles dense cudss	mumps	Solver for preconditioner (MUMPS, PARDISO, SPOOLES, Dense matrix, or cuDSS).
rhob	scalar > 1	1	Factor in linear error estimate.

TABLE 6-39: OPTIONAL DIRECT PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
clusterpardiso	on off	off	Use Parallel Direct Sparse Solver for Clusters (PARDISO).
cudssfactor	auto default modified	auto	Factorization algorithm (cuDSS).
cudssmatching	auto alg1 alg2 alg3 alg4 alg5	auto	Row reordering algorithm: alg1 and so on represent the cuDSS Algorithm 1 and so on (cuDSS).
cudssmemoryfactor	real 0–1	0.8	Memory factor for hybrid mode, if cudssusehybridmemory is set to auto or always (cuDSS).
cudsspivotepsilon	positive real	1e-13	Pivot perturbation (cuDSS).
cudsspivotthreshold	positive real	1	Pivot threshold (cuDSS).
cudssprecision	double single	double	Floating point precision (cuDSS).
cudssreorder	auto nd alg1 alg2 amd	auto	Preordering algorithm: automatic, nested dissection, COLAMB/block triangular, COLAMB/simple block triangular, or approximate minimum degree (cuDSS).
cudssreusereorder	true false	true	Reuse preordering (cuDSS).
cudssusehybridcompute	true false	false	Use hybrid compute mode (cuDSS).
cudssusehybridmemory	auto always never	off	Use hybrid memory mode (cuDSS).
cudssusematching	true false	false	Use row preordering (cuDSS).
cudssusermemorylimit	integer	4096	Memory limit in MB (cuDSS).
incore	auto manual	auto	In-core memory method (PARDISO, MUMPS).
internalmemusage	positive real	3	Internal memory usage factor (PARDISO, MUMPS).

TABLE 6-39: OPTIONAL DIRECT PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
memfracooc	scalar between 0 and 1	0.99	Fraction of memory to store out of core.
minicmemory	positive real	512.0	Minimum in-core memory in MB (PARDISO, MUMPS).
mumpsalloc	scalar at least 1	1.2	MUMPS memory allocation factor (MUMPS).
mumpsblr	on off	off	Block low rank factorization (MUMPS).
mumpsblrto1	scalar between 0 and 1	1e-8	Block low rank factorization tolerance (MUMPS).
mumpsblrtype	normal aggressive	normal	The compression type for the block low rank factorization (MUMPS).
mumpsmtfact	on off	on	Multithreaded matrix factorization (MUMPS).
mumpsmtsolve	on off	on	Multithreaded forward and backward solve (MUMPS).
mumpsreorder	auto amd amf qamd nd	auto	Preordering algorithm (MUMPS).
mumpsrreorder	on off	on	Row preordering (MUMPS).
ooc	automatic on off	automatic	Use out-of-core (PARDISO, MUMPS). The automatic option switches to out-of-core storage if needed.
oocmemory	positive real	512.0	In-core memory in MB (PARDISO, MUMPS).
pardmtsolve	on off	on	Multithreaded forward and backward solve (PARDISO).
pardreorder	mmd nd ndmt	nd	Preordering algorithm (PARDISO).
pardrreorder	on off	on	Row preordering algorithm (PARDISO).
pardschedule	auto one two	auto	Scheduling method (PARDISO).
pivotenable	on off	on	Use pivoting (MUMPS).
pivotperturb	scalar between 0 and 1	1e-8	Pivot perturbation threshold (PARDISO, MUMPS).
pivotrefines	nonnegative integer	0	Number of forced iterative refinements (PARDISO, MUMPS).
pivotstrategy	on off	off	Use 2-by-2 Bunch–Kaufman pivoting (on) or 1-by-1 diagonal pivoting (off) (PARDISO).
preorder	mmd nd ms bestof	nd	Preordering algorithm (SPOOLES).
reusereorder	true false	true	Reuse preordering (PARDISO, MUMPS).
reusepattern	true false	true	Reuse sparsity pattern, if reusereorder is set to true (PARDISO, MUMPS).
thresh	scalar between 0 and 1	0.1	Pivot threshold (MUMPS, SPOOLES).
usetotmemory	scalar between 0 and 1	0.8	Used fraction of total memory (PARDISO, MUMPS).

ITERATIVE PROPERTIES

TABLE 6-40: VALID ITERATIVE PROPERTIES (FOR ALL SOLVERS).

PROPERTY	VALUES	DEFAULT	DESCRIPTION
errorchk	off on auto	auto	Validate error estimate.
linsolver	gmres fgmres bicgstab tfqmr cg precondition	gmres, ilu (precond)	Method to use.
maxlinit	positive integer	100	Maximum number of intermediate iteration for the iterative solver in error checking. Available when errorchk is auto.
maxlinit	positive integer	10000 500 (coarse solver)	Maximum number of linear iterations (when used with a tolerance).
rhob	scalar > 1	1	Factor in linear error estimate.

TABLE 6-41: OPTIONAL ITERATIVE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
approxscgs	true false	false	Use approximate approximation, if vankavarsactive is set to true (SCGS).
droptol	scalar between 0 and 1	0.01 when used as preconditioner; 1 when used as presmoothing, postsmoothing, or coarse solver; 0.001 for Hierarchical LU and Sparse Localized Schur	Drop tolerance (SPOOLES, Hierarchical LU, Incomplete LU with tolerance element dropping strategy, and Sparse Localized Schur).
droptype	tol fill	tol	Element dropping strategy for Incomplete LU (Tolerance, Fill ratio).
eigk	positive integer	25	Number of approximate eigenvectors, when gcrodr is set to true (GMRES).
eigkrel	scalar between 0 and 1	0.5	Fraction of GMRES iterations when gcrodrrel is set to true (GMRES).
elimtol	scalar between 0 and 1	1	Elimination tolerance (Hierarchical LU).
fillratio	nonnegative integer	1	Fill ratio (Incomplete LU with Fill ratio element dropping strategy and SAI).
gcrodr	true false	true	Use GCRO-DR for an adaptive restarted GMRES solver (GMRES).
gcrodrrel	true false	false	Use relative subspace size when gcrodr is set to true (GMRES).
hybridcomp	vector of strings		Field/State components in step if hybridvarspec>manual.
hybridization	single multi	single	Use a single preconditioner or multiple preconditioner as a hybrid preconditioner.
hybridvar	vector of strings		Fields/States in step.

TABLE 6-41: OPTIONAL ITERATIVE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
hybridvarspec	all manual	all	Include all components or specify which manually.
irestol	scalar between 0 and 1	0.01	Residual tolerance when prefuntype is left (gmres, cg, bicgstab, tfqmr).
ilutdroptol	scalar between 0 and 1	0.01	Drop tolerance (ILUT).
ilutfillratio	nonnegative integer	1	Fill ratio (ILUT).
iter	nonnegative integer	2 (1 for ILU and ILUT)	Fixed number of iterations (when used as preconditioner, smoother, or coarse solver), for all iterative solvers except direct preconditioner when iterm is set to iter or itertol.
iterm	tol iter itertol	iter	Termination technique (when Incomplete LU, ILUT, or ILU0 are used as coarse solver or Localized Schur or Sparse Localized Schur are used as Schur solver).
itol	scalar between 0 and 1	0.1	Relative tolerance when iterm is set to tol or itertol.
itrestart	positive integer	50	Number of iterations before restart (gmres, fgmres).
keepy	auto on off	auto	Reuse the subspace when gcrodr is set to true (GMRES).
linesweepstype	ssor sor soru	ssor	The sweep type, which decides in which order the blocks are looped through (SCGS, SOR line, Vanka).
matrixformat	auto sparse filled matrixfree	auto	Matrix format.
maxlinit	positive integer	10	Maximum number of iterations when iterm is set to tol.
nrkeepss	positive integer	5	Minimum number of restarts for GCRO-DR, when keepy is set to auto.
prefun	ilu ilut ilu0 spooles	ilu	Solver (preconditioner) for Incomplete LU, ILUT (Intel MKL), ILU0 (Intel MKL), or SPOOLES).
prefuntype	auto (SAI only) left right	left (auto for SAI)	Left or right preconditioning (gmres, cg, bicgstab, tfqmr, SAI). For SAI, the default preconditioning type is auto.
relax	scalar between 0 and 2	1	Relaxation factor (Jacobi, SOR-based algorithms, ILU, ILUT, SAI, Vanka, and Hierarchical LU).
respectpattern	on off	on	Respect the matrix pattern (ILU).
reuselines	on off	on	Reuse lines of nodes (SOR line).
reusepattern	true false	true	Reuse sparsity pattern (SCGS).
reuseprolongators	on off	on	Reuse blocks of data (SCGS, SOR line, Vanka).
saicolmaxfactor	positive integer	5	Limit the maximum number of nonzero elements of each column in the SAI preconditioner matrix.

TABLE 6-41: OPTIONAL ITERATIVE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
saisymm	auto off on hermitian	auto	Symmetry characteristics of SAI method corresponding to Automatic, Nonsymmetric, Symmetric, and Hermitian, respectively.
saipattern	sysmat saipowa	sysmat	Sparsity pattern of SAI defined by system matrix or power of system matrix.
saipowera	positive integer	2	Power of system matrix for SAI sparsity pattern.
scgsblocked	true false	true	Use a blocked version (SCGS).
scgsdirectmaxsize	positive integer	150	Maximum size for direct stored factorization, if scgssolv is approx.
scgsfilter	true false	true	Apply filter to lines (SCGS)
scgsfilteraggressive	true false	true	Use an efficient smoother when a filter is applied to lines (SCGS).
scgsfilteramglvls	true false	false	Apply filter to AMG levels only, if scgsfilter is true (SCGS).
scgsfilterelems	true false	true	Filter individual elements, if scgsfilter is true (SCGS)
scgslinefiltertype	none filterstart filterall	filterstart	Apply filter to elements: none, unique start elements, or unique lines, if scgsfilter is true (SCGS).
scgsmethod	elements lines lines_vertices	elements	Method for SCGS: Mesh elements, Mesh element lines, or Mesh element lines and vertices (SCGS).
scgsmumpsminsize	positive integer	150	Maximum size for MUMPS factorization, if scgssolv is approx.
scgsrelax	scalar between 0 and 2	1	Relaxation factor (SCGS).
scgsrestart	positive integer	100	GMRES restart value (SCGS).
scgsreusemeshdata	true false	true	Reuse mesh data (SCGS).
scgssolv	gmres direct stored approx	gmres	Local block solver (SCGS).
scgstol	positive scalar	0.02	GMRES tolerance (SCGS).
seconditer	positive integer	1	Number of second iterations.
sorblocked	on off	on	Blocked SOR method.
sorvecdof	vector of strings		Vector element variables (SOR vector and SOR gauge algorithms).
symmetric	on off	off	Use symmetric form; ssor instead of sor, and so forth. (SOR, SORVector, SORGauge, SORLine).
thresh	scalar between 0 and 1	1	Pivot threshold (ILU).
transpose	on off	off	Use transposed form; soru instead of sor and so forth. (SOR, SORVector, SORGauge, SORLine).
usenlweights	true false	true	Terminate using nonlinear scales (GMRES with left preconditioning).
vankablocked	on off	on	Blocked Vanka method.

TABLE 6-41: OPTIONAL ITERATIVE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
vankadirectmaxsize	positive integer	150	Maximum size for direct stored factorization, if vankasolv is approx.
vankamumpsminsize	positive integer	150	Maximum size for MUMPS factorization, if vankasolv is approx.
vankarelay	scalar between 0 and 2	0.8	Relaxation factor for Vanka update.
vankarestart	positive integer	100	GMRES restart value (Vanka).
vankasolv	gmres direct stored approx	gmres	Local block solver (Vanka).
vankatol	positive scalar	0.02	GMRES tolerance (Vanka).
vankavars	vector of strings	{}	Lagrange multiplier variables (Vanka, SCGS).
vankavarsactive	true false	false	Use Vanka variables (SCGS).

The property `divcleantol` is used in the inequality $|T^T b| < \text{divcleantol} \cdot |b|$ to ensure that the numerical divergence after divergence cleaning is small enough; see [SOR Gauge](#) in the *COMSOL Multiphysics Reference Manual*.

MULTIGRID PROPERTIES

TABLE 6-42: VALID MULTIGRID PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
gmglevels	positive integer	1	Maximum number of geometric multigrid levels.
iter	integer	2	Fixed number of iterations (when used as preconditioner, smoother, or coarse solver).
linsolver	gm amg	gm	Method to use.
maxlinit	positive integer	500	Maximum number of linear iterations (when used with a tolerance).
mgcycle	v w f	v	Cycle type.
mglevels	positive integer	5	Maximum number of algebraic multigrid levels.
prefermatfree	true false	false	Prefer the free matrix format.
prefun	gm amg saamg	amg if used as Krylov preconditioner. gm, otherwise.	Solver for preconditioner (Geometric multigrid, Algebraic multigrid, or Smoothed aggregation AMG).
rhob	scalar > 1	1	Factor in linear error estimate.

TABLE 6-43: OPTIONAL MULTIGRID PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
agglevel	positive integer	2	Multigrid level to start smoothing at, if <code>aggmethod=manual</code> (saamg).
aggmethod	auto manual	auto	Smoothing of prolongations if <code>usesmooth=on</code> (saamg).
amgauto	integer from 1 to 10	3	Quality of multigrid hierarchy (amg).
amgcompwise	true false	true	Construct prolongators componentwise (amg).
assemall	on off	on	Assemble on the order-lowered levels when <code>loweramg</code> is on (amg and saamg).

TABLE 6-43: OPTIONAL MULTIGRID PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
coarseningmethod	parallel classic	parallel	The coarsening method to use (amg).
compactaggregation	on off	off	Use an aggregation algorithm that leads to a less rapid coarsening (saamg).
geomuse	vector of strings		Geometries for geometric multigrid hierarchy.
hybridization	single multi	single	Use a single preconditioner or multiple preconditioner as a hybrid preconditioner.
hybridvar	vector of strings		Fields/States in step.
hybridvarspec	all manual	all	Include all components or specify which manually.
hybridcomp	vector of strings		Field/State components in step if hybridvarspec>manual.
jacdamp	positive scalar	2/3	Jacobi damping factor if usesmooth=on (saamg).
kwslaplacemg	on off	off	Keep generated weak contribution.
loweramg	on off	on	Lower the element order first (any) (amg and saamg).
massem	on off	on	Assemble on multigrid levels (gmg).
maxcoarsedof	positive integer	5000	Maximum number of DOFs at coarsest level (amg, saamg).
mcaseassem	vector of strings		Multigrid levels where assemble should be performed (gmg, mcasegen>manual).
mcasegen	manual all any coarse coarseorder refine refineany refineall	any	Hierarchy generation method (gmg).
mcaseuse	vector of strings		Multigrid levels which should be used (gmg, mcasegen>manual).
mindofthread	positive integer	5000	Minimum number of DOFs per thread on multigrid levels (when prefun is not gmg).
mkeep	on off	off	Keep generated mesh cases (gmg).
nullspace	constant rbm		Null-space vectors: constant or rigid body modes (saamg).
prolongatoronlystrongconn	on off	on	Use a sparser prolongator to achieve better performance per iteration (amg).
prolongatortruncation	positive scalar	0.1	Prolongator truncation factor (amg).
reuseprolongators	on off	on	Reuse prolongators when possible (saamg).
rmethod	regular longest	regular	Mesh refinement method (gmg).
saamgcompwise	on off	off	Construct the SAAMG prolongators componentwise (saamg).
scale	vector of positive numbers	2	Mesh scale factor (gmg).
strconn	positive scalar	0.01	Strength of connection (saamg).

TABLE 6-43: OPTIONAL MULTIGRID PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
strconnamgp	positive scalar	0.25	Strength of connection (amg).
usefilter	on off	on	Use filtering if usesmooth=on (saamg).
usesmooth	on off	on	Use prolongation smoothing (saamg).

The useaggressive property is no longer used from version 5.3. For the geometric multigrid solver/preconditioners, the multigrid hierarchy is controlled in the following way (see also [Multigrid](#) in the *COMSOL Multiphysics Reference Manual*):

- If `mcasegen=all`, `any`, or `coarse`, `coarseorder`, then the multigrid hierarchy is automatically constructed starting from the mesh and discretization set by the study. The number of multigrid levels generated is given in the property `gmglevels`. The method `all` and `any` first tries to lower the discretization order for the shape functions used, and secondly coarsens the mesh. The method `all` lowers the order (by one) if all used shape functions can be lowered. The method `any` lowers the order (by one) if at least one shape function can be lowered. The method `coarse` does not lower the order, it only coarsens the mesh. The method `coarseorder` both lowers the order (for any shape functions that can be lowered by one) and coarsen the mesh, at the same time.
- If `mcasegen=refine`, `refineany`, or `refineall` then the multigrid hierarchy is automatically constructed by a combination of refining the mesh given by the study and changing the discretization. The number of multigrid levels generated is given in the property `gmglevels`. The refinement method can be specified using the property `rmethod`. The originally selected mesh for the study is used, in the case of refining the mesh, in a multigridlevel and the finest multigrid level generated is used for the study (solved for). The generated multigrid levels are kept in the model and the `mcasegen` property is changed into `manual`. The method `refine` only refines the mesh and does not change the shape function order. The method `refineany` and `refineall` first tries to lower the order, and secondly refines the mesh. The method `refineany` constructs a multigrid level by lowering the order (by one) if at least on shape function can be lowered. The method `refineall` generates multigrid levels by lowering the order (by one) if all used shape function can be lowered.
- If `mcasegen>manual`, then the existing multigrid levels (children to the current study) can be used. The subset to use is selected by giving their tags to the `mcaseuse` property.

The construction of coarse level matrices is controlled by the property `massem` and `mcaseassem`. The first property controls if the matrices should be assembled for the automatically generated levels. If set to `off`, prolongation and restriction matrices are used to project the matrices from the top level in the hierarchy. The second property controls which multigrid levels that should use the assemble technique in the `mcasegen>manual` case.

When an iterative solver is used as preconditioner, smoother, or coarse solver, you can choose whether to solve using a tolerance or to perform a fixed number of iterations. When used as a coarse solver, the default is to solve using a tolerance. When used as a preconditioner or smoother, the default is to perform a fixed number of iterations. If both properties `itol` and `iter` are given, the program solves using a tolerance.

DOMAIN DECOMPOSITION PROPERTIES (SCHWARZ AND SCHUR)

TABLE 6-44: VALID DOMAIN DECOMPOSITION PROPERTIES (SCHWARZ AND SCHUR).

PROPERTY	VALUE	DEFAULT	DESCRIPTION
agglevel	positive integer	2	Multigrid level to start smoothing at, if aggmeth = manual (aggregation) (Schwarz only).
aggmethod	auto manual	auto	Postpone prolongation smoothing (aggregation) (Schwarz only).
amgauto	integer from 1 to 10	3	Quality of multigrid hierarchy (algebraic) (Schwarz only).
amgauto	integer from 1 to 10	3	Quality of multigrid hierarchy (algebraic) (Schwarz only).
assemall	on off	on	Assemble on the order-lowered levels (algebraic, aggregation) if loweramgdd is on (Schwarz only).
compactaggregation	on off	off	Use an aggregation algorithm that leads to a less rapid coarsening (aggregation) (Schwarz only).
ddreordermethod	none dissection space	dissection	Element preordering method: none, nested dissection, or using a space-filling curve.
ddvisualize	boolean	false	Visualize domains in the domain decomposition instead of the finite element solution.
domdofmax	integer	1	Maximum number of nodes per subdomain.
domgeom	vector of strings		Partition geometries.
domnodesmax	integer	100000	Maximum number of DOFs per subdomain.
geomuse	vector of strings		Use coarse level in geometries (for mcasegen=coarseorder all any coarse refineall refineany refine) (Schwarz only).
hybridcomp	vector of strings		Field/state components in step / Preconditioner selection (for hybridvarspec= manual).
hybridization	single multi	single	Use a single preconditioner or multiple preconditioner as a hybrid preconditioner.
hybridvar	vector of strings		Fields/states in step / Preconditioner variables (for hybridization=multi).
hybridvarspec	all manual	all	Include all components or specify which manually (for hybridization=multi).
includeextradim	boolean	true	Include extra dimensions (if used) in the partitioning of domains for the domain decomposition.
iter	integer	1	Number of iterations.
jacdamp	positive scalar	2/3	Jacobi damping factor (aggregation) (Schwarz only).

TABLE 6-44: VALID DOMAIN DECOMPOSITION PROPERTIES (SCHWARZ AND SCHUR).

PROPERTY	VALUE	DEFAULT	DESCRIPTION
keepschur	on off	on	Keep Schur complement matrices (Schur only).
kwslaplacemg	on off	off	Keep generated weak contribution.
loweramgdd	on off	on	Lower element order first (algebraic, aggregation) (Schwarz only).
massem	on off	on	Assemble on coarse level (for mcasegen=coarseorder all any coarse refineall refineany refine manual) (Schwarz only).
maxcoarsedof	positive integer	5000	Maximum number of DOFs at coarsest level (algebraic, aggregation) if maxcoarsedofsactive is on (Schwarz only).
maxcoarsedofsactive	on off	off	Enable setting the maximum number of DOFs at coarsest level (Schwarz only).
mcasegen	coarseorder all any coarse refineall refineany refine manual	any	Coarse level generation method (Schwarz only).
mcaseuse	vector of strings		Coarse level that should be used (for mcasegen>manual) (Schwarz only).
mglevels	positive integer	5	Maximum number of algebraic multigrid levels (algebraic, aggregation) (Schwarz only).
mkeep	on off	on	Keep generated coarse level (for mcasegen=all any coarse) (Schwarz only).
ndom	integer	2 (Schwarz); 4 (Schur)	Minimum number of subdomains.
overlap	integer	1	Additional overlap.
overlapmethod	auto matrix mesh	auto	Overlap method (Schwarz only).
prefermatfree	on off	off	Automatically choose matrix free format (Schwarz only).
prefun (Schur)	ddschuradd ddschurm1	ddchurm1	Domain decomposition solver (Additive Schur/Multiplicative Schur), for Schur methods.
prefun (Schwarz)	ddadd ddm1 ddhyb ddsym	ddm1	Domain decomposition solver (Additive Schwarz/Multiplicative Schwarz/Hybrid Schwarz/ Symmetric Schwarz), for Schwarz methods.
reuseprolongators	on off	on	Reuse prolongators when possible (aggregation) (Schwarz only).
rmethod	longest regular	longest	Mesh refinement method (for mcasegen=refineall refineany refine) (Schwarz only).
scale	scalar > 0	2.0	Mesh coarsening factor (for mcasegen=coarseorder any all coarse) (Schwarz only).
strconn	positive scalar	0.01	Strength of connection (aggregation) (Schwarz only).

TABLE 6-44: VALID DOMAIN DECOMPOSITION PROPERTIES (SCHWARZ AND SCHUR).

PROPERTY	VALUE	DEFAULT	DESCRIPTION
useaggressive	on off	on	Use aggressive coarsening (aggregation) (Schwarz only).
usecoarse	on algebraic aggregation nicolaides off	on	Use coarse level: geometric (on), algebraic multigrid (algebraic), aggregations AMG (aggregation), nicolaides (Schwarz only), or off (Schwarz only).
usecoloring	on off	on	Use coloring (for prefun = ddmul ddsym) (Schwarz only).
usefilter	on off	on	Use filtering (aggregation) (Schwarz only).
userac	auto on off	auto	Recompute and clear subdomain data.
useschurcoloring	on off	on	Use coloring (for prefun = ddschurmul) (Schur only).

Example Code for Use With Java

```

model.sol("sol1").feature("s1").create("i1", "Iterative");
model.sol("sol1").feature("s1").feature("i1").create("dd1", "DomainDecomposition");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").feature("ds").
    create("mg1", "Multigrid");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").set("prefun", "ddadd");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").set("mcasegen", "refine");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").set("domnodesmax", "2");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").
    set("overlapmethod", "mesh");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").set("ndom", "16");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").
    set("domgeom", new String[]{"geom1"});
model.sol("sol1").feature("s1").feature("i1").feature("dd1").feature("cs").
    feature("dDef").set("linsolver", "pardiso");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").feature("ds").
    feature("mg1").set("gmglevels", "5");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").
    set("hybridization", "multi");
model.sol("sol1").feature("s1").feature("i1").feature("dd1").
    set("hybridcomp", new String[]{"comp1.u"});

```

SOR PREFUN PROPERTY

TABLE 6-45: SOR PREFUN PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prefun	ssor sor soru	ssor	Solver (SSOR, SOR, or SORU).

SSOR GAUGE PREFUN PROPERTY

TABLE 6-46: SSOR GAUGE PREFUN PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prefun	ssorgauge sorgauge sorugauge	sorgauge when used as presmoothing; sorugauge when used as postsmoothing; ssorgauge otherwise	Solver (SSOR gauge, SOR gauge, or SORU gauge).

SOR VECTOR PREFUN PROPERTY

TABLE 6-47: SOR VECTOR PREFUN PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
prefun	ssorvec sorvec soruvec	sorvec when used as presmoothing; soruvec when used as postsmoothing; ssorvec otherwise	Solver (SSOR vector, SOR vector, or SORU vector).

LowerLimit

Handle lower limits for segregated steps. This feature can be added as a subfeature to a Segregated feature.

SYNTAX

```
model.sol(sname).feature(solv).feature(segreated).create(fname, "LowerLimit")
model.sol(sname).feature(solv).feature(segreated).feature(fname).set(pname, value)
```

DESCRIPTION

This feature controls the lower limits for variables used in segregated steps.

TABLE 6-48: VALID LOWER LIMIT PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
lowerlimit	String		String of variable and lower limit pairs.

A valid lowerlimit string contains pairs of variables names and their associated lower limits. For example, to impose a lower limit of 0.25 for the value of the field u in Component 1, and a lower limit of 0.0 for the value of the field v in Component 1, use the string "comp1.u 0.25 comp1.v 0.0".

SEE ALSO

[Segreated](#), [UpperLimit](#)

LumpedStep

Handle a lumped solution step, which can be added as a subfeature to a Segreated feature.

SYNTAX

```
model.sol(sname).feature(solv).feature(segreated).create(fname, "LumpedStep")
model.sol(sname).feature(solv).feature(segreated).feature(fname).set(pname, value)
```


DESCRIPTION

This feature controls one lumped solution step.

TABLE 6-49: VALID LUMPED STEP PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
segcomp	vector of strings		Field/State components in step if segvarspec=manual
segvar	vector of strings		Fields/States in step
segvarspec	all manual	all	Include all components or specify which manually
submatrixformat	auto sparse filled free	auto	Matrix format: automatic, sparse, filled, or free.

See [SegregatedStep](#) for more information about these properties.

Modal

Solve parametric or time-dependent problem using the eigenmodal method.

SYNTAX

```
model.sol(sname).create(fname, "Modal")  
model.sol(sname).feature(fname).set(pname, pvalue)
```

DESCRIPTION

Operation feature. The following properties are accepted:

TABLE 6-50: VALID MODAL PROPERTIES FOR THE MODAL SOLVER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
analysistype	frequency transient	frequency	Solve for frequency response or transient response.
atolglobal	positive scalar	0.001	Absolute tolerance when atolglobalvaluemethod is set to manual.
atolglobalfactor	positive scalar	0.1	Tolerance factor when atolglobalvaluemethod is set to factor.
atolglobalmethod	scaled unscaled	scaled	Global method for the absolute tolerance when analysistype has been set to transient.
atolglobalvaluemethod	factor manual	factor	Tolerance method if analysistype has been set to transient.
clist	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .
cname	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use clist. See The clist and cname Properties .
complex	true false	false	Allow complex-valued data, if analysistype has been set to transient.
constrmodes	array of integers		The constraint modes to use, if constrmodeselection is set to manual.
constrmodeselection	all manual	all	The constraint modes to use.
constrsol	solution object or none		The solution to use for the constraint modes.

TABLE 6-50: VALID MODAL PROPERTIES FOR THE MODAL SOLVER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
dampratio	scalar numeric vector	0	Damping ratios for participating modes.
eigsol	solution object or none		Precomputed eigenpairs (or other vectors) to be used in the modal analysis.
eigsoluse	current manual an applicable solution	current	The parametric solution to use if such a solution has been set as the eigsol.
eigsolusesolnum	solution index		The solution index to use if eigsoluse is set to manual.
ewtrescale	true false	true	Update scaled absolute tolerance when analysistype has been set to transient.
initialdataextend	automatic true false	automatic	Extend basis with initial data when analysistype has been set to transient.
keeplog	on off	off	Keep warnings in stored log.
linearity	false linper	false	Use linear linearity or a linear perturbation when analysistype has been set to frequency.
linplistsolnum	Integer array	new int[] {1}	List of solutions to use for the linearization point, when linpsolnum is set to from_list.
linpmanualsolnum	Integer array	new int[] {1}	Solution number (index) to use for the linearization point, when linpsolnum is set to manual.
linpmethod	init sol	init	Method used for linearization point, which for sol is given by the linpsol property.
linpsol	zero solution object	zero	Linearization point solution.
linpsolnum	all auto first last from_list interp manual solution number	1	Indices to solutions to use as linearization points.
linpsoluse	current manual solution store tag	current	Subsolution that defines linearization point for the Eigenvalue study step.
linpsolusesolnum	integer		Index for the solution to use when linpsoluse is set to manual.
linpt	double	0	Interpolated point for the linearization point when linpsolnum is set to interp.
maxfreq	positive scalar	0	Maximum frequency if maxfreqactive is true.
maxfreqactive	true false	false	True to specify the maximum frequency manually.
maxstepbdf	positive scalar	1e - 1	Maximum time step when maxstepconstraintbdf is set to const.
maxstepconstraintbdf	auto const expr	auto	Maximum time step for time-dependent modal analysis: automatic (auto), constant (const), or an expression (expr).
maxstepexpressionbdf	String		Expression for the maximum time step when maxstepconstraintbdf is set to expr.

TABLE 6-50: VALID MODAL PROPERTIES FOR THE MODAL SOLVER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
message	String		The log message from the last solution process.
minfreq	positive scalar	0	Minimum frequency if maxfreqactive is true.
minfreqactive	true false	false	True to specify the minimum frequency manually.
modes	integer vector	all	Participating modes.
modeselection	all manual	all	The eigenpairs to use.
outsollinearized	du u	du	Store the total solution (u) or deviation and linearization point (du), when analysistype is set to frequency and storelinpoint is set to off.
plist	scalar numeric vector		Frequency list. Only applicable when analysistype has been set to frequency.
plistaddfreq	numeric list		List of additional frequencies when pout is set to fraction or spread.
pname	vector of strings		Parameter names.
pout	plist fraction spread	plist	Use plist as it stands or modify in relation to the participating modes.
punit	A valid frequency unit	Hz	Frequency unit. Only applicable when analysistype has been set to frequency.
romdata	String	new	Tag of the target container for the reduced model (new for a new reduced model).
romReconstruct	true false	true	Enable reconstruction in the produced reduced model.
rtol	scalar	0.01	Relative tolerance. Only applicable when analysistype has been set to transient.
soltypemat	on off	on	Store reduced-model matrices.
soltypeonline	on off	off	Create a reduced model.
soltypesol	on off	on	Perform frequency sweep or transient simulation using the modal solver.
storelinpoint	on off	off	Whether to store the linearization point.
storeudot	on off	on	Store time derivatives if analysistype has been set to transient.
tderglobalfactor	positive scalar	1	Time derivative factor when tdergloablmethode is set to manual.
tderglobalmethod	auto manual	auto	Derivative tolerance method when analysistype has been set to transient.
tlist	scalar numeric vector		Time list. Only applicable when analysistype has been set to transient.
tunit	A valid time unit	s	Time unit. Only applicable when analysistype has been set to transient.

In addition, the following properties are available for exporting matrices and vectors:

TABLE 6-51: VALID MODAL PROPERTIES FOR MATRIX AND VECTOR EXPORT FROM THE MODAL SOLVER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
AllL	on off	off	All load vectors, frequency.
B0r	on off	off	Reduced initial value input matrix, transient.
B0rdot	on off	off	Reduced initial time derivative input matrix, transient.
Br	on off	off	Reduced input matrix, transient and frequency.
Brdot	on off	off	Reduced time derivative input matrix, transient.
Brdotdot	on off	off	Reduced second time derivative input matrix, transient.
C	on off	off	Output matrix (state space).
Clmap	on off	off	Constraint modes to inputs map.
Cr	on off	off	Reduced output matrix, transient and frequency.
D	on off	off	Input feedback matrix (state space).
DPartSol	on off	off	Damping matrix times particular solution, frequency.
Dr	on off	off	Reduced damping matrix, transient and frequency.
Dra	on off	off	Damping ratio matrix, transient and frequency.
EPartSol	on off	off	Mass matrix times particular solution, frequency.
Er	on off	off	Reduced mass matrix, transient and frequency.
F	on off	off	Input feedback, transient and frequency.
Kr	on off	off	Reduced stiffness matrix, transient and frequency.
Kud	on off	off	Stiffness matrix times u_d , transient.
L	on off	off	Load vector, transient and frequency.
MA	on off	off	Stiffness matrix (state space).
MB	on off	off	Input matrix (state space).
Mc	on off	off	Mass matrix (state space).
Pm	on off	off	Projection matrix, transient and frequency.
U0	on off	off	Initial value vector, transient.
ud	on off	off	Particular solution (state space).
Udot0	on off	off	Initial derivative vector, transient.
x0	on off	off	Initial value (state space).
Y0	on off	off	Output bias, transient and frequency.

The reduced matrices are the matrices that result from the elimination of constraints in the discretized model.

For frequency response analysis, nonconstant Neumann boundary conditions and constant Dirichlet boundary conditions are supported. The only allowed type of parameter-dependent Dirichlet boundary condition are those that can be written as a constant vector times a scalar function. The scalar function is specified via the property `loadfact`. For transient response analysis only constant Dirichlet boundary conditions are supported. Neumann conditions that can be written as a constant vector times a scalar function are supported for transient response (specified in a Frequency Domain, Modal or Time Dependent, Modal study step).

The property `modes` is index 0 based.

If `pout` is set to `fraction` the output frequencies are the ones in `plist` multiplied by the absolute value of the largest eigenvalue in `eigsol` (or some other fraction of the largest participating eigenvalue of `eigsol`). The purpose of this property is to be able to automatically compute the frequency response for reasonable frequencies. If `pout` is set to `spread` then `plist` is interpreted as an interval around each participating eigenvalue. For example, if `plist`

is set to range(0.9,0.04,1.1) then each participating eigenvalue is multiplied by this list, and the resulting lists are concatenated into the plist that is used.

REMOVED PROPERTIES

TABLE 6-52: REMOVED PROPERTIES FOR THE MODAL SOLVER SEQUENCE FEATURE.

PROPERTY	REASON
Callblevel	Given by the solver sequence attribute feature.
soltype	Deprecated in version 5.3 and replaced with soltypesol, soltypemat, and soltypeonline.

ModalReduction

Solve parametric or time-dependent problem using modal reduction.

SYNTAX

```
model.sol(sname).create(fname,"ModalReduction")
model.sol(sname).feature(fname).set(pname,pvalue)
```

DESCRIPTION

Operation feature. The following properties are accepted:

TABLE 6-53: VALID MODAL PROPERTIES FOR THE MODAL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
analysistype	frequency transient	frequency	Solve for frequency response or transient response.
clist	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .
cname	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use clist. See The clist and cname Properties .
constrmodes	array of integers		The constraint modes to use, if constrmodeselection is set to manual.
constrmodeselection	all manual	all	The constraint modes to use.
constrsol	solution object or none		The solution to use for the constraint modes.
constrsoluse	current manual solution store	current	Subsolution to use for the constraint modes.
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
eigensoluse	current manual solution store	current	Subsolution to use for the eigenmodes.
eigsol	solution object		Precomputed eigenpairs (or other vectors) to be used in the modal analysis.
initialdataextend	automatic true false	automatic	Extend basis with initial data. For automatic, it is done only if it is large enough to make a difference.
keeplog	on off	off	Keep warnings in stored log.
linearity	false linper	false	Use linear linearity or a linear perturbation when analysistype has been set to frequency.

TABLE 6-53: VALID MODAL PROPERTIES FOR THE MODAL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
linplistsolnum	Integer array	new int[] {1}	List of solutions to use for the linearization point, when linsolnum is set to from_list.
linpmanualsolnum	Integer array	new int[] {1}	Solution number (index) to use for the linearization point, when linsolnum is set to manual.
linpmethod	init sol	init	Method used for linearization point, which for sol is given by the linsol property.
linsol	zero solution object	zero	Linearization point solution.
linsolnum	all auto first last from_list interp manual solution number	1	Indices to solutions to use as linearization points.
linsoluse	current manual solution store	current	Subsolution that defines linearization point for the Eigenvalue study step.
linsolusesolnum	integer		Index for the solution to use when linsoluse is set to manual.
linpt	double	0	Interpolated point for the linearization point when linsolnum is set to interp.
message	String		The log message from the last solution process.
modes	integer vector	all	Participating modes.
modeselection	all manual	all	The eigenpairs to use.
outsollinearized	du u	du	Store the total solution (u) or deviation and linearization point (du), when analysistype is set to frequency and storelinpoint is set to off.
plist	scalar numeric vector		Frequency list. Only applicable when analysistype has been set to frequency.
pname	vector of strings		Parameter names.
pout	plist fraction spread	plist	Use plist as it stands or modify in relation to the participating modes.
romdata	String	new	Tag of the target container for the reduced model (new for a new reduced model).
rominterf	stateless stateful	stateless	The reduced-order model interface if romdata is set to new.
romReconstruct	true false	true	Enable reconstruction in the produced reduced model.
rtol	scalar	0.01	Relative tolerance. Only applicable when analysistype has been set to transient.
soltypemat	on off	off	Store reduced-model matrices.
soltypeonline	on off	off	Create a reduced model.
soltypesol	on off	on	Perform frequency sweep or transient simulation using the modal solver.
storelinpoint	on off	off	Whether to store the linearization point.

TABLE 6-53: VALID MODAL PROPERTIES FOR THE MODAL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
tlist	scalar numeric vector		Time list. Only applicable when analysistype has been set to transient.
usesens	true false	false	If set to true, the sensitivity solution (when available) is used as constraint modes.

In addition, the following properties are available for exporting matrices and vectors:

TABLE 6-54: VALID MODAL PROPERTIES FOR MATRIX AND VECTOR EXPORT FROM THE MODAL SOLVER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
AllL	on off	off	All load vectors, frequency.
B0r	on off	off	Reduced initial value input matrix, transient.
B0rdot	on off	off	Reduced initial time derivative input matrix, transient.
Br	on off	off	Reduced input matrix, transient and frequency.
Brdot	on off	off	Reduced time derivative input matrix, transient.
Brdotdot	on off	off	Reduced second time derivative input matrix, transient.
C	on off	off	Output matrix (state space).
C_c	on off	off	Constrained part of output matrix.
Clmap	on off	off	Constraint modes to inputs map.
Cr	on off	off	Reduced output matrix, transient and frequency.
D	on off	off	Input feedback matrix (state space).
DPartSol	on off	off	Damping matrix times particular solution, frequency.
Dr	on off	off	Reduced damping matrix, transient and frequency.
Dr_c	on off	off	Constrained part of damping matrix.
Dra	on off	off	Damping ratio matrix, transient and frequency.
Dra_c	on off	off	Constrained part of damping ratio matrix.
EPartSol	on off	off	Mass matrix times particular solution, frequency.
Er	on off	off	Reduced mass matrix, transient and frequency.
Er_c	on off	off	Constrained part of mass matrix.
F	on off	off	Input feedback, transient and frequency.
Kr	on off	off	Reduced stiffness matrix, transient and frequency.
Kr_c	on off	off	Constrained part of stiffness matrix.
Kud	on off	off	Stiffness matrix times u_d , transient.
L	on off	off	Load vector, transient and frequency.
MA	on off	off	Stiffness matrix (state space).
MA_c	on off	off	Constrained part of stiffness matrix.
MB	on off	off	Input matrix (state space).
Mc	on off	off	Mass matrix (state space).
Mc_c	on off	off	Constrained part of mass matrix.
Pm	on off	off	Projection matrix, transient and frequency.
U0	on off	off	Initial value vector, transient.
ud	on off	off	Particular solution (state space).
Udot0	on off	off	Initial derivative vector, transient.
x0	on off	off	Initial value (state space).
Y0	on off	off	Output bias, transient and frequency.

The reduced matrices are the matrices that result from the elimination of constraints in the discretized model.

For frequency response analysis, nonconstant Neumann boundary conditions and constant Dirichlet boundary conditions are supported. The only allowed type of parameter-dependent Dirichlet boundary condition are those that can be written as a constant vector times a scalar function. The scalar function is specified via the property `loadfact`. For transient response analysis only constant Dirichlet boundary conditions are supported. Neumann conditions that can be written as a constant vector times a scalar function (which is specified via the property `loadfact`) are supported for transient response.

The property `modes` is index-0 based.

If `pout` is set to `fraction` the output frequencies are the ones in `plist` multiplied by the absolute value of the largest eigenvalue in `eigsol` (or some other fraction of the largest participating eigenvalue of `eigsol`). The purpose of this property is to be able to automatically compute the frequency response for reasonable frequencies. If `pout` is set to `spread` then `plist` is interpreted as an interval around each participating eigenvalue. For example, if `plist` is set to `range(0.9,0.04,1.1)` then each participating eigenvalue is multiplied by this list, and the resulting lists are concatenated into the `plist` that is used.

ModeFollowing

Use the Mode Following solver to track modes when solving parametric eigenvalue problems.

SYNTAX

```
model.sol(sname).feature(solv).create(fname,"ModeFollowing")
model.sol(sname).feature(solv).feature(fname).set(pname,value)
```

DESCRIPTION

This feature can be used as an attribute for parametric Eigenvalue features. The following properties are accepted:

TABLE 6-55: VALID MODEFOLLOWING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>detectifcannotfollowpreviousmodes</code>	<code>on</code> <code>off</code>	<code>on</code>	Detect new eigenpairs instead of always forcing continuation.
<code>extramodesallocationfactorvalue</code>	<code>double</code>	<code>1</code>	The extra modes allocation factor, if <code>selectionextramodes</code> is set to <code>extramodesallocationfactor</code> .
<code>followextramodes</code>	<code>on</code> <code>off</code>	<code>off</code>	Whether to follow extra modes.
<code>highsenseigddetection</code>	<code>on</code> <code>off</code> <code>warningneweigtracking</code>	<code>off</code>	Detect new eigenpairs by checking if the pseudoscalar product is small compared to all the previous eigenpairs. If set to <code>warningneweigtracking</code> , a warning is given if new modes converge. Available when <code>detectifcannotfollowpreviousmodes</code> is set to <code>on</code> .
<code>numberofextramodesvalue</code>	<code>nonnegative integer</code>	<code>0</code>	The number of expected extra modes, if <code>selectionextramodes</code> is set to <code>numberofexpectedextramodes</code> .

TABLE 6-55: VALID MODEFOLLOWING PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
selectionextramodes	extramodesallocationfactor numberofexpectedextramodes	extramodesallocationfactor	Method to enter the number of expected extra modes, by allocation factor or as an integer. Available if followextramodes is set to on.
thresholdmodefollowing	double	0.01	Tolerance factor for detecting if an eigenpair is new, when detectifcannotfollowprevious modes is set to on.

Optimization

Handle optimization solver properties.

SYNTAX

```
model.sol(sname).create(fname,"Optimization")
model.sol(sname).feature(fname).set(pname,value)
model.sol(sname).feature(fname).create(aname,SolverAttribute)
```

DESCRIPTION

Operation feature. Use this feature to solve PDE-constrained optimization problems. The computed solution object contains the PDE solution evaluated for the optimal set of design variables. When the gradient-evaluation method is analytic, it also returns the adjoint solution.

The Optimization Module includes IPOPT and several other optimization solvers. See the *Optimization Module Manual* for details.

To add a stationary solver, substitute SolverAttribute above with StationaryAttrib. For a time-dependent solver, replace SolverAttribute with TimeAttrib.

Choosing solver is done with the following property:

TABLE 6-56: PROPERTY TO SELECT OPTIMIZATION SOLVER.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
optmethod	bobyqa cobyla coordsearch lm gmma mma montecarlo neldermead ipopt ego	neldermead	Optimization solver

The following table includes general optimization solver properties, which are common to all optimization solvers:

TABLE 6-57: GENERAL OPTIMIZATION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
clist	String array	off	Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .
cname	String array	off	Provide names of constants as input parameters using a string array; for the corresponding constant values, use clist. See The clist and cname Properties .
control	String	user	Name of the controlling study step or user, if the feature is controlled manually.

TABLE 6-57: GENERAL OPTIMIZATION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
keeplog	on off	off	Keep warnings in stored log.
message	String		The log message from the last solution process.
objcontrib	all manual	all	Whether to use all objective contributions present or specify manually (not available if optmethod is lm).
opttol	real scalar	0.01	Optimality tolerance for derivative free optimization solvers.
opttolinner	real scalar	0.001	Optimality tolerance for gradient based optimization solvers.
gradientstep	String	The last study step in the list or none, if there is no supported study step.	Name of the study step to use for gradient computation.
useseed	on off	off	Random seed. Available for montecarlo.
randseed	integer	1000	Random seed. Available for montecarlo.
nsolvemax	integer	1000	Maximum number of model evaluations.
lsqdatamethod	manual lsq	manual	Least-squares time/parameter method. Available when there are lsq objectives in the model.
keepsol	auto last all	auto=last	Keep last or all solutions for derivative free optimization solvers.
keepsolgb	last lastn everynth	last	Keep last or all solutions for gradient based optimization solvers.
err	on off	on	Stop if error. Available for derivative free optimization solvers.
nkeepsols	integer	2	Number of solutions to save. Available for gradient based optimization solvers, when keepsolgb=lastn.
nskipols	integer	10	Save every Nth. Available for gradient based optimization solvers, when keepsolgb=everynth.
optobj	String array		User defined objective expressions.
descr	String array		Descriptions for used defined objective expressions.
optobjEvaluateFor	String array		Study to evaluate objective expressions for.
objectivetype	minimization maximization	sum	Determines whether the objective should be minimized or maximized.
multiobjective	sum min max	sum	Multiple objectives. Only sum is supported for gradient based optimization solvers.
objectivesolution	auto first last sum min max	auto	Solution. Handling of multiple solution vectors. Only auto available for IPOPT and LM.
outersolution	sum min max	sum	Outer solution. Available for derivative free optimization solvers, when there are Parametric Sweep study steps under Optimization.
objectivescaling	init manual none	none	Enable/disable objective scaling.
objscaleval	real scalar	1	Objective scale.

TABLE 6-57: GENERAL OPTIMIZATION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pname	String array		Global parameter names to use as control variables.
initval	String array		Initial values for global parameters used as control variables.
scale	String array		Scales for global parameters used as control variables.
lbound	String array		Lower bounds for global parameters used as control variables.
ubound	String array		Upper bounds for global parameters used as control variables.
constraint Expression	String array		Global constraint expressions.
constraintL bound	String array		Lower bounds for global constraint expressions.
constraintU bound	String array		Upper bounds for global constraint expressions.
constrmethod	penalty augLagrange	augLagrange for BOBYQA, penalty for others.	Constraint handling method, not available for COBYLA.
coefrho	auto manual	auto	Strategy for penalty parameter rho. Available when constrmethod=penalty.
rho	real scalar	10	Penalty parameter rho. Available when constrmethod=penalty and coefrho>manual.
tau	real scalar	0.5	Tolerance for the penalty parameter. Available when constrmethod=augLagrange.
gamma	real scalar	10	Multiplication factor for the penalty parameter. Available when constrmethod=augLagrange.
mu	real scalar	Max double value	Upper bound on Lagrange multipliers. Available when constrmethod=augLagrange.
nlagevals	integer	10	Maximum number of augmented iterations. Available when constrmethod=augLagrange.
deltachoice	dynamic dynamic2 static	dynamic	Strategy for the subsolver tolerance.
aCoef	real scalar	2	Tolerance parameter for the subsolver. Available when deltachoice=dynamic2.
delta	real scalar	0.001	Tolerance parameter for the subsolver. Available when deltachoice=static.
enfdesconstr	on off	on	Enforce design constraints strictly. Available when optmethod is equal to neldermead, cobyla, montecarlo, or coordsearch.
plot	on off	off	Enable/disable plot while solving.
plotgroup	String	default	Tag of the plotgroup to plot while solving.
probesel	all none manual	all	The probes to compute while solving.
probes	String array		Probes to use, if probesel>manual.
useobjtable	on off	on	Keep objective values in table.
objtable	String	new	Output table to use, when useobjtable=on.

TABLE 6-57: GENERAL OPTIMIZATION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
showindobj	on off	off	Show individual objective values in table.
plotobj	on off	off	Plot objective while solving.
window	String	new	Window to use, if plotobj=bj=on.
useconstrtable	on off	on	Keep values of constraints in table. Only available with the derivative free optimization solvers.
constrtable	String	new	Table to use, when useconstrtable=on.
pdistrib	on off	off	Distribute parametric sweep. Available when optmethod is equal to coordsearch, neldermead, or montecarlo.
nparallelsolve	integer	1	Maximum number of model evaluations in each parametric sweep, available when pdistrib=on.
convinfo	on off detailed	on	Optimization log.
keepaccumtable	on off	off	Keep accumulated probe table. Available for derivative free optimization solvers.

When the optimization solver is set to IPOPT (ipopt), the following properties are accepted:

TABLE 6-58: VALID PROPERTIES FOR OPTMETHOD IPOPT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
evaluate_orig_obj_at_resto_trial	on off	off	Determines if the original objective function should be evaluated at restoration phase trial points.
expect_infeasible_problem	on off	off	Enable heuristics to quickly detect an infeasible problem.
gradientipopt	analytic forward adjoint forward_numeric	analytic	Gradient/Jacobian evaluation method for IPOPT.
ipopt_acceptable_iter	integer	15	Number of acceptable iterates before triggering termination.
ipopt_acceptable_constr_viol_tol	real scalar	1	Constraint violation acceptance absolute tolerance factor
ipopt_acceptable_compl_inf_tol	real scalar	1	Complementarity conditions acceptance absolute tolerance factor
ipopt_acceptable_tol_fact	real scalar	10	Acceptable convergence tolerance (relative).
ipopt_dual_inf_tol	real scalar	1	Absolute tolerance on the dual infeasibility factor.
ipopt_expect_infeasible_problem_ctol	real scalar	1e-2	Threshold for disabling expect_infeasible_problem option.
ipopt_expect_infeasible_problem_ytol	real scalar	1e8	Multiplier threshold for activating expect_infeasible_problem option.
ipopt_file_print_level	integer	5	Verbosity level for output file.
ipopt_linear_solver	mumps pardiso	mumps	Linear solver used for step computations.
ipopt_mumps_mem_percent	real scalar	1000	Percentage increase in the estimated working space for MUMPS.

TABLE 6-58: VALID PROPERTIES FOR OPTMETHOD IPOPT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ipopt_mumps_pivotol	real scalar	1e-6	Pivot tolerance. Available for MUMPS.
ipopt_mumps_pivotolmax	real scalar	0.1	Maximum pivot tolerance. Available for MUMPS.
ipoptprintdir	String		For log files for IPOPT, specify the directory of the log file. Use the ipoptprintfile to specify the log file.
ipoptprintfile	String	ipoptout.txt	For log files for IPOPT, specify the name of the log file. Use the ipoptprintdir to specify the directory. If no name is specified, the log-file name is ipoptout.txt.
optobj	String		Objective function that is minimized when objcontrib>manual.
start_with_resto	on off	off	Tells algorithm to switch to restoration phase in first iteration.

The property `gradientipopt` is used to control if the gradient should be computed analytically (by solving the adjoint problem) or numerically. If the number of design variables is large, numerical computation of the gradient can be very time consuming. Analytic gradient is only supported when the underlying PDE-problem is stationary.

When the optimization solver is set to Levenberg–Marquardt (1m), the following properties are accepted:

TABLE 6-59: VALID PROPERTIES FOR OPTMETHOD LM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
cendiff	real scalar	6e-6	Central difference interval (only available when <code>gradientlm</code> is numeric and <code>gradorder</code> is second)
difint	real scalar	1.5e-8	Difference interval (only available when <code>gradientlm</code> is numeric and <code>gradorder</code> is first).
gammaf	real scalar	1	Defect reduction tolerance factor.
gammafactive	on off	off	Terminate also for defect reduction.
gammax	real scalar	1	Control variable tolerance factor.
gradientlm	analytic numeric forward adjoint forward_numeric	analytic	Gradient/Jacobian evaluation method.
gradorder	first second	first	Approximation order of the gradient.
lmfact	real scalar	1e-3	Initial Levenberg–Marquardt factor.
lmaxiter	integer	1	Maximum number of outer iterations.
lmaxiteractive	on off	off	Enable maximum outer iterations.
outfsens	on off onpws	on	Store functional sensitivity. <code>onpws</code> is equivalent to <code>on</code> , except the sensitivity is also stored in the solution while solving. This is relevant when using the results while solving functionality.

When the optimization solver is set to MMA, the following properties are accepted:

TABLE 6-60: VALID PROPERTIES FOR OPTMETHOD MMA.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
gradientmma	analytic forward numeric adjoint forward_numeric	analytic	Gradient/Jacobian evaluation method.
mmamaxiter	positive integer	1	Maximum outer iterations.

TABLE 6-60: VALID PROPERTIES FOR OPTMETHOD MMA.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mmamaxinteractive	on off	off	Enable maximum outer iterations.
mmainnmax	positive integer	10	Maximum inner iterations per outer iteration (only for gmma=off).
mmagepsfactor	real scalar	0.1	Internal tolerance factor.
mmacfactor	real scalar	1000	Constraint penalty factor.
mmaghinit	real scalar	0.5	Initial asymptote factor (ghinit).
mmaghdecr	real scalar	0.7	Decreasing asymptote factor (ghdecr).
mmaghincr	real scalar	1.2	Increasing asymptote factor (ghincr).
mmaasymin	real scalar	0.01	Minimum asymptote level (asymmin).
mmaasymax	real scalar	10	Maximum asymptote level (asymax).
mmaalbefa	real scalar	0.1	Bounds asymptote factor (albefa).
mmaxxmove	real scalar	0.5	Bounds control factor (xxmove).
mmaraai	real scalar	0.00001	Approximation increment (raai).
mmaraamin	real scalar	0.000001	Lower approximation bound (raamin).
mmalsq	on off	on	Automatically transform least-squares objectives to constraints for efficiency.
mmaminmax	on off	on	Automatically transform minimax and maximin problems to constraints for efficiency.
movelimit	real scalar	0.1	Limit the maximum absolute change of any scaled control variable between outer iterations.
movelimitactive	on off	off	Enable movelimit.

When the optimization solver is set to EGO, these additional properties are available:

TABLE 6-61: VALID PROPERTIES FOR OPTMETHOD EGO

PROPERTY	VALUE	DEFAULT	DESCRIPTION
acquisitionmaxmethod	direct montecarlo	direct	Optimization method for acquisition function.
covfunction	se matern32 matern52 nn	matern32	Type of covariance function to use in the Gaussian process function. Use se for Squared exponential, matern32 for Matérn 3/2, matern52 for Matérn 5/2 and nn for Single-layer neural network.
globalgpconstrfunction	new Gaussian Process function	new	Constraint Gaussian process function to use.
globalgpfunction	new Gaussian Process function	new	Gaussian process function to use.
gpusage	restart continue	restart	Usage: restart or continue to improve.
maxgpevalsacquisition	positive integer	10000	Maximum surrogate evaluations for optimization, if acquisitionmaxmethod is set to direct.
maxgpitersacquisition	positive integer	500	Maximum number of optimization iterations, if acquisitionmaxmethod is set to direct.
meanfunction	const linear quadratic	const	Defines the overall trend of the trained function(s).

TABLE 6-61: VALID PROPERTIES FOR OPTMETHOD EGO

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ninit sample	positive integer	10	Initial sample per control variable, if gpusage is set to restart.
nmontecarlo	positive integer	10000	Surrogate evaluations for optimization, if acquisitionmaxmethod is set to montecarlo.
restol	double	1e-16	Resolution tolerance for the Gaussian process function.

For a description of the optimization properties, see [Advanced Solver Properties](#) in the *Optimization Module User's Guide*.

TABLE 6-62: REMOVED FEMOPTIM PROPERTIES.

PROPERTY	REASON
Callblevel	Handled by attribute features
Solprop	Handled by stationary or time
Solcomp	Handled by variables
Report	Handled by variables
Out	Solution should be exported

Parametric

Handle parameters for parameter stepping for stationary problems.

SYNTAX

```
model.sol(sname).create(fname,"Stationary")
model.sol(sname).feature(fname).create(parname,"Parametric")
model.sol(sname).feature(fname).feature(parname).set(pname,pvalue)
```

DESCRIPTION

Attribute feature.

TABLE 6-63: PARAMETRIC PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
addsolutions	boolean	true	Add solution data points from the general noncontinuation parameter values lists, if uselsqdata is set to true.
continuationlsqall	boolean	true	Use all for continuation, if uselsqdata is set to true.
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
excludelsqvalues	boolean	true	Create bounding box from general parameter value lists, if uselsqdata is set to true.
initdampall	off on auto	auto	Setting this property to on the initial damping factor is used for all parameter steps.
paramtuning	on off	off	Setting this property to on enables the use of the properties pinitstep, pmaxstep, and pminstep.
pcontinuationmode	no last manual	last	Determines if a continuation sweep should be performed for one of the parameters in pname.
pcontinuation	String		For pcontinuationmode>manual this is one of the parameter names from pname.

TABLE 6-63: PARAMETRIC PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pdistrib	on off	off	If the solver should distribute the parameter sweep.
pinitstep	positive real		Initial step size for parameter. See paramtuning.
plist	real array		List of parameter values. Obsolete, use plistarr instead.
plistarr	real matrix		Lists of parameter values. One row of values for each parameter name.
plooporder	auto manual	auto	Parametric values loop order. If set to auto, a more efficient loop order is used if possible.
pmaxstep	positive real		Maximum step size for parameter. See paramtuning.
pminstep	positive real		Minimum step size for parameter. See paramtuning.
pname	vector of strings		Parameter names.
ponerror	stop skip empty	stop	Determines what the solver does when there is a solver error or when the continuation backtracking fails: Stop, skip the parameter with an error, or store an empty solution.
porder	constant linear	linear	Predictor order for parameter stepping.
pout	plist psteps	plist	When pout is set to plist, output solutions for the parameters in plist. When pout is set to psteps, output solutions at the parameter values chosen by the solver.
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
preusesol	no yes auto	no	Determines how the converged solutions are reused in the parameter sweep.
probefreq	pout psteps	pout	When to update probes.
probes	array of strings		Probes to use when probesel>manual.
probesel	all none manual	all	The probes to compute.
pwork	integer	1	Maximum number of distributed groups.
pworkactive	on off	off	Divide distributed parametric sweep work into distributed groups.
sweepstype	sparse filled	sparse	Method for doing the parameter variation. For sweepstype=sparse, the parameter tuples defined by the columns in plistarr are solved for. This method requires equal length for the rows. For sweepstype=filled, all parameter combinations given by plistarr are solved for.
useloadcase	on off	off	Define load cases.
uselsqdata	boolean	true	Use parameters from least-squares objectives (requires the Optimization Module).

PlugFlow

Solve a stationary plug flow problem.

SYNTAX

```
model.sol(sname).create(fname, "PlugFlow")
```


DESCRIPTION

This solver is a version of the Time-Dependent Solver (see [Time](#)). The difference being that it steps in volume instead of time. The available properties are those given in [Table 6-74](#), where the word *time* in the Description column should be understood as *volume*.

ProperOrthogonalDecomposition

Solve frequency-domain or time-dependent problem using POD (proper orthogonal decomposition) reduction.

SYNTAX

```
model.sol(sname).create(fname,"ProperOrthogonalDecomposition")
model.sol(sname).feature(fname).set(pname,pvalue)
```

DESCRIPTION

Operation feature. The following properties are accepted:

TABLE 6-64: VALID MODAL PROPERTIES FOR THE POD REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
analysistype	frequency transient	transient	Solve for frequency response or transient response.
clist	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .
cname	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use clist. See The clist and cname Properties .
constrmodes	array of integers		The constraint modes to use, if constrmodeselection is set to manual.
constrmodeselection	all manual	all	The constraint modes to use.
constrsol	solution object or none		The solution to use for the constraint modes.
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
groupmethod	one individual manual	one	Variable grouping method for training solution: one group, one group per variable, or manual.
keeplog	on off	off	Keep warnings in stored log.
linearity	false linper	false	Use linear linearity or a linear perturbation when analysistype has been set to frequency.
plist	scalar numeric vector		Frequency list. Only applicable when analysistype has been set to frequency.
pname	vector of strings		Parameter names.
podtrainingsol	String		Solution selection for POD training.
podtrainingsoluse	current manual solution tag	current	POD training solution to use.
podtrainingsolusesolnum	integer		Index of the POD solution to use, when podtrainingsoluse is set to manual.

TABLE 6-64: VALID MODAL PROPERTIES FOR THE POD REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
punit	String	s	Parameter unit. Only applicable when analysistype has been set to frequency.
romdata	String	new	Tag of the target container for the reduced model (new for a new reduced model).
rominterf	stateless stateful	stateless	The reduced-order model interface if romdata is set to new.
romReconstruct	true false	true	Enable reconstruction in the produced reduced model.
rtol	scalar	0.01	Relative tolerance. Only applicable when analysistype has been set to transient.
tlist	scalar numeric vector		Time list. Only applicable when analysistype has been set to transient.
truncationrtol	double	0.01	The relative truncation tolerance.
tunit	String	s	Time unit. Only applicable when analysistype has been set to transient.

Previous Solution

Compute solutions for previous parametric solution parameters and previous time-dependent solution parameters.

SYNTAX

```
model.sol(sname).create(fname, "Stationary")
model.sol(sname).feature(fname).create(parname, "Parametric")
model.sol(sname).feature(fname).feature(parname).create(pname, "PreviousSolution")
model.sol(sname).feature(fname).feature(parname).feature(pname).set(pname, pvalue)
```

```
model.sol(sname).create(fname, "Time");
model.sol(sname).feature(fname).create(pname, "PreviousSolution");
model.sol(sname).feature(fname).feature(pname).set(pname, pvalue);
```

DESCRIPTION

Attribute feature. After the solver has converged for a parameter step or a time step, the previous components are solved for in a separate solver step. These components are held fixed (not solved for) during the normal solver procedure.

Segregated

Handle the segregated solution approach.

SYNTAX

```
model.sol(sname).feature(solv).create(fname, "Segregated")
model.sol(sname).feature(solv).feature(fname).set(pname, value)
model.sol(sname).feature(solv).feature(fname).feature(fname2).set(pname, value)
```

DESCRIPTION

This feature can be used as an attribute for the Time and Stationary features. The approach taken is nonlinear Uzawa iterations in which user-defined groups of variables are solved for separately (a segregated step) while other

variables are held fixed. The segregated steps for the segregated solver is handled by subattributes of the sort of SegregatedStep, LumpedStep, LowerLimit, and UpperLimit.

The Segregated attribute supports the following properties (of which the ones that start with sub are properties for segregated steps);

TABLE 6-65: VALID SEGREGATED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
consistencycheck	none basic strictinitially strictalways	strictinitially	Depth of constraint consistency check.
maxsegiter	positive integer	100, 25 (Time)	Maximum number of segregated iterations.
ntolfact	positive scalar	1	Tolerance factor.
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probesel	all none manual	all	The probes to compute.
probes	array of strings		Probes to use when probesel>manual.
ratelimit	positive scalar	1 (Time)	Limit on nonlinear convergence rate.
ratelimitactive	on off	off, on (Time)	Use limit on nonlinear convergence rate.
segaaccdim	positive integer	10	Dimension of Anderson iteration space when segstabacc = segaacc.
segiter	positive integer	1	Fixed number of segregated iterations.
segreserrfact	positive scalar	1000	Residual factor when segtermmonres is set to auto or both (for stationary solvers).
segstabacc	none segcflcmp segaacc	none	Stabilization and acceleration: None, pseudo time stepping (for stationary solvers), or Anderson acceleration.
segterm	iter tol itertol	tol	Segregated solver termination technique.
segtermmonres	off on auto both	auto/off	Termination criterion: Solution, residual, solution or residual, or solution and residual. The default is auto for stationary problems but off for time-dependent problems.
segaaccdelay	nonnegative integer	0	Number of iterations between pseudo time stepping becomes inactive and Anderson acceleration becomes active when segstabacc = segaacc.
segaaccdim	positive integer	10	Dimension of Anderson iteration space when segstabacc = segaacc.
segaaccmix	scalar 0–1	1	Mixing parameter when segstabacc = segaacc.
segcflaadelay	nonnegative integer	0	Number of iterations between pseudo time stepping becomes inactive and Anderson acceleration becomes active when segstabacc = segcflcmp.
segcflaafact	nonnegative scalar	1	Threshold for Anderson step for pseudo time stepping.
segcflaacfl	positive scalar	100	CFL threshold when segstabacc = segcflcmp.
segcflaadim	positive integer	10	Dimension of Anderson iteration space when segstabacc = segcflcmp.

TABLE 6-65: VALID SEGREGATED PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
segcflaamix	scalar 0–1	1	Mixing parameter when segstabacc = segcflcmp.
segcflaaset	pid lock	pid	Above the CFL threshold, use the PID controller or lock to the target CFL number.
segcfljtech	true false	false	Override Jacobian update for step when segstabacc = segcflcmp.
segcfljtechval	onfirst minimal	onfirst	Jacobian update on first iteration or minimal when segstabacc = segcflcmp and segcfljtech = true.
segjtechcfl	positive scalar	100	CFL threshold for Jacobian update when segstabacc = segcflcmp and segcfljtech = true.
subadaptcfltol	boolean	true	Use an adaptive target error estimate for pseudo time stepping.
subcfltol	positive scalar	0.1	Target error estimate for pseudo time stepping.
subforcecfl	boolean	true	Limit to target CFL number for pseudo time stepping.
subinitcfl	positive scalar	5.0	Initial CFL number for pseudo time stepping.
subkdpid	positive scalar	0.05	PID controller - derivative for pseudo time stepping.
subkipid	positive scalar	0.05	PID controller - integral for pseudo time stepping.
subkppid	positive scalar	0.65	PID controller - proportional for pseudo time stepping.
submincfl	positive scalar	10000	Target CFD number for pseudo time stepping.
useheuristicfact	boolean	false (Time)	Use pre-6.2 linear heuristic for adaptive tolerance.

Termination of the segregated solver is controlled by the property `segterm`. The default setting is `tol`, in which case the segregated iterations are terminated when, for each group, the estimated error is below the corresponding tolerance set by the main tolerance for the parent solver multiplied with the nonlinear tolerance factor `ntolfact`. However, a maximum number of allowed segregated iterations is chosen through the property `maxsegiter`; if the maximum is reached, the iterations are terminated and an error message is displayed. Termination after a fixed number of segregated iterations is achieved by instead choosing `iter`. The number of segregated iterations is controlled by the property `segiter`. The third available option for `segterm` is `itertol`, which is a combination of the other two options; the segregated iterations are terminated when one of the two convergence criteria of `tol` and `iter` is met. The property `maxsegiter` is only supported when `tol` is used for termination. For both the settings `iter` and `itertol`, the number of iterations is controlled by the property `segiter`.

The nonlinear solver uses an adaptive tolerance for termination of iterative linear system solvers. This adaptive tolerance is based on the maximum of `ntol` and `itol`. During the nonlinear iterations, it can, however, be larger or smaller than this number. The segregated solver uses the same tolerance as the linear solver when constant damping is used. However, when automatically adjusted damping is used, the adaptive tolerance of the nonlinear solver is used. The parametric solver uses the same tolerance as the corresponding stationary solver.

The property `segstabacc` enables or disables pseudo time stepping (for stationary problems) or Anderson acceleration. When enabled the pseudo time stepping is controlled by the scalar-valued controller parameters `subcfltol`, `subinitcfl`, `subkdpid`, `subkipid`, and `subkppid`. For the Anderson acceleration, the parameter `segaaccdim` specifies the dimension of the Anderson iteration space.

The property `segtermmonres` controls the termination criterion for stationary problems when `segterm` is not `iter`. When `segtermmonres=off` the estimated error is solution based, with `segtermmonres=on` it is based on a relative residual and for `segtermmonres=auto` the estimated error is the minimum of the solution and residual based errors. For `segtermmonres=auto` the property `segreserrfact` is a scalar factor multiplying the relative residual error. For `subtermmonres=both`, both methods are used.

COMPATIBILITY

The property `subsecflcmp` from earlier versions of COMSOL Multiphysics is not used in version 5.0 and later versions. Use the property `segstabacc` instead.

SEE ALSO

[LowerLimit](#), [LumpedStep](#), [SegregatedStep](#), [UpperLimit](#)

SegregatedStep

Handle a segregated solution step, which can be added as a subfeature to a Segregated feature.

SYNTAX

```
model.sol(sname).feature(solv).feature(segregulated).create(fname, "SegregatedStep")
model.sol(sname).feature(solv).feature(segregulated).feature(fname).set(pname, value)
model.sol(sname).feature(solv).feature(segregulated).feature(fname).feature(sname).
    set(pname, value)
```

DESCRIPTION

This feature controls one segregated solution step.

TABLE 6-66: VALID SEGREGATED STEP PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>linsolver</code>	linear solver tag		Linear solver to use.
<code>maxsubiter</code>	integer	20	Maximum number of substep iterations.
<code>segcomp</code>	vector of strings		Field/State components in step if <code>segvarspec=manual</code> .
<code>segvar</code>	vector of strings		Fields/States in step.
<code>segvarspec</code>	<code>all</code> <code>manual</code>	<code>all</code>	Include all components or specify which manually.
<code>subaacc</code>	boolean	<code>false</code>	Use Anderson acceleration, when <code>subdtech</code> is set to <code>const</code> or <code>bcktrack</code> .
<code>subaccdelay</code>	nonnegative integer	0	Iteration delay for Anderson acceleration.
<code>subaaccdim</code>	positive integer	10	Dimension of iteration space for Anderson acceleration.
<code>subaaccfact</code>	nonnegative scalar	1	Threshold for Anderson step.
<code>subaaccmix</code>	scalar (0–1)	1	Mixing parameter for Anderson acceleration.
<code>subadapttol</code>	boolean	<code>false</code>	Use adaptive tolerance in the linear solver, when <code>subdtech</code> is set to <code>const</code> or <code>bcktrack</code> .
<code>subadapttolcflweight</code>	boolean	<code>true</code>	Apply CFL weighting for pseudo time stepping and if <code>subadapttol</code> is set to <code>true</code> .

TABLE 6-66: VALID SEGREGATED STEP PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
subadapttolerrweight	boolean	true	Apply nonlinear error weighting for models that do not use pseudo time stepping and if subadapttol is set to true.
subatolchoice	fullerrorest partialerrorest nonlinresidual onlycfl onlyerr	fullerrorest	The error estimation method to use if subadapttol is set to true. fullerrorest uses the full information about the linear and nonlinear error estimates to determine the tolerance. partialerrorest uses partial information about the linear and nonlinear error estimates to determine the tolerance. nonlinresidual uses the nonlinear residual only to determine the tolerance. onlycfl applies CFL suppression to the initial tolerance for models using pseudo time stepping. onlyerr applies a suppression mechanism based on the nonlinear error estimate for models with a stationary solver and not using pseudo time stepping.
subatolthreshold	positive value, 0–1	0.1	The threshold for using the safeguards when determining the tolerance. The higher the value, the faster the safeguards get disabled. Only available when subadapttol is set to true.
subbackdampstep	scalar (0–1)	0.5	Damping in one step, when subdtype is set to bcktrack and subbackmethod is set to conststep.
subbackmethod	fullest conststep	fullest	Backtracking method when subdtype is set to bcktrack: full estimation or constant step.
subbacktrackonce	boolean	false	Backtrack at least once when subdtype is set to bcktrack and subbackmethod is set to fullest.
subdamp	real	1.0	Substep damping factor.
subdampexponent	real scalar (1–2)	2	Damping exponent, if subatolchoice is set to nonlinresidual.
subdampfactor	real scalar (0–1)	1	Damping factor, if subatolchoice is set to nonlinresidual.
subddoginitdamp	nonnegative scalar	1	Initial damping factor for subdtype set to ddog.
subddogrestart	positive integer	7	Number of iterations before restart for subdtype set to ddog.
subdtype	const auto bcktrack hnlin ddog	const	Substep damping technique.
subetamax	scalar (0–1)	0.9	Maximal tolerance, if subadapttol is set to true.
subinitstep	real	1	Substep initial damping factor for when subdtype set to auto.

TABLE 6-66: VALID SEGREGATED STEP PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
subinitsteph	real	1e-4	Substep initial damping factor when subdtype set to hnlin.
subiter	integer	1	Substep iterations.
subjtech	minimal once onfirst onevery	see below	Substep Jacobian update technique when subdtype is set to const or bcktrack.
submatrixformat	auto sparse filled free	auto	Matrix format: automatic, sparse, filled, or free.
submaxdamp	scalar (0, 1]	1.0	The maximum damping factor. If backtracking returns a damping larger than this value, submaxdamp is used. Available only if subdtype is set to bcktrack.
submindamp	positive scalar	0.1	Substep minimum damping factor when subdtype is set to bcktrack.
subminstep	real	1e-4	Substep minimum damping factor when subdtype is set to auto.
subminsteph	real	1e-8	Substep minimum damping factor when subdtype is set to hnlin.
subntolfact	real	10	Substep tolerance factor.
subrelaxationressmooth	positive scalar	0.2	Relaxation factor, if subressmooth is set to true.
subreserrfact	positive scalar	100	Residual factor when subtermmonres is set to auto or both.
subressscale	scalefieldwise scaleuniform	scalefieldwise	Residual scaling technique for subdtype set to ddog.
subresssmooth	boolean	false	Add residual smoothing.
subrstep	real	10	Substep restrictions for step-size update.
subtermconst	iter tol itertol	iter	Substep termination technique when subdtype is set to const or bcktrack.
subtermauto	tol itertol	itertol	Substep termination technique when subdtype is set to auto or hnlin.
subtermmonres	off on auto both	auto/off	Termination criterion: Solution, residual, solution or residual, or solution and residual. The default is auto for stationary problems but off for time-dependent problems.
subupdweights	boolean	true	Update automatic scale factors in weights when subdtype is set to const or bcktrack.
subupdweightsauto	true false wthresh	true	Updates the weights for automatic scales if they change two orders of magnitude and then restart the Newton solver from the current solution if true. If set to wthresh, you can use thresholds for the weights. All updweights properties are available for stationary and parametric studies and for Newton solvers only (that is, subdtype is not ddog, bcktrack, or const).

TABLE 6-66: VALID SEGREGATED STEP PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
subupdweightsdamp	current manual	current	Available if subupdweightsauto is wthresh. Use fraction of the current damping factor at update (current) or a constant update damping factor (manual).
subupdweightsdampval	positive scalar	1	Fraction of current damping factor. If subupdweightsauto is wthresh and subupdweightsdamp is current.
subupdweightsdampvalconst	positive scalar	0.1	Update damping factor. If subupdweightsauto is wthresh and subupdweightsdamp is manual.
subupdweightsfac	scalar	100	Weights threshold factor for update. If subupdweightsauto is wthresh.

The fields/states to include in the step are defined through the property `segvar`. The property `segvarspec` controls which components of the fields/states in `segvar` to include in the step. By default `segvarspec` is `all`, in which case all components in the fields/states of `segvar` are included. By setting `segvarspec` to `manual`, a subset of the fields/states of `segvar` can be included in the step. The components to include in the step are then defined through the property `segcomp`.

Analogously, the property `subterm` controls how each substep is terminated through the properties `maxsubiter`, `subiter`, and `subntol/subntolfact` for a stationary or time-dependent problem.

The damping technique used in each substep is controlled by the property `subdtech`. The default setting is `const`, which means that damped Newton iterations with a fixed damping factor is used. The damping factor is set in the property `subdamp`. The other available damping technique is `autodamp` in which case the damping factor is automatically adjusted. For substeps which uses `autodamp`, four other properties are supported: `subhnl`, `subinitstep`, `subminstep`, and `subrstep`. For each substep, these properties set the properties `hnl`, `initstep`, `minstep`, and `rstep` supported by the nonlinear solver, see `FullyCoupled`.

In substeps with `subdtech=const` or `bcktrack`, the property `subjtech` controls how often the Jacobian is updated. The values `minimal`, `once`, and `onevery` give the same Jacobian update techniques as they do when applied to the coupled solver through the property `jtech`; see `FullyCoupled`. The value `onfirst` makes the solver update the Jacobian of the substep on the first subiteration each time the substep is solved for. Default value is `onevery` for stationary problems and `minimal` for time-dependent problems.

When `subdtech` is set to `ddog` (stationary problems), the double dogleg solver is used. The initial damping factor is controlled by the property `subddoginitdamp` and the property `subresscale` controls the residual scaling. The option `resscale=scalefieldwise` scales the equations based on the field-wise sizes of the initial residual. When the option `subresscale=scaleuniform` is selected the algorithm terminates on the relative residual based on the initial residual.

The property `subtermmonres` controls the termination criterion for stationary problems when `segterm` is not `iter`. When `subtermmonres=off` the estimated error is solution based, with `subtermmonres=on` it is based on a relative residual, and for `subtermmonres=auto` the estimated error is the minimum of the solution and residual based errors. For `subtermmonres=auto` the property `subreserrfact` is a scalar factor multiplying the relative residual error. For `subtermmonres=both`, both methods are used.

Sensitivity

Handle sensitivity solver parameters.

SYNTAX

```
model.sol(sname).feature(solv).create(fname, "Sensitivity")  
model.sol(sname).feature(solv).feature(fname).set(pname, value)
```

DESCRIPTION

Attribute feature. This feature can be used to make analytic forward or backward (adjoint) sensitivity analysis. This analysis is done after the main problem has converged. The solution approach (coupled or segregated, Jacobians, and so on) for the main problem is reused.

TABLE 6-67: VALID SENSITIVITY PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sensfunc	String		Sensitivity functional variable name
sensmethod	none adjoint forward numeric	none	Sensitivity analysis method

The forward numeric method (`numeric`) is a variant of the forward sensitivity method where the right-hand side of the sensitivity problem is computed by numerical differentiation.

StatAcceleration

Handle stationary acceleration for nonlinear problems with time-periodic stationary solution.

SYNTAX

```
model.sol(sname).feature(solv).create(fname, "StatAcceleration")
```

DESCRIPTION

This feature can be added as a subfeature to the Time-Dependent Solver and the Time Discrete Solver. Instead of time marching the problem from start to finish, the Stationary Acceleration node solves for a number of periods and then extrapolates the solution forward in time based on the average solution and the average time derivative. This solution process is repeated until the average time derivative has reached steady state.

TABLE 6-68: VALID PROPERTIES FOR STATACCELERATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
acccomp	vector of strings		Variable components to use stationary acceleration on if <code>accvarspec=manual</code> .
accvar	vector of strings		Variables to use stationary acceleration on.
accvarspec	all manual	all	Include all components or specify which manually.
stataccfreq	String	13.56[MHz]	Frequency of periodic solution.
stataccctol	double	0.01	Stationary tolerance.
stataccnperext	integer	50	Number of extrapolation cycles.
stataccnperavg	integer	5	Number of period averaging cycles.
stataccsmooth	integer	10	Number of smoothing cycles.

StateSpace

Assemble and store matrices that describe a model as a dynamic system.

SYNTAX

```

model.sol(sname).create(fname, "StateSpace")
model.sol(sname).feature(fname).set(pname, value)
model.sol(sname).feature(fname).getSparseMatrixVal(mname)
model.sol(sname).feature(fname).getSparseMatrixValImag(mname)
model.sol(sname).feature(fname).getSparseMatrixRow(mname)
model.sol(sname).feature(fname).getSparseMatrixCol(mname)
model.sol(sname).feature(fname).getVector(vname)
model.sol(sname).feature(fname).getVectorImag(vname)
model.sol(sname).feature(fname).getSparseMatrixValBlock(mname, start, stop)
model.sol(sname).feature(fname).getSparseMatrixValImagBlock(mname, start, stop)
model.sol(sname).feature(fname).getSparseMatrixRowBlock(mname, start, stop)
model.sol(sname).feature(fname).getSparseMatrixColBlock(mname, start, stop)
model.sol(sname).feature(fname).getVectorBlock(vname, start, stop)
model.sol(sname).feature(fname).getVectorImagBlock(vname, start, stop)
model.sol(sname).feature(fname).isReal(mname)
model.sol(sname).feature(fname).getM(mname)
model.sol(sname).feature(fname).getN(mname)

```

DESCRIPTION

State-space feature.

TABLE 6-69: VALID STATESPACE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
input	String array		The input parameters that affect the model.
output	String array		The output expressions.
static	on off	on	Static linearized model.
Mc	on off	off	Assemble the Mc matrix.
MA	on off	off	Assemble the McA matrix.
MB	on off	off	Assemble the McB matrix.
C	on off	off	Assemble the C matrix.
D	on off	off	Assemble the D matrix.
Null	on off	off	Assemble the Null matrix.
ud	on off	off	Assemble the ud vector.
x0	on off	off	Assemble the initial data.

The state-space feature assembles matrices that describe a model as a dynamic system when **Static** is **off**.

$$\begin{aligned}
 Mc\dot{x} &= McAx + McBu \\
 y &= Cx + Du
 \end{aligned}$$

In the case when **Static** is **on** a static linearized model of the system is described by

$$y = (D - C(McA)^{-1}McB)u$$

Let *Null* be the PDE constraint null-space matrix and *ud* a particular solution fulfilling the constraints. The solution vector *U* for the problem can then be written

$$U = \text{Null}x + ud + u0$$

where *u0* is the linearization point, which is determined by the current solution (that is, the solution computed by the previous feature in the sequence). The previous feature can, for example, be a solver or a Dependent Variable node. The Dependent Variable node gives control over which variables to solve for (compute the matrices for). The input linearization point is stored in the sequence after the state-space feature is run.

The input parameters `input` should contain all parameters that are of interest as input to the model. The output expressions `output` should contain a list of all expressions that are to be evaluated as output from the model.

Stationary

Solve a stationary problem with or without parameters, mesh adaptation, sensitivity, or optimization.

SYNTAX

```
model.sol(sname).create(fname, "Stationary")
model.sol(sname).feature(fname).set(pname, value)
```

DESCRIPTION

Operation feature.

The following properties are available.

TABLE 6-70: STATIONARY PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>clist</code>	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use <code>cname</code> . See The <code>clist</code> and <code>cname</code> Properties .
<code>cname</code>	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use <code>clist</code> . See The <code>clist</code> and <code>cname</code> Properties .
<code>control</code>	String	<code>user</code>	Name of the controlling study step or user if the feature is controlled manually.
<code>keeplog</code>	<code>on</code> <code>off</code>	<code>off</code>	Keep warnings in stored log.
<code>keepnotsolstatic</code>	<code>on</code> <code>off</code>	<code>off</code>	Used in Fatigue analysis to store all solnums from the source solution in the Fatigue solution object (default value is <code>on</code> for Fatigue analysis).
<code>linplistsolnum</code>	Integer array	<code>new int[] {1}</code>	List of solutions to use for the linearization point, when <code>linpsolnum</code> is set to <code>from_list</code> .
<code>linpmanualsolnum</code>	Integer array	<code>new int[] {1}</code>	Solution number (index) to use for the linearization point, when <code>linpsolnum</code> is set to <code>manual</code> .
<code>linpmethod</code>	<code>init</code> <code>sol</code>	<code>init</code>	Method used for linearization point, which for <code>sol</code> is given by the <code>linpsol</code> property.
<code>linpsol</code>	<code>zero</code> solution object	<code>zero</code>	Linearization point solution.
<code>linpsolnum</code>	<code>all</code> <code>auto</code> <code>first</code> <code>last</code> <code>from_list</code> <code>interp</code> <code>manual</code> solution number	<code>auto</code>	Indices to solutions to use as linearization points. The default value from a default solver is case dependent.
<code>linpsoluse</code>	<code>current</code> <code>manual</code> solution store tag	<code>current</code>	Linearization point solution to use. The solution store can be all solution objects that can be found in the solver sequence set by the <code>linpsol</code> property.
<code>linpsolusesolnum</code>	integer		Index for the solution to use when <code>linpsoluse</code> is set to <code>manual</code> .
<code>linpt</code>	double	<code>0</code>	Interpolated point for the linearization point when <code>linpsolnum</code> is set to <code>interp</code> .

TABLE 6-70: STATIONARY PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
listsolnum	vector of integers	{1}	Indices to solutions to use as linearization points when linpsolnum is set to from_list.
lumpedflux	on off	off	Use lumping when computing fluxes.
manualsolnum	vector of positive integers		Identifies the solutions used when linpsolnum is set to manual.
nonlin	auto on off linper	auto	Use the nonlinear solver.
message	String		The log message from the last solution process.
outsollinear	du u	u	Store the total solution (u) or deviation and linearization point (du), when nonlin is set to off and storelinpoint is set to off.
outsollinearized	du u	du	Store the total solution (u) or deviation and linearization point (du), when nonlin is set to linper and storelinpoint is set to off.
plot	on off	off	Whether to plot while solving.
plotgroup	String	default	Name of plot group for plot while solving.
probes	vector of strings		Probes to use if probesel=manual.
probesel	all none manual	all	The probes to compute.
reactf	on off	on	Compute reaction forces.
stol	positive real	1e-3	Relative tolerance, if control is set to user.
storelinpoint	on off	off	Whether to store the linearization point.
t	real	0	Interpolation time for linearization point from another solution, when linpsolnum is set to interp.

This solver uses a nonlinear solver if **nonlin** is **on**, and it uses the linear solver if **nonlin** is **off** or **linper**. If **nonlin** is set to **auto** an analysis is performed to automatically detect if the problem can be solved with a linear solver approach.

The automatic nonlinear/linear detection works in the following way. The linear solver is called if the residual Jacobian matrix (the stiffness matrix, K) and the constraint Jacobian matrix (the constraint matrix, N) are both found not solution dependent and if these matrices are detected as complete. In all other situations the nonlinear solver is used. The analysis is performed by a symbolic analysis of the expressions contributing to these matrices. Complete here means that in the residual and constraint vectors, only expressions were found for which COMSOL Multiphysics computes the correct Jacobian contribution.

Therefore, if you want to solve a linearized (nonlinear) problem, you must set **nonlin** to **off** or **linper**. The **off** option uses the linearization point for both the residual computation and for the Jacobian and the solution to the linear problem is added to the linearization point. This corresponds to one step in the Newton method. For **linper**, the linearization point is used for the Jacobian, the zero solution is used for the assembly of the residual and the solution to the linear problem is returned as the solution. Furthermore, the residual assembled for **linper** is computed using loads marked with the **linper** operator.

There are variables for which COMSOL Multiphysics is conservative and therefore flags these, and their Jacobian contribution, as solution dependent even though they not always are. For these situations, the nonlinear solver is used even though the linear solver could be used. This should only result in some extra computational effort, and should not influence the result. The opposite situation however, where the linear solver is used for a nonlinear problem is more dangerous. So, select **nonlin** to **off** with great care.

The property `reactf` controls the computation and storage of constraint reaction forces. The value `reactf=on` (default) means that the solver stores the FEM residual vector L in the solution object `model.sol`. Because $L = N_F \Lambda$ for a converged solution, the residual is the same as the constraint force. Only the components of L that correspond to nonzero rows of N_F are stored. The value `reactf=off` gives no computation or storage of the reaction force and saves some memory.

The linear solver uses the property `itol` for termination of iterative linear system solvers and for error checking for direct solvers (if enabled).

StopCondition

Handle stop conditions for time-dependent, time explicit, and parametric solver processes.

SYNTAX

```
model.sol(sname).create(fname,"Stationary")
model.sol(sname).feature(fname).create(pname,Parametric)
model.sol(sname).feature(fname).feature(pname).create(ocname,StopCondition)
model.sol(sname).feature(fname).feature(pname).feature(ocname).setIndex(pname,pvalue,ix)
```

DESCRIPTION

Attribute feature. Use the `StopCondition` feature to make sure the solver stops when a specified condition is fulfilled. When you provide a scalar expression, then the expression is evaluated after each time or parameter step. The stepping is stopped if the real part of the expression is evaluated to something negative. The corresponding solution, for which the expression is negative, is not returned. When you provide an integer, the solver stops when the corresponding implicit event is triggered. Use `setIndex` to set the stop condition properties for multiple stop conditions. For example,

```
model.sol("sol1").feature("t1").feature("st1").setIndex("stopcondarr", "(1/timestep)<200", 1);
```

specifies the second stop condition to be $(1/\text{timestep}) < 200$.

TABLE 6-71: VALID STOPCONDITION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>stopcondActive</code>	Vector with entries "on" and "off"		Vector describing which stop condition expression that are active.
<code>stopcondarr</code>	Vector of strings		Stop condition expressions.
<code>stopconddesc</code>	Vector of strings		Descriptions for stop condition expressions.
<code>stopcondition</code>	String		Stop condition expression (deprecated).
<code>stopcondterminateon</code>	Vector with entries "true" and "negative".		For "true" ("negative"), entries, the solver stops if the associated <code>stopcondarr</code> entry satisfies ≥ 1 (< 0).

StoreSolution

A placeholder for a solver sequence that is used to store a computed solution.

SYNTAX

```
model.sol(sname).create(fname,"StoreSolution")
model.sol(sname).feature(fname).getString("sol")
```

DESCRIPTION

The store solution feature stores a reference to a computed solution. Use the `sol` property to find out the name of the referenced solution.

TABLE 6-72: VALID STORESOLUTION PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>sol</code>	String	A solver sequence	Name of solver sequence that stores the solution.

In the case of a parametric sweep you can use the `StoreSolution` features to find the solutions created during the sweep. You first find out the solver sequence that holds the stored solutions

```
model.batch(pname).feature(fname).getString('psol')
```

where `pname` is the name of the parametric sweep feature that ran and `fname` is the name of the solution feature that stored the solutions. Use

```
model.sol(sname).feature().tags()
```

to find out the tags of the stored solutions. Use

```
model.sol(sname).feature(fname).getString('sol')
```

to find the solver sequence for a parameter. Use

```
model.sol(sname).getParamNames()
```

and

```
model.sol(sname).getParamVals()
```

to find the parameter values that created the solution object.

StudyStep

Specify which problem to use for subsequent solver operations.

SYNTAX

```
model.sol(sname).create(fname, "StudyStep")
model.sol(sname).feature(fname).set(pname, pvalue)
```

DESCRIPTION

Utility feature. This feature determines which problem to use for subsequent solver operations and appears as **Compile Equations** in the COMSOL Desktop. It contains a reference to a study and a reference to a study step within that study. When run, the corresponding low-level equation representation is compiled.

TABLE 6-73: VALID CONFIGURATION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>entset</code>	List of tags of named selections	{}	Selections whose geometric entities should be used. Used if <code>useent</code> is set to selected.
<code>keeplog</code>	boolean	false	Keep warnings in stored log.
<code>linsmallelem</code>	physicscontrolled usercontrolled false	physicscontrolled	Linearize small mesh elements: physics controlled, user controlled, of off.
<code>linsmallelemexpr</code>	double		Maximum size of mesh elements for linearization, when <code>linsmallelem</code> is set to usercontrolled. Specified using <code>setEntry</code> for each geometry in the model.

TABLE 6-73: VALID CONFIGURATION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
partmethod	parent mo nd wnd false	parent	Partitioning method for distributed computing: from parent, mesh ordering, nested dissection, weighted nested dissection, or off.
study	String		Name of study to use.
studystep	String		Name of study step to use.
splitcomplex	boolean	false	Represent complex variables by separate degrees of freedom for real and imaginary parts.
useent	all selected	all	Use all geometric entities or selected entities.

Time

Solve a time-dependent problem using implicit or explicit time stepping.

SYNTAX

```
model.sol(sname).create(fname,"Time")
model.sol(sname).feature(fname).set(pname,value)
```

DESCRIPTION

Operation feature.

The time interval and possible intermediate time values are given in the property Tlist. The output times are controlled by the property Tout.

The feature Time accepts the following properties and values:

TABLE 6-74: VALID PROPERTIES FOR TIME.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
algebraicsolveeverydt	positive double	0	Value of the period Δt , when algebraicsolvemethod is set to dt for Adams–Bashforth methods.
algebraicsolveeverydtRK	positive double	0	Value of the period Δt , when algebraicsolvemethod is set to dt for Runge–Kutta methods.
algebraicsolveeveryn	positive integer	1	Value of N , when algebraicsolvemethod is set to n for Adams–Bashforth methods.
algebraicsolveeverynRK	positive integer	1	Value of N , when algebraicsolvemethod is set to n for Runge–Kutta methods.
algebraicsolvemethod	n dt	n	Strategy used to solve the algebraic equations: every N th step or periodically for Adams–Bashforth methods.
algebraicsolvemethodRK	n dt	n	Strategy used to solve the algebraic equations: every N th step or periodically for Runge–Kutta methods.
assumeperiodic	boolean	false	Assume periodic values of variables not solved for.
atol	String	empty	Absolute tolerance per field. See below.

TABLE 6-74: VALID PROPERTIES FOR TIME.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
atolmethod	String	empty	How to interpret the atolfields value. See below.
atolglobal	positive scalar	1e-3	Global absolute tolerance, if atolglobalvaluemethod is manual.
atolglobalfactor	positive scalar	0.1	Global absolute tolerance as a factor of the relative tolerance, if atolglobalvaluemethod is factor.
atolglobalmethod	scaled unscaled	scaled	How to interpret the atolglobal value.
atolglobalvaluemethod	factor manual	factor	Use a factor of the relative tolerance or a user-defined value for the absolute tolerance.
atoludot	String	empty	Absolute tolerance for time derivatives per field. Only applicable if atoludotactive is on. See below.
atoludotactive	String	empty	Used to activate manual specification of absolute tolerance for time derivatives. See below.
bdforder	1–5	2	BDF order for manual BDF settings.
bwinitstepfrac	positive scalar	0.001	Fraction of initial step, when consistent is set to bweuler.
bwinitfactor	positive scalar	20	Safety factor used in the algebraic equation termination, when consistent is set to bweuler.
checkvalidity	true false	false	Check validity of coupled system, if odesolvertype is set to explicit.
complex	true false	false	Allow complex numbers.
consistent	off on bweuler	bweuler	Consistent initialization of DAE systems.
clist	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .
cname	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use clist. See The clist and cname Properties .
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
doprigrowmax	positive scalar	10	Maximum step size growth ratio for Dormand–Prince 5.
doprigrowmin	positive scalar	0.2	Minimum step size growth ratio for Dormand–Prince 5.
dopripicontrol	smooth quick disabled	smooth	Control behavior of the proportional-integral controller that adds damping on step size changes for Dormand–Prince 5.
doprisafe	positive scalar	0.9	Step size safety factor for Dormand–Prince 5.

TABLE 6-74: VALID PROPERTIES FOR TIME.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
endtimeinterpolation	boolean	true	Interpolate the last time specified in the list of output times, if true. If set to false, the last output time is solved and not interpolated. In particular, the solver does not take steps past the last time.
erkorder	integer, 1–4	4	Classical Runge–Kutta order.
estrat	include exclude	include	Error estimation strategy.
exprs	String		Expression for time stepping when <code>tstepping=elemexprs</code> .
exprsgeom	String array with model domain boundary edge point on each level	domain on all levels	The expression in <code>exprs</code> is evaluated for every mesh element in the geometric entity level specified here, when <code>timemethodexp</code> is <code>erk</code> , <code>ab3</code> , or <code>verlet</code> and <code>tstepping=elemexprs</code> .
exprsgeomab3loc	String array with domain on each level	domain on all levels	The expression in <code>exprs</code> is evaluated for every mesh element in the geometric entity level specified here, when <code>timemethodexp</code> is <code>ab3loc</code> .
exprsgporder	Integer array	2 on all levels	The Gauss-point order when <code>timemethodexp</code> is <code>erk</code> , <code>ab3</code> , or <code>verlet</code> ; <code>tstepping=elemexprs</code> ; and <code>updtstep=false</code> .
exprsgporderab3loc	Integer array	4 on all levels	The Gauss-point order when <code>timemethodexp</code> is <code>ab3loc</code> .
exprsgporderupdt	Integer array	0 on all levels	The Gauss-point order when <code>timemethodexp</code> is <code>erk</code> , <code>ab3</code> , or <code>verlet</code> ; <code>tstepping=elemexprs</code> ; and <code>updtstep>manual</code> .
eventtol	positive scalar	0.01	Event tolerance used for root finding of event conditions when using implicit events for BDF.
ewtrescale	on off	on	Update scaled absolute tolerance for BDF.
geometricNonlinearity	on off	off	Include geometric nonlinearity. Available if the license includes the Structural Mechanics Module, Acoustics Module, MEMS Module, or Multibody Dynamics Module.
incrdelay	positive integer	15	Number of time steps to delay a time step increase.
incrdelayactive	on off	off	Use delay in time step increase.
initialstepbdf	positive scalar	1e-3	Initial time step for BDF.
initialstepbdfactive	on off	off	Use an initial time step for BDF.
initialstepck5	positive scalar	1e-3	Initial time step for Cash–Karp 5.
initialstepdopri5	positive scalar	1e-3	Initial time step for Dormand–Prince 5.
initialsteprk34	positive scalar	1e-3	Initial time step for the RK34 method.
initialstepck5active	on off	off	Use an initial time step for Cash–Karp 5.
initialstepdopri5active	on off	off	Use an initial time step for Dormand–Prince 5.

TABLE 6-74: VALID PROPERTIES FOR TIME.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
initialsteprk34active	on off	off	Use an initial time step for RK34.
initialstepgenalpha	positive scalar	1e-3	Initial time step for generalized alpha.
initialstepgenalphaactive	on off	off	Use an initial time step for generalized alpha.
initialstepfractionbdf- <i>i</i>	positive scalar		The fraction of the time step for the initial step of a manual time stepping for BDF. The name and the default depend on the BDF order; for example, initialstepfractionbdf-2 for BDF order 2.
initialstepgrowthratebdf- <i>i</i>	positive scalar		The growth rate for the initial steps of a manual time stepping for BDF. The name and the default depend on the BDF order; for example, initialstepgrowthratebdf-2 for BDF order 2.
keeplog	boolean	false	Keep warnings in stored log.
lumpedflux	on off	off	Use lumping when computing fluxes.
masssingular	yes maybe	maybe	Singular mass matrix.
maxorder	integer between 1 and 5	5	Maximum BDF order.
maxstepbdf	positive scalar	1e-1	Maximum time step for BDF, when maxstepconstraintbdf is set to const.
maxstepconstraintbdf	auto const expr	auto	Maximum time step for BDF: automatic (auto), constant (const), or an expression (expr).
maxstepconstraintdopri5	auto const expr	auto	Maximum time step for Dormand–Prince 5: automatic (auto), constant (const), or an expression (expr).
maxstepconstraintgenalpha	auto const expr	auto	Maximum time step for generalized alpha: automatic (auto), constant (const), or an expression (expr).
maxstepdopri5	positive scalar	1e-1	Maximum time step for Dormand–Prince 5, when maxstepconstraintdopri5 is const.
maxstepexpressionbdf	String		Expression for the maximum time step for BDF, when maxstepconstraintbdf is expr.
maxstepexpressiongondopri5	String		Expression for the maximum time step for Dormand–Prince 5, when maxstepconstraintdopri5 is expr.
maxstepexpressiongenalpha	String		Expression for the maximum time step for generalized alpha, when maxstepconstraintgenalpha is expr.
maxstepgenalpha	positive scalar	1e-1	Maximum time step for generalized alpha, when maxstepconstraintgenalpha is const.
message	String		The log message from the last solution process.

TABLE 6-74: VALID PROPERTIES FOR TIME.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
minorder	1 2	1	Minimum BDF order.
mlinsolver	direct lumped	direct	Mass matrix solver: direct or lumped (if timemethodexp is not rk).
nlsolver	automatic manual	manual	Nonlinear solver settings.
ntimestepsupdate	positive integer	100	Number of time steps between updates, when updtlvl is set to manual when timemethodexp is not set to ab3loc.
ntimestepsupdateab3loc	positive integer	100	Number of time steps between updates, when updtlvl is set to manual when timemethodexp is set to ab3loc.
odesolvertype	implicit explicit	implicit	ODE solver type: using an implicit or explicit time stepping method.
plot	on off	off	Plot while solving.
plotfreq	tsteps tout	tout	Times to update plot.
plotgroup	String		Name of plot group for plot while solving.
probefreq	tsteps tout	tsteps	Times to update probe.
probes	vector of strings		Probed to use if probesel=manual.
probesel	all none manual	all	The probes to compute.
predictor	linear constant	linear	Predictor type to use (linear or constant),
reacf	on off	on	Compute reaction forces.
rescaleafterinitbw	on off	off	Rescale after initialization for the BDF and Generalized alpha methods, when consistent is set to bweuler.
rhoinf	numeric	0.75	Amplification factor for high frequencies.
rkmethord	rk34 ck5 dopri5	rk34	Runge–Kutta method: RK34, Cash–Karp 5, or Dormand–Prince 5. Only available when timemethodexp is set to rk.
rkstiffcheck	on off	on	Check for and stop if problem becomes numerically stiff for Runge–Kutta solvers.
rktimestep	String	1e-3	Time step for manual time stepping with the classical Runge–Kutta, Adam–Bashforth 3, and Verlet methods.
rtol	numeric	0.01	Relative tolerance.
rtstepab3loc	numeric	0.1	Relative time step change, when updtlvl is set to factor.
solref	true false	false	Use output solution for the time operators.
solutionperiod	numeric	0	The interval length for a periodic solution, when assumeperiodic is set to true.
stabcntrl	on off	off	Use a nonlinear controller for more efficient time-step control in the BDF method.
starttime	numeric	0	The start time for a period of a periodic solution, if assumeperiodic is set to true.
storeudot	on off	on	Store time derivatives.

TABLE 6-74: VALID PROPERTIES FOR TIME.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
tderglobalfactor	numeric	1	The global time derivative factor, if tderglobalmethod is set to manual.
tderglobalmethod	auto manual	auto	Global derivative tolerance method.
tderfactor	numeric	1	The time derivative factor, if tdermethod is set to manual.
tdermethod	auto manual	auto	Derivative tolerance method.
timemethod	bdf genalpha init	bdf	Implicit time-stepping method.
timemethodexp	rk erk ab3 ab3loc verlet	rk	Time-explicit solver: Runge–Kutta, classical Runge–Kutta, Adams–Bashforth 3, Adams–Bashforth 3 (local), or Verlet.
timestepbdf	numeric scalar numeric vector string with expression	0.01	Time step when manual time stepping using the BDF method.
timestepck5	numeric scalar numeric vector string with expression	0.01	Time step when manual time stepping using the Cash–Karp 5 Runge–Kutta method.
timestepdopri5	numeric scalar numeric vector string with expression	0.01	Time step when manual time stepping using the Dormand–Prince Runge–Kutta method.
timestepgenalpha	numeric scalar numeric vector string with expression	0.01	Time step when manual time stepping using the generalized alpha method.
timesteprk34	numeric scalar numeric vector string with expression	0.01	Time step when manual time stepping using the RK34 Runge–Kutta method.
tlist	numeric vector		List of output times.
tout	tlist tsteps tstepsclosest	tlist	Times to store: output times by interpolation; every N th step taken by solver; or steps taken by solver closest to output times.
tstepping	manual elemexprs	manual	Manual or from expressions time stepping.
tstepsbdf	free intermediate strict manual	free	Time-stepping mode when timemethod is set to bdf.
tstepsck5	free intermediate strict manual	free	Time-stepping mode when rkmethod is set to ck5.
tstepsdopri5	free intermediate strict manual	free	Time-stepping mode when rkmethod is set to dopri5.
tstepsgenalpha	free intermediate strict manual	free	Time-stepping mode when timemethod is set to genalpha.

TABLE 6-74: VALID PROPERTIES FOR TIME.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
tstepsrk34	free intermediate strict manual	free	Time-stepping mode when rkmethod is set to rk34.
tstepsstore	positive integer	1	Value of N for every N th step from the solver to store when tout is set to tsteps.
updtlvl	false manual factor	false	Update time step: off (false), manual, or factor when timemethodexp is set to ab3loc.
updtstep	false manual	false	Update time step: off (false) or manual, when tstepping is set to explicit for the timemethodexp methods erk and ab3.
verleiauxsolver	ab2 heun	ab2	Auxiliary solver, when timemethodexp is set to verlet.

By default, you can control the process of solving the linear or nonlinear system of equations in each time step manually. For a coupled problem, this is done through the properties Damp, Dtech, Hnlin, Initstep, Jtech, Maxiter, Minstep, and Rstep listed under femnlin. For a segregated problem, the properties listed under femstatic that are related to the segregated solver are available. When Timemethod is set to bdf it is possible to use the internal nonlinear solver of the time integrator. This can be achieved by setting Nlsolver to automatic.

The properties atol, atolmethod, atolglobal, atolglobalfactor, atolglobalmethod, atolglobalvaluemethod, atoludot, and atoludotactive require some additional explanation. The default value of the absolute tolerance for all fields is given by the property atolglobalfactor or atolglobal, depending on the setting for atolglobalvaluemethod. The modifier atolglobalmethod specifies whether the given value of atolglobal should be applied to scaled or unscaled variables. For variables where the automatic scaling does a good job, or where a manual scaling has been used, specifying the absolute tolerance in scaled variables is much easier. If either a different absolute value or scaling method than dictated by atolglobal and atolglobalmethod is wanted for one or several variables you can use the properties atol and atolmethod. Enter atol as a space-separated string with alternating field names and tolerances (for example, "u 1e-3 v 1e-6"). Enter atolmethod as a space-separated string with alternating field names and one of the strings global, scaled, or unscaled (for example, "u unscaled v scaled"). By default atolmethod is equal to global for all fields. The lists atol and atolmethod do not have to contain all fields. The ones not present get absolute tolerances as specified by atolglobal and atolglobalmethod. When solving wave-type equations, the time derivatives of all fields are also treated as unknowns, and therefore absolute tolerances have to be specified also for these components. By default these tolerances are chosen automatically. In some situations it might be necessary to specify them manually with the properties atoludot and atoludotactive. To turn on manual specification for, say, the two fields u and v, set the property atoludotactive to the string "u on v on". If atoludot is not specified, these two time-derivatives get the default absolute tolerance 1e-3. To specify other absolute tolerances, set atoludot to, for instance, the string "u 1e-4 v 1e-7". The absolute tolerance method for all time derivatives is the same as the method specified for the field itself.

The maximum allowed relative error in each time step (the local error) is specified using rtol. However, for small components of the solution vector U , the algorithm tries only to reduce the absolute local error in U below the given absolute tolerance.

There is no guarantee that the error tolerances are met strictly; that is, for hard problems they can be exceeded.

For the tolerance parameter in the convergence criterion for linear systems, the maximum of the numbers rtol and itol is used.

Use `complex=on` if complex numbers occur in the solution process.

The property `Consistent` controls the consistent initialization of a *differential algebraic equation* (DAE) system. The value `Consistent=off` means that the initial values are consistent (this is seldom the case because the initial value of the time derivative is 0). Otherwise, the solver tries to modify the initial values so that they become consistent. The value `consistent=on` can be used (when `timemethod=bdf` and `nlsolver=automatic`) for index-1 DAEs. Then the solver fixes the values of the differential DOFs and solves for the initial values of the algebraic DOFs and the time derivative of the differential DOFs. The value `Consistent=bweuler` can be used for both index-1 and index-2 DAEs. Then the solver perturbs the initial values of all DOFs by taking a backward Euler step.

For a DAE system, if `Estrat=exclude`, then the algebraic DOFs are excluded from the error norm of the time discretization error.

You can suggest a size of the initial time step using the property `initialstepbdf` when `timemethod` is set to `bdf` the property `initialstepdopri5` when `timemethod` is set to `dopri5`, and the property `initialstepgenalpha` when `timemethod` is set to `genalpha`. You also have to set one of the properties `initialstepbdfactive`, `initialstepdopri5active`, or `initialstepgenalphaactive` to `on` for the specified initial step to be active.

By default, the solver determines whether the system is differential-algebraic by looking after zero rows or columns in the mass matrix. If you have a DAE where the mass matrix has no zero rows or columns, put `masssingular=yes`.

The property `maxorder` gives the maximum degree of the interpolating polynomial in the BDF method (when `timemethod=bdf`).

If `timemethod=bdf` and `maxstepconstraintbdf=const`, then the property `maxstepbdf` put an upper limit on the time step size (this property is not allowed when `tstepsbdf>manual`). If instead `maxstepconstraintbdf=expr`, then the property `maxstepexpressionbdf` controls the maximum step size via an expression that is evaluated while solving. The same holds true for the associated `maxstep` properties if `timemethod=genalpha` or `timemethod=rk` and `rkmethod=dopri5`.

The `odesolver` property is used to select which time-stepping method to use for the ODEs:

- With `implicit` (the default), an implicit time-stepping method such as BDF is used.
- With `explicit`, an explicit time-stepping method, such as the Runge–Kutta family of explicit methods is used.

The `timemethod` property is used to select which implicit time-stepping method to use:

- With `timemethod=bdf`, the IDA solver (which uses a variable order backward differentiation formula) is used.
- With `timemethod=genalpha`, the generalized- α method is used. With generalized- α , the numerical damping can be controlled by giving a value, $0 \leq \rho_\infty \leq 1$, by which the amplitude of the highest possible frequency is multiplied each time step (hence, a small value corresponds to large damping while a value close to 1 corresponds to little damping). This is done through the property `rhoinf`. Also, the initial guess for the solution at the next time step (needed by the nonlinear solver) can be controlled through the property `predictor` when generalized- α is used. With `predictor=linear`, linear extrapolation using the current solution and time-derivative is used. With `predictor=constant`, the current solution is used as initial guess.
- When `timemethod` is set to `init` the solver computes consistent initial values (for the start time, as defined by the property `tlist`) for the system and then stop. Time derivatives of algebraic variables and indicator functions might still be uninitialized after this operation. Such uninitialized quantities are represented by NaN (not a number) in the solution object.

The `timemethodexp` property is used to select which explicit time-stepping method to use:

- With `timemethodexp=rk`, an explicit Runge–Kutta method is used: RK34, Cash–Karp 5, or Dormand–Prince 5 (specified using the `rkmethod` property).
- With `timemethodexp=erk`, a classical Runge–Kutta method of order 1–4 is used.

- With `timemethodexp=ab3`, an explicit Adams–Bashforth 3 method is used.
- With `timemethodexp=ab3loc`, a local time-stepping version of Adams–Bashforth 3 is used (for the Wave Form PDE interface).

The property `reactf` controls the computation and storage of the constraint reaction force. The value `reactf=on` (default) means that the solver stores the FEM residual vector L in the solution object. Because $L = N_F \Lambda$ for a converged solution, the residual is the same as the constraint force. Only the components of L that correspond to nonzero rows of N_F are stored. For each time for which the solution is requested an extra residual vector assembly is performed. The value `reactf=off` gives no computation or storage of the reaction force and can therefore save some computational time.

The property `tlist` must be a strictly monotone vector of real numbers. Commonly, the vector consists of a start time and a stop time. If more than two numbers are given, the intermediate times can be used as output times, or to control the size of the time steps (see below). If just a single number is given, it represents the stop time, and the start time is 0.

The property `tout` determines the times that occur in the output. If `tout=tsteps`, then the output contains every N th time steps (where N is specified using the `tstepsstore` property; default: 1) taken by the solver. If `tout=tlist`, then the output contains interpolated solutions for the times in the `tlist` property. If `tout=tstepsclosest`. The default is `tout=tlist`.

The properties `tstepsbdf` (applicable when `timemethod=bdf`), `tstepsdopri5` (applicable when `timemethod=dopri5`), and `tstepsgenalpha` (applicable when `timemethod=genalpha`) control the selection of time steps. If either of these properties is set to `free`, the solver selects the time steps according to its own logic, disregarding the intermediate times in the `tlist` vector. If either of the properties is set to `strict`, then time steps taken by the solver contain the times in `tlist`. If either of the properties is set to `intermediate`, then there is at least one time step in each interval of the `tlist` vector. If `tstepsgenalpha` has been set to `manual`, the solver follows the time step specified in the property `timestepgenalpha`. If `timestepgenalpha` is a scalar value, this time step is taken in the entire simulation. When `timestepgenalpha` is a (strictly monotone) numeric vector, the solver computes the solution at the times in the vector. The start time and stop time is still obtained from `tlist`; the vector given in `timestepgenalpha` is truncated or expanded using the first and last time step in the vector so that the start time and stop time agree with the values in `tlist`. Finally, an expression using variables with global scope and which results in a scalar can be used as `timestepgenalpha`.

For problems of wave type, the logic by which the solver selects the time step can sometimes result in a time step that oscillates in an inefficient manner. When `timemethod=genalpha` (the solver typically used for wave-type problems), you can avoid such oscillations in the time step using the properties `incrdelay` and `incrdelayactive`. When `incrdelayactive=on`, a counter keeps track of the number of consecutive time steps for which a time step increase has been warranted. When this counter exceeds the number given in the property `incrdelay`, the time step is increased and the counter is set to zero.

The order of the Runge–Kutta method can be set by the `erkorder` property. The size of the time step is controlled through the property `rktimestep` and can be given as a single scalar value, a (strictly monotone) numeric vector, or an expression using variables with global scope, which results in a scalar. For Adams–Bashforth 3, only a scalar constant value of the time step is allowed. Time stepping from expressions `tstepping=elemexprs` is useful for the Wave Form PDE interface. A local time-stepping version of Adams–Bashforth 3 is available for the Wave Form PDE interface by `timemethodexp=ab3loc`.

COMPATIBILITY

The `TimeExplicit` time-explicit solver has been removed from the Model Builder in version 6.1 but can still be used in the API. Instead, use the `Time` solver with an explicit time stepping. The `TimeExplicit` solver is still

available in the Model Builder if you open models created in versions before 6.1 that include a `TimeExplicit` solver.

	In structural mechanics models, the displacements are often quite small, and it is critical that a user-defined absolute tolerance value is chosen to be smaller than the actual displacements.
	For more information about the Time-Dependent solver; see Time-Dependent Solver in the <i>COMSOL Multiphysics Reference Manual</i> .

TimeAdaption

Handle time-dependent adaptive mesh refinement parameters.

SYNTAX

```
model.sol(sname).feature(solv).create(fname,"TimeAdaption")
model.sol(sname).feature(solv).feature(fname).set(pname,value)
```

DESCRIPTION

Handles settings for time-dependent adaptive mesh refinement. This feature can be added to a solver of the Time Dependent type.

The `TimeAdaption` feature splits the overall time range into subintervals, and in each interval an adapted mesh is generated and used. The meshes for these intervals as well as the solutions are added to the model. The solutions are stored in one container node (`model.sol()`) to facilitate the results processing.

The feature `TimeAdaption` accepts the following properties

TABLE 6-75: VALID TIMEADAPTION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>consistentrestart</code>	on off	off	Consistent initialization after restart.
<code>clist</code>	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use <code>cname</code> . See The clist and cname Properties .
<code>cname</code>	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use <code>clist</code> . See The clist and cname Properties .
<code>convertmesh</code>	on off	on	Convert to simplex mesh.
<code>eefuntime</code>	user	user	Error indicator function.
<code>eefuntime</code>	String		Error indicator name (eefuntime=user).
<code>elfrac</code>	positive scalar	0.2	Fraction of maximum refinement if <code>tauto=automatic</code> .
<code>elselect</code>	globalmin worst elements		Method for selecting elements to refine.
<code>elselectauto</code>	globalmin		Method for selecting elements to refine if <code>tauto=automatic</code> .
<code>gf</code>	positive scalar	2	Interval growth factor.
<code>globalminpar</code>	positive scalar		Controls refinement if <code>elselect=globalmin</code> .

TABLE 6-75: VALID TIMEADAPTION PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
globalminparauto	positive scalar		Controls refinement if <code>elselectauto=globalmin</code> .
initialsteprestart	positive scalar	0.001	Initial time step size after restart.
initialsteprestart active			
maxrefinement	scalar integer	5	Maximum number of element refinements.
message	String		The log message from the last solution process.
minti	positive scalar	0.01	Minimal length of adaptation time intervals.
rf	positive scalar	0.5	Interval reduction factor.
rmethod	regular longest	longest	Refinement method.
samplepts	scalar numeric vector	range(0,0.1,1)	Where to check the error in next subinterval.
tauto	manual automatic	manual	Time interval control.
tfrac	positive scalar	0.1	Length of adaptation time intervals.
tfracauto	positive scalar	0.1	Length of initial adaptation time interval if <code>tauto=automatic</code> .
timeadapgeom	String		Name of geometry sequence.
worstpar	positive scalar		Controls refinement if <code>elselect=worst</code> .

TIME ADAPTION

The TimeAdaption algorithm solves a sequence of problems on a sequence of adapted meshes. The first mesh, the base mesh, is obtained from the meshing sequence. The new adapted mesh is obtained by evaluating the mesh element error indicator, selecting a set of elements based on the element pick function, and then finally refining these elements. The solution to the problem on the previous mesh is then mapped to the new mesh and time integration continues until the next mesh adaptation takes place. The time of mesh adaptation can be determined manually or automatically.

The time adaptive solver works in one geometry at a time. You specify the name of the geometry sequence in the property `timeadapgeom`. The solver only supports simplex meshes, and if the base mesh is not simplex it can be converted by using the property `convertmesh`.

The length of the time interval using a fixed adapted spatial mesh can be controlled manually or automatically by the property `tauto`. If the time integrator runs into problems the computation is restarted at the beginning of the previous time interval. The length of the new interval is reduced to a fraction of the current interval length. This fraction is specified by the property `rf`. In the `tauto>manual` case the time interval length is given by property `tfrac`; if `tauto=automatic` the property `tfracauto` controls the initial interval length. For both cases the shortest possible interval length is given by the property `minti`.

If the property `tauto` is set to `automatic` the TimeAdaption algorithm tries to determine the length of the time interval according to the requested fraction of maximum refinement. The fraction is given by the value of the property `elfrac`. A value of zero means no refinement of the base mesh and a value of one means refinement everywhere with the maximum number of refinements (set through property `ngenlocal`). The algorithm strives to assume the given value of `elfrac` by controlling the size of the time interval. The shortening and lengthening of the interval is determined by the interval reduction and growth factors. These are the properties `rf` and `gf`, respectively.

The error indicator is specified using the property `eefuncntime`. A solution on the coarse base mesh is computed in the next time interval and the error indicator is evaluated at the points given by property `samplepts`. In this way a new adapted mesh appropriate for the next time interval can be generated and the computation on this new mesh

is then started. The sample points must be specified as a number between 0 and 1 because they are interpreted as being relative to the time interval under consideration. Entering a scalar value of 0.5 means that the error indicator is evaluated at the midpoint of the interval.

After each mesh adaptation the time integration is restarted and you can control the time stepping by the Time type analogous properties `consistentrestart` and `initialsteprestart`.

COMPATIBILITY

The property `ngenlocal` in versions prior to 5.5 has been renamed to `maxrefinement`.

TimeDiscrete

Solve a time-discretized problem.

SYNTAX

```
model.sol(sname).create(fname,"TimeDiscrete")
model.sol(sname).feature(fname).set(pname,value)
```

DESCRIPTION

Operation feature.

The output time interval and possible intermediate time values are given in the property `tlist`. The times to store are controlled by the property `tout`.

The feature `TimeDiscrete` accepts the following properties:

TABLE 6-76: VALID PROPERTIES FOR THE TIME-DISCRETE SOLVER.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
<code>atol</code>	String	empty	Absolute tolerance per field. See below,
<code>atolmethod</code>	String	empty	How to interpret the <code>atolfields</code> value. See below.
<code>atolglobal</code>	positive scalar	1e-3	Global absolute tolerance, if <code>atolglobalvaluemethod</code> is manual.
<code>atolglobalfactor</code>	positive scalar	0.1	Global absolute tolerance as a factor of the relative tolerance, if <code>atolglobalvaluemethod</code> is factor.
<code>atolglobalmethod</code>	scaled unscaled	scaled	How to interpret the <code>atolglobal</code> value.
<code>atolglobalvaluemethod</code>	factor manual	factor	Use a factor of the relative tolerance or a user-defined value for the absolute tolerance.
<code>clist</code>	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use <code>cname</code> . See The clist and cname Properties .
<code>cname</code>	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use <code>clist</code> . See The clist and cname Properties .
<code>control</code>	String	user	Name of the controlling study step or user if the feature is controlled manually.
<code>keeplog</code>	on off	off	Keep warnings in stored log.
<code>message</code>	String		The log message from the last solution process.
<code>plot</code>	on off	off	Plot while solving.
<code>plotfreq</code>	tsteps tout	tout	Times to update plot.

TABLE 6-76: VALID PROPERTIES FOR THE TIME-DISCRETE SOLVER.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
plotgroup	String		Name of plot group for plot while solving.
prevlevels	positive integer	2	Number of previous time levels to store.
probefreq	tsteps tout	tsteps	Times to update probe.
probes	vector of strings		Probed to use if probesel=manual.
probesel	all none manual	all	The probes to compute.
rtol	numeric	0.01	Relative tolerance.
timestepdiscrete	numeric scalar numeric vector string with expression	0.01	Time step when manual time stepping.
tlist	numeric vector		Time list.
tout	tlist tsteps tstepsclosest	tlist	Times to store: output times by interpolation; every <i>N</i> th step taken by solver; or steps taken by solver closest to output times.
tstepsstore	positive integer	1	Value of <i>N</i> for every <i>N</i> th step from the solver to store when tout is set to tsteps.

The `TimeDiscrete` solver is used for solving time-dependent PDEs that have already been discretized in time using, for example, the `prev` operator or the `bdf` operator. Such discretization requires the solution at previous time steps. Different discretizations require different number of previous time steps. For example, the first order accurate `bdf` method requires the solution at the previous time step, while the second-order accurate `bdf`-method requires the solution at the two preceding time steps. How many previous time steps should be accessible to the solver is controlled through the property `prevlevels`.

You can control the process of solving the linear or nonlinear system of equations in each time step manually. For a coupled problem, this is done through the properties `Damp`, `Dtech`, `Hnlin`, `Initstep`, `Jtech`, `Maxiter`, `Minstep`, and `Rstep` listed under `femnlin`. For a segregated problem, the properties listed under `femstatic` that are related to the segregated solver are available.

Because only manual time stepping is available, there is no estimation of the error made in a time step. However, the tolerances, specified through the properties `rtol`, `atol`, `atolmethod`, `atolglobal`, and `atolglobalmethod` are still important as tolerances when solving the nonlinear system of equations in each time step. For a description of these properties, see [Time](#). They should in general be set to the desired accuracy in the final solution.

The property `tlist` must be a strictly monotone vector of real numbers. Commonly, the vector consists of a start time and a stop time. If more than two numbers are given, the intermediate times can be used as output times, or to control the size of the time-steps (see below). If just a single number is given, it represents the stop time, and the start time is 0.

The property `tout` determines the times that occur in the output. If `tout=tsteps`, then the output contains every *N*th time steps (where *N* is specified using the `tstepsstore` property; default: 1) taken by the solver. If `tout=tlist`, then the output contains interpolated solutions for the times in the `tlist` property. If `tout=tstepsclosest`. The default is `tout=tlist`.

The size of the time step is controlled through the property `timestepdiscrete`. If `timestepdiscrete` is a scalar value, this time step is taken in the entire simulation. When `timestepdiscrete` is a (strictly monotone) numeric vector, the solver computes the solution at the times in the vector. The start time and stop time is still obtained from `tlist`; the vector given in `timestepdiscrete` is truncated and/or expanded using the first and/or last time

step in the vector so that the start time and stop time agrees with the values in `tlist`. Finally, an expression using variables with global scope and which results in a scalar can be used as `timestepdiscrete`.

	For more information about the time discrete solver; see Time-Discrete Solver in the <i>COMSOL Multiphysics Reference Manual</i> .
---	--

TimeExplicit

Solve time-dependent problems with explicit time stepping. Note that in the Model Builder, the **Time-Dependent Solver** node includes all time-explicit settings, and the **Time-Explicit Solver** is not available since version 6.1. You can use the API for Time instead.

SYNTAX

```
model.sol(sname).create(fname,"TimeExplicit")
model.sol(sname).feature(fname).set(pname,pvalue)
```

DESCRIPTION

Operation feature. The `TimeExplicit` solver is used for solving time-dependent PDEs using the classic Runge–Kutta or the Adams–Bashforth 3 explicit time-stepping schemes

TABLE 6-77: VALID TIMEEXPLICIT PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
<code>algebraicsolveeverydt</code>	positive double	0	Value of the period Δt , when <code>algebraicsolvemethod</code> is set to <code>dt</code> .
<code>algebraicsolveeveryn</code>	positive integer	1	Value of N , when <code>algebraicsolvemethod</code> is set to <code>n</code> .
<code>algebraicsolvemethod</code>	<code>n</code> <code>dt</code>	<code>n</code>	Strategy used to solve the algebraic equations: every N th step or periodically.
<code>clist</code>	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use <code>cname</code> . See The clist and cname Properties .
<code>cname</code>	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use <code>clist</code> . See The clist and cname Properties .
<code>control</code>	String	<code>user</code>	Name of the controlling study step or user if the feature is controlled manually.
<code>endtimeinterpolation</code>	boolean	<code>true</code>	Interpolate the last time specified in the list of output times, if <code>true</code> . If set to <code>false</code> , the last output time is solved and not interpolated. In particular, the solver does not take steps past the last time.
<code>erkorder</code>	integer between 1 and 4	4	Runge–Kutta order.
<code>exprs</code>	String		Expression for time stepping when <code>tstepping=elemexprs</code> .
<code>keeplog</code>	<code>on</code> <code>off</code>	<code>off</code>	Keep warnings in stored log.
<code>linsolver</code>	linear solver tag		Linear solver to use.
<code>message</code>	String		The log message from the last solution process.

TABLE 6-77: VALID TIMEEXPLICIT PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
ntimestepupdateab3loc	Positive integer	100	Number of time steps between updates, when updtlvl is set to manual.
odesolver	erk ab3 ab3loc	erk	Time explicit solver.
plot	on off	off	Plot while solving.
plotfreq	tsteps tout	tout	Times to update plot.
plotgroup	String		Name of plot group for plot while solving.
probefreq	tsteps tout	tsteps	Times to update probe.
probes	vector of strings		Probed to use if probesel=manual.
probesel	all none manual	all	The probes to compute.
rktimestep	positive scalar	1e-3	Time step.
storeudot	on off	on	Store time derivatives.
rtstepab3loc	positive scalar	0.1	Relative time step level, when updtlvl is set to factor.
tlist	vector of strings		Specified time list.
tout	tlist tsteps tstepsclosest	tlist	Times to store: output times by interpolation; every Nth step taken by solver; or steps taken by solver closest to output times.
tstepping	manual elemexprs	manual	Manual or from expressions time stepping.
tstepsstore	positive integer	1	Value of N for every Nth step from the solver to store when tout is set to tsteps.
updtlvl	false manual factor	false	Update time step: off (false), manual, or factor (factor only when odesolver is set to ab3loc).

The order of the Runge–Kutta method can be set by the `erkorder` property. The size of the time step is controlled through the property `rktimestep` and can be given as a single scalar value, a (strictly monotone) numeric vector, or an expression using variables with global scope, which results in a scalar. For Adams–Bashforth 3, only a scalar constant value of the time step is allowed. Time stepping from expressions `tstepping=elemexprs` is useful for the Wave Form PDE interface. A local time-stepping version of Adams–Bashforth 3 is available for the Wave Form PDE interface by `odesolver=ab3loc`.

COMPATIBILITY

The `TimeExplicit` time-explicit solver has been removed from the Model Builder in version 6.1 but can still be used in the API. Instead, use the `Time` solver with an explicit time stepping. The `TimeExplicit` solver is still available in the Model Builder if you open models created in versions before 6.1 that include a `TimeExplicit` solver.

TimeParametric

Handle properties for parameter stepping for a time-dependent problem.

SYNTAX

```
model.sol(sname).create(fname,"Time")
model.sol(sname).feature(fname).create(parname,"TimeParametric")
model.sol(sname).feature(fname).feature(parname).set(pname,pvalue)
```

DESCRIPTION

Attribute feature.

TABLE 6-78: TIME PARAMETRIC PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
addsolutions	boolean	true	Add solution data points from the general noncontinuation parameter values lists, if uselsqdata is set to true.
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
excludelsqvalues	boolean	true	Create bounding box from general parameter value lists, if uselsqdata is set to true.
pdistrib	on off	off	If the solver should distribute the parameter sweep.
plist	real array		List of parameter values. Obsolete, use plistarr instead.
plistarr	real matrix		Lists of parameter values. One row of values for each parameter name.
pname	vector of strings		Parameter names.
pwork	integer	1	Maximum number of distributed groups.
sweeptype	sparse filled	sparse	Method for doing the parameter variation. For sweeptype = sparse, the parameter tuples defined by the columns in plistarr are solved for. This method requires equal length for the rows. For sweeptype = filled, all parameter combinations given by plistarr are solved for.
uselsqdata	boolean	true	Use parameters from least-squares objectives (requires the Optimization Module).

UpperLimit

Handle upper limits for segregated steps. This feature can be added as a subfeature to a Segregated feature.

SYNTAX

```
model.sol(sname).feature(solv).feature(segregated).create(fname, "UpperLimit")
model.sol(sname).feature(solv).feature(segregated).feature(fname).set(pname, value)
```

DESCRIPTION

This feature controls the upper limits for variables used in segregated steps.

TABLE 6-79: VALID UPPER LIMIT PROPERTY.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
upperlimit	String		String of variable and upper limit pairs.

A valid upperlimit string contains pairs of variables names and their associated upper limits. For example, to impose an upper limit of 200 for the value of the field u in Component 1, and an upper limit of 90 for the value of the field v in Component 1, use the string "comp1.u 200 comp1.v 90".

SEE ALSO

[LowerLimit](#), [Segregated](#)

Variables

Handle initial data and scaling for variables solved, as well as how variables not solved for are computed. The methods are applied for the dependent variables present as Field or State subattributes. These attributes are automatically created and updated by this feature. So, if the Analysis (for the solver sequence) is altered or if a different Analysis is used in the sequence, then the Field attributes are changed accordingly.

SYNTAX

```
model.sol(sname).create(fname,"Variables")
model.sol(sname).feature(fname).set(pname,pval)
model.sol(sname).feature(fname).feature(varname).set(pname,pval)
model.sol(sname).feature(fname).updateVariables()
```

DESCRIPTION

Operation feature. Computes the initial values for the variables that are solved for and how the variables not solved for are computed. The variables handled are the ones present as Field or State attributes. The feature also handles scaling and which variables to store in output.

Use the `updateVariables` method if needed to perform a manual update of the variables after some change that affects them.

TABLE 6-80: VALID VARIABLES PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
clist	String array		Provide values for constants as input parameters using a string array; for the corresponding constant names, use cname. See The clist and cname Properties .
cname	String array		Provide names of constants as input parameters using a string array; for the corresponding constant values, use clist. See The clist and cname Properties .
control	String	user	Name of the controlling study step or user if the feature is controlled manually.
initmethod	init sol	init	Method used for initial value computation.
initsol	zero solution object	zero	Initial value solution object.
initsoluse	current manual solution store tag	current	Use current or stored values from the initial value solution object, or use manual to provide a solution index in the initsolusesolnum property.
initsolusesolnum	integer or parameter name		An outer solution index for a parametric sweep, such as a global parameter, when initsoluse is set to manual.
manualsolnum	positive integer		Identifies the solution used when solnum = manual.
notlistsolnum	vector of positive integers		List that identifies the solutions used when notsolnum = from_list.
notmanualsolnum	vector of positive integers		Identifies the solutions used when notsolnum = manual.
notsol	zero solution object	zero	Solution object for variables not solved for.
notsolmethod	init sol	init	Method used for variables not solved for.

TABLE 6-80: VALID VARIABLES PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
notsolnum	auto all first last from_list interp manual positive integer	auto	Which solnums from other solution to use for variables not solved for.
notsoluse	current solution store tag	current	Use current or stored values from the solution object for variables not solved for.
nott	double		The interpolation value used for values of variables not solved for when notsolnum = interp.
resscalemethod	auto manual	auto	Method used for scaling of residual.
resscaleval	scalar	1	Residual scaling value, if resscalemethod is manual.
scalemethod	auto init none manual	auto	Method used for scaling of variables.
scaleval	scalar	1	Global scaling value, if scalemethod is manual.
solnum	auto all first last interp manual positive integer	auto	The solnum from other solution to use for initial values of variables solved for.
t	double		The interpolation value used for initial values of variables solved for when solnum = interp.
useinitsol	on off	off	Use user-controlled initial values of variables solved for.
usesol	on off	off	Use user-controlled values of variables not solved for.

TABLE 6-81: VALID VARIABLE SUBATTRIBUTE FIELD/STATE PROPERTIES.

PROPERTY	VALUES	DEFAULT	DESCRIPTION
comp	vector of strings		Field/State components
out	on off	on	Store Field/State in output
reconstruct	String	none	Tag of a compatible reduced model to use for reconstruction when solvefor is off.
scalemethod	auto init none manual parent	parent	Method used for scaling of variables
scaleval	scalar	1	Scaling value
solvefor	on off	on	Solve for this Field/State
storesel	Array of selection strings		Selections defining which field data to store.
usestoresel	all selection	all	Store all field data or field data defined in selections in storesel.

The properties `initmethod`, `initsol`, and `initsoluse` determine the initial value for the solution components you solve for when `useinitsol` is set to `on`.

The properties `notsolmethod`, `notsol`, `notsoluse`, and `notsolnum` determine the value of solution components you do not solve when `usesol` is set to `on`.

Which variables to solve for, and which variables to store in the solution is controlled by the properties `solvefor` and `out` in the Field or State attributes.

The `reconstruct` property is only available when using model reduction and then to configure reconstruction for dependent variables not solved for.

The properties `scalemethod` and `scaleval` determine a scaling of the degrees of freedom that is applied in order to get a more well-conditioned system. The possible values of `scalemethod` are:

TABLE 6-82: VALUES FOR THE PROPERTY `SCALEMETHOD`.

VALUE	MEANING
auto	The scaling is automatically determined.
init	The scaling is determined from the initial value. Use this if the sizes of the components of the initial value give a good estimate of the order of magnitude of the solution.
none	No scaling is applied.
manual	The scaling is user controlled by setting the property <code>scaleval</code> .
parent	Scaling method is inherited from the parent Variables feature (only for subattributes).

XmeshInfo

Get extended mesh information.

SYNTAX

```
SolverFeature step = model.sol(seqTag).feature(compileEquationsTag);
XmeshInfo xmi = step.xmeshInfo();
XmeshInfo xmi = step.xmeshInfo(meshCase);
SolverFeature var = model.sol(seqTag).feature(variablesTag);
XmeshInfo xmi = var.xmeshInfo();
XmeshInfo xmi = var.xmeshInfo(meshCase);
XmeshInfo xmi = model.sol(seqTag).xmeshInfo();
XmeshInfo xmi = model.sol(seqTag).xmeshInfo(meshCase);

String[] mcases = xmi.meshCases();

int nDofs = xmi.nDofs();
String[] fieldNames = xmi.fieldNames();
int[] fieldNDofs = xmi.fieldNDofs();
String[] geomTags = xmi.geoms();
String[] meshTypes = xmi.meshTypes();
String[] meshTypes = xmi.meshTypes(geomTag);

XmeshInfoDofs dofs = xmi.dofs();
int[] geomNums = dofs.geomNums();
int[] nodes = dofs.nodes();
double[][] coords = dofs.coords();
String[] dofNames = dofs.dofNames();
int[] nameInds = dofs.nameInds();
int[] solVecInds = dofs.solVectorInds();

XmeshInfoNodes nodes = xmi.nodes();
XmeshInfoNodes nodes = xmi.nodes(geomTag);
double[][] coords = nodes.coords();
String[] dofNames = nodes.dofNames();
int[][] dofs = nodes.dofs();

XmeshInfoElements elems = xmi.elements(meshType);
XmeshInfoElements elems = xmi.elements(meshType,geomTag);
int[][] nodes = elems.nodes();
String[] locDofNames = elems.localDofNames();
double[][] locDofCoords = elems.localDofCoords();
int[][] dofs = elems.dofs();

model.sol(seqTag).feature(compileEquationsTag).clearXmesh();
model.sol(seqTag).feature(variablesTag).clearXmesh();
```

DESCRIPTION

The Xmesh information methods provide information about the numbering of elements, nodes, and degrees of freedom (DOFs) in the extended mesh and in the matrices returned by `Assemble` and the solvers.

```
SolverFeature step = model.sol(seqTag).feature(compileEquationsTag);
XmeshInfo xmi = step.xmeshInfo();
```

returns information about all degrees of freedom in the study step given by the compile equations feature, for the main mesh case. This includes information about internal degrees of freedom.

```
XmeshInfo xmi = step.xmeshInfo(meshCase);
```

returns information about the given mesh case. The string `meshCase` can be `main`, `adaptionresidual`, or a multigrid level tag.

```
SolverFeature var = model.sol(seqTag).feature(variablesTag);
XmeshInfo xmi = var.xmeshInfo();
XmeshInfo xmi = var.xmeshInfo(meshCase);
```

returns information about the degrees of freedom solved for in the given variables feature. That is, the numbering of the degrees of freedom span over the DOFs solved for, which is the indexing used in the matrices and vectors assembled by the solver. Internal degrees of freedom are not included.

```
XmeshInfo xmi = model.sol(seqTag).xmeshInfo();
XmeshInfo xmi = model.sol(seqTag).xmeshInfo(meshCase);
```

is equivalent to calling `xmeshInfo` on the last compile equations feature in the sequence.

```
model.sol(seqTag).feature(compileEquationsTag).clearXmesh();
model.sol(seqTag).feature(variablesTag).clearXmesh();
```

clears out the Xmesh object created by the call to `xmeshInfo`. After the required information has been obtained from the `XmeshInfo` object, this function should be called to release memory. When `xmeshInfo` is called on a solver sequence, an already existing Xmesh object is used, so there is no need to call `clearXmesh`.

General Information

`String[] mcases = xmi.meshCases()` returns a string vector containing tags of all mesh cases.

`int nDofs = xmi.nDofs()` returns the total number of DOFs.

`String[] fieldNames = xmi.fieldNames()` returns the field names, or the field names solved for.

`int[] fieldNDofs = xmi.fieldNDofs()` returns the number of DOFs for each field.

`String[] geomTags = xmi.geoms()` returns the tags of all geometries that exist in the xmesh.

`String[] meshTypes = xmi.meshTypes()` returns all mesh types.

`String[] meshTypes = xmi.meshTypes(geomTag)` returns all mesh types in geometry `geomTag` (a string). Possible mesh types are `vtx`, `edg`, `tri`, `quad`, `tet`, `hex`, `prism`, and `pyr`.

Information About Each DOF

`XmeshInfoDofs dofs = xmi.dofs()` returns information about each DOF.

The class `XmeshInfoDofs` has the following methods:

TABLE 6-83: XSMESHINFODOFS METHODS.

FIELD	CONTENTS
<code>int[] geomNums()</code>	1-based geometry numbers for all DOFs
<code>int[] nodes()</code>	0-based node numbers for all DOFs.
<code>double[][] coords()</code>	Global coordinates for all DOFs in the model length unit. The kth column of this matrix contains the coordinates of DOF number k.

TABLE 6-83: XMESHINFODOFS METHODS.

FIELD	CONTENTS
<code>double[][] coords(xdGeomTag)</code>	For DOFs in an extra dimension product, return the coordinates of each DOF in the extra dimension geometry <code>xdGeomTag</code> . For DOFs that are not located in an extra dimension product containing <code>xdGeomTag</code> , the value NaN is returned.
<code>double[][] gCoords()</code>	Same as <code>coords()</code> except that coordinates are given in the geometry length unit. If there is more than one geometry, the coordinates of each DOF are given in the length unit of the geometry of that DOF.
<code>String[] dofNames()</code>	DOF names
<code>int[] nameInds()</code>	0-based indices into <code>dofNames()</code> for all DOFs.
<code>int[] solVectorInds()</code>	0-based indices into solution vector for all DOFs.

Information About Each Node Point

`XmeshInfoNodes nodes = xmi.nodes()` returns information about nodes. This method issues an error if there is more than one geometry.

`XmeshInfoNodes nodes = xmi.nodes(geomTag)` returns information about nodes in geometry `geomTag` (a string).

The class `XmeshInfoNodes` has the following methods:

TABLE 6-84: XMESHINFONODES CLASS METHODS.

FIELD	CONTENTS
<code>double[][] coords()</code>	Global coordinates for all nodes. The <i>n</i> th column of the matrix <code>coords</code> contains the coordinates of node point number <i>n</i>
<code>double[][] gCoords()</code>	Same as <code>coords()</code> except that coordinates are given in the geometry length unit.
<code>String[] dofNames()</code>	DOF names in this geometry
<code>int[][] dofs()</code>	0-based DOF numbers for all nodes in this geometry. <code>dofs()[k][n]</code> is the DOF number for DOF name <code>dofNames()[k]</code> at node point <i>n</i> . A value of -1 means that there is no DOF with this name at the node. Note: If there is a slit, only one of the DOFs is given for each node point.

Information About Each Mesh Element

`XmeshInfoElements[] elems = xmi.elements(meshType)` returns information about mesh elements of type `meshType` (a string). This method issues an error if there is more than one geometry.

`XmeshInfoElements[] elems = xmi.elements(meshType,geomTag)` returns information about mesh elements of type `meshType` in geometry `geomTag`.

The `XmeshInfoElements` class has the following methods:

TABLE 6-85: XMESHINFOELEMENTS CLASS METHODS.

FIELD	CONTENTS
<code>double[][] localCoords()</code>	Local coordinates of nodes. The <i>k</i> th column of the matrix <code>localCoords()</code> contains the coordinates of local node point number <i>k</i> .
<code>int[][] nodes()</code>	0-based node point indices for all mesh elements of type <code>type()</code> . <code>nodes()[k][e1]</code> is the node point number within geometry <code>geomNum()</code> (see the output <code>xmi.nodes()</code>) for local node point <i>k</i> within mesh element <i>e1</i> . A value -1 means that there is no node point at this location.
<code>String[] localDofNames()</code>	The name for each local DOF.
<code>double[][] localDofCoords()</code>	The local coordinates for each local DOF (one column for each local DOF).
<code>int[][] dofs()</code>	0-based DOF numbers for all mesh elements of type <code>type()</code> . <code>dofs()[k][e1]</code> is the DOF number for local DOF <i>k</i> within mesh element <i>e1</i> . A value -1 means that there is no DOF at this location.

Studies and Study Steps

Introduction

In the COMSOL Desktop, you create one or more *studies*, each with one or more *study steps* to compute the solution to a model. The studies generate a solver sequence with the solvers and other solver features that correspond to the study steps in the study.



Study and Study Step Types in the *COMSOL Multiphysics Reference Manual*.

CREATING A STUDY AND ADDING STUDY STEP

The following commands create a study `std1` and adds a Stationary study step to that study:

```
model.study().create("std1");  
model.study("std1").create("stat", "Stationary");
```

RUNNING (COMPUTING) A STUDY

To run a study, use

```
model.study(<tag>).run();
```

The `run()` command corresponds to clicking **Compute** on a **Study** node in the COMSOL Desktop.

CREATING A REFERENCE TO A STUDY STEP

You can also create a reference to an existing study step from a solver sequence:

```
model.sol("sol1").create("st1", "StudyStep");  
model.sol("sol1").feature("st1").set("study", "std1");  
model.sol("sol1").feature("st1").set("studystep", "stat");
```

ATTACHING A STUDY

It is also possible to attach a study for making a solver sequence or batch feature part of the study sequence:

```
model.sol("sol1").attach("std1");
```

The `attach()` operation implies calling `study(<tag>)` (see below). You can only have one solver sequence attached to each study but you can have multiple batch features.

ASSOCIATING A SOLVER SEQUENCE WITH A STUDY

You can associate a solver sequence with a study using the following syntax:

```
model.sol("sol1").study("std1");
```

In contrast, `attach()` (see above) also makes the solver sequence part of the study sequence.

MAIN STUDY FEATURE SETTINGS

TABLE 6-86: METHODS FOR THE MAIN STUDY FEATURE SETTINGS.

METHOD	DESCRIPTION
<code>isGenConv()</code>	True if convergence plots should be generated.
<code>isGenIntermediatePlots()</code>	True if plots for intermediate Stored Solution nodes should be generated.
<code>isGenPlots()</code>	True if default results plots should be generated.
<code>isPlotUndefVals()</code>	True if plots should be generated showing locations of undefined values.
<code>isStoreSolution()</code>	True if a Store Solution node should be generated for each intermediate study step.
<code>setGenConv(boolean)</code>	Set to true if convergence plots should be generated.
<code>setGenIntermediatePlots(boolean)</code>	Set to true to generate plots for intermediate Store Solution nodes.
<code>setGenPlots(boolean)</code>	Set to true if default results plots should be generated.
<code>setPlotUndefVals(boolean)</code>	Set to true to generate plots showing the location of undefined values.
<code>setStoreCompleteHistory(boolean)</code>	Set to true to store the complete solver history.
<code>setStoreSolution(boolean)</code>	Set to true to generate a Store Solution node for each intermediate study step.

For example, use

```
model.study("std1").setGenConv(true);
```

to generate convergence plots when computing the solution.

COMMON STUDY STEP PROPERTIES

The following table explains properties available for all study steps defining equations:

TABLE 6-87: PHYSICS AND VARIABLES SELECTION

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>activate</code>	String Map		Choose physics interfaces and other features to solve for. Allowed keys are - A physics interface tag <code>frame</code>: followed by a <code>Frame</code> entity tag <code>rom</code>: followed by a <code>ReducedModel</code> entity tag - A component tag followed by <code>:shape</code> controlling shape optimization in the component - A component tag followed by <code>:topopt</code> controlling topology optimization in the component Allowed values are on or off. Default depends on the key.
<code>activateCoupling</code>	String Map		Choose coupling features to solve for. Allowed keys are <code>MultiphysicsCoupling</code> entity tags. Allowed values are on or off. Default is on.

TABLE 6-87: PHYSICS AND VARIABLES SELECTION

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activaterom	String Map		Choose reduced-order model output variables to solve for. Allowed keys are ReducedModel entity tags. Allowed values are on or off. Default is off.
disabledcommon	Reference Array		CommonFeature entities disabled when useadvanceddisable is on. Not all types can be disabled.
disabledcoordinatesystems	Reference Array		Coordinate system features disabled when useadvanceddisable is on. Only types representing PMLs, infinite elements and absorbing layers can be disabled.
disabledcoupling	Reference Array		MultiphysicsCoupling entities disabled when useadvanceddisable is on.
disabledpair	Reference Array		Pair entities disabled useadvanceddisable is on.
disabledphysics	Reference Array		Physics interfaces and features disabled when useadvanceddisable is on. Allowed reference tags are: • A physics interface tag • A path to a physics feature of the form phys_tag/feature_tag/...
disabledreduced	Reference Array		Reduced-order model features disabled when useadvanceddisable is on.
disabledvariables	Reference Array		Variables entities disabled when useadvanceddisable is on.
disableFrameControl	Reference Array		Entities which should not control any frame when useadvanceddisable is on. Allowed reference tags are: • A physics interface tag • frame: followed by a Frame entity tag • A component tag followed by :shape controlling shape optimization in the component
discretization	String Map		Select discretization for physics interfaces. Allowed keys are the same as in activate. Allowed values are physics (also default) and tags of any Discretization feature belonging to the key entity.
equationform	String Map		Override study step equation form for physics interfaces. Allowed keys are the same as in activate. Allowed values are physics (also default if the physics allows setting equation form) and any equation form allowed by the physics interface settings.

TABLE 6-87: PHYSICS AND VARIABLES SELECTION

PROPERTY	VALUE	DEFAULT	DESCRIPTION
equationform_freq	String Map		Specify frequency to use for a physics interface when its equation form is set to a frequency-domain type using the equationform property. Allowed keys are the same as in activate. Allowed values are parameter expressions. The default is the same as the default in the physics interface settings.
equationform_freq_src	String Map		Specify whether an equation-based physics interface should define time derivatives using the main frequency of a frequency-domain study step or a specific frequency given in equationform_freq. Allowed values are fromSolver (the default) and userdef.
reconstructors	String Map		Choose a reduced-order model to reconstruct dependent variables in a physics interface not solved for. Allowed keys are tags of physics interfaces not solved for and for which there exists at least one reconstructing reduced-order model. Allowed values are none (also default) and tags of ReducedModel entities capable of reconstructing the key interface's dependent variables.
useadvanceddisable	on off	off	Enable advanced frame control and disabling of model entities.

PROPERTIES OF SPECIFIC STUDY STEP TYPES

The following table lists the major study steps with links to documentation of available properties for each study step:

- [Batch](#)
- [Batch Sweep](#)
- [Bidirectionally Coupled Particle Tracing](#)
- [Bidirectionally Coupled Ray Tracing](#)
- [Cluster Computing](#)
- [Cluster Sweep](#)
- [Eigenfrequency](#)
- [Eigenvalue](#)
- [Stationary Then Eigenfrequency](#)
- [Frequency Domain and Frequency-Domain Perturbation](#)
- [Frequency to Time FFT](#)
- [Function Sweep](#)
- [Material Sweep](#)
- [Model Reduction](#)
- [Multigrid Level](#)
- [Parametric Sweep](#)
- [Ray Tracing](#)
- [Sensitivity](#)
- [Stationary](#)
- [Surrogate Model Training](#)
- [Time Dependent](#)
- [Time Discrete](#)
- [Time to Frequency FFT](#)

Batch

Use a Batch study step to start a COMSOL Multiphysics batch process that solves the current study on your computer.

SYNTAX

```
model.study(stdname).create(fname, "Batch");  
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-88: PROPERTIES FOR BATCH SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
batchdatastoragemode	filesystem database sameascurrentmodel	filesystem	Sets the storage mode for generated batch data during a batch run. filesystem will store the data on the file system in the configured batch directories. database will store the data in the same database and branch as where the original model is stored. sameascurrentmodel will store the data where the original model is stored, which can be either the file system or a database.
batchdir	String	Home directory	The directory to store files used by the batch job.
batchfile	String	batchmodel.mph	Name of batch model file.
batchmodelstoragemode	filesystem database sameascurrentmodel	filesystem	Sets the storage mode for generated models during a batch run. filesystem will store the models on the file system in the configured batch directories. database will store the models in the same database and branch as where the original model is stored. sameascurrentmodel will store the models where the original model is stored, which can be either the file system or a database.
clearmesh	on off	on	Clear meshes.
clearsol	on off	on	Clear solutions.
extsolveren	on off	off	Generate solver sequence in external process.
savefile	on off	off	Output model to file.
serverdir	String		Server directory.
specserverdir	on off	off	Specify server directory path.
syhaccumprobetable	on off	on	Synchronize accumulated probe table.
synchsolutions	on off	off	Synchronize solutions.

TABLE 6-89: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
graphics	on off	off	Use graphics.
maxalive	Integer		Alive time (seconds).

TABLE 6-89: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
maxallow	Integer		Maximum number of simultaneous jobs.
maxrestarts	Integer		Maximum number of job restarts.

Batch Sweep

Use the Batch Sweep study step to find the solution to a sequence of stationary or time-dependent simulations that arise when you vary some parameters of interest.

SYNTAX

```
model.study(stdname).create(fname, "BatchSweep");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-90: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.
sweepstype	filled sparse	sparse	Sweep type.

TABLE 6-91: PROPERTIES FOR OUTPUT WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
accumtable	String	new	Accumulated probe table.
accumtableall	on off	on	Use all probes for the accumulated probe table
probes	Vector of strings		Probes to use when probesel=manual.
probesel	all none manual	all	Probes to compute.
useaccumtable	on off	on	Use accumulated probe table.

TABLE 6-92: PROPERTIES FOR BATCH SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
batchdatastoragemode	filesystem database sameascurrentmodel	filesystem	Sets the storage mode for generated batch data during a batch run. filesystem will store the data on the file system in the configured batch directories. database will store the data in the same database and branch as where the original model is stored. sameascurrentmodel will store the data where the original model is stored, which can be either the file system or a database.
batchdir	String	Home directory	The directory to store files used by the batch job.
batchfile	String	batchmodel.mph	Name of batch model file.

TABLE 6-92: PROPERTIES FOR BATCH SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
batchmodelstoragemode	filesystem database sameascurrentmodel	filesystem	Sets the storage mode for generated models during a batch run. filesystem will store the models on the file system in the configured batch directories. database will store the models in the same database and branch as where the original model is stored. sameascurrentmodel will store the models where the original model is stored, which can be either the file system or a database.
clearmesh	on off	on	Clear meshes.
clearsol	on off	on	Clear solutions.
extsolvergen	on off	off	Generate solver sequence in external process.
paramfilename	on index	on	Add parameter names and values or shorter indices to the filename.
savefile	on off	off	Output model to file.
serverdir	String		Server directory.
specserverdir	on off	off	Specify server directory path.
sychaccumprobetable	on off	on	Synchronize accumulated probe table.
synchsolutions	on off	off	Synchronize solutions.

TABLE 6-93: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
graphics	on off	off	Use graphics.
maxalive	Integer		Alive time (seconds).
maxallow	Integer	1	Maximum number of simultaneous jobs.
maxrestarts	Integer		Maximum number of job restarts.

Bidirectionally Coupled Particle Tracing

The Bidirectionally Coupled Particle Tracing study step is a special case of the Time Dependent study step that is used to model bidirectionally coupled particle-field or fluid-particle interactions. It is available with the Particle Tracing Module. It is similar to the Time Dependent study step but has an additional section called Iterations, which determines the behavior of the iterative solver loop for self-consistently modeling particle-field interactions in the default solver sequence.

SYNTAX

```
model.study(stdname).create(fname, "BidirectionallyCoupledParticleTracing");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-94: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
tlist	Numeric vector		Time list.
usertol	on off	off	Physics-controlled or user-defined tolerance.
rtol	Positive scalar	0.01	Relative tolerance, if usertol is on.
tunit	String	s	Time unit.

TABLE 6-95: PROPERTIES FOR PLOT RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probefreq	pout psteps	pout	When to update probes.
probes	Vector of strings		Probes to use when probesel>manual.
probesel	all none manual	all	Probes to compute.
plotfreq	tout tsteps	tout	Where to update plot.

TABLE 6-96: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-97: PROPERTIES FOR ITERATIONS SECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
expr	String	1	Global expression used to compute relative error when method=convergence.
iter	Positive integer	5	Number of iterations of the solver loop when method=iterations.
maxiter	Positive integer	25	Maximum number of iterations in the solver loop when method=convergence.

TABLE 6-97: PROPERTIES FOR ITERATIONS SECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
method	convergence iterations	iterations	Choose whether termination of the solver loop is based on the convergence of a global variable or a fixed number of iterations.
miniter	Positive integer	1	Minimum number of iterations in the solver loop when method=convergence.
rtolterm	Positive real number	0.001	Relative tolerance for termination of the solver loop when method=convergence.
rtolthresh	Positive real number	1	Threshold used to avoid division by zero while computing the relative error when method=convergence.

TABLE 6-98: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.
useinitsol	on off	off	User-controlled initial values for variables solved for.
usesol	on off	off	User-controlled values for variables not solved for.

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-99: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[]{"geom1", "mesh1", "geom2", "mesh2"}.

TABLE 6-100: PROPERTIES FOR MESH ADAPTATION AND STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapgeom	String		Name of geometry to use.
adaption	on off	off	Adaptive mesh refinement.
adaptselection	selection	all()	The selection for the adaptation (the default is the entire geometry). See Selections .
autoremesh	on off	off	Automatic remeshing.
autoremeshgeom	String		Name of geometry to use.
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.
sweepstype	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
useparam	on off	off	Auxiliary sweep.

Bidirectionally Coupled Ray Tracing

The Bidirectionally Coupled Particle Tracing study step is a special case of the Ray Tracing study step that is used to model bidirectionally coupled particle-field or fluid-particle interactions. It is available with the Ray Optics Module. It is similar to the Ray Tracing study step but has an additional section called Iterations, which determines the behavior of the iterative solver loop for self-consistently modeling ray-field interactions in the default solver sequence.

SYNTAX

```
model.study(stdname).create(fname, "BidirectionallyCoupledRayTracing");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-101: PROPERTIES FOR BIDIRECTIONALLY COUPLED RAY TRACING STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
charvel	String	c_const or 343 m/s	Group velocity used to convert path lengths to solution times when timestepspec=specifylength.
l1list	Numeric vector		List of path lengths when timestepspec=specifylength.

TABLE 6-101: PROPERTIES FOR BIDIRECTIONALLY COUPLED RAY TRACING STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
lunit	String	m	Length unit when <code>timestepspec=specifylength</code> .
numberofreflections	Positive integer	5	Maximum number of reflections per ray for automatic stop conditions when <code>raystopcond=reflections</code> .
raystopcond	nostop noactive rayintensity reflections	nostop	Automatic stop condition in the default solver sequence.
rtol	Positive scalar	0.01	Relative tolerance, if <code>usertol</code> is on.
thresholdintensity	String	1 [W/m ²]	Threshold intensity for automatic stop conditions when <code>raystopcond=rayintensity</code> .
timestepspec	specifytime specifylength	specifytime	Determines whether the time intervals are entered directly or in terms of a path length.
tlist	Numeric vector		Time list when <code>timestepspec=specifytime</code> .
tunit	String	ns or ms	Time unit when <code>timestepspec=specifytime</code> .
usertol	on off	off	Physics-controlled or user-defined tolerance.

TABLE 6-102: PROPERTIES FOR PLOT RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probefreq	pout psteps	pout	When to update probes.
probes	Vector of strings		Probes to use when <code>probesel>manual</code> .
probesel	all none manual	all	Probes to compute.
plotfreq	tout tsteps	tout	Where to update plot.

TABLE 6-103: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .

TABLE 6-103: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-104: PROPERTIES FOR ITERATIONS SECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
expr	String	1	Global expression used to compute relative error when method=convergence.
iter	Positive integer	5	Number of iterations of the solver loop when method=iterations.
maxiter	Positive integer	25	Maximum number of iterations in the solver loop when method=convergence.
method	convergence iterations	iterations	Choose whether termination of the solver loop is based on the convergence of a global variable or a fixed number of iterations.
miniter	Positive integer	1	Minimum number of iterations in the solver loop when method=convergence.
rtolterm	Positive real number	0.001	Relative tolerance for termination of the solver loop when method=convergence.
rtolthresh	Positive real number	1	Threshold used to avoid division by zero while computing the relative error when method=convergence.

TABLE 6-105: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.

TABLE 6-105: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.
useinit sol	on off	off	User-controlled initial values for variables solved for.
usesol	on off	off	User-controlled values for variables not solved for.

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-106: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[] {"geom1", "mesh1", "geom2", "mesh2"}.

TABLE 6-107: PROPERTIES FOR MESH ADAPTATION AND STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapgeom	String		Name of geometry to use.
adaption	on off	off	Adaptive mesh refinement.
adaptselection	selection	all()	The selection for the adaptation (the default is the entire geometry). See Selections .
autoremesh	on off	off	Automatic remeshing.
autoremeshgeom	String		Name of geometry to use.
classicadap	on off	off	If on, use adaptive method for vector elements from version 6.0 and earlier.
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.
sweep type	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
useparam	on off	off	Auxiliary sweep.

Cluster Computing

Use the Cluster Computing study step when you want to submit COMSOL Multiphysics batch jobs to a job scheduler that in turn runs the batch job on a second computer or cluster.



The functionality described in this section requires a Floating Network License (FNL).

SYNTAX

```
model.study(stdname).create(fname, "ClusterComputing");  
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-108: PROPERTIES FOR CLUSTER COMPUTING SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
batchdatastoragemode	filesystem database sameascurrentmode 1	filesystem	Sets the storage mode for generated batch data during a batch run. filesystem will store the data on the file system in the configured batch directories. database will store the data in the same database and branch as where the original model is stored. sameascurrentmodel will store the data where the original model is stored, which can be either the file system or a database.
batchdir	String	Home directory	The directory to store files used by the batch job.
batchfile	String	batchmodel.mph	Name of batch model file.
batchmodelstoragemode	filesystem database sameascurrentmode 1	filesystem	Sets the storage mode for generated models during a batch run. filesystem will store the models on the file system in the configured batch directories. database will store the models in the same database and branch as where the original model is stored. sameascurrentmodel will store the models where the original model is stored, which can be either the file system or a database.
batchlic	on off		Use batch license.
clustertype	general whpc2008 sge slurm pbs lsf none		The type of cluster job.
extsolvergen	on off	off	Generate solver sequence in external process.
hostfile	String		Path to host file.
mpiargs	String		Additional MPI arguments.
mpibootstrap	String		Name of bootstrap server.
mpipath	String		Installation directory for MPI.
nn	Integer	1	Number of nodes.
rundir	String		The directory to store files used by the batch job.
schedargs	String		Additional scheduler arguments

TABLE 6-108: PROPERTIES FOR CLUSTER COMPUTING SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
scheduler	String		The scheduler for the batch job, if clustertype is whpc2008, slurm, or pbs, or lsf.
serverdir	String		Server directory.
sgequeue	String		The name for the cluster queue, if clustertype is sge, slurm, pbs, or lsf.
specbatchdir	on off		Specify different directory for batch process than used by the current process.
specserverdir	on off	off	Specify server directory path.
user	String		User account for submitting the job if clustertype is whpc2008, slurm, pbs, or lsf.

TABLE 6-109: PROPERTY FOR CLUSTER SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pdistrib	on off	off	Distribute parametric sweep.

TABLE 6-110: PROPERTIES FOR REMOTE AND CLOUD ACCESS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
filecmd	String		File for commands when remotecmd is file.
filetransfercmd	none scp file user	none	Command transfer file.
filetransferfrom	String		List of files to transfer from the other computer after the batch jobs have finished.
filetransferfromusercmd	String		Command to transfer files from remote location.
filetransferto	String		List of files to transfer to the other computer before running the batch job.
filetransfertousercmd	String		Command to transfer files to remote location.
remote	on off		Run on remote server.
remotecmd	none ssh file mpi user		Command to use when invoking a command on a remote server.
remotehosts	String		List of remote hostnames.
remoteos	native windows linux		Command transfer file.
remoteusercmd	String		Command to use when invoking a command on a remote server.
scpkey	String		SCP key file.
scppath	String		Directory where SCP resides.
scpuser	String		Username used by SCP.
scpusercmd	String		Command for copying files to remote location.
sshcmd	ssh putty user		SSH command.
sshkey	String		SSH key file.
sshpath	String		Directory where SSH resides.
sshport	String		Port host.

TABLE 6-110: PROPERTIES FOR REMOTE AND CLOUD ACCESS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sshports	String		Ports that should be forwarded by SSH.
sshuser	String		Username used by SSH.

TABLE 6-111: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
graphics	on off	off	Use graphics.
maxalive	Integer		Alive time (seconds).
maxallow	Integer	1	Maximum number of simultaneous jobs.
maxrestarts	Integer		Maximum number of job restarts.

Cluster Sweep

Use the Cluster Sweep study step when you want to study to solve several models in parallel where each model has a different set of parameters.



The functionality described in this section requires a Floating Network License (FNL).

SYNTAX

```
model.study(stdname).create(fname, "ClusterSweep");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-112: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.
sweepstype	filled sparse	sparse	Sweep type.

TABLE 6-113: PROPERTIES FOR OUTPUT WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
accumtable	String	new	Accumulated probe table.
accumtableall	on off	on	Use all probes for the accumulated probe table.
probes	Vector of strings		Probes to use when probesel>manual.
probesel	all none manual	all	Probes to compute.
useaccumtable	on off	on	Use an accumulated probe table.

TABLE 6-114: PROPERTIES FOR BATCH SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
batchdatastoragemode	filesystem database sameascurrentmode 1	filesystem	Sets the storage mode for generated batch data during a batch run. filesystem will store the data on the file system in the configured batch directories. database will store the data in the same database and branch as where the original model is stored. sameascurrentmode 1 will store the data where the original model is stored, which can be either the file system or a database.
batchdir	String	Home directory	The directory to store files used by the batch job.
batchfile	String	batchmodel.mph	Name of batch model file.
batchmodelstoragemode	filesystem database sameascurrentmode 1	filesystem	Sets the storage mode for generated models during a batch run. filesystem will store the models on the file system in the configured batch directories. database will store the models in the same database and branch as where the original model is stored. sameascurrentmode 1 will store the models where the original model is stored, which can be either the file system or a database.
batchlic	on off		Use batch license.
clearmesh	on off	on	Clear meshes.
clearsol	on off	on	Clear solutions.
clustertype	general whpc2008 sge slurm pbs lsf none		The type of cluster job.
extsolvergen	on off	off	Generate solver sequence in external process.
hostfile	String		Path to host file.
mpiargs	String		Additional MPI arguments.
mpibootstrap	String		Name of bootstrap server.
mpipath	String		Installation directory for MPI.
nn	Integer	1	Number of nodes.
paramfilename	on index	on	Add parameter names and values or shorter indices to the filename.
rundir	String		The directory to store files used by the batch job.
savefile	on off	off	Output model to file.
serverdir	String		Server directory.

TABLE 6-114: PROPERTIES FOR BATCH SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
specbatchdir	on off		Specify different directory for batch process than used by the current process.
specserverdir	on off	off	Specify server directory path.
synchaccumprobetable	on off	on	Synchronize accumulated probe table.
synchsolutions	on off	off	Synchronize solutions.

TABLE 6-115: PROPERTIES FOR REMOTE AND CLOUD ACCESS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
filetransfercmd	none scp user	none	Command transfer file.
filetransferfromusercmd	String		Command to transfer files from remote location.
filetransfertousercmd	String		Command to transfer files to remote location.
remote	on off		Run on remote server.
remotecmd	none ssh user		Command to use when invoking a command on a remote server.
remotehosts	String		List of remote hostnames.
remoteos	native windows linux		OS used on remote hosts.
remoteusercmd	String		Command to use when invoking a command on a remote server.
scpkey	String		SCP key file.
scppath	String		Directory where SCP resides.
scpuser	String		Username used by SCP.
scpusercmd	String		Command for copying files to remote location.
sshcmd	ssh putty user		SSH command.
sshkey	String		SSH key file.
sshpath	String		Directory where SSH resides.
sshporthost	String		Port host.
sshports	String		Ports that should be forwarded by SSH.
sshuser	String		Username used by SSH.

TABLE 6-116: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
graphics	on off	off	Use graphics.
maxalive	Integer		Alive time (seconds).
maxallow	Integer	1	Maximum number of simultaneous jobs.
maxrestarts	Integer		Maximum number of job restarts.

Eigenfrequency

The Eigenfrequency study step is used to compute eigenmodes and eigenfrequencies of a linear or linearized model.

SYNTAX

```
model.study(stdname).create(fname, "Eigenfrequency");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-117: PROPERTIES FOR EIGENFREQUENCY STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
appnreigs	Integer	20	Approximate number of eigenfrequencies.
approxnep	tol fixed	tol	Taylor expansion truncation using a tolerance or a fixed truncation, when eigsolver is set to arpacknonlinear.
chkeigregion	on off	on	Perform consistency check.
eigli	Real scalar	0	Largest imaginary part.
eiglr	Real scalar	0	Largest real part.
eigmethod	manual region all	manual	Eigenfrequency search method; the all method finds all eigenvalues for a full matrix and can only be used for small eigenfrequency problems.
eigsi	Real scalar	0	Smallest imaginary part.
eigsolver	arpack feast lapack arpacknonlinear	arpack	Which eigenfrequency solver to use: ARPACK, FEAST, LAPACK (filled matrix), or ARPACK nonlinear.
eigsr	Real scalar	0	Smallest real part.
eigwhich	lm lr sr li si	lm	Eigenfrequency search method around shift.
expdegneq	Integer	3	Degree of the Taylor expansion, if approxnep is set to fixed.
linplistsolnum	Integer array	new int[] {1}	List of solutions to use for the linearization point, when linpsolnum is set to from_list.
linpmanualsolnum	Integer array	new int[] {1}	Solution number (index) to use for the linearization point, when linpsolnum is set to manual.
linpmethod	init sol	init	Method used for linearization point, which for sol is given by the linpsol property.
linpsol	zero solution object	zero	Linearization point solution.
linpsolnum	all auto first last from_list interp manual solution number	1	Indices to solutions to use as linearization points.
linpsoluse	current manual solution store tag	current	Subsolution that defines linearization point for the Eigenfrequency study step.
linpsolusesolnum	Integer	1	Index for the solution to use when linpsoluse is set to manual.
linpstudy	String	zero	Study that defines linearization point for the Eigenfrequency study step.
linpt	double	0	Interpolated point for the linearization point when linpsolnum is set to interp.
maxdegneq	Integer	5	The maximum degree for the Taylor expansion truncation, if approxnep is set to tol.
maxnreigs	Integer	200	Maximum number of eigenfrequencies.
neigs	Integer	6	Desired number of eigenfrequencies.

TABLE 6-117: PROPERTIES FOR EIGENFREQUENCY STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
neigsactive	on off	off	Set desired number of eigenfrequencies.
pdistrib	on off	off	Distribute parametric sweep.
scalingnep	double	1	Eigenvalue scaling factor, when eigsolver is set to arpacknonlinear.
shift	Complex scalar	0	Shift.
shiftactive	on off	off	Use shift.
toltruncationnep	double	1E-12	Tolerance for the Taylor expansion truncation, if approxnep is set to tol.
useparam	on off	off	Auxiliary sweep.

TABLE 6-118: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-119: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.

TABLE 6-119: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-120: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[] {"geom1", "mesh1", "geom2", "mesh2"}.

The following ARPACK properties are available when eig solver is set to arpack and useparam (auxiliary sweep) is on:

TABLE 6-121: ARPACK PROPERTIES FOR AUXILIARY SWEEPS

PROPERTY	VALUE	DEFAULT	DESCRIPTION
auxsweepeigval	default preveigval	default	ARPACK starting vector: the default or from previous eigenfrequencies.
auxsweepeigvalshift	average closest	closest	ARPACK shift from previous eigenfrequencies: average of converged eigenfrequencies or eigenfrequency closest to the previous shift, if auxsweepeigvec is set to preveigvec.
auxsweepeigvec	default preveigvec	default	ARPACK shift: as specified (default) or based on eigenfrequencies from last parameter (preveigvec).
auxsweepeigvecstart	sum closest	sum	ARPACK starting vector from previous eigenfrequencies: summation of all eigenvalues or eigenvector with eigenfrequency closest to the shift, if auxsweepeigval is set to preveigval.

For the adaptation and error estimates settings, see [Table 6-172](#).

Eigenvalue

The Eigenvalue study step is used to compute eigenmodes and eigenvalues of a linear or linearized model.

SYNTAX

```
model.study(stdname).create(fname, "Eigenvalue");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-122: PROPERTIES FOR THE EIGENVALUE STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
appnreigs	Integer	20	Approximate number of eigenvalues.
approxnep	tol fixed	tol	Taylor expansion truncation using a tolerance or a fixed truncation, when eigsolver is set to arpacknonlinear.
chkeigregion	on off	on	Perform consistency check.
eigli	Real scalar	0	Largest imaginary part.
eiglrr	Real scalar	0	Largest real part.
eigmethod	manual region all	manual	Eigenvalue search method; the all method finds all eigenvalues for a full matrix and can only be used for small eigenvalue problems.
eigsi	Real scalar	0	Smallest imaginary part.
eigsolver	arpack feast lapack arpacknonlinear	arpack	Which eigenvalue solver to use: ARPACK, FEAST, LAPACK (filled matrix), or ARPACK nonlinear.
eigsr	Real scalar	0	Smallest real part.
eigwhich	lm lr sr li si	lm	Eigenfrequency search method around shift.
expdegneq	Integer	3	Degree of the Taylor expansion, if approxnep is set to fixed.
linplistsolnum	Integer array	new int[] {1}	List of solutions to use for the linearization point, when linpsolnum is set to from_list.
linpmanualsolnum	Integer array	new int[] {1}	Solution number (index) to use for the linearization point, when linpsolnum is set to manual.
linpmethod	init sol	init	Method used for linearization point, which for sol is given by the linpsol property.
linpsol	zero solution object	zero	Linearization point solution.
linpsolnum	all auto first last from_list interp manual solution number	1	Indices to solutions to use as linearization points.
linpsoluse	current manual solution store tag	current	Subsolution that defines linearization point for the Eigenvalue study step.
linpsolusesolnum	Integer	1	Index for the solution to use when linpsoluse is set to manual.
linpstudy	String	zero	Study that defines linearization point for the Eigenvalue study step.
linpt	double	0	Interpolated point for the linearization point when linpsolnum is set to interp.
maxdegneq	Integer	5	The maximum degree for the Taylor expansion truncation, if approxnep is set to tol.
maxnreigs	Integer	200	Maximum number of eigenvalues.
neigs	Integer	6	Desired number of eigenvalues.
neigsactive	on off	off	Set desired number of eigenvalues.
pdistrib	on off	off	Distribute parametric sweep.
scalingnep	double	1	Eigenvalue scaling factor, when eigsolver is set to arpacknonlinear.

TABLE 6-122: PROPERTIES FOR THE EIGENVALUE STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
shift	Complex scalar	0	Shift.
shiftactive	on off	off	Use shift.
toltruncationnep	double	1E-12	Tolerance for the Taylor expansion truncation, if approxnep is set to tol.
useparam	on off	off	Auxiliary sweep.

TABLE 6-123: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-124: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.

TABLE 6-124: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-125: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[]{"geom1", "mesh1", "geom2", "mesh2"}.

The following ARPACK properties are available when eig solver is set to arpack and useparam (auxiliary sweep) is on:

TABLE 6-126: ARPACK PROPERTIES FOR AUXILIARY SWEEPS

PROPERTY	VALUE	DEFAULT	DESCRIPTION
auxsweepeigval	default preveigval	default	ARPACK starting vector: the default or from previous eigenvalues.
auxsweepeigvalshift	average closest	closest	ARPACK shift from previous eigenvalues: average of converged eigenvalues or eigenvalue closest to the previous shift, if auxsweepeigvec is set to preveigvec.
auxsweepeigvec	default preveigvec	default	ARPACK shift: as specified (default) or based on eigenvalues from last parameter (preveigvec).
auxsweepeigvecstart	sum closest	sum	ARPACK starting vector from previous eigenfrequencies: summation of all eigenvalues or eigenvector with eigenvalue closest to the shift, if auxsweepeigval is set to preveigval.

For the adaptation and error estimates settings, see [Table 6-172](#).

Stationary Then Eigenfrequency

The Stationary Then Eigenfrequency study step is primarily intended for use with optimization, see the description of the [Stationary Then Eigenfrequency](#) study step in the reference manual.

SYNTAX

```
model.study(stdname).create(fname, "StationaryEigenfrequency");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The available properties are identical to those of the [Eigenfrequency](#) study step.

Frequency Domain and Frequency-Domain Perturbation

A Frequency Domain study step is used to compute the response of a linear or linearized model subjected to harmonic excitation for one or several frequencies. A Frequency-Domain Perturbation study step is used for studying small oscillations about a biased solution (small-signal analysis).

SYNTAX

```
model.study(stdname).create(fname, "Frequency");  
model.study(stdname).create(fname, "Frequencylinearized");  
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available. The properties for linearization point are only available for the Frequency-Domain Perturbation study step (see [Table 6-133](#) below).

TABLE 6-127: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
loadparameters	String		Load parameter values.
plist	Real vector		Frequencies.
preusesol	no yes auto	no	Reuse solution from previous step. The default is auto for MeanEnergies and ReducedElectricField studies.
punit	String	Hz	Frequency unit.
stol	Positive scalar	0.001	Relative tolerance, is usestol is set to true.
usestol	boolean	false	Use a user-defined relative tolerance.

TABLE 6-128: PROPERTIES FOR RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probes	Vector of strings		Probes to use when probesel>manual.
probesel	all none manual	all	Probes to compute.

TABLE 6-129: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .

TABLE 6-129: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-130: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.
useinitsol	on off	off	User-controlled initial values for variables solved for.
usesol	on off	off	User-controlled values for variables not solved for.

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-131: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[] {"geom1", "mesh1", "geom2", "mesh2"}.

TABLE 6-132: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
awe	on off	off	Use asymptotic waveform evaluation.
awefunc	Vector of strings		AWE expressions.
pcontinuation	String		Continuation parameter when pcontinuationmode = manual.
pcontinuationmode	no last manual	last	Determines if a continuation sweep should be performed for one of the parameters in pname.
pdistrib	on off	off	Distribute parametric sweep.
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	String		Parameter units.
sweepstype	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
useparam	on off	off	Auxiliary sweep.

TABLE 6-133: LINEARIZATION POINT PROPERTIES FOR THE FREQUENCY-DOMAIN PERTURBATION STUDY STEP

PROPERTY	VALUE	DEFAULT	DESCRIPTION
linplistsolnum	Integer array	new int[] {1}	List of solutions to use for the linearization point, when linpsolnum is set to from_list.
linpmanualsolnum	Integer array	new int[] {1}	Solution number (index) to use for the linearization point, when linpsolnum is set to manual.
linpmethod	init sol	init	Method used for linearization point, which for sol is given by the linpsol property.
linpsol	zero solution object	zero	Linearization point solution.
linpsolnum	all auto first last from_list interp manual solution number	1	Indices to solutions to use as linearization points.
linpsoluse	current manual solution store tag	current	Subsolution that defines linearization point for the Eigenvalue study step.
linpsolusesolnum	integer		Index for the solution to use when linpsoluse is set to manual.
linpstudy	String	zero	Study that defines the linearization point for the Frequency-Domain Perturbation study step.
linpt	double	0	Interpolated point for the linearization point when linpsolnum is set to interp.

For the adaptation and error estimates settings, see [Table 6-172](#).

Frequency to Time FFT

A Frequency to Time FFT study step, which you can add to a time-dependent study, performs an inverse FFT (or, alternatively, the nonuniform Fourier transform) from the frequency domain (input) to the time domain (output). As the default solver it adds an FFT solver.

SYNTAX

```
model.study(stdname).create(fname, "FreqToTimeFFT");  
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-134: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
addstatsol	on off	off	Add stationary solution.
fftinputmethod	sol init	sol	Prescribe the input values using a solution or an initial expression.
fftinputstudy	String	current	Study that defines input values for the FFT study step.
fftoutrange	Numeric vector		Output times.
fftscaling	cont discrete	cont	Use a discrete or continuous scaling for the Fourier transform.
fftwinalpha	Real scalar	0.5	Window parameter for a Tukey window.
fftwincenterinv	Real scalar	50	Window center for a Gaussian window function.
fftwincutoff	Real scalar	1	Cutoff fraction for window function in [0, 1].
fftwindev	Real scalar	1	Standard deviation for a Gaussian window function.
fftwindowinv	on off	off	Use window function.
fftwinexpr	String	1	Expression for window function (when set to fromexpr). Can be expressed in terms of t, freq, niterFFTin, and niterFFTout (if applicable).
fftwinmaxinv	Real scalar	100	Maximum (end) value for window.
fftwinmininv	Real scalar	0	Minimum (start) value for window.
fftwintypeinv	fromexpr cutoff rectangle gauss hamming hanning blackman tukey	fromexpr	Method for window function.
statmanualsolnum	Integer	1	Index to stationary solution to add.
statmethod	sol init	sol	For an added stationary solution, use a solution or initial expression.
statsolnum	auto first last interp manual positive integer	auto	Selection method for stationary solution to add.
statstudy	String	zero	Study from which the added stationary solution is chosen.
statt	String		Specify time for interpolated solution to add.

TABLE 6-134: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
tunit	String	s	Time unit.
winpunit	String	Hz	Frequency unit for window.

TABLE 6-135: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-136: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.

TABLE 6-136: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-137: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[]{"geom1", "mesh1", "geom2", "mesh2"}.

Function Sweep

A Function Sweep study step is a special case of a Parametric Sweep study step, where the solver sweeps over functions defined under a Switch node in the Model Builder.

SYNTAX

```
model.study(stdname).create(fname, "FunctionSweep");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-138: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pcase	Vector with entries "all" and "user".	["all"]	Cases, one per function switch.
plistarr	Vector of integer vectors		Integer case numbers. One list of integers (for example, range(1, 1, 10)) per function switch.
pname	Vector of strings		Function switches.

TABLE 6-139: ADVANCED AND OTHER PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
accumtable	String	new	Accumulated probe table.
accumtableall	on off	on	Use all probes for the accumulated probe table.
defsolvergen	global first last all	global	Control when a solver sequence should be generated during a function sweep using global parameters or the first, last, or each parameter tuple.
filename	String		Filename.
keepsol	last all		Keep solutions (only the last or all solutions).

TABLE 6-139: ADVANCED AND OTHER PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pdistrib	true false	false	Distribute parametric sweep.
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probes	Vector of strings		Probes to use when probesel>manual.
probesel	all none manual	all	Probes to compute.
save	on off		Save each solution as a model file.
useaccumtable	on off	on	Use an accumulated probe table.

Material Sweep

A Material Sweep study step is a special case of a Parametric Sweep study step, where the solver sweeps over materials defined under a Switch node in the Model Builder.

SYNTAX

```
model.study(stdname).create(fname, "MaterialSweep");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-140: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plistarr	Vector of integer vectors		Integer case numbers. One list of integers (for example, range(1, 1, 10)) per material switch.
pname	Vector of strings		Material switches.
pcase	Vector with entries "all" and "user".	["all"]	Cases, one per material switch.

TABLE 6-141: ADVANCED AND OTHER PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
accumtable	String	new	Accumulated probe table.
accumtableall	on off	on	Use all probes for the accumulated probe table.
defsolvergen	global first last all	global	Control when a solver sequence should be generated during a material sweep using global parameters or the first, last, or each parameter tuple.
filename	String		Filename.
keepsol	last all		Keep solutions (only the last or all solutions).
pdistrib	true false	false	Distribute parametric sweep.
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probes	Vector of strings		Probes to use when probesel>manual.
probesel	all none manual	all	Probes to compute.
save	on off		Save each solution as a model file.
useaccumtable	on off	on	Use an accumulated probe table.

Model Reduction

Use a Model Reduction study step to produce reduced models.

A Model Reduction study step involves the following settings:

- 1 Selecting the model-reduction method to apply.
- 2 If applicable: Selecting the training data (Select or generate Study and Study step reference).
- 3 Defining the unreduced (source) model (Study and Study step reference).
- 4 If applicable: Defining the objective function representing a quality measure to minimize.
- 5 Selecting to create or update an instance of the reduced model under reduced models for online use.
- 6 If applicable: Specifying if the reduced model should be capable of reconstruction.
- 7 Defining reduced-model control inputs.
- 8 Defining reduced-model outputs.

SYNTAX

```
model.study(stdname).create(fname, "ModelReduction");  
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step for model reduction. The following general and common properties are available.

TABLE 6-142: GENERAL AND COMMON PROPERTIES FOR MODEL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
reducedModelType	modal awe pod	modal	The model-reduction method to use.
romdata	String	new	Tag of the target container for the reduced model (new for a new reduced model).
useExtraResultsCompileEquations	true false	true	When true, an extra Compile Equations study-step feature is added last in the default solver sequence, which includes the reduced-order model feature in the final solution output by the study so that the reduced-order model's outputs can be evaluated directly using this solution.

The following properties are available for both the modal and pod model-reduction methods.

TABLE 6-143: PROPERTIES FOR MODAL AND POD MODEL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
romReconstruct	true false	true	Enable reconstruction in the produced reduced model.
qoidescr	String array	Empty array	Descriptions for user-defined output expressions.
qoiexpr	String array	Empty array	Global expressions defining outputs.
qoiname	String array	Empty array	Variable names for outputs.
trainingStepConstr	auto none a study step	none	Study step for constraint modes.
trainingStudyConstr	none a study	none	Training study for constraint modes.

TABLE 6-143: PROPERTIES FOR MODAL AND POD MODEL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
useInput	A string array of even length	A string array of twice the length of all global reduced-model inputs features' name string-array property.	A string array whose even and odd elements are, respectively, the global reduced-model input variable names (see GlobalReducedModelInputs) and on (default) or off, depending on whether the variable should be used or not.
trainingExpression	A string array of even length	A string array of twice the length of all global reduced-model inputs features' name string-array property.	A string array whose even and odd elements are, respectively, the global reduced-model input variable names (see GlobalReducedModelInputs) and the corresponding training expressions around which the model reduction will linearize the model.
pname	String array	Empty array	Use this property to add global parameter names to use as reduced-model input variables. When set, the variables will be added to the name property of the first global reduced-model inputs feature (see GlobalReducedModelInputs) as well as to useInput with the associated use value on.
initval	String array	Empty array	Initial values for the current pname variables.

For the modal model-reduction method, the following properties are available.

TABLE 6-144: PROPERTIES FOR MODAL MODEL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
trainingStep	auto a study step	auto	An Eigenfrequency or Eigenvalue study step used to produce the basis of the reduced method.
trainingStudy	none a study	none	Study containing a study step compatible with training data.
unreducedModelStep	none a study step	none	A Frequency Domain; Frequency-Domain Perturbation; Frequency Domain, Modal; or Time Dependent study step. Used to define the model to reduce.
unreducedModelStudy	none a study	none	Study containing a study step for reduction.

For the awe model-reduction method, the following properties are available.

TABLE 6-145: PROPERTIES FOR AWE MODEL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
awefunc	String array	Empty array	Error expressions.
awefuncdesc	String array	Empty array	Description of the error variables.
awefuncscale	String array	Empty array	Scale of the error variables.
awefuncuse	String array	Empty array	Use for adaptation.
awevar	String array	Empty array	Model error/output variables.
rtol	double	0.01	Relative tolerance for adaptation.

TABLE 6-145: PROPERTIES FOR AWE MODEL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
unreducedModelStepAWE	none a study step	none	A Frequency Domain or Frequency-Domain Perturbation study step. Used to define the model to reduce.
unreducedModelStudyAWE	none a study	none	Study containing at least one compatible study step.

For the pod model-reduction method, the following properties are available.

TABLE 6-146: PROPERTIES FOR POD MODEL REDUCTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
truncationrtol	double	0.01	The relative truncation tolerance.
unreducedModelStepPOD	none a study step	none	A Frequency Domain; Frequency-Domain Perturbation; Frequency Domain, Modal; or Time Dependent study step. Used to define the model to reduce.
unreducedModelStudyPOD	none a study	none	Study containing at least one compatible study step.

Multigrid Level

A Multigrid Level study substep specifies the geometric multigrid level used by the study step (a Stationary study step, for example).

SYNTAX

```
model.study(stdname).create(fname, "Stationary");
model.study(stdname).feature(fname).mglevel().create(mglname);
model.study(stdname).feature(fname).mglevel(mglname).set(pname, value);
```

DESCRIPTION

Study step attribute.

The following properties are available.

TABLE 6-147: PROPERTIES FOR PHYSICS SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
discretization	Vector of strings		Select discretizations for physics interfaces. The length of the vector is two times the number of physics interfaces. Example value when two physics interfaces g and c are available is: new String[]{"g", "disc1", "c", "disc2"}.

TABLE 6-148: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[]{"geom1", "mesh1", "geom2", "mesh2"}.

Parametric Sweep

Use the Parametric Sweep study step when you want to find the solution to a sequence of stationary or time-dependent problems that arise when you vary some parameters of interest.

SYNTAX

```
model.study(stdname).create(fname, "Parametric");  
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-149: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
keepbeforeandafter	on off	off	Store solutions not only before but also after the stop condition has been fulfilled (only for parametric sweep study steps in job configurations).
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.
stopcond	String		Stop condition expression (only for parametric sweep study steps in job configurations).
sweepstype	filled sparse switch	sparse	Sweep type.

TABLE 6-150: ADVANCED AND OTHER PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
accumtable	String	new	Accumulated probe table.
accumtableall	on off	on	Use all probes for the accumulated probe table.
clearprevrom	true false	true	Clear previously generated reduced-order models.
defsolvergen	global first last all	global	Control when a solver sequence should be generated during a parametric sweep using global parameters or the first, last, or each parameter tuple.
filename	String		Filename.
keepgenrom	true false	true	Keep the generating reduced-order model.
keeprom	on off	on	Keep reduced-order models.
keepsol	last all		Keep solutions (only the last or all solutions).
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probes	Vector of strings		Probes to use when probesel=manual.
probesel	all none manual	all	Probes to compute.
reusesol	true false	false	Reuse solution from previous step.

TABLE 6-150: ADVANCED AND OTHER PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
romtagsuffix	parindex parnameval parnamevalnz	parnameval	Add as suffix to tag: index, parameter name and value (the default), or name and nonzero value.
save	on off		Save each solution as a model file.
switchcase	Vector with entries "all" and "user".	{"all"}	Cases, one per parameter switch.
switchlistarr	Vector of integer vectors		Integer case numbers. One list of integers (for example, range(1, 1, 10)) per parameter switch.
switchname	Vector of strings		Parameter switches.
useaccumtable	on off	on	Use an accumulated probe table.

TABLE 6-151: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pdistrib	true false	false	Distribute parametric sweep.
paramselect	auto off	auto	Use parametric solver.

Ray Tracing

The Ray Tracing study step is a special case of the Time Dependent study step. It includes additional options for computing ray paths. The Ray Tracing study step is available with the Acoustics Module or the Ray Optics Module.

SYNTAX

```
model.study(stdname).create(fname, "RayTracing");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-152: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
charvel	String	c_const	Group velocity used to convert path lengths to solution times when timestepspec=specifylength.
l1list	Numeric vector		List of path lengths when timestepspec=specifylength.
lunit	String	m	Length unit when timestepspec=specifylength.
raystopcond	nostop noactive rayintensity		Automatic stop condition in the default solver sequence.
rtol	Positive scalar	0.01	Relative tolerance, if usertol is on.
thresholdintensity	String	1[W/m^2]	Threshold intensity for automatic stop conditions when raystopcond=rayintensity.
timestepspec	specifytime specifylength	specifytime	Determines whether the time intervals are entered directly or in terms of a path length.
t1list	Numeric vector		Time list when timestepspec=specifytime.

TABLE 6-152: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
tunit	String	s	Time unit when timestepspec=specifytime.
usertol	on off	off	Physics-controlled or user-defined tolerance.

TABLE 6-153: PROPERTIES FOR PLOT RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probefreq	pout psteps	pout	When to update probes.
probes	Vector of strings		Probes to use when probesel>manual.
probesel	all none manual	all	Probes to compute.
plotfreq	tout tsteps	tout	Where to update plot.

TABLE 6-154: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-155: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.

TABLE 6-155: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.
useinitisol	on off	off	User-controlled initial values for variables solved for.
usesol	on off	off	User-controlled values for variables not solved for.

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-156: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[] {"geom1", "mesh1", "geom2", "mesh2"}.

TABLE 6-157: PROPERTIES FOR MESH ADAPTATION AND STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapgeom	String		Name of geometry to use.
adaption	on off	off	Adaptive mesh refinement.
adaptselection	selection	all()	The selection for the adaptation (the default is the entire geometry). See Selections .
autoremesh	on off	off	Automatic remeshing.
autoremeshgeom	String		Name of geometry to use.
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.

TABLE 6-157: PROPERTIES FOR MESH ADAPTATION AND STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
punit	Vector of strings		Parameter units.
sweepstype	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
useparam	on off	off	Auxiliary sweep.

Schrödinger–Poisson

The Schrödinger–Poisson study step is a special case of the Eigenfrequency study step that is used to model bidirectionally coupled Schrödinger–Poisson systems. It is available with the Semiconductor Module. It is similar to the Eigenvalue study step but has an additional section called Iterations, which determines the behavior of the iterative solver loop for self-consistent modeling of the Schrödinger–Poisson system.

SYNTAX

```
model.study(stdname).create(fname, "SchrodingerPoisson");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-158: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
appnreigs	Integer	20	Approximate number of eigenfrequencies.
chkeigregion	on off	on	Perform consistency check.
eigli	Real scalar	0	Largest imaginary part.
eiglr	Real scalar	0	Largest real part.
eigmethod	manual region all	manual	Eigenvalue search method; the all method finds all eigenvalues for a full matrix and can only be used for small eigenfrequency problems.
eigsi	Real scalar	0	Smallest imaginary part.
eigsr	Real scalar	0	Smallest real part.
eigwhich	lm lr sr li si	lm	Eigenfrequency search method around shift.
maxnreigs	Integer	200	Maximum number of eigenfrequencies.
neigs	Integer	6	Desired number of eigenfrequencies.
neigsactive	on off	off	Set desired number of eigenfrequencies.
shift	Complex scalar	0	Shift.
shiftactive	on off	off	Use shift.

TABLE 6-159: PROPERTIES FOR PLOT RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probefreq	pout psteps	pout	When to update probes.
probes	Vector of strings		Probes to use when probesel>manual.
probesel	all none manual	all	Probes to compute.
plotfreq	tout tsteps	tout	Where to update plot.

TABLE 6-160: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-161: PROPERTIES FOR ITERATIONS SECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
atolterm	Positive real number	1E-6	Absolute tolerance for termination of the solver loop when method = minimization_of_global_variable.
expr	String	1	Global expression used to compute relative error when method!=iterations.
iter	Positive integer	5	Number of iterations of the solver loop when method=iterations.
maxiter	Positive integer	25	Maximum number of iterations in the solver loop when method!=iterations.
method	minimization_of_global_variable convergence_of_global_variable iterations	iterations	Choose whether termination of the solver loop is based on the convergence of a global variable, minimization of a global variable or a fixed number of iterations.
miniter	Positive integer	1	Minimum number of iterations in the solver loop when method!=iterations.
rtolterm	Positive real number	0.001	Relative tolerance for termination of the solver loop when method = convergence_of_global_variable.
rtolthresh	Positive real number	1	Threshold used to avoid division by zero while computing the relative error when method = convergence_of_global_variable.

TABLE 6-162: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.
useinitsol	on off	off	User-controlled initial values for variables solved for.
usesol	on off	off	User-controlled values for variables not solved for.

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-163: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: <code>new String[] {"geom1", "mesh1", "geom2", "mesh2"}</code> .

TABLE 6-164: PROPERTIES FOR MESH ADAPTATION AND STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapgeom	String		Name of geometry to use.
adaption	on off	off	Adaptive mesh refinement.
adaptselection	selection	all()	The selection for the adaptation (the default is the entire geometry). See Selections .
autoremesh	on off	off	Automatic remeshing.
autoremeshgeom	String		Name of geometry to use.
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.
sweepstype	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
useparam	on off	off	Auxiliary sweep.

Sensitivity

Use a Sensitivity study step to add sensitivity analysis to a study.

SYNTAX

```
model.study(stdname).create(fname, "Sensitivity");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-165: PROPERTIES FOR SENSITIVITY METHOD.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
descr	String		Objective function description.
gradientMethod	adjoint forward	adjoint	Gradient method.
gradientStep	String		Reference to study step.
objectiveActive	Vector with entries "on" and "off".	Vector with only "on".	Controls which objective functions from the physics interfaces that are active.
optobj	String		Objective function expression.

TABLE 6-166: PROPERTIES FOR CONTROL VARIABLES AND PARAMETERS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
controlVariableActive	Vector with entries "on" and "off".	Vector with only "on".	Controls which control variables from the physics interfaces that are active.
initval	Vector of scalars		Initial values, one per parameter.
pname	Vector of strings		Parameter names.
scale	Vector of type valuetype		Scales, one per parameter.
valuetype	Vector with entries "real" and "complex"	Vector with only "real".	Value types, one per parameter.

Stationary

A Stationary study step is intended for a stationary or steady-state situation where you can use a stationary solver, including mesh adaptation and error estimation (see [Table 6-172](#)).

SYNTAX

```
model.study(stdname).create(fname, "Stationary");  
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-167: PROPERTIES FOR STUDY SETTINGS

PROPERTY	VALUE	DEFAULT	DESCRIPTION
geometricNonlinearity	boolean	false	Include geometric nonlinearity. Available if the license includes the Structural Mechanics Module, Acoustics Module, MEMS Module, or Multibody Dynamics Module.
stol	Positive scalar	0.001	Relative tolerance, is usestol is set to true.
usestol	boolean	false	Use a user-defined relative tolerance.

TABLE 6-168: PROPERTIES FOR RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probefreq	pout psteps	pout	When to update probes.
probes	Vector of strings		Probes to use when probesel=manual.
probesel	all none manual	all	Probes to compute.

TABLE 6-169: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .

TABLE 6-169: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-170: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.
useinitsol	on off	off	User-controlled initial values for variables solved for.
usesol	on off	off	User-controlled values for variables not solved for.

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-171: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[] {"geom1", "mesh1", "geom2", "mesh2"}.

TABLE 6-172: PROPERTIES FOR MESH ADAPTATION AND ERROR ESTIMATES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapgeom	String		Adaptation in geometry.
adapsolnum	Array of integers>0	1	Indices.
adaptselection	selection	all()	The selection for the adaptation (the default is the entire geometry). See Selections .
adderrestvars	errandres err none	errandres	Add variables for error estimation: error estimates and residuals (errandres), error estimates only (err), or no variables (none).
adjppr	on off	on	Adjoint solution error estimate (for adaptation). The on value corresponds to PPR for Lagrange , and the off value corresponds to Interpolation error in the user interface.
allowcoarsening	on off	on	Controls if the mesh can be coarsened by the general modification method (meshadaptmethod set to modify).
elementspar	Positive scalar	0.5	Element fraction (for adaptation).
elselect	globalmin worst elements	globalmin	Method for selecting elements to refine (for adaptation).
erredim	domain boundary edge	domain	Geometric entity selection for the adaptation. edge is only available in 3D.
errestandadap	none adaption errest	none	Perform adaptation or error estimation (or none).
errestim	12errest goalerrest	12errest for adaption; goalerrest for errest	Error estimate.
gfunc	String		Functional when goalfunctype is set to gman.
goalerrest	on off	off	Goal-oriented error estimation.
goalfuncpredef	gfint gf12 gf11 gflinf	gfint	Predefined functionals for the solution when goalfunctype is set to gfpredef: integral (the default), L2 norm, L1 norm, or approximate max norm.

TABLE 6-172: PROPERTIES FOR MESH ADAPTATION AND ERROR ESTIMATES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
goalfunctype	gfpredef gfman	gfpredef	Functional type: predefined (the default) or manual.
goalorientedtermination	off auto manual	off	Activate goal-oriented termination. See below for additional properties when set to manual.
goeloglevel	minimal normal detailed	normal	Level of information in the log for adaptation using goal-oriented termination.
goeplot	boolean	true	Output goal-oriented termination increments.
goetableid	table name new	new	Table to use for convergence information from adaptation using goal-oriented termination.
globalminpar	Positive scalar	1.7	Element growth rate (for adaptation).
l2scale	String	1	Scaling factor.
l2staborder	String	2	Stability estimate derivative order
maxrefinement	scalar integer	5	Maximum number of element refinements (available when meshadaptmethod is not rebuild).
maxscale	Double	3	Maximum coarsening factor (for adaptation).
maxt	Double	1E7	Maximum number of elements (for adaptation).
meshadaptmethod	modify rebuild regular longest	longest	The refinement method for mesh adaptation (general mesh modification, rebuild mesh, regular refinement, or longest edge refinement)
ngen	Scalar integer	2	Maximum number of refinements (for adaptation).
resorder	String	0	Residual order.
savesolsref	true false	true	Save solution on every refined mesh (for adaptation).
selection	first last all manual	last	Solution selection: the first or last solution, all solutions, or manual, using weights and solution number indices in adapsolnum. Eigenfrequency studies use all with weights set to 1, which then uses the first solution only.
weights	double[] (positive values)	1.0	Weight for each selected solution (for adaptation).
window	graphics new	new	Plot window to use for displaying converge of goal-oriented termination.
worstpar	positive scalar		Controls refinement if elselect=worst (for adaptation).

When the `goalorientedtermination` property is set to `manual`, you can add goal-oriented termination expression using the `setIndex()` method:

- `goalorientedterminationexpression`: The expression for goal-oriented termination.
- `goalorientedterminationtolerance`: The tolerance for the goal-oriented termination.
- `goalorientedtolerancetype`: The tolerance type: "relative" or "absolute".
- `goalorientedterminationactive`: If the goal-oriented expression is active or not (true or false).

TABLE 6-173: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>constraintgroup</code>	Matrix with "on"/"off" entries	Matrix with only "off"	Constraint group status (active or not). One row per load case.
<code>loadcase</code>	Vector of strings		Load cases.
<code>loadgroup</code>	Matrix with "on"/"off" entries	Matrix with only "off"	Load group status (active or not). One row per load case.
<code>loadgroupweight</code>	Real matrix	off	Weight of each load case (as a scalar number). One row per load case.
<code>pcontinuation</code>	String		Continuation parameter when <code>pcontinuationmode = manual</code> .
<code>pcontinuationmode</code>	no last manual	last	Determines if a continuation sweep should be performed for one of the parameters in <code>pname</code> .
<code>pdistrib</code>	on off	off	Distribute parametric sweep.
<code>plistarr</code>	Real matrix		List of parameter values. One row of values per parameter name.
<code>pname</code>	Vector of strings		Parameter names.
<code>preusesol</code>	no yes auto	no	Determines how the converged solutions are reused in the parameter sweep.
<code>punit</code>	Vector of strings		Parameter units.
<code>sweepstype</code>	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
<code>useloadcase</code>	on off	off	Define load cases.
<code>useparam</code>	on off	off	Auxiliary sweep.

Surrogate Model Training

A Surrogate Model Training study step is intended for training a deep neural network and creating a surrogate model that can replace a full finite-element model.

SYNTAX

```
model.study(stdname).create(fname, "SurrogateModelTraining");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available:

TABLE 6-174: SURROGATEMODELTRAINING PROPERTIES

PROPERTY	VALUE	DEFAULT	DESCRIPTION
accumtable	new name of existing table	new	Output table to use for accumulated probe table.
accumtableall	boolean	false	Use all probes.
activation	string array	{"none", "tanh"}	Array of activation functions to use for each layer. Choose from none, relu, elu, sigmoid, tanh, softplus, leakyrelu, and gelu.
adpevals	nonnegative integer	10,000	Surrogate evaluations for optimization, when gpadpoptmethod is set to montecarlo.
automatictraining	boolean	false	Train all the generated surrogate models after the design of experiments.
basefilepath	string		File path to where all the external files will be created.
clearsynchdata	boolean	on	Clear the synchronization files left by batch jobs. Log files, .status, .recovery, and batch job MPH files are not removed. Only used when Surrogate Model Training is combined with a Batch or Cluster Computing node
computeaction	recompute append	recompute	Compute and build surrogate model (recompute), or improve and build it (append).
convinfo	of on detailed	on	Surrogate model training log level.
correlatedinput	boolean	false	Set to true if the input parameters are correlated.
covfunction	se matern32 matern52 nn	matern32	Type of covariance function to use when surrogatemodel is set to gp. Use se for Squared exponential, matern32 for Matérn 3/2, matern52 for Matérn 5/2 and nn for Single-layer neural network.
defsolvergen	global all	global	Use global parameters or each parameter tuple for the default solver sequence generation.
distributionselection	string array with keys col1, col2, col3, ... and values uniform, normal, lognormal, gamma, beta, weibull, or gumbel	{}	The distributions to use for each input parameter as an array of alternating keys and values.
errorhandling	immediate later	immediate	Use immediate to stop immediately if there is an error, or use later to skip problematic parameters.

TABLE 6-174: SURROGATEMODELTRAINING PROPERTIES

PROPERTY	VALUE	DEFAULT	DESCRIPTION
file	string map with keys 1, 2, 3, ... and values embedded or external	{}	Whether each quantity of interest should be stored embedded or external as an alternating array of keys and values.
filename	string array with keys 1, 2, 3, ... and valid filenames for the values	{}	The filename for each quantity of interest as an alternating array of keys and values.
funcname	string array with keys 1, 2, 3, ... and valid function names for the values	{}	The function name for each quantity of interest as an alternating array of keys and values.
geometrysampling	string array with keys 1, 2, 3, ... and values none or name of existing geometry sampling node	{}	Which Geometry Sampling node to use for each quantity of interest as an alternating array of keys and values.
globaldnnfunction	string array with keys 1, 2, 3, ... and values new or name of existing function	{}	Deep Neural Network function to use, when <code>surrogateModel</code> is set to <code>dnn</code> for each quantity of interest as an alternating array of keys and values.
globalgppfunction	string array with keys 1, 2, 3, ... and values new or name of existing function	{}	Gaussian Process function to use, when <code>surrogateModel</code> is set to <code>gp</code> for each quantity of interest as an alternating array of keys and values.
globalleastsquarefitfunction	string array with keys 1, 2, 3, ... and values new or name of existing function	{}	Least-Squares Fit function to use, when <code>surrogateModel</code> is set to <code>lsq</code> for each quantity of interest as an alternating array of keys and values.
globalpcefuction	string array with keys 1, 2, 3, ... and values new or name of existing function	{}	Polynomial Chaos Expansion (PCE) function to use, when <code>surrogateModel</code> is set to <code>pce</code> for each quantity of interest as an alternating array of keys and values.
gpadoptmethod	direct montecarlo	direct	Optimization method for error estimation, when <code>surrogateModel</code> is set to <code>gp</code> .
includeintraining	string matrix of on or off	{{}}	For each quantity of interest (row) and study-input parameter (column) whether the study-input parameter name should be included as input of the surrogate models or not.
innermostparameter	string array with keys 1, 2, 3, ... and values sum, min, max, last, first, all, or interp (time-dependent study only)	{}	The selection for the innermost parameter for each quantity of interest as an alternating array of keys and values.

TABLE 6-174: SURROGATEMODELTRAINING PROPERTIES

PROPERTY	VALUE	DEFAULT	DESCRIPTION
interp _{times}	string array with keys 1, 2, 3, ... and interpolation time expressions for values	{}	The interpolation times to sample for when <code>innermostparameter</code> is set to <code>interp</code> for each quantity of interest as an alternating array of keys and values.
keep _{sol}	last all	last	Keep only last or all model evaluations in memory.
layer _{type}	Array of dense and input for the first (input) layer	{"input", "dense"}	The type of layer in the DNN.
lb _{selection}	string array with keys col1, col2, col3, ... and values between 0 and 1 as strings	{}	The lower bound for the CDF for each input parameter distribution if <code>lcdfselection</code> is set to <code>manual</code> as an alternating array of keys and values.
lcdf _{selection}	string array with keys col1, col2, col3, ... and values 0.3, 0.1, 0.01, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, or manual	{}	The lower bound for the CDF for each input parameter distribution as an alternating array of keys and values.
lsq _{expression}	string		The definition of the least-squares fit function.
lsq _{lowerbound}	string array	{}	The lower bound for each least-squares parameter.
lsq _{parameternames}	string array	{}	The name for each least-squares parameter.
lsq _{parametervalues}	array of doubles	{}	The value for each least-squares parameter.
lsq _{scale}	string array	{}	The scaling for each least-squares parameter.
lsq _{upperbound}	string array	{}	The upper bound for each least-squares parameter.
max _{gpevals}	nonnegative integer	10,000	Maximum surrogate evaluations for optimization, when <code>gpadoptmethod</code> is set to <code>direct</code> .
max _{gpiters}	nonnegative integer	500	Maximum number of optimization iterations, when <code>gpadoptmethod</code> is set to <code>direct</code> .
mean _{function}	const linear quadratic	const	Defines the overall trend of the trained function(s) when <code>surrogatemodel</code> is set to <code>gp</code> .
nsolvenon _{adp}	positive integer	20	Number of input points when <code>computeaction</code> is set to <code>recompute</code> .
nsolvenon _{adpimprove}	positive integer	10	Number of input points when <code>computeaction</code> is set to <code>append</code> .
obj _{grp}	new name of table group	new	The output table group, when <code>surrogatemodel</code> is set to <code>none</code> .
out _{features}	array of positive integers	{0,0}	The number of output features for each layer in the DNN.

TABLE 6-174: SURROGATEMODELTRAINING PROPERTIES

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outputtable	string array with keys 1, 2, 3, ... and values new or name of existing table	{}	Output table to use, when surrogatemodel is set to none for each quantity of interest as an alternating array of keys and values.
parameterselection	string matrix of last or all	{{}}	Specify for each quantity of interest (row) and study-input parameter (column) the selection for the study-input parameter.
paramfilename	on index	index	Add parameter names and values or shorter indices to the filename. Only used when Surrogate Model Training is combined with a Batch or Cluster Computing node
pcesettings	auto manual	auto	PCE settings method. If set to auto (the default), the training will automatically determine the required polynomial degree needed to obtain suitable accuracy. If set to manual, the maximum polynomial degree is determined by the polydegreespce and qnorm settings.
pdistrib	boolean	false	Distribute model evaluation.
pname	string array	{}	Array of input parameter names.
polydegreespce	positive integer	30	Maximum polynomial degree, when pcesettings is set to manual.
punitselection	string array with keys col1, col2, col3, ... and units for the values	{}	The unit for each input parameter as an alternating array of keys and values.
qoconfigurestudyinput	string array with keys 1, 2, 3, ... and values reduce or configure	{}	For each quantity of interest, specify if it is global (reduce) or nonglobal (configure) as an array of alternating keys and values.
qoiexpression	string array	{}	Array of expressions for each quantity of interest.
qoisolutionindv	string array with keys 1, 2, 3, ... and values auto, sum, min, max, last, or first.	{}	Individual solution selection for each quantity of interest as an alternating array of keys and values.
qoisolutionouterindv	string array with keys 1, 2, 3, ... and values sum, min, or max.	{}	Individual outer solution selection for each quantity of interest as an alternating array of keys and values.
qnorm	double (0–1)	0.5	Q norm, when pcesettings is set to manual.
randseed	integer	1	Initial random seed when useseed is set to manual.
reusesol	boolean	false	Reuse solution from previous step.
s1selection	string array with keys col1, col2, col3, ... and double values as strings	{}	The first parameter for each input parameter distribution as an alternating array of keys and values.

TABLE 6-174: SURROGATEMODELTRAINING PROPERTIES

PROPERTY	VALUE	DEFAULT	DESCRIPTION
s2selection	string array with keys col1, col2, col3, ... and double values as strings	{}	The second parameter for each input parameter distribution as an alternating array of keys and values.
savefile	boolean	off	Output model to file.
savemodelaftersynch	boolean	off	Save the model to file before removing all synchronization data. Only used when Surrogate Model Training is combined with a Batch or Cluster Computing node
studyinputparamname	string array	{}	Array of study-input parameters.
surrogatemodel	none gp pce dnn lsq	none	Use gp (Gaussian Process), pce (Polynomial Chaos Expansion), dnn (Deep Neural Network), or lsq (Least-squares fit) to set up surrogate models after the design of experiments, or use none if no surrogate models should be created.
synchsolutions	on off	off	Synchronize solutions from the batch processes to the COMSOL Desktop. Only used when Surrogate Model Training is combined with a Batch or Cluster Computing node
uboundselection	string array with keys col1, col2, col3, ... and values between 0 and 1 as strings	{}	The upper bound for the CDF for each input parameter distribution if ucdfselection is manual as an alternating array of keys and values.
ucdfselection	string array with keys col1, col2, col3, ... and values 0.7, 0.9, 0.99, 1-1e-3, 1-1e-4, 1-1e-5, 1-1e-6, 1-1e-7, or manual	{}	The upper bound for the CDF for each input parameter distribution as an alternating array of keys and values.
useaccumtable	boolean	false	Use accumulated probe table.
useseed	automatic manual currenttime	automatic	Random seed type for input parameters sampling.

Time Dependent

A Time Dependent study step is intended for simulations where field variables vary over time and you can use a time-dependent solver.

SYNTAX

```
model.study(stdname).create(fname, "Transient");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-175: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
tlist	Numeric vector		Time list.
usertol	on off	off	Physics-controlled or user-defined tolerance.
rtol	Positive scalar	0.01	Relative tolerance, if usertol is on.
tunit	String	s	Time unit.

TABLE 6-176: PROPERTIES FOR PLOT RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probefreq	pout psteps	pout	When to update probes.
probes	Vector of strings		Probes to use when probesel=manual.
probesel	all none manual	all	Probes to compute.
plotfreq	tout tsteps	tout	Where to update plot.

TABLE 6-177: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-178: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.

TABLE 6-178: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.
useinitisol	on off	off	User-controlled initial values for variables solved for.
usesol	on off	off	User-controlled values for variables not solved for.

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-179: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[] { "geom1", "mesh1", "geom2", "mesh2" }.

TABLE 6-180: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
autoremesh	on off	off	Automatic remeshing.
autoremeshgeom	String		Name of geometry to use.
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.

TABLE 6-180: PROPERTIES FOR STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sweepstype	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
useparam	on off	off	Auxiliary sweep.

TABLE 6-181: PROPERTIES FOR TIME-DEPENDENT MESH ADAPTATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapgeom	String		Adaptation in geometry.
adapsolnum	Array of integers>0	1	Indices.
adaptselection	selection	all()	The selection for the adaptation (the default is the entire geometry). See Selections .
timeadaption	on off	off	Adaptive mesh refinement.

Time Discrete

A Time Discrete study step adds a Time Discrete Solver. Use it for performing time-dependent analysis using the projection method.

SYNTAX

```
model.study(stdname).create(fname, "TimeDiscrete");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-182: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
tlist	Numeric vector		Time list.
usertol	on off	off	Physics-controlled or user-defined tolerance.
rtol	Positive scalar	0.01	Relative tolerance, if usertol is on.
tunit	String	s	Time unit.

TABLE 6-183: PROPERTIES FOR PLOT RESULTS WHILE SOLVING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
plot	on off	off	Plot while solving.
plotgroup	String	default	Plot group to use for plot while solving.
probefreq	pout psteps	pout	When to update probes.
probes	Vector of strings		Probes to use when probesel=manual.
probesel	all none manual	all	Probes to compute.
plotfreq	tout tsteps	tout	Where to update plot.

TABLE 6-184: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .

TABLE 6-184: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvanceddisable	on off	off	See Physics and Variables Selection .

TABLE 6-185: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
initmethod	init sol	init	Method for initial values for variables solved for.
initstudy	String		Reference to study or "zero" for zero solution.
manualsolnum	Vector of integers	[1]	Index to solution for initial value for variables solved for.
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).
solnum	auto first last interp manual positive integer	auto	Solution selection of initial values for variables solved for.
t	String		Specify time for interpolated solution of initial value for variables solved for.

TABLE 6-185: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
useinit _{sol}	on off	off	User-controlled initial values for variables solved for.
uses _{sol}	on off	off	User-controlled values for variables not solved for.

See `setEntry()` for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-186: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries <code>geom1</code> and <code>geom2</code> are available is: <code>new String[] {"geom1", "mesh1", "geom2", "mesh2"}</code> .

TABLE 6-187: PROPERTIES FOR MESH ADAPTATION AND STUDY EXTENSIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
adapeom	String		Name of geometry to use.
adaption	on off	off	Adaptive mesh refinement.
adaptselection	selection	<code>all()</code>	The selection for the adaptation (the default is the entire geometry). See Selections .
plistarr	Real matrix		List of parameter values. One row of values per parameter name.
pname	Vector of strings		Parameter names.
punit	Vector of strings		Parameter units.
sweep _{type}	filled sparse	sparse	Sweep type: a filled or a sparse sweep.
useparam	on off	off	Auxiliary sweep.

Time to Frequency FFT

A Time to Frequency FFT study step, which you can add to a frequency-domain study, performs a forward FFT from the time domain (input) to the frequency domain (output). As the default solver it adds an FFT solver.

SYNTAX

```
model.study(stdname).create(fname, "TimeToFreqFFT");
model.study(stdname).feature(fname).set(pname, value);
```

DESCRIPTION

Study step.

The following properties are available.

TABLE 6-188: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
fftend _{time}	Real scalar	1.0	End time for the time interval.
fftinput _{method}	sol init	sol	Prescribe the input values using a solution or an initial expression.
fftinput _{study}	String	current	Study that defines input values for the FFT study step.

TABLE 6-188: PROPERTIES FOR STUDY SETTINGS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
fftmaxfreq	Real scalar	10	Maximum output frequency.
fftreastore	on off	on	Do not store negative frequencies for real input.
fftscaling	cont discrete	cont	Use a discrete or continuous scaling for the Fourier transform.
fftstarttime	Real scalar	0.0	Start time for the time interval.
fftwinalpha	Real scalar	0.5	Window parameter for a Tukey window.
fftwincenterfw	Real scalar	0.5	Window center for a Gaussian window function.
fftwincutoff	Real scalar	1	Cutoff fraction for window function in [0, 1].
fftwinddev	Real scalar	1	Standard deviation for a Gaussian window function.
fftwindowfw	on off	off	Use window function.
fftwinexpr	String	1	Expression for window function (when set to fromexpr). Can be expressed in terms of t, freq, niterFFTin, and niterFFTout (if applicable).
fftwinmaxfw	Real scalar	1	Maximum (end) value for window.
fftwinminfw	Real scalar	0	Minimum (start) value for window.
fftwintypefw	fromexpr cutoff rectangle gauss hamming hanning blackman tukey	fromexpr	Method for window function.
punit	String	Hz	Frequency unit.
tunit	String	s	Time unit.

TABLE 6-189: PROPERTIES FOR PHYSICS AND VARIABLES SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
activate	String Map		See Physics and Variables Selection .
activateCoupling	String Map		See Physics and Variables Selection .
activaterom	String Map		See Physics and Variables Selection .
disabledcommon	Reference Array		See Physics and Variables Selection .
disabledcoordinatesystems	Reference Array		See Physics and Variables Selection .
disabledcoupling	Reference Array		See Physics and Variables Selection .
disabledpair	Reference Array		See Physics and Variables Selection .
disabledphysics	Reference Array		See Physics and Variables Selection .
disabledreduced	Reference Array		See Physics and Variables Selection .
disabledvariables	Reference Array		See Physics and Variables Selection .
disableFrameControl	Reference Array		See Physics and Variables Selection .
discretization	String Map		See Physics and Variables Selection .
equationform	String Map		See Physics and Variables Selection .
equationform_freq	String Map		See Physics and Variables Selection .
equationform_freq_src	String Map		See Physics and Variables Selection .
reconstructors	String Map		See Physics and Variables Selection .
useadvancedisable	on off	off	See Physics and Variables Selection .

TABLE 6-190: PROPERTIES FOR VALUES OF DEPENDENT VARIABLES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
notlistsolnum	Vector of integers	[1]	Indices to selected solutions for values for variables not solved for.
notmanualsolnum	Vector of integers	[1]	Indices to solutions for values for variables not solved for.
notsolmethod	init sol	init	Method for values for variables not solved for.
notsolnum	auto all first last from_list interp manual positive integer	auto	Solution selection for values for variables not solved for.
notstudy	String		Reference to study or "zero" for zero solution.
nott	String		Specify time for interpolated solution of values for variables not solved for.
outputInterface	The interfaces to use as key		An output property (read only) for the physics interfaces in the output. It can be used as keys while setting the outputmap and outputselectionmap.
outputmap	String Map		See Store in Output for possible options and defaults (this property corresponds to the Output column). Possible values are physics, all, none, or selection.
outputselectionmap	String Map		An array of semicolon-separated strings of the output selections (specified with tags).

See [setEntry\(\)](#) for information about setting the values for a property that uses a String Map. If you use the `set(pname, value)` method, you have to provide all alternating property name and property value pairs.

TABLE 6-191: PROPERTY FOR MESH SELECTION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
mesh	Vector of strings		Select meshes for geometries. The length of the vector is two times the number of geometries. Example value when two geometries geom1 and geom2 are available is: new String[] {"geom1", "mesh1", "geom2", "mesh2"}.

Results

Detailed COMSOL[®] API reference information is included for the results features and utility methods for extracting, evaluating, and plotting results data from the simulations.

In this chapter:

- [About Results Commands](#)
- [Use of Datasets](#)
- [Extracting and Storing Plot Data](#)
- [Solution Selection](#)
- [Custom Color Tables](#)
- [Stepping Through Plots](#)
- [Special Formats and Symbols in Texts](#)
- [Results Commands](#)

About Results Commands

The commands in this chapter provide numerical results and plots for visualizing results. Note that `result` for the model object corresponds to Results in the Model Tree.

The following list includes the available commands for results evaluation and visualization (listed in alphabetical order):

- Animation
- Annotation
- AnnotationData
- Array 1D, Array 2D, Array3D
- ArrowData
- ArrowVolume, ArrowSurface, ArrowLine, ArrowPoint
- AvVolume, AvSurface, AvLine
- Average, Integral, Maximum, Minimum
- Beam
- Color
- Comparison
- Contour (Plot)
- Contour (Dataset)
- ContourSeries
- CoordSysLine, CoordSysSurface, CoordSysVolume, CoordSysPoint
- CutLine2D, CutLine3D
- CutPlane
- CutPoint1D, CutPoint2D, CutPoint3D
- Data
- Deform
- Directivity
- Edge2D, Edge3D
- Embed1D, Embed2D
- EnergyDecay
- ErrorBars
- Eval
- EvalAberration
- EvalGlobal
- EvalGlobalMatrix
- EvalGlobalSweep
- EvalPoint
- EvalPointMatrix
- EvaluationGroup
- Export
- Extrude1D, Extrude2D
- Filter (Dataset)
- Filter (Plot Attribute)
- Filter (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing)
- Filter (Table Graph)
- Function, Function2D, Function3D
- Global (Numerical)
- Global (Plot)
- Grain (1D Plot)
- Grain (Dataset)
- Grain (Evaluation)
- GrainBin
- GrainTrajectories
- GraphMarker
- Grid1D, Grid2D, Grid3D
- Height, AberrationHeight, HistogramHeight, TableHeight
- Histogram
- Image (Export)
- Image (Plot)
- ImageOverlay
- ImpulseResponse
- InterferencePattern
- Interp
- IntersectionPoint2D, IntersectionPoint3D
- IntVolume, IntSurface, IntLine
- Isosurface (Plot)
- Isosurface (Dataset)
- IsosurfaceSeries
- Join
- JointProbabilityDistribution
- KernelDensityEstimation
- LayeredMaterial
- LayeredMaterialSlice
- Line
- LineData
- LineGraph
- LineSegments
- Marker
- MassProperties
- MaterialAppearance
- MatrixHistogram

Results Commands (M to S)

- MaxMinVolume, MaxMinSurface, MaxMinLine, MaxMinPoint
- MaxVolume, MaxSurface, MaxLine, MinVolume, MinSurface, MinLine
- MeasureVolume, MeasureSurface, MeasureLine, MeasureDistance
- Mesh (Plot)
- Mesh (Dataset)
- Mesh (Export)
- MeshImportParameters
- Mirror2D, Mirror3D
- Multislice
- Nyquist
- OctaveBand
- OpticalAberration
- Parametric1D, Parametric2D
- ParCurve2D, ParCurve3D
- ParSurface
- Particle
- Particle (1D Plot)
- Particle (Dataset)
- Particle (Evaluation)
- ParticleBin
- ParticleMass
- ParticleTrajectories
- Partition
- Pellets
- PhasePortrait
- Pipe
- Plot
- PlotGroup1D, PlotGroup2D, PlotGroup3D
- PoincareMap
- Point
- PointData
- PointGraph
- PointTrajectories
- PolarGroup
- Polarization
- PrincipallLine, PrincipalSurface, PrincipalVolume
- RadiationPattern
- RadiationPattern (Export)
- Ray (1D Plot)
- Ray (Dataset)
- Ray (Evaluation)
- RayBin
- RayTrajectories
- Receiver2D, Receiver3D
- ReflectionGraph, ImpedanceGraph, AdmittanceGraph
- ResponseSpectrum2D, ResponseSpectrum3D
- Revolve1D, Revolve2D
- ScatterVolume, ScatterSurface
- Sector2D, Sector3D
- Selection (Dataset Attribute)
- Selection (Plot Attribute)
- Shell
- Slice
- SmithGroup
- SobolIndex
- Solution
- SolutionArray
- Spatial FFT
- Spot Diagram
- StdDevVolume, StdDevSurface, StdDevLine
- Streamline
- StreamlineMultislice
- StreamlineSurface
- Surface (Plot)
- Surface (Dataset)
- SurfaceData
- SurfaceSlit
- SurfaceSlit

Results Commands (T to Z)

- [Table](#)
- [Table \(Export\)](#)
- [Table \(Plot\)](#)
- [TableAnnotation](#)
- [TableArrow](#)
- [TableColor](#)
- [TableContour](#)
- [TableDOE](#)
- [TableErrorBars](#)
- [TableHistogram](#)
- [TablePoint](#)
- [TableSurface](#)
- [TableUQ](#)
- [TimeAverage, TimeIntegral](#)
- [ThroughThickness](#)
- [Touchstone](#)
- [Transformation2D, Transformation3D](#)
- [Transformation](#)
- [Transparency](#)
- [TubeData](#)
- [VisualEffects](#)
- [Volume](#)
- [Waterfall](#)
- [Whirl](#)

Commands Grouped by Function

PLOT ATTRIBUTES

FUNCTION	PURPOSE
Color	Color expression
Comparison	Compare data i graph plots to reference data in table graphs
Deform	Deformation
EnergyDecay	Energy decay for impulse response plots
ErrorBars	Error bars for 1D global and point graph plots
Export	Export expressions
Filter (Plot Attribute)	Filter
GraphMarker	Graph markers, such as maximum and minimum markers, for 1D graph plots
Height, AberrationHeight, HistogramHeight, TableHeight	Height for various plot types
ImageOverlay	Images overlaid in volume and surface plots
Marker	Maximum and minimum markers
MaterialAppearance	Use a material appearance in surface, volume, isosurface, slice, and many other types of plots
Selection (Plot Attribute)	Selection for various plot types
SolutionArray	Specify an array of solutions to use for a volume, surface, or line plot or a plot annotation
TableColor	Color expression for Table Arrow plots
TableErrorBars	Error bars for 1D table graph plots
Transformation	Transformation of various 2D and 3D plot types

FUNCTION	PURPOSE
Transparency	Transparency for various 3D plot types and 2D plots with a height expression
VisualEffects	Visual effects for various 3D plot types and 2D plots with a height expression

DATASETS

FUNCTION	PURPOSE
Array 1D, Array 2D, Array3D	Array datasets
Average, Integral, Maximum, Minimum	Evaluation datasets
Beam	Beam dataset
Contour (Dataset)	Contour dataset
CutLine2D, CutLine3D	Cut line datasets
CutPlane	Cut plane dataset
CutPoint1D, CutPoint2D, CutPoint3D	Cut point datasets
Edge2D, Edge3D	Edge datasets
Embed1D, Embed2D	Embed datasets
Extrude1D, Extrude2D	Extrusion datasets
Filter (Dataset)	Filter datasets
Grain (Dataset)	Grain dataset
GrainBin	Grain bin dataset
Grid1D, Grid2D, Grid3D	Grid datasets
IntersectionPoint2D, IntersectionPoint3D	Intersection point datasets
Isosurface (Dataset)	Isosurface dataset
Join	Join dataset for joining results from other datasets
KernelDensityEstimation	Kernel density estimation (KDE) dataset
LayeredMaterial	Layered shell dataset for layered material
Mesh (Dataset)	Mesh dataset
MeshImportParameters	Mesh import parameters as attribute to Filter and Partition datasets
Mirror2D, Mirror3D	Mirror dataset
Parametric1D, Parametric2D	Extends another dataset by using a parameter as dimension
ParCurve2D, ParCurve3D	Parametric curve datasets
ParSurface	Parametric surface dataset
Particle (Dataset)	Particle dataset
ParticleBin	Particle bin dataset
Partition	Dataset for partitioning a domain with respect to a set of isolevels
Pipe	Pipe dataset
Ray (Dataset)	Ray dataset
RayBin	Ray bin dataset
Receiver2D, Receiver3D	Receiver datasets
ResponseSpectrum2D, ResponseSpectrum3D	Response spectrum datasets
Revolve1D, Revolve2D	Revolution datasets
Sector2D, Sector3D	Sector symmetry datasets
Shell	Shell dataset for visualizing the top and bottom of a shell
Solution	Solution dataset
Surface (Dataset)	Surface dataset

FUNCTION	PURPOSE
TimeAverage , TimeIntegral	Time average and time integral datasets
Transformation2D , Transformation3D	Transformation 2D and transformation 3D datasets.

EXPORT FEATURES

FUNCTION	PURPOSE
Animation	Animation export or player
Data	Data export from datasets
Image (Export)	Image export
Mesh (Export)	Mesh export from datasets
Plot	Plot export
RadiationPattern (Export)	Radiation pattern export
Table (Export)	Table export
Touchstone	Touchstone export

NUMERICAL RESULTS

FUNCTION	PURPOSE
AvVolume , AvSurface , AvLine	Average
Eval	Generic evaluation
EvalAberration	Aberration evaluation
EvalGlobal	Global evaluation
EvalGlobalMatrix	Global matrix evaluation
EvalGlobalSweep	Global evaluation sweep
EvalPoint	Evaluation at points
EvalPointMatrix	Matrix evaluation at points
EvaluationGroup	Evaluation group
Global (Numerical)	Generic evaluation
Interp	Interpolation
IntVolume , IntSurface , IntLine	Integration
MassProperties	Mass properties
MaxVolume , MaxSurface , MaxLine , MinVolume , MinSurface , MinLine	Maximum and minimum
MeasureVolume , MeasureSurface , MeasureLine , MeasureDistance	Measure volumes, surface area, line lengths, and distances between points
Particle (Evaluation)	Evaluation on particle trajectories
Ray (Evaluation)	Evaluation on ray trajectories
SystemMatrix	Evaluation of system matrix

PLOTS

FUNCTION	PURPOSE
Annotation	Annotation plot
AnnotationData	Annotation data plot
ArrowData	Arrow data plot
ArrowVolume , ArrowSurface , ArrowLine , ArrowPoint	Arrow plots
Contour (Plot)	Contour plot
ContourSeries	Contour series plot
CoordSysLine , CoordSysSurface , CoordSysVolume , CoordSysPoint	Coordinate system plot
Directivity	Directivity plot
Function , Function2D	Function 1D and 2D plot
Global (Plot)	Global plot
Grain (1D Plot)	Grain plot
Grain (Dataset)	Grain dataset
Histogram	Histogram plot
Image (Plot)	Image plot
ImpulseResponse	Impulse response plot
InterferencePattern	Interference pattern plot
Isosurface (Plot)	Isosurface plot
IsosurfaceSeries	Isosurface series plot

JointProbabilityDistribution	Joint probability distribution plot
LayeredMaterialSlice	Layered shell slice plot
Line	Line plot
LineData	Line data plot
LineGraph	Line graph plot
LineSegments	Add line segments to 1D plots
MatrixHistogram	Matrix histogram plot
MaxMinVolume, MaxMinSurface, MaxMinLine, MaxMinPoint	Max/min marker plots
Mesh (Plot)	Mesh plot
Multislice	Multislice plot
Nyquist	Nyquist plot
OctaveBand	Octave band plot
OpticalAberration	Optical aberration plot for rays.
Particle	Massless particle tracing plot
Particle (1D Plot)	Particle plot
ParticleMass	Particle tracing plot with mass
ParticleTrajectories	Plot the data from a particle dataset
PhasePortrait	Phase portrait plot
PoincareMap	Plot a Poincaré map
Point	Point plot
PointData	Point data plot
PointGraph	Point graph plot
PointTrajectories	Point trajectories plot
Polarization	Polarization plot
Principalline, PrincipalSurface, PrincipalVolume	Principal stress/strain plots
Ray (1D Plot)	Ray plot
RayTrajectories	Plot the data from a ray dataset
ScatterVolume, ScatterSurface	Scatter plots
Slice	Slice plot
SobolIndex	Sobol index plot
Spot Diagram	Spot diagrams
Streamline	Streamline plot
StreamlineMultislice	Streamline plot on slices in 3D
StreamlineSurface	Streamline plot on surfaces in 3D
Surface (Plot)	Surface plot
SurfaceData	Surface data plot
SurfaceSlit	Surface slit plot
Table (Plot)	Table plot
TableAnnotation	Table annotation plot
TableContour	Table contour plot
TableHistogram	Table histogram plot
TablePoint	Table point plot
TableSurface	Table surface plot

ThroughThickness	Through-thickness plot
TubeData	Tube data plot
Volume	Volume plot
Waterfall	Waterfall plot
Whirl	Whirl plot for rotordynamics

TABLES

FUNCTION	PURPOSE
Table	Table containing real and imaginary data
TableDOE	Table containing data for design of experiments (DOE)
TableUQ	Table containing data for uncertainty quantification (UQ)

	Results Analysis and Plots in the <i>COMSOL Multiphysics Reference Manual</i>
---	---

Use of Datasets

Table 7-1 shows applicability of dataset output as datasets input for the most widely used datasets. Rows correspond to dataset output and columns to dataset input. Check marks appear if the row dataset output is accepted as input to the column dataset. *evaluation* indicates Average, Integral, Maximum, and Minimum datasets. Note that not all available datasets are included in this table.

TABLE 7-1: DATASET INPUT VERSUS OUTPUT.

DATASETS Input vs. output	ARRAY 1D	ARRAY 2D	ARRAY 3D	CONTOUR	CUTLINE2D	CUTLINE3D	CUTPLANE	CUTPOINT1D	CUTPOINT2D	CUTPOINT3D	EVALUATION	INTERSECTION POINT	ISOSURFACE	JOIN	MESH	MIRROR2D	MIRROR3D	PARAMETRIC1D	PARAMETRIC2D	PARCURVE2D	PARCURVE3D	PARSURFACE	PARTICLE	RAY	REVOLVE1D	REVOLVE2D	SOLUTION
Array1D	√							√																			
Array2D		√		√	√				√		√			√		√										√	
Array3D			√								√		√	√			√										
Contour											√																
CutLine2D		√						√			√					√									√		
CutLine3D			√					√			√						√										
CutPlane		√	√	√	√				√		√					√	√			√						√	
CutPoint1D	√										√																
CutPoint2D		√									√					√											
CutPoint3D			√								√						√										
Edge2D		√														√											√
Edge3D			√														√										√
<i>evaluation</i>											√																
Intersection Point2D/3D											√																
Isosurface			√	√							√						√										
Join																											
Mesh		√	√	√	√	√	√	√	√	√	√		√			√	√			√	√	√			√	√	
Parametric1D																											√
Parametric2D																											√
ParCurve2D		√					√	√			√		√			√											
ParCurve3D			√					√			√						√										
ParSurface		√	√	√	√				√		√					√	√			√							
Particle							√				√	√															
Ray							√				√	√															
Revolve1D		√		√	√				√		√					√				√							
Revolve2D			√			√	√			√	√		√				√				√	√					
Sector2D		√			√		√							√		√									√		√
Sector3D			√			√	√						√	√			√									√	√

TABLE 7-1: DATASET INPUT VERSUS OUTPUT.

DATASETS Input vs. output																				
	ARRAY ID	ARRAY2D	ARRAY3D	CONTOUR	CUTLINE2D	CUTLINE3D	CUTPLANE	CUTPOINT1D	CUTPOINT2D	CUTPOINT3D	EVALUATION	INTERSECTION POINT	ISOSURFACE	JOIN	MESH	MIRROR2D	MIRROR3D	PARAMETRIC1D	PARAMETRIC2D	PARCURVE2D
Solution		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓	✓		✓	✓	✓	✓	✓
Surface		✓		✓												✓				

Table 7-2 shows applicability of dataset output as input to plot features. Rows correspond to dataset output and columns to plot features. Check marks appear if the dataset output is accepted by the plot feature. Note that the table does not include all available datasets and plot types.

TABLE 7-2: DATASET OUTPUT AS INPUT TO PLOT FEATURES (SOME SPECIAL PLOT TYPES ARE NOT INCLUDED).

DATASET OUTPUT As input to plots																				
	ARROW	CONTOUR	GLOBAL	HISTOGRAM	INTERFERENCE PATTERN	ISOSURFACE	LINE	LINEGRAPH	MESH	MULTISLICE	PARTICLE	PARTICLE TRACING	PARTICLE TRAJECTORIES	PHASE PORTRAIT	POINTGRAPH	POINT TRAJECTORIES	RADIATION PATTERN	RAY	RAY TRAJECTORIES	SCATTER
Array2D	✓	✓		✓			✓													✓
Array3D	✓			✓		✓	✓			✓										✓
Contour	✓			✓			✓													
CutLine2D	✓			✓			✓	✓												✓
CutLine3D	✓			✓			✓	✓												✓
CutPlane	✓	✓		✓	✓		✓													✓
CutPoint1D				✓											✓	✓				
CutPoint2D				✓											✓	✓				
CutPoint3D				✓											✓	✓				
Edge2D				✓			✓	✓												
Edge3D				✓			✓	✓												
evaluation			✓	✓											✓					
IntersectionPoint2D				✓							✓		✓	✓		✓		✓	✓	
IntersectionPoint3D																				
Isosurface	✓	✓		✓			✓													✓
Join	✓	✓		✓		✓	✓	✓		✓					✓					✓
Mesh	✓	✓		✓		✓	✓	✓	✓	✓					✓					✓
Mirror2D	✓	✓		✓			✓													✓
Mirror3D	✓			✓		✓	✓			✓										✓
Parametric1D	✓	✓		✓			✓													✓
Parametric2D	✓	✓		✓		✓	✓			✓										✓
Particle			✓	✓							✓		✓	✓						
ParCurve2D	✓			✓			✓	✓												
ParCurve3D	✓			✓			✓	✓												
ParSurface	✓	✓		✓			✓													✓

TABLE 7-2: DATASET OUTPUT AS INPUT TO PLOT FEATURES (SOME SPECIAL PLOT TYPES ARE NOT INCLUDED).

DATASET OUTPUT As input to plots	ARROW		CONTOUR		GLOBAL		HISTOGRAM		INTERFERENCEPATTERN		ISOSURFACE		LINE		LINEGRAPH		MESH		MULTISLICE		PARTICLE		PARTICLETRACING		PARTICLETRAJECTORIES		PHASEPORTRAIT		POINTGRAPH		POINTTRAJECTORIES		RADIATION PATTERN		RAY		RAYTRAJECTORIES		SCATTER		SLICE		STREAMLINE		SURFACE		VOLUME		
Ray				✓	✓																				✓									✓	✓														
Revolve1D	✓	✓			✓						✓	✓																									✓			✓	✓								
Revolve2D	✓	✓			✓				✓	✓								✓																				✓		✓	✓	✓	✓	✓					
Sector2D	✓	✓			✓						✓	✓																											✓			✓	✓						
Sector3D	✓	✓			✓				✓	✓								✓																				✓			✓	✓	✓	✓	✓				
Solution	✓	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓						✓	✓	✓	✓								✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	
Surface	✓	✓			✓						✓																											✓		✓			✓						

Extracting and Storing Plot Data

This section describes how to retrieve results or plot data and how to control the update of plots and to store and clear plot data from the model directly through the COMSOL API.

- [Retrieving Plot Data](#)
- [Retrieving Numerical Results](#)
- [Updating Plots, Storing and Clearing Plot Data, and Previewing Datasets](#)

Retrieving Plot Data

Each plot is made up of one or more *groups* or parts. Most plots contain only one group, such as [Surface \(Plot\)](#) or [Line](#) plots, whereas for example [Slice](#) plots contain one group per slice. Retrieve the number of groups with the method

```
plot.getGroups(<renderIndex>)
```

where `plot` is any plot feature.

Retrieving the groups requires that you specify a *render index* for which you want the number of groups. Each plot is made up of one or more internal, or rendering, plot types. For example, particle tracing can contain both lines and spheres, which are separate rendering types. Most plot have only a single rendering type, in which case you can safely use 0 as the render index. The *group index* property indicates groups or parts within the rendering group index.

Extract the data contained in any plot feature using the following methods:

`p = plot.getVertices(<renderIndex>,<groupIndex>)` returns a float matrix containing one row for each dimension and one column for each vertex.

`t = getElements(<renderIndex>,<groupIndex>)` returns the indices to columns in `p` of a simplex mesh, each column in `t` representing a simplex.

`d = getData(<renderIndex>,<groupIndex>,<dataType>)` returns a float array containing the data for each point in `p`. The type of data to retrieve is given by the String `<dataType>:"Color", "Height",` and so forth.

To retrieve the available data types, use:

```
types = getDataTypes(<renderIndex>)
```

which returns a string array with the data types currently present in the plot.

For plot features, you can retrieve the point normals using the following method:

```
n = getNormals(<renderIndex>,<groupIndex>)
```

which returns a floating-point matrix of point normals with each column corresponding to a single vertex.

EXAMPLE

```
int renderDataGroups = plot.getRenderGroups();

for (int ri = 0; ri < renderDataGroups; ri++) {

    String[] dataTypes = plot.getDataTypes(ri);
    int plotDataGroups = plot.getGroups(ri);
    for (int gi = 0; gi < plotDataGroups; gi++) {
        p = plot.getVertices(ri,gi);
        t = plot.getElements(ri,gi);
    }
}
```

```

        //d = plot.getData(ri, gi, "Color");
        d = plot.getData(ri, gi, dataTypes[0]);
    }
}

```

RETRIEVING AXIS UNITS

Use `model.result(<feature>).getAxisUnits()` to return the units of the coordinate axes for the plot group to which the feature belong; null for 1D plot groups. This method returns a string array of length 2 or 3 containing LaTeX-formatted units.

RETRIEVING COLORS

The following methods are available for plot groups and plots:

```

double[] color = plot.getColor(<propertyname>, <useGraphicsTheme>)
double[] color = plot.getColor(<propertyname>, <colortheme>)

```

The return value is an array with the RGB values of the color.

The property name must be a property that defines a color. If the string value of the property is `fromtheme`, the color will be evaluated using the theme. If the string value is an explicit color, the RGB values for that color will be returned. In that case, the theme is not relevant.

In the first method, if `<useGraphicsTheme>` is set to `true`, it means that the model's graphics color theme will be used, and if it is set to `false`, it means that the model's image export color theme will be used.

Using the second method, you can specify the color theme to use, ignoring the model's theme.

RETRIEVING THE NUMBER OF PARAMETER VALUES THAT ARE AVAILABLE FOR STEPPING OVER

To retrieve the number of parameter values that are available for stepping over on a given level, returned as a nonnegative integer, use `np = plot.getStepCount(<level>)` or `np = plotGroup.getStepCount(<level>)`, where `<level>` is the level of a parametric sweep (0 is the top level). This information can be useful to, for example, manage sliders in apps.

Retrieving Numerical Results

It is possible to retrieve numerical results from any numerical feature, but there are a number of features especially designed for the COMSOL[®] API that are not present in the COMSOL Desktop[®]. These functions support multiple expressions, as well as some additional advanced properties, and a more convenient way to extract interpolated data. These two groups have slightly different access methods. Note that `numerical` in the model object corresponds to Derived Values in the Model Tree. In the API, code like `model.result().numerical(<ftag>)` returns objects of type `NumericalFeature`.

API-ONLY NUMERICAL FEATURES

TABLE 7-3: API-ONLY NUMERICAL FEATURE.

FUNCTION	PURPOSE
<code>Eval</code>	Generic evaluation
<code>Global (Numerical)</code>	Generic evaluation
<code>Interp</code>	Interpolation

`result = model.result().numerical(<ftag>).getData()` returns the real part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][coordinates]`.

`model.result().numerical(<ftag>).getData(<expressionIndex>)` returns the real part of the result for one expression, equivalent to `result[expressionIndex]`.

`result = model.result().numerical(<ftag>).getImagData()` returns the imaginary part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][coordinates]`.

`model.result().numerical(<ftag>).getImagData(<expressionIndex>)` returns the imaginary part of the result for one expression, equivalent to `result[expressionIndex]`.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. For models with outer solutions, `isComplex()` returns true if the result for the current value of `outersolnum` is complex.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution.

`model.result().numerical(<ftag>).getNData()` returns the number of points in the data vector.

`model.result().numerical(<ftag>).getCoordinates()` returns the coordinates of the evaluation or interpolation.

`model.result().numerical(<ftag>).getElements()` returns indices to columns in `p` of a simplex mesh.

`model.result().numerical(<ftag>).getVertexElements()` returns indices to mesh elements for each point.

More details can be found in the documentation for each feature type.

STANDARD NUMERICAL FEATURES

TABLE 7-4: STANDARD NUMERICAL FEATURES, ALSO PRESENT IN THE COMSOL MULTIPHYSICS UI.

FUNCTION	PURPOSE
EvalGlobal	Global evaluation
EvalPoint	Evaluation in points
IntVolume , IntSurface , IntLine	Integration

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to (`columnwise`) when `columnwise` is `false`. If `columnwise` is `true`, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate, columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is `true`, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` `true` and `columnwise` `false`.

`model.result().numerical(<ftag>).getComplex()` returns a complex-valued result (the real and imaginary values), as an array of length two, where the first element is the real part and the second element is the imaginary part. The second element is null if the return value is all-real. Data is ordered such that one row contains data for all solution numbers. This is identical to (`columnwise`) when `columnwise` is `false`. If `columnwise` is `true`, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getComplex(columnwise)` returns the real and imaginary part of a complex-valued result. If the Boolean `columnwise` is `false`, data is ordered such that each row contains the values for all solution numbers. If `true`, one column contains the values for all solution numbers.

`model.result().numerical(<ftag>).getComplex(<outersolnum>)` returns the real and imaginary part of a complex-valued result for the given outer solution number (the 1-indexed integer `<outersolnum>`).

`model.result().numerical(<ftag>).getComplex(columnwise,<outersolnum>)` returns the real and imaginary part of a complex-valued result for the given outer solution number in `<outersolnum>` using an order determined by the Boolean `columnwise` (see above).

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. For models with outer solutions, `isComplex()` returns true if the result is complex for any of the currently set `outersolnum`.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution (the 1-indexed integer `<outersolnum>`).

You can find more details in the documentation for each feature type.

Updating Plots, Storing and Clearing Plot Data, and Previewing Datasets

CONTROLLING AND CHECKING IF PLOTS ARE UPDATED

To specify if plots selected in the Model Builder only should be updated when the user explicitly requests it, use the following method:

```
model.result().setOnlyPlotWhenRequested(boolean value);
```

To check if plots selected in the Model Builder only should be updated when the user explicitly requests it, use the following method, which returns a Boolean value:

```
model.result().isOnlyPlotWhenRequested();
```

STORING AND CLEARING PLOT DATA

To avoid recreating plot data in a model, the plot data can be stored in the model for a plot group using the following property:

```
model.result("pg1").set("savedatainmodel", true);
```

which saves the plot data in the model for the plot group `pg1`. It can be useful for data that takes a long time to recompute but is not very large (such as some 1D plots, for example).

`model.result().clearStoredPlotData()` removes all stored plot data from the model.

CREATING A PREVIEW OF A DATASET

To create a preview of a dataset represented as a `DatasetFeature`, use the `createPlot` method:

```
DatasetFeature.createPlot()
```

Solution Selection

In this section:

- [About Selecting Solutions](#)
- [Selecting Solutions by Solution Number](#)
- [Selecting Solutions by Solution Level](#)
- [Choosing Solution Selection Method](#)
- [Results Configurations](#)

About Selecting Solutions

During the solution of, for example, time-dependent models or parametric sweep models, more than one solution is generated, and in order to do relevant results processing the correct set of solutions needs to be selected among them. How many solutions the result features can display vary, the main difference being between single-select and multiselect features. Single solution selection is available when only one solution at a time can be visualized, for example in 2D and 3D plots. In numerical features and 1D plots multiselect is used to be able to display plots relating to more than one parameter value at a time.

There are two ways to select solutions: by solution number and by solution level. COMSOL Multiphysics keeps them synchronized as far as possible.

Selecting solutions by solution number is a method that is tightly linked to the solution representation used by the solvers. The solution level method is structurally similar to the setup of the studies. Setting any one property belonging to either method triggers a synchronization in which all properties of that method are mapped to those of the other method. Mapping to the solution number method always works, since that case is the more general. If, on the other hand, mapping to the solution level method fails, its properties are set to their default settings.

The `solrepresentation` property is set to "solnum" automatically if the solution number method is the only one able to represent the selection made. On solving it is set to "solutioninfo" if the solution level method can be used to represent the previous selection. The `solrepresentation` property also decides which method is visible in the COMSOL Desktop.

Selecting Solutions by Solution Number

In the solution number method, the solutions are selected as a collection of inner and outer solutions. Inner solutions are generated by the solvers in one step. Outer solutions are generated by parametric sweeps wrapping the inner solutions. If there are more than one inner or outer parameter, they are represented and selected as tuples — that is, combinations of values of the different parameters. Depending on the setup of the studies the parameters can be either filled (all combinations) or sparse (a selection of combinations). In the single-selection case the properties used are `solnum`, `t` and `outersolnum`. In the multiselection case the valid properties are `innerinput`, `solnum`, `solnumindices`, `t`, `outerinput`, `outersolnum`, and `outerindices`.

Selecting Solutions by Solution Level

In contrast, the solution level method handles all filled parameters — that is, parameters defined individually in the studies — in one level each, up to a maximum of three levels. If more levels than that exist in the study the outermost levels are combined in the third level. Parameters that together form sparse tuples in the studies are always presented on the same level. In the single-selection case the properties used are `looplevel` and `interp`. In the multiselection case the valid properties are `looplevelinput`, `looplevel`, `looplevelindices`, and `interp`.

Choosing Solution Selection Method

Whether selection is made using the solution level method or the solution number method, the ways of selecting solutions are the same. In the single-selection case, solutions are selected by parameter index or, if the solution is time dependent, by value. In the multiselection case there are a number of selection input options all, first, last, by index from existing values, by manual indices, or, in time-dependent cases, by time.

Which method to use is not self-evident. The solution level method is perhaps the more intuitive of the two because it attempts to treat each individual level separately. The solution number method, however, is more general, and can pinpoint which tuples to use with more precision. The recommendation is to use the solution level method wherever possible, and resort to the solution number method only if its special characteristics are needed.

Results Configurations

You can create configuration features where you can define times, eigenvalue, or parameter tuples to evaluate for, graph plot style, and preferred units. Other results features such as data evaluation and plot groups can take their settings from the configuration features.

The following API syntax is available:

```
model.result().configuration();  
model.result().configuration(<ctag>);  
model.result().configuration(<ctag>).feature(<subtag>);
```

The following subfeatures are available:

- Multiselect Solution, for plots or evaluations that support multiple time steps, eigenvalues, or parameters:
`model.result().configuration().create(<ctag>, "MultiSelectSolution");`
- Single-Select Solution, for plots or evaluations that use a single time, eigenvalue, or parameter:
`model.result().configuration().create(<ctag>, "SingleSelectSolution");`
- Graph Plot Style, for coloring, style, and legend settings in graph plots:
`model.result().configuration().create(<ctag>, "GraphStyle");`
- Preferred Units, for specifying preferred units for results:
`model.result().configuration().create(<ctag>, "PreferredUnits");`

For Multiselect Solution and Single-Select Solution configurations, you specify the solution selections in these features (see the sections above about solution selections).

For Graph Plot Style configurations, they can be created and initialized from graph plots using the following method:

```
ConfigurationFeature createStyleConfiguration(String confTag)
```

For more information about Graph Plot Style configurations, see [Graph Plot Style](#).

For Preferred Units configurations, there is an `apply()` method, which applies the preferred units to the corresponding physical quantities.

Custom Color Tables

In this section:

- [Overview](#)
- [API Methods for colorTable](#)

Overview

Using the methods for `model.result().colorTable()`, you can perform the following color-table steps:

- Storing and managing imported color tables.
- Loading and managing color tables loaded from disk during the current session.
- Answering queries about color tables, both imported color tables and color tables loaded from disk.

The following section provides details about the API methods for `model.result().colorTable()`.

API Methods for colorTable

RETURNING COLOR TABLE INFORMATION

Use `String[] getColorTableNames()` to return the names of all color tables, both imported and loaded from disk. The names are the allowed values for the `colortable` property in, for example, Surface plots.

Use `double[][] getColorTableRGB(String colorTable)` to return an N -by-4 matrix. The matrix contains the raw RGB data that defines the color table (the fourth column contains color segment lengths and is optional). See [Color Tables and Color Themes](#) for how to interpret it. Also note that `colorTable` is case sensitive.

Use `String[] model.result().colorTable().getColorTableCategories()` to return an array of color table categories (which can be empty).

CHECKING COLOR TABLES

The boolean `hasColorTable(String colorTable)` method returns `true` if the color table exists.

The boolean `isColorTableContinuous(String colorTable)` method returns `true` if the color table exists and is continuous; otherwise, it returns `false`.

SAMPLING COLOR TABLES

Use `double[][] sampleColorTableRGB(String colorTable, int N)` to sample a color table in N uniformly distributed points. Returns an N -by-3-matrix containing RGB-values between 0 and 1.

Properties for colorTable

The `colorTable` method supports the following properties, using

```
model.result().colorTable(<ftag>).set(property,<value>);
```

TABLE 7-5: VALID PROPERTIES FOR COLORTABLE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
label	String		The label of the color table.
filename	String		The path to the file with the color table, if source is table.
input	decimal hexadecimal	decimal	The input format for RGB data.

TABLE 7-5: VALID PROPERTIES FOR COLORTABLE.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
maxvalue	1 255	1	Define the RGB values as a range between 0-1 or 0-255. Active when input is set to decimal.
source	table file	table	Define the color table through a local table or from a file.
type	continuous discrete	continuous	A continuous or discrete color table.

When input is decimal, use `setIndex("data", <value>, <row>, <column>)` to specify the RGB values (in the three leftmost columns) and an optional color segment length in the fourth column. When input is hexadecimal, use `setIndex("datahex", <value>, <row>, <column>)` to specify the hexadecimal color codes in the first column and an optional color segment length in the second column. See [setIndex\(\)](#) and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

Stepping Through Plots

You can use API commands to create the same effect as clicking the Plot Previous, Plot Next, Plot First, and Plot Last toolbar buttons in the plot group and plot nodes in the COMSOL Desktop. This functionality, for moving between solutions in a parametric, time-dependent, or eigenvalue model with multiple solutions, including models with several parametric levels, can be useful in COMSOL apps.

API Methods for Stepping Through Plots

The Boolean methods below are available for stepping through plots with multiple solutions. They return *true* if the parameter value was changed; otherwise, they return *false*. The methods are applicable to plot groups and plots.

TABLE 7-6: API METHODS FOR STEPPING THROUGH PLOTS.

METHOD	DESCRIPTION
<code>boolean stepPrevious(int level)</code>	Corresponds to Plot Previous.
<code>boolean stepNext(int level)</code>	Corresponds to Plot Next.
<code>boolean stepFirst(int level)</code>	Corresponds to Plot First.
<code>boolean stepLast(int level)</code>	Corresponds to Plot Last.

The input argument `level` is an integer that indicates the level in a model with several levels of parameters or other types of multiple solutions (such as a parametric sweep around a time-dependent simulation).

When using these methods in an app, it can also be useful to be able to test if these operations are applicable. For example, it is not possible to step to the previous parameter value if the current parameter value is the first parameter value. It can also be useful to be able to test if it is possible to step for a given level (as the level might be larger than the number of available parameters) and to test if it is possible to step at all. To test for such case, use the following API methods:

TABLE 7-7: API METHODS FOR CHECKING IF STEPPING IS POSSIBLE.

METHOD
<code>boolean canStepPrevious(int level)</code>
<code>boolean canStepNext(int level)</code>
<code>boolean canStepFirst(int level)</code>
<code>boolean canStepLast(int level)</code>

Being able to retrieve the number of levels that can be stepped over can be useful when the API methods described above are used. To do so, use the following method:

```
int getStepLevels()
```

Special Formats and Symbols in Texts

It is possible to add formatted text strings to plots using API commands. This includes adding HTML tags to display, for example, bold or italic fonts; mathematical symbols; and special characters. You can also use LaTeX Display and LaTeX Inline math modes to add mathematical expressions to texts. The LaTeX formatting modes are available for plot titles, color legend titles, axis labels, parameter indicators, and manual legend texts in graph plots.

Using Special Formats and Symbols

For some plot features you can use these LaTeX formatting modes for adding mathematical expressions to texts:

- To use LaTeX Display math mode, put `\\[ \\]` around the LaTeX mathematical expression. This mode uses more vertical space for the math expression. For example, to plot the expression

$$u + \frac{\log x^2}{4\pi}$$

use the following LaTeX text: `\\[u+\\frac{\\log x^2}{4 \\pi}]`

- To use LaTeX Inline math mode, put `\\$ \\$` around the LaTeX mathematical expression. This mode is meant to be included in line in the text (in normal LaTeX) and has a more compact presentation. To plot the same mathematical expression as in the other mode, use the following LaTeX text: `\\$u+\\frac{\\log x^2}{4 \\pi}\\$`

Note the use of double backslashes, which is needed in code for use with Java. In code for use with MATLAB only one backslash is needed.

SUPPORT FOR THE LATEX FORMATTING MODES

Use the LaTeX formatting modes with the following features:

- For titles in plot groups and plots when `titletype` is set to `manual`:
`model.result(<pgtag>).set("title",<value>);`
`model.result(<pgtag>).feature(<ftag>).set("title",<value>);`
- For parameter indicators in plot group titles when `titletype` is set to `manual`:
`model.result(<pgtag>).set("paramindicator",<value>);`
- For *x*-axis and *y*-axis labels in 1D and 2D plot groups when `xlabelactive` and `ylabelactive` are set to `on`:
`model.result(<pgtag>).set("xlabel",<value>);`
`model.result(<pgtag>).set("ylabel",<value>);`
- For secondary *y*-axis labels in 1D plot groups when `twoyaxes` and `yseclabel` are set to `on`:
`model.result(<pgtag>).set("yseclabel",<value>);`
- For color legend titles in plots when `showlegendstitle` in the parent plot group is set to `true`:
`model.result(<pgtag>).feature(<ftag>).set("legendtitle",<value>);`
- For legends in graph plots when `legendmethod` is set to `manual`:
`model.result(<pgtag>).feature(<ftag>).set("legends",<value>);`
- For legend prefixes and suffixes in graph plots when `legendmethod` is set to `automatic`:
`model.result(<pgtag>).feature(<ftag>).set("legendprefix",<value>);`
`model.result(<pgtag>).feature(<ftag>).set("legendsuffix",<value>);`

SYMBOLS AND SPECIAL CHARACTERS IN ANNOTATIONS

You can also include, for example, mathematical symbols and Greek letters in annotation texts in [Annotation](#) or [AnnotationData](#) features when the `latexmarkup` property is set to `on`. To include such symbols, surround the LaTeX syntax with `$` to indicate that the text inside of the `$` signs is LaTeX. For example, `$\\alpha = \\beta/\\pi$` appears as $\alpha = \beta/\pi$. Note the use of double backslashes, which is needed in code for use with Java. If the `latexmarkup` property is `on`, you can also add line breaks as `\\`. To add an annotation text string that can contain special characters and symbols to an annotation data plot or annotation feature named `<ftag>`, use:

```
model.result(<pgtag>).feature(<ftag>).set("text",<value>);
```

SEE ALSO

See [Mathematical Symbols and Special Characters](#) and [Using Special Formats and Symbols in Texts](#) in the *COMSOL Multiphysics Reference Manual* for information about available LaTeX characters and HTML tags.

Results Commands

Animation

Create animations of plots, saved to a file or shown in a player in the COMSOL Desktop main window.

SYNTAX

```
model.result().export().create(<ftag>,"Animation");
model.result().export(<ftag>).set(property,<value>);
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>,"Animation")` creates an animation feature with the name `<ftag>`.

`result().export(<ftag>).set("plotgroup",<ptag>)` changes the source of the animation to the plot group named `<ptag>`.

The following properties are available:

TABLE 7-8: VALID PROPERTY/VALUE PAIRS FOR ANIMATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
allarray	integer array	All solutions	The solutions to use, chosen from all combinations of outer and inner. Applicable when both inner and outer solutions exist and solnumtype is all.
alwaysask	on off	off	Always ask for filename when saving. This property is ignored when running without a user interface.
antialias	on off	on	Enable/disable antialiasing.
avifilename	String		The name of the AVI file produced if movietype is avi.
aviqual	double in [0,1]	0.75	The quality of the AVI file if movietype is avi; higher is better.
axes	on off	on	If options is on; enable/disable display of the coordinate axes. Used for 1D and 2D animations.
axisorientation	on off	on	If options is on; enable/disable display of the axis orientation indicator. Used for 3D animations.
background	current color fromtheme transparent	color	The background color.
bmpfilename	String		The name of the output file if imagetype is bmp.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.

TABLE 7-8: VALID PROPERTY/VALUE PAIRS FOR ANIMATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
cumulative	on off auto	auto	Control if the animation is cumulative or not, for a 1D graph plot group with a time-dependent, parametric, or eigenvalue tuple, if sweeptype is solutions.
customcolor	double array	{1, 1, 1}	If background is set to color, the red, green, and blue components of the background color.
cycletype	fullharm halfharm linear	fullharm	The transformation of the solution that is performed if sweeptype is dde.
fontsize	integer	9	The font size.
fps	positive integer	10	The number of frames per second.
framesel	all number	number	When sweeptype is solutions, framesel controls whether to look through all solutions or a specified number of frames.
frametime	positive double	0.1	The time in seconds that each frame is displayed when the target is set to player.
giffilename	String		The name of the GIF file produced if movietype is set to gif.
gifopen	on off	off	Enable/disable opening of the generated GIF file in a browser if movietype is set to gif.
globalpstart	double	0	The parameter's start value if sweeptype is globalparameter.
globalpstop	double	1	The parameter's stop value if sweeptype is globalparameter.
globalpunit	String		The unit of the parameter if sweeptype is globalparameter.
grid	on off	on	If options is on; enable/disable display of the coordinate grid. Used for 3D animations.
height	integer	480	The height in pixels of the frames of the animation, if size is manual.
highprecisioncolor	on off	off	Export 16-bit per channel PNG files. Available when the file extension of imagefilename is .png.
imagefilename	String		The base name of generated images if type is movieseq; the generated images have names of the form base1.ext, base2.ext, and so on.
interp	double array	Empty	The times to use, for transient levels, that the multilooplevel property points to. Available when the underlying data is transient, looplevelinput is interp.

TABLE 7-8: VALID PROPERTY/VALUE PAIRS FOR ANIMATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
lockratio	true false	false	If true, then the aspect ratio of the image is preserved when the width or the height is changed. Available if size is manual.
reverse	true false	false	Enable/disable reversal in time of the animation.
iterations	positive integer	1	The number of iterations for the playing, if repeat is set to iterations.
legend	on off	on	If options is on; enable/disable display of the legend.
logo	on off	on	If options is on; enable/disable display of the logo.
looplevel	integer array	All solutions	The solutions to use. Applicable to the level that the multilooplevel property points to, when looplevelinput is manual.
looplevelindices	integer array or String	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable to the level that the multilooplevel property points to, when looplevelinput is manualindices.
looplevelinput	all manual manualindices interp	all	How to input the solution to use, for the level that multilooplevel points to. manual indicates that looplevel is used for that level. manualindices indicates that looplevelindices is used for that level. interp indicates that interp is used for that level.
maxframes	integer >= 2	25	The number of frames.
movietype	gif avi webm	gif	The movie format to use if type is movie: GIF, AVI, or WebM.
multilooplevel	1 <= integer <= 3	1	The loop level to animate over. Only applied if solnumtype is pointing to the same level and solrepresentation is set to solutioninfo
options	on off	off	Enable/disable optional components of the image.
outerinnertype	first last all	last	When solnumtype is outer, outerinnertype controls whether to plot all, the first or the last inner solution. Applicable only for parametric sweep models.
outersolnumarray	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models when solnumtype is set to outer.

TABLE 7-8: VALID PROPERTY/VALUE PAIRS FOR ANIMATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outersolnumsingle	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models when solnumtype is set to inner.
parameter	String		The model parameter that varies if sweeptype is parameter.
plot	String		The streamline plot to animate if sweeptype is set to streamline.
plotgroup	String		The name of the plot group to animate.
pstart	double	0	The parameter's start value if sweeptype is set to parameter.
psstop	double	1	The parameter's stop value if sweeptype is set to parameter.
punit	String		The unit of the parameter if sweeptype is set to parameter.
repeat	off forever iterations	off	Whether to repeat from the beginning when the end is reached when playing. If forever, iterate until stopped. If iterations, as many times as given in iterations.
resolution	integer	96	The frame resolution in dots per inch.
reverse	true false	false	Enable/disable reversal in time of the animation.
showframe	positive integer	1	The shown frame when playing,
showgrid	on off	on	Include a grid.
singleinterp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels with the level pointed to by multilooplevel exempt. Available when the underlying data is transient.
singlelooplevel	array of nonnegative integers and strings	First solution for each level.	The index of the solution to use, per level with the level pointed to by multilooplevel exempt, or interp, but only when the underlying data is transient.
size	current hd manual	manual	The size of the animation frames, if target is file. The current size, a high-definition size, or a manually specified size.
solnum	integer array	all	The solutions to animate if sweeptype is solutions.
solnumtype	level1 level2 level3 all inner outer	all	Whether to sweep over a selected loop level or over all, inner or outer solutions, in a parametric sweep model.

TABLE 7-8: VALID PROPERTY/VALUE PAIRS FOR ANIMATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
sweeptype	solutions parameter globalparameter dde streamline	solutions	The parameter that varies during animation.
target	file player	file	Save the animation in a file or show it in a player in a COMSOL graphics window.
timeinterp	on off	off	Enable/disable sweep over interpolated times if sweeptype is solutions.
tint	double array		The interpolated times to animate if timeinterp is on.
title		on	If options is on; enable/disable display of the title.
type	imageseq movie	movie	The kind of output that is produced.
webmadvanced	automatic manual	automatic	Automatic or manual settings for advanced WebM settings.
webmcodec	vp8 vp9	vp9	Codec, when movietype is webm.
webmcqlevel	0–1	0.6	Quality level, when movietype is set to webm.
webmdither	0–5	0	Use dithering for WebM format (0 means no dithering).
webmfilename	String		The name of the WebM file produced if movietype is webm.
webmlossless	on off	off	Use lossless compression if movietype is webm.
webmparallelencoding	on off	on	Parallel encoding for the WebM format.
webmparalleldecoding	on off	off	Parallel decoding for WebM format (only available if parallel encoding is on).
webmquality	optimizeForQuality optimizeForSpeed	optimizeForSpeed	Optimize the WebM quality for speed (faster encoding) or quality (slower encoding).
webmyuvmatrix	BT601Limited BT601Full BT709Limited BT709Full	BT709Limited	YUV color space for WebM movies: BT.601 and BT.709 standards.
width	integer	640	The width in pixels of the frames of the animation, if size is manual
zoomextents	true false	false	Zoom to the extents of the plot's subject.

EXAMPLE

Create a 2D animation of a surface and contour plot:

Code for Use with Java

```
ExportFeature anim = model.result().export().create("c2", "pg1", "Animation");
anim.set("plotgroup", "pg1");
anim.run();
```

Code for Use with MATLAB

```
anim = model.result.export.create('c2','pg1','Animation');  
anim.set('plotgroup','pg1');  
anim.export('c2').run;
```

COMPATIBILITY

As of version 6.1, the Flash format is no longer available for animations, and the `flashfilename`, `flashinterp`, and `flashopen` properties are no longer used.

SEE ALSO

[Image \(Export\)](#)

Annotation

Add an annotation to a plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Annotation");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Annotation")` creates an annotation feature named `<ftag>` belonging to the plot group `<pgtag>`.

Annotations add text, including optional position coordinates, to plots.

The following properties are available:

TABLE 7-9: VALID PROPERTIES FOR ANNOTATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
<code>anchorpoint</code>	<code>upperright</code> <code>uppermiddle</code> <code>upperleft</code> <code>middleright</code> <code>center</code> <code>middleleft</code> <code>lowerright</code> <code>lowermiddle</code> <code>lowerleft</code>	<code>upperleft</code>	The position of the annotation text relative to the anchor point at the position given by <code>posexpr</code> .
<code>backgroundcolor</code>	<code>custom</code> <code>fromtheme</code> <code>black</code> <code>blue</code> <code>cyan</code> <code>gray</code> <code>green</code> <code>magenta</code> <code>red</code> <code>white</code> <code>yellow</code> <code>none</code>	<code>none</code>	The color to use for background rectangle.
<code>color</code>	<code>custom</code> <code>fromtheme</code> <code>black</code> <code>blue</code> <code>cyan</code> <code>gray</code> <code>green</code> <code>magenta</code> <code>red</code> <code>white</code> <code>yellow</code>	<code>fromtheme</code>	The color to use for the point where the annotation appears and the annotation text. The default takes the color from the current color theme.
<code>data</code>	<code>none</code> <code>parent</code> <code>dataset name</code>	<code>parent</code>	The dataset this feature refers to.
<code>descriptionintitle</code>	<code>on</code> <code>off</code>	<code>on</code>	Whether the title contribution should contain the description when <code>titletype</code> is <code>custom</code> .

TABLE 7-9: VALID PROPERTIES FOR ANNOTATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
expralwaysimaginary	true false	false	Always show the imaginary part when exprnumberformat is stopwatch or stopwatchsci.
expralwaysign	true false	false	Always show the sign when exprnumberformat is stopwatch or stopwatchsci.
expralwaysignexp	true false	false	Always show the sign of the exponent when exprnumberformat is stopwatchsci.
exprdecimals	positive integer	2	Number of decimals when exprnumberformat is stopwatch or stopwatchsci.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
exprexponentdigits	positive integer (1–3)	1	The number of exponent digits (1–3) for numerical values in the title when exprnumberformat is stopwatchsci.
exprnumberformat	default auto engineering scientific stopwatch stopwatchsci	auto	The number format for the annotation: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.
exprprecision	positive integer	6	Precision for the expressions used in annotations.
exprtrailingzeros	true false	false	Include trailing zeros when exprnumberformat is auto, engineering, or scientific.
inheritbackgroundcolor	boolean	true	If inheritplot is not none: Determines if the background color is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritframe	boolean	true	If inheritplot is not none: Determines if the show frame property is inherited.
inheritplot	none plot name	none	The plot that color is inherited from.
latexmarkup	on off	off	Include LaTeX markup language in annotation.
level	fromdataset global line point surface volume	fromdataset	The geometry level for evaluating expressions. The default is to take it from the dataset. volume is only available in 3D.
orientation	horizontal vertical	horizontal	Orient the annotation text horizontally or vertically.
plotonsecyaxis	boolean	false	Plot on secondary y-axis if twoyaxes is set to true in the parent plot group. For 1D graph plots and annotations only. An array, using setIndex syntax.
pointradius	double	2	A value between 0 and 10 for the point radius, if showpoint is true.
poseprx	String array	{"0"}	Position x-coordinates for the annotation.

TABLE 7-9: VALID PROPERTIES FOR ANNOTATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
posexp _y	String array	{ "0" }	Position y-coordinates for the annotation.
posexp _z	String array	{ "0" }	Position z-coordinates for the annotation (3D only).
posprecision	positive integer	6	Precision for the position values.
posalways _{sign}	true false	false	If prependcoords is set to true: Always show the sign when posnumberformat is stopwatch or stopwatchsci.
posalways _{signexp}	true false	false	If prependcoords is set to true: Always show the sign of the exponent when posnumberformat is stopwatchsci.
posdecimals	positive integer	2	If prependcoords is set to true: Number of decimals when posnumberformat is stopwatch or stopwatchsci.
posexponent _{digits}	positive integer (1–3)	1	If prependcoords is set to true: Number of exponent digits (1–3) for numerical values in the title when posnumberformat is stopwatchsci.
posinteger _{digits}	positive integer	3	If prependcoords is set to true: Number of integer digits when posnumberformat is stopwatch.
posnumberformat	default auto engineering scientific stopwatch stopwatchsci	auto	If prependcoords is set to true: The number format for the marker position: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.
posprecision	positive integer	6	If prependcoords is set to true: The precision for position values in the marker when posnumberformat is auto, engineering, or scientific.
postrailing _{zeros}	true false	false	If prependcoords is set to true: Include trailing zeros when posnumberformat is auto, engineering, or scientific.
prefix _{intitle}	String	Empty	A prefix text in a custom title.
prependcoords	true false	false	Include coordinates of annotation position before the annotation text.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere).
showframe	true false	false	Whether to show a frame around the annotation or not.
showpoint	true false	true	Whether to show the point for the position of the annotation or not.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffix _{intitle}	String	Empty	A suffix text in a custom title.
text	String	Empty	The text to display as an annotation.
title	String	The auto-title	The title to use when titletype is manual.

TABLE 7-9: VALID PROPERTIES FOR ANNOTATION.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

[Deform](#), [Transformation](#)

COMPATIBILITY

The property `allowexprs` property has been removed as of version 6.0. Evaluation on of expressions in the annotation's text using the `eval(expr)`, `eval(expr, unit)`, and `eval(expr, unit, expression)` syntax is now always allowed for the annotation text.

SEE ALSO

[MaxMinVolume](#), [MaxMinSurface](#), [MaxMinLine](#), [MaxMinPoint](#)

AnnotationData

Create an annotation data plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "AnnotationData");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "AnnotationData")` creates an annotation data plot feature named `<ftag>` belonging to the 2D or 3D plot group `<pgtag>`.

Annotation data plots are used to visualize raw annotation data given as points, text, and colors (see the example below). Annotation data plots can be added to 2D and 3D plot groups.

Set the `latexmarkup` property to on if you want to include mathematical symbols and Greek letters, for example, in the annotation. See [Symbols and Special Characters in Annotations](#) for more information.

The following properties are available:

TABLE 7-10: VALID PROPERTY/VALUE PAIRS FOR ANNOTATION DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
backgroundcolor	custom black blue cyan gray green magenta red white yellow none	none	The color to use for background rectangle.
color	custom black blue cyan gray green magenta red white yellow	red	The color to use for the point where the annotation appears and the annotation text.
latexmarkup	true false	false	Include LaTeX markup language in annotation.
pointradius	double	2	A value between 0 and 10 for the point radius, if showpoint is true.
showframe	true false	false	Whether to show a frame around the annotation or not.
showpoint	true false	true	Whether to show the point for the position of the annotation or not.
pos	double array		The position coordinates for the annotation data plot (x- and y-coordinates in 2D and x-, y-, and z-coordinates in 3D).
text	String		The text to display as an annotation.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.

ATTRIBUTES

None.

EXAMPLE

A method for creating letter in a circle in 2D.

Code for Use with Java

```
String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 2);

for (int i = 0; i < 26; i++) {
    double angle = 2*Math.PI*i/26;
    ResultFeature plot = pg.create("ann"+i, "AnnotationData");
    plot.set("pos", new double[]{Math.cos(angle), Math.sin(angle)})
        .set("text", "ABCDEFGHIJKLMNOPQRSTUVWXYZ".substring(i, i+1))
        .set("showpoint", false);
}
```

A method for adding Greek letters to the corners of a cube.

Code for Use with Java

```
String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 3);
```

```
String[] texts = {"\\alpha", "\\beta", "\\gamma", "\\delta", "\\epsilon", "\\zeta",
"\\eta", "\\theta"};
String[] colors = {"black", "blue", "cyan", "gray", "green", "magenta", "red", "yellow"};
for (int x = 0; x < 2; x++) {
    for (int y = 0; y < 2; y++) {
        for (int z = 0; z < 2; z++) {
            int index = x+2*y+4*z;
            ResultFeature plot = pg.create("ann"+index, "AnnotationData");
            plot.set("pos", new double[]{x, y, z})
                .set("text", "$"+texts[index]+"$")
                .set("latexmarkup", true)
                .set("color", colors[index]);
        }
    }
}
```

ATTRIBUTE

[Transparency](#)

SEE ALSO

[ArrowData](#), [LineData](#), [PointData](#), [SurfaceData](#), [TubeData](#)

Array 1D, Array 2D, Array3D

Create array datasets.

SYNTAX

```
model.result().dataset().create(<dtag>,"Array1D");
model.result().dataset().create(<dtag>,"Array2D");
model.result().dataset().create(<dtag>,"Array3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Array1D")` creates a 1D array dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"Array2D")` creates a 2D array dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"Array3D")` creates a 3D array dataset feature named `<dtag>`.

This dataset takes data from another dataset and adds an array of copies in cells as a linear, rectangular (2D), or three-dimensional (3D) array of cells.

The following properties are available for Array 1D, Array 2D, and Array 3D:

TABLE 7-11: VALID PROPERTIES FOR ARRAY DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
ax2	String array	{"0", "0"}	Theta and phi angles for the rotation, if type is set to linear and axistype is set to spherical for a 3D Array dataset.
ax3	String array	{"0", "0", "0"}	Axis coordinates (x, y, and z) for the rotation, if type is set to linear and axistype is set to cartesian for a 3D Array dataset.
axistype	x y z cartesian spherical	z	The axis type for a rotation, if type is set to linear for a 3D Array dataset.
base	corner center	corner	Base point for the position of the input cell if inputmethod is set to manual.

TABLE 7-II: VALID PROPERTIES FOR ARRAY DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
celllinearvar	String	Depends on the feature's tag	An integer variable that identify the current cell in the array.
cellvars	String array	Depends on the feature's tag	An integer array that identify the current cell in the array.
checkoverlap	boolean	true	Check for overlap between cells in the array.
data	none dataset name	First compatible dataset	The dataset this feature refers to.
displ	String array	{"0", "0"} (2D); {"0", "0", "0"} (3D)	The displacement used to move each cell in the array if displmethod is set to manual.
displmethod	auto manual	auto	Use automatic or manual definition of the displacement for the cells in the array.
floquetper	on off	off	Include Floquet–Bloch periodicity.
fullsize	array (integer or string)	1 (1D); {1, 1} (2D); {1, 1, 1} (3D)	Size of the array (number of cells in each direction) for a linear, rectangular, or three-dimensional array.
inputmethod	auto manual	auto	Use automatic or manual definition of the part of space that the array is created from,
linearsize	positive integer or string	1	Size of the linear array in number of cells.
pos	String array	{"0", "0"} (2D); {"0", "0", "0"} (3D)	If inputmethod is manual: The coordinates (position) of the base point. If base is set to corner, it is the lower-left corner of the base cell.
rotangle	double	0	The angle increment, in radians, for a rotation, if type is set to linear for a 2D or 3D Array dataset.
rotcenter	String array	{"0", "0"}	The center of rotation, if type is set to linear for a 2D Array dataset.
rotpos	String array	{"0", "0", "0"}	Point on axis of rotation, if type is set to linear for a 3D Array dataset.
size	String array	{"1", "1"} (2D); {"1", "1", "1"} (3D)	If inputmethod is manual: The size of the input.
spacevars	String array	Depends on the feature's tag	The name of space variables, which evaluate to the coordinates in the dataset's coordinate system.
type	linear rectangular three-dimensional	rectangular (2D); three-dimensional (3D)	The type of array: Linear, rectangular (2D only), or three-dimensional (3D only). Not available in 1D.
wavevector	String array		The wave vector for Floquet–Bloch periodicity as a 1-, 2-, or 3-element string array.

ATTRIBUTE[Selection \(Dataset Attribute\)](#)*ArrayFactor*

Create an array factor dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"ArrayFactor");  
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"ArrayFactor")` creates an array factor dataset feature named `<dtag>`.

This dataset takes data from another dataset evaluates an array factor for use in radiation pattern plots in RF simulations.

The following properties are available for ArrayFactor:

TABLE 7-12: VALID PROPERTIES FOR ARRAY FACTOR DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
displ	1x2 double array	0, 0	The displacement; specify using setIndex
function	String		The function to evaluate the array factor for.
normalization	boolean	false	Normalize the array factor evaluation.
phaseshift	1x2 double array	0, 0	The phase shift; specify using setIndex
plane	xy yz zx	xy	The plane for the array factor definition.
scale	linear dB	linear	Use a linear or dB scaling.
size	1x2 double array	1, 1	The array size; specify using setIndex.

ArrowData

Create an arrow data plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"ArrowData");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"ArrowData")` creates an arrow data plot feature named `<ftag>` belonging to the 2D or 3D plot group `<pgtag>`.

Arrow data plots are used to visualize raw vector data given as points, vectors, and colors (see the example below). Arrow data plots can be added to 2D and 3D plot groups.

The following properties are available:

TABLE 7-13: VALID PROPERTY/VALUE PAIRS FOR ARROW DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	tail	Whether the head, tail, or center of the arrow is located at the arrow position.
arrowlength	normalized proportional logarithmic	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.

TABLE 7-13: VALID PROPERTY/VALUE PAIRS FOR ARROW DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
colorcalibration	double	0	A color calibration value (-1.5 to 1.5) when colortabletrans is set to nonlinear or nonlinearsymmetric.
colordata	double ID array		The color data for the arrow data plot as a real N-vector.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.

TABLE 7-13: VALID PROPERTY/VALUE PAIRS FOR ARROW DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
pointdata	double 2D array		The point data for the arrows' starting points in the arrow data plot, as x and y-coordinates in 2D and x-, y-, and z-coordinates in 3D in an sdim-by-N real matrix.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
vectordata	double 2D array		The vector data for the arrow data plot, as sign and length of the arrows in the x and y directions in 2D and in the x, y, and z directions in 3D in an sdim-by-N real matrix.

ATTRIBUTES

MaterialAppearance, Transparency (3D only)

EXAMPLE

A method for creating an arrow circle in 2D.

Code for Use with Java

```
String pgTag = model.result().uniqueTag("pg");
ResultFeature pg = model.result().create(pgTag, 2);
ResultFeature plot = pg.create("arrow1", "ArrowData");
int N = 17;
double[][] p = new double[2][N];
double[][] vec = new double[2][N];
double len = 0.2;
for (int i = 0; i < N; i++) {
    double angle = 2*Math.PI*i/N;
    p[0][i] = Math.cos(angle);
    p[1][i] = Math.sin(angle);
    vec[0][i] = -len*p[0][i];
    vec[1][i] = -len*p[1][i];
}
plot.set("pointdata", p).set("vectordata", vec);
```

A method for creating a 3D logarithmic arrow spiral.

Code for Use with Java

```
String pgTag = model.result().uniqueTag("pg");
ResultFeature pg = model.result().create(pgTag, 3);
ResultFeature plot = pg.create("arrow1", "ArrowData");
int N = 1000;
double[][] p = new double[3][N];
double[][] vec = new double[3][N];
double[] color = new double[N];
for (int i = 0; i < N; i++) {
    double par = 0.005*i;
    p[0][i] = Math.exp(par)*Math.cos(10*par);
    p[1][i] = Math.exp(par)*Math.sin(10*par);
    p[2][i] = 0.1*i;
    double len = Math.sqrt(p[0][i]*p[0][i]+p[1][i]*p[1][i]+p[2][i]*p[2][i]);
    for (int j = 0; j < 3; j++) {
        vec[j][i] = 4*p[j][i]/len;
    }
    color[i] = i;
}
plot.set("pointdata", p)
    .set("vectordata", vec)
    .set("colordata", color)
    .set("coloring", "colortable");
```

SEE ALSO

[AnnotationData](#), [LineData](#), [PointData](#), [SurfaceData](#), [TubeData](#)

ArrowVolume, ArrowSurface, ArrowLine, ArrowPoint

Create volume, surface, line, or point arrow plots.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "ArrowVolume");
model.result(<pgtag>).create(<ftag>, "ArrowSurface");
model.result(<pgtag>).create(<ftag>, "ArrowLine");
model.result(<pgtag>).create(<ftag>, "ArrowPoint");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```


DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Arrow...")` creates an arrow plot feature named <ftag>.

Arrow plots are available in 2D and 3D. `ArrowPoint` plots arrows at points in 2D or 3D. `ArrowLine` plots arrows on lines in 2D or 3D such as geometry boundaries in 2D or cut plane edges in 3D. `ArrowSurface` plots arrows on surfaces in 2D or 3D. `ArrowVolume` plots arrows in a 3D volume.

The following properties are available:

TABLE 7-14: VALID PROPERTIES FOR ARROW PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	tail	Whether the head, tail, or center of the arrow is located at the arrow position.
arrowcount	positive integer	200	If placement is uniform: The approximate number of arrows. Available for <code>ArrowEdge</code> in 2D and 3D and for <code>ArrowSurface</code> in 3D.
arrowlength	normalized proportional logarithmic	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowtype	arrow arrowhead cone double	arrow	The type of arrow to draw.
arrowxmethod	number coord	number	Indicates whether the x-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for <code>ArrowSurface</code> in 2D and <code>ArrowVolume</code> in 3D.
arrowymethod	number coord	number	Indicates whether the y-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for <code>ArrowSurface</code> in 2D and <code>ArrowVolume</code> in 3D.
arrowzmethod	number coord	number	Indicates whether the z-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for <code>ArrowVolume</code> in 3D.
color	custom black blue cyan gray green magenta red white yellow	red	The color to use.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expressions in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is custom.

TABLE 7-14: VALID PROPERTIES FOR ARROW PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group. Active when placement is elements.
expr	String array of length 2 in 2D and 3 in 3D	Mode-dependent	The components of the vector to plot as arrows.
gporder	positive integer	1	If placement is gausspoints: The Gauss-point order.
inheritarrowsscale	boolean	true	If inheritplot is not none: Determines if arrow scale is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that arrow scale, color, and deformation scale are inherited from.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
maxpointcount	nonnegative integer	100000	The maximum number of points for arrow plots where placement is set to elements or gausspoints for ArrowVolume in 3D and Arrow Surface in 2D.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.

TABLE 7-14: VALID PROPERTIES FOR ARROW PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
placement	elements grid uniform uniformanis gausspoints	uniform (grid for ArrowVolume in 3D and ArrowSurface in 2D)	If uniform, then the arrows are distributed uniformly. If uniformanisotropic, then the arrows are distributed uniformly with weights taken from the weight property. If elements, then arrows are drawn in the mesh vertices for each element. If gausspoints, then the arrows are drawn in the Gauss points (of order set as gporder). The uniform and uniformanisotropic options are only available for ArrowEdge in 2D and 3D and for ArrowSurface in 3D. The grid option is only available for ArrowVolume in 3D and ArrowSurface in 2D.
planecoordsys	cutplane cartesian	cartesian	Coordinate system for arrow vectors. Only available for plots that use a cut plane dataset that points to a 3D solution or mesh dataset.
plotcomp	all normal tangential	all	Choose to plot arrows that represent all components of the vector in expr or only its normal or tangential components. Available for ArrowSurface in 3D.
prefixintitle	String	Empty	Added prefix to contribution to title.
revcoordsys	cylindrical cartesian	cartesian	Coordinate system for arrow vectors. Only available for plots that use a 1D or 2D revolution dataset that has default axis settings and points to a solution or mesh dataset for an axisymmetric geometry.
scale	positive double	1	If scaleactive is true: The length scale factor.
scaleactive	boolean	false	Whether to use manual scaling.
smooth	none material internal everywhere expression	everywhere	Smoothing settings when evaluationsettings is manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title

TABLE 7-14: VALID PROPERTIES FOR ARROW PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
weight	double array	1	If placement is uniform: The weights given to the different axis directions.
xnumber	nonnegative integer	15 (3D), 7 (2D)	Number of points in the x direction. Active when arrowxmethod is set to number. Available for ArrowSurface in 2D and ArrowVolume in 3D.
xcoord	String array	Empty	Absolute coordinates in the x direction, active when arrowxmethod is set to coord. Available for ArrowSurface in 2D and ArrowVolume in 3D.
ynumber	integer	15 (3D), 7 (2D)	Number of points in the y direction. Active when arrowymethod is set to number. Available for ArrowSurface in 2D and ArrowVolume in 3D.
ycoord	String array	Empty	Absolute coordinates in the z direction, active when arrowymethod is set to coord. Available for ArrowSurface in 2D and ArrowVolume in 3D.
znumber	integer	15 (3D), 7 (2D)	Number of points in the z direction. Active when arrowzmethod is set to number. Available for ArrowVolume in 3D.
zcoord	String array	Empty	Absolute coordinates in the z direction, active when arrowzmethod is set to coord. Available for ArrowVolume in 3D.

ATTRIBUTES

[Color](#), [Deform](#), [Filter \(Plot Attribute\)](#), [MaterialAppearance](#), [Selection \(Dataset Attribute\)](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[Line](#), [ScatterVolume](#), [ScatterSurface](#), [Surface \(Plot\)](#), [Volume](#)

Compute a volume average, surface average, or line average.

SYNTAX

```
model.result().numerical().create(<ftag>,"AvVolume");
model.result().numerical().create(<ftag>,"AvSurface");
model.result().numerical().create(<ftag>,"AvLine");
model.result().numerical(<ftag>).selection(...);
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"AvVolume")` creates a volume average feature with the name <ftag>.

`model.result().numerical().create(<ftag>,"AvSurface")` creates a surface average feature with the name <ftag>.

`model.result().numerical().create(<ftag>,"AvLine")` creates a line average feature with the name <ftag>.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is false. If `columnwise` is true, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is true, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` true and `columnwise` false.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. <outersolnum> is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).selection()` returns the selection of the geometry for the computation of the volume average, surface average, or line average. See [Selections](#) for more information about the available selection methods.

`model.result().numerical(<ftag>).set(property,<value>)` sets the value of a property of the volume average, surface average, or line average.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-15: VALID PROPERTIES FOR AVERAGES.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>data</code>	<code>none</code> <code>parent</code> <code>dataset name</code>	First compatible dataset	The dataset this feature refers to. <code>parent</code> is only applicable if used in an evaluation group.
<code>dataseries</code>	<code>average</code> <code>integral</code> <code>maximum</code> <code>minimum</code> <code>none</code> <code>rms</code> <code>stddev</code> <code>variance</code>	<code>none</code>	The transformation that is applied to the data series formed by the evaluation.
<code>dataseriescumulative</code>	<code>true</code> <code>false</code>	<code>false</code>	Make the table values as a cumulative integration, when <code>dataseries</code> is set to <code>integral</code> .
<code>dataseriesmethod</code>	<code>auto</code> <code>integration</code> <code>summation</code>	<code>auto</code>	The method to use for the data series: automatic, integration, or summation.
<code>const</code>	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
<code>descr</code>	String array	Model dependent	The descriptions of the expressions in <code>expr</code> . Is used in the automatic title.
<code>differential</code>	<code>on</code> <code>off</code>	<code>on</code>	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is <code>harmonic</code> .
<code>evalmethod</code>	<code>linpoint</code> <code>harmonic</code> <code>lintotal</code> <code>lintotalavg</code> <code>lintotalrms</code> <code>lintotalpeak</code>	<code>harmonic</code>	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
<code>expr</code>	String array	Model dependent	The expressions to evaluate.
<code>includeparam</code>	<code>true</code> <code>false</code>	<code>false</code>	Include the parameter value or time if <code>dataseries</code> is set to <code>maximum</code> or <code>minimum</code> for a parametric or time-dependent solution,
<code>innerinput</code>	<code>all</code> <code>first</code> <code>last</code> <code>manual</code> <code>manualindices</code> <code>interp</code>	<code>all</code>	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
<code>interp</code>	double row matrix	Empty on all levels	The times to use, for transient levels. Available when <code>data</code> is not <code>parent</code> and the underlying data is transient.
<code>intorder</code>	positive integer	4	The integration order.

TABLE 7-15: VALID PROPERTIES FOR AVERAGES.

NAME	VALUE	DEFAULT	DESCRIPTION
intorderactive	on off	off	Whether to use manually specified integration order.
intsurface	on off	off	Compute surface integral. Available for line integration features of axisymmetric models.
intvolume	on off	off	Compute volume integral. Available for surface integration features of axisymmetric models.
localzphys	array	0	The local z-coordinates, when locdef is set to physical.
localzrel	array of -1-1 numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition, if locinput is set to manual: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when using a Layered Material or Shell dataset. Not available for AvVolume.
locinput	fromdataset manual	fromdataset	The location input: from the dataset or manual. Only available when using a Layered Material or Shell dataset. Not available for AvVolume.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
maximumobj	abs real	real	The value being maximized if dataseries is set to maximum.
method	auto integration summation	auto	The integration method.
minimumobj	abs real	real	The value being minimized if dataseries is set to minimum.
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.

TABLE 7-15: VALID PROPERTIES FOR AVERAGES.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual configuration parent	manual	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
spliteval	on off	off	Evaluate each curve, surface, line, or plane separately. Not available for AvVolume.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
unit	String array	Model dependent	The units to use for the expressions in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[IntVolume](#), [IntSurface](#), [IntLine](#)

Average, Integral, Maximum, Minimum

Create average, integral, maximum, and minimum evaluation datasets.

SYNTAX

```
model.result().dataset().create(<dtag>,"Average");
model.result().dataset().create(<dtag>,"Integral");
model.result().dataset().create(<dtag>,"Maximum");
model.result().dataset().create(<dtag>,"Minimum");
model.result().dataset(<dtag>).set(property,<value>);
model.result().dataset(<dtag>).selection(...);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Average")` creates a dataset named `<dtag>` that computes the average of another dataset.

`model.result().dataset().create(<dtag>,"Integral")` creates a dataset named `<dtag>` that computes the integral of another dataset.

`model.result().dataset().create(<dtag>,"Maximum")` creates a dataset named `<dtag>` that computes the maximum of another dataset.

`model.result().dataset().create(<dtag>,"Minimum")` creates a dataset named `<dtag>` that computes the minimum of another dataset.

`model.result().dataset(<dtag>).selection()` returns the selection of the geometry for the dataset, which by default is the selected geometry in the dataset that this dataset refers to. See [Selections](#) for more information about the available selection methods.

`model.result().dataset(<dtag>).set(property,<value>)` sets the value of a property of the dataset.

The following common properties are available:

TABLE 7-16: COMMON PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
level	fromdataset volume surface edge point	fromdataset	The element dimension; if fromdataset, then the highest dimension supported by the dataset is used.

The following additional properties are available for some of the evaluation datasets:

TABLE 7-17: ADDITIONAL PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
intorderactive	on off	off	For Average and Integral: Whether to use manually specified integration order.
intorder	positive integer	4	For Average and Integral and for Maximum and Minimum when points is set to integration: The manually set integration order.
intsurface	boolean	false	For Average and Integral: Compute surface integral for 1D axisymmetric data.
intvolume	boolean	false	For Average and Integral: Compute volume integral for 2D axisymmetric data.
method	auto integration summation	auto	For Average and Integral: The integration method.

TABLE 7-17: ADDITIONAL PROPERTIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
points	lagrange node integration	lagrange	The evaluation points for the maximum or minimum: Lagrange points, node points, or integration points.
refine	nonnegative integer	2	For Maximum and Minimum when points is set to lagrange: The Lagrange order (element refinement) to use. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.

ATTRIBUTE[Selection \(Dataset Attribute\)](#)*Beam*

Add a beam dataset to visualize 3D geometries of beams.

SYNTAX

```
model.result().dataset().create(<dtag>, "Beam");
model.result().dataset(<dtag>).set(property, <value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>, "Beam")` creates a beam dataset with the name `<dtag>`.

Beam datasets can be used with stored solutions from models with beam physics. Use it to visualize beam structures in 3D using the beams' cross sections.

The following properties are available:

TABLE 7-18: VALID PROPERTIES FOR BEAM DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
physicsinterface	all physics interface name	all	The name of an applicable physics interface, or all for all applicable physics interfaces.
refinement	integer	2	The refinement of the cross section (1–5).
scale	double scalar	1	The scaling of the cross section.
spacevars	String array	Derived from feature name	The names of the space variables for the local beam coordinates.

SEE ALSO[Pipe](#)*Color*

Add a color expression attribute to add a coloring to the shapes defined by a plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>, "Color");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property, <value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "Color")` creates a color expression attribute named <atag> belonging to the plot feature <ftag>.

Use color expressions to add a coloring to the shapes defined by a plot. Color expressions can be added to surface plots; line plots; volume plots; arrow plots; contour plots; isosurface plots; particle tracing plots; and streamline, streamline multislice, and streamline surface plots.

The following properties are available:

TABLE 7-19: VALID PROPERTIES FOR COLOR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colordata	expr arrowlength	expr	Whether to color the arrows using an expression in expr (set to expr) or using the arrow length (set to arrowlength). This property is only available if the Color attribute is added to an arrow plot or a trajectory plot where pointtype is set to arrow.
coloring	colortable gradient	colortable	Whether to use a color table or a color gradient.
colorlegend	on off	on	Whether to show a color legend.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic.
colortable	color table name	Rainbow	The color table to use. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmet ric	none	Transformation of the color table.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.

TABLE 7-19: VALID PROPERTIES FOR COLOR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
prefixintitle	String	Empty	Added prefix to contribution to title.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

COLOR TABLES

The following color tables are available for use with the colortable property here and in other types of plots: AuroraAustralis, AuroraAustralisDark, AuroraBorealis, Cividis, Cyclic, CyclicClassic, Dipole, DipoleDark, Disco, DiscoClassic, DiscoDark, DiscoLight, DiscoLightClassic, Gaia, GaiaLight, GrayBody, GrayBodyLight, GrayPrint, GrayScale, HeatCamera, HeatCameraLight, Inferno, JupiterAuroraBorealis, Magma, Plasma, Prism, PrismDark, Rainbow, RainbowClassic, RainbowDark,

RainbowLight, RainbowLightClassic, Safety, SafetyBlue, Spectrum, SpectrumClassic, SpectrumLight, Thermal, ThermalClassic, ThermalDark, ThermalLight, ThermalLightClassic, ThermalWave, ThermalWaveDark, Traffic, TrafficClassic, TrafficFlow, TrafficLight, TrafficLightClassic, Twilight, Viridis, Wave, WaveClassic, WaveLight, and WaveLightClassic. See the *COMSOL Multiphysics Reference Manual* for more information about the color tables.

SEE ALSO

Deform, Height, AberrationHeight, HistogramHeight, TableHeight, ArrowVolume, ArrowSurface, ArrowLine, ArrowPoint, Contour (Plot), Isosurface (Plot), Particle, ParticleMass, Streamline, TableColor

Comparison

Create a comparison plot for graph plots, comparing it to the data in a table graph.

SYNTAX

```
model.result(<pgtag>).feature(<gtag>).create(<ftag>, "Comparison");
model.result(<pgtag>).feature(<gtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<gtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).feature(<gtag>).create(<ftag>, "Comparison")` creates a comparison subplot feature named `<ftag>` belonging to the plot group `<pgtag>` and graph plot `<gtag>`. Comparison subplot features are available for 1D global, line graph, and point graph plots. The comparison is made against the table data in a table graph plot.

The following properties are available::

TABLE 7-20: VALID PROPERTIES FOR COMPARISON.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
column	column number	first plotted	The column number to use in the table graph. Also for the row number if the table graph uses row-based data.
mapping	yfx xfy closest	yfx	The mapping: difference in the y direction, difference in the x direction, or the closest point.
metric	all maximum rms r2	maximum	The metric to use: all differences, maximum, RMS, or coefficient of determination
plot	table graph plot tag	none	The tag of the table graph with the data to compare to.

Contour (Plot)

Create a contour plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "Contour");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Contour")` creates a contour plot feature named `<ftag>` belonging to the plot group `<pgtag>`. Contour plots are available in 2D and 3D plot groups. Contour lines are colored by their level. Add a color expression attribute to color them by an arbitrary expression.

The following properties are available:

TABLE 7-21: VALID PROPERTIES FOR CONTOUR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the contours: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show a color legend.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
contourlabels	on off	off	Whether to display labels next to the contour lines. Only applicable if contourtype is lines.
contourtype	lines filled tubes	lines	Whether to display lines for each level or a surface with bands of color.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customlabelcolor	RGB-triplet	{1,0,0} or last used color	If contourlabels is on and labelcolor is custom: The label color to use.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.

TABLE 7-21: VALID PROPERTIES FOR CONTOUR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
includeoutside	on off	on	Fill surfaces outside of contour levels if contourtype is filled.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color and deformation scale are inherited from.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
labelcolor	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	If contourlabels is on: The label color to use. The default value uses a color from the color theme.
labelprec	positive	integer	If contourlabels is on: The number of significant digits in the labels.

TABLE 7-21: VALID PROPERTIES FOR CONTOUR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
legendtype	auto filled lines	auto	Whether to show the color legend as a filled legend or using lines. The auto setting provides a filled legend for filled contours and a line legend for line and tube contours.
levelmethod	number levels	number	How to specify contour levels.
levelrounding	boolean	true	Round the contour level values, if levelmethod is set to number.
levels	String array		The specific levels to plot. Active when levelmethod equals levels.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
number	positive integer	20	Total number of contour levels. Active when levelmethod equals number.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	1	Expression for the tube radius, when contourtype is set to tubes.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.

TABLE 7-21: VALID PROPERTIES FOR CONTOUR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.

TABLE 7-21: VALID PROPERTIES FOR CONTOUR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
title	String	The auto-title.	The title to use when <code>titletype</code> is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the <code>title</code> property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when <code>coloring</code> is gradient.
tuberadiusscale	double	Computed automatically	The scale factor for the tube radius when the contour type is set to tubes.
tuberadiusscaleactive	boolean	false	Whether the tube radius scaling is active or not. By default, the tube radius scaling is automatic.
tubescale	boolean	true	Whether the style inheritance of tube radius scale factor is active or not.
typeintitle	on off	on	Whether the title contribution should contain the type when <code>titletype</code> is custom.
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when <code>titletype</code> is custom.

ATTRIBUTES

[Color](#), [Deform](#), [Filter \(Plot Attribute\)](#), [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#) (2D only), [Marker](#), [Transformation](#), [Transparency](#) (3D and also 2D, if a Height attribute is used), [VisualEffects](#) (3D and also 2D, if a Height attribute is used)

SEE ALSO

[Contour \(Dataset\)](#), [ContourSeries](#), [Surface \(Plot\)](#)

Contour (Dataset)

Create a contour dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Contour");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>, "Contour")` creates a contour dataset feature named <dtag>.

Contour lines are in general not parameterizable, and this limits the set of plots and transformations that can be applied to them.

The following properties are available:

TABLE 7-22: VALID PROPERTY/VALUE PAIRS FOR CONTOUR DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
descr	String		Description of expr.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	u	Expression to find constant level for
level	double	0	Level that defines the dataset. Active when levelmethod is level.
levelmethod	number levels	number	How to specify contour levels
levelrounding	boolean	true	Round the contour level values, if levelmethod is set to number.
number	positive integer	20	Total number of contour levels. Active when levelmethod is number.
unit	String		Unit of expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
userder	boolean	true	Whether to use space derivatives to produce smoother plots.

SEE ALSO

[Contour \(Plot\)](#)

ContourSeries

Create a contour series plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "ContourSeries");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "ContourSeries")` creates a contour series plot feature named <ftag> belonging to the plot group <pgtag>. Contour series plots are available in 2D and 3D plot groups. Contour lines are colored by their level. Add a color expression attribute to color them by an arbitrary expression.

The following properties are available:

TABLE 7-23: VALID PROPERTIES FOR CONTOURSERIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the contours: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show a color legend.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
contourlabels	on off	off	Whether to display labels next to the contour lines. Only applicable if contourtype is lines.
contourtype	lines filled tubes	lines	Whether to display lines for each level or a surface with bands of color.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customlabelcolor	RGB-triplet	{1,0,0} or last used color	If contourlabels is on and labelcolor is custom: The label color to use.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.

TABLE 7-23: VALID PROPERTIES FOR CONTOURSERIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
includeoutside	on off	on	Fill surfaces outside of contour levels if contourtype is filled.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color and deformation scale are inherited from.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
labelcolor	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	If contourlabels is on: The label color to use. The default value uses a color from the color theme.
labelprec	positive	integer	If contourlabels is on: The number of significant digits in the labels.

TABLE 7-23: VALID PROPERTIES FOR CONTOURSERIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
legendtype	auto filled lines	auto	Whether to show the color legend as a filled legend or using lines. The auto setting provides a filled legend for filled contours and a line legend for line and tube contours.
levelmethod	number levels	number	How to specify contour levels.
levelrounding	boolean	true	Round the contour level values, if levelmethod is set to number.
levels	String array		The specific levels to plot. Active when levelmethod equals levels.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
number	positive integer	20	Total number of contour levels. Active when levelmethod equals number.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	1	Expression for the tube radius, when the contour type is set to tubes.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.

TABLE 7-23: VALID PROPERTIES FOR CONTOURSERIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.

TABLE 7-23: VALID PROPERTIES FOR CONTOURSERIES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
title	String	The auto-title.	The title to use when <code>titletype</code> is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the <code>title</code> property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when <code>coloring</code> is gradient.
tuberadiusscale	double	Computed automatically	The scale factor for the tube radius when the contour type is set to tubes.
tuberadiusscaleactive	boolean	false	Whether the tube radius scaling is active or not. By default, the tube radius scaling is automatic.
tubescale	boolean	true	Whether the style inheritance of tube radius scale factor is active or not.
typeintitle	on off	on	Whether the title contribution should contain the type when <code>titletype</code> is custom.
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when <code>titletype</code> is custom.

ATTRIBUTES

[Color](#), [Deform](#), [Filter \(Plot Attribute\)](#), [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#) (2D only), [Marker](#), [Transformation](#), [Transparency](#) (3D and also 2D, if a Height attribute is used), [VisualEffects](#) (3D and also 2D, if a Height attribute is used)

SEE ALSO

[Contour \(Plot\)](#), [Contour \(Dataset\)](#), [Surface \(Plot\)](#)

CoordSysLine, *CoordSysSurface*, *CoordSysVolume*, *CoordSysPoint*

Create a coordinate system point (2D and 3D), line (2D and 3D), surface (2D and 3D), or volume plot (3D only).

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"CoordSysVolume");
model.result(<pgtag>).create(<ftag>,"CoordSysSurface");
model.result(<pgtag>).create(<ftag>,"CoordSysLine");
model.result(<pgtag>).create(<ftag>,"CoordSysPoint");

model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"CoordSysVolume")` creates a coordinate system plot named `<ftag>` belonging to the plot group `<pgtag>`.

The following properties are available:

TABLE 7-24: VALID PROPERTIES FOR COORDINATE SYSTEM PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
arrowcount	positive integer	200	If placement is uniform: The approximate number of points to display coordinate systems in. Available for CoordSysLine in 2D and 3D and for CoordSysSurface 3D.
arrowlength	normalized proportional logarithmic	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowtype	arrow cone	arrow	The type of arrow to draw.
arrowxmethod	number coord	number	Indicates whether the x-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for CoordSysSurface in 2D and CoordSysVolume in 3D.
arrowymethod	number coord	number	Indicates whether the y-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for CoordSysSurface in 2D and CoordSysVolume in 3D.
arrowzmethod	number coord	number	Indicates whether the z-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for CoordSysVolume in 3D.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String		If mode is matrix: The description of the matrix expression to plot.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.

TABLE 7-24: VALID PROPERTIES FOR COORDINATE SYSTEM PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String		If mode is matrix: The matrix expression to plot.
inheritarrowsscale	boolean	true	If inheritplot is not none: Determines if arrow scale is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that arrow scale, color, and deformation scale are inherited from.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
mode	system matrix	system	Chooses whether what's plotted is taken from a coordinate system or a 2-by-2 or 3-by-3 matrix variable.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
placement	elements uniform uniformani	uniform	If uniform, then the arrows are distributed uniformly. If uniformanisotropic, then the arrows are distributed uniformly with weights taken from the weight property. If elements, then arrows are drawn in each element. Available for CoordSysLine in 2D and 3D and for CoordSysSurface in 3D.
plotvectors	rows columns	rows	If mode is matrix: Plot matrix as row vectors or column vectors.
scale	positive double	1	If scaleactive is true: The length scale factor.
scaleactive	boolean	false	Whether to use manual scaling.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.

TABLE 7-24: VALID PROPERTIES FOR COORDINATE SYSTEM PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
sys	none coordinate system name	none	The coordinate system to plot, if mode is set to system.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title should be used (the title property). none, if no title should be displayed.
type	stress strain	stress	Selection between a principal stress plot or a principal strain plot.
weight	double array	1	If placement is uniform: The weights given to the different axis directions.
xnumber	nonnegative integer	15 (3D), 7 (2D)	Number of points in the x direction. Active when arrowxmethod is set to number. Available for CoordSysSurface in 2D and CoordSysVolume in 3D.
xcoord	double array	Empty	Absolute coordinates in the x direction, active when arrowxmethod is set to coord. Available for CoordSysSurface in 2D and CoordSysVolume in 3D.
ynumber	integer	15 (3D), 7 (2D)	Number of points in the y direction. Active when arrowymethod is set to number. Available for CoordSysSurface in 2D and CoordSysVolume in 3D.
ycoord	double array	Empty	Absolute coordinates in the z direction, active when arrowymethod is set to coord. Available for CoordSysSurface in 2D and CoordSysVolume in 3D.
znumber	integer	15 (3D), 7 (2D)	Number of points in the z direction. Active when arrowzmethod is set to number. Available for CoordSysVolume in 3D.
zcoord	double array	Empty	Absolute coordinates in the z direction, active when arrowzmethod is set to coord. Available for CoordSysVolume in 3D.

ATTRIBUTES

Deform, Filter (Plot Attribute), Selection (Dataset Attribute), Transformation, Transparency (3D only)

CutLine2D, CutLine3D

Create a 2D or 3D cut line dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"CutLine2D");
model.result().dataset().create(<dtag>,"CutLine3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"CutLine2D")` creates a 2D cut line dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"CutLine3D")` creates a 3D cut line dataset feature named `<dtag>`.

The following properties are available:

TABLE 7-25: VALID PROPERTIES FOR CUT LINE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bndsnap	boolean	false	If true, each point is snapped to the closest boundary. For Cut Line 3D datasets only.
bounded	on off	on	Active when method equals twopoint, this property specifies if the line should be bounded by the two points or extend infinitely.
data	none dataset name	First compatible dataset	The dataset this feature refers to.
genparaactive	on off	off	Decides if parallel lines should be drawn.
genparadist	double array	{}	Active when genparaactive is on, this property contains the distances from the extra parallel lines to the base line.
genpoints	double matrix	{{0, 0, 0}, {1,0,0}}	Active when method equals twopoint, this property contains the coordinates of the two points in the two rows of the matrix.
linevar	String	c1n1number (for the first cut line dataset)	The line number variable for the lines defined by the cut line dataset.
method	twopoint pointdir	twopoint	Decides if the line should be specified by two points or through one point and a direction.
normal	String array of length 2	{"c1n1nx", "c1n1ny"} for the first Cut Line 2D dataset	For Cut Line 2D datasets only: Variables for the normal to the cut line.
orthvec	String array of length 3	{"0", "1", "0"}	Used in 3D when genparaactive is true: The direction that together with the cut line's direction spans the plane that contains the parallel lines.
pddir	String array	Zero string vector	Active when method equals pointdir, this property contains the direction.
pdpoint	String array	Zero string vector	Active when method equals pointdir, this property contains the coordinates of the point.
snapping	none boundary edge	none	No snapping, snap to closest boundary, or snap to closest edge (for CutLine3D only).

TABLE 7-25: VALID PROPERTIES FOR CUT LINE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
spacevars	String array of length 1	Created from the feature tag.	The name of the variable that evaluates to the line's parameterization.
tangent	String array of length 2 or 3	{"c1n1tx", "c1n1ty"} for the first Cut Line 2D dataset	Variables for the tangent to the cut line.

ATTRIBUTE

[Selection \(Dataset Attribute\)](#)

SEE ALSO

[LineGraph](#)

CutPlane

Create a cut plane dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"CutPlane");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"CutPlane")` creates a cut plane dataset with the name `<dtag>`.

The following properties are available:

TABLE 7-26: VALID PROPERTIES FOR CUT PLANE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
genparaactive	on off	off	Decides if parallel lines should be drawn.
genparadist	double array	{}	Active when <code>genparaactive</code> is on, this property contains the distances from the extra parallel planes to the base plane.
genmethod	threepoint pointnormal	threepoint	Active when <code>planetype</code> equals <code>general</code> , this property indicates whether the plane should be specified by three points or through one point and a normal.
genpoints	double matrix	{{0, 0, 0}, {1,0,0}, {0,1,0}}	Active when <code>method</code> equals <code>threepoint</code> , this property contains the coordinates of the three points in the rows of the matrix.
genpnpoint	String array of length three	Zero string vector	Active when <code>genmethod</code> equals <code>pointnormal</code> , this property contains the coordinates of the point.
genpnvec	String array of length three	Zero string vector	Active when <code>genmethod</code> equals <code>pointnormal</code> , this property contains the normal vector.
normal	String array of length three	Created from the feature tag.	The names of the variables that can be used to evaluate the plane's normal.
planetype	quick general	quick	Specify plane type.

TABLE 7-26: VALID PROPERTIES FOR CUT PLANE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
planevar	String	cp11number (for the first cut plane dataset)	The plane number variable for the planes defined by the cut plane dataset.
quickplane	xy yz zx yx zy xz	yz	Specify quick plane type. Active when planetype is quick.
quickx	String	"0"	x-coordinate, if planetype is yz or zy.
quicky	String	"0"	y-coordinate, if planetype is zx or xz.
quickz	String	"0"	z-coordinate, if planetype is xy or yx.
spacevars	String array of length 2	Created from the feature tag.	The names of the variables that evaluate to the plane's parameterization.

EXAMPLES

This example creates a cut plane and plots a surface plot and a contour plot on it.

Create a solution dataset and set it to point to the named solution Sol1 from a solver sequence:

Code for Use with Java

```
DatasetFeature ds = model.result().dataset().create("dset1", "Solution");
ds.set("solution", "sol1");
```

Code for Use with MATLAB

```
ds = model.result.dataset.create('dset1', 'Solution');
ds.set('solution', 'sol1');
```

Create 3D plot group:

Code for Use with Java

```
ResultFeature pg = result().create("pg1",3);
DatasetFeature ds = result().dataset().create("cutp1","CutPlane");
ds.set("data", "dset1");

pg.create("surf1","Surface");
pg.feature("surf1").set("data","cutp1");

pg.create("cont1","Contour");
pg.feature("cont1").set("data","cutp1");
```

Code for Use with MATLAB

```
pg = result.create('pg1',3);
ds = result.dataset.create('cutp1','CutPlane');
ds.set('data', 'dset1');

pg.create('surf1','Surface');
pg.feature('surf1').set('data','cutp1');

pg.create('cont1','Contour');
pg.feature('cont1').set('data','cutp1');
```

ATTRIBUTE

[Selection \(Dataset Attribute\)](#)

SEE ALSO

[Surface \(Plot\)](#)

CutPoint1D, CutPoint2D, CutPoint3D

Create a 1D, 2D, or 3D cut point dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"CutPoint1D");  
model.result().dataset().create(<dtag>,"CutPoint2D");  
model.result().dataset().create(<dtag>,"CutPoint3D");  
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"CutPoint1D")` creates a 1D cut point dataset with the name `<dtag>`.

`model.result().dataset().create(<dtag>,"CutPoint2D")` creates a 2D cut point dataset with the name `<dtag>`.

`model.result().dataset().create(<dtag>,"CutPoint3D")` creates a 3D cut point dataset with the name `<dtag>`.

A cut point is the 0D analog of cut lines and cut planes. A difference compared to cut lines and cut planes is that a cut point feature can contain an arbitrary number of points. Cut points can exist in 1D, 2D, and 3D.

The following properties are available:

TABLE 7-27: VALID PROPERTIES FOR CUT POINT DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
bndsnap	boolean	false	If true, each point is snapped to the closest boundary.
data	none dataset name	First compatible dataset	The dataset this feature refers to.
gridx	double array		If method is grid: The grid x-coordinates.
filename	String		If method is file: The file that contains the point coordinates in a spreadsheet format.
gridy	double array		If method is grid: The grid y-coordinates. (Only for CutPoint2D and CutPoint3D.)
gridz	double array		If method is grid: The grid z-coordinates. (Only for CutPoint3D.)
localzphys	array	0	The local z-coordinates, when locdef is set to physical.
localzrel	array of -1-1 numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition, in Cut Point 3D datasets where data is set to a Layered Material dataset and snapping is set to boundary or mesh: a reference surface, physical z-coordinate, or a relative z-coordinate.
method	coords file grid regulargrid	coords	The method used for defining the points.
pointvar	String	cpt1n (for the first cut point dataset)	Point number variable for the points defined by the cut point dataset.
pointx	String array	{}	If method is coords: The point x-coordinates.
pointy	String array	{}	If method is coords: The point y-coordinates. (Only for CutPoint2D and CutPoint3D.)
pointz	String array	{}	If method is coords: The point z-coordinates. (Only for CutPoint3D.)

TABLE 7-27: VALID PROPERTIES FOR CUT POINT DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
regulargridx	String	10	If method is regulargrid: The number of grid points in the x direction.
regulargridy	String	10	If method is regulargrid: The number of grid points in the y direction. (Only for CutPoint2D and CutPoint3D.)
regulargridz	String	10	If method is regulargrid: The number of grid points in the z direction. (Only for CutPoint3D.)
snapping	none boundary edge	none	No snapping, snap to closest boundary, or snap to closest edge (for CutPoint3D only).

SEE ALSO[PointGraph](#)*Data*

Export data to a file.

SYNTAX

```
model.result().export().create(<ftag>,"Data");
model.result().export().create(<ftag>,<dtag>,"Data");
model.result().export(<ftag>).set(property,<value>);
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>,"Data")` creates a data export feature with the name `<ftag>`.

`model.result().export().create(<ftag>,<dtag>,"Data")` creates a data export feature with the name `<ftag>` for the dataset `<dtag>`.

The following properties are available:

TABLE 7-28: VALID PROPERTIES FOR DATA EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alwaysask	on off	off	Always ask for filename when saving. This property is ignored when running without a UI.
coordfilename	String		If location is file: The file that contains coordinates to evaluate in.
config	configuration name		The configuration to use for the solution times or parameters, if <code>solutionparams</code> is set to configuration.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none dataset name	First compatible dataset	The name of the dataset to export.
descr	String array		Descriptions of the expressions in <code>expr</code> .
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is harmonic.

TABLE 7-28: VALID PROPERTIES FOR DATA EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
exporttype	text vtu	text	File type for the export: a text file or a VTU file.
expr	String array		The expressions to evaluate and export.
filename	String		The output file.
fullprec	on off	on	If on, floating-point numbers are written in full precision, otherwise they are written with six significant digits.
gpporder	positive integer	1	If pattern is gauss: The Gauss-point order.
gridstruct	grid spreadsheet	spreadsheet	If location is grid or regulargrid: The format of the exported data.
gridx1	double array		If location is grid and the dataset is 1D: x-coordinates to evaluate in.
gridx2	double array		If location is grid and the dataset is 2D: x-coordinates to evaluate in.
gridy2	double array		If location is grid and the dataset is 2D: y-coordinates to evaluate in.
gridx3	double array		If location is grid and the dataset is 3D: x-coordinates to evaluate in.
gridy3	double array		If location is grid and the dataset is 3D: y-coordinates to evaluate in.
gridz3	double array		If location is grid and the dataset is 3D: z-coordinates to evaluate in.
header	on off	off	Enable/disable a data header in the output file.
ifexists	append overwrite	overwrite	If the file exists, append to or overwrite the file contents.
includecoords	true false	true	Include spatial coordinate values in the data export.
includenan	true false	true	Include data points whose values are NaN in the data export.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
lagorder	integer	1	The Lagrange-point order to use if resolution is custom.
level	fromdataset volume surface line point	fromdataset	The geometry level to evaluate on.
location	fromdataset file grid regulargrid	fromdataset	The points evaluated in.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level

TABLE 7-28: VALID PROPERTIES FOR DATA EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>looplevelinput</code> is <code>manualindices</code> on a level.
looplevelinput	String array with <code>all</code> <code>first</code> <code>last</code> <code>manual</code> <code>manualindices</code> <code>interp</code> on each level	all on all levels	How to input the solution to use, per level. <code>manual</code> on a level indicates that <code>looplevel</code> is used on that level. <code>manualindices</code> on a level indicates that <code>looplevelindices</code> is used on that level. <code>interp</code> on a level indicates that <code>interp</code> is used on that level.
outerinput	<code>all</code> <code>first</code> <code>last</code> <code>manual</code> <code>manualindices</code>	all	How to input the outer solutions to use. Applicable only for parametric sweep models. <code>manual</code> indicates that <code>outersolnum</code> is used. <code>manualindices</code> indicates that <code>outersolnumindices</code> is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of <code>solnum</code> , the <code>solnum</code> property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>outerinput</code> is <code>manualindices</code> .
pattern	<code>gauss</code> <code>lagrange</code>	<code>lagrange</code>	If the dataset is a solution, particle, edge 2D or 3D, surface, or transformation 2D or 3D: Specifies if evaluation takes place in Lagrange points or in Gauss points.
recover	<code>off</code> <code>pprint</code> <code>ppr</code>	<code>off</code>	The derivative recovery method (<code>off</code> , within domains, or everywhere).
regulargridx1	String	10	If location is <code>regulargrid</code> and the dataset is 1D: The number of grid points along the x-axis.
regulargridx2	String	10	If location is <code>regulargrid</code> and the dataset is 2D: The number of grid points along the x-axis.
regulargridy2	String	10	If location is <code>regulargrid</code> and the dataset is 2D: The number of grid points along the y-axis.
regulargridx3	String	10	If location is <code>regulargrid</code> and the dataset is 3D: The number of grid points along the x-axis.
regulargridy3	String	10	If location is <code>regulargrid</code> and the dataset is 3D: The number of grid points along the y-axis.
regulargridz3	String	10	If location is <code>regulargrid</code> and the dataset is 3D: The number of grid points along the z-axis.
resolution	<code>normal</code> <code>fine</code> <code>finer</code> <code>custom</code>		The evaluation resolution.
separator	<code>C</code> <code>\t</code> <code>,</code> <code>;</code> <code>:</code> <code> </code>	<code>C</code>	The column separator for the data in the output text file. <code>C</code> means column; the second option is a space, and the third option is a tab.
sdim	<code>fromdataset</code> <code>0</code> <code>1</code> <code>2</code> <code>3</code>	<code>fromdataset</code>	The space dimension to evaluate the underlying dataset in.

TABLE 7-28: VALID PROPERTIES FOR DATA EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
solnum	integer array		Solution number to take values from.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1, 1, 20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.
sort	on off	off	Enable/disable sorting of the points with respect to the coordinates.
struct	sectionwise spreadsheet		If location is from dataset: Format of the exported data.
t	double array	Empty	The times to use. Available when the underlying solution is transient.

SEE ALSO

[Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#)

Deform

Add a deformation attribute to a plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>, "Deform");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property, <value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "Deform")` creates a deformation attribute feature with the name `<atag>`, belonging to the feature `<ftag>`.

Deformation attributes deform the coordinates of a plot feature according to an expression (typically, the displacement in a structural mechanics model). The deformation attribute can be added to arrow plots; contour plots; isosurface plots; particle plots; pellets plots; slice plots; streamline, streamline multislice, and streamline surface plots; surface plots; volume plots; and mesh plots.

The following properties are available:

TABLE 7-29: VALID PROPERTY/VALUE PAIRS FOR DEFORM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is <code>custom</code> .
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is <code>harmonic</code> .
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	Model dependent	The expression to plot.

TABLE 7-29: VALID PROPERTY/VALUE PAIRS FOR DEFORM.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
maxreldeformexpr	String	"0,1"	Maximum relative deformation as a fraction of the mean size of the geometry's bounding box.
prefixintitle	String	Empty	Added prefix to contribution to title.
revcoordsys	cylindrical cartesian	cartesian	Coordinate system for the deformation vector. Available for plots that use a Revolution 1D or Revolution 2D dataset that has default axis settings and points to a solution or mesh dataset for an axisymmetric geometry and for plots using a Mirror dataset applied to a Revolution 2D dataset.
scale	positive double	Model dependent	The scale factor for the deformation.
scaleactive	on off	off	Whether to use the automatically computed scale, or the scale in the scale property.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[Color](#), [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#), [ArrowVolume](#), [ArrowSurface](#), [ArrowLine](#), [ArrowPoint](#), [Contour \(Plot\)](#), [Contour \(Plot\)](#), [Isosurface \(Plot\)](#), [Line](#), [Mesh \(Plot\)](#), [Particle](#), [ParticleMass](#), [ScatterVolume](#), [ScatterSurface](#), [Slice](#), [Streamline](#), [StreamlineSurface](#), [Surface \(Plot\)](#), [Transformation](#), [Volume](#).

Directivity

Create a directivity plot. A directivity plot is an extension of far-field plots and is a common acoustic plot for speakers. This plot collects spatial information across frequencies and shows this data as a contour plot.

Directivity plots are available in 1D and 2D plot groups with the Acoustics Module.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Directivity");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Directivity")` creates a directivity plot feature named `<ftag>` belonging to the plot group `<pgtag>`. Directivity plots are available in 1D and 2D plot groups.

The following properties are available:

TABLE 7-30: VALID PROPERTIES FOR DIRECTIVITY PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
anglerestr	manual none	none	Use no angle restriction or a user-defined start angle and range.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
centerz	double	0	z-coordinate for center of evaluation.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
contourlabels	on off	off	Whether to display labels next to the contour lines. Only applicable if contourtype is lines.
contourtype	lines filled	lines	Whether to display lines for each level or a surface with bands of color.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customlabelcolor	RGB-triplet	{1,0,0} or last used color	If contourlabels is on and labelcolor is custom: The label color to use.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.

TABLE 7-30: VALID PROPERTIES FOR DIRECTIVITY PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none parent dataset name	parent	The dataset this feature refers to. From parent is only available in ID.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
filledlegend	on off	on	Whether to show the color legend as a filled legend.
frequenciescale	logarithmic linear	logarithmic	Whether to use a logarithmic or linear frequency scale in the plot (2D only).
includeoutside	on off	on	Fill surfaces outside of contour levels if contourtype is filled.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that color is inherited from.
interp	double array	Time corresponding to last selected solnum for transient levels	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
layout	frequencyx frequencyy	frequencyx	Plot the frequency on the x-axis or on the y-axis.
labelcolor	custom black blue cyan gray green magenta red white yellow	red	If contourlabels is on: The label color to use.
labelprec	positive integer	4	If contourlabels is on: The number of significant digits in the labels.
legendtitle	String		Color legend title, if showlegendtitle has been set to true in the parent plot group.

TABLE 7-30: VALID PROPERTIES FOR DIRECTIVITY PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
levelmethod	number levels	number	How to specify contour levels.
levelrounding	boolean	true	Round the contour level values, if levelmethod is set to number.
levels	String array		The specific levels to plot. Active when levelmethod equals levels.
linewidth	double preference	1	The line width. Only applicable in ID and if contourtype is lines.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
normal	double array of length 3	{0, 1, 0}	The normal direction for the evaluation.
normalization	angle max none	angle	Normalization of the expression values with respect to the angle (the default), with respect to the maximum value, or no normalization.
number	positive integer	20	Total number of contour levels. Active when levelmethod equals number.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
phidisc	integer >= 2	50	The angular discretization (resolution) in the phi direction.
phimin	double	0	If anglerestr is manual: The minimum phi angle in degrees.
phirange	double	360	If anglerestr is manual: The phi angle's range in degrees.
prefixintitle	String	Empty	Added prefix to contribution to title.
radius	double	1	Radius of evaluation distance.
refdir	double array of length 3	{0, 0, 1}	The reference direction for the evaluation.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-30: VALID PROPERTIES FOR DIRECTIVITY PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

None.

SEE ALSO[Contour \(Plot\)](#)*Edge2D, Edge3D*

Create a 2D or 3D edge dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Edge2D");
model.result().dataset().create(<dtag>,"Edge3D");

model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Edge2D")` creates a 2D edge dataset with the name `<dtag>`.

The edge dataset makes it possible to evaluate edges of a model in 1D or the model's dimension.

The following properties are available:

TABLE 7-31: VALID PROPERTY/VALUE PAIRS FOR EDGE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The solution dataset this feature refers to.
reverse	true false	false	Reverse the direction of the edges.

Embed1D, Embed2D

Create a 1D or 2D embed dataset for embedding 1D data into 2D (as a line in 2D) and 2D data into 3D (as a 2D surface in 3D).

SYNTAX

```
model.result().dataset().create(<dtag>,"Embed1D");
model.result().dataset().create(<dtag>,"Embed2D");

model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Embed2D")` creates a 2D embed dataset with the name `<dtag>`.

This dataset embeds 2D data into 3D (as a 2D surface in 3D). You can use an Embed2D dataset to plot 3D arrows for a 2D solution, for example.

The following properties are available:

TABLE 7-32: VALID PROPERTIES FOR EMBED DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
mapping	auto manual	auto	Automatic or manual mapping of the data to embed.
out_x	String	X in first direction, Y in second direction, and 0 in third direction (for Embed2D)	Output space expressions: out_0, out_1, and (Embed2D only) out_2, if mapping is set to manual.
spacevars	String array	Depends on the feature's tag	Names of variables that evaluate to the space coordinates in the space that the data is embedded into.

EnergyDecay

A plot attribute for adding energy decay to a 1D impulse response plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>, "EnergyDecay");  
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property, <value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "EnergyDecay")` adds energy decay information as a plot attribute named `<atag>` to an impulse response named `<ftag>`.

This plot attribute displays energy decay plot with quantities related to the energy decay.

The following properties are available:

TABLE 7-33: VALID PROPERTY/VALUE PAIRS FOR AN ENERGY DECAY ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
bandfreq	A frequency or All	lowest frequency	The band frequency when bandtype is set to indiband.
bandtype	broadband indiband	broadband	The band type: broadband or individual bands.
c50	boolean	true	Include c_{50} in the table.
c80	boolean	true	Include c_{80} in the table.
edt	boolean	true	Include EDT in the table.
plot	energydecay leveldecay mtf	energydecay	What to plot: the energy decay, the level decay, or the modulation transfer function.
snr	boolean	true	Include SNR in the table.
sti	boolean	true	Include STI in the table.
t20	boolean	true	Include t_{20} in the table.
t30	boolean	true	Include t_{30} in the table.
t60	boolean	true	Include t_{60} in the table.
ts	boolean	true	Include t_s in the table.

SEE ALSO

[ImpulseResponse](#)

ErrorBars

A plot attribute for adding error bars to 1D point graph, function, and global plots.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>, "ErrorBars");  
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property, <value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "ErrorBars")` creates error bars as a plot attribute named `<atag>` to a 1D global plot, function plot, or point graph named `<ftag>`.

This plot attribute displays vertical and, optionally, horizontal error bars in the parent graph.

The following properties are available:

TABLE 7-34: VALID PROPERTY/VALUE PAIRS FOR AN ERRORBARS ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
customlinecolor	RGB-triplet	{0,0,0} or last used linecolor	The color to use for the lines. Active when linecolor is set to custom.
errdown	String	0	The expression for the down direction of the error bars.
errleft	String	0	The expression for the left direction of the error bars, if xaxis is true.
errright	String	0	The expression for the right direction of the error bars, if xaxis is true.
errup	String	0	The expression for the up direction of the error bars.
customlinecolor	RGB-triplet	{0,0,0} or last used linecolor	The color to use for the error bars. Active when color is set to custom.
color	custom fromtheme black blue cyan gray green magenta red white yellow	black	How to color the lines or area for the error bars.
linestyle	solid dotted dashed dashdot	solid	The line style, if type is set to bar.
linewidth	integer preference	1	The line width as an integer between 1 and 10.
type	bar filled	bar	The type of plot: error bars or a filled area for the error.
xaxis	true false	false	Add x-axis error bars.

SEE ALSO

[Marker](#), [GraphMarker](#), [TableErrorBars](#), [Global \(Plot\)](#), [PointGraph](#), [Function](#)

Eval

Evaluate expressions in domains from datasets that directly map to a solution.

SYNTAX

```
model.result().numerical().create(<ftag>,"Eval");
model.result().numerical(<ftag>).selection(...);
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getData();
model.result().numerical(<ftag>).getData(<expressionIndex>);
model.result().numerical(<ftag>).getImagData();
model.result().numerical(<ftag>).getImagData(<expressionIndex>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).getNData();
model.result().numerical(<ftag>).getCoordinates();
model.result().numerical(<ftag>).getElements();
model.result().numerical(<ftag>).getVertexElements();
model.result().numerical(<ftag>).run();
```

DESCRIPTION

`model.result().numerical().create(<ftag>, "Eval")` creates an evaluation feature with the name <ftag>.

Eval is a feature made specifically for users of the COMSOL API and does not appear in the COMSOL Multiphysics Desktop. It is used to evaluate results directly on the solution.

`result = model.result().numerical(<ftag>).getData()` returns the real part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][coordinates]`.

`model.result().numerical(<ftag>).getData(<expressionIndex>)` returns the real part of the result for one expression, equivalent to `result[expressionIndex]`.

`result = model.result().numerical(<ftag>).getImagData()` returns the imaginary part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][coordinates]`.

`model.result().numerical(<ftag>).getImagData(<expressionIndex>)` returns the imaginary result for one expression, equivalent to `result[expressionIndex]`.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. <outersolnum> is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).getNData()` returns the number of points in the data vector.

`model.result().numerical(<ftag>).getCoordinates()` returns node point coordinates.

`model.result().numerical(<ftag>).getElements()` returns indices to columns in `p` of a simplex mesh.

`model.result().numerical(<ftag>).getVertexElements()` returns indices to mesh elements for each point.

The following properties are available:

TABLE 7-35: VALID PROPERTIES FOR EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an <i>N</i> -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none dataset name	First compatible dataset	The dataset this feature refers to. Only Solution, Particle, Shell, and Ray datasets are supported.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to evaluate.
complexfun	on off	on	Use complex-valued functions with real input.

TABLE 7-35: VALID PROPERTIES FOR EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
gporder	positive integer	1	If pattern is gauss: The Gauss-point order.
matherr	on off	off	Error for undefined operation or variable.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
refine	auto integer	auto	Refinement of elements for evaluation points.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere).
pattern	gauss lagrange	lagrange	Specifies if evaluation takes place in Lagrange points or in Gauss points.
phase	double	0	Evaluate solution at this angle, given in degrees.
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[Interp](#), [Global \(Numerical\)](#), [EvalPoint](#), [EvalGlobal](#)

EvalAberration

Perform optical aberration evaluations to compute a list of Zernike coefficients for Zernike polynomials that correspond to various types of monochromatic aberration, which arise when electromagnetic rays are focused by a system of lenses and mirrors. An Intersection Point 3D dataset (see [IntersectionPoint2D](#), [IntersectionPoint3D](#)) pointing to a [Ray \(Dataset\)](#) dataset must be used.



The EvalAberration optical aberration evaluation requires a license for the Ray Optics Module.

SYNTAX

```
model.result().numerical().create(<ftag>, "EvalAberration");
model.result().numerical(<ftag>).set(property, <value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name *<etag>*, replace *numerical(<ftag>)* with *evaluationGroup(<etag>).feature(<ftag>)*.

DESCRIPTION

model.result().numerical().create(<ftag>, "EvalAberration") creates an optical aberration evaluation feature with the name *<ftag>*.

model.result().numerical(<ftag>).getReal() returns the real-valued Zernike coefficients. Data is ordered so that there is one row with length equal to the number of Zernike polynomials up to the specified order.

model.result().numerical(<ftag>).getImag(allocate, columnwise) always returns null because the Zernike coefficients are always real valued.

model.result().numerical(<ftag>).isComplex() always returns false because the Zernike coefficients are real valued.

model.result().numerical(<ftag>).computeResult() returns the matrix of data containing the real and imaginary parts of the Zernike coefficients; the latter is always null.

model.result().numerical(<ftag>).setResult() and *model.result().numerical(<ftag>).appendResult()* evaluate the feature and set or append the result in the table indicated by the *table* property.

The following properties are available:

TABLE 7-36: VALID PROPERTIES FOR OPTICAL ABERRATION EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
additionallogical-expression	String	1	Additional logical expression that must be nonzero for rays to be included in the plot.
additionallogical-expressionactive	on off	off	If on, activates an additional logical expression for including rays in the Zernike coefficient calculation.
data	none dataset name	First compatible dataset	The intersection 3D dataset this feature refers to
filterrelease	positive integer	1	Index of the release feature of rays to include if <i>filterreleaseactive</i> is on.
filterreleaseactive	on off	off	If on, filter rays by release feature to determine which rays are included in the Zernike coefficient calculation.
filterwavelength	String	632.8[nm]	Wavelength of rays to include if <i>filterwavelengthactive</i> is on.
filterwavelengthactive	on off	off	If on, filter rays by wavelength to determine which rays are included in the Zernike coefficient calculation.

TABLE 7-36: VALID PROPERTIES FOR OPTICAL ABERRATION EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
filterwavelengthtol	String	1 [nm]	Numerical tolerance for filtering rays by wavelength if filterwavelengthactive is on.
lunit	any length unit	\u00b5m (micron)	Length unit for computing Zernike coefficients.
maxorder	2, 3, 4, or 5	5	Maximum polynomial order for the Zernike coefficients calculation,
numberofreflections	nonnegative integer	0	Rays that have reflected the specified number of times will be included in the plot. Used if numberofreflectionsactive is on.
numberofreflections -active	on off	off	If on, activates the filtering of rays based on number of reflections.

EvalGlobal

Evaluate global quantities.

SYNTAX

```

model.result().numerical().create(<ftag>,"EvalGlobal");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();

```

When added to an evaluation group of name <etag>, replace numerical(<ftag>) with evaluationGroup(<etag>).feature(<ftag>).

DESCRIPTION

model.result().numerical().create(<ftag>,"EvalGlobal") creates an evaluation feature with the name <ftag>.

model.result().numerical(<ftag>).getReal() returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to (columnwise) when columnwise is false. If columnwise is true, the ordering is the opposite: each column contains the values for all solution numbers.

model.result().numerical(<ftag>).getImag(allocate,columnwise) returns the imaginary part of the complex result, recomputing the feature if necessary. If allocate is true, a zero-valued matrix is allocated even when the result is real. getImag() uses allocate true and columnwise false.

model.result().numerical(<ftag>).isComplex() returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. `<outersolnum>` is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluate the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-37: VALID PROPERTY/VALUE PAIRS FOR GLOBAL EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an <i>N</i> -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	First compatible dataset	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
dataseries	average integral maximum minimum none rms stddev variance	none	The transformation that is applied to the data series formed by the evaluation.
dataseriescumulative	true false	false	Make the table values as a cumulative integration, when <code>dataseries</code> is set to <code>integral</code> .
dataseriesmethod	auto integration summation	auto	The method to use for the data series: automatic, integration, or summation.
descr	String array	Model dependent	The descriptions of the expressions in <code>expr</code> . Is used in the automatic title.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is <code>harmonic</code> .
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.
includeparam	true false	false	Include the parameter value or time if <code>dataseries</code> is set to <code>maximum</code> or <code>minimum</code> for a parametric or time-dependent solution,

TABLE 7-37: VALID PROPERTY/VALUE PAIRS FOR GLOBAL EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
maximumobj	abs real	real	The value being maximized if dataseries is set to maximum.
minimumobj	abs real	real	The value being minimized if dataseries is set to minimum.
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.

TABLE 7-37: VALID PROPERTY/VALUE PAIRS FOR GLOBAL EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and this feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
unit	String array	Model dependent	The units to use for the expressions in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[EvalGlobalMatrix](#), [Eval](#), [EvalPoint](#)

EvalGlobalMatrix

Evaluate global matrix quantities.

SYNTAX

```

model.result().numerical().create(<ftag>,"EvalGlobalMatrix");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();

```

DESCRIPTION

model.result().numerical().create(<ftag>,"EvalGlobalMatrix") creates an evaluation feature with the name <ftag>.

model.result().numerical(<ftag>).getReal() returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to (columnwise) when

`columnwise` is `false`. If `columnwise` is `true`, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate, columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If *allocate* is `true`, a zero-valued matrix is allocated even when the result is real. `getImag()` uses *allocate* `true` and *columnwise* `false`.

`model.result().numerical(<ftag>).isComplex()` returns `true` if the result is complex. The resulting value is a scalar, which `true` if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns `true` if the result is complex for the given outer solution. *<outersolnum>* is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-38: VALID PROPERTIES FOR GLOBAL MATRIX EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>const</code>	String matrix	Empty or physics dependent	Parameters to use in the expressions as an <i>N</i> -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
<code>data</code>	<code>none</code> <code>parent</code> <code>dataset name</code>	First compatible dataset	The dataset this feature refers to. <code>parent</code> is only applicable if used in an evaluation group.
<code>dataseries</code>	<code>average</code> <code>none</code> <code>sum</code>	<code>none</code>	The operation that is applied to the data series formed by the evaluation over inner parameters (solutions).
<code>descr</code>	String	Model dependent	The description of the expression in <code>expr</code> . Is used in the automatic title.
<code>differential</code>	<code>on</code> <code>off</code>	<code>on</code>	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is <code>harmonic</code> .
<code>evalmethod</code>	<code>linpoint</code> <code>harmonic</code> <code>lintotal</code> <code>lintotalavg</code> <code>lintotalrms</code> <code>lintotalpeak</code>	<code>harmonic</code>	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
<code>expr</code>	String	Model dependent	The matrix variable to plot.
<code>ignorenan</code>	<code>on</code> <code>off</code>	<code>on</code>	When <code>on</code> , NaN is ignored when taking the average in the inner or outer data series. If all entries are NaN for a given component, however, you get NaN as output.

TABLE 7-38: VALID PROPERTIES FOR GLOBAL MATRIX EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
outerdataseries	average none sum	none	The operation that is applied to the data series formed by the evaluation over outer parameters (solutions).
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.

TABLE 7-38: VALID PROPERTIES FOR GLOBAL MATRIX EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and this feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
symmetric	on off	off	Whether to make the matrix symmetric.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling <code>setResult()</code> or <code>appendResult()</code> . <code>new</code> indicates that a new table is created.
tablerows	inner outer	inner	Whether to use inner or outer solutions as rows in the table when calling <code>setResult()</code> or <code>appendResult()</code> . Applicable only for parametric sweep models.
trans	none inverse sy sz ys yz zs zy maxwellmutual mutualmaxwell invmaxwellmutual	none	The transformation to apply to the matrix. <code>sy</code> means a transformation from S to Y, and so on. The <code>maxwellmutual</code> <code>mutualmaxwell</code> , and <code>invmaxwellmutual</code> transformations are from Maxwell to mutual capacitance, and vice versa, and from an inverse Maxwell to mutual capacitance.
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.
y0	double array	Taken from the physics interfaces.	If <code>trans</code> is <code>sy</code> or <code>ys</code> : The characteristic admittance.
z0	double array	Taken from the physics interfaces.	If <code>trans</code> is <code>sz</code> or <code>zs</code> : The characteristic impedance.

SEE ALSO

[EvalGlobal](#), [Eval](#), [EvalPoint](#), [EvalPointMatrix](#), [EvalGlobalSweep](#)

EvalGlobalSweep

Vary a small number of model parameters as a global evaluation sweep.

SYNTAX

```
model.result().numerical().create(<ftag>,"EvalGlobalSweep");
model.result().numerical(<ftag>).setIndex(...);
model.result().numerical(<ftag>).set(property,<value>);
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.result().numerical().create(<ftag>,"EvalGlobalSweep")` creates a global evaluation sweep for sweeping over a small number of model parameters for results processing of reduced models, for example.

The following properties are available:

TABLE 7-39: VALID PROPERTIES FOR GLOBAL EVALUATION SWEEP.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none dataset name	First compatible dataset	The dataset this feature refers to.
descr	String array	Model dependent	The descriptions of the expressions in <code>expr</code> .
expr	String array	Model dependent	The expressions to plot.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or <code>interp</code> or <code>last</code> , but only for transient solutions. Available when <code>data</code> is not <code>parent</code> and the underlying data has multiple solutions.
plistarr	real matrix		Lists of parameter values. One row of values for each parameter name.
pname	vector of strings		Parameter names.
sweeptype	sparse filled	sparse	Method for doing the parameter variation. For <code>sweeptype = sparse</code> , the parameter tuples defined by the columns in <code>plistarr</code> are solved for. This method requires equal length for the rows. For <code>sweeptype = filled</code> , all parameter combinations given by <code>plistarr</code> are solved for.
unit	String array	Model dependent	The units to use for the expressions in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[EvalGlobal](#), [Eval](#), [EvalPoint](#), [EvalPointMatrix](#), [EvalGlobalMatrix](#)

EvalPoint

Evaluate quantities in points.

SYNTAX

```
model.result().numerical().create(<ftag>,"EvalPoint");
model.result().numerical(<ftag>).selection(...);
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"EvalPoint")` creates a point evaluation feature with the name <ftag>. Point evaluations can be performed both on points in a geometry or on cut points.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that there is one row per point, with one row containing data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is `false`. If `columnwise` is `true`, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is `true`, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` `true` and `columnwise` `false`.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. <outersolnum> is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-40: VALID PROPERTIES FOR EVALUATION IN POINTS.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	First compatible dataset	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
dataseries	average integral maximum minimum none rms stddev variance	none	The transformation that is applied to the data series formed by the evaluation.
dataseriescumulative	true false	false	Make the table values as a cumulative integration, when dataseries is set to integral.
dataseriesmethod	auto integration summation	auto	The method to use for the data series: automatic, integration, or summation.
descr	String array	Model dependent	The descriptions of the expressions in expr. Is used in the automatic title.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.
includeparam	true false	false	Include the parameter value or time if dataseries is set to maximum or minimum for a parametric or time-dependent solution,
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
localzphys	scalar	0	The local z-coordinates, when locdef is set to physical.
localzrel	scalar, -1-1	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition, if locinput is set to manual: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when using a Layered Material or Shell dataset.

TABLE 7-40: VALID PROPERTIES FOR EVALUATION IN POINTS.

NAME	VALUE	DEFAULT	DESCRIPTION
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
maximumobj	abs real	real	The value being maximized if dataseries is set to maximum.
minimumobj	abs real	real	The value being minimized if dataseries is set to minimum.
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and this feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.

TABLE 7-40: VALID PROPERTIES FOR EVALUATION IN POINTS.

NAME	VALUE	DEFAULT	DESCRIPTION
summethod	none sum average	none	Summation method to compute the sum or average for the expressions at all selected points.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
unit	String array	Model dependent	The units to use for the expressions in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[EvalGlobal](#), [EvalGlobalSweep](#)

EvalPointMatrix

Evaluate matrix quantities at points in the geometry.

SYNTAX

```

model.result().numerical().create(<ftag>,"EvalPointMatrix");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
model.result().numerical(<ftag>).selection();

```

DESCRIPTION

model.result().numerical().create(<ftag>,"EvalPointMatrix") creates an evaluation feature with the name <ftag>.

model.result().numerical(<ftag>).getReal() returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to (columnwise) when columnwise is false. If columnwise is true, the ordering is the opposite: each column contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate, columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If *allocate* is `true`, a zero-valued matrix is allocated even when the result is real. `getImag()` uses *allocate* `true` and *columnwise* `false`.

`model.result().numerical(<ftag>).isComplex()` returns `true` if the result is complex. The resulting value is a scalar, which `true` if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns `true` if the result is complex for the given outer solution. *<outersolnum>* is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the *table* property.

`model.result().numerical(<ftag>).selection()` selects the points where the matrix quantity should be evaluated. `model.result().numerical(<ftag>).selection().all()`; selects all points in the geometry.

The following properties are available:

TABLE 7-41: VALID PROPERTIES FOR GLOBAL MATRIX EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>const</code>	String matrix	Empty or physics dependent	Parameters to use in the expressions as an <i>N</i> -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
<code>data</code>	<code>none</code> <code>parent</code> <code>dataset name</code>	First compatible dataset	The dataset this feature refers to. <code>parent</code> is only applicable if used in an evaluation group.
<code>dataseries</code>	<code>average</code> <code>none</code> <code>sum</code>	<code>none</code>	The operation that is applied to the data series formed by the evaluation over inner solutions.
<code>descr</code>	String	Model dependent	The description of the expression in <code>expr</code> . Is used in the automatic title.
<code>differential</code>	<code>on</code> <code>off</code>	<code>on</code>	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is <code>harmonic</code> .
<code>evalmethod</code>	<code>linpoint</code> <code>harmonic</code> <code>lintotal</code> <code>lintotalavg</code> <code>lintotalrms</code> <code>lintotalpeak</code>	<code>harmonic</code>	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
<code>expr</code>	String	Model dependent	The matrix variable to plot.
<code>ignorenan</code>	<code>on</code> <code>off</code>	<code>on</code>	When <code>on</code> , NaN is ignored when taking the average in the inner or outer data series. If all entries are NaN for a given component, however, you get NaN as output.

TABLE 7-41: VALID PROPERTIES FOR GLOBAL MATRIX EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
localzphys	scalar	0	The local z-coordinates, when locdef is set to physical.
localzrel	scalar, -1-1	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition, if locinput is set to manual: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when using a Layered Material or Shell dataset.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
outerdataseries	average none sum	none	The operation that is applied to the data series formed by the evaluation over outer solutions.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.

TABLE 7-41: VALID PROPERTIES FOR GLOBAL MATRIX EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and this feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
symmetric	on off	off	Whether to make the matrix symmetric.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablerows	inner outer	inner	Whether to use inner or outer solutions as rows in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[EvalGlobal](#), [Eval](#), [EvalPoint](#), [EvalGlobalMatrix](#), [EvalGlobalSweep](#)

EvaluationGroup

Create an evaluation group for grouping related evaluation nodes and presenting the result in an evaluation group table.

SYNTAX

```
model.result().evaluationGroup().create(<egtag>,<eglabel>);
model.result().evaluationGroup(<egtag>).getRowHeaders()
model.result().evaluationGroup(<egtag>).set(property,<value>);
model.result().evaluationGroup(<egtag>).run();
```

DESCRIPTION

model.result().evaluationGroup().create(<egtag>,<egname>) creates an evaluation group named <egtag> with a label <eglabel>. An evaluation group is a group of evaluation nodes (derived values) that are evaluated and displayed in an evaluation group table. Use

model.result().evaluationGroup(<egtag>).getRowHeaders() to retrieve the row headers from an

evaluation group. Use `model.result().evaluationGroup(<egtag>).removeRows(<rows>)` to remove the rows listed as an array of positive integers as the input argument.

The following properties are available for evaluation groups:

TABLE 7-42: VALID PROPERTIES FOR EVALUATION GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
config	configuration name		The configuration to use for the solution times or parameters, if <code>solutionparams</code> is set to configuration.
concatenation	horizontal vertical	horizontal	Concatenate contributions from evaluation features horizontally or vertically.
data	none dataset name	First compatible dataset	The dataset this feature refers to. This is the default dataset for all plots in the group.
generalexpr	String		A general expression for transformation of evaluations (using the tags of the evaluation subfeatures). Active when <code>method</code> is set to <code>general</code> .
includeparameters	auto true false	auto	Include the parameter value columns from the first evaluation feature (auto), all parameter value columns (true), or no parameter value column (false).
innerinput	all first last manual manualindices interp	all	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when <code>data</code> is not none and the underlying data is transient.
keepchildnodes	boolean	false	Keep the evaluation of the child nodes when <code>type</code> is not set to none.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1, 1, 20)</code> . Applicable when <code>looplevelinput</code> is <code>manualindices</code> on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. <code>manual</code> on a level indicates that <code>looplevel</code> is used on that level. <code>manualindices</code> on a level indicates that <code>looplevelindices</code> is used on that level. <code>interp</code> on a level indicates that <code>interp</code> is used on that level.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. <code>manual</code> indicates that <code>outersolnum</code> is used. <code>manualindices</code> indicates that <code>outersolnumindices</code> is used.

TABLE 7-42: VALID PROPERTIES FOR EVALUATION GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
solnum	Integer array	All solutions	The solutions to plot. Available when the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.
storetable	inmodel inmodelandonfile onfile	inmodel	Controls whether evaluation group table is stored in model (inmodel), on file (onfile), or both (inmodelandonfile).
tablebuffersize	positive integer	Taken from buffer size preference. The default of which is 10000.	The size of the in-memory buffer size where the evaluation group table is stored, used when storetable is set to inmodel or inmodelandonfile. In the second case only the last tablebuffersize rows are kept in the model; the rest is read from file when necessary.
transpose	true false	false	Transpose the table data in the evaluation group so that the presentation contains row headers instead of column headers.
type	none sum average difference general	none	Transformation type for the evaluations: No transformation, sum, average, difference, or a general expression (the generalexpr property)

SEE ALSO

[AvVolume](#), [AvSurface](#), [AvLine](#), [EvalAberration](#), [EvalGlobal](#), [Eval](#), [EvalPoint](#), [MaxVolume](#), [MaxSurface](#), [MaxLine](#), [MinVolume](#), [MinSurface](#), [MinLine](#), [PlotGroup1D](#), [PlotGroup2D](#), [PlotGroup3D](#)

Export

Add an export expressions attribute to a plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Export");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "Export")` creates an export expressions attribute named `<atag>` belonging to the plot feature `<ftag>`.

Use export expressions to include additional quantities in plot data export.

Export expressions can be added to particle trajectories, point trajectories, radiation pattern, ray trajectories, streamline, streamline multislice, and streamline surface plots.

The following properties are available:

TABLE 7-43: VALID PROPERTY/VALUE PAIRS FOR EXPORT EXPRESSIONS ATTRIBUTES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
descr	String array		The description of the expressions in expr.
expr	String array		The expressions to export in addition to the plot data.
unit	String		The unit to use for the expression in expr.

Extrude1D, Extrude2D

Create a 1D or 2D extrude dataset for extruding data in results processing from 1D to 2D and from 2D to 3D.

SYNTAX

```
model.result().dataset().create(<dtag>, "Extrude1D");
model.result().dataset().create(<dtag>, "Extrude2D");

model.result().dataset(<dtag>).set(property, <value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>, "Extrude2D")` creates a 2D extrude dataset with the name `<dtag>`.

The extrude dataset makes it possible to postprocess a 1D or 2D solution that is extruded into 2D or 3D.

The following properties are available:

TABLE 7-44: VALID PROPERTIES FOR EXTRUDE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The solution dataset this feature refers to.
planemap	xy xz yz	xy	The extrusion direction as a plane to map the extrusion to: the xy-plane, xz-plane, or yz-plane (for extrusion 2D only).
res	integer, at least 2	10	The resolution as the number of layers in the extrusion.
seplevels	true false	false	Separate the levels to visualize data for each equidistant point in time, for example.
ymin	double	0	The minimum for the extrusion 1D range.
ymax	double	1	The maximum for the extrusion 1D range.
yvar	String	extr1y	The name of the variable for the extrusion direction for a 1D extrusion.
zmin	double	0	The minimum for the extrusion 2D range.
zmax	double	1	The maximum for the extrusion 2D range.

TABLE 7-44: VALID PROPERTIES FOR EXTRUDE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
zvar	String	extriz	The name of the variable for the extrusion direction for a 2D extrusion.
wavenumber	integer	0	The out-of-plane wave number for a 2D extrusion.

Filter (Dataset)

Create a filter dataset for filtering data using lower and upper bounds.

SYNTAX

```
model.result().dataset().create(<dtag>,"Filter");

model.result().dataset(<dtag>).set(property,<value>);

model.result().dataset(<dtag>).createMeshBasedGeom(<compTag>,<meshBasedGeomTag>);
model.result().dataset(<dtag>).createMesh(<compTag>,<geomTag>,<meshTag>,<importTag>);
model.result().dataset(<dtag>).createMeshPart(<meshTag>,<geomTag>,<meshPartTag>,<importTag>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Filter")` creates a filter dataset with the name `<dtag>`.

The filter dataset makes it possible to filter out values in an expression from another dataset using lower and upper bounds. Using the `createMeshBasedGeom`, `createMesh`, and `createMeshPart` methods, you can create a mesh-based geometry, mesh, or mesh part with a mesh import that refers to the Filter dataset.

The following properties are available:

TABLE 7-45: VALID PROPERTIES FOR FILTER DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bounds	lower upper lowerupper	lower	The type of bounds for the filter: a lower bound, an upper bound, or both lower and upper bounds.
data	none dataset name	First compatible dataset	The solution dataset this feature refers to.
descr	String		A description for the expression to filter.
expr	String		The expression for which to apply the filter.
level	fromdataset volume surface line point	fromdataset	The geometry level to evaluate on.
lowerexpr	String	0	A value or expression for the lower bound, if bounds is set to lower or lowerupper.
propagatetolower	true false	true	Propagate the filter to lower geometry dimensions.
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in <code>smoothexpr</code> .

TABLE 7-45: VALID PROPERTIES FOR FILTER DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
upperexpr	String	0	A value or expression for the upper bound, if bounds is set to upper or lowerupper.
useder	true false	true	Use cubic (Hermite) interpolation when applying the filter.

SEE ALSO[Partition](#)*Filter (Plot Attribute)*

Add a filter attribute to a plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Filter");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"Filter")` creates an element filter attribute named `<atag>` belonging to the plot feature `<ftag>`.

Use filters to limit plots to elements satisfying a condition.

Filters can be added to the following 1D, 2D, and 3D plots: arrow, contour, global, histogram, isosurface, line graph, line plot, point graph, slice, volume, max/min, Nyquist, admittance graph, impedance graph, reflection graph, streamline, streamline multislice, streamline surface, and through thickness plots.

The following properties are available:

TABLE 7-46: VALID PROPERTIES FOR FILTER ATTRIBUTES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
expr	String	1	The logical expression that must be satisfied for an element to be included in the plot.
nodespec	all any xor smooth	smooth	The set of nodes in an element that have to satisfy the logical expression for it to be included in the plot. The smooth option creates smooth edges by “cutting” through elements. Available in 2D and 3D only.
useder	true false	false	Use cubic (Hermite) interpolation in smooth plot filters; active if nodespec is set to smooth.
xdec	true false	true	Include line segments with decreasing x-coordinates in Global, PointGraph, and LineGraph 1D plots.
xinc	true false	true	Include line segments with increasing x-coordinates in Global, PointGraph, and LineGraph 1D plots.

TABLE 7-46: VALID PROPERTIES FOR FILTER ATTRIBUTES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
ydec	true false	true	Include line segments with decreasing y-coordinates in Global, PointGraph, and LineGraphID plots.
yinc	true false	true	Include line segments with increasing y-coordinates in Global, PointGraph, and LineGraphID plots.

Filter (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing)

Add a filter attribute to a grain, grain positions, particle trajectories, particle, point trajectories, ray, or ray tracing plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"GrainTrajectoriesFilter");
model.result(<pgtag>).feature(<ftag>).create(<atag>,"ParticleTrajectoriesFilter");
model.result(<pgtag>).feature(<ftag>).create(<atag>,"PointTrajectoriesFilter");
model.result(<pgtag>).feature(<ftag>).create(<atag>,"RayTrajectoriesFilter");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"ParticleTrajectoriesFilter")` creates a filter attribute named `<atag>` belonging to the plot feature `<ftag>`.

Use filters to plot a subset of grains, particles, points, or rays in a dataset.

Filters can be added to grain positions and grain plots; particle trajectories plots and particle plots; point trajectories plots; and ray trajectories plots and ray plots.

The following properties are available:

TABLE 7-47: VALID PROPERTIES FOR THE FILTERS FOR PARTICLE TRAJECTORIES, POINT TRAJECTORIES, AND RAY TRACING.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
evaluate	all fraction number	all	Determines which grains/particles/rays are rendered when creating the plot.
fraction	Double	1	Fraction of grains/particles/rays that are rendered. Active when evaluate is set to fraction.
logical	Double	1	Logical statement that must be true for a grain, particle, point, or ray to be included in the plot. Active when type is set to logical.
number	Integer	100	Number of grains, particles, points, or rays that are rendered. Active when evaluate is set to number.
type	all primary secondary logical	all	Determines which grains, particles, points, or rays that are included when creating the plot. primary and secondary are not available for point trajectories and grain positions.

Filter (Table Graph)

Add a filter attribute to a table graph plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"TableFilter");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"TableFilter")` creates a filter attribute named `<atag>` belonging to the Table Graph plot feature `<ftag>`.

Use filters to plot a subset of columns or rows in a table graph.

The following property is available:

TABLE 7-48: VALID PROPERTY FOR THE FILTERS FOR TABLE GRAPH PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
expr	String		Determines which columns or rows in the table to include in the table graph (specified as " <i>col1</i> , <i>col2</i> , ..." or similar for rows).

Function

Create a function plot as a graph plot of a defined function.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Function");
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Function")` creates a function plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

A function plot is used to visualize a function as a line, as points, or as its frequency spectrum. Function plots can be added to 1D plot groups.

The following properties are available:

TABLE 7-49: VALID PROPERTIES FOR FUNCTION PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
colorcycle	default long	default	The color cycle, when <code>linecolor</code> is set to <code>cycle</code> . With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when <code>linecolor</code> is set to <code>cyclecolortable</code> . See Color Tables for a list of color tables.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when <code>bottomcolor</code> is set to <code>custom</code> .

TABLE 7-49: VALID PROPERTIES FOR FUNCTION PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
dbmanualref	double	0	Reference value for display in dB, if dbref is manual (and indb is true).
dbref	maximum manual	maximum	Use the maximum value of a manual value as the dB reference when indb is true.
dbtype	10log 20log	10log	Use a 10 log or 20 log for the dB values when indb is true.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
display	line linepoints points fourier	line	How to display the function: as a line, as line and points, as points, or as a discrete Fourier transform (DFT).
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
extrapolation	none left right leftright	none	How to extrapolate the function outside the argument's bounds: no extrapolation or extrapolation to the left, to the right, or to both the left and the right of the bounds.
extrapolationcolor	custom black blue cyan gray green magenta red white yellow	red	The color of the extrapolation zones, if extrapolation is not none.
fouriershow	real imaginary amplitude phase spectrum	amplitude	When display is fourier: Show the DFT as real or imaginary part, amplitude, phase, or frequency spectrum.-
freqmax	integer		If fouriershow is spectrum and freqrangeactive is true: The upper frequency bound.
freqmin	integer		If fouriershow is spectrum and freqrangeactive is true: The lower frequency bound.
freqrangeactive	true false	false	If fouriershow is spectrum: Controls whether a manual frequency range is used.

TABLE 7-49: VALID PROPERTIES FOR FUNCTION PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
ignorelast	true false	true	If fouriershow is not spectrum: ignore last sample.
indb	true false	false	Display the amplitude or frequency spectrum in dB when fouriershow is amplitude or spectrum.
inverse	true false	false	If fouriershow is not spectrum: inverse the discrete Fourier transform.
legend	true false	false	Whether to show legends.
legendmethod	automatic manual	automatic	Whether to use the automatic legends or the legends supplied in the legends property.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolorortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
lowerbound	double	0	The lower bound for the function's argument.
maskdc	true false	false	If fouriershow is not spectrum: set the DC value (zero frequency component) to zero.
nfreqs	integer	1	If fouriershow is spectrum and nfreqsactive is true: The number of frequencies to plot.
nfreqsactive	true false	false	If fouriershow is spectrum: Controls whether the number of frequencies is set manually.
nsamples	integer	1	If fouriershow is not spectrum and nsamplesactive is true: The number of uniform samples to use.
nsamplesactive	true false	false	If fouriershow is not spectrum: Controls whether the number of uniform samples is set manually.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.

TABLE 7-49: VALID PROPERTIES FOR FUNCTION PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
pointdef	String		The tag of the dataset for the point definition, if display is set to linepoints or points.
pointcolor	custom fromtheme black blue cyan gray green magenta red white yellow	black	The point color.
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if twoyaxes is set to true in the parent plot group.
prefixintitle	String	Empty	Added prefix to contribution to title.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
scale	none multiplyperiod dividenfREQ	none	If fouriershow is spectrum, use no scaling, multiply by sampling period, or divide by number of frequencies.
scaleperiod	true false	true	If fouriershow is not spectrum: scale with sampling period.
shiftzero	true false	true	If fouriershow is not spectrum: shift zero frequency.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
upperbound	double	1	The upper bound for the function's argument.
xdataexpr	String	Model dependent	Expression for x-axis data.

TABLE 7-49: VALID PROPERTIES FOR FUNCTION PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
xdataexpr	String	Model dependent	Description of expression in xdataexpr.
xdataunit	String	Model dependent	The unit to use for the expression in xdataexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES[ErrorBars](#)**SEE ALSO**[Global \(Plot\)](#), [LineGraph](#), [Function2D](#), [Function3D](#)*Function2D*

Create a function 2D plot to visualize a function of two variables as a surface plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "Function2D");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Function2D")` creates a surface plot feature named `<ftag>` belonging to the 2D plot group `<pgtag>`.

Function 2D plots display a function of two variables as a colored 2D surface.

The following properties are available:

TABLE 7-50: VALID PROPERTIES FOR FUNCTION2D PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the surface: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.

TABLE 7-50: VALID PROPERTIES FOR FUNCTION2D PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expr	String	Model dependent	The expression to plot: a function of two variables such as $\sin(x, y)$.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
legendtitle	String		Color legend title, if showlegendstyle has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
prefixintitle	String	Empty	Added prefix to contribution to title.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.

TABLE 7-50: VALID PROPERTIES FOR FUNCTION2D PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String		The unit to use for the expression in expr.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xunit	String		The unit to use for the x-axis in the plot.
yunit	String		The unit to use for the y-axis in the plot.

ATTRIBUTES

[Filter \(Plot Attribute\)](#); [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#); [Transparency](#) (if a height attribute is used)

SEE ALSO

[Surface \(Plot\)](#), [Function](#), [Function3D](#)

Function3D

Create a function 3D plot to visualize a function of three variables as a slice plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "Function3D");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Function3D")` creates a slice plot feature named `<ftag>` belonging to the 3D plot group `<pgtag>`.

Function 3D plots display a function of three variables as a colored 3D slice plot.

The following properties are available:

TABLE 7-51: VALID PROPERTIES FOR FUNCTION3D PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the surface: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expr	String	Model dependent	The expression to plot: a function of two variables such as $an1(x,y)$.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
gendistance	double array	Empty	Vector of distances from base plane, indicating position of additional parallel planes. Active when genparaactive equals on and genpara equals number.

TABLE 7-51: VALID PROPERTIES FOR FUNCTION3D PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
genmethod	threepoint pointnormal	threepoint	Active when planetype equals general, this property indicates whether the plane should be specified by three points or through one point and a normal.
gennumber	positive integer	4	Number of additional parallel planes. Active when genparaactive equals on and genpara equals number.
genpara	gennumber gendistances	number	Active when planetype equals general, and genparaactive equals on, this property indicates whether the additional parallel planes should be specify by a number or by a vector of relative distances.
genparaactive	on off	off	Active when planetype equals general, this property indicates whether there should be additional parallel planes created.
genpnpoint	String array of length three	{"0", "0", "0"}	Active when genmethod equals pointnormal, this property contains the coordinates of the point.
genpnvec	String array of length three	{"0", "0", "0"}	Active when genmethod equals pointnormal, this property contains the normal vector.
genpoints	double matrix of size 3x3	{{0,0,0}, {1,0,0}, {0,1,0}}	Active when genmethod equals threepoint, this property contains the coordinates of the three points, with one row per point.
legendtitle	String		Color legend title, if showlegendtitle has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
planetype	quick general	yz	Specify plane type.
prefixintitle	String	Empty	Added prefix to contribution to title.
quickplane	xy yz zx	yz	Specify quick plane type. Active when planetype is quick.
quickx	String array	0	Absolute x-coordinates for planes, active when quickxmethod is set to coord.
quickxmethod	number coord	number	Active when planetype equals quick and quickplane equals yz, this property indicates whether the planes should be specified by one or more absolute coordinates along the x-axis, or a number indicating the number of planes to distribute evenly.

TABLE 7-51: VALID PROPERTIES FOR FUNCTION3D PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
quickxnumber	String	5	The number of planes when quickxmethod is set to number. It can contain a global expression for the number of slices.
quicky	String array	0	Absolute y-coordinates for planes, active when quickymethod is set to coord.
quickymethod	number coord	number	Active when planetype equals quick and quickplane equals zx, this property indicates whether the planes should be specified by one or more absolute coordinates along the y-axis, or a number indicating the number of planes to distribute evenly.
quickynumber	String	5	The number of planes when quickymethod is set to number. It can contain a global expression for the number of slices.
quickz	String array	0	Absolute z-coordinates for planes, active when quickzmethod is set to coord.
quickzmethod	number coord	number	Active when planetype equals quick and quickplane equals xy, this property indicates whether the planes should be specified by one or more absolute coordinates along the z-axis, or a number indicating the number of planes to distribute evenly.
quickznumber	String	5	The number of planes when quickzmethod is set to number. It can contain a global expression for the number of slices.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title.	The title to use when titletype is manual.

TABLE 7-51: VALID PROPERTIES FOR FUNCTION3D PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String		The unit to use for the expression in expr.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xunit	String		The unit to use for the x-axis in the plot.
yunit	String		The unit to use for the y-axis in the plot.
zunit	String		The unit to use for the z-axis in the plot.

ATTRIBUTES

[Filter](#) (Plot Attribute); [Transparency](#)

SEE ALSO

[Slice](#), [Function](#), [Function2D](#)

Global (Numerical)

Evaluate global quantities.

SYNTAX

```
model.result().numerical().create(<ftag>,"Global");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getData();
model.result().numerical(<ftag>).getData(<expressionIndex>);
model.result().numerical(<ftag>).getImagData();
model.result().numerical(<ftag>).getImagData(<expressionIndex>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).getNData();
model.result().numerical(<ftag>).run();
```

DESCRIPTION

`model.result().numerical().create(<ftag>,"Global")` creates an evaluation feature with the name `<ftag>`.

`Global` is a feature made specifically for users of the COMSOL API and does not appear in the COMSOL Desktop. `Global` features support multiple expressions as well as some additional advanced properties not available in [EvalGlobal](#).

`result = model.result().numerical(<ftag>).getData()` returns the real part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][value]`.

`model.result().numerical(<ftag>).getData(<expressionIndex>)` returns the real part of the result for one expression, equivalent to `result[expressionIndex]`.

`result = model.result().numerical(<ftag>).getImagData()` returns the imaginary part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][value]`.

`model.result().numerical(<ftag>).getImagData(<expressionIndex>)` returns the imaginary part of the result for one expression, equivalent to `result[expressionIndex]`.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. `<outersolnum>` is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).getNData()` returns the number of points in the data vector.

The following properties are available:

TABLE 7-52: VALID PROPERTIES FOR GLOBAL EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an <i>N</i> -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	First compatible dataset	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
descr	String array		Descriptions for the expressions to evaluate.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to evaluate.
complexfun	on off	on	Use complex-valued functions with real input.
matherr	on off	off	Error for undefined operation or variable.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
phase	double	0	Evaluate solution at this angle, given in degrees.
solnum	nonnegative integer array	All solutions	The solutions to use.
t	double array	Empty	The times to use. Available when the underlying solution is transient.

TABLE 7-52: VALID PROPERTIES FOR GLOBAL EVALUATION.

NAME	VALUE	DEFAULT	DESCRIPTION
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
unit	String array	Model dependent	The units to use for the expressions in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[Eval](#), [Interp](#), [EvalPoint](#), [EvalGlobal](#)

Global (Plot)

Create a global plot for 1D plot groups and polar plot groups.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Global");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Global")` creates a plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Global plot is used to visualize defined variables, such as solutions to ODEs or coupling operators with a destination. Global plots can be added to 1D and polar plot groups.

The following properties are available:

TABLE 7-53: VALID PROPERTIES FOR GLOBAL PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.

TABLE 7-53: VALID PROPERTIES FOR GLOBAL PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cycletlength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
dbmanualref	double	0	Reference value for display in dB, if dbref is manual (and indb is true).
dbref	maximum manual	maximum	Use the maximum value of a manual value as the dB reference when indb is true.
dbtype	10log 20log	10log	Use a 10 log or 20 log for the dB values when indb is true.
descr	String array	Model dependent	The description of the expressions in expr. Is used in the automatic legends.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
fouriershow	real imaginary amplitude phase spectrum	amplitude	When xdata is fourier: Show the DFT as real or imaginary part, amplitude, phase, or frequency spectrum.-
freqmax	integer		If fouriershow is spectrum and freqrangeactive is true: The upper frequency bound.

TABLE 7-53: VALID PROPERTIES FOR GLOBAL PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
frequmin	integer		If fouriershow is spectrum and freqrangearg is true: The lower frequency bound.
freqrangearg	true false	false	If fouriershow is spectrum: Controls whether a manual frequency range is used.
ignorelast	true false	true	If fouriershow is not spectrum: ignore last sample.
indb	true false	false	Display the amplitude or frequency spectrum in dB when fouriershow is amplitude or spectrum.
inverse	true false	false	If fouriershow is not spectrum: inverse the discrete Fourier transform.
functype	continuous discrete	continuous	Plot the function data as a continuous or a discrete function.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.

TABLE 7-53: VALID PROPERTIES FOR GLOBAL PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
maskdc	true false	false	If fouriershow is not spectrum: set the DC value (zero frequency component) to zero.
nfreqs	integer	1	If fouriershow is spectrum and nfreqsactive is true: The number of frequencies to plot.
nfreqsactive	true false	false	If fouriershow is spectrum: Controls whether the number of frequencies is set manually.
nsamples	integer	1	If fouriershow is not spectrum and nsamplesactive is true: The number of uniform samples to use.
nsamplesactive	true false	false	If fouriershow is not spectrum: Controls whether the number of uniform samples is set manually.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.

TABLE 7-53: VALID PROPERTIES FOR GLOBAL PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if twoyaxes is set to true in the parent plot group.
prefixintitle	String	Empty	Added prefix to contribution to title.
refine	nonnegative integer	1	Refinement of the time resolution for time-dependent studies (an integer 1–1000).
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
scale	none multiplyperiod dividenfREQ	none	If fouriershow is spectrum, use no scaling, multiply by sampling period, or divide by number of frequencies.
scaleperiod	true false	true	If fouriershow is not spectrum: scale with sampling period.
shiftzero	true false	true	If fouriershow is not spectrum: shift zero frequency.
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.

TABLE 7-53: VALID PROPERTIES FOR GLOBAL PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
title	String	The auto-title.	The title to use when <code>titletype</code> is <code>manual</code> .
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the <code>title</code> property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
typeintitle	on off	on	Whether the title contribution should contain the type when <code>titletype</code> is <code>custom</code> .
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when <code>titletype</code> is <code>custom</code> .
unwrapphase	boolean	false	Unwrap the phase, if the expression to plot has the unit of an angle.
xdata	expr solution fourier phase	solution	x-axis data. <code>expr</code> uses the expression in <code>xdataexpr</code> . <code>solution</code> uses the available solutions in the underlying dataset, such as time steps. <code>fourier</code> uses a discrete Fourier transform (DFT). <code>phase</code> uses a phase range, and rearranged phase plots.
xdataexpr	String	Model dependent	Expression for x-axis data.
xdatadescr	String	Model dependent	Description of expression in <code>xdataexpr</code> .
xdataphaserange	double array	$(\pi/10) * \text{range}(0,1,20)$	The phases for which the expressions should be evaluated when <code>xdata</code> is <code>phase</code> .
xdataphaseunit	String	rad	The unit in which <code>xdataphaserange</code> is described.
xdatasolnumtype	all inner outer valid level	outer	Whether the expression should be evaluated for every inner or every outer solution, or for a specific level (<code>level1</code> , <code>level2</code> , and so on). Applicable only for models containing multiple levels.
xdataunit	String	Model dependent	The unit to use for the expression in <code>xdataexpr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES

[Color](#), [Comparison](#), [ErrorBars](#), [Filter \(Plot Attribute\)](#), [GraphMarker](#)

SEE ALSO

[LineGraph](#), [PointGraph](#)

Grain (1D Plot)

Create a grain plot, which plots grain properties over time or compares grain properties against each other at a set of time steps.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "Grain1D");  
model.result(<pgtag>).feature(<ftag>).set(property, <value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Grain1D")` creates a grain plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

With the grain plot, you can plot properties of all grains in a dataset versus time, or plot two grain properties against each other at a set of selected times. When plotting grain properties versus time, it is possible to apply data series operations to the grain data. The grain plot is available in 1D plot groups.

The following properties are available:

TABLE 7-54: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
colorcycle	default long	default	The color cycle, when <code>linecolor</code> is set to <code>cycle</code> . With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when <code>linecolor</code> is set to <code>cyclecolortable</code> . See Color Tables for a list of color tables.

TABLE 7-54: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgecolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
dataseries	none average sum maximum minimum rms stddev variance	sum	The data series operation that is applied to all particles. Effective only when xdata is not expr.
descr	String array	Model dependent	The description of the expressions in expr. Is used in the automatic legends.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expr	String array	Model dependent	The expressions to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.

TABLE 7-54: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclength is manual.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.

TABLE 7-54: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>outerinput</code> is <code>manualindices</code> .
prefixintitle	String	Empty	Added prefix to contribution to title.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when <code>linecolor</code> is <code>cyclecolortable</code> .
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
solrepresentation	<code>solnum</code> <code>solutioninfo</code>	<code>solutioninfo</code>	Indicates which method of selecting solutions is active.
solutionparams	<code>manual</code> <code>parent</code>	<code>manual</code>	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when <code>titletype</code> is <code>manual</code> .
titletype	<code>auto</code> <code>custom</code> <code>label</code> <code>manual</code> <code>none</code>	<code>auto</code>	<code>auto</code> , if the title should be computed automatically. <code>custom</code> , if the title should be computed automatically, but customized. <code>label</code> , if the title should be the plot group's label. <code>manual</code> , if the manual title should be used (the <code>title</code> property). <code>none</code> , if no title should be displayed.
typeintitle	<code>on</code> <code>off</code>	<code>on</code>	Whether the title contribution should contain the type when <code>titletype</code> is <code>custom</code> .
topcolor	<code>custom</code> <code>black</code> <code>blue</code> <code>cyan</code> <code>gray</code> <code>green</code> <code>magenta</code> <code>red</code> <code>white</code> <code>yellow</code>	<code>white</code>	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	<code>on</code> <code>off</code>	<code>on</code>	Whether the title contribution should contain the unit when <code>titletype</code> is <code>custom</code> .

TABLE 7-54: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
xdata	expr solution	solution	x-axis data. expr uses the expression in xdataexpr. solution uses the available solutions in the underlying dataset, such as time steps.
xdataexpr	String	Model dependent	Expression for x-axis data.
xdatadescr	String	Model dependent	Description of expression in xdataexpr.
xdataphaseunit	String	rad	The unit in which xdataphaserange is described.
xdatasolnumtype	all inner outer valid level	outer	Whether the expression should be evaluated for every inner or every outer solution, or for a specific level (level1, level2, and so on). Applicable only for models containing multiple levels.
xdataunit	String	Model dependent	The unit to use for the expression in xdataexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES

[Color](#) (applies to lines), [Filter](#) (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing)

SEE ALSO

[Grain](#) (Evaluation), [GrainTrajectories](#), [GrainBin](#)

Grain (Dataset)

Create a grain dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Grain");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

model.result().dataset().create(<dtag>,"Grain") creates a grain dataset.

A grain dataset is required to plot grain positions. The grain dataset identifies the geometry in which the grain data is stored and the degrees of freedom which determine the position of each grain.

The following properties are available:

TABLE 7-55: VALID PROPERTIES FOR PARTICLE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
solution	String	first available solution	The solution this dataset refers to.
pgeomspec	manual fromphysics	fromphysics if an applicable physics interface is available, otherwise manual	The method of specifying the particle geometry and the names of the grain position degrees of freedom.

TABLE 7-55: VALID PROPERTIES FOR PARTICLE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
physicsinterface	none or the tag of a valid physics interface	none or first applicable physics interface	The physics interface from which the particle geometry and the names of the grain position degrees of freedom are defined.
pgeom	String	pgeom or pgeom_ followed by the tag of a granular flow physics interface	The geometry in which the grain degrees of freedom are defined. The correct name of this particle geometry is pgeom_<tag>, where <tag> is the name for the Granular Flow interface node.
posdof	String array	Position vector components of a granular flow physics interface	The position degrees of freedom of the grains.

SEE ALSO

[Grain \(Evaluation\)](#)

Grain (Evaluation)

Grain evaluations to evaluate quantities on grain positions.

SYNTAX

```

model.result().numerical().create(<ftag>,"Grain");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();

```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"Grain")` creates a grain evaluation feature with the name <ftag>. Grain evaluation can be performed on trajectories accessed through a grain dataset.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that there is one row per point, with one row containing data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is `false`. If `columnwise` is `true`, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is `true`, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate true` and `columnwise false`.

`model.result().numerical(<ftag>).isComplex()` returns `true` if the result is complex. The resulting value is a scalar, which `true` if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. `<outersolnum>` is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-56: VALID PROPERTIES FOR PARTICLE EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>data</code>	<code>none</code> dataset name	First grain compatible dataset	The grain dataset this feature refers to.
<code>descr</code>	String	Model dependent	The description of the expression in <code>expr</code> . Is used in the automatic title.
<code>evaluate</code>	<code>all</code> <code>fraction</code> <code>number</code>	<code>all</code>	What grains to evaluate.
<code>expr</code>	String	Model dependent	The expression to evaluate.
<code>fraction</code>	<code>double</code>	<code>1</code>	If <code>evaluate</code> is <code>fraction</code> : The fraction of grains to evaluate.
<code>innerinput</code>	<code>all</code> <code>first</code> <code>last</code> <code>manual</code> <code>manualindices</code> <code>interp</code>	<code>all</code>	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
<code>interp</code>	<code>double</code> row matrix	Empty on all levels	The times to use, for transient levels. Available when <code>data</code> is not parent and the underlying data is transient.
<code>looplevel</code>	<code>integer</code> row matrix	All solutions on all levels	The solutions to use, per level.
<code>looplevelindices</code>	<code>integer</code> row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>looplevelinput</code> is <code>manualindices</code> on a level.
<code>looplevelinput</code>	String array with <code>all</code> <code>first</code> <code>last</code> <code>manual</code> <code>manualindices</code> <code>interp</code> on each level	<code>all</code> on all levels	How to input the solution to use, per level. <code>manual</code> on a level indicates that <code>looplevel</code> is used on that level. <code>manualindices</code> on a level indicates that <code>looplevelindices</code> is used on that level. <code>interp</code> on a level indicates that <code>interp</code> is used on that level.
<code>number</code>	<code>double</code>	<code>100</code>	If <code>evaluate</code> is <code>number</code> : The number of grains to evaluate.

TABLE 7-56: VALID PROPERTIES FOR PARTICLE EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

GrainBin

Create a grain bin dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"GrainBin");  
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"GrainBin")` creates a grain bin set. This dataset evaluates an expression over all grains and then groups them into subintervals, or bins, based on which grains return similar values. If this dataset is then used in any other grain positions plot or evaluation, each bin produces a single grain having position and other properties equal to the average over grains in the bin.

The following properties are available:

TABLE 7-57: VALID PROPERTIES FOR PARTICLE BIN DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First grain dataset	The grain dataset this feature refers to
distribution	equalnumber equalwidth	equalnumber	Distribution of grain bins. Active when method is set to number.
limits	String		String listing the limits of all bins. Active when method is set to limits.
method	limits number tolerance	limits	The method used to define the limits of the grain bins
number	positive integer	10	Number of grain bins. Active when method is set to number.
tolerance	String		Maximum difference between expression values such that particles are placed in the same bin. Active when method is set to tolerance.

GrainTrajectories

Create a grain positions plot in 2D and 3D.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"GrainTrajectories");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"GrainTrajectories")` creates a grain positions plot feature named `<ftag>` belonging to plot group `<pgtag>`.

A grain dataset is required to plot the grain positions. The grain dataset identifies the geometry in which the grain data is stored and the degrees of freedom which determine the position of each grain.

The following properties are available:

TABLE 7-58: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	head	Whether the head, tail, or center of the arrow is located at the grain position.
arrowexpr	string array		Active when pointtype is set to arrow. Determines the direction in which the arrow points.

TABLE 7-58: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowlength	logarithmic normalized proportional	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowscale	positive double	1	If arrowscaleactive is true: the length scale factor.
arrowscaleactive	boolean	false	If true, arrowscale is used, otherwise the scale factor is computed automatically.
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.
comettailexpr	String array	inverse of velocity vector of the first available particle tracing interface	Active when pointtype is set to comettail. Determines the direction in which the comet tail points and the relative sizes of the tails of different grains.
data	none parent dataset name	parent	The dataset this feature refers to.
ellipsearrowbase	head tail	head	The base for the arrow to draw for ellipses.
ellipsearrowtype	arrow cone none	arrow	The type of arrow to draw for ellipses.
ellipsearrowscale	double		The ellipse arrow scale factor, if ellipsearrowscaleactive is set to true.
ellipsearrowscaleactive	boolean	false	Set to true to specify an ellipse arrow scale when pointtype is ellipse.
ellipsecount	positive integer	10	Maximum number of ellipses, when ellipseentrymethod is numberofellipses.
ellipseentrymethod	numberofellipses listoftimes	numberofellipses	The ellipse entry type when pointtype is ellipse: number of ellipses or a list of times.
ellipsetimes	String array		Collection of times for polarization ellipses when ellipseentrymethod is listoftimes.
extrasteps	none specifiedtimes proportional	proportional	Controls how the number of extra time steps rendered in the trajectory plot is calculated. These extra time steps typically correspond to the exact times of reflections or velocity reinitializations.
fixedpointsize	boolean	false	Use constant radii for the plotted spheres.

TABLE 7-58: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
illuminatelines	on off	on	Whether the plotted lines are affected by lighting when <code>linetype</code> is <code>line</code> . Available in 3D only.
inheritarrowsscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the arrow scale is inherited.
inheritcolor	boolean	true	If <code>inheritplot</code> is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the deformation scale is inherited.
inheritellipsescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the ellipse scale factor is inherited.
inheritplot	none plot name	none	The plot that color, color range, and deformation scale are inherited from.
inheritrange	boolean	true	If <code>inheritplot</code> is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the radius scale factor is inherited.
inherittailscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the tail scale factor is inherited.
inherittransparency	boolean	true	If <code>inheritplot</code> is not none: Determines if the transparency settings are inherited.
inherittubescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the tube radius scale factor is inherited.
interpcount	positive integer	100	Number of interpolated time, when interpolation is set to uniform.
interpolation	none uniform	none	The type of interpolation used to plot lines when <code>linetype</code> is <code>line</code> , <code>ribbon</code> , or <code>tube</code> .
linecolor	custom black blue cyan gray green magenta red white yellow	black (2D), red (3D)	The uniform color to use for lines.
linetype	none line ribbon tube	line	Plot grain traces as lines, ribbons, tubes, or not at all.
logrange	double > 1	100	If <code>arrowlength</code> is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.

TABLE 7-58: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
numextrasteps	nonnegative integer	100	If extrasteps is specifiedtimes, controls the maximum number of extra time steps to render in the plot.
pointautoscale	on off	on	If enabled, the point radii are scaled based on the size of the geometry.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.
pointlineanim	on off	off	When animating grain position plots with points, this property indicates whether the points at the current time should appear on top of the lines.
pointradiusexpr	String	0.001	The point radius expression.
pointtype	none point comettail arrow ellipse	none	Plot grain as points, with comet tails, with arrows, with polarization ellipses (3D only), or not at all.
propextrasteps	nonnegative integer	1	If extrasteps is proportional, controls the maximum number of extra time steps to render in the plot. The maximum number of extra steps is the product of this proportionality factor with the number of solution times.
radiusexpr	String	1	The tube radius expression
ribbondirexpr	String array		Expressions for the components of the ribbon direction.
semimajorexpr	String array		Expressions for the polarization ellipses' semi-major axis,
semiminorexpr	String array		Expressions for the polarization ellipses' semi-minor axis,
sphereradiusscale	positive double		The scale factor applied to the point radii if sphereradiusscaleactive is on.
sphereradiusscaleactive	on off	off	If on, sphereradiusscale is used; otherwise the scale factor is computed automatically.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
tailscale	positive double	1	Active when pointtype is set to comettail and tailscaleactive is on. Specifies the manual scale factor with which the comet tail expression is multiplied.

TABLE 7-58: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
tailscaleactive	on off	off	Active when pointtype is set to comettail. Specifies whether manual tail scaling is enabled.
title	String		Active when title is set to manual. Specifies the plot title.
titletype	automatic manual none	automatic	Determines how the plot title is specified.
tuberadiussscale	String	1	The scale factor applied to the tube radii if tuberadiusscaleactive is on.
tuberadiussscaleactive	on off	off	If on, tuberadiussscale is used; otherwise, the scale factor is computed automatically.
widthscale	double	1	The scale factor applied to the ribbon widths if widthscaleactive is true.
widthscaleactive	boolean	false	If true, widthscale is used; otherwise, the scale factor is computed automatically.

ATTRIBUTES

[Color](#), [Deform](#), [Export](#), [Filter](#) (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing), [MaterialAppearance](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[Grain](#) (Evaluation), [ParticleTrajectories](#), [RayTrajectories](#)

GraphMarker

A plot attribute for creating graph markers such as minimum and maximum markers for 1D point graph, line graph, table graph, global, and radiation pattern plots.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>, "GraphMarker");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property, <value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "GraphMarker")` creates a minimum and maximum or bandwidth marker plot named `<atag>` as an attribute to a 1D global plot or point graph named `<ftag>`.

This plot attribute displays the minimum and maximum, and the points there they are attained, or bandwidth of the expression in the parent graph.

The following properties are available:

TABLE 7-59: VALID PROPERTIES FOR A GRAPHMARKER ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
backgroundcolor	none fromtheme custom black blue cyan gray green magenta red white yellow	none	The color to use for a background rectangle, or none for no background.
color	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	The color with which markers are plotted. The default takes the color from the current color theme.
customlinecolor	RGB-triplet	{0,0,0} or last used linecolor	The color to use for the lines. Active when linecolor is set to custom.
cutoffmode	offsetfrompeak relativetopeak absolute	offsetfrompeak	Cutoff mode: offset from peak, relative to peak, or absolute. The absolute option is only available if the scope is set to global.
cutoffset	double	-3	The cutoff value, if cutoffmode is set to offsetfrompeak or relativetopeak.
cutoffvalabs	double	-3	The cutoff value, if cutoffmode is set to absolute.
display	minandmax min max	minandmax	The selection determines which minimum and maximum markers that are shown when displaymode is minmax.
displaymode	minmax widthmode intersection	minmax	The selection determines which type of markers that are shown: min and max (the default), width, or line intersection.
generalcoeffa	double		The A coefficient, if intersectionline is set to general.
generalcoeffb	double		The B coefficient, if intersectionline is set to general.
generalcoeffc	double		The C coefficient, if intersectionline is set to general.
includeunit	true false	false	Whether to include the unit for the values.
inclxcoord	true false	false	Whether to include x-coordinate for the maximum and minimum locations, when displaymode is set to minmax.
inclycoord	true false	true	Whether to include y-coordinate for the maximum and minimum locations, when displaymode is set to minmax.

TABLE 7-59: VALID PROPERTIES FOR A GRAPHMARKER ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
intersectionline	vertical horizontal identity general	vertical	The type of intersection lines: vertical (the default), horizontal, an identity line ($y=x$), or a general line $Ax+By+C=0$.
intersectionx	double array		The x-coordinates for vertical lines, if intersectionline is set to vertical.
intersectiony	double array		The y-coordinates for horizontal lines, if intersectionline is set to horizontal.
linecolor	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	How to color the lines in the graph. The default takes the color from the current color theme.
linewidth	double preference	1	The line width.
linestyle	solid dotted dashed dashdot	solid	The line style.
orientation	horizontal vertical	horizontal	Orient the marker text horizontally or vertically.
precision	integer > 0	6	The number of decimals displayed in the labels in the UI.
prependcoords	true false	false	Prepend the position of the marker, if displaymode is set to minmax.
rangetype	width widthoutside	width	The type of bandwidth plot, when displaymode is widthmode.
relativetopeak	true false	false	If the cutoff value is relative to the peak value, when displaymode is widthmode.
scope	global local	global	The scope of the markers. Not available when rangetype is intersection.
showframe	true false	false	Show a rectangular frame around the marker text.
showlines	true false	false	Show the intersection lines, if displaymode is set to intersection.
threshold	double	-3	The cutoff value, when displaymode is widthmode.
usecubicinterpolation	true false	false	Use cubic interpolation to find minimum and maximum points, when displaymode is set to minmax.

SEE ALSO

[Marker](#), [MaxMinVolume](#), [MaxMinSurface](#), [MaxMinLine](#), [MaxMinPoint](#)

Grid1D, Grid2D, Grid3D

Create a dataset that can evaluate 1D, 2D, or 3D functions or other datasets on a domain with a grid. For example, you can use these datasets to evaluate BEM and far-field operators.

SYNTAX

```
model.result().dataset().create(<dtag>,"Grid1D");
model.result().dataset().create(<dtag>,"Grid2D");
model.result().dataset().create(<dtag>,"Grid3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Grid1D")` creates a 1D grid dataset feature named `<dtag>`.

This dataset provides support for evaluation of functions or other datasets on a domain with a grid. All functions in the same function list as the selected function can also be evaluated. The domain is an interval for `Grid1D`, a rectangle for `Grid2D`, and a block for `Grid3D`. The domain does not need to have the same dimension as the number of arguments to the function.

The following properties are available:

TABLE 7-60: VALID PROPERTIES FOR GRID DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
adaptive	true false	true	For <code>Grid1D</code> only. If active, it can downsample 1D datasets by removing unnecessary evaluation points to improve graph plots.
auxvar1	String	Depends on dimension	Auxiliary variable for the first dimension, if <code>defineauxvars</code> is set to true.
auxvar2	String	Depends on dimension	Auxiliary variable for the second dimension, if <code>defineauxvars</code> is set to true. (For <code>Grid2D</code> and <code>Grid3D</code> .)
auxvar3	String	Depends on dimension	Auxiliary variable for the third dimension, if <code>defineauxvars</code> is set to true. (For <code>Grid3D</code> .)
auxunit1	String		Unit for auxiliary variable for the first dimension, if <code>defineauxvars</code> is set to true.
auxunit2	String		Unit for auxiliary variable for the second dimension, if <code>defineauxvars</code> is set to true. (For <code>Grid2D</code> and <code>Grid3D</code> .)
auxunit3	String		Unit for auxiliary variable for the third dimension, if <code>defineauxvars</code> is set to true. (For <code>Grid3D</code> .)
data	none dataset tag	none	The dataset this feature refers to.
defineauxvars	true false	false	Define auxiliary variables.
distribution	uniform exponential mixed	uniform	The point distribution: uniform, exponential, or mixed uniform/exponential.
function	none all function tag	none	The function this feature refers to.
par _i	String	Model dependent	The name of the parameter. <code>par1</code> , <code>par2</code> , and <code>par3</code> for <code>Grid3D</code> , for example.
parmin1	double	0	Lower bound for the first dimension of the domain.
parmax1	double	1	Upper bound for the first dimension of the domain.
parmin2	double	0	Lower bound for the second dimension of the domain. (For <code>Grid2D</code> and <code>Grid3D</code> .)

TABLE 7-60: VALID PROPERTIES FOR GRID DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
parmax2	double	1	Upper bound for the second dimension of the domain. (For Grid2D and Grid3D.)
parmin3	double	0	Lower bound for the third dimension of the domain. (For Grid3D.)
parmax3	double	1	Upper bound for the third dimension of the domain. (For Grid3D.)
res _i	integer >= 2	1000 for 1D, 100 for 2D, and 30 for 3D	The number of points into which each dimension is discretized. res1, res2, and res3 for Grid 3D, for example.
source	function data	function	Use a function or a dataset as the source.

Height, AberrationHeight, HistogramHeight, TableHeight

Add a height attribute for 2D contour, line, and surface plots; for 2D optical aberration plots; for 2D histogram, matrix histogram, and table histogram plots, and for 2D table surface plots.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Height");
model.result(<pgtag>).feature(<ftag>).create(<atag>,"AberrationHeight");
model.result(<pgtag>).feature(<ftag>).create(<atag>,"HistogramHeight");
model.result(<pgtag>).feature(<ftag>).create(<atag>,"TableHeight");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"Height")` creates a Height attribute feature with the name <atag>, belonging to the feature <ftag>.

Height attributes are available for 2D contour, line, and surface plots, and with slightly different sets of properties, for 2D optical aberration, histogram, matrix histogram, table histogram, and table surface plots. Adding a height attribute changes the rendered view to 3D. The plot group, however, still remains a 2D plot group. Any other plots in the group that do not have a height attribute are rendered at $z = 0$. 2D plots with a height attribute can also use a transparency attribute.

The following properties are available:

TABLE 7-61: VALID PROPERTIES FOR HEIGHT ATTRIBUTES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coldata	integer		Table data for the height for a table surface with the columns table format. An integer that specifies the column number to use.
data	none filled table data	none	Table data for the height for a table surface with the filled table format.
descr	String	Model dependent	The description of the expression in expr. It is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.

TABLE 7-61: VALID PROPERTIES FOR HEIGHT ATTRIBUTES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	Model dependent	The expression to plot. For Height attributes only.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
heightdata	parent expr or parent data	parent	parent uses the expression and description in the plot the height feature has been added to. expr uses the expr and descr properties (for Height attributes only). data uses some specified data from the data or coldata properties (for TableHeight attributes only).
imagplot	on off	off	Plot the imaginary part of the height data (for TableHeight attributes only).
offset	double	0	The offset along the z-axis where to place the height plot.
prefixintitle	String	Empty	Added prefix to contribution to title.
preprocz	none linear	none	Preprocess the height data with a scaling factor and a shift, if set to linear (for TableHeight attributes only).
scale	positive double	1	If scaleactive is true: The scale factor for the height.
scaleactive	boolean	false	Whether to use manual scaling.
scalingz	double	1	The scaling factor for the preprocessing, if preprocz is set to linear (for TableHeight attributes only).
shiftz	double	0	The shift for the preprocessing, if preprocz is set to linear (for TableHeight attributes only).
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed. custom is available for Height attributes only.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-61: VALID PROPERTIES FOR HEIGHT ATTRIBUTES.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

SEE ALSO

[Color](#), [Deform](#), [Histogram](#), [Table](#), [TableSurface](#), [Transparency](#)

Histogram

Create a histogram plot for 1D and 2D plot groups.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Histogram");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Histogram")` creates a histogram plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Histograms are used to visualize the distribution of the range of an expression. The result is a plot with the expression's range on the *x*-axis and element length, area, or volume on the *y*-axis. Such histogram plots can be added to 1D plot groups. In 2D histograms, which you can add to 2D plot groups, the *x*-axis and *y*-axis represent the values of two quantities (as a number of bins or a range of values), and the color surface represents the count of the total element volume in each "bin".

The following properties are available:

TABLE 7-62: VALID PROPERTIES FOR HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an <i>N</i> -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
cumulative	boolean	false	Plot a cumulative histogram.

TABLE 7-62: VALID PROPERTIES FOR HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
function	continuous discrete	continuous	Use a continuous function or a discrete function (that is, using a constant level in each bin) for the histogram.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
level	fromdataset volume surface line point	fromdataset	The geometry level to evaluate on.
limits	double array	0 1	The bin limits if method is limit.

TABLE 7-62: VALID PROPERTIES FOR HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
measure	auto integral count	auto	The measure for evaluation of the histogram data. Choose integral for volume-based data; choose count of element-based data. auto is count for Mesh datasets; integral otherwise.
method	limit number	number	The method used to specify the bins in which the expression's range is split
normalization	integralsum integralscaled none peak	none	The normalization of the values plot on the y-axis.
number	nonnegative integer	10	The number of bins if method is number.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.

TABLE 7-62: VALID PROPERTIES FOR HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if twoyaxes is set to true in the parent plot group for 1D histogram plots.
prefixintitle	String	Empty	Added prefix to contribution to title
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values. This and other range settings are available for 2D Histogram plots.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.

TABLE 7-62: VALID PROPERTIES FOR HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
sdim	fromdataset 0 1 2 3	fromdataset	The space dimension to evaluate the underlying dataset in
solnum	integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
type	curve solid	curve	Use a filled histogram (solid) or a histogram drawn using lines (curve). This property is only available when function is discrete.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-62: VALID PROPERTIES FOR HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

[Filter](#) (Plot Attribute); [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#) (2D only); [Transparency](#) (if a height attribute is used)

COMPATIBILITY

The normalization property `integral` in previous versions of COMSOL Multiphysics has been replaced by `integralsum` in version 5.5.

SEE ALSO

[TableHistogram](#), [MatrixHistogram](#)

Image (Export)

Export an image.

SYNTAX

```
model.result().export().create(<ftag>,"Image");
model.result().export().create(<ftag>,<pgtag>,"Image");
model.result().export(<ftag>).set(property,<value>);
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>,"Image")` creates an image feature with the name `<ftag>`.

`model.result().export().create(<ftag>,<pgtag>,"Image")` creates an image feature with the name `<ftag>` for the plot group `<pgtag>`.

`result().export(<ftag>).set("plotgroup",<ptag>)` changes the source of the image to the plot group named `<ptag>`.

Image features can be used both to export images and to have ready-made views of plot groups.

The following properties are available:

TABLE 7-63: VALID PROPERTY/VALUE PAIRS FOR IMAGE EXPORT.

NAME	VALUE	DEFAULT	DESCRIPTION
alwaysask	on off	off	Always ask for filename when saving if target is set to file. This property is ignored when running without a UI.
antialias	on off	on	Enable/disable antialiasing.
axes1d	on off	on	If <code>options1d</code> is on; enable/disable display of the coordinate axes in 1D.
axes2d	on off	on	If <code>options2d</code> is on; enable/disable display of the coordinate axes in 2D.

TABLE 7-63: VALID PROPERTY/VALUE PAIRS FOR IMAGE EXPORT.

NAME	VALUE	DEFAULT	DESCRIPTION
axisorientation	on off	on	If options is on; enable/disable display of the axis orientation indicator. Used for 3D plots.
background	current color transparent	color	The background color.
bmpfilename	String		The name of the output file if imagetype is bmp.
customcolor	double array	{1, 1, 1}	If background is color; the red, green, and blue components of the background color.
epsfilename	String		The name of the output file if imagetype is eps.
fontsize	integer	9	The font size.
gltfincludelines	on off	on	Include line segments if imagetype is gltf.
gltffilename	String		The name of the output file if imagetype is gltf.
grid	on off	on	If options is on; enable/disable display of the coordinate grid. Used for 3D plots.
height	double	600	The height of the image.
highprecisioncolor	on off	off	Export a 16-bit per channel PNG file when imagetype is png.
imagetype	png eps jpeg bmp gltf	png	The type of image to export if target is set to file. eps can only be used for 1D plots. gltf can only be used for 3D plots.
jpegfilename	String		The name of the output file if imagetype is jpeg.
legend1d	on off	on	If options1d is on: enable or disable display of the legend in 1D.
legend2d	on off	on	If options2d is on: enable or disable display of the legend in 2D.
legend3d	on off	on	If options3d is on: enable or disable display of the legend in 3D.
lockratio	true false	false	If true, then the aspect ratio of the image is preserved when the width or the height is changed.
lockview	true false	false	Lock the view in PowerPoint or Word if target is set to linked. Saved camera settings are then used when you update the image from PowerPoint or Word.
logo1d	on off	on	If options1d is on: enable or disable display of the logo in 1D.
logo2d	on off	on	If options2d is on: enable or disable display of the logo in 2D.
logo3d	on off	on	If options3d is on: enable or disable display of the logo in 3D.
options1d	on off	off	Enable/disable optional components of the image in 1D.

TABLE 7-63: VALID PROPERTY/VALUE PAIRS FOR IMAGE EXPORT.

NAME	VALUE	DEFAULT	DESCRIPTION
options2d	on off	off	Enable/disable optional components of the image in 2D.
options3d	on off	off	Enable/disable optional components of the image in 3D.
pngfilename	String		The name of the output file if imagetype is png.
qualityactive	on off	off	If a quality level (quantitylevel) should be active when imagetype is jpeg.
quantitylevel	integer	92	The quality level when imagetype is jpeg.
resolution	integer	96	The frame resolution in dots per inch.
plotgroup	String		The plot group to export.
size	manualprint manualweb current presentation any user-defined preset sizes.	current	The preset size. The presentation preset option is intended for export to PowerPoint and Word.
sourceobject	The tag of the node for the plot.		The tag of a node (such as a plotgroup like "pg1 ") with the plot for the image export.
sourcetype	plotgroup geometry other	plotgroup	The type of source for the image: A plot group, the geometry, or some other type (such as a selection or mesh),
target	file clipboard linked	file	The target for the image: a file, the Windows clipboard, or linked to PowerPoint and Word (the latter two options are only available on Windows),
title1d	on off	on	If options1d is on: enable or disable display of the title in 1D.
title2d	on off	on	If options2d is on: enable or disable display of the title in 2D.
title3d	on off	on	If options3d is on: enable or disable display of the title in 3D.
unit	px mm in	mm	The unit for the dimensions of the image.
view	auto new any other existing view	auto	The view for the image: with auto it is typically taken from the plot group or from the geometry.
width	double	800	The width of the image.
zoomextents	on off	off	Perform a zoom to extents for the image.



The default values listed above are valid before any image has been exported successfully. After that, the settings from the last successful image export are used as default values the next time an image export feature is created.

Changing plot group after creation does not reset plot group-dependent default settings (title, colorlegend).

SEE ALSO

[Animation](#)

Image (Plot)

Embed an image in a surface or volume plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Image");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).selection()...
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"Image")` creates an image feature with the name `<ftag>`.

You can add an image a to a 3D or 2D plot group to plot an image on the geometry's surface.

For example, `model.result("pg1").feature("img1").selection().set(12);` selects boundary 12 as the boundary for which to display the image called `img1` in the plot group `pg1`.

The following properties are available:

TABLE 7-64: VALID PROPERTIES FOR IMAGE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
ax2	double array	{0, 0}	The theta and phi angles (in radians), when <code>axistype</code> is set spherical.
ax3	double array	{0, 0, 1}	Axis direction, when <code>axistype</code> is set cartesian.
axistype	cartesian spherical x y z	z	The axis type: Cartesian, spherical, X-axis, Y-axis, or Z-axis, if mapping is set to cylindrical or spherical.
coordinterpretation	relative pixels	off	Interpret the u- and v-coordinates as relative values or pixel coordinates when mapping is set to manual.
data	none parent dataset name	parent	The dataset this feature refers to.
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is custom.
displacement	double	0	The displacement for the cylinder, when <code>heightmode</code> is set to manual.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expressionintitle	on off	off	Whether the title contribution should contain the expression when <code>titletype</code> is custom.
extrapolation	repeat mirroredrepeat clamp	repeat	How the image is extrapolated. With <code>clamp</code> , each coordinate is mapped to the closest valid value at an edge of the image.
filename	String		The filename for the image to add.
heightmode	auto fit manual	auto	How to determine the height: automatically, to fit to the object, or manually, when mapping is set to cylindrical.

TABLE 7-64: VALID PROPERTIES FOR IMAGE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
heightval	double	1	The height for the cylinder, when heightmode is set to manual.
importedheight	String		The height, in pixels, for example, of the imported image file.
importedname	String		Name of the imported image file.
importedwidth	String		The width, in pixels, for example, of the imported image file.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color and color range are inherited from.
mapping	auto manual planar cylindrical spherical rectangular	auto	The surface mapping of the image (automatic or manual). For 3D only, you can also choose a planar, cylindrical, or spherical mapping. For 2D only, you can also choose a rectangular mapping.
normal	double array	{0, 0, 1}	Normal vector, when planetype is set to general.
planetype	xy yz zx yx zy xz auto general	xy	The surface mapping of the image (automatic or manual). For 3D only, you can also choose a planar, cylindrical, or spherical mapping.
pos	double array	{0, 0, 0}	Axis direction, when mapping is set to cylindrical or spherical.
phirange	double	360	The angle range (in degrees), when mapping is set to cylindrical.
preserveaspect	true false	true	Preserve the aspect ratio when mapping is set to auto.
prefixintitle	String		Added prefix to contribution to title.
projsize	double	1	Projected size, when mapping is set to planar.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.

TABLE 7-64: VALID PROPERTIES FOR IMAGE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
rot	double	0	The rotation of the image mapping in radians, when mapping is set to planar, cylindrical, or spherical.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
uexp	String	0	Expression for the u-coordinate when mapping is set to manual.
vexp	String	0	Expression for the v-coordinate when mapping is set to manual.
width	double	1	The width of the image, when mapping is set to rectangular or planar.

ATTRIBUTES

[Deform](#), [MaterialAppearance](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[ImageOverlay](#), [MaterialAppearance](#), [Transparency](#)

ImageOverlay

Embed an image overlay in a surface or volume plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"ImageOverlay");  
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"ImageOverlay")` creates an image overlay attribute feature with the name `<ftag>`.

You can add an image overlay as an attribute to a 3D volume plot or a 2D or 3D surface plot, overlaying the image in the plot's surface.

The following properties are available:

TABLE 7-65: VALID PROPERTIES FOR IMAGEOVERLAY ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
ax2	double array	{0, 0}	The theta and phi angles (in radians), when <code>axistype</code> is set spherical.
ax3	double array	{0, 0, 1}	Axis direction, when <code>axistype</code> is set cartesian.
axistype	cartesian spherical x y z	z	The axis type: Cartesian, spherical, X-axis, Y-axis, or Z-axis, if mapping is set to cylindrical or spherical.
coordinterpretation	relative pixels	off	Interpret the u- and v-coordinates as relative values or pixel coordinates when mapping is set to manual.
extrapolation	repeat mirroredrepeat clamp	repeat	How the image is extrapolated. With <code>clamp</code> , each coordinate is mapped to the closest valid value at an edge of the image.
filename	String		The filename for the image to add.
importedheight	String		The height, in pixels, for example, of the imported image file.
importedname	String		Name of the imported image file.
importedwidth	String		The width, in pixels, for example, of the imported image file.
mapping	auto manual planar cylindrical spherical rectangular	auto	The surface mapping of the image (automatic or manual). For 3D only, you can also choose a planar, cylindrical, or spherical mapping. For 2D only, you can also choose a rectangular mapping.
normal	double array	{0, 0, 1}	Normal vector, when <code>planetype</code> is set to general.
planetype	xy yz zx yx zy xz auto general	xy	The surface mapping of the image (automatic or manual). For 3D only, you can also choose a planar, cylindrical, or spherical mapping.
pos	double array	{0, 0, 0}	Axis direction, when mapping is set to cylindrical or spherical.
preserveaspect	true false	true	Preserve the aspect ratio when mapping is set to auto.
projsize	double	1	Projected size, when mapping is set to planar.

TABLE 7-65: VALID PROPERTIES FOR IMAGEOVERLAY ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
rot	double	0	The rotation of the image mapping in radians, when mapping is set to planar, cylindrical, or spherical.
uexp	String	0	Expression for the u-coordinate when mapping is set to manual.
vexp	String	0	Expression for the v-coordinate when mapping is set to manual.
width	double	1	The width of the image, when mapping is set to rectangular or planar.

SEE ALSO

[Image \(Plot\)](#), [MaterialAppearance](#)

ImpulseResponse

Create an impulse response plot for 1D plot groups. This plot type requires the Acoustics Module.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"ImpulseResponse");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"ImpulseResponse")` creates an impulse response plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Impulse response plots are primarily used for results processing of acoustics simulations. Impulse response plots can be added to 1D plot groups.

The following properties are available:

TABLE 7-66: VALID PROPERTIES FOR IMPULSE RESPONSE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.

TABLE 7-66: VALID PROPERTIES FOR IMPULSE RESPONSE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
dbmanualref	double	0	Reference value for display in dB, if dbref is manual (and indb is true).
dbref	maximum manual	maximum	Use the maximum value of a manual value as the dB reference when indb is true.
dbtype	10log 20log	10log	Use a 10 log or 20 log for the dB values when indb is true.
density	String		The density of the frequency expression.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
fouriershow	real imaginary amplitude phase spectrum	amplitude	When transform is fourier: Show the DFT as real or imaginary part, amplitude, phase, or frequency spectrum.-
freqmax	integer		If fouriershow is spectrum and freqrangeactive is true: The upper frequency bound.
freqmin	integer		If fouriershow is spectrum and freqrangeactive is true: The lower frequency bound.
freqrangeactive	true false	false	If fouriershow is spectrum: Controls whether a manual frequency range is used.
frequency	String		The frequency expression.
freqtype	octave octave3 octave6	octave	The frequency interpretation: octave, 1/3 octave, or 1/6 octave.
ignorelast	true false	true	If fouriershow is not spectrum: ignore last sample.
indb	true false	false	Display the amplitude or frequency spectrum in dB when fouriershow is amplitude or spectrum.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.

TABLE 7-66: VALID PROPERTIES FOR IMPULSE RESPONSE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
maskdc	true false	false	If fouriershow is not spectrum: set the DC value (zero frequency component) to zero.
movingaverage	true false	false	If transform is fourier: Compute a moving average if set to true.
n	integer	1	If type is octave: The number for the denominator in a fractional octave band.
nfreqs	integer	1	If fouriershow is spectrum and nfreqsactive is true: The number of frequencies to plot.
nfreqsactive	true false	false	If fouriershow is spectrum: Controls whether the number of frequencies is set manually.

TABLE 7-66: VALID PROPERTIES FOR IMPULSE RESPONSE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
nsamples	integer	1	If <code>fouriershow</code> is not spectrum and <code>nsamplesactive</code> is true: The number of uniform samples to use.
nsamplesactive	true false	false	If <code>fouriershow</code> is not spectrum: Controls whether the number of uniform samples is set manually.
numberofcolors	integer	7	The number of colors in the color cycle when <code>cyclelength</code> is manual.
prefixintitle	String		Added prefix to contribution to title.
power	String		The power of the frequency expression.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when <code>linecolor</code> is <code>cyclecolortable</code> .
ripplefactor	double	0.05	The ripple factor.
scale	none multiplyperiod dividenfreq	none	If <code>fouriershow</code> is spectrum, use no scaling, multiply by sampling period, or divide by number of frequencies.
scaleperiod	true false	true	If <code>fouriershow</code> is not spectrum: scale with sampling period.
shiftzero	true false	true	If <code>fouriershow</code> is not spectrum: shift zero frequency.
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when <code>innerinput</code> is <code>manualindices</code> .
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
soundspeed	String		The speed of sound.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title.	The title to use when <code>titletype</code> is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
transform	none fourier	none	Transformation of x-axis date.

TABLE 7-66: VALID PROPERTIES FOR IMPULSE RESPONSE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
type	linear octave	linear	If movingaverage is true: Use a linear or 1/n octave moving average.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

[Color](#), [EnergyDecay](#)

SEE ALSO

[LineGraph](#)

InterferencePattern

Plot the interference pattern resulting from the intersection of rays with a cut plane.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"InterferencePattern");
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"InterferencePattern")` creates an interference pattern plot feature named `<ftag>` belonging to the plot group `<pgtag>`. The interference pattern plot is used to visualize the interference of multiple rays as they intersect a cut plane. Interference pattern plots can be added to 2D plot groups.

The following properties are available:

TABLE 7-67: VALID PROPERTIES FOR INTERFERENCE PATTERN PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.

TABLE 7-67: VALID PROPERTIES FOR INTERFERENCE PATTERN PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that color and color range are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
interp	double array	Time corresponding to last selected solnum for transient levels	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
originspec	manual index intensity	manual	Determines how the origin of the interference plot is defined.

TABLE 7-67: VALID PROPERTIES FOR INTERFERENCE PATTERN PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
originx	double	0	If originspec is manual, determines the x-coordinate of the origin.
originy	double	0	If originspec is manual, determines the y-coordinate of the origin.
originz	double	0	If originspec is manual, determines the z-coordinate of the origin.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
rangexactive	on off	off	Whether to use a manual range of x-coordinates for the grid points.
rangexmax	double	Plot dependent	The maximum x-coordinate in the local coordinate system of the interference pattern plot. Active when rangexactive is on.
rangexmin	double	Plot dependent	The minimum x-coordinate in the local coordinate system of the interference pattern plot. Active when rangexactive is on.
rangeyactive	on off	off	Whether to use a manual range of y-coordinates for the grid points.
rangeymax	double	Plot dependent	The maximum y-coordinate in the local coordinate system of the interference pattern plot. Active when rangeyactive is on.
rangeymin	double	Plot dependent	The minimum y-coordinate in the local coordinate system of the interference pattern plot. Active when rangeyactive is on.

TABLE 7-67: VALID PROPERTIES FOR INTERFERENCE PATTERN PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
rayindex	int	1	The index of the ray used to define the location of the origin of the interference pattern plot. The origin is located at the intersection of the specified ray with the cut plane. Active when originspec is index.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the title property). none, if no title contribution should be used.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
xyresolution	int	200	The number of grid points in each direction to use when rendering the interference pattern. Cannot be greater than 1000.
legendtitle	String		Color legend title, if showlegendstyle has been set to true in the parent plot group.

Interp

Evaluate expressions in arbitrary points or datasets using interpolation.

SYNTAX

```
model.result().numerical().create(<ftag>,"Interp");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getCoordinates();
model.result().numerical(<ftag>).getData();
model.result().numerical(<ftag>).getData(<expressionIndex>);
model.result().numerical(<ftag>).getImagData();
model.result().numerical(<ftag>).getImagData(<expressionIndex>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).getElements();
model.result().numerical(<ftag>).getNData();
model.result().numerical(<ftag>).run();
model.result().numerical(<ftag>).setInterpolationCoordinates(<value>);
```

DESCRIPTION

`model.result().numerical().create(<ftag>,"Interp")` creates an interpolation feature with the name `<ftag>`.

Interp is a feature made specifically for users of the COMSOL API and does not appear in the COMSOL Multiphysics UI. Interp combines cut points and evaluation features, as well as allowing evaluation of arbitrary datasets. It supports multiple expressions and some additional advanced properties not available when using cut points and [EvalPoint](#) features.

`result = model.result().numerical(<ftag>).getData()` returns the real part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][coordinates]`.

`model.result().numerical(<ftag>).getData(<expressionIndex>)` returns the real part of the result for one expression, equivalent to `result[expressionIndex]`.

`result = model.result().numerical(<ftag>).getImagData()` returns the imaginary part of the result, recomputing the feature if necessary. `result` is a three-dimensional double matrix ordered `result[expression][solnum][coordinates]`.

`model.result().numerical(<ftag>).getImagData(<expressionIndex>)` returns the imaginary part of the result for one expression, equivalent to `result[expressionIndex]`.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. `<outersolnum>` is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).getElements()` returns indices to columns in `p` of a simplex mesh.

`model.result().numerical(<ftag>).getNData()` returns the number of points in the data vector.

`model.result().numerical(<ftag>).getCoordinates()` returns the coordinates of the interpolation.

`model.result().numerical(<ftag>).setInterpolationCoordinates(<value>)` is identical to `model.result().numerical(<ftag>).set(coord,<value>)`, when `value` is of type `double[][]`.

The columns of the `coord` property are the coordinates for the evaluation points. If the number of rows in `coord` equals the space dimension, then `coord` are global coordinates, and the selection (or the property `edim`) determines the dimension in which the expressions are evaluated. For instance, dimension 2 means that the expressions are evaluated on boundaries in a 3D model. If the dimension to evaluate on is less than the space dimension, then the points in `coord` are projected onto the closest point on a domain of that dimension. It is further possible to select

a single geometric entity of a given dimension. If so, then the closest point on that domain in the given dimension is used.

If the number of rows in `coord` is less than the space dimension, then these coordinates are parameter values on a geometry face or edge. In that case, the domain number for that face or edge must be specified in the selection.

For datasets that do not support selections, the `edim` property must always be used, and no selections are possible.

The following properties are available:

TABLE 7-68: VALID PROPERTIES FOR INTERP.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>complexfun</code>	<code>on off</code>	<code>on</code>	Use complex-valued functions with real input
<code>const</code>	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
<code>coord</code>	double matrix	Empty	The coordinates to evaluate in. Datasets that define an interpolation in themselves, such as cut planes, revolve and so forth, can be evaluated directly in the coordinates that define their shapes, without specifying <code>coord</code> .
<code>coorderr</code>	<code>on off</code>	<code>on</code>	If on, evaluating for a set of points that all fall outside the geometry results in an error being reported.
<code>data</code>	<code>none dataset name</code>	First compatible dataset	The dataset this feature refers to.
<code>differential</code>	<code>on off</code>	<code>on</code>	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is <code>harmonic</code> .
<code>edim</code>	<code>auto 0 1 2 3</code>	<code>auto</code>	Element dimension. This is used for arbitrary datasets. Solution datasets use selections as usual to specify where to perform the evaluation, and ignore the value of the <code>edim</code> property.
<code>evalmethod</code>	<code>linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak</code>	<code>harmonic</code>	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
<code>expr</code>	String array	Model dependent	The expressions to evaluate.
<code>ext</code>	double between 0 and 1	0.1	Extrapolation distance: How much outside the mesh that the interpolation searches. The scale is in terms of the local element size.
<code>matherr</code>	<code>on off</code>	<code>off</code>	Error for undefined operation or variable
<code>outersolnum</code>	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
<code>phase</code>	double	0	Evaluate solution at this angle, given in degrees.
<code>refine</code>	<code>auto integer</code>	<code>auto</code>	Refinement of elements for evaluation points.
<code>recover</code>	<code>off pprint ppr</code>	<code>off</code>	The derivative recovery method (<code>off</code> , within domains, or everywhere).
<code>solnum</code>	nonnegative integer array	All solutions	The solutions to use.

TABLE 7-68: VALID PROPERTIES FOR INTERP.

NAME	VALUE	DEFAULT	DESCRIPTION
t	double array	Empty	The times to use. Available when the underlying solution is transient.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[Eval](#), [Global \(Numerical\)](#), [EvalPoint](#), [EvalGlobal](#)

IntersectionPoint2D, IntersectionPoint3D

Create a dataset that enables the evaluation of expressions at the intersection points of particle or ray trajectories and a curve or surface. These datasets can use a Ray or Particle dataset as their input.

SYNTAX

```
model.result().dataset().create(<dtag>,"InsterSectionPoint3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"IntersectionPoint2D")` creates an intersection point 2D dataset with the name `<dtag>`.

The following properties are available:

TABLE 7-69: VALID PROPERTY/VALUE PAIRS FOR INTERSECTION POINT 2D AND INTERSECTION POINT 3D DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	double array	{0,1}	Axis direction for a semicircle used as the curve type or a hemisphere used as the plane type.
bounded	on off	on	Control if a line used as the curve type should be bounded by the points or not.
center	double array	{0,0}	Center coordinates for a circle, semicircle, sphere, or hemisphere as the curve or plane type.
data	none dataset name	First compatible dataset	The dataset this feature refers to.
expr	String		Expression for a general curve or plane.
extrasteps	none specifiedtimes proportional	specifiedtimes	Controls how the number of extra time steps to find all intersections are added. These extra time steps typically correspond to the exact times of reflections or velocity reinitializations.
genparaactive	on off	off	Decides if parallel lines or planes should be drawn.
genparadist	double array	{}	Active when genparaactive is on, this property contains the distances from the extra parallel lines or planes to the base plane.
genmethod	threepoint pointnormal	threepoint	Active when planetype equals general, this property indicates whether the plane should be specified by three points or through one point and a normal.

TABLE 7-69: VALID PROPERTY/VALUE PAIRS FOR INTERSECTION POINT 2D AND INTERSECTION POINT 3D DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
genpoints	double	0 or 1, depending on index position ([0, 0, 0; 1 0 0; 0 1 0])	Active when method equals threepoint, this property contains the coordinates of the three points in the rows of the matrix. See below for the use of this property and the setIndex method.
genpnpoint	String array of length three	Zero string vector	Active when genmethod equals pointnormal, this property contains the coordinates of the point.
genpnvec	String array of length three	Zero string vector	Active when genmethod equals pointnormal, this property contains the normal vector.
interpolation	linear cubic	linear	The polynomial order of the method used to interpolate between time steps to evaluate expressions at the intersection points. If cubic is selected, the derivatives at the time steps are also taken into account.
normal	String array of length two or three	Created from the feature tag.	The names of the variables that can be used to evaluate the curve's or plane's normal.
numextrasteps	nonnegative integer	100	If extrasteps is specifiedtimes, controls the maximum number of extra time steps to use to find all intersections.
planetype	quick general	quick	Specify plane type for the surface type plane (3D).
propextrasteps	nonnegative integer	1	If extrasteps is proportional, controls the maximum number of extra time steps to use to find all intersections. The maximum number of extra steps is the product of this proportionality factor with the number of solution times.
radius	double	1	Radius for a circle, semicircle, sphere, or hemisphere as the curve or plane type.
quickplane	xy yz zx yx zy xz	yz	Specify quick plane type. Active when planetype is quick.
quickx	String	"0"	x-coordinate if planetype is yz or zy.
quicky	String	"0"	y-coordinate if planetype is zx or xz.
quickz	String	"0"	z-coordinate if planetype is xy or yx.
spacevars	String array of length 2	Created from the feature tag.	The names of the variables that evaluate to the plane's parameterization.
type	line circle semicircle general (2D); plane sphere hemisphere general (3D)	line (2D); plane (3D)	The type of curve (2D) or surface (3D).

For specifying the coordinates for a line curve type, for example, use the setIndex method:

```
model.result().dataset("ip2").setIndex("genpoints", "1.3", 0, 1);
```

That line sets the y-coordinate of point one to the value 1.3.

Add a volume, surface, or line integration to evaluate numerical results.

SYNTAX

```
model.result().numerical().create(<ftag>,"IntVolume");
model.result().numerical().create(<ftag>,"IntSurface");
model.result().numerical().create(<ftag>,"IntLine");
model.result().numerical(<ftag>).selection(...);
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"IntVolume")` creates a volume integral feature with the name <ftag>.

`model.result().numerical().create(<ftag>,"IntSurface")` creates a surface integral feature with the name <ftag>.

`model.result().numerical().create(<ftag>,"IntLine")` creates a line integral feature with the name <ftag>.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is false. If `columnwise` is true, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is true, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` true and `columnwise` false.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. <outersolnum> is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-70: VALID PROPERTIES FOR INTEGRATION.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	First compatible dataset	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
dataseries	average integral maximum minimum none rms stddev variance	none	The transformation that is applied to the data series formed by the evaluation
dataseriescumulative	true false	false	Make the table values as a cumulative integration, when dataseries is set to integral.
dataseriesmethod	auto integration summation	auto	The method to use for the data series: automatic, integration, or summation.
descr	String array	Model dependent	The descriptions of the expression in expr. Is used in the automatic title.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.
includeparam	true false	false	Include the parameter value or time if dataseries is set to maximum or minimum for a parametric or time-dependent solution,
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
intorder	positive integer	4	The integration order
intorderactive	on off	off	Whether to use manually specified integration order

TABLE 7-70: VALID PROPERTIES FOR INTEGRATION.

NAME	VALUE	DEFAULT	DESCRIPTION
intsurface	on off	off	Compute surface integral. Available for line integration features of axisymmetric models.
intvolume	on off	off	Compute volume integral. Available for surface integration features of axisymmetric models.
localzphys	array	0	The local z-coordinates, when locdef is set to physical.
localzrel	array of -1-1 numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition, if locinput is set to manual: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when using a Layered Material or Shell dataset. Not available for IntVolume.
locinput	fromdataset manual	fromdataset	The location input: from the dataset or manual. Only available when using a Layered Material or Shell dataset. Not available for IntVolume.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
maximumobj	abs real	real	The value being maximized if dataseries is set to maximum.
method	auto integration summation	auto	The integration method.
minimumobj	abs real	real	The value being minimized if dataseries is set to minimum.
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.

TABLE 7-70: VALID PROPERTIES FOR INTEGRATION.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
spliteval	on off	off	Evaluate each curve, surface, line, or plane separately. Not available for IntVolume.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
unit	String array	Model dependent	The units to use for the expressions in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[AvVolume](#), [AvSurface](#), [AvLine](#), [EvalGlobal](#), [EvalPoint](#), [Global \(Numerical\)](#)

Isosurface (Plot)

Create an isosurface plot in 3D.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Isosurface");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Isosurface")` creates an isosurface plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

The following properties are available:

TABLE 7-71: VALID PROPERTIES FOR ISOSURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the isosurfaces: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
contourlabels	on off	off	Whether to display labels next to the isosurfaces, if surfacetype is set to surface.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.

TABLE 7-71: VALID PROPERTIES FOR ISOSURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
filledlegend	on off	on	Whether to show the color legend as a filled legend.
includeinteriorboundaries	on off	off	Include interior boundaries, if surfacetype is set to filled.
includeoutside	on off	on	Include filled volumes outside of the isosurface levels, if surfacetype is set to filled.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color and deformation scale are inherited from.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interactive	on off	off	If true, the isosurfaces can be moved interactively after the plot has been made.

TABLE 7-71: VALID PROPERTIES FOR ISOSURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
labelcolor	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	If contourlabels is on: The label color to use. The default value uses a color from the color theme.
labelprec	positive integer	4	If contourlabels is on: The number of significant digits in the isosurface labels.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
legendtype	filled lines	lines	Whether to show the color legend as a filled legend or using lines.
levelmethod	number levels	number	How to enter isosurface levels.
levelrounding	boolean	true	Round the isosurface level values, if levelmethod is set to number.
levels	String array		The levels to plot. Active when levelmethod equals levels.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
number	positive integer	5	Total number of isosurface levels. Active when levelmethod equals number.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.

TABLE 7-71: VALID PROPERTIES FOR ISOSURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
shift	double	0	If interactive is on: The shift that is applied to the level values.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
surfacetype	surface filled	surface	Whether to show the isosurfaces as surfaces or as filled volumes.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.

TABLE 7-71: VALID PROPERTIES FOR ISOSURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
useder	boolean	true	If true, space derivatives of the isosurface expression are used to produce smoother plots.

ATTRIBUTES

[Color](#), [Deform](#), [Filter](#) (Plot Attribute), [Marker](#), [MaterialAppearance](#), [Selection](#) (Dataset Attribute), [Transformation](#), [Transparency](#), [VisualEffects](#)

SEE ALSO

[Isosurface](#) (Dataset), [IsosurfaceSeries](#), [Slice](#), [Surface](#) (Plot), [Volume](#)

Isosurface (Dataset)

Create an isosurface dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Isosurface");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Isosurface")` creates an isosurface dataset feature named `<dtag>`.

The following properties are available:

TABLE 7-72: VALID PROPERTIES FOR ISOSURFACE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
descr	String		Description of expr.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	u	Expression to find constant level for.
level	double array	{0}	Levels that defines the dataset if levelmethod is levels.
levelmethod	number levels	number	How to specify levels.
levelrounding	boolean	true	Round the isosurface level values, if levelmethod is set to number.
number	positive integer	5	Total number of contour levels. Active when levelmethod is number.
unit	String		Unit of expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
useder	boolean	true	Whether to use space derivatives to produce smoother plots.

SEE ALSO

[Isosurface \(Plot\)](#)

IsosurfaceSeries

Create an isosurface series plot in 3D.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"IsosurfaceSeries");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"IsosurfaceSeries")` creates an isosurface series plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

The following properties are available:

TABLE 7-73: VALID PROPERTIES FOR ISOSURFACESERIES PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
color	custom black blue cyan gray green magenta red white yellow	red	The color to use.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
contourlabels	on off	off	Whether to display labels next to the isosurfaces, if surfacetype is set to surface.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.

TABLE 7-73: VALID PROPERTIES FOR ISOSURFACESERIES PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color and deformation scale are inherited from.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
level	nonnegative integer	0	The number of levels.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range (1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.

TABLE 7-73: VALID PROPERTIES FOR ISOSURFACESERIES PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
surfacetype	surface filled	surface	Whether to show the isosurfaces as surfaces or as filled volumes.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.

TABLE 7-73: VALID PROPERTIES FOR ISOSURFACESERIES PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
useder	boolean	true	If true, space derivatives of the isosurface expression are used to produce smoother plots.

ATTRIBUTES

[Color](#), [Deform](#), [Filter \(Plot Attribute\)](#), [Marker](#), [MaterialAppearance](#), [Selection \(Dataset Attribute\)](#), [Transformation](#), [Transparency](#), [VisualEffects](#)

SEE ALSO

[Isosurface \(Plot\)](#), [Isosurface \(Dataset\)](#), [Slice](#)

Join

Join datasets for joining two datasets to form the difference, for example.

SYNTAX

```
model.result().dataset().create(<dtag>,"Join");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

model.result().dataset().create(<dtag>,"Join") creates a join dataset feature named <dtag>.

The following properties are available:

TABLE 7-74: VALID PROPERTY/VALUE PAIRS FOR JOIN DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	None	The first source dataset.
data2	none dataset name	None	The second source dataset.
expr	String	data1-data2	When method is general: The way the evaluations in the first and second datasets is combined. data1 and data2 can be used as symbols for the values from the two datasets, respectively.

TABLE 7-74: VALID PROPERTY/VALUE PAIRS FOR JOIN DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
method	difference differencenorm explicit general product quotient sum	difference	How to combine the results from evaluating in the two source datasets.
solutions	all one	all	Whether to expose one or all solutions from the first dataset.
solutions2	all one	all	Whether to expose one or all solutions from the second dataset.

ATTRIBUTE

Selection (*Dataset Attribute*) (if its two source datasets use the same geometry)

JointProbabilityDistribution

Plot a joint probability distribution.

	The JointProbabilityDistribution plot requires a license for the Uncertainty Quantification Module.
---	---

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"JointProbabilityDistribution");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result().dataset().create(<ftag>,"JointProbabilityDistribution");` creates a joint probability distribution plot in 2D plot groups as a results-processing step for models using uncertainty quantification.

The following properties are available:

TABLE 7-75: VALID PROPERTY/VALUE PAIRS FOR JOINT PROBABILITY DISTRIBUTION PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
abovediagonal	true false	true	Include parameter pairs above the diagonal, if mode is set to overview.
colortable	color table name	Rainbow	The color table to use when plotcontours is set to true. See Color Tables for a list of color tables.
colx	Table column number		The x-axis column for the parameter pair when mode is set to pair.
coly	Table column number		The y-axis column for the parameter pair when mode is set to pair.
curvecolor	custom black blue cyan gray green magenta red white yellow	blue	The estimation distribution curve color to use. Active when plotestimdistr is true.
customcurvecolor	1-by-3 double (RGB values)		The RGB values for a custom estimation distribution curve color, when curvecolor is set to custom.

TABLE 7-75: VALID PROPERTY/VALUE PAIRS FOR JOINT PROBABILITY DISTRIBUTION PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
customhistogramcolor	1-by-3 double (RGB values)		The RGB values for a custom histogram color, when histogramcolor is set to custom.
customjointcolor	1-by-3 double (RGB values)		The RGB values for a custom point color, when jointcolor is set to custom.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
histogramcolor	custom black blue cyan gray green magenta red white yellow	green	The histogram color to use. Active when plothistogram is true.
jointcolor	custom black blue cyan gray green magenta red white yellow	gray	The point color to use. Active when plotpoints is true.
mode	overview pair	overview	The plot mode: overview for an overview, or pair for an enlarged view of the scatter plot for the selected pair.
numberoflevels	positive integer	10	The number of contour levels, if plotcontours is set to true.
padding	double	0.02	Padding of parameter ranges.
plotcontours	true false	true	Plot contours.
plotestimdistr	true false	true	Plot estimated distribution, if mode is set to overview.
plothistogram	true false	true	Plot histogram, if mode is set to overview.
plotpoints	true false	true	Plot points.
table	none table name		The table this feature refers to.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
xpadding	double	0.02	x-axis padding, when mode is set to pair.
xspacing	double	0.05	Horizontal spacing between plots, when mode is set to pair.
ypadding	double	0.01	y-axis padding, when mode is set to pair.
yspacing	double	0.05	Vertical spacing between plots, when mode is set to pair.

ATTRIBUTES

None.

KernelDensityEstimation

Compute a kernel density estimation (KDE) from a table of values.

SYNTAX

```
model.result().dataset().create(<dtag>,"KernelDensityEstimation");
model.result().dataset(<dtag>).set(property,<value>);
model.result().dataset(<dtag>).setIndex(property,<value>,<index>);
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.result().dataset().create(<dtag>,"KernelDensityEstimation");` creates a kernel density evaluation dataset.

Kernel density estimation datasets take a table and compute the kernel density estimation (KDE), which is a nonparametric way to estimate the probability density function of a random variable.

The following properties are available:

TABLE 7-76: VALID PROPERTY/VALUE PAIRS FOR KERNELDENSITYESTIMATION DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
autores	boolean	true	Use an automatic or manual resolution for the variables.
bandwidth	auto manual	auto	Use automatic computation of the bandwidth using Silverman's rule of thumb or provide the bandwidth manually.
bandwidth1d	scalar double	1	The bandwidth for the KDE kernel, if bandwidth is set to manual and dimension is 1.
bandwidth2d	double 2-by-2 array	{1 0; 0 1}	The bandwidth for the KDE kernel, if bandwidth is set to manual and dimension is 2. Specify using <code>setIndex</code> syntax.
bandwidth3d	double 3-by-3 array	{1 0 0; 0 1 0; 0 0 1}	The bandwidth for the KDE kernel, if bandwidth is set to manual and dimension is 3. Specify using <code>setIndex</code> syntax.
dimension	1, 2 or 3	1	The dimension of the input data.
evaluationgroup	tag of evaluation group none		The evaluation group to use as input data, when source is set to <code>evaluationgroup</code> .
kerneltype	gaussian parabolic	gaussian	The type of kernel: Gaussian (normal) or parabolic (Epanechnikov).
outvar	String		Name of the output variable with the KDE value.
resx	positive integer	10	Resolution for the x variable, if <code>autores</code> is true.
resy	positive integer	10	Resolution for the y variable, if <code>autores</code> is true and dimension is 2 or 3.
resz	positive integer	10	Resolution for the z variable, if <code>autores</code> is true and dimension is 3.
source	table evaluationgroup	table	The data source: a table or an evaluation group.
table	tag of table none		The table to use as input data, when source is set to <code>table</code> .
xvar	String		Name of input variable for x.

TABLE 7-76: VALID PROPERTY/VALUE PAIRS FOR KERNELDENSITYESTIMATION DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
yvar	String		Name of input variable for y, if dimension is 2 or 3.
zvar	String		Name of input variable for z, if dimension is 3.

LayeredMaterial

Create a layered material dataset.



The LayeredMaterial dataset requires a license for the AC/DC Module, Composite Materials Module, or Heat Transfer Module.

SYNTAX

```
model.result().dataset().create(<dtag>,"LayeredMaterial");
model.result().dataset(<dtag>).set(property,<value>);
model.result().dataset(<dtag>).setIndex(property,<value>,<index>);
model.result().dataset(<dtag>).selection(...);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"LayeredMaterial")` creates a layered material dataset.

Layered material datasets refer to another dataset and are used to create a volume or domain (3D) dataset corresponding to a layered shell physics defined using a surface or boundary (2D) geometry and a built-in extra dimension (1D) geometry. This dataset is used to plot or evaluate the quantities on a domain level or its lower dimensions.

The following properties are available:

TABLE 7-77: VALID PROPERTY/VALUE PAIRS FOR LAYEREDMATERIAL DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	none	The dataset that this dataset refers to.
evaluatein	meshnodes interfaces layermidplanes	meshnodes	The location of layers to evaluate in: node points of the mesh, interfaces, or layer midplanes.
layeredmaterial	none layered material		The layered material that this layer or interface selection refers to.
resolution	Integer 1–10	1	Refinement level (1–10) when evaluatein is set to meshnodes.
scale	scalar	1	Scale factor for the layers.
seplevels	true false	false	Separate the levels to create slices on the node points in the through-thickness direction when evaluatein is set to meshnodes.
usealllayers	true false	false	Use all layers.

When `evaluatein` is set to `meshnodes` or `layermidplanes`, you can use the `setIndex` selection method to specify the layers to include with the property `layerselection`. Likewise, when `evaluatein` is set to `interfaces`, you can use the `setIndex` selection method to specify the layer interfaces to include with the property `interfaceselection`. See [setIndex\(\)](#) for more information about that method.

ATTRIBUTE

[Selection \(Dataset Attribute\)](#)

LayeredMaterialSlice

Create a layered material slice plot.



The LayeredMaterialSlice plot requires a license for the AC/DC Module, Composite Materials Module, Heat Transfer Module, or Structural Mechanics Module.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"LayeredMaterialSlice");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"LayeredMaterialSlice")` creates a layered material slice plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

A layered material slice plot displays a layered material quantity on a slice created at a specified through-thickness location in 2D or 3D.

The following properties are available:

TABLE 7-78: VALID PROPERTIES FOR LAYERED MATERIAL SLICE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.

TABLE 7-78: VALID PROPERTIES FOR LAYERED MATERIAL SLICE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. It is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
descriptionseparation	positive scalar	0.2	Relative separation for the descriptions, when showdescriptions is true.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, height scale, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.

TABLE 7-78: VALID PROPERTIES FOR LAYERED MATERIAL SLICE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
localzphys	array	0	The local z-coordinates, when locdef is set to physical.
localzrel	array of -1-1 numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative interfaces layermidplanes	reference	The location definition: a reference surface, physical z-coordinate, a relative z-coordinate, layer interfaces, and layer midplanes.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
orientationlinear	x y z diagonal	x (in 2D); z (in 3D)	The orientation for the linear displacement, when slicedisplacement is set to linear. In 2D, only x and y are available.
orientationrectangular	xy yz zx yx zy xz	xy	The orientation for the rectangular displacement, when slicedisplacement is set to rectangular. In 2D, only xy and yx are available.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.

TABLE 7-78: VALID PROPERTIES FOR LAYERED MATERIAL SLICE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
showdescriptions	true false	false	Show descriptions (annotations) for the slices.
slicedisplacement	none linear rectangular	none	Displacement of slices: no displacement, linear displacement, or rectangular displacements.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.

TABLE 7-78: VALID PROPERTIES FOR LAYERED MATERIAL SLICE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
wireframe	on off	off	Whether to plot filled elements or only their edges.
xseparation	positive scalar	0.15	Relative x-separation, when applicable for a linear or rectangular displacement.
yseparation	positive scalar	0.15	Relative y-separation, when applicable for a linear or rectangular displacement.
zseparation	positive scalar	0.15	Relative z-separation, when applicable for a linear or rectangular displacement.

ATTRIBUTES

Deform, Filter (Plot Attribute), Marker, MaterialAppearance, Selection (Dataset Attribute), Transformation, Transparency (3D only)

SEE ALSO

Surface (Plot)

Line

Create a line plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Line");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Line")` creates a line plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Line plots display a quantity on lines, curves and edges in 2D or 3D.

The following properties are available:

TABLE 7-79: VALID PROPERTIES FOR LINE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.

TABLE 7-79: VALID PROPERTIES FOR LINE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, tube scale, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
inheritubescale	boolean	true	If inheritplot is not none and linetype is tube: Determines if the tube scale is inherited.
interp	double array	Time corresponding to last selected solnum for transient levels	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
linetype	line tube	line	Plot lines or tubes.

TABLE 7-79: VALID PROPERTIES FOR LINE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	1	The tube radius. Active when linetype is tubes.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.

TABLE 7-79: VALID PROPERTIES FOR LINE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0–1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.

TABLE 7-79: VALID PROPERTIES FOR LINE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
tubeendcaps	true false	true	Use rounded end caps for tubes, when linetype is set to tube.
tuberadiussscale	double	1	The scale factor applied to the tube radii if tuberadiussscaleactive is true.
tuberadiussscaleactive	boolean	false	If true, tuberadiussscale is used, otherwise the scale factor is computed automatically.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
wireframe	on off	off	Whether to plot filled elements or only their edges.

ATTRIBUTES

Deform; Filter (Plot Attribute), Height, AberrationHeight, HistogramHeight, TableHeight (2D only); Marker, MaterialAppearance; Selection (Dataset Attribute); Transformation; Transparency (3D only)

EXAMPLES

Line plot on 2D solution:

Code for Use with Java

```
DatasetFeature ds = model.result().dataset().create("dset1", "Solution");
ds.set("solution", "sol1");

ResultFeature pg = model.result().create("pg1", 2);
pg.set("data", "dset1");
pg.create("line1", "Line");
pg.feature("line1").set("expr", "3*u");
```

Code for Use with MATLAB

```
ds = model.result.dataset.create('dset1', 'Solution');
ds.set('solution', 'sol1');

pg = model.result.create('pg1', 2);
pg.set('data', 'dset1');
pg.create('line1', 'Line');
pg.feature('line1').set('expr', '3*u');
```

Line plot on cut plane in 3D, using the Thermal color table:

Code for Use with Java

```
DatasetFeature ds2 = model.result().dataset().create("cp1", "CutPlane");
ds2.set("data", "dset2");

ResultFeature pg2 = model.result().create("pg2", 3);
pg2.create("line2", "Line");
pg2.feature("line2").set("data", "cp1");
pg2.feature("line2").set("expr", "2*u");
```

```
pg2.feature("line2").set("colortable", "Thermal");

pg2.run();
```

Code for Use with MATLAB

```
ds2 = model.result.dataset.create('cp1', 'CutPlane');
ds2.set('data', 'dset2');

pg2 = model.result.create('pg2',3);
pg2.create('line2','Line');
pg2.feature('line2').set('data', 'cp1');
pg2.feature('line2').set('expr', '2*u');
pg2.feature('line2').set('colortable', 'Thermal');

pg2.run;
```

SEE ALSO

[LineGraph](#)

LineData

Create a line data plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"LineData");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"LineData")` creates a line data plot feature named `<ftag>` belonging to the 2D or 3D plot group `<pgtag>`.

Line data plots are used to visualize raw point data given as points, elements, and colors as line segments (see the examples below). Line data plots can be added to 2D and 3D plot groups.

The following properties are available:

TABLE 7-80: VALID PROPERTY/VALUE PAIRS FOR LINE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
colordata	double 1D array		The color data for the line data plot as a real N-vector.
coloring	colortable uniform gradient	colortable	How to color the lines: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.

TABLE 7-80: VALID PROPERTY/VALUE PAIRS FOR LINE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
elementdata	integer 2D array		The element data for the line data plot, providing connections between the points. The data is an (edim+1)-by-M integer matrix with values that are 0-based indices into pointdata. The right-hand rule defines the normal direction in 3D. In 2D the order does not matter.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
pointdata	double 2D array		The point data for the line data plot, as x- and y-coordinates in 2D and x-, y-, and z-coordinates in 3D in an sdim-by-N real matrix.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.

TABLE 7-80: VALID PROPERTY/VALUE PAIRS FOR LINE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.

ATTRIBUTES

Marker, MaterialAppearance, Transparency (3D only)

EXAMPLES

A method for creating a line data plot in 2D for data representing a sine curve ($x, \sin(x)$):

Code for Use with Java

```
String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 2);
ResultFeature plot = pg.create("line1", "LineData");
int N = 100;
double[][] p = new double[2][N];
int[][] t = new int[2][N - 1];
double[] color = new double[N];
for (int i = 0; i < N; i++) {
    double x = 4 * Math.PI * i / N;
    p[0][i] = x;
    p[1][i] = Math.sin(x);
    if (i > 0) {
        t[0][i - 1] = i - 1;
        t[1][i - 1] = i;
    }
}
plot.set("pointdata", p)
    .set("elementdata", t)
    .set("colordata", color);
plot.run();
```

Code for Use with MATLAB

This variant, for use with MATLAB[®], sets a fixed blue color for the line and plots it:

```
import com.comsol.model.util.*
```

```

model = ModelUtil.create('Model');
pg = model.result.create('pg', 3);
pl = pg.create('line', 'LineData');
x = 0:100;
y = [sqrt(x); sin(x); cos(x)];
t = [1:99; 2:100];
pl.set('pointdata', y);
pl.set('elementdata', t);
pl.set('color', 'blue')
mphplot(model, 'pg')

```

A method for creating a line data plot in 3D for data representing a curve $(x, x^{1.3}, x^{1.6})$:

Code for Use with Java

```

String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 3);
ResultFeature plot = pg.create("line1", "LineData");
int N = 100;
double[][] p = new double[3][N];
int[][] t = new int[2][N - 1];
double[] color = new double[N];
for (int i = 0; i < N; i++) {
    p[0][i] = i;
    p[1][i] = Math.pow(i, 1.3);
    p[2][i] = Math.pow(i, 1.6);
    if (i > 0) {
        t[0][i - 1] = i - 1;
        t[1][i - 1] = i;
    }
}
plot.set("pointdata", p)
    .set("elementdata", t)
    .set("colordata", color);
plot.run();

```

SEE ALSO

[AnnotationData](#), [ArrowData](#), [PointData](#), [SurfaceData](#), [TubeData](#)

LineGraph

Create a line graph plot.

SYNTAX

```

model.result(<pgtag>).create(<ftag>, "LineGraph");
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();

```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "LineGraph")` creates a line graph plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

A line graph plot is used to visualize quantities on lines, either cut lines or boundaries (2D) and edges (3D) in a geometry. Line graph plots can be added to 1D plot groups.

The following properties are available:

TABLE 7-81: VALID PROPERTIES FOR LINE GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgecolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.

TABLE 7-81: VALID PROPERTIES FOR LINE GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.

TABLE 7-81: VALID PROPERTIES FOR LINE GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
localzphys	array	0	The local z-coordinates, when locdef is set to physical.
localzrel	array of -1-1 numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when data is set to a Layered Material dataset.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
normalizearclength	on off	off	Normalize the arc length when xdata is arc or reversedarc.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.

TABLE 7-81: VALID PROPERTIES FOR LINE GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if twoyaxes is set to true in the parent plot group.
prefixintitle	String	Empty	Added prefix to contribution to title.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
threshold	manual none	none	Use a smoothing threshold.

TABLE 7-81: VALID PROPERTIES FOR LINE GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xdata	arc reversedarc spacevar expr	solution	x-axis data. expr uses the expression in xdataexpr. arc uses the curve's arc length, reversedarc uses the arc length measured from the curve's endpoint, and spacevar uses the space variable from the dataset specified with the data property.
xdataexpr	String	Model dependent	Expression for x-axis data.
xdatadescr	String	Model dependent	Description of expression in xdataexpr.
xdataunit	String	Model dependent	The unit to use for the expression in xdataexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES

[Color](#), [Comparison](#), [Filter \(Plot Attribute\)](#), [GraphMarker](#)

SEE ALSO

[Global \(Plot\)](#), [Line](#), [PointGraph](#)

LineSegments

Create a line segments plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"LineSegments");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"LineSegments")` creates a line plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Line segments plots display line segments in 1D plots.

The following properties are available:

TABLE 7-82: VALID PROPERTIES FOR LINESEGMENTS PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendmethod	automatic manual	automatic	Whether to use the automatic legends or the legends supplied in the legends property.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.

TABLE 7-82: VALID PROPERTIES FOR LINESEGMENTS PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line segments feature in a ID plot. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.

TABLE 7-82: VALID PROPERTIES FOR LINESEGMENTS PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
xdescr	String array		Descriptions for the x-coordinates. Specify them using the setIndex method.
xexpr	Expression array		Expressions for the x-coordinates. Specify them using the setIndex method.
xunit	String array		Units for the x-coordinates. Specify them using the setIndex method.
ydescr	String array		Descriptions for the y-coordinates. Specify them using the setIndex method.
yexpr	Expression array		Expressions for the y-coordinates. Specify them using the setIndex method.
yunit	String array		Units for the y-coordinates. Specify them using the setIndex method.

ATTRIBUTES

None.

SEE ALSO

[Line](#)

Marker

A plot attribute for creating maximum and minimum markers for 3D volume, 2D and 3D surface and surface data, 2D and 3D line and line data, contour and contour series, isosurface and isosurface series, layered material slice, multislice, point data, slice, surface slit, and table contour, table point, table surface, and tube data plots.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Marker");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

model.result(<pgtag>).feature(<ftag>).create(<atag>,"Marker") creates a maximum and minimum marker plot named <atag> as an attribute to a plot named <ftag>.

This plot attribute displays the maximum and minimum of the expression in the parent plot and the points there they are attained.

The following properties are available:

TABLE 7-83: VALID PROPERTIES FOR A MARKER ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
alwaysign	true false	false	Always show the sign when numberformat is stopwatch or stopwatchsci.
alwaysignexp	true false	false	Always show the sign of the exponent when numberformat is stopwatchsci.
anchorpoint	upperright uppermiddle upperleft middleright center middleleft lowerright lowermiddle lowerleft	upperleft	The position of the marker text relative to the anchor point at the position given by posexpr.
backgroundcolor	none fromtheme custom black blue cyan gray green magenta red white yellow	none	The color to use for a background rectangle, or none for no background.
color	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	The color with which markers are plotted. The default takes the color from the current color theme.
customcolor	RGB-triplet	{1,0,0} or last used color	The color to use when color is set to custom.
decimals	positive integer	2	Number of decimals when numberformat is stopwatch or stopwatchsci.
display	minmax min max evaluation	minmax	The selection determines which markers are shown. Choose evaluation to add points where the markers should appear.
exponentdigits	integer (1–3)	1	Number of exponent digits when numberformat is stopwatchsci.
includeunit	true false	false	Whether to include the unit for the maximum and minimum values.
integerdigits	positive integer	3	Number of integer digits when numberformat is stopwatch.
labelprefix	String		Add an optional prefix to the maximum and minimum labels.
labelsuffix	String		Add an optional suffix to the maximum and minimum labels.
localtablematrix	2D or 3D matrix of doubles		The coordinate values for points where markers should appear, if display is set to evaluation. Specify the values using the setIndex method.

TABLE 7-83: VALID PROPERTIES FOR A MARKER ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
numberformat	default auto engineering scientific stopwatch stopwatchsci	auto	The number format for the marker: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.
orientation	horizontal vertical	horizontal	Orient the marker text horizontally or vertically.
pointradius	double	2	A value between 0 and 10 for the point radius, if showpoint is true.
posalwaysign	true false	false	If prependcoords is set to true: Always show the sign when posnumberformat is stopwatch or stopwatchsci.
posalwaysignexp	true false	false	If prependcoords is set to true: Always show the sign of the exponent when posnumberformat is stopwatchsci.
posdecimals	positive integer	2	If prependcoords is set to true: Number of decimals when posnumberformat is stopwatch or stopwatchsci.
posintegerdigits	positive integer	3	If prependcoords is set to true: Number of integer digits when posnumberformat is stopwatch.
posnumberformat	default auto engineering scientific stopwatch stopwatchsci	auto	If prependcoords is set to true: The number format for the marker position: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.
posprecision	positive integer	6	If prependcoords is set to true: The precision for position values in the marker when posnumberformat is auto, engineering, or scientific.
postrailingzeros	true false	false	If prependcoords is set to true: Include trailing zeros when posnumberformat is auto, engineering, or scientific.
precision	positive integer	6	The precision for numerical values in the marker when numberformat is auto, engineering, or scientific.
prependcoords	true false	false	Prepend the position of the marker.
showframe	true false	false	Show a rectangular frame around the maximum and minimum values.
showpoint	true false	true	Whether to show the point for the position of the maximum and minimum values or not.
trailingzeros	true false	false	Include trailing zeros when numberformat is auto, engineering, or scientific.

SEE ALSO

[GraphMarker](#), [MaxMinVolume](#), [MaxMinSurface](#), [MaxMinLine](#), [MaxMinPoint](#)

MassProperties

Add a mass properties evaluation to evaluate volume, mass, and center of mass for a model geometry.

SYNTAX

```
model.result().numerical().create(<ftag>, "MassProperties");
model.result().numerical(<ftag>).selection(...);
model.result().numerical(<ftag>).set(property, <value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise, <outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate, columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate, columnwise, <outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>, "MassProperties")` creates a mass properties feature with the name <ftag>.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is false. If `columnwise` is true, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate, columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is true, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` true and `columnwise` false.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. <outersolnum> is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluate the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-84: VALID PROPERTIES FOR MEASUREMENTS.

NAME	VALUE	DEFAULT	DESCRIPTION
contributionenabled	boolean	true	Which mass properties to include, set with setIndex for the mass property that you want to include or exclude.
data	none parent dataset name	First compatible dataset	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
dataseries	average integral maximum minimum none rms stddev variance	none	The transformation that is applied to the data series formed by the evaluation.
dataseriesmethod	auto integration summation	auto	The method to use for the data series: automatic, integration, or summation.
descr	String	Volume, Area, Length, or Distance	A description for the table column header.
includeparam	true false	false	Include the parameter value or time if dataseries is set to maximum or minimum for a parametric or time-dependent solution,
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
intorder	positive integer	4	The integration order
intorderactive	on off	off	Whether to use manually specified integration order
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
method	auto integration summation	auto	The integration method, when applicable.
minimumobj	abs real	real	The value being minimized if dataseries is set to minimum.

TABLE 7-84: VALID PROPERTIES FOR MEASUREMENTS.

NAME	VALUE	DEFAULT	DESCRIPTION
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.

TABLE 7-84: VALID PROPERTIES FOR MEASUREMENTS.

NAME	VALUE	DEFAULT	DESCRIPTION
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
unit	String array of length 3	Model dependent	The unit to use for the measurement, set with setIndex for the mass property that you want to set a unit for.

SEE ALSO

[AvVolume](#), [AvSurface](#), [AvLine](#), [EvalGlobal](#), [EvalPoint](#), [Global \(Numerical\)](#), [IntVolume](#), [IntSurface](#), [IntLine](#), [MeasureVolume](#), [MeasureSurface](#), [MeasureLine](#), [MeasureDistance](#)

MaterialAppearance

A plot attribute for adding material appearance to 3D volume plots; 2D and 3D surface plots; isosurface plots; and radiation pattern plots; streamline, streamline multislice, and streamline surface plots; and surface data plots.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"MaterialAppearance");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"MaterialAppearance")` adds a material appearance plot named `<atag>` as an attribute to a volume, surface or surface data, isosurface, or radiation pattern plot named `<ftag>`.

This plot attribute replaces the standard results plot with a material appearance.

The following properties are available:

TABLE 7-85: VALID PROPERTIES FOR A MATERIALAPPEARANCE ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
appearance	material custom	material	Take the appearance from a material or define a custom appearance.
color	color	white	A color, if family is set to plastic, defaultplot, or other material family that supports a color.
family	material type	plastic	The family of materials, if appearance is set to custom.
material	tag of a material or none		The tag of the material with the appearance to use, when appearance is set to material.
usematerialsel	true false	true	Use the selection for the material specified for the material property. Not available when material is set to none.
useplotcolors	true false	false	Use the color from the parent plot instead of the material's color. Also, set this property to true to overlay an image.

If appearance is set to custom, the following additional properties are available:

TABLE 7-86: PROPERTIES FOR CUSTOM APPEARANCE OF MATERIALS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alpha	double	1	Diffuse and ambient color opacity.
ambient	string[] (a color or custom)	custom	Ambient light color.
customambient	double[]		RGB values for a custom ambient color.
customdiffuse	double[]		RGB values for a custom diffuse color.
customspecular	double[]		RGB values for a custom specular color.
diffuse	string[] (a color or custom)	custom	Diffuse light color.
fresnel	double	0.5	Reflectance at normal incidence for Cook–Torrance.
lighting	phong cooktorrance simple	phong	Lighting model.
noise	on off	off	Add noise to the appearance.
noisefreq	double[]	1	Normal vector noise scale.
noisescale	double[]	0	Normal vector noise scale.
roughness	double	0.1	Surface roughness for Cook–Torrance lighting model.
shininess	double	64	Specular exponent for Blinn–Phong lighting model.
specular	string[] (a color or custom)	custom	Specular light color.

SEE ALSO

[ImageOverlay](#)

MatrixHistogram

Create a matrix histogram plot for 2D plot groups.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"MatrixHistogram");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"MatrixHistogram")` creates a histogram plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Use a matrix histogram if you have a precomputed matrix that you want to visualize as a 2D histogram. For example, in a fatigue analysis, you can use it for rainflow counting to visualize the distribution of stress amplitudes and mean stresses. The matrix then contains data points in the *xy*-plane, where the *x*- and *y*-values are stresses. Matrix histogram plots can be added to 2D plot groups.

The following properties are available:

TABLE 7-87: VALID PROPERTIES FOR MATRIX HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
axisunit	String	Empty	Unit for the histogram axes.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.

TABLE 7-87: VALID PROPERTIES FOR MATRIX HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
gridcolor	custom black blue cyan gray green magenta red white yellow	gray	The grid color, if showgrid is set to true.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
looplevel	integer row matrix	all on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.

TABLE 7-87: VALID PROPERTIES FOR MATRIX HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
prefixintitle	String	Empty	Added prefix to contribution to title.
showgrid	true false	false	Whether to include a grid in the matrix histogram plot.
solnum	integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when titletype is manual

TABLE 7-87: VALID PROPERTIES FOR MATRIX HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xdata	String	Empty	Data for the histogram's x-axis.
ydata	String	Empty	Data for the histogram's y-axis.

ATTRIBUTES

[Filter](#) (Plot Attribute); [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#); [Transparency](#) (if a height attribute is used)

SEE ALSO

[Histogram](#)

MaxMinVolume, MaxMinSurface, MaxMinLine, MaxMinPoint

Create max/min marker plots.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"MaxMinVolume");
model.result(<pgtag>).create(<ftag>,"MaxMinSurface");
model.result(<pgtag>).create(<ftag>,"MaxMinLine");
model.result(<pgtag>).create(<ftag>,"MaxMinPoint");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"MaxMinVolume")` creates a max/min marker plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

This plot type displays the maximum and minimum of an expression and the points there they are attained.

The following properties are available:

TABLE 7-88: VALID PROPERTIES FOR MAX/MIN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
alwaysysign	true false	false	Always show the sign when numberformat is stopwatch or stopwatchsci.
alwaysysignexp	true false	false	Always show the sign of the exponent when numberformat is stopwatchsci.
anchorpoint	upperright uppermiddle upperleft middleright center middleleft lowerright lowermiddle lowerleft	upperleft	The position of the marker text relative to the anchor point at the position given by posexpr.
backgroundcolor	none fromtheme custom black blue cyan gray green magenta red white yellow	none	The color to use for a background rectangle, or none for no background.
color	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	The color with which markers are plotted. The default takes the color from the current color theme.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
customcolor	RGB-triplet	{1,0,0} or last used color	The color to use when color is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
decimals	positive integer	2	Number of decimals when numberformat is stopwatch or stopwatchsci.
integerdigits	positive integer	3	Number of integer digits when numberformat is stopwatch.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
display	minmax min max	minmax	The selection determines which markers are shown.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.

TABLE 7-88: VALID PROPERTIES FOR MAX/MIN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
exponentdigits	Integer (1–3)	1	Number of exponent digits when numberformat is stopwatchsci.
expr	String		The expression to compute the maximum and minimum for.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
includeunit	true false	false	Whether to include the unit for the maximum and minimum values.
inheritbackgroundcolor	boolean	true	If inheritplot is not none: Determines if the background color is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that the color and deformation scale are inherited from.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
integerdigits	positive integer	3	Number of integer digits when numberformat is stopwatch.
intorder	nonnegative integer	4	The integration order to use when points is set to integration.
labelprefix	String		Add an optional prefix to the maximum and minimum labels.
labelsuffix	String		Add an optional suffix to the maximum and minimum labels.
numberformat	default auto engineering scientific stopwatch stopwatchsci	auto	The number format for the maximum and minimum values: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.
orientation	horizontal vertical	horizontal	Orient the marker text horizontally or vertically.
precision	integer > 0	6	The number of decimals displayed in the labels in the UI.
prefixintitle	String	Empty	Added prefix to contribution to title.
pointradius	double	2	A value between 0 and 10 for the point radius, if showpoint is true.

TABLE 7-88: VALID PROPERTIES FOR MAX/MIN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
points	lagrange node integration	lagrange	The evaluation points for the maximum or minimum: Lagrange points, node points, or integration points.
posalwaysign	true false	false	If prependcoords is set to true: Always show the sign when posnumberformat is stopwatch or stopwatchsci.
posalwaysignexp	true false	false	If prependcoords is set to true: Always show the sign of the exponent when posnumberformat is stopwatchsci.
posdecimals	positive integer	2	If prependcoords is set to true: Number of decimals when posnumberformat is stopwatch or stopwatchsci.
posexponentdigits	positive integer (1–3)	1	If prependcoords is set to true: Number of exponent digits (1–3) for numerical values in the title when posnumberformat is stopwatchsci.
posintegerdigits	positive integer	3	If prependcoords is set to true: Number of integer digits when posnumberformat is stopwatch.
posnumberformat	default auto engineering scientific stopwatch stopwatchsci	auto	If prependcoords is set to true: The number format for the marker position: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.
posprecision	positive integer	6	If prependcoords is set to true: The precision for position values in the marker when posnumberformat is auto, engineering, or scientific.
postrailingzeros	true false	false	If prependcoords is set to true: Include trailing zeros when posnumberformat is auto, engineering, or scientific.
precision	positive integer	6	The precision for numerical values in the marker when numberformat is auto, engineering, or scientific.
refine	nonnegative integer	2	When points is set to lagrange, or when there is no solution data: The Lagrange order (number of refinements of each mesh element) when computing the maximum and minimum.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere).
showframe	true false	false	Show a rectangular frame around the maximum and minimum values.
showpoint	true false	true	Whether to show the point for the position of the maximum and minimum values or not.

TABLE 7-88: VALID PROPERTIES FOR MAX/MIN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
trailingzeros	true false	false	Include trailing zeros when numberformat is auto, engineering, or scientific.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

[Deform](#), [Filter](#) (Plot Attribute), [Selection](#) (Dataset Attribute), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[GraphMarker](#), [Marker](#)

MaxVolume, *MaxSurface*, *MaxLine*, *MinVolume*, *MinSurface*, *MinLine*

Find extremal values (maximum or minimum).

SYNTAX

```
model.result().numerical().create(<ftag>,"MaxVolume");
model.result().numerical().create(<ftag>,"MinVolume");
model.result().numerical().create(<ftag>,"MaxSurface");
model.result().numerical().create(<ftag>,"MinSurface");
model.result().numerical().create(<ftag>,"MaxLine");
model.result().numerical().create(<ftag>,"MinLine");
model.result().numerical(<ftag>).selection(...);
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"MaxVolume")` creates a volume maximum feature with the name <ftag>, and similarly for "MinVolume".

`model.result().numerical().create(<ftag>,"MaxSurface")` creates a surface maximum feature with the name <ftag>, and similarly for "MinSurface".

`model.result().numerical().create(<ftag>,"MaxLine")` creates a line maximum feature with the name <ftag>, and similarly for "MinLine".

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is false. If `columnwise` is true, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is true, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` true and `columnwise` false.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. <outersolnum> is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and

`model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-89: VALID PROPERTIES FOR MAXIMUM AND MINIMUM.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	First compatible dataset	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
dataseries	average integral maximum minimum none rms stddev variance	none	The transformation that is applied to the data series formed by the evaluation.
dataseriescumulative	true false	false	Make the table values as a cumulative integration, when dataseries is set to integral.
dataseriesmethod	auto integration summation	auto	The method to use for the data series: automatic, integration, or summation.
descr	String array	Model dependent	The descriptions of the expressions in expr. Is used in the automatic title.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.
includeparam	true false	false	Include the parameter value or time if dataseries is set to maximum or minimum for a parametric or time-dependent solution,
includepos	true false	false	Include the coordinates for the positions of the maximum and minimum.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
intorder	nonnegative integer	4	The integration order to use when points is set to integration.
localzphys	array	0	The local z-coordinates, when locdef is set to physical.

TABLE 7-89: VALID PROPERTIES FOR MAXIMUM AND MINIMUM.

NAME	VALUE	DEFAULT	DESCRIPTION
localzrel	array of -1-1 numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition, if locinput is set to manual: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when using a Layered Material or Shell dataset. Not available for MaxVolume and MinVolume.
locinput	fromdataset manual	fromdataset	The location input: from the dataset or manual. Only available when using a Layered Material or Shell dataset. Not available for MaxVolume and MinVolume.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation.
obj	abs real	real	The function of each real or complex number that is maximized/minimized.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.

TABLE 7-89: VALID PROPERTIES FOR MAXIMUM AND MINIMUM.

NAME	VALUE	DEFAULT	DESCRIPTION
points	lagrange node integration	lagrange	The evaluation points for the maximum or minimum: Lagrange points, node points, or integration points (Gauss points).
prependcoords	true false	false	Prepend the position of the maximum and minimum values.
refine	nonnegative integer	5	The Lagrange order (number of refinements) to use when points is set to lagrange. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
spliteval	on off	off	Evaluate each curve, surface, line, or plane separately. Not available for MaxVolume and MinVolume.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
unit	String array	Model dependent	The units to use for the expressions in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[AvVolume](#), [AvSurface](#), [AvLine](#), [IntVolume](#), [IntSurface](#), [IntLine](#)

MeasureVolume, *MeasureSurface*, *MeasureLine*, *MeasureDistance*

Add a volume, surface, or line measurement to evaluate a volume, surface area, or line length for a model geometry.

SYNTAX

```
model.result().numerical().create(<ftag>,"MeasureVolume");
model.result().numerical().create(<ftag>,"MeasureSurface");
model.result().numerical().create(<ftag>,"MeasureLine");
model.result().numerical().create(<ftag>,"MeasureDistance");
model.result().numerical(<ftag>).selection(...);
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name *<etag>*, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"MeasureVolume")` creates a volume measurement feature with the name *<ftag>*.

`model.result().numerical().create(<ftag>,"MeasureSurface")` creates a surface measurement feature with the name *<ftag>*.

`model.result().numerical().create(<ftag>,"MeasureLine")` creates a line measurement feature with the name *<ftag>*.

`model.result().numerical().create(<ftag>,"MeasureDistance")` creates a distance measurement feature with the name *<ftag>*.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is false. If `columnwise` is true, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is true, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` true and `columnwise` false.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. `<outersolnum>` is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluate the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-90: VALID PROPERTIES FOR MEASUREMENTS.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	First compatible dataset	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
dataseries	average integral maximum minimum none rms stddev variance	none	The transformation that is applied to the data series formed by the evaluation.
dataseriesmethod	auto integration summation	auto	The method to use for the data series: automatic, integration, or summation.
descr	String	Volume, Area, Length, or Distance	A description for the table column header.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
frametype	mesh material spatial geometry fromdataset	spatial	The frame used for the measurement.
includeparam	true false	false	Include the parameter value or time if dataseries is set to maximum or minimum for a parametric or time-dependent solution,
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.

TABLE 7-90: VALID PROPERTIES FOR MEASUREMENTS.

NAME	VALUE	DEFAULT	DESCRIPTION
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
intorder	positive integer	4	The integration order.
intorderactive	on off	off	Whether to use manually specified integration order.
intsurface	on off	off	Compute surface integral. Available for line integration features of axisymmetric models.
intvolume	on off	off	Compute volume integral. Available for surface integration features of axisymmetric models.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
method	auto integration summation	auto	The integration method, when applicable.
minimumobj	abs real	real	The value being minimized if dataseries is set to minimum.
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.

TABLE 7-90: VALID PROPERTIES FOR MEASUREMENTS.

NAME	VALUE	DEFAULT	DESCRIPTION
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
solrepresentation	<code>solnum</code> <code>solutioninfo</code>	<code>solutioninfo</code>	Indicates which method of selecting solutions is active.
solutionparams	<code>manual</code> <code>parent</code>	<code>manual</code>	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
spliteval	<code>on</code> <code>off</code>	<code>off</code>	Evaluate each curve, surface, line, or plane separately. Available for <code>MeasureSurface</code> and <code>MeasureLine</code> .
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	<code>new</code> table name	<code>new</code>	The table to use when calling <code>setResult()</code> or <code>appendResult()</code> . <code>new</code> indicates that a new table is created.
tablecols	<code>inner</code> <code>outer</code> <code>data</code> <code>level1</code>	<code>data</code>	Whether to use inner or outer solutions as columns in the table when calling <code>setResult()</code> or <code>appendResult()</code> . Applicable only for parametric sweep models. The level values (<code>level1</code> , <code>level2</code> , and so on) are the levels in the parametric sweep.
timeinterp	<code>on</code> <code>off</code>	<code>off</code>	<code>on</code> if <code>t</code> is used to determine time steps; <code>off</code> if <code>solnum</code> is used.
unit	String array	Model dependent	The unit to use for the measurement.

SEE ALSO

[AvVolume](#), [AvSurface](#), [AvLine](#), [EvalGlobal](#), [EvalPoint](#), [Global \(Numerical\)](#), [IntVolume](#), [IntSurface](#), [IntLine](#), [MassProperties](#)

Mesh (Plot)

Create a mesh plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Mesh");
model.result(<pgtag>).feature(<ftag>).set(property <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Mesh")` creates a mesh plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

This plot type provides visualization of meshes.

The following properties are available:

TABLE 7-91: VALID PROPERTIES FOR MESH PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
color	color definition	black	Specify a color for 1D mesh plots.
colorlegend	on off	on	Whether to show color legend when elemcolor is set to quality or size.
colorrangeunitinterval	true false	true	Use a color range from 0 to 1, corresponding to the potentially minimum and maximum mesh quality values.
colortable	color table name	TrafficFlow	The element color table to use when elemcolor is set to quality or size. See Color Tables for a list of color tables. 2D and 3D only.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custompointcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when pointcolor is set to custom.
customwireframecolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when wireframecolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
elemcolor	quality size type color	quality	How to color the elements 2D and 3D only. The default, quality, uses a mesh quality measure. size colors the elements according to the local mesh element size. type colors the elements according to their mesh element type.
elemfilter	random quality qualityrev size expression logicalexpression	random	If filteractive is on: The expression to use for filtering when only a subset of the elements are shown. 2D and 3D only.
elemscale	double in [0,1]	1	The factor with which the elements are scaled before display. 2D and 3D only.
elemtype2	all tri quad	all	2D mesh element types to plot.
elemtype3	all tet pyr prism hex	all	3D mesh element types to plot: all, tetrahedron, pyramid, prism, or hex.
filteractive	on off	off	Whether to use element filtering.
filterexpr	String	x	The expression to use for filtering when elemfilter is set to expression. 2D and 3D only.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.

TABLE 7-91: VALID PROPERTIES FOR MESH PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
logfilterexpr	String	'1'	The logical expression to use for filtering when elemfilter is set to 'logicalexpression'. 2D and 3D only.
meshdomain	all point edge surface volume (if 3D)	all	Mesh domain level(s) to plot. 2D and 3D only.
nodepoints	none geom physics field name	none	When set to geom or the name of a physics field included in the solution dataset, such as comp1.u, show node points, including interior node points for higher-order elements when data is a solution dataset.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if twoyaxes is set to true in the parent plot group. For 1D graph plots only.
pointcolor	custom color	black	How to color the node points if nodepoints is geom or a physics field. 2D and 3D only.
qualexpr	String		A custom expression for a mesh quality measure (used when qualmeasure is set to custom).
qualmeasure	skewness maxangle volcircum vollength condition growth custom	skewness	The mesh quality measure used when plotting the mesh quality: equiangular skewness, maximum angle, volume versus circumradius, volume versus length, condition number, element growth rate, or a custom expression.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the mesh plot's resolution in mesh plots where the geometry order is greater than one. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
sphereradiusscale	positive double	1.5	The scale factor applied to the point radii if nodepoints is geom or a physics field.

TABLE 7-91: VALID PROPERTIES FOR MESH PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
title	String	The auto-title	The title to use when titletype is manual. 2D and 3D only.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title should be used (the title property). none, if no title should be displayed. 2D and 3D only.
wireframecolor	custom none color	black	How to color the wireframe mesh; none means that it is not displayed at all. 2D and 3D only.
1dfilter	size none	none	Plot the mesh size as the y-axis data. 1D only.

ATTRIBUTES

[Deform](#), [Filter \(Plot Attribute\)](#), [Selection \(Dataset Attribute\)](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[Mesh \(Dataset\)](#)

Mesh (Dataset)

Create a mesh dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Mesh");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Mesh")` creates a mesh dataset feature named `<dtag>`.

This dataset provides support for evaluation of spatial coordinates and mesh variables on a mesh.

The following properties are available:

TABLE 7-92: VALID PROPERTIES FOR MESH DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
mesh	String	Empty	The mesh that this dataset refers to.
sorder	linear quadratic cubic quartic quintic sextic septic linear_serp quadratic_serp cubic_serp quartic_serp	linear	The geometry shape function. The first seven are Lagrange shape functions, and the last four are serendipity shape functions.

ATTRIBUTE

[Selection \(Dataset Attribute\)](#)

SEE ALSO

[Mesh \(Plot\)](#)

Mesh (Export)

Export a mesh.

SYNTAX

```
model.result().export().create(<ftag>, "Mesh");  
model.result().export().create(<ftag>, <dtag>, "Mesh");  
model.result().export(<ftag>).set(property, <value>);  
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>, "Mesh")` creates a mesh export feature with the name `<ftag>`.

`model.result().export().create(<ftag>, <dtag>, "Mesh")` creates a mesh export feature with the name `<ftag>` for the dataset `<dtag>`.

The following properties are available:

TABLE 7-93: VALID PROPERTIES FOR MESH EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alwaysask	on off	off	Always ask for filename when saving. This property is ignored when running without a UI.
data	String		The name of the dataset to export.
filename	String		The output file.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
solnum	integer array		Solution number to take values from.
stlformat	binary text	binary	STL file format.
t	String		Time to evaluate in if <code>timeinterp</code> is on.
timeinterp	on off	off	Enable/disable explicit time to evaluate in.

SEE ALSO

[Data](#)

MeshImportParameters

Add a mesh import parameters attribute to Filter and Partition datasets.

SYNTAX

```
model.result().dataset().create(<dtag>, "MeshImportParameters");  
model.result().dataset(<dtag>).set(property, <value>);
```

DESCRIPTION

`model.result().dataset(<dtag>).feature().create(<atag>, "MeshImportParameters")` creates a mesh import parameters attribute to a Filter or Partition dataset feature named `<dtag>`.

In this feature, you pick a parameter from a list using the `setIndex` method, such as

```
model.result().dataset(<dtag>).feature(<atag>).setIndex("looplevel", 3, 0);
```

Mirror2D, Mirror3D

Create 2D and 3D mirror datasets.

SYNTAX

```
model.result().dataset().create(<dtag>,"Mirror2D");
model.result().dataset().create(<dtag>,"Mirror3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Mirror2D")` creates a 2D mirror dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"Mirror3D")` creates a 3D mirror dataset feature named `<dtag>`.

This dataset takes data from another dataset and adds a mirror copy with respect to an axis or plane of reflection (for 2D and 3D, respectively).

The following properties are available for Mirror2D and Mirror3D:

TABLE 7-94: VALID PROPERTIES FOR MIRROR DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to
genpoints	double matrix	{{0, 0, 0}, {1,0,0}}	Active when method equals twopoint, this property contains the coordinates of the two points in the two rows of the matrix.
method	twopoint pointdir	twopoint	Decides if the line should be specified by two points or through one point and a direction
pddir	String array	{"0", "1"}	Active when method equals pointdir, this property contains the direction.
pdpoint	String array	{"0", "0"}	Active when method equals pointdir, this property contains the coordinates of the point.
removesymelem	boolean	false	Remove elements on the symmetry plane.
sidevar	String		The name of the positive side variable, which is 1 on the side where the original data resides and 0 on the other side.
spacevars	String array	Depends on the feature's tag	The name of space variables, which evaluate to the coordinates after the transformation performed by the mirror dataset.
vectortrans	symmetric antisymmetric	symmetric	Use a symmetric or antisymmetric vector transformation for the mirror operation.

The following properties are available for Mirror 3D:

TABLE 7-95: VALID PROPERTIES FOR MIRROR3D.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
genmethod	threepoint pointnormal	threepoint	Active when planetype equals general, this property indicates whether the plane should be specified by three points or through one point and a normal.

TABLE 7-95: VALID PROPERTIES FOR MIRROR3D.

NAME	VALUE	DEFAULT	DESCRIPTION
genpoints	double matrix	{{0, 0, 0}, {1,0,0}, {0,1,0}}	Active when method equals threepoint, this property contains the coordinates of the three points in the rows of the matrix.
genpnpoint	String array of length three	Zero string vector	Active when genmethod equals pointnormal, this property contains the coordinates of the point.
genpnvec	String array of length three	Zero string vector	Active when genmethod equals pointnormal, this property contains the normal vector.
planetype	quick general	yz	Specify plane type.
quickplane	xy yz zx	yz	Specify quick plane type. Active when planetype is quick.
quickx	String	"0"	x-coordinate if planetype is yz.
quicky	String	"0"	y-coordinate if planetype is zx.
quickz	String	"0"	z-coordinate if planetype is xy.
sidevar	String		The name of the positive side variable, which is 1 on the side where the original data resides and 0 on the other side.
spacevars	String array	Depends on the feature's tag	The names of space variables, which evaluate to the coordinates after the transformation performed by the mirror dataset.

ATTRIBUTE

[Selection \(Dataset Attribute\)](#)

Multislice

Create a slice plot in multiple directions at once.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Multislice");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Multislice")` creates a slice feature in multiple directions named `<ftag>` belonging to the plot group `<pgtag>`.

The following properties are available:

TABLE 7-96: VALID PROPERTIES FOR MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.

TABLE 7-96: VALID PROPERTIES FOR MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
coloring	colortable uniform gradient	colortable	How to color the slices: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.

TABLE 7-96: VALID PROPERTIES FOR MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
expressionintitle	on off	off	Whether the title contribution should contain the expression when <code>titletype</code> is <code>custom</code>
inheritcolor	boolean	true	If <code>inheritplot</code> is not <code>none</code> : Determines if the color is inherited.
inheritdeformscale	boolean	true	If <code>inheritplot</code> is not <code>none</code> : Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, and deformation scale are inherited from.
inheritrange	boolean	true	If <code>inheritplot</code> is not <code>none</code> : Determines if the color and data ranges are inherited.
inherittransparency	boolean	true	If <code>inheritplot</code> is not <code>none</code> : Determines if the transparency settings are inherited.
interp	double array	Time corresponding to last selected <code>solnum</code> for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if <code>showlegendtitle</code> has been set to <code>true</code> in the parent plot group.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or <code>interp</code> or <code>last</code> , but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
multiplanexmethod	number coord	number	Indicates whether the x-coordinates of the x-planes should be specified by number of planes to distribute evenly across the data or by coordinates.
multiplaneymethod	number coord	number	Indicates whether the y-coordinates of the y-planes should be specified by number of planes to distribute evenly across the data or by coordinates.
multiplanezmethod	number coord	number	Indicates whether the z-coordinates of the z-planes should be specified by number of planes to distribute evenly across the data or by coordinates.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to <code>colortable</code> or <code>gradient</code> and <code>colortabletrans</code> is set to <code>nonlinear</code> or <code>nonlinearsymmetric</code> .

TABLE 7-96: VALID PROPERTIES FOR MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.

TABLE 7-96: VALID PROPERTIES FOR MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
suffixintitle	String	Empty	Added suffix to contribution to title.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-96: VALID PROPERTIES FOR MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xnumber	String	1	Number of planes in the x direction. Active when multiplanexmethod is set to number.
xcoord	String array	Empty	Absolute coordinates in the x direction, active when multiplanexmethod is set to coord.
ynumber	String	1	Number of planes in the y direction. Active when multiplaneymethod is set to number.
ycoord	String array	Empty	Absolute coordinates in the z direction, active when multiplaneymethod is set to coord.
znumber	String	1	Number of planes in the z direction. Active when multiplanezmethod is set to number.
zcoord	String array	Empty	Absolute coordinates in the z direction, active when multiplanezmethod is set to coord.

ATTRIBUTES

[Deform](#), [Filter \(Plot Attribute\)](#), [Marker](#), [MaterialAppearance](#), [Selection \(Dataset Attribute\)](#), [Transformation](#), [Transparency](#)

SEE ALSO

[Slice](#), [Volume](#)

Nyquist

Create a Nyquist plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Nyquist");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

model.result(<pgtag>).create(<ftag>,"Nyquist") creates a Nyquist plot feature named <ftag> belonging to the plot group <pgtag>.

Nyquist plots are used to visualize complex-valued variables. Nyquist plots can be added to 1D plot groups.

The following properties are available:

TABLE 7-97: VALID PROPERTIES FOR NYQUIST PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	on off	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	on off	Model dependent	Whether the automatic legends should include the expressions.
autounit	on off	off	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String array	Model dependent	The description of the expressions in expr. Is used in the automatic legends.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.

TABLE 7-97: VALID PROPERTIES FOR NYQUIST PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
expressionintitle	on off	off	Whether the title contribution should contain the expression when <code>titletype</code> is <code>custom</code> .
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when <code>legendmethod</code> is set to <code>evaluated</code> .
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when <code>legendmethod</code> is set to <code>evaluated</code> .
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends.	Manual legends active when <code>legendmethod</code> is set to <code>manual</code> .
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use <code>cyclereset</code> to restart the cycling of colors from the first color. Use <code>fromtheme</code> to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in <code>markers</code>). Markers are visible when <code>linemarker</code> is set.
markers	integer	8	The number of markers to show. Markers are visible when <code>linemarker</code> is set.
numberofcolors	integer	7	The number of colors in the color cycle when <code>cyclength</code> is <code>manual</code> .
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of <code>solnum</code> , the <code>solnum</code> property is not used.

TABLE 7-97: VALID PROPERTIES FOR NYQUIST PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, <code>range(1, 1, 20)</code> . Applicable when <code>outerinput</code> is <code>manualindices</code> .
prefixintitle	String	Empty	Added prefix to contribution to title
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when <code>linecolor</code> is <code>cyclecolortable</code> .
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1, 1, 20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when <code>titletype</code> is <code>manual</code> .
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the <code>title</code> property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
typeintitle	on off	on	Whether the title contribution should contain the type when <code>titletype</code> is <code>custom</code> .
unit	String array	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.
unitcircle	boolean	false	Whether to plot the unit circle along with the axis.
unitintitle	on off	on	Whether the title contribution should contain the unit when <code>titletype</code> is <code>custom</code> .

OctaveBand

Create an Octave Band Plot.



The Octave Band Plot requires a license for the Acoustics Module, MEMS Module, or Structural Mechanics Module.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"OctaveBand");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"OctaveBand")` creates an octave band plot feature named `<ftag>` belonging to the 1D plot group `<pgtag>`.

The octave band plots a weighted frequency response in bands (octave or 1/3 octave) or as a continuous frequency response. The frequency can be weighted.

The following properties are available:

TABLE 7-98: VALID PROPERTIES FOR OCTAVE BAND PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
amplituderef	double	1	Reference level for the amplitude expression type.
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
bandtype	octave octave3 octave6 octave12	octave	The band type: octave bands, 1/3 octave bands, 1/6 octave bands, or 1/12 octave bands, if quantity is <code>bandtype</code> or <code>bandaveragepsd</code> .
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
colorcycle	default long	default	The color cycle, when <code>linecolor</code> is set to <code>cycle</code> . With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when <code>linecolor</code> is set to <code>cyclecolortable</code> . See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.

TABLE 7-98: VALID PROPERTIES FOR OCTAVE BAND PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expr	String	Model dependent	The expression used to transform the input to plot.
exprdomain	frequency time	frequency	Use a frequency-domain expression or a time-domain expression.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
exprtype	pressure ampl power transfer	pressure	Type of expression: pressure, amplitude, power, or transfer function. Only pressure and amplitude are available if exprdomain is set to time.
inband	on off	on	Whether to use in-band data only.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legends	String array	The last computed automatic legends	Manual legends active when legendmethod is set to manual.
levelref	double	0	Reference level for the transfer function expression type.

TABLE 7-98: VALID PROPERTIES FOR OCTAVE BAND PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width, when type is set to line.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.

TABLE 7-98: VALID PROPERTIES FOR OCTAVE BAND PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
powerref	double	1	Reference level for the power expression type.
prefixintitle	String	Empty	Added prefix to contribution to title
pressureref	double	<i>acpr</i> .pref_SPL	Reference level for the pressure expression type.
quantity	continuous bandpower bandaveragepsd	bandaveragepsd	The quantity to plot: a continuous power spectral density (PSD), the band power, or the band average PSD
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
sdim	fromdataset 0 1 2 3	fromdataset	The space dimension to evaluate the underlying dataset in
solnum	integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
type	bar outline line	bar	Use a filled histogram (bar), a histogram drawn using lines (outline), or a graph connecting the centerpoints of the bands (line). This property is only available when function is discrete.

TABLE 7-98: VALID PROPERTIES FOR OCTAVE BAND PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
weightexpr	expression		Value or expression to use as weighting.
weighting	a c z expression	z	Weighting function for octave band plot: A-weighting, C-weighting, Z-weighting, or a user-defined expression.

ATTRIBUTE[GraphMarker](#)*OpticalAberration*

Plot various types of monochromatic aberration that arise when electromagnetic rays are focused by a system of lenses and mirrors.

	The Optical Aberration plot is available with the Ray Optics Module.
---	--

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"OpticalAberration");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
model.result(<pgtag>).feature(<ftag>).runCommand("createReferenceHemisphereDataset");
model.result(<pgtag>).feature(<ftag>).runCommand("recomputeReferenceHemisphereDataset");
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"OpticalAberration")` creates an optical aberration plot feature named `<ftag>` belonging to the plot group `<pgtag>`. The optical aberration plot is used to visualize various types of monochromatic aberration. Optical aberration plots can be added to 2D plot groups.

The following properties are available:

TABLE 7-99: VALID PROPERTIES FOR OPTICAL ABERRATION PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
additionallogical-expression	String	1	Additional logical expression that must be nonzero for rays to be included in the plot.
additionallogical-expressionactive	on off	off	If on, activates an additional logical expression for including rays in the Zernike coefficient calculation.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.

TABLE 7-99: VALID PROPERTIES FOR OPTICAL ABERRATION PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to, which must be an Intersection Point 3D dataset. This Intersection Point 3D dataset should describe the intersection points of rays with a Gaussian reference hemisphere centered at the focus.
filterrelease	positive integer	1	Index of the release feature of rays to include if filterreleaseactive is on.
filterreleaseactive	on off	off	If on, filter rays by release feature to determine which rays are included in the Zernike coefficient calculation.
filterwavelength	String	632.8[nm]	Wavelength of rays to include if filterwavelengthactive is on.
filterwavelengthactive	on off	off	If on, filter rays by wavelength to determine which rays are included in the Zernike coefficient calculation.
filterwavelengthtol	String	1[nm]	Numerical tolerance for filtering rays by wavelength if filterwavelengthactive is on.
hemisphereradius	String	50[mm]	Radius of the Gaussian reference hemisphere. This is copied to an Intersection Point 3D dataset when automatically computing the focus.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that color and color range are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.

TABLE 7-99: VALID PROPERTIES FOR OPTICAL ABERRATION PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
legendtitle	String		Color legend title, if showlegendtitle has been set to true in the parent plot group.
lunit	Any length unit	\u00b5m (micron)	Length unit for plotting combinations of Zernike polynomials.
maxorder	2, 3, 4, or 5	5	Maximum polynomial order for the Zernike coefficients calculation,
ngrid	int	1000	The number of grid points to plot on the unit circle when rendering the optical aberration plot. Use an integer value between 100 and 1,000,000.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
normal	directionbased positionbased userdefined	direction--based	Determines how the normal direction to the focal plane is calculated.
normalexpr	String array of length 3	{0,0,1}	Expression for the normal direction to the focal plane, active if normal is userdefined.
numberofreflections	nonnegative integer	0	Rays that have reflected the specified number of times will be included in the plot. Used if numberofreflectionsactive is on.
numberofreflections--active	on off	off	If on, activates the filtering of rays based on number of reflections.
posexpr	String array	("0", "0")	Position of the center of the unit circle in which the Zernike polynomials are plotted. By specifying nonzero components for some instances of the Optical Aberration plot it is possible to display multiple such plots side-by-side in the Graphics window.
terms	all higherorder selectindividual	all	Terms in Zernike polynomial to include: all, higher-order terms, or individually selected terms.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the title property). none, if no title contribution should be used.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
z00	boolean	false	Include the z(0,0), piston term (if terms is set to selectindividual).

TABLE 7-99: VALID PROPERTIES FOR OPTICAL ABERRATION PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
z1m1	boolean	false	Include the $z(1,-1)$, vertical tilt term (if terms is set to selectindividual).
z11	boolean	false	Include the $z(1,1)$, horizontal tilt term (if terms is set to selectindividual).
z2m2	boolean	false	Include the $z(2,-2)$, oblique astigmatism term (if terms is set to selectindividual).
z20	boolean	false	Include the $z(2,0)$, defocus term (if terms is set to selectindividual).
z22	boolean	false	Include the $z(2,2)$, astigmatism term (if terms is set to selectindividual).
z3m3	boolean	false	Include the $z(3,-3)$, oblique trefoil term (if terms is set to selectindividual).
z3m1	boolean	false	Include the $z(3,-1)$, vertical coma term (if terms is set to selectindividual).
z31	boolean	false	Include the $z(3,1)$, horizontal coma term (if terms is set to selectindividual).
z33	boolean	false	Include the $z(3,3)$, horizontal trefoil term (if terms is set to selectindividual).
z4m4	boolean	false	Include the $z(4,-4)$, oblique quatrefoil term (if terms is set to selectindividual).
z4m2	boolean	false	Include the $z(4,-2)$, oblique secondary astigmatism term (if terms is set to selectindividual).
z40	boolean	false	Include the $z(4,0)$, spherical aberration term (if terms is set to selectindividual).
z42	boolean	false	Include the $z(4,2)$, secondary astigmatism term (if terms is set to selectindividual).
z44	boolean	false	Include the $z(4,4)$, horizontal quatrefoil term (if terms is set to selectindividual).
z5m5	boolean	false	Include the $z(5,-5)$ term (if terms is set to selectindividual).
z5m3	boolean	false	Include the $z(5,-3)$ term (if terms is set to selectindividual).
z5m1	boolean	false	Include the $z(5,-1)$ term (if terms is set to selectindividual).
z51	boolean	false	Include the $z(5,1)$ term (if terms is set to selectindividual).
z53	boolean	false	Include the $z(5,3)$ term (if terms is set to selectindividual).
z55	boolean	false	Include the $z(5,5)$ term (if terms is set to selectindividual).

ATTRIBUTES

Height, AberrationHeight, HistogramHeight, TableHeight; Transparency (if a height attribute is used)

Parametric1D, Parametric2D

Extend (extrude) a 1D or 2D dataset by using a parameter as dimension.

SYNTAX

```
model.result().dataset().create(<dtag>,"Parametric1D");
model.result().dataset().create(<dtag>,"Parametric2D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Parametric2D")` creates a parametric extrusion dataset feature named `<dtag>`. This dataset extends a dataset by using a parameter, such as time, as a dimension.

The following properties are available:

TABLE 7-100: VALID PROPERTIES FOR THE PARAMETRIC1D AND PARAMETRIC2D DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
levelscale	double	1	The scaling factor applied to the levels if <code>levelscaleactive</code> is true.
levelscaleactive	boolean	false	If true, <code>levelscale</code> is used to scale the levels; otherwise the scale factor is computed automatically.
leveltrans	none expression	none	Use an expression in terms of the level variable to define a level transformation.
solnum	Integer array	All solutions	The solutions to use for extrapolation.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
seplevels	true false	true	Separate the levels to visualize data for each equidistant point in time, for example.
t	double array	Empty	The times to use as intermediate layers in the extrusion.
transexpr	String	level	Level transformation expression (as a function of <code>level</code> ; the default value means no transformation).

ParCurve2D, ParCurve3D

Create a 2D or 3D parametric curve dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"ParCurve2D");
model.result().dataset().create(<dtag>,"ParCurve3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>, "ParCurve2D")` creates a 2D parametric curve dataset feature named <dtag>.

`model.result().dataset().create(<dtag>, "ParCurve3D")` creates a 3D parametric curve dataset feature named <dtag>.

Evaluation is made along an arbitrary parametric curve in 2D or 3D.

The following properties are available: Add option for evaluating outside the mesh:

TABLE 7-101: VALID PROPERTIES FOR PARAMETRIC CURVE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
name	String	none	The name of this feature.
bndsnap	boolean	false	If true, each point is snapped to the closest boundary. Available in 3D only.
data	none dataset name	First compatible dataset	The dataset this feature refers to.
global	boolean	false	If set to true, the dataset only evaluates globally defined expressions.
par1	String	t	The curve parameter.
parmin1	double	0	The minimum value for par.
parmax1	double	1	The maximum value for par.
exprx	String	0	The expression for x(par).
expry	String	0	The expression for y(par).
exprz	String	0	The expression for z(par).
res	integer	1000	Resolution (number of discretization points along the curve).
tangent	String array of length 2 or 3	{"c1n1tx", "c1n1ty"} for the first Parametric Curve 2D dataset	Variables for the tangent to the parametric curve.

The `global` property can be useful for BEM models, for example, to be able to evaluate globally defined expressions outside the mesh.

SEE ALSO

[ParSurface](#)

ParSurface

Create a parametric surface dataset.

SYNTAX

```
model.result().dataset().create(<dtag>, "ParSurface");  
model.result().dataset(<dtag>).set(property, <value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>, "ParSurface")` creates a parametric surface dataset feature named <dtag>.

Evaluation is made along an arbitrary parametric surface in 3D.

The following properties are available:

TABLE 7-102: VALID PROPERTIES FOR A PARAMETRIC CURVE DATASET.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to
exprx	String	0	The expression for x(par1, par2)
expry	String	0	The expression for y(par1, par2)
exprz	String	0	The expression for z(par1, par2)
global	boolean	false	If set to true, the dataset only evaluates globally defined expressions
par1	String	s	The first surface parameter
par2	String	t	The second surface parameter
parmin1	double	0	The minimum value for par1.
parmax1	double	1	The maximum value for par1
parmin2	double	0	The minimum value for par2
parmax2	double	1	The maximum value for par2
res	integer	200	Resolution (the number of discretization points for each parameter)

The `global` property can be useful for BEM models and for far-field results processing to, for example, be able to evaluate globally defined expressions outside the mesh.

SEE ALSO

[ParCurve2D](#), [ParCurve3D](#)

Particle

Create a massless particle tracing plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Particle");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Particle")` creates a massless particle tracing plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

With massless particle tracing, you can visualize pathlines (that is, trajectories of particles released in a flow field), which can be time-dependent or static. For time-dependent flows you can also use a snapshot in time of the flow field as a static field. The motion of the particles does not affect the flow field. Particle tracing is available in 2D and 3D plot groups.

The following properties are available:

TABLE 7-103: VALID PROPERTIES FOR PARTICLE TRACING PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	head	Whether the head, tail, or center of the arrow is located at the particle position.
arrowexpr	string array		Active when <code>pointtype</code> is set to <code>arrow</code> . Determines the direction in which the arrow points.

TABLE 7-103: VALID PROPERTIES FOR PARTICLE TRACING PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
arrowlength	logarithmic normalized proportional	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowscale	positive double	1	If arrowscaleactive is true: the length scale factor
arrowscaleactive	boolean	false	If true, arrowscale is used; otherwise, the scale factor is computed automatically.
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.
atol	manual automatic	automatic	Whether to specify a manual absolute tolerance for the position. automatic indicates that the absolute tolerance for the position is the mean of the lengths of the bounding box of the geometry multiplied by the relative tolerance.
atolpos	positive double	0.001	Absolute tolerance for the position. Active when atol is set to manual.
bndcoord	double array	0	Vector of values from 0 to 1 to describe the starting points. Active when bndmethod is set to coord. Available in 2D only.
bndmethod	number coord	number	Active when posmethod is set to bnd. Available in 2D only.
bndnumber	positive integer	10	The number of equidistant starting points along the selected boundaries. Available in 2D only.
bndselection	Reference to a named selection on boundaries	First available named selection	The boundaries to start from. Available in 2D only.
comettailexpr	String array		Active when pointtype is set to comettail. Determines the direction in which the comet tail points.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
customlinecolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use for lines. Active when linecolor is set to custom.
custompointcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use for points. Active when pointcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expressions in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
dropfreq	positive double	1	Particles are released at these intervals when dropmethod is set to freq.
dropmethod	once freq times	once	Method to use when releasing particles.

TABLE 7-103: VALID PROPERTIES FOR PARTICLE TRACING PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
droptimes	double array	1	The specific times when to release particles. Used when dropmethod is set to times.
edgetol	positive double	0.001	The absolute tolerance controlling how close to the geometry boundary the pathlines are cut when they exit the geometry. A lower value cuts the line closer to the geometry boundary.
expr	String array of length 2 in 2D and 3 in 3D	Model dependent	Expressions for the components to plot.
hmax	double	0.1	The maximum time step. Active when stepsize is set to manual.
hstart	double	0.1	The initial time step. Active when stepsize is set to manual.
inheritarrowsscale	boolean	true	If inheritplot is not none: Determines if the arrow scale is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that sphere scale, tube scale, and deformation scale are inherited from.
inheritspherescale	boolean	true	If inheritplot is not none: Determines if the sphere scale factor is inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
inherit tubescale	boolean	true	If inheritplot is not none and linetype is tube: Determines if the tube scale is inherited.
interp	double array	Time corresponding to last selected solnum for transient levels	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
linecolor	custom black blue cyan gray green magenta red white yellow	black (2D), red (3D)	The uniform color to use for lines.
linetype	none line tube	line	Plot particle traces as lines or tubes, or not at all.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
maxsteps	positive integer	1000	The maximum number of steps to use in the particle simulation. Active when maxstepsactive is set to on.

TABLE 7-103: VALID PROPERTIES FOR PARTICLE TRACING PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
maxstepsactive	on off	off	Whether to manually specify the maximum number of steps used in particle simulation. off indicates that the limit varies in this way: for static flow fields, the algorithm uses the value 1000; for time-dependent flows, there is no upper limit of the number of steps, and the particle simulation goes on until it reaches the end time.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.
pointdom	disappear stick	stick	stick plots the points on the boundary at the exit point, disappear does not render these points at all.
pointlineanim	on off	off	When animating particle tracing plots with points, this property indicates whether the points at the current time should appear on top of the lines.
pointautoscale	on off	on	If enabled, the point radii are scaled so that the maximum radius is 0.01 of the plot scale.
pointradiusexpr	String	1	The point radius expression.
pointtype	none point comettail	none	Selects between plotting the pathlines' endpoints as points, comet tails, or not at all.
posmethod	start bnd	start	The type of particle tracing positioning. Choose specific starting points or a boundary to start from. Available in 2D only.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	1	The tube radius.
resolution	coarser coarse normal fine finer extrafine		Affects the number of output points by adding points between each time step taken in the ODE solver, using a 4th-order interpolation, to produce a smoother output.
rtol	positive double	0.001	The relative error tolerance that the ODE solver uses.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.

TABLE 7-103: VALID PROPERTIES FOR PARTICLE TRACING PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
static	on off	off	Only affects time-dependent problems. When set to on, this property freezes the time and considers this a static flow field.
statictend	double	1	The maximum time at which to end the particle-tracing simulation for static flow fields. Active when statictendactive is set to on.
statictendactive	on off	off	Whether to specify the end time for static flow fields automatically. (For time-dependent flows, the automatic end time is the last time that the time-dependent solver returns.) When set to off, there is no upper limit of the time. In this case, the particle simulation goes on until all particles exit the geometry or the simulation reaches the maximum number of steps.
stepsize	automatic manual	automatic	Whether to specify initial (hstart) and maximum (hmax) time step manually.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
tailscale	positive double	1	Active when pointtype is set to comettail and tailscaleactive is on. Specifies the manual scale factor with which the comet tail expression is multiplied.
tailscaleactive	on off	off	Active when pointtype is set to comettail. Specifies whether manual tail scaling is enabled.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
tstart	double	0	Manual start time. Active when tstartactive is set to on.
tstartactive	on off	off	on indicates a manual start time (set in the tstart property). off indicates that the start time becomes either the first time value that the time-dependent solver returns or 0 for stationary flows. Available when dropmethod is set to once or freq.
tubeautoscale	on off	on	If enabled, the tube radii are scaled so that the maximum tube radius is 0.02 of the plot scale.
tvar	String	partt	The name of the variable for time. Normally there is no need to change the default name. You can use this names in expressions as well as for the color when coloring the pathlines according to an expression.

TABLE 7-103: VALID PROPERTIES FOR PARTICLE TRACING PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
xcoord	double array	Empty	Absolute coordinates in the x direction, active when posmethod is set to start.
ycoord	double array	Empty	Absolute coordinates in the y direction, active when postmethod is set to start.
zcoord	double array	Empty	Absolute coordinates in the z direction, active when posmethod is set to coord. Available in 3D only.

ATTRIBUTES

[Color](#) (applies to lines), [Deform](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[ParticleMass](#), [Streamline](#)

Particle (1D Plot)

Create a particle plot, which plots particle properties over time or compares particle properties against each other at a set of time steps.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "Particle1D");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Particle1D")` creates a particle plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

With the particle plot, you can plot properties of all particles in a dataset versus time, or plot two particle properties against each other at a set of selected times. When plotting particle properties versus time, it is possible to apply data series operations to the particle data. The particle plot is available in 1D plot groups.

The following properties are available:

TABLE 7-104: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.

TABLE 7-104: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
dataseries	none average sum maximum minimum rms stddev variance	sum	The data series operation that is applied to all particles. Effective only when xdata is not expr.
descr	String array	Model dependent	The description of the expressions in expr. Is used in the automatic legends.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expr	String array	Model dependent	The expressions to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.

TABLE 7-104: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.

TABLE 7-104: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
prefixintitle	String	Empty	Added prefix to contribution to title.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-104: VALID PROPERTIES FOR PARTICLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xdata	expr solution	solution	x-axis data. expr uses the expression in xdataexpr. solution uses the available solutions in the underlying dataset, such as time steps.
xdataexpr	String	Model dependent	Expression for x-axis data.
xdatadescr	String	Model dependent	Description of expression in xdataexpr.
xdataphaseunit	String	rad	The unit in which xdataphaserange is described.
xdatasolnumtype	all inner outer valid level	outer	Whether the expression should be evaluated for every inner or every outer solution, or for a specific level (level1, level2, and so on). Applicable only for models containing multiple levels.
xdataunit	String	Model dependent	The unit to use for the expression in xdataexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES

[Color](#) (applies to lines), [Filter](#) (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing)

SEE ALSO

[Particle](#) (Evaluation), [ParticleTrajectories](#), [ParticleBin](#)

Particle (Dataset)

Create a particle dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Particle");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

model.result().dataset().create(<dtag>,"Particle") creates a particle dataset.

A particle dataset is required to plot particle trajectories. The particle dataset identifies the geometry in which the particle data is stored and the degrees of freedom which determine the position of each particle.

The following properties are available:

TABLE 7-105: VALID PROPERTIES FOR PARTICLE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
solution	String	first available solution	The solution this dataset refers to.
pgeomspec	manual fromphysics	fromphysics if an applicable physics interface is available, otherwise manual	The method of specifying the particle geometry and the names of the particle position degrees of freedom.
physicsinterface	none or the tag of a valid physics interface	none or first applicable physics interface	The physics interface from which the particle geometry and the names of the particle position degrees of freedom are defined.
pgeom	String	pgeom or pgeom_ followed by the tag of a particle tracing physics interface	The geometry in which the particle degrees of freedom are defined. The correct name of this particle geometry is pgeom_<tag>, where <tag> is the name for the Particle Tracing interface node.
posdof	String array	Position vector components of a particle tracing physics interface	The position degrees of freedom of the particles.

SEE ALSO

[Particle \(Evaluation\)](#)

Particle (Evaluation)

Particle evaluations to evaluate quantities on particle trajectories.

SYNTAX

```
model.result().numerical().create(<ftag>,"Particle");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"Particle")` creates a particle evaluation feature with the name <ftag>. Particle evaluation can be performed on trajectories accessed through a particle dataset.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that there is one row per point, with one row containing data for all solution numbers. This

is identical to (*columnwise*) when *columnwise* is **false**. If *columnwise* is **true**, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate, columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If *allocate* is **true**, a zero-valued matrix is allocated even when the result is real. `getImag()` uses *allocate* **true** and *columnwise* **false**.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. *<outersolnum>* is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the *table* property.

The following properties are available:

TABLE 7-106: VALID PROPERTIES FOR PARTICLE EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First particle compatible dataset	The particle dataset this feature refers to.
descr	String	Model dependent	The description of the expression in <i>expr</i> . Is used in the automatic title.
evaluate	all fraction number	all	What particles to evaluate.
expr	String	Model dependent	The expression to evaluate.
fraction	double	1	If <i>evaluate</i> is <i>fraction</i> : The fraction of particles to evaluate.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. <i>manual</i> indicates that <i>solnum</i> is used. <i>manualindices</i> indicates that <i>solnumindices</i> is used. <i>interp</i> indicates that <i>t</i> is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when <i>data</i> is not <i>parent</i> and the underlying data is transient.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <i>looplevelinput</i> is <i>manualindices</i> on a level.

TABLE 7-106: VALID PROPERTIES FOR PARTICLE EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
number	double	100	If evaluate is number: The number of particles to evaluate.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.

TABLE 7-106: VALID PROPERTIES FOR PARTICLE EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ParticleBin

Create a particle bin dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"ParticleBin");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

model.result().dataset().create(<dtag>,"ParticleBin") creates a particle bin set. This dataset evaluates an expression over all particles and then groups them into subintervals, or bins, based on which particles return similar values. If this dataset is then used in any other particle tracing plot or evaluation, each bin produces a single particle having position and other properties equal to the average over particles in the bin.

The following properties are available:

TABLE 7-107: VALID PROPERTIES FOR PARTICLE BIN DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First particle dataset	The particle dataset this feature refers to
distribution	equalnumber equalwidth	equalnumber	Distribution of particle bins. Active when method is set to number.
limits	String		String listing the limits of all bins. Active when method is set to limits.
method	limits number tolerance	limits	The method used to define the limits of the particle bins
number	positive integer	10	Number of particle bins. Active when method is set to number.
tolerance	String		Maximum difference between expression values such that particles are placed in the same bin. Active when method is set to tolerance.

ParticleMass

Create a particle tracing plot with mass.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"ParticleMass");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "ParticleMass")` creates a particle tracing plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

With particle tracing, you can visualize pathlines (that is, trajectories of particles released in a flow field), which can be time dependent or static. For time-dependent flows you can also use a snapshot in time of the flow field as a static field. The motion of the particles does not affect the flow field. Particle tracing is available in 2D and 3D plot groups.

The following properties are available:

TABLE 7-108: VALID PROPERTIES PAIRS FOR PARTICLE TRACING PLOTS WITH MASS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
arrowbase	heat tail center	head	Whether the head, tail, or center of the arrow is located at the particle position.
arrowexpr	string array		Active when pointtype is set to arrow. Determines the direction in which the arrow points.
arrowlength	logarithmic normalized proportional	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowscale	positive double	1	If arrowscaleactive is true: the length scale factor.
arrowscaleactive	boolean	false	If true, arrowscale is used; otherwise, the scale factor is computed automatically.
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.
atol	manual automatic	automatic	Whether to specify a manual absolute tolerance for the position and velocity. automatic indicates that the absolute tolerance for the position is the mean of the lengths of the bounding box of the geometry multiplied by the relative tolerance.
atolpos	positive double	0.001	Absolute tolerance for the position. Active when atol is set to manual.
atolvel	positive double	0.001	Absolute tolerance for the velocity. Active when atol is set to manual.
bndcoord	double array	0	Vector of values from 0 to 1 to describe the starting points. Active when bndmethod is set to coord. Available in 2D only.
bndmethod	number coord	number	Active when posmethod is set to bnd. Available in 2D only.
bndnumber	positive integer	10	The number of equidistant starting points along the selected boundaries. Available in 2D only.
bndselection	Reference to a named selection on boundaries	First available named selection	The boundaries to start from. Available in 2D only.

TABLE 7-108: VALID PROPERTIES PAIRS FOR PARTICLE TRACING PLOTS WITH MASS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
comettailexpr	String array		Active when pointtype is set to comettail. Determines the direction in which the comet tail points.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
customlinecolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use for lines. Active when linecolor is set to custom.
custompointcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use for points. Active when pointcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	Description of forces to plot.
dropfreq	positive double	1	Particles are released at these intervals when dropmethod is set to freq.
dropmethod	once freq times	once	Method to use when releasing particles.
droptimes	double array	1	The specific times when to release particles. Used when dropmethod is set to times.
edgetol	positive double	0.001	The absolute tolerance controlling how close to the geometry boundary the pathlines are cut when they exit the geometry. A lower value cuts the line closer to the geometry boundary.
fx	String	Mode-dependent	x-component for the force expression.
fy	String	Mode-dependent	y-component for the force expression.
fz	String	Model dependent	z-component for the force expression. In 2D, this property is only used if the underlying geometry is axisymmetric.
hmax	double	0.1	The maximum time step. Active when stepsize is set to manual.
hstart	double	0.1	The initial time step. Active when stepsize is set to manual.
inheritarrowsscale	boolean	true	If inheritplot is not none: Determines if the arrow scale is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that sphere scale, tube scale, and deformation scale are inherited from.
inheritspherescale	boolean	true	If inheritplot is not none: Determines if the sphere scale is inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.

TABLE 7-108: VALID PROPERTIES PAIRS FOR PARTICLE TRACING PLOTS WITH MASS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
inheritubescale	boolean	true	If inheritplot is not none and linetype is tube: Determines if the tube scale is inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
linecolor	custom black blue cyan gray green magenta red white yellow	black (2D), red (3D)	The uniform color to use for lines.
linetype	none line tube	line	Plot particle tracing lines as lines or tubes, or not at all.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
mass	String	1	The particle mass.
maxsteps	positive integer	1000	The maximum number of steps to use in the particle simulation. Active when maxstepsactive is set to on.
maxstepsactive	on off	off	Whether to manually specify the maximum number of steps used in particle simulation. off indicates that the limit varies in this way: for static flow fields, the algorithm uses the value 1000; for time-dependent flows, there is no upper limit of the number of steps, and the particle simulation goes on until it reaches the end time.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.
pointdom	disappear stick	stick	stick plots the points on the boundary at the exit point, and disappear does not render these points at all.

TABLE 7-108: VALID PROPERTIES PAIRS FOR PARTICLE TRACING PLOTS WITH MASS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
pointlineanim	on off	off	When animating particle tracing plots with points, this property indicates whether the points at the current time should appear on top of the lines.
pointtype	none point comettail	none	Selects between plotting the pathlines' endpoints as points, comet tails, or not at all.
posmethod	start bnd (in 2D)	start	The type of particle tracing positioning. Choose specific starting points or a boundary to start from. Available in 2D only.
radiusexpr	String	1	The tube radius.
pointautoscale	on off	on	If enabled, the point radii are scaled so that the maximum radius is 0.01 of the plot scale.
pointradiusexpr	String	1	The point radius expression.
resolution	coarser coarse normal fine finer extrafine		Affects the number of output points by adding points between each time step taken in the ODE solver, using a 4th-order interpolation, to produce a smoother output.
rtol	positive double	0.001	The relative error tolerance that the ODE solver uses.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
static	on off	off	Only affects time-dependent problems. When set to on, this property freezes the time and considers this a static flow field.
statictend	double	1	The maximum time at which to end the particle-tracing simulation for static flow fields. Active when statictendactive is set to on.
statictendactive	on off	off	Whether to specify the end time for static flow fields automatically. (For time-dependent flows, the automatic end time is the last time that the time-dependent solver returns.) When set to off, there is no upper limit of the time. In this case, the particle simulation goes on until all particles exit the geometry or the simulation reaches the maximum number of steps.
stepsize	automatic manual	automatic	Whether to specify initial (hstart) and maximum (hmax) time step manually.

TABLE 7-108: VALID PROPERTIES PAIRS FOR PARTICLE TRACING PLOTS WITH MASS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
tailscale	positive double	1	Active when pointtype is set to comettail and tailscaleactive is on. Specifies the manual scale factor with which the comet tail expression is multiplied.
tailscaleactive	on off	off	Active when pointtype is set to comettail. Specifies whether manual tail scaling is enabled.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title should be used (the title property). none, if no title should be displayed.
tstart	double	0	Manual start time. Active when tstartactive is set to on.
tubeautoscale	on off	on	If enabled, the tube radii are scaled so that the maximum tube radius is 0.02 of the plot scale.
tstartactive	on off	off	On indicates a manual start time (set in the tstart property). Off indicates that the start time becomes either the first time value that the time-dependent solver returns or 0 for stationary flows. Available when dropmethod is set to once or freq.
tvar	String	partt	The name of the variable for time. Normally there is no need to change the default name. You can use this names in expressions as well as for the color when coloring the pathlines according to an expression.
velvarx	String	partu	The name of the variable for the x-component of the velocity.
velvary	String	partv	The name of the variable for the y-component of the velocity.
velvarz	String	partw	The name of the variable for the z-component of the velocity. In 2D, this property is only used if the underlying geometry is axisymmetric.
velstartx	String	0	x-component for initial velocity.
velstarty	String	0	y-component for initial velocity.
velstartz	String	0	z-component for initial velocity. In 2D, this property is only used if the underlying geometry is axisymmetric.

TABLE 7-108: VALID PROPERTIES PAIRS FOR PARTICLE TRACING PLOTS WITH MASS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
xcoord	double array	Empty	Absolute coordinates in the x direction, active when posmethod is set to start.
ycoord	double array	Empty	Absolute coordinates in the y direction, active when postmethod is set to start.
zcoord	double array	Empty	Absolute coordinates in the z direction, active when posmethod is set to coord. Available in 3D only.

ATTRIBUTES

[Color](#) (applies to lines), [Deform](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[Particle](#), [PointTrajectories](#), [Streamline](#)

ParticleTrajectories

Create a particle trajectories plot in 2D and 3D.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"ParticleTrajectories");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"ParticleTrajectories")` creates a particle trajectories plot feature named `<ftag>` belonging to plot group `<pgtag>`.

A particle dataset is required to plot particle trajectories. The particle dataset identifies the geometry in which the particle data is stored and the degrees of freedom which determine the position of each particle.

The following properties are available:

TABLE 7-109: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	head	Whether the head, tail, or center of the arrow is located at the particle position.
arrowexpr	string array		Active when pointtype is set to arrow. Determines the direction in which the arrow points.
arrowlength	logarithmic normalized proportional	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowscale	positive double	1	If <code>arrowscaleactive</code> is true: the length scale factor.
arrowscaleactive	boolean	false	If true, <code>arrowscale</code> is used, otherwise the scale factor is computed automatically.

TABLE 7-109: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.
comettailexpr	String array	inverse of velocity vector of the first available particle tracing interface	Active when pointtype is set to comettail. Determines the direction in which the comet tail points and the relative sizes of the tails of different particles.
data	none parent dataset name	parent	The dataset this feature refers to.
ellipsearrowbase	head tail	head	The base for the arrow to draw for ellipses.
ellipsearrowtype	arrow cone none	arrow	The type of arrow to draw for ellipses.
ellipsearrowscale	double		The ellipse arrow scale factor, if ellipsearrowscaleactive is set to true.
ellipsearrowscaleactive	boolean	false	Set to true to specify an ellipse arrow scale when pointtype is ellipse.
ellipsecount	positive integer	10	Maximum number of ellipses, when ellipseentrymethod is numberofellipses.
ellipseentrymethod	numberofellipses listoftimes	numberofellipses	The ellipse entry type when pointtype is ellipse: number of ellipses or a list of times.
ellipsetimes	String array		Collection of times for polarization ellipses when ellipseentrymethod is listoftimes.
extrasteps	none specifiedtimes proportional	proportional	Controls how the number of extra time steps rendered in the trajectory plot is calculated. These extra time steps typically correspond to the exact times of reflections or velocity reinitializations.
fixedpointsize	boolean	false	Use constant radii for the plotted spheres.
illuminatelines	on off	on	Whether the plotted lines are affected by lighting when linetype is line. Available in 3D only.
inheritarrowscale	boolean	true	If inheritplot is not none: Determines if the arrow scale is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.

TABLE 7-109: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
inheritellipsescale	boolean	true	If inheritplot is not none: Determines if the ellipse scale factor is inherited.
inheritplot	none plot name	none	The plot that color, color range, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If inheritplot is not none: Determines if the radius scale factor is inherited.
inherittailscale	boolean	true	If inheritplot is not none: Determines if the tail scale factor is inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
inherittubescale	boolean	true	If inheritplot is not none: Determines if the tube radius scale factor is inherited.
interpcount	positive integer	100	Number of interpolated time, when interpolation is set to uniform.
interpolation	none uniform	none	The type of interpolation used to plot lines when linetype is line, ribbon, or tube.
linecolor	custom black blue cyan gray green magenta red white yellow	black (2D), red (3D)	The uniform color to use for lines.
linetype	none line ribbon tube	line	Plot particle traces as lines, ribbons, tubes, or not at all.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
numextrasteps	nonnegative integer	100	If extrasteps is specifiedtimes, controls the maximum number of extra time steps to render in the plot.
pointautoscale	on off	on	If enabled, the point radii are scaled based on the size of the geometry.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.

TABLE 7-109: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
pointlineanim	on off	off	When animating particle tracing plots with points, this property indicates whether the points at the current time should appear on top of the lines.
pointradiusexpr	String	0.001	The point radius expression.
pointtype	none point comettail arrow ellipse	none	Plot particles as points, with comet tails, with arrows, with polarization ellipses (3D only), or not at all.
propextrasteps	nonnegative integer	1	If extrasteps is proportional, controls the maximum number of extra time steps to render in the plot. The maximum number of extra steps is the product of this proportionality factor with the number of solution times.
radiusexpr	String	1	The tube radius expression
ribbondirexpr	String array		Expressions for the components of the ribbon direction.
semimajorexpr	String array		Expressions for the polarization ellipses' semi-major axis,
semiminorexpr	String array		Expressions for the polarization ellipses' semi-minor axis,
sphereradiusscale	positive double		The scale factor applied to the point radii if sphereradiusscaleactive is on.
sphereradiusscaleactive	on off	off	If on, sphereradiusscale is used; otherwise the scale factor is computed automatically.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
tailscale	positive double	1	Active when pointtype is set to comettail and tailscaleactive is on. Specifies the manual scale factor with which the comet tail expression is multiplied.
tailscaleactive	on off	off	Active when pointtype is set to comettail. Specifies whether manual tail scaling is enabled.
title	String		Active when title is set to manual. Specifies the plot title.
titletype	automatic manual none	automatic	Determines how the plot title is specified.
tuberadiusscale	String	1	The scale factor applied to the tube radii if tuberadiusscaleactive is on.

TABLE 7-109: VALID PROPERTIES FOR PARTICLE TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
tuberadiusscaleactive	on off	off	If on, tuberadiusscale is used; otherwise, the scale factor is computed automatically.
widthscale	double	1	The scale factor applied to the ribbon widths if widthscaleactive is true.
widthscaleactive	boolean	false	If true, widthscale is used; otherwise, the scale factor is computed automatically.

ATTRIBUTES

[Color](#), [Deform](#), [Export](#), [Filter](#) (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing), [MaterialAppearance](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[Particle](#) (Evaluation), [PointTrajectories](#), [RayTrajectories](#)

Partition

Partition a domain with respect to a set of isolevels and add elements corresponding to the isosurfaces or contours.

SYNTAX

```
model.result().dataset().create(<dtag>,"Partition");

model.result().dataset(<dtag>).set(property,<value>);

model.result().dataset(<dtag>).createMeshBasedGeom(<compTag>,<meshBasedGeomTag>);
model.result().dataset(<dtag>).createMesh(<compTag>,<geomTag>,<meshTag>,<importTag>);
model.result().dataset(<dtag>).createMeshPart(<compTag>,<geomTag>,<meshTag>,<importTag>);
```

DESCRIPTION

model.result().dataset().create(<dtag>,"Partition") creates a partition dataset with the name <dtag>.

The partition dataset makes it possible to partition a domain with respect to a set of isolevels and add elements corresponding to the isosurfaces or contours, using a scalar expression for which to apply the partitioning. Using the createMeshBasedGeom, createMesh, and createMeshPart methods, you can create a mesh-based geometry, mesh, or mesh part with a mesh import that refers to the Partition dataset.

The following properties are available:

TABLE 7-110: VALID PROPERTIES FOR PARTITION DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The solution dataset this feature refers to.
descr	String		A description for the expression to use for the partitioning.
expr	String		The expression for which to apply the partitioning.
isolevel	double array	Empty	The isolevels, if levelmethod is levels.
level	fromdataset volume surface line point	fromdataset	The geometry level to evaluate on.

TABLE 7-110: VALID PROPERTIES FOR PARTITION DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
levelmethod	number levels	numbers	The definition of levels: a number of levels or a list of levels.
number	integer	1	The number of levels, if levelmethod is number.
numberrounding	true false	true	Round the levels, if levelmethod is number.
propagatetolower	true false	true	Propagate the filter to lower geometry dimensions.
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
useder	true false	true	Use cubic (Hermite) interpolation when applying the filter.

SEE ALSO[Filter \(Dataset\)](#)*Pellets*

Create a pellets plot in 2D and 3D plot groups.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Pellets");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Pellets")` creates a plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

You can use pellets plots in 3D models to visualize the flow of pellets and similar objects.

TABLE 7-III: VALID PROPERTIES FOR PELLETS PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
axis	x y z	y	The main axis for pellet spheres, if type is set to sphere.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is set to gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is set to uniform.
coloring	colortable uniform gradient	colortable	How to color the surface: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
corecolor	custom black blue cyan gray green magenta red white yellow	black	The core color to use. Active when shrinkingcore is set to true.
coreradiusexpr	String	0	The core radius expression, when shrinkingcore is set to true.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.

TABLE 7-III: VALID PROPERTIES FOR PELLETS PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, height scale, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendtitle has been set to true in the parent plot group.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-III: VALID PROPERTIES FOR PELLETS PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
revangle	double	240	The revolution angle (in degrees) for pellet spheres, if type is sphere.
shrinkingcore	boolean	false	Enable a shrinking core with a space-dependent radius scale factor.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.

TABLE 7-III: VALID PROPERTIES FOR PELLETS PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
startangle	double	210	The start angle (in degrees) for the revolution of pellet spheres, if type is sphere.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
type	sphere slice	sphere	The type of pellet: sphere or slices (three orthogonal filled circles).
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

[Deform](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[Particle](#)

PhasePortrait

Create a phase portrait plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"PhasePortrait");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "PhasePortrait")` creates a plot feature named <ftag> belonging to the plot group <pgtag>.

You can use phase portraits in 2D and 2D axisymmetric models to visualize large datasets of particle trajectories.

The following properties are available:

TABLE 7-112: VALID PROPERTIES FOR PHASE PORTRAIT.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none parent dataset name	parent	The dataset this feature refers to.
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is <code>custom</code> .
fixedpointsize	boolean	false	Use constant radii for the plotted spheres.
inheritcolor	boolean	true	If <code>inheritplot</code> is not none: Determines if the color is inherited. Available in 2D and 3D.
inheritrange	boolean	true	If <code>inheritplot</code> is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the radius scale factor is inherited.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
looplevel	integer row matrix	all on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>looplevelinput</code> is <code>manualindices</code> on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. <code>manual</code> on a level indicates that <code>looplevel</code> is used on that level. <code>manualindices</code> on a level indicates that <code>looplevelindices</code> is used on that level. <code>interp</code> on a level indicates that <code>interp</code> is used on that level.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. <code>manual</code> indicates that <code>outersolnum</code> is used. <code>manualindices</code> indicates that <code>outersolnumindices</code> is used.

TABLE 7-112: VALID PROPERTIES FOR PHASE PORTRAIT.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
prefixintitle	String	Empty	Added prefix to contribution to title.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.
pointradiusexpr	String	1	The point radius expression.
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
sphereradiusscale	positive double		The scale factor applied to the point radii if sphereradiusscaleactive is on.
sphereradiusscaleactive	boolean	false	If true, sphereradiusscale is used; otherwise the scale factor is computed automatically.
suffixintitle	String	Empty	Added suffix to contribution to title
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when titletype is manual
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the title property). none, if no title contribution should be used.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-112: VALID PROPERTIES FOR PHASE PORTRAIT.

NAME	VALUE	DEFAULT	DESCRIPTION
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom
xdata	position manual	position	Whether to use the particle position or user-defined x-axis data.
xmanual	String	Empty	User-defined x-axis data used if xdata is set to manual.
ydata	position manual	position	Whether to use the particle position or user-defined y-axis data.
ymanual	String	Empty	User-defined y-axis data used if ydata is set to manual.

ATTRIBUTES

[Color](#), [MaterialAppearance](#)

*Pipe***SYNTAX**

```
model.result().dataset().create(<dtag>,"Pipe");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Pipe")` creates a pipe dataset with the name `<dtag>`.

Pipe datasets can be used with stored solutions from models with pipe physics. Use it to visualize pipe structures in 3D using the pipes' cross sections.

The following properties are available:

TABLE 7-113: VALID PROPERTIES FOR PIPE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
physicsinterface	all physics interface name	all	The name of an applicable physics interface, or all for all applicable physics interfaces.
refinement	integer	2	The refinement of the cross section (1–5).
scale	double scalar	1	The scaling of the cross section.
spacevars	String array	Derived from feature name	The names of the space variables for the local pipe coordinates.
wallstol	double scalar	1e - 4	Walls relative tolerance.

SEE ALSO

[Beam](#)

Plot

Export plots to files.

SYNTAX

```
model.result().export().create(<ftag>, "Plot");  
model.result().export().create(<ftag>, <pgtag>, <plottag>, "Plot");  
model.result().export(<ftag>).set(property, <value>);  
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>, "Plot")` creates a plot export feature with the name `<ftag>`.

`model.result().export().create(<ftag>, <pgtag>, <plottag>, "Plot")` creates a plot export feature with the name `<ftag>` for the plot `<plottag>` in the plot group `<pgtag>`.

To export a 3D surface from a plot to an STL file, use `.stl` as the filename extension.

The following properties are available:

TABLE 7-114: VALID PROPERTIES FOR PLOT EXPORTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alwaysask	on off	off	Always ask for filename when saving. This property is ignored when running without a UI.
color	boolean	false	For PLY and 3MD files: include vertex colors in the data export.
exporttype	text vtu stlascii stlbin plyascii plybin 3mf wav	text	File type for the export: a text file, VTU file, STL ASCII or binary file (3D surfaces only), PLY ASCII or binary file (3D surfaces only), 3MF file (3D surfaces only), or WAV audio files (1D graph plots, function plots, table plots, and impulse response plots only).
ifexists	append overwrite	overwrite	If the file exists, append to or overwrite the file contents. Not available for VTU files.
plotgroup	String		The name of the plot group containing the plot to export
plot	String		The name of the plot (in plotgroup) to export
compact	boolean	false	Only applicable for streamline and particle tracing plots. If true, the exported data contains one line per streamline or particle rather than one line per point on a streamline or particle trajectory.
filename	String		The output file.
fullprec	on off	on	If on, floating-point numbers are written in full precision, otherwise they are written with six significant digits. Not for STL files.
header	on off	off	Enable/disable a data header in the output file (for text files only).
multiplecurves	asrows ascolumns	asrows	For 1D plot groups with multiple curves using the same <i>x</i> -coordinates: Save as rows with two columns or save as columns, with one column for the <i>x</i> -coordinates and one column for the <i>y</i> -coordinates for each curve.
paramvalues	true false	true	Whether to include parameter values in data headers (for text files only).
quantlevel	8bit 16bit 32bitfloat		The quantization level, if exporttype is wav.
repeat	on off	off	Enable/disable repeated playing of the animation.

TABLE 7-114: VALID PROPERTIES FOR PLOT EXPORTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
samplingfreq	integer		The sampling frequency, if samplingfreqmode is user.
samplingfreqmode	auto user	auto	The sampling frequency mode (automatic or user defined), if exporttype is wav.
separator	C \t , ; :	C	The column separator for the data in the output text file. C means column; the second option is a space, and the third option is a tab.
sort	on off	off	Enable/disable sorting of the points with respect to the coordinates.
struct	sectionwise spreadsheet	spreadsheet	Format of the exported data.
wavnorm	maxampl user	maxampl	The normalization of WAV audio data: maximum amplitude or a user-defined value, if exporttype is wav.
wavnormval	double	1	The normalization value, if wavnorm is user.

SEE ALSO

[Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#), [RadiationPattern \(Export\)](#)

PlotGroup1D, PlotGroup2D, PlotGroup3D

Create a 1D, 2D, or 3D plot group.

SYNTAX

```
model.result().create(<pgtag>, dim);
model.result(<pgtag>).set(property, <value>);
model.result(<pgtag>).selection()...;
model.result(<pgtag>).run();
```

DESCRIPTION

`model.result().create(<pgtag>, dim)` creates a plot group named `<pgtag>` of view dimension `dim`. A plot group is a group of plots that are shown together in a graphics or plot window.

2D and 3D plot groups can include selections of geometric entities. See [Selections](#).

1D plot groups contain graph plots. In the list of properties below, properties related to manual color legend settings are marked with an asterisk (*) and are only available if the 1D plot group includes any plot that has a color expression.

The following properties are available for 1D plot groups:

TABLE 7-115: VALID PROPERTIES FOR 1D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
axisactive	on off	off	Use manual axis formatting settings.
axiscommonexp	on off	on	Use a common exponent for all axis values.
axisnotation	automatic engineering scientific	automatic	Use an automatic notation (decimal or scientific) for axis values or always use engineering or scientific notation.
axisprecision	positive integer	4	Precision for number format (number of digits).
axistrailingzeros	on off	off	Show trailing zeros for axis values.

TABLE 7-115: VALID PROPERTIES FOR 1D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
colorlegendpos	alternating bottom left leftdouble right rightdouble	right	The position of the legends for all plots in this group (when a Color Expression has been added).
config	configuration name		The configuration to use for the solution times or parameters, if solutionparams is set to configuration.
data	none dataset name	First compatible dataset	The dataset this feature refers to. This is the default dataset for all plots in the group.
datasetintitle	on off	off	Whether the title should contain the dataset when titletype is custom.
dateintitle	on off	off	Whether the title contribution of plots in this group should contain the current date when titletype is custom.
descriptionintitle	on off	on	Whether the title contribution of plots in this group should contain the description when titletype is custom.
expressionintitle	on off	off	Whether the title contribution of plots in this group should contain the expression when titletype is custom.
filenameintitle	on off	off	Whether the title contribution of plots in this group should contain the name of the MPH file when titletype is custom.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not none and the underlying data is transient.
legendactive*	on off	off	Use manual color legend formatting settings.
legendbackground	true false	true	Whether the legend background and frame should be visible.
legendcolumncount	positive integer	1	The number of columns of legends. If legendposoutside is set to bottom or top, use legendrowcount instead.
legendcommonexp*	on off	on	Use a common exponent for all color legend values.
legendlayout	inside outside	inside	Show legends inside of or outside of the graphics axis area.
legendmaxwidthinside	between 0 and 1	0.5	Maximum relative legend width, if legendlayout is set to inside.
legendnotation*	automatic engineering scientific	automatic	Use an automatic notation (decimal or scientific) for color legend values or always use engineering or scientific notation.

TABLE 7-115: VALID PROPERTIES FOR 1D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
legendpos	upperright middleright lowerright upperleft middleleft lowerleft middleright center middleleft manual	upperright	The position of the legends for all plots in this group, if legendlayout is set to inside. Choose manual to specify the exact position.
legendposoutside	bottom left right top	right	The position of the legends for all plots in this group, if legendlayout is set to outside.
legendposx	double	1	The x-position (0–1) for the legend position when legendpos is set to manual.
legendposy	double	1	The y-position (0–1) for the legend position when legendpos is set to manual.
legendprecision*	positive integer	3	Precision for number format (number of digits).
legendrowcount	positive integer	1	The number of rows of legends if legendposoutside is set to bottom or top.
legendtrailingzeros*	on off	off	Show trailing zeros for axis values.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
manualgrid	on off	off	Whether to use the automatic grid spacing or the grid settings specified in xspacing and yspacing.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.

TABLE 7-115: VALID PROPERTIES FOR 1D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
paramindicator	String		The parameter indicator to use when titletype is manual.
phase	double	0	The phase angle for the solution, if phasetype is manual.
phaseintitle	on off	off	Whether the title should contain the phase when titletype is custom.
phasetype	fromdataset customperplot manual	fromdataset	The source for the phase (solution at angle): from dataset, custom per plot in the plot group, or manual, if the solution dataset is applicable. If you choose customperplot, the phasetype property with the options fromdataset and manual (with a phase property) is available for the plots in the plot group.
prefixintitle	String	Empty	Added prefix to group contribution to title.
preserveaspect	on off	off	Whether the x- and y-axis should have equal scale.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere).
refine	nonnegative integer	1	The element refinement to use, if resolution is set to custom.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the resolution of the plots in the plot group. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
savedatainmodel	true false	false	Save plot data in the model.
showgrid	true false	true	Display the grid in the plot.
showlegendsmaxmin	true false	false	Show maximum and minimum values for color legends (when a Color Expression has been added).
showlegendstitle	true false	false	Show legend title (when a Color Expression has been added).
showlegendsunit	true false	false	Show units for color legends (when a Color Expression has been added).
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.

TABLE 7-115: VALID PROPERTIES FOR 1D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
solnum	Integer array	All solutions	The solutions to plot. Available when the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.
suffixintitle	String	Empty	Added suffix to group contribution to title.
switchxy	on off	off	Switch the data on the x- and y-axes in 1D plot groups.
t	double array	Empty	The times to plot. Available when the underlying solution is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeintitle	on off	off	Whether the title contribution of plots in this group should contain the current time when titletype is custom.
title	String	The auto-title.	The title to use when titletype is manual.
titlealwaysimaginary	true false	false	Always show the imaginary part when titlenumberformat is stopwatch or stopwatchsci.
titlealwaysign	true false	false	Always show the sign when titlenumberformat is stopwatch or stopwatchsci.
titlealwaysignexp	true false	false	Always show the sign of the exponent when titlenumberformat is stopwatchsci.
titlecolor	custom black blue cyan gray green magenta red white yellow	black	The color of the title.
titledecimals	positive integer	3	The number of decimals for numerical values in the title when titlenumberformat is stopwatch or stopwatchsci.
titleexponentdigits	positive integer (1–3)	1	The number of exponent digits (1–3) for numerical values in the title when titlenumberformat is stopwatchsci.
titleintegerdigits	positive integer	3	The number of integer digits for numerical values in the title when titlenumberformat is stopwatch.

TABLE 7-115: VALID PROPERTIES FOR 1D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
titlenumberformat	default auto engineering scientific stopwatch stopwatchsci	default	The number format for the title: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.
titleparamindicator	true false	true	Add a parameter indicator to the plot title, when titletype is custom or label.
titleprecision	positive integer	6	The precision for numerical values in the title when titlenumberformat is auto, engineering, or scientific.
titletrailingzeros	true false	false	Include trailing zeros when titlenumberformat is auto, engineering, or scientific.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
twoyaxes	true false	false	Use two y-axes.
typeintitle	on off	on	Whether the title contribution of plots in this group should contain the type when titletype is custom.
unitintitle	on off	on	Whether the title contribution of plots in this group should contain the unit when titletype is custom.
xextra	double array	Empty	Extra grid points to include along the x-axis.
xlabel	String	Empty	The label on the x-axis.
xlabelactive	on off	off	on if a manual x-axis label should be used, off if it should be computed automatically.
xlog	on off	off	Whether to use a logarithmic scale along the x-axis.
xmax	double	Computed automatically	The maximum value of the x-axis.
xmin	double	Computed automatically	The minimum value of the x-axis.
xspacing	double	Computed automatically	The grid spacing on the x-axis. Used if manualgrid is set to on.
yextra	double array	Empty	Extra grid points to include along the y-axis
ylabel	String	Empty	The label on the y-axis.
ylabelactive	on off	off	on if a manual y-axis label should be used, off if it should be computed automatically.
ylog	on off	off	Whether to use a logarithmic scale along the y-axis.

TABLE 7-115: VALID PROPERTIES FOR 1D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
ylogsec	on off	off	Whether to use a logarithmic scale along the secondary y-axis.
ymax	double	Computed automatically	The maximum value of the y-axis.
ymaxsec	double	Computed automatically	The maximum value of the secondary y-axis.
ymin	double	Computed automatically	The minimum value of the y-axis.
yminsec	double	Computed automatically	The minimum value of the secondary y-axis.
ysecextra	double array	Empty	Extra grid points to include along the secondary y-axis.
yseclabel	String	Empty	The label on the secondary y-axis.
yseclabelactive	on off	off	on if a manual secondary y-axis label should be used, off if it should be computed automatically.
ysecspacing	double	Computed automatically	The grid spacing on the secondary y-axis. Used of manualgrid is set to on.
yspacing	double	Computed automatically	The grid spacing on the y-axis. Used of manualgrid is set to on.
window	graphics new windowX, where X is an integer	graphics	The window where the plot group is displayed. When plotting on a graphics server, graphics is equivalent to new.
windowtitle	String	Computed automatically	The title to use for the window where the plot group is displayed. It is not possible to change the title of the graphics window.
windowtitleactive	on off	off	Set windowtitleactive to on to enter a manual window title in windowtitle.

The following properties are available for 2D and 3D plot groups:

TABLE 7-116: VALID PROPERTIES FOR 2D AND 3D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrayaxis	x y z (3D only)	x	The array axis, if plotarraytype is linear.
arrayplane	xy yz xz	xy	The array plane, if plotarraytype is square in a 3D plot group.
axisactive	on off	off	Use manual axis formatting settings.
axiscommonexp	on off	on	Use a common exponent for all axis values.
axisnotation	automatic engineering scientific	automatic	Use an automatic notation (decimal or scientific) for axis values or always use engineering or scientific notation.
axisprecision	positive integer	4	Precision for number format (number of digits).
axistrailingzeros	on off	off	Show trailing zeros for axis values.
celldisp	double	0	The cell displacement, if displacementlinear is set to absolute.

TABLE 7-116: VALID PROPERTIES FOR 2D AND 3D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
columnndisp	double	0	The column cell displacement, if displacementsquare is set to absolute.
columnpadding	double	0	Absolute column padding for a square plot array.
config	configuration name		The configuration to use for the solution times or parameters, if solutionparams is set to configuration.
customedgecolor	RGB-triplet	{1,1,1} or last used edgecolor	The color to use when plotting dataset edges. Active when edgecolor is set to custom.
data	none dataset name	parent	The dataset this feature refers to. This is the default dataset for all plots in the group.
datasetintitle	on off	off	Whether the title should contain the dataset when titletype is custom.
displacementlinear	relative absolute	relative	Displacement of a linear plot array: relative or absolute.
displacementsquare	relative absolute	relative	Displacement of a square plot array: relative or absolute.
edgecolor	custom black blue cyan gray green magenta red white yellow	black	The color to use when plotting dataset edges.
edges	on off	on	Whether to plot dataset edges.
expressionintitle	on off	off	Whether the title contribution of plots in this group should contain the expression when titletype is custom.
frametype	mesh material spatial geometry fromdataset	spatial	The frame used when dataset edges are plotted. Use fromdataset to take the frame from the dataset.
inherithide	on off	off	Whether to propagate hiding of objects to lower dimensions in plots.
interp	double array	Time corresponding to last selected solnum for transient levels	The time to use, for transient levels. Available when data is not none and the underlying data is transient.
legendactive	on off	off	Use manual color legend formatting settings.
legendcolor	custom black blue cyan gray green magenta red white yellow	black	How to color the values in the color legend.
legendcommonexp	on off	on	Use a common exponent for all color legend values.
legendnotation	automatic engineering scientific	automatic	Use an automatic notation (decimal or scientific) for color legend values or always use engineering or scientific notation.

TABLE 7-116: VALID PROPERTIES FOR 2D AND 3D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
legendprecision	positive integer	3	Precision for number format (number of digits).
legendpos	alternating bottom left leftdouble right rightdouble	right	The position of the legends for all plots in this group.
legendtrailingzeros	on off	off	Show trailing zeros for axis values.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not none and the underlying data has multiple solutions.
order	rowmajor columnmajor	rowmajor	The ordering of a square plot array: row-major or column-major.
outersolnum	nonnegative integer	1	The index of the outer solutions to use Applicable only for parametric sweep models.
padding	double	0	Absolute padding for a linear plot array.
paddinglinear	relative absolute	relative	Padding of a linear plot array, if displacementlinear is set to relative: relative or absolute.
paddingsquare	relative absolute	relative	Padding of a square plot array, if displacementsquare is set to relative: relative or absolute.
paramindicator	String		The parameter indicator to use when titletype is manual.
phase	double	0	The phase angle for the solution, if phasetype is manual.
phaseintitle	true false	false	Whether the title should contain the phase when titletype is custom.
phasetype	fromdataset customperplot manual	fromdataset	The source for the phase (solution at angle): from dataset, custom per plot in the plot group, or manual, if the solution dataset is applicable. If you choose customperplot, the phasetype property with the options fromdataset and manual (with a phase property) is available for the plots in the plot group.
plotarraytype	none linear square	none	The plot array type.
prefixintitle	String	Empty	Added prefix to group contribution to title.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere).
refine	nonnegative integer	1	The element refinement to use, if resolution is set to custom.
relcolumnpadding	double	0.3	Relative column padding for a square plot array.
relpadding	double	0.3	Relative padding for a linear plot array.

TABLE 7-116: VALID PROPERTIES FOR 2D AND 3D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
relrowpadding	double	0.3	Relative row padding for a square plot array.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the resolution of the plots in the plot group. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
rowdisp	double	0	The row cell displacement, if displacementsquare is set to absolute.
rowpadding	double	0	Absolute row padding for a square plot array.
showhiddenobjects	true false	false	Whether hidden objects should be visible in plots.
showlegends	true false	true	Show color legends.
showlegendsmaxmin	true false	false	Show maximum and minimum values in color legends.
showlegendstitle	true false	false	Show color legend title.
showlegendsunit	true false	false	Show the unit above the color legend.
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionintitle	on off	on	Whether the title should contain the solution when titletype is custom.
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.
suffixintitle	String	Empty	Added suffix to group contribution to title.
symmetryaxis	true false	false	Show the symmetry axis in the plot (for 2D axisymmetric plot only).
t	double	Time corresponding to last selected solnum	The time to use, for transient problems. If it is not available in any solnum, the time is interpolated in the solution.

TABLE 7-116: VALID PROPERTIES FOR 2D AND 3D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title	The title to use when titletype is manual.
titleparamindicator	true false	true	Add a parameter indicator to the plot title, when titletype is custom.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution of plots in this group should contain the type when titletype is custom.
unitintitle	on off	on	Whether the title contribution of plots in this group should contain the unit when titletype is custom.
view	auto new view name	auto	The view settings to use when displaying this plot group. auto indicates that the view is selected automatically, or be created if there is none. new creates a new view. Typically geometry-based datasets such as solution or mesh datasets use the current geometry view. Surface plots with height attributes do not use the view setting.
window	graphics new windowX, where X is an integer	graphics	The window where the plot group is displayed. When plotting on a graphics server, graphics is equivalent to new.
windowtitle	String	Computed automatically	The title to use for the window where the plot group is displayed. It is not possible to change the title of the graphics window.
windowtitleactive	on off	off	Set windowtitleactive to on to enter a manual window title in windowtitle.
xlabel	String	Empty	The label on the x-axis. Available for 2D plot groups.
xlabelactive	on off	off	on if a manual x-axis label should be used, off if it should be computed automatically. Available for 2D plot groups.

TABLE 7-116: VALID PROPERTIES FOR 2D AND 3D PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
ylabel	String	Empty	The label on the y-axis. Available for 2D plot groups.
ylabelactive	on off	off	on if a manual y-axis label should be used, off if it should be computed automatically. Available for 2D plot groups.

2D and 3D Plot Properties for Plot Arrays

When you have enabled the `plotarrayenable` property for a 2D or 3D plot group, all plots in that plot group support a few extra properties related to the plot array. See the following table for information about those properties:

TABLE 7-117: PROPERTIES FOR PLOTS IN 2D AND 3D PLOT GROUPS WITH PLOT ARRAYS.

NAME	VALUE	DEFAULT	DESCRIPTION
applytodatasetedgesplotarr	true false	true	Make the plot array apply to dataset edges.
arrayindex	0 or positive integer	0	The array index if <code>manualindexing</code> is set to true for a linear plot array.
belongstoplotarray	true false	true	True if the plot belongs to the plot array.
colindex	0 or positive integer	0	The column index if <code>manualindexing</code> is set to true for a square plot array.
manualindexing	true false	false	Use manual 0-based indexing for the position of this plot.
rowindex	0 or positive integer	0	The row index if <code>manualindexing</code> is set to true for a square plot array.

SEE ALSO

[EvaluationGroup](#), [PolarGroup](#), [Solution](#), [SmithGroup](#)

PoincareMap

Create a plot of a Poincaré map.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "PoincareMap");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "PoincareMap")` creates a plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Plots of a Poincaré map are used to visualize where particle trajectories intersect a given plane, for example, to illustrate where a ray passes or where particles are deposited. They can be added to 2D and 3D plot groups.

The following properties are available:

TABLE 7-118: VALID PROPERTIES FOR POINCARÉ MAP.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none parent dataset name	parent	The dataset this feature refers to.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
fixedpointsize	boolean	false	Use constant radii for the plotted spheres.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited. Available in 2D and 3D.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If inheritplot is not none: Determines if the radius scale factor is inherited.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
looplevel	integer row matrix	all on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.

TABLE 7-118: VALID PROPERTIES FOR POINCARÉ MAP.

NAME	VALUE	DEFAULT	DESCRIPTION
prefixintitle	String	Empty	Added prefix to contribution to title.
pointradiusexpr	String	1	The point radius expression.
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1, 1, 20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
sphereradiusscale	positive double		The scale factor applied to the point radii if <code>sphereradiusscaleactive</code> is on.
sphereradiusscaleactive	boolean	false	If true, <code>sphereradiusscale</code> is used; otherwise the scale factor is computed automatically.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title	The title to use when <code>titletype</code> is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the <code>title</code> property). none, if no title contribution should be used.
typeintitle	on off	on	Whether the title contribution should contain the type when <code>titletype</code> is custom.
unitintitle	on off	on	Whether the title contribution should contain the unit when <code>titletype</code> is custom.

ATTRIBUTES

[Color](#), [MaterialAppearance](#), [Transparency](#)

Point

Create a point plot in 2D or 3D.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "Point");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Point")` creates a point plot feature named <ftag> belonging to the 2D or 3D plot group <pgtag>.

Point plots are used to visualize points using different geometric shapes and to plot torque arrows in 2D. Point plots can be added to 2D and 3D plot groups.

The following properties are available:

TABLE 7-119: VALID PROPERTIES FOR POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
alwaysassign	true false	false	Always show the sign when numberformat is stopwatch or stopwatchsci.
alwaysassignexp	true false	false	Always show the sign of the exponent when numberformat is stopwatchsci.
anchorpoint	upperright uppermiddle upperleft middleright center middleleft lowerright lowermiddle lowerleft	upperleft	The position of the point text relative to the point, if pointtype is set to text.
backgroundcolor	custom fromtheme black blue cyan gray green magenta red white yellow none	none	The color to use for background rectangle if pointtype is set to text.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the points: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.

TABLE 7-119: VALID PROPERTIES FOR POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
decimals	positive integer	2	Number of decimals when numberformat is stopwatch or stopwatchsci.
expr	String	Model dependent	The expression to plot.
fixedpointsize	true false	false	Whether to use a fixed point size.
gporder	positive integer	1	If placement is gausspoints: The Gauss-point order.
heightexpr	String	1	Height expression, if pointtype is rectangle or ellipse.
includeunit	true false	false	Whether to include the unit for the values if pointtype is set to text.
integerdigits	positive integer	3	Number of integer digits when numberformat is stopwatch.
labelprefix	String		Add an optional prefix to the point plot if pointtype is set to text.
labelsuffix	String		Add an optional suffix to the point plot if pointtype is set to text..
lagorder	integer	1	The Lagrange-point order to use if resolution is custom.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
level	line point surface volume	point	The geometry level for evaluating point data. The default is to take it from points. volume is only available in 3D.
maxpointcount	nonnegative integer	100000	The maximum number of points for point plots when level is not point.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
numberformat	default auto engineering scientific stopwatch stopwatchsci	auto	The number format for the point text: default, automatic, engineering, scientific, stopwatch, or scientific stopwatch.

TABLE 7-119: VALID PROPERTIES FOR POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
orientation	horizontal vertical	horizontal	Orient the point text horizontally or vertically, if pointtype is set to text.
placement	meshnodes gausspoints	meshnodes	Placement of point evaluation, node points or Gauss points, if level is not point.
pointtype	point text block (3D) ellipsoid (3D) torque (2D) rectangle (2D) ellipse (2D)	point	The point type: point, text, block, or ellipsoid in 3D; point, text, torque, rectangle, or ellipse in 2D.
precision	positive integer	6	The precision for numerical values in the point text when numberformat is auto, engineering, or scientific.
radiusexpr	String	1	Point radius expression, if pointtype is point or torque.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when placement is set to meshnodes.
resolution	normal fine finer custom		The evaluation resolution, if placement is set to meshnodes
rotexpr	String	0	Rotation expression, if pointtype is rectangle or ellipse.
showframe	true false	false	Whether to show a frame around the point text or not, if pointtype is set to text.
sphereradiusscale	positive double	1	The scale factor applied to the point radii.

TABLE 7-119: VALID PROPERTIES FOR POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
smooth	none material internal everywhere expression	material	Smoothing settings, if level is not point. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
textcolor	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	The color to use for the texts when pointtype is set to text.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual and level is not point.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
trailingzeros	true false	false	Include trailing zeros when numberformat is auto, engineering, or scientific.
transformexpr	3-by-3 array	{ "1", "0", "0"; "0", "1", "0"; "0", "0", "1" }	The three axis vectors determining the size and rotation for 3D points with pointtype set to block or ellipsoid. Specify them using the setIndex method.
widthexpr	String	1	Width expression, if pointtype is rectangle or ellipse.

ATTRIBUTES

Deform, Filter (Plot Attribute), Selection (Dataset Attribute), Transformation, Transparency (3D only)

SEE ALSO

[PointData](#)

PointData

Create a point data plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "PointData");  
model.result(<pgtag>).feature(<ftag>).set(property, <value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "PointData")` creates a point data plot feature named `<ftag>` belonging to the 2D or 3D plot group `<pgtag>`.

Point data plots are used to visualize raw point data given as points and colors (see the example below). Point data plots can be added to 2D and 3D plot groups.

The following properties are available:

TABLE 7-120: VALID PROPERTIES FOR POINT DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
colordata	double 1D array		The color data for the point data plot as a real N-vector.
coloring	colortable uniform gradient	colortable	How to color the points: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.

TABLE 7-120: VALID PROPERTIES FOR POINT DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
pointdata	double 2D array		The point data for the point data plot, as x and y-coordinates in 2D and x-, y-, and z-coordinates in 3D in an sdm-by-N real matrix.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
sphereradiusscale	positive double	1	The scale factor applied to the point radii.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.

ATTRIBUTES

[Marker](#), [MaterialAppearance](#), [Transparency](#)

EXAMPLE

A method for creating a circle of points in 2D.

Code for Use with Java

```
String pgTag = model.result().uniqueTag("pg");
ResultFeature pg = model.result().create(pgTag, 2);
ResultFeature plot = pg.create("pt1", "PointData");
int N = 17;
double[][] p = new double[2][N];
double[] color = new double[N];
double R = 1000;
for (int i = 0; i < N; i++) {
    double angle = i * 2 * Math.PI / N;
    p[0][i] = R * Math.cos(angle);
    p[1][i] = R * Math.sin(angle);
    color[i] = p[1][i];
}
plot.set("pointdata", p)
    .set("colordata", color)
    .set("coloring", "colortable");
plot.run();
```

SEE ALSO

[AnnotationData](#), [ArrowData](#), [LineData](#), [Point](#), [SurfaceData](#), [TubeData](#)

PointGraph

Create a point graph plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "PointGraph");
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "PointGraph")` creates a graph point plot feature named `<ftag>` belonging to the 1D plot group `<pgtag>`.

Point plot is used to visualize quantities on points, either cut points or points in a geometry. Point plots can be added to 1D plot groups.

The following properties are available:

TABLE 7-121: VALID PROPERTIES FOR POINT GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autopoint	true false	false	Whether the automatic legends should include the point.

TABLE 7-121: VALID PROPERTIES FOR POINT GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
dbmanualref	double	0	Reference value for display in dB, if dbref is manual (and indb is true).
dbref	maximum manual	maximum	Use the maximum value of a manual value as the dB reference when indb is true.
dbtype	10log 20log	10log	Use a 10 log or 20 log for the dB values when indb is true.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	Model dependent	The expression to plot.

TABLE 7-121: VALID PROPERTIES FOR POINT GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
fouriershow	real imaginary amplitude phase spectrum	amplitude	When xdata is fourier: Show the DFT as real or imaginary part, amplitude, phase, or frequency spectrum.-
freqmax	integer		If fouriershow is spectrum and freqrangeactive is true: The upper frequency bound.
freqmin	integer		If fouriershow is spectrum and freqrangeactive is true: The lower frequency bound.
freqrangeactive	true false	false	If fouriershow is spectrum: Controls whether a manual frequency range is used.
functype	continuous discrete	continuous	Plot the function data as a continuous or a discrete function.
ignorelast	true false	true	If fouriershow is not spectrum: ignore last sample.
indb	true false	false	Display the amplitude or frequency spectrum in dB when fouriershow is amplitude or spectrum.
inverse	true false	false	If fouriershow is not spectrum: inverse the discrete Fourier transform.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.

TABLE 7-121: VALID PROPERTIES FOR POINT GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
localzphys	array	0	The local z-coordinates, when locdef is set to physical.
localzrel	array of -l-l numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when data is set to a Layered Material dataset.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
maskdc	true false	false	If fouriershow is not spectrum: set the DC value (zero frequency component) to zero.
nfreqs	integer	1	If fouriershow is spectrum and nfreqsactive is true: The number of frequencies to plot.

TABLE 7-121: VALID PROPERTIES FOR POINT GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
nfreqsactive	true false	false	If fouriershow is spectrum: Controls whether the number of frequencies is set manually.
nsamples	integer	1	If fouriershow is not spectrum and nsamplesactive is true: The number of uniform samples to use.
nsamplesactive	true false	false	If fouriershow is not spectrum: Controls whether the number of uniform samples is set manually.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if twoyaxes is set to true in the parent plot group.
prefixintitle	String	Empty	Added prefix to contribution to title.
refine	nonnegative integer	1	Refinement of the time resolution for time-dependent studies (an integer 1–1000).
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
scale	none multiplyperiod dividenfreq	none	If fouriershow is spectrum, use no scaling, multiply by sampling period, or divide by number of frequencies.
scaleperiod	true false	true	If fouriershow is not spectrum: scale with sampling period.
shiftzero	true false	true	If fouriershow is not spectrum: shift zero frequency.
solnum	integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active

TABLE 7-121: VALID PROPERTIES FOR POINT GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when titletype is manual
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom
unwrapphase	boolean	false	Unwrap the phase, if the expression to plot has the unit of an angle.
xdata	expr solution fourier phase	solution	x-axis data. expr uses the expression in xdataexpr. solution uses the available solutions in the underlying dataset, for example, time steps. fourier uses a discrete Fourier transform (DFT). phase uses a phase range, and rearranged phase plots.
xdataexpr	String	Model dependent	Expression for x-axis data.
xdatadescr	String	Model dependent	Description of expression in xdataexpr.
xdataphaserange	double array	$(\pi/10) * \text{range}(0, 1, 20)$	The phases for which the expression should be evaluated when xdata is phase.
xdataphaseunit	String	rad	The unit in which xdataphaserange is described.

TABLE 7-121: VALID PROPERTIES FOR POINT GRAPHS.

NAME	VALUE	DEFAULT	DESCRIPTION
xdatasolnumtype	all inner outer valid level	outer	Whether the expression should be evaluated for every inner or every outer solution, or for a specific level (level1, level2, and so on). Applicable only for models containing multiple levels.
xdataunit	String	Model dependent	The unit to use for the expression in xdataexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES

[Color](#), [Comparison](#), [ErrorBars](#), [Filter \(Plot Attribute\)](#), [GraphMarker](#)

SEE ALSO

[LineGraph](#), [Global \(Plot\)](#)

PointTrajectories

Create a point trajectories plot in 2D and 3D.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"PointTrajectories");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"PointTrajectories")` creates a point trajectories plot feature named `<ftag>` belonging to plot group `<pgtag>`.

`model.result(<pgtag>).feature(<ftag>).selection()` returns the selection of points for which to plot trajectories. Selections are only available when you use points as the plot data. See [Selections](#) for more information about the available selection methods.

A solution or cut point dataset is required to plot point trajectories.

The following properties are available:

TABLE 7-122: VALID PROPERTIES FOR POINT TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	head	Whether the head, tail, or center of the arrow is located at the arrow position.
arrowexpr	string array		Active when pointtype is set to arrow. Determines the direction in which the arrow points.
arrowlength	logarithmic normalized proportional	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.

TABLE 7-122: VALID PROPERTIES FOR POINT TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowscale	positive double	1	If arrowscaleactive is true: the length scale factor.
arrowscaleactive	boolean	false	If true, arrowscale is used, otherwise the scale factor is computed automatically.
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.
comettailexpr	String array	inverse of velocity vector of the first available particle tracing interface	Active when pointtype is set to comettail. Determines the direction in which the comet tail points and the relative sizes of the tails of different point trajectories.
data	none parent dataset name	parent	The dataset this feature refers to.
ellipsearrowbase	head tail	head	The base for the arrow to draw for ellipses.
ellipsearrowtype	arrow cone none	arrow	The type of arrow to draw for ellipses.
ellipsearrowscale	double		The ellipse arrow scale factor, if ellipsearrowscaleactive is set to true.
ellipsearrowscaleactive	boolean	false	Set to true to specify an ellipse arrow scale when pointtype is ellipse.
ellipsecount	positive integer	10	Maximum number of ellipses, when ellipseentrymethod is numberofellipses.
ellipseentrymethod	numberofellipses listoftimes	numberofellipses	The ellipse entry type when pointtype is ellipse: number of ellipses or a list of times.
ellipsetimes	String array		Collection of times for polarization ellipses when ellipseentrymethod is listoftimes.
extrasteps	none specifieldtimes proportional	proportional	Controls how the number of extra time steps rendered in the trajectory plot is calculated. These extra time steps typically correspond to the exact times of reflections or velocity reinitializations.
expr	String array		The x-, y-, and z-expressions for the trajectory data (mapped from globalexpr or pointsexpr depending on the plotdata property's setting).
fixedpointsize	boolean	false	Use constant radii for the plotted spheres.
globalexpr	String array		The x-, y-, and z-expressions for global plot data.

TABLE 7-122: VALID PROPERTIES FOR POINT TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
illuminatelines	on off	on	Whether the plotted lines are affected by lighting when <code>linetype</code> is <code>line</code> . Available in 3D only.
inheritarrowsscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the arrow scale is inherited.
inheritcolor	boolean	true	If <code>inheritplot</code> is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the deformation scale is inherited.
inheritellipsescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the ellipse scale factor is inherited.
inheritplot	none plot name	none	The plot that color, color range, and deformation scale are inherited from.
inheritrange	boolean	true	If <code>inheritplot</code> is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the radius scale factor is inherited.
inherittailscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the tail scale factor is inherited.
inherittransparency	boolean	true	If <code>inheritplot</code> is not none: Determines if the transparency settings are inherited.
inherittubescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the tube radius scale factor is inherited.
interpcount	positive integer	100	Number of interpolated time, when interpolation is set to uniform.
interpolation	none uniform	none	The type of interpolation used to plot lines when <code>linetype</code> is <code>line</code> or <code>tube</code> .
linecolor	custom black blue cyan gray green magenta red white yellow	black (2D), red (3D)	The uniform color to use for lines.
linetype	none line ribbon tube	line	Plot particle traces as lines, ribbons, or tubes, or not at all.
logrange	double > 1	100	If <code>arrowlength</code> is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.

TABLE 7-122: VALID PROPERTIES FOR POINT TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
plotdata	global points	points	The plot data used for the point trajectories: points in the geometry (points) or a user-defined global expression (global).
pointautoscale	on off	on	If enabled, the point radii are scaled based on the size of the geometry.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.
pointlineanim	on off	off	When animating particle tracing plots with points, this property indicates whether the points at the current time should appear on top of the lines.
pointradiusexpr	String	0.001	The point radius expression.
pointsexpr	String array		The x-, y-, and z-expressions for points as the plot data.
pointtype	none point comettail arrow ellipse	none	Plot particles as points, with comet tails, with arrows, with polarization ellipses (3D only), or not at all.
radiusexpr	String	1	The tube radius expression
semimajorexpr	String array		Expressions for the polarization ellipses' semimajor axis,
semiminorexpr	String array		Expressions for the polarization ellipses' semiminor axis.
sphereradiusscale	positive double		The scale factor applied to the point radii if sphereradiusscaleactive is on.
sphereradiusscaleactive	on off	off	If on, sphereradiusscale is used; otherwise the scale factor is computed automatically.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
tailscale	positive double	1	Active when pointtype is set to comettail and tailscaleactive is on. Specifies the manual scale factor with which the comet tail expression is multiplied.
tailscaleactive	on off	off	Active when pointtype is set to comettail. Specifies whether manual tail scaling is enabled.
title	String		Active when title is set to manual. Specifies the plot title.
titletype	automatic manual none	automatic	Determines how the plot title is specified.

TABLE 7-122: VALID PROPERTIES FOR POINT TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
tuberadiusscale	String	1	The scale factor applied to the tube radii if tuberadiusscaleactive is on.
tuberadiusscaleactive	on off	off	If on, tuberadiusscale is used, otherwise the scale factor is computed automatically.

ATTRIBUTES

[Color](#), [Deform](#), [Export](#), [Filter](#) (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing), [MaterialAppearance](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[ParticleTrajectories](#), [RayTrajectories](#)

PolarGroup

Create a polar plot group.

SYNTAX

```
model.result().create(<pgtag>, "PolarGroup");
model.result(<pgtag>).set(property, <value>);
model.result(<pgtag>).run();
```

DESCRIPTION

`model.result().create(<pgtag>, "PolarGroup")` creates a polar plot group named `<pgtag>`. A polar plot group displays the containing graph plots in a polar coordinate system.

The following properties are available for polar plot groups:

TABLE 7-123: VALID PROPERTY/VALUE PAIRS FOR POLAR PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
angularunit	degrees radians	degrees	The angular unit: degrees or radians.
config	configuration name		The configuration to use for the solution times or parameters, if <code>solutionparams</code> is set to configuration.
data	none dataset name	First compatible dataset	The dataset this feature refers to. This is the default dataset for all plots in the group.
datasetintitle	on off	off	Whether the title should contain the dataset when <code>titletype</code> is custom
expressionintitle	on off	off	Whether the title contribution of plots in this group should contain the expression when <code>titletype</code> is custom
innerinput	all first last manual manualindices interp	all	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when <code>data</code> is not none and the underlying data is transient.

TABLE 7-123: VALID PROPERTY/VALUE PAIRS FOR POLAR PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
legendpos	upperright middleright lowerright upperleft middleleft lowerleft middleright center middleleft manual	upperright	The position of the legends for all plots in this group. Choose manual to specify the exact position.
legendposx	double	1	The x-position (0-1) for the legend position when legendpos is set to manual.
legendposy	double	1	The y-position (0-1) for the legend position when legendpos is set to manual.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
manualgrid	on off	off	Whether to use the automatic grid spacing or the grid settings specified in rspacing and tspacing.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
phaseintitle	on off	off	Whether the title should contain the phase when titletype is custom.
prefixintitle	String	Empty	Added prefix to group contribution to title.

TABLE 7-123: VALID PROPERTY/VALUE PAIRS FOR POLAR PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere).
refine	nonnegative integer	1	The element refinement to use, if resolution is set to custom.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the resolution of the plots in the plot group. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
rextra	double array	Empty	Extra grid points to include along the r-axis
rmax	double	Computed automatically	The maximum value of the r-axis
rmin	double	Computed automatically	The minimum value of the r-axis
rotmdir	ccw cw	ccw	Rotation direction: counterclockwise or clockwise.
rspaceing	double	Computed automatically	The grid spacing on the r-axis. Used if manualgrid is set to on.
showlegendsmaxmin	on off	off	Show maximum and minimum values for color legends (when a Color Expression has been added).
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	Integer array	All solutions	The solutions to plot. Available when the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range (1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.
suffixintitle	String	Empty	Added suffix to group contribution to title
symmetricangle	on off	off	Display angles as +-180 degrees around zero instead as a full 360 degree revolution.
t	double array	Empty	The times to plot. Available when the underlying solution is transient.

TABLE 7-123: VALID PROPERTY/VALUE PAIRS FOR POLAR PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
textra	double array	Empty	Extra grid points to include along the θ -axis, in degrees
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
title	String	The auto-title.	The title to use when titletype is manual
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
tspacing	double	Computed automatically	The grid spacing on the θ -axis. Used if manualgrid is set to on.
typeintitle	on off	on	Whether the title contribution of plots in this group should contain the type when titletype is custom
unitintitle	on off	on	Whether the title contribution of plots in this group should contain the unit when titletype is custom
window	graphics new windowX, where X is an integer	graphics	The window where the plot group is displayed. When plotting on a graphics server, graphics is equivalent to new.
windowtitle	String	Computed automatically	The title to use for the window where the plot group is displayed. It is not possible to change the title of the graphics window.
windowtitleactive	on off	off	Set windowtitleactive to on to enter a manual window title in windowtitle
zeroangle	right up left down	right	Direction of the zero angle.

SEE ALSO

[PlotGroup1D](#), [PlotGroup2D](#), [PlotGroup3D](#), [SmithGroup](#)

Polarization

Create a polarization plot to visualize the polarization state for diffraction orders, simulated for periodic structures.



The Polarization plot requires a license for the RF Module or Wave Optics Module.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Polarization");
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Polarization")` creates a polarization plot feature named <ftag> belonging to the plot group <pgtag>.

A polarization plot is used to visualize the polarization state using polarization ellipses. Polarization plots can be added to 1D plot groups.

The following properties are available:

TABLE 7-124: VALID PROPERTIES FOR POLARIZATION.

NAME	VALUE	DEFAULT	DESCRIPTION
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
centerpos	auto user	auto	The center position for the polarization ellipses. If set to user, enter the center position's components using the xcenter and ycenter properties.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
display	none manual model dependent	model dependent	What to display as polarization ellipses. If you choose manual, enter the expression's components using the xexpr and yexpr properties.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.

TABLE 7-124: VALID PROPERTIES FOR POLARIZATION.

NAME	VALUE	DEFAULT	DESCRIPTION
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.

TABLE 7-124: VALID PROPERTIES FOR POLARIZATION.

NAME	VALUE	DEFAULT	DESCRIPTION
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
normalization	manual currentorder allorders	currentorder	Normalize the polarization ellipses with respect to the current order, with respect to all orders, or use manual scaling.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
plotscale	double	0.5	The scaling of the polarization ellipses, when normalization is set to manual.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
showrotmdir	true false	true	Whether to show the direction of rotation.
solnum	integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.

TABLE 7-124: VALID PROPERTIES FOR POLARIZATION.

NAME	VALUE	DEFAULT	DESCRIPTION
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
xcenter	String	0	The x-coordinate for the center position, when centerpos is set to user.
xexpr	String	Model dependent	The x-coordinate for the polarization ellipses, when display is set to manual.
ycenter	String	0	The y-coordinate for the center position, when centerpos is set to user.
yexpr	String	Model dependent	The y-coordinate for the polarization ellipses, when display is set to manual.

ATTRIBUTE[Color](#)**SEE ALSO**[LineGraph](#)*PrincipalLine, PrincipalSurface, PrincipalVolume*

Create a principal stress or principal strain plot in volumes, on surfaces, or on lines (edges).

SYNTAX

```

model.result(<pgtag>).create(<ftag>,"PrincipalVolume");
model.result(<pgtag>).create(<ftag>,"PrincipalSurface");
model.result(<pgtag>).create(<ftag>,"PrincipalLine");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();

```

DESCRIPTION

model.result(<pgtag>).create(<ftag>,"PrincipalVolume") create a volume principal stress or principal strain plot named <ftag> belonging to the plot group <pgtag>.

The following properties are available:

TABLE 7-125: VALID PROPERTIES FOR PRINCIPAL STRESS/STRAIN PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
arrowcount	positive integer	200	The number of arrows. Available for PrincipalSurface in 3D.
arrowlength	normalized proportional logarithmic	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowtype	arrow cone	arrow	The type of arrow to draw.
arrowxmethod	number coord	number	Indicates whether the x-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for PrincipalSurface in 2D and PrincipalVolume in 3D.
arrowymethod	number coord	number	Indicates whether the y-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for PrincipalSurface in 2D and PrincipalVolume in 3D.
arrowzmethod	number coord	number	Indicates whether the z-coordinates of the arrows should be specified by number of points to distribute evenly across the data or by coordinates. Available for PrincipalVolume in 3D.
color	custom black blue cyan gray green magenta red white yellow	red	The color to use.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
gporder	positive integer	1	If pattern is gauss: The Gauss-point order.
inheritarrowsscale	boolean	true	If inheritplot is not none: Determines if arrow scale is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that arrow scale and color are inherited from.
interp	double array	Time corresponding to last selected solnum for transient levels	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.

TABLE 7-125: VALID PROPERTIES FOR PRINCIPAL STRESS/STRAIN PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
placement	uniform uniformani gausspoints	uniform	If uniform, then the arrows are distributed uniformly. If uniformanisotropic, then the arrows are distributed uniformly with weights taken from the weight property. If gausspoints, the arrows are drawn in the Gauss points (with order set by gporder). Available for PrincipalSurface in 3D.
prncdirstrainexpr	3-by-3 string matrix	Physics-dependent	If type is strain: The expressions for the principal directions.
prncdirstressexpr	3-by-3 string matrix	Physics-dependent	If type is stress: The expressions for the principal directions.
prncstraindata	manual any available principal strain data	First available principal strain data	The data used for the principal strain plot. If set to manual, specify principal values and directions. The data is referred to using the physics interface and an integer: solid_1, for example, is the first available principal strain data from a Solid Mechanics interface.
prncstressdata	manual any available principal stress data	First available principal stress data	The data used for the principal stress plot. If set to manual, specify principal values and directions. The data is referred to using the physics interface and an integer: solid_1, for example, is the first available principal stress data from a Solid Mechanics interface.
prncvalstrainexpr	1-by-3 string matrix	Physics-dependent	If type is strain: The expressions for the principal values.
prncvalstressexpr	1-by-3 string matrix	Physics-dependent	If type is stress: The expressions for the principal values.
revcoordsys	cylindrical cartesian	cartesian	Coordinate system for principal stress/strain vectors. Only available for plots that use a 1D or 2D revolution dataset that has default axis settings and points to a solution or mesh dataset for an axisymmetric geometry.
scale	positive double	1	If scaleactive is true: The length scale factor.
scaleactive	boolean	false	Whether to use manual scaling.

TABLE 7-125: VALID PROPERTIES FOR PRINCIPAL STRESS/STRAIN PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
t	double	Time corresponding to last selected solnum	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title should be used (the title property). none, if no title should be displayed.
type	stress strain	stress	Selection between principal stress and principal strain plot.
weight	double array	1	If placement is uniform: The weights given to the different axis directions.
xnumber	nonnegative integer	15 (3D), 7 (2D)	Number of points in the x direction. Active when arrowxmethod is set to number. Available for PrincipalSurface in 2D and PrincipalVolume in 3D.
xcoord	double array	Empty	Absolute coordinates in the x direction, active when arrowxmethod is set to coord. Available for PrincipalSurface in 2D and PrincipalVolume in 3D.
ynumber	integer	15 (3D), 7 (2D)	Number of points in the y direction. Active when arrowymethod is set to number. Available for PrincipalSurface in 2D and PrincipalVolume in 3D.
ycoord	double array	Empty	Absolute coordinates in the z direction, active when arrowymethod is set to coord. Available for PrincipalSurface in 2D and PrincipalVolume in 3D.
znumber	integer	15 (3D), 7 (2D)	Number of points in the z direction. Active when arrowzmethod is set to number. Available for PrincipalVolume in 3D.
zcoord	double array	Empty	Absolute coordinates in the z direction, active when arrowzmethod is set to coord. Available for PrincipalVolume in 3D.

ATTRIBUTES

Color, Deform, Filter (Plot Attribute), MaterialAppearance, Selection (Dataset Attribute)
Transformation, Transparency (3D only)

RadiationPattern

Create a radiation pattern plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"RadiationPattern");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"RadiationPattern")` creates a radiation pattern plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Radiation pattern plots are used to visualize radiation patterns in the far-field for antennas, for example. They can be added to all types of plot groups.

The following properties are available:

TABLE 7-126: VALID PROPERTIES FOR RADIATION PATTERN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
anglerestr	manual none	none	Whether there is a restriction on the angle range that is evaluated.
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions. Available in 1D.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions. Available in 1D.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label. Available in 1D.
autosolution	true false	true	Whether the automatic legends should include the solution. Available in 1D.
autounit	true false	false	Whether the automatic legends should include the unit. Available in 1D.
beamwidth	boolean	false	Compute the beamwidth (when set to true). Available in 1D.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> . Available in 1D.
center	double array of length 3	{0, 0, 0}	If <code>circle</code> is manual: The center of the circle to evaluate on. Available in 2D and 3D.
center2	double array of length 2	{0, 0}	If <code>circle</code> is manual and the dataset is 2D: The center of the circle to evaluate on. Available in 1D.
center3	double array of length 3	{0, 0, 0}	If <code>circle</code> is manual and the dataset is 3D: The center of the circle to evaluate on. Available in 1D.
circle	manual unit	unit	The circle to evaluate on. Available in 1D.

TABLE 7-126: VALID PROPERTIES FOR RADIATION PATTERN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colorexpr	String	Model dependent	The color of the far-field surface. Available in 3D.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables. Available in 1D.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom. Available in 1D.
customgridcolor	RGB-triplet	{0,0,1}	f grid is fine, normal, or coarse and gridcolor is custom: The grid's color. Available in 3D.
customlinecolor	RGB-triplet	{0,0,1} or last used edgecolor	The color to use for the lines. Active when linecolor is set to custom. Available in 1D.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom. Available in 1D. Available in 1D.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient. Available in 1D.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String array	Model dependent	The description of the expressions in expr. Is used in the automatic legends.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
directivity	on off	off	Whether to compute the maximum directivity and the direction where it is attained. Available in 2D and 3D when anglerestr is none.
directivityexpr	String array	Model dependent	The expressions to use for the directivity computation.

TABLE 7-126: VALID PROPERTIES FOR RADIATION PATTERN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
grid	fine normal coarse none	none	The type of grid to display on the far-field surface. Available in 3D.
gridcolor	custom black blue cyan gray green magenta red white yellow	black	If grid is fine, normal, or coarse: The grid's color. Available in 3D.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited. Available in 2D and 3D.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends. Available in 1D.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated. Available in 1D.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual. Available in 1D.
legendsuffix	String		A suffix added to the automatic legend.
leveldown	double (nonnegative)	0	The level down from the reference direction to compute the beamwidth for (when beamwidth is true). Available in 1D.

TABLE 7-126: VALID PROPERTIES FOR RADIATION PATTERN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme. Available in ID.
linewidth	double preference	1	The line width. Available in ID.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line. Available in ID.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line. Available in ID.
looplevel	integer row matrix	all on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range (1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set. Available in ID.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set. Available in ID.
normal	double array of length 3	{0, 0, 1} for 3D, {0, 1, 0} for 2D axisymmetry.	If the dataset is 3D or 2D axially symmetric: The normal of the circle to evaluate on. Available in ID.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual. Available in ID.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.

TABLE 7-126: VALID PROPERTIES FOR RADIATION PATTERN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
phidisc	integer ≥ 2	50 (1D); 20 (2D and 3D)	The angular discretization (number of angles) in the phi direction.
phimin	double	0	If anglerestr is manual: The minimum phi angle in degrees.
phirange	double	360	If anglerestr is manual: The phi angle's range in degrees.
phivar	String	Empty	For 2D axisymmetric radiation pattern plots only, evaluate the azimuthal angle once for each angle using the variable name in phivar. If empty, an evaluate-once-and-revolve approach is used instead.
prefixintitle	String	Empty	Added prefix to contribution to title.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable. Available in 1D.
radius	String	1	If circle is manual: The radius of the circle to evaluate on.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. The default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
refdir	double array of length 3	{1, 0, 0} for 3D, {0, 0, 1} for 2D axisymmetry, {1, 0} for 2D	If the dataset is 3D, 2D axially symmetric, or 2D: The reference direction of the circle to evaluate on. Available in 1D.

TABLE 7-126: VALID PROPERTIES FOR RADIATION PATTERN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
sphere	manual unit	unit	The sphere to evaluate on. Available in 2D and 3D.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
thetadisc	integer ≥ 2	10	The angular discretization (number of elevation angles) in the theta direction. Available in 2D and 3D.
thetamin	double	0	If anglerestr is manual: The minimum theta angle in degrees. Available in 2D and 3D.
thetarange	double	360	If anglerestr is manual: The phi angle's range in degrees. Available in 2D and 3D.
threshold	double		Threshold value for the evaluated radius maps to the plotted radius 0 if thresholdactive is on (3D only).
thresholdactive	on off	off	Use a manual threshold value (3D only).
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient. Available in 1D.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-126: VALID PROPERTIES FOR RADIATION PATTERN PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
useradiusascolor	on off	off	Use the expression in expr as the colorexpr. Available in 3D only.

ATTRIBUTES

[Deform](#) (2D and 3D only), [Export](#), [GraphMarker](#) (1D only), [MaterialAppearance](#), [Transformation](#) (2D and 3D only), [Transparency](#) (3D only)

SEE ALSO

[RadiationPattern \(Export\)](#)

RadiationPattern (Export)

Export data from radiation pattern plots to files.

SYNTAX

```
model.result().export().create(<ftag>,"RadiationPattern");
model.result().export().create(<ftag>,<pgtag>,<plottag>,"RadiationPattern");
model.result().export(<ftag>).set(property,<value>);
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>,"RadiationPattern")` creates a radiation pattern export feature with the name `<ftag>`.

`model.result().export().create(<ftag>,<pgtag>,<plottag>,"RadiationPattern")` creates a radiation pattern export feature with the name `<ftag>` for the radiation pattern plot `<plottag>` in the plot group `<pgtag>`.

The following properties are available:

TABLE 7-127: VALID PROPERTIES FOR RADIATION PATTERN EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alwaysask	on off	off	Always ask for filename when saving. This property is ignored when running without a UI.
anglerestr	none manual	none	Use manual angle restriction. If set to manual, you can specify an angle restriction.
center	double 3-by-1 array	{0, 0, 0}	If sphere is set to manual: The center of the sphere to evaluate on. Available in 3D.
centerz	double	0	If sphere is set to manual. It is the z-component in the cylindrical coordinates for an axisymmetric model.
config	configuration name		The configuration to use for the solution times or parameters, if solutionparams is set to configuration.

TABLE 7-127: VALID PROPERTIES FOR RADIATION PATTERN EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The name of the solution dataset to export the radiation pattern for.
descr	String array		Descriptions of the expressions in expr.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array		The expressions to evaluate and export.
filename	String		The output file.
fullprec	on off	on	If on, floating-point numbers are written in full precision, otherwise they are written with six significant digits.
header	on off	off	Enable/disable a data header in the output file.
ifexists	append overwrite	overwrite	If the file exists, append to or overwrite the file contents.
includenan	true false	true	Include data points whose values are NaN in the data export.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.

TABLE 7-127: VALID PROPERTIES FOR RADIATION PATTERN EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range (1,1,20). Applicable when outerinput is manualindices.
phidisc	integer >= 2	50 (1D); 20 (2D and 3D)	The angular discretization (number of angles) in the phi direction.
phimin	double	0	If anglerestr is manual: The minimum phi angle in degrees.
phirange	double	360	If anglerestr is manual: The phi angle's range in degrees.
phivar	String	Empty	For 2D axisymmetric radiation pattern plots only, evaluate the azimuthal angle once for each angle using the variable name in phivar. If empty, an evaluate-once-and-revolve approach is used instead.
radius	String	1	If sphere is manual: The radius of the sphere to evaluate on.
separator	C \t , ; :	C	The column separator for the data in the output text file. C means column; the second option is a space, and the third option is a tab.
solnum	integer array		Solution number to take values from.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range (1,1,20). Applicable when innerinput is manualindices.
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.
sort	on off	off	Enable/disable sorting of the points with respect to the coordinates.
sphere	manual unit	unit	The sphere to evaluate on. Available in 2D and 3D.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
thetadisc	integer >= 2	10	The angular discretization (number of elevation angles) in the theta direction. Available in 2D and 3D.
thetamin	double	0	If anglerestr is manual: The minimum theta angle in degrees. Available in 2D and 3D.
thetarange	double	360	If anglerestr is manual: The phi angle's range in degrees. Available in 2D and 3D.

SEE ALSO

[Plot](#), [RadiationPattern](#)

Ray (1D Plot)

Create a ray plot, which plots ray properties over time or compares ray properties against each other at a set of time steps.

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "Ray1D");  
model.result(<pgtag>).feature(<ftag>).set(property, <value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "Ray1D")` creates a ray plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

With the ray plot, you can plot properties of all rays in a dataset versus time, or plot two ray properties against each other at a set of selected times. When plotting ray properties versus time, it is possible to apply data series operations to the ray data. The ray plot is available in 1D plot groups.

The following properties are available:

TABLE 7-128: VALID PROPERTIES FOR RAY PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
colorcycle	default long	default	The color cycle, when <code>linecolor</code> is set to <code>cycle</code> . With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when <code>linecolor</code> is set to <code>cyclecolortable</code> . See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when <code>bottomcolor</code> is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when <code>linecolor</code> is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when <code>topcolor</code> is set to custom.
cyclelength	auto manual	auto	The length of color cycle when <code>linecolor</code> is <code>cyclegradient</code> .

TABLE 7-128: VALID PROPERTIES FOR RAY PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none parent dataset name	parent	The dataset this feature refers to.
dataseries	none average sum maximum minimum rms stddev variance	sum	The data series operation that is applied to all rays. Effective only when xdata is not expr.
descr	String array	Model dependent	The description of the expressions in expr. Is used in the automatic legends.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expr	String array	Model dependent	The expressions to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.

TABLE 7-128: VALID PROPERTIES FOR RAY PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
prefixintitle	String	Empty	Added prefix to contribution to title.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-128: VALID PROPERTIES FOR RAY PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1, 1, 20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
solrepresentation	<code>solnum</code> <code>solutioninfo</code>	<code>solutioninfo</code>	Indicates which method of selecting solutions is active.
solutionparams	<code>manual</code> <code>parent</code>	<code>manual</code>	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when <code>titletype</code> is <code>manual</code> .
titletype	<code>auto</code> <code>custom</code> <code>label</code> <code>manual</code> <code>none</code>	<code>auto</code>	<code>auto</code> , if the title should be computed automatically. <code>custom</code> , if the title should be computed automatically, but customized. <code>label</code> , if the title should be the plot group's label. <code>manual</code> , if the manual title should be used (the <code>title</code> property). <code>none</code> , if no title should be displayed.
topcolor	<code>custom</code> <code>black</code> <code>blue</code> <code>cyan</code> <code>gray</code> <code>green</code> <code>magenta</code> <code>red</code> <code>white</code> <code>yellow</code>	<code>white</code>	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
typeintitle	<code>on</code> <code>off</code>	<code>on</code>	Whether the title contribution should contain the type when <code>titletype</code> is <code>custom</code> .
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	<code>on</code> <code>off</code>	<code>on</code>	Whether the title contribution should contain the unit when <code>titletype</code> is <code>custom</code> .
xdata	<code>expr</code> <code>solution</code>	<code>solution</code>	x-axis data. <code>expr</code> uses the expression in <code>xdataexpr</code> . <code>solution</code> uses the available solutions in the underlying dataset, such as time steps.
xdataexpr	String	Model dependent	Expression for x-axis data.
xdatadescr	String	Model dependent	Description of expression in <code>xdataexpr</code> .
xdataphaseunit	String	<code>rad</code>	The unit in which <code>xdataphaserange</code> is described.

TABLE 7-128: VALID PROPERTIES FOR RAY PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
xdatasolnumtype	all inner outer valid level	outer	Whether the expression should be evaluated for every inner or every outer solution, or for a specific level (level1, level2, and so on). Applicable only for models containing multiple levels.
xdataunit	String	Model dependent	The unit to use for the expression in xdataexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES

[Color](#) (applies to lines), [Global \(Numerical\)](#)

SEE ALSO

[Ray \(Evaluation\)](#), [RayTrajectories](#), [RayBin](#)

Ray (Dataset)

Create a ray dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Ray");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

model.result().dataset().create(<dtag>,"Ray") creates a ray dataset.

A ray dataset is required to plot ray trajectories. The ray dataset identifies the geometry in which the ray data is stored and the degrees of freedom which determine the position of each ray.

The following properties are available:

TABLE 7-129: VALID PROPERTY/VALUE PAIRS FOR RAY DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
solution	String	first available solution	The solution this dataset refers to.
pgeomspec	manual fromphysics	fromphysics if an applicable physics interface is available, otherwise manual	The method of specifying the ray geometry and the names of the ray position degrees of freedom.
physicsinterface	none or the tag of a valid physics interface	none or first applicable physics interface	The physics interface from which the ray geometry and the names of the ray position degrees of freedom are defined.
rgeom	String	rgeom or pgeom_ followed by the tag of a Geometrical Optics or Ray Acoustics interface	The geometry in which the ray degrees of freedom are defined. The correct name of this ray geometry is pgeom_<name>, where <name> is the name for the Geometrical Optics or Ray Acoustics interface node.
posdof	String array	Position vector components in an applicable physics interface	The position degrees of freedom of the rays.

Ray evaluations to evaluate quantities on ray trajectories.

SYNTAX

```
model.result().numerical().create(<ftag>,"Ray");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getReal();
model.result().numerical(<ftag>).getReal(columnwise);
model.result().numerical(<ftag>).getReal(<outersolnum>);
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);
model.result().numerical(<ftag>).getImag();
model.result().numerical(<ftag>).getImag(allocate,columnwise);
model.result().numerical(<ftag>).getImag(<outersolnum>);
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).computeResult();
model.result().numerical(<ftag>).setResult();
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name <etag>, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"Ray")` creates a ray evaluation feature with the name <ftag>. Ray evaluations can be performed on trajectories accessed through a ray dataset.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that there is one row per point, with one row containing data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is `false`. If `columnwise` is `true`, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is `true`, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` `true` and `columnwise` `false`.

`model.result().numerical(<ftag>).isComplex()` returns true if the result is complex. The resulting value is a scalar, which true if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns true if the result is complex for the given outer solution. <outersolnum> is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and `model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-130: VALID PROPERTY/VALUE PAIRS FOR RAY EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First ray compatible dataset	The ray dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
evaluate	all fraction number	all	What ray to evaluate.
expr	String	Model dependent	The expression to evaluate.
fraction	double	1	If evaluate is fraction: The fraction of rays to evaluate.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
number	double	100	If evaluate is number: The number of particles to evaluate.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.

TABLE 7-130: VALID PROPERTY/VALUE PAIRS FOR RAY EVALUATIONS.

NAME	VALUE	DEFAULT	DESCRIPTION
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

RayBin

Create a ray bin dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"RayBin");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

model.result().dataset().create(<dtag>,"RayBin") creates a ray bin dataset. This dataset evaluates an expression over all rays and then groups them into subintervals, or bins, based on which rays return similar values. If this dataset is then used in any other ray plot or evaluation, each bin produces a single ray having position and other properties equal to the average over rays in the bin.

The following properties are available:

TABLE 7-131: VALID PROPERTY/VALUE PAIRS FOR RAY BIN DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First ray dataset	The ray dataset this feature refers to.
distribution	equalnumber equalwidth	equalnumber	Distribution of ray bins. Active when method is set to number.

TABLE 7-131: VALID PROPERTY/VALUE PAIRS FOR RAY BIN DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
limits	String		String listing the limits of all bins. Active when method is set to limits.
method	limits number tolerance	limits	The method used to define the limits of the ray bins.
number	positive integer	10	Number of ray bins. Active when method is set to number.
tolerance	String		Maximum difference between expression values such that rays are placed in the same bin. Active when method is set to tolerance.

RayTrajectories

Create a ray trajectories plot in 2D and 3D

SYNTAX

```
model.result(<pgtag>).create(<ftag>, "RayTrajectories");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "RayTrajectories")` creates a ray trajectories plot feature named `<ftag>` belonging to plot group `<pgtag>`.

A ray dataset is required to plot ray trajectories. The ray dataset identifies the geometry in which the ray data is stored and the degrees of freedom which determine the position of each ray.

The following properties are available:

TABLE 7-132: VALID PROPERTIES FOR RAY TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	head	Whether the head, tail, or center of the arrow is located at the ray position.
arrowexpr	string array		Active when pointtype is set to arrow. Determines the direction in which the arrow points.
arrowlength	logarithmic normalized proportional	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowscale	positive double	1	If <code>arrowscaleactive</code> is true: the length scale factor.
arrowscaleactive	boolean	false	If true, <code>arrowscale</code> is used, otherwise the scale factor is computed automatically.
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.

TABLE 7-132: VALID PROPERTIES FOR RAY TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
comettailexpr	String array	inverse of velocity vector of the first available ray tracing interface	Active when pointtype is set to comettail. Determines the direction in which the comet tail points and the relative sizes of the tails of different rays.
data	none parent dataset name	parent	The dataset this feature refers to.
ellipsearrowbase	head tail	head	The base for the arrow to draw for ellipses.
ellipsearrowtype	arrow cone none	arrow	The type of arrow to draw for ellipses.
ellipsearrowscale	double		The ellipse arrow scale factor, if ellipsearrowscaleactive is set to true.
ellipsearrowscaleactive	boolean	false	Set to true to specify an ellipse arrow scale when pointtype is ellipse.
ellipsecount	positive integer	10	Maximum number of ellipses, when ellipseentrymethod is numberofellipses.
ellipseentrymethod	numberofellipses listoftimes	numberofellipses	The ellipse entry type when pointtype is ellipse: number of ellipses or a list of times.
ellipsetimes	String array		Collection of times for polarization ellipses when ellipseentrymethod is listoftimes.
extrasteps	none specifiedtimes proportional	proportional	Controls how the number of extra time steps rendered in the trajectory plot is calculated. These extra time steps typically correspond to the exact times of reflections and refractions.
fixedpointsize	boolean	false	Use constant radii for the plotted spheres.
illuminatelines	on off	on	Whether the plotted lines are affected by lighting when linetype is line. Available in 3D only.
inheritarrowscale	boolean	true	If inheritplot is not none: Determines if the arrow scale is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritellipsescale	boolean	true	If inheritplot is not none: Determines if the ellipse scale factor is inherited.

TABLE 7-132: VALID PROPERTIES FOR RAY TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
inheritplot	none plot name	none	The plot that color, color range, and plot scales are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If inheritplot is not none: Determines if the radius scale factor is inherited.
inherittailscale	boolean	true	If inheritplot is not none: Determines if the tail scale factor is inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
inherittubescale	boolean	true	If inheritplot is not none: Determines if the tube radius scale factor is inherited.
interpcount	positive integer	100	Number of interpolated time, when interpolation is set to uniform.
interpolation	none uniform	none	The type of interpolation used to plot lines when linetype is line or tube.
linecolor	custom black blue cyan gray green magenta red white yellow	black (2D), red (3D)	The uniform color to use for lines.
linetype	none line ribbon tube	line	Plot ray traces as lines, ribbons, or tubes, or not at all.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
numextrasteps	nonnegative integer	100	If extrasteps is specifiedtimes, controls the maximum number of extra time steps to render in the plot.
pointautoscale	on off	on	If enabled, the point radii are scaled based on the size of the geometry.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.
pointlineanim	on off	off	When animating ray trajectories plots with points, this property indicates whether the points at the current time should appear on top of the lines.
pointradiusexpr	String	0.001	The point radius expression.

TABLE 7-132: VALID PROPERTIES FOR RAY TRAJECTORIES PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
pointtype	none point comettail arrow ellipse	none	Plot particles as points, with comet tails, with arrows, with polarization ellipses (3D only), or not at all.
propextrasteps	nonnegative integer	1	If extrasteps is proportional, controls the maximum number of extra time steps to render in the plot. The maximum number of extra steps is the product of this proportionality factor with the number of solution times.
radiusexpr	String	1	The tube radius expression.
sphereradiusscale	positive double		The scale factor applied to the point radii if sphereradiusscaleactive is on.
sphereradiusscaleactive	on off	off	If on, sphereradiusscale is used; otherwise the scale factor is computed automatically.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
tailscale	positive double	1	Active when pointtype is set to comettail and tailscaleactive is on. Specifies the manual scale factor with which the comet tail expression is multiplied.
tailscaleactive	on off	off	Active when pointtype is set to comettail. Specifies whether manual tail scaling is enabled.
title	String		Active when title is set to manual. Specifies the plot title.
titletype	automatic manual none	automatic	Determines how the plot title is specified.
tuberadiusscale	String	1	The scale factor applied to the tube radii if tuberadiusscaleactive is on.
tuberadiusscaleactive	on off	off	If on, tuberadiusscale is used; otherwise, the scale factor is computed automatically.

ATTRIBUTES

[Color](#), [Deform](#), [Export](#), [Filter](#) (Grain Positions, Particle Tracing, Point Trajectories, Ray Tracing), [MaterialAppearance](#), [Transformation](#), [Transparency](#) (3D only)

SEE ALSO

[ParticleTrajectories](#), [PointTrajectories](#), [Ray](#) (Dataset), [Ray](#) (1D Plot), [Ray](#) (Evaluation)

SYNTAX

```
model.result().dataset().create(<dtag>,"Receiver2D");
model.result().dataset().create(<dtag>,"Receiver3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Receiver2D")` and `model.result().dataset().create(<dtag>,"Receiver3D")` create 2D and 3D receiver datasets, respectively. Use these datasets to collect the data necessary to visualize the impulse response for a Ray Acoustics simulation (of room acoustics, for example), using an Impulse Response plot. The Impulse Response plot uses the data from a Receiver dataset as input.

The following properties are available:

TABLE 7-133: VALID PROPERTY/VALUE PAIRS FOR RECEIVER2D AND RECEIVER3D DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
center	double array	{0, 0,} (2D); {0, 0, 0} (3D)	Center coordinates.
data	none dataset name	First ray dataset	The ray dataset this feature refers to.
directivity	omnidirectional user	omnidirectional	The directivity type: omnidirectional, or a user-defined expression (in dB),
directivityexpr	double	0	An expression for the directivity. Active when directivity is set to user.
dirvar	String	<dtag>dir	Variable for the directivity.
distance	double	0	Source-receiver distance. Active when radiusinput is set to expr.
distvar	String	<dtag>dist	Variable for the distance traveled.
extrasteps	all specifiedtimes proportional none	all	Specification of the maximum number of extra time steps rendered: all, as a specified number of times, as proportional to the number of solution times, or none.
firstvar	String	<dtag>first	Variable for the first ray arrival time.
interpolation	linear cubic	linear	Interpolation between time steps: linear or cubic.
nrays	integer	0	Number of rays. Active when radiusinput is set to expr.
numextrasteps	nonnegative integer	100	Maximum number of extra time steps. Active when extrasteps is set to specifiedtimes.
propextrasteps	nonnegative integer	1	Proportionality factor for extra time steps. Active when extrasteps is set to proportional.
radius	double	0	Radius. Active when radiusinput is set to user.
radiusinput	expr user	expr	Define the input for the radius using an expression or a user-defined radius.
volume	double	0	Room volume. Active when radiusinput is set to expr.
volvar	String	<dtag>vol	Variable for the volume.

ReflectionGraph, ImpedanceGraph, AdmittanceGraph

Create a reflection graph, impedance graph, or admittance graph in a Smith plot. You can add a Reflection Graph, Impedance Graph, and Admittance Graph to a [SmithGroup](#).

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"ReflectionGraph");
model.result(<pgtag>).create(<ftag>,"ImpedanceGraph");
model.result(<pgtag>).create(<ftag>,"AdmittanceGraph");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

The reflection graph, impedance graph, and admittance graph are all graphs that appear in a Smith plot. The expressions to use can be S-parameters from an RF simulation, for example. For an impedance plot you can also provide a reference impedance; for an admittance plot you can also provide a reference admittance.

The following properties are available:

TABLE 7-134: VALID PROPERTIES FOR REFLECTION GRAPH, IMPEDANCE GRAPH, AND ADMITTANCE GRAPH.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autosolution	true false	true	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgecolor	The color to use for the lines. Active when linecolor is set to custom.

TABLE 7-134: VALID PROPERTIES FOR REFLECTION GRAPH, IMPEDANCE GRAPH, AND ADMITTANCE GRAPH.

NAME	VALUE	DEFAULT	DESCRIPTION
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String array	Model dependent	The description of the expressions in expr. Is used in the automatic legends.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when legendmethod is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when legendmethod is set to evaluated.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.

TABLE 7-134: VALID PROPERTIES FOR REFLECTION GRAPH, IMPEDANCE GRAPH, AND ADMITTANCE GRAPH.

NAME	VALUE	DEFAULT	DESCRIPTION
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	0.5	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.

TABLE 7-134: VALID PROPERTIES FOR REFLECTION GRAPH, IMPEDANCE GRAPH, AND ADMITTANCE GRAPH.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
prefixintitle	String	Empty	Added prefix to contribution to title.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
solnum	nonnegative integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

TABLE 7-134: VALID PROPERTIES FOR REFLECTION GRAPH, IMPEDANCE GRAPH, AND ADMITTANCE GRAPH.

NAME	VALUE	DEFAULT	DESCRIPTION
y0	double	0.02	Reference admittance (Admittance Graph only).
z0	double	50	Reference impedance (Impedance Graph only).

ATTRIBUTES

[Color](#), [Filter](#) (Plot Attribute)

SEE ALSO

[SmithGroup](#)

ResponseSpectrum2D, ResponseSpectrum3D

Create a response spectrum 2D or 3D dataset. A response spectrum analysis is a modal-based method for estimating the structural response to a transient, nondeterministic event. Typical applications are designing against earthquakes and shocks.



The ResponseSpectrum2D and ResponseSpectrum3D datasets require a license for the Structural Mechanics Module.

SYNTAX

```
model.result().dataset().create(<dtag>,"ResponseSpectrum2D");
model.result().dataset().create(<dtag>,"ResponseSpectrum3D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"ResponseSpectrum2D")` creates a 2D response spectrum dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"ResponseSpectrum3D")` creates a 3D response spectrum dataset feature named `<dtag>`.

The input to a response spectrum is the results from an eigenfrequency computation.

The following properties are available for ResponseSpectrum2D and ResponseSpectrum3D:

TABLE 7-135: VALID PROPERTIES FOR RESPONSE SPECTRUM2D AND RESPONSE SPECTRUM3D DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
abscouplingterms	boolean	false	Use absolute value for coupling terms, when modecomb is cqc, grouping, or tenpercent.
abscouplingtermsdoublesum	boolean	false	Use absolute value for coupling terms (NRC RG 1.92 Rev. I), when modecomb is doublesum.
augmentwithrigidresponse	boolean	false	Augment with rigid response, when spatialcomb is cqc3 or srss3 (3D only).
data	none dataset name	First compatible dataset	The eigenvalue dataset this feature refers to.

TABLE 7-135: VALID PROPERTIES FOR RESPONSE SPECTRUM2D AND RESPONSE SPECTRUM3D DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
durationtime	nonnegative scalar	0	The time of duration (in seconds) when modecomb is doublesum.
freqlimitperiodic	nonnegative scalar	0	Frequency limit for purely periodic modes (when rigidmodes is not none).
freqlimitrigid	nonnegative scalar	0	Frequency limit for purely rigid modes (when rigidmodes is gupta).
masscorrectiongupta	missingmass none	none	Mass correction when rigidmodes is gupta: none or missing mass method.
masscorrectionlindleyow	staticzpa missingmass none	none	Mass correction when rigidmodes is lindleyow: none, missing mass method, or static ZPA method.
modecomb	cqc abssum srss doublesum grouping tenpercent	cqc	Mode combination: CQC (Der Kiureghian), absolute value sum, SRSS, double sum (Rosenblueth), grouping method, or ten percent method. Available when spatialcomb is srss or percent.
primaxisrotation	angle	0[deg]	Primary axis rotation (3D only).
primhorizspectrum	none function name	none	Horizontal spectrum function in 2D; primary horizontal spectrum function in 3D.
reldamp	0–1	0.05	Damping ratio when modecomb is cqc or doublesum.
rigidmodes	lindleyow gupta none	none	Rigid modes.
sechorizscale	nonnegative scalar	0.5	Secondary horizontal spectrum scale factor, when spatialcomb is cqc3 or srss3 (3D only).
sechorizspectrum	none function name	none	Secondary horizontal spectrum function (3D only).
spatialcomb	srss percent cqc3 srss3	srss	Spatial combination: SRSS, percent method, CQC3 (3D only), or SRSS3 (3D only).
spectrumfunof	frequency periodtime	frequency	If the spectrum depends on frequency or period time.
spectrumtype	pseudoacc displacement pseudovel	pseudoacc	Spectrum type: pseudoacceleration, displacement, or pseudovelocity spectrum.
vertspectrum	none function name	none	Vertical spectrum function.

TABLE 7-135: VALID PROPERTIES FOR RESPONSE SPECTRUM2D AND RESPONSE SPECTRUM3D DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
wgtsmallresponse	0–1 (0-100%)	40[%]	Weight factor for smaller response (when spatialcomb is percent).
zeroperiodaccfreq	nonnegative scalar	0	Zero period acceleration frequency (when rigidmodes is not none). Available when spatialcomb is srss or percent.

SEE ALSO[Solution](#)*Revolve1D, Revolve2D*

Create a 1D or 2D revolve dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Revolve1D");
model.result().dataset().create(<dtag>,"Revolve2D");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Revolve1D")` creates a 1D revolution dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"Revolve2D")` creates a 2D revolution dataset feature named `<dtag>`.

The revolution dataset needs to supply variables that cannot be expressed in terms of the coordinates of the underlying dataset: The spatial coordinates and the rotation angle. Therefore, the interpolation or evaluation has to be extended with support for supplying the values of these variables in a given set of points.

The source datasets for the revolutions can be any 1D and 2D datasets although these datasets are primarily intended for axisymmetric coordinate systems.

The following properties are available for Revolve1D and Revolve2D datasets:

TABLE 7-136: VALID PROPERTIES FOR REVOLVE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
anglevar	String	Derived from feature name	The name of the angle variable in the revolved coordinate system.
data	none dataset name	First compatible dataset	The dataset this feature refers to.
endcaps	boolean	true	If true, add end caps if the revolution is not full.
modenumber	integer	0	The azimuthal mode number.
point	double	0	The point that is the axis of revolution (1D only).
revangle	double	360	The revolution angle (degrees).
revlayers	integer >= 3	50	The number of revolution layers.

TABLE 7-136: VALID PROPERTIES FOR REVOLVE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
spacevars	String array	Derived from feature name	The names of the space variables in the revolved coordinate system.
startangle	double	0	The angle where the revolved sector begins (degrees).

The following additional properties are only available for Revolve2D datasets:

TABLE 7-137: VALID PROPERTY/VALUE PAIRS FOR 2D REVOLVE DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
method	twopoint pointdir	twopoint	Decides if the line should be specified by two points or through one point and a direction.
genpoints	double matrix	{{0, 0, 0}, {1,0,0}}	Active when method equals twopoint, this property contains the coordinates of the two points in the two rows of the matrix.
layermethod	coarse custom fine normal	normal	The method used for choosing how many layers to use.
pddir	String array	{"0", "1"}	Active when method is set to pointdir, this property contains the direction.
pdpoint	String array	{"0", "0"}	Active when method equals pointdir, this property contains the coordinates of the point.
planemap	xy xz	xz	The 3D plane that the 2D xy plane is mapped to.
revlayers	integer >= 3	50	The number of revolution layers if layermethod is custom.

ATTRIBUTE

[Selection \(Dataset Attribute\)](#)

SEE ALSO

[Solution](#)

ScatterVolume, ScatterSurface

Create a scatter plot on surfaces or in volumes.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"ScatterVolume");
model.result(<pgtag>).create(<ftag>,"ScatterSurface");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Scatter...")` creates a scatter plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Scatter plots are available in 2D (`ScatterSurface`) and 3D (`ScatterVolume`),

The following properties are available:

TABLE 7-138: VALID PROPERTIES FOR SCATTER PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
arrowxmethod	number coord	number	Indicates whether the x-coordinates of the spheres should be specified by number of points to distribute evenly across the data or by coordinates.
arrowymethod	number coord	number	Indicates whether the y-coordinates of the spheres should be specified by number of points to distribute evenly across the data or by coordinates.
arrowzmethod	number coord	number	Indicates whether the z-coordinates of the spheres should be specified by number of points to distribute evenly across the data or by coordinates. Available for ScatterVolume.
color	custom black blue cyan gray green magenta red white yellow	red	The color to use.
colorexpr	String		The expression for the colors of the spheres.
colordescr	String		A description of colorexpr.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expressions in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array of length 2 in 2D and 3 in 3D	Mode-dependent	The components to plot.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.

TABLE 7-138: VALID PROPERTIES FOR SCATTER PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
inheritplot	none plot name	none	The plot that color and sphere scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If inheritplot is not none: Determines if the sphere scale is inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
planecoordsys	cutplane cartesian	cartesian	Coordinate system for scatter plot vectors. Only available for plots that use a cut plane dataset that points to a 3D solution or mesh dataset.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String		The expression for the radii of the spheres.
radiusdescr	String		A description of radiusexpr.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maxim value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.

TABLE 7-138: VALID PROPERTIES FOR SCATTER PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
revcoordsys	cylindrical cartesian	cartesian	Coordinate system for scatter plot vectors. Only available for plots that use a 1D or 2D revolution dataset that has default axis settings and points to a solution or mesh dataset for an axisymmetric geometry.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
sphereradiusscale	double	1	The scale factor applied to the sphere radii if sphereradiusscaleactive is true.
sphereradiusscaleactive	boolean	false	If true, sphereradiusscale is used; otherwise the scale factor is computed automatically.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
xnumber	nonnegative integer	15 (3D), 7 (2D)	Number of points in the x direction. Active when arrowxmethod is set to number.

TABLE 7-138: VALID PROPERTIES FOR SCATTER PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
xcoord	double array	Empty	Absolute coordinates in the x direction, active when arrowxmethod is set to coord.
ynumber	integer	15 (3D), 7 (2D)	Number of points in the y direction. Active when arrowymethod is set to number.
ycoord	double array	Empty	Absolute coordinates in the z direction, active when arrowymethod is set to coord.
znumber	integer	15 (3D), 7 (2D)	Number of points in the z direction. Active when arrowzmethod is set to number. Available for ScatterVolume.
zcoord	double array	Empty	Absolute coordinates in the z direction, active when arrowzmethod is set to coord. Available for ScatterVolume.

ATTRIBUTES

[MaterialAppearance](#), [Transparency](#) (ScatterVolume only)

SEE ALSO

[Surface \(Plot\)](#), [Volume](#)

Sector2D, Sector3D

Create a 2D or 3D sector symmetry dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Sector2D");
model.result().dataset().create(<dtag>,"Sector3D");
model.result().dataset(<dtag>).set(property,<value>);
model.result().dataset(<dtag>).run();
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Sector2D")` creates a 2D sector symmetry dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"Sector3D")` creates a 3D sector symmetry feature named `<dtag>`.

This dataset takes data from another datasets and expands it using sector symmetry around an axis.

The following properties are available for Sector 2D and Sector 3D:

TABLE 7-139: VALID PROPERTIES FOR SECTOR SYMMETRY DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
genpoints	double matrix	{{0, 0, 0}, {1,0,0}}	When method is twopoint: The coordinates of two points on the symmetry axis. Available for Sector3D.

TABLE 7-139: VALID PROPERTIES FOR SECTOR SYMMETRY DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
include	all manual	all	Use all sectors or specify manually which ones to include.
method	twopoint pointdir	twopoint	Decides if the line should be specified by two points or through one point and a direction. Available for Sector3D.
mode	integer	0	If sectors is odd or trans is rot: The azimuthal mode number use to transform the solution's phase between sectors.
pddir	String array	{"0", "1"}	When method is pointdir: The direction in which the symmetry axis points. Available for Sector3D.
pdpoint	String array	{"0", "0"}	When method is pointdir: The coordinates of a point on the symmetry axis. Available for Sector3D.
point	double array of length 2	{0, 0}	The coordinates of the symmetry axis. Available for Sector2D.
reflaxis	String array of length 2 in 2D and length 3 in 3D	{"1", "0"} in 2D, {"1", "0", "0"} in 3D	If trans is rotrfl: The direction in which the reflection axis/plane (for 2D and 3D respectively) points.
sectors	String	2	The number of sectors as an integer ≥ 2 or variable.
sectorsinclude	String	sectors	The number of sectors to include when include is set to manual. It can contain a variable for the number of sectors.
sectorvar	String	Created from feature's tag	Name of variable that evaluates to the sector number, which starts at 0 for the first sector.
spacevars	String array of length 2 in 2D and length 3 in 3D	Created from feature's tag	Names of variables that evaluate to the space coordinates in the coordinate system after sector symmetry expansion.
startsector	String	0	Starting sector for manual selection of sectors to include when include is set to manual. It can contain a variable for the starting sector.
trans	rot rotrfl	rot	If sectors is even or a global expression: Controls whether the transformation performed between consecutive sectors is rotation or rotation and reflection.

ATTRIBUTE[Selection \(Dataset Attribute\)](#)*Selection (Dataset Attribute)*

Add a selection to a dataset.

SYNTAX

```
model.result(<pgtag>).dataset(<dtag>).selection().geom(...);
model.result(<pgtag>).dataset(<dtag>).selection().set(...);
```

DESCRIPTION

Use selections to limit data in a dataset to a certain selection of geometric entities such as domains or boundaries.

	See model.selection() for more information about selection.
---	---

Selection attributes can be added to solution, mesh, cut line, cut plane, shell, layered material, average, minimum, maximum, and integral dataset.

Selection (Plot Attribute)

Add a selection attribute to a plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Selection");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).selection().set(...);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"Selection")` creates a selection attribute named `<atag>` belonging to the plot feature `<ftag>`.

Use selections to limit plots to a certain selection of geometric entities such as domains or boundaries.

	See model.selection() for more information about selection.
--	---

Selection attributes can be added to arrow plots, contour plots, coordinate system plots, isosurface plots, line plots, max/min plots, mesh plots, multislice plots, principal stress plots, slice plots, streamline and streamline multislice plots, surface plots, and volume plots.

The following properties are available with Selection attributes for applicable 3D plots using a Revolve2D dataset:

TABLE 7-140: VALID PROPERTY/VALUE PAIRS FOR SELECTION ATTRIBUTES TO PLOTS USING A REVOLVE2D DATASET.

NAME	VALUE	DEFAULT	DESCRIPTION
evalmantle	true false	true	Evaluate and plot on the revolved geometry's mantle.
evalstartcap	true false	true	Evaluate and plot on the revolved geometry's start cap (at the start angle for the revolution).
evalendcap	true false	true	Evaluate and plot on the revolved geometry's end cap (at the ending angle for the revolution).

The following properties are available for plots that evaluate on boundaries and where the selection is done domains.

TABLE 7-141: VALID PROPERTY/VALUE PAIRS FOR SELECTION ATTRIBUTES EVALUATING ON BOUNDARIES.

NAME	VALUE	DEFAULT	DESCRIPTION
evaluatein	domains adjacent	adjacent	Whether to map the evaluation to the selected domains or on the adjacent boundaries.
exterior	on off	on	Controls if boundaries external to the selection are included in the plot, when evaluatein is domains.
interior	on off	off	Controls if boundaries internal to the selection are included in the plot, when evaluatein is domains.

Shell

Create a shell dataset.



The Shell dataset requires a license for the Structural Mechanics Module.

SYNTAX

```
model.result().dataset().create(<dtag>,"Shell");  
model.result().dataset(<dtag>).set(property,<value>);  
model.result().dataset(<dtag>).run();
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Shell")` creates a shell dataset. A shell dataset makes it possible to visualize the top and bottom surfaces of a shell in 3D. The following properties are available:

TABLE 7-142: VALID PROPERTY/VALUE PAIRS FOR SHELL DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to. Only 3D and 2D axisymmetric datasets with one or more surfaces can be used.
areascalefactor	scalar	1	The area scale factor.
bottomconst	4-by-1 real array		Array of bottom shell surface parameters: Local z-coordinate for thickness-dependent results and the x-, y-, and z-coordinates of the reference point for moment computations.
displacementexpr	3-by-1 string array	All 0s	String with an expression for the displacement normals (apply as an additive correction to the shell's orientation when evaluating in the spatial frame).
distanceexpr	String		String with an expression for the displacement magnitude.
linescalefactor	scalar	1	The line scale factor.
orientationexpr	3-by-1 string array		String array of expressions for the components of the displacement direction vector.
resolution	Integer 1–10	1	Refinement level (1–10).
seplevels	true false	false	Separate the levels to create slices in the through-thickness direction.
topconst	4-by-1 real array		Array of top shell surface parameters: Local z-coordinate for thickness-dependent results and the x-, y-, and z-coordinates of the reference point for moment computations.

SEE ALSO

[Solution](#)

Slice

Create a slice plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Slice");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Slice")` creates a slice feature named `<ftag>` belonging to the plot group `<pgtag>`.



Only one direction of slices is possible in one plot. Use different slice plots together to accomplish several slices in different directions.

The following properties are available:

TABLE 7-143: VALID PROPERTIES FOR SLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the slices: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.

TABLE 7-143: VALID PROPERTIES FOR SLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. It is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
gendistance	double array	Empty	Vector of distances from base plane, indicating position of additional parallel planes. Active when genparaactive equals on and genpara equals number.
genmethod	threepoint pointnormal	threepoint	Active when planetype equals general, this property indicates whether the plane should be specified by three points or through one point and a normal.
gennumber	positive integer	4	Number of additional parallel planes. Active when genparaactive equals on and genpara equals number.
genpara	gennumber gendistances	number	Active when planetype equals general, and genparaactive equals on, this property indicates whether the additional parallel planes should be specify by a number or by a vector of relative distances.

TABLE 7-143: VALID PROPERTIES FOR SLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
genparaactive	on off	off	Active when planetype equals general, this property indicates whether there should be additional parallel planes created.
genpnpoint	String array of length three	{"0", "0", "0"}	Active when genmethod equals pointnormal, this property contains the coordinates of the point.
genpnvec	String array of length three	{"0", "0", "0"}	Active when genmethod equals pointnormal, this property contains the normal vector.
genpoints	double matrix of size 3x3	{{0,0,0}, {1,0,0}, {0,1,0}}	Active when genmethod equals threepoint, this property contains the coordinates of the three points, with one row per point.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interactive	on off	off	If true, the isosurfaces can be moved interactively after the plot has been made.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendtitle has been set to true in the parent plot group.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-143: VALID PROPERTIES FOR SLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
planetype	quick general	yz	Specify plane type.
prefixintitle	String	Empty	Added prefix to contribution to title.
quickplane	xy yz zx	yz	Specify quick plane type. Active when planetype is quick.
quickx	String array	0	Absolute x-coordinates for planes, active when quickxmethod is set to coord.
quickxmethod	number coord	number	Active when planetype equals quick and quickplane equals yz, this property indicates whether the planes should be specified by one or more absolute coordinates along the x-axis, or a number indicating the number of planes to distribute evenly.
quickxnumber	String	5	The number of planes when quickxmethod is set to number. It can contain a global expression for the number of slices.
quicky	String array	0	Absolute y-coordinates for planes, active when quickymethod is set to coord.
quickymethod	number coord	number	Active when planetype equals quick and quickplane equals zx, this property indicates whether the planes should be specified by one or more absolute coordinates along the y-axis, or a number indicating the number of planes to distribute evenly.
quickynumber	String	5	The number of planes when quickymethod is set to number. It can contain a global expression for the number of slices.
quickz	String array	0	Absolute z-coordinates for planes, active when quickzmethod is set to coord.

TABLE 7-143: VALID PROPERTIES FOR SLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
quickzmethod	number coord	number	Active when planetype equals quick and quickplane equals xy, this property indicates whether the planes should be specified by one or more absolute coordinates along the z-axis, or a number indicating the number of planes to distribute evenly.
quickznumber	String	5	The number of planes when quickzmethod is set to number. It can contain a global expression for the number of slices.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.

TABLE 7-143: VALID PROPERTIES FOR SLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
shift	double	0	If interactive is on: The shift that is applied to the level values.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps: off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.

TABLE 7-143: VALID PROPERTIES FOR SLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

Deform, Filter (Plot Attribute), Marker, MaterialAppearance, Selection (Dataset Attribute), Transformation, Transparency

SEE ALSO

Volume

SmithGroup

Create a Smith plot group.

	The Smith Plot Group requires a license for the AC/DC Module, RF Module, MEMS Module, or Plasma Module.
---	---

SYNTAX

```
model.result().create(<pgtag>,"SmithGroup");
model.result(<pgtag>).set(property,<value>);
model.result(<pgtag>).run();
```

DESCRIPTION

model.result().create(<pgtag>,"SmithGroup") creates a Smith plot group named <pgtag>. A Smith plot group displays the containing graph plots as Smith plots.

The following properties are available for Smith plot groups:

TABLE 7-144: VALID PROPERTIES FOR SMITH PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
admittance	on off	off	Add grid for an admittance Smith plot.
config	configuration name		The configuration to use for the solution times or parameters, if solutionparams is set to configuration.
data	none dataset name	First compatible dataset	The dataset this feature refers to. This is the default dataset for all plots in the group.
datasetintitle	on off	off	Whether the title should contain the dataset when titletype is custom
expressionintitle	on off	off	Whether the title contribution of plots in this group should contain the expression when titletype is custom
impedance	on off	on	Add grid for an impedance Smith plot.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double row matrix	Empty on all levels.	The times to use, for transient levels. Available when data is not none and the underlying data is transient.
legendpos	upperright middleright lowerright upperleft middleleft lowerleft middleright center middleleft manual	upperright	The position of the legends for all plots in this group. Choose manual to specify the exact position.
legendposx	double	1	The x-position (0-1) for the legend position when legendpos is set to manual.
legendposy	double	1	The y-position (0-1) for the legend position when legendpos is set to manual.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range (1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.

TABLE 7-144: VALID PROPERTIES FOR SMITH PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when outerinput is manualindices.
phaseintitle	on off	off	Whether the title should contain the phase when titletype is custom.
prefixintitle	String	Empty	Added prefix to group contribution to title.
resolution	fine normal coarse	normal	Grid resolution for Smith plot coordinate grid.
showlegendsmaxmin	on off	off	Show maximum and minimum values for color legends (when a Color Expression has been added).
solnum	Integer array	All solutions	The solutions to plot. Available when the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.
suffixintitle	String	Empty	Added suffix to group contribution to title
t	double array	Empty	The times to plot. Available when the underlying solution is transient.
title	String	The auto-title.	The title to use when titletype is manual
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution of plots in this group should contain the type when titletype is custom

TABLE 7-144: VALID PROPERTIES FOR SMITH PLOT GROUPS.

NAME	VALUE	DEFAULT	DESCRIPTION
unitintitle	on off	on	Whether the title contribution of plots in this group should contain the unit when titletype is custom
window	graphics new windowX, where X is an integer	graphics	The window where the plot group is displayed. When plotting on a graphics server, graphics is equivalent to new.
windowtitle	String	Computed automatically	The title to use for the window where the plot group is displayed. It is not possible to change the title of the graphics window.
windowtitleactive	on off	off	Set windowtitleactive to on to enter a manual window title in windowtitle

SEE ALSO

[PlotGroup1D](#), [PlotGroup2D](#), [PlotGroup3D](#), [PolarGroup](#)

SobolIndex

Create a Sobol index plot to visualize Sobol indices for a sensitivity analysis in an uncertainty quantification study. This plot type can be added to 1D plot groups.



The Sobol index plot requires a license for the Uncertainty Quantification Module.

SYNTAX

```
model.result().create(<pgtag>,"SobolIndex");
model.result(<pgtag>).set(property,<value>);
model.result(<pgtag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"SobolIndex")` creates a Sobol index feature named `<ftag>` belonging to the 1D plot group `<pgtag>`. The plot takes a quantity of interest from a table created by an uncertainty quantification study.

The following properties are available:

TABLE 7-145: VALID PROPERTIES FOR SOBOL INDEX PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autoheaders	true false	true	Whether the automatic legends should include the column headers.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.

TABLE 7-145: VALID PROPERTIES FOR SOBOLE INDEX PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
legend	on off	off	Whether to show legends.
legendmethod	automatic manual	automatic	Whether to use the automatic legends or the legends supplied in the legends property.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
quantityofinterest	positive integer	1	The column in the table that contains the quantity of interest.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
sort	boolean	false	Sort parameters in decreasing order with respect to the total Sobol index for the quantity of interest.
table	none table feature name	First available table	The table feature this plot refers to.
title	String	The auto-title	The title to use when titletype is manual.

TABLE 7-145: VALID PROPERTIES FOR SOBOL INDEX PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.

SEE ALSO

[Table](#)

Solution

Create a solution dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Solution");
model.result().dataset(<dtag>).set(property,<value>);
model.result().dataset(<dtag>).selection(...);
model.result().dataset(<dtag>).createDeformedConfig(<gtag>,<mtag>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Solution")` creates a solution dataset.

Solution datasets refer to a solution in a solver sequence. All result features that perform evaluations on a solution refer to these datasets rather than directly to the solution itself. The solution datasets specify which geometry and frame to use for evaluation and whether to evaluate only on a specific selection, among other things. It is possible to have multiple dataset solutions referring to the same solution.

The following properties are available:

TABLE 7-146: VALID PROPERTY/VALUE PAIRS FOR SOLUTION DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
comp	String (component name, or None)	First available component, if any.	The component this dataset refers to.
frametype	mesh material spatial geometry	material or spatial	The frame type that this dataset uses for evaluation of spatial coordinates.
geom	String	First available geometry, if any.	The geometry this dataset refers to. Only applicable for components with more than one geometry.
phase	double	0	Evaluate solution at this angle, given in degrees.
scalefactor	double	1	Multiply solution by this scaling factor.
solution	String	First compatible solution	The solution this dataset refers to.

`model.result().dataset(<dtag>).createDeformedConfig(<gtag>,<mtag>)` creates a new geometry sequence (deformed configuration) tagged <gtag> together with a meshing sequence tagged <mtag> to use for remeshing of a deformed mesh. The created geometry sequence only contains a FromMesh feature.

EXAMPLE

Create a solution dataset and set it to point to the second component in the solver sequence sol1:

Code for Use with Java

```
DatasetFeature ds = model.result().dataset().create("dset1", "Solution");
ds.set("solution", "sol1");
ds.set("comp", "comp2");
```

Code for Use with MATLAB

```
ds = model.result.dataset.create('dset1', 'Solution');
ds.set('solution', 'sol1');
ds.set('comp', 'comp2');
```

ATTRIBUTE

[Selection \(Dataset Attribute\)](#)

SEE ALSO

[PlotGroup1D](#), [PlotGroup2D](#), [PlotGroup3D](#)

SolutionArray

Create an array of plots for a selection of multiple times, eigenvalues, or parameter values in 2D and 3D.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>, "SolutionArray");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property, <value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "SolutionArray")` adds a solution array named `<atag>` as an attribute to a 3D plot named `<ftag>` or to a 2D plot named `<ftag>`.

The following properties are available:

TABLE 7-147: VALID PROPERTY/VALUE PAIRS FOR SOLUTIONARRAY.

NAME	VALUE	DEFAULT	DESCRIPTION
arrayshape	linear square	linear	The array shape when <code>plotarraytype</code> in the parent plot group is square.
arrayaxis	x y	x	The array axis when <code>arrayshape</code> is linear.
data	none solution dataset name	The first solution dataset	The dataset this feature refers to.
innerinput	all first last manual manualindices	all	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
interp	double array	Time corresponding to last selected <code>solnum</code> for transient levels.	The time to use, for transient levels. Available when <code>data</code> is not parent and the underlying data is transient.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.

TABLE 7-147: VALID PROPERTY/VALUE PAIRS FOR SOLUTIONARRAY.

NAME	VALUE	DEFAULT	DESCRIPTION
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>looplevelinput</code> is <code>manualindices</code> on a level.
looplevelinput	String array with <code>all</code> <code>first</code> <code>last</code> <code>manual</code> <code>manualindices</code> <code>interp</code> on each level	<code>all</code> on all levels	How to input the solution to use, per level. <code>manual</code> on a level indicates that <code>looplevel</code> is used on that level. <code>manualindices</code> on a level indicates that <code>looplevelindices</code> is used on that level. <code>interp</code> on a level indicates that <code>interp</code> is used on that level.
order	<code>rowmajor</code> <code>columnmajor</code>	<code>rowmajor</code>	The ordering of a square plot array: row-major or column-major.
outerinput	<code>all</code> <code>first</code> <code>last</code> <code>manual</code> <code>manualindices</code>	<code>all</code>	How to input the outer solutions to use. Applicable only for parametric sweep models. <code>manual</code> indicates that <code>outersolnum</code> is used. <code>manualindices</code> indicates that <code>outersolnumindices</code> is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>outerinput</code> is <code>manualindices</code> .
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, <code>range(1,1,20)</code> . Applicable when <code>innerinput</code> is <code>manualindices</code> .
t	double array	Empty	The times to use. Available when the underlying solution is transient.

SEE ALSO

[PlotGroup1D](#), [PlotGroup2D](#), [PlotGroup3D](#)

Spatial FFT

Create a spatial FFT dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"SpatialFFT");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"SpatialFFT")` creates a spatial FFT dataset.

Spatial FFT datasets compute the spatial Fourier transform of a variable or expression.

The following properties are available:

TABLE 7-148: VALID PROPERTY/VALUE PAIRS FOR SPATIAL FFT DATASETS.

NAME	VALUE	DEFAULT	DESCRIPTION
backward	true false	true	Use a negative frequency (changed sign).
data	none dataset name	First compatible dataset	The dataset this feature refers to.
domain	frequency index	frequency	Axes values corresponding to the spatial frequencies or as the 0-based DFT indices.
gridres	auto manual	automatic	Create the spatial grid resolution automatically or by entering the resolution manually.
fxvar	String	sfft1fx (for an sfft1 dataset)	Fourier space x component variable name.
fyvar	String	sfft1fy (for an sfft1 dataset)	Fourier space y component variable name.
fzvar	String	sfft1fz (for an sfft1 dataset)	Fourier space z component variable name.
layout	normal periodic padding	automatic	Use a normal layout, a periodic layout, or use zero padding.
maskdc	true false	true	Mask the DC part.
padx	integer	10	The zero padding in the x direction if layout is set to padding.
pady	integer	10	The zero padding in the y direction if layout is set to padding.
padz	integer	10	The zero padding in the z direction if layout is set to padding.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the resolution. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
sampresx	integer	10	The sampling resolution in the x direction if gridres is set to manual.
sampresy	integer	10	The sampling resolution in the y direction if gridres is set to manual.
sampresz	integer	10	The sampling resolution in the z direction if gridres is set to manual.

Spot Diagram

Create a spot diagram plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"SpotDiagram");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
model.result(<pgtag>).feature(<ftag>).runCommand("createFocalPlaneDataset");
model.result(<pgtag>).feature(<ftag>).runCommand("recomputeFocalPlaneDataset");
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "SpotDiagram")` creates a Spot Diagram plot named <ftag> belonging to the plot group <pgtag>.

A Spot Diagram can be added to a 2D Plot Group but the associated dataset must either be a Ray dataset solved for in a 3D model component, or an Intersection Point 3D dataset.

The following properties are available:

TABLE 7-149: VALID PROPERTIES FOR PHASE PORTRAIT.

NAME	VALUE	DEFAULT	DESCRIPTION
additionallogical-expression	String	1	Additional logical expression that must be nonzero for rays to be included in the plot.
additionallogical-expressionactive	on off	off	If on, activates an additional logical expression for including rays in the plot.
arrangement	release wavelength single	release	Controls how the rays are sorted before plotting, possibly by grouping them into separate spots.
circleactive	on off	off	If on, draw a circle at the origin of the spot diagram.
circleradiusexpr	String	1[um]	Radius of the circle to be plotted at the origin, if circleactive is on.
columns	positive integer	3	Maximum number of vertical columns of spots. Used if layout is rectangular.
data	none parent dataset name	parent	The dataset this feature refers to.
distribution	equalnumber equalwidth	equal-number	Used when sorting rays by wavelength into a specified number of intervals. Determines how the interval endpoints are calculated.
filterrelease	positive integer	1	Index of the release feature of rays to plot if filterreleaseactive is on.
filterreleaseactive	on off	off	If on, filter rays by release feature to determine which rays appear in the plot.
filterwavelength	String	632.8[nm]	Wavelength of rays to plot if filterwavelengthactive is on.
filterwavelengthactive	on off	off	If on, filter rays by wavelength to determine which rays appear in the plot.
filterwavelengthtol	String	1[nm]	Numerical tolerance for filtering rays by wavelength if filterwavelengthactive is on.
fitannotationstospot	on off	off	If on, move text closer to some spots in the array when the spots have unequal sizes.

TABLE 7-149: VALID PROPERTIES FOR PHASE PORTRAIT.

NAME	VALUE	DEFAULT	DESCRIPTION
layout	automatic rectangular	automatic	Method for positioning multiple spots in the Graphics window, if arrangement is release or wavelength.
limits	String		Limits of intervals for sorting spots by wavelength.
origin	rays area	rays	Determines how the center of each spot is determined when positioning it relative to other spots, if arrangement is release or wavelength.
method	limits number tolerance	tolerance	Method for sorting rays by wavelength. Used if arrangement is wavelength.
normal	directionbased positionbased userdefined	direction-based	Determines how the normal direction to the plane is calculated.
normalexpr	String array of length 3	{0,0,1}	Expression for the normal direction to the surface, active if normal is userdefined.
number	positive integer	10	Number of wavelength intervals. Used when sorting rays by wavelength if method is number.
numberofreflections	nonnegative integer	0	Rays that have reflected the specified number of times will be included in the plot. Used if numberofreflectionsactive is on.
numberofreflectionsactive	on off	off	If on, activates the filtering of rays based on number of reflections.
paddinghoriz	double	0.5	Padding factor for horizontal spacing between spots, if arrangement is release or wavelength.
paddingvert	double	0.5	Padding factor for vertical spacing between plots, if arrangement is release or wavelength.
pointcolor	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use for points.
posexpr	String array of length 2	{0,0}	Expressions for the spot diagram origin location in local coordinates.
showframe	on off	off	If on, draw a box around each text annotation in the Graphics window. Applies to every type of annotation.
spotcoordsactive	on off	off	If on, show a text annotation for the coordinates of the center of each spot.

TABLE 7-149: VALID PROPERTIES FOR PHASE PORTRAIT.

NAME	VALUE	DEFAULT	DESCRIPTION
spotcoordsposition	above below	above	Position of the annotations for spot coordinates, if spotcoordsactive is on.
spotcoordsprecision	positive integer	3	Numerical precision of the annotations for spot coordinates, if spotcoordsactive is on.
spotcoordssystem	local global	local	Coordinate system to use when displaying annotations for spot coordinates, if spotcoordsactive is on.
spotcoordsunit	length unit	mm	Length unit used in the annotations for spot coordinates, if spotcoordsactive is on.
spotsizeactive	on off	on	If on, show a text annotation for the RMS spot size next to every spot.
spotsizeposition	above below	above	Position of the annotations for RMS spot size, if spotsizeactive is on.
spotsizeprecision	positive integer	3	Numerical precision of the annotations for RMS spot size, if spotsizeactive is on.
spotsizeunit	length unit	um	Length unit of the annotations for RMS spot size, if spotsizeactive is on.
tolerance	String	1 [nm]	Tolerance for sorting rays by wavelength.
transverse	automatic user defined	automatic	Determines how the transverse direction in the plane is calculated.
transverseexpr	String array of length 3	{1,0,0}	Transverse direction in the surface if transverse is userdefined.
wavelengthactive	on off	off	If on, show a text annotation for the vacuum wavelength next to every spot.
wavelengthposition	above below	below	Position of the annotations for wavelength, if wavelengthactive is on.
wavelengthprecision	positive integer	4	Numerical precision of the annotations for wavelength, if wavelengthactive is on.
wavelengthunit	length unit	nm	Length unit of the annotations for wavelength, if wavelengthactive is on.

ATTRIBUTES[Color](#)*StdDevVolume, StdDevSurface, StdDevLine*

Add a derived-values feature for evaluating the standard deviation of an expression over a volume, surface, or line.

SYNTAX

```
model.result().numerical().create(<ftag>,"StdDevVolume");  
model.result().numerical().create(<ftag>,"StdDevSurface");  
model.result().numerical().create(<ftag>,"StdDevLine");  
model.result().numerical(<ftag>).selection(...);  
model.result().numerical(<ftag>).set(property,<value>);  
model.result().numerical(<ftag>).getReal();  
model.result().numerical(<ftag>).getReal(columnwise);  
model.result().numerical(<ftag>).getReal(<outersolnum>);  
model.result().numerical(<ftag>).getReal(columnwise,<outersolnum>);  
model.result().numerical(<ftag>).getImag();  
model.result().numerical(<ftag>).getImag(allocate,columnwise);  
model.result().numerical(<ftag>).getImag(<outersolnum>);  
model.result().numerical(<ftag>).getImag(allocate,columnwise,<outersolnum>);  
model.result().numerical(<ftag>).isComplex();  
model.result().numerical(<ftag>).isComplex(<outersolnum>);  
model.result().numerical(<ftag>).computeResult();  
model.result().numerical(<ftag>).setResult();  
model.result().numerical(<ftag>).appendResult();
```

When added to an evaluation group of name *<etag>*, replace `numerical(<ftag>)` with `evaluationGroup(<etag>).feature(<ftag>)`.

DESCRIPTION

`model.result().numerical().create(<ftag>,"StdDevVolume")` creates a volume standard-deviation feature with the name *<ftag>*.

`model.result().numerical().create(<ftag>,"StdDevSurface")` creates a surface standard-deviation feature with the name *<ftag>*.

`model.result().numerical().create(<ftag>,"StdDevLine")` creates a line standard-deviation feature with the name *<ftag>*.

`model.result().numerical(<ftag>).getReal()` returns the real result, recomputing the feature if necessary. Data is ordered such that one row contains data for all solution numbers. This is identical to `(columnwise)` when `columnwise` is `false`. If `columnwise` is `true`, the ordering is the opposite: each *column* contains the values for all solution numbers.

`model.result().numerical(<ftag>).getImag(allocate,columnwise)` returns the imaginary part of the complex result, recomputing the feature if necessary. If `allocate` is `true`, a zero-valued matrix is allocated even when the result is real. `getImag()` uses `allocate` `true` and `columnwise` `false`.

`model.result().numerical(<ftag>).isComplex()` returns `true` if the result is complex. The resulting value is a scalar, which `true` if any of the expressions in an array is complex valued.

`model.result().numerical(<ftag>).isComplex(<outersolnum>)` returns `true` if the result is complex for the given outer solution. *<outersolnum>* is applicable only for parametric sweep solutions and is the index of each outer parametric solution.

`model.result().numerical(<ftag>).computeResult()` returns the matrix of data that the `setResult` method adds to a table. The matrix includes data only, not the parameter columns, and it does not use any table-specific settings.

`model.result().numerical(<ftag>).setResult()` and

`model.result().numerical(<ftag>).appendResult()` evaluates the feature and set or append the result in the table indicated by the `table` property.

The following properties are available:

TABLE 7-150: VALID PROPERTIES FOR STANDARD DEVIATION FEATURES.

NAME	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	none	The dataset this feature refers to. parent is only applicable if used in an evaluation group.
dataseries	average integral maximum minimum none rms stddev variance	none	The transformation that is applied to the data series formed by the evaluation.
dataseriescumulative	true false	false	Make the table values as a cumulative integration, when dataseries is set to integral.
dataseriesmethod	auto integration summation	auto	The method to use for the data series: automatic, integration, or summation.
descr	String array	Model dependent	The descriptions of the expressions in expr. for use in the automatic title
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String array	Model dependent	The expressions to evaluate
includeparam	true false	false	Include the parameter value or time if dataseries is set to maximum or minimum for a parametric or time-dependent solution,
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
intorder	positive integer	4	The integration order
intorderactive	on off	off	Whether to use manually specified integration order

TABLE 7-150: VALID PROPERTIES FOR STANDARD DEVIATION FEATURES.

NAME	VALUE	DEFAULT	DESCRIPTION
intsurface	on off	off	Compute surface integral. Available for line integration features of axisymmetric models.
intvolume	on off	off	Compute volume integral. Available for surface integration features of axisymmetric models.
localzphys	array	0	The local z-coordinates, when locdef is set to physical.
localzrel	array of -1-1 numbers	0	The local relative z-coordinates, when locdef is set to relative.
locdef	reference physical relative	reference	The location definition, if locinput is set to manual: a reference surface, physical z-coordinate, or a relative z-coordinate. Only available when using a Layered Material or Shell dataset. Not available for StdDevVolume.
locinput	fromdataset manual	fromdataset	The location input: from the dataset or manual. Only available when using a Layered Material or Shell dataset. Not available for StdDevVolume.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
maximumobj	abs real	real	The value being maximized if dataseries is set to maximum.
minimumobj	abs real	real	The value being minimized if dataseries is set to minimum.
normalization	first last maximum none	none	The normalization of the columns that is applied to the data series formed by the evaluation
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.

TABLE 7-150: VALID PROPERTIES FOR STANDARD DEVIATION FEATURES.

NAME	VALUE	DEFAULT	DESCRIPTION
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
solnum	nonnegative integer array	All solutions	The solutions to use.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual configuration parent	manual	If data is a local dataset with nontrivial data and the feature is part of an evaluation group, specify solutions manually or take them from the parent evaluation group.
t	double array	Empty	The times to use. Available when the underlying solution is transient.
table	new table name	new	The table to use when calling setResult() or appendResult(). new indicates that a new table is created.
tablecols	inner outer data level1	data	Whether to use inner or outer solutions as columns in the table when calling setResult() or appendResult(). Applicable only for parametric sweep models. The level values (level1, level2, and so on) are the levels in the parametric sweep.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.

SEE ALSO

[AvVolume](#), [AvSurface](#), [AvLine](#)

Streamline

Create a streamline plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Streamline");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Streamline")` creates a streamline plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Streamlines visualize a vector quantity on domains. A streamline is a curve everywhere tangent to an instantaneous vector field. Streamlines are available in 2D and 3D plot groups.

The following properties are available:

TABLE 7-151: VALID PROPERTIES FOR STREAMLINE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>adaptivestep</code>	<code>on</code> <code>off</code>	<code>on</code>	Use adaptive integration step length.
<code>arrowcount</code>	positive integer		The number of arrows, if <code>pointtype</code> is <code>arrow</code> and <code>arrowcountactive</code> is <code>on</code> .
<code>arrowcountactive</code>	<code>on</code> <code>off</code>	<code>off</code>	Allow specification of the number of arrow.
<code>arrowdistr</code>	<code>equalarc</code> <code>equaltime</code> <code>equalinvtime</code>	<code>equalarc</code>	Arrow distribution: equal in arc length, time, or inverse time, if <code>pointtype</code> is <code>arrow</code> .
<code>arrowlength</code>	<code>normalized</code> <code>proportional</code> <code>logarithmic</code>	<code>normalized</code>	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
<code>arrowscale</code>	positive double	<code>1</code>	The arrow length scale factor.
<code>arrowtype</code>	<code>arrow</code> <code>arrowhead</code> <code>cone</code>	<code>arrow</code>	The type of arrow to draw.
<code>back</code>	<code>on</code> <code>off</code>	<code>on</code>	Allow backward time integration. Only available for time-dependent datasets.
<code>color</code>	<code>custom</code> <code>black</code> <code>blue</code> <code>cyan</code> <code>fromtheme</code> <code>gray</code> <code>green</code> <code>magenta</code> <code>red</code> <code>white</code> <code>yellow</code>	<code>black (2D), red (3D)</code>	The uniform color to use for the streamlines.
<code>const</code>	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
<code>customcolor</code>	RGB-triplet	<code>{1,0,0}</code> or last used color	The uniform color to use. Active when <code>color</code> is set to <code>custom</code> .
<code>data</code>	<code>none</code> <code>parent</code> <code>dataset name</code>	<code>parent</code>	The dataset this feature refers to.
<code>descr</code>	String	Model dependent	The description of the expression in expr. Is used in the automatic title.

TABLE 7-151: VALID PROPERTIES FOR STREAMLINE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String array of length 2 in 2D and 3 in 3D	Model dependent	The components to plot.
fixedpointsize	boolean	false	Use constant radii for the plotted spheres. Available if <code>pointtype</code> is <code>interactivepoint</code> .
illuminatelines	on off	on	Whether the plotted lines are affected by lighting when <code>linetype</code> is <code>line</code> . Available in 3D only.
inheritarrowsscale	boolean	true	If <code>inheritplot</code> is not none: Determines if arrow scale is inherited.
inheritcolor	boolean	true	If <code>inheritplot</code> is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, tube scale, and deformation scale are inherited from.
inheritrange	boolean	true	If <code>inheritplot</code> is not none: Determines if the color and data ranges are inherited.
inheritribbonscale	boolean	true	If <code>inheritplot</code> is not none: Determines if the ribbon scale is inherited.
inherittransparency	boolean	true	If <code>inheritplot</code> is not none: Determines if the transparency settings are inherited.
inherit tubescale	boolean	true	If <code>inheritplot</code> is not none and <code>linetype</code> is <code>tube</code> or <code>ribbon</code> : Determines if the tube or ribbon scale is inherited.
integrator	rk2 rk4	rk2	The Runge-Kutta integration method to use: second-order or fourth-order.

TABLE 7-151: VALID PROPERTIES FOR STREAMLINE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
linetype	line tube ribbon none	line	Plot streamlines as lines, tubes, or ribbons (3D). Use none to plot arrows only.
localtime	double	0	The local time for the streamline time integration, if pointtype is interactivearrow or interactivepoint.
localtimeshifts	double array		Extra release times of arrows or points, if pointtype is interactivearrow or interactivepoint.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
looptol	positive double	0.01	Loop tolerance.
madv	manual automatic	automatic	Whether to specify advanced parameters when positioning method is magnitude or uniform.
magdensity	double array of length two containing positive doubles	{5, 12.3} (2D), {5, 9} (3D)	Minimum and maximum density levels of the streamlines. Active when posmethod is set to magnitude.
magexprmode	vectorfieldnorm custom	vectorfieldnorm	Whether to specify a custom expression to control the density of streamline starting points when selpointdistr is selmagctrl.
maxintegrationsteps	positive integer	1000000 (2D), 200000 (3D)	The maximum number of integration steps.
maxlen	positive double	Inf	The length of the streamlines as a fraction of the mean bounding box's size.
maxtime	positive double	Inf	Maximum integration time.
mdistend	nonnegative double	0.5	Terminating distance factor. Active when madv is set to manual.
minitref	positive integer	1	Boundary element refinement. Active when madv is set to manual.
msatfactor	nonnegative double	1.3	Starting distance factor. Active when madv is set to manual.

TABLE 7-151: VALID PROPERTIES FOR STREAMLINE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
mseed	double array of length 2 or 3, depending on space dimension.	$\{0,0\}$ (2D), $\{0,0,0\}$ (3D)	The first starting point. Active when mseedactive is set to on.
mseedactive	on off	off	Whether to manually specify first starting point. Active when madv is set to manual.
number	integer	20	Approximate number of streamlines. Active when startmethod is set to number.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
planecoordsys	cutplane cartesian	cartesian	Coordinate system for streamline vectors. Only available for plots that use a cut plane dataset that points to a 3D solution or mesh dataset.
pointradiusexpr	String	0.001	Point radius expression, if pointtype is interactivepoint.
pointtype	none arrow interactivearrow interactivepoint	none	Use no arrows or points, static arrows that are tangential to the streamlines, or interactive (moving) arrows or points that are tangential to the streamlines.
posmethod	magnitude start uniform selection	selection	The type of streamline positioning. Either starting-point controlled, uniform density, magnitude controlled, or starting from selected lines (boundaries) (2D) or surfaces (boundaries) or edges (3D). Use selection methods to specify the boundaries or edges.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	1	The tube radius.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.

TABLE 7-151: VALID PROPERTIES FOR STREAMLINE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
revcoordsys	cylindrical cartesian	cartesian	Coordinate system for streamline vectors. Only available for plots that use a 1D or 2D revolution dataset that has default axis settings and points to a solution or mesh dataset for an axisymmetric geometry.
selexpr	String	1	The expression to control the density of streamline starting points when magexprmode is custom.
selmdist	double array of length two containing positive doubles	{0.02, 0.1}	Minimum and maximum distance between streamline starting points relative to the size of the geometry. Active when selpointdistr is set to selmagctrl.
selmeshrefine	nonnegative integer	1	If posmethod is selection and selpointdistr is meshbased. The refinement of mesh-based point distribution on the selected edges (2D) or surfaces (3D).
selnumber	nonnegative integer	20	If posmethod is selection and selpointdistr is uniform. The approximate number of streamlines placed on the selected edges (2D) or surfaces (3D).
selpointdistr	uniform selmagctrl meshbased	uniform	If posmethod is selection: Use a uniform point distribution, a magnitude-controlled point distribution, or a mesh-based point distribution.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.

TABLE 7-151: VALID PROPERTIES FOR STREAMLINE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
sphereradiusscale	positive double		The scale factor applied to the point radii if sphereradiusscaleactive is on.
sphereradiusscaleactive	on off	off	If on, sphereradiusscale is used; otherwise the scale factor is computed automatically. Available if pointtype is interactivepoint.
startdata	none cut line or cut plane name	none	Active when posmethod is start: The line or plane where the starting points are distributed evenly.
startmethod	number coord	number	Active when posmethod is start. Indicates whether the starting points should be specified by number of points to distribute evenly or by coordinates.
steplen	positive double	0.1	The step length. Edit to specify how accurately streamlines are computed.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
tubeendcaps	true false	true	Add end caps to the streamline tubes, if linetype is set to tube.

TABLE 7-151: VALID PROPERTIES FOR STREAMLINE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
tuberadiusscale	double	1	The scale factor applied to the tube radii if tuberadiusscaleactive is true.
tuberadiusscaleactive	boolean	false	If true, tuberadiusscale is used; otherwise, the scale factor is computed automatically, if linetype is set to tube.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
uadv	manual automatic	automatic	Whether to specify advanced parameters when positioning method is uniform.
udensity	positive double	5	Density level of the streamlines. Active when posmethod is set to uniform.
udistend	nonnegative double	0.5	Terminating distance factor. Active when uadv is set to manual.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
usatfactor	nonnegative double	1.3	Starting distance factor. Active when uadv is set to manual.
useed	double array of length 2 or 3, depending on space dimension.	{0,0} (2D) or {0,0,0} (3D)	The first starting point. Active when useedactive is set to on.
useedactive	on off	off	Whether to manually specify first starting point. Active when uadv is set to manual.
widthexpr	String	1	The ribbon width.
widthscale	double	1	The scale factor applied to the ribbon widths if widthscaleactive is true.
widthscaleactive	boolean	false	If true, widthscale is used; otherwise, the scale factor is computed automatically.
xcoord	double array	Empty	Absolute coordinates in the x direction, active when startmethod is set to coord.
ycoord	double array	Empty	Absolute coordinates in the y direction, active when startmethod is set to coord.
zcoord	double array	Empty	Absolute coordinates in the z direction, active when startmethod is set to coord. Available in 3D only.

ATTRIBUTES

[Color](#), [Deform](#), [Export](#), [Filter](#) (Plot Attribute), [MaterialAppearance](#), [Selection](#) (Dataset Attribute), [Transformation](#), [Transparency](#) (3D only)

COMPATIBILITY

When `posmethod` is set to `magnitude`, the `mdist` property for 2D and 3D streamlines has been removed from version 6.4. Likewise, when `posmethod` is set to `uniform`, the `udist` property for 2D and 3D streamlines has been removed from version 6.4. Use the `magdensity` and `udensity` properties instead.

The `uinitref` and `uignoredist` properties for 3D streamlines in earlier versions have been removed from version 6.2.

When `posmethod` is set to `magnitude`, the `mdensity` property for 2D streamlines has been removed from version 6.1. Instead, use the `mdist` property.

The `lineararrowtype` property in previous versions of COMSOL Multiphysics has been replaced and extended by `pointtype` in version 5.5.

SEE ALSO

[Particle](#), [ParticleMass](#), [StreamlineSurface](#)

StreamlineMultislice

Create a streamline multislice plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"StreamlineMultislice");
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"StreamlineMultislice")` creates a streamline multislice plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Streamline plots visualize a vector quantity as streamlines on slices. A streamline is a curve everywhere tangent to an instantaneous vector field. Streamline multislice plots are available in 3D plot groups.

The following properties are available:

TABLE 7-152: VALID PROPERTIES FOR STREAMLINE MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>adaptivestep</code>	<code>on</code> <code>off</code>	<code>on</code>	Use adaptive integration step length. Active when
<code>anglethreshold</code>	<code>double</code>	<code>0.05[rad]</code>	The minimum angle threshold (in degrees).
<code>arrowcount</code>	<code>positive integer</code>		The number of arrows.
<code>arrowdistr</code>	<code>equalarc</code> <code>equaltime</code> <code>equalinvtime</code>	<code>equalarc</code>	Arrow distribution: equal in arc length, time, or inverse time.
<code>arrowlength</code>	<code>normalized</code> <code>proportional</code> <code>logarithmic</code>	<code>normalized</code>	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.

TABLE 7-152: VALID PROPERTIES FOR STREAMLINE MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowscale	positive double	1	The arrow length scale factor.
arrowtype	arrow arrowhead cone	arrow	The type of arrow to draw.
back	on off	on	Allow backward time integration. Only available for time-dependent datasets.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String array of length 2 in 2D and 3 in 3D	Model dependent	The components to plot.
illuminatelines	on off	on	Whether the plotted lines are affected by lighting when linetype is line.
inheritarrowscale	boolean	true	If inheritplot is not none: Determines if arrow scale is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.

TABLE 7-152: VALID PROPERTIES FOR STREAMLINE MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inheritplot	none plot name	none	The plot that color, tube scale, and deformation scale are inherited from.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
inherit tubescale	boolean	true	If inheritplot is not none and linetype is tube or ribbon: Determines if the tube or ribbon scale is inherited.
integrator	rk2 rk4	rk2	The Runge-Kutta integration method to use: second-order or fourth-order.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
linetype	line tube none	line	Plot streamlines as lines or tubes, or use no streamlines to plot arrows only.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
looptol	positive double	0.01	Loop tolerance.
madv	manual automatic	automatic	Whether to specify advanced parameters when positioning method is magnitude.
magdensity	double array of length two containing positive doubles	{5, 12.3}	Minimum and maximum density levels of the streamlines. Active when posmethod is set to magnitude.
maxintegrationsteps	positive integer	1000000	The maximum number of integration steps.
maxlen	positive double	Inf	The length of the streamlines as a fraction of the mean bounding box's size.
maxtime	positive double	Inf	Maximum integration time.
minitref	positive integer	1	Boundary element refinement. Active when madv is set to manual.

TABLE 7-152: VALID PROPERTIES FOR STREAMLINE MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
multiplanexmethod	number coord	number	Indicates whether the x-coordinates of the x-planes should be specified by number of planes to distribute evenly across the data or by coordinates.
multiplaneymethod	number coord	number	Indicates whether the y-coordinates of the y-planes should be specified by number of planes to distribute evenly across the data or by coordinates.
multiplanezmethod	number coord	number	Indicates whether the z-coordinates of the z-planes should be specified by number of planes to distribute evenly across the data or by coordinates.
number	integer	20	Approximate number of streamlines. Active when startmethod is set to number.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
planecoordsys	cutplane cartesian	cartesian	Coordinate system for streamline vectors. Only available for plots that use a cut plane dataset that points to a 3D solution or mesh dataset.
posmethod	magnitude start uniform	selection	The type of streamline positioning. Either starting-point controlled, uniform density, or magnitude controlled.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	1	The tube radius.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.

TABLE 7-152: VALID PROPERTIES FOR STREAMLINE MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
steplen	positive double	0.1	The step length. Edit to specify how accurately streamlines are computed.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0–1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
tuberadiusscale	double	1	The scale factor applied to the tube radii if tuberadiusscaleactive is true.

TABLE 7-152: VALID PROPERTIES FOR STREAMLINE MULTISLICE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
tuberadiusscaleactive	boolean	false	If true, tuberadiusscale is used, otherwise the scale factor is computed automatically.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
uadv	manual automatic	automatic	Whether to specify advanced parameters when positioning method is uniform.
udensity	positive double	5	Density level of the streamlines. Active when posmethod is set to uniform.
udistend	nonnegative double	0.5	Terminating distance factor. Active when uadv is set to manual.
uignoredist	nonnegative double	0.5	Fraction of streamline length to ignore. Active when uadv is set to manual.
uinitref	positive integer	1	Boundary element refinement. Active when uadv is set to manual.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
usatfactor	nonnegative double	1.3	Starting distance factor. Active when uadv is set to manual.
xnumber	String	1	Number of planes in the x direction. Active when multiplanexmethod is set to number.
xcoord	String array	Empty	Absolute coordinates in the x direction, active when multiplanexmethod is set to coord.
ynumber	String	1	Number of planes in the y direction. Active when multiplaneymethod is set to number.
ycoord	String array	Empty	Absolute coordinates in the z direction, active when multiplaneymethod is set to coord.
znumber	String	1	Number of planes in the z direction. Active when multiplanezmethod is set to number.
zcoord	String array	Empty	Absolute coordinates in the z direction, active when multiplanezmethod is set to coord.

ATTRIBUTES

[Color](#), [Deform](#), [Export](#), [Filter \(Plot Attribute\)](#), [MaterialAppearance](#), [Selection \(Dataset Attribute\)](#), [Transformation](#), [Transparency](#)

COMPATIBILITY

When `posmethod` is set to `magnitude`, the `mdist` property has been removed from version 6.4. Likewise, when `posmethod` is set to `uniform`, the `udist` property has been removed from version 6.4. Use the `magdensity` and `udensity` properties instead.

When `posmethod` is set to `magnitude`, the `mdensity` property has been removed from version 6.1. Instead, use the `mdist` property.

StreamlineSurface

Create a streamline surface plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"StreamlineSurface");
model.result(<pgtag>).feature(<ftag>).selection(...);
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"StreamlineSurface")` creates a streamline surface plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Streamline plots visualize a vector quantity as streamlines on plane surfaces. A streamline is a curve everywhere tangent to an instantaneous vector field. Streamline surface plots are available in 3D plot groups.

The following properties are available:

TABLE 7-153: VALID PROPERTIES FOR STREAMLINE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
<code>adaptivestep</code>	<code>on</code> <code>off</code>	<code>on</code>	Use adaptive integration step length.
<code>arrowcount</code>	positive integer		The number of arrows.
<code>arrowdistr</code>	<code>equalarc</code> <code>equaltime</code> <code>equalinvtime</code>	<code>equalarc</code>	Arrow distribution: equal in arc length, time, or inverse time.
<code>arrowlength</code>	<code>normalized</code> <code>proportional</code> <code>logarithmic</code>	<code>normalized</code>	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
<code>arrowscale</code>	positive double	<code>1</code>	The arrow length scale factor.
<code>arrowtype</code>	<code>arrow</code> <code>arrowhead</code> <code>cone</code>	<code>arrow</code>	The type of arrow to draw.
<code>back</code>	<code>on</code> <code>off</code>	<code>on</code>	Allow backward time integration. Only available for time-dependent datasets.
<code>color</code>	<code>custom</code> <code>black</code> <code>blue</code> <code>cyan</code> <code>gray</code> <code>green</code> <code>magenta</code> <code>red</code> <code>white</code> <code>yellow</code>	<code>red</code>	The uniform color to use.
<code>const</code>	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.

TABLE 7-153: VALID PROPERTIES FOR STREAMLINE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String array of length 2 in 2D and 3 in 3D	Model dependent	The components to plot.
illuminatelines	on off	on	Whether the plotted lines are affected by lighting when linetype is line.
inheritarrowsscale	boolean	true	If inheritplot is not none: Determines if arrow scale is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inheritplot	none plot name	none	The plot that color, tube scale, and deformation scale are inherited from.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
inherit tubescale	boolean	true	If inheritplot is not none and linetype is tube or ribbon: Determines if the tube or ribbon scale is inherited.

TABLE 7-153: VALID PROPERTIES FOR STREAMLINE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
integrator	rk2 rk4	rk2	The Runge-Kutta integration method to use: second-order or fourth-order.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
linetype	line tube none	line	Plot streamlines as lines or tubes, or use no streamlines to plot arrows only.
logrange	double > 1	100	If arrowlength is logarithmic: The ratio between the maximum arrow length and the arrow length below which no arrow is drawn.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
looptol	positive double	0.01	Loop tolerance.
madv	manual automatic	automatic	Whether to specify advanced parameters when positioning method is magnitude.
magdensity	double array of length two containing positive doubles	{5, 12.3}	Minimum and maximum density levels of the streamlines. Active when posmethod is set to magnitude.
magexprmode	vectorfieldnorm custom	vectorfieldnorm	Whether to specify a custom expression to control the density of streamline starting points when selpointdistr is selmagctrl.
maxintegrationsteps	positive integer	1000000	The maximum number of integration steps.
maxlen	positive double	Inf	The length of the streamlines as a fraction of the mean bounding box's size.
maxtime	positive double	Inf	Maximum integration time.
minitref	positive integer	1	Boundary element refinement. Active when madv is set to manual.
number	integer	20	Approximate number of streamlines. Active when startmethod is set to number.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.

TABLE 7-153: VALID PROPERTIES FOR STREAMLINE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
planecoordsys	cutplane cartesian	cartesian	Coordinate system for streamline vectors. Only available for plots that use a cut plane dataset that points to a 3D solution or mesh dataset.
posmethod	magnitude start uniform selection	start	The type of streamline positioning. Either starting-point controlled, uniform density, magnitude controlled, or starting from selected edges.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	1	The tube radius.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
selexpr	String	1	The expression to control the density of streamline starting points when magexprmode is custom.
selmdist	double array of length two containing positive doubles	{0.02, 0.1}	Minimum and maximum distance between streamline starting points relative to the size of the geometry. Active when selpointdistr is set to selmagctrl.
selmeshrefine	nonnegative integer	1	If posmethod is selection and selpointdistr is meshbased. The refinement of mesh-based point distribution on the selected edges.
selnumber	nonnegative integer	20	If posmethod is selection and selpointdistr is uniform. The approximate number of streamlines placed on the selected edges.
selpointdistr	uniform selmagctrl meshbased	uniform	If posmethod is selection: Use a uniform point distribution, a magnitude-controlled point distribution, or a mesh-based point distribution.

TABLE 7-153: VALID PROPERTIES FOR STREAMLINE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
startdata	none cut line name	none	Active when posmethod is start: The line where the starting points are distributed evenly.
startmethod	number coord	number	Active when posmethod is start. Indicates whether the starting points should be specified by number of points to distribute evenly or by coordinates.
steplen	positive double	0.1	The step length. Edit to specify how accurately streamlines are computed.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0–1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.

TABLE 7-153: VALID PROPERTIES FOR STREAMLINE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
tuberadiusscale	double	1	The scale factor applied to the tube radii if tuberadiusscaleactive is true.
tuberadiusscaleactive	boolean	false	If true, tuberadiusscale is used, otherwise the scale factor is computed automatically.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
uadv	manual automatic	automatic	Whether to specify advanced parameters when positioning method is uniform.
udensity	positive double	5	Density level of the streamlines. Active when posmethod is set to uniform.
udistend	nonnegative double	0.5	Terminating distance factor. Active when uadv is set to manual.
uignoredist	nonnegative double	0.5	Fraction of streamline length to ignore. Active when uadv is set to manual.
uinitref	positive integer	1	Boundary element refinement. Active when uadv is set to manual.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
usatfactor	nonnegative double	1.3	Starting distance factor. Active when uadv is set to manual.
xcoord	double array	Empty	Absolute coordinates in the x direction, active when startmethod is set to coord.
ycoord	double array	Empty	Absolute coordinates in the y direction, active when startmethod is set to coord.
zcoord	double array	Empty	Absolute coordinates in the z direction, active when startmethod is set to coord.

ATTRIBUTES

[Color](#), [Deform](#), [Export](#), [Filter](#) (Plot Attribute), [MaterialAppearance](#), [Transformation](#), [Transparency](#)

COMPATIBILITY

When `posmethod` is set to `magnitude`, the `mdist` property has been removed from version 6.4. Likewise, when `posmethod` is set to `uniform`, the `udist` property has been removed from version 6.4. Use the `magdensity` and `udensity` properties instead.

When `posmethod` is set to `magnitude`, the `mdensity` property has been removed from version 6.1. Instead, use the `mdist` property.

SEE ALSO

[Particle](#), [ParticleMass](#), [Streamline](#)

Surface (Plot)

Create a surface plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Surface");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Surface")` creates a surface plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Surface plots display a quantity as a colored 2D or 3D surface.

The following properties are available:

TABLE 7-154: VALID PROPERTIES FOR SURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bandcount	positive integer	10	The number of discrete color bands, if <code>colortabletype</code> is set to <code>discrete</code> .
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the surface: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to <code>colortable</code> .
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to <code>colortable</code> or <code>gradient</code> .
colortable	color table name	Rainbow	The color table to use when coloring is set to <code>colortable</code> . See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to <code>colortable</code> or <code>gradient</code> .

TABLE 7-154: VALID PROPERTIES FOR SURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colortabletype	continuous discrete	continuous	The color table type: continuous or discrete.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, height scale, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.

TABLE 7-154: VALID PROPERTIES FOR SURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendtitle has been set to true in the parent plot group.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.

TABLE 7-154: VALID PROPERTIES FOR SURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0–1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.

TABLE 7-154: VALID PROPERTIES FOR SURFACE PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
wireframe	on off	off	Whether to plot filled elements or only their edges.

ATTRIBUTES

Deform, Filter (Plot Attribute), Height, AberrationHeight, HistogramHeight, TableHeight (2D only), Image (Plot), Marker, MaterialAppearance, Selection (Dataset Attribute), Transformation, Transparency (3D and 2D, if a height attribute is used)

EXAMPLES

Surface plot on 2D solution:

Code for Use with Java

```
DatasetFeature ds = model.result().dataset().create("dset1", "Solution");
ds.set("Solution", "sol1");

ResultFeature pg = result().create("pg1",2);
pg.set("data","dset1");
pg.create("surf1","Surface");
pg.feature("surf1").set("expr", "3*u");
```

Code for Use with MATLAB

```
ds = model.result.dataset.create('dset1', 'Solution');
ds.set('Solution', 'sol1');

ResultFeature pg = result().create('pg1',2);
pg.set('data','dset1');
pg.create('surf1','Surface');
pg.feature('surf1').set('expr', '3*u');
```

Surface plot on cut plane in 3D using a Thermal color table:

Code for Use with Java

```
result().dataset().create("cp1", "CutPlane");

ResultFeature pg2 = result().create("pg2",3);
```

```
pg2.create("surf2","Surface");
pg2.feature("surf2").set("data", "cp1");
pg2.feature("surf1").set("expr", "2*u");
pg2.feature("surf1").set("colortable", "Thermal");

pg2.run();
```

Code for Use with MATLAB

```
result.dataset.create('cp1', 'CutPlane');

pg2 = result.create('pg2',3);
pg2.create('surf2','Surface');
pg2.feature('surf2').set('data', 'cp1');
pg2.feature('surf1').set('expr', '2*u');
pg2.feature('surf1').set('colortable', 'Thermal');

pg2.run;
```

SEE ALSO

[SurfaceSlit](#), [Volume](#), [Isosurface \(Plot\)](#), [Function2D](#)

Surface (Dataset)

Create a surface dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Surface");
model.result().dataset(<dtag>).set(property,<value>);
```

DESCRIPTION

`model.result().dataset().create(<dtag>,"Surface")` creates a surface dataset with the name `<dtag>`.

The surface dataset makes it possible to evaluate surfaces of a 3D model in 2D or 3D. For evaluation in 2D, different parameterizations of the 2D projection are available.

The following properties are available:

TABLE 7-155: VALID PROPERTIES FOR SURFACE DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The solution dataset this feature refers to.
param	face xy yz zx yx zy xz expr	face	The projection used when evaluating in 2D.
exprx	String		When param is expr: The x-axis expression when evaluating in 2D.
expy	String		When param is expr: The y-axis expression when evaluating in 2D.
spacevars	String array	Depends on the feature's tag	The name of space variables, which evaluate to the coordinates in the dataset's coordinate system.

SurfaceData

Create a surface data plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"SurfaceData");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"SurfaceData")` creates a surface data plot feature named `<ftag>` belonging to the 2D or 3D plot group `<pgtag>`.

Surface data plots are used to visualize raw point data given as points, elements, normals (3D only), and colors as surfaces (see the examples below). Surface data plots can be added to 2D and 3D plot groups.

The following properties are available:

TABLE 7-156: VALID PROPERTIES FOR SURFACE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
colordata	double ID array		The color data for the surface data plot as a real N-vector.
coloring	colortable uniform gradient	colortable	How to color the surfaces: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.

TABLE 7-156: VALID PROPERTIES FOR SURFACE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
elementdata	integer 2D array		The element data for the surface data plot, providing connections between the points. The data is an (edim+1)-by-M integer matrix with values that are 0-based indices into pointdata. The right-hand rule defines the normal direction in 3D. In 2D the order does not matter.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
normaldata	double 2D array		A 3-by-N real matrix for the normals in 3D surface data plots only. This property is optional and can either be empty or have the same length as the number of columns in pointdata. If it is empty, the normals are computed by the standard algorithm used throughout results processing.
pointdata	double 2D array		The point data for the surface data plot, as x- and y-coordinates in 2D and x-, y-, and z-coordinates in 3D in an sdim-by-N real matrix.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.

TABLE 7-156: VALID PROPERTIES FOR SURFACE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
title	String	The auto-title.	The title to use when <code>titletype</code> is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the <code>title</code> property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.

ATTRIBUTES

[Marker](#), [MaterialAppearance](#), [Transparency](#)

EXAMPLES

A method for creating a pentagon as a 2D surface data plot:

Code for Use with Java

```
String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 2);
ResultFeature plot = pg.create("surf1", "SurfaceData");
int N = 5;
double[][] p = new double[2][N+1];
int[][] t = new int[3][N];
double[] color = new double[N+1];
p[0][0] = 0;
p[1][0] = 0;
for (int i = 0; i < N; i++) {
    double angle = i * 2 * Math.PI / N;
    p[0][i + 1] = Math.cos(angle);
    p[1][i + 1] = Math.sin(angle);
    t[0][i] = 0;
    t[1][i] = i + 1;
    t[2][i] = 1 + (i + 1) % N;
}
plot.set("pointdata", p)
    .set("elementdata", t)
    .set("colordata", color);
plot.run();
```

A method for creating a surface data plot in 3D for data representing the sinc function (sampling function) as function of the radius r :

Code for Use with Java

```
String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 2);
ResultFeature plot = pg.create("surf1", "SurfaceData");
int Nx = 51;
int Ny = 51;
double[][] p = new double[3][Nx * Ny];
int[][] t = new int[3][2 * (Nx-1) * (Ny-1)];
double[] color = new double[Nx*Ny];
int pos = 0;
for (int i = 0; i < Ny; i++) {
    for (int j = 0; j < Nx; j++) {
        double x = 20 * (j - Nx / 2) / Nx;
```

```

        double y = 20 * (i - Ny / 2) / Ny;
        double r = Math.sqrt(x * x + y * y);
        double z = 4 * ((r == 0) ? 1 : (Math.sin(r) / r));
        p[0][pos] = x;
        p[1][pos] = y;
        p[2][pos] = z;
        color[pos] = z;
        pos++;
    }
}
pos = 0;
for (int i = 0; i < Ny - 1; i++) {
    for (int j = 0; j < Nx - 1; j++) {
        int p00 = Nx * i + j;
        int p01 = Nx * i + j + 1;
        int p10 = Nx * (i + 1) + j;
        int p11 = Nx * (i + 1) + j + 1;
        t[0][pos] = p00;
        t[1][pos] = p01;
        t[2][pos] = p11;
        pos++;
        t[0][pos] = p00;
        t[1][pos] = p11;
        t[2][pos] = p10;
        pos++;
    }
}
plot.set("pointdata", p)
    .set("elementdata", t)
    .set("colordata", color);
plot.run();

```

The coloring of the sphere is based on the z -coordinate of each triangle point.

SEE ALSO

[AnnotationData](#), [ArrowData](#), [LineData](#), [PointData](#), [TubeData](#)

SurfaceSlit

Create a surface slit plot, for evaluating one expression on the upside and another expression on the downside of a boundary in 3D.

SYNTAX

```

model.result(<pgtag>).create(<ftag>, "SurfaceSlit");
model.result(<pgtag>).feature(<ftag>).set(property, <value>);
model.result(<pgtag>).feature(<ftag>).run();

```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>, "SurfaceSlit")` creates a surface slit plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Surface slit plots display quantities as colored surfaces on the upside and downside of selected boundaries in a 3D model.

The following properties are available:

TABLE 7-157: VALID PROPERTIES FOR SURFACE SLIT PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the surfaces: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
downdescr	String	Model dependent	The description of the expression in downexpr. Is used in the automatic title.
downexpr	String	Model dependent	The expression to plot on the downside.

TABLE 7-157: VALID PROPERTIES FOR SURFACE SLIT PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
downunit	String	Model dependent	The unit to use for the expression in downexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritheightscale	boolean	true	If inheritplot is not none: Determines if height scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, height scale, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-157: VALID PROPERTIES FOR SURFACE SLIT PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.

TABLE 7-157: VALID PROPERTIES FOR SURFACE SLIT PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

TABLE 7-157: VALID PROPERTIES FOR SURFACE SLIT PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
updescr	String	Model dependent	The description of the expression in upexpr. Is used in the automatic title.
upexpr	String	Model dependent	The expression to plot on the upside.
upunit	String	Model dependent	The unit to use for the expression in upexpr. If the old unit is not valid when the expression changes, the unit property is reset to default.

ATTRIBUTES

[Deform](#), [Filter \(Plot Attribute\)](#), [Marker](#), [MaterialAppearance](#), [Selection \(Dataset Attribute\)](#), [Transformation](#), [Transparency](#)

SEE ALSO

[Surface \(Plot\)](#), [Volume](#)

SystemMatrix

Create a numerical evaluation of system matrix.

SYNTAX

```
model.result().numerical().create(<ftag>,"SystemMatrix");
model.result().numerical(<ftag>).set(property,<value>);
model.result().numerical(<ftag>).getData();
model.result().numerical(<ftag>).getData(<expressionIndex>);
model.result().numerical(<ftag>).getImagData();
model.result().numerical(<ftag>).getImagData(<expressionIndex>);
model.result().numerical(<ftag>).isComplex();
model.result().numerical(<ftag>).isComplex(<outersolnum>);
model.result().numerical(<ftag>).getNData();
model.result().numerical(<ftag>).run();
```

DESCRIPTION

`model.result().numerical().create(<ftag>,"SystemMatrix")` create a system matrix evaluation feature with the name `<ftag>`.

The system matrix evaluation makes it possible to retrieve matrices directly from Assemble and Modal features in a solver sequence. You can also use this methods to retrieve matrices and vectors directly from reduced model datasets.

The following properties are available:

TABLE 7-158: VALID PROPERTIES FOR SYSTEM MATRIX EVALUATIONS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
feature	String	The first Assemble or Modal feature in the chosen solution.	The solver feature from which the matrix is retrieved.
matrixassem	constraint constraintforcejac constraintforcenullbasis constraintjac constraintnullbasis damping elimdamping elimload elimmass elimstiffness load lowerboundconstraint mass optconstraint optconstraintjac partsol stiffness upperboundconstraint uscale	stiffness	The matrix to retrieve if the solver feature is an Assemble feature. See also the documentation for Assemble in the <i>COMSOL Multiphysics Reference Manual</i> .
matrixmodal	allloads damping dampingratio ddinctrl dinctrl dinctrl0 dpartsol epartsol inctrl inctrl0 inoutput kud load mass projection stiffness output outputbias u0 udot0 u1	stiffness	The matrix to retrieve if the solver feature is a frequency-dependent Modal Solver feature.
matrixmodalstate	allloads damping dampingratio ddinctrl dinctrl dinctrl0 dpartsol epartsol inctrl inctrl0 inoutput kud load mass projection ssc ssd ssma ssmb ssmc ssnul1 ssud ssx0 stiffness output outputbias u0 udot0 u1	stiffness	The matrix to retrieve if the solver feature is a time-dependent Modal Solver feature.
matrixstatespace	ssc ssd ssma ssmb ssmc ssnul1 ssud ssx0	ssma	The matrix to retrieve if the solver feature is a State Space feature.
reducedmodelmatrix	allloads damping dampingratio ddinctrl dinctrl dinctrl0 dpartsol epartsol inctrl inctrl0 inoutput kud load mass projection ssc ssd ssma ssmb ssmc ssnul1 ssud ssx0 stiffness output outputbias u0 udot0 u1	stiffness	The matrix to retrieve if the solution is from a Reduced Model dataset.
solution	String	First compatible solution	The solution this dataset refers to.
threshold	nonnegative double	0	Sets a cutoff threshold where elements below the assigned value are considered as zero.

Table

Create a feature containing a table of data.

SYNTAX

```
model.result().table().create(<ftag>,"Table");
model.result().table(<ftag>).setColumnHeaders(<headers>);
model.result().table(<ftag>).setTableData(<realData>);
model.result().table(<ftag>).setTableData(<realData>,<imagData>);
model.result().table(<ftag>).addColumnns(<headers>,<realData>);
model.result().table(<ftag>).addColumnns(<headers>,<realData>,<imagData>);
model.result().table(<ftag>).addRow(<realData>);
model.result().table(<ftag>).addRow(<realData>,<imagData>);
model.result().table(<ftag>).addRows(<realData>);
model.result().table(<ftag>).addRows(<realData>,<imagData>);
model.result().table(<ftag>).addRowsAndWarnings(<realData>,<warnings>));
model.result().table(<ftag>).addRowAndWarnings(<realData>,<imagData>,<warnings>);
model.result().table(<ftag>).removeRow(<index>);
model.result().table(<ftag>).removeRow(<index_array>);
model.result().table(<ftag>).getColumnHeaders();
model.result().table(<ftag>).getReal();
model.result().table(<ftag>).getRowHeaders();
model.result().table(<ftag>).getImag();
model.result().table(<ftag>).Row(<index>);
model.result().table(<ftag>).getImagRow(<index>);
model.result().table(<ftag>).getTableData(<fullPrecision>);
model.result().table(<ftag>).getTableRow(<index>,<fullPrecision>);
model.result().table(<ftag>).getNRows();
model.result().table(<ftag>).isComplex();
model.result().table(<ftag>).clearTableData();
model.result().table(<ftag>).set(property,<value>);
model.result().table(<ftag>).save(<filename>);
model.result().table(<ftag>).save(<filename>,<fullPrecision>);
model.result().table(<ftag>).loadFile(tempFile,...);
model.result().table(<ftag>).saveFile(tempFile,...);
```

DESCRIPTION

`model.result().table().create(<ftag>,"Table")` creates a table feature named `<ftag>`. Tables support two data formats, *all* or *filled*. Filled data is typically produced from parametric sweeps and makes it possible to retrieve data for a pair of parameters on a matrix format. Filled tables can be used to make response surface plots. (See [TableSurface](#).)

`model.result().table(<ftag>).setColumnHeaders(<headers>)` sets the table's column headers from the string array `<headers>`.

`model.result().table(<ftag>).setTableData(<realData>)` sets the table content from a double matrix containing real data. Any previous real or imaginary data is removed.

`model.result().table(<ftag>).setTableData(<realData>,<imagData>)` sets both real and imaginary data from the double matrices `<realData>` and `<imagData>`, which must be of the same size. `<imagData>` can be null to indicate that there is no imaginary data.

`model.result().table(<ftag>).getColumnHeaders()` retrieves the column headers.

`model.result().table(<ftag>).addColumnns(<headers>,<realData>)` adds one or more columns and associated real data to the table.

`model.result().table(<ftag>).addColumnns(<headers>,<realData>,<imagData>)` adds one or more columns and associated real data and imaginary data to the table.

`model.result().table(<ftag>).addRow(<realData>)` adds one row of real data to the table.

`model.result().table(<ftag>).addRow(<realData>,<imagData>)` adds one row of real and imaginary data to the table.

Similarly `model.result().table(<ftag>).addRows(<realData>)` and `model.result().table(<ftag>).addRows(<realData>,<imagData>)` add multiple rows of real or real and imaginary data to the table, and `model.result().table(<ftag>).addRows(<realData>,<warnings>)` and `model.result().table(<ftag>).addRows(<realData>,<imagData>,<warnings>)` add multiple rows of real or real and imaginary data as well as Boolean warnings to the table.

`model.result().table(<ftag>).removeRow(<index>)` removes the row with a given index from the table. If the row index is out of bounds, nothing happens.

`model.result().table(<ftag>).removeRows(<index_array>)` removes the rows listed as an array of positive integers as the input argument.

`model.result().table(<ftag>).getColumnHeaders()` returns the column headers in the table.

`model.result().table(<ftag>).getReal()` returns the real part of the table content.

`model.result().table(<ftag>).getImag()` returns the imaginary part of the table content. Note: this method allocates imaginary data if there was none. Check for imaginary content with the `isComplex` method before calling this method if you want to avoid this.

`model.result().table(<ftag>).getRealRow(<index>)` returns the real data in one row.

`model.result().table(<ftag>).getImagRow(<index>)` returns the imaginary data in one row.

`model.result().table(<ftag>).getFilledReal()` returns the real part of the table content on a filled format, when available.

`model.result().table(<ftag>).getFilledImag()` returns the imaginary part of the table content on a filled format, when available.

`model.result().table(<ftag>).getTableData(<fullPrecision>)` returns the table data as a string matrix, with limited or full precision as specified by the Boolean `<fullPrecision>`.

`model.result().table(<ftag>).getTableRow(<index>,<fullPrecision>)` returns the table data for one row as a string array, with limited or full precision.

`model.result().table(<ftag>).getNRows()` returns the number of rows in the table.

`model.result().table(<ftag>).isComplex()` returns true if the table contains imaginary data. This method checks whether imaginary data has been allocated, not if it is different from 0.

`model.result().table(<ftag>).clearTableData()` removes all table data and column headers.

`model.result().table(<ftag>).save(<filename>)` saves the table `<ftag>` content to the text file `<filename>` in full precision.

For `model.result().table(<ftag>).loadFile()` and `model.result().table(<ftag>).saveFile()`, see [The loadFile and saveFile Methods](#).

The following properties are available:

TABLE 7-159: VALID PROPERTIES FOR TABLES.

NAME	VALUE	DEFAULT	DESCRIPTION
cols	parameters in table none	none	For tables with filled data, cols controls which parameter is used on the columns. Cannot be the same parameter as in rows.
datacol	positive integer	1	For tables with filled data, datacol controls which data column should be used to populate the filled table.

TABLE 7-159: VALID PROPERTIES FOR TABLES.

NAME	VALUE	DEFAULT	DESCRIPTION
descr	none data manual	data	For tables with filled data, descr controls which description to use for the columns, in addition to the parameter values. none gives no description, manual uses the description in descrmanual, and data uses the description in the table data.
descrmanual	String	Empty	Manual column header for tables with filled data, used when descr is set to manual.
filename	String	Empty	The name of the file on which the table is stored, used when storetable is set to inmodelandonfile or onfile.
filterstringdata	on off	off	For tables with filled data, filterstringdata controls whether to filter the input column using showrowstep.
format	all filled	all	Controls whether you retrieve all or filled table data when calling the getTableData method.
param	positive integer	1	For tables with filled data containing more than two parameters, param is the index indicating which values to use for fixing additional parameters. Additional parameters are ordered from left to right in the original table data.
rows	parameters in table none	none	For tables with filled data, rows controls which parameter is used on the rows.
showrowstep	positive integer	1	When retrieving filled data using the getTableData method, showrowstep controls whether to leave all but every showrowstep input cells empty, for example, every third cell in the first column. This refers to the input column only. Only applied when filterstringdata is on.
storetable	inmodel inmodelandonfile onfile	inmodel	Controls whether table is stored in model(inmodel) or on file(onfile), or both(inmodelandonfile).
tablebuffersize	positive integer	Taken from buffer size preference. The default of which is 10000.	The size of the in-memory buffer size where the table is stored, used when storetable is set to inmodel or inmodelandonfile. In the second case only the last tablebuffersize rows are kept in the model; the rest is read from file when necessary.

SEE ALSO

[Table \(Plot\)](#), [TableAnnotation](#), [TableDOE](#), [TableSurface](#), [TableUQ](#)

Table (Export)

Export data from tables to files.

SYNTAX

```
model.result().export().create(<ftag>,"Table");  
model.result().export(<ftag>).set(property,<value>);  
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>,"Table")` creates a table export feature with the name `<ftag>`.

The following properties are available:

TABLE 7-160: VALID PROPERTIES FOR TABLE EXPORTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alwaysask	on off	off	Always ask for filename when saving. This property is ignored when running without a UI.
complexnotation	fromprefs rectangular polar	fromprefs	Use a complex notation from the Table window (fromprefs) or a rectangular or polar notation.
evaluationgroup	none evaluation group feature name	First available evaluation group	The evaluation group feature this feature refers to, if source is evaluationgroup.
ifexists	append overwrite	overwrite	If the file exists, append to or overwrite the file contents.
filename	String		The output file.
header	on off	off	Enable/disable a data header in the output file.
manualprec	positive integer	5	The numerical precision (between 1 and 17), when prec is set to manual.
notation	fromtable automatic engineering scientific decimal	fromtable	Use a numeric notation from the Table window, an automatic notation, or an engineering, scientific, or decimal notation.
prec	fromtable full manual	fromtable	Use the precision for floating-point numbers from the Table window, using full precision, or specified manually.
sort	on off	off	Enable/disable sorting of the points with respect to the coordinates.
source	table evaluationgroup	table	Use table data from a table or an evaluation group.
table	none table feature name	First available table	The table feature this feature refers to, if source is table.
target	file clipboard linked	file	The target for the table: a file, the Windows clipboard, or linked to PowerPoint and Word (the latter two options are only available on Windows),

SEE ALSO

[Data](#), [Table](#)

Table (Plot)

Plot data from a table in a graph plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Table");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Table")` creates a table graph plot feature named `<ftag>`.

Table graphs can be added to 1D and polar plot groups and displays data from a table.

The following properties are available:

TABLE 7-161: VALID PROPERTIES FOR TABLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autoheaders	true false	true	Whether the automatic legends should include the column headers.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgecolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
dbmanualref	double	0	Reference value for display in dB, if dbref is manual (and indb is true).
dbref	maximum manual	maximum	Use the maximum value of a manual value as the dB reference when indb is true.
dbtype	10log 20log	10log	Use a 10 log or 20 log for the dB values when indb is true.
evaluationgroup	none evaluation group feature name	First available evaluation group	The evaluation group feature this plot refers to, if source is evaluationgroup.
fouriershow	real imaginary amplitude phase spectrum	amplitude	When transform is fourier: Show the DFT as real or imaginary part, amplitude, phase, or frequency spectrum.-
freqmax	integer		If fouriershow is spectrum and freqrangeactive is true: The upper frequency bound.
freqmin	integer		If fouriershow is spectrum and freqrangeactive is true: The lower frequency bound.

TABLE 7-161: VALID PROPERTIES FOR TABLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
freqrangeactive	true false	false	If fouriershow is spectrum: Controls whether a manual frequency range is used.
ignorelast	true false	true	If fouriershow is not spectrum: ignore last sample.
imagplot	on off	off	If transform is none: When on, the imaginary part of the data in the table is plotted.
indb	true false	false	Display the amplitude or frequency spectrum in dB when fouriershow is amplitude or spectrum.
inverse	true false	false	If fouriershow is not spectrum: inverse the discrete Fourier transform.
legend	on off	off	Whether to show legends.
legendmethod	automatic manual	automatic	Whether to use the automatic legends or the legends supplied in the legends property.
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends.	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
maskdc	true false	false	If fouriershow is not spectrum: set the DC value (zero frequency component) to zero.

TABLE 7-161: VALID PROPERTIES FOR TABLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
nfreqs	integer	1	If <code>fouriershow</code> is <code>spectrum</code> and <code>nfreqsactive</code> is <code>true</code> : The number of frequencies to plot.
nfreqsactive	true false	false	If <code>fouriershow</code> is <code>spectrum</code> : Controls whether the number of frequencies is set manually.
nsamples	integer	1	If <code>fouriershow</code> is not <code>spectrum</code> and <code>nsamplesactive</code> is <code>true</code> : The number of uniform samples to use.
nsamplesactive	true false	false	If <code>fouriershow</code> is not <code>spectrum</code> : Controls whether the number of uniform samples is set manually.
numberofcolors	integer	7	The number of colors in the color cycle when <code>cyclelength</code> is manual.
plotcolumninput	all manual	all	The columns or rows to plot. <code>all</code> indicates all columns excluding those used in <code>xaxisdata</code> , <code>manual</code> indicates the columns specified in <code>plotcolumns</code> .
plotcolumns	positive integer array	Empty	The columns or rows to plot when <code>plotcolumninput</code> is <code>manual</code> .
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if <code>twoaxes</code> is set to <code>true</code> in the parent plot group.
preproc{x y}	none linear	none	Use a linear preprocessing of the table data values, which you specify using the scaling and shift properties.
rangex	all manual	all	Use all values in the range for the x-axis column, or specify a manual range.
rangey	all manual	all	Use all values in the range for the y-axis column, or specify a manual range.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when <code>linecolor</code> is <code>cyclecolortable</code> .
rowbased	boolean	false	Plot row-based table data.
scale	none multiplyperiod dividenfreq	none	If <code>fouriershow</code> is <code>spectrum</code> , use no scaling, multiply by sampling period, or divide by number of frequencies.
scaleperiod	true false	true	If <code>fouriershow</code> is not <code>spectrum</code> : scale with sampling period.
scaling{x y}	double	1.0	Real scaling value for data preprocessing
shift{x y}	double	0.0	Real shift value for data preprocessing
shiftzero	true false	true	If <code>fouriershow</code> is not <code>spectrum</code> : shift zero frequency.
sorting	none x y	none	Sort the data based on the x- or y-coordinates.
source	table evaluationgroup	table	Use table data from a table or an evaluation group.
table	none table feature name	First available table	The table feature this plot refers to, if source is <code>table</code> .

TABLE 7-161: VALID PROPERTIES FOR TABLE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
transform	none fourier	none	The transformation to apply to the data before plotting. None, or a discrete Fourier transform (DFT):
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
xaxisdata	auto rowindex positive integer	auto	The column supplying x-axis data. auto determines input based on evaluated result, for example, using an index if the table contains output from a parametric sweep with multiple parameters. rowindex means that all columns are used as data and are plotted against the row number in the table.
xmin	double	-Inf	The minimum value for the x-column data range if rangex is set to manual.
xmax	double	Inf	The maximum value for the x-column data range if rangex is set to manual.
ymin	double	-Inf	The minimum value for the y-column data range if rangey is set to manual.
ymax	double	Inf	The maximum value for the y-column data range if rangey is set to manual.

ATTRIBUTES

[GraphMarker](#), [TableErrorBars](#), [Filter \(Table Graph\)](#)

SEE ALSO

[Table](#), [TableAnnotation](#), [TableArrow](#), [TableSurface](#)

TableAnnotation

Add annotations from a table to a plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"TableAnnotation");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"TableAnnotation")` creates a table annotation plot feature named `<ftag>`.

Table annotations can be added to 1D, 2D, and 3D plot groups.

The following properties are available:

TABLE 7-162: VALID PROPERTIES FOR TABLE ANNOTATION PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
anchorpoint	upperright uppermiddle upperleft middleright center middleleft lowerright lowermiddle lowerleft	upperleft	The position of the annotation text relative to the anchor point.
annotation	rowheaders rownnumbers label manual	rowheaders	Whether to use the row headers, the row numbers, the plot's label, or a manual input (the text property) as annotation. Available if source is table or evaluationgroup.
backgroundcolor	custom fromtheme black blue cyan gray green magenta red white yellow none	none	The color to use for background rectangle.
color	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	The color to use for the annotation texts.
custombackgroundcolor	RGB-triplet	{1,0,0} or last used color	The background color to use. Active when backgroundColor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The color to use. Active when color is set to custom.
evaluationgroup	none evaluation group feature name	First available evaluation group	The evaluation group feature this plot refers to, if source is evaluationgroup.
inheritbackgroundcolor	boolean	true	If inheritplot is not none: Determines if the background color is inherited.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritframe	boolean	true	If inheritplot is not none: Determines if the show frame property is inherited.
inheritplot	none plot name	none	The plot that background color, color, deform scale factor, and frame is inherited from.
latexmarkup	on off	off	Include LaTeX markup language in annotation.
localtablematrix	l-by-3 array		Content of the local matrix, when source is set to localtable. Specify them using the setIndex method.

TABLE 7-162: VALID PROPERTIES FOR TABLE ANNOTATION PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
relpadding	boolean	false	Add relative padding.
showframe	true false	false	Whether to show a frame around the annotation or not.
source	table evaluationgroup localtable	table	Use table data from a table or an evaluation group, or define a local table.
table	none table feature name	First available table	The table feature this plot refers to, if source is table.
text	String	Empty	The text to display as an annotation, if annotation is set to manual.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the title property). none, if no title contribution should be used.
xaxisdata	auto rowindex	auto	The column supplying x-coordinates. auto determines input based on evaluated result. rowindex means that all columns are used as data.
xpadding	positive value	0.1	The padding in the x direction, if relpadding is true.
yaxisdata	auto rowindex	auto	The column supplying y-coordinates. auto determines input based on evaluated result. rowindex means that all columns are used as data.
ypadding	positive value	0.1	The padding in the y direction, if relpadding is true.
zaxisdata	auto rowindex	auto	The column supplying z-coordinates (3D only). auto determines input based on evaluated result. rowindex means that all columns are used as data.
zpadding	positive value	0.1	The padding in the z direction, if relpadding is true (for 2D and 3D only).

ATTRIBUTE[Transparency](#) (3D only)**SEE ALSO**[Table](#), [Table \(Plot\)](#), [TableArrow](#), [TableSurface](#)*TableArrow*

Create a table arrow plot for 2D and 3D plot groups.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"TableArrow");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"TableArrow")` creates a table arrow plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

Use a table arrow plot to visualize data from a table or evaluation group in 2D or 3D.

The following properties are available:

TABLE 7-163: VALID PROPERTIES FOR TABLE ARROW PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
arrowbase	head tail center	tail	Whether the head, tail, or center of the arrow is located at the arrow position.
arrowlength	normalized proportional logarithmic	proportional	The arrow scaling: Proportional uses the actual arrow length, normalized a unit length, and logarithmic a length proportional to the logarithm of the arrow length.
arrowtype	arrow arrowhead cone double	arrow	The type of arrow to draw.
color	custom black blue cyan gray green magenta red white yellow	red	The color to use.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
evaluationgroup	none evaluation group feature name	First available evaluation group	The evaluation group feature this plot refers to, if source is evaluationgroup.
imagplot	on off	off	When on, the imaginary part of the data in the table is plotted.
inheritarrowsscale	boolean	true	Determines if arrow scale is inherited.
inheritcolor	boolean	true	Determines if the color is inherited.
inheritplot	none plot name	none	The plot that arrow scale and color are inherited from.
plotcolumninput	auto manual	auto	Whether the table data is automatically interpreted or manually specified by column.
preproc	none linear	none	Use a linear preprocessing of the coordinate data values, which you specify using the scaling and shift properties.
preprocdir	none linear	none	Use a linear preprocessing of the direction data values, which you specify using the scaling and shift properties.
scale	positive double	1	If scaleactive is true: The length scale factor.
scaleactive	boolean	false	Whether to use manual scaling.
scaling	double	1	Real scaling value for data preprocessing, if preproc is linear.

TABLE 7-163: VALID PROPERTIES FOR TABLE ARROW PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
scalingdir	double	1	Real scaling value for data preprocessing, if preprocdir is linear.
shift	double	0	Real shift value for data preprocessing, if preproc is linear.
shiftdir	double	0	Real shift value for data preprocessing, if preprocdir is linear.
source	table evaluationgroup	table	Use table data from a table or an evaluation group.
table	none table feature name	First available table	The table feature this plot refers to, if source is table.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
xdata	positive integer	1	The column to take x-coordinate data from, when plotcolumninput is manual.
xdirdata	positive integer	1	The column to take x-direction data from, when plotcolumninput is manual.
ydata	positive integer	1	The column to take y-coordinate data from, when plotcolumninput is manual.
ydirdata	positive integer	1	The column to take y-direction data from, when plotcolumninput is manual.
zdata	positive integer	1	The column to take z-coordinate data from, when plotcolumninput is manual.
zdirdata	positive integer	1	The column to take z-direction data from, when plotcolumninput is manual.

ATTRIBUTE

[TableColor](#), [Transparency](#) (3D only), [VisualEffects](#) (3D only)

SEE ALSO

[Table](#), [Table \(Plot\)](#), [TableAnnotation](#), [TableColor](#), [TableSurface](#)

TableColor

Add a color expression attribute to add coloring to arrows in a table arrow plot.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"TableColor");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "TableColor")` creates a color expression attribute named <atag> belonging to the table arrow plot feature <ftag>.

The following properties are available:

TABLE 7-164: VALID PROPERTIES FOR TABLECOLOR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
coldata	positive integer	1	The column to take color data from, when <code>plotcolumninput</code> is manual.
colordata	arrowlength data	arrowlength	Whether to color the arrows using data from a table column or using the arrow length.
coloring	colortable gradient	colortable	Whether to use a color table or a color gradient.
colorlegend	on off	on	Whether to show a color legend.
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic.
colortable	color table name	Rainbow	The color table to use. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table.
colortabletype	continuous discrete	continuous	Whether the color table should be continuous or discrete.
descr	String	Model dependent	The description of the expression in <code>expr</code> . Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is harmonic.
imagplot	on off	off	When on, the imaginary part of the data in the table is plotted.
legendtitle	String		Color legend title, if <code>showlegendstitle</code> has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when <code>coloring</code> is set to <code>colortable</code> or <code>gradient</code> and <code>colortabletrans</code> is set to <code>nonlinear</code> or <code>nonlinearsymmetric</code> .
plotcolumninput	auto manual	auto	Whether the color data is taken from the table automatically or from the column supplied in the <code>coldata</code> property, if <code>colordata</code> is set to <code>data</code> .
preproc	none linear	none	Use a linear preprocessing of the color data values, which you specify using the scaling and shift properties.

TABLE 7-164: VALID PROPERTIES FOR TABLECOLOR.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
scaling	double	1	Real scaling value for data preprocessing, if preproc is linear.
shift	double	0	Real shift value for data preprocessing, if preproc is linear.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.

SEE ALSO

[TableArrow](#), [Color](#)

TableContour

Create a table contour plot for 2D plot groups.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"TableContour");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"TableContour")` creates a table contour plot feature named `<ftag>` belonging to the 2D plot group `<pgtag>`.

Table contour plots are used to visualize data from a table or evaluation group as a, possibly filled, 2D contour plot.

The following properties are available:

TABLE 7-165: VALID PROPERTIES FOR TABLE CONTOUR PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the contours: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
contourlabels	on off	off	Whether to display labels next to the contour lines. Only applicable if contourtype is lines.
contourtype	lines filled	lines	Whether to display lines for each level or a surface with bands of color.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform bottom color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform top color to use. Active when topcolor is set to custom.
datacol	positive integer	1	For tables with filled data, datacol controls which data column should be used to populate the filled table.
dataformat	filledtable columns cells	Model dependent	The format of the table data. The default value depends on the table data: Filled table if the data in the table is structurally filled; otherwise, columns if the table contains at least three columns; otherwise, cells. For evaluation groups, filledtable is not available.

TABLE 7-165: VALID PROPERTIES FOR TABLE CONTOUR PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
descr	none data manual	data	For tables with filled data, descr controls which description to use for the columns, in addition to the parameter values. none gives no description, manual uses the description in descrmanual, and data uses the description in the table data.
descrmanual	String	Empty	Manual column header for tables with filled data, used when descr is set to manual.
evaluationgroup	none evaluation group feature name	First available evaluation group	The evaluation group feature this plot refers to, if source is evaluationgroup.
filterstringdata	on off	off	For tables with filled data, filterstringdata controls whether to filter the input column using showrowstep.
format	all filled	all	Controls whether you retrieve all or filled table data when calling the getTableData method.
imagplot	on off	off	If transform is none: When on, the imaginary part of the data in the table is plotted.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritheightscale	boolean	true	If inheritplot is not none: Determines if height scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, and height scale is inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
labelcolor	custom fromtheme black blue cyan gray green magenta red white yellow	fromtheme	If contourlabels is on: The label color to use. The default value uses a color from the color theme.
labelprec	positive	integer	If contourlabels is on: The number of significant digits in the labels.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
legendtype	auto filled lines	auto	Whether to show the color legend as a filled legend or using lines. The auto setting provides a filled legend for filled contours and a line legend for line and tube contours.
levelmethod	number levels	number	How to specify contour levels.

TABLE 7-165: VALID PROPERTIES FOR TABLE CONTOUR PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
levelrounding	boolean	true	Round the contour level values, if levelmethod is set to number.
levels	String array		The specific levels to plot. Active when levelmethod equals levels.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
param	positive integer	1	For tables with filled data containing more than two parameters, param is the index indicating which values to use for fixing additional parameters. Additional parameters are ordered from left to right in the original table data.
plotdata	table manual	table	For tables with filled data containing more than two parameters, plotdata controls whether to use manual settings in the plot (table row, column, and so forth), or the filled settings in the table feature. In the latter case, the table's properties are automatically synchronized with the plot's properties.
preproc{x y data}	none linear	none	Use a linear preprocessing of the table data values, which you specify using the scaling and shift properties.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maxim value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
scaling{x y data}	double	1.0	Real scaling value for data preprocessing.

TABLE 7-165: VALID PROPERTIES FOR TABLE CONTOUR PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
shift{x y data}	double	0.0	Real shift value for data preprocessing.
showrowstep	positive integer	1	When retrieving filled data using the <code>getTableData</code> method, <code>showrowstep</code> controls whether to leave all but every <code>showrowstep</code> input cells empty, for example, every third cell in the first column. This refers to the input column only. Only applied when <code>filterstringdata</code> is on.
source	table evaluationgroup	table	Use table data from a table or an evaluation group.
table	none table feature name	First available table	The table feature this plot refers to, if source is table.
title	String	The auto-title.	The title to use when <code>titletype</code> is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the <code>title</code> property). none, if no title contribution should be used.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
xaxisdata	Parameters in table none	none	For tables with filled data, <code>xaxisdata</code> controls which parameter is used on the x-axis.
yaxisdata	Parameters in table none	none	For tables with filled data, <code>yaxisdata</code> controls which parameter is used on the y-axis. Cannot be the same parameter as in <code>xaxisdata</code> .
wireframe	on off	off	Whether to plot filled elements or only their edges,

ATTRIBUTES

[Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#); [Marker](#); [MaterialAppearance](#); [Transparency](#) (if a height attribute is used)

SEE ALSO

[Table](#), [Table \(Plot\)](#), [TableAnnotation](#), [TableSurface](#), [Contour \(Plot\)](#), [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#)

TableDOE

Create a feature containing a table of data for design of experiments (DOE).

SYNTAX

```
model.result().table().create(<ftag>,"TableDOE");
model.result().table(<ftag>).setColumnHeaders(<headers>);
model.result().table(<ftag>).setTableData(<realData>);
model.result().table(<ftag>).setTableData(<realData>,<imagData>);
model.result().table(<ftag>).addColumnns(<headers>,<realData>);
model.result().table(<ftag>).addColumnns(<headers>,<realData>,<imagData>);
model.result().table(<ftag>).addRow(<realData>);
model.result().table(<ftag>).addRow(<realData>,<imagData>);
model.result().table(<ftag>).addRows(<realData>);
model.result().table(<ftag>).addRows(<realData>,<imagData>);
model.result().table(<ftag>).addRowsAndWarnings(<realData>,<warnings>));
model.result().table(<ftag>).addRowAndWarnings(<realData>,<imagData>,<warnings>);
model.result().table(<ftag>).removeRow(<index>);
model.result().table(<ftag>).removeRow(<index_array>);
model.result().table(<ftag>).getColumnHeaders();
model.result().table(<ftag>).getReal();
model.result().table(<ftag>).getRowHeaders()
model.result().table(<ftag>).getImag();
model.result().table(<ftag>).Row(<index>);
model.result().table(<ftag>).getImagRow(<index>);
model.result().table(<ftag>).getTableData(<fullPrecision>);
model.result().table(<ftag>).getTableRow(<index>,<fullPrecision>);
model.result().table(<ftag>).getNRows();
model.result().table(<ftag>).isComplex();
model.result().table(<ftag>).clearTableData();
model.result().table(<ftag>).set(property,<value>);
model.result().table(<ftag>).save(<filename>);
model.result().table(<ftag>).save(<filename>,<fullPrecision>);
model.result().table(<ftag>).loadFile(tempFile,...);
model.result().table(<ftag>).saveFile(tempFile,...);
```

DESCRIPTION

`model.result().table().create(<ftag>,"TableDOE")` creates a table feature for DOE named `<ftag>`. Tables support two data formats, *all* or *filled*. Filled data is typically produced from parametric sweeps and makes it possible to retrieve data for a pair of parameters on a matrix format. Filled tables can be used to make response surface plots. (See [TableSurface](#).)

The following properties are available:

TABLE 7-166: VALID PROPERTIES FOR TABLE DOE.

NAME	VALUE	DEFAULT	DESCRIPTION
cols	parameters in table none	none	For tables with filled data, cols controls which parameter is used on the columns. Cannot be the same parameter as in rows.
datacol	positive integer	1	For tables with filled data, datacol controls which data column should be used to populate the filled table.
descr	none data manual	data	For tables with filled data, descr controls which description to use for the columns, in addition to the parameter values. none gives no description, manual uses the description in descrmanual, and data uses the description in the table data.
descrmanual	String	Empty	Manual column header for tables with filled data, used when descr is set to manual.

TABLE 7-166: VALID PROPERTIES FOR TABLE DOE.

NAME	VALUE	DEFAULT	DESCRIPTION
filename	String	Empty	The name of the file on which the table is stored, used when storetable is set to inmodelandonfile or onfile.
filterstringdata	on off	off	For tables with filled data, filterstringdata controls whether to filter the input column using showrowstep.
format	all filled	all	Controls whether you retrieve all or filled table data when calling the getTableData method.
param	positive integer	1	For tables with filled data containing more than two parameters, param is the index indicating which values to use for fixing additional parameters. Additional parameters are ordered from left to right in the original table data.
rows	parameters in table none	none	For tables with filled data, rows controls which parameter is used on the rows.
showrowstep	positive integer	1	When retrieving filled data using the getTableData method, showrowstep controls whether to leave all but every showrowstep input cells empty, for example, every third cell in the first column. This refers to the input column only. Only applied when filterstringdata is on.
storetable	inmodel inmodelandonfile onfile	inmodel	Controls whether table is stored in model(inmodel) or on file(onfile), or both(inmodelandonfile).
tablebuffersize	positive integer	Taken from buffer size preference. The default of which is 10000.	The size of the in-memory buffer size where the table is stored, used when storetable is set to inmodel or inmodelandonfile. In the second case only the last tablebuffersize rows are kept in the model; the rest is read from file when necessary.

SEE ALSO

[Table](#), [Table \(Plot\)](#), [TableAnnotation](#), [TableUQ](#), [TableSurface](#)

TableErrorBars

A plot attribute for adding error bars to 1D table graph plots.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"TableErrorBars");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"TableErrorBars")` creates error bars as a plot attribute named <atag> to a 1D table graph named <ftag>.

This plot attribute displays error bars in the parent graph.

The following properties are available:

TABLE 7-167: VALID PROPERTY/VALUE PAIRS FOR A TABLEERRORBARS ATTRIBUTE.

NAME	VALUE	DEFAULT	DESCRIPTION
coldown	"auto" or column number	"auto"	The column for the lower bound.
colup	"auto" or column number	"auto"	The column for the upper bound.
customlinecolor	RGB-triplet	{0,0,0} or last used linecolor	The color to use for the error bars. Active when color is set to custom.
color	custom fromtheme black blue cyan gray green magenta red white yellow	black	How to color the lines or area for the error bars.
linestyle	solid dotted dashed dashdot	solid	The line style, if type is set to bar.
linewidth	integer preference	1	The line width as an integer between 1 and 10.
scalingdown	scalar	1	Scaling of the column values for the lower bound.
scalingup	scalar	1	Scaling of the column values for the upper bound.
type	bar filled	bar	The type of plot: error bars or a filled area for the error.

SEE ALSO

[ErrorBars](#), [Marker](#), [GraphMarker](#)

TableHistogram

Create a table histogram plot for 1D and 2D plot groups.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"TableHistogram");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"TableHistogram")` creates a table histogram plot feature named <ftag> belonging to the plot group <pgtag>.

Table histograms are used to visualize the distribution of the range of data taken from a table or evaluation group. The result is a plot with the data range on the x-axis and the data length, area, or volume on the y-axis. Such histogram plots can be added to 1D plot groups. In 2D histograms, which you can add to 2D plot groups, the x-axis and y-axis represent the values of two quantities (as a number of bins or a range of values), and the color surface represents the count of the total element volume in each “bin”.

The following properties are available:

TABLE 7-168: VALID PROPERTIES FOR TABLE HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
cumulative	boolean	false	Plot a cumulative histogram.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
evaluationgroup	Tag of evaluation group or none	none	The evaluation group to take the data from, when source is set to evaluationgroup.
function	continuous discrete	continuous	Use a continuous function or a discrete function (that is, using a constant level in each bin) for the histogram.
limits	double array	0 1	The bin limits if method is limits.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.

TABLE 7-168: VALID PROPERTIES FOR TABLE HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
measure	auto integral count	auto	The measure for evaluation of the histogram data. Choose <code>integral</code> for volume-based data; choose <code>count</code> of element-based data. <code>auto</code> is <code>count</code> for Mesh datasets; <code>integral</code> otherwise.
method	limit number	number	The method used to specify the bins in which the expression's range is split
normalization	integralsum integralscaled none peak	none	The normalization of the values plot on the y-axis.
number	nonnegative integer	10	The number of bins if method is <code>number</code> .
numberofcolors	integer	7	The number of colors in the color cycle when <code>cyclelength</code> is <code>manual</code> .
prefixintitle	String	Empty	Added prefix to contribution to title
rangecoloractive	on off	off	Whether to use the manual color range specified in <code>rangecolormin</code> and <code>rangecolormax</code> . The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values. This and other range settings are available for 2D Histogram plots.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when <code>linecolor</code> is <code>cyclecolortable</code> .
source	table evaluationgroup	table	The data source for the histogram: a table or an evaluation group.
suffixintitle	String	Empty	Added suffix to contribution to title
table	Tag of table or none	none	The table to take the data from, when <code>source</code> is set to <code>table</code> .
title	String	The auto-title.	The title to use when <code>titletype</code> is <code>manual</code> .
titletype	auto custom label manual none	auto	<code>auto</code> , if the title should be computed automatically. <code>custom</code> , if the title should be computed automatically, but customized. <code>label</code> , if the title should be the plot group's label. <code>manual</code> , if the manual title should be used (the <code>title</code> property). <code>none</code> , if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when <code>linecolor</code> is <code>cyclegradient</code> .
type	curve solid	curve	Use a filled histogram (<code>solid</code>) or a histogram drawn using lines (<code>curve</code>). This property is only available when function is discrete.
typeintitle	on off	on	Whether the title contribution should contain the type when <code>titletype</code> is <code>custom</code> .

TABLE 7-168: VALID PROPERTIES FOR TABLE HISTOGRAM PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xaxisdata	Column integer		An integer representing the table column to use as the x-coordinate.
yaxisdata	Column integer		An integer representing the table column to use as the y-coordinate (2D only).

ATTRIBUTES

[Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#) (2D only); [Transparency](#) (if a height attribute is used)

SEE ALSO

[Histogram](#), [MatrixHistogram](#)

TablePoint

Plot data from a table in a point plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"TablePoint");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"TablePoint")` creates a table point plot feature named `<ftag>`.

Table point plots can be used with tables to plot their data as points. They can be added to 2D plot groups.

The following properties are available:

TABLE 7-169: VALID PROPERTIES FOR TABLE POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bandcount	positive integer	10	The number of discrete color bands, if <code>colortabletype</code> is set to discrete.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the surface: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.

TABLE 7-169: VALID PROPERTIES FOR TABLE POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
colortabletype	continuous discrete	continuous	The color table type: continuous or discrete.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform bottom color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform top color to use. Active when topcolor is set to custom.
datacol	positive integer	1	For tables with filled data, datacol controls which data column should be used to populate the filled table.
dataformat	filledtable columns cells	Model dependent	The format of the table data. The default value depends on the table data: Filled table if the data in the table is structurally filled; otherwise, columns if the table contains at least three columns; otherwise, cells. For evaluation groups, filledtable is not available.
descr	none data manual	data	For tables with filled data, descr controls which description to use for the columns, in addition to the parameter values. none gives no description, manual uses the description in descrmanual, and data uses the description in the table data.
descrmanual	String	Empty	Manual column header for tables with filled data, used when descr is set to manual.
evaluationgroup	none evaluation group feature name	First available evaluation group	The evaluation group feature this plot refers to, if source is evaluationgroup.
filterstringdata	on off	off	For tables with filled data, filterstringdata controls whether to filter the input column using showrowstep.
fixedpointsize	boolean	false	Use constant radii for the plotted points.

TABLE 7-169: VALID PROPERTIES FOR TABLE POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
format	all filled	all	Controls whether you retrieve all or filled table data when calling the <code>getTableData</code> method.
function	continuous discrete	continuous	Considering the table data as samples of a continuous function or treating the samples as discrete and draw them as large pixels.
imagplot	on off	off	If transform is none: When on, the imaginary part of the data in the table is plotted.
inheritcolor	boolean	true	If <code>inheritplot</code> is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that color, color range, and height scale is inherited from.
inheritrange	boolean	true	If <code>inheritplot</code> is not none: Determines if the color and data ranges are inherited.
inheritspherescale	boolean	true	If <code>inheritplot</code> is not none: Determines if the sphere scale factor is inherited.
legendtitle	String		Color legend title, if <code>showlegendstitle</code> has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to <code>colortable</code> or <code>gradient</code> and <code>colortabletrans</code> is set to <code>nonlinear</code> or <code>nonlinearsymmetric</code> .
param	positive integer	1	For tables with filled data containing more than two parameters, <code>param</code> is the index indicating which values to use for fixing additional parameters. Additional parameters are ordered from left to right in the original table data.
plotdata	table manual	table	For tables with filled data containing more than two parameters, <code>plotdata</code> controls whether to use manual settings in the plot (table row, column, and so forth), or the filled settings in the table feature. In the latter case, the table's properties are automatically synchronized with the plot's properties.
preproc{x y data}	none linear	none	Use a linear preprocessing of the table data values, which you specify using the scaling and shift properties.

TABLE 7-169: VALID PROPERTIES FOR TABLE POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
scaling{x y data}	double	1.0	Real scaling value for data preprocessing.
shift{x y data}	double	0.0	Real shift value for data preprocessing.
showrowstep	positive integer	1	When retrieving filled data using the getTableData method, showrowstep controls whether to leave all but every showrowstep input cells empty, for example, every third cell in the first column. This refers to the input column only. Only applied when filterstringdata is on.
source	table evaluationgroup	table	Use table data from a table or an evaluation group.
table	none table feature name	First available table	The table feature this plot refers to, if source is table.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the title property). none, if no title contribution should be used.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.

TABLE 7-169: VALID PROPERTIES FOR TABLE POINT PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
xaxisdata	Parameters in table none	none	For tables with filled data, xaxisdata controls which parameter is used on the x-axis.
yaxisdata	Parameters in table none	none	For tables with filled data, yaxisdata controls which parameter is used on the y-axis. Cannot be the same parameter as in xaxisdata.

ATTRIBUTE[Marker](#)**SEE ALSO**[Table](#), [Table \(Plot\)](#), [TableAnnotation](#), [TableContour](#), [TableSurface](#), [Point](#)*TableSurface*

Plot data from a table in a surface plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"TableSurface");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"TableSurface")` creates a table surface plot feature named `<ftag>`.

Table surface plots can only be used with tables containing filled data, produced from a parametric sweep. They can be added to 2D plot groups.

The following properties are available:

TABLE 7-170: VALID PROPERTIES FOR TABLE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bandcount	positive integer	10	The number of discrete color bands, if <code>colortabletype</code> is set to discrete.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the surface: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.

TABLE 7-170: VALID PROPERTIES FOR TABLE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
colorscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
colortabletype	continuous discrete	continuous	The color table type: continuous or discrete.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform bottom color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform top color to use. Active when topcolor is set to custom.
datacol	positive integer	1	For tables with filled data, datacol controls which data column should be used to populate the filled table.
dataformat	filledtable columns cells	Model dependent	The format of the table data. The default value depends on the table data: Filled table if the data in the table is structurally filled; otherwise, columns if the table contains at least three columns; otherwise, cells. For evaluation groups, filledtable is not available.
descr	none data manual	data	For tables with filled data, descr controls which description to use for the columns, in addition to the parameter values. none gives no description, manual uses the description in descrmanual, and data uses the description in the table data.
descrmanual	String	Empty	Manual column header for tables with filled data, used when descr is set to manual.
evaluationgroup	none evaluation group feature name	First available evaluation group	The evaluation group feature this plot refers to, if source is evaluationgroup.
filterstringdata	on off	off	For tables with filled data, filterstringdata controls whether to filter the input column using showrowstep.
format	all filled	all	Controls whether you retrieve all or filled table data when calling the getTableData method.

TABLE 7-170: VALID PROPERTIES FOR TABLE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
function	continuous discrete	continuous	Considering the table data as samples of a continuous function or treating the samples as discrete and draw them as large pixels.
imagplot	on off	off	If transform is none: When on, the imaginary part of the data in the table is plotted.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritheightscale	boolean	true	If inheritplot is not none: Determines if height scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, and height scale is inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
param	positive integer	1	For tables with filled data containing more than two parameters, param is the index indicating which values to use for fixing additional parameters. Additional parameters are ordered from left to right in the original table data.
plotdata	table manual	table	For tables with filled data containing more than two parameters, plotdata controls whether to use manual settings in the plot (table row, column, and so forth), or the filled settings in the table feature. In the latter case, the table's properties are automatically synchronized with the plot's properties.
preproc{x y data}	none linear	none	Use a linear preprocessing of the table data values, which you specify using the scaling and shift properties.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.

TABLE 7-170: VALID PROPERTIES FOR TABLE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
scaling{x y data}	double	1.0	Real scaling value for data preprocessing.
shift{x y data}	double	0.0	Real shift value for data preprocessing.
showrowstep	positive integer	1	When retrieving filled data using the getTableData method, showrowstep controls whether to leave all but every showrowstep input cells empty, for example, every third cell in the first column. This refers to the input column only. Only applied when filterstringdata is on.
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
source	table evaluationgroup	table	Use table data from a table or an evaluation group.
table	none table feature name	First available table	The table feature this plot refers to, if source is table.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0–1), if threshold is set to manual.
title	String	The auto-title.	The title to use when titletype is manual.

TABLE 7-170: VALID PROPERTIES FOR TABLE SURFACE PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
titletype	auto manual none	auto	auto, if the title contribution should be computed automatically, possibly using the group's customization. manual, if the manual title contribution should be used (the title property). none, if no title contribution should be used.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
xaxisdata	Parameters in table none	none	For tables with filled data, xaxisdata controls which parameter is used on the x-axis.
yaxisdata	Parameters in table none	none	For tables with filled data, yaxisdata controls which parameter is used on the y-axis. Cannot be the same parameter as in xaxisdata.
wireframe	on off	off	Whether to plot filled elements or only their edges,

ATTRIBUTES

[Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#); [MaterialAppearance](#); [Marker](#); [Transparency](#) (if a height attribute is used)

SEE ALSO

[Table](#), [Table \(Plot\)](#), [TableAnnotation](#), [TableContour](#), [TablePoint](#), [Surface \(Plot\)](#), [Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#)

TableUQ

Create a feature containing a table of data for uncertainty quantification (UQ).

SYNTAX

```

model.result().table().create(<ftag>,"TableUQ");
model.result().table(<ftag>).setColumnHeaders(<headers>);
model.result().table(<ftag>).setTableData(<realData>);
model.result().table(<ftag>).setTableData(<realData>,<imagData>);
model.result().table(<ftag>).addColumnns(<headers>,<realData>);
model.result().table(<ftag>).addColumnns(<headers>,<realData>,<imagData>);
model.result().table(<ftag>).addRow(<realData>);
model.result().table(<ftag>).addRow(<realData>,<imagData>);
model.result().table(<ftag>).addRows(<realData>);
model.result().table(<ftag>).addRows(<realData>,<imagData>);
model.result().table(<ftag>).addRowsAndWarnings(<realData>,<warnings>));
model.result().table(<ftag>).addRowAndWarnings(<realData>,<imagData>,<warnings>);
model.result().table(<ftag>).removeRow(<index>);
model.result().table(<ftag>).removeRow(<index_array>);
model.result().table(<ftag>).getColumnHeaders();
model.result().table(<ftag>).getReal();
model.result().table(<ftag>).getRowHeaders()
model.result().table(<ftag>).getImag();
model.result().table(<ftag>).Row(<index>);
model.result().table(<ftag>).getImagRow(<index>);
model.result().table(<ftag>).getTableData(<fullPrecision>);
model.result().table(<ftag>).getTableRow(<index>,<fullPrecision>);
model.result().table(<ftag>).getNRows();
model.result().table(<ftag>).isComplex();
model.result().table(<ftag>).clearTableData();
model.result().table(<ftag>).set(property,<value>);
model.result().table(<ftag>).save(<filename>);
model.result().table(<ftag>).save(<filename>,<fullPrecision>);
model.result().table(<ftag>).loadFile(tempFile,...);
model.result().table(<ftag>).saveFile(tempFile,...);

```

DESCRIPTION

`model.result().table().create(<ftag>,"TableDOE")` creates a table feature for UQ named `<ftag>`. Tables support two data formats, *all* or *filled*. Filled data is typically produced from parametric sweeps and makes it possible to retrieve data for a pair of parameters on a matrix format. Filled tables can be used to make response surface plots. (See [TableSurface](#).)

The following properties are available:

TABLE 7-171: VALID PROPERTIES FOR TABLE UQ.

NAME	VALUE	DEFAULT	DESCRIPTION
cols	parameters in table none	none	For tables with filled data, cols controls which parameter is used on the columns. Cannot be the same parameter as in rows.
datacol	positive integer	1	For tables with filled data, datacol controls which data column should be used to populate the filled table.
descr	none data manual	data	For tables with filled data, descr controls which description to use for the columns, in addition to the parameter values. none gives no description, manual uses the description in descrmanual, and data uses the description in the table data.
descrmanual	String	Empty	Manual column header for tables with filled data, used when descr is set to manual.

TABLE 7-171: VALID PROPERTIES FOR TABLE UQ.

NAME	VALUE	DEFAULT	DESCRIPTION
filename	String	Empty	The name of the file on which the table is stored, used when storetable is set to inmodelandonfile or onfile.
filterstringdata	on off	off	For tables with filled data, filterstringdata controls whether to filter the input column using showrowstep.
format	all filled	all	Controls whether you retrieve all or filled table data when calling the getTableData method.
param	positive integer	1	For tables with filled data containing more than two parameters, param is the index indicating which values to use for fixing additional parameters. Additional parameters are ordered from left to right in the original table data.
rows	parameters in table none	none	For tables with filled data, rows controls which parameter is used on the rows.
showrowstep	positive integer	1	When retrieving filled data using the getTableData method, showrowstep controls whether to leave all but every showrowstep input cells empty, for example, every third cell in the first column. This refers to the input column only. Only applied when filterstringdata is on.
storetable	inmodel inmodelandonfile onfile	inmodel	Controls whether table is stored in model(inmodel) or on file(onfile), or both(inmodelandonfile).
tablebuffersize	positive integer	Taken from buffer size preference. The default of which is 10000.	The size of the in-memory buffer size where the table is stored, used when storetable is set to inmodel or inmodelandonfile. In the second case only the last tablebuffersize rows are kept in the model; the rest is read from file when necessary.

SEE ALSO

[Table](#), [Table \(Plot\)](#), [TableAnnotation](#), [TableDOE](#), [TableSurface](#)

TimeAverage, TimeIntegral

Create a time average or time integral dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"TimeAverage");
model.result().dataset().create(<dtag>,"TimeIntegral");
model.result().dataset(<dtag>).set(property,<value>);
```


DESCRIPTION

`model.result().dataset().create(<dtag>,"TimeAverage")` creates a time average dataset with the name <dtag>.

`model.result().dataset().create(<dtag>,"TimeIntegral")` creates a time integral dataset with the name <dtag>.

The time average and time integral datasets make it possible to compute time averages and time integrals of some time-dependent data in another dataset, for example.

The following properties are available:

TABLE 7-172: VALID PROPERTY/VALUE PAIRS FOR TIME AVERAGE AND TIME INTEGRAL DATASETS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
data	none dataset name	First compatible dataset	The dataset this feature refers to.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. manual indicates that solnum is used. manualindices indicates that solnumindices is used. interp indicates that t is used.
minlen	nonnegative double	1e-4	Minimum interval length relative to the length of the time interval, if usetimeinterpolation is set to true.
rtol	nonnegative double	1e-3	Relative tolerance, if usetimeinterpolation is set to true.
solnum	Integer array	All solutions	The solutions to use for extrapolation.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
t	double array	Empty	The times to use as intermediate layers in the extrusion.
usetimeinterpolation	true false	true	Use interpolation between stored time steps.

ThroughThickness

Create a through-thickness plot to display the variation of a layered shell quantity in its thickness direction at specified points.



The ThroughThickness plot requires a license for the AC/DC Module, Composite Materials Module, Heat Transfer Module, or Structural Mechanics Module.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"ThroughThickness");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"ThroughThickness")` creates a through-thickness plot feature named <ftag> belonging to the plot group <pgtag>.

A through-thickness plot is used to visualize the variation of a layered shell quantity in its thickness direction. Through-thickness plots can be added to 1D plot groups. The following properties are available:

TABLE 7-173: VALID PROPERTIES FOR THROUGH-THICKNESS PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
autodescr	true false	Model dependent	Whether the automatic legends should include the expression descriptions.
autoexpr	true false	Model dependent	Whether the automatic legends should include the expressions.
autoplotlabel	true false	false	Whether the automatic legends should include the plot label.
autopoint	true false	false	Whether the automatic legends should include the point.
autosolution	true false	false	Whether the automatic legends should include the solution.
autounit	true false	false	Whether the automatic legends should include the unit.
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
colorcycle	default long	default	The color cycle, when linecolor is set to cycle. With default, it contains 7 colors; with long, it contains 19 colors.
colortable	color table name	Rainbow	The color table to use when linecolor is set to cyclecolortable. See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customlinecolor	RGB-triplet	{0,0,1} or last used edgcolor	The color to use for the lines. Active when linecolor is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
cyclelength	auto manual	auto	The length of color cycle when linecolor is cyclegradient.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.

TABLE 7-173: VALID PROPERTIES FOR THROUGH-THICKNESS PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when <code>titletype</code> is custom.
includeinterfaces	off all exterior	off	Whether to include interface positions in the plot: No interfaces, all interfaces, or interfaces between layered materials.
innerinput	all first last manual manualindices interp	all	How to input the solution to use. <code>manual</code> indicates that <code>solnum</code> is used. <code>manualindices</code> indicates that <code>solnumindices</code> is used. <code>interp</code> indicates that <code>t</code> is used.
interfacecolor	custom black blue cyan gray green magenta red white yellow	magenta	How to color the interface positions, when <code>includeinterfaces</code> is true.
interfacelinestyle	solid dotted dashed dashdot	dotted	The line style for the lines representing the interface positions, when <code>includeinterfaces</code> is true.
interp	double row matrix	Empty on all levels	The times to use, for transient levels. Available when data is not parent and the underlying data is transient.
legend	on off	off	Whether to show legends.
legendexprprecision	positive integer	3	The precision for the evaluated expression when <code>legendmethod</code> is set to evaluated.
legendmethod	automatic manual evaluated	automatic	Whether to use the automatic legends or the legends added manually or evaluated.
legendpattern	String		A legend string to be evaluated when <code>legendmethod</code> is set to evaluated.

TABLE 7-173: VALID PROPERTIES FOR THROUGH-THICKNESS PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
legendprefix	String		A prefix added to the automatic legend.
legends	String array	The last computed automatic legends	Manual legends active when legendmethod is set to manual.
legendsuffix	String		A suffix added to the automatic legend.
linecolor	custom cycle cyclereset cyclecolortable cyclegradient fromtheme black blue cyan gray green magenta red white yellow	cycle	How to color the lines in the graph. Cycle indicates that the colors is different for each line. Use cyclereset to restart the cycling of colors from the first color. Use fromtheme to take the color from the selected color theme.
linewidth	double preference	1	The line width.
linemarker	none cycle asterisk circle diamond plus point square star triangle	none	The line markers, if any. Cycle indicates that the marker is different for each line.
looplevel	integer row matrix	All solutions on all levels	The solutions to use, per level.
looplevelindices	integer row matrix	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range (1, 1, 20). Applicable when looplevelinput is manualindices on a level.
looplevelinput	String array with all first last manual manualindices interp on each level	all on all levels	How to input the solution to use, per level. manual on a level indicates that looplevel is used on that level. manualindices on a level indicates that looplevelindices is used on that level. interp on a level indicates that interp is used on that level.
markerpos	interp datapoints	interp	Controls whether the positions of the markers are in the data points of the plot, or interpolated depending on the number (set in markers). Markers are visible when linemarker is set.
markers	integer	8	The number of markers to show. Markers are visible when linemarker is set.
numberofcolors	integer	7	The number of colors in the color cycle when cyclelength is manual.
linestyle	none cycle solid dotted dashed dashdot	solid	The line style, if any. Cycle indicates that the line style is different for each line.

TABLE 7-173: VALID PROPERTIES FOR THROUGH-THICKNESS PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
outerinput	all first last manual manualindices	all	How to input the outer solutions to use. Applicable only for parametric sweep models. manual indicates that outersolnum is used. manualindices indicates that outersolnumindices is used.
outersolnum	nonnegative integer array	1	The index of the outer solutions to use. Applicable only for parametric sweep models. Since the various outer solutions can have different number of solnum, the solnum property is not used.
outersolnumindices	String or integer array	Empty	An alternative way to specify the outer solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when outerinput is manualindices.
posentry	coordinates selection	selection	Determine the position using a selection or by providing coordinates.
plotonsecyaxis	boolean	true	Plot on secondary y-axis, if twoyaxes is set to true in the parent plot group.
point3x	scalar	0	The x-coordinate for the position when posentry is set to coordinates.
point3y	scalar	0	The y-coordinate for the position when posentry is set to coordinates.
point3z	scalar	0	The z-coordinate for the position when posentry is set to coordinates.
prefixintitle	String	Empty	Added prefix to contribution to title.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.

TABLE 7-173: VALID PROPERTIES FOR THROUGH-THICKNESS PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
reversecolortable	boolean	false	Whether to reverse the order of the colors in the color table. Active when linecolor is cyclecolortable.
smooth	true false	false	Apply smoothing to the plot data, when evaluationsettings is set to manual.
solnum	integer array	All solutions	The solutions to plot. Available when data is not parent and the underlying data has multiple solutions.
solnumindices	String or integer array	Empty	An alternative way to specify the solutions to use, allowing you to enter, for example, range(1,1,20). Applicable when innerinput is manualindices.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double array	Empty	The times to plot. Available when data is not parent and the underlying solution is transient.
thicknesscoordinateunit	length unit	The default length unit	The length unit to use if ydata is set to thicknesscoordinate.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The bottom color to use for a color gradient. Active when linecolor is cyclegradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-173: VALID PROPERTIES FOR THROUGH-THICKNESS PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the <code>unit</code> property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when <code>titletype</code> is custom.
xdata	arc reversedarc <code>expr</code>	solution	x-axis data. <code>expr</code> uses the expression in <code>xdataexpr</code> . <code>arc</code> uses the curve's arc length, and <code>reversedarc</code> uses the arc length measured from the curve's endpoint.
xdataexpr	String	Model dependent	Expression for x-axis data.
xdatadescr	String	Model dependent	Description of expression in <code>xdataexpr</code> .
xdataunit	String	Model dependent	The unit to use for the expression in <code>xdataexpr</code> . If the old unit is not valid when the expression changes, the <code>unit</code> property is reset to default.
ydata	<code>expr</code> <code>thicknesscoordinate</code>	thickness coordinate	Take the y-data from an expression or as the thickness coordinate.
ydataexpr	String	Model dependent	Expression for y-axis data when <code>ydata</code> is set to <code>expr</code> .
ydatadescr	String	Model dependent	Description of expression in <code>ydataexpr</code> .

ATTRIBUTES[Color](#)**SEE ALSO**[LineGraph](#)*Touchstone*

Export data for parameters in port networks as Touchstone files.

SYNTAX

```
model.result().export().create(<ftag>,"Touchstone");
model.result().export().create(<ftag>,<dtag>,"Touchstone");
model.result().export(<ftag>).set(property,<value>);
model.result().export(<ftag>).run();
```

DESCRIPTION

`model.result().export().create(<ftag>,"Touchstone")` creates a Touchstone export feature with the name `<ftag>`.

`model.result().export().create(<ftag>,<dtag>,"Touchstone")` creates a Touchstone export feature with the name `<ftag>` for the dataset `<dtag>`.

The following properties are available:

TABLE 7-174: VALID PROPERTIES FOR TOUCHSTONE EXPORT.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
alwaysask	on off	off	Always ask for filename when saving. This property is ignored when running without a UI.
config	configuration name		The configuration to use for the solution times or parameters, if solutionparams is set to configuration.
data	String		The name of the solution dataset to export.
filename	String		The output file.
format	String ("MA", "RI", or "DB")	MA	The output format: Magnitude and angel (MA), real and imaginary parts (RI), or magnitude in dB and angle (DB).
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
parameter	String		One of "S", "Y", "Z", "G", and "H", depending on the support network parameters in the physics interface.
physicsinterface	Physics interface name (string)		The physics interfaces from which to take the data.
solnum	integer array		Solution number to take values from.
solutionparams	manual configuration	manual	Specify solution parameters directly (manual) or from a configuration.

SEE ALSO

[Data](#)

Transformation2D, Transformation3D

Create a transformation 2D or transformation 3D dataset. Use a transformation dataset to scale, rotate, and move a geometry in another dataset. If you use multiple transformations, note that they are performed in that order and that they do not always commute. If you need a transformation in another order, use multiple transformation datasets in succession with only one type of transformation in each dataset.

SYNTAX

```
model.result().dataset().create(<dtag>,"Transformation2D");
model.result().dataset().create(<dtag>,"Transformation3D");
model.result().dataset(<dtag>).setIndex(...);
model.result().dataset(<dtag>).set(property,<value>);
```

See [set\(\)](#), [setIndex\(\)](#), and [Methods Associated to Set, SetIndex, and the Various Get Methods](#) for more information.

DESCRIPTION

`model.result().dataset().create(<dtag>,"Transformation2D")` creates a 2D transformation dataset feature named `<dtag>`.

`model.result().dataset().create(<dtag>,"Transformation3D")` creates a 3D transformation dataset feature named `<dtag>`.

This dataset takes a geometry from another dataset and adds a transformation that can include scaling, rotation, and translation (moving).

The following properties are available for Transformation2D and Transformation3D:

TABLE 7-175: VALID PROPERTIES FOR TRANSFORMATION DATASETS (IF NOT NOTED, PROPERTIES ARE FOR THE STRUCTURED TRANSFORMATION TYPE).

NAME	VALUE	DEFAULT	DESCRIPTION
ax2	double array	{0,0}	Theta and phi angles for the rotation, if type is set to linear and axistype is set to spherical for a Rotation3D dataset, and if enablerot is true.
ax3	double array	{0, 0, 0}	Axis coordinates (x, y, and z) for the rotation, if type is set to linear and axistype is set to cartesian for a Rotation3D dataset, and if enablerot is true.
axistype	x y z cartesian spherical	z	The axis type for a rotation, if type is set to linear for a Rotation3D dataset, if enablerot is true.
data	none dataset name	First compatible dataset	The dataset this feature refers to
enablemove	boolean	false	Enable moving (translating) the geometry.
enablerot	boolean	false	Enable rotating the geometry.
enablescale	boolean	false	Enable scaling of the geometry.
move	double array	{0, 0} (2D), {0, 0, 0} (3D)	The move (translation), if transtype is general, if enablemove is true.
rotangle	double	0	The angle increment for a rotation, if enablerot is true.
rotcenter	double array	{0, 0}	The rotation center for a Rotation2D dataset.
scale	Numerical array	Depends on the feature's tag	The scaling in the x, y (2D and 3D), and z (3D only) directions, if enablescale is true.
spacevars	String array	Depends on the feature's tag	The name of space variables, which evaluate to the coordinates after the transformation.
translation	double array	{0, 0} (2D), {0, 0, 0} (3D)	The translation, if transtype is general.
transmatrix	double array	2-by-2 or 3-by-3 matrix of zeros	The base vectors of a transformation, specified using setIndex syntax, if transtype is general.
transtype	structured general	structured	The transformation type.

SEE ALSO

[Transformation](#)

Transformation

Transform (scale, move, and rotate) some 2D and 3D plot types.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Transformation");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

model.result(<pgtag>).feature(<ftag>).create(<atag>,"Transformation") creates a transformation attribute feature with the name <atag>, belonging to the feature <ftag>.

Transformation attributes can transform the coordinates of a plot feature by specified values for scaling, rotation, and moving. The transformation attribute can be added to arrow plots; contour plots; isosurface plots; particle plots; pellets plots; slice plots; streamline, streamline multislice, and streamline surface plots; surface plots; volume plots; and mesh plots.

The following properties are available:

TABLE 7-176: VALID PROPERTIES FOR TRANSFORMATION PLOT ATTRIBUTES (IF NOT NOTED, PROPERTIES ARE FOR THE STRUCTURED TRANSFORMATION TYPE).

NAME	VALUE	DEFAULT	DESCRIPTION
applytodatasetedges	boolean	true	Apply transformation to dataset edges (available for both transformation types).
ax2	double array	{0, 0}	Theta and phi angles for the rotation, if type is set to linear and axistype is set to spherical for a Rotation3D dataset, and if enablerot is true.
ax3	double array	{0, 0, 0}	Axis coordinates (x, y, and z) for the rotation, if type is set to linear and axistype is set to cartesian for a Rotation3D dataset, and if enablerot is true.
axistype	x y z cartesian spherical	z	The axis type for a rotation, if type is set to linear for a Rotation3D dataset, if enablerot is true.
enablemove	boolean	false	Enable moving (translating) the geometry.
enablerot	boolean	false	Enable rotating the geometry.
enablescale	boolean	false	Enable scaling of the geometry.
move	double array	{0, 0} (2D), {0, 0, 0} (3D)	The move (translation), if transtype is general, if enablemove is true.
rotangle	double	0	The angle increment for a rotation, if enablerot is true.
rotcenter	double array	{0, 0}	The rotation center for a Rotation2D dataset.
scale	Numerical array	Depends on the feature's tag	The scaling in the x, y (2D and 3D), and z (3D only) directions, if enablescale is true.
translation	double array	{0, 0} (2D), {0, 0, 0} (3D)	The translation, if transtype is general.
transmatrix	double array	2-by-2 or 3-by-3 matrix of zeros	The base vectors of a transformation, specified using setIndex syntax, if transtype is general.
transtype	structured general	structured	The transformation type.

SEE ALSO

[Transformation2D](#), [Transformation3D](#)

Transparency

Add transparency to some 3D plot types and 2D plot types with a height attribute.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"Transparency");
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>, "Transparency")` adds transparency named `<atag>` as an attribute to a 3D plot named `<ftag>` or to a 2D plot named `<ftag>` and that is using a height expression attribute.

The following properties are available:

TABLE 7-177: VALID PROPERTIES FOR TRANSPARENCY.

NAME	VALUE	DEFAULT	DESCRIPTION
descr	String	Model dependent	The description of the expression in <code>expr</code> , if <code>descractive</code> is on.
descractive	on off	off	Add a description when input is <code>expr</code> .
expr	String	Model dependent	The transparency expression, if input is <code>expr</code> .
input	constant <code>expr</code>	constant	Whether the transparency is constant or defined by the expression in <code>expr</code> .
mapping	auto explicit	auto	auto to map the values of the expression to a transparency range, or explicit to use the value of the expression directly as transparency. Active when input is <code>expr</code> .
maxtransparency	double (0-1)	1	The value to map the maximum value of the expression to, where 0 means no transparency and 1 means fully transparent. Active when mapping is auto.
mintransparency	double (0-1)	0	The value to map the minimum value of the expression to, where 0 means no transparency and 1 means fully transparent. Active when mapping is auto.
rangevalueactive	on off	off	Set a manual value range, if mapping is auto.
rangevaluemax	double	Model dependent	The maximum value of the manual value range, if <code>rangevalueactive</code> is on.
rangevaluemin	double	Model dependent	The minimum value of the manual value range, if <code>rangevalueactive</code> is on.
reverse	on off	off	Whether to reverse the mapping of the transparency, if mapping is auto.
transparency	scalar between 0 and 1	0.5	The constant transparency, where 0 means no transparency and 1 makes the plot fully transparent. Active when input is constant.
transparencymode	linear logarithmic	linear	Use linear or logarithmic transformation when mapping is auto.
uniformblending	scalar between 0 and 1	0	The Fresnel transmittance, where 0 means no transmittance and 1 means full transmittance.
unit	String	Model dependent	The unit to use for the expression in <code>expr</code> . If the old unit is not valid when the expression changes, the unit property is reset to default. Active when input is <code>expr</code> .

SEE ALSO

[Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#), [MaterialAppearance](#), [Surface \(Plot\)](#), [VisualEffects](#), [Volume](#)

TubeData

Create a tube data plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"TubeData");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"TubeData")` creates a tube data plot feature named `<ftag>` belonging to the 2D or 3D plot group `<pgtag>`.

Tube data plots are used to visualize raw point data given as points, elements, and colors as tubes (see the examples below). Tube data plots can be added to 2D and 3D plot groups.

The following properties are available:

TABLE 7-178: VALID PROPERTY/VALUE PAIRS FOR TUBE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
colordata	double 1D array		The color data for the tube data plot as a real N-vector.
coloring	colortable uniform gradient	colortable	How to color the tubes: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.

TABLE 7-178: VALID PROPERTY/VALUE PAIRS FOR TUBE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
pointdata	double 2D array		The point data for the tube data plot, as x- and y-coordinates in 2D and x-, y-, and z-coordinates in 3D in an sdim-by-N real matrix.
radiusdata	double 1D array		The radius data for the tube data plot as a real N-vector or a single double value for a fixed tube radius.
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maxim value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range are not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto label manual none	auto	auto, if the title should be computed automatically. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.

TABLE 7-178: VALID PROPERTY/VALUE PAIRS FOR TUBE DATA PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
tubeendcaps	true false	true	Use rounded end caps.

ATTRIBUTE[Marker, MaterialAppearance](#)**EXAMPLES**

A method for creating a logarithmic tube spiral in 2D:

Code for Use with Java

```
String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 2);
ResultFeature plot = pg.create("tube1", "TubeData");
int N = 100;
double[][] p = new double[2][N];
double[] radius = new double[N];
for (int i = 0; i < N; i++) {
    double par = 0.05*i;
    p[0][i] = Math.exp(par)*Math.cos(3*par);
    p[1][i] = Math.exp(par)*Math.sin(3*par);
    radius[i] = 0.3;
}
plot.set("pointdata", p).set("radiusdata", radius);
```

A method for creating a logarithmic tube spiral in 3D:

Code for Use with Java

```
String pgTag = model.result().uniquetag("pg");
ResultFeature pg = model.result().create(pgTag, 3);
ResultFeature plot = pg.create("tube1", "TubeData");
int N = 1000;
double[][] p = new double[3][N];
double[] radius = new double[N];
double[] color = new double[N];
for (int i = 0; i < N; i++) {
    double par = 0.005*i;
    p[0][i] = Math.exp(par)*Math.cos(10*par);
    p[1][i] = Math.exp(par)*Math.sin(10*par);
    p[2][i] = 0.1*i;
    radius[i] = 0.2*Math.sqrt(i+1);
    color[i] = i;
}
plot.set("pointdata", p)
    .set("radiusdata", radius)
    .set("colordata", color)
    .set("coloring", "colortable");
```

SEE ALSO

[AnnotationData](#), [ArrowData](#), [LineData](#), [PointData](#), [SurfaceData](#)

VisualEffects

Add or turn off visual effects such as lighting and shadows for some 3D plot types and 2D plot types with a height attribute.

SYNTAX

```
model.result(<pgtag>).feature(<ftag>).create(<atag>,"VisualEffects");  
model.result(<pgtag>).feature(<ftag>).feature(<atag>).set(property,<value>);
```

DESCRIPTION

`model.result(<pgtag>).feature(<ftag>).create(<atag>,"VisualEffects")` adds a feature for adding or turning off visual effects named `<atag>` as an attribute to an annotation, arrow, contour, coordinate system, image, isosurface, layered material slice, line, max/min, mesh, multislice, particle tracing, particle tracing with mass, particle trajectories, pellets, Poincaré map, point, point trajectories, principal stress, radiation pattern, ray trajectories, scatter volume, slice, streamline, streamline multislice, streamline surface, surface, surface slit, table annotation, table arrow, volume, waterfall, or whirl 3D plot named `<ftag>` or to a contour, function, histogram, matrix histogram, table contour, table histogram, line, optical aberration, surface, or table surface 2D plot named `<ftag>` and using a height expression attribute. The visual effects may not be visible for all plot types that support them, but the effects will be visible for all plots that include surfaces.

The following properties are available:

TABLE 7-179: VALID PROPERTIES FOR VISUALEFFECTS.

NAME	VALUE	DEFAULT	DESCRIPTION
directcasts	true false	true	Cast shadows for direct shadows, if directmode is manual.
directmode	auto manual	auto	Use automatic or manual settings for direct shadows.
directreceives	true false	true	Receive shadows for direct shadows, if directmode is manual.
lighting	true false	true	Affected by lighting or not.
ssaocasts	true false	true	Cast shadows for ambient occlusion, if ssaomode is manual.
ssaomode	auto manual	auto	Use automatic or manual settings for ambient occlusion.
ssaoreceives	true false	true	Receive shadows for ambient occlusion, if ssaomode is manual.

SEE ALSO

[Height](#), [AberrationHeight](#), [HistogramHeight](#), [TableHeight](#), [MaterialAppearance](#), [Surface \(Plot\)](#), [Transparency](#), [Volume](#)

Volume

Create a volume plot.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Volume");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Volume")` creates a volume plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

A volume plot shows an expression evaluated in all elements of a 3D volume.

The following properties are available:

TABLE 7-180: VALID PROPERTIES FOR VOLUME PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
bottomcolor	custom black blue cyan gray green magenta red white yellow	black	The bottom color to use for a color gradient. Active when coloring is gradient.
color	custom black blue cyan gray green magenta red white yellow	red	The uniform color to use. Active when coloring is uniform.
coloring	colortable uniform gradient	colortable	How to color the plot: using a color table, a uniform color, or a color gradient.
colorlegend	on off	on	Whether to show color legend, when coloring is set to colortable.
colourscalemode	linear linearsymmetric logarithmic	linear	Scaling of the color table: linear, linear symmetric around 0, or logarithmic, when coloring is set to colortable or gradient.
colortable	color table name	Rainbow	The color table to use when coloring is set to colortable. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
custombottomcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when bottomcolor is set to custom.
customcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when color is set to custom.
customtopcolor	RGB-triplet	{1,0,0} or last used color	The uniform color to use. Active when topcolor is set to custom.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
elemfilter	random expression	random	If filteractive is on: The expression to use for filtering when only a subset of the elements are shown.
elemscale	double in [0,1]	1	The scale factor with which each element is scaled.

TABLE 7-180: VALID PROPERTIES FOR VOLUME PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
filteractive	on off	off	Whether to use element filtering.
filterexpr	String	x	The expression to use for filtering when elemfilter is set to expression.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritdeformscale	boolean	true	If inheritplot is not none: Determines if the deformation scale is inherited.
inheritplot	none plot name	none	The plot that color, color range, and deformation scale are inherited from.
inheritrange	boolean	true	If inheritplot is not none: Determines if the color and data ranges are inherited.
inherittransparency	boolean	true	If inheritplot is not none: Determines if the transparency settings are inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
legendtitle	String		Color legend title, if showlegendstitle has been set to true in the parent plot group.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.

TABLE 7-180: VALID PROPERTIES FOR VOLUME PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
rangecoloractive	on off	off	Whether to use the manual color range specified in rangecolormin and rangecolormax. The color range specifies the minimum and maximum value in the plotted colors. Default is the minimum and maximum data values.
rangecolormax	double	Plot dependent	The maximum color range value. Active when rangecoloractive is on.
rangecolormin	double	Plot dependent	The minimum color range value. Active when rangecoloractive is on.
rangedataactive	on off	off	Whether to use the manual data range specified in rangedatamin and rangedatamax. Values outside the data range is not plotted.
rangedatamax	double	Plot dependent	The maximum data value. Active when rangedataactive is on.
rangedatamin	double	Plot dependent	The minimum data value. Active when rangedataactive is on.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
tetkeep	double in [0, 1]	1	The fraction of the elements to display.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-180: VALID PROPERTIES FOR VOLUME PLOTS.

NAME	VALUE	DEFAULT	DESCRIPTION
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0-1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
topcolor	custom black blue cyan gray green magenta red white yellow	white	The top color to use for a color gradient. Active when coloring is gradient.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
wireframe	on off	off	Whether to plot filled elements or only their lines.

ATTRIBUTES

[Deform](#), [Filter \(Plot Attribute\)](#), [Image \(Plot\)](#), [Marker](#), [MaterialAppearance](#), [Selection \(Dataset Attribute\)](#), [Transformation](#), [Transparency](#)

Waterfall

Create a waterfall plot in a 3D plot group. The waterfall plot creates a waterfall diagram, which is a plot that can illustrate how an expression depends on two parameters in a sweep.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Waterfall");  
model.result(<pgtag>).feature(<ftag>).set(property,<value>);  
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

`model.result(<pgtag>).create(<ftag>,"Waterfall")` creates a waterfall plot feature named `<ftag>` belonging to the plot group `<pgtag>`.

The following properties are available:

TABLE 7-181: VALID PROPERTIES FOR WATERFALL PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colortable	color table name	Rainbow	The color table to use when <code>styletype</code> is set to <code>surface</code> or <code>both</code> . See Color Tables for a list of color tables.
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in <code>expr</code> . Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when <code>titletype</code> is <code>custom</code> .
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if <code>evalmethod</code> is <code>harmonic</code> .
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
expr	String	Model dependent	The expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when <code>titletype</code> is <code>custom</code> .
inheritcolor	boolean	true	If <code>inheritplot</code> is not <code>none</code> : Determines if the color is inherited.
inheritplot	none plot name	none	The plot that color is inherited from.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when <code>data</code> is not <code>parent</code> and the underlying data is transient.

TABLE 7-181: VALID PROPERTIES FOR WATERFALL PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
smooth	none material internal everywhere expression	material	Smoothing settings. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.
solrepresentation	solnum solutioninfo	solutioninfo	Indicates which method of selecting solutions is active.
styletype	both line surface	both	Use a line plot, a surface plot, or both.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title.	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.

TABLE 7-181: VALID PROPERTIES FOR WATERFALL PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
unit	String	Model dependent	The unit to use for the expression in expr. If the old unit is not valid when the expression changes, the unit property is reset to default.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.
xdescr	String		A description for the x-axis data expression.
xdescractive	on off	off	Enable a description for the x-axis data expression.
xexpr	String		Expression for the x-axis data.
xunit	String		The unit for the x-axis data expression.
ydescr	String		A description for the y-axis data expression.
ydescractive	on off	off	Enable a description for the y-axis data expression.
yexpr	String		Expression for the y-axis data.
yunit	String		The unit for the y-axis data expression.

ATTRIBUTES

[Color](#), [MaterialAppearance](#), [Transparency](#)

Whirl

Create a whirl plot in a 3D plot group. This plot type is only available with a license for the Rotordynamics Module.

SYNTAX

```
model.result(<pgtag>).create(<ftag>,"Whirl");
model.result(<pgtag>).feature(<ftag>).set(property,<value>);
model.result(<pgtag>).feature(<ftag>).run();
```

DESCRIPTION

model.result(<pgtag>).create(<ftag>,"Whirl") creates a whirl plot feature named <ftag> belonging to the plot group <pgtag>.

The whirl plot creates a plot of the mode shapes of a rotor rotated about the rotor axis at discrete rotation intervals.

The following properties are available:

TABLE 7-182: VALID PROPERTIES FOR WHIRL PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
colorcalibration	double	0	A color calibration value (-1.5 to 1.5) when colortabletrans is set to nonlinear or nonlinearsymmetric.
colortable	color table name	Rainbow	The color table to use. See Color Tables for a list of color tables.
colortabletrans	none reverse nonlinear nonlinearsymmetric	none	Transformation of the color table, when coloring is set to colortable or gradient.

TABLE 7-182: VALID PROPERTIES FOR WHIRL PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
const	String matrix	Empty or physics dependent	Parameters to use in the expressions as an N -by-3 matrix containing name, value, and description. Only parameters provided by the physics interfaces in the model can be set.
data	none parent dataset name	parent	The dataset this feature refers to.
descr	String	Model dependent	The description of the expression in expr. Is used in the automatic title.
descriptionintitle	on off	on	Whether the title contribution should contain the description when titletype is custom.
differential	on off	on	Whether the expression should be linearized at the linearization point. Applicable only if evalmethod is harmonic.
evalmethod	linpoint harmonic lintotal lintotalavg lintotalrms lintotalpeak	harmonic	Applicable only for solutions with a stored linearization point. Controls if the linearization point, the perturbation, or a combination should be used when evaluating the expression.
evaluationsettings	parent manual	manual	Whether quality settings should be set in the plot or be inherited from the parent plot group.
expr	String array	Model dependent	The x, y, and z components of the expression to plot.
expressionintitle	on off	off	Whether the title contribution should contain the expression when titletype is custom.
inheritcolor	boolean	true	If inheritplot is not none: Determines if the color is inherited.
inheritplot	none plot name	none	The plot that color and tube radius scale are inherited from.
inheritradius	boolean	true	If inheritplot is not none: Determines if the tube radius scale is inherited.
interp	double array	Time corresponding to last selected solnum for transient levels.	The time to use, for transient levels. Available when data is not parent and the underlying data is transient.
looplevel	array of nonnegative integers and strings	Last solution for transient or parametric solution for each level. Otherwise, first solution for each level.	The index of the solution to use, per level, or interp or last, but only for transient solutions. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-182: VALID PROPERTIES FOR WHIRL PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
nonlinearcolortablerev	on off	off	Whether to reverse to color table or gradient when coloring is set to colortable or gradient and colortabletrans is set to nonlinear or nonlinearsymmetric.
nplanes	nonnegative integer	4	Number of planes in the whirl plot.
nrings	nonnegative integer	4	Number of rings in the whirl plot.
outersolnum	nonnegative integer	1	The index of the outer solutions to use. Applicable only for parametric sweep models.
prefixintitle	String	Empty	Added prefix to contribution to title.
radiusexpr	String	Empty	The tube radius expression.
recover	off pprint ppr	off	The derivative recovery method (off, within domains, or everywhere), when evaluationsettings is set to manual.
refine	nonnegative integer	1	The element refinement to use, if resolution is set to manual. Bear in mind that this is the refinement used for the base dataset, so the number of elements in the model can increase radically if the plot uses, for example, a revolve dataset.
resolution	norefine coarse normal fine finer extrafine custom	normal	Controls the plot's resolution, when evaluationsettings is set to manual. A finer setting results in a higher resolution by modifying the internally computed default refinement. Use custom to enter your own refinement in the refine property.
ringcolor	custom black blue cyan gray green magenta red white yellow	blue	The color to use for the rings in the whirl plot.
smooth	none material internal everywhere expression	material	Smoothing settings, when evaluationsettings is set to manual. With material, smoothing is done inside domains with the same material. With internal, smoothing is done inside geometry domains. With expression, the smoothing is based on the expression in smoothexpr.
smoothexpr	String	dom	The expression to use for smoothing when smooth is set to expression.
solnum	nonnegative integer	Last solution for transient or parametric solution. Otherwise, first solution.	The index of the solution to use. Available when data is not parent and the underlying data has multiple solutions.

TABLE 7-182: VALID PROPERTIES FOR WHIRL PLOTS.

PROPERTY	VALUE	DEFAULT	DESCRIPTION
solutionparams	manual parent	manual	If data is a local dataset with nontrivial data, specify solutions manually or take them from the parent plot group.
suffixintitle	String	Empty	Added suffix to contribution to title.
t	double	Time corresponding to last selected solnum.	The time to use, for transient problems. Available when data is not parent and the underlying data is transient.
threshold	manual none	none	Use a smoothing threshold.
thresholdvalue	double	0.1	Threshold value (0–1), if threshold is set to manual.
timeinterp	on off	off	on if t is used to determine time steps; off if solnum is used.
title	String	The auto-title	The title to use when titletype is manual.
titletype	auto custom label manual none	auto	auto, if the title should be computed automatically. custom, if the title should be computed automatically, but customized. label, if the title should be the plot group's label. manual, if the manual title should be used (the title property). none, if no title should be displayed.
tuberadiussscale	double	1	The scale factor applied to the tube radii if tuberadiusscaleactive is true.
tuberadiussscaleactive	boolean	false	If true, tuberadiussscale is used, otherwise the scale factor is computed automatically.
typeintitle	on off	on	Whether the title contribution should contain the type when titletype is custom.
unitintitle	on off	on	Whether the title contribution should contain the unit when titletype is custom.

ATTRIBUTES

[Color](#), [MaterialAppearance](#), [Transparency](#)

The COMSOL File Formats

This chapter describes the COMSOL native data file formats, including COMSOL Multiphysics® files in native binary data format and native text data format.

In this chapter:

- [File Formats](#)
- [Data Formats](#)
- [Color Tables and Color Themes](#)
- [Native Binary Data Files and Text Data Files](#)
- [Serializable Classes](#)

File Formats

The following table shows the file types that COMSOL Multiphysics can read and write and provides pointers to documentation:

TABLE 8-1: FILE FORMATS SUMMARY.

FILE FORMAT	EXTENSION	PRODUCT	READ	WRITE	DOCUMENTATION
COMSOL Model Files	mph	MPH	Yes	Yes	N/A
Model files for Java	java	MPH	No	Yes	About the COMSOL Model File Formats
Compiled model files for Java (class files)	class	MPH	Yes	No	N/A
Model files for MATLAB (M-files)	m	LLMATLAB	No	Yes	About the COMSOL Model File Formats
Native binary data files	mphbin	MPH	Yes	Yes	Native Binary Data Files and Text Data Files
Native text data files	mphtxt	MPH	Yes	Yes	Native Binary Data Files and Text Data Files
Materials	xml	MPH	Yes	Yes	Materials
Spreadsheet file	txt	MPH	Yes	Yes	Spreadsheet Data Format
Grid files	txt	MPH	Yes	Yes	Grid Data Format
Sectionwise files	txt	MPH	Yes	Yes	Sectionwise Data Format
CGNS data files (Windows® only)	cgns	ACOUSTICS, CFD	Yes	No	CGNS
Continuous color tables		MPH	Yes	No	Continuous Color Tables
Discrete color tables		MPH	Yes	No	Discrete Color Tables
Color themes		MPH	Yes	No	About Color Themes
Microsoft Excel® files	xlsx, xls, xlsb, xlsm	LLEXCEL	Yes	Yes	Supported Microsoft Excel File Types

Product keys:

- MPH: COMSOL Multiphysics
- ACOUSTICS: The Acoustics Module
- CFD: The CFD Module
- LLMATLAB: COMSOL LiveLink™ for MATLAB®
- LLEXCEL: COMSOL LiveLink™ for Excel®

Data Formats

The data formats in COMSOL Multiphysics are used for exporting results data to file as well as representing input to interpolation functions in COMSOL. For all data formats, the exported file can contain a number of header rows starting with %, which contain information about the model and the exported data.

In this section:

- [Spreadsheet Data Format](#)
- [Grid Data Format](#)
- [Sectionwise Data Format](#)
- [CGNS](#)



In the *COMSOL Multiphysics Reference Manual*:

- [Examples of Spreadsheet, Sectionwise, and Grid File Formats](#)
- [Exporting Data and Images](#)

Spreadsheet Data Format

This data format is used for importing unstructured data and exporting results data:

TABLE 8-2: DATA.

SECTION	DESCRIPTION
Data	Data values separated by spaces

Each row of the file defines the coordinates and data values for the function in one point. A file used to define a function of three variables can begin as follows:

```
0 0 0.12 0.34
0 1 0.52 1.50
1 0 0.67 0.91
...
```

The first variable (input argument) appears in the first (leftmost) column, the second variable in the second column, the third variable in the third column, and the function values in the fourth (rightmost) column. The variables can be any function inputs. For space-dependent functions $f(x, y, z)$ they are the x -, y -, and z -coordinates.

It is possible to define several functions in one file by providing more than one data column after the input variables (which, in many cases, are coordinates). When exporting several expressions to a file, COMSOL Multiphysics generates files with this structure.



You can use the % character to indicate that a row contains comments and not data.

Grid Data Format



The grid data format can only be used for import of data.

The following table shows the format for results data stored as grid points and corresponding data values:

TABLE 8-3: GRID, DATA.

SECTION	NUMBER OF ROWS	DESCRIPTION
% Grid	1–3	x grid points separated by spaces. y grid points separated by spaces (optional). z grid points separated by spaces (optional).
% Data	Number of y grid points (2D) or number of y grid points times number of z grid points (3)	Data values separated by spaces.

Each row contains values for different x grid points for fixed values of y and z . The rows first increase the y grid value and then the z grid value. The grid points can also represent another independent variable that the data values depend on. For example, the “grid points” can be temperature values and the data values the thermal conductivity at these temperatures.



It is important to use a comment line starting with % to separate the grid points or other interpolation points and the data values that are associated with these coordinates or interpolation points.

It is possible to include more than one function in the file as long as a % Data header separates them one from the other.



In the *COMSOL Multiphysics Reference Manual*:

- [Interpolation](#)
- [Examples of Spreadsheet, Sectionwise, and Grid File Formats](#)
- [Exporting Data and Images](#)



Rock Fracture Flow: Application Library path **COMSOL_Multiphysics/Geophysics/rock_fracture_flow**

Sectionwise Data Format

The following table shows the format for results data stored as node coordinates, elements, and corresponding data values:

TABLE 8-4: NODES, ELEMENTS, DATA.

SECTION	NO. COLUMNS	DESCRIPTION
%Coordinates	1–3	One to three columns containing x , y (optional), and z (optional)
%Elements	3 (2D), 4 (3D)	Triangulation where each row contains the row indices of the points in the Coordinates section that make up a single element — triangular in 2D, tetrahedral in 3D
%Data	1	Column of data values

This format can also be used to import data for unstructured interpolation. It has the advantage over the [Spreadsheet Data Format](#) in that it also contains the exact mesh used to perform the interpolation.

	Duplicate values (that is, evaluations with the same coordinates and the same values of the evaluated expressions) are removed before the sectionwise data is exported to file.
	In the <i>COMSOL Multiphysics Reference Manual</i>: • Examples of Spreadsheet, Sectionwise, and Grid File Formats • Exporting Data and Images

CGNS

If you are using Windows and your license includes the Acoustics Module or the CFD Module, you can import CFD analysis data in the CGNS (CFD General Notation System) format. For an extensive description of the CGNS standard and its implementations, see cgns.org.

Supported Microsoft Excel File Types

If your license includes the LiveLink™ for Excel®, you can import data to COMSOL from files with the following Excel® formats:

- For a client with Excel: Excel, Excel 97, Excel binary, and Excel macro
- For a client without Excel: Excel and Excel macro

Color Tables and Color Themes

About Color Tables

Color table files are used when plotting. When COMSOL Multiphysics is loaded, it reads all files in the following directories:

- `data/colortables/` in the directory where COMSOL is installed.
- The user settings directory `.comsol/v6.4/colortables` under your local home directory.

Most settings for colors are in the `Default.txt` file.

Files that adhere to the format specified in this section are made available as color tables, including any user-defined color table files that you have added.

In the examples below, a space is used as a separator in the color table files. You can also use a tab, comma, or semicolon as the separator.

In this section:

- [Continuous Color Tables](#)
- [Discrete Color Tables](#)

You can also add color tables to a model through the COMSOL API and in the COMSOL Desktop. See [Custom Color Tables](#) in this book and [Custom Color Tables](#) in the *COMSOL Multiphysics Reference Manual* for more information.

For information about available color tables, see [Selecting Color Tables](#) in the *COMSOL Multiphysics Reference Manual*.

Continuous Color Tables

An example of a color table that defines a transition from blue to red:

```
% Continuous
0 0 1
1 0 0
```

Each line, from left to right, contains the red, green, and blue components (RGB) of a color. The components are floating-point values between 0 and 1. The table can contain an arbitrary number of colors.

Each color can contain an optional fourth component, which represents the length of the interval between the two colors. For N colors, there are $N - 1$ intervals. An example of a color table that defines a sharp transition from blue to magenta followed by a slow transition from magenta to red:

```
% Continuous
0 0 1 1
0.5 0 0.5 10
1 0 0
```

Discrete Color Tables

An example of a color table for which the lower half of the legend is blue and the upper half is red:

```
% Discrete
0 0 1
1 0 0
```


Each color can contain an optional fourth component, the length of the interval occupied by that color. For N colors, there are N lengths. An example of a color table that defines a small blue interval, a long green interval, and a medium red interval:

```
% Discrete
0 0 1 1
0 1 0 10
1 0 0 5
```

About Color Themes

Color themes can be used to change the look of the COMSOL Desktop and to color selections in the **Graphics** window for easier identification of different parts of the model geometry. You can access and store color theme files in the following locations:

- `data/colors` (similar to `data/colortables`; for predefined color themes).
- `.comsol/v64/colors` (for user-defined color themes).

It is recommended to use one of the existing color themes, which are designed to consider clashes between theme colors and selection colors. It is up to you to make sure the colors work well together if you modify an existing color theme or add your own.

The COMSOL Multiphysics software will use the default theme color for any color not read successfully from the color theme file, so you do not need to define all colors. Likewise, if a theme used in a model is not available in the `data/colorthemes` folder, you visually get the default theme.

The color theme files contain three sections.

- The first section, `Graphics colors`, defines all the colors used by selections and contains keys for each color, similar to the preferences file. This section contains the theme colors that can be used for geometry coloring. This section can contain an arbitrary number of colors, which will cycle automatically. The colors are defined using RGB data in the range of 0–255.
- The second section, `Theme colors`, defines the selection theme colors.
- The third section, `Graph cycle colors`, contains colors for graph plots.

For a user-defined color theme, you can add a section called `Palette` first in the file, where you can define colors to use in one of the other sections. For example,

```
% Palette
MyColor=100 120 130

% Graphics colors
key=MyColor
```

The color `MyColor`, defined under `Palette`, is then used as the color for a key under `Graphics colors`.

The names of the theme files will appear in the theme-selection lists in the COMSOL Desktop.

Below you find an example file for a custom color theme with suggested names:

```
% Graphics colors

default_surface=200 200 200
default_line=0 0 0
selected_surface=140 140 242
selected_line=0 0 255
hover_surface=255 120 120 100
hover_line=255, 60, 60, 120
hover_surface_simple=242 140 140
hover_line_simple=255 0 0
```

```

hover_selected_surface=190 250 190 100
hover_selected_line=100 250 100 120
hover_selected_surface_simple=140 242 140
hover_selected_line_simple=0 153 0
feature_selection1_surface=250 222 87
feature_selection1_line=188 106 0
feature_selection2_surface=255 148 255
feature_selection2_line=155 58 165
feature_selection3_surface=247 147 30
feature_selection3_line=227 127 10
feature_selection4_surface=140 98 57
feature_selection4_line=120 78 37

```

```
% Theme colors
```

```

Red=255 0 0
Green=0 255 0
Blue=0 0 255
Yellow=255 255 0
Cyan=0 255 255
Magenta=255 0 255
Chartreuse=127 255 0
Azure=0 127 255
Rose=255 0 127
Orange=255 127 0
Spring green=0 255 127
Violet=127 0 255
Harlequin=63 255 0
Cerulean=0 63 255
Crimson=255 0 63
Lime=207 255 0
Capri=0 191 255
Cerise=255 0 191
Vermilion=255 63 0
Erin=0 255 63
Ultramarine=63 0 255
Amber=255 191 0
Aquamarine=0 255 191
Purple=191 0 255

```

```
% Graph cycle colors
```

```

% Range maxRGB
R1 G1 B1
R2 G2 B2
R3 G3 B3
...

```

where *R1*, *G1*, and *B1* are the RGB (red, green, and blue) values for the first color; *R2*, *G2*, and *B2* are the RGB values for the second color; and so on. The range on the first row of that section is the maximum value for an RGB component (*maxRGB* is an integer value). That row is optional; it defaults to 255 if not set.

It is possible to skip giving colors a name in the theme section, in which case they will be displayed as **Color 1**, **Color 2**, and so on. The syntax of the second section would then be:

```

255 191 0
0 255 191
191 0 255
...

```

These names only appear in the Linux and macOS version of the COMSOL Desktop, where the colors are displayed in a list. They do not appear in the Windows version.

The **simple** suffix in the selection color names means that the color is used for software rendering, as well as for OpenGL when **Optimize for** is set to **Performance** on the **Graphics** page in the **Preferences** window (and possibly also in some other instances when some OpenGL requirements fail).

Materials

The Model XML-File Format

Materials can be imported and exported using a Model XML-file format, which you can save and open as XML files with the .xml extension.

A typical Model XML-file can look like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<archive>
  <model>
    <material>
      <label label="AC/DC library (History XML)"/>
    </material>
    <material tag="ndfeb_bmn_35" type="Common" component="">
      <propertyGroup tag="RemanentFluxDensity" descr="Remanent flux density">
        <func tag="int1" oper="Interpolation">
          <set name="funcname" value="Br"/>
          <set name="table" value="{ '293.15', '1.220' }, { '353.15', '1.13' }"/>
          <set name="extrap" value="linear"/>
          <set name="fununit" value="{ 'T' }"/>
          <set name="argunit" value="{ 'K' }"/>
        </func>
        <set name="murec" value="{ '1.05', '0', '0', '0', '1.05', '0', '0', '0', '1.05' }"/>
        <set name="normBr" value="Br(T)"/>
        <addInput quantity="temperature"/>
      </propertyGroup>
      <label label="BMN-35"/>
      <set name="family" value="chrome"/>
      <propertyGroup tag="def">
        <set name="thermalconductivity" value="{ '9.0[W/(m*K)]', '0', '0', '0', '9.0[W/(m*K)]', '0', '0', '0', '9.0[W/(m*K)]' }"/>
        <set name="density" value="7.55[g/cm^3]"/>
        <set name="heatcapacity" value="440[J/(kg*K)]"/>
        <set name="electricconductivity" value="{ '1/1.50[uohm*m]', '0', '0', '0', '1/1.50[uohm*m]', '0', '0', '0', '1/1.50[uohm*m]' }"/>
        <set name="relpermittivity" value="{ '1', '0', '0', '0', '1', '0', '0', '0', '1' }"/>
      </propertyGroup>
    </material>
    <!-- Node groups are added last and are used as categories in the material library -->
    <nodeGroup tag="bomatec_magnetic_materials" type="GlobalDefinitions">
      <!-- A type change is necessary to only allow materials -->
      <set name="type" value="material"/>
      <!-- This is a subcategory of the top-level category -->
      <feature tag="ndfeb">
        <label label="NdFeB"/>
        <!-- These elements adds the material to the group -->
        <add tag="ndfeb_bmn_35"/>
        <add tag="ndfeb_bmn_38"/>
      </feature>
    </nodeGroup>
  </model>
</archive>
```

Each element represents a method in Java, and the element attributes represents arguments to the methods. Below are some more details about parts of this example. The statement

```
<material>
  <label label="AC/DC library (History XML)"/>
</material>
```

accesses the material list of the model object and sets the label of that list to the value of the label attribute. When you use a material element with attributes it refers to a material instead, see the following example:

```
<material tag="ndfeb_bmn_35" type="Common" component="">
```

During import, this element first creates a material of type `Common`. The `tag` and `component` attributes are not strictly enforced during import because the import decides their values depending to what list you import the materials to. The given `tag` can be occupied by another material, and the `component` is specified by the target. The content of the material element defines what property groups and functions the material contains. The `propertyGroup` element and the `func` element work in the same way as the material element — they represent new entities under the entity created by the parent element.

The `propertyGroup` element contains a lot of elements named `set` and `addInput`. The `set` element represents setting the value of a material property, where the `name` attribute is the name of the property and the `value` attribute is its value. Tensor-valued properties use curly braces to specify tensors. Here the syntax follows the standards that COMSOL Multiphysics uses in its API. If you give one value to a 3-by-3 tensor property it will become an isotropic tensor. Similarly, a value of length 3 is a diagonal tensor, and a value of length 6 represents a symmetric tensor. Give 9 values (as in the example above) to specify a full anisotropic tensor. The convention to use a vector of values to a tensor is something that the COMSOL Multiphysics software uses for property groups and also settings in physics features and multiphysics couplings. Other matrix-style values, such as the interpolation table for the `func` element, use a matrix specification using two levels of curly braces. The `addInput` element adds a model input to the material, and it expects a valid physical quantity ID for its `quantity` attribute.

The material element also supports a few `set` elements. Typically for the appearance settings such as the one named `family` in the example above.

See the following sections for tables that list all supported material types and settings; property groups and material properties; physical quantities for model inputs; and functions and settings.

Supported Material Types and Settings

The following table includes the supported material types and settings with their data type and feature ID:

TABLE 8-5: SUPPORTED MATERIAL TYPES AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
<code>alpha</code>	Opacity	Double		Applies to all ID:s
<code>ambient</code>	Ambient color	String		Applies to all ID:s
<code>basis</code>	Custom basis for brush lines	String		Applies to all ID:s
<code>basisx</code>	xm-axis	DoubleArray		Applies to all ID:s
<code>basisy</code>	ym-axis	DoubleArray		Applies to all ID:s
<code>callCleanup</code>	Call <code>cleanup()</code>	Boolean	External	External material
<code>callInit</code>	Call <code>init()</code>	Boolean	External	External material
<code>clearcoat</code>	Clear coat	Double		Applies to all ID:s
<code>color</code>	Color	String		Applies to all ID:s
<code>colornoise</code>	Additional color	String		Applies to all ID:s
<code>colornoisebrush</code>	Brush lines	String		Applies to all ID:s
<code>colornoisefrequency</code>	Noise frequency	Double		Applies to all ID:s
<code>colornoisenormalscale</code>	Normal vector noise scale	Double		Applies to all ID:s
<code>colornoisescale</code>	Noise scale	Double		Applies to all ID:s

TABLE 8-5: SUPPORTED MATERIAL TYPES AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
colornoisetype	Noise type	String		Applies to all ID:s
complex	Pass arguments as complex	Boolean	External	External material
customambient	Custom color	DoubleArray		Applies to all ID:s
customcolor	Custom color	DoubleArray		Applies to all ID:s
customdiffuse	Custom color	DoubleArray		Applies to all ID:s
customnoiseicolor	Custom color	DoubleArray		Applies to all ID:s
customspecular	Custom color	DoubleArray		Applies to all ID:s
diffuse	Diffuse color	String		Applies to all ID:s
diffusewrap	Diffuse wrap	Double		Applies to all ID:s
equation		String	PorousMedia	Porous material
errorMessage	Return error, warning, and log details	Boolean	External	External material
errorMessageSize	Maximum error message length	Int	External	External material
family	Material type	String		Applies to all ID:s
filenameeglInxa64	Filename	File	External	External material
filenameeglInxarm64	Filename	File	External	External material
filenameemacarm64	Filename	File	External	External material
filenameemaci64	Filename	File	External	External material
filenameewin64	Filename	File	External	External material
fresnel	Reflectance at normal incidence	Double		Applies to all ID:s
importednameeglInxa64	Filename	String	External	External material
importednameeglInxarm64	Filename	String	External	External material
importednamemacarm64	Filename	String	External	External material
importednamemaci64	Filename	String	External	External material
importednamewin64	Filename	String	External	External material
info	Material information	StringMatrix	Common	Material
info	Material information	StringMatrix	External	External material
info	Material information	StringMatrix	LayeredMaterial	Layered material
intname	Interface	StringArray	LayeredMaterial	Layered material
intname_D	Label	String	SingleLayerMaterial	Single layer material
intname_U	Label	String	SingleLayerMaterial	Single layer material
layername	Label	String	SingleLayerMaterial	Single layer material
layername	Layer	StringArray	LayeredMaterial	Layered material
lighting	Lighting model	String		Applies to all ID:s
link	Material	String	Link	Material link
link	Material	String	TopologyLink	Topology link
link	Material	String	SingleLayerMaterial	Single layer material
link	Material	String	LayeredMaterialLink	Layered material link

TABLE 8-5: SUPPORTED MATERIAL TYPES AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
link	Material	StringArray	LayeredMaterial	Layered material
linkBase	Material	String	SingleLayerMaterial	Single layer material
linkBase	Material	String	LayeredMaterialLink	Layered material link
linkBase	Material	String	LayeredMaterialStack	Layered material stack
linkBase	Material	String	PorousMedia	Porous material
matLink_int	Material	StringArray	LayeredMaterial	Layered material
matLink_int_D	Material	String	SingleLayerMaterial	Single layer material
matLink_int_U	Material	String	SingleLayerMaterial	Single layer material
merge	Merge middle layers	Boolean	LayeredMaterialLink	Layered material link
merge	Merge middle layers	Boolean	LayeredMaterialStack	Layered material stack
meshPoints	Mesh elements	Double	SingleLayerMaterial	Single layer material
meshPoints	Mesh elements	DoubleArray	LayeredMaterial	Layered material
metallic	Metallic	Double		Applies to all ID:s
middlePlane	Position	String	Common	Material
middlePlane	Position	String	SingleLayerMaterial	Single layer material
middlePlane	Position	String	LayeredMaterialLink	Layered material link
middlePlane	Position	String	LayeredMaterialStack	Layered material stack
mirror	Mirror in	String	LayeredMaterialLink	Layered material link
mirror	Mirror in	String	LayeredMaterialStack	Layered material stack
noise	Normal mapping	String		Applies to all ID:s
noisecolor	Color	String		Applies to all ID:s
noisecolorblend	Color blend	Double		Applies to all ID:s
noisefreq	Normal vector noise frequency	Double		Applies to all ID:s
noisescale	Normal vector noise scale	Double		Applies to all ID:s
normalnoisebrush	Brush lines	String		Applies to all ID:s
normalnoisetype	Noise type	String		Applies to all ID:s
offset	Relative midsurface offset	Double	Common	Material
offset	Relative midsurface offset	Double	SingleLayerMaterial	Single layer material
offset	Relative midsurface offset	Double	LayeredMaterialLink	Layered material link
offset	Relative midsurface offset	Double	LayeredMaterialStack	Layered material stack

TABLE 8-5: SUPPORTED MATERIAL TYPES AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
orientDist	Distance between the orientation lines	Double	LayeredMaterial	Layered material
orientDist	Distance between the orientation lines	Double	LayeredMaterialLink	Layered material link
orientDist	Distance between the orientation lines	Double	LayeredMaterialStack	Layered material stack
orientation	Orientation/variation	String	Common	Material
orientation	Orientation/variation	String	External	External material
orientation	Orientation/variation	String	LayeredMaterial	Layered material
origin	Origin	DoubleArray		Applies to all ID:s
outputQuantities	Output quantities	StringMatrix	External	External material
pearl	Pearl	Double		Applies to all ID:s
phase	Phase	String	Common	Material
phase	Phase	String	External	External material
phase	Phase	String	LayeredMaterial	Layered material
porosity	Porosity	String	PorousMedia	Porous material
position	Position	StringArray	LayeredMaterial	Layered material
reflectance	Reflectance	Double		Applies to all ID:s
repeated	Number of repeats	Double	LayeredMaterialLink	Layered material link
repeated	Number of repeats	Double	LayeredMaterialStack	Layered material stack
requiredInputQuantities	Required input quantities	StringMatrix	External	External material
rotation	Rotation (deg)	DoubleArray	LayeredMaterial	Layered material
rotation	Rotation	Double	SingleLayerMaterial	Single layer material
roughness	Surface roughness	Double		Applies to all ID:s
shininess	Specular exponent	Double		Applies to all ID:s
showLabels	Shows labels in cross-section plot	Boolean	Common	Material
showLabels	Shows labels in cross-section plot	Boolean	LayeredMaterial	Layered material
showLabels	Shows labels in cross-section plot	Boolean	SingleLayerMaterial	Single layer material
showLabels	Shows labels in cross-section plot	Boolean	LayeredMaterialLink	Layered material link
showLabels	Shows labels in cross-section plot	Boolean	LayeredMaterialStack	Layered material stack
socket	Interface type	String	External	External material
solidInfo	Porosity	StringMatrix	PorousMedia	Porous material
specifybasisy	Specify ym-axis	String		Applies to all ID:s
specular	Specular color	String		Applies to all ID:s
stack	Stack member	String	Common	Material
stack	Stack member	String	SingleLayerMaterial	Single layer material

TABLE 8-5: SUPPORTED MATERIAL TYPES AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
stack	Stack member	String	LayeredMaterialLink	Layered material link
stack	Stack member	String	LayeredMaterialStack	Layered material stack
stackAlign	Automatic alignment when the selected stack member is not available	Boolean	Common	Material
stackAlign	Automatic alignment when the selected stack member is not available	Boolean	SingleLayerMaterial	Single layer material
stackAlign	Automatic alignment when the selected stack member is not available	Boolean	LayeredMaterialLink	Layered material link
stackAlign	Automatic alignment when the selected stack member is not available	Boolean	LayeredMaterialStack	Layered material stack
stackInfo	Stack zone definition	StringMatrix	LayeredMaterialStack	Layered material stack
stateInit	Initial state values	StringArray	External	External material
stateName	State base name	StringArray	External	External material
sys	Coordinate system	String	Common	Material
sys	Coordinate system	String	SingleLayerMaterial	Single layer material
sys	Coordinate system	String	LayeredMaterialLink	Layered material link
sys	Coordinate system	String	LayeredMaterialStack	Layered material stack
thickness	Thickness	Double	Common	Material
thickness	Thickness	Double	SingleLayerMaterial	Single layer material
thickness	Thickness	DoubleArray	LayeredMaterial	Layered material
thicknessScale	Scale	String	LayeredMaterialLink	Layered material link
thicknessScale	Scale	String	LayeredMaterialStack	Layered material stack
threadSafe	Thread safe	Boolean	External	External material
topologySource	Topology source	String	TopologyLink	Topology link
transform	Transform	String	LayeredMaterialLink	Layered material link
transform	Transform	String	LayeredMaterialStack	Layered material stack
widthRatio	Thickness-to-width ratio	Double	Common	Material
widthRatio	Thickness-to-width ratio	Double	LayeredMaterial	Layered material
widthRatio	Thickness-to-width ratio	Double	SingleLayerMaterial	Single layer material

TABLE 8-5: SUPPORTED MATERIAL TYPES AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
widthRatio	Thickness-to-width ratio	Double	LayeredMaterialLink	Layered material link
widthRatio	Thickness-to-width ratio	Double	LayeredMaterialStack	Layered material stack

Supported Property Groups and Material Properties

The following table includes all supported property groups and material properties with their property group, property name, size, and SI unit:

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Electrochemistry	ElectrodePotential	Equilibrium potential	Eeq	Equilibrium potential	1x1	V
Electrochemistry	ElectrodePotential	Equilibrium potential	dEeqdT	Temperature derivative of equilibrium potential	1x1	V/K
Electrochemistry	ElectrodePotential	Equilibrium potential	cEeqref	Reference concentration	1x1	mol/m ³
Electrochemistry	OperationalSOC	Operational electrode state-of-charge	socmax	Maximum electrode state-of-charge	1x1	1
Electrochemistry	OperationalSOC	Operational electrode state-of-charge	socmin	Minimum electrode state-of-charge	1x1	1
Electrochemistry	ElectrolyteConductivity	Electrolyte conductivity	sigmal	Electrolyte conductivity	3x3	S/m
Electrochemistry	ElectrolyteSaltConcentration	Electrolyte salt concentration	cElsalt	Electrolyte salt concentration	1x1	mol/m ³
Electrochemistry	linearizedRes	Linearized resistivity	rho0	Reference resistivity	1x1	ohm·m
Electrochemistry	linearizedRes	Linearized resistivity	alpha	Resistivity temperature coefficient	1x1	1/K
Electrochemistry	linearizedRes	Linearized resistivity	Tref	Reference temperature	1x1	K
Electrochemistry	SpeciesProperties	Species properties	transpNum	Transport number	1x1	1
Electrochemistry	SpeciesProperties	Species properties	fcl	Activity dependence	1x1	1
Electrochemistry	ic	Intercalation strain	dvol	Volumetric strain	1x1	1
Electrochemistry	LocalCurrentDensity	Local current density	ilocmat	Local current density expression	1x1	A/m ²
Electrochemistry	PolymerElectrolyteWaterTransport	Polymer electrolyte water transport	alpha	Water transport coefficient	1x1	s·mol ² /(kg·m ³)

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Electrochemistry	PolymerElectrolyteWaterTransport	Polymer electrolyte water transport	xi	Electroosmotic coefficient	1x1	l
Electrochemistry	MembraneCrossover	Membrane crossover	Psi_H2	Permeation coefficient	1x1	s·mol/kg
Electrochemistry	MembraneCrossover	Membrane crossover	Psi_O2	Permeation coefficient	1x1	s·mol/kg
Electrochemistry	MembraneCrossover	Membrane crossover	Psi_N2	Permeation coefficient	1x1	s·mol/kg
Electrochemistry	PolymerElectrolyteWaterAbsorptionDesorption	Polymer electrolyte water absorption-desorption	k_abs_dsp	Absorption-desorption rate constant	1x1	mol/(m ² ·s)
Geometric properties	shell	Shell	lth	Thickness	1x1	m
Geometric properties	shell	Shell	lrot	Rotation	1x1	rad
Geometric properties	shell	Shell	lne	Mesh elements	1x1	l
Electromagnetic models	linzRes	Linearized resistivity	rho0	Reference resistivity	1x1	ohm·m
Electromagnetic models	linzRes	Linearized resistivity	alpha	Resistivity temperature coefficient	1x1	l/K
Electromagnetic models	linzRes	Linearized resistivity	Tref	Reference temperature	1x1	K
Electromagnetic models	LossTangent	Loss tangent, loss angle	delta	Loss tangent, loss angle	1x1	rad
Electromagnetic models	LossTangent	Loss tangent, loss angle	epsilonPrim	Relative permittivity (real part)	3x3	l
Electromagnetic models	LossTangentDF	Loss tangent, dissipation factor	tanDelta	Loss tangent, dissipation factor	1x1	l
Electromagnetic models	LossTangentDF	Loss tangent, dissipation factor	epsilonPrim	Relative permittivity (real part)	3x3	l
Electromagnetic models	DielectricLoss	Dielectric losses	epsilonBis	Relative permittivity (imaginary part)	3x3	l
Electromagnetic models	DielectricLoss	Dielectric losses	epsilonPrim	Relative permittivity (real part)	3x3	l
Electromagnetic models	DielectricLoss	Dielectric losses	eta_epsilon	Dielectric loss factor	3x3	l
Electromagnetic models	RefractiveIndex	Refractive index	n	Refractive index, real part	3x3	l

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Electromagnetic models	RefractiveIndex	Refractive index	ki	Refractive index, imaginary part	3x3	I
Electromagnetic models	MagneticLosses	Magnetic losses	murPrim	Relative permeability (real part)	1x1	I
Electromagnetic models	MagneticLosses	Magnetic losses	murBis	Relative permeability (imaginary part)	1x1	I
Electromagnetic models	HBCurve	H-B curve	normH	Magnetic field norm	1x1	A/m
Electromagnetic models	EffectiveHBCurve	Effective H-B curve	normHeff	Effective magnetic field norm	1x1	A/m
Electromagnetic models	BHCurve	B-H curve	normB	Magnetic flux density norm	1x1	T
Electromagnetic models	BHCurve	B-H curve	normH	Magnetic field norm	1x1	A/m
Electromagnetic models	BHCurve	B-H curve	Wpm	Magnetic coenergy density	1x1	J/m ³
Electromagnetic models	EffectiveBHCurve	Effective B-H curve	normBeff	Effective magnetic flux density norm	1x1	T
Electromagnetic models	EffectiveBHCurve	Effective B-H curve	normHeff	Effective magnetic field norm	1x1	A/m
Electromagnetic models	RemanentFluxDensity	Remanent flux density	murec	Recoil permeability	3x3	I
Electromagnetic models	RemanentFluxDensity	Remanent flux density	normBr	Remanent flux density norm	1x1	T
Electromagnetic models	BHsCurve	B-Hs curve	normB	Magnetic flux density norm	1x1	T
Electromagnetic models	BHsCurve	B-Hs curve	normHs	Shifted magnetic field norm	1x1	A/m
Electromagnetic models	BHsCurve	B-Hs curve	Hc	Coercive magnetic field	1x1	A/m
Electromagnetic models	BHsCurve	B-Hs curve	Wpm	Magnetic coenergy density	1x1	J/m ³
Electromagnetic models	EJCurve	E-J characteristic	normE	Electric field norm	1x1	V/m
Piezoresistive models	PiezoresistanceForm	Piezoresistance form	Pil	Piezoresistive coupling matrix, Voigt notation	6x6	m ⁴ /(s·A ²)

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Piezoresistive models	ElastoresistanceForm	Elastoresistance form	ml	Elastoresistive coupling matrix, Voigt notation	6x6	?*m
Electromagnetic models	JilesAtherton	Jiles–Atherton model parameters	MsJA	Saturation magnetization	3x3	A/m
Electromagnetic models	JilesAtherton	Jiles–Atherton model parameters	aJA	Domain wall density	3x3	A/m
Electromagnetic models	JilesAtherton	Jiles–Atherton model parameters	kJA	Pinning loss	3x3	A/m
Electromagnetic models	JilesAtherton	Jiles–Atherton model parameters	cJA	Magnetization reversibility	3x3	I
Electromagnetic models	JilesAtherton	Jiles–Atherton model parameters	alphaJA	Interdomain coupling	3x3	I
Electromagnetic models	PMHBCurve	Permanent magnet shifted H-B curve	normHs	Shifted magnetic field norm	1x1	A/m
Electromagnetic models	PMBHCurve	Permanent magnet shifted B-H curve	normBpm	Magnetic flux density norm	1x1	T
Electromagnetic models	Hc	Coercive magnetic field	Hc	Coercive magnetic field	1x1	A/m
Electromagnetic models/Optical dispersion models	DispersionModelCauchy	Cauchy	ODcca	Cauchy dispersion coefficients	3x1	I
Electromagnetic models/Optical dispersion models	DispersionModelCauchy	Cauchy	Trefcca	Reference temperature	1x1	K
Electromagnetic models/Optical dispersion models	DispersionModelCauchy	Cauchy	Prefcca	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelConrad y	Conrad y	ODcra	Conrad y dispersion coefficients	3x1	I
Electromagnetic models/Optical dispersion models	DispersionModelConrad y	Conrad y	Trefcra	Reference temperature	1x1	K

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Electromagnetic models/Optical dispersion models	DispersionModelConrady	Conrady	Prefcra	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelHerzberger	Herzberger	ODhza	Herzberger dispersion coefficients	5x1	l
Electromagnetic models/Optical dispersion models	DispersionModelHerzberger	Herzberger	Trefhza	Reference temperature	1x1	K
Electromagnetic models/Optical dispersion models	DispersionModelHerzberger	Herzberger	Prefhza	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelSchottStandard	Schott	ODsca	Schott dispersion coefficients	6x1	l
Electromagnetic models/Optical dispersion models	DispersionModelSchottStandard	Schott	Trefsca	Reference temperature	1x1	K
Electromagnetic models/Optical dispersion models	DispersionModelSchottStandard	Schott	Prefsca	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelSchottExtended	Schott extended	ODscb	Schott extended dispersion coefficients	9x1	l
Electromagnetic models/Optical dispersion models	DispersionModelSchottExtended	Schott extended	Trefscb	Reference temperature	1x1	K
Electromagnetic models/Optical dispersion models	DispersionModelSchottExtended	Schott extended	Prefscb	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierStandard	Sellmeier	ODsma	Sellmeier dispersion coefficients	6x1	l
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierStandard	Sellmeier	Trefsma	Reference temperature	1x1	K

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierStandard	Sellmeier	Prefsma	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierModified1	Sellmeier modified, type 1	ODsmb	Sellmeier modified, type 1 dispersion coefficients	6x1	l
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierModified1	Sellmeier modified, type 1	Trefsmb	Reference temperature	1x1	K
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierModified1	Sellmeier modified, type 1	Prefsmb	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierModified2	Sellmeier modified, type 2	ODsmc	Sellmeier modified, type 2 dispersion coefficients	5x1	l
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierModified2	Sellmeier modified, type 2	Trefsmc	Reference temperature	1x1	K
Electromagnetic models/Optical dispersion models	DispersionModelSellmeierModified2	Sellmeier modified, type 2	Prefsmc	Reference pressure	1x1	Pa
Electromagnetic models/Optical dispersion models	DispersionModelTemperatureDependentSellmeier	Temperature-dependent Sellmeier	ODtds	Temperature-dependent Sellmeier dispersion coefficients	30x1	l
Electromagnetic models/Thermo optic dispersion models	ThermoOpticDispersionModelSchott	Schott thermo-optic	TOsco	Schott thermo-optic dispersion coefficients	6x1	l
Electromagnetic models/Thermo optic dispersion models	ThermoOpticDispersionModelSchott	Schott thermo-optic	Trefsco	Reference temperature	1x1	K
Electromagnetic models/Optical attenuation models	AttenuationCoefficient	Attenuation coefficient	alphal	Attenuation coefficient	1x1	l/m
Electromagnetic models/Optical attenuation models	InternalTransmittance2	Internal transmittance, 2 mm sample thickness	taui2	Internal transmittance, 2 mm sample thickness	1x1	l

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Electromagnetic models/Optical attenuation models	InternalTransmittance5	Internal transmittance, 5 mm sample thickness	taui5	Internal transmittance, 5 mm sample thickness	l x l	l
Electromagnetic models/Optical attenuation models	InternalTransmittance10	Internal transmittance, 10 mm sample thickness	taui10	Internal transmittance, 10 mm sample thickness	l x l	l
Electromagnetic models/Optical attenuation models	InternalTransmittance25	Internal transmittance, 25 mm sample thickness	taui25	Internal transmittance, 25 mm sample thickness	l x l	l
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	An_jr	Jain–Roulston coefficient (n-type), A	l x l	V
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	Bn_jr	Jain–Roulston coefficient (n-type), B	l x l	V
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	Cn_jr	Jain–Roulston coefficient (n-type), C	l x l	V
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	Ap_jr	Jain–Roulston coefficient (p-type), A	l x l	V
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	Bp_jr	Jain–Roulston coefficient (p-type), B	l x l	V
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	Cp_jr	Jain–Roulston coefficient (p-type), C	l x l	V
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	Nref_jr	Band-gap narrowing reference concentration	l x l	l/m ³
Semiconductors/ Band-gap narrowing models	JainRoulstonModel	Jain–Roulston model	alpha_jr	Conduction band fraction	l x l	l
Semiconductors/ Band-gap narrowing models	SlotboomModel	Slotboom model	Eref_sb	Band-gap narrowing reference energy	l x l	V

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Semiconductors/ Band-gap narrowing models	SlotboomModel	Slotboom model	Nref_sb	Band-gap narrowing reference concentration	1x1	1/m ³
Semiconductors/ Band-gap narrowing models	SlotboomModel	Slotboom model	alpha_sb	Conduction band fraction	1x1	1
Semiconductors/ Generation– Recombination	SRH	Shockley– Read–Hall recombination	taun	Electron lifetime, SRH	1x1	s
Semiconductors/ Generation– Recombination	SRH	Shockley– Read–Hall recombination	taup	Hole lifetime, SRH	1x1	s
Semiconductors/ Generation– Recombination	Direct	Direct recombination	C	Direct recombination factor	1x1	m ³ /s
Semiconductors/ Generation– Recombination	Auger	Auger recombination	Cn	Auger recombination factor, electrons	1x1	m ⁶ /s
Semiconductors/ Generation– Recombination	Auger	Auger recombination	Cp	Auger recombination factor, holes	1x1	m ⁶ /s
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	an	a factor, electrons, impact ionization	1x1	1/V
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	ap	a factor, holes, impact ionization	1x1	1/V
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	bn	b factor, electrons, impact ionization	1x1	V/m
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	bp	b factor, holes, impact ionization	1x1	V/m
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	cnii	c factor, electrons, impact ionization	1x1	1/K
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	cpii	c factor, holes, impact ionization	1x1	1/K
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	dn	d factor, electrons, impact ionization	1x1	1/K

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Semiconductors/ Generation– Recombination	ImpactIonization	Impact ionization	dp	d factor, holes, impact ionization	1x1	1/K
Semiconductors	SemicondMaterial	Semiconductor material	Eg0	Band gap	1x1	V
Semiconductors	SemicondMaterial	Semiconductor material	chi0	Electron affinity	1x1	V
Semiconductors	SemicondMaterial	Semiconductor material	Nv	Effective density of states, valence band	1x1	1/m ³
Semiconductors	SemicondMaterial	Semiconductor material	Nc	Effective density of states, conduction band	1x1	1/m ³
Semiconductors	SemicondMaterial	Semiconductor material	mun	Electron mobility	1x1	m ² / (V·s)
Semiconductors	SemicondMaterial	Semiconductor material	mup	Hole mobility	1x1	m ² / (V·s)
Semiconductors	SemicondMaterialDG	Semiconductor material, density-gradient	meDG	Electron effective mass, density- gradient	3x3	kg
Semiconductors	SemicondMaterialDG	Semiconductor material, density-gradient	mhDG	Hole effective mass, density- gradient	3x3	kg
Semiconductors	SemicondMaterialDG	Semiconductor material, density-gradient	meOx	Electron effective mass, density- gradient, for penetration depth in barrier	3x3	kg
Semiconductors	SemicondMaterialDG	Semiconductor material, density-gradient	mhOx	Hole effective mass, density- gradient, for penetration depth in barrier	3x3	kg
Semiconductors	SemicondMaterialDG	Semiconductor material, density-gradient	meOxStar	Electron effective mass, density- gradient, for flux into barrier	3x3	kg
Semiconductors	SemicondMaterialDG	Semiconductor material, density-gradient	mhOxStar	Hole effective mass, density- gradient, for flux into barrier	3x3	kg

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Semiconductors/ Mobility models	PowerLawMobilityModel	Power law mobility model	mun0_pl	Electron mobility reference	1x1	$\text{m}^2/(\text{V}\cdot\text{s})$
Semiconductors/ Mobility models	PowerLawMobilityModel	Power law mobility model	mup0_pl	Hole mobility reference	1x1	$\text{m}^2/(\text{V}\cdot\text{s})$
Semiconductors/ Mobility models	PowerLawMobilityModel	Power law mobility model	alphan_pl	Electron exponent	1x1	1
Semiconductors/ Mobility models	PowerLawMobilityModel	Power law mobility model	alphap_pl	Hole exponent	1x1	1
Semiconductors/ Mobility models	PowerLawMobilityModel	Power law mobility model	Tref_pl	Reference temperature	1x1	K
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	mun0_ref_a rora	Electron mobility reference	1x1	$\text{m}^2/(\text{V}\cdot\text{s})$
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	mup0_ref_a rora	Hole mobility reference	1x1	$\text{m}^2/(\text{V}\cdot\text{s})$
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	mun_min_r ef_arora	Electron mobility reference minimum	1x1	$\text{m}^2/(\text{V}\cdot\text{s})$
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	mup_min_r ef_arora	Hole mobility reference minimum	1x1	$\text{m}^2/(\text{V}\cdot\text{s})$
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	Nn0_ref_ar ora	Electron reference impurity concentration	1x1	$1/\text{m}^3$
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	Np0_ref_ar ora	Hole reference impurity concentration	1x1	$1/\text{m}^3$
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	alpha0_aror a	Alpha coefficient	1x1	1
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	beta1_arora	Mobility reference minimum exponent	1x1	1
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	beta2_arora	Mobility reference exponent	1x1	1
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	beta3_arora	Impurity concentration reference exponent	1x1	1
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	beta4_arora	Alpha coefficient exponent	1x1	1
Semiconductors/ Mobility models	AroraMobilityModel	Arora mobility model	Tref_arora	Reference temperature	1x1	K

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Semiconductors/ Mobility models	FletcherMobilityModel	Fletcher mobility model	F1_fl	Fletcher mobility coefficient 1	1x1	1/(V·m·s)
Semiconductors/ Mobility models	FletcherMobilityModel	Fletcher mobility model	F2_fl	Fletcher mobility coefficient 2	1x1	1/m ²
Semiconductors/ Mobility models	FletcherMobilityModel	Fletcher mobility model	Tref_fl	Reference temperature	1x1	K
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	alphan0_ct	Electron alpha coefficient	1x1	1
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	alphap0_ct	Hole alpha coefficient	1x1	1
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	vn0_ct	Electron saturation velocity	1x1	m/s
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	vp0_ct	Hole saturation velocity	1x1	m/s
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	betan1_ct	Electron alpha exponent	1x1	1
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	betap1_ct	Hole alpha exponent	1x1	1
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	betan2_ct	Electron velocity saturation exponent	1x1	1
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	betap2_ct	Hole velocity saturation exponent	1x1	1
Semiconductors/ Mobility models	CaugheyThomasMobilityModel	Caughey–Thomas mobility model	Tref_ct	Reference temperature	1x1	K
Semiconductors/ Mobility models	LombardiSurfaceMobilityModel	Lombardi surface mobility model	deltan_ls	Electron delta coefficient	1x1	V/s
Semiconductors/ Mobility models	LombardiSurfaceMobilityModel	Lombardi surface mobility model	deltap_ls	Hole delta coefficient	1x1	V/s
Semiconductors/ Mobility models	LombardiSurfaceMobilityModel	Lombardi surface mobility model	mun1_ls	Electron mobility reference 1	1x1	m ² /(V·s)
Semiconductors/ Mobility models	LombardiSurfaceMobilityModel	Lombardi surface mobility model	mup1_ls	Hole mobility reference 1	1x1	m ² /(V·s)

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Semiconductors/ Mobility models	LombardiSurfaceMobility Model	Lombardi surface mobility model	mun2_ls	Electron mobility reference 2	1x1	m ² / (V·s)
Semiconductors/ Mobility models	LombardiSurfaceMobility Model	Lombardi surface mobility model	mup2_ls	Hole mobility reference 2	1x1	m ² / (V·s)
Semiconductors/ Mobility models	LombardiSurfaceMobility Model	Lombardi surface mobility model	alphan_ls	Electron alpha coefficient	1x1	1
Semiconductors/ Mobility models	LombardiSurfaceMobility Model	Lombardi surface mobility model	alphap_ls	Hole alpha coefficient	1x1	1
Semiconductors/ Mobility models	LombardiSurfaceMobility Model	Lombardi surface mobility model	Tref_ls	Reference temperature	1x1	K
Semiconductors/ Mobility models	LombardiSurfaceMobility Model	Lombardi surface mobility model	Eref_ls	Electric field reference	1x1	V/m
Semiconductors/ Mobility models	LombardiSurfaceMobility Model	Lombardi surface mobility model	Nref_ls	Doping concentration reference	1x1	1/m ³
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	T_ref_kl	Reference temperature	1x1	K
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	mu_e_max_kl	Klaassen max electron mobility	1x1	m ² / (V·s)
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	mu_h_max_kl	Klaassen max hole mobility	1x1	m ² / (V·s)
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	mu_e_min_kl	Klaassen min electron mobility	1x1	m ² / (V·s)
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	mu_h_min_kl	Klaassen min hole mobility	1x1	m ² / (V·s)
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	theta_e_kl	Klaassen lattice mobility electron exponent	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	theta_h_kl	Klaassen lattice mobility hole exponent	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	alpha_e_l_kl	Klaassen dopant and carrier mobility electron exponent	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	alpha_h_l_kl	Klaassen dopant and carrier mobility hole exponent	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	N_ref_e_l_kl	Klaassen electron reference density	1x1	1/m ³
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	N_ref_h_l_kl	Klaassen hole reference density	1x1	1/m ³
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	c_D_kl	Klaassen cluster function donor coefficient	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	c_A_kl	Klaassen cluster function acceptor coefficient	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	N_ref_D_kl	Klaassen cluster function donor reference density	1x1	1/m ³
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	N_ref_A_kl	Klaassen cluster function acceptor reference density	1x1	1/m ³
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	f_BH_kl	Klaassen P parameter BH weight	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	f_CW_kl	Klaassen P parameter CW weight	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	N_BH_kl	Klaassen P parameter BH prefactor	1x1	1/m ³
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	P_CW_kl	Klaassen P parameter CW prefactor	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	s_1_kl	Klaassen 1st s parameter	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	s_2_kl	Klaassen 2nd s parameter	1x1	1
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	s_3_kl	Klaassen 3rd s parameter	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	s_4_kl	Klaassen 4th s parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	s_5_kl	Klaassen 5th s parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	s_6_kl	Klaassen 6th s parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	s_7_kl	Klaassen 7th s parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	r_1_kl	Klaassen 1st r parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	r_2_kl	Klaassen 2nd r parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	r_3_kl	Klaassen 3rd r parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	r_4_kl	Klaassen 4th r parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	r_5_kl	Klaassen 5th r parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	r_6_kl	Klaassen 6th r parameter	1x1	l
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	m_e_kl	Klaassen electron mobility mass	1x1	kg
Semiconductors/ Mobility models	KlaassenUnifiedMobility Model	Klaassen unified mobility model	m_h_kl	Klaassen hole mobility mass	1x1	kg
Equilibrium discharge	RadiationHeatTransfer	Radiation heat transfer	Qrad	Total volumetric emission coefficient	1x1	W/m ³
Solid mechanics/ Linear elastic material	Enu	Young's modulus and Poisson's ratio	E	Young's modulus	1x1	Pa
Solid mechanics/ Linear elastic material	Enu	Young's modulus and Poisson's ratio	nu	Poisson's ratio	1x1	l
Solid mechanics/ Linear elastic material	Lame	Lamé parameters	lambLame	Lamé parameter λ	1x1	N/m ²
Solid mechanics/ Linear elastic material	Lame	Lamé parameters	muLame	Lamé parameter μ	1x1	N/m ²
Solid mechanics/ Linear elastic material	KG	Bulk modulus and shear modulus	K	Bulk modulus	1x1	N/m ²
Solid mechanics/ Linear elastic material	KG	Bulk modulus and shear modulus	G	Shear modulus	1x1	N/m ²

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Linear elastic material	EG	Young's modulus and shear modulus	E	Young's modulus	1x1	Pa
Solid mechanics/ Linear elastic material	EG	Young's modulus and shear modulus	G	Shear modulus	1x1	N/m ²
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	DA	Extensional stiffness matrix	3x3	N/m
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	DB	Bending-extensional stiffness matrix	3x3	N
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	DD	Bending stiffness matrix	3x3	N·m
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	DAs	Shear stiffness matrix	2x2	N/m
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	I0	Translational inertia	1x1	kg/m ²
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	I1	Rotational-translational inertia matrix	3x3	kg/m
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	I2	Rotational inertia matrix	3x3	kg
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	eta_DA	Loss factor for stiffness matrix DA	3x3	I
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	eta_DB	Loss factor for stiffness matrix DB	3x3	I
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	eta_DD	Loss factor for stiffness matrix DD	3x3	I
Solid mechanics/ Linear elastic material	DABD	Effective stiffness matrices	eta_DAs	Loss factor for stiffness matrix DAs	2x2	I
Solid mechanics/ Linear elastic material	Dabd	Effective flexibility matrices	Da	Extensional flexibility matrix	3x3	s ² /kg
Solid mechanics/ Linear elastic material	Dabd	Effective flexibility matrices	Db	Bending-extensional flexibility matrix	3x3	s ² /(kg·m)
Solid mechanics/ Linear elastic material	Dabd	Effective flexibility matrices	Dd	Bending flexibility matrix	3x3	s ² /(kg·m ²)

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Linear elastic material	DabD	Effective flexibility matrices	Das	Shear flexibility matrix	2x2	s ² /kg
Solid mechanics/ Linear elastic material	CpCs	Pressure-wave and shear-wave speeds	cp	Pressure-wave speed	1x1	m/s
Solid mechanics/ Linear elastic material	CpCs	Pressure-wave and shear-wave speeds	cs	Shear-wave speed	1x1	m/s
Solid mechanics/ Linear elastic material	Cubic	Cubic	c11	Elasticity constant c11	1x1	Pa
Solid mechanics/ Linear elastic material	Cubic	Cubic	c12	Elasticity constant c12	1x1	Pa
Solid mechanics/ Linear elastic material	Cubic	Cubic	c44	Elasticity constant c44	1x1	Pa
Solid mechanics/ Linear elastic material	Orthotropic	Orthotropic	Evector	Young's modulus	3x1	Pa
Solid mechanics/ Linear elastic material	Orthotropic	Orthotropic	nuvector	Poisson's ratio	3x1	I
Solid mechanics/ Linear elastic material	Orthotropic	Orthotropic	Gvector	Shear modulus	3x1	N/m ²
Solid mechanics/ Linear elastic material	Orthotropic	Orthotropic	eta_Evector	Loss factor for orthotropic Young's modulus	3x1	I
Solid mechanics/ Linear elastic material	Orthotropic	Orthotropic	eta_Gvector	Loss factor for orthotropic shear modulus	3x1	I
Solid mechanics/ Linear elastic material	OrthotropicVoGrp	Orthotropic, Voigt notation	GvectorVo	Shear modulus, Voigt notation	3x1	N/m ²
Solid mechanics/ Linear elastic material	OrthotropicVoGrp	Orthotropic, Voigt notation	eta_GvectorVo	Loss factor for orthotropic shear modulus, Voigt notation	3x1	I
Solid mechanics/ Elastoplastic Material	ElastoplasticModel	Elastoplastic material model	sigmags	Initial yield stress	1x1	Pa
Solid mechanics/ Elastoplastic Material	ElastoplasticModel	Elastoplastic material model	Et	Isotropic tangent modulus	1x1	Pa
Solid mechanics/ Elastoplastic Material	ElastoplasticModel	Elastoplastic material model	Ek	Kinematic tangent modulus	1x1	Pa

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Elastoplastic Material	ElastoplasticModel	Elastoplastic material model	sigmagh	Hardening function	1x1	Pa
Solid mechanics/ Elastoplastic Material	ElastoplasticModel	Elastoplastic material model	Hillcoefficients	Hill's coefficients	6x1	$\text{m}^2\cdot\text{s}^4/\text{kg}^2$
Solid mechanics/ Elastoplastic Material	ElastoplasticModel	Elastoplastic material model	ys	Initial tensile and shear yield stresses	6x1	N/m^2
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	sigmags	Initial yield stress	1x1	Pa
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	sigmah	Hardening function	1x1	Pa
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	alphaShima	Shima–Oyane alpha parameter	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	gammaShima	Shima–Oyane gamma parameter	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	mShima	Shima–Oyane m parameter	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	nH	Hardening exponent	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	Et	Isotropic tangent modulus	1x1	Pa
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	k_lud	Strength coefficient	1x1	Pa
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	f0	Initial void volume fraction	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	q1GTN	Tvergaard correction coefficient q1	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	q2GTN	Tvergaard correction coefficient q2	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	q3GTN	Tvergaard correction coefficient q3	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	fc	Critical void volume fraction	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	ff	Failure void volume fraction	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	fmax	Maximum void volume fraction	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	fn	Void volume fraction of nucleating voids	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	snn	Standard deviation for void nucleation	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	enn	Mean strain for void nucleation	1x1	1
Solid mechanics/ Poroplastic Material	PoroplasticModel	Poroplastic material model	kw	Void growth rate parameter	1x1	1
Solid mechanics/ Geomechanics material	MohrCoulomb	Mohr–Coulomb	cohesion	Cohesion	1x1	Pa
Solid mechanics/ Geomechanics material	MohrCoulomb	Mohr–Coulomb	internalphi	Angle of internal friction	1x1	rad
Solid mechanics/ Geomechanics material	MohrCoulomb	Mohr–Coulomb	psid	Dilatation angle	1x1	rad
Solid mechanics/ Geomechanics material	MohrCoulomb	Mohr–Coulomb	Kiso	Isotropic hardening modulus	1x1	N/m ²
Solid mechanics/ Geomechanics material	MohrCoulomb	Mohr–Coulomb	epvolmax	Maximum plastic volumetric strain	1x1	1
Solid mechanics/ Geomechanics material	MohrCoulomb	Mohr–Coulomb	Rcap	Ellipse aspect ratio	1x1	1
Solid mechanics/ Geomechanics material	DruckerPrager	Drucker–Prager	alphaDrucker	Drucker– Prager alpha coefficient	1x1	1
Solid mechanics/ Geomechanics material	DruckerPrager	Drucker–Prager	kDrucker	Drucker– Prager k coefficient	1x1	Pa
Solid mechanics/ Geomechanics material	MatsuokaNakai	Matsuoka– Nakai	muMatsuoka	Matsuoka– Nakai mu coefficient	1x1	1
Solid mechanics/ Geomechanics material	LadeDuncan	Lade–Duncan	kLade	Lade–Duncan k coefficient	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Geomechanics material	Ottosen	Ottosen	aOttosen	Ottosen a parameter	1x1	l
Solid mechanics/ Geomechanics material	Ottosen	Ottosen	bOttosen	Ottosen b parameter	1x1	l
Solid mechanics/ Geomechanics material	Ottosen	Ottosen	k1Ottosen	Size factor	1x1	l
Solid mechanics/ Geomechanics material	Ottosen	Ottosen	k2Ottosen	Shape factor	1x1	l
Solid mechanics/ Geomechanics material	HoekBrown	Hoek Brown	mHB	Hoek–Brown m parameter	1x1	l
Solid mechanics/ Geomechanics material	HoekBrown	Hoek Brown	sHB	Hoek–Brown s parameter	1x1	l
Solid mechanics/ Geomechanics material	HoekBrown	Hoek Brown	GSI	Geological strength index	1x1	l
Solid mechanics/ Geomechanics material	HoekBrown	Hoek Brown	Dfactor	Disturbance factor	1x1	l
Solid mechanics/ Geomechanics material	HoekBrown	Hoek Brown	miHB	Intact rock parameter	1x1	l
Solid mechanics/ Geomechanics material	YieldStressParameters	Yield stress parameters	sigmaut	Tensile strength	1x1	Pa
Solid mechanics/ Geomechanics material	YieldStressParameters	Yield stress parameters	sigmauc	Compressive strength	1x1	Pa
Solid mechanics/ Geomechanics material	YieldStressParameters	Yield stress parameters	sigmabc	Biaxial compressive strength	1x1	Pa
Solid mechanics/ Geomechanics material	CamClayModel	Cam-clay	kappaSwelli ng	Swelling index	1x1	l
Solid mechanics/ Geomechanics material	CamClayModel	Cam-clay	lambdaCom p	Compression index	1x1	l
Solid mechanics/ Geomechanics material	CamClayModel	Cam-clay	evoidref	Void ratio at reference pressure	1x1	l
Solid mechanics/ Geomechanics material	CamClayModel	Cam-clay	evoid0	Initial void ratio	1x1	l

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Geomechanics material	CamClayModel	Cam-clay	M	Slope of critical state line	1x1	1
Solid mechanics/ Linear elastic material	Anisotropic	Anisotropic	D	Elasticity matrix	6x6	Pa
Solid mechanics/ Linear elastic material	Anisotropic	Anisotropic	eta_D	Loss factor for elasticity matrix D	6x6	1
Solid mechanics/ Strength limits	AnisotropicStrengthParameters	Anisotropic strength parameters, Voigt notation	F_f	Fourth rank tensor, Voigt notation	6x6	$\text{m}^2 \cdot \text{s}^4 / \text{kg}^2$
Solid mechanics/ Strength limits	AnisotropicStrengthParameters	Anisotropic strength parameters, Voigt notation	F_s	Second rank tensor, Voigt notation	6x1	1/Pa
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	sigmats	Tensile strengths	3x1	Pa
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	sigmacs	Compressive strengths	3x1	Pa
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	sigmass	Shear strengths	3x1	Pa
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	sigmaID	Linear degradation stress	1x1	N/m^2
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	epsilon_t1	Ultimate tensile strain in longitudinal direction	1x1	1
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	epsilon_c1	Ultimate compressive strain in longitudinal direction	1x1	1
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	Ef1	Young's modulus of fiber in longitudinal direction	1x1	Pa
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	nuf12	In-plane Poisson's ratio of fiber	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	sigmat ₂₂	In situ transverse tensile strength	1x1	Pa
Solid mechanics/ Strength limits	OrthotropicStrengthParameters	Orthotropic strength parameters, Voigt notation	sigmas ₁₂	In situ in-plane shear strength	1x1	Pa
Solid mechanics/ Strength limits	IsotropicStrengthParameters	Isotropic strength parameters	sigmat	Tensile strength	1x1	Pa
Solid mechanics/ Strength limits	IsotropicStrengthParameters	Isotropic strength parameters	sigmac	Compressive strength	1x1	Pa
Solid mechanics/ Strength limits	IsotropicStrengthParameters	Isotropic strength parameters	sigmac _b	Biaxial compressive strength	1x1	Pa
Solid mechanics/ Strength limits	OrthotropicStrainParameters	Orthotropic ultimate strains, Voigt notation	epsilon _t	Ultimate tensile strains	3x1	1
Solid mechanics/ Strength limits	OrthotropicStrainParameters	Orthotropic ultimate strains, Voigt notation	epsilon _c	Ultimate compressive strains	3x1	1
Solid mechanics/ Strength limits	OrthotropicStrainParameters	Orthotropic ultimate strains, Voigt notation	gamma _{ss}	Ultimate shear strains	3x1	1
Solid mechanics/ Strength limits	IsotropicStrainParameters	Isotropic ultimate strains	epsilon _t	Ultimate tensile strain	1x1	1
Solid mechanics/ Strength limits	IsotropicStrainParameters	Isotropic ultimate strains	epsilon _c	Ultimate compressive strain	1x1	1
Solid mechanics/ Linear elastic material	AnisotropicVoGrp	Anisotropic, Voigt notation	D _{Vo}	Elasticity matrix, Voigt notation	6x6	Pa
Solid mechanics/ Linear elastic material	AnisotropicVoGrp	Anisotropic, Voigt notation	eta_D _{Vo}	Loss factor for elasticity matrix D, Voigt notation	6x6	1
Solid mechanics	ViscoelasticModel	Linear viscoelastic material	G _v	Shear modulus	1x1	N/m ²
Solid mechanics	ViscoelasticModel	Linear viscoelastic material	K	Bulk modulus	1x1	N/m ²
Piezoelectric models	StressCharge	Stress–charge form	cE	Elasticity matrix, Voigt notation	6x6	Pa
Piezoelectric models	StressCharge	Stress–charge form	eES	Coupling matrix, Voigt notation	3x6	C/m ²

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Piezoelectric models	StressCharge	Stress–charge form	epsilon _r S	Relative permittivity	3x3	I
Piezoelectric models	StressCharge	Stress–charge form	eta_cE	Loss factor for elasticity matrix cE	6x6	I
Piezoelectric models	StressCharge	Stress–charge form	eta_eES	Loss factor for coupling matrix eES	3x6	I
Piezoelectric models	StressCharge	Stress–charge form	eta_epsilon _S	Loss factor for electric permittivity ?S	3x3	I
Piezoelectric models	StrainCharge	Strain–charge form	sE	Compliance matrix, Voigt notation	6x6	I/Pa
Piezoelectric models	StrainCharge	Strain–charge form	dET	Coupling matrix, Voigt notation	3x6	C/N
Piezoelectric models	StrainCharge	Strain–charge form	epsilon _r T	Relative permittivity	3x3	I
Piezoelectric models	StrainCharge	Strain–charge form	eta_sE	Loss factor for compliance matrix sE	6x6	I
Piezoelectric models	StrainCharge	Strain–charge form	eta_dET	Loss factor for coupling matrix dET	3x6	I
Piezoelectric models	StrainCharge	Strain–charge form	eta_epsilon _T	Loss factor for electric permittivity ?T	3x3	I
Solid mechanics/ Hyperelastic material	Gent	Gent	muG	Macroscopic shear modulus	1x1	Pa
Solid mechanics/ Hyperelastic material	Gent	Gent	jmG	Model parameter	1x1	I
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C10	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C01	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C11	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C20	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C02	Model parameters	1x1	Pa

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C2I	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C12	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C30	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	MooneyRivlin	Mooney–Rivlin	C03	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	Murnaghan	Murnaghan	l	Murnaghan third-order elastic moduli	1x1	N/m ²
Solid mechanics/ Hyperelastic material	Murnaghan	Murnaghan	m	Murnaghan third-order elastic moduli	1x1	N/m ²
Solid mechanics/ Hyperelastic material	Murnaghan	Murnaghan	n	Murnaghan third-order elastic moduli	1x1	N/m ²
Solid mechanics/ Hyperelastic material	ArrudaBoyce	Arruda–Boyce	Nseg	Number of segments	1x1	l
Solid mechanics/ Hyperelastic material	ArrudaBoyce	Arruda–Boyce	mu0	Macroscopic shear modulus	1x1	N/m ²
Solid mechanics/ Hyperelastic material	Varga	Varga	c1VA	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	Varga	Varga	c2VA	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	Yeoh	Yeoh	c1YE	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	Yeoh	Yeoh	c2YE	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	Yeoh	Yeoh	c3YE	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	BlatzKo	Blatz–Ko	phiBK	Model parameters	1x1	l
Solid mechanics/ Hyperelastic material	BlatzKo	Blatz–Ko	betaBK	Model parameters	1x1	l

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Hyperelastic material	BlatzKo	Blatz–Ko	muBK	Shear modulus	1x1	Pa
Solid mechanics/ Hyperelastic material	Gao	Gao	aG	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	Gao	Gao	nG	Model parameters	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	E50Ref	Reference stiffness for primary loading	1x1	Pa
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	EurRef	Reference stiffness for unloading and reloading	1x1	Pa
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	evoid0	Initial void ratio	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	mH	Stress exponent	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	Kc	Bulk modulus in compression	1x1	N/m ²
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	evoidm	Maximum void ratio	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	Rcap	Ellipse aspect ratio	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	K0nc	Coefficient of earth pressure at rest	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	HardeningSoilModel	Hardening Soil	rsc	Swelling to compression ratio	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	kappaSwelling	Swelling index	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	kappaSwelli ngs	Swelling index for changes in suction	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	lambdaCom p0	Compression index at saturation	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	lambdaCom pss	Compression index for changes in suction	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	evoidref0	Void ratio at reference pressure and saturation	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	evoid0	Initial void ratio	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	M	Slope of critical state line	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	wB	Weight parameter	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	mB	Soil stiffness parameter	1x1	Pa
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	bB	Plastic potential smoothing parameter	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	sy0	Initial yield value for suction	1x1	Pa
Solid mechanics/ Nonlinear elastoplastic material	BarcelonaBasicModel	Barcelona Basic	kB	Tension to suction ratio	1x1	l
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayMod el	Structured Cam-clay	kappaSwelli ngS	Swelling index for structured clay	1x1	l

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	lambdaCompS	Compression index for destructured clay	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	evoidrefS	Void ratio at reference pressure for destructured clay	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	evoid0	Initial void ratio	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	M	Slope of critical state line	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	dvS	Destructuring index for volumetric deformation	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	dsS	Destructuring index for shear deformation	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	Deltaei	Additional void ratio at initial yielding	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	pbi	Initial structure strength	1x1	Pa
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	zetaS	Plastic potential shape parameter	1x1	1
Solid mechanics/ Nonlinear elastoplastic material	StructuredCamClayModel	Structured Cam-clay	epdevc	Critical equivalent deviatoric plastic strain	1x1	1
Solid mechanics	GeneralStressStrain	General stress– strain relation	S	Second Piola– Kirchhoff stress	3x3	N/m ²
Solid mechanics	PoroelasticModel	Poroelastic material	alphaB	Biot–Willis coefficient	1x1	1
Solid mechanics	ThermalExpansion	Thermal expansion	alphatan	Tangent coefficient of thermal expansion	3x3	1/K

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics	ThermalExpansion	Thermal expansion	dL	Thermal strain	3x3	I
Piezoelectric models	Ferroelectric	Ferroelectric	Psat	Saturation polarization	1x1	C/m ²
Piezoelectric models	Ferroelectric	Ferroelectric	chie0	Initial electric susceptibility	3x3	I
Piezoelectric models	Ferroelectric	Ferroelectric	lambdase	Saturation electrostriction	1x1	I
Piezoelectric models	Ferroelectric	Ferroelectric	kJAe	Pinning loss	3x3	V/m
Piezoelectric models	Ferroelectric	Ferroelectric	cJAe	Polarization reversibility	3x3	I
Piezoelectric models	Ferroelectric	Ferroelectric	alphaJAe	Interdomain coupling	3x3	m/F
Piezoelectric models	Ferroelectric	Ferroelectric	aJAe	Domain wall density	3x3	V/m
Piezoelectric models	Ferroelectric	Ferroelectric	Qe	Electrostrictive coupling matrix	6x6	m ⁴ /C ²
Piezoelectric models	Ferroelectric	Ferroelectric	QVe	Electrostrictive coupling matrix, Voigt notation	6x6	m ⁴ /C ²
Magnetostrictive models	Magnetostrictive	Magnetostrictive	Ms	Saturation magnetization	1x1	A/m
Magnetostrictive models	Magnetostrictive	Magnetostrictive	chi0	Initial magnetic susceptibility	3x3	I
Magnetostrictive models	Magnetostrictive	Magnetostrictive	lambdas	Saturation magnetostriction	1x1	I
Magnetostrictive models	Magnetostrictive	Magnetostrictive	lambda100	Magnetostriction constants	1x1	I
Magnetostrictive models	Magnetostrictive	Magnetostrictive	lambda111	Magnetostriction constants	1x1	I
Magnetostrictive models	StressMagnetization	Stress–magnetization form	cH	Elasticity matrix, Voigt notation	6x6	Pa
Magnetostrictive models	StressMagnetization	Stress–magnetization form	eta_cH	Loss factor for elasticity matrix cH	6x6	I
Magnetostrictive models	StressMagnetization	Stress–magnetization form	eHS	Piezomagnetic coupling matrix, Voigt notation	3x6	T
Magnetostrictive models	StressMagnetization	Stress–magnetization form	murS	Relative permeability	3x3	I

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Magnetostrictive models	StrainMagnetization	Strain–magnetization form	sH	Compliance matrix, Voigt notation	6x6	1/Pa
Magnetostrictive models	StrainMagnetization	Strain–magnetization form	eta_sH	Loss factor for compliance matrix sH	6x6	1
Magnetostrictive models	StrainMagnetization	Strain–magnetization form	dHT	Piezomagnetic coupling matrix, Voigt notation	3x6	m/A
Magnetostrictive models	StrainMagnetization	Strain–magnetization form	murT	Relative permeability	3x3	1
Acoustics	PoroacousticsModel	Poroacoustics model	K_eff	Effective bulk modulus	1x1	Pa
Acoustics	PoroacousticsModel	Poroacoustics model	Rf	Flow resistivity	1x1	Pa·s/m ²
Acoustics	PoroacousticsModel	Poroacoustics model	Lth	Thermal characteristic length	1x1	m
Acoustics	PoroacousticsModel	Poroacoustics model	Lv	Viscous characteristic length	1x1	m
Acoustics	PoroacousticsModel	Poroacoustics model	tau	Tortuosity factor	1x1	1
Acoustics	PoroacousticsModel	Poroacoustics model	tau0	Static viscous tortuosity	1x1	1
Acoustics	PoroacousticsModel	Poroacoustics model	taup0_th	Static thermal tortuosity	1x1	1
Acoustics	PoroacousticsModel	Poroacoustics model	kp0_th	Static thermal permeability	1x1	m ²
Acoustics	PoroacousticsModel	Poroacoustics model	Hr	Hydraulic radius of pores	1x1	m
Acoustics	PoroacousticsModel	Poroacoustics model	K_inf	Bulk modulus infinite frequency limit	1x1	Pa
Acoustics	PoroacousticsModel	Poroacoustics model	rho_inf	Density infinite frequency limit	1x1	kg/m ³
Acoustics	PoroacousticsModel	Poroacoustics model	tau_ent	Entropy-mode relaxation time	1x1	s
Acoustics	PoroacousticsModel	Poroacoustics model	tau_vor	Vorticity-mode relaxation time	1x1	s
Acoustics	PoroacousticsModel	Poroacoustics model	K_f	Fluid bulk modulus	1x1	Pa
Acoustics	PoroacousticsModel	Poroacoustics model	K_gr	Grain bulk modulus	1x1	Pa

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Acoustics	PoroacousticsModel	Poroacoustics model	Ks	Bulk modulus of skeleton material	1x1	Pa
Acoustics	PoroacousticsModel	Poroacoustics model	s_m	Median pore size	1x1	m
Acoustics	PoroacousticsModel	Poroacoustics model	sigma_s	Standard deviation in pore size distribution	1x1	l
Acoustics	NonlinearModel	Nonlinear model	BA	Parameter of nonlinearity	1x1	l
Acoustics	AttenuationDissipationModel	Attenuation and dissipation model	delta_diff	Sound diffusivity	1x1	m ² /s
Acoustics	AnisotropicAcousticsModel	Anisotropic acoustics model	rho_eff	Effective density	3x3	kg/m ³
Acoustics	AnisotropicAcousticsModel	Anisotropic acoustics model	K_eff	Effective bulk modulus	1x1	Pa
External material parameters	comcomsolgeneralStressStrain	General stress-strain relation	par	Material model parameters	1x1	
External material parameters	comcomsolgeneralStressStrain	General stress-strain relation	args	Extra library function string arguments	1x1	
External material parameters	comcomsolgeneralStressDeformation	General stress-deformation relation	par	Material model parameters	1x1	
External material parameters	comcomsolgeneralStressDeformation	General stress-deformation relation	args	Extra library function string arguments	1x1	
External material parameters	comcomsolinelasticResidualStrain	Inelastic residual strain	par	Material model parameters	1x1	
External material parameters	comcomsolinelasticResidualStrain	Inelastic residual strain	args	Extra library function string arguments	1x1	
External material parameters	comcomsolinelasticResidualDeformation	Inelastic residual deformation	par	Material model parameters	1x1	
External material parameters	comcomsolinelasticResidualDeformation	Inelastic residual deformation	args	Extra library function string arguments	1x1	
External material parameters	comcomsolgeneralHBRelation	General H(B) relation	par	Material model parameters	1x1	
External material parameters	comcomsolgeneralHBRelation	General H(B) relation	args	Extra library function string arguments	1x1	
External material parameters	comcomsolgeneralBHRelation	General B(H) relation	par	Material model parameters	1x1	

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
External material parameters	comcomsolgeneralBHRelation	General B(H) relation	args	Extra library function string arguments	1x1	
External material parameters	comcomsolpolyumodlagrangian	Lagrangian interface to PolyUMod	par	Material model parameters	1x1	
External material parameters	comcomsolpolyumodlagrangian	Lagrangian interface to PolyUMod	args	Extra library function string arguments	1x1	
External material parameters	comcomsolelasticSolid	Elastic stress–strain relation	E	Young’s modulus	1x1	Pa
External material parameters	comcomsolelasticSolid	Elastic stress–strain relation	nu	Poisson’s ratio	1x1	1
Solid mechanics/Hyperelastic material	HolzapfelGasserOgden	Holzapfel–Gasser–Ogden	k1HGO	Fiber stiffness	1x1	Pa
Solid mechanics/Hyperelastic material	HolzapfelGasserOgden	Holzapfel–Gasser–Ogden	k2HGO	Model parameter	1x1	1
Solid mechanics/Hyperelastic material	HolzapfelGasserOgden	Holzapfel–Gasser–Ogden	k3HGO	Fiber dispersion	1x1	1
Solid mechanics/Shape memory alloy material	ShapeMemoryAlloyThermalExpansionAustenite	Thermal expansion, austenite phase	alpha_A	Coefficient of thermal expansion	3x3	1/K
Solid mechanics/Shape memory alloy material	ShapeMemoryAlloyThermalExpansionAustenite	Thermal expansion, austenite phase	alphatan_A	Tangent coefficient of thermal expansion	3x3	1/K
Solid mechanics/Shape memory alloy material	ShapeMemoryAlloyThermalExpansionAustenite	Thermal expansion, austenite phase	dL_A	Thermal strain	3x3	1
Solid mechanics/Shape memory alloy material	ShapeMemoryAlloyThermalExpansionMartensite	Thermal expansion, martensite phase	alpha_M	Coefficient of thermal expansion	3x3	1/K
Solid mechanics/Shape memory alloy material	ShapeMemoryAlloyThermalExpansionMartensite	Thermal expansion, martensite phase	alphatan_M	Tangent coefficient of thermal expansion	3x3	1/K
Solid mechanics/Shape memory alloy material	ShapeMemoryAlloyThermalExpansionMartensite	Thermal expansion, martensite phase	dL_M	Thermal strain	3x3	1
Solid mechanics/Viscoplastic Material	Anand	Anand viscoplasticity	A_ana	Viscoplastic rate coefficient	1x1	1/s
Solid mechanics/Viscoplastic Material	Anand	Anand viscoplasticity	Q_ana	Activation energy	1x1	J/mol

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Viscoplastic Material	Anand	Anand viscoplasticity	xi_ana	Stress multiplier	1x1	1
Solid mechanics/ Viscoplastic Material	Anand	Anand viscoplasticity	m_ana	Stress sensitivity	1x1	1
Solid mechanics/ Viscoplastic Material	Anand	Anand viscoplasticity	ssat_ana	Deformation resistance saturation coefficient	1x1	N/m ²
Solid mechanics/ Viscoplastic Material	Anand	Anand viscoplasticity	sa_init	Deformation resistance initial value	1x1	N/m ²
Solid mechanics/ Viscoplastic Material	Anand	Anand viscoplasticity	h0_ana	Hardening coefficient	1x1	N/m ²
Solid mechanics/ Viscoplastic Material	Anand	Anand viscoplasticity	a_ana	Hardening sensitivity	1x1	1
Solid mechanics/ Viscoplastic Material	Anand	Anand viscoplasticity	n_ana	Deformation resistance sensitivity	1x1	1
Solid mechanics/ Viscoplastic Material	ChabocheViscoplasticity	Chaboche viscoplasticity	A_cha	Viscoplastic rate coefficient	1x1	1/s
Solid mechanics/ Viscoplastic Material	ChabocheViscoplasticity	Chaboche viscoplasticity	sigRef_cha	Reference stress	1x1	N/m ²
Solid mechanics/ Viscoplastic Material	ChabocheViscoplasticity	Chaboche viscoplasticity	n_cha	Stress exponent	1x1	1
Solid mechanics/ Viscoplastic Material	Perzyna	Perzyna viscoplasticity	A_per	Viscoplastic rate coefficient	1x1	1/s
Solid mechanics/ Viscoplastic Material	Perzyna	Perzyna viscoplasticity	sigRef_per	Reference stress	1x1	N/m ²
Solid mechanics/ Creep material	Norton	Norton	A_nor	Creep rate coefficient	1x1	1/s
Solid mechanics/ Creep material	Norton	Norton	sigRef_nor	Reference stress	1x1	N/m ²
Solid mechanics/ Creep material	Norton	Norton	n_nor	Stress exponent	1x1	1
Solid mechanics/ Creep material	Garofalo	Garofalo (hyperbolic sine)	A_gar	Creep rate coefficient	1x1	1/s
Solid mechanics/ Creep material	Garofalo	Garofalo (hyperbolic sine)	sigRef_gar	Reference stress	1x1	N/m ²

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Creep material	Garofalo	Garofalo (hyperbolic sine)	n_gar	Stress exponent	1x1	1
Solid mechanics/ Creep material	NabarroHerring	Nabarro– Herring	D_nav	Volume diffusivity	1x1	m ² /s
Solid mechanics/ Creep material	NabarroHerring	Nabarro– Herring	b_nav	Burgers vector	1x1	m
Solid mechanics/ Creep material	NabarroHerring	Nabarro– Herring	dg_nav	Grain diameter	1x1	m
Solid mechanics/ Creep material	Coble	Coble	D_cob	Grain boundary diffusivity	1x1	m ² /s
Solid mechanics/ Creep material	Coble	Coble	b_cob	Burgers vector	1x1	m
Solid mechanics/ Creep material	Coble	Coble	dg_cob	Grain diameter	1x1	m
Solid mechanics/ Creep material	Weertman	Weertman	D_wee	Diffusivity	1x1	m ² /s
Solid mechanics/ Creep material	Weertman	Weertman	b_wee	Burgers vector	1x1	m
Solid mechanics/ Creep material	Weertman	Weertman	n_wee	Stress exponent	1x1	1
Solid mechanics/ Creep material	Weertman	Weertman	sigRef_wee	Reference stress	1x1	N/m ²
Solid mechanics/ Creep material	Hillcoefficients	Hill's coefficients	Hillcoefficients	Hill's coefficients	6x1	m ² ·s ⁴ /kg ²
Solid mechanics/ Hyperelastic material	HyperelasticityFung	Fung	AFung	Coefficient matrix	6x6	1
Solid mechanics/ Hyperelastic material	HyperelasticityFung	Fung	cFung	Fung parameter c	1x1	Pa
Solid mechanics/ Hyperelastic material	HyperelasticityDelfino	Delfino	aDelf	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	HyperelasticityDelfino	Delfino	bDelf	Model parameters	1x1	1
Solid mechanics/ Hyperelastic material	HyperelasticityExtended Tube	Extended tube	GcET	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	HyperelasticityExtended Tube	Extended tube	GeET	Model parameters	1x1	Pa
Solid mechanics/ Hyperelastic material	HyperelasticityExtended Tube	Extended tube	alphaET	Model parameters	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Hyperelastic material	HyperelasticityExtended Tube	Extended tube	betaET	Model parameters	1x1	1
Solid mechanics/ Hyperelastic material	HyperelasticityVanDer Waals	van der Waals	muW	Shear modulus	1x1	Pa
Solid mechanics/ Hyperelastic material	HyperelasticityVanDer Waals	van der Waals	lambdaW	Maximum chain stretch	1x1	1
Solid mechanics/ Hyperelastic material	HyperelasticityVanDer Waals	van der Waals	alphaW	Chain network interaction	1x1	1
Solid mechanics/ Hyperelastic material	HyperelasticityVanDer Waals	van der Waals	betaW	Weight	1x1	1
Solid mechanics/ Fatigue Behavior	fatigueGeneral	General	sigmae	Endurance limit	1x1	Pa
Solid mechanics/ Elastoplastic Material	elastoplasticRambergOsgood	Ramberg–Osgood	K_ROcyclic	Cyclic hardening coefficient	1x1	Pa
Solid mechanics/ Elastoplastic Material	elastoplasticRambergOsgood	Ramberg–Osgood	n_ROcyclic	Cyclic hardening exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Strain Based	fatigueStrainCoffinManson	Coffin–Manson	epsilonf_CM	Fatigue ductility coefficient	1x1	1
Solid mechanics/ Fatigue Behavior/Strain Based	fatigueStrainCoffinManson	Coffin–Manson	c_CM	Fatigue ductility exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Strain Based	fatigueStrainCoffinManson	Coffin–Manson	gammaf_CM	Shear fatigue ductility coefficient	1x1	1
Solid mechanics/ Fatigue Behavior/Strain Based	fatigueStrainCoffinManson	Coffin–Manson	cgamma_CM	Shear fatigue ductility exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Strain Based	fatigueStrainWangBrown	Wang–Brown	S_WB	Normal strain sensitivity coefficient	1x1	1
Solid mechanics/ Fatigue Behavior/Strain Based	fatigueStrainFatemiSocie	Fatemi–Socie	k_FS	Normal stress sensitivity coefficient	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressBasquin	Basquin	sigmaf_Basquin	Fatigue strength coefficient	1x1	Pa
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressBasquin	Basquin	b_Basquin	Fatigue strength exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressBasquin	Basquin	tauf_Basquin	Shear fatigue strength coefficient	1x1	Pa
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressBasquin	Basquin	bgamma_Basquin	Shear fatigue strength exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressFindley	Findley	k_Findley	Normal stress sensitivity coefficient	1x1	1
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressFindley	Findley	f_Findley	Limit factor	1x1	Pa
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressMatake	Matake	k_Matake	Normal stress sensitivity coefficient	1x1	1
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressMatake	Matake	f_Matake	Limit factor	1x1	Pa
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressNormalStress	Normal stress	f_NormalStress	Limit factor	1x1	Pa
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressDangVan	Dang Van	a_DangVan	Hydrostatic stress sensitivity coefficient	1x1	1
Solid mechanics/ Fatigue Behavior/Stress Based	fatigueStressDangVan	Dang Van	b_DangVan	Limit factor	1x1	Pa
Solid mechanics/ Fatigue Behavior/Energy Based	fatigueEnergyMorrow	Morrow	Wf_Morrow	Fatigue energy coefficient	1x1	J/m ³

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Fatigue Behavior/Energy Based	fatigueEnergyMorrow	Morrow	m_Morrow	Fatigue energy exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Energy Based	fatigueEnergyDarveaux	Darveaux	K1_Darveaux	Crack initiation energy coefficient	1x1	1
Solid mechanics/ Fatigue Behavior/Energy Based	fatigueEnergyDarveaux	Darveaux	k2_Darveaux	Crack initiation energy exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Energy Based	fatigueEnergyDarveaux	Darveaux	K3_Darveaux	Crack propagation energy coefficient	1x1	m
Solid mechanics/ Fatigue Behavior/Energy Based	fatigueEnergyDarveaux	Darveaux	k4_Darveaux	Crack propagation energy exponent	1x1	1
Solid mechanics/ Fatigue Behavior/Energy Based	fatigueEnergyDarveaux	Darveaux	Wref_Darveaux	Reference energy density	1x1	J/m ³
Solid mechanics/ Fatigue Behavior/Fatigue Behavior	fatigueApproximateSN	Approximate S-N curve	sigmat	Transition stress	1x1	Pa
Solid mechanics/ Fatigue Behavior/Fatigue Behavior	fatigueApproximateSN	Approximate S-N curve	Nt	Transition life	1x1	
Solid mechanics/ Fatigue Behavior/Fatigue Behavior	fatigueApproximateSN	Approximate S-N curve	Ne	Endurance life	1x1	
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	sigRef	Reference stress	1x1	N/m ²
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	eRef	Reference strain	1x1	1
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	gammaRef	Reference shear strain	1x1	1
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	n_stress	Stress exponent	1x1	1
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	n_strain	Strain exponent	1x1	1
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	Kt	Bulk modulus in tension	1x1	N/m ²
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	Kc	Bulk modulus in compression	1x1	N/m ²

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	q_ult	Ultimate deviatoric stress	1x1	N/m ²
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	e_ult	Ultimate strain	1x1	1
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	sax	Uniaxial stress function	1x1	N/m ²
Solid mechanics	NonlinearElasticMaterial	Nonlinear elastic material	tau	Shear stress function	1x1	N/m ²
Solid mechanics/ Elastoplastic Material	Ludwik	Ludwik	k_lud	Strength coefficient	1x1	Pa
Solid mechanics/ Elastoplastic Material	Ludwik	Ludwik	n_lud	Hardening exponent	1x1	1
Solid mechanics/ Elastoplastic Material	Voce	Voce	sigma_voc	Saturation flow stress	1x1	Pa
Solid mechanics/ Elastoplastic Material	Voce	Voce	beta_voc	Saturation exponent	1x1	1
Solid mechanics/ Elastoplastic Material	Swift	Swift	e0_swi	Reference strain	1x1	1
Solid mechanics/ Elastoplastic Material	Swift	Swift	n_swi	Hardening exponent	1x1	1
Solid mechanics/ Elastoplastic Material	HockettSherby	Hockett– Sherby	sigma_hoc	Steady-state flow stress	1x1	Pa
Solid mechanics/ Elastoplastic Material	HockettSherby	Hockett– Sherby	m_hoc	Saturation coefficient	1x1	1
Solid mechanics/ Elastoplastic Material	HockettSherby	Hockett– Sherby	n_hoc	Saturation exponent	1x1	1
Solid mechanics/ Elastoplastic Material	ArmstrongFrederick	Armstrong– Frederick	Ck	Kinematic hardening modulus	1x1	Pa
Solid mechanics/ Elastoplastic Material	ArmstrongFrederick	Armstrong– Frederick	gammak	Kinematic hardening parameter	1x1	1
Solid mechanics/ Elastoplastic Material	Chaboche	Chaboche	Ck0_cha	Kinematic hardening modulus	1x1	Pa
Solid mechanics/ Elastoplastic Material	JohnsonCook	Johnson–Cook	k_jcook	Strength coefficient	1x1	Pa

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Elastoplastic Material	JohnsonCook	Johnson–Cook	n_jcook	Hardening exponent	1x1	1
Solid mechanics/ Elastoplastic Material	JohnsonCook	Johnson–Cook	C_jcook	Strain rate strength coefficient	1x1	1
Solid mechanics/ Elastoplastic Material	JohnsonCook	Johnson–Cook	epet0_jcook	Reference strain rate	1x1	1/s
Solid mechanics/ Elastoplastic Material	JohnsonCook	Johnson–Cook	m_jcook	Temperature exponent	1x1	
Solid mechanics/ Shape memory alloy material	ShapeMemoryAlloyAustenite	Austenite phase	E_A	Young's modulus	1x1	Pa
Solid mechanics/ Shape memory alloy material	ShapeMemoryAlloyAustenite	Austenite phase	Cp_A	Heat capacity at constant pressure	1x1	J/(kg·K)
Solid mechanics/ Shape memory alloy material	ShapeMemoryAlloyMartensite	Martensite phase	E_M	Young's modulus	1x1	Pa
Solid mechanics/ Shape memory alloy material	ShapeMemoryAlloyMartensite	Martensite phase	Cp_M	Heat capacity at constant pressure	1x1	J/(kg·K)
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	T0	Shape memory alloy reference temperature	1x1	K
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	TM _s	Martensite start temperature	1x1	K
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	TM _f	Martensite finish temperature	1x1	K
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	TA _s	Austenite start temperature	1x1	K
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	TA _f	Austenite finish temperature	1x1	K
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	CM	Slope of martensite limit curve	1x1	Pa/K
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	CA	Slope of austenite limit curve	1x1	Pa/K
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	etrmaxLagoudas	Maximum transformation strain	1x1	1

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Solid mechanics/ Shape memory alloy material	LagoudasModel	Lagoudas model	sigmaStar	Calibration stress level	1x1	N/m ²
Solid mechanics/ Shape memory alloy material	LagoudasModelStress	Lagoudas model, stress	sMs	Martensite start stress	1x1	N/m ²
Solid mechanics/ Shape memory alloy material	LagoudasModelStress	Lagoudas model, stress	sMf	Martensite finish stress	1x1	N/m ²
Solid mechanics/ Shape memory alloy material	LagoudasModelStress	Lagoudas model, stress	sAs	Austenite start stress	1x1	N/m ²
Solid mechanics/ Shape memory alloy material	LagoudasModelStress	Lagoudas model, stress	sAf	Austenite finish stress	1x1	N/m ²
Solid mechanics/ Shape memory alloy material	LagoudasModelStress	Lagoudas model, stress	Tstress	Measurement temperature	1x1	
Solid mechanics/ Shape memory alloy material	LagoudasExponentialLaw	Lagoudas, exponential law	etrmin	Initial maximum transformation strain	1x1	1
Solid mechanics/ Shape memory alloy material	LagoudasExponentialLaw	Lagoudas, exponential law	etrsat	Ultimate transformation strain	1x1	1
Solid mechanics/ Shape memory alloy material	LagoudasExponentialLaw	Lagoudas, exponential law	scrit	Critical stress	1x1	N/m ²
Solid mechanics/ Shape memory alloy material	LagoudasExponentialLaw	Lagoudas, exponential law	kcrit	Saturation exponent	1x1	1/Pa
Solid mechanics/ Shape memory alloy material	SouzaAuricchioModel	Souza–Auricchio model	TStar	Reference temperature	1x1	K
Solid mechanics/ Shape memory alloy material	SouzaAuricchioModel	Souza–Auricchio model	beta	Slope of limit curve	1x1	Pa/K
Solid mechanics/ Shape memory alloy material	SouzaAuricchioModel	Souza–Auricchio model	etrmaxAuricchio	Maximum transformation strain	1x1	1
Solid mechanics/ Shape memory alloy material	SouzaAuricchioModel	Souza–Auricchio model	sig0_SA	Elastic domain radius	1x1	N/m ²
Solid mechanics/ Shape memory alloy material	SouzaAuricchioModel	Souza–Auricchio model	Hk	Hardening modulus	1x1	N/m ²
Gas models	idealGas	Ideal gas	Rs	Specific gas constant	1x1	J/(kg·K)

TABLE 8-6: SUPPORTED PROPERTY GROUPS AND MATERIAL PROPERTIES.

CATEGORY	PROPERTY GROUP (API NAME)	PROPERTY GROUP (DESCRIPTION)	PROPERTY (NAME)	PROPERTY (DESCRIPTION)	SIZE	UNIT
Fluid flow/ Inelastic Non- Newtonian	PowerLaw	Power law	m_pow	Fluid consistency coefficient	1x1	Pa·s
Fluid flow/ Inelastic Non- Newtonian	PowerLaw	Power law	n_pow	Flow behavior index	1x1	1
Fluid flow/ Inelastic Non- Newtonian	Carreau	Carreau model	mu0	Zero shear rate viscosity	1x1	Pa·s
Fluid flow/ Inelastic Non- Newtonian	Carreau	Carreau model	mu_inf	Infinite shear rate viscosity	1x1	Pa·s
Fluid flow/ Inelastic Non- Newtonian	Carreau	Carreau model	lam_car	Relaxation time	1x1	s
Fluid flow/ Inelastic Non- Newtonian	Carreau	Carreau model	n_car	Power index	1x1	1
Fluid flow/ Inelastic Non- Newtonian	Carreau	Carreau model	a_car	Transition parameter	1x1	1
Fluid flow/ Inelastic Non- Newtonian	Ellis	Ellis model	tau_half	Shear stress parameter	1x1	N/m ²
Fluid flow/ Inelastic Non- Newtonian	Ellis	Ellis model	a_e	Power parameter	1x1	1
Fluid flow/ Inelastic Non- Newtonian	DeKee	DeKee model	lam_DK	Relaxation time	1x1	s
Fluid flow/ Inelastic Non- Newtonian	DeKee	DeKee model	mu_DK	Shear rate viscosity parameter	1x1	Pa·s
Fluid flow/ Inelastic Non- Newtonian	Cross	Cross model	tau_tr	Critical stress	1x1	N/m ²
Fluid flow/ Inelastic Non- Newtonian	Viscoplastic	Viscoplastic	tau_y	Yield stress	1x1	N/m ²
Fluid flow/ Inelastic Non- Newtonian	Viscoplastic	Viscoplastic	mu_p	Plastic viscosity	1x1	Pa·s

Supported Physical Quantities for Model Inputs

The following table lists all supported physical quantities for use as model inputs with their category, base name, and SI unit:

TABLE 8-7: SUPPORTED PHYSICAL QUANTITIES FOR MODEL INPUTS.

API NAME	DESCRIPTION	CATEGORY	BASE NAME	UNIT
numberdensityacceptor	Acceptor concentration	Transport	Na	l/m^3
angularfrequency	Angular frequency	General	omega	l/s
catalyticactivity	Catalytic activity	Transport	a	mol/s
charge	Charge	Electromagnetics	Q	C
inelasticpowerloss	Collisional power loss	Transport	Sen	W/m^3
concentration	Concentration	General	c	mol/m^3
massconcentration	Concentration	Transport	cm	kg/m^3
conversion	Conversion	Transport	cr	l
correctedpressure	Corrected pressure	Transport	pc	Pa
correctedvelocity	Corrected velocity field	Transport	uc	m/s
current	Current	General	I	A
currentdensity	Current density	Electromagnetics	J	A/m^2
currentsource	Current source	Electromagnetics	Qj	A/m^3
depth	Depth	Acoustics	D	m
dispersedphasevolume fraction	Dispersed phase volume fraction	Transport	phid	l
displacement	Displacement	General	u_d	m
numberdensitydonor	Donor concentration	Transport	Nd	l/m^3
downsidetemperature	Downside temperature	Hidden	Td	K
effectivegasdensity	Effective gas density	Transport	rhogeff	kg/m^3
elastichearstrain	Elastic shear strain	Solid mechanics	esh	l
uniaxialstrain	Elastic uniaxial strain	Solid mechanics	eax	l
displacementfield	Electric displacement field	Electromagnetics	Df	C/m^2
electricfield	Electric field	Electromagnetics	E	V/m
electricpotential	Electric potential	Electromagnetics	V	V
electrolyteconductivity	Electrolyte conductivity	Hidden	signal	S/m
electricpotentialionicphase	Electrolyte potential	Hidden	phil	V
electrondensity	Electron density	Transport	ne	l/m^3
electronenergydensity	Electron energy density	Transport	en	$\text{kg}/(\text{m}\cdot\text{s}^3\cdot\text{A})$
energy	Energy	General	Ene	J
energydensity	Energy density	Electromagnetics	W	Pa
effectiveplasticstrain	Equivalent plastic strain	Solid mechanics	epe	l
effectiveplasticstraininmatrix	Equivalent plastic strain in the matrix material	Solid mechanics	epm	l
effectiveplasticstrainrate	Equivalent plastic strain rate	Solid mechanics	epet	l/s
effectiveviscoplasticstrain	Equivalent viscoplastic strain	Solid mechanics	evpe	l
effectiveviscoplasticstrainrate	Equivalent viscoplastic strain rate	Solid mechanics	evpet	l/s
fluidconductance	Fluid conductance	Transport	Rb	l/s
forcedensity	Force density	Transport	f	N/m^3

TABLE 8-7: SUPPORTED PHYSICAL QUANTITIES FOR MODEL INPUTS.

API NAME	DESCRIPTION	CATEGORY	BASE NAME	UNIT
frequency	Frequency	General	freq	1/s
generalizedchemicalpotential	Generalized chemical potential	Transport	eta	Pa
henrysconstant	Henry's constant	Transport	kh	J/mol
hydraulicconductivity	Hydraulic conductivity	Transport	K	m/s
initialelectrondensity	Initial electron density	Transport	neinit	1/m ³
heatflux	Inward heat flux	Transport	q0	W/m ²
kinematicviscosity	Kinematic viscosity	Transport	nu	m ² /s
length	Length	General	l	m
levelsetvariable	Level set variable	Transport	phils	l
linecharge	Line charge	Electromagnetics	Ql	C/m
linecurrentsource	Line current source	Electromagnetics	Qjl	A/m
logelectrondensity	Log of electron density	Transport	Ne	l
logelectronenergydensity	Log of electron energy density	Transport	En	l
luminousintensity	Luminous intensity	General	li	cd
magneticfield	Magnetic field	Electromagnetics	H	A/m
magneticfluxdensity	Magnetic flux density	Electromagnetics	B	T
massfraction	Mass fraction	Transport	w	l
masssource	Mass source	Transport	Qm	kg/(m ³ ·s)
meanelectronenergy	Mean electron energy	Transport	ebar	V
energyperamount	Molar enthalpy	Transport	h	J/mol
molarsurfaceflux	Molar flux	Transport	Ns	mol/(m·s)
molarvolume	Molar volume	Transport	Vmol	m ³ /mol
normalelectroncurrentdensity	Normal electron current density	Electromagnetics	nFe	A/m ²
normalioncurrentdensity	Normal ion current density	Electromagnetics	nF	A/m ²
numberdensity	Number density	Transport	nd	1/m ³
phasefieldhelpvariable	Phase field help variable	Transport	psi	l
phasefieldvariable	Phase field variable	Transport	phipf	l
pointcurrentsource	Point current source	Electromagnetics	I0	A
powerflow	Power flow	Electromagnetics	pFlow	W/m ²
practicalsalinity	Practical salinity	Acoustics	Spr	l
pressure	Pressure	General	pA	Pa
heatproduction	Production/absorption coefficient	Transport	qs	W/(m ³ ·K)
reducedelectricfield	Reduced electric field	Transport	EN	V·m ²
reduceddiffusivity	Reduced electron diffusivity	Transport	DeN	l/(m·s)
reducedmobility	Reduced electron mobility	Transport	muN	l/(V·m·s)
relativehumidity	Relative humidity	General	phi	l
salinity	Salinity	Acoustics	Sp	l
energyflux	Secondary emission energy flux	Transport	seflux	V/(m ² ·s)
particleflux	Secondary emission flux	Transport	sflux	l/(m ² ·s)
solidangle	Solid angle	General	sr	l

TABLE 8-7: SUPPORTED PHYSICAL QUANTITIES FOR MODEL INPUTS.

API NAME	DESCRIPTION	CATEGORY	BASE NAME	UNIT
spacechargedensity	Space charge density	Electromagnetics	rhoq	C/m ³
specificenergy	Specific energy	Transport	es	J/kg
slipvelocity	Squared slip velocity	Transport	slipvel	J/kg
surfacechargedensity	Surface charge density	Electromagnetics	rhoqs	C/m ²
surfacecurrentdensity	Surface current density	Electromagnetics	Js	A/m
surfagemagneticcurrentdensity	Surface magnetic current density	Electromagnetics	Jms	V/m
surfaceconcentration	Surface site concentration	Transport	gamma	mol/m ²
temperature	Temperature	General	T	K
timechangeinpressurehead	Time change in pressure head	Transport	dHpdt	m/s
turbulentdissipationrate	Turbulent dissipation rate	Transport	ep	W/kg
turbulentkinematicviscosity	Undamped turbulent kinematic viscosity	Transport	nutilde	m ² /s
upsidetemperature	Upside temperature	Hidden	Tu	K
velocity	Velocity	General	u	m/s
volumefraction	Volume fraction	Transport	Vf	I
strainreferencetemperature	Volume reference temperature	General	Tempref	K
volumetricheatcapacity	Volumetric heat capacity	Transport	Cvol	J/(m ³ ·K)

Supported Functions and Settings

The following table includes the supported functions and settings with their data type and feature ID:

TABLE 8-8: SUPPORTED FUNCTIONS AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
argders	Arguments	StringMatrix	Analytic	Analytic
args	Arguments	StringArray	Analytic	Analytic
argunit	Arguments	StringArray	Analytic	Analytic
complex	May produce complex output for real arguments	Boolean	Analytic	Analytic
dermethod	Derivatives	String	Analytic	Analytic
expr	Expression	String	Analytic	Analytic
fununit	Function	String	Analytic	Analytic
periodic	Make periodic	String	Analytic	Analytic
periodiclower	Lower limit	Double	Analytic	Analytic
periodicupper	Upper limit	Double	Analytic	Analytic
plotargs	Arguments	StringMatrix	Analytic	Analytic
plotaxis	Argument	BooleanArray	Analytic	Analytic
argunit	Argument	StringArray	Interpolation	Interpolation
defineinv	Define inverse function	String	Interpolation	Interpolation
defineprimfun	Define primitive function	String	Interpolation	Interpolation
definerandom	Define random function	String	Interpolation	Interpolation
defvars	Use spatial coordinates as arguments	String	Interpolation	Interpolation
dseparator	Decimal separator	String	Interpolation	Interpolation

TABLE 8-8: SUPPORTED FUNCTIONS AND SETTINGS.

SETTING	DESCRIPTION	TYPE	FEATURE ID	FEATURE DESCRIPTION
exportfilename	Export...	None	Interpolation	Interpolation
extrap	Extrapolation	String	Interpolation	Interpolation
extrapvalue	Value outside range	Double	Interpolation	Interpolation
filename	Filename	File	Interpolation	Interpolation
frame	Frame	String	Interpolation	Interpolation
funcinvname	Inverse function name	String	Interpolation	Interpolation
funcname	Function name	String	Interpolation	Interpolation
funcs	Functions	StringMatrix	Interpolation	Interpolation
fununit	Function	StringArray	Interpolation	Interpolation
importeddin	Dimension	String	Interpolation	Interpolation
importedname	Filename	String	Interpolation	Interpolation
importedstruct	Data type	String	Interpolation	Interpolation
interp	Interpolation	String	Interpolation	Interpolation
leftend	Lower limit	Double	Interpolation	Interpolation
nargs	Number of arguments	Int	Interpolation	Interpolation
plotinv	Plot the inverse function	Boolean	Interpolation	Interpolation
primfunname	Primitive function name	String	Interpolation	Interpolation
randomname	Random function name	String	Interpolation	Interpolation
randomnargs	Number of arguments	Int	Interpolation	Interpolation
randomrange	Range	String	Interpolation	Interpolation
range	Range	String	Interpolation	Interpolation
resultTable	Table from	String	Interpolation	Interpolation
rightend	Upper limit	Double	Interpolation	Interpolation
scaledata	Internal scaling of data points	String	Interpolation	Interpolation
sheet	Sheet	String	Interpolation	Interpolation
source	Data source	String	Interpolation	Interpolation
struct	Data format	String	Interpolation	Interpolation
table	Function values	StringMatrix	Interpolation	Interpolation
arg	Argument	String	Piecewise	Piecewise
argunit	Arguments	String	Piecewise	Piecewise
extrap	Extrapolation	String	Piecewise	Piecewise
extrapvalue	Value outside range	Double	Piecewise	Piecewise
fununit	Function	String	Piecewise	Piecewise
pieces	Intervals	StringMatrix	Piecewise	Piecewise
smooth	Smoothing	String	Piecewise	Piecewise
smoothends	Smooth at endpoints	Boolean	Piecewise	Piecewise
smoothzone	Size of transition zone	Double	Piecewise	Piecewise
zonelengthtype	Transition zone	String	Piecewise	Piecewise

Native Binary Data Files and Text Data Files

A COMSOL Multiphysics native data file is used to store COMSOL data. This file format is suitable for exchange of mesh or CAD data between COMSOL Multiphysics and other software systems. It is possible to save a COMSOL Multiphysics native data file in a native text data file format, using the extension `.mphtxt`, or in a native binary data file format, using the extension `.mphbin`. The file formats contain the same data in the same order.

In this section:

- [File Structure](#)
- [Objects](#)
- [Terminology](#)
- [Text File Format](#)
- [Binary File Format](#)

File Structure

The COMSOL Multiphysics native data file format has a global version number, so that it is possible to revise the whole structure. The first entry in each file is the file format, indicated by two integers. The first integer is the major file version and the second is referred to as the minor file version. For the current version, the first two entries in a file is 0 1.

The following sections describe the file structure of the supported version.

FILE VERSION 0.1

After the file version, the file contains three groups of data:

- A number of *tags* stored as strings, which are used so that objects can refer to each other.
- A number of *types*, which are strings that can be used in serializing the object. The types are currently not used by the COMSOL Multiphysics software.
- *Objects*, where each object starts with the header 0 0 1, followed by a string that defines which type of object that follows.

Example When using `model.mesh(<tag>).export(<filename>)` or **Mesh > Export** to save a COMSOL mesh, the tag equals the variable name (`m1`), the type is set to `obj` (but this is not used), and the file contains the serialization of the mesh object, including point coordinates and element data of the mesh. In this case, the file also contains a selection object. See some of the entries in [Serializable Classes](#) for more examples of COMSOL Multiphysics native text data file content.

```
# Created by COMSOL Multiphysics.

# Major & minor version
0 1
2 # number of tags
# Tags
5 mesh1
10 mesh1_sel1
2 # number of types
# Types
3 obj
3 obj

# ----- Object 0 -----
```

```

0 0 1
4 Mesh # class
8 # version
3 # sdim

# Include geometric model
1

# Geometric model

4 # Number of geometric entities per dimension
14
15
6
0

1 # Voids are labeled

1 # Number of finite voids

6 # Up and down domains for boundaries
0 -1
0 -1
0 -1
0 -1
0 -1
0 -1

3 # Isolated edges in domains
0 0
6 -1
13 0

2 # Isolated vertices in domains
6 -1
13 0

# Mesh data

514 # number of mesh vertices

# Mesh vertex coordinates
0.92206542189421015 0.91795822469634891 1
1 0.9220130709632427 0.91794866649210038
0.91012354689345587 0.81224942182395221 1
...
...
# ----- Object 1 -----

0 0 1
9 Selection # class
0 # Version
7 Outer boundaries # Label
5 mesh1 # Geometry/mesh tag
2 # Dimension
6 # Number of entities
# Entities
0
1
2
3
4

```

Objects

The objects section contains the serialization data for each serialized object, instantiated from a serializable class. The serialization of each object start with a version number. With this version number, the serialization can be revised in future versions while maintaining backward compatibility.

The following section describes the format of the supported version:

OBJECT VERSION 0

The following table contains the fields of the objects:

DATA TYPE	VARIABLE	DESCRIPTION
Integer		Version.
Integer		Not used.
Integer	type	Serialization type, 1 for Serializable.
Serializable	obj	If type equals 1, this field follows.

Serialization type 1 indicates that the following field is a subtype to [Serializable](#).

Terminology

The following data types are used in the serialization:

- *Boolean* refers to an 8-bit signed character which must be 0 or 1.
- *Character* refers to an 8-bit signed character.
- *Integer* refers to a 32-bit signed integer.
- *Double* refers to a 64-bit double.

Matrices are stored in row-major order. In this documentation brackets are used to indicate a matrix. Hence, `integer[3][4]` means that 12 integers representing a matrix are stored in the file. The first four entries correspond to the integers in the first row of the matrix, and so on.

Text File Format

COMSOL Multiphysics text file, using the file extension `.mphtxt`, are text files where values are stored as text separated by whitespace characters.

Lexical conventions:

- Strings are serialized as the length of the string followed by a space and then the characters of the string, for example, “6 COMSOL”. This is the only place where whitespace matters.
- The software ignores everything following a `#` on a line except when reading a string. This makes it possible to store comments in the file.

Binary File Format

COMSOL Multiphysics binary file, using the extension `.mphbin`, are binary files with the following data representation:

- Integers and doubles are stored in little-endian byte order.
- Strings are stored as the length of the string (integer) followed by the characters of the string (integers).

Serializable Classes

In this section:

- [BezierCurve](#)
- [BezierSurf](#)
- [BezierTri](#)
- [BSplineCurve](#)
- [BSplineSurf](#)
- [Ellipse](#)
- [Geom1](#)
- [Geom2](#)
- [Geom3](#)
- [GeomCurve](#)
- [GeomSurf](#)
- [Mesh](#)
- [MeshCurve](#)
- [MeshSurf](#)
- [PolChain](#)
- [Selection](#)
- [Serializable](#)
- [Straight](#)
- [Transform](#)

The [Serializable](#) class is the base type, and all other types are subtypes of [Serializable](#) or of its subtypes.

	In the Fields sections for each type below, the fields appear in the table in the exact order that they must appear in the data files. The Variable column lists the internal variables used in the descriptions of data types and in the definitions of what the class implements.
--	---

	The serialization of a subtype of Serializable begins with Serializable's serialization; that is, it begins with the type ID string. The type ID strings are the same as the headers of the following sections.
---	--

For an example of the serialization format — specifically, of a file containing a 3D mesh with tetrahedral and prism elements — see `mesh_example_4.mph.txt` in the Application Libraries folder `COMSOL_Multiphysics/Meshing_Tutorials`.

	For geometry types, you can import and export older versions of the serializable classes.
---	--

BezierCurve

CURRENT VERSION

1

SUBTYPE OF

[GeomCurve](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	d	Space dimension.
boolean		1 if the curve is rational, 0 if the curve is polynomial (nonrational).
integer	p	Degree.
double[p+1][k]	Pw	The control points. If the curve is rational, then the points are given in homogeneous coordinates and k = d+1. If the curve is polynomial, the points are given in Cartesian coordinates and k = d.

DESCRIPTION

A rational Bézier curve is a parameterized curve of the form

$$\mathbf{C}(t) = \frac{\sum_{i=0}^p \mathbf{P}_i w_i B_i^p(t)}{\sum_{i=0}^p w_i B_i^p(t)}, \quad 0 \leq t \leq 1$$

where the functions

$$B_i^p(t) = \binom{p}{i} t^i (1-t)^{p-i}$$

are the *Bernstein basis* functions of *degree* p , $\mathbf{P}_i = (x_1, \dots, x_d)$ are the control points in the d -dimensional space, and w_i are the weights, which should always be positive real numbers to get a properly defined rational Bézier curve. A rational Bézier curve has a direction defined by the parameter t . The homogeneous control points $\text{Pw}[i]$ used in the serialization of a rational curve have the components:

$$\begin{aligned} \text{Pw}[i][k] &= w_i P_{i,k}, \quad 0 \leq k < d \\ \text{Pw}[i][d] &= w_i \end{aligned}$$

A polynomial curve has all weights equal to 1.

EXAMPLE

The following example illustrates a Bézier curve:

```
11 BezierCurve # class
1 # version
3 # sdim
1 # rational?
2 # degree
# homogeneous control points
-1 0 0 1
-0.70710678118654757 -0.70710678118654757 0 0.70710678118654757
0 -1 0 1
```

SEE ALSO

[BSplineCurve](#)

CURRENT VERSION

1

SUBTYPE OF

GeomSurf

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
boolean		1 if the surface is rational, 0 if it is polynomial (nonrational).
integer	p	Degree in the first parameter.
integer	q	Degree in the second parameter.
double[q+1][p+1][k]	Pw	The surface control points. If the surface is rational, these are given in homogeneous coordinates and k = 4. If the surface is polynomial, these are given in Cartesian coordinates and k = 3. The value of p must be greater than or equal to the value of q.

DESCRIPTION

A rectangular rational Bézier surface of degree p -by- q is described by

$$\mathbf{S}(s,t) = \frac{\sum_{i=0}^p \sum_{j=0}^q \mathbf{P}_{i,j} w_{i,j} B_i^p(s) B_j^q(t)}{\sum_{i=0}^p \sum_{j=0}^q w_{i,j} B_i^p(s) B_j^q(t)}, \quad 0 \leq s, t \leq 1,$$

where B_i^p and B_j^q are the Bernstein basis functions of degree p and q , respectively, as described in the entry of [BezierCurve](#). This surface description is called rectangular because the parameter domain is rectangular; that is, the two parameters s and t can vary freely in given intervals. The homogeneous control points $\text{Pw}[j][i]$ used in the serialization of a rational surface have the components:

$$\begin{aligned} \text{Pw}[j][i][k] &= w_{i,j} P_{i,j,k}, \quad 0 \leq k < 3 \\ \text{Pw}[j][i][d] &= w_{i,j} \end{aligned}$$

A polynomial surface has all weights equal to 1.

SEE ALSO

[BSplineSurf](#), [BezierTri](#)

CURRENT VERSION

1

SUBTYPE OF

GeomSurf

FIELDS

The class is defined by the following fields:

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
double[3][3]	P	Control points P[0], P[1], and P[2].

DESCRIPTION

This type represents a triangular planar surface through three control points, defined as

$$S(s, t) = (1 - s - t)P_0 + sP_1 + tP_2$$

where

$$\begin{cases} 0 \leq s, t \\ s + t \leq 1 \end{cases}$$

SEE ALSO

[BezierSurf](#)

BSplineCurve

CURRENT VERSIONS

2

SUBTYPE OF

[GeomCurve](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	d	Space dimension.
boolean		1 if the curve is rational, 0 if the curve is polynomial (nonrational).
boolean		1 if the curve is periodic, 0 if it is not periodic.
integer	p	Degree.
integer	m	Length of knot vector.
double[m]	U	Knot vector.
double[n][k]	Pw	The control points of the curve. The number of control points, n, is given by n = m - p - 1. If the curve is rational, these are given in homogeneous coordinates and k = d + 1. If the curve is polynomial, these are given in Cartesian coordinates and k = d.

DESCRIPTION

The BSplineCurve describes a general spline curve using B-spline basis functions. Splines on this form are often referred to as B-splines.

A *p*th-degree spline curve is defined by

$$\mathbf{C}(u) = \frac{\sum_{i=0}^{n-1} N_i^p(u) w_i \mathbf{P}_i}{\sum_{i=0}^{n-1} N_i^p(u) w_i}, \quad a \leq u \leq b$$

where $\mathbf{P}_i$ are the control points, the w_i are the weights, and the N_i^p are the p th degree B-spline basis functions defined in the nonperiodic and nonuniform knot vector

$$U = \{a, \dots, a, u_{p+1}, \dots, u_{m-p-1}, b, \dots, b\}$$

For $N_i^p(u)$, the following definition is used:

$$N_i^0(u) = \begin{cases} 1 & u_i \leq u < u_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

$$N_i^p(u) = \frac{u - u_i}{u_{i+p} - u_i} N_i^{p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1}^{p-1}(u)$$

For nonrational B-splines, all weights are equal to 1 and the curve can be expressed as

$$\mathbf{C}(u) = \sum_{i=0}^{n-1} N_i^p(u) \mathbf{P}_i, \quad a \leq u \leq b$$

The homogeneous control points $\text{Pw}[i]$ used in the serialization of a rational curve have the components:

$$\begin{aligned} \text{Pw}[i][k] &= w_i P_{i,k}, \quad 0 \leq k < d \\ \text{Pw}[i][d] &= w_i \end{aligned}$$

A polynomial curve has all weights equal to 1.

EXAMPLE

```
12 BSplineCurve # class
2 # version
3 # sdim
0 # rational?
0 # periodic?
3 # degree
# knot vector
8 0 0 0 0 1 1 1 1
# control points
1 0 0
1 0.3333333333333333 0
1 0.6666666666666666 0.3333333333333333
1 1 1
```

SEE ALSO

[BezierCurve](#)

CURRENT VERSION

2

SUBTYPE OF

GeomSurf

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
boolean		1 if the surface is rational, 0 if the surface is polynomial (nonrational).
integer	p	Degree in first parameter.
integer	q	Degree in second parameter.
boolean		1 if the surface is periodic in the first parameter; 0 otherwise.
boolean		1 if the surface is periodic in the second parameter; 0 otherwise.
integer	m1	Length of first knot vector.
double[m1]	U	First knot vector.
integer	m2	Length of second knot vector.
double[m2]	V	Second knot vector.
double[n2][n1][k]	Pw	The control points of the surface. The number of control points, n1 and n2, are given by n1 = m1 - p - 1 and n2 = m2 - q - 1. If the surface is rational these are given in homogeneous coordinates and k = d + 1. If the surface is polynomial these are given in Cartesian coordinates and k = d.

DESCRIPTION

The generalization of B-spline curves to surfaces is a tensor product surfaces given by

$$\mathbf{S}(s, t) = \frac{\sum_{i=0}^{n1-1} \sum_{j=0}^{n2-1} \mathbf{P}_{i,j} w_{i,j} N_i^{p(s)} N_j^{q(t)}}{\sum_{i=0}^{n1-1} \sum_{j=0}^{n2-1} w_{i,j} N_i^{p(s)} N_j^{q(t)}}$$

For N_i^p , the following definition is used:

$$N_i^0(u) = \begin{cases} 1 & u_i \leq u < u_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

$$N_i^p(u) = \frac{u - u_i}{u_{i+p} - u_i} N_i^{p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1}^{p-1}(u)$$

The homogeneous control points $\text{Pw}[j][i]$ used in the serialization of a rational surface have the components:

$$\text{Pw}[j][i][k] = w_{i,j} P_{i,j,k}, 0 \leq k < 3$$

$$\text{Pw}[j][i][d] = w_{i,j}$$

A polynomial surface has all weights equal to 1.

SEE ALSO
[BezierSurf](#)

Ellipse

CURRENT VERSION
1

SUBTYPE OF
[GeomCurve](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	d	Space dimension.
Transform		Transformation class. Only present if d = 3.
double[d]	center	Centerpoint.
boolean		1 if clockwise rotation; 0 otherwise. Only present if d = 2.
double[d]	normal	Normal vector. Only present if d = 3.
double[d]	M	Major axis.
double	rat	Ratio of minor axis length to major axis length.
double	offset	Parameter at the end of major axis.

DESCRIPTION

This curve defines an ellipse in the two or three dimensional space.

In 2D, an ellipse is defined by a centerpoint `center`, a vector defining the major axis `M` of the ellipse (including the magnitude of the major axis), the radius ratio of the minor axis length to the major axis length `rat`, the direction of the ellipse, and the parameter `offset` at the major axis `offset`.

In 3D, an ellipse is defined by a centerpoint `center`, a unit vector normal to the plane of the ellipse `normal`, a vector defining the major axis of the ellipse `M` (including the magnitude of the major axis), the radius ratio, and the parameter `offset` at the major axis `offset`. The direction of the ellipse is defined by the right-hand rule using the normal vector.

An ellipse is a closed curve that has a period of 2π . It is parameterized as:

$$\text{point} = \text{center} + M \cos(t - \text{offset}) + N \sin(t - \text{offset})$$

where `M` and `N` are the major and minor axes, respectively.

Geom1

CURRENT VERSION
2

SUBTYPE OF
[Serializable](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	type	Object type: 0 for a point object, 1 for a curve object, 2 for a solid object, and -1 for a mixed object.
boolean		1 if void regions are labeled; 0 otherwise.
double		Relative geometry tolerance.
integer	nv	Number of vertices.
double[nv]	vtx	Vector of vertex coordinates.
integer[nv][2]	ud	Matrix of integers giving domains on upside and downside of each vertex.
integer	na	Number of attributes.
Attributes[na]		Attributes. The Attribute type is undocumented because it is only used internally.

DESCRIPTION

The `Geom1` type represents a geometry object in 1D that is not an assembly. The geometry object must not be self-intersecting.

EXAMPLE

A solid 1D object (an interval):

```
# Major & minor version
0 1
1 # number of tags
# Tags
5 geom1
1 # number of types
# Types
3 obj

0 0 1
5 Geom1 # class
2 # version
1 # type
1 # voidsLabeled
1e-010 # gtol
3 # number of vertices
# Vertex coordinates
0
1
3
# Vertex up/down
1 0
2 1
0 2
```

Geom2

CURRENT VERSION

2

SUBTYPE OF

[Serializable](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer		Object type: 0 for a point object, 1 for a curve object, 2 for a solid object, and -1 for a mixed object.
boolean		1 if void regions are labeled; 0 otherwise.
double		Relative geometry tolerance.
double		Relative resolution tolerance.
integer	nv	Number of vertices.
VertexData[nv]	vertex	Matrix of vertex data.
integer	ne	Number of edges.
EdgeData[ne]	edge	Matrix of edge data.
integer	nc	Number of curves.
integer	na	Number of attributes.
Attribute[na]		Attributes. The Attribute type is undocumented because it is only used internally.

In the table above, the entity types are defined as follows:

VertexData

ENTITY/OBJECT	VARIABLE	DESCRIPTION
double[2]		Coordinates of the vertex.
int		The domain number if the vertex is isolated. If the vertex is not isolated this value is unspecified.
double		Relative tolerance of the vertex.

EdgeData

ENTITY/OBJECT	VARIABLE	DESCRIPTION
int[2]		Start and end vertex of the edge, respectively (1-based).
double[2]		Parameter values of the two vertices.
int[2]		The left and right domain number of the edge, respectively.
int		Signed index of the underlying curve (1-based). The sign indicates the direction of the edge relative the curve.
double		Relative tolerance of the edge.

DESCRIPTION

The Geom2 type represents a geometry object in 2D that is not an assembly and is not virtual. The geometry object must not be self-intersecting.

EXAMPLE

A straight line in 2D:

```
# Major & minor version
0 1
1 # number of tags
# Tags
3 1s1
1 # number of types
# Types
```



```

3 obj

# ----- Object 0 -----

0 0 1
5 Geom2 # class
2 # version
1 # type
1 # voidsLabeled
1e-10 # gtol
0.0001 # resTol

2 # number of vertices
# Vertices
# X Y dom tol
0 0 -1 NAN
1 1.5 -1 NAN

1 # number of edges
# Edges
# vtx1 vtx2 s1 s2 up down curve tol
1 2 0 1 0 0 1 NAN

1 # number of curves
# Curves

# Curve 1
11 BezierCurve # class
1 # version
2 # sdim
0 # rational?
1 # degree
# homogeneous control points
0 0
1 1.5
# Attributes
0 # nof attributes

```

Geom3

CURRENT VERSION

3

SUBTYPE OF

[Serializable](#)

FIELDS

DATA TYPE	VARIABLE	DESCRIPTION
integer		Version.
integer	type	Object type: 0 for a point object, 1 for a curve object, 2 for a surface object, 3 for a solid object, and -1 for a mixed object.
boolean		1 if void regions are labeled; 0 otherwise.
double		Relative geometry tolerance.
double		Relative resolution tolerance.
integer	nv	Number of vertices.
VertexData[nv]	vertex	Matrix of vertex data.
integer	npv	Number of parameter vertices.
ParamVertexData[npv]	pvertex	Matrix of parameter vertex data.
integer	ne	Number of edges.
EdgeData[ne]	edge	Matrix of edge data.
integer	npe	Number of parameter edges.
ParamEdgeData[npe]	pedge	Matrix of parameter edge data.
integer	nf	Number of faces.
FaceData[nf]	face	Matrix of face data.
integer	nc	Number of curves.
GeomCurve[nc]	curves	Vector of curves.
integer	ns	Number of surfaces.
GeomSurf[ns]	surfaces	Vector of surfaces.
integer	npc	Number of parameter curves.
GeomCurve[npc]	pcurves	Vector of parameter curves.
integer	na	Number of attributes.
Attributes[na]		Attributes. The Attribute type is undocumented because it is only used internally.

In the table above, the entity types are defined as follows:

VertexData

ENTITY/OBJECT	VARIABLE	DESCRIPTION
double[3]		Coordinates of the vertex.
int		The domain number if the vertex is isolated. If the vertex is not isolated this value is unspecified.
double		Relative tolerance of the vertex.

ParamVertexData

ENTITY/OBJECT	VARIABLE	DESCRIPTION
int		Vertex index (1-based).
double[2]		Parameter values in the surface.
int		Face index (1-based).
int		Surface index (1-based).
double		Relative tolerance for the vertex.

EdgeData

ENTITY/OBJECT	VARIABLE	DESCRIPTION
int[2]		Start and end vertex of the edge, respectively (1-based).
double[2]		Parameter values of the start and end vertices.
int		Index of a domain if the edge is not adjacent to a face, and is unspecified otherwise.
int		Signed index of the underlying curve (1-based). The sign indicates the direction of the edge relative to the curve. Is 0 if there is no curve.
double		Relative tolerance of the edge.

ParamEdgeData

ENTITY/OBJECT	VARIABLE	DESCRIPTION
int		Index of the corresponding 3D edge (1-based).
int[2]		Indices of the start and end parameter vertices (1-based index in the vector of ParamVertexData).
double[2]		Parameter values of the start and end vertices.
int[2]		Indices of the left and right faces, respectively (1-based). This is 0 if there is no face.
int		Signed index of the underlying parameter curve. The sign indicates the direction of the edge relative to the curve.
int		Index of the surface (1-based).
double		Relative tolerance of the edge.

FaceData

ENTITY/OBJECT	VARIABLE	DESCRIPTION
int[2]		The up and down domain index of the face.
int		Surface index of the face (1-based).
double		Relative tolerance of the edge.

DESCRIPTION

The `Geom3` type represents a geometry object in 3D represented using the COMSOL kernel that is not an assembly and is not virtual. The geometry object must not be self-intersecting.

GeomCurve

SUBTYPE OF

[Serializable](#)

DESCRIPTION

`GeomCurve` is the abstract base type for all curve types. It has no fields (except for the type ID from `Serializable`).

GeomSurf

SUBTYPE OF

[Serializable](#)

DESCRIPTION

`GeomSurf` is the abstract base type for all surface types. It has no fields (except for the type ID from `Serializable`).

Mesh

CURRENT VERSION

8

SUBTYPE OF

[Serializable](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	sdim	Space dimension (if equal to 0 no more fields).
integer		Include geometric model.
integer		Number of geometric entities per dimension. Used when Include geometric model is 1.
integer		Voids are labeled. Used when Include geometric model is 1.
integer		Number of finite voids. Used when Include geometric model is 1 and Voids are labeled is 1.
integer		Up and down domains for boundaries. Used when Include geometric model is 1.
integer		Isolated edges in domains. Used in 3D when Include geometric model is 1.
integer		Isolated vertices in domains. Used in 2D and 3D and when Include geometric model is 1.
integer	np	Number of mesh vertices.
double[d][np]	p	Mesh points.
integer	nt	Number of element types (equals the number of repeats of the following fields).
string		Element type.
integer	nep	Number of vertices per element.
integer	ne	Number of elements.
integer[ne][nep]	elem	Matrix of point indices for each element.
integer	ndom	Number of geometric entity values.
integer[ndom]	dom	Vector of geometric entity labels for each element.

DESCRIPTION

The geometric entity numbering for points, edges, and boundaries must start from 0 and the geometric entity numbering for solid domains must start from 1 when defining a mesh through a COMSOL Multiphysics mesh file. Domain 0 is reserved for the infinite void and negative numbers are used for finite voids.

For information about the local mesh numbering for the mesh element type in COMSOL Multiphysics, see [Mesh Element Numbering Conventions](#).

EXAMPLE

The following displays a mesh with triangular elements on a unit square. Neither point nor edge elements are present.

```
# Major & minor version
0 1
1 # number of tags
# Tags
5 mesh1
1 # number of types
# Types
3 obj

# ----- Object 0 -----

0 0 1
4 Mesh # class
8 # version
2 # sdim

# Include geometric model
```

```

1
# Geometric model

3 # Number of geometric entities per dimension
0
0
1

0 # Voids are labeled

0 # Up and down domains for boundaries

0 # Isolated vertices in domains


# Mesh data

4 # number of mesh vertices

# Mesh vertex coordinates
0 1
0 0
1 1
1 0

1 # number of element types

# Type #0

3 tri # type name


3 # number of vertices per element
2 # number of elements
# Elements
0 1 2
3 2 1


2 # number of geometric entity indices
# Geometric entity indices
1
1

```

MeshCurve

CURRENT VERSION

1

SUBTYPE OF

[GeomCurve](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	d	Space dimension.
integer	np	Number of interpolation points.
double[np][d+1]	P	Coordinates and parameters for each interpolation point.
BSplineCurve		Interpolating curve.

In the matrix P , the first d values of each row are the coordinates for the point and the $(d+1)^{\text{th}}$ value is the parameter.

DESCRIPTION

A MeshCurve represents a cubic spline interpolation curve through given interpolation points. The corresponding parameter values are also given as an increasing sequence of numbers.

SEE ALSO

[BSplineCurve](#)

MeshSurf

CURRENT VERSIONS

1

SUBCLASS OF

[GeomSurf](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	nv	Number of mesh vertices.
double[nv][5]	p	Coordinates and parameters for each mesh vertex.
integer	nt	Number of mesh triangles.
integers[nt][3]	elem	Matrix of (0-based) mesh vertex indices for each mesh triangle.

DESCRIPTION

A MeshSurf represents a continuously differentiable surface formed by using piecewise quadratic interpolation on a surface mesh. The surface mesh is given by a set of mesh vertices and a matrix `elem` defining the triangle connectivity. The surface parameter values of each mesh vertex is also given. For each mesh triangle, a quadratic interpolation is used on each of the four subtriangles.

SEE ALSO

[Mesh](#)

PolChain

CURRENT VERSION

1

SUBTYPE OF

[GeomCurve](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	np	Number of polygon points.
double[np][2]	pol	Coordinates of the polygon points.

DESCRIPTION

A PolChain represents a polygon in 2D with a parameter interval $[i/(np-1), (i+1)/(np-1)]$ on the i th polygon segment. Using $np > 2$ is not recommended because it gives a curve with tangent discontinuities.

SEE ALSO

[MeshCurve](#)

Selection

CURRENT VERSION

0

SUBTYPE OF

[Serializable](#)

FIELDS

ENTITY/OBJECT	DESCRIPTION
integer	Version.
string	Selection label. The string is encoded in UTF-8.
string	Tag of corresponding object (mesh) in file.
integer	Dimension of selection (0: vertex; 1: edge; 2: face; 3: domain in 3D).
integer	Number of entities.
integer[]	The indices of the entities for the selection. The integers specify the 0-based indices of the entities (1-based for domains).

DESCRIPTION

Selections can appear in files containing a mesh. Each selection refers to a set of entities that needs to be defined by the mesh in the file.

EXAMPLE

The following example displays a domain selection in 3D named Fluid, specifying domains 1 and 3:

```
0 # Version
5 Fluid # Label
5 mesh1 # Geometry/mesh tag
3 # Dimension
2 # Number of entities
# Entities
1
3
```


Serializable

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
String		Type ID for the subtype.

The subtypes have additional fields.

DESCRIPTION

Serializable is the abstract base type for all other types.

Straight

CURRENT VERSION

1

SUBTYPE OF

[GeomCurve](#)

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	d	Space dimension.
double[d]	root	The point on the line which corresponds to the parameter value 0.
double[d]	dir	The direction in vector of the line.
double	pscale	Parameter scale.

DESCRIPTION

This curve defines an infinite straight line in the two-dimensional or three-dimensional space. It is represented by a point and a unit vector specifying the direction. A straight also has a scale factor for the parameterization, so that the parameter values can be made invariant under transformation.

A straight line is an open curve that is not periodic. It is parameterized as:

$$\text{pos} = \text{root} + u * \text{pscale} * \text{dir}$$

where u is the parameter.

Transform

CURRENT VERSION

0

FIELDS

ENTITY/OBJECT	VARIABLE	DESCRIPTION
integer		Version.
integer	d	Space dimension.
boolean		1 if transformation is a unit transformation; 0 otherwise. If the value is 1, no more fields are present.
double [d+1][d+1]	M	Values in transformation matrix.
boolean		1 if determinant is positive; 0 otherwise.
boolean		1 if matrix is isotropic; 0 otherwise.

DESCRIPTION

The transformation class is defined by the transformation matrix, which operates as a premultiplier on column vectors containing homogeneous coordinates thus

$$\begin{bmatrix} x' & y' & z' & s' \end{bmatrix} = M \cdot \begin{bmatrix} x & y & z & s \end{bmatrix}'$$

where the conventional 3D coordinates are

$$\begin{bmatrix} x & y & z \\ s & s & s \end{bmatrix}$$

The matrix thus consists of

$$\begin{bmatrix} & & & T_x \\ & \mathbf{R} & & T_y \\ & & & T_z \\ 0 & 0 & 0 & S \end{bmatrix}$$

where **R** is a nonsingular transformation matrix, containing the rotation, reflection, nonuniform scaling, and shearing components; **T** is a translation vector; and *S* is a global scaling factor greater than zero.

Example of the Serialization Format

To illustrate the use of the serialization format, the following example shows how to create a planar surface object with one face:

```
A PLANAR SURFACE OBJECT WITH ONE FACE
# Major & minor version
0 1
1 # number of tags
# Tags
3 fin
1 # number of types
# Types
3 obj

# ----- Object 0 -----

0 0 1
5 Geom3 # class
3 # version
```

```

2 # type
1 # voidsLabeled
1e-10 # gtol
0.0001 # resTol

4 # number of vertices
# Vertices
# X Y Z dom tol
0 0 0 -1 NAN
0 1 0 -1 NAN
2 0 0 -1 NAN
2 1 0 -1 NAN

4 # number of parameter vertices
# Parameter vertices
# vtx s t fac surf tol
1 0.25 0.25 -1 1 NAN
2 0.25 0.375 -1 1 NAN
3 0.5 0.25 -1 1 NAN
4 0.5 0.375 -1 1 NAN

4 # number of edges
# Edges
# vtx1 vtx2 s1 s2 dom curve tol
2 1 1 0 -1 -4 NAN
1 3 1 0 -1 -1 NAN
4 2 1 0 -1 -3 NAN
3 4 1 0 -1 -2 NAN

4 # number of parameter edges
# Parameter edges
# edg v1 v2 s1 s2 up down pcurve surf tol
1 2 1 0 1 1 0 1 1 NAN
2 1 3 0 1 1 0 2 1 NAN
3 4 2 0 1 1 0 3 1 NAN
4 3 4 0 1 1 0 4 1 NAN

1 # number of faces
# Faces
# up down surf tol
0 0 1 NAN

4 # number of curves
# Curves

# Curve 1
11 BezierCurve # class
1 # version
3 # sdim
0 # rational?
1 # degree
# control points
2 0 0
0 0 0

# Curve 2
11 BezierCurve # class
1 # version
3 # sdim
0 # rational?
1 # degree
# control points
2 1 0
2 0 0

```

```

# Curve 3
11 BezierCurve # class
1 # version
3 # sdim
0 # rational?
1 # degree
# control points
0 1 0
2 1 0

# Curve 4
11 BezierCurve # class
1 # version
3 # sdim
0 # rational?
1 # degree
# control points
0 0 0
0 1 0

1 # number of surfaces
# Surfaces

# Surface 1
9 BezierTri # class
1 # version
# control points
-2 -2 0
6 -2 0
-2 6 0

4 # number of parameter curves
# Parameter curves

# Parameter curve 1
11 BezierCurve # class
1 # version
2 # sdim
0 # rational?
1 # degree
# control points
0.25 0.375
0.25 0.25

# Parameter curve 2
11 BezierCurve # class
1 # version
2 # sdim
0 # rational?
1 # degree
# control points
0.25 0.25
0.5 0.25

# Parameter curve 3
11 BezierCurve # class
1 # version
2 # sdim
0 # rational?
1 # degree
# control points
0.5 0.375
0.25 0.375

```

```
# Parameter curve 4
11 BezierCurve # class
1 # version
2 # sdim
0 # rational?
1 # degree
# control points
0.5 0.25
0.5 0.375

# Attributes
0 # nof attributes
```


I n d e x

ID

- asymmetric geometry 137
- cut point dataset 783
- cut point datasets 782
- function plots 820
- geometric entity counter methods 272
- geometric primitives 249
- global plots 832
- grid datasets 853
- histogram plots 856
- impulse response 868
- intervals 356, 358
- line graphs 911
- matrix representing vertices 275
- nonlocal couplings 95
- plot groups 1000, 1007
- point graphs 1020
- points, reflecting 365
- polarization plots 1034
- revolved datasets 1069
- table histogram plots 1155
- table plots 1139
- through-thickness plot 1170

2D

- asymmetric geometry 137
- Bézier polygons 292
- boundary similarity 100
- chamfers 297
- circles 299
- circular arcs 301
- cut point dataset 783
- cut point datasets 782
- DXF files, importing 348
- ellipses 328
- fillets 337
- geometric entity counter methods 272
- getVertex 275
- grid datasets 853
- height attribute 854
- lines, reflecting 365
- matrix histogram plots 926
- nonlocal couplings 95
- objects, rotating 396
- parameterized curve datasets 966
- parametric curves 370
- plot groups 1000
- Polygons 384

- rational Bézier curves 317, 388
- rectangles 389
- rectangular arrays 287
- revolved datasets 1069
- squares 404
- tangents 408
- work planes 418

3D

- Bézier polygons 292
- block shaped arrays 287
- blocks 294
- boundary similarity 99
- cut point dataset 783
- cut point datasets 782
- cylinders 319
- ellipsoids 330
- extruding planar objects 334
- geometric entity counter methods 272
- getVertex 275
- grid datasets 853
- helix 340
- nonlocal couplings 95
- objects, rotating 396
- parameterized curve datasets 966
- parametric curves 370
- parametric datasets 966
- parametric surfaces 372
- planar objects, revolving 392
- planes, reflecting 365
- plot groups 1000
- Polygons 384
- pyramids 386
- rational Bézier curves 317, 388
- rectangular arrays 287
- spheres 400
- sweeps 406
- tetrahedra 411
- torus 413
- work planes 418

A AberrationHeight 854
absolute repair tolerance 264
absorbing layer 94
access methods 29
Adapt (meshes) 455
Adaption (meshes) 640
adaptively refined meshes 438
adding

- geometry 255
 - height attributes to 2D plots 854
 - meshing sequences 428
- adjacency information, geometry 272
- AdjacentSelection (meshes) 456
- Advanced (solvers) 556
- analytic functions 112
- angular units 137, 262
- Animation (plots) 737
- Annotation (plots) 742
- annotation data plots 745
- AnnotationData (plots) 745
- application library examples
 - grid data formats 1198
- Array 287
- arrays, geometry 275
- arrow data plots 749
- ArrowData (plots) 749
- ArrowLine (plots) 752
- ArrowPoint (plots) 752
- ArrowSurface (plots) 752
- ArrowVolume (plots) 752
- Assemble (solvers) 559
- assembly geometry type, checking for 253
- asymptotic waveform evaluation (AWE) 561
- Atolglobalmethod (solvers) 637
- attribute feature, meshes 428
- attributes, for geometry 257
- attributes, methods for in geometry 260
- automatic rescaling of linear equations 556
- Average (datasets) 761
- average coupling operator 96
- AvVolume, AvSurface, AvLine 757
- AWE (solvers) 561
- axisymmetric geometries 137
- B**
 - Ball (meshes) 457
 - ball, meshes 457
 - BallSelection (meshes) 458
 - base-vector coordinate system 88
 - Batch (job type) 51
 - batch job 49
 - batch jobs
 - running compiled model files 22
 - Beam (datasets) 762
 - Bernstein basis 1257
 - Bézier curve, file format 1257
 - Bézier surface rectangular, file format 1258
 - BezierPolygon 292
 - binary data file formats 1253
 - Block 294
 - block shaped arrays 287
 - block size, assembly 557
 - block versions, meshes 447
 - blocking, other clients from a server 41
 - BndLayer (meshes) 461
 - BndLayerProp (meshes) 463
 - Boolean operations 306
 - boundary coordinate systems 90
 - boundary layers, meshes 461
 - boundary probes 182
 - boundary similarity coupling 99
 - Box (meshes) 465
 - box, meshes 464
 - BoxSelection (meshes) 458
 - B-spline curve 1259
 - B-spline surface 1261
 - build status, meshes 160, 431
 - building
 - geometry 139, 257
 - meshes 160, 430
 - Bunch–Kaufman pivoting 590
- C**
 - C linkage 120
 - C matrix 626
 - cache, clearing 41
 - CAD Import Module
 - 3D kernels 263
 - arrays 275
 - cadps 138
 - defeaturing 140
 - geometry, exporting 280
 - geometry, importing 354
 - hasCadRep 253
 - cadps 138
 - CentroidMeasurement 297, 323
 - CGNS function 123
 - Chamfer 298
 - changing units 262
 - check for tag 33
 - Circle 299
 - circleperpendicular, plane type 419
 - CircularArc 301
 - class files 18, 22
 - clearing cache 41
 - clipping tools 236
 - Cluster (job type) 52
 - coarse grid solver types 587
 - coefficient form 59
 - coefficient form equations 59

- Color 762
- color table files 1200
 - separators in 1200
- color tables
 - methods for 732
- combined coordinate systems 92
- CombineSolution (solvers) 564
- combining solutions 564
- Comparison 765
- compiled model files
 - running as batch jobs 22
- compiled model files for Java
 - running from Desktop 22
- compiling model files for Java 20
- ComplementSelection (meshes) 526
- component
 - getting the type of 84
- component nodes 83
- Compose (geometry) 305
- composite coordinate systems 92
- COMSOL Desktop 20
- COMSOL Multiphysics files 1253
- COMSOL server 43
- Cone 311
- connecting to a server 43
- constraint force Jacobian
 - input matrix 586
- constraint Jacobian
 - assemble 553, 559
 - input matrix 586
- constraint vector
 - assemble 553, 559
 - input matrix 586
- constraints 85
- contact pairs 171
- continuing operations, meshes 450
- Contour 765, 771
- control points 1257
- Convert (meshes) 466
- converting objects 314
- coordinate names 110
- coordinate system 87
- coordinate system from geometry 91
- coordinates, mesh vertices 444
- coordinates, plane type 419
- CoordSysLine (plots) 777
- CoordSysPoint (plots) 777
- CoordSysSurface (plots) 777
- CoordSysVolume (plots) 777
- Copy 472
 - Copy (geometry) 367
 - copy, of solution 568
 - CopyDomain 471
 - CopyEdge 468
 - CopyFace 469
 - CornerProp (meshes) 474
 - CornerRefinement (meshes) 475
 - cpl_BoundarySimilarity 99
 - CreateDomains (meshes) 475
 - CreateEdges (meshes) 476
 - CreateFaces (meshes) 478
 - CreateVertices (meshes) 479
 - CubicBezier 317
 - cumulative selection 268
 - curves, creating 292, 384
 - CutLine2D (datasets) 780
 - CutLine3D (datasets) 780
 - CutPlane (datasets) 781
 - CutPoint (datasets) 783
 - Cylinder 319
 - Cylinder (meshes) 480
 - cylinder, meshes 480
 - CylinderSelection (meshes) 458
 - cylindrical coordinate systems 90

D

- D matrix 626
- DAE 167, 638
- damped Newton method 581
- damping matrix
 - assemble 553, 559
 - input matrix 586
- damping ratios 602
- Data (plots, exporting) 784
- data job type 54
- data types 32
- data, mesh 445
- datasets 194
- deep neural network function 132
- defeaturing, in CAD 140
- Deform (plots) 787
- degree, rational Bézier curve and 1257
- Delete (geometry) 321
- DeleteEntities (meshes) 482
- deleting
 - geometry 259
 - meshes 431
 - meshing sequences 159
- dependent variables 143
- derived values 727
- description of model 45

- destination map
 - extrusion coupling 97
 - projection coupling 100
- DetectFaces (meshes) 482
- Difference (geometry) 305
- DifferenceSelection (meshes) 526
- differential algebraic equation 638
- Direct (solvers) 587
- direct properties 589
- Directivity 788
- Dirichlet boundary conditions 604, 608
- disabling
 - geometry 259
 - mesh features 431
- discontinuous Lagrange elements 534
- DiskSelection (meshes) 458
- Distribution (meshes) 483
- dom, meshes 475
- domain mesh, copying 470
- domain probes 182
- domains, deleting 321
- drop tolerance 591
- duplicates, sectionwise data and 1199
- DXF, CAD drawing 348
- E**
 - eccentric oblique cones, creating 324
 - Eclipse
 - setting up for compiling Java files 22
 - ECone 324
 - edg, meshes 476
 - Edge (datasets) 793
 - Edge (meshes) 484
 - Edge 2D and 3D datasets 792
 - edge elements, meshes 448
 - edge evaluation, geometry 273
 - edge map 99
 - edge meshes, copying 468
 - edge probes 182
 - edgeangle, plane type 419
 - EdgeGroup 485
 - EdgeMap 486
 - edgeparallel, plane type 419
 - edges, deleting 321
 - edited status, meshes 431
 - editing
 - geometry 256
 - meshes 429
 - eigenmodal method 601
 - eigenpairs 602, 605
 - Eigenvalue 568
 - Eigenvalue (solvers) 556
 - element 102
 - element sets 105
 - elevation function 122
 - Ellipse 328
 - ellipse, file format 1262
 - Ellipsoid 330
 - emailing COMSOL 18
 - Embed (datasets) 793
 - Embed 1D and 2D datasets 793
 - EnergyDecay plot attribute 793
 - environment mapping 237
 - error features 257
 - error status, meshes 431, 450
 - ErrorBars plot attribute 794, 1154
 - Euler step 638
 - Eval 795
 - EvalGlobal 799
 - EvalPoint 806
 - evaluate expressions 796
 - evaluation groups 813
 - event 215
 - Excel formats 1199
 - excluding computed data 45
 - expand (meshes) 511
 - explicit element distribution 484
 - explicit time stepping 644
 - ExplicitSelection (meshes) 487–488
 - exploding objects 402
 - Export (plot attribute) 815
 - export feature 194
 - exporting
 - data 1197
 - geometry 140, 280
 - meshes 161, 452
 - exporting images 240
 - external function 120
 - external material 148
 - extrapolation tolerance 97
 - Extrude 334
 - Extrude (datasets) 816
 - extruding, work planes 266
 - extrusion coupling operator 97
- F**
 - fac, meshes 478
 - face evaluation, geometry 274
 - face mesh, copying 469
 - faceparallel, plane type 419
 - faces, deleting 321
 - fast Fourier transform 575

- feature status 258
- FFT 575
 - spatial 1091
- field 107
- file formats
 - binary data 1253
 - text 1255
 - text data 1253
- fill, meshes 488
- Fillet 337
- Filter (dataset) 817
- Filter (Particle) (plot attribute) 819
- Filter (plot attribute) 818
- Filter (Ray) (plot attribute) 819
- Filter (Table Graph) (plot attribute) 819
- finalized geometry 255
- finite voids 272
- FixedMesh (meshes) 489
- flattened corners, creating 297
- fonts 45
- force null-space basis 553, 559
- frame feature 109
- frames 109
- FreeQuad (meshes) 489
- FreeTet (meshes) 491
- FreeTri (meshes) 492
- FromMesh 340
- frustums, creating 324, 386
- FullyCoupled (solvers) 579
- function switch 111
- Function2D (plots) 824
- Function3D (plots) 826
- functions 111
- G** Gaussian pulse function 116
- geneal contact pairs 171
- general extrusion couplings 98
- general matrix information 552
- general projection couplings 101
- geometric entities 35, 272
- geometry 137
 - arrays 275
 - exporting 280
 - name 110
 - object names 258
 - object selection 140
 - objects, moving and copying 367
 - representation 263
 - selection 35
- geometry cleanup 269
- geometry features 137
- geometry modeling kernels 263
- geometry objects
 - importing 353
- geometry sequences
 - constructing 255
 - importing 350
- geometry shape function 84
- get* 29
- global
 - attribute features 428
 - selections 35
 - variable probes 182
- Global (plot) 832
- global equations 166
- Global numerical) 830
- gradient/Jacobian evaluation method 613
- GraphMarker (plots) 850
- Grid (datasets) 853
- grid data formats 1198
- grouping, of nodes 165
- groups 143
- growth rate
 - mesh 442
- H** hasTag 33
- Height 854
- Helix 340
- hexahedral elements, numbering 449
- Hexahedron 343
- highly nonlinear 584
- Histogram (plot) 856
- HistogramHeight 854
- hostname 43
- I** identity mapping couplings 100
- identity pairs 171
- IFFT 575
- Image (exporting) 861
- Image (plot attribute) 866
- Image (plot) 864
- image function 122
- images
 - exporting 240
 - plotting 240
- images, exporting 861
- Import (geometry) 348
- Import (meshes) 494
- importing
 - data 1197
 - geometry objects 353

- geometry sequences 350
 - materials 150
 - meshes 429, 494
 - mphbin 353
- importing data from file 111, 180
- Imprint (meshes) 500
- imprints, creating 339
- impulse response 868
- ImpulseResponse (plot) 868
- infinite elements 93
- infinite void 272
- initial values 143
- installation path 22
- Integral (datasets) 761
- integration coupling operator 96
- integration rules 59, 144
- intermediate space 97, 100
- internet resources 18
- Interp (datasets) 875
- interpolation functions 113
- Intersection (geometry) 305
- IntersectionPoint2D (datasets) 878
- IntersectionPoint3D (datasets) 878
- IntersectionSelection (meshes) 526
- IntersectLine (meshes) 501, 503
- Interval (geometry) 357
- IntLine (results feature) 757, 880, 935
- IntSurface (results feature) 757, 880, 935
- IntVolume (results feature) 757, 880, 935
- inverse fast Fourier transform 575
- inverted elements 441
- Isosurface (datasets) 888
- Isosurface (plots) 884, 889
- Iterative (solvers) 587
- iterative properties 591
- J**
 - Jacobian update technique
 - fully coupled 582
 - segregated step 623
 - Java
 - memory, meshing limitations 447
 - syntax 31
 - job type 49
 - join, meshes 506
 - JoinEntities (meshes) 506
- K**
 - kernels 263
 - knowledge base, COMSOL 19
 - Krylov space 571
- L**
 - LayeredMaterial (datasets) 897
 - LayeredMaterialSlice (plots) 898
 - length units 137, 262
 - Line (plots) 903, 916
 - line data plots 908
 - line integration coupling operator 97
 - Linear (solvers) 586
 - linear extrusion couplings 99
 - linear extrusion map properties 98
 - linear projection coupling operator 101
 - linearization point method 571
 - LineData (plots) 908
 - LineGraph 911
 - LineSegment (geometry) 358
 - load group 143
 - load vector
 - assemble 552, 559
 - input matrix 586
 - local attribute feature 428
 - LogicalExpression (meshes) 506
 - lower bound constraint vector 553, 559
 - LowerLimit (solvers) 600
 - LumpedStep (solvers) 600
- M**
 - Map (meshes) 507
 - mapped coordinate systems 88
 - Marker (plots) 919
 - mass matrix
 - assemble 553, 559
 - input matrix 586
 - mass properties 145
 - MassProperties (results feature) 922
 - material frames 109
 - material link 148
 - material models 147
 - material switch 148
 - MaterialAppearance (plots) 925
 - materials 148
 - importing 150
- MATLAB**
 - functions, general 122
 - LiveLink for 18
 - syntax 31
 - syntax for data types 31
- matrix assembly 559
- matrix creation 554
- matrix data 553
- Matrix Histogram (plot) 926
- matrix properties 32
- Maximum (datasets) 761
- maximum and minimum couplings 96
- MaxMinLine (plots) 930

- MaxMinPoint (plots) 930
- MaxMinSurface (plots) 930
- MaxMinVolume (plots) 930
- Mc, McA, McB matrices 626
- MeasureLine (results feature) 939
- MeasureSurface (results feature) 939
- MeasureVolume (results feature) 939
- measuring
 - entities 277
 - geometric entities 277
 - geometry 140
 - objects 278
- measuring average position 297
- measuring distance 323
- MergeEntities (meshes) 508
- mesh
 - growth rate 442
 - importing 494
- Mesh (datasets) 945
- Mesh (exporting) 946
- Mesh (plots) 942
- mesh component 430
- mesh control entity 270
- mesh element sets 105
- mesh parts 430
- mesh, copying
 - copying meshes 472
- mesh, repressing a geometry 253
- MeshError 450
- meshes
 - frames 109
 - importing 494
 - refining 512, 640
 - remeshing domains 514
 - remeshing edges 514
 - remeshing faces 515
 - sequences for 159
 - setting data 161
- MeshInformation 450
- meshing sequences 159
- MeshWarning 450
- Minimum (datasets) 761
- Mirror (datasets) 947
- Mirror (geometry) 365
- Modal (solvers) 601
- modal reduction 605, 617
- ModalReduction (solvers) 605
- ModeFollowing (solvers) 608
- model
 - 44
- .attr() 47
- .attr(<tag>) 48
- .batch 49
- .bem 56
- .coeff 58
- .common 61
- .component 82
- .constr 85
- .coordSystem 87
- .cpl 94
- .elem 101
- .elementSet 104
- .field 107
- .frame 108
- .func 110
- .geom 135
- .init 143
- .intRule 144
- .material 146
- .mesh 158
- .methodCall 163
- .modelName 164–165
- .multiphysics 164
- .ode 166
- .opt 168
- .pair 170
- .param 174
- .physics 176
- .probe 181
- .reduced 184
- .result 193
- .savePointt 199
- .selection 200
- .shape 209
- .sol 211
- .solverEvent 215
- .study 217
- .thermodynamics 222
- .unitSystem 224–225
- .variable 226
- .view 228
- .weak 238
- Model Builder 20
- model directory 40
- model entity 48
- model entity list 47
- model files for Java
 - compiling 20
 - structure of 21
- model methods, in Model Java-files 164

- model object 20
- model path 46
- model tree 83
- models, adding geometry 255
- ModelUtil 39
- ModelUtil.connect 41
- Move (geometry) 367
- mphbin 354
- mphbin files 1255
- mphtxt 354
- mphtxt files 1255
- Multigrid (solvers) 587
- multigrid levels 219
- multigrid properties 594
- multisolutions 546
- MUMPS 590
- N**
 - named selection 200, 289, 360, 416, 458
 - naming, geometry objects 259
 - NASTRAN bulk data format 500
 - NASTRAN files 453
 - needs rebuild status, meshes 431
 - Neumann boundary conditions 604, 608
 - Newton iterations 582
 - nodal discontinuous Lagrange elements 534
 - node groups 165
 - nonlinear solver 579
 - nonlocal couplings 95
 - normal distribution function 119
 - normal, plane type 419
 - null matrix 626
 - null-space basis 553, 559
 - null-space function 557
 - number of elements, meshes 440
 - number of parameter values, for stepping over 727
 - numbering conventions, meshes 448
 - numerical results feature 194
 - NURBS curve 1259
 - NURBS surface 1261
 - Nyquist plots 953
- O**
 - objective functions 168
 - objects, deleting 259, 321
 - objects, file structure 1253
 - OctaveBand 957
 - ODE 167
 - one point map 99
 - OnePointMap (meshes) 509
 - operation features, geometry 255
 - operation features, meshes 428
 - Optimization 609
 - optimization 168
 - Optimization (study node) 50
 - optimization constraint 553, 559
 - orthonormal system 88
 - out-of-core solvers 590
- P**
 - pair 171
 - parameter checks 370
 - parameters 175
 - Parametric (datasets) 965
 - Parametric (job type) 49
 - Parametric (solvers) 615
 - ParametricCurve (geometry) 371
 - Parasolid kernel 271
 - ParCurve (datasets) 965
 - PARDISO 590
 - parent entity, getting 30
 - ParSurface (datasets) 966
 - partial fraction fit function 135
 - Particle (1D) (plots) 972
 - Particle (datasets) 976
 - Particle (plots) 967
 - Particle Bin (datasets) 980
 - ParticleMass (plots) 980
 - ParticleTrajectories (plots) 986
 - ParticleTrajectoriesFilter (plot attribute) 819
 - PartInstance (geometry) 374
 - Partition (dataset) 990
 - password protection 40
 - perfectly matched layer 93
 - physical quantities, list of 153
 - physics interface 178
 - physics-controlled meshing 436
 - piecewise function 115
 - Pipe (datasets) 998
 - pivot perturbation threshold 590
 - planar objects 334
 - Player (plots) 737
 - Plot (exporting) 998
 - plot arrays, properties for 1011
 - plot group 194
 - plot, meshes 442
 - PlotGroup 813, 1000
 - plots, color attribute 763
 - plotting images 240
 - plotting, retrieving the view to use 194
 - PlugFlow 616
 - PML 93
 - Point (geometry) 383
 - Point (plots) 1013

- point data plots 1018
- point normals 726
- point plots 1014
- point probes 182
- PointData (plots) 1018
- PointGraph 1020
- PointTrajectories (plots) 1026
- PointTrajectoriesFilter (plot attribute) 819
- polar coordinate systems 1030
- PolarGroup 1030
- polarization plots 1033
- Polygon 384
- polynomial chaos expansion (PCE) function 128
- porous materials 148
- port number 43
- preconditioner feature types 587
- preordering algorithm 590
- presentation 195
- PreviousSolution (solvers) 618
- Principalline (plots) 1037
- PrincipalSurface (plots) 1037
- PrincipalVolume (plots) 1037
- prism elements, numbering 449
- probe 181
- processes 54
- projection coupling operator 100
- ProperOrthogonalDecomposition (solvers) 617
- properties, boundary layers 464
- Pyramid 386
- pyramid elements, numbering 449
- Q** QuadraticBezier 388
 - quadrilateral elements, numbering 448
 - quality of elements, meshes 440
 - quality of elements, statistics 439
 - quick, plane type 419
- R** RadiationPattern (exporting) 1047
 - ramp function 116
 - random function 119
 - rational Bézier curves 317, 388
 - Ray (1D) (plots) 1049
 - Ray (datasets) 1054
 - Ray Bin (datasets) 1057
 - RayTrajectories (plots) 1058
 - RayTrajectoriesFilter (plot attribute) 819
 - reaction forces
 - computation and storage of, stationary 629
 - computation and storage of, time dependent 639
 - Rectangle (geometry) 389
 - rectangle function 117
 - rectangular arrays 287
 - Reference (meshes) 511
 - Refine (meshes) 512
 - reflecting objects 365
 - registry key 22
 - relative repair tolerance 138, 264
 - relative tolerance 572
 - relaxation factor 592
 - RemeshDomains (meshes) 514
 - RemeshEdges (meshes) 514
 - RemeshFaces (meshes) 515
 - RemoveDetails (geometry) 391
 - render index 726
 - repair tolerances 264
 - report 195
 - ResponseSpectrum (2D and 3D datasets) 1067
 - results 712
 - Revolve (datasets) 1069
 - Revolve (geometry) 392
 - revolving, work planes 266
 - RGB color values 1200
 - RigidTransform (geometry) 395
 - Rotate (geometry) 396
 - rotated coordinate systems 89
 - Runge–Kutta explicit time stepping 644
- S** SAI preconditioner 588
 - save point model 199
 - saving
 - COMSOL Multiphysics native data files 1253
 - Scale (geometry) 398
 - Scale (meshes) 516
 - scale vector 553, 559
 - scaling coordinate systems 92
 - scaling values 262
 - ScatterSurface (plots) 1070
 - ScatterVolume (plots) 1070
 - second-order elements, checking for 440
 - sectionwise data formats 1198
 - sector symmetry pairs 171
 - Segregated (solvers) 618
 - SegregatedStep (solvers) 621
 - selection
 - named 200, 289, 360, 416, 458
 - selection 34
 - Selection (dataset attribute) 1075
 - Selection (plot attribute) 1076
 - selections
 - editing 256
 - frames and 110

- meshes, and 432
- selections, in geometry 256
- sensitivity 168
- Sensitivity (solvers) 624
- separators, in color table files 1200
- sequences of operations 20
- sequences, meshing 436
- serialization
 - objects 1253
 - terminology 1255
 - types 1253
- serialization, tags 1253
- servers, connecting 41
- set (assign parameters) 30
- setIndex(data types) 32
- shape function 210
- shape functions, constraints and 85
- sharg_2_5 Argyris element 531
- shbub bubble element 532
- shcurl 533
- shcurl2 534
- shdens density element 535
- shdisc discontinuous element 534
- shdiv divergence element 536
- shdiv2 divergence type 2 element 537
- shgp Gauss point data elements 535
- shherm Hermite element 532
- shlag Lagrange element 530
- shnsepr nodal serendipity element 530
- simplex meshes, converting 466
- Size (meshes) 517
- SizeExpression (meshes) 520
- Slice (plots) 1077
- SmithGroup 1084
- SobolIndex (datasets) 1087
- solid polygons, creating 292, 384
- Solution (datasets) 1089
- solution copy 568
- SolutionCopy (solvers) 568
- solutions
 - combining 564
 - creating 551
 - object data 545
- solver sequence 212
- solver sequences 545
- source map, extrusion coupling 97
- source map, projection coupling 100
- sparse approximate inverse preconditioner 588
- spatial coordinates 110
- Spatial FFT (datasets) 1091

- spatial Fourier transform 1091
- spatial frames 109
- Sphere 400
- spherical coordinate systems 91
- spline curve 1259
- spline surface 1261
- Split (geometry) 402
- splitmethod (meshes) 466
- SPOOLES 590
- SpotDiagram (plots) 1092
- spreadsheet data formats 1197
- Square 404
- StatAcceleration (solvers) 625
- StateSpace 625
- static linearized model 626
- Stationary (solvers) 556, 627
- statistics, meshes 161, 439
- status, meshes 431, 442
- step function 117
- stiffness matrix
 - assemble 552, 559
 - input matrix 586
- StopCondition (solvers) 629
- stopping operations, meshes 450
- storing, mesh data 445
- Streamline (plots) 1099
- StreamlineMultislice (plots) 1107
- StreamlineSurface (plots) 1113
- structured quadrilateral mesh, creating 508
- study sequence 219
- study types 218
- StudyStep (solvers) 212, 630
- summary of commands 712
- Surface (datasets) 1124
- Surface (plots) 1119
- surface data plots 1125
- Surface Slit (plots) 1128
- SurfaceData (plots) 1124
- Sweep (geometry) 406
- Sweep (meshes) 521
- symmetric matrices 558

T

- Table (data) 1134
- Table (exporting) 1138
- Table (plots) 1138
- table feature 195
- TableAnnotation (plots) 1142
- TableArrow (plot) 1144
- TableColor 1146
- TableContour (plot) 1148

- TableDOE (data) 1152
- TableFilter (plot attribute) 819
- TableHeight 854
- TableHistogram (plot) 1155
- TablePoint (plots) 1158
- TableSurface (plots) 1162
- TableUQ (data) 1166
- tag
 - checking for 33
- tags, file structure and 1253
- tags, naming 27
- Tangent (geometry) 408
- tangential derivative variables
 - for Argyris elements 531
 - for bubble elements 533
 - for curl elements 534
 - for discontinuous elements 535
 - for divergence elements 536
 - for Hermite elements 531–532
 - for Lagrange elements 530
- task 53
- task type 53
- task type property 54
- technical support, COMSOL 19
- tetrahedral elements, numbering 448
- tetrahedral mesh, creating 490
- Tetrahedron (geometry) 411
- text data file format 1253
- text files 1255
- Time (solvers) 631
- Time Average (dataset) 1168
- Time Integral (dataset) 1168
- TimeDiscrete (solvers) 642
- TimeParametric (solvers) 574, 645
- title, of model 46
- torque arrows 1014
- Torus 413
- Touchstone (export) 1175
- transferring, mesh data 445
- Transform (meshes) 523
- Transformation2D (datasets) 1176
- Transformation3D (datasets) 1176
- transformed, plane type 419
- Transparency (plots) 1178
- transposed form 593
- triangle function 118
- triangular elements, numbering 448
- triangular meshes, unstructured 492
- tube data plots 1180
- TubeData (plots) 1180
- two point map 100
- TwoPointMap (meshes) 524
- type of component 84
- U**
 - undefined numerical values, checking 558
 - Union (geometry) 305
 - Union (meshes) 466, 525
 - UnionSelection (meshes) 526
 - unit system entity 225
 - unit systems 225
 - unit systems, model objects 44
 - unstructured quadrilateral mesh, creating 489
 - upper bound constraint vector 553, 559
 - UpperLimit (solvers) 646
 - Uzawa iterations 618
- V**
 - Vanka method 593
 - variable name suffix 110
 - variables 227
 - file formats and 1197
 - Variables (solvers) 647
 - Vertex (meshes) 527
 - vertices, deleting 321
 - vertices, plane type 419
 - view, for use when plotting 194
 - views 228
 - virtual composite edge, face, domain, and entity 270
 - virtual geometry 253
 - virtual operations 255, 270
 - VisualEffects (plots) 1182
 - voids
 - entity numbers for 36
 - selecting all 35
 - Volume (plots) 1183
 - volume of elements, meshes 442
 - vtx, meshes 479
- W**
 - warning status, meshes 431, 450
 - warnings, build operations 257
 - Waterfall (plots) 1187
 - Wave Form PDE 60
 - wave functions 118
 - weak equations 167
 - weak expressions 238
 - weak expressions, global equations and 167
 - weak form 238
 - weak form equations 238
 - websites, COMSOL 19
 - work planes 266, 418
 - WorkPlane (geometry) 418
- X**
 - XmeshInfo (solvers) 649

